

A UNIFIED THEORY OF LEARNING THROUGH SIMILARITY

KERNELS

The Geometry of Learning

Foundations, Algorithms, and the Frontiers
of Machine Intelligence

Similarity becomes geometry. Geometry becomes learning.

WRITTEN BY

Taha Bouhsine

AzettaAI · tahabouhsine.com

Dedication

For my grandmother, **Henya**.
Your strength became inheritance.
Your love became direction.
Whatever light this book carries began with you.
— Taha

Preface

A kernel is a decision about what the world is allowed to call similar. From that decision come a geometry, a notion of simplicity, and the patterns a machine can learn.

Every learning system begins before the optimization, before the data split, and even before the model. It begins with a choice about resemblance. Which differences matter? Which variations should disappear? What structure should survive when an example is compared with another it has never seen?

Kernel methods turn those questions into mathematics. A similarity function becomes an inner product; the inner product opens a Hilbert space; and geometry becomes prediction, testing, inference, or control. The same idea reaches from support vector machines and Gaussian processes to distributions, graphs, sequences, operators, dynamical systems, and the infinite-width limits of neural networks. Few ideas in machine learning travel so far while keeping their mathematical spine intact.

This book is an attempt to show that spine whole.

Why this book

Kernel methods are often presented as a finished chapter in the history of machine learning: elegant, useful, and overtaken. That view mistakes a set of famous algorithms for the deeper idea that produced them. Kernels are not a museum of pre-deep-learning techniques. They are a language for designing geometry, controlling complexity, representing structured objects, reasoning about functions, and understanding what modern learning systems do.

The subject has also become fragmented. The foundations live in functional analysis; the algorithms in optimization; generalization in probability; Gaussian processes in Bayesian statistics; mean embeddings in modern statistics; graph and sequence kernels in specialized literatures; neural tangent kernels and feature learning at the frontier. A reader can master one region and still miss the bridges that make the field coherent.

The purpose of *Kernels: The Geometry of Learning* is to build those bridges. It develops the theory from first principles, follows it into computation, and keeps going until the classical and modern views meet. The goal is not merely to catalogue methods. It is to reveal the few structural ideas that generate many of them.

The book's promises

The book makes three promises to its reader.

First, **rigor without ritual**. Definitions appear before they are needed; formal results state their assumptions and proof status; arguments are included because they explain, not because a textbook is expected to contain them.

Second, **practice without black boxes**. Algorithms are connected to their objectives, conditioning, complexity, diagnostics, and failure modes. Computed examples and companion laboratories are designed to make mathematical claims answerable to numerical evidence.

Third, **breadth without losing the thread**. Classical machines, distributional methods, probabilistic models, scientific applications, and modern neural kernels are treated as parts of one story: choose a geometry, understand the function class it creates, and learn responsibly inside it.

These promises are standards the book is built to meet, not claims of infallibility. Proofs can be sharpened, examples can be improved, and the frontier will move. The provenance records, review states, tests, revision history, and errata process make that unfinished work visible. Intellectual honesty is part of the architecture.

Three ways through one book

Graduate readers can follow the foundational path, beginning with the mathematical preliminaries and moving through RKHS theory, supervised learning, generalization, spectral methods, kernel construction, scaling, and Gaussian processes. Practitioners can move between core explanations and reproducible labs, emphasizing model selection, diagnostics, approximation, and case studies. Researchers can read the complete sequence, including provenance, proof boundaries, advanced constructions, and open problems.

These are paths through one canonical text, not diluted editions for different audiences. The mathematics does not change when the route does.

Sources and gratitude

This book did not invent its subject; it tries to carry it faithfully. Three works, above all, taught it how to think. The lecture course *Machine Learning with Kernel Methods* by Julien Mairal and Jean-Philippe Vert gave the through-line from positive-definite functions to modern applications. *Learning with Kernels* by Bernhard Schölkopf and Alexander Smola gave the regularization view and the geometry of the feature space. *Kernel Methods for Pattern Analysis* by John Shawe-Taylor and Nello Cristianini gave the modular separation of kernel from algorithm that organizes much of this book. Where the exposition here is clear, it is often because these came first.

Beyond those foundations, the book rests on the primary literature — hundreds of papers whose theorems, algorithms, and counterexamples are attributed at the point of use. The bibliography is meant to be read as a record of debt as much as a list of references. To every author whose result appears in these pages: thank you. The synthesis is this edition's; the mathematics is yours.

And a broader thanks is owed to the community this book belongs to — the one that insists on explaining learning with mathematics. It states its assumptions instead of hiding them, proves what it claims instead of asserting it, and reaches for a theorem before a metaphor. Metaphors are how we start to understand; they are not where understanding should stop. The discipline of turning intuition into a definition, and a definition into a proof that anyone can check, is what makes a field cumulative rather than fashionable. This book is only possible because so many people have done that work, in the open, for decades. It is written in gratitude to them, and in the hope of being useful to whoever does it next.

An invitation

The central idea of this book is simple enough to say in one line:

Similarity becomes geometry. Geometry becomes learning.

But simple ideas become powerful only when followed without compromise. The chapters ahead follow this one from Gram matrices to operators, from margins to probability measures, from finite samples to infinite-width networks, and from theorem to working system.

If the book succeeds, kernels will no longer look like a collection of tricks. They will look like what they are: one of the clearest ways we have learned to turn structure into intelligence.

Taha Bouhsine
tahabouhsine.com · AzettaAI ((azettaai?))
July 2026

Contents

Preface	v
Why this book	v
The book's promises	v
Three ways through one book	vi
Sources and gratitude	vi
An invitation	vi
1 Mathematical Preliminaries	1
1.1 Linear algebra and matrices	1
1.2 Real analysis, metrics, and Hilbert spaces	2
1.3 Measure, integration, and Hilbert-valued expectation	3
1.4 Operators, spectra, and Fourier analysis	4
1.5 Probability and statistics	4
1.6 Convex optimization and duality	5
1.7 More probability: conditioning, scores, and U-statistics	6
1.8 Graphs, manifolds, and groups	8
1.9 Couplings and linear programs	9
1.10 Paths, tensors, and iterated integrals	9
1.11 Indefinite inner products	9
1.12 Numerical linear algebra clinic	10
1.13 Measure and probability clinic	10
1.14 Functional and convex analysis clinic	10
1.15 Structured-domain clinic	11
1.16 Summary and further reading	11
1.17 Notation and conventions	11
1.18 Common mistakes and practical implications	11
1.19 Exercises	12
2 Introduction: Learning by Comparison	13
2.1 The promise: patterns are relations	13
2.2 Three views of the same object	13
2.2.1 The geometric view: a kernel is an inner product	14
2.2.2 The regularization view: the norm is smoothness	14
2.2.3 The modular view: kernel plus algorithm	15
2.3 The one obstacle: the solve	15
2.4 How to read this book	15
2.5 Common mistakes and practical implications	17
2.6 Summary and further reading	17
2.7 Exercises	17
3 Positive Definite Kernels and RKHS	19
3.1 Why pairwise comparisons?	19
3.2 Positive definite kernels	19
3.2.1 The simplest p.d. kernels	20
3.2.2 A first nonlinear example: the polynomial kernel	22
3.3 Kernels as inner products: Aronszajn's theorem	23
3.3.1 In case of need: Hilbert spaces	23

3.3.2	Proof of the converse: the finite case	24
3.3.3	The general case: a short history	24
3.4	Reproducing kernel Hilbert spaces	25
3.4.1	Mapping data points to functions	25
3.4.2	Definition and the reproducing property	26
3.4.3	An equivalent definition: continuous evaluation	26
3.4.4	Uniqueness of the kernel and of the space	28
3.5	The equivalence: positive definite equals reproducing	28
3.5.1	Application: back to Aronszajn's theorem	30
3.6	Examples: kernels and their RKHS	30
3.6.1	The linear kernel	31
3.6.2	The polynomial kernel of degree two	31
3.6.3	The Gaussian kernel	32
3.7	Combining kernels: the cone of p.d. kernels	33
3.8	Worked examples: a kernel quiz	34
3.8.1	Series, exponentials and separable factors (1, 2, 3, 5)	35
3.8.2	Trigonometric kernels (6, 7)	35
3.8.3	Minima and maxima (8, 9, 10)	36
3.8.4	Arithmetic kernels (11, 12, 13)	36
3.8.5	A cautionary counterexample (4)	37
3.9	The RKHS norm as a smoothness functional	37
3.10	Summary	38
3.11	Common mistakes and practical implications	39
3.12	Exercises	39
4	The Kernel Trick and the Representer Theorem	41
4.1	Why map data to a Hilbert space?	41
4.1.1	Two purposes of the embedding	42
4.2	The kernel trick	43
4.2.1	Example 1: distances in the feature space	43
4.2.2	Example 2: distance between a point and a set	45
4.2.3	What the choice of kernel does: illustrations	46
4.2.4	A first pattern-analysis algorithm: novelty detection	47
4.2.5	Example 3: centering data in the feature space	47
4.2.6	Summary: three ways to use the trick	48
4.2.7	Structured inputs and outputs	49
4.3	The representer theorem	49
4.3.1	Scope and the regularization reading	51
4.3.2	Practical use: from $\mathcal{H}$ to $\mathbb{R}^n$	52
4.4	A general recipe for kernelizing an algorithm	53
4.5	Common mistakes and practical implications	54
4.6	Summary and further reading	54
4.7	Exercises	55
5	Kernel Ridge Regression and Smooth Losses	57
5.1	The supervised learning problem	57
5.1.1	Supervised learning with kernels: general principles	58
5.2	Kernel ridge regression	59
5.2.1	The regression setup and least squares	59
5.2.2	The KRR problem and the effect of the representer theorem	60
5.2.3	Solving KRR: a single linear system	60
5.2.4	Worked example: KRR with the Gaussian RBF kernel	62
5.2.5	Uniqueness of the solution	63
5.2.6	Link with standard ridge regression	64

5.2.7	Three readings of the smoother: Gaussian processes, degrees of freedom, cross-validation	66
5.2.8	Leave-one-out cross-validation in closed form	66
5.2.9	Beyond the squared loss: robust regression	68
5.2.10	Weighted regression	69
5.3	Kernel logistic regression	71
5.3.1	Binary classification and the trouble with the 0/1 loss	71
5.3.2	The logistic model and the logistic loss	71
5.3.3	Reduction to a finite-dimensional convex problem	72
5.3.4	Newton's method and the quadratic approximation	73
5.3.5	Solving KLR by iteratively reweighted least squares	74
5.4	Large-margin classifiers: a first look	75
5.4.1	Losses as functions of the margin	75
5.4.2	Solving large-margin classifiers in general	77
5.4.3	A tiny bit of learning theory: risk, Bayes risk, and the ϕ -risk	78
5.5	Common mistakes and practical implications	79
5.6	Summary and further reading	79
5.7	Exercises	79
6	Support Vector Machines	83
6.1	From margins to the hinge loss	83
6.2	The primal problem	84
6.2.1	Reduction to finite dimension	84
6.2.2	Slack variables and smooth constraints	84
6.3	Lagrangian duality and the dual problem	85
6.3.1	The Lagrangian	85
6.3.2	Minimizing over α	86
6.3.3	Minimizing over ξ	86
6.3.4	The dual function	87
6.3.5	Eliminating ν	87
6.3.6	Back to the primal: the dual in terms of α	87
6.4	Complementary slackness and the KKT conditions	88
6.4.1	Reading off the three regimes	89
6.4.2	The same conclusions without KKT	89
6.5	Geometric interpretation	90
6.5.1	Why a wide margin generalizes	91
6.5.2	A worked two-point example	92
6.6	Support vectors	93
6.7	Practical remarks and variants	94
6.7.1	The C -SVM parameterization	94
6.7.2	The ν -SVM	95
6.7.3	The 2-SVM	96
6.8	Multiclass SVM	96
6.8.1	One-vs-all	96
6.8.2	One-vs-one	97
6.8.3	Single-machine formulations	98
6.9	Probabilistic outputs	99
6.9.1	What calibration can and cannot promise	100
6.10	Model selection	101
6.10.1	Cross-validation	101
6.10.2	Leave-one-out bounds from a single training	101
6.11	Summary	104
6.12	Common mistakes and practical implications	105
6.13	Exercises	105

7	Support Vector Regression	109
7.1	The ε -insensitive loss and where sparsity comes from	109
7.2	The primal program for SV regression	110
7.3	The dual and its box constraints	110
7.4	Geometry of the ε -tube: support vectors, sparsity, and resistance	111
7.5	General losses and the ridge connection	113
7.6	Quantile regression with the pinball loss	113
7.7	Expectile regression	116
7.8	Noncrossing distributional prediction	116
7.9	ν -SV regression	116
7.10	Parametric insensitivity models	118
7.11	Applications	119
7.12	Common mistakes and practical implications	119
7.13	Summary and further reading	119
7.14	Exercises	119
8	One-Class SVMs and Novelty Detection	121
8.1	Estimating the support of a distribution	121
8.2	The smallest enclosing hypersphere	122
8.2.1	The soft hypersphere	123
8.2.2	SVDD with negative examples	124
8.3	Separating the data from the origin	125
8.4	The nu-property	126
8.4.1	Connection to classification and robustness	127
8.5	Hypersphere equals hyperplane for RBF kernels	128
8.6	Density level sets and the Parzen connection	128
8.7	Uniqueness, generalization, and experiments	129
8.8	Common mistakes and practical implications	130
8.9	Summary and further reading	130
8.10	Exercises	131
9	Ranking and Ordinal Regression	133
9.1	The ranking problem	133
9.2	Reducing ranking to classification on pairs	133
9.2.1	The ranking risk and misordered pairs	134
9.3	The soft-margin ranking SVM	135
9.4	Online ranking: a perceptron for preferences	136
9.5	Ordinal regression: thresholds on the line	137
9.6	Ranking and the area under the ROC curve	138
9.7	Common mistakes and practical implications	139
9.8	Summary and further reading	139
9.9	Exercises	140
10	Solving the SVM: Decomposition and SMO	141
10.1	Why the dual is hard at scale	141
10.2	The KKT conditions as the stopping test	141
10.3	Decomposition and chunking	142
10.3.1	Working-set selection	143
10.4	Sequential minimal optimization	144
10.4.1	The analytic two-variable step	144
10.5	Caching and shrinking	146
10.6	Interior-point methods, the alternative	146
10.7	SMO for regression	147
10.8	Summary	147

10.9 Common mistakes and practical implications	148
10.10 Summary and further reading	148
10.11 Exercises	148
11 Online Kernel Learning	151
11.1 The online learning setting	151
11.2 The kernel perceptron	151
11.2.1 The dual update and the kernel expansion	151
11.3 Novikoff's mistake bound	153
11.3.1 Voted and averaged perceptrons	154
11.4 The kernel adatron	155
11.5 Online support vector regression	156
11.6 NORMA and passive-aggressive updates	157
11.7 The budget problem	157
11.8 Summary	158
11.9 Common mistakes and practical implications	158
11.10 Summary and further reading	158
11.11 Exercises	159
12 Learning Theory in RKHS Balls	161
12.1 From the 0/1 loss to convex surrogates	161
12.2 Calibration: a small φ -risk ensures a small 0/1 risk	162
12.3 Empirical risk minimization	164
12.4 Rademacher complexity	165
12.5 A basic learning bound for ERM	166
12.6 ERM in balls of an RKHS	167
12.7 Fast rates: local complexity and stability	170
12.7.1 Localization and the variance condition	170
12.7.2 The sub-root fixed point that sets the rate	171
12.7.3 Generalization from algorithmic stability	172
12.8 From constrained ERM to penalized ERM: regularization	174
12.9 Interlude: convex optimization and Lagrangian duality	175
12.9.1 Primal, dual, and the duality gap	175
12.9.2 The Lagrangian and the dual function	176
12.9.3 Weak and strong duality	177
12.9.4 Dual optimal pairs and complementary slackness	178
12.10 The support vector machine in this light	179
12.11 Common mistakes and practical implications	179
12.12 Summary and further reading	180
12.13 Exercises	180
13 VC Theory and Generalization	183
13.1 The risk we want and the risk we can measure	183
13.2 Concentration and the law of large numbers	183
13.3 Consistency and uniform convergence	185
13.4 The growth function and the VC dimension	185
13.5 Deriving a VC bound	187
13.6 Structural risk minimization	189
13.7 The margin controls capacity	189
13.8 Common mistakes and practical implications	190
13.9 Summary and further reading	191
13.10 Exercises	191

14 Kernel PCA and Denoising	193
14.1 Principal component analysis	193
14.1.1 Formalization	193
14.1.2 Solution	194
14.2 Kernel PCA	194
14.2.1 Formalization in the RKHS	194
14.2.2 Sanity check: kernel PCA with the linear kernel is PCA	195
14.2.3 The representer argument	195
14.2.4 Solving the finite-dimensional problem	196
14.2.5 Reconstruction error: the second face of the spectrum	200
14.2.6 Denoising: projecting away the noise directions	201
14.2.7 The pre-image problem	202
14.3 Out-of-sample extension and the Nyström view	203
14.3.1 Projecting a new point	203
14.3.2 The Nyström view and its cost	205
14.3.3 The general Nyström-extension principle	206
14.3.4 Robust and sparse variants	207
14.4 Common mistakes and practical implications	207
14.5 Summary and further reading	207
14.6 Exercises	207
15 Kernel Clustering and Spectral Methods	211
15.1 The K-means algorithm	211
15.1.1 An optimization point of view	211
15.2 The kernel K-means algorithm	212
15.2.1 The two steps, in kernel form	213
15.2.2 An equivalent objective	213
15.3 Spectral clustering	214
15.3.1 A matrix form of the objective	214
15.3.2 The relaxation	215
15.3.3 From eigenvectors back to a partition	216
15.4 The graph-cut view: normalized cuts	216
15.5 The relaxation to the Fiedler vector	217
15.5.1 The normalized Laplacian and the Fiedler vector	219
15.5.2 What the relaxation gives up	220
15.6 Normalized-cut spectral clustering	220
15.7 Kernel K-means and normalized cut are one objective	221
15.8 Out-of-sample extension	222
15.9 Common mistakes and practical implications	222
15.10 Summary and further reading	223
15.11 Exercises	223
16 Kernel CCA and Correlation Analysis	225
16.1 Kernel canonical correlation analysis	225
16.1.1 Classical CCA	225
16.1.2 CCA is a generalized eigenvalue problem	226
16.1.3 Kernelizing CCA	226
16.1.4 What goes wrong, and why we must regularize	227
16.1.5 Variance, covariance, and correlation as one family	228
16.2 Why regularization is mandatory	228
16.2.1 Characteristic kernels force the saturation	229
16.2.2 The shrinkage form of the regularized eigenproblem	229
16.2.3 A worked example: saturation and its cure	230
16.3 Multi-view learning and deep CCA	231

16.4	Common mistakes and practical implications	231
16.5	Summary and further reading	232
16.6	Exercises	232
17	Kernel Discriminants and Projections	235
17.1	From variance to prediction	235
17.2	The generalized eigenvalue problem and deflation	235
17.3	Kernel Fisher discriminant analysis	237
17.3.1	Between-class and within-class scatter	237
17.3.2	The dual, and why regularization is unavoidable	238
17.4	Principal components regression	239
17.5	Kernel partial least squares	240
17.5.1	Directions of maximum covariance	241
17.5.2	Primal PLS and its deflation	241
17.5.3	The dual and kernel PLS	242
17.6	One engine, three methods	243
17.7	Summary	243
17.8	Common mistakes and practical implications	243
17.9	Summary and further reading	244
17.10	Exercises	244
18	Data Visualization and Kernel MDS	247
18.1	The visualization problem	247
18.2	From distances to inner products: classical MDS	248
18.2.1	The double-centering identity	248
18.2.2	The eigendecomposition and the embedding	249
18.3	MDS is PCA is kernel PCA	250
18.4	Kernel MDS: visualizing the feature space	251
18.5	Metric and non-metric MDS	252
18.6	Manifold learning as kernel PCA	253
18.6.1	Isomap: the centered geodesic kernel	253
18.6.2	Locally linear embedding: the max-eigenvalue-shift kernel	254
18.6.3	Laplacian eigenmaps: the pseudo-inverse Laplacian kernel	254
18.6.4	Where t-SNE and UMAP part company	256
18.7	Common mistakes and practical implications	257
18.8	Summary and further reading	257
18.9	Exercises	257
19	Mercer's Theorem, Spectra, and Rates	259
19.1	From smoothness functionals to kernels	259
19.1.1	A first example: the min kernel on $[0, 1]$	259
19.1.2	Green functions of differential operators	261
19.2	Mercer kernels and their integral operator	262
19.2.1	Mercer kernels	262
19.2.2	Compact, self-adjoint, positive operators	263
19.2.3	The integral operator of a kernel	263
19.2.4	Spectral decomposition of the operator	266
19.3	Mercer's theorem	267
19.3.1	Mercer kernels as inner products in ℓ^2	270
19.3.2	Scope and limits of the construction	271
19.4	Worked example 1: periodic kernels on $[0, 1]$	271
19.4.1	Fourier eigenfunctions	271
19.4.2	Polynomial decay of eigenvalues: Bernoulli kernels	272
19.4.3	Exponential decay of eigenvalues	273

19.5	Worked example 2: dot-product kernels on the sphere S^{d-1}	273
19.5.1	Spherical harmonics	274
19.5.2	The Funk-Hecke theorem	274
19.5.3	A fully worked case: the quadratic kernel on the circle	275
19.6	The RKHS built from the spectrum	276
19.6.1	The geometry of the RKHS	278
19.6.2	Example: Sobolev spaces of periodic functions	278
19.7	Convergence rates of kernel ridge regression	279
19.7.1	The isometry between $\mathcal{H}$ and $L^2_{\nu}(\mathcal{X})$	279
19.7.2	KRR and the statistical model	279
19.7.3	The bias-variance decomposition of the excess MSE	280
19.7.4	A simplification: replacing $\Phi_n^{\top} \Phi_n$ by nT	282
19.7.5	Upper bounds on bias and variance	282
19.7.6	Rates of convergence	284
19.7.7	Optimality and adaptivity	286
19.8	The integral-operator minimax framework: source and capacity	286
19.8.1	The source condition	286
19.8.2	The capacity condition and the effective dimension	287
19.8.3	The minimax rate and the saturation of KRR	287
19.9	Beyond compactness: an outlook	289
19.10	Common mistakes and practical implications	290
19.11	Summary and further reading	290
19.12	Exercises	290
20	Translation-Invariant, Semigroup, and Probabilistic Kernels	293
20.1	Translation-invariant kernels on the integers	293
20.1.1	The Fourier-Stieltjes transform on the torus	294
20.1.2	Two examples	294
20.1.3	Proof of Herglotz's theorem	295
20.2	Translation-invariant kernels on $\mathbb{R}^d$	296
20.2.1	The Fourier-Stieltjes transform on $\mathbb{R}^d$	297
20.2.2	Proof of Bochner's theorem	299
20.3	The RKHS of a translation-invariant kernel	300
20.3.1	Reading off the smoothness class	301
20.3.2	The Matern family	302
20.3.3	A recap on the Green, Mercer, and Bochner families	302
20.4	Parametric spectral families: Matern and spectral mixtures	303
20.4.1	The Matern spectrum and mean-square smoothness	303
20.4.2	Worked example: the smoothness ladder in numbers	304
20.4.3	Spectral mixture kernels	305
20.4.4	Non-stationary kernels: varying length scales and warping	306
20.5	Conditionally positive definite kernels	307
20.6	Generalization to semigroups	308
20.6.1	Semicharacters	309
20.6.2	Integral representation of positive definite functions	310
20.6.3	Example: the half-line $(\mathbb{R}_+, +, \text{Id})$	310
20.6.4	Example: kernels for finite measures	311
20.7	Kernels from probabilistic models	313
20.7.1	The Fisher kernel	313
20.7.2	A worked example: a Gaussian data model	315
20.7.3	Application: aggregation of visual words	315
20.7.4	Relation to classification with generative models	316
20.7.5	Mutual information kernels	317
20.7.6	Marginalized kernels	318

20.8	The algebra of kernel construction	319
20.8.1	Interpolating with ANOVA kernels, and normalization	319
20.9	Summary	320
20.10	Common mistakes and practical implications	320
20.11	Exercises	321
21	Invariances and the Pre-Image Problem	323
21.1	Prior knowledge and invariance	323
21.2	The Virtual Support Vector method	324
21.3	Tangent vectors, invariance kernels, and jittering	325
21.3.1	Tangent distance and translation-invariant kernels	325
21.3.2	Jittering kernels	325
21.4	The pre-image problem	326
21.5	Finding approximate pre-images	327
21.6	Reduced set methods	328
21.6.1	Sequential evaluation and face detection	330
21.7	Summary	330
21.8	Common mistakes and practical implications	330
21.9	Summary and further reading	330
21.10	Exercises	331
22	Kernels for Sequences	333
22.1	Why sequences, and why kernels	333
22.2	Supervised classification by vector embedding	334
22.2.1	Physico-chemical embeddings	334
22.2.2	The vector space model and text kernels	334
22.3	Substring indexation and the spectrum kernel	335
22.3.1	The spectrum kernel	335
22.4	The subsequence kernel and its dynamic program	337
22.4.1	Definition	337
22.4.2	Computing the kernel by dynamic programming	338
22.4.3	Dictionary-based indexation	341
22.5	Kernels from generative models	341
22.5.1	P-kernels and conditional independence	341
22.5.2	Context-tree models and the context-tree kernel	342
22.5.3	Marginalized kernels	343
22.5.4	Fixed-length and pair hidden Markov model kernels	344
22.5.5	Stochastic context-free grammars for RNA	344
22.5.6	The Fisher kernel for sequences	345
22.6	The local alignment kernel	345
22.6.1	From Smith-Waterman to a kernel	346
22.6.2	The local alignment kernel is positive definite	347
22.6.3	Computing the LA kernel by dynamic programming	349
22.7	Application: remote homology detection	350
22.8	Summary	351
22.9	Common mistakes and practical implications	351
22.10	Exercises	352
23	Efficient String and Tree Kernels	355
23.1	From feature counts to dynamic programming	355
23.2	The all-subsequences kernel	355
23.3	The fixed-length subsequences kernel	357
23.4	The gap-weighted subsequences kernel	357
23.4.1	Variants: character weightings, gap counts, and soft matching	359

23.5 Beyond dynamic programming: trie-based kernels	360
23.6 Kernels for trees	361
23.7 The unifying view: convolution kernels	362
23.8 Summary	363
23.9 Common mistakes and practical implications	363
23.10 Summary and further reading	363
23.11 Exercises	363
24 Kernels for Text	365
24.1 From documents to vectors: the bag of words	365
24.2 Term weighting and tf-idf	366
24.3 Semantic smoothing with a proximity matrix	368
24.4 The generalised vector space model	368
24.5 Latent semantic kernels	369
24.6 Semantic diffusion kernels	371
24.7 Neural embeddings and optimal transport	372
24.7.1 Word and document embeddings as a linear kernel	372
24.7.2 The Word Mover's Distance	373
24.7.3 Contextual and sentence embeddings as a cosine kernel	374
24.8 String and word-sequence kernels	375
24.9 Summary	375
24.10 Common mistakes and practical implications	375
24.11 Summary and further reading	375
24.12 Exercises	376
25 Kernels for and on Graphs	377
25.1 Part A: Kernels between graphs	377
25.1.1 Explicit features: indexing by substructures	377
25.1.2 The expressiveness-complexity tradeoff	379
25.1.3 Walk kernels	381
25.1.4 The product graph	382
25.1.5 Extensions: labels, tottering, and subtrees	384
25.1.6 The Weisfeiler-Leman hierarchy	385
25.1.7 Message-passing GNNs and the 1-WL ceiling	387
25.1.8 Kernelized 1-WL: from histograms to optimal transport	388
25.1.9 The convolution kernel: one framework behind them all	389
25.1.10 Applications and summary	391
25.2 Part B: Kernels on a graph	391
25.2.1 Graph distance and conditionally p.d. kernels	392
25.2.2 Construction by regularization: the graph Laplacian	394
25.2.3 The diffusion kernel	396
25.2.4 Harmonic analysis on graphs	398
25.2.5 Applications	399
25.3 Common mistakes and practical implications	400
25.4 Summary and further reading	400
25.5 Exercises	400
26 Kernels from Generative Models	403
26.1 P-kernels: a probability as an inner product	403
26.1.1 Conditional independence and marginalization	404
26.2 The fixed-length marginalization kernel	404
26.3 The hidden Markov model kernel	405
26.4 The pair hidden Markov model kernel	406
26.5 The hidden tree model kernel	407

26.6	The Fisher kernel	409
26.7	Further reading	410
26.8	Common mistakes and practical implications	410
26.9	Summary and further reading	411
26.10	Exercises	411
27	Signature and Sequence-Path Kernels	413
27.1	From points to paths	413
27.2	The path signature	413
27.2.1	One straight segment: the tensor exponential	415
27.3	Three structural properties	415
27.3.1	Invariance to reparametrization	415
27.3.2	Chen's identity: concatenation multiplies	416
27.3.3	The shuffle identity	417
27.3.4	Uniqueness up to tree-like equivalence	417
27.4	Computing the truncated signature	418
27.5	The truncated signature kernel	418
27.6	The untruncated signature kernel as a Goursat PDE	419
27.7	Alignment kernels: warping done positively	420
27.7.1	Why dynamic time warping is not a kernel	420
27.7.2	The global alignment kernel	421
27.7.3	Why the global alignment kernel is positive definite	422
27.8	Reservoir and random-feature sequence kernels	423
27.9	Summary	423
27.10	Common mistakes and practical implications	423
27.11	Summary and further reading	424
27.12	Exercises	424
28	Geometric and Equivariant Kernels	427
28.1	Why the domain's geometry matters	427
28.2	The Matern family as a stochastic differential equation	427
28.3	Kernels on a Riemannian manifold	428
28.3.1	The Laplace-Beltrami operator and its spectrum	428
28.3.2	The manifold Matern and heat kernels	428
28.4	Kernels on a weighted graph	429
28.5	Lie groups and homogeneous spaces	431
28.6	Group-invariant kernels by averaging	432
28.7	Summary	434
28.8	Common mistakes and practical implications	434
28.9	Summary and further reading	435
28.10	Exercises	435
29	Indefinite and Krein-Space Kernels	437
29.1	When the similarity is not a kernel	437
29.2	Krein spaces and the $k = k_+ - k_-$ decomposition	438
29.2.1	Indefinite inner products	438
29.2.2	Reproducing kernel Krein spaces	439
29.3	The representer theorem becomes a stabilization	440
29.4	Spectrum transformations	441
29.5	Learning the SVM in a Krein space	443
29.6	Summary	445
29.7	Common mistakes and practical implications	445
29.8	Summary and further reading	445
29.9	Exercises	446

30 Mean Embeddings, MMD, and Characteristic Kernels	449
30.1 Comparing two distributions from samples	449
30.2 The kernel mean embedding	450
30.2.1 Existence of the mean embedding	451
30.3 The Maximum Mean Discrepancy	452
30.3.1 Estimating the MMD from samples	454
30.4 MMD as an integral probability metric	454
30.5 A two-sample test using the MMD	455
30.5.1 The threshold and the permutation test	456
30.6 Independence testing with HSIC	457
30.7 Learning generative models with MMD	458
30.7.1 The adversarial view and the GAN connection	458
30.8 Characteristic kernels	459
30.9 Characteristic kernels via universality	460
30.9.1 Stone–Weierstrass and a general criterion	460
30.9.2 Universality from a Taylor series	461
30.9.3 Universality from a Fourier series	462
30.10 Characteristic kernels via the Fourier transform	463
30.11 Summary	464
30.12 Common mistakes and practical implications	464
30.13 Exercises	464
31 Kernel Hypothesis Testing	467
31.1 From a discrepancy to a decision	467
31.2 The V-statistic and the U-statistic	468
31.3 Why the null law is a chi-square mixture	469
31.4 Calibrating the null by permutation	470
31.5 Test power and the objective for kernel choice	471
31.6 The median heuristic and its limits	472
31.7 Learned and deep kernels	473
31.8 Linear-time versus quadratic-time estimators	474
31.9 Aggregating over kernels: MMDagg	474
31.10 Independence and interaction testing	475
31.11 Relative tests and goodness-of-fit	475
31.12 Block estimators and streaming tests	476
31.13 Dependent observations and the wild bootstrap	476
31.14 Robustness, multiple testing, and selective kernel choice	477
31.15 Conditional and localized two-sample questions	477
31.16 Summary	477
31.17 Common mistakes and practical implications	478
31.18 Summary and further reading	478
31.19 Exercises	478
32 Optimal Transport and Kernels	481
32.1 Two ways to measure the distance between distributions	481
32.2 The Monge and Kantorovich problems	481
32.2.1 Monge’s map	481
32.2.2 Kantorovich’s relaxation: transport plans	482
32.3 Kantorovich duality and the IPM connection	482
32.3.1 Kantorovich–Rubinstein: W_1 as an IPM over 1-Lipschitz functions	483
32.4 The one-dimensional case: transport by sorting	484
32.5 Sample complexity: the price of geometry	485
32.6 Entropic regularization and the Sinkhorn algorithm	485
32.6.1 The scaling form of the solution	486

32.7 Sinkhorn divergences: debiasing and the bridge to MMD	488
32.8 When to prefer optimal transport, when to prefer MMD	489
32.9 Summary	489
32.10 Common mistakes and practical implications	490
32.11 Summary and further reading	490
32.12 Exercises	490
33 Kernel Quadrature and Herding	493
33.1 The integration problem, and why randomness is wasteful	493
33.2 Worst-case error is a maximum mean discrepancy	494
33.3 Optimally weighted quadrature	496
33.4 Bayesian quadrature	497
33.5 Kernel herding	498
33.5.1 Super-samples and the rate, stated carefully	500
33.5.2 Optimally weighted herding is Bayesian quadrature	500
33.6 Optimal nodes, leverage scores, and determinantal sampling	501
33.7 A note on coresets	501
33.8 Summary	501
33.9 Common mistakes and practical implications	502
33.10 Summary and further reading	502
33.11 Exercises	502
34 Conditional Mean Embeddings and Kernel Bayes' Rule	505
34.1 From marginal to conditional embeddings	505
34.2 The conditional mean embedding operator	506
34.3 The empirical estimate	507
34.4 The regression view	509
34.5 Kernel probability rules	509
34.5.1 The kernel sum rule	509
34.5.2 The kernel chain rule	510
34.6 Kernel Bayes' Rule	510
34.7 Kernelized inference and the kernel Bayes filter	511
34.8 Summary	512
34.9 Common mistakes and practical implications	512
34.10 Summary and further reading	512
34.11 Exercises	512
35 Kernel Stein Discrepancy and Stein Methods	515
35.1 Goodness of fit for a model you can only score	515
35.2 Stein's identity and the Stein operator	516
35.3 From the identity to a discrepancy	517
35.4 The kernel Stein discrepancy	517
35.4.1 The Stein kernel and its closed form	518
35.4.2 Worked closed form: RBF kernel against a Gaussian target	519
35.5 Empirical KSD: U-statistics and V-statistics	520
35.6 A goodness-of-fit test for unnormalized models	521
35.7 Stein variational gradient descent	522
35.8 Choosing the kernel, and a bridge to quadrature	524
35.9 Summary	524
35.10 Common mistakes and practical implications	525
35.11 Exercises	525
36 Causal Inference with Kernels	527
36.1 From dependence to intervention	527
36.2 Dependence as a distance between embeddings	528

36.3	Conditional independence in an RKHS	528
36.4	Estimating effects under confounding	530
36.4.1	Kernel instrumental variables	530
36.4.2	Proximal causal learning	532
36.5	Beyond the mean: distributional and counterfactual effects	532
36.6	What kernels buy, and what they do not	532
36.7	Summary	533
36.8	Common mistakes and practical implications	533
36.9	Summary and further reading	533
36.10	Exercises	534
37	Distribution Regression and Functional Data	535
37.1	When the input is a distribution	535
37.1.1	The two-stage sampled model	535
37.2	Representing each input by its mean embedding	536
37.3	A kernel on distributions: the kernel of kernels	536
37.4	The two-stage estimator	539
37.5	Consistency and the two error sources	540
37.6	Set kernels and support measure machines	541
37.7	Functional data: when the input is a function	542
37.8	Summary	543
37.9	Common mistakes and practical implications	543
37.10	Summary and further reading	543
37.11	Exercises	543
38	Large-Scale Kernel Machines	545
38.1	The scaling problem	545
38.2	Interlude: large-scale learning with linear models	546
38.2.1	Why convexity, and two inequalities	546
38.2.2	Gradient descent and its rate	548
38.2.3	Stochastic gradient descent	549
38.2.4	Randomized incremental methods for finite sums	550
38.3	Matrix-free computation and EigenPro	551
38.4	Before approximation: decomposition and interior-point solvers	552
38.5	The Nystrom approximation	553
38.5.1	The principle: projection onto a small subspace	553
38.5.2	Choosing the anchors by kernel PCA	554
38.5.3	Choosing the anchors by random sampling	555
38.5.4	Greedy and K-means anchors	556
38.6	Choosing landmarks by leverage scores	557
38.6.1	Ridge leverage scores and the effective dimension	557
38.6.2	Sampling proportionally to leverage	558
38.7	FALKON and preconditioned solvers	560
38.7.1	The landmark system and its conditioning	560
38.7.2	A preconditioner from the landmarks alone	560
38.7.3	Optimal rates: approximation as regularization	562
38.8	Random Fourier features	563
38.8.1	From Bochner’s theorem to a random feature	563
38.8.2	The random feature map	564
38.8.3	The approximation guarantee	565
38.8.4	Reduction to a linear model	566
38.9	Nystrom versus random features: pricing the choice	567
38.9.1	Structured and quasi-random features	567
38.10	Block Krylov methods and many right-hand sides	568

38.11	Fast kernel products without global low rank	569
38.12	Mixed precision and reliable stopping	569
38.13	Distributed, federated, and streaming kernels	569
38.14	Common mistakes and practical implications	570
38.15	Summary and further reading	570
38.16	Exercises	570
39	Multiple Kernel Learning	573
39.1	Choosing or combining kernels	573
39.1.1	The setting: learning with one kernel	573
39.2	The sum kernel	574
39.2.1	Summing kernels is concatenating features	575
39.2.2	Example: data integration in bioinformatics	576
39.2.3	The sum kernel from the functional side	576
39.3	The weighted sum kernel	577
39.4	Learning the kernel	579
39.4.1	The value of the problem is convex in the kernel	579
39.4.2	Multiple Kernel Learning	580
39.4.3	Example: genomic data fusion for protein annotation	580
39.4.4	Example: image classification	581
39.5	MKL is a group lasso	581
39.5.1	Sum kernel versus MKL, side by side	583
39.5.2	Example: ridge versus lasso regression	583
39.6	Algorithm and extensions	584
39.6.1	The simpleMKL algorithm	584
39.6.2	Beyond the simplex: the ℓ_r extension	585
39.6.3	The ℓ_p -norm generalization	585
39.6.4	Does learning the kernel actually help?	586
39.7	Centered kernel alignment as a selection criterion	586
39.7.1	Kernel alignment and the ideal kernel	586
39.7.2	Why centering is essential	587
39.7.3	What alignment guarantees	588
39.7.4	Two-stage alignment-based kernel weighting	589
39.8	Common mistakes and practical implications	590
39.9	Summary and further reading	590
39.10	Exercises	590
40	Deep Learning from a Kernel Point of View	593
40.1	What it means to do deep learning with kernels	593
40.2	Deep kernel machines and dot-product kernels	594
40.2.1	A catalog of dot-product kernels	594
40.3	Random feature kernels: the infinite-width limit	595
40.3.1	The deep limit: the NNGP covariance recursion	598
40.4	Neural tangent kernels: the trajectory of gradient descent	599
40.4.1	Depth: the neural tangent kernel recursion	600
40.5	The Nyström bridge and end-to-end learning	601
40.6	Convolutional kernel networks	602
40.7	Constructing the RKHS for continuous signals	603
40.7.1	Patch extraction P_k	603
40.7.2	Nonlinear mapping M_k	603
40.7.3	Pooling A_k	604
40.7.4	The multilayer representation and the CKN kernel	604
40.8	Convolutional kernel networks in practice	604

40.9	Deep learning and stability to deformations	605
40.9.1	Smooth homogeneous activations	605
40.9.2	Deformations and the notion of stability	605
40.9.3	Warmup: translation invariance	606
40.9.4	Stability to deformations	606
40.9.5	Invariance to other groups	607
40.10	Discretization and signal preservation	607
40.11	What the patch RKHS contains, and CNNs inside it	608
40.11.1	Link with generalization	609
40.12	Application to graphs: graph convolutional kernel networks	609
40.12.1	From the path kernel to its relaxation	610
40.12.2	One layer of GCKN	610
40.12.3	Stacking layers	610
40.12.4	Scalable approximation	611
40.12.5	GCKN versus GNN	611
40.12.6	Experiments and interpretation	611
40.13	Application to biological sequences	612
40.13.1	A convolutional kernel network for sequences	612
40.13.2	Gapped k-mers and recurrent kernel networks	613
40.14	Why a fixed kernel makes a poor hidden neuron	613
40.14.1	The activation, and its gradient, vanish almost everywhere	614
40.14.2	Locality does not compose the way piecewise-linearity does	614
40.14.3	Bandwidth is a knife-edge, and scale is unmanaged	614
40.14.4	A kernel is a readout, not a representation	615
40.15	Conclusion	615
40.16	Common mistakes and practical implications	615
40.17	Summary and further reading	616
40.18	Exercises	616
41	Gaussian Processes and the RVM	619
41.1	The Bayesian view: a prior over functions	619
41.2	The Gaussian process	620
41.3	Gaussian process regression	621
41.4	Kernel ridge regression is the GP posterior mean	622
41.5	The marginal likelihood and hyperparameter learning	623
41.6	Gaussian process classification	624
41.7	Laplace mechanics: from mode to prediction	624
41.8	Sparse Bayesian priors and the Relevance Vector Machine	626
41.9	Sparse and variational Gaussian processes	627
41.9.1	The variational answer: sparsity without changing the model	628
41.9.2	Stochastic variational Gaussian processes	630
41.10	Deep, multi-output, and spectral-mixture processes	630
41.10.1	Learning the kernel: spectral mixtures	631
41.10.2	Many outputs: coregionalization	631
41.10.3	Depth: compositions of processes	631
41.11	Common mistakes and practical implications	632
41.12	Summary and further reading	632
41.13	Exercises	632
42	Kernelized Bandits and Bayesian Optimization	635
42.1	Sequential optimization of an unknown function	635
42.2	The surrogate: a running Gaussian process belief	635
42.3	Acquisition functions and the exploration-exploitation tradeoff	636
42.3.1	GP-UCB: optimism in the face of uncertainty	636

42.3.2	Expected improvement	637
42.3.3	Thompson sampling and probability of improvement	638
42.4	Regret and the maximum information gain	639
42.4.1	Sharper confidence: IGP-UCB	642
42.5	Active learning and experimental design	643
42.6	Constrained and safe optimization	643
42.7	Multi-fidelity and cost-aware search	644
42.8	Batch, asynchronous, and high-dimensional Bayesian optimization	644
42.9	Summary	644
42.10	Common mistakes and practical implications	645
42.11	Summary and further reading	645
42.12	Exercises	645
43	Modern Generalization Theory	647
43.1	The interpolation revolution	647
43.1.1	The minimum-norm interpolant	647
43.2	Double descent	648
43.2.1	The risk of ridgeless least squares	648
43.2.2	Why regularization flattens the peak	649
43.3	Benign overfitting	650
43.4	Spectral learning curves and scaling laws	652
43.5	PAC-Bayes and data-dependent certificates	653
43.6	Compression and algorithm-dependent complexity	654
43.7	Random-matrix equivalents and finite-sample diagnostics	654
43.8	Limits, lower bounds, and what cannot be universal	655
43.9	Where this connects	655
43.10	Common mistakes and practical implications	655
43.11	Summary and further reading	656
43.12	Exercises	656
44	The Frontier: Feature Learning and Beyond	657
44.1	Two limits of one wide network	657
44.1.1	The lazy regime and the neural tangent kernel	657
44.1.2	Lazy training is about scale, not depth	659
44.1.3	The parametrization decides the regime	659
44.2	Why a fixed kernel cannot learn features	660
44.3	Kernels that grow depth and learn parts	660
44.3.1	The neural-network Gaussian process hierarchy	660
44.3.2	Convolutional kernel networks	661
44.4	Attention is a kernel smoother	661
44.5	Kernels on curved and discrete domains	663
44.6	Quantum feature maps and kernel estimation	663
44.7	Foundation-model representations as learned kernels	664
44.8	Privacy, federated learning, and fairness	664
44.9	Operator algebra and noncommutative data	664
44.10	Frontier status and annual update policy	665
44.11	The synthesis: what the kernel view keeps, and where it stops	665
44.12	Common mistakes and practical implications	665
44.13	Summary and further reading	666
44.14	Exercises	666
45	Applications and Practice	667
45.1	The anatomy of a kernel-method project	667

45.2	Three applications, three kernels	667
45.2.1	Remote protein homology with string and mismatch kernels	667
45.2.2	Text categorization with the vector-space and string kernels	668
45.2.3	Molecular property prediction with graph kernels	669
45.3	The practical recipe	669
45.3.1	Choosing a kernel for your data	669
45.3.2	Normalization and centering	670
45.3.3	The model-selection loop	671
45.3.4	Diagnosing under and overfitting through the spectrum	672
45.3.5	Exact, Nystrom, or random features	672
45.4	Case study: spatial exposure mapping	673
45.5	Case study: a scientific inverse problem	673
45.6	Case study: dynamics and off-policy evaluation	673
45.7	Case study: distribution shift and calibrated decisions	674
45.8	Case study: censored and time-to-event outcomes	674
45.9	Model cards and reproducibility packets	674
45.10A	map of the software	674
45.11	Summary	675
45.12	Common mistakes and practical implications	676
45.13	Summary and further reading	676
45.14	Exercises	676
46	Vector- and Operator-Valued Kernels	679
46.1	From scalar to vector-valued reproduction	679
46.2	The vector-valued representer theorem	679
46.3	Separable and nonseparable constructions	680
46.4	Functional responses and structured outputs	681
46.5	Learning the output geometry	681
46.6	Differentially constrained vector fields	681
46.7	Structured prediction beyond vector regression	682
46.8	Functional responses and discretization	682
46.9	Scalable block solvers	682
46.10	Generalization and negative transfer	683
46.11	Common mistakes and practical implications	683
46.12	Summary and further reading	683
46.13	Exercises	684
47	Semi-Supervised and Manifold Regularization	685
47.1	What unlabeled data can and cannot identify	685
47.2	Graph energy and label propagation	685
47.3	Manifold regularization	686
47.4	Consistency and graph choices	686
47.5	Harmonic extension and label propagation	686
47.6	Laplacian regularized least squares and SVMs	687
47.7	When unlabeled data help or hurt	687
47.8	Graph limits and density bias	688
47.9	Out-of-sample extensions	688
47.10	Scaling sparse graphs	688
47.11	Common mistakes and practical implications	689
47.12	Summary and further reading	689
47.13	Exercises	689
48	Inverse Learning and Spectral Regularization	691
48.1	Learning as an inverse problem	691

48.2	Iterative regularization and early stopping	691
48.3	Source conditions, qualification, and saturation	692
48.4	A finite spectral diagnostic	692
48.5	A catalog of spectral filters	692
48.6	Conjugate gradients as regularization	693
48.7	Choosing the regularization level	693
48.8	Statistical rates and effective dimension	694
48.9	Stochastic and online iterative regularization	694
48.10	Beyond scalar regression	694
48.11	Common mistakes and practical implications	695
48.12	Summary and further reading	695
48.13	Exercises	695
49	Deep Kernel Learning	697
49.1	The model and its distinction from NTK	697
49.2	Marginal-likelihood training	697
49.3	Exact and approximate computation	698
49.4	Calibration and failure modes	698
49.5	Non-Gaussian likelihoods	698
49.6	Variational inducing-point DKL	699
49.7	Identifiability and geometry control	699
49.8	Derivatives, invariances, and structured inputs	699
49.9	OOD uncertainty and conformal calibration	700
49.10	Comparing DKL with neighboring models	700
49.11	Common mistakes and practical implications	701
49.12	Summary and further reading	701
49.13	Exercises	701
50	Smoothing Splines and Additive RKHS Models	703
50.1	From roughness to a finite estimator	703
50.2	The spline representer theorem	703
50.3	Green functions, splines, and kernel ridge regression	704
50.4	The smoother matrix and effective complexity	704
50.5	Smoothing-spline ANOVA	705
50.6	Bayesian interpretation and uncertainty	706
50.7	Common mistakes and practical implications	706
50.8	Summary and further reading	706
50.9	Exercises	707
51	Spatial and Spatiotemporal Kernel Models	709
51.1	Random fields, covariance, and variograms	709
51.2	Kriging as constrained kernel prediction	709
51.3	Anisotropy and nonstationarity	710
51.4	Matérn fields and the SPDE connection	710
51.5	Space-time covariance design	711
51.6	Multivariate fields and change of support	711
51.7	Common mistakes and practical implications	712
51.8	Summary and further reading	712
51.9	Exercises	712
52	Distribution Shift, Robustness, and Conformal Prediction	713
52.1	A taxonomy of distribution change	713
52.2	Detecting shift with kernel embeddings	713
52.3	Kernel mean matching	714
52.4	Robust losses and distributional neighborhoods	715

52.5	Conformal prediction from exchangeable scores	715
52.6	Conformal prediction under shift	716
52.7	Common mistakes and practical implications	716
52.8	Summary and further reading	716
52.9	Exercises	717
53	Kernels for Dynamical Systems, Control, and Reinforcement Learning	719
53.1	Sequential data and four sources of error	719
53.2	Gaussian-process transition models	719
53.3	Koopman and transfer operators	720
53.4	Kernel EDMD	720
53.5	Value functions and Bellman operators	721
53.6	Off-policy evaluation and coverage	721
53.7	Control with learned kernel models	721
53.8	Common mistakes and practical implications	722
53.9	Summary and further reading	722
53.10	Exercises	722
54	Kernels for Scientific Computing and Operator Learning	725
54.1	Linear information about an unknown function	725
54.2	Symmetric kernel collocation	726
54.3	Green kernels and physics-informed covariance	726
54.4	Gaussian processes with differential observations	727
54.5	Probabilistic numerical methods	727
54.6	Inverse scientific problems	727
54.7	Learning operators between function spaces	728
54.8	Common mistakes and practical implications	728
54.9	Summary and further reading	729
54.10	Exercises	729
55	Reproducing-Kernel Banach and Variation Spaces	731
55.1	What the Hilbert structure was buying	731
55.2	Semi-inner products and duality mappings	731
55.3	Banach-space representer theorems	732
55.4	Nonsmooth norms and sparse atoms	732
55.5	Variation spaces and ridge splines	733
55.6	Algorithms: from duality to column generation	733
55.7	Statistical and computational tradeoffs	734
55.8	Common mistakes and practical implications	734
55.9	Summary and further reading	734
55.10	Exercises	734
56	Accountable Kernels: Uncertainty, Explanation, and Audit	735
56.1	Accountability is a property of the model, not a wrapper	735
56.2	Uncertainty you can put in a report	736
56.2.1	The Gaussian-process interval, and its honest limits	736
56.2.2	Conformal prediction: a distribution-free coverage guarantee	737
56.3	Explanation for free: the representer theorem as attribution	738
56.3.1	Prediction as a weighted vote over training points	738
56.3.2	Exact influence and closed-form leave-one-out	739
56.4	Auditing a deployed kernel model	740
56.4.1	Is the input still the training distribution? Kernel drift monitoring	740
56.4.2	Independence and fairness audits with HSIC	741
56.4.3	Reproducibility and the audit trail	742
56.5	Uncertainty that drives the next experiment	742

56.6 Summary	743
56.7 Exercises	744
57 Kernels in Science and Space	745
57.1 Why these domains demand accountable models	745
57.2 Space and astronomy	745
57.2.1 Stellar rotation and exoplanet light curves	745
57.2.2 Disentangling a planet from stellar activity	747
57.2.3 Gravitational waves: the noise-weighted inner product is a kernel	747
57.2.4 Spacecraft telemetry and orbit uncertainty	748
57.3 Molecules and materials	748
57.3.1 Potentials from a kernel that knows the symmetries	748
57.3.2 The killer app: variance triggers a calculation	749
57.3.3 Bayesian optimization for discovery	750
57.4 Earth observation	751
57.5 What the cases share	751
57.6 Summary	752
57.7 Exercises	752

1 Mathematical Preliminaries

Kernel methods sit at a crossroads of linear algebra, real and functional analysis, probability, and convex optimization, and the book's later parts reach further, into spectral graph theory, groups and their actions, couplings of measures, the tensor calculus of paths, and inner products without positivity. This chapter gathers, in one place, the results the rest of the book uses by name. It is a reference, not a course: each section states the definitions and facts a later chapter leans on, with just enough intuition to make them usable. Read it straight through for a refresher, or jump to a result when a chapter cites it.

1.1 Linear algebra and matrices

The Gram matrix is the object every kernel algorithm actually computes with, so the linear algebra of symmetric matrices is the bedrock of the whole subject. We work in $\mathbb{R}^d$ with the standard inner product $\langle x, y \rangle = x^\top y = \sum_{i=1}^d x_i y_i$ and Euclidean norm $\|x\| = \sqrt{x^\top x}$.

Symmetric and positive semidefinite matrices

A real matrix $A \in \mathbb{R}^{n \times n}$ is **symmetric** if $A = A^\top$. A symmetric A is **positive semidefinite** (PSD), written $A \geq 0$, if $v^\top A v \geq 0$ for every $v \in \mathbb{R}^n$; it is **strictly positive definite** (or just positive definite in the matrix sense), written $A > 0$, if the inequality is strict, $v^\top A v > 0$ for every $v \neq 0$. The two are genuinely different properties: a PSD matrix may be singular, while a strictly positive definite one is invertible.

A word of caution on terminology, because the two adjacent uses of "positive definite" are a standing source of confusion. In the kernel-methods literature we follow Aronszajn's convention: a **positive definite kernel** is one whose every Gram matrix $K_{ij} = k(x_i, x_j)$ is positive semidefinite ($K \geq 0$). So the phrase "positive definite kernel," used throughout this book, produces PSD Gram matrices, not strictly positive definite ones. When we need the stronger matrix property, that a particular Gram matrix is invertible, we say **strictly positive definite** explicitly. Positive semidefiniteness is thus the matrix shadow of a valid kernel: a kernel is positive definite (Aronszajn) exactly when every Gram matrix it produces is PSD.

Spectral theorem (symmetric case)

Every symmetric $A \in \mathbb{R}^{n \times n}$ has an orthonormal basis of eigenvectors: there is an orthogonal matrix U (so $U^\top U = I$) and a real diagonal matrix $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ with

$$A = U \Lambda U^\top = \sum_{k=1}^n \lambda_k u_k u_k^\top.$$

The eigenvalues λ_k are real. $A \geq 0$ iff all $\lambda_k \geq 0$, and $A > 0$ iff all $\lambda_k > 0$. This decomposition is the finite-dimensional ancestor of Mercer's theorem: the eigenvectors are the discrete eigenfunctions, and $\sqrt{\lambda_k} u_k$ gives an explicit feature map.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Several consequences recur. A PSD matrix has a unique PSD **square root** $A^{1/2} = U \Lambda^{1/2} U^\top$, and it factors as $A = B^\top B$ (take $B = \Lambda^{1/2} U^\top$); conversely any $B^\top B$ is PSD, which is why a Gram matrix $K = \Phi \Phi^\top$ is always PSD. The **trace** $\text{tr}(A) = \sum_i A_{ii} = \sum_k \lambda_k$ is the sum of eigenvalues and is cyclic, $\text{tr}(AB) = \text{tr}(BA)$; the **determinant**

$\det(A) = \prod_k \lambda_k$. The **rank** is the number of nonzero eigenvalues. For the eigenvalue extremes, the **Rayleigh quotient** gives

$$\lambda_{\max}(A) = \max_{v \neq 0} \frac{v^T A v}{v^T v}, \quad \lambda_{\min}(A) = \min_{v \neq 0} \frac{v^T A v}{v^T v},$$

and the higher eigenvalues follow the Courant-Fischer min-max principle. Maximizing a Rayleigh quotient is exactly what kernel PCA and spectral clustering do.

Singular value decomposition and pseudo-inverse

Any $M \in \mathbb{R}^{m \times n}$ factors as $M = U \Sigma V^T$ with U, V orthogonal and Σ diagonal with nonnegative **singular values** $\sigma_1 \geq \dots \geq 0$. When M is not invertible, the **Moore-Penrose pseudo-inverse** $M^+ = V \Sigma^+ U^T$ (inverting the nonzero singular values) gives the minimum-norm least-squares solution of $Mx = b$; it is what "solving" a rank-deficient normal equation means.

Numerical solvability is not the same as algebraic invertibility. For a strictly positive definite symmetric matrix, the spectral **condition number** is $\kappa_2(A) = \lambda_{\max}(A)/\lambda_{\min}(A)$. Relative perturbations can be amplified by roughly this factor. Kernel matrices with repeated or nearly repeated observations can be positive definite yet numerically unusable. Prefer a Cholesky or QR solve to forming A^{-1} , add a documented ridge or jitter only when the model permits it, and report the residual in the original system.

Two identities save real computation in the book. The **matrix inversion lemma** (Sherman-Morrison-Woodbury),

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1},$$

lets kernel ridge regression switch between an $n \times n$ solve in dual variables and a $d \times d$ solve in primal variables, whichever is smaller. The **Schur (Hadamard) product** $A \circ B$, the entrywise product, preserves positive semidefiniteness: if $A, B \geq 0$ then $A \circ B \geq 0$. That single fact is why the product of two kernels is a kernel.

1.2 Real analysis, metrics, and Hilbert spaces

Feature spaces are infinite-dimensional, so the finite intuition above has to survive a limit. The right setting is a Hilbert space, and the facts below are what make an infinite-dimensional "space of features" behave like Euclidean space.

Metric and normed spaces; completeness

A **metric** d on a set X is a nonnegative, symmetric function with $d(x, y) = 0 \iff x = y$ that satisfies the triangle inequality $d(x, z) \leq d(x, y) + d(y, z)$. A **norm** $\|\cdot\|$ on a vector space induces the metric $d(x, y) = \|x - y\|$. A sequence is **Cauchy** if its terms get arbitrarily close to each other; a space is **complete** if every Cauchy sequence converges to a point of the space. Completeness is the property that rules out "holes," and it is exactly the condition an inner-product space must add to become a Hilbert space.

Inner product and Hilbert space

An **inner product** $\langle \cdot, \cdot \rangle$ on a real vector space H is symmetric, linear in each argument, and positive: $\langle f, f \rangle \geq 0$ with equality only at $f = 0$. It induces the norm $\|f\| = \sqrt{\langle f, f \rangle}$. A **Hilbert space** is an inner-product space that is complete in this norm. $\mathbb{R}^d$ and the sequence space ℓ^2 of square-summable sequences are the model examples; a reproducing kernel Hilbert space is a Hilbert space of functions.

Three inequalities are used constantly. **Cauchy-Schwarz**, $|\langle f, g \rangle| \leq \|f\| \|g\|$, with equality iff f and g are parallel, is the workhorse behind nearly every bound in the book (it turns the reproducing property into the pointwise bound $|f(x)| \leq \|f\| \sqrt{k(x, x)}$). The **triangle inequality** $\|f + g\| \leq \|f\| + \|g\|$ follows from it. And the **Pythagorean identity** $\|f + g\|^2 = \|f\|^2 + \|g\|^2$ when $\langle f, g \rangle = 0$ drives the orthogonal-decomposition proof of the representer

theorem. All three lean on positivity of the inner product; the section on indefinite inner products at the end of this chapter records what breaks when positivity is dropped.

Orthogonal projection and the Riesz representation theorem

In a Hilbert space, any closed subspace V admits a unique **orthogonal projection**: every f splits as $f = f_V + f_\perp$ with $f_V \in V$, $f_\perp \perp V$, and f_V the closest point of V to f . The **Riesz representation theorem** says every continuous linear functional $L : H \rightarrow \mathbb{R}$ is an inner product against a unique vector: $L(f) = \langle f, g_L \rangle$ for some $g_L \in H$. Applied to the evaluation functional $f \mapsto f(x)$, Riesz produces the reproducing kernel; that is the definitional route to an RKHS.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

A few pointwise-analysis notions round this out. A function is **continuous** if small input changes give small output changes, and **Lipschitz** with constant L if $|f(x) - f(y)| \leq L d(x, y)$. For an RKHS function the reproducing property and Cauchy-Schwarz give the exact bound $|f(x) - f(x')| \leq \|f\|_{\mathcal{H}} d_k(x, x')$, where $d_k(x, x')^2 = k(x, x) - 2k(x, x') + k(x', x')$ is the metric induced by the kernel's own feature map. This is automatic and needs no smoothness of k . Turning it into Lipschitz-ness in the ordinary metric $d(x, x')$, so that $\|f\|_{\mathcal{H}}$ bounds a genuine Lipschitz constant, needs a further hypothesis on the kernel: that k is smooth enough (for instance differentiable with bounded gradient) that $d_k(x, x') \leq C d(x, x')$. Under that hypothesis the RKHS norm bounds a Lipschitz constant, which is the precise sense in which "small norm means smooth." A set is **compact** if every sequence has a convergent subsequence (on $\mathbb{R}^d$, compact means closed and bounded); compactness of the domain is the hypothesis under which Mercer's theorem holds. A basis (e_k) of a Hilbert space is **orthonormal** if $\langle e_i, e_j \rangle = \delta_{ij}$, and then $f = \sum_k \langle f, e_k \rangle e_k$ with $\|f\|^2 = \sum_k \langle f, e_k \rangle^2$ (Parseval); the eigenfunction expansion of Mercer's theorem is one such basis.

1.3 Measure, integration, and Hilbert-valued expectation

A probability space is a triple $(\Omega, \mathcal{F}, P)$: $\mathcal{F}$ is a sigma-algebra of events and P is a countably additive measure with total mass one. A random variable is a measurable map. Statements that hold **almost everywhere** may fail on a set of measure zero. The spaces $L^p(P)$ identify functions that agree almost everywhere and satisfy $\int |f|^p dP < \infty$; conditional expectations live in L^2 , while kernel mean existence begins with an L^1 condition.

Three exchange rules recur. Tonelli permits swapping integrals of a nonnegative measurable function. Fubini permits the same for an absolutely integrable function. Dominated convergence permits moving a limit through an integral when the sequence converges almost everywhere and is bounded by an integrable envelope. Each rule has a hypothesis; expectation notation alone does not make an interchange legal.

Bochner integrability

Let $Z : \Omega \rightarrow \mathcal{H}$ take values in a separable Hilbert space. If Z is strongly measurable and $\mathbb{E}\|Z\|_{\mathcal{H}} < \infty$, its **Bochner expectation** $\mathbb{E}Z \in \mathcal{H}$ is the norm limit of expectations of simple functions. It satisfies

$$\|\mathbb{E}Z\|_{\mathcal{H}} \leq \mathbb{E}\|Z\|_{\mathcal{H}}, \quad A(\mathbb{E}Z) = \mathbb{E}[AZ]$$

for every bounded linear operator A . For $Z = k(X, \cdot)$, a sufficient condition is $\mathbb{E}\sqrt{k(X, X)} < \infty$, and $\mathbb{E}Z$ is the kernel mean embedding.

1.4 Operators, spectra, and Fourier analysis

Mercer's theorem and the translation-invariant kernels of Part V need a little operator theory and harmonic analysis. The finite spectral theorem generalizes to certain operators on function spaces, and Fourier analysis is what characterizes which functions are valid kernels.

Bounded, compact, self-adjoint, positive operators

A linear operator $T : H \rightarrow H$ is **bounded** if $\|Tf\| \leq C\|f\|$ for some C ; **self-adjoint** if $\langle Tf, g \rangle = \langle f, Tg \rangle$ (the operator analogue of a symmetric matrix); **positive** if $\langle Tf, f \rangle \geq 0$; and **compact** if it maps bounded sets to sets whose closure is compact. Compactness is the infinite-dimensional substitute for finite rank: it is what forces a discrete spectrum.

Spectral theorem for compact self-adjoint operators

A compact self-adjoint operator T on a Hilbert space has a countable orthonormal system of eigenfunctions (ψ_k) with real eigenvalues $\lambda_k \rightarrow 0$, and $Tf = \sum_k \lambda_k \langle f, \psi_k \rangle \psi_k$. The **integral operator** $(T_k f)(x) = \int k(x, y) f(y) d\nu(y)$ of a continuous PSD kernel on a compact domain is compact, self-adjoint, and positive, so this theorem applies; its eigenvalues and eigenfunctions are precisely the pieces of Mercer's decomposition $k(x, y) = \sum_k \lambda_k \psi_k(x) \psi_k(y)$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

For kernels that depend only on the difference $x - y$, the relevant spectrum is the Fourier one. The **Fourier transform** of an integrable function f on $\mathbb{R}^d$ is $\hat{f}(\omega) = \int f(x) e^{-i\omega^\top x} dx$; it diagonalizes convolution and translation. A finite nonnegative measure has a Fourier transform that is a positive definite function, and **Bochner's theorem** is the converse: a continuous translation-invariant function $k(x, y) = \kappa(x - y)$ is positive definite exactly when κ is the Fourier transform of a finite nonnegative measure. That characterization is the source of the Gaussian and Laplace kernels and the starting point for random Fourier features.

1.5 Probability and statistics

Learning is estimation from samples, so probability supplies both the objects (distributions, expectations) and the guarantees (concentration). We treat data as independent draws from an unknown distribution and ask what can be inferred.

Distributions, expectation, and independence

A random variable X has a distribution P ; its **expectation** is $\mathbb{E}[X] = \int x dP(x)$, a linear operator, and its **variance** is $\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2]$. Variables are **independent** if their joint distribution factors; a sample is **i.i.d.** when its points are independent and identically distributed. The kernel mean embedding $\mu_P = \mathbb{E}_{X \sim P}[k(\cdot, X)]$ of Part VI is just an expectation taken in an RKHS, and the maximum mean discrepancy compares two such embeddings.

Two families of results are used to turn samples into guarantees. The **laws of large numbers** say the empirical average $\frac{1}{n} \sum_i X_i$ converges to $\mathbb{E}[X]$ as $n \rightarrow \infty$, and the **central limit theorem** says its fluctuations are Gaussian of size $1/\sqrt{n}$; this $1/\sqrt{n}$ rate is the one that appears in every generalization bound. Sharper, non-asymptotic control comes from **concentration inequalities**. **Hoeffding's inequality** bounds the deviation of a bounded average, and its generalization, **McDiarmid's bounded-differences inequality**, bounds any function of independent variables that changes little when one variable is altered:

$$\Pr(f(X_1, \dots, X_n) - \mathbb{E}f \geq t) \leq \exp\left(\frac{-2t^2}{\sum_i c_i^2}\right),$$

where c_i bounds the change from moving coordinate i . McDiarmid applied to the worst-case empirical error is the engine behind the Rademacher bounds of Chapter 11, Learning Theory in RKHS Balls. A last modeling link: minimizing a loss is often maximum-likelihood estimation in disguise, since a loss $\ell(y, f)$ equal to $-\log p(y | f)$ makes empirical risk minimization the same as fitting a probabilistic noise model, the reading that connects the squared loss to Gaussian noise and the logistic loss to a Bernoulli model.

This section covers what empirical risk minimization needs. The inference chapters ask for more, and the deeper material, Gaussian conditioning, Bayes' rule, score functions, U-statistics, and a precise statement of Hoeffding's inequality, is collected in the section on conditioning, scores, and U-statistics below.

1.6 Convex optimization and duality

Fitting a kernel machine is solving an optimization problem, and almost always a convex one, which is why the solutions are unique and computable. The vocabulary of convexity and Lagrangian duality is used throughout Parts II and VII.

Convex sets, convex functions, and gradients

A set is **convex** if it contains the segment between any two of its points. A function f is **convex** if its graph lies below its chords, $f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$; equivalently, for differentiable f , the graph lies above every tangent, $f(y) \geq f(x) + \nabla f(x)^\top (y - x)$. A convex function has no local minima other than global ones, so gradient methods find the optimum. The **gradient** ∇f is the vector of partial derivatives; where f is not differentiable, a **subgradient** is any g with $f(y) \geq f(x) + g^\top (y - x)$, which is how the hinge loss is handled at its kink.

Lagrangian duality and the KKT conditions

For a constrained problem $\min_x f(x)$ subject to inequality constraints $g_i(x) \leq 0$ and equality constraints $h_j(x) = 0$, the **Lagrangian** is $L(x, \alpha, \nu) = f(x) + \sum_i \alpha_i g_i(x) + \sum_j \nu_j h_j(x)$, where the inequality multipliers satisfy $\alpha_i \geq 0$ while the equality multipliers ν_j are unrestricted in sign. The **dual function** $q(\alpha, \nu) = \min_x L(x, \alpha, \nu)$ lower-bounds the optimum (weak duality), and for convex problems meeting a regularity condition the bound is tight (strong duality). At an optimum the **Karush-Kuhn-Tucker (KKT) conditions** hold:

$$\nabla f(x) + \sum_i \alpha_i \nabla g_i(x) + \sum_j \nu_j \nabla h_j(x) = 0 \quad (\text{stationarity}),$$

$$g_i(x) \leq 0, \quad h_j(x) = 0 \quad (\text{primal feasibility}),$$

$$\alpha_i \geq 0 \quad (\text{dual feasibility}), \quad \alpha_i g_i(x) = 0 \quad (\text{complementary slackness}).$$

Complementary slackness is the operative one for kernel machines: the SVM's support vectors are exactly the points whose inequality multiplier α_i is nonzero, read straight off this condition.

The algorithms that solve these problems at scale are covered in Chapter 37, Large-Scale Kernel Machines, but their basics belong here. **Gradient descent** steps against the gradient, $x_{t+1} = x_t - \eta \nabla f(x_t)$, and converges on convex, smooth objectives; **stochastic gradient descent** replaces the full gradient with a cheap unbiased estimate from one example or a minibatch, trading exactness for a per-step cost independent of n . Smoothness (a Lipschitz gradient) and strong convexity (a curvature lower bound) are the two properties that set the convergence rate. Finally, a **quadratic program**, minimizing a convex quadratic subject to linear constraints, is the exact form of the SVM dual, and interior-point and decomposition methods are how it is solved.

The **Fenchel conjugate** of a proper convex function f is $f^*(u) = \sup_x \{\langle u, x \rangle - f(x)\}$. Fenchel-Young gives $f(x) + f^*(u) \geq \langle u, x \rangle$, with equality exactly when $u \in \partial f(x)$. This is the compact route from a primal loss to

its dual constraint. Strong Fenchel duality still needs a qualification condition; convexity alone does not erase a duality gap.

1.7 More probability: conditioning, scores, and U-statistics

The distributional chapters of the book do inference, not only estimation: they condition one Gaussian block on another, differentiate log-densities they cannot normalize, and average symmetric functions over pairs of samples. The five facts below are their toolkit.

Conditioning a joint Gaussian

Partition a Gaussian vector as $x = (x_1, x_2) \sim \mathcal{N}(\mu, \Sigma)$ with

$$\mu = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}, \quad \Sigma_{22} > 0.$$

Then conditionally on x_2 , the block x_1 is again Gaussian, with

$$\mathbb{E}[x_1 \mid x_2] = \mu_1 + \Sigma_{12} \Sigma_{22}^{-1} (x_2 - \mu_2), \quad \text{Cov}(x_1 \mid x_2) = \Sigma_{11} - \Sigma_{12} \Sigma_{22}^{-1} \Sigma_{21}.$$

The conditional mean is affine in the observed block, and the conditional covariance is the **Schur complement** of Σ_{22} in Σ ; it does not depend on the observed value at all.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Read with Σ a kernel Gram matrix over training and test points, this theorem is Gaussian process regression: the posterior mean is a linear smoother in the observations and the posterior variance is a Schur complement, which is how Chapter 40, Gaussian Processes and the RVM derives its predictive equations and how the acquisition rules of Chapter 41, Kernelized Bandits and Bayesian Optimization know where they are uncertain. The Schur complement is also the block that appears when the matrix inversion lemma of the linear-algebra section inverts a partitioned matrix.

Bayes' rule and conditional expectation

For random variables X, Y with joint density $p(x, y)$, the **conditional density** of y given x is $p(y \mid x) = p(x, y)/p(x)$ wherever $p(x) > 0$, and **Bayes' rule** reverses the conditioning,

$$p(y \mid x) = \frac{p(x \mid y) p(y)}{\int p(x \mid y') p(y') dy'}.$$

The **conditional expectation** $\mathbb{E}[f(Y) \mid X = x] = \int f(y) p(y \mid x) dy$ is, as a function of x , the best mean-square predictor of $f(Y)$: it minimizes $\mathbb{E}[(f(Y) - g(X))^2]$ over all measurable g , and it obeys the tower property $\mathbb{E}[\mathbb{E}[f(Y) \mid X]] = \mathbb{E}[f(Y)]$.

The conditional mean embeddings of Chapter 33, Conditional Mean Embeddings and Kernel Bayes' Rule represent the map $x \mapsto \mathbb{E}[\varphi(Y) \mid X = x]$ as an RKHS-valued regression and turn Bayes' rule into linear algebra on Gram matrices; the adjustment formulas of Chapter 35, Causal Inference with Kernels are conditional expectations averaged over confounders.

The score function and integration by parts

Let p be a continuously differentiable, everywhere positive density on $\mathbb{R}^d$. Its **score function** is

$$s_p(x) = \nabla_x \log p(x) = \frac{\nabla p(x)}{p(x)},$$

which is unchanged if p is multiplied by a constant: the score is computable even when the normalizer is not. If f is differentiable, $\mathbb{E}_p \|s_p(X) f(X)\|$ and $\mathbb{E}_p \|\nabla f(X)\|$ are finite, and $p(x) f(x) \rightarrow 0$ as $\|x\| \rightarrow \infty$, then

$$\mathbb{E}_p [s_p(X) f(X) + \nabla f(X)] = 0.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

In each coordinate i , $\mathbb{E}_p [\partial_i \log p \cdot f + \partial_i f] = \int ((\partial_i p) f + p \partial_i f) dx = \int \partial_i (p f) dx = 0$, the last step by integration by parts using the decay of $p f$ at infinity. $\square$

The identity says the operator $f \mapsto s_p f + \nabla f$ has mean zero under p for a whole class of test functions, a fingerprint of p that never touches its normalizing constant; running the test functions over an RKHS ball turns the largest violated mean into the computable discrepancy of Chapter 34, Kernel Stein Discrepancy and Stein Methods.

U-statistics and V-statistics

Let $h(x, x')$ be a symmetric function, called the **kernel** of the statistic (an older usage, distinct from a positive definite kernel), and let $X_1, \dots, X_n$ be i.i.d. from P . The **U-statistic** and **V-statistic** with kernel h are

$$U_n = \frac{1}{n(n-1)} \sum_{i \neq j} h(X_i, X_j), \quad V_n = \frac{1}{n^2} \sum_{i,j=1}^n h(X_i, X_j).$$

Every term of U_n has expectation $\theta = \mathbb{E}[h(X, X')]$ for independent $X, X' \sim P$, so the U-statistic is an **unbiased** estimator of θ for every n . The V-statistic keeps the diagonal terms $h(X_i, X_i)$ and is biased by $O(1/n)$, in exchange for being a plug-in: V_n is θ computed under the empirical distribution. Kernels of order m are averaged over m -tuples of distinct indices in the same way.

The squared maximum mean discrepancy of Chapter 29, Mean Embeddings, MMD, and Characteristic Kernels is estimated exactly this way, as a two-sample U- or V-statistic whose kernel is built from k ; the null distributions that calibrate the tests of Chapter 30, Kernel Hypothesis Testing come from the degenerate case of U-statistic limit theory, where the usual Gaussian limit collapses and a weighted sum of χ^2 variables takes its place.

Hoeffding's inequality

Let $X_1, \dots, X_n$ be independent with $X_i \in [a_i, b_i]$ almost surely, and let $\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$. For every $t > 0$,

$$\Pr(\bar{X} - \mathbb{E}[\bar{X}] \geq t) \leq \exp\left(\frac{-2n^2 t^2}{\sum_{i=1}^n (b_i - a_i)^2}\right),$$

and the same bound holds for the deviation below the mean. When all ranges equal $[a, b]$ the exponent is $-2nt^2/(b-a)^2$: a bounded average deviates by more than t with probability exponentially small in n .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

McDiarmid's bounded-differences inequality, displayed in the probability section above, generalizes this from averages to any stable function of independent variables (take $f = \bar{X}$, so $c_i = (b_i - a_i)/n$); it is proved in Chapter 12, VC Theory and Generalization. Hoeffding is the bound behind the single-function deviation estimates of Chapter 11, Learning Theory in RKHS Balls and the sampling-error control of the randomized approximations in Chapter 37, Large-Scale Kernel Machines.

1.8 Graphs, manifolds, and groups

Several chapters replace $\mathbb{R}^d$ by a structured domain: a similarity graph built on the sample, a curved surface, or a space acted on by a symmetry group. One matrix and one averaging construction cover most of what they need.

The graph Laplacian

Let $W = (w_{ij})$ be symmetric nonnegative edge weights on vertices $\{1, \dots, n\}$, with degrees $d_i = \sum_j w_{ij}$ and degree matrix $D = \text{diag}(d_1, \dots, d_n)$. The **graph Laplacian** is $L = D - W$. Its quadratic form is

$$f^\top L f = \frac{1}{2} \sum_{i,j=1}^n w_{ij} (f_i - f_j)^2 \geq 0,$$

so L is symmetric PSD, and $f^\top L f$ measures how much f varies across heavy edges. The constant vector satisfies $L\mathbf{1} = 0$, and the multiplicity of the eigenvalue 0 is the number of connected components. The **normalized Laplacians** $L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}$ and $L_{\text{rw}} = I - D^{-1} W$ (defined when all $d_i > 0$) rescale by degree and inherit the same structure.

Eigenvectors of L with small eigenvalue are the smoothest functions on the graph, which is why spectral clustering in Chapter 14, Kernel Clustering and Spectral Methods cuts along them; matrix functions of L such as the heat kernel e^{-tL} and the regularized inverse are the kernels on graphs of Chapter 24, Kernels for and on Graphs; and L is the discrete stand-in for the manifold operator of the next paragraph in Chapter 27, Geometric and Equivariant Kernels.

On a smooth compact Riemannian manifold the role of L is played by the **Laplace-Beltrami operator**, the divergence of the gradient computed in the manifold's own metric. It is self-adjoint and positive, its spectrum is discrete, and its eigenfunctions generalize the Fourier modes: on the circle they are the sines and cosines, on the sphere the spherical harmonics. Heat and Matérn kernels on manifolds are built from these eigenpairs in Chapter 27, Geometric and Equivariant Kernels, and graph Laplacians estimated from samples approximate the operator, the reading behind the spectral embeddings of Chapter 17, Data Visualization and Kernel MDS.

Group actions, invariance, and averaging

A group G **acts** on a set X when each $g \in G$ is assigned a transformation $x \mapsto g \cdot x$ with $e \cdot x = x$ and $g \cdot (h \cdot x) = (gh) \cdot x$. A function f on X is **invariant** if $f(g \cdot x) = f(x)$ for every g and x . For a finite group, the **averaging operator**

$$(\bar{f})(x) = \frac{1}{|G|} \sum_{g \in G} f(g \cdot x)$$

always produces an invariant function, fixes every function that is already invariant, and, when the action preserves the underlying measure, is idempotent and self-adjoint: it is the orthogonal projection onto the subspace of invariant functions. For a compact group the sum becomes an integral against the **Haar measure**, the unique probability measure on G invariant under the group's own translations, and the projection statement is unchanged.

Averaging a kernel over the action in each argument yields an invariant kernel whose RKHS consists of invariant functions, the construction at the center of Chapter 27, Geometric and Equivariant Kernels; the virtual-sample and jittered kernels of Chapter 20, Invariances and the Pre-Image Problem are this projection applied approximately, over a subset of the group.

1.9 Couplings and linear programs

Comparing two distributions by physically moving the mass of one onto the other requires a name for a joint plan of movement.

Couplings of two probability measures

A **coupling** of probability measures P on X and Q on Y is a probability measure π on the product $X \times Y$ whose marginals are P and Q : $\pi(A \times Y) = P(A)$ and $\pi(X \times B) = Q(B)$ for all measurable A, B . The set $\Pi(P, Q)$ of couplings is convex, since a mixture of couplings has the same marginals, and it is never empty: the **independent coupling** $P \otimes Q$ always belongs to it.

Optimal transport selects the coupling minimizing an expected cost $\mathbb{E}_\pi[c(X, Y)]$; for empirical measures this is a linear program over transport matrices with fixed row and column sums, the problem Chapter 31, Optimal Transport and Kernels solves, regularizes, and compares against the MMD. A linear program is the Lagrangian framework of the optimization section with every function affine: its dual is again a linear program, strong duality holds with no extra regularity, and dualizing the marginal constraints of the transport program is precisely what produces the Kantorovich dual used there.

1.10 Paths, tensors, and iterated integrals

A time series is a path, and the natural polynomial features of a path are integrals against its own increments, which live in tensor powers of the state space. The **tensor product** $V \otimes W$ of two vector spaces is the space of formal bilinear combinations of symbols $v \otimes w$; for $V = W = \mathbb{R}^d$ with basis (e_i) , the m -fold power $(\mathbb{R}^d)^{\otimes m}$ has basis $e_{i_1} \otimes \cdots \otimes e_{i_m}$, so its elements are m -way arrays of d^m coordinates.

Iterated integrals of a path

Let $x : [0, T] \rightarrow \mathbb{R}^d$ be a continuous path of bounded variation, so that integration against its increments dx_t is well defined. For each multi-index $(i_1, \dots, i_m) \in \{1, \dots, d\}^m$, the **iterated integral** of x is the number

$$S^{(i_1, \dots, i_m)}(x) = \int_{0 < t_1 < t_2 < \dots < t_m < T} dx_{t_1}^{i_1} dx_{t_2}^{i_2} \cdots dx_{t_m}^{i_m},$$

the integral over the ordered simplex of times. For $m = 1$ these are the total increments $x_T^i - x_0^i$; for $m = 2$ they record the ordered co-movement of pairs of coordinates. Collecting the degree- m integrals into a tensor $S^{(m)}(x) \in (\mathbb{R}^d)^{\otimes m}$ for every m gives the raw material of the path's **signature**.

The signature $(1, S^{(1)}(x), S^{(2)}(x), \dots)$ is a feature expansion of the path that is invariant to time reparameterization and determines the path up to a natural equivalence; inner products of signatures define the sequence kernels of Chapter 26, Signature and Sequence-Path Kernels, computed there by recursion on the data rather than by materializing any tensor.

1.11 Indefinite inner products

Some natural similarity functions fail positive semidefiniteness, and the geometry that accommodates them keeps the bilinear algebra of an inner product while giving up positivity.

Indefinite and Krein inner products

An **indefinite inner product** on a vector space $\mathcal{K}$ is a symmetric bilinear form $\langle \cdot, \cdot \rangle_{\mathcal{K}}$ with no positivity requirement, so $\langle f, f \rangle_{\mathcal{K}}$ may be negative, or zero for $f \neq 0$. The workable case is a **Krein space**: $\mathcal{K}$ decomposes as an orthogonal direct sum of two Hilbert spaces $\mathcal{H}_+$ and $\mathcal{H}_-$ with

$$\langle f, g \rangle_{\mathcal{K}} = \langle f_+, g_+ \rangle_{\mathcal{H}_+} - \langle f_-, g_- \rangle_{\mathcal{H}_-},$$

a difference of two genuine inner products.

Cauchy-Schwarz fails outright: $\langle f, f \rangle_{\mathcal{K}}$ can vanish on nonzero (neutral) vectors, so it no longer induces a norm and nothing bounds $|\langle f, g \rangle_{\mathcal{K}}|$; topology and projections must instead be borrowed from the associated Hilbert space $\mathcal{H}_+ \oplus \mathcal{H}_-$. Kernels that decompose as a difference $k_+ - k_-$ of two positive definite kernels reproduce in exactly such spaces, the subject of Chapter 28, Indefinite and Krein-Space Kernels.

1.12 Numerical linear algebra clinic

Exact identities do not specify stable algorithms. Normal equations square the condition number, explicit inverses waste work and magnify rounding error, and a small ridge changes the statistical problem even when introduced as “jitter.” For every Gram solve, distinguish four quantities: backward error, forward coefficient error, prediction error, and optimization residual. They need not move together.

A dependable workflow scales features, inspects the Gram diagonal and a spectral estimate, uses a factorization or matrix-free solver appropriate to definiteness, and checks the residual in the original system. Cholesky is preferred for a well-conditioned positive-definite matrix; pivoted Cholesky or eigendecomposition exposes low rank; QR is safer for rectangular least squares; iterative methods require a stopping rule and preconditioner. The scaling chapter Chapter 37, Large-Scale Kernel Machines turns these choices into algorithms.

1.13 Measure and probability clinic

Three distinctions prevent many later errors. Almost-sure equality is not pointwise equality; convergence in probability is weaker than almost-sure convergence and does not by itself permit exchanging a limit and expectation; and an expectation under Q can be rewritten under P only when Q is absolutely continuous with respect to P . In that last case the Radon-Nikodym derivative dQ/dP is the abstract density ratio behind importance weighting.

Conditional expectation is defined relative to a sigma-algebra, even when no density exists. Regular conditional distributions need assumptions on the measurable spaces. Hilbert-valued expectations need measurability and norm integrability, not only formal linearity. When a proof moves an operator, limit, derivative, or supremum through an expectation, name the theorem that permits it.

1.14 Functional and convex analysis clinic

Finite-dimensional intuition fails in two recurring ways. Bounded closed sets need not be compact in an infinite-dimensional norm topology, and a bounded sequence can converge weakly without converging in norm. Compact operators convert some weak convergence into strong convergence and make inverse problems spectrally tractable, but their inverses are typically unbounded on their ranges.

For convex programs, KKT conditions are necessary and sufficient only under the relevant convexity and qualification conditions. Slater’s condition is a common sufficient qualification for convex inequality constraints. Existence also needs coercivity, compactness, or lower-semicontinuity plus an appropriate compactness argument. Algorithmic convergence adds smoothness, step-size, stochastic-noise, and stopping assumptions beyond the existence of an optimum.

1.15 Structured-domain clinic

Graphs, manifolds, groups, tensors, and paths each have more than one natural normalization. A graph Laplacian depends on the affinity scale and degree convention; a manifold kernel depends on metric, boundary, and volume measure; group averaging depends on the action and invariant measure; tensor features grow exponentially with degree; path signatures depend on augmentation and truncation. State these choices before invoking a theorem.

The test for a structured kernel remains the ordinary Gram test: for every finite input set, the matrix must be PSD. Structure supplies principled constructions, not an exemption from positivity, measurability, conditioning, or validation.

1.16 Summary and further reading

Kernel methods reduce nonlinear learning to linear algebra in Hilbert spaces, but the reduction remains sound only when its analytic hypotheses travel with it. Spectra govern geometry and conditioning; bounded evaluation and projection produce RKHS representations; measure theory and Bochner integration make distribution embeddings well defined; convex duality turns losses into coefficient problems; and structured domains contribute their own operators and symmetries. Readers needing proof-level measure theory, functional analysis, or convex analysis should consult a dedicated graduate text before relying on an interchange, compactness claim, or duality theorem.

1.17 Notation and conventions

A few conventions hold throughout the book. Vectors are columns; $x^\top y$ is an inner product and $xy^\top$ an outer product. $\mathbf{K}$ or K is the Gram matrix with entries $K_{ij} = k(x_i, x_j)$; $\mathcal{H}$ is a reproducing kernel Hilbert space with inner product $\langle \cdot, \cdot \rangle_{\mathcal{H}}$ and norm $\|\cdot\|_{\mathcal{H}}$; Φ is a feature map with $k(x, x') = \langle \Phi(x), \Phi(x') \rangle$. Training data is $(x_i, y_i)_{i=1}^n$, and n is the sample size, d the input dimension. The symbols λ (a regularization weight or an eigenvalue), α (dual coefficients), and σ (a kernel bandwidth) recur; where an overload could confuse, the surrounding text disambiguates. The full symbol table and a glossary of terms are collected in the [Notation and Glossary](#).

How to use this chapter

Nothing here is proved in full (the one-line integration by parts above is the exception); each result is stated so a later chapter can cite it and keep moving. When a chapter writes "by the spectral theorem" or "Bochner's theorem gives," this is where the statement lives. The first five sections are the core, used from the opening chapters onward. The sections from conditioning, scores, and U-statistics through indefinite inner products serve the later parts of the book, on probabilistic inference, structured domains, transport, paths, and indefinite geometry; read them when the consuming chapter sends you here. Standard references for the material are, for linear algebra and analysis, any graduate text; for the probability and optimization used in learning, the treatments in the three sources this book synthesizes, and in particular the statistical-learning and optimization chapters of Schölkopf and Smola (2002) and the appendices of Shawe-Taylor and Cristianini (2004).

1.18 Common mistakes and practical implications

For **Mathematical Preliminaries**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

1.19 Exercises

1. computation A Gram matrix has eigenvalues $10, 1, 10^{-8}$. Compute its condition number, then compute the condition number after adding ridge $10^{-3}I$. Explain why the regularized solve is more stable.
2. proof Starting from the projection theorem, prove the Pythagorean identity for $f = f_V + f_\perp$ and identify the exact step used in a representer-theorem proof.
3. proof Let $Z = k(X, \cdot)$. Under $\mathbb{E}\sqrt{k(X, X)} < \infty$, use reproduction and the Bochner operator rule to show $\langle f, \mathbb{E}Z \rangle = \mathbb{E}f(X)$.
4. computation Compute the Fenchel conjugate of $f(x) = x^2/2$ and verify Fenchel-Young equality at $u = f'(x)$.
5. synthesis For a graph-kernel pipeline, list one assumption or diagnostic drawn from each of linear algebra, probability, convex optimization, and graph geometry.
6. challenge Give an example in which pointwise convergence does not justify exchanging limit and expectation, then state a domination condition that repairs the argument.

2 Introduction: Learning by Comparison

Almost every kernel method is one idea worn many ways: if you can say how similar two examples are, you can learn from them. This short chapter states that idea, unpacks the three viewpoints that the rest of the book develops in turn, and lays out the path from here to deep networks, modern theory, and practice.

2.1 The promise: patterns are relations

Suppose you are handed a thousand molecules and told which inhibit a target, or a stack of images to sort into cats and dogs, or a genome to scan for genes. What these tasks share is not their raw data, which could hardly be more different, but their shape: in each case you are looking for a *pattern*, a regularity that lets you say something about an example you have not seen. Shawe-Taylor and Cristianini (2004) make this the organizing idea of the whole field, pattern analysis, the search for stable relations in data. A classifier is a pattern that separates two groups; a regression is a pattern that ties inputs to a number; a clustering is a pattern of proximity; a correlation is a pattern shared between two views of the same objects.

The difficulty is that the data themselves rarely arrive as tidy vectors on which such patterns are visible. A molecule is a graph, a protein is a string, an image is a grid of pixels whose interesting structure lives in no single coordinate. The traditional response is feature engineering: hand-build a map from raw data into a vector space where the pattern shows. Kernel methods make a quieter, more powerful bargain. They ask for one thing only.

The single ingredient

A **kernel** is a function $k(x, x')$ that scores how similar two examples x and x' are. Give a kernel method nothing but the table of pairwise scores on your data, the $n \times n$ **Gram matrix** $K_{ij} = k(x_i, x_j)$, and it will find the pattern. The examples may be vectors, strings, graphs, distributions, or anything at all; the algorithm never sees them, only their comparisons.

The Gram matrix is the interface between your data and every algorithm in this book. Below it is drawn for twelve points on a line: slide the kernel's bandwidth and watch the table of similarities reshape, the only channel through which a kernel method perceives its world.

2.2 Three views of the same object

Why should comparing examples be enough to learn from them? The three sources this book draws on answer the question from three angles, and the answers are the same theorem seen from three sides. Understanding all three is the fastest route to fluency, so the book is built to develop each in turn. These views are also what let the subject outgrow its first home in classification. In the later parts the space itself comes under study, the objects being compared become whole probability distributions, the questions become inferential and causal, and the answers arrive with calibrated uncertainty and working code.

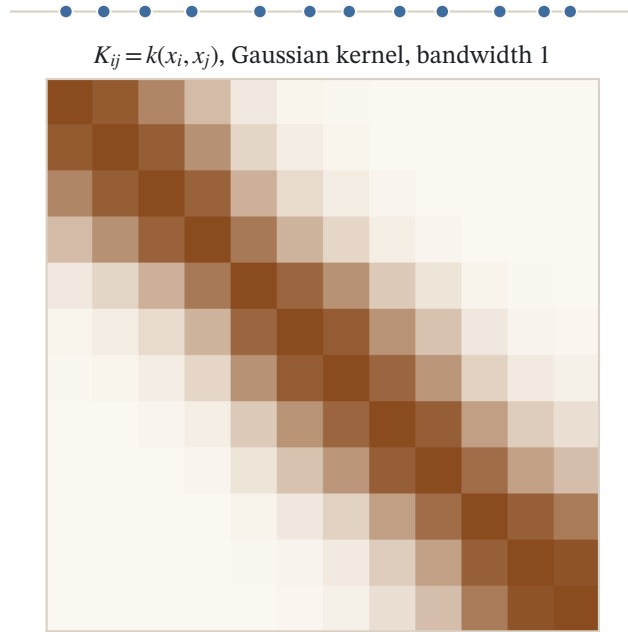


Figure 2.1: Twelve points on a line and their Gram matrix $K_{ij} = k(x_i, x_j)$ for a Gaussian kernel at bandwidth 1. A kernel is positive definite exactly when this matrix is positive semidefinite for every sample; narrowing the bandwidth drives it toward the identity.

2.2.1 The geometric view: a kernel is an inner product

The first answer is that a valid kernel is never merely a similarity score; it is secretly a *dot product* in some space of features. Whenever k is *positive definite*, there is a map Φ sending each example into a (possibly infinite-dimensional) inner-product space, its *feature space*, such that

$$k(x, x') = \langle \Phi(x), \Phi(x') \rangle.$$

Comparing with k is thus doing geometry, measuring angles and lengths, in a space you never have to build. Distances, projections, and separating planes all become available through the kernel alone. Lifting the data this way is also what makes a separating plane likely to exist at all: Cover (1965) showed that patterns cast nonlinearly into a higher-dimensional space are far more probably linearly separable there than in their native coordinates, which is exactly the room a rich feature space buys. This is the view of Part I, and its precise form, the reproducing kernel Hilbert space, is where the book begins.

2.2.2 The regularization view: the norm is smoothness

The second answer, developed at length by Schölkopf and Smola (2002), is that the feature space is not just any inner-product space but a space of *functions*, and its norm measures how wildly a function wiggles. Learning is then a tug of war written as

$$\min_f \underbrace{\frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i))}_{\text{fit the data}} + \underbrace{\lambda \|f\|^2}_{\text{stay smooth}},$$

fit the training data, but pay for roughness. The kernel decides what "rough" means, and the penalty is what keeps the fit from memorizing noise. This regularized-risk template is the engine of Part II, and the reason kernel methods generalize rather than merely interpolate.

2.2.3 The modular view: kernel plus algorithm

The third answer, the pedagogical heart of Shawe-Taylor and Cristianini, is a clean separation of concerns. A kernel method has two independent halves: a *kernel* that encodes what the data are and how they compare, and an *algorithm* that consumes a Gram matrix and returns a pattern. Because the algorithm only ever touches the matrix, the two halves mix and match freely.

The two-part recipe

Any pattern-analysis task decomposes as **a kernel** (choose or design it for your data type) plus **an algorithm** (ridge regression, an SVM, PCA, a two-sample test,...). Swap the kernel and the same algorithm now works on strings or graphs; swap the algorithm and the same kernel now clusters instead of classifies. This modularity is why one small theory covers so much ground, and why the book can treat algorithms (Parts II, III, and V) separately from the kernels that feed them (Part VII).

2.3 The one obstacle: the solve

There is a price for working through the Gram matrix, and naming it early explains the shape of the later chapters. The classical way to fit a kernel machine is to solve a linear system, or a quadratic program, over the whole $n \times n$ matrix at once. That matrix costs n^2 memory to store and its solve costs on the order of n^3 time, and neither exponent has an n you can escape, because the method is defined over all pairs of examples together. For a few thousand examples this is nothing; for a few million it is a wall. Much of the modern subject, and all of Part X, is about walking through that wall while keeping the kernel, whether by approximating the matrix, learning the kernel, or recognizing that a deep network was a kernel machine all along.

2.4 How to read this book

The fourteen parts follow one arc: state the idea and build its space, learn with a fixed kernel, understand why learning works, design kernels for real data, push comparison beyond points to whole distributions and to inference, and close with scale, uncertainty, the present day, and practice. Part 0 is a reference to lean on, not a prerequisite to clear; each later part can be read on its own once Part I is in hand, but the order below is the intended path.

- **Part 0, Preliminaries.** A self-contained reference for the mathematics the book leans on, from linear algebra and Hilbert spaces through probability, optimization, and functional analysis. Read it front to back as a refresher, or dip in when a later chapter cites a result by name.
- **Part I, The Idea.** Positive definite kernels and the reproducing kernel Hilbert space they secretly define (Chapter 2, Positive Definite Kernels and RKHS), then the kernel trick and the representer theorem that collapses an infinite-dimensional search to n numbers.
- **Part II, Supervised Machines with a Fixed Kernel.** Fix a kernel and vary the loss: ridge regression, the support vector machine (Chapter 5, Support Vector Machines), and its siblings for regression, novelty detection, and ranking, the workhorse algorithms as one recipe.
- **Part III, Optimization and Implementation.** A kernel machine is only as useful as the solver behind it. This part opens the quadratic program the dual defines and builds the algorithms that make it tractable, from decomposition and sequential minimal optimization to online updates that never store the whole Gram matrix.

- **Part IV, Learning Theory.** Why should a machine fit on a sample predict on the unseen? Two lineages answer: modern Rademacher and margin bounds for RKHS balls, and the classical VC and structural-risk theory that framed the field.
- **Part V, Structure and Subspaces.** The same Gram matrix that drives supervised learning drives the unsupervised kind: kernel PCA, clustering, canonical correlation, discriminants, and multidimensional scaling, one family of subspace methods read through the kernel.
- **Part VI, The Geometry of the Space.** Why did any of it work? Mercer’s theorem opens the RKHS, the spectrum of a kernel becomes a scale of smoothness, and eigenvalue decay turns into concrete rates of convergence (Chapter 18, Mercer’s Theorem, Spectra, and Rates).
- **Part VII, Designing Kernels for Data.** Everything so far assumed a kernel was handed to you. This part builds them: for vectors by symmetry and probability, for prior knowledge through invariance, for sequences, text, and graphs, for paths and curved domains, and even for the indefinite similarities that fall outside the positive definite world.
- **Part VIII, Kernels and Distributions.** A kernel can embed not just a point but an entire probability distribution (Chapter 29, Mean Embeddings, MMD, and Characteristic Kernels). This part builds the mean embedding, tests whether two samples share a law, meets optimal transport, and integrates against distributions with kernel quadrature.
- **Part IX, Kernel Probabilistic Inference.** Embeddings turn probability into linear algebra, and that opens inference: conditional distributions become operators (Chapter 33, Conditional Mean Embeddings and Kernel Bayes’ Rule), Stein operators test unnormalized models, causal questions get RKHS answers, and regression learns from training examples that are themselves distributions.
- **Part X, Scaling and Learning the Kernel.** The exact solve does not survive a million examples, and the kernel need not be fixed or shallow. This part removes the quadratic wall, learns the kernel itself, and reads deep networks as kernel machines.
- **Part XI, The Bayesian View.** The same kernel that defines a norm defines a covariance: kernel ridge regression becomes the posterior mean of a Gaussian process (Chapter 40, Gaussian Processes and the RVM), every prediction carries a probability, and that uncertainty becomes a rule for what to measure next.
- **Part XII, Kernels Now.** The two standard textbooks close around 2004, before the deep-learning era rewrote what kernels are for. This part is a guide to what came after: the interpolation revolution that revised the book’s own theory of generalization (Chapter 42, Modern Generalization Theory), and the frontier where fixed kernels give way to learned features.
- **Part XIII, Practice.** Theory meets the terminal: end-to-end applications, the model-selection and implementation choices that decide whether a kernel method works, and a map of the software that puts all of it a few lines of code away (Chapter 44, Applications and Practice).

A note on the mathematics: the book assumes comfort with linear algebra and probability, states results precisely, and gives the proofs the sources give, with the longest deferred to expandable blocks. Definitions, theorems, and examples sit in numbered boxes that can be cited and linked from anywhere in the book, and the interactive [dependency map](#) charts which earlier results each theorem leans on, so any proof can be traced back to its foundations. Procedures get algorithm boxes; worked examples display only numbers that were actually computed, each reproduced by a verification script committed alongside the text; and interactive figures, which run the real kernel arithmetic live, appear where a picture teaches faster than a paragraph. Every chapter ends with pointers into the literature, whose full details are in the [bibliography](#). With the single idea in hand, comparison as the basis of learning, we can begin.

Further reading

The three sources this book synthesizes are complementary points of entry. Shawe-Taylor and Cristianini (2004), *Kernel Methods for Pattern Analysis*, opens with the pattern-analysis framing and the modular kernel-plus-algorithm view used above. Schölkopf and Smola (2002), *Learning with Kernels*, develops the regularization and support-vector viewpoint in depth. The lecture course of Mairal and Vert, *Machine Learning with Kernel Methods*, gives the broadest modern survey, including the kernel families, distribution embeddings, and deep-learning connections of the later parts. The book itself now reaches well past both textbooks: Parts VIII through XIII draw on the post-2004 literature, from mean embeddings and Stein methods to Gaussian processes and the modern theory of interpolation, cited chapter by chapter where the results appear.

2.5 Common mistakes and practical implications

For **Introduction: Learning by Comparison**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

2.6 Summary and further reading

This chapter established explain the central definitions and claims in Introduction: Learning by Comparison; Apply the chapter’s principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Cover 1965), (Schölkopf and Smola 2002), (Shawe-Taylor and Cristianini 2004).

2.7 Exercises

1. warm-up The chapter frames classification, regression, clustering, and correlation as four kinds of pattern. For each of the following, name which kind of pattern is being sought: sorting a stack of images into cats and dogs; predicting a molecule’s boiling point from its structure; grouping customers by how close their buying habits are; measuring how strongly the text of an article moves with the photograph beside it.
2. warm-up A kernel method is told that it will only ever be shown the $n \times n$ Gram matrix $K_{ij} = k(x_i, x_j)$, never the examples x_i themselves. Explain in one or two sentences why this is what lets the same algorithm run unchanged on vectors, strings, and graphs. Then state, in your own words, the one property the entries of K must carry so that the geometric view of Section 2 applies, that is, so that $k(x, x') = \langle \Phi(x), \Phi(x') \rangle$ for some feature map Φ . Hint

The algorithm’s independence from the raw data is exactly the modular kernel-plus-algorithm split. The property you need on K is the one the chapter names when it says a valid kernel is “secretly a dot product.”

3. warm-up The chapter says the classical way to fit a kernel machine solves a linear system over the whole Gram matrix, at a cost of about n^2 memory and n^3 time, and that “neither exponent has an n you can escape.” Roughly how many times more time does the solve take when the dataset grows from one thousand to one million examples? Explain why the modular view of Section 2 does not, on its own, remove this wall, and name which part of the book is devoted to walking through it. Hint

A factor of 10^3 more examples raises an n^3 cost by $(10^3)^3$. The wall lives in the algorithm’s use of the matrix, not in the choice of kernel, so swapping kernels leaves it untouched.

4. exploration Open the interactive Gram-matrix figure for the twelve points on a line and narrow the kernel’s bandwidth toward zero while watching the table of similarities. Describe what the matrix approaches, and say which matrix it comes to resemble. Then argue, from that shape alone, why a kernel with a very small bandwidth “sees” each point as similar only to itself, and why an algorithm handed such a matrix has almost no relations left to learn from. Hint

Watch the off-diagonal entries as the bandwidth shrinks: a point stops resembling even its close neighbors. What is left is large only where $i = j$. Compare the limiting table to the identity matrix.

3 Positive Definite Kernels and RKHS

Machine learning algorithms are usually written for vectors. Real data are strings, graphs, images, molecules, time series. Kernel methods resolve this mismatch with one idea: never touch the data directly, only compare pairs of points. This chapter develops the mathematical foundation of that idea. We define positive definite kernels, prove Aronszajn's theorem identifying every such kernel with an inner product after an embedding into a Hilbert space, construct the reproducing kernel Hilbert space (RKHS) that realizes this embedding canonically as a space of functions, build a toolbox for combining and recognizing kernels, work out the RKHS of the classical examples, and show that the RKHS norm measures the smoothness of a function. Everything that follows in this book rests on these objects.

3.1 Why pairwise comparisons?

We want versatile algorithms: methods that process and analyze data without making any assumption on the type of the data, whether vectors, strings, graphs or images. The approach developed here is to build methods on pairwise comparisons. By imposing constraints on the pairwise comparison function, the positive definiteness studied below, we will obtain a general framework for learning from data, namely optimization in a reproducing kernel Hilbert space.

Concretely, we define a comparison function $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ on the data space $\mathcal{X}$, and we represent a set of n data points $\mathcal{S} = \{x_1, x_2, \dots, x_n\}$ by the $n \times n$ matrix of all pairwise comparisons:

$$[\mathbf{K}]_{ij} := K(x_i, x_j).$$

For instance, a set of three DNA-like strings such as $\mathcal{S} = (\text{aatcgagtcac}, \text{atggacgtct}, \text{tgcactact})$ becomes, once a comparison function between strings is chosen, an ordinary 3×3 matrix of real numbers, and from that point on the strings themselves never need to be looked at again.

Three remarks explain why this representation is so attractive, and what it costs.

- **Genericity.** The matrix $\mathbf{K}$ is always an $n \times n$ matrix of real numbers, whatever the nature of the data. The same algorithm will therefore work for any type of data: vectors, strings, graphs, and so on.
- **Modularity.** There is total modularity between the choice of the comparison function K , which encodes what we know about the data, and the choice of the learning algorithm, which operates only on the matrix.
- **Scalability.** The price is poor scalability with respect to the dataset size: the matrix takes n^2 operations to compute and n^2 memory to store. Chapter 37, Large-Scale Kernel Machines shows how to deal with large-scale problems; for now we accept the cost.

Not every comparison function will do. We now restrict ourselves to a particular class of pairwise comparison functions, and this restriction is the source of all the structure to come.

3.2 Positive definite kernels

A useful comparison function should behave like a similarity measure, and the algebraic way to say "behaves like an inner product" is positive definiteness.

Definition (positive definite kernel)

A *positive definite (p.d.) kernel* on a set $\mathcal{X}$ is a function $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ that is symmetric,

$$\forall(x, x') \in \mathcal{X}^2, \quad K(x, x') = K(x', x),$$

and which satisfies, for all $N \in \mathbb{N}$, all points $(x_1, x_2, \dots, x_N) \in \mathcal{X}^N$ and all coefficients $(a_1, a_2, \dots, a_N) \in \mathbb{R}^N$:

$$\sum_{i=1}^N \sum_{j=1}^N a_i a_j K(x_i, x_j) \geq 0.$$

Remark (similarity matrices)

Equivalently, a kernel K is p.d. if and only if, for any $N \in \mathbb{N}$ and any set of points $(x_1, x_2, \dots, x_N) \in \mathcal{X}^N$, the similarity matrix $[\mathbf{K}]_{ij} := K(x_i, x_j)$ is symmetric positive semidefinite. Indeed, the double sum in the definition is exactly the quadratic form $\mathbf{a}^\top \mathbf{K} \mathbf{a}$. Kernel methods are algorithms that take such matrices as input.

Note the slightly unfortunate but standard terminology: a positive definite *kernel* produces positive *semidefinite* matrices; the quadratic form is allowed to vanish on nonzero coefficient vectors.

Remark (terminology and history)

The word *kernel* predates machine learning by decades. As Schölkopf and Smola (2002) recall, it entered mathematics through the theory of integral operators studied by Hilbert and others, where a function K is the kernel of the operator

$$(T_K f)(x) = \int K(x, x') f(x') dx'.$$

The same object travels under many names across communities: reproducing kernel, Mercer kernel, admissible kernel, support vector kernel, nonnegative definite kernel, and, in statistics, covariance function. Schölkopf and Smola also caution that the name “positive definite kernel” sits slightly awry with matrix theory, where “definite” is often reserved for the strict case in which the quadratic form vanishes only for the zero coefficient vector. Our kernels are merely “semidefinite” in that stricter sense, which is exactly why their Gram matrices are p.s.d. rather than p.d.

What does the inequality actually demand? It is best read as a promise about squared lengths. If the kernel were secretly an inner product, $K(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$, then the double sum would telescope into $\left\| \sum_i a_i \Phi(x_i) \right\|^2$, which can never be negative. Positive definiteness is exactly the algebraic shadow of that geometric fact: it is the one condition, stated without ever naming a feature map Φ , that guarantees a Φ can be found. Aronszajn’s theorem below turns the promise into a construction; for now, the condition is the entrance ticket.

Concretely, positive definiteness is something one can watch: choose a handful of points, form their Gram matrix, and check whether that matrix is positive semidefinite. The next figure does this live for points on a line, and lets you see how the choice of kernel and its bandwidth reshape the entries and the eigenvalues of the matrix.

3.2.1 The simplest p.d. kernels

The definition is best absorbed through three foundational examples, in increasing generality.

Lemma (product kernel on $\mathbb{R}$)

Let $\mathcal{X} = \mathbb{R}$. The function $K : \mathbb{R}^2 \rightarrow \mathbb{R}$ defined by

$$\forall(x, x') \in \mathbb{R}^2, \quad K(x, x') = xx'$$

is p.d.

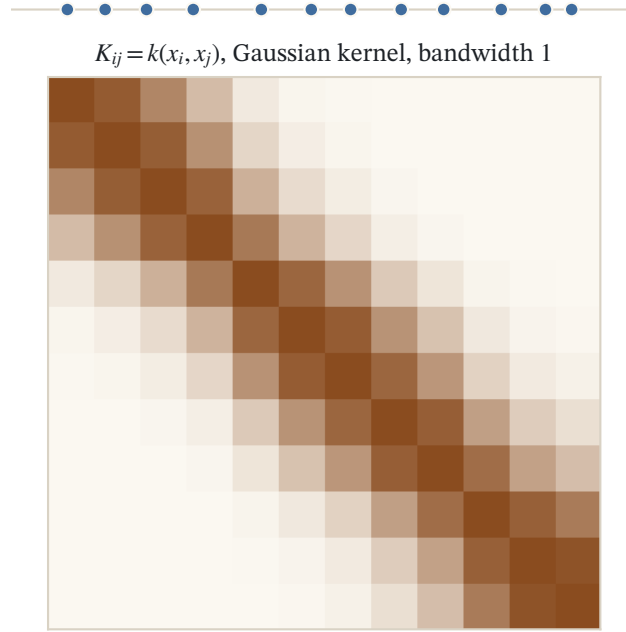


Figure 3.1: Twelve points on a line and their Gram matrix $K_{ij} = k(x_i, x_j)$ for a Gaussian kernel at bandwidth 1. A kernel is positive definite exactly when this matrix is positive semidefinite for every sample; narrowing the bandwidth drives it toward the identity.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Symmetry is clear: $xx' = x'x$. For positivity, the double sum collapses into a square:

$$\sum_{i=1}^N \sum_{j=1}^N a_i a_j x_i x_j = \left(\sum_{i=1}^N a_i x_i \right)^2 \geq 0.$$

□

Lemma (linear kernel on $\mathbb{R}^d$)

Let $\mathcal{X} = \mathbb{R}^d$. The function $K : \mathcal{X}^2 \rightarrow \mathbb{R}$ defined by

$$\forall (x, x') \in \mathcal{X}^2, \quad K(x, x') = \langle x, x' \rangle_{\mathbb{R}^d}$$

is p.d. It is often called the *linear kernel*.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Symmetry: $\langle x, x' \rangle_{\mathbb{R}^d} = \langle x', x \rangle_{\mathbb{R}^d}$. Positivity, by bilinearity of the inner product:

$$\sum_{i=1}^N \sum_{j=1}^N a_i a_j \langle x_i, x_j \rangle_{\mathbb{R}^d} = \left\| \sum_{i=1}^N a_i x_i \right\|_{\mathbb{R}^d}^2 \geq 0.$$

□

The same computation proves something far more ambitious. Nothing in the argument used the structure of $\mathcal{X}$; it only used that the compared objects live in an inner product space. So we may first move the data anywhere we like.

Lemma (kernels from embeddings)

Let $\mathcal{X}$ be any set, and $\Phi : \mathcal{X} \rightarrow \mathbb{R}^d$ any mapping. Then the function $K : \mathcal{X}^2 \rightarrow \mathbb{R}$ defined as follows is p.d.:

$$\forall(x, x') \in \mathcal{X}^2, \quad K(x, x') = \langle \Phi(x), \Phi(x') \rangle_{\mathbb{R}^d}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Symmetry: $\langle \Phi(x), \Phi(x') \rangle_{\mathbb{R}^d} = \langle \Phi(x'), \Phi(x) \rangle_{\mathbb{R}^d}$. Positivity:

$$\sum_{i=1}^N \sum_{j=1}^N a_i a_j \langle \Phi(x_i), \Phi(x_j) \rangle_{\mathbb{R}^d} = \left\| \sum_{i=1}^N a_i \Phi(x_i) \right\|_{\mathbb{R}^d}^2 \geq 0.$$

□

This lemma is the picture to keep in mind throughout: points x in an arbitrary set $\mathcal{X}$ are mapped by a feature map Φ into a space where inner products make sense, and the kernel is the inner product of the images. The remarkable fact, proved below, is that *every* p.d. kernel arises this way.

3.2.2 A first nonlinear example: the polynomial kernel

Embeddings let us build kernels that are nonlinear in the original variables. For $x = (x_1, x_2)^\top \in \mathbb{R}^2$, consider the map into $\mathbb{R}^3$ of degree-two monomials, $\Phi(x) = (x_1^2, \sqrt{2} x_1 x_2, x_2^2)$. Then

$$K(x, x') = \langle \Phi(x), \Phi(x') \rangle_{\mathbb{R}^3} = x_1^2 x_1'^2 + 2x_1 x_2 x_1' x_2' + x_2^2 x_2'^2 = (x_1 x_1' + x_2 x_2')^2 = \langle x, x' \rangle_{\mathbb{R}^2}^2.$$

The squared inner product is therefore p.d. on $\mathbb{R}^2$ by the embedding lemma, and it lets linear machinery capture quadratic structure: a function that is linear in $\Phi(x)$ is a quadratic function of x . Geometrically, a curved boundary in the plane, such as an ellipse, becomes a hyperplane in the three-dimensional feature space.

The feature map that realizes a kernel is far from unique. Schölkopf and Smola (2002) make this vivid for the polynomial kernel by exhibiting two different maps that produce the same kernel. One lists the *ordered* products of coordinates, $[x]_{j_1} [x]_{j_2} \cdots [x]_{j_d}$, landing in a space of dimension N^d ; the other keeps only the distinct *unordered* monomials and compensates for their multiplicities by weighting each with the square root of the number of orderings that generate it. In the plane at degree two the second map is exactly the $\Phi(x) = (x_1^2, \sqrt{2} x_1 x_2, x_2^2)$ above, the $\sqrt{2}$ being the square root of the two orderings of the product $x_1 x_2$. Both maps compute $\langle x, x' \rangle_{\mathbb{R}^2}^2$, so a single kernel lives at once in feature spaces of different dimensions. What the kernel pins down is the geometry, not the coordinates.

Remark (why the $\sqrt{2}$?)

Those square-root-of-multiplicity weights are not cosmetic. Schölkopf and Smola (2002) observe that they are precisely what makes the feature map commute with rotations: up to an overall scale, the homogeneous kernel $\langle x, x' \rangle^d$ is the only kernel whose degree- d monomial feature map is invariant under orthogonal transformations of the input space. So the natural weighting of the monomials is forced by the demand that relabeling the coordinate axes, an operation that changes nothing about the data, must not change the geometry the kernel induces.

Exercise

Show that $K(x, x') = \langle x, x' \rangle_{\mathbb{R}^d}^d$ is p.d. on $\mathcal{X} = \mathbb{R}^d$ for any $d \in \mathbb{N}$. (A one-line proof will be available once we know that products of p.d. kernels are p.d.; a direct proof expands the power into monomial features, generalizing the computation above.)

Verification artifact. checks/example-ch01-example-ch01-1.json records the example source hash and verification scope.

3.3 Kernels as inner products: Aronszajn's theorem

The embedding lemma produced p.d. kernels from feature maps. The converse question is the deep one: given a p.d. kernel, defined only by its algebraic properties, is it secretly an inner product after some embedding? The answer is yes, always, provided we allow the target space to be an infinite-dimensional generalization of Euclidean space.

Theorem (Aronszajn, 1950)

K is a p.d. kernel on the set $\mathcal{X}$ if and only if there exists a Hilbert space $\mathcal{H}$ and a mapping

$$\Phi : \mathcal{X} \rightarrow \mathcal{H}$$

such that, for any x, x' in $\mathcal{X}$:

$$K(x, x') = \langle \Phi(x), \Phi(x') \rangle_{\mathcal{H}}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

One direction is already done: the proof of the embedding lemma works verbatim with $\mathbb{R}^d$ replaced by any Hilbert space $\mathcal{H}$, since it only uses bilinearity, symmetry and nonnegativity of the squared norm. The content of the theorem is the other direction. Before proving it, we recall the definitions it involves.

3.3.1 In case of need: Hilbert spaces

Definitions (inner product, Hilbert space)

An *inner product* on an $\mathbb{R}$ -vector space $\mathcal{H}$ is a mapping $(f, g) \mapsto \langle f, g \rangle_{\mathcal{H}}$ from $\mathcal{H}^2$ to $\mathbb{R}$ that is bilinear, symmetric, and such that $\langle f, f \rangle_{\mathcal{H}} > 0$ for all $f \in \mathcal{H} \setminus \{0\}$. A vector space endowed with an inner product is called *pre-Hilbert*. It carries the norm $\|f\|_{\mathcal{H}} = \langle f, f \rangle_{\mathcal{H}}^{1/2}$. A *Cauchy sequence* $(f_n)_{n \geq 0}$ is a sequence whose elements become progressively arbitrarily close to each other:

$$\lim_{N \rightarrow +\infty} \sup_{n, m \geq N} \|f_n - f_m\|_{\mathcal{H}} = 0.$$

A *Hilbert space* is a pre-Hilbert space that is complete for the norm $\|\cdot\|_{\mathcal{H}}$: every Cauchy sequence in $\mathcal{H}$ converges in $\mathcal{H}$.

In one sentence: a Hilbert space is Euclidean geometry, with its lengths, angles and projections, that survives the passage to infinitely many dimensions. The inner product supplies the geometry, and completeness is the safety net.

Completeness is not a technical luxury. It is necessary to keep the good convergence properties of Euclidean spaces in an infinite-dimensional context; without it, limits that "should" exist may fall outside the space, and basic tools such as orthogonal projections and the Riesz representation theorem fail. The rational numbers are the cautionary analogy: a sequence of rationals can march steadily toward $\sqrt{2}$ yet have no rational limit, and a space of functions can misbehave in the same way unless we complete it, which is exactly the final step of the RKHS construction below.

3.3.2 Proof of the converse: the finite case

When $\mathcal{X}$ is finite the theorem is pure linear algebra, and the proof shows exactly where the feature map comes from: the spectral decomposition of the kernel matrix.

Proof (finite case)

Assume $\mathcal{X} = \{x_1, x_2, \dots, x_N\}$ is finite of size N . Any p.d. kernel $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is entirely defined by the $N \times N$ symmetric positive semidefinite matrix $[\mathbf{K}]_{ij} := K(x_i, x_j)$. This matrix can be diagonalized on an orthonormal basis of eigenvectors $(u_1, u_2, \dots, u_N)$ with nonnegative eigenvalues $0 \leq \lambda_1 \leq \dots \leq \lambda_N$, that is,

$$K(x_i, x_j) = \left[\sum_{l=1}^N \lambda_l u_l u_l^\top \right]_{ij} = \sum_{l=1}^N \lambda_l [u_l]_i [u_l]_j = \langle \Phi(x_i), \Phi(x_j) \rangle_{\mathbb{R}^N},$$

with

$$\Phi(x_i) = \begin{pmatrix} \sqrt{\lambda_1} [u_1]_i \\ \vdots \\ \sqrt{\lambda_N} [u_N]_i \end{pmatrix}.$$

□

It is worth reading off what this proof actually did, because the idea recurs in a more abstract guise for infinite $\mathcal{X}$. The feature map is nothing but the square root of the kernel matrix, taken through its eigenbasis: the coordinates of $\Phi(x_i)$ are the entries of the i -th data point along each principal axis u_l , rescaled by $\sqrt{\lambda_l}$. Directions with large eigenvalue stretch the embedding, directions with zero eigenvalue collapse it. The nonnegativity of the eigenvalues, which is precisely positive semidefiniteness, is exactly what makes those square roots real, and hence what makes the embedding possible at all.

3.3.3 The general case: a short history

For infinite $\mathcal{X}$ the result was established in stages: Mercer (1909) proved it for $\mathcal{X} = [a, b] \subset \mathbb{R}$, and more generally for $\mathcal{X}$ compact, with K continuous; Kolmogorov (1941) for $\mathcal{X}$ countable; and Aronszajn (1944, 1950) in the general case. We will go through the proof of the general case by introducing the concept at the heart of the theory, the reproducing kernel Hilbert space, which provides a canonical choice of $\mathcal{H}$ valid without any restriction on $\mathcal{X}$ or on K .

Mercer's special case deserves a closer look, because it produces the embedding by a route that is the exact continuous analogue of the finite spectral construction: eigenvectors are replaced by eigenfunctions. Both Schölkopf and Smola (2002) and Shawe-Taylor and Cristianini (2004) develop it, and it is the standard way to see a kernel as a weighted sum of feature coordinates.

Theorem (Mercer, 1909)

Let $\mathcal{X}$ be a compact subset of $\mathbb{R}^n$ and let $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ be continuous and symmetric, with a positive integral operator, that is,

$$\int_{\mathcal{X} \times \mathcal{X}} K(x, x') f(x) f(x') dx dx' \geq 0 \quad \text{for all } f \in L^2(\mathcal{X}).$$

Then K admits a uniformly convergent expansion

$$K(x, x') = \sum_{j=1}^{\infty} \phi_j(x) \phi_j(x'),$$

where the feature coordinates are $\phi_j = \sqrt{\mu_j} \psi_j$, built from the eigenvalues $\mu_j \geq 0$ and the eigenfunctions ψ_j of the integral operator. The eigenfunctions $(\psi_j)_{j \geq 1}$ are orthonormal in $L^2(\mathcal{X})$; the scaled coordinates $(\phi_j)_{j \geq 1}$ are therefore orthogonal, but not orthonormal, with squared norm $\|\phi_j\|_{L^2}^2 = \mu_j$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The expansion is the promised embedding: the map $x \mapsto (\phi_1(x), \phi_2(x), \dots)$ into the sequence space ℓ^2 realizes K as an inner product, exactly as the map $x_i \mapsto (\sqrt{\lambda_i} [u_i]_i)_i$ did in the finite case, with the eigenfunctions ψ_j now playing the role of the principal axes u_i and the eigenvalues μ_j the role of the λ_i . This is the Mercer route to the feature space. Its price is the hypotheses it needs, compactness of $\mathcal{X}$, continuity of K , and a reference measure; the RKHS route developed below reaches the same conclusion while asking for none of them.

3.4 Reproducing kernel Hilbert spaces

3.4.1 Mapping data points to functions

Among the Hilbert spaces $\mathcal{H}$ whose existence Aronszajn's theorem asserts, one particular realization will be of special interest to us. It is a space of functions from $\mathcal{X}$ to $\mathbb{R}$: each data point $x \in \mathcal{X}$ will be represented by a function $\Phi(x) = K_x$ in $\mathcal{H}$.

To see what this could mean, consider $\mathcal{X} = \mathbb{R}$. We could decide to represent each scalar $x \in \mathbb{R}$ by a Gaussian function centered at x :

$$K_x : t \mapsto e^{-\frac{1}{2\alpha^2}(x-t)^2}.$$

Two points x and y then become two Gaussian bumps sitting at x and y on the real line, and their overlap measures their similarity. The questions this raises are exactly the ones the RKHS construction answers: what is the corresponding space $\mathcal{H}$, if it exists, and what is its inner product?

One point deserves emphasis before the formal definition. The data are mapped to Gaussian functions living in a Hilbert space $\mathcal{H}$, but $\mathcal{H}$ is much richer and contains much more than Gaussian bumps. It is in this larger space that prediction functions f will live, and they will interact with the embedded data through the inner product: $f(x) = \langle f, \Phi(x) \rangle_{\mathcal{H}}$. This identity, evaluation as an inner product, is the defining property we now formalize.

3.4.2 Definition and the reproducing property

Definition (reproducing kernel, RKHS)

Let $\mathcal{X}$ be a set and $\mathcal{H} \subset \mathbb{R}^{\mathcal{X}}$ be a class of functions forming a (real) Hilbert space with inner product $\langle \cdot, \cdot \rangle_{\mathcal{H}}$. The function $K : \mathcal{X}^2 \rightarrow \mathbb{R}$ is called a *reproducing kernel* (r.k.) of $\mathcal{H}$ if

1. $\mathcal{H}$ contains all functions of the form

$$\forall x \in \mathcal{X}, \quad K_x : t \mapsto K(x, t);$$

2. for every $x \in \mathcal{X}$ and $f \in \mathcal{H}$, the *reproducing property* holds:

$$f(x) = \langle f, K_x \rangle_{\mathcal{H}}.$$

If a r.k. exists, then $\mathcal{H}$ is called a *reproducing kernel Hilbert space* (RKHS).

The second condition is the whole point, so it is worth saying in words. It asserts that evaluating a function f at a point x , an operation that looks nothing like an inner product, is in fact the inner product of f against one special function, the kernel section K_x . Reading a function's value at x becomes measuring its overlap with the bump sitting at x . The kernel therefore plays a double role: each K_x is at once a member of the space $\mathcal{H}$ (a function we can evaluate elsewhere) and the probe that reports the value of any other member at x . That single identity is what will let us turn nonlinear learning in $\mathcal{X}$ into linear algebra in $\mathcal{H}$.

Why do we care? Because the RKHS gives us a simple recipe to do machine learning:

- Map the data $x \in \mathcal{X}$ to a high-dimensional Hilbert space $\mathcal{H}$, the RKHS, through the kernel mapping $\Phi : \mathcal{X} \rightarrow \mathcal{H}$ with $\Phi(x) = K_x$.
- In $\mathcal{H}$, consider only simple *linear* models $f(x) = \langle f, \Phi(x) \rangle_{\mathcal{H}}$; this is literally the reproducing property.
- If $\mathcal{X} = \mathbb{R}^p$, a function that is linear in $\Phi(x)$ may be highly nonlinear in x , as the polynomial example already showed.

For instance, for supervised learning, given training data $(y_i, x_i)_{i=1, \dots, n}$, we may want to minimize the regularized empirical risk

$$\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2,$$

where L is a loss function and $\lambda \geq 0$ a regularization parameter. Making sense of this optimization problem, and solving it, occupies the coming chapters; the smoothness interpretation of the penalty $\|f\|_{\mathcal{H}}^2$ closes the present one.

3.4.3 An equivalent definition: continuous evaluation

The definition of a r.k. looks tailored to the kernel K . In fact, being an RKHS is an intrinsic property of the Hilbert space $\mathcal{H}$, independent of any particular kernel: it says that pointwise evaluation is a well-behaved operation.

Theorem (RKHS = continuous evaluation functionals)

The Hilbert space $\mathcal{H} \subset \mathbb{R}^{\mathcal{X}}$ is a RKHS if and only if for any $x \in \mathcal{X}$, the linear mapping

$$F : \mathcal{H} \rightarrow \mathbb{R}, \quad f \mapsto f(x)$$

is continuous.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

In plain terms, the theorem says that the RKHS property is not really about kernels at all: a Hilbert space of functions is an RKHS precisely when knowing that two functions are close in norm already tells us their values are close at every point. The kernel is then a byproduct, handed to us by the Riesz representation theorem as the vector that represents the evaluation functional. This is a reassuring characterization, because it lets us recognize an RKHS from the intrinsic behavior of the space, without having to exhibit the kernel first.

Corollary (norm convergence implies pointwise convergence)

Convergence in a RKHS implies pointwise convergence: if $(f_n)_{n \in \mathbb{N}}$ converges to f in $\mathcal{H}$, then $(f_n(x))_{n \in \mathbb{N}}$ converges to $f(x)$ for any $x \in \mathcal{X}$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

This corollary is what makes an RKHS safe for machine learning: functions close in the norm we optimize are close as predictors, at every single point. Nothing of the sort holds in a space like L^2 , whose elements are not even defined pointwise.

Proof (if $\mathcal{H}$ is a RKHS then $f \mapsto f(x)$ is continuous)

If a r.k. K exists, then for any $(x, f) \in \mathcal{X} \times \mathcal{H}$:

$$|f(x)| = |\langle f, K_x \rangle_{\mathcal{H}}| \leq \|f\|_{\mathcal{H}} \cdot \|K_x\|_{\mathcal{H}} = \|f\|_{\mathcal{H}} \cdot K(x, x)^{1/2},$$

where the inequality is Cauchy-Schwarz and the last equality holds because $\|K_x\|_{\mathcal{H}}^2 = \langle K_x, K_x \rangle_{\mathcal{H}} = K(x, x)$ by the reproducing property applied to $f = K_x$. The evaluation map is linear, and this bound shows it is bounded, hence continuous. (Since F is linear, it is indeed sufficient to check continuity at zero, that is, $f \rightarrow 0 \Rightarrow f(x) \rightarrow 0$, which the bound gives immediately.) The corollary follows by applying the bound to $f_n - f$:

$$|f_n(x) - f(x)| \leq \|f_n - f\|_{\mathcal{H}} K(x, x)^{1/2} \rightarrow 0.$$

□

Proof (if $f \mapsto f(x)$ is continuous then $\mathcal{H}$ is a RKHS)

Conversely, assume that for any $x \in \mathcal{X}$ the linear form $f \in \mathcal{H} \mapsto f(x)$ is continuous. Then, by the Riesz representation theorem, a general property of Hilbert spaces stating that every continuous linear form is an inner product against a fixed element, there exists a unique $g_x \in \mathcal{H}$ such that

$$f(x) = \langle f, g_x \rangle_{\mathcal{H}} \quad \text{for all } f \in \mathcal{H}.$$

The function $K(x, y) := g_x(y)$ is then a r.k. for $\mathcal{H}$: the function $K_x = g_x$ belongs to $\mathcal{H}$ by construction, and the displayed identity is the reproducing property. □

3.4.4 Uniqueness of the kernel and of the space

The article "a" in "a reproducing kernel" is unnecessarily cautious: the kernel of an RKHS is unique, and an RKHS is determined by its kernel.

Theorem (uniqueness)

If $\mathcal{H}$ is a RKHS, then it has a unique r.k. Conversely, a function K can be the r.k. of at most one RKHS.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof (a r.k., if it exists, is unique)

Let K and K' be two reproducing kernels of the same RKHS $\mathcal{H}$. Then, for any $x \in \mathcal{X}$, using bilinearity:

$$\|K_x - K'_x\|_{\mathcal{H}}^2 = \langle K_x - K'_x, K_x - K'_x \rangle_{\mathcal{H}} = \langle K_x - K'_x, K_x \rangle_{\mathcal{H}} - \langle K_x - K'_x, K'_x \rangle_{\mathcal{H}}.$$

The first term equals $K_x(x) - K'_x(x)$ by the reproducing property of K applied to the function $K_x - K'_x \in \mathcal{H}$, and the second term equals $K_x(x) - K'_x(x)$ by the reproducing property of K' . Hence

$$\|K_x - K'_x\|_{\mathcal{H}}^2 = K_x(x) - K'_x(x) - K_x(x) + K'_x(x) = 0.$$

This shows that $K_x = K'_x$ as functions, i.e., $K_x(y) = K'_x(y)$ for any $y \in \mathcal{X}$. In other words, $K = K'$. $\square$

The converse statement, that the RKHS of a given r.k. is unique, is left as an exercise; here is the idea. If $\mathcal{H}_1$ and $\mathcal{H}_2$ are two RKHS with the same kernel K , both contain the linear span $\mathcal{H}_0$ of the functions $\{K_x\}_{x \in \mathcal{X}}$, and on $\mathcal{H}_0$ both inner products are computed from K alone, since $\langle K_x, K_y \rangle = K(x, y)$ in each space. Moreover $\mathcal{H}_0$ is dense in each space: a function f orthogonal to every K_x satisfies $f(x) = \langle f, K_x \rangle = 0$ for all x , hence $f = 0$. Since evaluation functionals are continuous, norm limits of sequences in $\mathcal{H}_0$ are determined pointwise, so the two completions coincide as sets of functions and carry the same norm.

Consequence

We can talk of "the" kernel of a RKHS, and of "the" RKHS of a kernel.

3.5 The equivalence: positive definite equals reproducing

We now connect the two notions introduced so far. This single result, due to Moore and Aronszajn and known as the Moore-Aronszajn theorem, simultaneously proves Aronszajn's embedding theorem and constructs the canonical feature space.

Theorem

A function $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is p.d. if and only if it is a r.k.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The easy direction repeats a computation we have already made twice.

Proof (a r.k. is p.d.)

Let K be the r.k. of a RKHS $\mathcal{H}$.

1. K is symmetric because, for any $(x, y) \in \mathcal{X}^2$, the reproducing property applied to $f = K_x$ gives

$$K(x, y) = \langle K_x, K_y \rangle_{\mathcal{H}} = \langle K_y, K_x \rangle_{\mathcal{H}} = K(y, x).$$

2. It is p.d. because for any $N \in \mathbb{N}$, $(x_1, x_2, \dots, x_N) \in \mathcal{X}^N$ and $(a_1, a_2, \dots, a_N) \in \mathbb{R}^N$:

$$\sum_{i,j=1}^N a_i a_j K(x_i, x_j) = \sum_{i,j=1}^N a_i a_j \langle K_{x_i}, K_{x_j} \rangle_{\mathcal{H}} = \left\| \sum_{i=1}^N a_i K_{x_i} \right\|_{\mathcal{H}}^2 \geq 0.$$

□

The converse is the constructive heart of the chapter: starting from nothing but the p.d. function K , we build the space $\mathcal{H}$ by hand. The plan is natural. The functions K_x must belong to $\mathcal{H}$, so we start from their span; the reproducing property forces the value of the inner product on this span; positivity of K makes this forced bilinear form a genuine inner product; and a completion step delivers a Hilbert space.

Proof (a p.d. kernel is a r.k.)

Step 1: the candidate space and inner product. Let $\mathcal{H}_0 \subset \mathbb{R}^{\mathcal{X}}$ be the vector subspace spanned by the functions $\{K_x\}_{x \in \mathcal{X}}$. For any $f, g \in \mathcal{H}_0$, given by finite expansions

$$f = \sum_{i=1}^m a_i K_{x_i}, \quad g = \sum_{j=1}^n b_j K_{y_j},$$

define

$$\langle f, g \rangle_{\mathcal{H}_0} := \sum_{i,j} a_i b_j K(x_i, y_j).$$

Step 2: the definition is legitimate. A function of $\mathcal{H}_0$ may admit several expansions, so we must check that $\langle f, g \rangle_{\mathcal{H}_0}$ does not depend on the expansions chosen for f and g . Regrouping the double sum in two ways,

$$\langle f, g \rangle_{\mathcal{H}_0} = \sum_{i=1}^m a_i \left(\sum_{j=1}^n b_j K(x_i, y_j) \right) = \sum_{i=1}^m a_i g(x_i), \quad \langle f, g \rangle_{\mathcal{H}_0} = \sum_{j=1}^n b_j \left(\sum_{i=1}^m a_i K(x_i, y_j) \right) = \sum_{j=1}^n b_j f(y_j).$$

The first expression depends on g only through its values as a function, so the result is unchanged if the expansion of g is replaced by another one; the second expression shows the same for f . Hence $\langle \cdot, \cdot \rangle_{\mathcal{H}_0}$ is well defined, and it is clearly a symmetric bilinear form. Taking $g = K_x$ in the first identity also shows that for any $x \in \mathcal{X}$ and $f \in \mathcal{H}_0$:

$$\langle f, K_x \rangle_{\mathcal{H}_0} = f(x),$$

which is the reproducing property on $\mathcal{H}_0$.

Step 3: it is an inner product. Since K is assumed p.d.,

$$\|f\|_{\mathcal{H}_0}^2 = \sum_{i,j=1}^m a_i a_j K(x_i, x_j) \geq 0,$$

so the form is positive semidefinite. For such forms the Cauchy-Schwarz inequality is valid; combined with the reproducing property it yields, for all $x \in \mathcal{X}$:

$$|f(x)| = |\langle f, K_x \rangle_{\mathcal{H}_0}| \leq \|f\|_{\mathcal{H}_0} \cdot K(x, x)^{1/2}.$$

In particular, $\|f\|_{\mathcal{H}_0} = 0$ implies $f(x) = 0$ for every x , that is, $f = 0$. The form is therefore positive definite, and $\mathcal{H}_0$ is a pre-Hilbert space endowed with the inner product $\langle \cdot, \cdot \rangle_{\mathcal{H}_0}$.

Step 4: completion. For any Cauchy sequence $(f_n)_{n \geq 0}$ in $(\mathcal{H}_0, \langle \cdot, \cdot \rangle_{\mathcal{H}_0})$, the bound of Step 3 gives

$$\forall (x, m, n) \in \mathcal{X} \times \mathbb{N}^2, \quad |f_m(x) - f_n(x)| \leq \|f_m - f_n\|_{\mathcal{H}_0} \cdot K(x, x)^{1/2}.$$

Therefore, for any x , the real sequence $(f_n(x))_{n \geq 0}$ is Cauchy in $\mathbb{R}$ and has a limit; call it $f(x)$. Let $\mathcal{H}$ be the space obtained by adding to $\mathcal{H}_0$ all functions defined as such pointwise limits of Cauchy sequences. The inner product extends to $\mathcal{H}$ by $\langle f, g \rangle_{\mathcal{H}} := \lim_n \langle f_n, g_n \rangle_{\mathcal{H}_0}$ for Cauchy sequences $(f_n), (g_n)$ converging pointwise to f, g ; one checks that this limit exists, does not depend on the chosen sequences, and defines an inner product for which $\mathcal{H}$ is complete, so that $\mathcal{H}$ is a Hilbert space in which $\mathcal{H}_0$ is dense. Finally, the reproducing property passes to the limit: for $f \in \mathcal{H}$ obtained from the Cauchy sequence (f_n) ,

$$f(x) = \lim_n f_n(x) = \lim_n \langle f_n, K_x \rangle_{\mathcal{H}_0} = \langle f, K_x \rangle_{\mathcal{H}},$$

so K is a r.k. of $\mathcal{H}$ (up to a few technicalities in the verification of the completion, left as an exercise). $\square$

3.5.1 Application: back to Aronszajn's theorem

The equivalence just proved delivers the missing direction of Aronszajn's theorem, with an explicit and canonical embedding.

Proof of Aronszajn's theorem (converse direction)

If K is p.d. over a set $\mathcal{X}$, then by the previous theorem it is the r.k. of a Hilbert space $\mathcal{H} \subset \mathbb{R}^{\mathcal{X}}$. Define the mapping $\Phi : \mathcal{X} \rightarrow \mathcal{H}$ by

$$\forall x \in \mathcal{X}, \quad \Phi(x) = K_x.$$

By the reproducing property, for all $(x, y) \in \mathcal{X}^2$:

$$\langle \Phi(x), \Phi(y) \rangle_{\mathcal{H}} = \langle K_x, K_y \rangle_{\mathcal{H}} = K(x, y).$$

$\square$

So every p.d. kernel is an inner product after an embedding, and one universal choice of embedding always works: send each point to its own kernel function. This is exactly the "Gaussian bump" picture made rigorous.

3.6 Examples: kernels and their RKHS

The completion construction is abstract; for concrete kernels one can often describe the RKHS explicitly. Thanks to the uniqueness theorem, the strategy is simple: guess a Hilbert space of functions, then verify the two axioms of the definition. If they hold, the guessed space is the RKHS, since there can be no other.

3.6.1 The linear kernel

Take $\mathcal{X} = \mathbb{R}^d$ and the linear kernel $K(x, y) = \langle x, y \rangle_{\mathbb{R}^d}$.

Theorem (RKHS of the linear kernel)

The RKHS of the linear kernel is the set of linear functions of the form

$$f_w(x) = \langle w, x \rangle_{\mathbb{R}^d} \quad \text{for } w \in \mathbb{R}^d,$$

endowed with the inner product

$$\forall w, v \in \mathbb{R}^d, \quad \langle f_w, f_v \rangle_{\mathcal{H}} = \langle w, v \rangle_{\mathbb{R}^d},$$

and corresponding norm $\|f_w\|_{\mathcal{H}} = \|w\|_2$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The set $\mathcal{H} = \{f_w : w \in \mathbb{R}^d\}$ described in the theorem is the dual of $\mathbb{R}^d$, hence a Hilbert space with the stated inner product (the map $w \mapsto f_w$ is a linear isomorphism onto $\mathcal{H}$, so the inner product is well defined). We check the two RKHS axioms. First, $\mathcal{H}$ contains all the functions $K_x : t \mapsto K(x, t) = \langle x, t \rangle_{\mathbb{R}^d}$, since $K_x = f_x$. Second, for every $x \in \mathbb{R}^d$ and $f_w \in \mathcal{H}$,

$$f_w(x) = \langle w, x \rangle_{\mathbb{R}^d} = \langle f_w, f_x \rangle_{\mathcal{H}} = \langle f_w, K_x \rangle_{\mathcal{H}},$$

which is the reproducing property. $\mathcal{H}$ is thus the RKHS of the linear kernel. $\square$

3.6.2 The polynomial kernel of degree two

Let us find the RKHS of the polynomial kernel of degree 2 on $\mathcal{X} = \mathbb{R}^d$:

$$\forall x, y \in \mathbb{R}^d, \quad K(x, y) = \langle x, y \rangle_{\mathbb{R}^d}^2 = (x^\top y)^2.$$

The derivation illustrates a reusable four-step method.

First step: look for an inner product. Using the invariance of the trace under cyclic permutations and the fact that $x^\top y$ is a scalar,

$$K(x, y) = \text{trace}(x^\top y x^\top y) = \text{trace}(y^\top x x^\top y) = \text{trace}(x x^\top y y^\top) = \langle x x^\top, y y^\top \rangle_F,$$

where $\langle \cdot, \cdot \rangle_F$ is the Frobenius inner product for matrices in $\mathbb{R}^{d \times d}$. Note that in passing we have proven that K is p.d.: it is an inner product after the embedding $x \mapsto x x^\top$.

Second step: propose a candidate RKHS. We know that $\mathcal{H}$ must contain all finite expansions

$$f(x) = \sum_i a_i K(x_i, x) = \sum_i a_i \langle x_i x_i^\top, x x^\top \rangle_F = \left\langle \sum_i a_i x_i x_i^\top, x x^\top \right\rangle_F.$$

Any symmetric matrix in $\mathbb{R}^{d \times d}$ may be decomposed as $\sum_i a_i x_i x_i^\top$ (diagonalize, and take the eigenvectors with their signed eigenvalues as coefficients). Our candidate RKHS is therefore the set of quadratic functions

$$\mathcal{H} = \{f_S : x \mapsto \langle S, xx^\top \rangle_F = x^\top Sx \mid S \in \mathcal{S}^{d \times d}\},$$

where $\mathcal{S}^{d \times d}$ is the set of *symmetric* matrices in $\mathbb{R}^{d \times d}$, endowed with the inner product $\langle f_{S_1}, f_{S_2} \rangle_{\mathcal{H}} = \langle S_1, S_2 \rangle_F$.

Remark (why symmetric matrices?)

The restriction to symmetric matrices is what makes the inner product well defined. A general matrix S and its symmetrization $(S + S^\top)/2$ define the same quadratic function $x \mapsto x^\top Sx$, so on all of $\mathbb{R}^{d \times d}$ the map $S \mapsto f_S$ is not injective, and $\langle S_1, S_2 \rangle_F$ would depend on the matrix representative rather than on the function. On $\mathcal{S}^{d \times d}$ the map is one-to-one: if $x^\top Sx = x^\top S'x$ for all x with S, S' symmetric, then $S = S'$ (polarize).

Third step: check that the candidate is a Hilbert space. This step is trivial in the present case, since $\mathcal{H}$ is a finite-dimensional Euclidean space, isomorphic to $\mathcal{S}^{d \times d}$ via $\Phi : S \mapsto f_S$. Sometimes things are not so simple, and completeness must be proved explicitly.

Fourth step: check that $\mathcal{H}$ is the RKHS.

Proof

1. $\mathcal{H}$ contains all the functions $K_x : t \mapsto K(x, t) = \langle xx^\top, tt^\top \rangle_F$, since $K_x = f_{xx^\top}$ and $xx^\top$ is symmetric.
2. For all $f_S \in \mathcal{H}$ and $x \in \mathcal{X}$,

$$f_S(x) = \langle S, xx^\top \rangle_F = \langle f_S, f_{xx^\top} \rangle_{\mathcal{H}} = \langle f_S, K_x \rangle_{\mathcal{H}}.$$

By uniqueness, $\mathcal{H}$ is the RKHS of the degree-two polynomial kernel. $\square$

Remark (pre-images are rare)

All points $x \in \mathcal{X}$ are mapped to a rank-one matrix $xx^\top$, hence to the function $K_x = f_{xx^\top}$ in $\mathcal{H}$. However, most points of $\mathcal{H}$ do not admit a pre-image in $\mathcal{X}$: a generic symmetric matrix has rank larger than one, or a negative eigenvalue, and therefore cannot be written as $xx^\top$. The image $\Phi(\mathcal{X})$ is a thin curved subset of the feature space, a theme that returns when we study pre-image problems.

Exercise

What is the RKHS of the general polynomial kernel $K(x, y) = \langle x, y \rangle_{\mathbb{R}^d}^k$?

Verification artifact. checks/example-ch01-example-ch01-2.json records the example source hash and verification scope.

3.6.3 The Gaussian kernel

The Gaussian kernel on $\mathcal{X} = \mathbb{R}$ (or, with the same formula, on $\mathbb{R}^d$),

$$K(x, y) = e^{-\frac{1}{2\sigma^2}(x-y)^2},$$

is the kernel behind the bump picture that opened the RKHS section: its canonical feature map sends each point x to the Gaussian function K_x centered at x . It is indeed p.d.; the proof is a two-line application of the combination rules of the next section, so we present it in the worked quiz below. Its RKHS contains far more than the Gaussian bumps themselves, and its precise description is postponed to a later chapter; what matters already is that the general theory guarantees this space exists, is unique, and satisfies $f(x) = \langle f, K_x \rangle_{\mathcal{H}}$ for all its members.

Remark (geometry of the Gaussian embedding)

Schölkopf and Smola (2002) record three facts about this feature map that are worth keeping in mind. First, since $K(x, x) = 1$ for every x , each point maps to a unit-length element of $\mathcal{H}$: the Gaussian embedding sends all of $\mathcal{X}$

to the unit sphere of the RKHS. Second, since $K(x, x') > 0$ for all x, x' , the cosine of the angle between any two images, $\langle \Phi(x), \Phi(x') \rangle_{\mathcal{H}}$, is strictly positive, so the images all sit within a single orthant, subtending pairwise angles below $\pi/2$. Third, pulling the other way, for any finite set of distinct points the Gaussian Gram matrix has full rank, so the images $\Phi(x_1), \dots, \Phi(x_m)$ are always linearly independent. A Gaussian kernel on an infinite domain therefore has an infinite-dimensional RKHS: no finite feature space can reproduce it. The apparent tension, images confined to a patch of a sphere yet never linearly dependent, is resolved by the rapid decay of the eigenvalues, which stacks ever more dimensions into an ever thinner shell.

3.7 Combining kernels: the cone of p.d. kernels

Checking the definition of positive definiteness directly is painful for all but the simplest kernels. In practice one builds complicated kernels from simple ones, using stability properties of the class of p.d. kernels.

Theorem (closure properties)

If K_1 and K_2 are p.d. kernels on $\mathcal{X}$, then

- $K_1 + K_2$,
- $K_1 K_2$ (the pointwise product), and
- cK_1 , for $c \geq 0$,

are also p.d. kernels. Moreover, if $(K_i)_{i \geq 1}$ is a sequence of p.d. kernels that converges pointwise to a function K ,

$$\forall (x, x') \in \mathcal{X}^2, \quad K(x, x') = \lim_{i \rightarrow \infty} K_i(x, x'),$$

then K is also a p.d. kernel.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The sum, scaling and limit statements are left as exercises, and they are one-liners from the definition: the quadratic form $\sum_{i,j} a_i a_j K(x_i, x_j)$ is additive in K , homogeneous under $K \mapsto cK$, and passes to pointwise limits since a limit of nonnegative numbers is nonnegative (symmetry also passes to the limit). Together they say that the p.d. kernels on $\mathcal{X}$ form a *convex cone, closed under pointwise convergence*, and the product rule adds that this cone is stable under multiplication. This cone is the playground of kernel design: the entire kernel jungle of the later chapters lives inside it.

The only nontrivial statement is the product, a result also known as the Schur product theorem. The proof is a factorization argument.

Proof ($K_1 K_2$ is p.d.)

Consider n points in $\mathcal{X}$ and the corresponding $n \times n$ positive semidefinite kernel matrices $\mathbf{K}_1$ and $\mathbf{K}_2$. As p.s.d. matrices, they admit factorizations $\mathbf{K}_1 = X^T X$ and $\mathbf{K}_2 = Y^T Y$ (for instance via the spectral decomposition). Writing x_i and y_i for the i -th columns of X and Y respectively, the entrywise product matrix $\mathbf{K}$ satisfies

$$[\mathbf{K}]_{ij} = [\mathbf{K}_1]_{ij} [\mathbf{K}_2]_{ij} = \text{trace}((x_i^T x_j)(y_j^T y_i)) = \text{trace}((y_i x_i^T)(x_j y_j^T)) = \langle x_i y_i^T, x_j y_j^T \rangle_F = \langle z_i, z_j \rangle_{\mathbb{R}^{n^2}},$$

where $z_i = \text{vec}(x_i y_i^T)$. The matrix $\mathbf{K}$ is therefore a Gram matrix of the vectors $z_1, \dots, z_n$, hence p.s.d., and since the points were arbitrary, $K = K_1 K_2$ is a p.d. kernel. $\square$

Remark (normalisation, and a decomposition)

Two consequences of the closure rules recur often enough to deserve names. Shawe-Taylor and Cristianini (2004) call the first *normalisation*. Writing $g(x) = K(x, x)^{1/2}$, the normalised kernel

$$\tilde{K}(x, x') = \frac{K(x, x')}{K(x, x)^{1/2} K(x', x')^{1/2}}$$

is the product of K with the separable kernel $(x, x') \mapsto g(x)^{-1}g(x')^{-1}$, hence p.d. by the product rule; it is the kernel of the feature map $x \mapsto \Phi(x)/\|\Phi(x)\|_{\mathcal{H}}$, the embedded points pushed onto the unit sphere, and its diagonal is identically 1, the very property that made the Gaussian images unit vectors. Read backwards, the same identity is a *decomposition*: every kernel factors as the product of its normalisation and the rank-one kernel $g(x)g(x')$, which Shawe-Taylor and Cristianini call the Schur-product form of the kernel.

A first payoff: the exercise on polynomial kernels is now immediate, since $\langle x, x' \rangle^d$ is a d -fold product of the linear kernel with itself. A second, more spectacular payoff combines products, sums and limits.

Theorem (exponential of a kernel)

If K is a kernel, then e^K , defined by $e^K(x, x') = \exp(K(x, x'))$, is a kernel too.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Write the exponential as a pointwise limit of its Taylor partial sums:

$$e^{K(x, x')} = \lim_{n \rightarrow +\infty} \sum_{i=0}^n \frac{K(x, x')^i}{i!}.$$

Each power K^i is p.d. by the product rule (and $K^0 = 1$ is p.d., being the inner product after the constant embedding $\Phi(x) = 1$); each partial sum is a combination of p.d. kernels with nonnegative coefficients $1/i!$, hence p.d.; and the pointwise limit of p.d. kernels is p.d. $\square$

3.8 Worked examples: a kernel quiz

Here is a quiz: which of the following thirteen functions are p.d. kernels? The answers exercise every tool assembled so far, embeddings, closure rules, and the humble necessary conditions hidden in the definition. Before reading the solutions, recall the two cheapest ways to disqualify a candidate: taking $N = 1$ in the definition shows that a p.d. kernel must satisfy $K(x, x) \geq 0$ on the diagonal, and taking $N = 2$ shows that every 2×2 similarity matrix must be p.s.d., i.e., $K(x, x)K(x', x') \geq K(x, x')^2$.

#	$\mathcal{X}$	$K(x, x')$	p.d.?	Key argument
1	$(-1, 1)$	$\frac{1}{1 - xx'}$	Yes	geometric series of p.d. kernels
2	$\mathbb{N}$	$2^{x+x'}$	Yes	separable: $\Phi(x) = 2^x$
3	$\mathbb{N}$	$2^{xx'}$	Yes	exponential of a p.d. kernel
4	$\mathbb{R}_+$	$\log(1 + xx')$	No	a 2×2 matrix fails
5	$\mathbb{R}$	$\exp(- x - x' ^2)$	Yes	separable factor times $e^{2xx'}$
6	$\mathbb{R}$	$\cos(x + x')$	No	negative on the diagonal
7	$\mathbb{R}$	$\cos(x - x')$	Yes	$\Phi(x) = (\cos x, \sin x)$
8	$\mathbb{R}_+$	$\min(x, x')$	Yes	overlap of indicators in L^2
9	$\mathbb{R}_+$	$\max(x, x')$	No	a 2×2 matrix fails
10	$\mathbb{R}_+$	$\min(x, x')/\max(x, x')$	Yes	product of two p.d. kernels
11	$\mathbb{N}$	$\text{GCD}(x, x')$	Yes	Euler totient expansion
12	$\mathbb{N}$	$\text{LCM}(x, x')$	No	a 2×2 matrix fails
13	$\mathbb{N}$	$\text{GCD}(x, x')/\text{LCM}(x, x')$	Yes	GCD^2 times a separable kernel

#	$\mathcal{X}$	$K(x, x')$	p.d.?	Key argument
---	---------------	------------	-------	--------------

We now justify each verdict, grouping the kernels by the technique that decides them. (For the kernels involving quotients on $\mathbb{R}_+$, and those on $\mathbb{N}$, we take the points to be positive so that all expressions are defined.)

3.8.1 Series, exponentials and separable factors (1, 2, 3, 5)

1. $K(x, x') = \frac{1}{1-xx'}$ **on** $(-1, 1)$: **p.d.** Since $|xx'| < 1$, the geometric series converges:

$$\frac{1}{1-xx'} = \sum_{n=0}^{\infty} (xx')^n = \lim_{N \rightarrow \infty} \sum_{n=0}^N (xx')^n.$$

Each term $(xx')^n = x^n \cdot x'^n$ is p.d. (it is the product kernel of $\mathbb{R}$ after the embedding $x \mapsto x^n$, or the n -fold product of the kernel xx' with itself); each partial sum is p.d. by the sum rule; and the pointwise limit is p.d. by the limit rule.

2. $K(x, x') = 2^{x+x'}$ **on** $\mathbb{N}$: **p.d.** It factorizes as a separable, rank-one kernel:

$$2^{x+x'} = 2^x \cdot 2^{x'} = \Phi(x) \Phi(x'), \quad \Phi(x) = 2^x \in \mathbb{R}.$$

This is an inner product after an embedding into the one-dimensional Hilbert space $\mathbb{R}$, hence p.d. The same argument shows that any kernel of the form $g(x)g(x')$ is p.d., a fact reused below.

3. $K(x, x') = 2^{xx'}$ **on** $\mathbb{N}$: **p.d.** Write $2^{xx'} = e^{(\ln 2)xx'}$. The kernel xx' is p.d., the scaling $(\ln 2)xx'$ is p.d. since $\ln 2 \geq 0$, and the exponential of a p.d. kernel is p.d.

5. **The Gaussian kernel** $K(x, x') = \exp(-|x - x'|^2)$ **on** $\mathbb{R}$: **p.d.** Expand the square in the exponent:

$$e^{-(x-x')^2} = e^{-x^2} \cdot e^{-x'^2} \cdot e^{2xx'}.$$

The factor $e^{-x^2}e^{-x'^2}$ is separable of the form $g(x)g(x')$, hence p.d.; the factor $e^{2xx'}$ is the exponential of the p.d. kernel $2xx'$, hence p.d.; and the product of two p.d. kernels is p.d. The same argument works in $\mathbb{R}^d$ and for any bandwidth, and settles the kernel behind the bump picture of the RKHS section.

3.8.2 Trigonometric kernels (6, 7)

7. $K(x, x') = \cos(x - x')$ **on** $\mathbb{R}$: **p.d.** The addition formula exhibits an explicit two-dimensional feature map:

$$\cos(x - x') = \cos x \cos x' + \sin x \sin x' = \langle \Phi(x), \Phi(x') \rangle_{\mathbb{R}^2}, \quad \Phi(x) = (\cos x, \sin x).$$

6. $K(x, x') = \cos(x + x')$ **on** $\mathbb{R}$: **not p.d.** It fails on the diagonal: $K(x, x) = \cos(2x)$, so at $x = \pi/2$ we get $K(x, x) = \cos \pi = -1 < 0$, violating the definition already with $N = 1$ and $a_1 = 1$. The sign flip relative to the previous case, $\cos(x + x') = \cos x \cos x' - \sin x \sin x'$, is a *difference* of p.d. kernels, and differences leave the cone.

3.8.3 Minima and maxima (8, 9, 10)

8. $K(x, x') = \min(x, x')$ **on** $\mathbb{R}_+$: **p.d.** Embed each $x \geq 0$ as the indicator function of the interval $[0, x]$ in the Hilbert space $L^2(\mathbb{R}_+)$, that is, $\Phi(x) = \mathbf{1}_{[0, x]}$. Then

$$\langle \Phi(x), \Phi(x') \rangle_{L^2} = \int_0^\infty \mathbf{1}_{\{t \leq x\}} \mathbf{1}_{\{t \leq x'\}} dt = \min(x, x'),$$

an inner product after an embedding into a Hilbert space, hence p.d. by Aronszajn's theorem. (Readers who know stochastic processes will recognize the covariance function of Brownian motion.)

9. $K(x, x') = \max(x, x')$ **on** $\mathbb{R}_+$: **not p.d.** Take the two points $x_1 = 0, x_2 = 1$ and the coefficients $a = (1, -1)$:

$$\sum_{i,j} a_i a_j K(x_i, x_j) = \max(0, 0) - 2 \max(0, 1) + \max(1, 1) = 0 - 2 + 1 = -1 < 0.$$

Equivalently, the similarity matrix of these two points has determinant $0 \cdot 1 - 1 \cdot 1 = -1$ and is not p.s.d.

10. $K(x, x') = \min(x, x') / \max(x, x')$ **on positive reals**: **p.d.** Write it as a product of two p.d. kernels:

$$\frac{\min(x, x')}{\max(x, x')} = \min(x, x') \cdot \min\left(\frac{1}{x}, \frac{1}{x'}\right),$$

using the identity $1/\max(x, x') = \min(1/x, 1/x')$ for positive numbers. The first factor is p.d. by example 8. The second factor is the p.d. kernel $\min$ composed with the map $\sigma(x) = 1/x$; and if K is p.d. then $(x, x') \mapsto K(\sigma(x), \sigma(x'))$ is p.d. for *any* map σ , since the definition only involves evaluations at finitely many points, which σ merely relabels. The product rule concludes.

3.8.4 Arithmetic kernels (11, 12, 13)

11. $K(x, x') = \text{GCD}(x, x')$ **on positive integers**: **p.d.** Gauss's identity for the Euler totient function φ states that $n = \sum_{d|n} \varphi(d)$ for every positive integer n . Applying it to $n = \text{gcd}(x, x')$, and noting that d divides $\text{gcd}(x, x')$ exactly when d divides both x and x' :

$$\text{gcd}(x, x') = \sum_{d=1}^{\infty} \varphi(d) \mathbf{1}_{\{d \mid x\}} \mathbf{1}_{\{d \mid x'\}} = \langle \Phi(x), \Phi(x') \rangle, \quad \Phi(x) = (\sqrt{\varphi(d)} \mathbf{1}_{\{d \mid x\}})_{d \geq 1},$$

a sum with nonnegative weights of separable p.d. kernels (each vector $\Phi(x)$ has finitely many nonzero entries, so the inner product is a finite sum). Hence GCD is p.d.

12. $K(x, x') = \text{LCM}(x, x')$ **on positive integers**: **not p.d.** The same counterexample shape as for $\max$: take $x_1 = 1, x_2 = 2$ and $a = (1, -1)$:

$$\text{lcm}(1, 1) - 2 \text{lcm}(1, 2) + \text{lcm}(2, 2) = 1 - 4 + 2 = -1 < 0.$$

13. $K(x, x') = \text{GCD}(x, x') / \text{LCM}(x, x')$ **on positive integers**: **p.d.** Use the identity $\text{gcd}(x, x') \cdot \text{lcm}(x, x') = xx'$ to rewrite

$$\frac{\text{gcd}(x, x')}{\text{lcm}(x, x')} = \frac{\text{gcd}(x, x')^2}{xx'} = \text{gcd}(x, x')^2 \cdot \frac{1}{x} \cdot \frac{1}{x'}.$$

The factor gcd^2 is a product of two p.d. kernels (example 11, squared by the product rule), and the factor $(1/x)(1/x')$ is separable of the form $g(x)g(x')$, hence p.d. The product rule concludes.

3.8.5 A cautionary counterexample (4)

4. $K(x, x') = \log(1 + xx')$ on $\mathbb{R}_+$: not p.d. The kernel looks tempting because xx' is p.d. and the logarithm grows slowly, but no combination rule applies: the Taylor series $\log(1 + u) = \sum_{n \geq 1} (-1)^{n+1} u^n / n$ has alternating signs, so the series argument that worked for $1/(1 - xx')$ and e^K breaks down. And indeed the kernel is not p.d. Take the two points $x_1 = 1$ and $x_2 = 2$. The similarity matrix is

$$\begin{pmatrix} \log 2 & \log 3 \\ \log 3 & \log 5 \end{pmatrix},$$

whose determinant is $\log 2 \cdot \log 5 - (\log 3)^2 \approx 1.1156 - 1.2069 < 0$. A symmetric 2×2 matrix with negative determinant has a negative eigenvalue, so the matrix is not p.s.d. and K is not p.d. The moral: positive definiteness must be proved, not assumed from the shape of the formula, and a small explicit matrix is often the fastest judge.

3.9 The RKHS norm as a smoothness functional

We closed the definition of the RKHS with a learning objective penalized by $\|f\|_{\mathcal{H}}^2$. Why is this norm a sensible thing to penalize? Because the RKHS norm measures how fast a function can vary.

Remember the RKHS of the linear kernel: for $K_{\text{lin}}(x, x') = x^\top x'$, the functions are $f(x) = w^\top x$ with $\|f\|_{\mathcal{H}} = \|w\|_2$. Picture the level lines of f drawn for $\|f\| = 0.5, 1$ and 2 : the larger the norm, the steeper the linear function and the more tightly packed its level sets, while a small-norm function is nearly flat. The general phenomenon is a direct consequence of the reproducing property.

By Cauchy-Schwarz we have, for any function $f \in \mathcal{H}$ and any two points $x, x' \in \mathcal{X}$:

$$|f(x) - f(x')| = |\langle f, K_x - K_{x'} \rangle_{\mathcal{H}}| \leq \|f\|_{\mathcal{H}} \times \|K_x - K_{x'}\|_{\mathcal{H}} = \|f\|_{\mathcal{H}} \times d_K(x, x'),$$

where the first equality uses the reproducing property at x and at x' , and where

$$d_K(x, x') := \|K_x - K_{x'}\|_{\mathcal{H}} = (K(x, x) - 2K(x, x') + K(x', x'))^{1/2}$$

is the distance between the embedded points $\Phi(x) = K_x$ and $\Phi(x') = K_{x'}$. In other words, the kernel equips the raw set $\mathcal{X}$ with a geometry, and every function of the RKHS is Lipschitz with constant $\|f\|_{\mathcal{H}}$ with respect to that geometry: the norm of a function in the RKHS controls how fast the function varies over $\mathcal{X}$.

Important message

Small norm $\implies$ slow variations. Penalizing $\|f\|_{\mathcal{H}}^2$ in the empirical risk is a smoothness penalty, with smoothness measured in the metric d_K that the kernel itself defines.

This is the interpretation that makes the learning objective of the RKHS section trustworthy: raising the penalty weight λ forces $\|f\|_{\mathcal{H}}$ down, and a smaller norm buys a flatter, more slowly varying predictor. The figure below fits a kernel model to points you can drag, and lets you turn both the kernel bandwidth and the ridge weight λ ; watch the fitted function stiffen into a smooth curve as λ grows, and relax toward interpolation as λ shrinks.

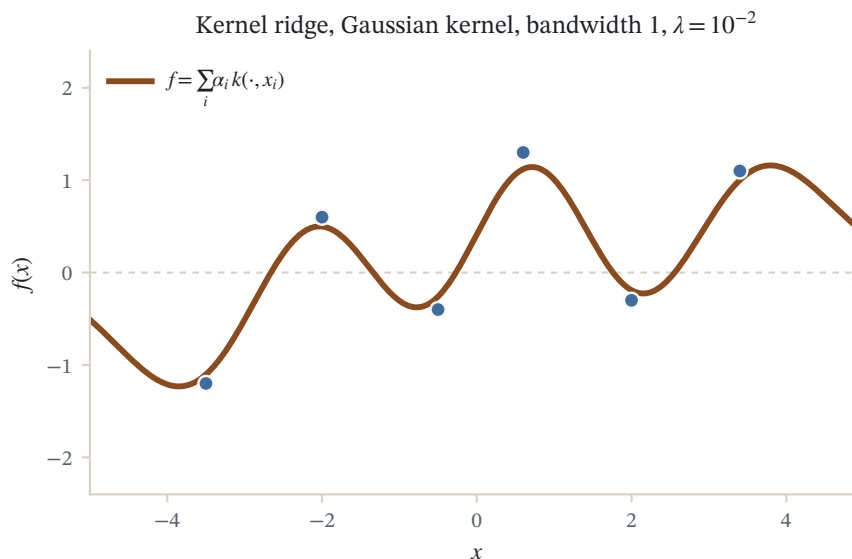


Figure 3.2: Kernel ridge regression on six points with a Gaussian kernel (bandwidth 1, ridge $\lambda = 10^{-2}$). The curve is the exact solution $f = \sum_i \alpha_i k(\cdot, x_i)$ with $\alpha = (K + \lambda n I)^{-1} y$.

3.10 Summary

The chapter can be compressed into four statements, each of which the sequel uses constantly.

- Positive definite kernels can be thought of as inner products after embedding the data space $\mathcal{X}$ into some Hilbert space. As such, a p.d. kernel defines a metric on $\mathcal{X}$.
- A realization of this embedding is the RKHS, valid without restriction on the space $\mathcal{X}$ nor on the kernel: p.d. kernels and reproducing kernels are the same objects, and each kernel has exactly one RKHS.
- The RKHS is a space of functions over $\mathcal{X}$, in which evaluation is the inner product against the embedded points: $f(x) = \langle f, K_x \rangle_{\mathcal{H}}$.
- The norm of a function in the RKHS is related to its degree of smoothness with respect to the metric defined by the kernel on $\mathcal{X}$, which is why it serves as a regularizer.

We will now see applications of kernels and RKHS in statistics, before coming back to the problem of choosing, and eventually designing, the kernel. Readers who want the same foundations developed at book length, with many further examples, will find them in the standard monographs of Schölkopf and Smola (2002) and of Shawe-Taylor and Cristianini (2004).

Further reading

The reproducing-kernel foundations of this chapter are developed at book length by Schölkopf and Smola (2002), *Learning with Kernels*, whose chapter 2 introduces positive definite kernels, the reproducing kernel Hilbert space, and the basic constructions, and by Shawe-Taylor and Cristianini (2004), *Kernel Methods for Pattern Analysis*, whose chapters 2 and 3 build the Gram matrix, the feature-space viewpoint, and the properties of kernels. The original source for the correspondence between positive definite kernels and Hilbert spaces of functions is Aronszajn (1950). The parallel development of reproducing kernels in statistics, where the RKHS norm reappears as the roughness penalty of a smoothing spline, is collected in the monograph of Wahba (1990).

3.11 Common mistakes and practical implications

For **Positive Definite Kernels and RKHS**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

3.12 Exercises

1. warm-up Two necessary conditions fall out of the definition by choosing small coefficient vectors. Taking $N = 1$, show that any p.d. kernel satisfies $K(x, x) \geq 0$ for every $x \in \mathcal{X}$. Taking $N = 2$ with points x, x' and the matrix it produces, show that $K(x, x')^2 \leq K(x, x)K(x', x')$. Then use the first condition alone to explain in one line why $K(x, x') = \cos(x + x')$ is not p.d. on $\mathbb{R}$.
2. computation Let $\mathcal{X} = \mathbb{R}_+$ and $K(x, x') = \min(x, x')$. Write down the 3×3 Gram matrix $\mathbf{K}$ for the points $x_1 = 1, x_2 = 2, x_3 = 3$. Then exhibit three vectors $z_1, z_2, z_3 \in \mathbb{R}^3$ with $\langle z_i, z_j \rangle_{\mathbb{R}^3} = [\mathbf{K}]_{ij}$, and explain how they are the finite-dimensional shadow of the feature map $\Phi(x) = \mathbf{1}_{[0, x]}$ used to prove that $\min$ is p.d. Hint

The matrix has entry $\min(i, j)$. Try the staircase vectors $z_i = (\underbrace{1, \dots, 1}_i, 0, \dots)$; the number of shared leading ones between z_i and z_j is $\min(i, j)$, exactly as $\int \mathbf{1}\{t \leq x\} \mathbf{1}\{t \leq x'\} dt = \min(x, x')$.

3. computation Consider the inhomogeneous kernel $K(x, x') = (1 + xx')^2$ on $\mathcal{X} = \mathbb{R}$. Expand it into monomials, read off an explicit feature map $\Phi : \mathbb{R} \rightarrow \mathbb{R}^3$ with $K(x, x') = \langle \Phi(x), \Phi(x') \rangle_{\mathbb{R}^3}$, and conclude that K is p.d. two ways: from the feature map, and from the closure rules applied to the constant, linear, and squared-linear kernels. Hint

$(1 + xx')^2 = 1 + 2xx' + x^2x'^2$. Match this against $\langle \Phi(x), \Phi(x') \rangle$ with $\Phi(x) = (1, \sqrt{2}x, x^2)$; the $\sqrt{2}$ plays the same multiplicity-balancing role as in the degree-two map of the text.

4. proof Prove directly from the definition that the p.d. kernels on a set $\mathcal{X}$ form a convex cone: if K_1, K_2 are p.d. and $c_1, c_2 \geq 0$, then $c_1K_1 + c_2K_2$ is p.d. Then show that a pointwise limit of p.d. kernels is p.d. (You may take symmetry of each K_i as given; check that it too passes to the combination.) Hint

The quadratic form $\sum_{i,j} a_i a_j K(x_i, x_j)$ is linear in K , so it splits as $c_1(\geq 0) + c_2(\geq 0) \geq 0$. For the limit, a limit of nonnegative reals is nonnegative.

5. proof Verify the reproducing property by hand in the RKHS of the linear kernel on $\mathbb{R}^2$, where $K(x, x') = \langle x, x' \rangle$ and functions are $f_w(x) = \langle w, x \rangle$ with $\langle f_w, f_v \rangle_{\mathcal{H}} = \langle w, v \rangle$. Fix $w = (2, -1)$ and $x = (3, 5)$. Compute $f_w(x)$ directly, then identify the kernel section K_x as some f_v , compute $\langle f_w, K_x \rangle_{\mathcal{H}}$, and confirm the two numbers agree. State which v realizes K_x . Hint

$K_x(t) = \langle x, t \rangle = f_x(t)$, so $K_x = f_x$ with $v = x$. Then $\langle f_w, K_x \rangle_{\mathcal{H}} = \langle w, x \rangle = f_w(x)$; both should give $6 - 5 = 1$.

6. proof Show that $K(x, x') = (x - x')^2$ is *not* a p.d. kernel on $\mathbb{R}$, even though it is symmetric and its diagonal vanishes. Do so by writing the 2×2 Gram matrix for the points $x_1 = 0, x_2 = 1$, reading off its eigenvalues, and exhibiting coefficients (a_1, a_2) that make the quadratic form strictly negative. Contrast this with $\min(x, x')$, which the text shows is p.d. Hint

The matrix is $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, with eigenvalues ± 1 ; the eigenvector for -1 gives $a = (1, -1)$ and quadratic form $-2 < 0$.

3 Positive Definite Kernels and RKHS

7. exploration Open the two interactive figures of this chapter. In the Gram-matrix heatmap, place a handful of points on the line, switch between the linear, polynomial, and Gaussian kernels, and sweep the bandwidth: confirm that the smallest eigenvalue of the displayed matrix never drops below zero, and describe what shrinking the Gaussian bandwidth does to the off-diagonal entries and to the spread of eigenvalues. Then in the kernel-lab figure, drag a few data points and raise the ridge weight λ : report what happens to the fitted curve and relate the stiffening you observe to the bound $|f(x) - f(x')| \leq \|f\|_{\mathcal{H}} d_K(x, x')$ from the smoothness section.
8. challenge The kernel equips $\mathcal{X}$ with the distance $d_K(x, x') = \|K_x - K_{x'}\|_{\mathcal{H}} = (K(x, x) - 2K(x, x') + K(x', x'))^{1/2}$. Using the feature map $\Phi(x) = K_x$ into the Hilbert space $\mathcal{H}$, prove that d_K satisfies the triangle inequality $d_K(x, z) \leq d_K(x, y) + d_K(y, z)$ for all $x, y, z \in \mathcal{X}$. Then argue that d_K is only a pseudometric in general by describing when $d_K(x, x') = 0$ can occur for distinct $x \neq x'$, and give one concrete kernel where this happens. Hint

$d_K(x, y) = \|\Phi(x) - \Phi(y)\|_{\mathcal{H}}$ is a distance induced by a norm, so the triangle inequality is inherited from $\|\cdot\|_{\mathcal{H}}$. It degenerates exactly when $\Phi(x) = \Phi(x')$; for instance a constant kernel $K \equiv 1$ collapses every point to the same image.

4 The Kernel Trick and the Representer Theorem

Chapter 2, Positive Definite Kernels and RKHS built the theory: positive definite kernels are exactly inner products after an embedding $\Phi : \mathcal{X} \rightarrow \mathcal{H}$, and each kernel carries its own function space, the RKHS. This chapter turns that theory into a way of computing. Two results do all the work. The kernel trick says that any algorithm which touches its data only through pairwise inner products can be run, unchanged, in the feature space of any p.d. kernel, even an infinite-dimensional one. The representer theorem says that a large class of learning problems posed over the entire RKHS actually have their solutions in an n -dimensional subspace spanned by the training points. Together they explain how kernel methods can be simultaneously nonlinear, nonparametric, and computationally concrete, and they give a general recipe, used throughout the rest of the book, for turning a linear algorithm into a kernel method.

4.1 Why map data to a Hilbert space?

Before stating the trick, it helps to recall the kind of problem it is designed for. In supervised learning we want a prediction function $f : \mathcal{X} \rightarrow \mathcal{Y}$, given labeled training data $(x_i, y_i)_{i=1, \dots, n}$ with $x_i \in \mathcal{X}$ and $y_i \in \mathcal{Y}$. The dominant template, going back to Vapnik (1995), is regularized empirical risk minimization:

$$\min_{f \in \mathcal{F}} \underbrace{\frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i))}_{\text{empirical risk, data fit}} + \underbrace{\lambda \Omega(f)}_{\text{regularization}}.$$

The first term asks f to fit the data; the second penalizes complexity so that f also behaves sensibly away from the data. The output set $\mathcal{Y}$ determines the flavor of the problem: $y_i \in \{-1, +1\}$ for binary classification, $y_i \in \{1, \dots, K\}$ for multi-class classification, $y_i \in \mathbb{R}$ for regression, and $y_i \in \mathbb{R}^k$ for multivariate regression.

The most familiar instance is the linear model, as in logistic regression: one assumes a linear relation between y and features $x \in \mathbb{R}^p$, takes $f(x) = w^\top x + b$ parametrized by $(w, b) \in \mathbb{R}^{p+1}$, chooses L to be a convex loss, and takes $\Omega(f)$ to be the squared ℓ_2 -norm $\|w\|^2$. Linear models are easy to fit and easy to regularize, but they are limited twice over: they only express linear relationships, and they only apply when $\mathcal{X}$ is a vector space in the first place.

Kernel methods remove both limitations at once by moving the problem into the RKHS. We map the data into a Hilbert space and work with linear forms there:

$$\Phi : \mathcal{X} \rightarrow \mathcal{H}, \quad f(x) = \langle \Phi(x), f \rangle_{\mathcal{H}},$$

and we solve the learning problem over the RKHS, with the RKHS norm as the regularizer:

$$\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2.$$

The picture to keep in mind is a cloud of raw objects x in $\mathcal{X}$, which may have no useful geometry of its own, sent by Φ into a Hilbert space $\mathcal{H}$ where each image $\Phi(x)$ is a point in an honest vector space and predictors are simply inner products against a fixed vector f .

4.1.1 Two purposes of the embedding

Why is this a good idea? There are two distinct motivations, and it is worth keeping them separate.

First purpose: gain a geometry. Embedding data in a vector space gives access to the entire toolbox of Euclidean and Hilbertian geometry: angle computation, projection onto linear subspaces, definition of barycenters, and so on. It lets us learn potentially rich, infinite-dimensional models, and it makes regularization natural and theoretically grounded, since $\|f\|_{\mathcal{H}}$ measures the smoothness of the predictor. Crucially, the principle is generic: it assumes nothing about the nature of the set $\mathcal{X}$, which may consist of vectors, sets, graphs, or sequences. Any set carrying a p.d. kernel inherits this geometry.

Second purpose: fix a geometry you do not like. Even when $\mathcal{X} = \mathbb{R}^p$ already is a vector space, its Euclidean structure may be the wrong one for the task. We can then lift the data to a higher-dimensional space with nicer properties, such as linear separability or a clearer clustering structure. The classic two-dimensional illustration: data (x_1, x_2) that are separated by a circle of radius R in the plane, hence not linearly separable there, become linearly separable after the map $(x_1, x_2) \mapsto (x_1^2, x_2^2)$, because the circle $x_1^2 + x_2^2 = R^2$ turns into a straight line in the new coordinates. The linear form $f(x) = \langle \Phi(x), f \rangle_{\mathcal{H}}$ in $\mathcal{H}$ then corresponds to a nonlinear model back in $\mathcal{X}$.

It helps to see the arithmetic once. Consider the map $\Phi(x) = (x_1, x_2, x_1^2 + x_2^2)$ that appends to each plane point the squared radius on which it sits. Two classes separated by a circle, an inner blob inside a surrounding ring, cannot be told apart by any straight line in the plane. But after the lift the ring is pushed up along the new third axis while the inner blob stays low, so a single flat plane, which is nothing but a threshold on $x_1^2 + x_2^2$, slices them apart. That plane is a linear predictor in $\mathcal{H}$ and a circle back in $\mathcal{X}$. The inner product in the lifted space works out to $\langle \Phi(x), \Phi(x') \rangle = x_1 x_1' + x_2 x_2' + (x_1^2 + x_2^2)(x_1'^2 + x_2'^2)$, a formula written entirely in the original coordinates: we never had to store the extra coordinate to make use of it. That last observation, that the inner product in the enlarged space is a closed-form function of the raw inputs, is exactly the economy the next section turns into a general principle.

The figure below lets you raise the lift by hand and watch the separating plane, which no line in the plane could match, appear as the two rings pull apart along the new dimension.

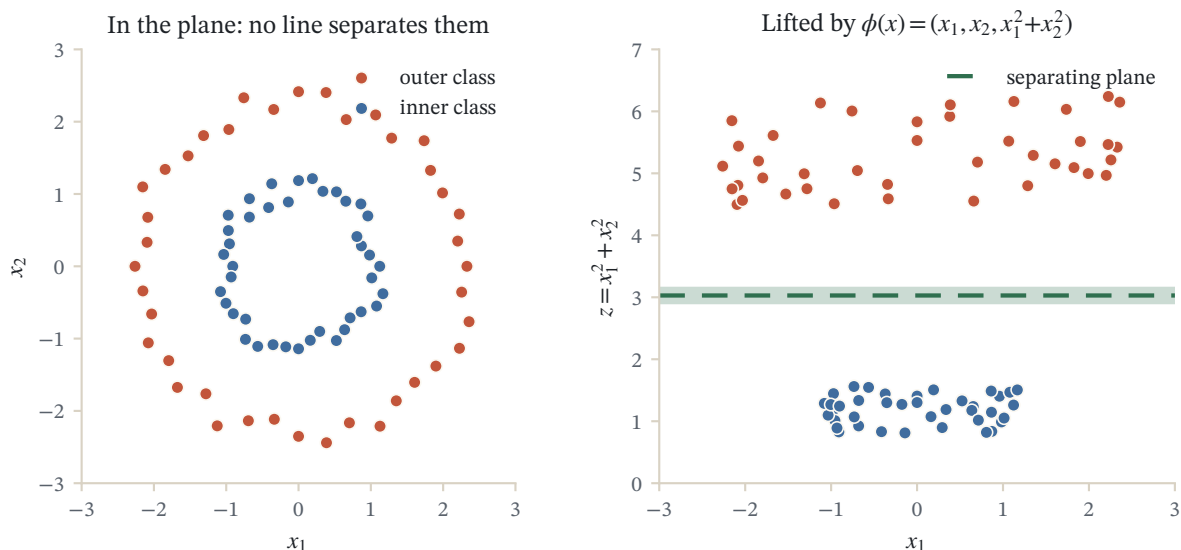


Figure 4.1: Two classes that no line separates in the plane become linearly separable under the lift $\phi(x) = (x_1, x_2, x_1^2 + x_2^2)$: a flat plane slices the inner ring off cleanly. The polynomial kernel computes $\langle \phi(x), \phi(x') \rangle$ without ever building the third coordinate.

All of this raises an obvious objection: the feature space may be huge, even infinite-dimensional, so how could we ever compute in it? Two theoretical results underpin the family of algorithms known collectively as kernel

methods, whose standard book-length treatment is Schölkopf and Smola (2002), and together they answer this objection. The kernel trick rests on the representation of p.d. kernels as inner products; the representer theorem rests on properties of the regularization functional defined by the RKHS norm. We take them in turn.

4.2 The kernel trick

The first result is almost embarrassingly simple to state and prove, which should not distract from how much it changes in practice. The idea has a long history. Aizerman, Braverman, and Rozonoer (1964) already used kernels as inner products in an implicit feature space in their method of potential functions, and Boser, Guyon, and Vapnik (1992) brought the same device into the maximum-margin classifier, where it became the engine of the support vector machine and set off the modern development of kernel methods.

Proposition (the kernel trick)

Any algorithm to process finite-dimensional vectors that can be expressed only in terms of pairwise inner products can be applied to potentially infinite-dimensional vectors in the feature space of a p.d. kernel, by replacing each inner product evaluation by a kernel evaluation.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The proof is trivial, because the kernel *is* exactly the inner product in the feature space: by the Aronszajn (1950) theorem of Chapter 2, Positive Definite Kernels and RKHS, $K(x, x') = \langle \Phi(x), \Phi(x') \rangle_{\mathcal{H}}$. If an algorithm never looks at coordinates of its input vectors and only ever asks for inner products between pairs of them, then feeding it the values $K(x_i, x_j)$ is indistinguishable, from the algorithm's point of view, from running it on the vectors $\Phi(x_1), \dots, \Phi(x_n)$ themselves.

Despite its triviality, the trick has huge practical applications, and one conceptual consequence worth stating explicitly: vectors in the feature space are only ever manipulated *implicitly*, through pairwise inner products. We never write down $\Phi(x)$, we never need to, and in many cases we could not even if we wanted to. The rest of this section makes this concrete through three worked examples: computing distances, comparing a point to a set, and centering data, all in the feature space, all without ever leaving the world of kernel evaluations.

Read operationally, the proposition is a test followed by a procedure. The test: does the algorithm ever inspect a coordinate of an input vector, or does it only ever ask for numbers of the form $\langle x_i, x_j \rangle$? If only the latter, the procedure is to compute the $n \times n$ matrix of kernel values $K(x_i, x_j)$ once, then feed those numbers to the algorithm wherever it expected inner products. The code of the algorithm does not change; only the meaning of the numbers it consumes does. Nearest-neighbor classification is the fastest illustration: it needs only distances between points, and we are about to see that distances are assembled from kernel values, so it kernelizes for free.

4.2.1 Example 1: distances in the feature space

The most basic geometric quantity is the distance between the images of two points $x_1, x_2 \in \mathcal{X}$. Expanding the squared norm through the inner product:

$$\begin{aligned} d_K(x_1, x_2)^2 &= \|\Phi(x_1) - \Phi(x_2)\|_{\mathcal{H}}^2 \\ &= \langle \Phi(x_1) - \Phi(x_2), \Phi(x_1) - \Phi(x_2) \rangle_{\mathcal{H}} \\ &= \langle \Phi(x_1), \Phi(x_1) \rangle_{\mathcal{H}} + \langle \Phi(x_2), \Phi(x_2) \rangle_{\mathcal{H}} - 2\langle \Phi(x_1), \Phi(x_2) \rangle_{\mathcal{H}}, \end{aligned}$$

and every term on the right is a kernel evaluation:

$$d_K(x_1, x_2)^2 = K(x_1, x_1) + K(x_2, x_2) - 2K(x_1, x_2).$$

4 The Kernel Trick and the Representer Theorem

So every p.d. kernel on $\mathcal{X}$ induces a distance on $\mathcal{X}$, computable from three kernel values, whatever $\mathcal{X}$ is and however large $\mathcal{H}$ is.

Example (distance for the Gaussian kernel)

Take the Gaussian kernel with bandwidth σ on $\mathbb{R}^d$:

$$K(x, y) = e^{-\frac{\|x-y\|^2}{2\sigma^2}}.$$

Since $K(x, x) = 1 = \|\Phi(x)\|_{\mathcal{H}}^2$ for every x , all points are mapped onto the unit sphere of the feature space. The distance between the images of two points x and y is

$$d_K(x, y) = \sqrt{2 \left(1 - e^{-\frac{\|x-y\|^2}{2\sigma^2}}\right)}.$$

As a function of the Euclidean distance $\|x - y\|$, this grows from 0, is approximately proportional to $\|x - y\|/\sigma$ for nearby points, and saturates at $\sqrt{2}$ as the points move far apart: beyond a few bandwidths, all pairs look maximally dissimilar. The bandwidth σ sets the scale at which the feature-space geometry stops resolving distances in the input space.

Verification artifact. checks/example-ch02-example-3-1.json records the example source hash and verification scope.

The Gaussian example is abstract because its feature map is infinite-dimensional. To watch the trick actually reproduce a feature-space computation, we take a kernel whose feature map is small enough to write down, and compute the same two quantities both ways.

Example (the kernel trick on a polynomial kernel, two ways)

Take the homogeneous quadratic kernel $K(x, y) = \langle x, y \rangle^2$ on $\mathbb{R}^2$ and the two points $x_a = (1, 2)$ and $x_b = (2, 1)$. This kernel has an explicit finite feature map $\Phi(x) = (x_1^2, \sqrt{2} x_1 x_2, x_2^2) \in \mathbb{R}^3$, because $\langle \Phi(x), \Phi(y) \rangle = x_1^2 y_1^2 + 2x_1 x_2 y_1 y_2 + x_2^2 y_2^2 = \langle x, y \rangle^2$. We compute one inner product and one distance both through Φ and directly through K , and watch them agree.

1. Build the feature vectors. $\Phi(x_a) = (1, 2\sqrt{2}, 4) \approx (1, 2.828, 4)$ and $\Phi(x_b) = (4, 2\sqrt{2}, 1) \approx (4, 2.828, 1)$.
2. Inner product, route A (explicit). In the feature space, $\langle \Phi(x_a), \Phi(x_b) \rangle = 1 \cdot 4 + (2\sqrt{2})^2 + 4 \cdot 1 = 4 + 8 + 4 = 16$.
3. Inner product, route B (kernel). Through the kernel, $\langle x_a, x_b \rangle = 1 \cdot 2 + 2 \cdot 1 = 4$, so $K(x_a, x_b) = 4^2 = 16$. The two routes match, and route B never built the 3-vector.
4. Assemble the diagonal. $K(x_a, x_a) = (1 + 4)^2 = 25$ and $K(x_b, x_b) = (4 + 1)^2 = 25$.
5. Distance, both ways. The kernel formula gives $d_K(x_a, x_b)^2 = 25 + 25 - 2 \cdot 16 = 18$; the explicit vectors give $\|\Phi(x_a) - \Phi(x_b)\|^2 = (1 - 4)^2 + 0 + (4 - 1)^2 = 9 + 9 = 18$. Hence $d_K(x_a, x_b) = \sqrt{18} \approx 4.243$.

Reading. Every feature-space quantity the trick promises, an inner product here and a distance there, is reproduced exactly by kernel evaluations that never inspect the coordinates of Φ . For this quadratic kernel we could still afford to write Φ down; the force of the identity $K(x, y) = \langle \Phi(x), \Phi(y) \rangle$ is that for the Gaussian kernel, whose Φ is infinite-dimensional, the kernel column is the only one we could ever compute, and it suffices.

Verification artifact. checks/example-ch02-example-3-2.json records the example source hash and verification scope.

Remark (normalisation puts every image on the unit sphere)

The Gaussian kernel landed every image on the unit sphere for free, because $K(x, x) = 1$. We can arrange the same thing for any kernel by *normalisation*: replace K by

$$\hat{K}(x, z) = \frac{K(x, z)}{\sqrt{K(x, x) K(z, z)}},$$

which is exactly the kernel of the rescaled feature map $\Phi(x) \mapsto \Phi(x)/\|\Phi(x)\|_{\mathcal{H}}$ and therefore sends every image to the unit sphere of $\mathcal{H}$. After normalisation $\hat{K}(x, z) = \langle \Phi(x), \Phi(z) \rangle / (\|\Phi(x)\| \|\Phi(z)\|)$ is the cosine of the angle between the two feature vectors, and the distance $d_{\hat{K}}(x, z)^2 = 2(1 - \hat{K}(x, z))$ depends only on that angle. Shawe-Taylor and Cristianini (2004) present this as the simplest example of a recurring pattern: many useful transformations of the data in feature space are realised by an operation on the Gram matrix, without ever touching coordinates.

4.2.2 Example 2: distance between a point and a set

Distances between single points are only the beginning; the feature space also contains objects that have no counterpart in $\mathcal{X}$ at all. Here is a natural problem. Let $S = (x_1, \dots, x_n)$ be a finite set of points in $\mathcal{X}$. How can we define, and compute, the similarity between an arbitrary point $x \in \mathcal{X}$ and the set S ?

A solution: map all points to the feature space, summarize S by the barycenter of its images,

$$\mu := \frac{1}{n} \sum_{i=1}^n \Phi(x_i),$$

and define the distance between x and S as the feature-space distance to that barycenter:

$$d_K(x, S) := \|\Phi(x) - \mu\|_{\mathcal{H}}.$$

Expanding the squared norm exactly as before, every cross term is again an inner product between feature vectors, hence a kernel value:

$$d_K(x, S) = \left\| \Phi(x) - \frac{1}{n} \sum_{i=1}^n \Phi(x_i) \right\|_{\mathcal{H}} = \sqrt{K(x, x) - \frac{2}{n} \sum_{i=1}^n K(x, x_i) + \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n K(x_i, x_j)}.$$

Remark (no pre-image)

The barycenter μ exists only in the feature space in general: it need not have a *pre-image*, that is, a point $x_\mu \in \mathcal{X}$ such that $\Phi(x_\mu) = \mu$. The images $\Phi(x)$ of input points typically form a thin, curved subset of $\mathcal{H}$ (for the Gaussian kernel, a subset of the unit sphere), and averages of points on that subset fall off it. This is the first instance of a recurring theme: the kernel trick lets us compute with feature-space objects, like μ , that we could never write down as elements of $\mathcal{X}$, because everything we ever ask about them reduces to kernel evaluations.

The same inner-product bookkeeping yields two companion identities, noted by Shawe-Taylor and Cristianini (2004), that turn global properties of the whole set S into simple averages over the Gram matrix.

Proposition (mean and variance in the feature space)

Let $\mu = \frac{1}{n} \sum_{i=1}^n \Phi(x_i)$ be the barycenter of the images of S . Then the squared length of the barycenter is the average of all entries of the Gram matrix,

$$\|\mu\|_{\mathcal{H}}^2 = \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n K(x_i, x_j),$$

which is therefore always nonnegative, and equals zero exactly when the data are already centred, $\mu = 0$. The average squared distance of the images from their barycenter, the *variance of the data in the feature space*, is the average of the diagonal of the Gram matrix minus the average of all its entries,

$$\frac{1}{n} \sum_{s=1}^n \|\Phi(x_s) - \mu\|_{\mathcal{H}}^2 = \frac{1}{n} \sum_{s=1}^n K(x_s, x_s) - \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n K(x_i, x_j).$$

4 The Kernel Trick and the Representer Theorem

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Both drop out of the expansion $\|\Phi(x_s) - \mu\|^2 = \langle \Phi(x_s), \Phi(x_s) \rangle - 2\langle \Phi(x_s), \mu \rangle + \langle \mu, \mu \rangle$, averaged over s , with the cross terms collapsing onto $\|\mu\|^2$. The second identity is the striking one: the spread of the data around its centre of mass, an intrinsically feature-space quantity with no coordinates to inspect, is read straight off the Gram matrix. For the Gaussian kernel the diagonal is all ones, so the variance is 1 minus the average Gram entry: a set whose images cluster tightly (Gram entries near 1) has small feature-space variance, while a well-spread set of near-orthogonal images (Gram entries near 0) has variance approaching 1.

4.2.3 What the choice of kernel does: illustrations

The formula for $d_K(x, S)$ is the same for every kernel; the geometry it induces is not. We can illustrate this by fixing a small set S and plotting $f(x) = d_K(x, S)$ for several kernels.

One dimension. Take $S = \{2, 3\}$ on the real line and plot $x \mapsto d_K(x, S)$ on $[0, 5]$ for three kernels: the linear kernel $K(x, y) = xy$, and the Gaussian kernel $K(x, y) = e^{-(x-y)^2/(2\sigma^2)}$ with $\sigma = 1$ and with $\sigma = 0.2$.

- For the linear kernel, the feature space is $\mathcal{H} = \mathbb{R}$ itself, the barycenter has the pre-image $\mu = 2.5$, and the distance is simply $d_K(x, S) = |x - \mu|$: a V-shaped curve with its minimum at the ordinary mean of S . Nothing nonlinear has happened.
- For the Gaussian kernel, since $K(x, x) = 1$, the formula collapses to

$$d_K(x, S) = \sqrt{C - \frac{2}{n} \sum_{i=1}^n K(x_i, x)},$$

where $C = 1 + \frac{1}{n^2} \sum_{i,j} K(x_i, x_j)$ does not depend on x . The distance is therefore a constant minus a sum of Gaussian bumps centered at the points of S . With $\sigma = 1$ the two bumps merge and the curve has a single smooth dip spanning the whole set. With $\sigma = 0.2$ the curve is essentially flat at its saturation value except for two sharp dips, one at $x = 2$ and one at $x = 3$: the kernel now resolves the individual points rather than the set as a whole.

Two dimensions. The same experiment with $S = \{(1, 1)', (1, 2)', (2, 2)'\}$ in the plane shows the level sets of $x \mapsto d_K(x, S)$. For the linear kernel they are the level sets of the distance to the barycenter computed in $\mathbb{R}^2$. For the Gaussian kernel with $\sigma = 1$ they form a single smooth basin around the cloud, and with $\sigma = 0.2$ the basin fragments into three local wells, one around each point of S . As before, the barycenter μ in $\mathcal{H}$, a single point in the feature space, may carry a lot of information about the training data: with a Gaussian kernel, distance to μ behaves like a density model of the set.

A basic application to discrimination. The point-to-set distance immediately yields a classifier. Take two labeled sets $S_1 = \{(1, 1)', (1, 2)'\}$ and $S_2 = \{(1, 3)', (2, 2)'\}$ and score a new point by

$$f(x) = d_K(x, S_1)^2 - d_K(x, S_2)^2,$$

assigning x to the class whose set is nearer in the feature space. With the linear kernel the terms $K(x, x)$ cancel and f is an affine function of x : the decision boundary is a straight line, the perpendicular bisector geometry of the two barycenters. With the Gaussian kernel at $\sigma = 1$ the boundary bends to follow the shape of the two clouds, and at $\sigma = 0.2$ the classifier becomes local, carving out neighborhoods around individual training points. One formula, three classifiers; the kernel is where the modeling happens.

4.2.4 A first pattern-analysis algorithm: novelty detection

The point-to-set distance already supports a genuine, if simple, learning algorithm. Shawe-Taylor and Cristianini (2004) use it for *novelty detection*: deciding whether a fresh observation resembles a reference sample or should be flagged as anomalous. The construction reuses everything above and adds nothing but a threshold.

Regard the training set $S = (x_1, \dots, x_n)$ as a sample scattered around its feature-space centre of mass μ , and enclose the images in the ball centred at μ whose radius is the largest training distance,

$$R_S = \max_{1 \leq i \leq n} \|\Phi(x_i) - \mu\|_{\mathcal{H}}.$$

Both R_S and, for any test point x , the distance $\|\Phi(x) - \mu\|_{\mathcal{H}}$ are computed from kernel values by the point-to-set formula, so no vector in $\mathcal{H}$ is ever written down. Declare a new point x_{n+1} novel when its image falls outside the ball,

$$\|\Phi(x_{n+1}) - \mu\|_{\mathcal{H}} > R_S.$$

What makes this more than a heuristic is that it comes with a guarantee. The catch is that μ is only the *empirical* centre of mass, an estimate of the true mean $\mathbb{E}[\Phi(x)]$ of the distribution, and the two differ. But the estimate is good in a way that does not depend on the dimension of $\mathcal{H}$: if the support of the distribution lies in a ball of radius R about the origin, then with probability at least $1 - \delta$ over the sample the empirical centre lies within

$$\sqrt{\frac{2R^2}{n}} \left(\sqrt{2} + \sqrt{\ln \frac{1}{\delta}} \right)$$

of the true mean. Widening the accept region by twice this margin gives a threshold for which a genuinely new point drawn from the same distribution is wrongly flagged with probability at most $1/(n+1)$, a controlled false-alarm rate. The implicit model of "normal" is a single sphere in the feature space; when the kernel is Gaussian that sphere is a flexible curved region back in $\mathcal{X}$, which is why the same construction can flag departures from a bent data cloud that no sphere in the input space could describe. It is, in the terminology of Shawe-Taylor and Cristianini (2004), one of the first pattern-analysis algorithms one can write down, and it is built entirely from the Gram matrix.

4.2.5 Example 3: centering data in the feature space

Many classical data-analysis procedures, principal component analysis first among them, begin with the instruction "center the data". Our third example shows that this too survives the move to an implicit feature space.

Everything in this example operates on a single object, the Gram matrix of kernel values, so it is worth seeing how that matrix answers to the choice of kernel and bandwidth before we start transforming it.

Problem (centered Gram matrix)

Let $S = (x_1, \dots, x_n)$ be a finite set of points in $\mathcal{X}$ endowed with a p.d. kernel K , and let $\mathbf{K}$ be their $n \times n$ Gram matrix, $[\mathbf{K}]_{ij} = K(x_i, x_j)$. Let $\mu = \frac{1}{n} \sum_{i=1}^n \Phi(x_i)$ be the barycenter, and let $u_i = \Phi(x_i) - \mu$, for $i = 1, \dots, n$, be the centered data in $\mathcal{H}$. How can we compute the centered Gram matrix $[\mathbf{K}^c]_{ij} = \langle u_i, u_j \rangle_{\mathcal{H}}$?

We cannot form the vectors u_i explicitly, but their pairwise inner products expand into kernel evaluations. A direct computation gives, for $1 \leq i, j \leq n$:

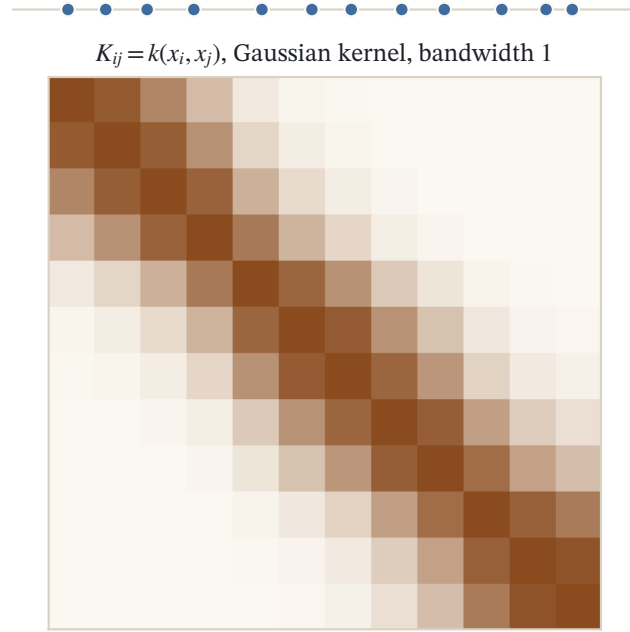


Figure 4.2: Twelve points on a line and their Gram matrix $K_{ij} = k(x_i, x_j)$ for a Gaussian kernel at bandwidth 1. A kernel is positive definite exactly when this matrix is positive semidefinite for every sample; narrowing the bandwidth drives it toward the identity.

$$\begin{aligned}
 \mathbf{K}_{ij}^c &= \langle \Phi(x_i) - \mu, \Phi(x_j) - \mu \rangle_{\mathcal{H}} \\
 &= \langle \Phi(x_i), \Phi(x_j) \rangle_{\mathcal{H}} - \langle \mu, \Phi(x_i) + \Phi(x_j) \rangle_{\mathcal{H}} + \langle \mu, \mu \rangle_{\mathcal{H}} \\
 &= \mathbf{K}_{ij} - \frac{1}{n} \sum_{k=1}^n (\mathbf{K}_{ik} + \mathbf{K}_{jk}) + \frac{1}{n^2} \sum_{k,l=1}^n \mathbf{K}_{kl}.
 \end{aligned}$$

Each entry subtracts the corresponding row and column means and adds back the grand mean. This can be rewritten in matrix form: with U the $n \times n$ matrix whose entries are all $U_{ij} = 1/n$,

$$\mathbf{K}^c = \mathbf{K} - U\mathbf{K} - \mathbf{K}U + U\mathbf{K}U = (I - U)\mathbf{K}(I - U).$$

A quick way to remember the operator: $I - U$ is the familiar mean-subtracting projection, the very matrix that centers ordinary data in $\mathbb{R}^n$. Sandwiching $\mathbf{K}$ between two copies of it centers the feature vectors on both sides of the inner product at once, which is why the row mean, the column mean, and the grand mean all appear together in the entrywise formula. The consistency check is immediate: if the data were already centered in the feature space then $\mu = 0$, every mean would vanish, and $\mathbf{K}^c$ would collapse back to $\mathbf{K}$.

Centering in the feature space is thus a fixed, data-independent linear operation on the Gram matrix. We will use this formula verbatim when we kernelize PCA later in the book.

4.2.6 Summary: three ways to use the trick

The kernel trick is a trivial statement with important applications, and the examples above illustrate its three main uses.

- It can be used to obtain **nonlinear versions of well-known linear algorithms**, for instance by replacing the classical inner product with a Gaussian kernel, as the discrimination example showed.
- It can be used to apply classical algorithms to **non-vectorial data** such as strings or graphs, by again replacing the classical inner product with a valid kernel for that data type. The formulas for d_K , for $d_K(x, S)$, and for $\mathbf{K}^c$ never used anything about $\mathcal{X}$.
- It allows us in some cases to **embed the initial space into a larger feature space and manipulate points that have no pre-image**, the barycenter μ being the standard example.

4.2.7 Structured inputs and outputs

The second and third uses deserve emphasis, because they extend beyond exotic inputs to exotic *outputs*. Nothing in the framework requires the prediction targets to be numbers or class labels: since a kernel can be defined on any set, it can in particular be defined on a set of input-output *pairs*.

A running illustration is machine translation, say from Japanese to French. Here an input x is a Japanese sentence and the desired output y is a French sentence: both spaces are discrete, combinatorially huge, and nothing like $\mathbb{R}^p$. The kernel approach is to define a p.d. kernel K on the product set $\mathcal{X} \times \mathcal{Y}$ of sentence pairs, for example built from string kernels that measure shared subsequences, and to learn a real-valued compatibility function $g(x, y) = \langle \Phi(x, y), g \rangle_{\mathcal{H}}$ that scores how well the candidate translation y matches the source sentence x . Training fits g so that correct pairs from a bilingual corpus score higher than incorrect ones, and prediction returns the highest-scoring output,

$$\hat{y}(x) = \arg \max_{y \in \mathcal{Y}} g(x, y).$$

Learning the scoring function is a kernel method like any other: the algorithm only ever touches sentence pairs through kernel evaluations. The structured nature of the problem is absorbed entirely by the kernel on pairs, plus a search procedure for the $\arg \max$. This pattern, kernels on $\mathcal{X} \times \mathcal{Y}$ for structured output prediction, will return when we discuss the kernel jungle; for now it stands as the clearest demonstration that "data" in the kernel trick means any set we can measure similarity on, on either side of the prediction.

4.3 The representer theorem

The kernel trick handles algorithms already phrased in terms of inner products. But our motivating problem was an optimization over a function space:

$$\min_{f \in \mathcal{H}} \underbrace{\frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i))}_{\text{empirical risk, data fit}} + \underbrace{\lambda \|f\|_{\mathcal{H}}^2}_{\text{regularization}}$$

for a loss function ℓ such as $\ell(y, t) = (y - t)^2$. An RKHS is a space of potentially nonlinear functions, and $\|f\|_{\mathcal{H}}$ measures the smoothness of f , so this is a natural way to estimate a regression function $f : \mathcal{X} \rightarrow \mathbb{R}$ from data $(x_i \in \mathcal{X}, y_i \in \mathbb{R})_{i=1, \dots, n}$. But how do we solve such a problem in practice, when $\mathcal{H}$ is infinite-dimensional? The second pillar of kernel methods answers this: the search space collapses, exactly, to a subspace of dimension at most n .

Theorem (representer theorem)

Let $\mathcal{X}$ be a set endowed with a p.d. kernel K , let $\mathcal{H}$ be the corresponding RKHS, and let $S = \{x_1, \dots, x_n\} \subseteq \mathcal{X}$ be a finite set of points in $\mathcal{X}$. Let $\Psi : \mathbb{R}^{n+1} \rightarrow \mathbb{R}$ be a function of $n + 1$ variables, strictly increasing with respect to its last variable. Then any solution of the optimization problem

4 The Kernel Trick and the Representer Theorem

$$\min_{f \in \mathcal{H}} \Psi(f(x_1), \dots, f(x_n), \|f\|_{\mathcal{H}})$$

admits a representation of the form

$$\forall x \in \mathcal{X}, \quad f(x) = \sum_{i=1}^n \alpha_i K(x_i, x) = \sum_{i=1}^n \alpha_i K_{x_i}(x).$$

In other words, the solution lives in a finite-dimensional subspace: $f \in \text{Span}(K_{x_1}, \dots, K_{x_n})$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Before the hypotheses, the plain reading. We set out to search an infinite-dimensional space of functions, and the theorem tells us the size of that search was an illusion: whatever the winning function turns out to be, it can be written as a weighted sum of the n kernel functions $K_{x_i} = K(x_i, \cdot)$ planted one at each training point, with a single weight α_i per point. Learning an unknown function over $\mathcal{H}$ has quietly become learning n numbers. The smallest case makes the mechanism visible. With a single training point x_1 , any candidate f splits into a part pointing along K_{x_1} , which alone fixes the value $f(x_1)$, and a part orthogonal to it, which leaves $f(x_1)$ untouched yet still adds to $\|f\|_{\mathcal{H}}$; since the objective rewards a smaller norm, it deletes the orthogonal part outright, and the solution is the lone bump $f = \alpha_1 K_{x_1}$.

Note how weak the hypotheses are. The theorem asks nothing of the first n arguments of Ψ : the data-fit term need not be convex, differentiable, or even continuous. All that matters is that the objective strictly prefers a smaller RKHS norm when the values at the data points are held fixed. The regularized empirical risk above is the special case $\Psi(t_1, \dots, t_n, u) = \frac{1}{n} \sum_i \ell(y_i, t_i) + \lambda u^2$. The result in this generality is due to Kimeldorf and Wahba (1971) for the quadratic case, with the general form popularized by Schölkopf, Herbrich, and Smola (2001).

Proof

Let $\xi(f)$ denote the functional being minimized, $\xi(f) = \Psi(f(x_1), \dots, f(x_n), \|f\|_{\mathcal{H}})$, and let $\mathcal{H}_S$ be the linear span in $\mathcal{H}$ of the vectors K_{x_i} :

$$\mathcal{H}_S = \left\{ f \in \mathcal{H} : f(x) = \sum_{i=1}^n \alpha_i K(x_i, x), (\alpha_1, \dots, \alpha_n) \in \mathbb{R}^n \right\}.$$

$\mathcal{H}_S$ is a finite-dimensional subspace of the Hilbert space $\mathcal{H}$, hence closed, so any $f \in \mathcal{H}$ decomposes uniquely by orthogonal projection as

$$f = f_S + f_{\perp}, \quad f_S \in \mathcal{H}_S, \quad f_{\perp} \perp \mathcal{H}_S.$$

Now the reproducing property enters. Since $\mathcal{H}$ is an RKHS, point evaluation is an inner product against $K_{x_i} = K(x_i, \cdot)$, and $K_{x_i} \in \mathcal{H}_S$ while $f_{\perp} \perp \mathcal{H}_S$, so

$$\forall i = 1, \dots, n, \quad f_{\perp}(x_i) = \langle f_{\perp}, K_{x_i} \rangle_{\mathcal{H}} = 0,$$

and therefore $f(x_i) = f_S(x_i)$ for all $i = 1, \dots, n$: the orthogonal part is invisible at the data points, so the first n arguments of Ψ are unchanged when f is replaced by f_S . Pythagoras' theorem in $\mathcal{H}$ then gives

$$\|f\|_{\mathcal{H}}^2 = \|f_S\|_{\mathcal{H}}^2 + \|f_{\perp}\|_{\mathcal{H}}^2,$$

so $\|f_S\|_{\mathcal{H}} \leq \|f\|_{\mathcal{H}}$, with equality if and only if $\|f_{\perp}\|_{\mathcal{H}} = 0$. Since Ψ is strictly increasing in its last argument, $\xi(f) \geq \xi(f_S)$, with equality if and only if $\|f_{\perp}\|_{\mathcal{H}} = 0$. Any minimizer of ξ must therefore satisfy $f_{\perp} = 0$, that is, lie in $\mathcal{H}_S$. $\square$

Remark (why the theorem is "trivial", and why it matters)

The representer theorem has important consequences, but it is in fact rather simple once the reproducing property is internalized. We are looking for a function $f \in \mathcal{H}$ that interacts with the world only through the values $f(x) = \langle K_x, f \rangle_{\mathcal{H}}$ at the training points: the part $f_{\perp}$ orthogonal to the K_{x_i} is useless for explaining the training data, and the regularizer strictly punishes carrying it around. The theorem simply says the optimum carries no dead weight.

Remark (the support vector expansion, sparsity, and coupled losses)

Schölkopf and Smola (2002) state the theorem in the form used throughout the support-vector literature, and three of their observations are worth carrying forward. First, the loss need not be a sum of per-example terms: their version allows an arbitrary cost that may *couple* the training examples, some $c(x_1, y_1, f(x_1), \dots, x_n, y_n, f(x_n))$, and the conclusion is unchanged, because the proof only ever uses that the objective sees f through its n point values and its norm. Second, in the SVM setting the resulting expansion $f = \sum_i \alpha_i K(x_i, \cdot)$ is what they call the *support vector expansion*, and for well-chosen losses (the hinge loss of classification, the ε -insensitive loss of regression) many of the coefficients α_i come out exactly zero. The solution is then *sparse*, supported on a subset of the training points, the support vectors; this is a property of the loss rather than of the theorem, but it is what makes the finite representation cheap to store and fast to evaluate. Third, one may append an unregularized parametric part, a few fixed basis functions $\psi_1, \dots, \psi_M$ such as a constant offset, and a *semiparametric* representer theorem then guarantees a mixed solution $f = \sum_{i=1}^n \alpha_i K(x_i, \cdot) + \sum_{p=1}^M \beta_p \psi_p$, a kernel expansion plus a parametric term. The finite reduction survives all of these extensions intact.

4.3.1 Scope and the regularization reading

Often the function Ψ has the separable form

$$\Psi(f(x_1), \dots, f(x_n), \|f\|_{\mathcal{H}}) = c(f(x_1), \dots, f(x_n)) + \lambda \Omega(\|f\|_{\mathcal{H}}),$$

where $c(\cdot)$ measures the fit of f to a given problem (regression, classification, dimension reduction, and so on) and Ω is strictly increasing. This formulation has two important consequences.

- **Theoretically**, the minimization will enforce the norm $\|f\|_{\mathcal{H}}$ to be small, which can be beneficial by ensuring a sufficient level of smoothness for the solution. This is the regularization effect, and it is what connects the optimization problem to the statistical goal of generalizing beyond the sample.
- **Practically**, we know by the representer theorem that the solution lives in a subspace of dimension n , which can lead to efficient algorithms even though the RKHS itself can be of infinite dimension.

Two boundary notes on scope. First, strict monotonicity in the norm is what forces *every* minimizer into $\mathcal{H}_S$; if Ψ is merely nondecreasing in its last variable (for instance $\lambda = 0$, no regularization), the proof still shows that *some* minimizer lies in $\mathcal{H}_S$ whenever a minimizer exists, but uniqueness of the representation is lost. Second, the objective must depend on f only through finitely many point evaluations and the norm; a data-fit term involving, say, derivatives of f or an integral of f over $\mathcal{X}$ falls outside the statement.

Remark (when the representer theorem needs care)

The monotonicity hypothesis is not a technical convenience but essentially the exact condition for the finite expansion to hold. Argyriou, Micchelli, and Pontil (2009) characterise which regularizers admit a representer theorem: for a differentiable regularizer, every minimizer is guaranteed to lie in $\text{Span}(K_{x_1}, \dots, K_{x_n})$ for all data if and only if the regularizer is a nondecreasing function of the RKHS norm. An arbitrary penalty on f does not qualify. If the penalty is not monotone in $\|f\|_{\mathcal{H}}$, or if it rewards structure orthogonal to the training kernels, the optimum can pick up a component outside the span and the clean n -term representation no longer applies. The

4 The Kernel Trick and the Representer Theorem

collapse from $\mathcal{H}$ to $\mathbb{R}^n$ is thus a property of norm-based regularization specifically, not of RKHS optimization in general.

4.3.2 Practical use: from $\mathcal{H}$ to $\mathbb{R}^n$

The representer theorem is not just a structural statement; it is a computational device. When it holds, we may restrict the search to functions of the form

$$f(x) = \sum_{i=1}^n \alpha_i K(x_i, x), \quad \alpha \in \mathbb{R}^n,$$

and rewrite every ingredient of the objective in terms of α and the Gram matrix $\mathbf{K}$. For any $j = 1, \dots, n$, the value at a training point is a matrix-vector product:

$$f(x_j) = \sum_{i=1}^n \alpha_i K(x_i, x_j) = [\mathbf{K}\alpha]_j.$$

Furthermore, the RKHS norm becomes a quadratic form, again by the reproducing property applied pairwise:

$$\|f\|_{\mathcal{H}}^2 = \left\| \sum_{i=1}^n \alpha_i K_{x_i} \right\|_{\mathcal{H}}^2 = \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j K(x_i, x_j) = \alpha^\top \mathbf{K} \alpha.$$

Therefore a problem of the form

$$\min_{f \in \mathcal{H}} \Psi(f(x_1), \dots, f(x_n), \|f\|_{\mathcal{H}}^2)$$

is equivalent to the following n -dimensional optimization problem:

$$\min_{\alpha \in \mathbb{R}^n} \Psi([\mathbf{K}\alpha]_1, \dots, [\mathbf{K}\alpha]_n, \alpha^\top \mathbf{K} \alpha).$$

It is worth pausing on what was and was not assumed. We never needed a formula for Ψ ; we needed only that the objective sees f through its n values at the training points and through its norm, and both of those turned into functions of α and $\mathbf{K}$. When the loss is nice enough the finite problem even has a closed form. Take the squared loss $\ell(y, t) = (y - t)^2$ with regularizer $\lambda \|f\|_{\mathcal{H}}^2$: the objective becomes $\frac{1}{n} \|y - \mathbf{K}\alpha\|^2 + \lambda \alpha^\top \mathbf{K} \alpha$, a convex quadratic in α whose gradient vanishes at the solution of the linear system $(\mathbf{K} + n\lambda I) \alpha = y$. That is kernel ridge regression, the subject of the next chapter, already dropping out of the recipe.

This problem can usually be solved analytically or by numerical methods; the following chapters work through many examples, kernel ridge regression and the support vector machine chief among them. An infinite-dimensional variational problem has become a finite-dimensional one whose data are exactly the $n \times n$ Gram matrix: once again, the algorithm touches the data only through pairwise kernel evaluations.

It is worth carrying out the reduction once with numbers, on a kernel small enough that we can also stand outside the RKHS and check the same predictor as a plain inner product in the feature space.

Example (the representer expansion is a feature-space inner product)

Reuse the quadratic kernel $K(x, y) = \langle x, y \rangle^2$ on $\mathbb{R}^2$, with feature map $\Phi(x) = (x_1^2, \sqrt{2} x_1 x_2, x_2^2)$. Take three training points $x_1 = (1, 0)$, $x_2 = (0, 1)$, $x_3 = (1, 1)$ with targets $y = (1, 2, 3)$, and a fresh test point $x_\star = (2, 1)$. The minimum-norm interpolant supplied by the representer theorem has the form $f = \sum_i \alpha_i K(x_i, \cdot)$ with coefficients solving $\mathbf{K}\alpha = y$. We evaluate $f(x_\star)$ both as a kernel expansion and as an honest inner product in $\mathbb{R}^3$.

1. Form the Gram matrix. With $K(x_i, x_j) = \langle x_i, x_j \rangle^2$, the entries are $\mathbf{K} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 4 \end{bmatrix}$, which has $\det \mathbf{K} = 2 \neq 0$.
2. Solve for the weights. The system $\mathbf{K}\alpha = y = (1, 2, 3)$ gives $\alpha = (1, 2, 0)$; indeed $\mathbf{K}\alpha = (1, 2, 3)$ reproduces the targets, so f interpolates the data.
3. Evaluate through the kernel (route 1). The kernel row at $x_\star$ is $k_\star = (K(x_1, x_\star), K(x_2, x_\star), K(x_3, x_\star)) = (4, 1, 9)$, so $f(x_\star) = k_\star^\top \alpha = 4 + 2 + 0 = 6$.
4. Evaluate through the feature space (route 2). Assemble $w = \sum_i \alpha_i \Phi(x_i) = 1 \cdot (1, 0, 0) + 2 \cdot (0, 0, 1) + 0 = (1, 0, 2)$. With $\Phi(x_\star) = (4, 2\sqrt{2}, 1)$, $\langle \Phi(x_\star), w \rangle = 4 + 0 + 2 = 6$, the same value.
5. Check the norm identity. The RKHS norm reads off both ways: $\alpha^\top \mathbf{K} \alpha = 5$ and $\|w\|^2 = 1^2 + 0 + 2^2 = 5$.

Reading. The predictor $f = \sum_i \alpha_i K(x_i, \cdot)$ that the representer theorem hands us is nothing but the linear form $x \mapsto \langle \Phi(x), w \rangle$ with weight vector $w = \sum_i \alpha_i \Phi(x_i)$ living in the span of the training images: $f(x_\star) = k_\star^\top \alpha = \langle \Phi(x_\star), w \rangle$ and $\|f\|_{\mathcal{H}}^2 = \alpha^\top \mathbf{K} \alpha = \|w\|^2$. The n coefficients α and the feature-space vector w are two encodings of one function, and the Gram matrix is the dictionary between them.

Verification artifact. checks/example-ch02-example-3-3.json records the example source hash and verification scope.

4.4 A general recipe for kernelizing an algorithm

We can now step back and describe the pattern that the rest of this book instantiates over and over. Most kernel methods admit two complementary interpretations, and it pays to be fluent in both.

Remark (dual interpretations of kernel methods)

Most kernel methods can be read in two ways:

- A **geometric interpretation in the feature space**, thanks to the kernel trick. Even when the feature space is large, most kernel methods work in the linear span of the embeddings of the available points.
- A **functional interpretation**, often as an optimization problem over (subsets of) the RKHS associated with the kernel, with the representer theorem collapsing that problem to finite dimension.

These are two descriptions of the same object: the span of $\Phi(x_1), \dots, \Phi(x_n)$ in the feature space corresponds, through the isometry of Chapter 2, Positive Definite Kernels and RKHS, to the span of $K_{x_1}, \dots, K_{x_n}$ in the RKHS.

Remark (two interchangeable modules)

There is a second, more operational sense in which kernel methods decompose, emphasised by Shawe-Taylor and Cristianini (2004): a kernel method is literally two modules bolted together through a single interface. One module is the *kernel*, which is data-specific and carries the domain knowledge about which objects should count as similar. The other is the *pattern-analysis algorithm*, which is general purpose and typically comes with its own analysis of statistical stability. They meet only at the $n \times n$ Gram matrix: the algorithm consumes kernel values and never inspects the raw objects, and the kernel produces those values and never inspects the algorithm. Because the interface is so narrow, either module can be swapped without touching the other. The same regression or classification code runs on vectors, strings, or graphs by changing only the kernel, and one fixed kernel can be handed to any inner-product algorithm to perform a different task. This reusability, rather than any single algorithm, is what earns kernels the status of a methodology instead of a trick.

Correspondingly, there are two routes for turning a classical linear algorithm into a kernel method, and the recipe is short enough to state as pseudocode:

To kernelize an algorithm A operating on data $x_1, \dots, x_n$:

Route 1 (kernel trick, geometric):

1. Rewrite A so that it accesses the data ONLY through pairwise inner products $\langle x_i, x_j \rangle$ [and inner products

4 The Kernel Trick and the Representer Theorem

- (x, x_i) with any test point x].
2. Replace every inner product by a kernel evaluation $K(x_i, x_j)$, i.e., precompute the Gram matrix K .
 3. Run A unchanged; feature-space objects (barycenters, projections, centered data) remain implicit.

Route 2 (representer theorem, functional):

1. Pose A as minimizing $\Psi(f(x_1), \dots, f(x_n), \|f\|_H)$ over the RKHS H , with Ψ strictly increasing in $\|f\|_H$.
2. Substitute $f = \sum_i \alpha_i K(x_i, \cdot)$, so that $f(x_j) = [K \alpha]_j$ and $\|f\|_H^2 = \alpha' K \alpha$.
3. Solve the resulting n -dimensional problem in α .
4. Predict at a new point x via $f(x) = \sum_i \alpha_i K(x_i, x)$.

The two routes meet in the middle: both reduce the algorithm to computations on the Gram matrix, and both output predictors expressed as kernel expansions over the training points. Route 1 is the natural reading for procedures defined by geometry, such as centering, projections, or nearest-barycenter classification; Route 2 is the natural reading for procedures defined by an objective function, such as regularized regression and classification. Everything we kernelize in the coming chapters, ridge regression, support vector machines, PCA, canonical correlation analysis, will be an instance of one route or the other, and usually both at once.

The cost of this generality is also visible in the recipe: kernel methods manipulate $n \times n$ Gram matrices, so their complexity scales with the number of data points rather than the dimension of the data, a trade we will revisit when we discuss large-scale approximations such as random features (Rahimi and Recht, 2007). But the conceptual gain is settled: with the kernel trick and the representer theorem in hand, linearity in the feature space buys nonlinearity in the input space, and infinite-dimensional function classes come with finite-dimensional algorithms.

Further reading

The kernel trick and the representer theorem are treated in Schölkopf and Smola (2002), *Learning with Kernels*, chapters 2 and 4, and in Shawe-Taylor and Cristianini (2004), *Kernel Methods for Pattern Analysis*, chapter 2, both of which develop the feature-space computations and the finite reduction used throughout the rest of the subject. The generalized representer theorem is due to Schölkopf, Herbrich, and Smola (2001), building on the spline result of Kimeldorf and Wahba (1971).

4.5 Common mistakes and practical implications

For **The Kernel Trick and the Representer Theorem**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

4.6 Summary and further reading

This chapter established explain the central definitions and claims in The Kernel Trick and the Representer Theorem; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Vapnik 1995), (Schölkopf and Smola 2002), (Aronszajn 1950).

4.7 Exercises

1. warm-up Using only $K(x, x') = \langle \Phi(x), \Phi(x') \rangle_{\mathcal{H}}$, verify the two properties a distance must have from the kernel formula $d_K(x, x')^2 = K(x, x) + K(x', x') - 2K(x, x')$: that $d_K(x, x) = 0$ for every x , and that d_K is symmetric, $d_K(x, x') = d_K(x', x)$. Why does positive definiteness of K guarantee that the quantity under the square root is never negative?
2. warm-up For the normalised kernel $\hat{K}(x, z) = K(x, z)/\sqrt{K(x, x)K(z, z)}$, show that every feature vector $\Phi(x)/\|\Phi(x)\|_{\mathcal{H}}$ has unit length, so that $\hat{K}(x, z)$ is the cosine of the angle between $\Phi(x)$ and $\Phi(z)$. Deduce that $d_{\hat{K}}(x, z)^2 = 2(1 - \hat{K}(x, z))$, and conclude that $0 \leq d_{\hat{K}}(x, z) \leq 2$, with the maximum attained exactly when the two images are antipodal on the unit sphere.
3. computation Take the real line with the linear kernel $K(x, y) = xy$ and the set $S = \{2, 3\}$. Compute the 2×2 Gram matrix, then use the mean and variance identities to find the squared length $\|\mu\|_{\mathcal{H}}^2$ of the barycenter and the feature-space variance $\frac{1}{n} \sum_s \|\Phi(x_s) - \mu\|_{\mathcal{H}}^2$. Finally evaluate the point-to-set distance $d_K(x, S)$ at $x = 0$ and at $x = 2.5$, and check that the minimum sits at the ordinary mean of S .
4. proof Prove the variance identity of the chapter: with $\mu = \frac{1}{n} \sum_{i=1}^n \Phi(x_i)$,

$$\frac{1}{n} \sum_{s=1}^n \|\Phi(x_s) - \mu\|_{\mathcal{H}}^2 = \frac{1}{n} \sum_{s=1}^n K(x_s, x_s) - \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n K(x_i, x_j).$$

Work entirely with inner products and kernel values; you may never write $\Phi(x_i)$ in coordinates. Explain in one sentence why the right-hand side is an intrinsically feature-space quantity that is nonetheless read straight off the Gram matrix. Hint

Expand $\|\Phi(x_s) - \mu\|^2 = \langle \Phi(x_s), \Phi(x_s) \rangle - 2\langle \Phi(x_s), \mu \rangle + \langle \mu, \mu \rangle$, average over s , and use $\frac{1}{n} \sum_s \Phi(x_s) = \mu$ so the middle term collapses onto $\langle \mu, \mu \rangle = \|\mu\|^2$, which by the mean identity is the average of all Gram entries.

5. proof Prove the centering identity. Let U be the $n \times n$ matrix with every entry $1/n$, and $u_i = \Phi(x_i) - \mu$. First show the entrywise formula $\mathbf{K}_{ij}^c = \langle u_i, u_j \rangle_{\mathcal{H}} = \mathbf{K}_{ij} - \frac{1}{n} \sum_k (\mathbf{K}_{ik} + \mathbf{K}_{jk}) + \frac{1}{n^2} \sum_{k,l} \mathbf{K}_{kl}$, then recognise this as the matrix product $\mathbf{K}^c = (I - U)\mathbf{K}(I - U)$. Verify that $U^2 = U$ and UKU is the constant matrix of the grand mean, and check the consistency case: if the data are already centred, $\mu = 0$, then $\mathbf{K}^c = \mathbf{K}$. Hint

Substitute $\mu = \frac{1}{n} \sum_k \Phi(x_k)$ into $\langle \Phi(x_i) - \mu, \Phi(x_j) - \mu \rangle$ and expand the four inner products; each becomes an average of a row, a column, or the whole Gram matrix. For the matrix form, note that left-multiplication by U replaces each column by its mean and right-multiplication by U replaces each row by its mean.

6. proof Reconstruct the orthogonal-decomposition step of the representer theorem. Let $\mathcal{H}_S = \text{Span}(K_{x_1}, \dots, K_{x_n})$, write any $f \in \mathcal{H}$ as $f = f_S + f_{\perp}$ with $f_S \in \mathcal{H}_S$ and $f_{\perp} \perp \mathcal{H}_S$, and use the reproducing property to prove that $f_{\perp}(x_i) = 0$ for every i , hence $f(x_i) = f_S(x_i)$. Then invoke Pythagoras to show $\|f_S\|_{\mathcal{H}} \leq \|f\|_{\mathcal{H}}$ with equality iff $f_{\perp} = 0$, and explain why strict monotonicity of Ψ in its last argument forces every minimiser into $\mathcal{H}_S$. Where exactly would the argument break if Ψ were only nondecreasing in the norm? Hint

Point evaluation is an inner product: $f_{\perp}(x_i) = \langle f_{\perp}, K_{x_i} \rangle_{\mathcal{H}}$. Since $K_{x_i} \in \mathcal{H}_S$ and $f_{\perp} \perp \mathcal{H}_S$, this inner product is 0. With merely nondecreasing Ψ you still get $\xi(f_S) \leq \xi(f)$, but no longer strict inequality when $f_{\perp} \neq 0$, so a minimiser outside $\mathcal{H}_S$ is not excluded.

7. exploration Open the interactive figure above in the section on the two purposes of the embedding. Two classes, an inner blob surrounded by a ring, cannot be separated by any straight line in the plane. Raise the lift by hand and watch the third coordinate $x_1^2 + x_2^2$ push the outer ring up while the inner blob stays low, until a single flat plane slices them apart. Answer three questions from what you see: (a) what does that separating plane correspond to back in the original plane $\mathcal{X}$? (b) the inner product in the lifted space is $\langle \Phi(x), \Phi(x') \rangle = x_1 x'_1 + x_2 x'_2 + (x_1^2 + x_2^2)(x_1'^2 + x_2'^2)$, written entirely in the raw coordinates, so which theorem of this chapter lets you run a linear separator in the lifted space without ever storing the extra

4 The Kernel Trick and the Representer Theorem

coordinate? (c) as you shrink the gap between the rings, does the lift still separate them, and what would that say about the limits of a fixed feature map?

8. challenge Carry out the finite reduction by hand for kernel ridge regression. Substitute the representer form $f(x) = \sum_{i=1}^n \alpha_i K(x_i, x)$ into the objective $\frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2$. Show that $f(x_j) = [\mathbf{K}\alpha]_j$ and $\|f\|_{\mathcal{H}}^2 = \alpha^\top \mathbf{K}\alpha$, so the problem becomes $\min_{\alpha \in \mathbb{R}^n} \frac{1}{n} \|y - \mathbf{K}\alpha\|^2 + \lambda \alpha^\top \mathbf{K}\alpha$. Differentiate and conclude that a minimiser solves the linear system $(\mathbf{K} + n\lambda I)\alpha = y$. Then reflect: the support vector expansion $f = \sum_i \alpha_i K(x_i, \cdot)$ can have many coefficients α_i equal to zero for other losses (hinge, ε -insensitive) but generically not here; what feature of the squared loss is responsible for that difference? Hint

For the norm use the reproducing property pairwise: $\langle K_{x_i}, K_{x_j} \rangle_{\mathcal{H}} = K(x_i, x_j)$. The gradient of $\frac{1}{n} \|y - \mathbf{K}\alpha\|^2 + \lambda \alpha^\top \mathbf{K}\alpha$ is $\frac{2}{n} \mathbf{K}(\mathbf{K}\alpha - y) + 2\lambda \mathbf{K}\alpha$; setting it to zero and cancelling a factor of $\mathbf{K}$ gives the stated system. Sparsity comes from losses that are flat on an interval (a subgradient can be zero away from the fit), which the smooth quadratic loss never is.

5 Kernel Ridge Regression and Smooth Losses

The first two chapters built the machinery: positive definite kernels, the reproducing kernel Hilbert spaces they generate, and the representer theorem that collapses infinite-dimensional optimization problems onto the training points. This chapter puts that machinery to work on supervised learning. We formulate learning as regularized empirical risk minimization in an RKHS, then solve it exactly for the squared loss (kernel ridge regression, a single linear system), extend the solution to weighted squared losses, and finally show that kernel logistic regression, whose loss is smooth but not quadratic, reduces to a sequence of weighted ridge regressions through Newton's method. The chapter closes by placing these methods inside the general family of large-margin classifiers and asking the question that motivates the learning theory to come: when does minimizing a surrogate loss actually control the number of classification errors?

5.1 The supervised learning problem

Everything in this chapter starts from the same practical situation: we observe examples of inputs paired with outputs, and we want a rule that predicts the output for inputs we have never seen. To reason about this cleanly we fix the vocabulary once and for all.

Definition (supervised learning)

Given

- $\mathcal{X}$, a space of inputs,
- $\mathcal{Y}$, a space of outputs,
- $S_n = (x_i, y_i)_{i=1, \dots, n}$, a training set of (input, output) pairs,

the supervised learning problem is to estimate a function $h : \mathcal{X} \rightarrow \mathcal{Y}$ to predict the output for any future input.

Depending on the nature of the output space, this single definition covers several familiar tasks:

- **Regression**, when $\mathcal{Y} = \mathbb{R}$;
- **Classification**, when $\mathcal{Y} = \{-1, 1\}$ or any set of two labels;
- **Structured output** regression or classification, when $\mathcal{Y}$ is more general.

Three examples illustrate how varied the spaces $\mathcal{X}$ and $\mathcal{Y}$ can be, and why a framework that never manipulates inputs directly, only through a kernel, is attractive.

Example (regression)

Task: predict the capacity of a small molecule to inhibit a drug target. Here $\mathcal{X}$ is a set of molecular structures, naturally represented as graphs rather than vectors, and $\mathcal{Y} = \mathbb{R}$. The inputs live in a discrete, combinatorial space; no coordinate representation is given a priori.

Verification artifact. checks/example-ch03-example-4-1.json records the example source hash and verification scope.

Example (classification)

Task: recognize whether an image shows a dog or a cat. Here $\mathcal{X}$ is a set of images, which can be flattened into vectors in $\mathbb{R}^d$ for large d , and $\mathcal{Y} = \{\text{cat}, \text{dog}\}$, a two-element label set that we may encode as $\{-1, 1\}$.

Verification artifact. checks/example-ch03-example-4-2.json records the example source hash and verification scope.

Example (structured output)

Task: translate from Japanese to French. Here $\mathcal{X}$ is the set of finite-length strings of Japanese characters and $\mathcal{Y}$ is the set of finite-length strings of French characters. Both spaces are infinite and discrete, and the output itself has rich internal structure.

Verification artifact. checks/example-ch03-example-4-3.json records the example source hash and verification scope.

5.1.1 Supervised learning with kernels: general principles

Kernel methods attack all of these problems with one uniform recipe, built in three steps. The key idea in the first step is that whatever $\mathcal{Y}$ is, we can always reduce the problem to learning a *real-valued* function $f : \mathcal{Z} \rightarrow \mathbb{R}$ on a suitable space $\mathcal{Z}$, and real-valued functions are exactly what RKHSs contain.

Step 1: express $h : \mathcal{X} \rightarrow \mathcal{Y}$ through a real-valued function $f : \mathcal{Z} \rightarrow \mathbb{R}$.

- Regression ($\mathcal{Y} = \mathbb{R}$): take $\mathcal{Z} = \mathcal{X}$ and predict directly,

$$h(x) = f(x) \quad \text{with } f : \mathcal{X} \rightarrow \mathbb{R}.$$

- Classification ($\mathcal{Y} = \{-1, 1\}$): take $\mathcal{Z} = \mathcal{X}$ and threshold,

$$h(x) = \text{sign}(f(x)) \quad \text{with } f : \mathcal{X} \rightarrow \mathbb{R}.$$

- Structured output: take $\mathcal{Z} = \mathcal{X} \times \mathcal{Y}$, score every candidate output, and predict the best one,

$$h(x) = \arg \max_{y \in \mathcal{Y}} f(x, y) \quad \text{with } f : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}.$$

Step 2: define an empirical risk. To assess how good a candidate function f is on the training set S_n , we choose a loss function ℓ that compares the prediction $f(x_i)$ with the observed output y_i , and average it over the training set:

$$R_n(f) := \frac{1}{n} \sum_{i=1}^n \ell(f(x_i), y_i).$$

The empirical risk measures data fit; different losses will give different algorithms, and much of this chapter is a tour of what happens for particular choices of ℓ .

Step 3: regularize in an RKHS. Define a p.d. kernel K on $\mathcal{Z}$, let $\mathcal{H}$ be its RKHS, and solve either the constrained problem or its penalized counterpart:

$$\min_{f \in \mathcal{H}, \|f\|_{\mathcal{H}} \leq B} R_n(f) \quad \text{or} \quad \min_{f \in \mathcal{H}} R_n(f) + \lambda \|f\|_{\mathcal{H}}^2.$$

We will work mostly with the penalized form,

$$\min_{f \in \mathcal{H}} \underbrace{\frac{1}{n} \sum_{i=1}^n \ell(f(x_i), y_i)}_{\text{empirical risk, data fit}} + \underbrace{\lambda \|f\|_{\mathcal{H}}^2}_{\text{regularization}}.$$

Remark (why this formulation is the right one)

Three observations explain the appeal of this template.

- **Regularization is important**, particularly in high dimension, to prevent overfitting: without the penalty, a rich $\mathcal{H}$ can interpolate the training data and generalize poorly.
- **It contains linear models as a special case.** When $\mathcal{Z} = \mathbb{R}^d$ and K is the linear kernel, every $f \in \mathcal{H}$ is a linear model $f = f_{\mathbf{w}}$ with $f_{\mathbf{w}}(x) = \mathbf{w}^\top x$, and the regularization is exactly the familiar $\|\mathbf{w}\|^2$.
- **It goes far beyond linear models.** Using more general spaces $\mathcal{Z}$ and kernels K lets us learn non-linear functions over a functional space endowed with a natural regularization (recall from Chapter 2, Positive Definite Kernels and RKHS that a small norm in the RKHS means a smooth function), and lets us learn functions over non-vectorial data such as strings and graphs, since the kernel is the only interface to the data.

This three-step template, data fit through a loss plus regularization by the RKHS norm, is the organizing idea behind almost every kernel method for supervised learning; the unifying references are Schölkopf and Smola (2002) and Shawe-Taylor and Cristianini (2004). What changes from one method to the next is only the loss ℓ : the squared loss gives ridge regression, the logistic loss gives penalized logistic regression, the hinge loss gives the support vector machine. The kernel and the regularizer stay fixed, and, as we are about to see repeatedly, the representer theorem turns each of these infinite-dimensional problems into the same kind of finite computation. We now examine a few methods from this family in detail, starting with the one that admits a closed-form solution.

5.2 Kernel ridge regression

5.2.1 The regression setup and least squares

Consider first the regression setting:

- $\mathcal{X}$, a set of inputs;
- $\mathcal{Y} = \mathbb{R}$, real-valued outputs;
- $S_n = (x_i, y_i)_{i=1, \dots, n} \in (\mathcal{X} \times \mathbb{R})^n$, a training set of n pairs.

The goal is to find a function $f : \mathcal{X} \rightarrow \mathbb{R}$ that predicts y by $f(x)$. A convenient running example is a one-dimensional cloud of noisy points (x_i, y_i) with x ranging over $[0, 10]$, through which we want to pass a sensible curve.

The most classical way to quantify the error made when f predicts $f(x)$ instead of y is the squared error:

$$\ell(f(x), y) = (y - f(x))^2.$$

Fixing a set of functions $\mathcal{H}$, least-square regression amounts to finding the function in $\mathcal{H}$ with the smallest empirical risk, called in this case the mean squared error (MSE):

$$\hat{f} \in \arg \min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2.$$

This raw formulation has well-known issues: it is unstable, especially in large dimensions, and it overfits if $\mathcal{H}$ is too large. Both problems are addressed by the same modification.

5.2.2 The KRR problem and the effect of the representer theorem

Let us now take $\mathcal{H}$ to be the RKHS associated to a p.d. kernel K on $\mathcal{X}$. Kernel ridge regression (KRR) is obtained by regularizing the MSE criterion by the RKHS norm.

Definition (kernel ridge regression)

Given a p.d. kernel K on $\mathcal{X}$ with RKHS $\mathcal{H}$, a training set S_n and a regularization parameter $\lambda > 0$, kernel ridge regression estimates

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2. \quad (\text{KRR})$$

The penalty has two distinct effects, and it is worth keeping them apart.

First effect: statistical. The penalty prevents overfitting by penalizing non-smooth functions. Since the RKHS norm controls the smoothness of f (for the Gaussian kernel, for instance, it controls all derivatives), the objective trades data fit against regularity, and the parameter λ sets the exchange rate.

Second effect: computational, simplifying the solution. The objective is a function of f only through the values $f(x_1), \dots, f(x_n)$ plus a strictly increasing function of $\|f\|_{\mathcal{H}}$, which is exactly the hypothesis of the representer theorem (Kimeldorf and Wahba, 1971; Schölkopf, Herbrich, and Smola, 2001). By that theorem, any solution of (KRR) can therefore be expanded as

$$\hat{f}(x) = \sum_{i=1}^n \alpha_i K(x_i, x)$$

for some vector $\alpha \in \mathbb{R}^n$. This is the pivotal move: an optimization problem over a possibly infinite-dimensional function space has become a problem over n real coefficients, one per training point. The unknown function $\hat{f}$ is now nothing but a weighted sum of kernel bumps $K(x_i, \cdot)$, each centered at a training input, and learning it means fixing the n weights α_i .

5.2.3 Solving KRR: a single linear system

Let us make the finite-dimensional problem explicit. Write

- $\mathbf{y} = (y_1, \dots, y_n)^\top \in \mathbb{R}^n$, the vector of outputs,
- $\alpha = (\alpha_1, \dots, \alpha_n)^\top \in \mathbb{R}^n$, the vector of coefficients,
- $\mathbf{K}$, the $n \times n$ Gram matrix with entries $\mathbf{K}_{ij} = K(x_i, x_j)$.

For $\hat{f} = \sum_i \alpha_i K(x_i, \cdot)$, the vector of training predictions is

$$(\hat{f}(x_1), \dots, \hat{f}(x_n))^\top = \mathbf{K}\alpha,$$

since $\hat{f}(x_j) = \sum_i \alpha_i K(x_i, x_j) = [\mathbf{K}\alpha]_j$. As usual, the reproducing property also gives the norm:

$$\|\hat{f}\|_{\mathcal{H}}^2 = \left\langle \sum_i \alpha_i K(x_i, \cdot), \sum_j \alpha_j K(x_j, \cdot) \right\rangle_{\mathcal{H}} = \sum_{i,j} \alpha_i \alpha_j K(x_i, x_j) = \alpha^\top \mathbf{K} \alpha.$$

The KRR problem is therefore equivalent to the finite-dimensional problem

$$\arg \min_{\alpha \in \mathbb{R}^n} \frac{1}{n} (\mathbf{K}\alpha - \mathbf{y})^\top (\mathbf{K}\alpha - \mathbf{y}) + \lambda \alpha^\top \mathbf{K} \alpha.$$

Proposition (closed form of KRR)

For $\lambda > 0$, the vector

$$\alpha = (\mathbf{K} + \lambda n I)^{-1} \mathbf{y}$$

solves the KRR problem, and $\hat{f}(x) = \sum_{i=1}^n \alpha_i K(x_i, x)$ is the (unique) KRR estimator.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The objective

$$J(\alpha) = \frac{1}{n} (\mathbf{K}\alpha - \mathbf{y})^\top (\mathbf{K}\alpha - \mathbf{y}) + \lambda \alpha^\top \mathbf{K}\alpha$$

is a convex and differentiable function of α : the first term is a convex quadratic composed with a linear map, and the second is convex because $\mathbf{K}$ is positive semidefinite. Its minimum is therefore attained exactly where the gradient in α vanishes. Differentiating,

$$0 = \nabla J(\alpha) = \frac{2}{n} \mathbf{K}(\mathbf{K}\alpha - \mathbf{y}) + 2\lambda \mathbf{K}\alpha = \frac{2}{n} \mathbf{K}[(\mathbf{K} + \lambda n I)\alpha - \mathbf{y}].$$

For $\lambda > 0$, the matrix $\mathbf{K} + \lambda n I$ is invertible: $\mathbf{K}$ is positive semidefinite, so every eigenvalue of $\mathbf{K} + \lambda n I$ is at least $\lambda n > 0$. Hence one solution of the stationarity equation is obtained by making the bracket itself vanish:

$$\alpha = (\mathbf{K} + \lambda n I)^{-1} \mathbf{y}.$$

□

So the entire learning algorithm is: form the Gram matrix, solve one $n \times n$ linear system, and predict a new point x by $\hat{f}(x) = \sum_i \alpha_i K(x_i, x)$. Note that the proof only exhibits *one* solution in α ; when $\mathbf{K}$ is singular the stationarity equation has other solutions, a point we settle below.

It is worth reading the closed form $\alpha = (\mathbf{K} + \lambda n I)^{-1} \mathbf{y}$ slowly, because it says something concrete. Without regularization we would want $\mathbf{K}\alpha = \mathbf{y}$, that is, a curve passing exactly through every observation; the coefficients would be $\alpha = \mathbf{K}^{-1} \mathbf{y}$, which can be enormous and wildly oscillating when two inputs are close and $\mathbf{K}$ is nearly singular. Adding $\lambda n I$ lifts every eigenvalue of $\mathbf{K}$ away from zero, so the inverse can no longer blow up: the coefficients stay bounded and the fit is allowed to miss the data points a little in exchange for a smoother curve. In the eigenbasis of $\mathbf{K}$, the component of $\mathbf{y}$ along an eigenvector with eigenvalue μ is scaled by $\mu/(\mu + \lambda n)$, close to 1 for the large-eigenvalue directions (the smooth, well-supported ones) and close to 0 for the small-eigenvalue directions (the noisy, high-frequency ones). Ridge regression is therefore a soft spectral filter: it keeps the directions the data speaks to loudly and damps the rest.

Remark (the penalty as a prior, and what it measures)

The idea of stabilizing an unstable fit by restricting the admissible functions predates kernels: it is Tikhonov regularization, introduced by Tikhonov and Arsenin (1977) for ill-posed inverse problems and transported here into an RKHS. Two complementary readings explain what the penalty $\lambda \|f\|_{\mathcal{H}}^2$ is doing. Probabilistically, the whole objective is a maximum a posteriori estimate: the empirical risk is a negative log-likelihood of the data and the penalty is a negative log-prior favoring simple functions, so minimizing risk plus penalty is maximizing a posterior. Analytically, Schölkopf and Smola (2002) note that the RKHS norm can be written as $\|f\|_{\mathcal{H}}^2 = \langle Yf, Yf \rangle$ for a regularization operator Y that extracts, roughly, the derivatives of f ; for a translation-invariant kernel Y acts as a multiplier in the Fourier domain, and for the Gaussian kernel it penalizes a weighted sum of all the derivatives of f , damping the high frequencies most. This regularization-operator reading, in which the choice of Y fixes at

once the penalty and a matching network architecture, was developed by Girosi, Jones, and Poggio (1995), and Evgeniou, Pontil, and Poggio (2000) used it to place such regularization networks and support vector machines inside a single Tikhonov framework. This is the precise sense in which a small RKHS norm means a smooth function, and it dovetails with the spectral-filter picture above: the high-frequency directions that the operator Y penalizes are exactly the small-eigenvalue directions of $\mathbf{K}$ that the closed form shrinks toward zero.

Example (the smallest KRR by hand)

Take two training points, $x_1 = 0$ with $y_1 = 1$ and $x_2 = 2$ with $y_2 = 0$, a Gaussian kernel with $\sigma^2 = 2$, and λ chosen so that $\lambda n = 0.5$ (here $n = 2$). Then $K(x_1, x_1) = K(x_2, x_2) = 1$ and $K(x_1, x_2) = e^{-(2)^2/(2 \cdot 2)} = e^{-1} \approx 0.37$, so

$$\mathbf{K} + \lambda n I = \begin{pmatrix} 1.5 & 0.37 \\ 0.37 & 1.5 \end{pmatrix}, \quad \alpha = (\mathbf{K} + \lambda n I)^{-1} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \approx \begin{pmatrix} 0.71 \\ -0.17 \end{pmatrix}.$$

The fitted curve is $\hat{f}(x) = 0.71 K(x, 0) - 0.17 K(x, 2)$. Evaluating at the data, $\hat{f}(0) \approx 0.65$ and $\hat{f}(2) \approx 0.09$: the predictions have been pulled in from the targets 1 and 0 toward each other. That shrinkage is the regularization at work; sending $\lambda \rightarrow 0$ would drive $\hat{f}(0) \rightarrow 1$ and $\hat{f}(2) \rightarrow 0$, a perfect but far less stable interpolation.

Verification artifact. checks/example-ch03-example-4-4.json records the example source hash and verification scope.

The next figure is this algorithm itself, running live rather than described. The curve you see is the exact kernel ridge solution $\alpha = (\mathbf{K} + \lambda n I)^{-1} \mathbf{y}$, re-solved every time you move a point or a slider.

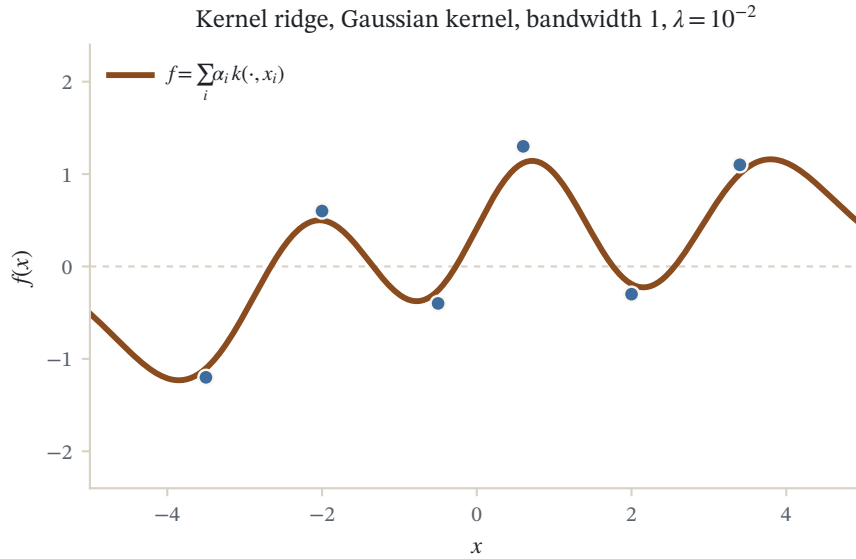


Figure 5.1: Kernel ridge regression on six points with a Gaussian kernel (bandwidth 1, ridge $\lambda = 10^{-2}$). The curve is the exact solution $f = \sum_i \alpha_i k(\cdot, x_i)$ with $\alpha = (\mathbf{K} + \lambda n I)^{-1} \mathbf{y}$.

5.2.4 Worked example: KRR with the Gaussian RBF kernel

It is instructive to run KRR on the one-dimensional noisy dataset described above, using the Gaussian RBF kernel

$$K(x, x') = \exp\left(-\frac{(x - x')^2}{2\sigma^2}\right),$$

and sweeping the regularization parameter over ten orders of magnitude, $\lambda = 10^3, 10^2, 10, 1, 10^{-1}, 10^{-2}, \dots, 10^{-7}$. Each value of λ gives one linear system and one fitted curve overlaid on the data, and the sequence of plots tells the whole bias-variance story of the method:

- For very large λ (say $\lambda = 1000$), the penalty dominates the objective. The fitted curve is nearly flat, close to the zero function: the model underfits, barely reacting to the data.
- As λ decreases through 100, 10, 1, the curve progressively bends to follow the main trend of the point cloud, capturing the low-frequency structure while ignoring the noise. Intermediate values give a visually convincing fit.
- As λ continues to shrink through $10^{-2}, 10^{-4}$, down to 10^{-7} , the penalty becomes negligible and the curve develops increasingly sharp wiggles, chasing individual noisy observations. In the limit the estimator approaches the minimum-norm interpolant of the data: it overfits.

The lesson is that λ is not a nuisance constant but the dial that selects the complexity of the fitted function, and in practice it is chosen by model selection techniques such as cross-validation. The kernel bandwidth σ plays a similar complexity-controlling role: a small σ makes each kernel bump narrow, so the curve can turn quickly and overfit, while a large σ forces neighboring bumps to overlap and blend into a smooth, slowly varying fit. The two knobs interact, which is exactly why it helps to feel them rather than only read about them.

The widget below is the same live kernel ridge solver, offered here for the regularization story specifically: leave the points fixed, push λ down to watch the curve grow sharp wiggles that chase individual observations, then raise it and watch those wiggles get damped out of the fit.

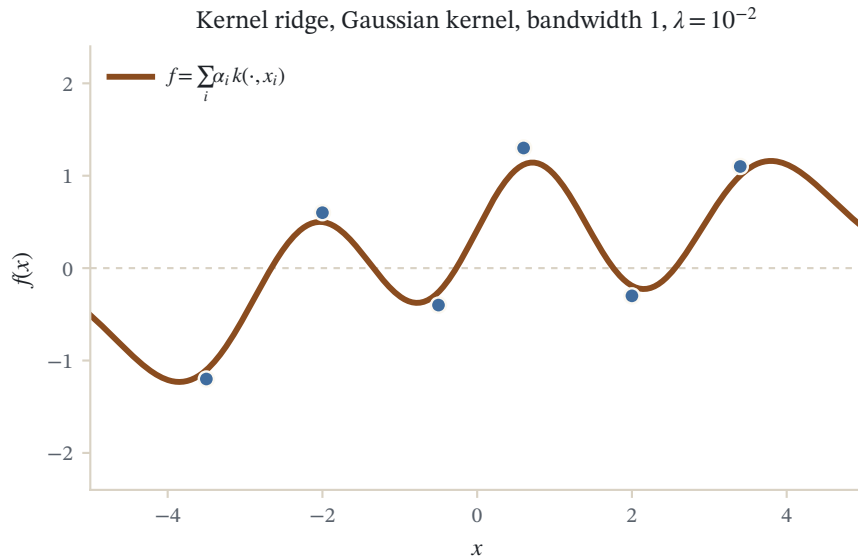


Figure 5.2: Kernel ridge regression on six points with a Gaussian kernel (bandwidth 1, ridge $\lambda = 10^{-2}$). The curve is the exact solution $f = \sum_i \alpha_i k(\cdot, x_i)$ with $\alpha = (K + \lambda nI)^{-1}y$.

5.2.5 Uniqueness of the solution

The stationarity condition $\mathbf{K}[(\mathbf{K} + \lambda nI)\alpha - \mathbf{y}] = 0$ does not force the bracket to vanish when $\mathbf{K}$ is singular, so several coefficient vectors α may be optimal. It is natural to ask whether these different vectors describe different functions. They do not.

Proposition (uniqueness)

For $\lambda > 0$, the set of solutions of $\mathbf{K}[(\mathbf{K} + \lambda nI)\alpha - \mathbf{y}] = 0$ is

$$\alpha = (\mathbf{K} + \lambda n I)^{-1} \mathbf{y} + \epsilon, \quad \text{with } \mathbf{K}\epsilon = 0,$$

and all these vectors define the same function of $\mathcal{H}$. KRR therefore has a unique solution $\hat{f} \in \mathcal{H}$, which can possibly be expressed by several α 's if $\mathbf{K}$ is singular.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

$\mathbf{K}$ being a symmetric matrix, it can be diagonalized in an orthonormal basis, and $\text{Ker}(\mathbf{K}) \perp \text{Im}(\mathbf{K})$. In this eigenbasis, $\mathbf{K} + \lambda n I$ acts diagonally (adding λn to each eigenvalue of $\mathbf{K}$), so $(\mathbf{K} + \lambda n I)^{-1}$ leaves both $\text{Im}(\mathbf{K})$ and $\text{Ker}(\mathbf{K})$ invariant; on $\text{Ker}(\mathbf{K})$ it is simply multiplication by $(\lambda n)^{-1}$.

The stationarity condition says exactly that $(\mathbf{K} + \lambda n I)\alpha - \mathbf{y}$ lies in $\text{Ker}(\mathbf{K})$. Applying the invertible map $(\mathbf{K} + \lambda n I)^{-1}$, which maps $\text{Ker}(\mathbf{K})$ onto itself, this is equivalent to

$$\alpha - (\mathbf{K} + \lambda n I)^{-1} \mathbf{y} \in \text{Ker}(\mathbf{K}),$$

that is,

$$\alpha = (\mathbf{K} + \lambda n I)^{-1} \mathbf{y} + \epsilon, \quad \mathbf{K}\epsilon = 0.$$

Now let $\alpha' = \alpha + \epsilon$ with $\mathbf{K}\epsilon = 0$, and let f, f' be the corresponding functions, $f = \sum_i \alpha_i K(x_i, \cdot)$ and $f' = \sum_i \alpha'_i K(x_i, \cdot)$. Then

$$\|f - f'\|_{\mathcal{H}}^2 = (\alpha - \alpha')^\top \mathbf{K} (\alpha - \alpha') = \epsilon^\top \mathbf{K} \epsilon = 0,$$

therefore $f = f'$ as elements of $\mathcal{H}$. $\square$

5.2.6 Link with standard ridge regression

KRR should reduce to ordinary ridge regression, the classical penalized least-squares estimator of Hoerl and Kennard (1970), when the kernel is linear; checking that it does is instructive, and the check produces a genuinely useful computational rule. The dual, kernelized reading of ridge regression developed here goes back to Saunders, Gamerman, and Vovk (1998), who wrote ridge regression purely in terms of the Gram matrix.

Take $\mathcal{X} = \mathbb{R}^d$ and the linear kernel $K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}'$. Let $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_n)^\top$ be the $n \times d$ data matrix, so that the kernel matrix is $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$. The function learned by KRR is then linear, $f_{\text{KRR}}(\mathbf{x}) = \mathbf{w}_{\text{KRR}}^\top \mathbf{x}$, with

$$\mathbf{w}_{\text{KRR}} = \sum_{i=1}^n \alpha_i \mathbf{x}_i = \mathbf{X}^\top \alpha = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top + \lambda n I)^{-1} \mathbf{y}.$$

On the other hand, for the linear kernel the RKHS is the set of linear functions $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ and the RKHS norm is $\|f\|_{\mathcal{H}} = \|\mathbf{w}\|$. We can therefore rewrite the original KRR problem directly in terms of $\mathbf{w}$:

$$\arg \min_{\mathbf{w} \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|^2 = \arg \min_{\mathbf{w} \in \mathbb{R}^d} \frac{1}{n} (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda \mathbf{w}^\top \mathbf{w}.$$

This is standard ridge regression. Setting the gradient to zero,

$$-\frac{2}{n}\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) + 2\lambda\mathbf{w} = 0 \implies \mathbf{w}_{\text{RR}} = (\mathbf{X}^\top\mathbf{X} + \lambda n\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}.$$

At first sight this looks different from $\mathbf{w}_{\text{KRR}} = \mathbf{X}^\top(\mathbf{X}\mathbf{X}^\top + \lambda n\mathbf{I})^{-1}\mathbf{y}$: the inverted matrix is $d \times d$ in one case and $n \times n$ in the other. The two are reconciled by a classical identity.

Lemma (push-through identity)

For any matrices B and C , and $\gamma > 0$, the following holds (whenever the products and inverses make sense):

$$B(CB + \gamma I)^{-1} = (BC + \gamma I)^{-1}B.$$

This is the push-through identity: it pushes the factor B through the inverse, exchanging CB for BC inside it. It is not the Sherman-Morrison-Woodbury matrix inversion lemma, which rewrites the inverse of a low-rank update; here nothing is inverted twice, and the two sides simply relate the $d \times d$ and $n \times n$ resolvents.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Both $CB + \gamma I$ and $BC + \gamma I$ are assumed invertible. Start from the identity

$$(BC + \gamma I)B = BCB + \gamma B = B(CB + \gamma I).$$

Multiplying on the left by $(BC + \gamma I)^{-1}$ and on the right by $(CB + \gamma I)^{-1}$ gives the claim. $\square$

Applying the lemma with $B = \mathbf{X}^\top$, $C = \mathbf{X}$ and $\gamma = \lambda n$, we deduce (of course, since both formulas minimize the same strictly convex objective):

$$\mathbf{w}_{\text{RR}} = \underbrace{(\mathbf{X}^\top\mathbf{X} + \lambda n\mathbf{I})^{-1}}_{d \times d} \mathbf{X}^\top\mathbf{y} = \mathbf{X}^\top \underbrace{(\mathbf{X}\mathbf{X}^\top + \lambda n\mathbf{I})^{-1}}_{n \times n} \mathbf{y} = \mathbf{w}_{\text{KRR}}.$$

Remark (which formula to compute)

Computationally, inverting the matrix (or rather, solving the corresponding linear system) is the expensive part. The two equivalent formulas involve matrices of different sizes, which suggests the following rule:

Regime	Formula	System size
$d > n$ (high dimension, few points)	KRR form, invert $\mathbf{X}\mathbf{X}^\top + \lambda n\mathbf{I}$	$n \times n$
$d < n$ (many points, moderate dimension)	RR form, invert $\mathbf{X}^\top\mathbf{X} + \lambda n\mathbf{I}$	$d \times d$

This is a first instance of a recurring theme: kernelization trades the dimension of the feature space for the number of training points.

Remark (a stability bound for ridge regression)

The Gram-matrix reading of ridge regression is also the natural setting for a generalization guarantee. Shawe-Taylor and Cristianini (2004) show, through a stability analysis of the least-squares fit, that the expected squared error of the learned function is bounded by its empirical squared error on the training set plus a complexity term that grows with the trace of the kernel matrix, $\text{tr}(\mathbf{K}) = \sum_i K(x_i, x_i)$, and with the bound B imposed on the RKHS norm through $\|f\|_{\mathcal{H}} \leq B$. The shape of the bound is the whole justification for regularizing: shrinking B shrinks the complexity term, so controlling the norm controls the gap between training error and test error, which is precisely the overfitting the penalty $\lambda\|f\|_{\mathcal{H}}^2$ is there to prevent. The learning-theoretic reason to regularize and the numerical reason, a well-conditioned linear system, are two views of the same parameter λ .

5.2.7 Three readings of the smoother: Gaussian processes, degrees of freedom, cross-validation

The KRR fit is a linear function of the outputs. Writing the training predictions as

$$\hat{\mathbf{y}} = \mathbf{K}\alpha = \mathbf{K}(\mathbf{K} + \lambda n I)^{-1} \mathbf{y} =: \mathbf{S}_\lambda \mathbf{y},$$

introduces the smoother matrix $\mathbf{S}_\lambda$, which maps observed outputs to fitted values. Three standard readings of $\mathbf{S}_\lambda$ round out the picture and, in the last case, hand us a way to choose λ .

Remark (kernel ridge regression is a Gaussian process posterior mean)

Place a Gaussian process prior $f \sim \mathcal{GP}(0, K)$ on the unknown function, so that any finite collection of values $(f(x_1), \dots, f(x_n))$ is jointly Gaussian with covariance $\mathbf{K}$, and suppose the outputs are noisy observations $y_i = f(x_i) + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ independent. The posterior mean of f at a test point x is then

$$\mathbb{E}[f(x) \mid \mathbf{S}_n] = \mathbf{k}(x)^\top (\mathbf{K} + \sigma^2 I)^{-1} \mathbf{y}, \quad \mathbf{k}(x) = (K(x_1, x), \dots, K(x_n, x))^\top.$$

This is exactly the KRR predictor $\hat{f}(x) = \sum_i \alpha_i K(x_i, x) = \mathbf{k}(x)^\top (\mathbf{K} + \lambda n I)^{-1} \mathbf{y}$ once we set the observation-noise variance to $\sigma^2 = \lambda n$. Kernel ridge regression is therefore the posterior mean of a Gaussian process with covariance kernel K , and the regularization parameter is a noise level: more assumed noise means more smoothing. The full Bayesian treatment, which also delivers the posterior variance as an error bar, is developed by Rasmussen and Williams (2006).

Remark (effective degrees of freedom)

Since $\mathbf{S}_\lambda$ plays the role that the projection onto the fitted subspace plays in ordinary least squares, its trace measures how many parameters the fit effectively spends. The effective degrees of freedom of KRR is

$$\text{df}(\lambda) = \text{tr}(\mathbf{S}_\lambda) = \text{tr}(\mathbf{K}(\mathbf{K} + \lambda n I)^{-1}) = \sum_{i=1}^n \frac{\mu_i}{\mu_i + \lambda n},$$

where the μ_i are the eigenvalues of $\mathbf{K}$. Each term is the spectral-filter factor of one eigendirection, so $\text{df}(\lambda)$ counts the directions the data actually determines: it ranges from n as $\lambda \rightarrow 0$ (every direction fitted) down to 0 as $\lambda \rightarrow \infty$ (the flat fit), and it is the honest replacement for the number of parameters when comparing fits at different λ .

5.2.8 Leave-one-out cross-validation in closed form

Cross-validation estimates test error by refitting on data with some points held out. Leave-one-out cross-validation takes this to its limit, holding out a single point at a time, and so appears to demand n separate fits, one per point. For a linear smoother it demands none beyond the first. Because the KRR fitted values are $\hat{\mathbf{y}} = \mathbf{S}_\lambda \mathbf{y}$ with a smoother $\mathbf{S}_\lambda$ that does not depend on the responses, the effect of deleting a point can be undone analytically, and all n leave-one-out errors fall out of the one fit already computed. This is the third reading of the smoother, and the one that hands us a principled rule for choosing λ .

Proposition (closed-form leave-one-out for KRR)

Let $\mathbf{S}_\lambda = \mathbf{K}(\mathbf{K} + \lambda n I)^{-1}$ be the KRR smoother, $\hat{\mathbf{y}} = \mathbf{S}_\lambda \mathbf{y}$ the fitted training values, and $e_i = y_i - \hat{f}(x_i)$ the in-sample residuals. Let $\hat{f}^{(-i)}$ be the KRR estimator obtained from the same problem with the i th pair (x_i, y_i) removed, using the same kernel, the same λ , and the same $1/n$ normalization. Then for every i with $[\mathbf{S}_\lambda]_{ii} \neq 1$,

$$y_i - \hat{f}^{(-i)}(x_i) = \frac{e_i}{1 - [\mathbf{S}_\lambda]_{ii}}.$$

Consequently the leave-one-out score is a function of the single fit at that λ , with no refitting:

$$\text{LOO}(\lambda) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}^{(-i)}(x_i))^2 = \frac{1}{n} \sum_{i=1}^n \left(\frac{e_i}{1 - [\mathbf{S}_\lambda]_{ii}} \right)^2.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Fix an index i and abbreviate $\mathbf{S} = \mathbf{S}_\lambda$. Build a phantom response vector $\tilde{\mathbf{y}}$ equal to $\mathbf{y}$ in every coordinate except the i th, where we substitute the leave-one-out prediction itself, $\tilde{y}_i := \hat{f}^{(-i)}(x_i)$.

Step 1: the phantom fit is the leave-one-out fit. Kernel ridge regression maps a response vector $\mathbf{v}$ to the minimizer over $f \in \mathcal{H}$ of $\frac{1}{n} \sum_{j=1}^n (v_j - f(x_j))^2 + \lambda \|f\|_{\mathcal{H}}^2$. Split the sum for $\mathbf{v} = \tilde{\mathbf{y}}$ into the terms $j \neq i$ and the term $j = i$. The estimator $\hat{f}^{(-i)}$ minimizes exactly the $j \neq i$ part plus the penalty. At $f = \hat{f}^{(-i)}$ the remaining term is $(\tilde{y}_i - \hat{f}^{(-i)}(x_i))^2 = 0$, its least possible value. A function that minimizes the $j \neq i$ part and simultaneously drives the nonnegative $j = i$ term to zero minimizes the whole objective, so the KRR fit on $\tilde{\mathbf{y}}$ is $\hat{f}^{(-i)}$.

Step 2: use linearity of the smoother. The fitted value at x_i is the linear map $\mathbf{v} \mapsto [\mathbf{S}\mathbf{v}]_i$, and $\mathbf{S}$ depends only on $\mathbf{K}$ and λ , not on the responses. Applying it to $\mathbf{y}$ gives $\hat{f}(x_i) = [\mathbf{S}\mathbf{y}]_i$, and applying it to $\tilde{\mathbf{y}}$ gives, by Step 1, $[\mathbf{S}\tilde{\mathbf{y}}]_i = \hat{f}^{(-i)}(x_i) = \tilde{y}_i$.

Step 3: solve for the leave-one-out value. The vectors $\mathbf{y}$ and $\tilde{\mathbf{y}}$ differ only in coordinate i , so $[\mathbf{S}\tilde{\mathbf{y}}]_i - [\mathbf{S}\mathbf{y}]_i = [\mathbf{S}]_{ii}(\tilde{y}_i - y_i)$. Substituting the two evaluations from Step 2,

$$\tilde{y}_i - \hat{f}(x_i) = [\mathbf{S}]_{ii}(\tilde{y}_i - y_i),$$

which rearranges to $\tilde{y}_i(1 - [\mathbf{S}]_{ii}) = \hat{f}(x_i) - [\mathbf{S}]_{ii} y_i$. Therefore

$$y_i - \hat{f}^{(-i)}(x_i) = y_i - \tilde{y}_i = \frac{y_i(1 - [\mathbf{S}]_{ii}) - \hat{f}(x_i) + [\mathbf{S}]_{ii} y_i}{1 - [\mathbf{S}]_{ii}} = \frac{y_i - \hat{f}(x_i)}{1 - [\mathbf{S}]_{ii}} = \frac{e_i}{1 - [\mathbf{S}]_{ii}}.$$

□

The identity is the reproducing-kernel instance of the classical leave-one-out formula for linear smoothers (Wahba, 1990; Rifkin and Lippert, 2007). It has a clean reading: the leave-one-out residual inflates each in-sample residual by the factor $1/(1 - [\mathbf{S}_\lambda]_{ii}) \geq 1$. A point on which the fit leans heavily, so that $[\mathbf{S}_\lambda]_{ii}$ is close to one, is one the model would badly miss if it were removed, and the naive resubstitution error $\frac{1}{n} \sum_i e_i^2$ understates the true error by ignoring exactly this leverage.

Replacing every diagonal $[\mathbf{S}_\lambda]_{ii}$ by their common average $\text{tr}(\mathbf{S}_\lambda)/n = \text{df}(\lambda)/n$ turns the leave-one-out score into generalized cross-validation,

$$\text{GCV}(\lambda) = \frac{1}{n} \sum_{i=1}^n \frac{(y_i - \hat{f}(x_i))^2}{(1 - \text{df}(\lambda)/n)^2} = \frac{\frac{1}{n} \|\mathbf{I} - \mathbf{S}_\lambda\|_{\mathbf{y}}^2}{(1 - \text{df}(\lambda)/n)^2},$$

the criterion of Craven and Wahba (1979) and Golub, Heath, and Wahba (1979). Averaging the diagonal makes GCV invariant to an orthogonal rotation of the fitting problem and lets it be read off from the single scalar $\text{tr}(\mathbf{S}_\lambda)$, the effective degrees of freedom $\text{df}(\lambda)$ of the remark above, so the model-selection criterion is tied directly to model complexity. Either score reduces the choice of λ to a one-dimensional search that never holds out data by hand.

Example (leave-one-out for KRR: one fit versus five)

Five points $x = (0, 1, 2, 3, 4)$ with a symmetric W-shaped target $\mathbf{y} = (0.2, 1.0, 0.3, 1.0, 0.2)$, a Gaussian kernel $K(x, x') = e^{-(x-x')^2/2}$ (so $\sigma^2 = 1$, with $K_{i,i+1} = 0.6065$ and $K_{i,i+2} = 0.1353$), and $\lambda = 0.3$, so $\lambda n = 1.5$. Relative to the kernel width the two peaks at $x = 1, 3$ are high-frequency structure.

1. Fit once. Solve $\alpha = (\mathbf{K} + 1.5 \mathbf{I})^{-1} \mathbf{y}$ and form the fitted values $\hat{\mathbf{y}} = \mathbf{S}_\lambda \mathbf{y} = (0.2222, 0.4007, 0.4084, 0.4007, 0.2222)$, giving residuals $\mathbf{e} = (-0.0222, 0.5993, -0.1084, 0.5993, -0.0222)$. The two peaks are the badly fitted points.
2. Read the smoother diagonal. The hat matrix $\mathbf{S}_\lambda = \mathbf{K}(\mathbf{K} + 1.5 \mathbf{I})^{-1}$ has diagonal $[\mathbf{S}_\lambda]_{ii} = (0.3624, 0.3245, 0.3246, 0.3245, 0.3624)$, so the inflation denominators are $1 - [\mathbf{S}_\lambda]_{ii} = (0.6376, 0.6755, 0.6754, 0.6755, 0.6376)$, and the trace is $\text{tr}(\mathbf{S}_\lambda) = \text{df}(0.3) = 1.6984$.
3. Inflate each residual. Dividing termwise, the closed-form leave-one-out residuals are $e_i / (1 - [\mathbf{S}_\lambda]_{ii}) = (-0.0348, 0.8873, -0.1605, 0.8873, -0.0348)$. Each peak point, poorly reconstructed by its neighbours once removed, carries a leave-one-out residual near 0.89, well above its training residual of 0.60.
4. Check against five refits. Deleting each point in turn and refitting KRR on the other four, keeping the same λ so that the penalty added to the 4×4 Gram is still $\lambda n = 1.5$, then predicting the held-out point, reproduces $(-0.0348, 0.8873, -0.1605, 0.8873, -0.0348)$ to machine precision, the largest discrepancy being 2×10^{-16} . The single fit already held all five answers.
5. Score. Averaging the squares gives $\text{LOO}(0.3) = 0.3205$, whereas the naive resubstitution error is $\frac{1}{n} \sum_i e_i^2 = 0.1462$: the honest estimate is more than twice the training error. Using the average $\text{df}/n = 1.6984/5$ in place of the individual diagonals gives $\text{GCV}(0.3) = 0.3354$, close to LOO.
6. Sweep λ . Recomputing from one fit per λ , the leave-one-out curve is U-shaped with an interior minimum at $\lambda = 0.3$, while the degrees of freedom climb toward $n = 5$ as λ falls:

λ	$\text{df}(\lambda) = \text{tr}(\mathbf{S}_\lambda)$	$\text{LOO}(\lambda)$	$\text{GCV}(\lambda)$
3	0.3015	0.3998	0.4002
1	0.7643	0.3582	0.3610
0.3	1.6984	0.3205	0.3354
0.1	2.7556	0.3730	0.4196
0.03	3.7791	0.5568	0.6642
0.01	4.4221	0.7512	0.9025

The minimizing row is in bold; LOO worsens on both sides of $\lambda = 0.3$.

Reading. One matrix inversion produced the entire leave-one-out curve: no data was held out and no model refit. Its minimizer at $\lambda = 0.3$ trades the underfitting of large λ , which flattens the two peaks, against the overfitting of small λ , which chases every point through a kernel too narrow to interpolate them, and the effective degrees of freedom read the chosen complexity as about 1.7 parameters out of a possible five.

Verification artifact. checks/example-ch03-example-4-5.json records the example source hash and verification scope.

5.2.9 Beyond the squared loss: robust regression

The squared error $\ell(t, y) = (t - y)^2$ is a convention, not a law of nature, and it is sensitive to outliers: a single grossly wrong observation contributes quadratically to the risk and can drag the whole fit toward it. The reason is exactly the quadratic growth, since the derivative $2(t - y)$ grows without bound, so a point far from the curve exerts an unbounded pull. Many other loss functions exist for regression, and the cure is always the same idea, namely to make the loss grow more slowly than a parabola once the residual is large. Three common alternatives illustrate the range. The absolute loss $|t - y|$ grows only linearly, so a distant outlier pulls with bounded force. The Huber loss (Huber, 1964) interpolates between the two, staying quadratic for small residuals, where quadratic

behavior is statistically efficient, and switching to linear growth beyond a threshold, where robustness matters. The ε -insensitive loss used by support vector regression (Vapnik, 1995) goes further and charges nothing at all while the residual stays within a tube of width ε , which additionally makes the solution sparse in the training points. Any such loss leads to a valid kernel method through the general recipe of the first section, and it is usually solved by numerical optimization, since there is usually no analytical solution beyond the squared error. The next two sections develop exactly this program: first for a weighted squared loss (still analytical), then for the logistic loss (numerical, but reducible to the weighted squared case).

There is a deeper way to read this menu of losses, due to Schölkopf and Smola (2002): every regression loss is secretly a noise model. Suppose the outputs are generated as $y = f(x) + \xi$ with the noise ξ drawn from a density p . The maximum-likelihood estimate of f then minimizes $\sum_i -\ln p(y_i - f(x_i))$, so identifying the loss $\ell(r)$ in the residual $r = y - f(x)$ with $-\ln p(r)$ turns the choice of a loss into the choice of a noise distribution. The common losses line up with familiar densities.

Loss $\ell(r)$	Implied noise density $p(r) \propto$	Character
Squared, r^2	$e^{-r^2/(2\sigma^2)}$, Gaussian	light tails, efficient, fragile to outliers
Absolute, $ r $	$e^{- r }$, Laplacian	heavy tails, robust, non-smooth at 0
Huber	Gaussian core, exponential tails	efficient near 0, robust far out
ε -insensitive	flat on $[-\varepsilon, \varepsilon]$, exponential outside	robust, and sparse in the data

The squared loss is thus the maximum-likelihood choice exactly when the noise is Gaussian, which is both why least squares feels so natural and why it is so sensitive to outliers: the Gaussian assigns vanishing probability to large residuals, so a single distant point is treated as almost impossible and the fit strains to accommodate it. Choosing a heavier-tailed loss, absolute or Huber or ε -insensitive, is choosing to disbelieve extreme residuals, and that disbelief is exactly what robustness means. The Huber loss is the cleanest illustration: its Gaussian core keeps the statistical efficiency of least squares where residuals are small, while its exponential tails cap the influence any one gross outlier can have.

Remark (why some losses give sparse solutions)

The ε -insensitive loss was singled out above as producing a solution that is sparse in the training points, and Shawe-Taylor and Cristianini (2004) explain the mechanism in a way that unifies several observations of this chapter. When a loss has a flat region, an interval of residuals on which it is identically zero and so has zero derivative, the optimality conditions force the coefficient α_i of any training point that lands in that region to vanish: such a point exerts no force on the fit and can be removed without changing the solution. The surviving points, those on the boundary of or outside the flat region, are the support vectors. The squared loss has no flat region, its derivative $2r$ vanishes only at the single point $r = 0$, so generically every α_i is nonzero and kernel ridge regression yields a dense expansion in which all n training points must be stored and evaluated at test time. The ε -insensitive loss trades a little accuracy for a wide flat tube and, with it, a sparse and cheaply evaluated predictor. This is the regression counterpart of the support-vector sparsity we will meet for the hinge loss in classification.

5.2.10 Weighted regression

Sometimes not all training points deserve the same influence: observations may come with known noise levels, or, as we will see for logistic regression, an outer algorithm may want to reweight points at each iteration. Given weights $W_1, \dots, W_n \in \mathbb{R}$, a variant of ridge regression weights the error differently at different points:

$$\arg \min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n W_i (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2.$$

The objective still depends on f only through its values at the training points and its norm, so by the representer theorem the solution is again $f(x) = \sum_{i=1}^n \alpha_i K(x_i, x)$, where α now solves, with $\mathbf{W} = \text{diag}(W_1, \dots, W_n)$:

$$\arg \min_{\alpha \in \mathbb{R}^n} \frac{1}{n} (\mathbf{K}\alpha - \mathbf{y})^\top \mathbf{W} (\mathbf{K}\alpha - \mathbf{y}) + \lambda \alpha^\top \mathbf{K} \alpha.$$

Proposition (weighted KRR)

Assume $W_i > 0$ for all i and $\lambda > 0$. A solution of the weighted KRR problem is

$$\alpha = \mathbf{W}^{\frac{1}{2}} \left(\mathbf{W}^{\frac{1}{2}} \mathbf{K} \mathbf{W}^{\frac{1}{2}} + n\lambda I \right)^{-1} \mathbf{W}^{\frac{1}{2}} \mathbf{y}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The objective is convex and differentiable in α (the first term is a positive semidefinite quadratic form since $\mathbf{W} \geq 0$), so it suffices to find a zero of the gradient. Setting the gradient to zero gives

$$0 = \frac{2}{n} (\mathbf{K} \mathbf{W} \mathbf{K} \alpha - \mathbf{K} \mathbf{W} \mathbf{y}) + 2\lambda \mathbf{K} \alpha.$$

Since the W_i are positive, $\mathbf{W}^{\frac{1}{2}} = \text{diag}(W_1^{1/2}, \dots, W_n^{1/2})$ is well defined and invertible, and we can factor the right-hand side:

$$0 = \frac{2}{n} \mathbf{K} \mathbf{W}^{\frac{1}{2}} \left[\left(\mathbf{W}^{\frac{1}{2}} \mathbf{K} \mathbf{W}^{\frac{1}{2}} + n\lambda I \right) \mathbf{W}^{-\frac{1}{2}} \alpha - \mathbf{W}^{\frac{1}{2}} \mathbf{y} \right].$$

Indeed, expanding the bracket and multiplying through by $\mathbf{K} \mathbf{W}^{\frac{1}{2}}$ recovers $\mathbf{K} \mathbf{W} \mathbf{K} \alpha + n\lambda \mathbf{K} \alpha - \mathbf{K} \mathbf{W} \mathbf{y}$, which matches the gradient up to the factor $2/n$. A sufficient condition for optimality is therefore that the bracket vanishes:

$$\left(\mathbf{W}^{\frac{1}{2}} \mathbf{K} \mathbf{W}^{\frac{1}{2}} + n\lambda I \right) \mathbf{W}^{-\frac{1}{2}} \alpha - \mathbf{W}^{\frac{1}{2}} \mathbf{y} = 0.$$

The matrix $\mathbf{W}^{\frac{1}{2}} \mathbf{K} \mathbf{W}^{\frac{1}{2}}$ is positive semidefinite (it is a congruence of $\mathbf{K}$), so adding $n\lambda I$ makes it invertible, and

$$\mathbf{W}^{-\frac{1}{2}} \alpha = \left(\mathbf{W}^{\frac{1}{2}} \mathbf{K} \mathbf{W}^{\frac{1}{2}} + n\lambda I \right)^{-1} \mathbf{W}^{\frac{1}{2}} \mathbf{y},$$

which is the announced formula after multiplying by $\mathbf{W}^{\frac{1}{2}}$. $\square$

Remark

The symmetrization by $\mathbf{W}^{\frac{1}{2}}$ is not cosmetic: it keeps the matrix to invert symmetric positive definite, which is what standard linear solvers (Cholesky, conjugate gradient) want. When $\mathbf{W} = I$ the formula collapses back to the unweighted solution $\alpha = (\mathbf{K} + n\lambda I)^{-1} \mathbf{y}$. Keep the weighted solver in mind: it is the inner engine of kernel logistic regression below.

5.3 Kernel logistic regression

5.3.1 Binary classification and the trouble with the 0/1 loss

We now turn to classification:

- $\mathcal{X}$, a set of inputs;
- $\mathcal{Y} = \{-1, 1\}$, binary outputs;
- $S_n = (x_i, y_i)_{i=1, \dots, n} \in (\mathcal{X} \times \mathcal{Y})^n$, a training set of n pairs.

Following the general recipe, the goal is to find a real-valued function $f : \mathcal{X} \rightarrow \mathbb{R}$ and to predict y by $\text{sign}(f(x))$.

The most natural loss simply measures whether a prediction is correct or not. The 0/1 loss is

$$\ell_{0/1}(f(x), y) = \mathbf{1}(yf(x) < 0) = \begin{cases} 0 & \text{amp; if } y = \text{sign}(f(x)), \\ 1 & \text{amp; otherwise.} \end{cases}$$

It is then tempting to learn f by solving

$$\min_{f \in \mathcal{H}} \underbrace{\frac{1}{n} \sum_{i=1}^n \ell_{0/1}(f(x_i), y_i)}_{\text{misclassification rate}} + \underbrace{\lambda \|f\|_{\mathcal{H}}^2}_{\text{regularization}}.$$

However, this direct approach fails for three compounding reasons:

- **The problem is non-smooth, and typically NP-hard to solve.** The 0/1 loss is piecewise constant, so gradients carry no information, and minimizing the misclassification count is combinatorial.
- **The regularization has no effect**, since the 0/1 loss is invariant by scaling of f : for any $t > 0$, $\text{sign}(tf(x)) = \text{sign}(f(x))$, so $R_n(tf) = R_n(f)$ while $\lambda \|tf\|_{\mathcal{H}}^2 = t^2 \lambda \|f\|_{\mathcal{H}}^2$ can be made arbitrarily small. The penalty cannot discriminate between functions with the same sign pattern.
- **In fact, no function achieves the minimum when $\lambda > 0$.** The scaling argument shows why: given any candidate f with $\|f\|_{\mathcal{H}} > 0$, the function tf with $0 < t < 1$ has the same empirical 0/1 risk and a strictly smaller penalty, so f cannot be a minimizer. The infimum of the objective equals the infimum of the misclassification rate alone, approached along sequences tf with $t \rightarrow 0$ whose limit carries no classification information.

The way out is to replace the 0/1 loss by a smooth, convex proxy. Logistic regression provides a principled one.

5.3.2 The logistic model and the logistic loss

An alternative is to define a probabilistic model of y parametrized by $f(x)$. Let σ denote the sigmoid function, and model

$$\forall y \in \{-1, 1\}, \quad p(y | f(x)) = \frac{1}{1 + e^{-yf(x)}} = \sigma(yf(x)).$$

This is a bona fide conditional distribution: $\sigma(u) + \sigma(-u) = 1$, so the two probabilities $p(1 | f(x)) = \sigma(f(x))$ and $p(-1 | f(x)) = \sigma(-f(x))$ sum to one. Large positive $f(x)$ means the model is confident that $y = 1$; $f(x) = 0$ means indifference. Plotting $\sigma(u)$ and $\sigma(-u)$ gives two mirror-image S-shaped curves crossing at height 1/2 at $u = 0$.

The *logistic loss* is the negative conditional log-likelihood of this model:

$$\ell_{\text{logistic}}(f(x), y) = -\ln p(y | f(x)) = \ln(1 + e^{-yf(x)}).$$

It is a smooth, convex, decreasing function of the product $yf(x)$: confident correct predictions cost almost nothing, confident wrong predictions cost roughly $|f(x)|$, and the transition is smooth. It upper-bounds a scaled 0/1 loss, so pushing it down pushes errors down too.

Reading off a few values fixes the scale. The loss depends only on the margin $u = yf(x)$. At $u = 0$, where the model is undecided, it charges $\ln 2 \approx 0.69$. A correctly and confidently classified point at $u = 2$ pays $\ln(1 + e^{-2}) \approx 0.13$, and one at $u = 4$ pays only $\ln(1 + e^{-4}) \approx 0.018$: the cost decays toward zero but never quite reaches it. A confidently wrong point at $u = -2$ pays $\ln(1 + e^2) \approx 2.13$, close to the value $|u| = 2$ that the loss shadows on the wrong side. The small residual cost even for correct answers is the price of smoothness, and it is exactly why, as we will see, every training point keeps exerting a little pull on the fitted function.

Definition (kernel logistic regression)

Kernel logistic regression (KLR) estimates

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \ell_{\text{logistic}}(f(x_i), y_i) + \lambda \|f\|_{\mathcal{H}}^2 = \arg \min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \ln(1 + e^{-y_i f(x_i)}) + \lambda \|f\|_{\mathcal{H}}^2.$$

Two remarks on this objective. First, it can be interpreted as a regularized conditional maximum likelihood estimator: minimizing the average negative log-likelihood is maximizing the likelihood of the observed labels under the sigmoid model, and the norm penalty plays the role of a prior favoring smooth score functions. Second, unlike KRR there is no explicit solution, but the problem is a smooth convex optimization problem that can be solved numerically, and we will see that the numerics are pleasantly structured. The kernelized form of the estimator is due to Zhu and Hastie (2005), and it inherits a well-established algorithmic idea, namely the iteratively reweighted least-squares scheme long used to fit generalized linear models (Green, 1984), which we develop next.

5.3.3 Reduction to a finite-dimensional convex problem

The KLR objective depends on f only through its values at the training points and through $\|f\|_{\mathcal{H}}$, so the representer theorem applies once more: any solution can be expanded as

$$\hat{f}(x) = \sum_{i=1}^n \alpha_i K(x_i, x),$$

and as always,

$$(\hat{f}(x_1), \dots, \hat{f}(x_n))^T = \mathbf{K}\alpha \quad \text{and} \quad \|\hat{f}\|_{\mathcal{H}}^2 = \alpha^T \mathbf{K}\alpha.$$

To find α we therefore need to solve

$$\min_{\alpha \in \mathbb{R}^n} \frac{1}{n} \sum_{i=1}^n \ln(1 + e^{-y_i [\mathbf{K}\alpha]_i}) + \lambda \alpha^T \mathbf{K}\alpha.$$

Before attacking this problem, we record the calculus facts about the sigmoid and the logistic loss that make everything below mechanical. Write $\ell_{\text{logistic}}(u) = \ln(1 + e^{-u})$ for the loss as a function of the margin variable $u = yf(x)$.

Lemma (technical facts)

For all $u \in \mathbb{R}$:

- Sigmoid: $\sigma(u) = \frac{1}{1 + e^{-u}}$, $\sigma(-u) = 1 - \sigma(u)$, $\sigma'(u) = \sigma(u)\sigma(-u) \geq 0$.
- Logistic loss: $\ell_{\text{logistic}}(u) = \ln(1 + e^{-u})$, $\ell'_{\text{logistic}}(u) = -\sigma(-u)$, $\ell''_{\text{logistic}}(u) = \sigma(u)\sigma(-u) \geq 0$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

That $\sigma(-u) = 1 - \sigma(u)$ follows by direct computation: $\sigma(u) + \sigma(-u) = \frac{1}{1+e^{-u}} + \frac{1}{1+e^u} = \frac{e^u}{e^u+1} + \frac{1}{1+e^u} = 1$. Differentiating $\sigma(u) = (1 + e^{-u})^{-1}$ gives $\sigma'(u) = \frac{e^{-u}}{(1+e^{-u})^2} = \sigma(u) \cdot \frac{e^{-u}}{1+e^{-u}} = \sigma(u)\sigma(-u)$, which is nonnegative. For the loss, $\ell'_{\text{logistic}}(u) = \frac{-e^{-u}}{1+e^{-u}} = -\sigma(-u)$, and differentiating once more, $\ell''_{\text{logistic}}(u) = -\sigma'(-u) \cdot (-1) = \sigma(-u)\sigma(u) \geq 0$. In particular ℓ_{logistic} is convex. $\square$

Note also that $0 < \ell''_{\text{logistic}}(u) = \sigma(u)\sigma(-u) \leq 1/4$, so the loss is convex with bounded curvature; combined with the positive semidefinite quadratic penalty, the KLR objective in α is convex and smooth.

5.3.4 Newton's method and the quadratic approximation

Write the objective as

$$J(\alpha) = \frac{1}{n} \sum_{i=1}^n \ell_{\text{logistic}}(y_i [\mathbf{K}\alpha]_i) + \lambda \alpha^\top \mathbf{K}\alpha.$$

This is a smooth convex optimization problem that many numerical methods can solve. Let us make one of them explicit: Newton's method, which iteratively approximates J by a quadratic function and solves the quadratic problem. The quadratic approximation of J near a point α^0 is

$$J_q(\alpha) = J(\alpha^0) + (\alpha - \alpha^0)^\top \nabla J(\alpha^0) + \frac{1}{2}(\alpha - \alpha^0)^\top \nabla^2 J(\alpha^0)(\alpha - \alpha^0),$$

and one Newton step replaces α^0 by the minimizer of J_q . We need the gradient and the Hessian.

Gradient. By the chain rule, using $[\mathbf{K}\alpha]_i = \sum_j K_{ij} \alpha_j$,

$$\frac{\partial J}{\partial \alpha_j} = \frac{1}{n} \sum_{i=1}^n \underbrace{\ell'_{\text{logistic}}(y_i [\mathbf{K}\alpha]_i)}_{P_i(\alpha)} y_i K_{ij} + \lambda [\mathbf{K}\alpha]_j,$$

therefore, collecting the per-example derivatives into the diagonal matrix $\mathbf{P}(\alpha) = \text{diag}(P_1(\alpha), \dots, P_n(\alpha))$ and using the symmetry of $\mathbf{K}$,

$$\nabla J(\alpha) = \frac{1}{n} \mathbf{K} \mathbf{P}(\alpha) \mathbf{y} + 2\lambda \mathbf{K}\alpha.$$

Hessian. Differentiating once more, and using $y_i^2 = 1$,

$$\frac{\partial^2 J}{\partial \alpha_j \partial \alpha_l} = \frac{1}{n} \sum_{i=1}^n \underbrace{\ell''_{\text{logistic}}(y_i [\mathbf{K}\alpha]_i)}_{W_i(\alpha)} y_i K_{ij} y_i K_{il} + 2\lambda K_{jl},$$

therefore, with $\mathbf{W}(\alpha) = \text{diag}(W_1(\alpha), \dots, W_n(\alpha))$,

$$\nabla^2 J(\alpha) = \frac{1}{n} \mathbf{K} \mathbf{W}(\alpha) \mathbf{K} + 2\lambda \mathbf{K}.$$

Note that each curvature weight $W_i(\alpha) = \sigma([\mathbf{K}\alpha]_i)\sigma(-[\mathbf{K}\alpha]_i)$ is strictly positive, and largest for points near the decision boundary, where the model is most uncertain.

Assembling the quadratic model. Fix α^0 and abbreviate $\mathbf{P} = \mathbf{P}(\alpha^0)$, $\mathbf{W} = \mathbf{W}(\alpha^0)$. In $J_q(\alpha)$, let us isolate the terms that depend on α (all others are constants for the minimization):

$$\alpha^\top \nabla J(\alpha^0) = \frac{1}{n} \alpha^\top \mathbf{K} \mathbf{P} \mathbf{y} + 2\lambda \alpha^\top \mathbf{K} \alpha^0,$$

$$\frac{1}{2} \alpha^\top \nabla^2 J(\alpha^0) \alpha = \frac{1}{2n} \alpha^\top \mathbf{K} \mathbf{W} \mathbf{K} \alpha + \lambda \alpha^\top \mathbf{K} \alpha,$$

$$-\alpha^\top \nabla^2 J(\alpha^0) \alpha^0 = -\frac{1}{n} \alpha^\top \mathbf{K} \mathbf{W} \mathbf{K} \alpha^0 - 2\lambda \alpha^\top \mathbf{K} \alpha^0.$$

Summing the three contributions (the cross terms $2\lambda \alpha^\top \mathbf{K} \alpha^0$ cancel) and multiplying by 2, we obtain, up to an additive constant C independent of α :

$$2J_q(\alpha) = -\frac{2}{n} \alpha^\top \mathbf{K} \mathbf{W} \underbrace{(\mathbf{K} \alpha^0 - \mathbf{W}^{-1} \mathbf{P} \mathbf{y})}_{:= \mathbf{z}} + \frac{1}{n} \alpha^\top \mathbf{K} \mathbf{W} \mathbf{K} \alpha + 2\lambda \alpha^\top \mathbf{K} \alpha + C.$$

Completing the square in the first two terms against the vector $\mathbf{z}$ (which only changes the constant),

$$2J_q(\alpha) = \frac{1}{n} (\mathbf{K} \alpha - \mathbf{z})^\top \mathbf{W} (\mathbf{K} \alpha - \mathbf{z}) + 2\lambda \alpha^\top \mathbf{K} \alpha + C.$$

This is exactly a weighted kernel ridge regression (WKRR) problem, with regularization parameter 2λ : the Newton step for KLR is a weighted least-squares fit of the working responses $\mathbf{z}$ with weights $\mathbf{W}$, and we already know its closed-form solution from the previous section.

5.3.5 Solving KLR by iteratively reweighted least squares

In summary, one way to solve KLR is to iteratively solve a WKRR problem until convergence. Each iteration linearizes the logistic model around the current predictions and solves the resulting quadratic problem exactly. Read as an algorithm, every round does three plain things: it reads off the current scores $m_i = [\mathbf{K}\alpha]_i$, it manufactures a per-point weight W_i and a target z_i that together encode what the logistic loss would like to happen next, and it hands those to the ordinary weighted ridge solver. The weights $W_i = \sigma(m_i)\sigma(-m_i)$ are large for uncertain points near the boundary and small for confidently classified ones, so each step pays most attention exactly where the decision is still in doubt.

KLR by kernelized IRLS

Input: Gram matrix $\mathbf{K}$, labels $\mathbf{y}$, regularization λ , initial α_0

Repeat for $t = 0, 1, 2, \dots$ until convergence:

current predictions at the training points

for $i = 1..n$:

$m_i \leftarrow [\mathbf{K} \alpha_t]_i$

$P_i^t \leftarrow l'(y_i m_i) = -\sigma(-y_i m_i)$ # gradient weights

$W_i^t \leftarrow l''(y_i m_i) = \sigma(m_i) \sigma(-m_i)$ # curvature weights

```

z_i^t <- m_i - P_i^t y_i / W_i^t = m_i + y_i / sigma(y_i m_i)
# Newton step = one weighted kernel ridge regression
alpha_{t+1} <- solveWKRR(K, W^t, z^t) with regularization 2 lambda
= (W^t)^{1/2} ((W^t)^{1/2} K (W^t)^{1/2} + 2 n lambda I)^{-1} (W^t)^{1/2} z^t
Output: f(x) = sum_i alpha_i K(x_i, x)

```

The simplification of the working response uses the technical facts: with $u_i = y_i m_i$,

$$z_i = m_i - \frac{P_i y_i}{W_i} = m_i + \frac{\sigma(-u_i) y_i}{\sigma(u_i) \sigma(-u_i)} = m_i + \frac{y_i}{\sigma(y_i m_i)},$$

where we used $y_i^2 = 1$ and $\sigma(m_i) \sigma(-m_i) = \sigma(u_i) \sigma(-u_i)$. Intuitively, z_i shifts the current prediction m_i in the direction of the label y_i , by a large amount when the model currently assigns low probability to the correct label and by roughly y_i when it is already confident.

Two concrete points make the update tangible. Suppose a point has true label $y_i = 1$ but the current score is $m_i = -1$, so the model is confidently wrong: here $\sigma(y_i m_i) = \sigma(-1) \approx 0.27$, so the working response is $z_i = -1 + 1/0.27 \approx 2.7$, a target well above the current score and on the correct side, while the curvature weight $W_i = \sigma(-1) \sigma(1) \approx 0.20$ is sizeable. Contrast a confidently correct point with $y_i = 1$ and $m_i = 3$: now $z_i = 3 + 1/\sigma(3) \approx 4.05$, essentially its own score nudged up by about one, and the weight $W_i = \sigma(3) \sigma(-3) \approx 0.045$ is tiny, so the next weighted fit barely bothers with it. The Newton step therefore rebuilds the regression targets so that a single ordinary weighted ridge solve pulls hardest precisely on the points the current model gets wrong, and all but ignores those it already handles well.

Remark

This is the kernelized version of the famous iteratively reweighted least-squares (IRLS) method used to solve standard linear logistic regression. As a Newton method on a smooth convex objective it converges quadratically near the optimum, and in practice a handful of WKRR solves suffice. The structural lesson generalizes: for any smooth convex loss, the Newton step of the kernelized problem is a weighted kernel ridge regression with loss-dependent weights and working responses, so a KRR solver is the basic computational primitive for the whole family of smooth-loss kernel methods.

5.4 Large-margin classifiers: a first look

5.4.1 Losses as functions of the margin

We have now met two loss functions for binary classification:

- the 0/1 loss, $\ell_{0/1}(f(x), y) = \mathbf{1}(yf(x) < 0)$;
- the logistic loss, $\ell_{\text{logistic}}(f(x), y) = \ln(1 + e^{-yf(x)})$.

In both cases the loss depends on $f(x)$ and y only through their product. This quantity deserves a name.

Definition (margin)

In binary classification ($\mathcal{Y} = \{-1, 1\}$), the margin of the function f for a pair (x, y) is

$$y f(x).$$

The margin is positive exactly when the prediction is correct, and its magnitude measures the confidence of the prediction. In both examples above the loss is a decreasing function of the margin:

$$\ell(f(x), y) = \varphi(yf(x)), \quad \text{with } \varphi \text{ non-increasing.}$$

Correct confident predictions are cheap; wrong confident predictions are expensive. This immediately suggests a whole family of methods obtained by varying φ . The classical choices are:

Method	$\varphi(u)$
Kernel logistic regression	$\ln(1 + e^{-u})$
Support vector machine (1-SVM)	$\max(1 - u, 0)$
Support vector machine (2-SVM)	$\max(1 - u, 0)^2$
Boosting	e^{-u}

Reading the table as four ways to be gentle on confident correct predictions and harsh on confident mistakes is the quickest way to see what unites these methods, and reading the differences tells you how each one behaves. All four are non-increasing in the margin $u = yf(x)$, so all four reward a large positive margin. Where they differ is in how they treat the two ends of the axis.

- **Logistic loss**, $\ln(1 + e^{-u})$: smooth everywhere, strictly positive even for large u , and asymptotically linear in $-u$ on the wrong side. It never quite reaches zero, so every point keeps nudging the fit, which is why the solution is not sparse.
- **Hinge loss** (1-SVM), $\max(1 - u, 0)$: exactly zero once the margin exceeds 1, then linear below it. The flat region is what makes the SVM solution depend only on the points near the boundary, the support vectors; the kink at $u = 1$ is also what makes the loss non-differentiable and forces the dual treatment of the next chapter.
- **Squared hinge** (2-SVM), $\max(1 - u, 0)^2$: same flat region beyond the margin, but the penalty on violations grows quadratically and, being smooth at the kink, it can be handled by the same Newton-type machinery as the logistic loss.
- **Exponential loss** (boosting), e^{-u} : the most aggressive of the four, punishing wrong-side margins exponentially. That severity makes boosting sensitive to mislabeled points, which is the flip side of its fast early progress.

Plugging in two margin values turns the comparison into numbers. Take a point on the wrong side by one unit, $u = -1$. The logistic loss charges $\ln(1 + e) \approx 1.31$, the hinge loss charges $\max(1 - (-1), 0) = 2$, the squared hinge charges $2^2 = 4$, and the exponential charges $e^1 \approx 2.72$: the two smooth-tailed losses and especially the squared hinge escalate fast on misclassified points. Now take a comfortably correct margin $u = 2$: the logistic loss still charges $\ln(1 + e^{-2}) \approx 0.13$ and the exponential still charges $e^{-2} \approx 0.14$, while both hinge losses charge exactly 0. That last contrast is the whole story: the hinge losses switch off completely once a point is safely beyond the margin, which is what leaves the SVM solution supported on only the boundary points, whereas the logistic and exponential losses never fully release a point, which keeps their solutions dense and, for the exponential loss, easily swayed by outliers.

Remark (which margin losses are likelihoods)

The reading of a loss as a negative log-likelihood, used earlier to attach a noise model to each regression loss, also discriminates among these classification losses. The logistic loss is by construction the negative log-likelihood of the sigmoid model, so its fitted score turns into a calibrated class probability $\sigma(f(x))$. Schölkopf and Smola (2002) point out that the hinge loss enjoys no such interpretation: there is no conditional distribution over $\{-1, 1\}$ whose negative log-likelihood equals $\max(1 - yf(x), 0)$, since a proper model would need the two label probabilities to sum to one for every value of $f(x)$, which the hinge loss cannot enforce. The support vector machine therefore buys sparsity and a geometric margin at the price of direct probabilistic output, whereas kernel logistic regression keeps the probabilities but gives up sparsity. The loss-versus-noise-model correspondence is what makes this trade-off precise.

Definition (large-margin classifier)

Given a non-increasing function $\varphi : \mathbb{R} \rightarrow \mathbb{R}_+$, a (kernel) large-margin classifier is an algorithm that estimates a function $f : \mathcal{X} \rightarrow \mathbb{R}$ by solving

$$\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \varphi(y_i f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2.$$

Hence KLR is a large-margin classifier, corresponding to $\varphi(u) = \ln(1 + e^{-u})$, and many more are possible. Two questions naturally arise:

1. Can we solve the optimization problem for other φ 's, in particular the non-differentiable hinge loss of the SVM?
2. Is it a good idea to optimize this objective at all, if at the end of the day we are interested in the 0/1 loss, that is, in learning models that make few errors?

The first question is algorithmic and occupies the next chapter, via convex duality and the support vector machine. The second is statistical, and we set up its vocabulary now.

5.4.2 Solving large-margin classifiers in general

The optimization side, at least, follows a by now familiar pattern. The objective

$$\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \varphi(y_i f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2$$

depends on f only through its values at the training points and its RKHS norm, so by the representer theorem the solution of the unconstrained problem can be expanded as

$$f(x) = \sum_{i=1}^n \alpha_i K(x_i, x).$$

Plugging this expansion into the original problem, we obtain the following unconstrained optimization problem in $\mathbb{R}^n$:

$$\min_{\alpha \in \mathbb{R}^n} \left\{ \frac{1}{n} \sum_{i=1}^n \varphi(y_i [\mathbf{K}\alpha]_i) + \lambda \alpha^\top \mathbf{K} \alpha \right\}.$$

When φ is convex, this is a convex problem (each summand is a convex function composed with a linear map, and the penalty is a positive semidefinite quadratic form), and it can be solved using general tools for convex optimization, or by specific algorithms exploiting the structure of the loss; for the SVM we will see a dedicated dual algorithm later. The smooth cases, as we saw with the logistic loss, can be handled by Newton-type methods built on weighted kernel ridge regression.

5.4.3 A tiny bit of learning theory: risk, Bayes risk, and the φ -risk

To make the second question precise, we need a probabilistic model of where the data comes from and a notion of the best achievable performance.

Assumptions and notations. Let P be an (unknown) distribution on $\mathcal{X} \times \mathcal{Y}$, and let

$$\eta(x) = P(Y = 1 \mid X = x)$$

be a measurable version of the conditional distribution of Y given X . Assume the training points $S_n = (X_i, Y_i)_{i=1, \dots, n}$ are i.i.d. random variables distributed according to P .

Definition (risk, Bayes risk, empirical risk)

The risk of a classifier $f : \mathcal{X} \rightarrow \mathbb{R}$ is its probability of error,

$$R(f) = P(\text{sign}(f(X)) \neq Y).$$

The Bayes risk is the smallest achievable risk over all measurable classifiers,

$$R^* = \inf_{f \text{ measurable}} R(f).$$

The empirical risk of f is its error rate on the sample,

$$R_n(f) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(\text{sign}(f(X_i)) \neq Y_i).$$

The Bayes risk is attained by the classifier

$$f^*(x) = \eta(x) - \frac{1}{2},$$

which predicts the label 1 exactly on the region where $\eta(x) > 1/2$, that is, wherever the label 1 is conditionally more probable than the label -1 . No classifier, however clever, can beat f^* , because on each x the best possible decision is to bet on the more probable label; the residual error comes from the intrinsic randomness of Y given X . This is the standard starting point of statistical classification theory (Devroye, Györfi, and Lugosi, 1996).

Large-margin classifiers, however, do not minimize the empirical version of $R(f)$; they minimize a surrogate built from φ . Let the *empirical φ -risk* be the quantity a large-margin classifier actually optimizes (before regularization),

$$R_n^\varphi(f) = \frac{1}{n} \sum_{i=1}^n \varphi(Y_i f(X_i)),$$

which is the empirical version of the φ -risk

$$R_\varphi(f) = \mathbb{E}[\varphi(Yf(X))].$$

The question left hanging, and answered in the next chapter's development of learning theory for large-margin classifiers, is the following: can we hope to have a small risk $R(f)$ if we focus instead on the φ -risk $R_\varphi(f)$? In other words, does driving the surrogate risk toward its minimum drive the true probability of error toward the Bayes risk R^* ? For the convex margin losses in the table above the answer will turn out to be yes: the surrogate losses are what the literature calls classification-calibrated, so driving the φ -risk to its minimum does drive the

true risk to the Bayes risk (Zhang, 2004; Bartlett, Jordan, and McAuliffe, 2006). This is what ultimately justifies the practical program of this chapter: replace the intractable 0/1 objective by a convex margin loss, solve the resulting finite-dimensional convex problem through the representer theorem, and, for smooth losses, do it with nothing more exotic than a sequence of ridge regressions.

Further reading

The regularized-risk viewpoint of this chapter, with the RKHS norm as the penalty, is developed at length by Schölkopf and Smola (2002), *Learning with Kernels*, whose chapter 4 treats regularization and the link between penalties, priors, and smoothness, and whose regression material covers kernel ridge regression and its variants in detail. Shawe-Taylor and Cristianini (2004), *Kernel Methods for Pattern Analysis*, give a complementary pattern-analysis account: their chapter 7 builds least-squares and ridge regression directly from the Gram matrix and places them among the other kernel pattern-analysis algorithms. Both are the standard textbook treatments of the material here, and each carries the large-margin and logistic-loss threads forward into the support vector machines of the next chapter.

5.5 Common mistakes and practical implications

For **Kernel Ridge Regression and Smooth Losses**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

5.6 Summary and further reading

This chapter established explain the central definitions and claims in Kernel Ridge Regression and Smooth Losses; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Schölkopf and Smola 2002), (Shawe-Taylor and Cristianini 2004), (Kimeldorf and Wahba 1971).

5.7 Exercises

1. warm-up **Ridge as a spectral filter**. The KRR predictions at the training points are $\mathbf{K}\alpha = \mathbf{K}(\mathbf{K} + \lambda n\mathbf{I})^{-1}\mathbf{y}$. Diagonalize $\mathbf{K} = \sum_k \mu_k u_k u_k^\top$ in an orthonormal eigenbasis, with eigenvalues $\mu_k \geq 0$. (a) Show that the component of the fitted training values along u_k is the component of $\mathbf{y}$ along u_k multiplied by the factor $\mu_k/(\mu_k + \lambda n)$. (b) What are the limits of this factor as $\lambda \rightarrow 0^+$ and as $\lambda \rightarrow \infty$? (c) For fixed λ , explain in one sentence why the factor is close to 1 for large μ_k and close to 0 for small μ_k , and connect this to the claim that ridge regression keeps the smooth, well-supported directions and damps the noisy ones.
2. computation **A ridge fit by hand, two ways**. Take $\mathcal{X} = \mathbb{R}$, the linear kernel $K(x, x') = xx'$, and three points $x_1 = 1, x_2 = 2, x_3 = 3$ with outputs $\mathbf{y} = (1, 2, 2)^\top$. Fix the regularization so that $\lambda n = 7$. (a) Using the primal ridge form $\mathbf{w}_{\text{RR}} = (\mathbf{X}^\top \mathbf{X} + \lambda n\mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$ with the 3×1 design matrix $\mathbf{X} = (1, 2, 3)^\top$, show that the scalar weight is $w = 11/21 \approx 0.52$, and compare it with the unregularized least-squares slope $11/14 \approx 0.79$. (b) The push-through identity says the same predictor is $\mathbf{w}_{\text{KRR}} = \mathbf{X}^\top (\mathbf{K} + \lambda n\mathbf{I})^{-1} \mathbf{y}$ with the 3×3 Gram matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$. Without inverting the 3×3 matrix, argue from the lemma why the two formulas must give the same w , and state which one you would actually compute here and why.

3. proof **Deriving the KRR closed form**. Starting from the representer-theorem expansion $\hat{f} = \sum_i \alpha_i K(x_i, \cdot)$, the KRR objective becomes

$$J(\alpha) = \frac{1}{n}(\mathbf{K}\alpha - \mathbf{y})^\top (\mathbf{K}\alpha - \mathbf{y}) + \lambda \alpha^\top \mathbf{K}\alpha.$$

(a) Show that J is convex in α , using that $\mathbf{K}$ is positive semidefinite. (b) Compute $\nabla J(\alpha)$ and show it factors as $\nabla J(\alpha) = \frac{2}{n} \mathbf{K}[(\mathbf{K} + \lambda n \mathbf{I})\alpha - \mathbf{y}]$. (c) Deduce that $\alpha = (\mathbf{K} + \lambda n \mathbf{I})^{-1} \mathbf{y}$ is a stationary point, and explain why $\mathbf{K} + \lambda n \mathbf{I}$ is invertible for $\lambda > 0$. (d) Why does part (b) show that this α need not be the only solution of $\nabla J(\alpha) = 0$, and why does the fitted function $\hat{f}$ not depend on that ambiguity? Hint

For (b) use $\nabla_\alpha (\mathbf{K}\alpha - \mathbf{y})^\top (\mathbf{K}\alpha - \mathbf{y}) = 2\mathbf{K}(\mathbf{K}\alpha - \mathbf{y})$ and $\nabla_\alpha \alpha^\top \mathbf{K}\alpha = 2\mathbf{K}\alpha$, then pull $\mathbf{K}$ out in front. For (c), every eigenvalue of $\mathbf{K} + \lambda n \mathbf{I}$ is at least $\lambda n > 0$. For (d), the left factor $\mathbf{K}$ kills anything in $\text{Ker}(\mathbf{K})$; recall $\|f - f'\|_{\mathcal{H}}^2 = \epsilon^\top \mathbf{K} \epsilon = 0$ when $\mathbf{K} \epsilon = 0$.

4. proof **The two ridge limits**. Let $\alpha(\lambda) = (\mathbf{K} + \lambda n \mathbf{I})^{-1} \mathbf{y}$ and $\hat{f}_\lambda = \sum_i \alpha_i(\lambda) K(x_i, \cdot)$. (a) Show that $\|\hat{f}_\lambda\|_{\mathcal{H}} \rightarrow 0$ as $\lambda \rightarrow \infty$, so the fit collapses to the zero function and the model underfits. (b) Assume now that $\mathbf{K}$ is invertible. Show that as $\lambda \rightarrow 0^+$ the coefficients converge to $\alpha \rightarrow \mathbf{K}^{-1} \mathbf{y}$, and hence that the fitted training values $\mathbf{K}\alpha \rightarrow \mathbf{y}$: the limit interpolates the data exactly. (c) Relate both limits to the bias-variance description of the λ sweep given in the RBF worked example. Hint

Work in the eigenbasis of $\mathbf{K}$. Along the eigenvector with eigenvalue μ_k , the coefficient of $\mathbf{y}$ is scaled by $1/(\mu_k + \lambda n)$ in $\alpha(\lambda)$, and by $\mu_k/(\mu_k + \lambda n)$ in $\mathbf{K}\alpha(\lambda)$. Take the two limits componentwise. For (a), note $\|\hat{f}_\lambda\|_{\mathcal{H}}^2 = \alpha^\top \mathbf{K} \alpha = \sum_k \mu_k (u_k^\top \mathbf{y})^2 / (\mu_k + \lambda n)^2$.

5. computation **Losses as noise models, and the Huber loss**. Recall that identifying a regression loss $\ell(r)$ in the residual $r = y - f(x)$ with $-\ln p(r)$ turns a loss into a noise density $p(r) \propto e^{-\ell(r)}$. (a) Verify that the squared loss $\ell(r) = r^2/(2\sigma^2)$ corresponds to a Gaussian noise density and the absolute loss $\ell(r) = |r|$ to a Laplacian one. (b) The Huber loss with threshold $\delta > 0$ is

$$h_\delta(r) = \begin{cases} \frac{1}{2}r^2, & \text{amp; } |r| \leq \delta, \\ \delta(|r| - \frac{1}{2}\delta), & \text{amp; } |r| > \delta. \end{cases}$$

Show that h_δ is continuous and continuously differentiable at $|r| = \delta$ (compute the one-sided values and derivatives), so that it interpolates smoothly between the quadratic and absolute regimes. (c) Sketch the implied density $p(r) \propto e^{-h_\delta(r)}$ in words: Gaussian core, exponential (Laplacian) tails, and explain in one sentence why this makes the Huber fit robust to outliers where least squares is not.

6. exploration **Feeling the two knobs**. Use the interactive kernel-lab figure in the section "Solving KRR: a single linear system" (the live solver of $\alpha = (\mathbf{K} + \lambda n \mathbf{I})^{-1} \mathbf{y}$). Keep the data points fixed and vary the two controls in turn. (a) With the bandwidth σ held fixed, push λ from large to small and describe what happens to the curve: where does it start (nearly flat) and where does it end (sharp wiggles chasing individual points)? Name which regime is high bias and which is high variance. (b) Now hold λ fixed and vary σ : explain why a small σ lets the curve turn quickly while a large σ forces a smooth, slowly varying fit. (c) Find a pair (σ, λ) you would call a good fit, then show that you can partly undo an increase in λ by decreasing σ (or vice versa). Write two or three sentences on how the two knobs together control the smoothness of the fitted function.
7. proof **One Newton step of KLR is a weighted ridge regression**. For kernel logistic regression the finite-dimensional objective is $J(\alpha) = \frac{1}{n} \sum_i \ell_{\text{logistic}}(y_i [\mathbf{K}\alpha]_i) + \lambda \alpha^\top \mathbf{K}\alpha$, with $\ell_{\text{logistic}}(u) = \ln(1 + e^{-u})$. Write $m_i = [\mathbf{K}\alpha^0]_i$, $P_i = \ell'_{\text{logistic}}(y_i m_i) = -\sigma(-y_i m_i)$, $W_i = \ell''_{\text{logistic}}(y_i m_i) = \sigma(m_i)\sigma(-m_i)$, and set $\mathbf{P} = \text{diag}(P_i)$, $\mathbf{W} = \text{diag}(W_i)$. (a) Derive $\nabla J(\alpha) = \frac{1}{n} \mathbf{K} \mathbf{P} \mathbf{y} + 2\lambda \mathbf{K} \alpha$ and $\nabla^2 J(\alpha) = \frac{1}{n} \mathbf{K} \mathbf{W} \mathbf{K} + 2\lambda \mathbf{K}$ (use $y_i^2 = 1$). (b) Substitute these into the quadratic model J_q around α^0 and, by completing the square, show that minimizing J_q is exactly the weighted KRR problem

$$\min_{\alpha} \frac{1}{n} (\mathbf{K}\alpha - \mathbf{z})^\top \mathbf{W} (\mathbf{K}\alpha - \mathbf{z}) + 2\lambda \alpha^\top \mathbf{K}\alpha, \quad \mathbf{z} = \mathbf{K}\alpha^0 - \mathbf{W}^{-1} \mathbf{P} \mathbf{y}.$$

c Using the technical facts, simplify the working response to $z_i = m_i + y_i/\sigma(y_i m_i)$, and interpret it: in which direction, and by how much, does z_i move the current score when the model is confidently wrong versus confidently right? Hint

For (b), keep only the α -dependent terms of $J_q(\alpha) = J(\alpha^0) + (\alpha - \alpha^0)^\top \nabla J(\alpha^0) + \frac{1}{2}(\alpha - \alpha^0)^\top \nabla^2 J(\alpha^0)(\alpha - \alpha^0)$; the cross terms $2\lambda\alpha^\top \mathbf{K}\alpha^0$ cancel, leaving a form you can complete the square against $\mathbf{z}$. For (c), $P_i/W_i = -\sigma(-u_i)/[\sigma(u_i)\sigma(-u_i)] = -1/\sigma(u_i)$ with $u_i = y_i m_i$, and $y_i^2 = 1$.

8. challenge **Flat loss regions, sparsity, and what is not a likelihood.** (a) Suppose a convex regression loss ℓ is differentiable and identically zero on an interval, as with the ε -insensitive loss $\ell(r) = \max(|r| - \varepsilon, 0)$ on $[-\varepsilon, \varepsilon]$. Argue at the level of the stationarity conditions why a training point whose residual lands strictly inside the flat region can be assigned coefficient $\alpha_i = 0$, so that it is not a support vector, and contrast this with the squared loss, whose derivative $2r$ vanishes only at $r = 0$ and which therefore yields a dense expansion. (b) Show that the hinge loss $\varphi(u) = \max(1 - u, 0)$ is not the negative log-likelihood of any conditional model of $y \in \{-1, 1\}$ given $f(x)$: if $p(y | f) = e^{-\varphi(yf)}/Z$ were such a model, the requirement $p(1 | f) + p(-1 | f) = 1$ for every f would have to hold; exhibit values of f where $e^{-\varphi(f)} + e^{-\varphi(-f)}$ is not constant, so no single normalization Z works. Conclude why kernel logistic regression yields calibrated class probabilities while the SVM does not. Hint

For (a), the point contributes a term with derivative $\ell'(r_i) = 0$ to the optimality condition, so removing it (setting $\alpha_i = 0$) leaves the conditions satisfied. For (b), evaluate $g(f) = e^{-\varphi(f)} + e^{-\varphi(-f)}$ at $f = 0$, $f = 1$, and $f = 2$: for $f \geq 1$, $\varphi(f) = 0$ and $\varphi(-f) = 1 + f$, so $g(f) = 1 + e^{-(1+f)}$ is not constant in f .

9. computation **Leave-one-out from a single fit.** A kernel ridge fit at some λ produces, from the one solve, the in-sample residuals $\mathbf{e} = (-0.0222, 0.5993, -0.1084, 0.5993, -0.0222)$ and the smoother diagonal $[\mathbf{S}_\lambda]_{ii} = (0.3624, 0.3245, 0.3246, 0.3245, 0.3624)$. (a) Without any refitting, compute the five leave-one-out residuals $e_i/(1 - [\mathbf{S}_\lambda]_{ii})$ and the score $\text{LOO}(\lambda) = \frac{1}{5} \sum_i (\cdot)^2$; you should land on 0.3205, more than double the resubstitution error $\frac{1}{5} \sum_i e_i^2 = 0.1462$. (b) For KRR the smoother satisfies $\mathbf{0} \leq \mathbf{S}_\lambda \leq \mathbf{I}$; use this to show $0 \leq [\mathbf{S}_\lambda]_{ii} < 1$, hence $1/(1 - [\mathbf{S}_\lambda]_{ii}) \geq 1$ and therefore $\text{LOO}(\lambda) \geq \frac{1}{n} \sum_i e_i^2$ always: the training error can only understate the leave-one-out error. (c) Explain in one sentence why a coordinate with $[\mathbf{S}_\lambda]_{ii}$ near 1 is a point the fit leans on, and what happens to the inflation factors as $\lambda \rightarrow 0^+$. Hint

For (a), $[\mathbf{S}_\lambda]_{ii} = 0.3245$ gives denominator 0.6755, so the peak residual 0.5993 inflates to 0.8873. For (b), $[\mathbf{S}_\lambda]_{ii} = \mathbf{u}_i^\top \mathbf{S}_\lambda \mathbf{u}_i$ for the i th coordinate vector $\mathbf{u}_i$ is a Rayleigh quotient, bounded by the extreme eigenvalues $\mu_k/(\mu_k + \lambda n) \in [0, 1)$. For (c), $\mathbf{S}_\lambda \rightarrow \mathbf{I}$ on $\text{Im}(\mathbf{K})$ as $\lambda \rightarrow 0$, so the diagonals approach 1 and the denominators collapse.

10. proof **Generalized cross-validation and the degrees of freedom.** Let $\mathbf{S}_\lambda = \mathbf{K}(\mathbf{K} + \lambda n \mathbf{I})^{-1}$, and diagonalize $\mathbf{K} = \sum_k \mu_k \mathbf{u}_k \mathbf{u}_k^\top$ with eigenvalues $\mu_k \geq 0$. (a) Show that $\text{df}(\lambda) = \text{tr}(\mathbf{S}_\lambda) = \sum_k \mu_k/(\mu_k + \lambda n)$, and that it decreases monotonically from n as $\lambda \rightarrow 0^+$ (assuming $\mathbf{K} > 0$) to 0 as $\lambda \rightarrow \infty$. (b) Starting from the leave-one-out score, replace every diagonal $[\mathbf{S}_\lambda]_{ii}$ by the common value $\text{tr}(\mathbf{S}_\lambda)/n$ and show that the result is

$$\text{GCV}(\lambda) = \frac{\frac{1}{n} \|(\mathbf{I} - \mathbf{S}_\lambda)\mathbf{y}\|^2}{(1 - \text{df}(\lambda)/n)^2}.$$

c Give one reason GCV is attractive when n is large: it needs only the scalar $\text{tr}(\mathbf{S}_\lambda)$, not the n individual diagonals, and it is invariant to an orthogonal rotation of the fitting problem (Golub, Heath, and Wahba, 1979). Hint

For (a), the trace is the sum of eigenvalues, and $\mathbf{S}_\lambda$ shares the eigenvectors of $\mathbf{K}$ with eigenvalues $\mu_k/(\mu_k + \lambda n)$; each term decreases in λ . For (b), $\sum_i (y_i - \hat{f}(x_i))^2 = \|(\mathbf{I} - \mathbf{S}_\lambda)\mathbf{y}\|^2$, and the shared denominator $(1 - \text{df}(\lambda)/n)^2$ factors out of the sum.

6 Support Vector Machines

- Kernel ridge regression taught us a recipe: pick a convex loss, add a squared RKHS norm, and let the representer theorem collapse the infinite-dimensional problem to a finite-dimensional one. This chapter applies the recipe to classification with a loss built for margins, the hinge loss, and obtains the support vector machine. The hinge loss is not smooth, so the optimization deserves more care than in the ridge case: we reformulate the problem as a quadratic program, derive its Lagrangian dual in full, and read off from the optimality conditions the single most celebrated property of the SVM, that its solution is supported on a small set of training points, the support vectors. Along the way we meet the geometric picture of margin maximization, the equivalent C -parameterization used by most software, and the 2-SVM variant obtained from the squared hinge loss.
-

6.1 From margins to the hinge loss

Consider binary classification: training points $x_1, \dots, x_n \in \mathcal{X}$ with labels $y_1, \dots, y_n \in \{-1, +1\}$, and a real-valued function $f : \mathcal{X} \rightarrow \mathbb{R}$ whose sign will be the predicted label. The natural quantity to look at is the *margin* $y_i f(x_i)$: it is positive when f classifies x_i correctly, and its size measures how confidently. The 0-1 loss $\mathbf{1}(yf(x) \leq 0)$ only asks the margin to be positive, but it is neither convex nor continuous, so minimizing it directly is intractable. We therefore replace it by a convex surrogate that not only rewards a correct sign but keeps pushing until the margin reaches 1.

Definition (hinge loss)

The *hinge loss* is the function $\varphi_{\text{hinge}} : \mathbb{R} \rightarrow \mathbb{R}_+$ defined by

$$\varphi_{\text{hinge}}(u) = \max(1 - u, 0) = \begin{cases} 0 & \text{if } u \geq 1, \\ 1 - u & \text{otherwise.} \end{cases}$$

Plotted against the margin $u = yf(x)$, the hinge loss is a convex, piecewise-linear upper bound on the 0-1 loss: it vanishes on $[1, \infty)$ and grows linearly to the left of 1. A point pays nothing once its margin exceeds 1, pays a little when it is correctly classified but inside the margin band, and pays proportionally to how far it sits on the wrong side. This "flat then linear" shape is exactly what will later make the solution sparse.

Why the threshold 1, rather than some other positive number? It is purely a matter of scale. Rescaling f by a positive constant rescales the whole margin picture with it, so demanding a target margin of 1 while letting the regularizer decide how large f may grow is equivalent to fixing the size of f and letting the margin float. We adopt the first convention because it keeps the loss fixed and pushes all the tuning into a single regularization parameter. The geometric section below cashes out what "margin 1" means as an actual distance in space.

Definition (support vector machine)

Given a positive definite kernel K on $\mathcal{X}$ with RKHS $\mathcal{H}$, the *support vector machine* (SVM) is the large-margin classifier obtained by solving

$$\min_{f \in \mathcal{H}} \left\{ \frac{1}{n} \sum_{i=1}^n \varphi_{\text{hinge}}(y_i f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2 \right\},$$

and classifying a new point x by the sign of the minimizer $\hat{f}(x)$.

This is the same regularized empirical risk template we used for kernel ridge regression, with the squared loss traded for the hinge loss. The regularization term $\lambda \|f\|_{\mathcal{H}}^2$ controls the smoothness of f , and, as we will see in the geometric interpretation below, it is precisely what makes the SVM a *large-margin* classifier.

Historically the pieces arrived in almost the reverse of this logical order. Boser, Guyon, and Vapnik (1992) introduced the maximum-margin classifier together with the kernel trick that lets it operate in a high-dimensional feature space while touching the data only through inner products. Cortes and Vapnik (1995) added the slack variables that let the machine tolerate points which cannot be separated cleanly, producing the soft-margin SVM that we actually solve here. Vapnik (1995) set the whole construction inside statistical learning theory, which explains why maximizing a margin is a sound way to control generalization and not merely an aesthetic preference. The RKHS-and-hinge-loss presentation used in this chapter, which recasts the SVM as one instance of the regularized empirical risk recipe, is the modern synthesis expounded by Schölkopf and Smola (2002).

6.2 The primal problem

6.2.1 Reduction to finite dimension

The objective is the sum of an empirical term depending on f only through the values $f(x_1), \dots, f(x_n)$ and a strictly increasing function of $\|f\|_{\mathcal{H}}$, so the representer theorem applies: the solution lives in the span of the kernel functions at the training points,

$$\hat{f}(x) = \sum_{i=1}^n \hat{\alpha}_i K(x_i, x),$$

for some coefficient vector $\hat{\alpha} \in \mathbb{R}^n$. Writing $K \in \mathbb{R}^{n \times n}$ for the Gram matrix $K_{ij} = K(x_i, x_j)$, we have $f(x_i) = [K\alpha]_i$ and $\|f\|_{\mathcal{H}}^2 = \alpha^\top K\alpha$, so $\hat{\alpha}$ solves

$$\min_{\alpha \in \mathbb{R}^n} \left\{ \frac{1}{n} \sum_{i=1}^n \varphi_{\text{hinge}}(y_i [K\alpha]_i) + \lambda \alpha^\top K\alpha \right\}.$$

This is a convex optimization problem in n variables. Unlike the ridge case, however, the objective is not smooth: the hinge loss has a kink at $u = 1$, so we cannot simply set a gradient to zero and solve a linear system.

6.2.2 Slack variables and smooth constraints

The standard cure for a piecewise-linear objective is to move the nonsmoothness into constraints. Introduce slack variables $\xi_1, \dots, \xi_n \in \mathbb{R}$, one per training point, meant to dominate the individual losses. The problem is equivalent to

$$\min_{\alpha \in \mathbb{R}^n, \xi \in \mathbb{R}^n} \left\{ \frac{1}{n} \sum_{i=1}^n \xi_i + \lambda \alpha^\top K\alpha \right\} \quad \text{subject to} \quad \xi_i \geq \varphi_{\text{hinge}}(y_i [K\alpha]_i),$$

because at the optimum each ξ_i is pushed down onto the loss it dominates. This is the *epigraph trick*: minimizing a function is the same as minimizing a fresh variable constrained to lie above the function's graph, and since nothing in the objective rewards a large ξ_i , the minimizer squeezes each slack down until it rests exactly on the hinge. The objective is now smooth (linear plus quadratic), but the constraints still carry the hinge. Fortunately a hinge constraint splits into two linear ones: for any $u, v \in \mathbb{R}$,

$$u \geq \varphi_{\text{hinge}}(v) \iff \begin{cases} u \geq 1 - v, \\ u \geq 0. \end{cases}$$

Indeed $\varphi_{\text{hinge}}(v) = \max(1 - v, 0)$, and dominating a maximum of two quantities is the same as dominating both. Applying this with $u = \xi_i$ and $v = y_i[K\alpha]_i$ yields a fully smooth formulation.

Proposition (SVM, primal formulation)

The SVM solution is $\hat{f}(x) = \sum_{i=1}^n \hat{\alpha}_i K(x_i, x)$, where $\hat{\alpha}$ solves

$$\min_{\alpha \in \mathbb{R}^n, \xi \in \mathbb{R}^n} \frac{1}{n} \sum_{i=1}^n \xi_i + \lambda \alpha^\top K \alpha,$$

subject to

$$\begin{cases} y_i[K\alpha]_i + \xi_i - 1 \geq 0, & \text{for } i = 1, \dots, n, \\ \xi_i \geq 0, & \text{for } i = 1, \dots, n. \end{cases}$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

This is a classical *quadratic program* (QP): minimization of a convex quadratic function under linear inequality constraints. In principle any out-of-the-box optimization package can solve it. In practice the problem has $2n$ variables and $2n$ constraints, where n is the number of training points, and general-purpose QP solvers start to struggle when n exceeds a few thousand. This is our motivation for deriving the dual: the dual is also a QP, but its structure, in particular the sparsity of the final solution, leads to much faster specialized algorithms.

6.3 Lagrangian duality and the dual problem

The derivation that follows runs over four subsections, so it helps to have the whole arc in view before the algebra begins. We attach a price to every constraint and fold the constraints into the objective, which frees the primal variables α and ξ to be minimized without any restriction at all. Minimizing over α pins it to the signed prices through the relation $\alpha = \text{diag}(y) \mu / 2\lambda$; minimizing over the slacks ξ , which enter the Lagrangian only linearly, gives a finite value only when the two prices attached to each point sum to $1/n$, a bookkeeping identity that then lets us discard the multipliers v altogether. What survives is a concave function of the price vector μ alone, to be maximized over the simple box $0 \leq \mu \leq \mathbf{1}/n$. Translating that box back through the same relation $\alpha = \text{diag}(y) \mu / 2\lambda$ turns the whole apparatus into a clean quadratic program in the expansion coefficients themselves. Each subsection below carries out exactly one of these moves.

6.3.1 The Lagrangian

Duality trades the constrained primal for an unconstrained inner minimization weighted by multipliers. The multipliers are worth a word of interpretation before the algebra: think of each one as a *price* charged for violating its constraint. Once the constraints are folded into the objective at these prices, the primal variables are set free to roam, but every unit by which a point dips below its margin requirement now costs μ_i , and every unit of negative slack costs v_i . If the prices are chosen well, the freely-optimizing variables have no incentive to break any constraint, and the penalized problem returns the same value as the constrained one. Finding those best prices is the dual problem, and, as we confirm at the end of this section, its payoff is a far simpler feasible set.

6 Support Vector Machines

Introduce Lagrange multipliers $\mu \in \mathbb{R}^n$ for the margin constraints $y_i[K\alpha]_i + \xi_i - 1 \geq 0$ and $\nu \in \mathbb{R}^n$ for the positivity constraints $\xi_i \geq 0$. The Lagrangian of the problem is

$$L(\alpha, \xi, \mu, \nu) = \frac{1}{n} \sum_{i=1}^n \xi_i + \lambda \alpha^\top K \alpha - \sum_{i=1}^n \mu_i [y_i [K\alpha]_i + \xi_i - 1] - \sum_{i=1}^n \nu_i \xi_i,$$

or, in matrix notation, writing $\mathbf{1}$ for the all-ones vector and $\text{diag}(y)$ for the diagonal matrix with the labels on its diagonal,

$$L(\alpha, \xi, \mu, \nu) = \frac{\xi^\top \mathbf{1}}{n} + \lambda \alpha^\top K \alpha - (\text{diag}(y) \mu)^\top K \alpha - (\mu + \nu)^\top \xi + \mu^\top \mathbf{1}.$$

The dual function is $q(\mu, \nu) = \inf_{\alpha, \xi} L(\alpha, \xi, \mu, \nu)$, which we now compute by minimizing over α and ξ separately, since the Lagrangian decouples.

6.3.2 Minimizing over α

As a function of α , the Lagrangian is a convex quadratic (the matrix K is positive semidefinite), so it is minimized wherever its gradient vanishes:

$$\nabla_\alpha L = 2\lambda K \alpha - K \text{diag}(y) \mu = K(2\lambda \alpha - \text{diag}(y) \mu).$$

The following choice solves $\nabla_\alpha L = 0$:

$$\alpha^* = \frac{\text{diag}(y) \mu}{2\lambda}, \quad \text{that is,} \quad \alpha_i^* = \frac{y_i \mu_i}{2\lambda}.$$

(When K is singular the stationary point is not unique, but any solution of $\nabla_\alpha L = 0$ attains the same minimal value, so this choice is enough to evaluate the infimum.)

It is worth reading this relation forward rather than treating it as a bookkeeping step. It says the optimal α is nothing but the vector of signed multipliers, rescaled by 2λ . In other words the coefficients of the kernel expansion and the dual prices are the same objects up to the label y_i and a constant. Everything the optimality analysis will later say about support vectors is, at bottom, a statement about which of these prices are nonzero, so it pays to keep the identification $\alpha_i = y_i \mu_i / 2\lambda$ in view.

6.3.3 Minimizing over ξ

As a function of ξ , the Lagrangian is linear, with slope

$$\nabla_\xi L = \frac{\mathbf{1}}{n} - \mu - \nu.$$

A linear function has infimum $-\infty$ unless it is constant, so the minimum over ξ is finite exactly when

$$\mu + \nu = \frac{\mathbf{1}}{n},$$

in which case all the ξ -terms of the Lagrangian cancel.

6.3.4 The dual function

Substituting α^* into the remaining terms, the quadratic part becomes

$$\lambda \alpha^{*\top} K \alpha^* - (\text{diag}(y)\mu)^\top K \alpha^* = \frac{1}{4\lambda} \mu^\top \text{diag}(y) K \text{diag}(y) \mu - \frac{1}{2\lambda} \mu^\top \text{diag}(y) K \text{diag}(y) \mu,$$

and we obtain the Lagrange dual function

$$q(\mu, \nu) = \inf_{\alpha \in \mathbb{R}^n, \xi \in \mathbb{R}^n} L(\alpha, \xi, \mu, \nu) = \begin{cases} \mu^\top \mathbf{1} - \frac{1}{4\lambda} \mu^\top \text{diag}(y) K \text{diag}(y) \mu & \text{if } \mu + \nu = \frac{\mathbf{1}}{n}, \\ -\infty & \text{otherwise.} \end{cases}$$

The dual problem is to maximize $q(\mu, \nu)$ subject to $\mu \geq 0$ and $\nu \geq 0$ (the multipliers of inequality constraints must be nonnegative). The quadratic form appearing in q is $\mu^\top \text{diag}(y) K \text{diag}(y) \mu$, the Gram matrix reweighted by the labels; since K is positive semidefinite this form is too, so q is concave and we are maximizing a concave function over a convex set, exactly the well-posed shape a dual problem should have.

6.3.5 Eliminating ν

The variable ν only appears through the linkage $\mu + \nu = \mathbf{1}/n$, so we can eliminate it. If $\mu_i > 1/n$ for some i , then no $\nu_i \geq 0$ satisfies $\mu_i + \nu_i = 1/n$, and $q(\mu, \nu) = -\infty$ for every admissible ν ; such μ can be discarded. If instead $0 \leq \mu_i \leq 1/n$ for all i , then taking $\nu_i = 1/n - \mu_i \geq 0$ makes the dual function finite, with a value depending on μ alone. The dual problem is therefore equivalent to

$$\max_{0 \leq \mu \leq \mathbf{1}/n} \mu^\top \mathbf{1} - \frac{1}{4\lambda} \mu^\top \text{diag}(y) K \text{diag}(y) \mu,$$

or, written out with indices,

$$\max_{0 \leq \mu \leq \mathbf{1}/n} \sum_{i=1}^n \mu_i - \frac{1}{4\lambda} \sum_{i,j=1}^n y_i y_j \mu_i \mu_j K(x_i, x_j).$$

6.3.6 Back to the primal: the dual in terms of α

Once the dual is solved in μ , a primal solution is recovered through the stationarity relation $\alpha = \text{diag}(y)\mu/2\lambda$. The link is so simple, and invertible since $y_i^2 = 1$ gives $\mu_i = 2\lambda y_i \alpha_i$, that we may change variables directly in the dual and obtain the QP that α itself must solve. The linear term becomes $\mu^\top \mathbf{1} = 2\lambda \sum_i y_i \alpha_i$, the quadratic term becomes $\lambda \alpha^\top K \alpha$, and after dividing by the positive constant λ the box constraint $0 \leq \mu_i \leq 1/n$ reads $0 \leq y_i \alpha_i \leq 1/(2\lambda n)$.

Proposition (SVM, dual formulation)

The coefficient vector $\hat{\alpha}$ of the SVM solution $\hat{f} = \sum_i \hat{\alpha}_i K(x_i, \cdot)$ solves

$$\max_{\alpha \in \mathbb{R}^n} 2 \sum_{i=1}^n \alpha_i y_i - \sum_{i,j=1}^n \alpha_i \alpha_j K(x_i, x_j) = 2 \alpha^\top y - \alpha^\top K \alpha,$$

subject to

$$0 \leq y_i \alpha_i \leq \frac{1}{2\lambda n}, \quad \text{for } i = 1, \dots, n.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The primal of the preceding proposition is a convex quadratic program, and the point $\alpha = 0$, $\xi = \mathbf{1}$ satisfies every constraint (indeed $y_i[K\alpha]_i + \xi_i - 1 = 0$ and $\xi_i = 1 > 0$), so Slater's condition holds and strong duality lets us solve the dual in its place. Attach multipliers $\mu \geq 0$ to the margin constraints and $\nu \geq 0$ to the positivity constraints $\xi_i \geq 0$, giving the Lagrangian $L(\alpha, \xi, \mu, \nu) = \frac{1}{n} \xi^\top \mathbf{1} + \lambda \alpha^\top K \alpha - (\text{diag}(y)\mu)^\top K \alpha - (\mu + \nu)^\top \xi + \mu^\top \mathbf{1}$. It is convex in α and linear in ξ , so its infimum splits across the two blocks. Stationarity in α , $\nabla_\alpha L = K(2\lambda\alpha - \text{diag}(y)\mu) = 0$, is solved by $\alpha^* = \text{diag}(y)\mu/2\lambda$, and every solution attains the same value. Stationarity in ξ , $\nabla_\xi L = \mathbf{1}/n - \mu - \nu = 0$, is the only way the ξ -linear part stays bounded below, forcing $\mu + \nu = \mathbf{1}/n$; otherwise the infimum is $-\infty$. Substituting α^* collapses the two quadratic terms to a single $-\frac{1}{4\lambda} \mu^\top \text{diag}(y)K\text{diag}(y)\mu$, so the dual function is $q(\mu) = \mu^\top \mathbf{1} - \frac{1}{4\lambda} \mu^\top \text{diag}(y)K\text{diag}(y)\mu$. The requirement $\nu \geq 0$ with $\nu = \mathbf{1}/n - \mu$ reads $\mu \leq \mathbf{1}/n$, and $\mu \geq 0$ was imposed from the start, so the feasible set is the box $0 \leq \mu \leq \mathbf{1}/n$. Finally the invertible change of variables $\mu_i = 2\lambda y_i \alpha_i$ (using $y_i^2 = 1$) sends $\mu^\top \mathbf{1}$ to $2\lambda \sum_i y_i \alpha_i$, the quadratic form to $\lambda \alpha^\top K \alpha$, and the box $0 \leq \mu_i \leq 1/n$ to $0 \leq y_i \alpha_i \leq 1/(2\lambda n)$; dividing the objective by the positive constant λ leaves the stated program. $\square$

Notice what duality has bought us. The dual is a QP in only n variables (the slacks are gone), its constraints are simple box constraints rather than general linear inequalities, and, remarkably, the expansion coefficients of $\hat{f}$ coincide with the dual variables up to the fixed relabeling $\mu_i = 2\lambda y_i \alpha_i$. Strong duality holds here (the primal is a convex QP with feasible points, e.g. $\alpha = 0$, $\xi = \mathbf{1}$, satisfying the constraints), so solving the dual really does solve the SVM.

The box constraint $0 \leq y_i \alpha_i \leq 1/(2\lambda n)$ repays a moment of reading, since it is the whole geometry of the machine compressed into two inequalities per point. The lower bound $0 \leq y_i \alpha_i$ forces each coefficient to agree in sign with its label: a positive point may only pull the decision function up and a negative point may only pull it down, so every training point can lobby the boundary in favour of its own class and never against it. The upper bound $y_i \alpha_i \leq 1/(2\lambda n)$ caps how hard any single point is allowed to pull. This cap is exactly what lets the soft-margin SVM absorb an outlier instead of being dominated by it: a badly placed point may press against the bound, but once there it can press no harder, and the regularizer λ sets the ceiling. Sending $\lambda \rightarrow 0$ lifts the ceiling to infinity and recovers the hard-margin machine, in which a single unseparable point could in principle acquire unbounded influence.

6.4 Complementary slackness and the KKT conditions

The Karush-Kuhn-Tucker conditions state that at the optimum, each multiplier and its constraint cannot both be "active on the wrong side": the product of each multiplier with its constraint value vanishes. The price metaphor makes this an either/or. A multiplier is the price the optimum pays to hold a constraint down; if a constraint is slack, meaning it holds with strict inequality and room to spare, then relaxing it a little buys nothing, so its price must be zero. Read the other way, a nonzero price can attach only to a constraint that is exactly tight. Thus for each point either its margin constraint binds, with $y_i f(x_i) + \xi_i = 1$, or the price μ_i on that constraint is zero; and independently, either the slack ξ_i is zero or the price ν_i guarding $\xi_i \geq 0$ is zero. The three regimes worked out below are simply the consistent ways of setting these two either/or switches. For our two families of constraints, the complementary slackness conditions read, for $i = 1, \dots, n$,

$$\begin{cases} \mu_i [y_i f(x_i) + \xi_i - 1] = 0, \\ \nu_i \xi_i = 0. \end{cases}$$

Rewriting with $\mu_i = 2\lambda y_i \alpha_i$ and $\nu_i = 1/n - \mu_i$, and dropping harmless nonzero factors, this becomes

$$\begin{cases} \alpha_i [y_i f(x_i) + \xi_i - 1] = 0, \\ \left(\alpha_i - \frac{y_i}{2\lambda n} \right) \xi_i = 0. \end{cases}$$

6.4.1 Reading off the three regimes

Combining these two conditions with primal feasibility ($y_i f(x_i) + \xi_i \geq 1$, $\xi_i \geq 0$) and the dual box constraint $0 \leq y_i \alpha_i \leq 1/(2\lambda n)$, every training point falls into one of three regimes, indexed by where its coefficient sits in the box.

- **Interior of the flat region**, $\alpha_i = 0$. Then $\alpha_i - y_i/(2\lambda n) \neq 0$, so the second condition forces $\xi_i = 0$, and feasibility gives $y_i f(x_i) \geq 1$. The point sits on the correct side, at or beyond the margin, and contributes nothing to $\hat{f}$.
- **Strictly inside the box**, $0 < y_i \alpha_i < \frac{1}{2\lambda n}$. Both multipliers μ_i and ν_i are strictly positive, so both constraints are active: $\xi_i = 0$ and $y_i f(x_i) + \xi_i - 1 = 0$. Hence $y_i f(x_i) = 1$: the point lies exactly on the margin.
- **At the upper bound**, $\alpha_i = \frac{y_i}{2\lambda n}$. The second condition is void ($\xi_i \geq 0$ is unconstrained by it), while $\mu_i > 0$ makes the first constraint active: $y_i f(x_i) + \xi_i = 1$ with $\xi_i \geq 0$, so $y_i f(x_i) \leq 1$. The point is on the margin or inside it, possibly misclassified.

6.4.2 The same conclusions without KKT

The three regimes can also be derived from first principles, without invoking the KKT machinery, by staring at the dual as a box-constrained minimization. Changing sign, the dual is equivalent to

$$\min_{\alpha \in \mathbb{R}^n} q(\alpha) = \frac{1}{2} \alpha^\top K \alpha - \alpha^\top y \quad \text{s.t.} \quad \forall i, 0 \leq y_i \alpha_i \leq C,$$

with $C = 1/(2\lambda n)$. Its gradient has the striking form

$$\nabla q(\alpha) = K\alpha - y = [f(x_i) - y_i]_{i=1, \dots, n} :$$

the i -th partial derivative is the residual of the current function at the i -th training point. Now fix a coordinate i , assume $y_i = 1$ (the case $y_i = -1$ is symmetric), and consider the one-dimensional restriction of q to α_i over the interval $[0, C]$, a convex parabola. Locating the constrained minimizer α_i^* on this interval translates into three statements about the sign of the partial derivative at the optimum.

- **Case 1:** $0 < y_i \alpha_i^* < C$. The minimizer is interior, so the partial derivative vanishes, $[\nabla q(\alpha^*)]_i = 0$, i.e. $f(x_i) = y_i$, hence $y_i f(x_i) = 1$.
- **Case 2:** $y_i \alpha_i^* = C$. The minimizer is pushed against the upper bound, which can only happen if the parabola is still decreasing there: $[\nabla q(\alpha^*)]_i \leq 0$, i.e. $f(x_i) \leq y_i$, hence $y_i f(x_i) \leq 1$.
- **Case 3:** $\alpha_i^* = 0$. The minimizer is pushed against the lower bound, so the parabola is nondecreasing there: $[\nabla q(\alpha^*)]_i \geq 0$, i.e. $f(x_i) \geq y_i$, hence $y_i f(x_i) \geq 1$.

This is exactly the KKT trichotomy again, obtained from nothing more than the geometry of minimizing a parabola over an interval.

6.5 Geometric interpretation

A picture of a two-class point cloud makes this analysis concrete. Draw the level sets $f(x) = 0$ (the decision boundary) and $f(x) = +1, f(x) = -1$ (the edges of the *margin band*). The three regimes of the previous section stratify the training set geometrically:

- points with $\alpha_i = 0$ lie strictly outside the margin band, on the correct side ($y_i f(x_i) \geq 1$);
- points with $0 < y_i \alpha_i < \frac{1}{2\lambda n}$ lie exactly on the edge of the band, $y_i f(x_i) = 1$;
- points with $y_i \alpha_i = \frac{1}{2\lambda n}$ lie on or inside the band, or on the wrong side of the boundary altogether ($y_i f(x_i) \leq 1$).

The picture also explains the name "large-margin classifier". Consider the linear kernel on $\mathbb{R}^d$, so that $f(x) = \langle w, x \rangle$ with $\|f\|_{\mathcal{H}} = \|w\|$. The two hyperplanes $f(x) = +1$ and $f(x) = -1$ are parallel, at Euclidean distance $2/\|w\|$ from each other. To see where that distance comes from, walk from the hyperplane $\langle w, x \rangle = -1$ toward $\langle w, x \rangle = +1$ along the unit normal direction $w/\|w\|$. Along that walk the value $\langle w, x \rangle$ increases at rate $\|w\|$ per unit of length, since a step of length t in direction $w/\|w\|$ raises the inner product by $t \langle w, w/\|w\| \rangle = t \|w\|$. Covering the gap of 2 in the value of f therefore takes a walk of length $2/\|w\|$, which is the width of the band. Points incur no hinge loss precisely when they lie beyond their side of this band. Minimizing the hinge loss plus $\lambda \|w\|^2$ therefore trades off two desiderata: keep the training points out of the band (small loss) while making the band as wide as possible (small $\|w\|$, i.e. large geometric margin $2/\|w\|$). In an RKHS the same picture holds with f in place of w : the regularizer $\|f\|_{\mathcal{H}}^2$ is the inverse squared margin measured in the geometry of $\mathcal{H}$.

Schölkopf and Smola (2002) organize this scale bookkeeping around a single definition, worth importing here because it explains the target margin of 1 we simply postulated at the start. Give a labelled point its signed distance to the boundary.

Definition (geometric margin)

For a hyperplane $\{x : \langle w, x \rangle + b = 0\}$, the *geometric margin* of a labelled point (x, y) is

$$\frac{y(\langle w, x \rangle + b)}{\|w\|},$$

the Euclidean distance from x to the boundary, signed to be positive exactly when x is correctly classified. The geometric margin of a training sample is the minimum of this quantity over its points.

A hyperplane is unchanged when w and b are scaled by a common positive constant, so the pair (w, b) carries one unit of redundant freedom, a "gauge" in the physicists' phrase. Schölkopf and Smola remove it by calling the pair *canonical* with respect to the sample when $\min_i |\langle w, x_i \rangle + b| = 1$, that is, when the point closest to the boundary sits at output value exactly 1. For a canonical hyperplane the geometric margin of the closest point is precisely $1/\|w\|$, so the full band between $f = +1$ and $f = -1$ has width $2/\|w\|$, matching the walk computed above. Fixing the target margin at 1 and letting $\|w\|$ float, our convention since the definition of the hinge loss, is therefore not a loss-function quirk but the choice of a canonical representative, and it is exactly what turns $\|w\|$ itself into a geometric quantity, the inverse margin.

Proposition (the margin is $1/\|w\|$)

Let $\{x : \langle w, x \rangle + b = 0\}$ be a hyperplane, canonical with respect to the sample, that is $\min_i |\langle w, x_i \rangle + b| = 1$. Then the training point nearest the boundary lies at Euclidean distance $1/\|w\|$ from it, and the two margin hyperplanes $\langle w, x \rangle + b = \pm 1$ are parallel at distance $2/\|w\|$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The Euclidean distance from a point x_0 to the hyperplane $\langle w, x \rangle + b = 0$ is $|\langle w, x_0 \rangle + b|/\|w\|$. To see this, decompose $x_0 = x_{\perp} + t w/\|w\|$ with $x_{\perp}$ on the hyperplane, so that $\langle w, x_0 \rangle + b = \langle w, x_{\perp} \rangle + b + t \langle w, w \rangle/\|w\| = t \|w\|$, whence

the signed distance to the boundary is $t = (\langle w, x_0 \rangle + b)/\|w\|$. Minimizing its absolute value over the sample and using canonicity gives the nearest distance $\min_i |\langle w, x_i \rangle + b|/\|w\| = 1/\|w\|$. For the band, start from any point on $\langle w, x \rangle + b = -1$ and step along the unit normal $w/\|w\|$: a step of length s raises $\langle w, x \rangle + b$ by $s \langle w, w/\|w\| \rangle = s \|w\|$, so covering the rise of 2 from -1 to $+1$ takes $s = 2/\|w\|$, the width of the band. It is widest exactly when $\|w\| = \|f\|_{\mathcal{H}}$ is smallest, which is why the regularizer $\lambda \|f\|_{\mathcal{H}}^2$ is a drive toward a large margin. $\square$

The reason a wide band is worth wanting is robustness. A point that sits a full margin width from the boundary can be perturbed by any displacement smaller than that width without crossing to the wrong side, so a large geometric margin is a quantitative promise that small changes in the input, or the arrival of a nearby unseen test point, will not flip the prediction. This is the geometric shadow of the statistical fact, placed by Vapnik (1995) inside learning theory and developed in full in Chapter 11, Learning Theory in RKHS Balls, that the margin controls generalization: the objective is not merely fitting the training labels but fitting them with room to spare.

6.5.1 Why a wide margin generalizes

The robustness argument is the geometric shadow of a genuine generalization guarantee. Chapter 11, Learning Theory in RKHS Balls develops such guarantees in full; it is worth previewing one bound here so that "the margin controls generalization" acquires content beyond intuition. Schölkopf and Smola (2002) record the following, whose striking feature is that it never mentions the dimension of the feature space.

Theorem (margin error bound, Schölkopf and Smola 2002)

Fix $\rho > 0$ and consider linear classifiers $x \mapsto \text{sgn}\langle w, x \rangle$ with $\|w\| \leq \Lambda$, applied to data with $\|x\| \leq R$. Let the *margin error* be the fraction of training points whose margin $y_i \langle w, x_i \rangle$ falls below ρ . Then for every data distribution, with probability at least $1 - \delta$ over a sample of size n , the probability that a fresh point is misclassified is at most

$$(\text{margin error}) + \sqrt{\frac{c}{n} \left(\frac{R^2 \Lambda^2}{\rho^2} \ln^2 n + \ln \frac{1}{\delta} \right)},$$

for a universal constant c .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Read the two terms together. The first is the training performance graded at confidence level ρ ; the second is a complexity penalty that shrinks as the margin ρ grows relative to the data radius R and the weight norm Λ . A machine that separates the sample with a wide margin pays a small penalty even though the bound has said nothing about how many parameters it has, which is precisely why controlling $\|w\|$, and hence the inverse margin, is a legitimate stand-in for controlling dimension. The bound also explains the otherwise mysterious restriction $\|x\| \leq R$: only the ratio R/ρ of radius to margin appears, so a margin is meaningful only once the scale of the inputs is pinned.

Shawe-Taylor and Cristianini (2004) carry the same style of analysis through to the soft-margin machine we actually solve, where a few points are allowed to violate the margin. Writing γ for the achieved geometric margin and ξ for the vector of margin slacks $\xi_i = (\gamma - y_i f(x_i))_+$, their bound for the box-constrained (or "1-norm soft margin") SVM has the schematic form

$$\frac{\|\xi\|_1}{n\gamma} + \frac{4}{n\gamma} \sqrt{\text{tr}(K)} + 3\sqrt{\frac{\ln(2/\delta)}{2n}},$$

with K the Gram matrix. The first term is the total margin violation per unit margin, the empirical price of the outliers the machine chose to tolerate; the second is again a margin-scaled complexity term, now driven by $\text{tr}(K) = \sum_i K(x_i, x_i)$ in place of R^2 . Minimizing the soft-margin objective is, in this light, a direct attempt to keep the sum of these terms small: shrink the slacks while holding the margin wide.

6.5.2 A worked two-point example

A two-point example makes every one of these quantities concrete.

Example (two points, solved by hand)

Take the linear kernel $K(x, x') = \langle x, x' \rangle$ on $\mathbb{R}^2$ and just two training points, $x_1 = (1, 0)$ with $y_1 = +1$ and $x_2 = (-1, 0)$ with $y_2 = -1$. The classifier is $f(x) = \alpha_1 \langle x_1, x \rangle + \alpha_2 \langle x_2, x \rangle = \langle w, x \rangle$ with weight vector $w = \alpha_1 x_1 + \alpha_2 x_2 = (\alpha_1 - \alpha_2, 0)$. By symmetry the decision boundary must be the vertical axis, and demanding the margin condition $y_i f(x_i) = 1$ at both points gives $w = (1, 0)$, hence $\alpha_1 - \alpha_2 = 1$. The sign constraint $0 \leq y_i \alpha_i$ asks for $\alpha_1 \geq 0$ and $\alpha_2 \leq 0$, and the symmetric choice $\alpha_1 = \frac{1}{2}$, $\alpha_2 = -\frac{1}{2}$ meets every requirement. Both coefficients are nonzero, so both points are support vectors, each sitting exactly on its own margin edge $f = +1$ and $f = -1$; the geometric margin is $2/\|w\| = 2$, precisely the distance between the two points. As long as $\lambda \leq \frac{1}{2}$, so that the shared value $y_i \alpha_i = \frac{1}{2}$ stays below the cap $1/(2\lambda n) = 1/(4\lambda)$, neither coefficient is pinned to the upper bound, and both points fall in the strictly-interior regime of the KKT trichotomy, exactly on the margin.

Verification artifact. checks/example-ch05-example-5-1.json records the example source hash and verification scope.

The two-point example kept its boundary through the origin. Adding an offset b and a point the machine will ignore turns the support-vector bookkeeping into numbers. This is the classic maximum-margin machine $f(x) = \langle w, x \rangle + b$ of Boser, Guyon, and Vapnik (1992), the hard-margin limit $\lambda \rightarrow 0$ of the dual; its only change from the offset-free form of the previous sections is the extra equality constraint $\sum_i \alpha_i y_i = 0$, the footprint of optimizing over b .

Example (hard-margin SVM on four points)

Linear kernel $K(x, x') = \langle x, x' \rangle$ on $\mathbb{R}^2$; four points $x_1 = (0, 0)$, $x_2 = (4, 0)$ with $y_1 = y_2 = -1$, and $x_3 = (1, 2)$, $x_4 = (1, 6)$ with $y_3 = y_4 = +1$. The hard-margin dual is

$$\max_{\alpha \geq 0, \sum_i \alpha_i y_i = 0} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle, \quad w = \sum_i \alpha_i y_i x_i.$$

1. Build the reweighted Gram matrix. The inner products are $\langle x_2, x_2 \rangle = 16$, $\langle x_2, x_3 \rangle = \langle x_2, x_4 \rangle = 4$, $\langle x_3, x_3 \rangle = 5$, $\langle x_3, x_4 \rangle = 13$, $\langle x_4, x_4 \rangle = 37$, and $x_1 = 0$ kills its whole row. The matrix $Q_{ij} = y_i y_j \langle x_i, x_j \rangle$ that enters the quadratic form flips the signs of the mixed-class entries.
2. Solve the dual. The maximizer is $\alpha = (\frac{3}{8}, \frac{1}{8}, \frac{1}{2}, 0)$. It respects the equality constraint, $\sum_i \alpha_i y_i = -\frac{3}{8} - \frac{1}{8} + \frac{1}{2} + 0 = 0$, and every $\alpha_i \geq 0$.
3. Recover the weight vector. $w = \frac{3}{8}(-1)(0, 0) + \frac{1}{8}(-1)(4, 0) + \frac{1}{2}(+1)(1, 2) + 0 = (-\frac{1}{2} + \frac{1}{2}, 1) = (0, 1)$, so $\|w\| = 1$.
4. Recover the offset. Any support vector lies exactly on its margin, $y_k(\langle w, x_k \rangle + b) = 1$. Using x_3 : $\langle w, x_3 \rangle = 2$, so $2 + b = 1$ gives $b = -1$; x_1 and x_2 give the same b .
5. Read the margins. With $f(x) = \langle w, x \rangle + b = x^{(2)} - 1$ (the second coordinate minus one), $f(x_1, \dots, x_4) = (-1, -1, 1, 5)$, hence $y_i f(x_i) = (1, 1, 1, 5)$. The geometric margin is $1/\|w\| = 1$ and the band width is $2/\|w\| = 2$.
6. Sort the points. The three points x_1, x_2, x_3 have $\alpha_i \neq 0$ and $y_i f(x_i) = 1$: support vectors, one from each corner, sitting on the margin. The far point x_4 has $\alpha_4 = 0$ and $y_4 f(x_4) = 5 > 1$: it lies well outside the band and drops out of the model, the $\alpha_i = 0$ regime of the trichotomy.

Reading. The machine remembers only the three points pinned to the margin; the comfortably classified x_4 is discarded. The unequal coefficients $\frac{3}{8}$ and $\frac{1}{8}$ on the two negative base points show them pulling with different force because the positive apex x_3 sits off-center, closer to x_1 , while the sum rule $\sum_i \alpha_i y_i = 0$ keeps the two classes' total pull in balance.

Verification artifact. checks/example-ch05-example-5-2.json records the example source hash and verification scope.

Nothing conveys this trade-off as directly as moving the points yourself and watching the boundary respond.

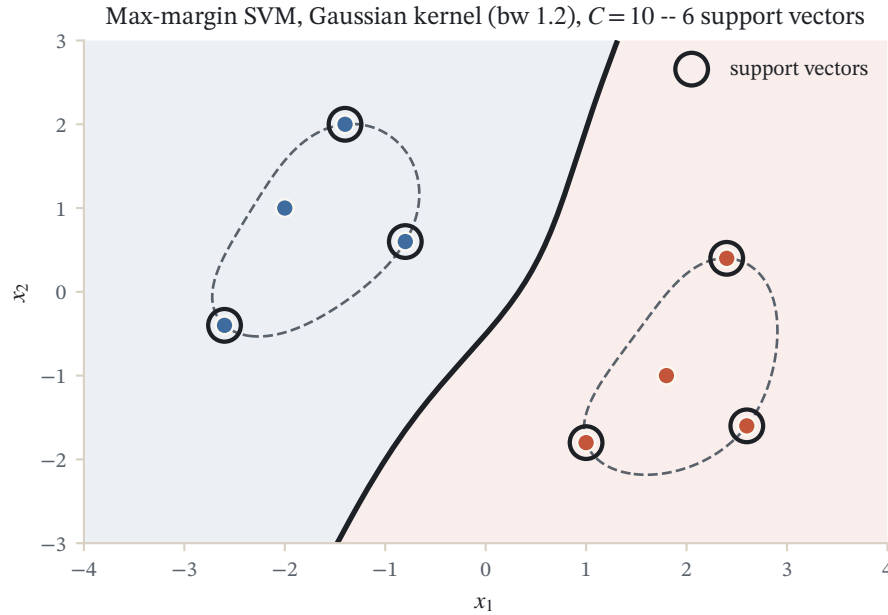


Figure 6.1: The maximum-margin decision boundary (solid) and its ± 1 margins for two classes under a Gaussian kernel. Circled points are the support vectors, the only ones with $\alpha_i > 0$.

6.6 Support vectors

Definition (support vectors)

The training points with $\alpha_i \neq 0$ are called the *support vectors* of the SVM.

Proposition (the solution depends only on the support vectors)

Every training point with $y_i f(x_i) > 1$ has $\alpha_i = 0$. Consequently the support vectors all satisfy $y_i f(x_i) \leq 1$, lying on or inside the margin band, and $f = \sum_{i \in \text{SV}} \alpha_i K(x_i, \cdot)$ is a combination of the support vectors alone.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Complementary slackness on the margin constraint reads $\alpha_i [y_i f(x_i) + \xi_i - 1] = 0$. Suppose $y_i f(x_i) > 1$. Primal feasibility gives $\xi_i \geq 0$, so the bracket obeys $y_i f(x_i) + \xi_i - 1 \geq y_i f(x_i) - 1 > 0$ and is strictly positive; for the product to vanish the other factor must, forcing $\alpha_i = 0$. Reading this contrapositively, any support vector, a point with $\alpha_i \neq 0$, satisfies $y_i f(x_i) \leq 1$. Since the zero-coefficient points contribute nothing to $f(x) = \sum_i \alpha_i K(x_i, x)$, the sum collapses onto $\text{SV} = \{i : \alpha_i \neq 0\}$. $\square$

Complementary slackness gives the cleanest reading of why this set is the one that matters. The first condition, $\alpha_i [y_i f(x_i) + \xi_i - 1] = 0$, is a switch: either the price α_i is zero, or the bracket is zero and the margin constraint binds. A support vector is therefore, by definition, a point whose margin constraint is exactly tight, a point the machine is actively pressing against; a nonzero coefficient is precisely the force of that press. Points the machine feels no need to press on pay the zero price the switch allows them and drop out of the model entirely. Sparsity is thus not an accident of the solver but a direct consequence of the optimality conditions: the only points that can carry weight are the ones sitting on or inside the margin.

The KKT analysis shows that support vectors are exactly the points that touch or violate the margin: those on the margin edge (interior coefficients) and those inside or beyond it (coefficients at the bound). All the comfortably

classified points, the ones the parabola argument parks at $\alpha_i = 0$, simply disappear from the model. Only support vectors matter for classifying new points:

$$\forall x \in \mathcal{X}, \quad f(x) = \sum_{i=1}^n \alpha_i K(x_i, x) = \sum_{i \in \text{SV}} \alpha_i K(x_i, x),$$

where SV denotes the set of support vectors. Two practical consequences follow.

- **Fast training.** The solution is sparse in α . Decomposition methods exploit this by optimizing over small working sets of coordinates at a time, which is what makes SVM solvers scale far beyond what a generic $2n$ -dimensional QP solver on the primal could handle. The best known such method is sequential minimal optimization (Platt 1998), which shrinks the working set all the way down to two coordinates at a time and solves each tiny subproblem in closed form; it is developed in full in Chapter 9, Solving the SVM: Decomposition and SMO.
- **Fast prediction.** Classifying a new point requires kernel evaluations against the support vectors only, not against the whole training set.

Note where the sparsity comes from: the flat part of the hinge loss. A loss that is strictly curved everywhere, like the squared loss of ridge regression, gives every training point a nonzero coefficient; the hinge loss lets well-classified points drop out exactly.

6.7 Practical remarks and variants

6.7.1 The C -SVM parameterization

Remark (C-SVM)

The SVM optimization problem is often written in terms of a regularization parameter C instead of λ , as

$$\arg \min_{f \in \mathcal{H}} \frac{1}{2} \|f\|_{\mathcal{H}}^2 + C \sum_{i=1}^n L_{\text{hinge}}(f(x_i), y_i).$$

This is equivalent to our formulation with

$$C = \frac{1}{2n\lambda}.$$

The dual problem then reads

$$\max_{\alpha \in \mathbb{R}^n} 2 \sum_{i=1}^n \alpha_i y_i - \sum_{i,j=1}^n \alpha_i \alpha_j K(x_i, x_j), \quad \text{subject to} \quad 0 \leq y_i \alpha_i \leq C, \quad i = 1, \dots, n.$$

This formulation is often called the C -SVM; it is the convention used by most software packages. Large C means weak regularization (a hard, narrow margin), small C means strong regularization (a soft, wide margin).

As a procedure the C -SVM is a single box-constrained quadratic program followed by a reading of the support set. Stated as pseudocode it makes explicit where the specialized solver of the next chapter plugs in.

Algorithm (C-SVM dual)

Input Gram matrix $K \in \mathbb{R}^{n \times n}$ with $K_{ij} = K(x_i, x_j)$; labels $y \in \{-1, +1\}^n$; penalty $C = 1/(2n\lambda) > 0$.

Output coefficients $\alpha \in \mathbb{R}^n$ of $\hat{f} = \sum_i \alpha_i K(x_i, \cdot)$ and the support set SV.

1. Change variables to $\beta_i = y_i \alpha_i$ and form the label-reweighted matrix $Q = \text{diag}(y) K \text{diag}(y)$, so that $Q_{ij} = y_i y_j K(x_i, x_j)$.
2. Solve the box-constrained quadratic program $\max_{\beta \in \mathbb{R}^n} 2 \mathbf{1}^\top \beta - \beta^\top Q \beta$ subject to $0 \leq \beta_i \leq C$, which is the dual of the remark above rewritten in β .
3. Do not hand this to a generic QP solver: exploit the anticipated sparsity by decomposing over small working sets, in the limit two coordinates at a time in closed form (SMO); this is the subject of Chapter 9, Solving the SVM: Decomposition and SMO.
4. Recover $\alpha_i = y_i \beta_i$ and set $SV = \{i : \alpha_i \neq 0\}$; the complementary-slackness trichotomy labels each index as off-margin ($\beta_i = 0$), on-margin ($0 < \beta_i < C$), or capped ($\beta_i = C$).
5. Classify a new point x by $\text{sgn } \hat{f}(x) = \text{sgn } \sum_{i \in SV} \alpha_i K(x_i, x)$, touching the support vectors only.

6.7.2 The ν -SVM

Both C and λ share an awkward property: they are raw prices with no natural scale, so a value that regularizes one dataset well can be far off for another, and there is no a priori way to choose one. Schölkopf, Smola, Williamson, and Bartlett (2000) proposed a reparameterization, the ν -SVM, that trades the price for a parameter $\nu \in (0, 1]$ with a directly interpretable meaning. Instead of a fixed target margin of 1, the ν -SVM promotes the margin to a variable $\rho \geq 0$ to be maximized, and charges for points that fall short of it:

$$\min_{f \in \mathcal{H}, \rho \geq 0, \xi \geq 0} \frac{1}{2} \|f\|_{\mathcal{H}}^2 - \nu \rho + \frac{1}{n} \sum_{i=1}^n \xi_i \quad \text{subject to} \quad y_i f(x_i) \geq \rho - \xi_i.$$

Here ρ locates the margin (the band is now $f = \pm \rho$ rather than $f = \pm 1$), the reward $-\nu \rho$ presses to widen it, and the slacks ξ_i absorb the points that cannot keep up. Carrying the same Lagrangian steps through, the dual is once more a box-constrained quadratic program, now with two extra scalar constraints:

$$\max_{\mu \in \mathbb{R}^n} -\frac{1}{2} \sum_{i,j=1}^n \mu_i \mu_j y_i y_j K(x_i, x_j) \quad \text{subject to} \quad 0 \leq \mu_i \leq \frac{1}{n}, \quad \sum_i \mu_i y_i = 0, \quad \sum_i \mu_i \geq \nu.$$

The box $0 \leq \mu_i \leq 1/n$ is exactly the one we met in the dual of the ordinary SVM (the prices μ of the margin constraints); what is new is that the linear term $\mu^\top \mathbf{1}$ has left the objective and reappeared as the constraint $\sum_i \mu_i \geq \nu$, while the equality $\sum_i \mu_i y_i = 0$ is the footprint of the offset b that the ν -formulation carries.

Proposition (the ν -property, Schölkopf, Smola, Williamson, and Bartlett 2000)

At a solution of the ν -SVM with $\rho > 0$:

- ν is an upper bound on the fraction of *margin errors*, the points with $\xi_i > 0$ (those inside the band or misclassified);
- ν is a lower bound on the fraction of support vectors;
- under mild non-degeneracy of the data-generating distribution, both fractions converge to ν as $n \rightarrow \infty$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The first two claims are a short counting argument on the constraints, and they are worth seeing because they pin the connection between the top of the box and the outliers that the C -SVM only hinted at. At the optimum the constraint $\sum_i \mu_i \geq \nu$ is tight whenever $\rho > 0$, so $\sum_i \mu_i = \nu$. A margin error has its price pinned to the ceiling, $\mu_i = 1/n$, so if n_{err} points are margin errors then $n_{\text{err}}/n \leq \sum_i \mu_i = \nu$; a support vector merely has $\mu_i > 0$, so $\nu = \sum_i \mu_i \leq n_{\text{SV}}/n$. The user therefore sets, before training, a ceiling on the tolerated margin errors and a floor on the number of support vectors, both far more legible than a bare value of C . The same reading sharpens the earlier remark that the box cap controls robustness: the points pinned at the top of the box are exactly the margin errors, and ν is the dial that bounds how many of them there may be.

6.7.3 The 2-SVM

Remark (2-SVM)

A variant of the SVM, sometimes called the 2-SVM, is obtained by replacing the hinge loss by the *squared* hinge loss:

$$\min_{f \in \mathcal{H}} \left\{ \frac{1}{n} \sum_{i=1}^n \varphi_{\text{hinge}}(y_i f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2 \right\}.$$

After some computation (following the same slack-variable and Lagrangian steps as above, and left as an exercise below), the dual problem of the 2-SVM turns out to be

$$\max_{\alpha \in \mathbb{R}^n} 2 \alpha^\top y - \alpha^\top (K + n\lambda I) \alpha, \quad \text{subject to } 0 \leq y_i \alpha_i, \quad i = 1, \dots, n.$$

This is exactly the dual of the ordinary SVM run with the modified kernel $K + n\lambda I$ and $C = +\infty$: squaring the hinge removes the upper box constraint and shifts the kernel by a ridge on the diagonal. The 2-SVM thus stays inside the same algorithmic family, at the cost of losing some sparsity, since the quadratic growth of the loss softens the flat region that produced exact zeros.

6.8 Multiclass SVM

Everything so far is resolutely binary: the machine outputs a sign, and a sign can tell exactly two classes apart. Most classification problems that matter come with more, ten digits, dozens of phonemes, thousands of topics, so let the labels now range over $y_i \in \{1, \dots, M\}$ with $M > 2$. Two roads lead from the binary machine to a multiclass one. The first keeps the binary SVM as a black box and *reduces* the M -class problem to a collection of two-class problems, in one of two standard ways. The second rebuilds the margin idea natively for M classes and trains one joint machine with a single coupled quadratic program. Both roads end at the same kind of decision rule, a family of per-class score functions $f_1, \dots, f_M$ compared by $\arg \max_m f_m(x)$; they differ in what is optimized on the way there.

6.8.1 One-vs-all

The *one-vs-all* (or one-versus-rest) reduction trains M binary machines. Machine m relabels the sample: class m becomes +1 and the other $M - 1$ classes together become -1; running the binary SVM of this chapter on the relabeled data produces a score function f_m . A new point is assigned to the class whose machine claims it most confidently,

$$\hat{y}(x) = \arg \max_{m \in \{1, \dots, M\}} f_m(x).$$

The maximum, and not the individual signs, is essential. Nothing prevents two machines from both reporting a positive score on the same x , or all M of them from reporting a negative one, so the signs alone can be ambiguous; the maximum always names exactly one class, up to exact ties.

Two frictions are built into this reduction. First, the decision rule compares raw margins produced by M separately trained optimization problems, and nothing in the training aligned their scales: a margin of 0.7 from machine 3 and a margin of 0.5 from machine 8 are measured in different units. Second, each binary problem opposes one class to $M - 1$, so the relabeled data are imbalanced by construction. Neither friction is fatal. Rifkin and Klautau (2004) argue, against a decade of more elaborate proposals, that when each binary machine is properly regularized and tuned, one-vs-all is as accurate as any of its competitors, and it has the merit of M independent, embarrassingly parallel trainings, each over all n points.

6.8.2 One-vs-one

The *one-vs-one* (or all-pairs) reduction trains one binary machine per unordered pair of classes, $M(M - 1)/2$ machines in all, and machine $\{a, b\}$ sees only the training points of classes a and b . At prediction time every machine casts one vote, for whichever of its two classes its sign favors, and the class with the most votes wins. More machines, but far smaller ones: each pairwise QP involves only the two classes' points, roughly $2n/M$ of them under balanced classes, and since SVM training cost grows faster than linearly in the sample size, many small duals are typically cheaper than a few big ones. This is the reduction implemented as the default in the standard SVM software libraries, following the empirical comparison of Hsu and Lin (2002), who found the accuracies of all the multiclass constructions of this section to be close on standard benchmarks, and the pairwise reduction the most practical to train.

Algorithm (one-vs-one training and max-wins voting)

Input training sample $(x_i, y_i)_{i=1}^n$ with $y_i \in \{1, \dots, M\}$; kernel K ; penalty C .

Output a predicted class $\hat{y}(x)$ for any query x .

1. For every unordered pair of classes $\{a, b\}$ with $a < b$, repeat step 2.
2. Extract the training points of classes a and b only, relabel class a as $+1$ and class b as -1 , and train a binary C -SVM on them, obtaining the decision function f_{ab} .
3. At a query x , collect one vote per machine: machine $\{a, b\}$ votes for class a if $f_{ab}(x) > 0$ and for class b otherwise.
4. Predict the class with the most votes; break any remaining tie by a fixed convention, for instance the head-to-head result between two tied leaders.

The vote is worth watching once in complete detail, because it exhibits the one structural oddity of the reduction: machines are forced to vote on points from classes they have never seen.

Example (a three-class one-vs-one vote)

Linear kernel on $\mathbb{R}^2$, three classes of two points each: class 1 at $(0, 0)$ and $(0, 2)$, class 2 at $(6, 0)$ and $(6, 2)$, class 3 at $(3, 5)$ and $(3, 7)$. Query point $x^* = (1, 1)$. Train the three pairwise hard-margin machines (the box cap inactive) in the offset-carrying canonical form $f(x) = \langle w, x \rangle + b$ of the four-point example, the lower-indexed class relabeled $+1$.

1. Train machine 1v2. Its four points are mirror-image clusters at $x^{(1)} = 0$ and $x^{(1)} = 6$, so the widest band is vertical: $w = (-\frac{1}{3}, 0)$, $b = 1$, giving $f_{12}(x) = 1 - x^{(1)}/3$, boundary at $x^{(1)} = 3$, and all four points sitting on the margins.
2. Train machine 1v3. The closest opposite pair is $(0, 2)$ and $(3, 5)$, and the machine bisects it: $w = (-\frac{1}{3}, -\frac{1}{3})$, $b = \frac{5}{3}$. Check the margins: $f_{13}(0, 2) = +1$ and $f_{13}(3, 5) = -1$, the two support vectors.
3. Train machine 2v3. Symmetrically, $w = (\frac{1}{3}, -\frac{1}{3})$, $b = -\frac{1}{3}$, with support vectors $(6, 2)$ at margin $+1$ and $(3, 5)$ at -1 .
4. Vote at the query. $f_{12}(x^*) = 1 - \frac{1}{3} = +\frac{2}{3}$, a vote for class 1; $f_{13}(x^*) = -\frac{2}{3} + \frac{5}{3} = +1$, a vote for class 1, the query sitting exactly on this machine's margin edge; $f_{23}(x^*) = 0 - \frac{1}{3} = -\frac{1}{3}$, a vote for class 3.
5. Tally. Class 1 collects two votes, class 2 none, class 3 one: predict class 1.

Reading. The 2v3 machine has never seen a class-1 point, yet it must vote, and it votes for class 3, the nearer of the two classes it knows. Max-wins absorbs such bystander votes: class 1 swept the two contests it took part in, so no rival can reach its count of $M - 1 = 2$. Note also how small the bystander's score is, $f_{23}(x^*) = -\frac{1}{3}$: the query lies deep inside that machine's margin band, and the vote is cast without conviction.

Verification artifact. checks/example-ch05-example-5-3.json records the example source hash and verification scope.

6.8.3 Single-machine formulations

Both reductions optimize proxies: no pairwise or one-vs-rest training run ever sees the quantity the final decision rule actually uses. The natural multiclass margin of a labeled point under the rule $\arg \max_m f_m$ is

$$\gamma_i = f_{y_i}(x_i) - \max_{m \neq y_i} f_m(x_i),$$

positive exactly when the point is classified correctly, and measuring by how much its own class outscores the best impostor. A single-machine multiclass SVM regularizes all M functions jointly and presses all these margins up in one quadratic program. The two classical formulations differ in how they charge a violation.

Weston and Watkins (1999) charge every offending class separately. Writing the linear case with per-class weight vectors and offsets, $f_m(x) = \langle w_m, x \rangle + b_m$, their primal is

$$\min_{\{w_m, b_m\}, \xi} \quad \frac{1}{2} \sum_{m=1}^M \|w_m\|^2 + C \sum_{i=1}^n \sum_{m \neq y_i} \xi_i^m$$

subject to

$$\langle w_{y_i}, x_i \rangle + b_{y_i} \geq \langle w_m, x_i \rangle + b_m + 2 - \xi_i^m, \quad \xi_i^m \geq 0, \quad \text{for all } i \text{ and } m \neq y_i.$$

Each training point carries $M - 1$ slacks, one per class it must outscore, and pays their sum. The target gap of 2 is Weston and Watkins's normalization: at $M = 2$ the constraints see only the difference $w_1 - w_2$ while the regularizer also charges the sum, so the optimum sets $w_2 = -w_1$; the gap-of-2 constraint then reads $y_i f_1(x_i) \geq 1 - \xi_i/2$, and the formulation reduces to the binary machine of this chapter. As with the binary margin of 1, any positive constant would do, absorbed by rescaling.

Crammer and Singer (2001) charge only the worst offender. Their machine carries a single slack per training point and no offsets:

$$\min_{\{w_m\}, \xi} \quad \frac{1}{2} \sum_{m=1}^M \|w_m\|^2 + C \sum_{i=1}^n \xi_i$$

subject to

$$\langle w_{y_i}, x_i \rangle - \langle w_m, x_i \rangle \geq 1 - \delta_{y_i, m} - \xi_i \quad \text{for all } i \text{ and all } m,$$

where $\delta_{y_i, m}$ is the Kronecker delta; the constraint for $m = y_i$ just reads $\xi_i \geq 0$. At the optimum each slack rests on the hinge of the multiclass margin,

$$\xi_i = \max \left(0, 1 + \max_{m \neq y_i} \langle w_m, x_i \rangle - \langle w_{y_i}, x_i \rangle \right),$$

the exact multiclass analogue of φ_{hinge} . The payoff of the single-slack design is structural: the dual splits into one small simplex-constrained block per training point, and Crammer and Singer exploit this with a decomposition method that sweeps the data updating one block at a time, the multiclass cousin of the working-set methods of Chapter 9, Solving the SVM: Decomposition and SMO. Both formulations kernelize as before, each f_m expanding over the training points so that data enter only through K ; both classify by $\arg \max_m f_m(x)$; and both charge at least a unit whenever the decision rule errs, so each training objective dominates the multiclass 0-1 loss.

method	problems solved	problem size	decision rule
one-vs-all	M binary QPs	all n points each	$\arg \max_m f_m(x)$, raw scores compared across machines
one-vs-one	$M(M-1)/2$ binary QPs	the two classes' points only, about $2n/M$	max-wins vote of the pairwise signs
Weston-Watkins	one joint QP	$n(M-1)$ slacks, all classes coupled	$\arg \max_m f_m(x)$
Crammer-Singer	one joint QP	nM dual variables, one simplex per point	$\arg \max_m f_m(x)$

The comparison of Hsu and Lin (2002) is the standard field guide to this table: on benchmark after benchmark the four constructions land within a whisker of one another in accuracy, while the pairwise reduction trains fastest and the joint machines, for all their conceptual cleanliness, cost the most. The lasting argument for the single-machine view is not speed but honesty of formulation, one optimization problem whose objective is the thing the decision rule cares about. One loose end remains: the one-vs-all rule compares raw margins across independently trained machines, a comparison of quantities with no common scale. The next section builds the currency conversion it quietly assumed, a map from margins to probabilities.

6.9 Probabilistic outputs

The SVM answers "which side" with a sign and "how confidently" with a margin, measured in units of its own band. Many consumers of a classifier need a third answer, $P(y = +1 | x)$: a medical screen whose two errors cost differently, a pipeline that defers to a human below a confidence floor, a one-vs-all rule that wants the M scores on one scale. The margin is not that probability, and the failure is structural, not cosmetic. Fix a point x and write $\eta = P(y = +1 | x)$; the population hinge risk at x , as a function of the predicted value f , is

$$\eta \varphi_{\text{hinge}}(f) + (1 - \eta) \varphi_{\text{hinge}}(-f) = \eta \max(1 - f, 0) + (1 - \eta) \max(1 + f, 0),$$

a piecewise-linear function whose middle segment, on $(-1, 1)$, has slope $1 - 2\eta$. For $\eta > \frac{1}{2}$ the slope is negative and the minimum sits at $f = +1$; for $\eta < \frac{1}{2}$, at $f = -1$. The population-optimal SVM output is the bare sign of $2\eta - 1$: the hinge loss discards exactly the quantity a probability would report, how far η is from $\frac{1}{2}$. If probabilities are wanted, they must be grafted on after training, by a second, separately fitted stage.

Definition (Platt scaling, Platt 2000)

Let f be a trained SVM decision function and $(f_i, y_i)_{i=1}^N$, with $f_i = f(x_i)$, the scores and labels of a *calibration set* disjoint from the training set, containing N_+ positives and N_- negatives. *Platt scaling* posits the sigmoid model

$$P(y = +1 | f) = \frac{1}{1 + \exp(Af + B)},$$

and fits the two parameters (A, B) by maximum likelihood on the calibration set: with the smoothed targets

$$t_i = \begin{cases} \frac{N_+ + 1}{N_+ + 2} & \text{if } y_i = +1, \\ \frac{1}{N_- + 2} & \text{if } y_i = -1, \end{cases}$$

minimize the cross-entropy $-\sum_i [t_i \ln p_i + (1 - t_i) \ln(1 - p_i)]$ over (A, B) , where $p_i = 1/(1 + \exp(Af_i + B))$.

This is logistic regression with a single input, the score itself, so the fit is a two-parameter convex problem; Newton's method with its 2×2 Hessian settles it in a handful of iterations, and a sensible fit has $A < 0$, making the probability increase with the margin. Platt (2000) motivates the sigmoid shape empirically: across SVMs on real data, the class-conditional densities of the scores between the margins are close to exponential, and a sigmoid posterior is exactly what a pair of exponential tails produces. The two departures from textbook logistic regression are both deliberate. The targets t_i are not the raw labels 0 and 1 but shrunk versions of them, the posterior means of the label frequencies under a uniform prior (an exercise derives them); this keeps the fitted probabilities away from hard 0 and 1 and stabilizes small calibration sets. And the fit must not reuse the training scores: every on-margin support vector sits at exactly $y_i f(x_i) = 1$, so the training-set margin distribution is a spike at ± 1 plus violations, a distortion no fresh test point will reproduce. Platt recommends a held-out calibration set or cross-validated scores.

Example (Platt scaling of eight held-out scores)

A trained SVM assigns eight calibration points the scores f_i below, with labels y_i : $N_+ = 4$ positives and $N_- = 4$ negatives. The table's last two rows are filled in by the steps.

f_i	-2.0	-1.2	-0.4	0.1	0.5	1.0	1.8	2.4
y_i	-1	-1	-1	+1	-1	+1	+1	+1
t_i	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{5}{6}$	$\frac{1}{6}$	$\frac{5}{6}$	$\frac{5}{6}$	$\frac{5}{6}$
p_i	0.1111	0.2056	0.3489	0.4578	0.5486	0.6569	0.7986	0.8725

1. Form the targets. With $N_+ = N_- = 4$, $t_+ = (4 + 1)/(4 + 2) = \frac{5}{6} \approx 0.8333$ for the positives and $t_- = 1/(4 + 2) = \frac{1}{6} \approx 0.1667$ for the negatives: the second row of the table.
2. Fit the sigmoid by Newton. The cross-entropy has gradient $(\sum_i (t_i - p_i)f_i, \sum_i (t_i - p_i))$ and Hessian built from the weights $p_i(1 - p_i)$. Starting from $(A, B) = (0, 0)$, six Newton steps reach machine precision: $A = -0.909732$, $B = 0.260042$, with cross-entropy 4.408633 at the optimum.
3. Read off the probabilities. The last row of the table is $p_i = 1/(1 + \exp(Af_i + B))$: the extreme scores earn 0.1111 and 0.8725, the scores near the boundary hover near $\frac{1}{2}$.
4. Locate the probability threshold. $p = \frac{1}{2}$ where $Af + B = 0$, that is at $f_0 = -B/A = 0.285845$, not at $f = 0$. The rule "predict +1 when $p \geq \frac{1}{2}$ " disagrees with $\text{sgn } f$ on every score in the window $(0, 0.285845)$.

Reading. The sigmoid is monotone, so it can reshape confidence but never reorder scores. The calibration data contain a misordered pair, a negative at $f = 0.5$ scored above a positive at $f = 0.1$ (the shaded cells), and after calibration the pair is still misordered, $0.5486 > 0.4578$; the fit has split the difference by parking both near $\frac{1}{2}$ and sliding the $p = \frac{1}{2}$ crossing to 0.2858. Calibration converts margins into usable probabilities; it cannot repair the classifier underneath.

Verification artifact. checks/example-ch05-example-5-4.json records the example source hash and verification scope.

6.9.1 What calibration can and cannot promise

The worked example already displayed the two caveats that matter most. First, the map is monotone by construction, so Platt scaling changes no ranking: a score ordering that misplaces a pair leaves a probability ordering that misplaces the same pair, and any quantity computed from the ranking alone is untouched by calibration. Second, the probability threshold moves: $p \geq \frac{1}{2}$ corresponds to $f \geq -B/A$, which is zero only by accident, so the calibrated classifier and the raw sign can disagree on a window of borderline scores. This is not a malfunction; the calibration set testified that scores just above zero were more often negative than positive, and the fit listened. A user who needs the original decision boundary preserved should keep $\text{sgn } f$ for decisions and use p only as a confidence report.

Two further cautions. The sigmoid is a two-parameter parametric family, a modeling choice, not a theorem: when the true score-to-frequency relation is not sigmoidal, the fit inherits the mismatch, and its quality should be checked like any other model's, for instance on a reliability plot of predicted against empirical frequencies. And the innocuous-looking objective repays careful implementation: the naive expression for the cross-entropy loses precision catastrophically for large $|Af + B|$, and Lin, Lin, and Weng (2007) give the standard reformulation and a robust Newton variant, along with evidence that the sigmoid fit is well behaved across a wide range of problems. For probabilities that emerge from the model itself rather than from an appended stage, the Bayesian route through Gaussian processes is developed in Chapter 40, Gaussian Processes and the RVM.

6.10 Model selection

Two numbers have ridden along unchosen through the whole chapter: the regularization level, C or equivalently λ , and whatever parameters the kernel itself carries, above all a bandwidth σ for the Gaussian kernel. The theory says what the machine does at fixed dials and is silent about setting them, yet everything visible in practice depends on the setting: too small a bandwidth makes every training point its own island, all of them support vectors of a memorized boundary, while too large a one flattens the geometry until the machine is nearly linear; and C sets the price ceiling that decides how much margin violation is tolerated. The margin bound of the geometric section names the quantity at stake, a radius-to-margin ratio, and this section makes the choice of dials operational: first by the general-purpose tool, cross-validation, then by bounds special to the SVM that price a whole leave-one-out run from a single training.

6.10.1 Cross-validation

The k -fold protocol is the workhorse. Split the sample into k folds of equal size; for each candidate pair (C, σ) and each fold, train on the other $k - 1$ folds and record the error on the held-out fold; average the k error rates; pick the candidate with the smallest average and retrain on all the data at that setting. Candidates are laid out on logarithmic grids, since both dials act multiplicatively, and a coarse grid refined around its winner costs far less than a fine grid everywhere. Each surrogate machine sees only $(1 - 1/k)n$ points, so the estimate is slightly pessimistic about the final machine trained on all n ; the common choices $k = 5$ or 10 keep that bias small at k trainings per candidate. The extreme $k = n$ is the *leave-one-out* (LOO) estimate: train n times, each time on $n - 1$ points, and test on the point left out. Its virtue is statistical, the LOO error is an almost unbiased estimate of the expected error of a machine trained on $n - 1$ points, an observation going back to Luntz and Brailovsky recalled by Chapelle, Vapnik, Bousquet, and Mukherjee (2002); its vice is arithmetic, n retrainings per candidate. And every grid suffers the same disease: its size is exponential in the number of dials, fine for two, hopeless for the per-feature bandwidths we will meet in a moment.

6.10.2 Leave-one-out bounds from a single training

The SVM's structure rescues the LOO estimate from its cost: properties of one trained machine bound the outcome of all n retrainings. The first such certificate is the sparsity itself.

Proposition (the support-vector count bounds the LOO error)

Fix C and run the C -SVM. Every leave-one-out round that leaves out a point with $\alpha_i = 0$ classifies that point correctly, so the number of LOO errors is at most the number of support vectors:

$$\#\{\text{LOO errors}\} \leq \#\text{SV}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Work in the dual of the C-SVM remark, maximizing $G(\alpha) = 2\alpha^\top y - \alpha^\top K\alpha$ over the box $0 \leq y_j \alpha_j \leq C$, and let $\hat{\alpha}$ be a maximizer with $\hat{\alpha}_i = 0$, so that x_i is not a support vector and, by the KKT trichotomy, $y_i \hat{f}(x_i) \geq 1$. The leave-one-out round for x_i solves the same program over the remaining coordinates, with row and column i of K deleted. The restriction of $\hat{\alpha}$ is feasible for it, and it is optimal: any feasible α' of the reduced problem extends, by inserting $\alpha_i = 0$, to a feasible point of the full problem with the same objective value, every term added by the extension carrying the factor $\alpha_i = 0$; if α' beat the restriction of $\hat{\alpha}$, its extension would beat $\hat{\alpha}$. The reduced problem can have several maximizers, but they all define the same function: if α^1, α^2 are two of them, concavity forces G to be constant on the segment between them, so the quadratic term along the segment vanishes, $\delta^\top K \delta = 0$ for $\delta = \alpha^2 - \alpha^1$; and then $g = \sum_j \delta_j K(x_j, \cdot)$ has $\|g\|_{\mathcal{H}}^2 = \delta^\top K \delta = 0$, so $g = 0$ as a function. The machine retrained without x_i therefore has the same decision function $\hat{f}$, and it classifies the left-out point correctly, since $y_i \hat{f}(x_i) \geq 1 > 0$. Every LOO error thus occurs at a support vector. $\square$

Vapnik (1995) turns the count into a generalization statement: taking expectations over samples, the expected test error of the machine trained on $n - 1$ points is at most $E[\#SV]/n$. Sparsity is thus not only a computational grace but a statistical one. As a selection criterion, though, the count is blunt: it sees how many points matter, not how precariously they are placed, and on small samples it saturates, as the worked example below shows. The second certificate reads the geometry instead.

Theorem (radius-margin bound, Vapnik 1998)

Train the hard-margin SVM on a separable sample, let $\gamma = 1/\|w\|$ be its margin, and let R be the radius of the smallest ball of $\mathcal{H}$ containing the points $K(x_i, \cdot)$. Then

$$\#\{\text{LOO errors}\} \leq \frac{R^2}{\gamma^2} = R^2 \|w\|^2.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Variants of the statement move a small constant factor around, depending on whether the machine carries an offset; the invariant content is the ratio. The proof couples two facts. A leave-one-out error at x_p forces that point's coefficient to be large, of order $1/R^2$: the remaining machine must be re-bent across a ball of radius R to misplace it. And the hard-margin coefficients altogether obey $\sum_p \alpha_p = \|w\|^2$: multiply the on-margin identity $y_p f(x_p) = 1$ by α_p , sum over support vectors, and the double sum collapses to $\|w\|^2$ plus an offset term killed by $\sum_p \alpha_p y_p = 0$. A budget of $\|w\|^2$ split into pieces of size at least $1/R^2$ buys at most $R^2 \|w\|^2$ pieces. The radius has its own dual QP over the Gram matrix, the *minimum enclosing ball* problem

$$R^2 = \max_{\beta \geq 0, \sum_i \beta_i = 1} \sum_{i=1}^n \beta_i K(x_i, x_i) - \sum_{i,j=1}^n \beta_i \beta_j K(x_i, x_j),$$

with center $\sum_i \beta_i K(x_i, \cdot)$: a box-and-simplex QP of the same family as the SVM itself, and the same ball, worn as a data description rather than a certificate, returns in Chapter 7, One-Class SVMs and Novelty Detection. Two properties make $R^2 \|w\|^2$ a good selection score. It is dimensionless and scale-invariant, rescaling the data or the kernel moves R^2 and $\|w\|^2$ in exactly cancelling ways, so kernels at different scales are compared fairly. And it is never smaller than 1 (an exercise), so its value reads as a multiple of the best achievable geometry. The bound is stated for the hard-margin machine, but the soft-margin case folds in through a trick this chapter already owns: the 2-SVM remark showed the squared-hinge machine to be a hard-margin machine for the shifted kernel $K + n\lambda I$, and it is in exactly this form that the criterion is applied with a regularization dial.

Between the blunt count and the coarse ratio sits the sharpest of the one-training certificates, which asks of each support vector how replaceable it is.

Definition (span of a support vector, Vapnik and Chapelle 2000)

Let a hard-margin SVM have support vectors $\{x_p\}_{p \in \text{SV}}$ with coefficients $\alpha_p > 0$ in the classical dual (the four-point example's parameterization). For $p \in \text{SV}$, let

$$\Lambda_p = \left\{ \sum_{j \in \text{SV}, j \neq p} \lambda_j K(x_j, \cdot) : \sum_{j \neq p} \lambda_j = 1, \alpha_j + y_j y_p \alpha_p \lambda_j \geq 0 \text{ for all } j \neq p \right\},$$

the set of reallocations of x_p 's load onto the other support vectors that keep every coefficient admissible. The *span* of x_p is the distance $S_p = d(K(x_p, \cdot), \Lambda_p)$ in $\mathcal{H}$.

Theorem (span bound, Vapnik and Chapelle 2000)

Run the leave-one-out procedure on a hard-margin SVM and suppose the set of support vectors is unchanged by each single removal. If the round that leaves out the support vector x_p misclassifies it, then $\alpha_p S_p^2 \geq 1$. Consequently

$$\#\{\text{LOO errors}\} \leq \#\{p \in \text{SV} : \alpha_p S_p^2 \geq 1\}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Read the condition as leverage times irreplaceability. A support vector whose image lies close to an admissible combination of its colleagues has small span: remove it, the colleagues absorb its load, the boundary barely moves, and the left-out point is still classified correctly. An LOO error needs both a heavy coefficient and a position no colleague can cover. Computationally, dropping the inequality constraints relaxes Λ_p to the affine hull of the other support vectors, whose distance to $K(x_p, \cdot)$ has a closed form, $\tilde{S}_p^2 = 1/[(\tilde{K}_{\text{SV}}^{-1})_{pp}]$, where $\tilde{K}_{\text{SV}}$ is the support-vector Gram matrix bordered by a row and column of ones with a zero corner; Vapnik and Chapelle show the relaxation is typically tight, so all the spans cost one matrix inversion. Since every span is at most the diameter of the support-vector set, the span certificate is never harder to earn than a radius-type one, and summing the conditions with $S_p \leq 2R$ recovers a radius-margin count up to the constant; in the experiments of Vapnik and Chapelle (2000) the span count follows the true LOO error curve closely where the radius-margin curve is a looser envelope.

The point of these certificates for model selection is not their absolute values but their shape as functions of the dials. Chapelle, Vapnik, Bousquet, and Mukherjee (2002) made this the whole method: treat $T(\theta) = R^2 \|w\|^2$, or a smoothed span count, as an objective over the hyperparameters θ , differentiate it, both factors are solutions of QPs whose optima have computable derivatives, and run gradient descent. Where a grid pays exponentially in the number of dials, a gradient pays linearly, and with one bandwidth per input feature the same machinery scales to dozens of hyperparameters and quietly performs feature selection, coordinates whose bandwidth the descent sends to zero have been judged irrelevant. Learning the kernel itself, as a weighted combination of a dictionary of kernels, continues this thread in Chapter 38, Multiple Kernel Learning.

Algorithm (radius-margin model selection)

Input data $(x_i, y_i)_{i=1}^n$; a kernel family K_θ , for instance Gaussian with bandwidth σ , with the regularization dial folded in through the diagonal shift $K + n\lambda I$ of the 2-SVM remark; a search grid or an initial θ .

Output selected hyperparameters $\hat{\theta}$ and the machine trained at $\hat{\theta}$.

1. Form the Gram matrix K_θ and train the hard-margin machine once, reading off $\|w_\theta\|^2 = \sum_p \alpha_p$.
2. Solve the enclosing-ball QP for R_θ^2 on the same Gram matrix.
3. Score the setting by $T(\theta) = R_\theta^2 \|w_\theta\|^2$.
4. Move θ and repeat: across a log-spaced grid keep the minimizer, or follow the gradient $\partial T / \partial \theta$ (Chapelle, Vapnik, Bousquet, and Mukherjee 2002).
5. Retrain at $\hat{\theta}$ and confirm the choice on data the search never touched.

Example (two leave-one-out certificates from one training run)

Linear kernel; four points $x_1 = (0, 0)$, $x_2 = (0, 1)$ with $y_1 = y_2 = -1$ and $x_3 = (10, 0)$, $x_4 = (10, 1)$ with $y_3 = y_4 = +1$; Gram matrix

$$K = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 100 & 100 \\ 0 & 1 & 100 & 101 \end{pmatrix}.$$

Train the hard-margin machine once, then price its leave-one-out error two ways.

1. Train. The hard-margin dual (as in the four-point example) returns $\alpha = (0.01, 0.01, 0.01, 0.01)$, so $w = \sum_i \alpha_i y_i x_i = (0.2, 0)$ and $b = -1$: the boundary is $x^{(1)} = 5$, with $\|w\|^2 = 0.04$ and margin $\gamma = 1/\|w\| = 5$. Check the budget identity: $\sum_i \alpha_i = 0.04 = \|w\|^2$.
2. Count support vectors. All four points sit exactly on the margin, $y_i f(x_i) = 1$, so $\#SV = 4$ and the count certificate promises only $\#SV/n = 1.0$: vacuous.
3. Solve the enclosing-ball QP. The maximizer is $\beta = (\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$, center $\sum_i \beta_i x_i = (5, 0.5)$, and $R^2 = \sum_i \beta_i K_{ii} - \beta^\top K \beta = 25.25$.
4. Assemble the radius-margin certificate. $R^2 \|w\|^2 = 25.25 \times 0.04 = 1.01$: at most one of the four rounds can err, an LOO rate of at most $1.01/4 = 0.2525$.
5. Compare with the truth. Retraining four times, every left-out point is re-predicted correctly, its twin on the same corner pinning the boundary at $x^{(1)} = 5$ again. True LOO errors: 0.

Reading. One training run priced all four retrainsings, and the two certificates price the same machine very differently. The count saturates: with $n = 4$ and every point on the margin it cannot say better than "all four might err". The radius-margin score sees the geometry: $R^2 \|w\|^2 = 1.01$ sits just above its floor of 1, the enclosing ball of radius $\sqrt{25.25} \approx 5.02$ barely exceeding the margin 5, so the certificate already rules out all but one error. Neither is a point estimate of the true 0; their job is comparative, curves over (C, σ) whose minima locate good dials at one training per candidate instead of n .

Verification artifact. checks/example-ch05-example-5-5.json records the example source hash and verification scope.

6.11 Summary

The support vector machine is regularized empirical risk minimization in an RKHS with the hinge loss. The representer theorem reduces it to a convex but nonsmooth problem in $\mathbb{R}^n$; slack variables turn it into a QP with $2n$ variables; Lagrangian duality compresses it back to a box-constrained QP in n variables whose solution coincides, after the relabeling $\mu_i = 2\lambda y_i \alpha_i$, with the expansion coefficients of $\hat{f}$. Complementary slackness, or equivalently an elementary parabola-on-an-interval argument, sorts the training points into three regimes: $\alpha_i = 0$ off the margin, coefficients strictly inside the box exactly on the margin, and coefficients at the bound on or beyond it. The nonzero-coefficient points, the support vectors, are all the machine remembers, which is what makes both training and prediction fast. The C -SVM is the same machine reparameterized with $C = 1/(2n\lambda)$; the ν -SVM trades that price for a parameter ν that upper bounds the fraction of margin errors and lower bounds the fraction of support vectors; and the 2-SVM, built on the squared hinge, is again the same machine with kernel $K + n\lambda I$ and $C = +\infty$. Beyond two classes, the one-vs-all and one-vs-one reductions reuse the binary machine as a component, while the Weston-Watkins and Crammer-Singer formulations optimize the multiclass margin in one joint program; all decide by $\arg \max_m f_m(x)$ or a pairwise vote. Platt scaling grafts calibrated probabilities onto the margins, a two-parameter sigmoid fitted by maximum likelihood on held-out scores, monotone and therefore powerless to reorder what the machine got backwards. The dials C and bandwidth are set by cross-validation or, at one training per candidate, by the leave-one-out certificates: the support-vector count, the radius-margin ratio $R^2 \|w\|^2$, and the span bound, the latter two smooth enough to be minimized by gradient descent over many hyperparameters at

once. The wide margin these machines seek is not an aesthetic preference but a quantity that provably controls generalization, through margin-based bounds that never invoke the dimension of the feature space. The same slack-variable and duality pipeline, run with the ϵ -insensitive loss in place of the hinge, produces support vector regression, the regression counterpart taken up in Chapter 6, Support Vector Regression.

Further reading

The support vector machine is treated at book length in two standard references. Schölkopf and Smola (2002), *Learning with Kernels*, develops pattern-recognition SVMs in its chapter 7 and gives its optimization chapters over to the quadratic programs and decomposition algorithms that solve them in practice, which makes it the natural place to go deeper on the primal-dual machinery sketched here. Shawe-Taylor and Cristianini (2004), *Kernel Methods for Pattern Analysis*, presents support vector machines in its chapter 7 with an emphasis on the margin-based statistical analysis that justifies the construction. Both books situate the SVM inside the same broad RKHS and regularization framework used throughout this chapter. Two shorter expositions are standard entry points in their own right: the widely cited tutorial of Burges (1998), which works carefully through the geometry of the margin, the dual problem, and the Karush-Kuhn-Tucker conditions, and the introductory monograph of Cristianini and Shawe-Taylor (2000), the first textbook devoted to support vector machines.

6.12 Common mistakes and practical implications

For **Support Vector Machines**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

6.13 Exercises

1. warm-up Evaluate the hinge loss $\varphi_{\text{hinge}}(u) = \max(1 - u, 0)$ at the margins $u = yf(x) \in \{-1, 0, \frac{1}{2}, 1, 2\}$, and say which of these points pay no penalty. The loss has a kink at $u = 1$: write down its derivative on each side of the kink, and give one valid subgradient at $u = 1$ itself. Finally, explain in a sentence why fixing the target margin at 1, rather than at some other positive number, costs no generality.
2. warm-up The chapter shows that the C -SVM and the λ -form are the same machine, related by $C = 1/(2n\lambda)$. For a training set of $n = 100$ points regularized with $\lambda = 10^{-2}$, compute C and the width $1/(2\lambda n)$ of the dual box $0 \leq y_i \alpha_i \leq 1/(2\lambda n)$. Does raising λ widen or narrow this box, and does that correspond to a harder or a softer margin? Reconcile your answer with the rule that large C means weak regularization.
3. computation Take the linear kernel $K(x, x') = \langle x, x' \rangle$ on $\mathbb{R}^2$ and three training points: $x_1 = (1, 0)$ with $y_1 = +1$, $x_2 = (-1, 0)$ with $y_2 = -1$, and $x_3 = (3, 0)$ with $y_3 = +1$. Assume weak regularization, so the upper box cap $1/(2\lambda n)$ is inactive. The classifier $f(x) = \sum_i \alpha_i \langle x_i, x \rangle = \langle w, x \rangle$ carries no offset, so its boundary passes through the origin. Argue from the two nearest opposite-class points that the boundary is the vertical axis and $w = (1, 0)$. Then find every coefficient α_i , identify the support vectors, report the geometric margin $2/\|w\|$, and confirm that x_3 falls in the $\alpha_i = 0$ regime of the KKT trichotomy. Hint

By symmetry about the vertical axis, x_1 and x_2 play the role of the chapter's two-point example, giving $\alpha_1 = \frac{1}{2}, \alpha_2 = -\frac{1}{2}$. Since $w = \alpha_1 x_1 + \alpha_2 x_2 + \alpha_3 x_3$, the well-separated point contributes only if $\alpha_3 \neq 0$. Check its margin: $y_3 f(x_3) = 3 > 1$, so it lies strictly outside the band, which by the trichotomy forces $\alpha_3 = 0$.

4. proof The chapter states the dual of the 2-SVM, built on the squared hinge $\varphi_{\text{hinge}}(y_i f(x_i))^2$, but leaves the derivation as an exercise. Supply it. Introduce one slack $\xi_i \geq \varphi_{\text{hinge}}(y_i [K\alpha]_i)$ per point, so the primal reads

$$\min_{\alpha, \xi} \frac{1}{n} \sum_{i=1}^n \xi_i^2 + \lambda \alpha^\top K \alpha \quad \text{s.t.} \quad y_i [K\alpha]_i + \xi_i - 1 \geq 0, \quad \xi_i \geq 0.$$

Form the Lagrangian with multipliers $\mu, \nu \geq 0$, minimize over α and ξ , and show the dual is

$$\max_{\alpha \in \mathbb{R}^n} 2 \alpha^\top y - \alpha^\top (K + n\lambda I) \alpha \quad \text{s.t.} \quad 0 \leq y_i \alpha_i, \quad i = 1, \dots, n.$$

Explain why the upper box constraint has disappeared, so that the 2-SVM behaves like the ordinary SVM with kernel $K + n\lambda I$ and $C = +\infty$. Hint

Minimizing over α still gives $\alpha^* = \text{diag}(y)\mu/2\lambda$. The change is in ξ : the objective is now quadratic in ξ , so $\partial_{\xi_i} L = (2/n)\xi_i - \mu_i - \nu_i = 0$ has a finite solution $\xi_i^* = n(\mu_i + \nu_i)/2$ for every $\mu_i \geq 0$. There is no longer a linear-in- ξ condition $\mu + \nu = 1/n$ capping μ , which is why the ceiling on $y_i \alpha_i$ vanishes; substituting ξ_i^* back contributes the diagonal shift $n\lambda I$.

5. proof Work directly from the complementary slackness conditions

$$\alpha_i [y_i f(x_i) + \xi_i - 1] = 0, \quad \left(\alpha_i - \frac{y_i}{2\lambda n} \right) \xi_i = 0,$$

together with primal feasibility $y_i f(x_i) + \xi_i \geq 1$, $\xi_i \geq 0$ and the box $0 \leq y_i \alpha_i \leq 1/(2\lambda n)$. Prove: (a) any point strictly outside its margin band, $y_i f(x_i) > 1$, must have $\alpha_i = 0$, hence is not a support vector; (b) any point whose coefficient sits strictly inside the box, $0 < y_i \alpha_i < 1/(2\lambda n)$, lies exactly on the margin, $y_i f(x_i) = 1$. Conclude that the support vectors are precisely the points that touch or cross the margin. Hint

For (a): if $y_i f(x_i) > 1$ then feasibility with $\xi_i \geq 0$ makes the bracket $y_i f(x_i) + \xi_i - 1$ strictly positive, so the first condition forces $\alpha_i = 0$. For (b): $0 < y_i \alpha_i$ makes the first bracket vanish, while $y_i \alpha_i < 1/(2\lambda n)$ makes the factor $\alpha_i - y_i/(2\lambda n) \neq 0$ in the second, forcing $\xi_i = 0$; combine the two.

6. proof Prove the two counting bounds of the ν -property. At a ν -SVM optimum with $\rho > 0$, the constraint $\sum_i \mu_i \geq \nu$ is tight, so $\sum_i \mu_i = \nu$, with the box $0 \leq \mu_i \leq 1/n$ in force. A margin error (a point with $\xi_i > 0$) has its price pinned to the ceiling, $\mu_i = 1/n$, whereas every support vector merely has $\mu_i > 0$. From these facts alone, show that the fraction of margin errors is at most ν and the fraction of support vectors is at least ν . Hint

If n_{err} points are margin errors, they alone contribute n_{err}/n to the sum $\sum_i \mu_i = \nu$, and the remaining prices are nonnegative, giving $n_{\text{err}}/n \leq \nu$. For the support vectors, note that only points with $\mu_i > 0$ contribute anything to $\sum_i \mu_i = \nu$, and each such contribution is at most $1/n$, so $\nu \leq n_{\text{SV}}/n$.

7. exploration Open the interactive SVM figure above and drag the training points. Begin with two well-separated clouds and note which points light up as support vectors, those sitting on or inside the margin band, and which drop out with $\alpha_i = 0$. Now drag a single point across the boundary into the opposing cloud and watch it become a margin error pinned against the top of the box. Next sweep the C control: raise C (weaken regularization) and describe how the band narrows and the support set shrinks toward the borderline points; lower C and watch the band widen and gather more support vectors. Finally switch the kernel from linear to Gaussian and observe how the boundary bends around each cloud. Match each behaviour you see to one of the three KKT regimes.
8. challenge The upper box cap is $1/(2\lambda n)$, and the chapter notes that sending $\lambda \rightarrow 0$ (equivalently $C \rightarrow \infty$) lifts this ceiling to infinity and recovers the hard-margin machine. (a) Argue from the box constraint alone why, in this limit, a single point that cannot be cleanly separated can acquire an unbounded coefficient α_i , and hence dominate $\hat{f} = \sum_i \alpha_i K(x_i, \cdot)$. (b) Show that with finite λ no coefficient can exceed $1/(2\lambda n)$ in magnitude, and interpret this cap as the mechanism by which the soft-margin machine absorbs an outlier

rather than being ruled by it. (c) Connect this to the ν -SVM reading, in which the points pinned at the top of the box are exactly the margin errors and ν is the dial that bounds how many of them there may be. Hint

For (a), an unseparable point keeps its margin constraint tight while wanting more slack, so nothing stops its price, and hence $y_i \alpha_i$, from growing once the ceiling is removed. For (b), the cap is literally the box inequality $0 \leq y_i \alpha_i \leq 1/(2\lambda n)$; since $y_i^2 = 1$, $|\alpha_i| = |y_i \alpha_i|$ is bounded by the same number. For (c), recall from the previous exercise that margin errors carry $\mu_i = 1/n$, the top of the box.

9. computation A ten-class problem has $n = 10000$ training points, 1000 per class. (a) How many binary machines does each reduction train, and on how many points each: one-vs-all, then one-vs-one? How many dual variables does the Crammer-Singer joint problem carry? (b) Prove the max-wins guarantee: in a one-vs-one vote, if some class wins all $M - 1$ of the contests it takes part in, it strictly wins the tally. (c) In the worked one-vs-one example the 2v3 machine votes for class 3 on a class-1 query; explain why such bystander votes can never overturn a class that sweeps its own contests, and construct a three-class outcome in which no class sweeps and the vote ends in a 1–1–1 tie. Hint

For (a): 10 machines of 10000 points; $10 \cdot 9/2 = 45$ machines of 2000 points; the Crammer-Singer dual carries $nM = 100000$ variables, one simplex of size 10 per training point. For (b): every rival lost at least one contest, the one against the sweeping class, so it collects at most $M - 2$ of its $M - 1$ possible votes. For (c): make the pairwise winners a cycle, 1 beats 2, 2 beats 3, 3 beats 1.

10. proof Platt scaling, twice examined. (a) Derive the smoothed target $t_+ = (N_+ + 1)/(N_+ + 2)$: model the labels of the positive-scored calibration cell as Bernoulli with unknown parameter θ under a uniform prior on $[0, 1]$, observe N_+ positives out of N_+ , and compute the posterior mean of θ ; check that $t_- = 1/(N_- + 2)$ is the same computation seen from the negative side. (b) Show that $p(f) = 1/(1 + e^{Af+B})$ is strictly increasing in f exactly when $A < 0$, and conclude that Platt scaling never changes the ordering of scores: any negative point ranked above a positive point stays ranked above it in probability. (c) With the worked example's fit, $A = -0.909732$ and $B = 0.260042$, verify the crossover $f_0 = -B/A = 0.285845$ and identify the interval of scores on which the rule $p \geq \frac{1}{2}$ and the rule $\text{sgn } f$ disagree. Hint

For (a): with a Beta(1, 1) prior and s successes in m trials the posterior is Beta($s + 1$, $m - s + 1$), with mean $(s + 1)/(m + 2)$; set $s = m = N_+$. For (b): $p'(f) = -A p(1 - p)$, and $p(1 - p) > 0$. For (c): the two rules disagree exactly on $f \in (0, f_0)$, scores the SVM calls positive and the calibrated probability calls negative.

11. challenge The radius-margin score has a floor. (a) Prove that for the hard-margin machine $R \geq \gamma$, hence $R^2 \|w\|^2 \geq 1$: the hard-margin optimum has support vectors on both margin hyperplanes, any two points on opposite hyperplanes are at distance at least 2γ , and both lie in the enclosing ball. (b) Show the score is scale-invariant: replacing K by $c^2 K$ (equivalently rescaling the feature map by $c > 0$) multiplies R^2 by c^2 and $\|w\|^2$ by $1/c^2$, leaving $R^2 \|w\|^2$ fixed, so the criterion compares kernels at different scales fairly, which $\|w\|^2$ alone would not. (c) In the worked example $R^2 \|w\|^2 = 1.01$: conclude from integrality that at most one LOO error is possible there, and compare with the true count of 0. Hint

For (a): if one class had no support vector on its margin hyperplane, the boundary could be translated toward that class and the band widened, contradicting optimality; then take one support vector on each hyperplane, at mutual distance at least 2γ but at most $2R$. For (b): w solves a problem whose constraints scale with c , so $\|w\|$ scales as $1/c$; distances in $\mathcal{H}$, and with them R , scale as c . For (c): the bound 1.01 caps an integer count.

7 Support Vector Regression

The support vector machine of Chapter 5, Support Vector Machines learns a real-valued decision function but uses only its sign. This chapter keeps the function and throws away the sign: we estimate a real target $y \in \mathbb{R}$ instead of a label $y \in \{\pm 1\}$, and we ask that the estimator inherit the one property that made the classifier so useful, sparsity. The engine of that inheritance is a single change to the loss. Where least squares charges for every deviation, Vapnik's ε -insensitive loss forgives small ones, opening a flat-bottomed valley of zero cost around the target. Points that fall into the valley cost nothing, contribute nothing, and drop out of the solution, so the regressor is written in terms of a small subset of the data, the support vectors, exactly as in classification. We build the primal program, derive its dual and the box constraints that structure it, read the geometry of the resulting ε -tube, and then meet the ν -variant that trades the awkward parameter ε for a parameter ν with a clean statistical meaning: it bounds the fraction of points allowed outside the tube. We follow Vapnik (1995) and Schölkopf and Smola (2002) throughout, with the ν -formulation from Schölkopf, Smola, Williamson, and Bartlett (2000).

7.1 The ε -insensitive loss and where sparsity comes from

Recall why the support vector classifier is sparse. A correctly classified pattern sitting beyond the margin incurs zero hinge loss. Because the decision surface is computed by minimizing that very loss, a pattern which contributes nothing to the objective also carries no information about where the surface should go: we could delete it and recover the same solution. Such patterns therefore do not appear in the final expansion. The mechanism is entirely the flat region of the loss.

To carry this over to regression we need a loss for real targets that also has a flat region, a zone of deviations we simply do not charge for. That is precisely Vapnik's proposal.

Definition (ε -insensitive loss, Vapnik 1995)

For a fixed tolerance $\varepsilon \geq 0$, chosen a priori, the ε -insensitive loss of predicting $f(x)$ when the target is y is

$$|y - f(x)|_\varepsilon := \max(0, |y - f(x)| - \varepsilon).$$

Deviations up to ε cost nothing; beyond ε the loss grows linearly with the excess.

Support vector regression was introduced by Drucker, Burges, Kaufman, Smola, and Vapnik (1997), who carried the margin construction of the classifier across to real-valued targets; the ε -insensitive loss on which it rests is due to Vapnik (1995) and is developed at length in Vapnik (1998). Everything that follows, the primal program, its dual, the tube geometry, and the ν -variant, is the account systematized in the monograph of Schölkopf and Smola (2002).

The graph is a trough: a flat floor of zero on the interval $[-\varepsilon, \varepsilon]$, then two straight ramps of unit slope. The flat floor is the regression analogue of the classifier's margin region, and it will do the same work. A point whose prediction already lands within ε of its target sits on the floor, contributes zero to the objective, and, as we will prove from the optimality conditions, receives a zero coefficient. The prediction is then reconstructed from the remaining points alone. Setting $\varepsilon = 0$ recovers the ordinary ℓ_1 (least absolute deviations) loss, which is robust but not sparse; the sparsity is bought entirely by the width of the insensitive zone.

7.2 The primal program for SV regression

We estimate an affine function in a feature space induced by a kernel,

$$f(x) = \langle w, x \rangle + b,$$

where, as in Chapter 5, Support Vector Machines, we write x for the (possibly mapped) input and apply the kernel trick only to the linear part. Two goals pull against each other. We want f to be flat, which for the same reason as in classification means small $\|w\|$: a flat function has large geometric margin and low capacity, so the regularizer $\frac{1}{2}\|w\|^2$ is shared verbatim with the classifier of Chapter 5, Support Vector Machines. And we want f to stay within ε of the data. The direct objective

$$\frac{1}{2}\|w\|^2 + \frac{C}{m} \sum_{i=1}^m |y_i - f(x_i)|_\varepsilon$$

already encodes the trade-off, with $C > 0$ setting how dearly we pay for leaving the tube. To turn the nonsmooth loss into a smooth quadratic program we split each excess deviation into two slack variables, one for overshoot and one for undershoot, exactly as the soft margin splits margin violations.

Definition (primal program, ε -SVR)

Given data $(x_1, y_1), \dots, (x_m, y_m)$, tolerance $\varepsilon \geq 0$, and penalty $C > 0$, solve

$$\min_{w, b, \xi, \xi^*} \frac{1}{2}\|w\|^2 + \frac{C}{m} \sum_{i=1}^m (\xi_i + \xi_i^*)$$

subject to, for every $i = 1, \dots, m$,

$$\langle w, x_i \rangle + b - y_i \leq \varepsilon + \xi_i, \quad y_i - \langle w, x_i \rangle - b \leq \varepsilon + \xi_i^*, \quad \xi_i, \xi_i^* \geq 0.$$

The two constraint families measure the two ways a point can leave the tube: ξ_i records how far $f(x_i)$ overshoots $y_i + \varepsilon$, and ξ_i^* how far it undershoots $y_i - \varepsilon$. A point inside the tube needs neither slack, because any deviation below ε already satisfies both inequalities with $\xi_i = \xi_i^* = 0$, and so does not enter the objective at all. This is the flat floor of the loss written as a feasibility condition, and it is the first sign of the sparsity to come.

7.3 The dual and its box constraints

As with the classifier, the kernel enters only after we pass to the dual, where the inputs appear solely through inner products. We introduce a nonnegative multiplier for each constraint: α_i for the overshoot family, α_i^* for the undershoot family, and η_i, η_i^* for the two positivity constraints on the slacks. The derivation is the Lagrangian saddle-point calculation laid out in the tutorial of Smola and Schölkopf (2004), and it produces the box constraints that give the method its structure.

Theorem (dual program, ε -SVR)

The dual of the primal program is

$$\max_{\alpha, \alpha^*} -\frac{1}{2} \sum_{i,j=1}^m (\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*) k(x_i, x_j) - \varepsilon \sum_{i=1}^m (\alpha_i + \alpha_i^*) + \sum_{i=1}^m (\alpha_i - \alpha_i^*) y_i$$

subject to

$$\sum_{i=1}^m (\alpha_i - \alpha_i^*) = 0, \quad \alpha_i, \alpha_i^* \in \left[0, \frac{C}{m}\right].$$

The solution is $f(x) = \sum_{i=1}^m (\alpha_i - \alpha_i^*) k(x_i, x) + b$, with $w = \sum_i (\alpha_i - \alpha_i^*) x_i$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Form the Lagrangian, subtracting each constraint (written as a nonnegative quantity) against its multiplier:

$$L = \frac{1}{2} \|w\|^2 + \frac{C}{m} \sum_i (\xi_i + \xi_i^*) - \sum_i (\eta_i \xi_i + \eta_i^* \xi_i^*) - \sum_i \alpha_i (\varepsilon + \xi_i - y_i + \langle w, x_i \rangle + b) - \sum_i \alpha_i^* (\varepsilon + \xi_i^* + y_i - \langle w, x_i \rangle - b),$$

with all of $\alpha_i, \alpha_i^*, \eta_i, \eta_i^* \geq 0$. At a saddle point the derivatives in the primal variables vanish. Differentiating in b gives $\sum_i (\alpha_i - \alpha_i^*) = 0$. Differentiating in w gives $w = \sum_i (\alpha_i - \alpha_i^*) x_i$, the support vector expansion. Differentiating in ξ_i and ξ_i^* gives $\frac{C}{m} - \alpha_i - \eta_i = 0$ and $\frac{C}{m} - \alpha_i^* - \eta_i^* = 0$; since $\eta_i, \eta_i^* \geq 0$, this forces $\alpha_i, \alpha_i^* \leq \frac{C}{m}$, the upper box. Substituting these relations back into L annihilates every slack and offset term (each ξ_i multiplies $\frac{C}{m} - \alpha_i - \eta_i = 0$, and b multiplies $\sum_i (\alpha_i - \alpha_i^*) = 0$), leaving exactly the stated objective once inner products $\langle x_i, x_j \rangle$ are replaced by $k(x_i, x_j)$. $\square$

Three features of this dual deserve names. First, the variables enter only through the differences $\beta_i := \alpha_i - \alpha_i^*$, and the equality constraint $\sum_i \beta_i = 0$ is the exact analogue of $\sum_i \alpha_i y_i = 0$ in the classifier dual; it is what makes the offset b free. Second, each multiplier is confined to the box $[0, C/m]$, so no single point can pull on the solution with unbounded force; the cap C/m is the largest influence one datum may exert, and it is the seed of the robustness we discuss below. Third, the objective is a quadratic program in $2m$ variables with one equality and $2m$ box constraints, solved by the same decomposition machinery, sequential minimal optimization and its relatives, that we developed for the classifier in Chapter 9, Solving the SVM: Decomposition and SMO.

Algorithm (ε -SVR dual QP)

Input data $\{(x_i, y_i)\}_{i=1}^m$, kernel k , penalty C , tube width $\varepsilon \geq 0$.

Output coefficients $\beta_i = \alpha_i - \alpha_i^*$ and offset b , giving $f(x) = \sum_i \beta_i k(x_i, x) + b$.

1. Form the Gram matrix $K_{ij} = k(x_i, x_j)$.
2. Solve the quadratic program of the dual theorem for (α, α^*) subject to $\sum_i (\alpha_i - \alpha_i^*) = 0$ and $0 \leq \alpha_i, \alpha_i^* \leq C/m$.
3. Set $\beta_i = \alpha_i - \alpha_i^*$; the support vectors are the indices with $\beta_i \neq 0$.
4. Pick any in-bound support vector, one with $0 < \alpha_i < C/m$ (upper edge) or $0 < \alpha_i^* < C/m$ (lower edge), and set $b = y_i - \sum_j \beta_j k(x_j, x_i) \mp \varepsilon$, minus for an upper-edge, plus for a lower-edge SV.
5. Repeat step 4 over several in-bound SVs and average, to stabilize b .

7.4 Geometry of the ε -tube: support vectors, sparsity, and resistance

The Karush-Kuhn-Tucker conditions turn the dual solution into a picture. At optimality each multiplier times its constraint slack vanishes:

$$\alpha_i (\varepsilon + \xi_i - y_i + \langle w, x_i \rangle + b) = 0, \quad \alpha_i^* (\varepsilon + \xi_i^* + y_i - \langle w, x_i \rangle - b) = 0,$$

together with $(\frac{C}{m} - \alpha_i) \xi_i = 0$ and the same for the starred pair. Reading these off gives the anatomy of the solution.

The tube is a slab. The region $|y - f(x)| \leq \varepsilon$ is a band of vertical half-width ε tracking the graph of f . In more than one input dimension it is a slab between two parallel hyperplanes offset in y , so "tube" is a mild abuse; the picture on the line is a genuine tube.

Only boundary and exterior points count. If (x_i, y_i) lies strictly inside the tube, then $\varepsilon - |y_i - f(x_i)| > 0$ while $\xi_i = \xi_i^* = 0$, so the parenthesized factor in each KKT equation is strictly positive and forces $\alpha_i = \alpha_i^* = 0$. Interior points receive zero coefficient. The support vectors, the points with $\beta_i \neq 0$, are exactly those on the edge of the tube or outside it. This is the promised sparsity: it is geometrically plausible because deleting a strictly-interior point leaves the optimum unchanged, hence that point carried no information about the fit. One further identity always holds, $\alpha_i \alpha_i^* = 0$: a point cannot simultaneously overshoot and undershoot, so at most one of the two multipliers is ever nonzero (Problem 9.1 of the source).

Computing the offset. A support vector strictly inside the box, $0 < \alpha_i < C/m$, has $\xi_i = 0$ and sits exactly on the tube edge, so its KKT equation reads $\varepsilon - y_i + \langle w, x_i \rangle + b = 0$, giving $b = y_i - \langle w, x_i \rangle - \varepsilon$; a lower-edge in-bound SV gives $b = y_i - \langle w, x_i \rangle + \varepsilon$. Any such point determines b ; averaging several guards against roundoff.

We can watch all of this happen on a dataset small enough to solve by hand.

Example (a five-point ε -SVR fit with a linear kernel)

Five points on a line, $x = (1, 2, 3, 4, 5)$, $y = (1, 2, 3, 4, 5)$, with linear kernel $k(x, x') = xx'$, penalty $C = 10$ so $C/m = 2$, and tube width $\varepsilon = 0.5$. We solve the dual QP and read off the fit.

1. Solve the dual. The QP returns $\alpha_1^* = 0.1875$ and $\alpha_5 = 0.1875$, all other multipliers zero, so $\beta = (-0.1875, 0, 0, 0, 0.1875)$. The equality holds: $\sum_i \beta_i = 0$.
2. Recover the line. From $w = \sum_i \beta_i x_i = -0.1875(1) + 0.1875(5) = 0.75$. Using the upper-edge SV x_5 , $b = y_5 - wx_5 - \varepsilon = 5 - 3.75 - 0.5 = 0.75$. The lower-edge SV x_1 gives the same: $b = 1 - 0.75 + 0.5 = 0.75$. So $f(x) = 0.75x + 0.75$.
3. Classify the points. The residuals $y_i - f(x_i)$ are $(-0.5, -0.25, 0, +0.25, +0.5)$. Points x_2, x_3, x_4 lie strictly inside the tube ($|\cdot| < 0.5$) and get $\beta_i = 0$; points x_1 and x_5 sit exactly on the lower and upper edges and are the two support vectors. The SV fraction is $2/5 = 0.4$.
4. Check strong duality. The dual objective evaluates to $0.75 - 0.1875 - 0.28125 = 0.28125$; the primal objective is $\frac{1}{2}w^2 + 0 = 0.5(0.5625) = 0.28125$ since no slack is used. The duality gap is zero.

Reading. A perfectly linear dataset is fit not by the least-squares slope 1.0 but by the flatter slope 0.75: the tube lets the line relax toward horizontal until its two extreme points touch the edges, and those two points, alone, pin the solution. Everything in between is inert.

Verification artifact. checks/example-ch-svr-example-6-1.json records the example source hash and verification scope.

The bounded influence C/m buys a robustness property that is worth stating precisely, because it is often misread. Since the loss grows only linearly outside the tube, the multiplier of an exterior point is pinned at the ceiling C/m no matter how far out the point lies. Moving such a point further away does not change its force on the fit.

Proposition (resistance of SV regression, Schölkopf and Smola 2002)

With the ε -insensitive loss, a local move of the target value y_i of a point that lies strictly outside the tube does not change the regression.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Shifting y_i slightly keeps (x_i, y_i) outside the tube, so its dual multiplier stays at the bound C/m and the dual solution remains feasible. The primal solution, with $\xi_i^{(*)}$ adjusted by the shift, is also feasible, and the KKT conditions still hold because the exterior point's multiplier is at the same bound as before. By convexity the pair is still optimal, so f is unchanged. $\square$

Thus SV regression behaves like a trimmed estimator of the mean: informally it discards the most extreme residuals and fits to the boundary points. This is the opposite of least squares, where a single far outlier dominates. The parameter ε is related to the breakdown behavior, since it governs what fraction of the data may sit outside the tube as arbitrary outliers.

7.5 General losses and the ridge connection

The ε -insensitive loss is one member of a family. The same derivation goes through for any convex, symmetric loss $c(x, y, f(x)) = \tilde{c}(|y - f(x)|)$ that vanishes on $[-\varepsilon, \varepsilon]$: each choice changes only the box on the dual variables and adds a term $T(\alpha^{(*)})$ to the objective. Schölkopf and Smola (2002) tabulate the cases; the pattern is that the maximum slope $s = \sup_{\xi} \tilde{c}'(\xi)$ of the loss sets the ceiling $[0, Cs]$ on the multipliers, so a bounded slope means bounded influence and hence a robust estimator.

Loss $\tilde{c}(\xi)$	box on $\alpha^{(*)}$	extra term T
ε -insensitive, $ \xi _{\varepsilon}$	$[0, C]$	0
Laplacian, $ \xi $	$[0, C]$	0
Gaussian, $\frac{1}{2}\xi^2$	$[0, \infty)$	$\frac{1}{2C} \alpha^2$
Huber's robust loss	$[0, C]$	$\frac{1}{2C} \alpha^2$

The Gaussian row is the bridge to the rest of the book. With squared loss and $\varepsilon = 0$, the primal objective $\frac{1}{2}\|w\|^2 + \frac{C}{m} \sum_i (y_i - f(x_i))^2$ is exactly ridge regression, and its kernelized form is the kernel ridge regression of Chapter 4, Kernel Ridge Regression and Smooth Losses. So SV regression and kernel ridge regression are the same regularized risk minimization with different losses: squared loss gives the closed-form linear solve of KRR but a dense expansion in which every point is a support vector, while the ε -insensitive loss gives a quadratic program but a sparse expansion. One trades a linear system for a QP and, in return, gets to store only the support vectors. The cautionary note in the source is exactly this: any loss with $\varepsilon = 0$ loses sparsity, which may be acceptable for small data but slows prediction otherwise.

7.6 Quantile regression with the pinball loss

Every loss met so far is symmetric: it charges overshoot and undershoot alike, so the fitted function tracks the centre of the data and says nothing about its spread. Yet many questions are about the spread. A delivery estimate is useful only with a near-worst-case upper bound; a risk model needs the fifth and ninety-fifth percentiles, not the average. The clean way to read such a bound off a model in Chapter 40, Gaussian Processes and the RVM is to assume a Gaussian predictive density and take its quantiles, but that answer is only as trustworthy as the Gaussian assumption, and real residuals are often skewed or heavy-tailed. Quantile regression asks for the conditional quantile directly, distribution-free, through a single change of loss: make the trough of the previous sections lopsided.

Definition (pinball loss, Koenker and Bassett 1978)

For a quantile level $\tau \in (0, 1)$, the pinball (or check) loss of the residual $u = y - f(x)$ is

$$\rho_{\tau}(u) = \max(\tau u, (\tau - 1)u) = \begin{cases} \tau u & u \geq 0, \\ (1 - \tau) |u| & u < 0. \end{cases}$$

An underprediction ($u > 0$, the fit lying below the target) is charged at rate τ ; an overprediction ($u < 0$) at rate $1 - \tau$.

The graph is again two straight ramps meeting at the origin, but now with unequal slopes τ and $1 - \tau$ and no flat floor between them: the tolerance is $\varepsilon = 0$. The lopsided slopes are the whole idea. Where a symmetric loss pulls the fit to the middle, the pinball loss makes it cheaper to sit below the data when $\tau > \frac{1}{2}$ and cheaper to sit above when $\tau < \frac{1}{2}$, so the minimizer settles at a level with a controlled fraction of the mass beneath it. Koenker and Bassett (1978) proved the population statement that anchors the method: the function minimizing the expected pinball loss $\mathbb{E} \rho_\tau(Y - f(X))$ is exactly the conditional τ -quantile of Y given X . At $\tau = \frac{1}{2}$ the two slopes are equal, $\rho_{1/2}(u) = \frac{1}{2}|u|$, and we are back at the median, the Laplacian (least absolute deviations) row of the table above.

Kernelizing this in the manner of Chapter 5, Support Vector Machines is now routine. We regularize by flatness exactly as before and split each residual into an over- and an under-shoot slack, but we weight the two slacks unequally, by τ and $1 - \tau$, to match the loss.

Definition (primal program, quantile SVR)

Given data $(x_1, y_1), \dots, (x_m, y_m)$, level $\tau \in (0, 1)$, and penalty $C > 0$, solve

$$\min_{w, b, \xi, \xi^*} \frac{1}{2} \|w\|^2 + \frac{C}{m} \sum_{i=1}^m (\tau \xi_i + (1 - \tau) \xi_i^*)$$

subject to, for every $i = 1, \dots, m$,

$$y_i - \langle w, x_i \rangle - b \leq \xi_i, \quad \langle w, x_i \rangle + b - y_i \leq \xi_i^*, \quad \xi_i, \xi_i^* \geq 0.$$

Passing to the dual repeats the calculation behind the ε -SVR dual almost verbatim. The only two changes are that the tube width is gone, $\varepsilon = 0$, and that the unequal slack weights push the two ceilings apart.

Theorem (dual program, quantile SVR, Takeuchi, Le, Sears, and Smola 2006)

The dual is

$$\max_{\alpha, \alpha^*} \sum_{i=1}^m (\alpha_i - \alpha_i^*) y_i - \frac{1}{2} \sum_{i,j=1}^m (\alpha_i - \alpha_i^*) (\alpha_j - \alpha_j^*) k(x_i, x_j)$$

subject to

$$\sum_{i=1}^m (\alpha_i - \alpha_i^*) = 0, \quad \alpha_i \in \left[0, \frac{C}{m} \tau\right], \quad \alpha_i^* \in \left[0, \frac{C}{m} (1 - \tau)\right].$$

It is precisely the ε -SVR dual with $\varepsilon = 0$ and the single symmetric box $[0, C/m]$ replaced by the asymmetric pair. The fit is $f(x) = \sum_i (\alpha_i - \alpha_i^*) k(x_i, x) + b$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Introduce multipliers $\alpha_i, \alpha_i^* \geq 0$ for the two tube constraints and $\eta_i, \eta_i^* \geq 0$ for the slack positivity, and form the Lagrangian

$$L = \frac{1}{2} \|w\|^2 + \frac{C}{m} \sum_i (\tau \xi_i + (1 - \tau) \xi_i^*) - \sum_i (\eta_i \xi_i + \eta_i^* \xi_i^*) + \sum_i \alpha_i (y_i - \langle w, x_i \rangle - b - \xi_i) + \sum_i \alpha_i^* (\langle w, x_i \rangle + b - y_i - \xi_i^*).$$

Stationarity in w gives $w = \sum_i (\alpha_i - \alpha_i^*) x_i$, and stationarity in b gives $\sum_i (\alpha_i - \alpha_i^*) = 0$, exactly as in the ε -SVR derivation. Stationarity in the slacks now reads $\frac{C}{m} \tau - \alpha_i - \eta_i = 0$ and $\frac{C}{m} (1 - \tau) - \alpha_i^* - \eta_i^* = 0$; since $\eta_i, \eta_i^* \geq 0$,

these force $\alpha_i \leq \frac{C}{m}\tau$ and $\alpha_i^* \leq \frac{C}{m}(1 - \tau)$, the asymmetric ceilings. Substituting the relations back annihilates every slack and offset term, and, because $\varepsilon = 0$, no $-\varepsilon \sum_i (\alpha_i + \alpha_i^*)$ term survives, leaving the stated objective once $\langle x_i, x_j \rangle$ is replaced by $k(x_i, x_j)$. $\square$

Reading the box is again the whole story, just as the budget constraint was for ν -SVR. The upper multiplier α_i , which attaches to points sitting above the fit, is capped at $\frac{C}{m}\tau$; the lower multiplier α_i^* , for points below the fit, is capped at $\frac{C}{m}(1 - \tau)$. A large τ gives the points above the fit more pull and those below it less, so the fit rises until only a τ -fraction of the data is left beneath it; a small τ does the reverse. The Karush-Kuhn-Tucker conditions make the accounting exact: a point strictly below the fit has its α_i^* pinned at the ceiling $\frac{C}{m}(1 - \tau)$, a point strictly above has α_i pinned at $\frac{C}{m}\tau$, and a point on the fit is an in-bound support vector that fixes the offset through $b = y_i - \langle w, x_i \rangle$. Because the loss has no flat floor, no point sits in a cost-free interior, so every point is a support vector: quantile regression, like every $\varepsilon = 0$ entry in the loss table, trades the sparsity of the tube for the ability to chase a quantile. The tie to Chapter 4, Kernel Ridge Regression and Smooth Losses is the one drawn above, moved one loss to the side: kernel ridge minimizes the squared residual and estimates the conditional mean, quantile SVR minimizes the pinball residual and estimates a conditional quantile, and the median case $\tau = \frac{1}{2}$ is the robust absolute-deviation midpoint between them.

With finitely many points the fraction beneath the fit cannot equal τ exactly, but it is trapped around it. Points lying exactly on the fit are ties that may be counted on either side, so writing #below for the strictly-below points and #on for those on the fit, every solution obeys

$$\frac{\text{\#below}}{m} \leq \tau \leq \frac{\text{\#below} + \text{\#on}}{m},$$

the quantile analogue of the ν -property bracket, which follows from the equality $\sum_i (\alpha_i - \alpha_i^*) = 0$ and the asymmetric ceilings (Exercise 10). We watch two levels bracket a small data set.

Example (a quantile fit at two levels)

Six points $x = (1, 2, 3, 4, 5, 6)$, $y = (1.0, 2.6, 2.0, 4.2, 3.4, 5.6)$, linear kernel $k(x, x') = xx'$, penalty $C = 12$ so $C/m = 2$. We fit the pinball loss at $\tau = 0.25$ and $\tau = 0.75$ and read off the two lines.

1. Set the asymmetric box. At $\tau = 0.25$ the ceilings are $\alpha_i \in [0, \frac{C}{m}\tau] = [0, 0.5]$ and $\alpha_i^* \in [0, \frac{C}{m}(1 - \tau)] = [0, 1.5]$; at $\tau = 0.75$ they swap to $[0, 1.5]$ and $[0, 0.5]$. Everything else in the dual is identical.
2. Fit both levels. Solving the dual QP gives the low line $f(x) = 0.6x + 0.4$ at $\tau = 0.25$ and the high line $f(x) = 0.75x + 1.1$ at $\tau = 0.75$. The low line lies below the high line at every x , so the two bracket the data.
3. Count what is beneath. At $\tau = 0.25$ the residuals $y_i - f(x_i)$ are $(0, 1.0, -0.2, 1.4, 0, 1.6)$: one point (x_3) sits below the line, two (x_1, x_5) on it, three above. At $\tau = 0.75$ the residuals are $(-0.85, 0, -1.35, 0.1, -1.45, 0)$: three points below, two on, one above. The fraction beneath rises from $1/6$ to $3/6$ as τ grows.
4. Check the quantile bracket. At $\tau = 0.25$, $\frac{1}{6} \leq 0.25 \leq \frac{3}{6}$, that is $0.167 \leq 0.25 \leq 0.500$; at $\tau = 0.75$, $\frac{3}{6} \leq 0.75 \leq \frac{5}{6}$, that is $0.500 \leq 0.75 \leq 0.833$. The level is trapped between the strictly-below and the on-or-below fractions in both rows, and strong duality holds, the dual and primal objectives agreeing at 2.480 and 2.256.

Reading. One dial, τ , slides the fit up or down through the data with no tube at all: the asymmetry of the box does what the width ε did before. Two levels give two lines that fence the points between them, a distribution-free prediction band built from the same quadratic program as the classifier.

Verification artifact. checks/example-ch-svr-example-6-2.json records the example source hash and verification scope.

7.7 Expectile regression

Quantiles minimize an asymmetric absolute loss. Expectiles replace the absolute deviation by an asymmetrically weighted square (Newey and Powell 1987). For level $\tau \in (0, 1)$, define

$$\ell_\tau^{\text{exp}}(y, f) = |\tau - \mathbf{1}\{y < f\}|(y - f)^2.$$

At $\tau = 1/2$ the population minimizer is the conditional mean; away from one half, large residuals receive quadratic influence and pull the estimate toward the corresponding tail. Expectiles are smooth away from zero residual and often easier to optimize than pinball loss, but they are less robust to outliers and are not quantiles. Their interpretation is through asymmetric squared error, not a direct conditional coverage probability.

Kernel expectile regression minimizes the empirical expectile loss plus $\lambda \|f\|_{\mathcal{H}}^2$, so the representer theorem gives the same finite expansion as KRR and quantile regression. Iteratively reweighted least squares alternates between residual-dependent weights and a weighted kernel ridge solve. Stop on objective and KKT change, and guard against nearly zero weights and poorly conditioned weighted systems.

7.8 Noncrossing distributional prediction

Fitting many quantile levels independently can produce crossings, such as an estimated upper quantile below a lower one. Remedies include joint optimization with monotonicity constraints, rearrangement after fitting, or a model of the entire conditional distribution. Each changes the estimator: post-hoc sorting repairs order but does not reproduce the jointly constrained optimum.

Evaluate a quantile model across levels with pinball loss, empirical conditional coverage, interval width, and crossing frequency. Marginal calibration alone can conceal errors across covariate regions. If intervals are required rather than quantiles, conformal calibration in Chapter 51, Distribution Shift, Robustness, and Conformal Prediction can wrap a fitted kernel regressor under its own exchangeability or shift assumptions.

Algorithm (fit a kernel quantile family)

1. Declare quantile levels and the deployment cost of under- and overprediction.
2. Tune kernel and regularization on training folds using aggregate pinball loss.
3. Fit levels jointly with noncrossing constraints, or declare and test a rearrangement rule.
4. Evaluate on untouched data by level and relevant subgroup.
5. Report crossings, interval width, tail sample size, solver tolerances, and sensitivity to outliers.

7.9 ν -SV regression

The tolerance ε is useful when the desired accuracy is known in advance, but often it is not, and choosing it blindly is delicate: a value that is ideal for a clean signal is far too tight for a noisy one, and vice versa. It would be better to let the algorithm choose ε itself, controlled by a parameter with a meaning we can reason about. This is the ν -trick, transplanted from the ν -classifier of Chapter 5, Support Vector Machines: promote ε to a variable, pay for it linearly, and let a new parameter ν govern the trade.

Definition (primal program, ν -SVR)

For $\nu \in (0, 1]$ and $C > 0$, solve over $w, b, \varepsilon \geq 0, \xi, \xi^*$

$$\min \frac{1}{2} \|w\|^2 + C \left(\nu \varepsilon + \frac{1}{m} \sum_{i=1}^m (\xi_i + \xi_i^*) \right)$$

subject to the same tube constraints as before, now with ε a decision variable rather than a constant.

The term $\nu\varepsilon$ charges for the tube width: a wide tube is cheap in slack but expensive in this term, and the optimizer settles the balance. Passing to the dual, the calculation of the previous section repeats almost verbatim, but the derivative in the new variable ε contributes one extra constraint and, remarkably, removes the ε term from the objective.

Theorem (dual program, ν -SVR)

The dual is

$$\max_{\alpha, \alpha^*} \sum_{i=1}^m (\alpha_i - \alpha_i^*) y_i - \frac{1}{2} \sum_{i,j=1}^m (\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*) k(x_i, x_j)$$

subject to

$$\sum_{i=1}^m (\alpha_i - \alpha_i^*) = 0, \quad \alpha_i, \alpha_i^* \in \left[0, \frac{C}{m}\right], \quad \sum_{i=1}^m (\alpha_i + \alpha_i^*) \leq C\nu.$$

The width ε and offset b are recovered afterward from any pair of in-bound support vectors, one on each edge of the tube.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The ε that vanished from the objective reappears as the Lagrange multiplier of the new budget constraint $\sum_i (\alpha_i + \alpha_i^*) \leq C\nu$. That single constraint is the whole story of the ν -trick, and reading it against the box $[0, C/m]$ gives ν its meaning. This is the counting argument that names ν .

Proposition (ν -property, Schölkopf, Smola, Williamson, Bartlett 2000)

Suppose ν -SVR is applied and the resulting ε is nonzero. Then

i ν is an upper bound on the fraction of errors (points strictly outside the tube);

ii ν is a lower bound on the fraction of support vectors;

iii if the data are drawn i.i.d. from a distribution $P(x, y) = P(x)P(y | x)$ with $P(y | x)$ having no atoms (a density), then with probability one, asymptotically as $m \rightarrow \infty$, ν equals both the fraction of support vectors and the fraction of errors.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof (parts i and ii)

When $\varepsilon > 0$ the budget constraint is active, so $\sum_i (\alpha_i + \alpha_i^*) = C\nu$. A point strictly outside the tube has a positive slack, and by the KKT slack condition $(\frac{C}{m} - \alpha_i^{(*)})\xi_i^{(*)} = 0$ its active multiplier is pinned at the ceiling C/m , contributing exactly C/m to the sum. Hence $\frac{C}{m} \cdot (\text{number of errors}) \leq \sum_i (\alpha_i + \alpha_i^*) = C\nu$, which gives $(\text{number of errors}) \leq m\nu$, the fraction bound (i). For (ii), every multiplier obeys $\alpha_i^{(*)} \leq C/m$, so the same total satisfies $C\nu = \sum_i (\alpha_i + \alpha_i^*) \leq \frac{C}{m} \cdot (\text{number of SVs})$, giving $(\text{number of SVs}) \geq m\nu$. $\square$

So ν is squeezed between the two counts: errors on the low side, support vectors on the high side, and asymptotically it pins both. The bound is tightest when m is large relative to C and loosens otherwise, which is the content of the source's asymptotic table. Because $0 \leq \text{errors} \leq \text{SVs}$, only $\nu \in (0, 1]$ is meaningful; values $\nu \geq 1$ force $\varepsilon = 0$ and reduce to plain ℓ_1 regression. The algorithm is identical in shape to the ε -version, with the one extra constraint and the post hoc recovery of ε .

7 Support Vector Regression

Algorithm (ν -SV regression)

Input data $\{(x_i, y_i)\}_{i=1}^m$, kernel k , penalty C , fraction parameter $\nu \in (0, 1]$.

Output coefficients $\beta_i = \alpha_i - \alpha_i^*$, offset b , and the automatically chosen width ε .

1. Form the Gram matrix $K_{ij} = k(x_i, x_j)$.
2. Solve the ν -SVR dual QP for (α, α^*) under $\sum_i (\alpha_i - \alpha_i^*) = 0$, $0 \leq \alpha_i, \alpha_i^* \leq C/m$, and $\sum_i (\alpha_i + \alpha_i^*) \leq C\nu$.
3. Set $\beta_i = \alpha_i - \alpha_i^*$; the SVs are the indices with $\beta_i \neq 0$, the errors those with $|\beta_i| = C/m$.
4. Take one in-bound SV i on the upper edge and one j on the lower edge; with $g_k = \sum_l \beta_l k(x_l, x_k)$, solve $\varepsilon = \frac{1}{2}[(y_i - g_i) - (y_j - g_j)]$ and $b = \frac{1}{2}[(y_i - g_i) + (y_j - g_j)]$.
5. Report $f(x) = \sum_i \beta_i k(x_i, x) + b$ and ε ; to sweep accuracy, repeat for several ν .

Example (ν sets the tube width, and the ν -property)

Five points $x = (0, 1, 2, 3, 4)$, $y = (0, 0.9, 0.2, -0.8, 0.3)$, a Gaussian kernel $k(x, x') = e^{-(x-x')^2/2\sigma^2}$ with $\sigma = 1.5$ (so the Gram matrix is full rank and the dual is unique), and $C = m = 5$ so $C/m = 1$. We solve ν -SVR for three values of ν and read off the tube.

1. Sweep ν . Solving the dual gives width $\varepsilon = 0.565$ at $\nu = 0.2$, $\varepsilon = 0.502$ at $\nu = 0.4$, and $\varepsilon = 0.330$ at $\nu = 0.6$. The tube narrows monotonically as ν grows.
2. Count support vectors and errors. At $\nu = 0.2$: 3 SVs, 0 errors, so fractions 0.60 and 0.00. At $\nu = 0.4$: 4 SVs, 0 errors (0.80, 0.00). At $\nu = 0.6$: 4 SVs, 2 errors (0.80, 0.40).
3. Check the bounds. In every row the fraction of errors $\leq \nu \leq$ the fraction of SVs: $0.00 \leq 0.2 \leq 0.60$, then $0.00 \leq 0.4 \leq 0.80$, then $0.40 \leq 0.6 \leq 0.80$. The proposition holds line by line.
4. Inspect $\nu = 0.6$. Here $\beta = (-0.5, 1, 0, -1, 0.5)$, so $\sum_i \beta_i = 0$ and the budget $\sum_i |\beta_i| = 3.0$ equals $C\nu = 3.0$ exactly, confirming the constraint is active. Points x_1, x_3 hit the ceiling $|\beta| = 1$ and lie outside the tube (the two errors); x_0, x_4 are in-bound SVs on the edges; x_2 is inside. The recovered offset is $b = 0.15$.

Reading. Turning one dial, ν , simultaneously widens or narrows the tube and controls how many points are allowed to escape it. You never specify ε ; you specify the fraction of the data you are willing to treat as errors, and the geometry follows.

Verification artifact. checks/example-ch-svr-example-6-3.json records the example source hash and verification scope.

7.10 Parametric insensitivity models

A constant tube width assumes the noise has the same scale everywhere, which is rarely true: measurements are often noisier in some regions of input space than in others. The remedy is to let the width vary with x . Replace the constant ε by a flexible model $\sum_{q=1}^p \zeta_q g_q(x)$ built from a fixed set of positive functions g_q , and let the algorithm learn the coefficients ζ_q . The constraints become $\langle w, x_i \rangle + b - y_i \leq \sum_q \zeta_q g_q(x_i) + \xi_i$ and its mirror, so the tube can breathe with the local scale of the data.

Carrying this through the same Lagrangian gives a dual that is unchanged except for the budget constraint, which now reads, for each q ,

$$\sum_{i=1}^m (\alpha_i + \alpha_i^*) g_q(x_i) \leq C \zeta_q,$$

still linear in the multipliers. The ordinary ν -SVR is the special case $p = 1$, $g_1 \equiv 1$, which recovers the single budget $\sum_i (\alpha_i + \alpha_i^*) \leq C\nu$. With a shared width function on both edges, the computation of b and of the width stays a small linear solve, and a version of the ν -property continues to hold for each ζ_q . This is the natural tool for heteroscedastic noise, where the tube should be a fitted envelope rather than a constant band.

7.11 Applications

Two settings in the source show the method at work. On the Boston housing benchmark, ν -SVR with a Gaussian kernel matches the best results obtainable by tuning ϵ directly against the test set, across a wide range of ν ; crucially, the fitted fractions of errors and support vectors track ν as the proposition predicts, so ν is not just a knob but a readable summary of the fit. The second setting is time series prediction, where regression on lagged windows is a staple; there the sparsity of the expansion is a practical asset, since prediction cost scales with the number of support vectors, not with the size of the training set. In both cases the ability of ν to adapt the tube to the unknown noise level is what makes the method convenient in practice, and it is why one reaches for ν -SVR when the target accuracy is not known in advance.

A tempting idea, storing only the support vectors as a compressed encoding of the data, works less well than one might hope: for noisy high-dimensional problems fit to moderate accuracy, the number of support vectors can be large, so the compression is modest. The support vectors are the right summary of the *function*, not necessarily a small summary of the *data*. The generalization behavior behind these fits is the subject of Chapter 11, Learning Theory in RKHS Balls, and the choice of kernel k draws on the catalog of Chapter 19, Translation-Invariant, Semigroup, and Probabilistic Kernels.

7.12 Common mistakes and practical implications

For **Support Vector Regression**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

7.13 Summary and further reading

This chapter established explain the central definitions and claims in Support Vector Regression; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Vapnik 1995), (Vapnik 1998), (Drucker and Vapnik 1997).

7.14 Exercises

1. warm-up **The trough**. Sketch $|y - f|_\epsilon$ as a function of f for fixed y and $\epsilon = 0.5$. Mark the flat floor and the two ramps, and explain in one sentence why a point landing on the floor gets a zero dual coefficient.
2. computation **Redo the fit**. For the five points of the first worked example but with $\epsilon = 1$, find the flattest line whose tube contains all points, identify the support vectors, and give w and b . Compare the SV count to the $\epsilon = 0.5$ case. Hint

The tube of half-width 1 is wide enough that a flatter slope suffices; solve $\min \frac{1}{2}w^2$ subject to $|y_i - wx_i - b| \leq 1$. The binding points are again the two extremes.

7 Support Vector Regression

3. proof **Never both**. Show that at the optimum of the ε -SVR dual, $\alpha_i \alpha_i^* = 0$ for every i . Argue that if some solution had α_i, α_i^* both positive, subtracting $\min(\alpha_i, \alpha_i^*)$ from both leaves the objective no smaller and stays feasible. Hint

The difference $\alpha_i - \alpha_i^*$ and the tube constraints are unaffected by the subtraction, while the term $-\varepsilon(\alpha_i + \alpha_i^*)$ can only increase.

4. proof **Fewer slacks**. Prove geometrically that $\xi_i \xi_i^* = 0$, so a single slack per point suffices, and rederive the dual in the signed variable $\beta_i = \alpha_i - \alpha_i^*$, whose box is $-C/m \leq \beta_i \leq C/m$ (source Problem 9.2).
5. computation **Budget is active**. In the $\nu = 0.6$ row of the second example, verify by hand that $\sum_i |\beta_i| = C\nu$ and $\sum_i \beta_i = 0$, and recompute ε from the two in-bound support vectors using $\varepsilon = \frac{1}{2}[(y_i - g_i) - (y_j - g_j)]$.
6. exploration **Ridge in disguise**. Take the Gaussian-loss row with $\varepsilon = 0$ and show that the primal is kernel ridge regression from Chapter 4, Kernel Ridge Regression and Smooth Losses. Explain why every point becomes a support vector, and connect the loss of sparsity to the missing flat floor.
7. exploration **One-sided regression**. First show that a naive free-offset formulation seeking a flat function above all observations is degenerate: $w = 0$ and a sufficiently large unpenalized b has zero objective. Repair the formulation by fixing or penalizing the offset (equivalently, include the constant function in the penalized RKHS), keep only the slack family for observations that fall above the fitted function, derive the resulting one-sided dual, and relate its single multiplier family to ν -SVR with unequal budgets on the two edges (source Problem 9.4). Hint

Keep only the ξ_i^* family, or equivalently drive the upper multipliers to zero; the tube degenerates into a single supporting hyperplane.

8. challenge **Heteroscedastic tube**. With a two-function width model $g_1 \equiv 1, g_2(x) = x$, work out how the fitted envelope $\zeta_1 + \zeta_2 x$ tilts on a dataset whose noise grows with x , and state the modified budget constraints that the dual must satisfy. Hint

The dual budget splits into one linear constraint per g_q : $\sum_i (\alpha_i + \alpha_i^*) g_q(x_i) \leq C\zeta_q$. A growing envelope means $\zeta_2 > 0$.

9. computation **Median line**. Set $\tau = \frac{1}{2}$ in the quantile example. Show the box collapses to the symmetric $[0, \frac{C}{2m}]$ on both sides and the pinball loss becomes $\frac{1}{2}|u|$. The fit is $f(x) = 0.92x + 0.08$; check that it lies between the $\tau = 0.25$ and $\tau = 0.75$ lines, and name the row of the loss table it realizes. Hint

Equal slopes give the Laplacian loss $|u|$, so $\tau = \frac{1}{2}$ is a regularized least-absolute-deviations fit, the robust analogue of the mean-fitting ridge of Chapter 4, Kernel Ridge Regression and Smooth Losses.

10. proof **The quantile bracket**. From the dual equality $\sum_i (\alpha_i - \alpha_i^*) = 0$ and the asymmetric ceilings, prove that any quantile-SVR fit satisfies $\frac{\#below}{m} \leq \tau \leq \frac{\#below + \#on}{m}$, then verify both inequalities on the two levels of the worked example. Hint

A strictly-below point carries $\alpha_i^* = \frac{C}{m}(1 - \tau)$ and $\alpha_i = 0$; a strictly-above point carries $\alpha_i = \frac{C}{m}\tau$ and $\alpha_i^* = 0$. Split $\sum_i \alpha_i = \sum_i \alpha_i^*$ over the below, on, and above sets and use $\#below + \#on + \#above = m$.

8 One-Class SVMs and Novelty Detection

Every algorithm so far in this book has been handed labels: a target value to regress, a class to separate, a pair of views to correlate. This chapter drops the labels entirely. We are given only a sample of points from some distribution and asked a deceptively simple question: which region of space does this distribution actually occupy? Answering it lets us flag a future point as normal or novel, the task of novelty detection, without ever having seen an example of the novel class. The kernel machinery of the support vector machine turns out to solve this almost verbatim, in two guises that look different but coincide for the kernels we use most. We build the smallest enclosing hypersphere of Tax and Duin (2004), the origin-separating one-class SVM of Schölkopf, Platt, Shawe-Taylor, Smola and Williamson (2001), prove the beautiful ν -property that makes a single knob control the fraction of outliers, and show that for translation-invariant kernels the sphere and the hyperplane are the same machine.

8.1 Estimating the support of a distribution

Suppose points $x_1, \dots, x_m$ are drawn independently from an unknown distribution P on a space $\mathcal{X}$, and we want to summarize where P lives. The most ambitious answer would estimate the full density p , from which every downstream question could in principle be read off. But density estimation is hard, sometimes impossible: a density exists only when P is absolutely continuous, and estimating the measure of an arbitrary Borel set is not a solvable problem in general (Vapnik 1995). Novelty detection does not need any of that. It needs only a region, a set $S \subseteq \mathcal{X}$ that captures most of the mass of P , so that a test point falling outside S can be declared abnormal. This is Vapnik's principle in action: never solve a harder problem than the one you actually face.

The right formalization is a quantile. Fix a class $\mathcal{C}$ of candidate regions and a size functional λ (usually volume). For a target mass $\mu \in (0, 1]$, the quantile function returns the smallest region in $\mathcal{C}$ that still captures mass μ .

Definition (multi-dimensional quantile)

For a distribution P , a class $\mathcal{C}$ of measurable sets, and a size functional $\lambda : \mathcal{C} \rightarrow \mathbb{R}$, the *quantile function* is

$$U(\mu) = \inf\{\lambda(C) : P(C) \geq \mu, C \in \mathcal{C}\},$$

and $C(\mu)$ denotes a set attaining the infimum. With λ the Lebesgue measure, $C(\mu)$ is the *minimum-volume set* holding a fraction μ of the mass; as $\mu \rightarrow 1$ it approaches the support of P .

Replacing P by the empirical distribution $P_m^{\text{emp}}(C) = \frac{1}{m} \sum_i \mathbf{1}_C(x_i)$ gives an estimator, but a naive one is useless: an unconstrained $\mathcal{C}$ lets us wrap the region tightly around the exact training points, capturing every one while generalizing to nothing. As always, we need to restrict $\mathcal{C}$, and as always the restriction will be a smoothness penalty in a kernel feature space rather than a hard limit on shape. The two algorithms of this chapter implement exactly this: they fix the fraction of training points to capture and then find the smoothest region with that property, where smoothness is measured by a support-vector regularizer. Both connect back to support estimation via embeddings, and both, as we will see, are unlabeled cousins of the classifiers from the previous chapters.

8.2 The smallest enclosing hypersphere

The most concrete way to describe where the data lives is to draw a ball around it. Map the points into the feature space $\mathcal{H}$ of a kernel k through ϕ , and ask for the smallest sphere that contains all of the images. A test point is called novel when its image lands outside. Shawe-Taylor and Cristianini (2004) motivate this by two defects of a fixed-center ball: its center should be free to move to shrink the radius, and one unlucky point should not be able to inflate the radius arbitrarily. We first solve the hard problem, then soften it.

Write $\phi_i = \phi(x_i)$. The hard problem minimizes the squared radius subject to containment:

$$\min_{R \in \mathbb{R}, c \in \mathcal{H}} R^2 \quad \text{subject to} \quad \|\phi_i - c\|^2 \leq R^2, \quad i \in [m].$$

Introducing multipliers $\alpha_i \geq 0$ gives the Lagrangian $L = R^2 + \sum_i \alpha_i (\|\phi_i - c\|^2 - R^2)$. Setting $\partial L / \partial R = 2R(1 - \sum_i \alpha_i) = 0$ forces $\sum_i \alpha_i = 1$, and $\partial L / \partial c = -2 \sum_i \alpha_i (\phi_i - c) = 0$ gives the center as a convex combination of the data,

$$c = \sum_i \alpha_i \phi_i.$$

The center lies in the convex hull of the training images, hence in their span, so it has a dual representation and everything can be written with the kernel alone. Substituting back and using $\sum_i \alpha_i = 1$ collapses the Lagrangian to a function of α only.

Proposition (SVDD dual)

The squared radius of the smallest enclosing hypersphere equals the optimal value of

$$\max_{\alpha} \sum_i \alpha_i k(x_i, x_i) - \sum_{i,j} \alpha_i \alpha_j k(x_i, x_j) \quad \text{s.t.} \quad \sum_i \alpha_i = 1, \quad \alpha_i \geq 0.$$

Its center is $c = \sum_i \alpha_i \phi_i$, and a point x_i lies exactly on the sphere if and only if $\alpha_i > 0$. Such points are the support vectors.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Expanding $\|\phi_i - c\|^2 = k(x_i, x_i) - 2\langle \phi_i, c \rangle + \|c\|^2$ with $c = \sum_j \alpha_j \phi_j$ gives $\langle \phi_i, c \rangle = \sum_j \alpha_j k(x_i, x_j)$ and $\|c\|^2 = \sum_{j,l} \alpha_j \alpha_l k(x_j, x_l)$. Then

$$L = \sum_i \alpha_i \|\phi_i - c\|^2 = \sum_i \alpha_i k(x_i, x_i) - 2 \sum_{i,j} \alpha_i \alpha_j k(x_i, x_j) + \sum_{j,l} \alpha_j \alpha_l k(x_j, x_l),$$

and the last two sums combine, using $\sum_i \alpha_i = 1$, into a single $-\sum_{i,j} \alpha_i \alpha_j k(x_i, x_j)$, which is the stated objective.

The Karush-Kuhn-Tucker complementarity condition $\alpha_i (\|\phi_i - c\|^2 - R^2) = 0$ forces $\|\phi_i - c\| = R$ whenever $\alpha_i > 0$, so every support vector sits on the boundary. $\square$

The objective is concave (its Hessian is minus twice the kernel matrix, which is positive semidefinite), so the problem is a convex quadratic program with a unique optimal value and no spurious local minima. The radius is recovered from any support vector x_i by $R^2 = \|\phi_i - c\|^2 = k(x_i, x_i) - 2 \sum_j \alpha_j k(x_i, x_j) + \sum_{j,l} \alpha_j \alpha_l k(x_j, x_l)$, and the containment test for a new point x compares $\|\phi(x) - c\|^2$ against R^2 .

Example (smallest enclosing ball of five points)

Take a linear kernel $k(x, x') = \langle x, x' \rangle$, so the feature space is the input plane and the ball is an ordinary disc. The five points are the corners of a square and its center:

$$x_1 = (0, 0), x_2 = (2, 0), x_3 = (2, 2), x_4 = (0, 2), x_5 = (1, 1).$$

The diagonal of the Gram matrix is $k(x_i, x_i) = \|x_i\|^2 = (0, 4, 8, 4, 2)$.

1. Solve the dual. Maximizing $\sum_i \alpha_i \|x_i\|^2 - \|\sum_i \alpha_i x_i\|^2$ under $\sum_i \alpha_i = 1, \alpha_i \geq 0$ returns $\alpha = (0.25, 0.25, 0.25, 0.25, 0)$. The four corners share the weight equally; the center point gets none.
2. Read off the center. $c = \sum_i \alpha_i x_i = 0.25 [(0, 0) + (2, 0) + (2, 2) + (0, 2)] = (1, 1)$, a convex combination of the corners.
3. Read off the radius. The optimal dual value is $\sum_i \alpha_i \|x_i\|^2 - \|c\|^2 = 4 - 2 = 2$, so $R^2 = 2$ and $R = \sqrt{2} \approx 1.4142$.
4. Classify the points. The squared distances to c are $(2, 2, 2, 2, 0)$. The four corners sit exactly on the boundary ($\alpha_i > 0$, support vectors); the center x_5 is strictly inside with $\alpha_5 = 0$.

Reading. The dual weights are supported entirely on the boundary. Four points define the disc and the fifth is redundant, exactly the sparsity that makes the solution cheap to store and to test against.

Verification artifact. checks/example-ch-oneclass-example-7-1.json records the example source hash and verification scope.

8.2.1 The soft hypersphere

Requiring the ball to swallow every point makes it hostage to a single outlier. The fix is the same slack device used for soft-margin classification: let points fall outside, but pay for it. With slack variables $\xi_i \geq 0$ and a trade-off constant C ,

$$\min_{R, c, \xi} R^2 + C \sum_i \xi_i \quad \text{s.t.} \quad \|\phi_i - c\|^2 \leq R^2 + \xi_i, \xi_i \geq 0.$$

The derivation is unchanged except that the multiplier on the slack constraint, $\beta_i = C - \alpha_i \geq 0$, now caps each dual variable at C . The dual is the SVDD objective above with the box constraint $0 \leq \alpha_i \leq C$ in place of $\alpha_i \geq 0$. A point with nonzero slack lies outside the sphere and, by complementarity, sits at the ceiling $\alpha_i = C$; the radius is still read from any non-bound support vector, one with $0 < \alpha_i < C$.

Setting $C = 1/(\nu\ell)$ reparametrizes the trade-off in the way that will matter below. Since $\sum_i \alpha_i = 1$ and each $\alpha_i \leq 1/(\nu\ell)$, the constant $\nu \in (0, 1]$ turns out to bound the fraction of excluded points directly. We collect the resulting procedure.

Algorithm (smallest enclosing hypersphere, SVDD)

Input Kernel matrix $K_{ij} = k(x_i, x_j)$; parameter $\nu \in (0, 1]$ (use $\nu = 0$, i.e. $\alpha_i \geq 0$, for the hard sphere).

Output Center weights α , radius R , and a test $f(x)$ that flags novelty.

1. Solve the QP $\max_{\alpha} \sum_i \alpha_i K_{ii} - \sum_{i,j} \alpha_i \alpha_j K_{ij}$ subject to $\sum_i \alpha_i = 1$ and $0 \leq \alpha_i \leq 1/(\nu\ell)$.
2. Pick any non-bound support vector x_s with $0 < \alpha_s < 1/(\nu\ell)$.
3. Set $R^2 = K_{ss} - 2 \sum_j \alpha_j K_{js} + \sum_{i,j} \alpha_i \alpha_j K_{ij}$.
4. Return $f(x) = \text{sgn}(k(x, x) - 2 \sum_i \alpha_i k(x_i, x) + \sum_{i,j} \alpha_i \alpha_j K_{ij} - R^2)$, which is positive exactly when x is novel.

8.2.2 SVDD with negative examples

So far the only information was a sample of normal points. Sometimes a few examples of the novel class are also at hand: confirmed frauds in a transaction log, known faults on a production line, a handful of images from outside the target category. Discarding them would be wasteful, yet they are far too few and too unrepresentative to train a balanced two-class classifier against. Tax and Duin (2004) fold them into the sphere directly. The ball should still wrap the normal points, the *targets*, but now it must also draw its boundary in so as to leave the labelled outliers, the *negatives*, on the outside.

Write $y_i = +1$ for a target and $y_i = -1$ for a negative. Targets want to sit inside the sphere and negatives outside it, so the two families carry containment constraints that point in opposite directions. With separate penalties C_1, C_2 for the two kinds of violation,

$$\min_{R, c, \xi} R^2 + C_1 \sum_{i: y_i=+1} \xi_i + C_2 \sum_{l: y_l=-1} \xi_l \quad \text{s.t.} \quad \begin{cases} \|\phi_i - c\|^2 \leq R^2 + \xi_i, & y_i = +1, \\ \|\phi_l - c\|^2 \geq R^2 - \xi_l, & y_l = -1, \end{cases} \quad \xi \geq 0.$$

The target constraint is the soft-sphere constraint from before; the negative constraint is its mirror image, demanding a squared distance of at least R^2 rather than at most. Carrying the derivation through, the dual keeps the exact shape of the SVDD dual but signs every negative.

Proposition (SVDD dual with negatives)

With multipliers $\alpha_i \geq 0$ the smallest sphere containing the targets and excluding the negatives solves

$$\max_{\alpha} \sum_i y_i \alpha_i k(x_i, x_i) - \sum_{i,j} y_i y_j \alpha_i \alpha_j k(x_i, x_j) \quad \text{s.t.} \quad \sum_i y_i \alpha_i = 1, \quad 0 \leq \alpha_i \leq C_{y_i},$$

where C_{y_i} is C_1 for a target and C_2 for a negative. The center is $c = \sum_i y_i \alpha_i \phi_i$, a signed combination in which each negative pulls the center away from itself, and any non-bound point $0 < \alpha_i < C_{y_i}$ lies exactly on the sphere.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Form the Lagrangian with $\alpha_i, \gamma_i \geq 0$,

$$L = R^2 + C_1 \sum_{y_i=+1} \xi_i + C_2 \sum_{y_l=-1} \xi_l - \sum_{y_i=+1} \alpha_i (R^2 + \xi_i - \|\phi_i - c\|^2) - \sum_{y_l=-1} \alpha_l (\|\phi_l - c\|^2 - R^2 + \xi_l) - \sum_i \gamma_i \xi_i.$$

Then $\partial L / \partial R = 2R(1 - \sum_{y_i=+1} \alpha_i + \sum_{y_l=-1} \alpha_l) = 0$ gives $\sum_i y_i \alpha_i = 1$, and $\partial L / \partial c = 0$ gives $c = \sum_{y_i=+1} \alpha_i \phi_i - \sum_{y_l=-1} \alpha_l \phi_l = \sum_i y_i \alpha_i \phi_i$. The slack derivatives give $\alpha_i = C_{y_i} - \gamma_i$, hence $0 \leq \alpha_i \leq C_{y_i}$. Substituting c and expanding $\|\phi_i - c\|^2$ exactly as in the hard sphere, now with the signs carried through $\langle \phi_i, c \rangle = \sum_j y_j \alpha_j k(x_i, x_j)$ and $\|c\|^2 = \sum_{i,j} y_i y_j \alpha_i \alpha_j k(x_i, x_j)$, collapses L to the stated objective under $\sum_i y_i \alpha_i = 1$. Complementarity forces $\|\phi_i - c\|^2 = R^2$ at every point with $0 < \alpha_i < C_{y_i}$, so each non-bound support vector, target or negative, sits on the boundary. $\square$

The reading is that the negatives enter every formula with a flipped sign. The center $c = \sum_i y_i \alpha_i \phi_i$ is an affine rather than a convex combination, so a labelled outlier repels the center instead of attracting it, and the radius adjusts just enough to leave it out. The decision function keeps its form, $f(x) = \text{sgn}(k(x, x) - 2 \sum_i y_i \alpha_i k(x_i, x) + \sum_{i,j} y_i y_j \alpha_i \alpha_j k(x_i, x_j) - R^2)$, positive on the novel side.

This is more than a cosmetic sign change: it is the bridge to ordinary two-class classification. The quadratic term $\sum_{i,j} y_i y_j \alpha_i \alpha_j k(x_i, x_j)$ is exactly the support vector machine's dual form. When the kernel is normalized so that $k(x, x) = \kappa(0)$ is constant, the linear term $\sum_i y_i \alpha_i k(x_i, x) = \kappa(0) \sum_i y_i \alpha_i = \kappa(0)$ is frozen on the feasible

set, and maximizing what remains is the same as minimizing $\frac{1}{2} \sum_{i,j} y_i y_j \alpha_i \alpha_j k(x_i, x_j)$ subject to $\sum_i y_i \alpha_i = 1$ and $0 \leq \alpha_i \leq C$. That is the two-class SVM dual, differing only in that the equality constraint reads $\sum_i y_i \alpha_i = 1$ rather than $= 0$, because the radius is free where the classifier's bias is free. SVDD with negatives thus interpolates between the two worlds: with no negatives it is the one-class sphere, and as the negatives grow into a full second sample it becomes the maximum-margin classifier.

Example (one negative pushes the ball)

A linear kernel $k(x, x') = \langle x, x' \rangle$, so the ball is a disc. Three targets and one negative:

$$x_1 = (-2, 0), x_2 = (2, 0), x_3 = (0, 1) \text{ (} y = +1 \text{)}, \quad x_4 = (0, -1) \text{ (} y = -1 \text{)}.$$

The Gram diagonal is $k(x_i, x_i) = \|x_i\|^2 = (4, 4, 1, 1)$.

1. Fit the targets alone. The plain SVDD on x_1, x_2, x_3 returns $\alpha = (0.5, 0.5, 0)$, center $c = (0, 0)$, and $R^2 = 4$, so $R = 2$. The two far targets are diametrically opposite and pin the disc; x_3 is interior.
2. Locate the negative. Its squared distance to that center is $\|x_4 - c\|^2 = 1 < 4 = R^2$, so the negative sits *inside* the target-only ball. Ignored, it would be accepted as normal.
3. Refit with the negative. The signed dual returns $\alpha = (1.25, 1.25, 0, 1.5)$, which meets $\sum_i y_i \alpha_i = 1.25 + 1.25 + 0 - 1.5 = 1$. The center moves to $c = \sum_i y_i \alpha_i x_i = (0, 1.5)$, lifted away from the outlier below.
4. Read the new sphere. The optimal value is $R^2 = 6.25$, so $R = 2.5$. The squared distances to the new center are $(6.25, 6.25, 0.25, 6.25)$: the two far targets and the negative all sit exactly on the boundary, while x_3 is strictly inside. The negative now lands on the sphere and is excluded.

Reading. One labelled outlier moved the center from $(0, 0)$ to $(0, 1.5)$ and grew the radius from 2 to 2.5, just enough to expel it. The negative carries weight $\alpha_4 = 1.5$ with a minus sign, the only new ingredient, and it earns a place on the boundary exactly as a support vector does.

Verification artifact. checks/example-ch-oneclass-example-7-2.json records the example source hash and verification scope.

8.3 Separating the data from the origin

Schölkopf, Platt, Shawe-Taylor, Smola and Williamson (2001) reach the same goal by a different picture. Instead of wrapping a ball around the data, push a hyperplane between the data and the origin, as far from the origin as possible. Whatever lands on the origin side of the plane is novel. The strategy is a direct transcription of the maximum-margin idea: the training images should sit on the far side by a margin $\rho/\|w\|$, and a small $\|w\|$ means a large margin of separation from the origin.

With slack variables and the ν -parametrization the primal is

$$\min_{w \in \mathcal{H}, \rho \in \mathbb{R}, \xi} \frac{1}{2} \|w\|^2 + \frac{1}{\nu m} \sum_i \xi_i - \rho \quad \text{s.t.} \quad \langle w, \phi_i \rangle \geq \rho - \xi_i, \xi_i \geq 0.$$

The decision function is $f(x) = \text{sgn}(\langle w, \phi(x) \rangle - \rho)$: it returns +1 inside the estimated region and -1 outside. Forming the Lagrangian with multipliers $\alpha_i, \beta_i \geq 0$ and differentiating gives the support-vector expansion $w = \sum_i \alpha_i \phi_i$, the equality $\sum_i \alpha_i = 1$ (from $\partial/\partial \rho$), and $\alpha_i = 1/(\nu m) - \beta_i$, so $0 \leq \alpha_i \leq 1/(\nu m)$. Substituting yields a dual that is pure quadratic in α , with no linear term.

Algorithm (ν -one-class SVM)

Input Kernel matrix $K_{ij} = k(x_i, x_j)$; parameter $\nu \in (0, 1]$.

Output Dual weights α , offset ρ , and decision function f .

1. Solve the QP $\min_{\alpha} \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j K_{ij}$ subject to $\sum_i \alpha_i = 1$ and $0 \leq \alpha_i \leq 1/(\nu m)$.
2. Pick any non-bound support vector x_s with $0 < \alpha_s < 1/(\nu m)$.
3. Set $\rho = \sum_j \alpha_j K_{js}$, the value $\langle w, \phi_s \rangle$ where the constraint is tight.
4. Return $f(x) = \text{sgn}(\sum_i \alpha_i k(x_i, x) - \rho)$; points with $f(x) < 0$ are novel.
5. Repeat steps 1 to 3 with a sequential-minimal-optimization solver (Platt 1998), optimizing over pairs (α_i, α_j) to respect $\sum_i \alpha_i = 1$, until no KKT violation exceeds the tolerance.

The single equality constraint $\sum_i \alpha_i = 1$ is what makes the pairwise SMO update natural: one cannot change a single α_i without breaking the sum, so the smallest legal step moves two at once. This is the same structure as the C-SVM dual of the previous chapter, which is why the classifier's solver carries over almost unchanged (Schölkopf and Smola 2002). The same ν -parametrization reappears in nu-support vector regression, where ν controls the fraction of points outside the regression tube.

There is a revealing special case. If the kernel is normalized as a density, such as a Gaussian, and we force $\nu \rightarrow$ its extreme so that the box constraint $\alpha_i \leq 1/(\nu m)$ becomes $\alpha_i \leq 1/m$, then the equality $\sum_i \alpha_i = 1$ admits only the uniform solution $\alpha_i = 1/m$. The decision function $f(x) = \text{sgn}(\frac{1}{m} \sum_i k(x_i, x) - \rho)$ is then a thresholded Parzen-windows density estimate. For smaller ν the same expansion survives but only a subset of the training points carry weight: the one-class SVM is a sparse, thresholded density estimator that spends its budget only on the points that pin down the boundary.

8.4 The nu-property

The parameter ν is not just a knob to be tuned by trial and error; it has a precise meaning. Call a training point an *outlier* when it falls on the novel side of the solution, that is when $\xi_i > 0$. The following proposition, the analogue for one-class problems of the ν -property for classifiers (Schölkopf and Smola 2000), pins ν between two observable fractions.

Proposition (ν -property)

Assume the solution of the ν -one-class SVM satisfies $\rho > 0$. Then: (i) ν is an upper bound on the fraction of outliers; (ii) ν is a lower bound on the fraction of support vectors; (iii) if the data are drawn from a distribution without discrete components and the kernel is analytic and nonconstant, then with probability one, asymptotically, ν equals both fractions.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof (of (i) and (ii))

By KKT complementarity, a point with positive slack $\xi_i > 0$ has its slack multiplier $\beta_i = 0$, hence $\alpha_i = 1/(\nu m) - \beta_i = 1/(\nu m)$, the upper bound. So every outlier sits at the ceiling. Because $\sum_i \alpha_i = 1$ and each term is at most $1/(\nu m)$, the number of points at the ceiling is at most νm ; since outliers are a subset of these, the fraction of outliers is at most ν , proving (i). For (ii), $\sum_i \alpha_i = 1$ with every $\alpha_i \leq 1/(\nu m)$ cannot be met by fewer than νm nonzero terms, so at least νm points have $\alpha_i > 0$; these are the support vectors, and their fraction is at least ν . $\square$

Example (a nu-sweep and the sandwich)

Ten points on a line, a cluster with two stragglers,

$$x = (0, 0.3, 0.6, 0.9, 1.2, 1.5, 1.8, 2.1, 3.5, 4.2),$$

with a Gaussian kernel $k(x, x') = e^{-(x-x')^2/c}$, width $c = 1$. Solve the ν -one-class SVM for three values of ν and count outliers and support vectors.

1. Sweep the parameter. The solver returns the following, where each point deemed an outlier sits at the ceiling $\alpha_i = 1/(\nu m)$:

ν	$1/(\nu m)$	ρ	frac. SVs	frac. outliers
0.2	0.500	0.3095	0.60	0.00
0.4	0.250	0.3126	0.60	0.10
0.5	0.200	0.3244	0.70	0.30

2. Check the sandwich. In every row the fraction of outliers is at most ν and the fraction of support vectors is at least ν : $0 \leq 0.2 \leq 0.6$, then $0.1 \leq 0.4 \leq 0.6$, then $0.3 \leq 0.5 \leq 0.7$.
3. Watch the boundary tighten. At $\nu = 0.2$ no point is excluded. Raising ν lowers the ceiling $1/(\nu m)$, forcing more weight to the bound: at $\nu = 0.4$ the leftmost point $x = 0.0$ becomes an outlier, and at $\nu = 0.5$ the extremes $x = 0.0, 2.1$ and the straggler $x = 4.2$ are excluded.

Reading. One number ν squeezes the answer from both sides. It is an upper bound on how much data you throw away and a lower bound on how many points define the region, so setting ν is the same as declaring, in advance, the outlier rate you are willing to tolerate.

Verification artifact. checks/example-ch-oneclass-example-7-3.json records the example source hash and verification scope.

8.4.1 Connection to classification and robustness

The origin-separating view exposes a clean link to ordinary two-class learning. Reflect the data through the origin to form the labeled set $\{(\phi_i, +1)\} \cup \{(-\phi_i, -1)\}$. By symmetry the optimal separating hyperplane for this set passes through the origin, and it is exactly the supporting hyperplane of the one-class problem (Schölkopf and Smola 2002). Novelty detection is thus the maximum-margin separation of the data from its own mirror image, which is why the whole apparatus of margins, support vectors, and SMO transfers.

A second dividend is robustness. Because outliers already sit at the ceiling $\alpha_i = 1/(\nu m)$, nudging one of them further into the novel region changes nothing.

Proposition (resistance)

Local movements of an outlier in the direction of w do not change the supporting hyperplane.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Let x_o be an outlier, so $\xi_o > 0$ and $\alpha_o = 1/(\nu m)$. Translate its image to $\phi_o + \delta w$ with $\delta > 0$. Its constraint value $\langle w, \phi_o + \delta w \rangle = \langle w, \phi_o \rangle + \delta \|w\|^2$ increases, so the slack it needs, $\xi'_o = \rho - \langle w, \phi_o + \delta w \rangle$, stays nonnegative and the point remains an outlier with α_o still at the ceiling. Every KKT condition still holds with the same (w, ρ) , so the previous solution is still optimal and the hyperplane is unchanged. $\square$

Contrast this with the hard enclosing sphere, whose radius any single far point can inflate without limit. The ν -softening is precisely what buys the resistance.

8.5 Hypersphere equals hyperplane for RBF kernels

We now have two algorithms, a ball and a plane, that seem to describe the data differently. For the translation-invariant kernels that dominate practice, they are the same machine.

Theorem (equivalence)

If the kernel is translation invariant, $k(x, x') = \kappa(x - x')$, so that $k(x, x) = \kappa(0)$ is a constant, then the soft SVDD dual and the ν -one-class SVM dual have the same optimizers and induce the same decision boundary.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

In the SVDD objective the linear term is $\sum_i \alpha_i k(x_i, x_i) = \kappa(0) \sum_i \alpha_i = \kappa(0)$, a constant on the feasible set $\sum_i \alpha_i = 1$. Dropping it, maximizing $-\sum_{i,j} \alpha_i \alpha_j k(x_i, x_j)$ is the same as minimizing $\frac{1}{2} \sum_{i,j} \alpha_i \alpha_j k(x_i, x_j)$, which is the one-class SVM objective, under the identical constraints $\sum_i \alpha_i = 1$, $0 \leq \alpha_i \leq 1/(\nu \ell)$. Hence the optimal α coincide. For the boundary, the SVDD test compares $\|\phi(x) - c\|^2 = \kappa(0) - 2 \sum_i \alpha_i k(x_i, x) + \|c\|^2$ against R^2 ; since $\kappa(0)$ and $\|c\|^2$ are constants, this is a threshold on $\sum_i \alpha_i k(x_i, x)$, the same quantity thresholded by the hyperplane test $\langle w, \phi(x) \rangle \geq \rho$. The two decision functions agree. $\square$

Geometrically the reason is vivid. A translation-invariant kernel with $\kappa(0) = 1$ maps every point onto the unit sphere of feature space, since $\|\phi(x)\|^2 = k(x, x) = 1$. All images lie on one sphere, so the smallest ball enclosing them cuts off a spherical cap, and the plane that slices off the smallest cap is exactly the hyperplane with maximum margin from the origin. On the round sphere, the tightest ball and the farthest plane trace the same circle. This is why the Gaussian and other RBF kernels let one move freely between the two formulations, and why a Gaussian kernel always makes the data separable from the origin: all images sit in a single orthant of unit vectors.

8.6 Density level sets and the Parzen connection

The equivalence just proved says that for an RBF kernel the whole decision reduces to thresholding a single function of the test point, $g(x) = \sum_i \alpha_i k(x_i, x)$. Written out, g is a sum of identical bumps, one centered on each training point, weighted by α_i . That is the very shape of a kernel density estimate, and following the thread back to a classical estimator makes the meaning of the one-class boundary exact: it is a contour of an estimated density.

The classical estimator is the Parzen window (Parzen 1962). Given a normalized window k_h of bandwidth h , the density estimate at a point is the average height of the windows centered on the data,

$$\hat{p}_h(x) = \frac{1}{m} \sum_{i=1}^m k_h(x - x_i), \quad k_h(u) = \frac{1}{(2\pi h^2)^{d/2}} e^{-\|u\|^2/2h^2}$$

for the Gaussian window, and under the usual schedule $h \rightarrow 0$ with $mh^d \rightarrow \infty$ it converges to the true density p . Now compare it with the one-class decision. In the uniform limit found above, where the box constraint pins every $\alpha_i = 1/m$, the machine tests $f(x) = \text{sgn}(\frac{1}{m} \sum_i k(x_i, x) - \rho)$. The bare Gaussian kernel $k(x, x') = e^{-\|x-x'\|^2/2h^2}$ is the Parzen window stripped of its constant $Z = (2\pi h^2)^{-d/2}$, so $\hat{p}_h(x) = Z \cdot \frac{1}{m} \sum_i k(x_i, x)$, and the accepted region is

$$\{x : f(x) \geq 0\} = \left\{x : \frac{1}{m} \sum_i k(x_i, x) \geq \rho\right\} = \{x : \hat{p}_h(x) \geq Z\rho\}.$$

The normal region is a *super-level set* of the Parzen density, and its boundary $\{x : \hat{p}_h(x) = Z\rho\}$ is a density contour. The offset ρ is a density threshold in disguise.

This is the precise sense in which support estimation is level-set estimation. Recall the minimum-volume set $C(\mu)$ of the first section, the smallest region holding mass μ . For a density p with no flat plateau of positive measure, that minimum-volume set is exactly a super-level set $\{p \geq \tau\}$, with τ fixed so that the set carries mass μ : among all regions of a given probability, the one of least volume is bounded by a density contour, since trading any interior sliver for an equal-probability sliver at higher density shrinks the volume. Estimating the support at level μ is therefore estimating the level set $\{p \geq \tau\}$, and the one-class SVM does this by thresholding the plug-in estimate $\hat{p}_h$. The general, sparse solution is the same estimator run economically: in place of the flat weights $1/m$ that rebuild the full Parzen sum, the optimized α_i concentrate on the points near the contour, spending the representation only where the level set actually bends.

That the substitution is legitimate, not merely suggestive, is the content of Vert and Vert (2006). They prove that the Gaussian one-class SVM, with bandwidth and regularization sent to zero at appropriate rates, is a consistent estimator of density level sets, and they trace exactly how its solution relates to the Parzen estimate it thresholds. So the one-class SVM inherits the guarantees of kernel density estimation while keeping the sparsity and the convex program of the support vector machine. Seen through the mean embedding, the thresholded quantity is the empirical kernel mean $\frac{1}{m} \sum_i k(x_i, \cdot)$, which is why the Gaussian and its relatives, the standard Parzen windows, are also the standard one-class kernels.

Example (a Parzen level set is the one-class boundary)

Four points on a line, a cluster of three with one straggler,

$$x = (0, 1, 2, 5),$$

a Gaussian window of bandwidth $h = 1$, so $Z = 1/(\sqrt{2\pi}h) = 0.3989$. Estimate the density, threshold it, and match the boundary to the one-class test.

1. Estimate the density. The Parzen estimate $\hat{p}_h(x) = \frac{1}{m} \sum_i Z e^{-(x-x_i)^2/2}$ at the four points is (0.1737, 0.2208, 0.1748, 0.1009). The cluster rides a ridge near 0.17 to 0.22; the lone point at $x = 5$ drops to 0.1009.
2. Threshold it. Take the level $\tau = 0.15$. The three cluster points clear it and are accepted; the straggler at $x = 5$, with $\hat{p}_h = 0.1009 < 0.15$, falls below and is flagged novel, an empirical outlier fraction of $1/4 = 0.25$.
3. Trace the level set. Solving $\hat{p}_h(x) = 0.15$ on the line gives two crossings, $x = -0.2527$ and $x = 2.2773$. The accepted region is the single interval $[-0.2527, 2.2773]$, the super-level set $\{\hat{p}_h \geq 0.15\}$, whose two endpoints are the density contour.
4. Match the one-class test. The one-class boundary $\frac{1}{m} \sum_i k(x_i, x) = \rho$ with $\rho = \tau/Z = 0.376$ crosses at $x = -0.2527$ and $x = 2.2773$, identical to the density contour. Thresholding $\hat{p}_h$ and thresholding the kernel sum draw the same boundary.

Reading. The one-class decision is a thresholded Parzen estimate to the last digit: the offset $\rho = 0.376$ is the density level $\tau = 0.15$ divided by the window constant Z , and the estimated support is the density super-level set $\{\hat{p}_h \geq 0.15\}$. Novelty detection is level-set estimation.

Verification artifact. checks/example-ch-oneclass-example-7-4.json records the example source hash and verification scope.

8.7 Uniqueness, generalization, and experiments

Two theoretical guarantees make the method trustworthy. The first is that the solution is well defined.

Proposition (supporting hyperplane)

If the data set X is separable from the origin, there is a unique hyperplane that separates all of the data from the origin and whose distance to the origin is maximal, given by $\min_w \frac{1}{2} \|w\|^2$ subject to $\langle w, \phi_i \rangle \geq 1$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Separability means the origin is not in the convex hull of the images, so by the supporting-hyperplane theorem a separating hyperplane exists, and rescaling w lets us demand $\langle w, \phi_i \rangle \geq 1$ for all i . Among all such w the distance from the plane $\{z : \langle w, z \rangle = 1\}$ to the origin is $1/\|w\|$, which is maximized by minimizing $\|w\|$ over the convex feasible set. A strictly convex objective on a convex set has a unique minimizer. $\square$

The second guarantee bounds how often a fresh point escapes the region. There is no margin between two classes here, since there is only one, but we can still leave a safety margin γ and control the mass falling outside a region slightly larger than the estimated one. The stability analysis of Shawe-Taylor and Cristianini (2004) for the soft sphere gives, with probability at least $1 - \delta$, a bound on the probability that a point is flagged novel of the form

$$P(f(x) = 1) \leq \frac{1}{\gamma \ell} \|\xi^*\|_1 + \frac{6R^2}{\gamma \sqrt{\ell}} + 3\sqrt{\frac{\ln(2/\delta)}{2\ell}},$$

where R bounds the radius of the data in feature space and $\|\xi^*\|_1$ is the total slack. The one-class bound of Schölkopf and Smola (2002) has the same shape: it shrinks as the margin $\rho/\|w\|$ grows, which is exactly why minimizing $\|w\|^2$ is the right regularizer, and it suggests reporting a slightly reduced offset $\rho - \gamma$ rather than the raw ρ , so the guarantee applies to the region actually used.

The combinatorial version of the problem, fixing ν and finding the truly minimal region excluding exactly a ν -fraction, is intractable: Ben-David, Eiron and Simon (2002) show that even approximating the densest region of fixed radius is NP-hard. The ν -one-class SVM sidesteps this by solving a convex relaxation, trading the exact minimal region for a smooth one that a quadratic program can find in polynomial time.

On real data the theory bears out. Schölkopf, Platt, Shawe-Taylor, Smola and Williamson (2001) trained the one-class SVM on the digit 0 of the USPS handwritten-digit set with a Gaussian kernel. At $\nu = 50\%$ the machine captured a tight description of "0-ness," recognizing 44% of test zeros with zero false positives on the other nine digits it had never seen. Lowering ν to 5% relaxed the region and lifted true-digit recognition to 91% at a modest 7% false-positive rate. Turned on the full test set as a pure novelty detector, the algorithm surfaced exactly the mislabeled and malformed digits as its highest-scoring outliers, and across a sweep of ν the measured fractions of outliers and support vectors tracked ν from below and above as the proposition promises, confirming that the single parameter does the job it was designed for.

8.8 Common mistakes and practical implications

For **One-Class SVMs and Novelty Detection**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

8.9 Summary and further reading

This chapter established explain the central definitions and claims in One-Class SVMs and Novelty Detection; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it

to a new setting. For primary and extended treatments, consult (Schölkopf and Williamson 2001), (Tax and Duin 2004), (Schölkopf and Bartlett 2000).

8.10 Exercises

1. warm-up **Center in the hull.** Show directly from $\sum_i \alpha_i = 1$ and $\alpha_i \geq 0$ that the SVDD center $c = \sum_i \alpha_i \phi_i$ lies in the convex hull of the training images, and conclude it lies in their span.
2. warm-up **Reading the radius.** For the five-point example, verify by hand that $\|x_2 - c\|^2 = 2$ with $c = (1, 1)$, and confirm that using any of the four corners to compute R^2 gives the same value.
3. warm-up **Parzen limit.** Show that when the box constraint forces $\alpha_i \leq 1/m$, the only feasible point of the one-class dual is $\alpha_i = 1/m$, and hence that the decision function reduces to a thresholded Parzen-windows estimate.
4. computation **Soft-sphere dual.** Starting from the soft SVDD primal $\min_{R,c,\xi} R^2 + C \sum_i \xi_i$ subject to $\|\phi_i - c\|^2 \leq R^2 + \xi_i$, $\xi_i \geq 0$, derive the dual and show the only change from the hard case is the box constraint $0 \leq \alpha_i \leq C$.
5. computation **Positivity of the offset.** Argue that the one-class SVM solution has $\rho > 0$ whenever the data are separable from the origin, and explain where the assumption $\rho > 0$ is used in the proof of the ν -property. Hint: relate ρ to the margin $\rho/\|w\|$ and use Proposition (supporting hyperplane).
6. computation **Hard-margin blow-up.** Show that forcing $\rho \geq 0$ in the primal changes the equality constraint from $\sum_i \alpha_i = 1$ to $\sum_i \alpha_i \geq 1$, and argue geometrically that the hard-margin limit $\nu \rightarrow 0$ can become infeasible while the free-offset problem stays feasible. Hint: a large negative ρ always satisfies the constraints.
7. challenge **Prove the equivalence in the decision function.** For a translation-invariant kernel with $\kappa(0) = 1$, show that the SVDD test $\|\phi(x) - c\|^2 \geq R^2$ and the one-class test $\langle w, \phi(x) \rangle \geq \rho$ define the same boundary, and express the SVDD threshold R^2 in terms of ρ and constants. Hint: expand $\|\phi(x) - c\|^2$ and collect the terms independent of x .
8. challenge **Separation from a reference set.** Modify the one-class SVM to separate the data not from the origin but from the mean $\frac{1}{t} \sum_{n=1}^t \phi(z_n)$ of a second set $\{z_n\}$. Derive the dual and show the objective gains a linear term $-q_i$ with $q_i = \frac{1}{t} \sum_n k(x_i, z_n)$, so that a weak model of the "other" class can be folded in. Hint: replace $\langle w, \phi_i \rangle$ by $\langle w, \phi_i - \bar{\phi}_z \rangle$ in the margin constraint.
9. computation **A negative that does not matter.** In the SVDD-with-negatives example, move the negative to $x_4 = (0, -3)$, well below the targets. Show that the target-only ball (center $(0, 0)$, $R = 2$) already leaves it outside, so its dual weight vanishes, $\alpha_4 = 0$, and the solution collapses to the plain three-target SVDD. Explain through complementarity why a negative that is already outside the target ball carries no weight, and contrast this with the original negative at $(0, -1)$.
10. challenge **Minimum-volume sets are level sets.** Let p be a density on $\mathbb{R}$ with no interval of constant positive value. Show that among all measurable sets C with $\int_C p = \mu$, the one of least Lebesgue measure is a super-level set $\{p \geq \tau\}$ for some $\tau \geq 0$. Conclude that the estimated support returned by the RBF one-class SVM is, in the population limit, a density level set. Hint: if C omits a point where p is high while including one where p is lower, swap two slivers of equal probability and compare their volumes.

9 Ranking and Ordinal Regression

A search engine does not need to know how relevant each page is on some absolute scale; it needs to put the best page first. A recommender does not have to predict a star rating exactly; it has to order the films you have not seen so the one you would love sits at the top. These are ranking problems, and they are subtly different from the classification and regression tasks of the previous chapters: the object we want to learn is an ordering, not a label and not a number. This chapter builds ranking on the kernel machinery already in hand. The key move is to reduce a preference between two items to a single classification on their difference vector, which turns the whole apparatus of the support vector machine loose on orderings. We derive the soft-margin ranking SVM and its dual, a perceptron-style online ranking rule, the threshold model of ordinal regression that partitions the real line into ranks, and the exact identity that ties the number of misordered pairs to the area under the ROC curve.

9.1 The ranking problem

In a ranking problem the training data comes with a relative ordering rather than an isolated target. Following the setup of Shawe-Taylor and Cristianini (2004), we are given instance and rank pairs (x_i, y_i) , where each instance lives in an implicit kernel-defined feature space through a map ϕ , and each rank y_i is drawn from a finite set Y carrying a total order. We say x_i is preferred over x_j , written $x_i > x_j$, when $y_i > y_j$; the two items are incomparable when $y_i = y_j$. The order on Y therefore induces a partial order on the instances that partitions them into equivalence classes, one per rank.

The goal is a ranking rule, a map $r : X \rightarrow Y$ that assigns each instance a rank consistent with the preferences seen in training and, we hope, with the preferences of unseen data. Two features of the problem distinguish it from ordinary classification. First, the labels are not exchangeable: confusing rank 5 with rank 4 is a milder error than confusing rank 5 with rank 1, because the labels themselves are ordered. Second, what we ultimately care about is often not the absolute rank but the relative order of pairs, which is exactly what a search or recommendation system exposes to a user. Herbrich, Graepel, and Obermayer (2000) introduced the large-margin treatment of this problem under the name ordinal regression, and Joachims (2002) showed that framing web-search relevance as a ranking task, learned from click-through pairs, outperforms treating it as regression on relevance scores.

Definition (preference relation and ranking rule)

Let Y be a finite set with a total order $<$. Given instances with ranks (x_i, y_i) , the induced preference relation on instances is $x_i > x_j \iff y_i > y_j$, and instances with equal rank are incomparable. A *ranking rule* is a map $r : X \rightarrow Y$; it is consistent with a preference $x_i > x_j$ when $r(x_i) > r(x_j)$.

There are two natural ways to attack this. One is to learn the ranks directly, embedding each instance on the real line and cutting that line into ranks with thresholds; this is the ordinal regression route of Section ordinal regression. The other, described by Shawe-Taylor and Cristianini (2004) as an alternative reduction, is to predict the relative ordering of every pair of examples and so obtain an ordinary two-class classification problem. The pairwise route is where the kernel machinery attaches most directly, so we take it first.

9.2 Reducing ranking to classification on pairs

Suppose we score instances with a linear function in the feature space, $f(x) = \langle w, \phi(x) \rangle$, and rank them by the value of f : larger score means more preferred. Then a single preference is a statement about a difference of scores. The item x_i is placed above x_j exactly when

$$f(x_i) > f(x_j) \iff \langle w, \phi(x_i) \rangle - \langle w, \phi(x_j) \rangle > 0 \iff \langle w, \phi(x_i) - \phi(x_j) \rangle > 0.$$

The linearity of the inner product collapses the comparison of two scores into the sign of w against one vector, the difference $\phi(x_i) - \phi(x_j)$. Every preferred pair (i, j) thus becomes a positively labelled training point for an ordinary linear classifier through the origin, with input the difference vector and label +1. This is the reduction of ranking to classification of Herbrich, Graepel, and Obermayer (2000) and Joachims (2002).

Definition (pairwise ranking problem)

Given ranked data $\{(x_i, y_i)\}$, let $P = \{(i, j) : y_i > y_j\}$ be the set of preferred pairs. The *pairwise ranking problem* is the classification problem on the difference vectors

$$z_{ij} = \phi(x_i) - \phi(x_j), \quad \text{all with label } +1, \quad (i, j) \in P,$$

seeking w with $\langle w, z_{ij} \rangle > 0$ for as many pairs as possible. A ranking function $f(x) = \langle w, \phi(x) \rangle$ orders a pair correctly iff its difference vector is classified positively.

Because the reduction never touches ϕ except through inner products, it stays inside the kernel world. The inner product between two difference vectors expands into four kernel evaluations,

$$\langle z_{ij}, z_{kl} \rangle = \langle \phi(x_i) - \phi(x_j), \phi(x_k) - \phi(x_l) \rangle = K_{ik} - K_{il} - K_{jk} + K_{jl},$$

where $K_{ab} = \kappa(x_a, x_b)$ is the Gram matrix of the original data. So the pairwise problem has its own kernel, computed from the base kernel with no explicit features, and any dual algorithm of Chapter Support Vector Machines runs on it unchanged. The one cost is size: with ℓ instances there can be on the order of ℓ^2 preferred pairs, so the pairwise sample grows quadratically, a point flagged by Shawe-Taylor and Cristianini (2004) and addressed in practice by Joachims (2002) through a decomposition that never materialises all pairs at once.

9.2.1 The ranking risk and misordered pairs

What should a ranking function minimise? For classification we counted misclassified points; the honest analogue for ranking counts misordered pairs. A pair $(i, j) \in P$ is *misordered*, or discordant, by the scorer f when f fails to place the preferred item above the other, that is when $\langle w, z_{ij} \rangle \leq 0$. The empirical ranking risk is the fraction of preferred pairs that come out discordant.

Definition (ranking risk)

For a scorer $f(x) = \langle w, \phi(x) \rangle$ and preferred pairs P , the empirical *ranking risk* is

$$\hat{R}(f) = \frac{1}{|P|} \sum_{(i,j) \in P} \mathbf{1}[\langle w, z_{ij} \rangle \leq 0],$$

the fraction of preferred pairs the scorer misorders. The population ranking risk is the probability that an independently drawn preferred pair is misordered.

This is exactly the training error of the pairwise classifier of the previous definition, so every generalisation bound for margin classifiers transfers to it. The indicator is not convex, so, as with the SVM, we shall upper bound it by a hinge loss and control the norm of w ; the margin that appears is a margin between the two items of a pair.

Proposition (the margin of a pair)

For a unit-norm scorer w the signed distance by which f separates the preferred pair (i, j) is

$$\gamma_{ij} = \frac{\langle w, \phi(x_i) - \phi(x_j) \rangle}{\|w\|} = f(x_i) - f(x_j) \quad (\text{for } \|w\| = 1).$$

The pair is correctly ordered with margin at least γ iff $f(x_i) - f(x_j) \geq \gamma$, and the overall ranking margin is $\gamma = \min_{(i,j) \in P} \gamma_{ij}$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The difference vector z_{ij} is the point the pairwise classifier sees, and the functional margin of a linear classifier through the origin at a point z with label $+1$ is $\langle w, z \rangle$; dividing by $\|w\|$ gives the geometric margin, the Euclidean distance from z to the hyperplane $\langle w, \cdot \rangle = 0$. Substituting $z_{ij} = \phi(x_i) - \phi(x_j)$ and using linearity gives $\langle w, z_{ij} \rangle = f(x_i) - f(x_j)$. The minimum over pairs is the smallest such gap, which is the ranking margin. $\square$

We can now make the reduction concrete on a tiny set, forming every difference vector, checking its margin, and counting how many pairs a candidate direction misorders.

Example (difference vectors and misordered pairs)

Four items in $\mathbb{R}^2$ with the linear kernel $\kappa(x, x') = \langle x, x' \rangle$:

$$x_1 = (4, 1), \quad x_2 = (3, 3), \quad x_3 = (2, 0), \quad x_4 = (1, 2),$$

with true order $x_1 > x_2 > x_3 > x_4$. The candidate scoring direction is $w = (1, -1)$. The six preferred pairs are $(1>2), (1>3), (1>4), (2>3), (2>4), (3>4)$.

1. Score the items. $f(x_i) = \langle w, x_i \rangle$ gives $f = (3, 0, 2, -1)$. By score the order is $x_1 > x_3 > x_2 > x_4$, so x_2 and x_3 look swapped.
2. Form the difference vectors. For each preferred pair, $z_{ij} = x_i - x_j$: $z_{12} = (1, -2)$, $z_{13} = (2, 1)$, $z_{14} = (3, -1)$, $z_{23} = (1, 3)$, $z_{24} = (2, 1)$, $z_{34} = (1, -2)$.
3. Read the pair margins. $\langle w, z_{ij} \rangle$ equals $+3, +1, +4, -2, +1, +3$ for the six pairs. Only $(2>3)$ is nonpositive, at -2 .
4. Count the misordered pairs. Exactly one pair, $(2>3)$, is misordered, so the ranking risk is $\hat{R}(f) = 1/6 \approx 0.1667$.
5. Check the pair kernel. Between pairs $(2>3)$ and $(1>4)$, $\langle z_{23}, z_{14} \rangle = (1)(3) + (3)(-1) = 0$; via the Gram matrix, $K_{21} - K_{24} - K_{31} + K_{34} = 15 - 9 - 8 + 2 = 0$, matching.

Reading. The whole ranking task has become a linear classification of six difference vectors, and the ranking risk is just their training error, here one pair in six. The pair kernel reproduces the difference-vector inner product from base-kernel entries alone, so nothing here needed the features explicitly.

Verification artifact. checks/example-ch-ranking-example-8-1.json records the example source hash and verification scope.

9.3 The soft-margin ranking SVM

The ranking risk is a count of misordered pairs and hence not convex, so we do exactly what the support vector machine does for classification: replace the count by a hinge upper bound and trade it against the norm of w . Requiring each preferred pair to be ordered with margin one, $\langle w, z_{ij} \rangle \geq 1$, and allowing a slack $\xi_{ij} \geq 0$ where that fails, gives the soft-margin ranking SVM of Herbrich, Graepel, and Obermayer (2000) and Joachims (2002). Minimising $\frac{1}{2}\|w\|^2$ maximises the ranking margin of Proposition above, while C sets the price of a misordered pair.

9 Ranking and Ordinal Regression

Algorithm (soft-margin ranking SVM)

Input Ranked data $\{(x_i, y_i)\}_{i=1}^\ell$, base kernel κ , penalty $C > 0$; preferred pairs $P = \{(i, j) : y_i > y_j\}$.

Output Dual weights $\alpha_{ij} \geq 0$ defining the scorer $f(x) = \sum_{(i,j) \in P} \alpha_{ij}(\kappa(x_i, x) - \kappa(x_j, x))$.

1. Form the pair Gram matrix $G_{(ij),(kl)} = K_{ik} - K_{il} - K_{jk} + K_{jl}$ over pairs in P .
2. Solve the primal $\min_{w, \xi} \frac{1}{2} \|w\|^2 + C \sum_{(i,j) \in P} \xi_{ij}$ subject to $\langle w, z_{ij} \rangle \geq 1 - \xi_{ij}$ and $\xi_{ij} \geq 0$, equivalently its dual $\max_{\alpha} \sum_{(ij)} \alpha_{ij} - \frac{1}{2} \sum_{(ij),(kl)} \alpha_{ij} \alpha_{kl} G_{(ij),(kl)}$ with $0 \leq \alpha_{ij} \leq C$.
3. Iterate the dual ascent (SMO-style working set) until no KKT violation exceeds tolerance τ .
4. Recover $w = \sum_{(ij)} \alpha_{ij} z_{ij}$ implicitly; keep the α_{ij} for kernel evaluation.
5. Rank a new x by the value of $f(x)$; larger is more preferred.

The dual is worth deriving, because it shows the pair kernel is all the optimiser ever needs. The Lagrangian of the primal, with multipliers $\alpha_{ij} \geq 0$ for the margin constraints and $\mu_{ij} \geq 0$ for the slack nonnegativity, is

$$L = \frac{1}{2} \|w\|^2 + C \sum_{(ij)} \xi_{ij} - \sum_{(ij)} \alpha_{ij} (\langle w, z_{ij} \rangle - 1 + \xi_{ij}) - \sum_{(ij)} \mu_{ij} \xi_{ij}.$$

Stationarity in w gives $w = \sum_{(ij)} \alpha_{ij} z_{ij}$, a weighted sum of difference vectors, and stationarity in ξ_{ij} gives $\alpha_{ij} + \mu_{ij} = C$, hence the box constraint $0 \leq \alpha_{ij} \leq C$. Substituting back and using $\langle z_{ij}, z_{kl} \rangle = G_{(ij),(kl)}$,

$$\max_{\alpha} \sum_{(ij) \in P} \alpha_{ij} - \frac{1}{2} \sum_{(ij),(kl) \in P} \alpha_{ij} \alpha_{kl} G_{(ij),(kl)}, \quad 0 \leq \alpha_{ij} \leq C.$$

This is precisely a support vector machine dual whose Gram matrix is the pair kernel, so any solver from Chapter Support Vector Machines applies, and the resulting w lies in the span of the difference vectors, giving the kernelised scorer in the algorithm's output. Notice there is no equality constraint $\sum_i \alpha_i y_i = 0$: every pairwise label is +1 and the classifier passes through the origin, so the bias term is absent.

Shawe-Taylor and Cristianini (2004) reach an equivalent optimiser from a stability bound rather than from a margin postulate. They upper bound the ranking risk by a Rademacher-style quantity built from the pairwise slacks and the trace of the kernel, then minimise that bound; the program that results is the same soft ranking objective, with a ν -parametrisation (in the style of Chapter Support Vector Machines) in which at most a fraction ν of the constraints are allowed nonzero slack while at least a fraction ν sit on the margin. The connection to generalisation is therefore not incidental: minimising the norm of w is minimising an upper bound on the probability that a fresh pair is misordered, the theme developed in Chapter Learning Theory in RKHS Balls.

9.4 Online ranking: a perceptron for preferences

The batch SVM must hold all pairs in memory, which is costly when the data streams in. The perceptron idea of Chapter Online Kernel Learning adapts to ranking with almost no change: scan the preferred pairs, and whenever the current w misorders one, nudge w toward ordering it correctly by adding the pair's difference vector. Each update is the standard perceptron correction applied to the difference vector z_{ij} , which by construction carries label +1.

Algorithm (online ranking perceptron)

Input Stream of preferred pairs (i, j) with difference vectors $z_{ij} = \phi(x_i) - \phi(x_j)$; learning rate $\eta > 0$.

Output Weight vector w (in dual form, a set of pair coefficients) defining $f(x) = \langle w, \phi(x) \rangle$.

1. Initialise $w \leftarrow 0$.

2. For each incoming preferred pair (i, j) : compute the margin $m = \langle w, z_{ij} \rangle$.
3. If $m \leq 0$ (the pair is misordered), update $w \leftarrow w + \eta z_{ij}$; otherwise leave w unchanged.
4. Repeat over the stream (or over passes through a fixed set) until no pair is misordered or a pass budget is reached.

Because w is only ever incremented by difference vectors, it stays in their span, $w = \sum_{(i,j)} \alpha_{ij} z_{ij}$ with nonnegative integer-multiple coefficients when $\eta = 1$, so the margin evaluates through the pair kernel exactly as in the batch case, and the update is kernelisable. The convergence guarantee is inherited wholesale from the perceptron: if the preferred pairs are separable with ranking margin γ and the difference vectors have norm at most R , the number of updates is at most $(R/\gamma)^2$, by the Novikoff argument of Chapter Online Kernel Learning applied to the difference vectors. Shawe-Taylor and Cristianini (2004) give the matching stability bound for the ranking perceptron, and Crammer and Singer (2002) analyse the closely related online ordinal algorithm PRank that we meet in the next section.

Example (one update fixes a swapped pair)

Three items in $\mathbb{R}^2$, linear kernel, true order $x_1 > x_2 > x_3$:

$$x_1 = (4, 4), x_2 = (3, 0), x_3 = (1, 1).$$

Start from $w_0 = (0, 1)$, learning rate $\eta = 1$. Preferred pairs: $(1>2), (1>3), (2>3)$.

1. Score with w_0 . $f = \langle w_0, x_i \rangle = (4, 0, 1)$. Then $f(x_3) = 1 > f(x_2) = 0$, so the pair $(2>3)$ is misordered; the other two pairs are fine. Misordered count: one.
2. Form the offending difference vector. $z_{23} = x_2 - x_3 = (2, -1)$, with margin $\langle w_0, z_{23} \rangle = (0)(2) + (1)(-1) = -1 \leq 0$, confirming the violation.
3. Apply the perceptron update. $w_1 = w_0 + \eta z_{23} = (0, 1) + (2, -1) = (2, 0)$. The new margin on that pair is $\langle w_1, z_{23} \rangle = (2)(2) + (0)(-1) = +4 > 0$, so the pair is now ordered.
4. Re-count over all pairs. With $w_1 = (2, 0)$, $f = (8, 6, 2)$, giving order $x_1 > x_2 > x_3$. All three preferred pairs are correct: misordered count is zero.

Reading. A single correction along the difference vector of the swapped pair flips its margin from -1 to $+4$ and, here, repairs the entire ranking. This is the perceptron of Chapter Online Kernel Learning acting on pairs, and its update count is bounded by $(R/\gamma)^2$ just as in classification.

Verification artifact. checks/example-ch-ranking-example-8-2.json records the example source hash and verification scope.

9.5 Ordinal regression: thresholds on the line

The pairwise view predicts orderings; sometimes we instead want to output an actual rank label, for example a star rating from one to five. The direct route, ordinal regression in the sense of Herbrich, Graepel, and Obermayer (2000), keeps the single scoring function $f(x) = \langle w, \phi(x) \rangle$ but cuts the real line into $|Y|$ intervals with an ordered set of thresholds, one interval per rank. An instance is given the rank of the interval its score falls into.

Definition (linear ranking rule with thresholds)

A *linear ranking rule* embeds instances on the real line by $f(x) = \langle w, \phi(x) \rangle$ and converts the score to a rank with thresholds b_y , one per rank $y \in Y$, that respect the order: $y < y'$ implies $b_y \leq b_{y'}$. The rank assigned to x is

$$r_{w,b}(x) = \min\{y \in Y : f(x) < b_y\},$$

with the largest label given a threshold large enough that the minimum always exists. In dual form $w = \sum_i \alpha_i \phi(x_i)$, so $f(x) = \sum_i \alpha_i \kappa(x_i, x)$ and only kernel evaluations are needed.

Ordered thresholds partition the input space into $|Y|$ regions, and within a region the value of f even induces a finer ordering, which we discard when we report only the rank. The learning problem is now to choose w and the thresholds so that each training instance lands in the correct interval with a margin. Shawe-Taylor and Cristianini (2004) recode this so it becomes two ordinary classifications per instance. For an instance of rank y , correctness means the score sits above the lower threshold b_{y-1} and below the upper threshold b_y ; each inequality is a linear classification in an augmented feature space that appends a one-hot indicator of the rank to $\phi(x)$. Concretely, with $\hat{w}_b = (w, -b)$ and the augmented vector $\phi(x, y) = (\phi(x), e_y)$, the instance is correctly ranked iff $\langle \hat{w}_b, \phi(x, y) \rangle < 0$ and $\langle \hat{w}_b, \phi(x, y-1) \rangle \geq 0$, so a bound on the two classifier error rates bounds the ranking error, at worst doubling it because either half can fail.

The soft ordinal program then reads: maximise a common margin γ while paying for the upper and lower slacks ξ_i^u, ξ_i^l by which each instance falls short of its two thresholds,

$$\min_{w, b, \gamma, \xi} -\gamma + C \sum_{i=1}^{\ell} (\xi_i^u + \xi_i^l) \quad \text{s.t.} \quad \langle w, \phi(x_i) \rangle \leq b_{y_i} - \gamma + \xi_i^l, \quad \langle w, \phi(x_i) \rangle \geq b_{y_i-1} + \gamma - \xi_i^u, \quad \|w\|^2 = 1,$$

the soft ranking computation of Shawe-Taylor and Cristianini (2004). Its dual multipliers satisfy $\sum_i (\alpha_i^u + \alpha_i^l) = 1$, and, with $C = 1/(\nu\ell)$, at most a fraction ν of the instances miss the margin at both adjacent thresholds while at least a fraction ν achieve it, the familiar ν -property transported to ranking.

The online counterpart is the PRank algorithm of Crammer and Singer (2002), which maintains w together with the whole ordered vector of thresholds and updates both on a mistake. When an instance of true rank y_i is predicted at rank $y = r_{\alpha, b}(x_i) \neq y_i$, the coefficient of x_i is adjusted by $y_i - y$ and each threshold strictly between the predicted and true ranks is shifted one step toward correcting the error. A short case analysis, given in Shawe-Taylor and Cristianini (2004), shows the shifts can never cross two thresholds, so the update preserves $b_y \leq b_{y'}$ for $y < y'$: the thresholds stay ordered, and the rule remains a valid ranking rule after every step. The stability of PRank follows from the perceptron mistake bound exactly as in the pairwise case, now with a factor $|Y| - 1$ counting the thresholds an error can involve.

9.6 Ranking and the area under the ROC curve

When there are only two ranks, preferred and not, ranking becomes bipartite: we want every positive item scored above every negative item. This is the setting of information retrieval and of any detector evaluated by its ROC curve, and it exposes a clean identity. The area under the ROC curve, the AUC, counts the fraction of positive and negative pairs the scorer orders correctly, which is one minus the bipartite ranking risk. This is the Wilcoxon-Mann-Whitney statistic, and it is what RankBoost of Freund, Iyer, Schapire, and Singer (2003) optimises directly.

Proposition (AUC as concordant-pair count)

Let a scorer assign real values to m_+ positive and m_- negative items, with no ties. Then

$$\text{AUC} = \frac{1}{m_+ m_-} \sum_{p \in \text{pos}} \sum_{n \in \text{neg}} \mathbf{1}[f(p) > f(n)] = 1 - \hat{R}_{\text{bip}},$$

where $\hat{R}_{\text{bip}}$ is the fraction of positive-negative pairs the scorer misorders. With ties, each contributes $\frac{1}{2}$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The empirical ROC curve is the step function traced as the decision threshold sweeps from $+\infty$ to $-\infty$; each positive crossed raises the curve by $1/m_+$ and each negative crossed advances it by $1/m_-$. The area accumulated

when the threshold passes a given negative equals $1/m_-$ times the fraction of positives already above it, so summing over negatives gives $\frac{1}{m_+m_-}$ times the number of positive-negative pairs with the positive scored higher. That count over the total m_+m_- pairs is the concordant fraction, and its complement is the misordered fraction $\hat{R}_{\text{bip}}$. Ties split a pair evenly between the two sides, contributing $\frac{1}{2}$. $\square$

So maximising AUC and minimising the pairwise ranking risk are the same objective, and the ranking SVM and ranking perceptron of the previous sections are AUC optimisers in disguise, on the convex hinge relaxation. The identity also gives a one-line evaluation recipe: to score a bipartite ranker, count concordant pairs.

Example (AUC as a pair count)

Three positive items with scores (0.9, 0.6, 0.4) and three negative items with scores (0.7, 0.5, 0.2). There are $m_+m_- = 9$ positive-negative pairs.

1. Count for the top positive. Score 0.9 beats all of 0.7, 0.5, 0.2: three concordant pairs.
2. Count for the middle positive. Score 0.6 beats 0.5 and 0.2 but loses to 0.7: two concordant, one discordant.
3. Count for the bottom positive. Score 0.4 beats only 0.2: one concordant, two discordant.
4. Total the pairs. Concordant = $3 + 2 + 1 = 6$, discordant = 3, ties = 0, so the Mann-Whitney statistic is $U = 6$.
5. Form the AUC. $\text{AUC} = U/9 = 6/9 \approx 0.6667$, while the bipartite ranking risk is $3/9 \approx 0.3333$, and indeed $\text{AUC} + \hat{R}_{\text{bip}} = 1$.

Reading. No curve needs to be drawn: the AUC is a count of correctly ordered positive-negative pairs divided by the total. Because that count is exactly the complement of misordered pairs, a ranker trained to minimise pairwise risk is training its AUC upward.

Verification artifact. checks/example-ch-ranking-example-8-3.json records the example source hash and verification scope.

9.7 Common mistakes and practical implications

For **Ranking and Ordinal Regression**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

9.8 Summary and further reading

This chapter established explain the central definitions and claims in Ranking and Ordinal Regression; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Herbrich and Obermayer 2000), (Joachims 2002), (Freund and Singer 2003).

9.9 Exercises

1. warm-up For the four items of the difference-vector example, verify by hand that the score order induced by $w = (1, -1)$ is $x_1 \succ x_3 \succ x_2 \succ x_4$, and confirm that exactly the pair $(2 \succ 3)$ is inverted relative to the true order.
2. warm-up Show that if κ is a positive definite kernel then the pair kernel $G_{(ij),(kl)} = K_{ik} - K_{il} - K_{jk} + K_{jl}$ is also positive definite on the set of difference vectors. (Hint: it is the ordinary Gram matrix of the vectors $z_{ij} = \phi(x_i) - \phi(x_j)$, so write a quadratic form $\sum \alpha_{ij} \alpha_{kl} G$ as a squared norm.)
3. computation A constant shift of the scorer, $f \mapsto f + c$, and a positive rescaling, $f \mapsto af$ with $a > 0$, leave every pairwise comparison unchanged. Use this to explain why the pairwise ranking SVM has no bias term and why the induced ranking rule is identifiable only up to positive scaling. Then explain how the unit-margin constraint and norm penalty nevertheless choose a particular scale for the optimizer.
4. computation With ℓ instances spread over $|Y|$ ranks of equal size, count the number of preferred pairs $|P|$ as a function of ℓ and $|Y|$, and show it is $\Theta(\ell^2)$ for fixed $|Y|$. Contrast this with the $\Theta(\ell)$ derived examples of the threshold ordinal-regression formulation, and comment on when each reduction is cheaper.
5. computation Redo the perceptron-update example with learning rate $\eta = 2$ instead of 1. Does the single update on the pair $(2 \succ 3)$ still repair the full ranking? Compute the new scores and misordered count, and relate any overshoot to the fact that the perceptron only guarantees the updated pair's margin becomes positive, not that other pairs are preserved.
6. challenge Prove the perceptron ranking mistake bound: if the difference vectors z_{ij} satisfy $\|z_{ij}\| \leq R$ and there is a unit vector $w^\star$ with $\langle w^\star, z_{ij} \rangle \geq \gamma > 0$ for all preferred pairs, then the online ranking algorithm with $\eta = 1$ makes at most $(R/\gamma)^2$ updates. (Hint: track $\langle w, w^\star \rangle$ and $\|w\|^2$ across updates exactly as in Novikoff's theorem for the classification perceptron in Chapter Online Kernel Learning.)
7. challenge Show that the AUC identity extends to the smoothed hinge surrogate: the convex objective $\sum_{p,n} \max(0, 1 - (f(p) - f(n)))$ upper bounds $m_+ m_-$ times the bipartite ranking risk, so minimising it minimises an upper bound on $1 - \text{AUC}$. Identify where the constant-1 margin enters and why any positive margin would serve.
8. challenge In the threshold model, suppose PRank predicts rank $y = y_i + 1$ for an instance of true rank y_i . Write out the coefficient and threshold updates of Crammer and Singer (2002), and prove that the single crossed threshold b_{y_i} cannot pass its neighbour b_{y_i+1} , so the thresholds remain ordered. (Hint: only thresholds strictly between predicted and true ranks move, and each moves by one integer step.)

10 Solving the SVM: Decomposition and SMO

The support vector machine of the previous chapter leaves us with a beautiful object and a computational headache: a convex quadratic program whose only unknowns are the dual coefficients, but whose cost matrix is the dense $n \times n$ Gram matrix of the training set. For a few thousand points a general-purpose solver handles it; for a few hundred thousand the matrix does not even fit in memory. This chapter is about how the quadratic program is actually solved. We start from the Karush-Kuhn-Tucker conditions, which double as an exact stopping test, then build up the two ideas that made kernel machines practical: decomposition, which optimizes a handful of variables at a time while freezing the rest, and its limiting case, sequential minimal optimization, whose working set of two variables is so small that the subproblem has a closed-form solution and no inner solver is needed at all. We derive that analytic step in full, including the clipping to a box, the bias update, and the rule for choosing which pair to move, and we close with the interior-point alternative and the regression variant. The material follows Schölkopf and Smola (2002).

10.1 Why the dual is hard at scale

Recall the soft-margin support vector classifier. With training pairs $(x_1, y_1), \dots, (x_n, y_n)$, labels $y_i \in \{-1, +1\}$, a kernel K with Gram matrix $K_{ij} = K(x_i, x_j)$, and a penalty $C > 0$, the dual problem is the quadratic program

$$\max_{\alpha \in \mathbb{R}^n} W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j K_{ij} \quad \text{subject to} \quad \sum_{i=1}^n \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C.$$

The decision function is $f(x) = \sum_{i=1}^n \alpha_i y_i K(x_i, x) + b$, so once the α_i are known the machine is determined up to the scalar threshold b . This is a concave maximization over a box intersected with a single hyperplane, a genuinely easy problem in the textbook sense: the objective is smooth, the feasible set is convex and compact, and any local optimum is global. The difficulty is entirely one of size.

The cost sits in the quadratic form. The matrix of that form is $Q_{ij} = y_i y_j K_{ij}$, an $n \times n$ dense matrix. Merely storing it costs $O(n^2)$ memory, and a direct quadratic-program solver that factorizes it costs $O(n^3)$ time. At $n = 10^5$, the Gram matrix alone is 10^{10} double-precision numbers, roughly eighty gigabytes, and the cubic factorization is out of reach. Even forming a single row of Q requires n kernel evaluations. Any usable algorithm must therefore avoid ever materializing the whole matrix, and must instead touch the kernel only where it is needed. This constraint, not the shape of the objective, is what has shaped every serious SVM solver, and it is the reason the naive approach of handing the program to a black-box optimizer fails past a few thousand points. The escape routes split into two families, examined in this chapter: decomposition, which solves the exact dual on a small changing subset of the variables, and interior-point methods, which solve a sequence of easier smooth problems. A third route, replacing K by a low-rank surrogate, belongs to the chapter on large-scale kernels.

10.2 The KKT conditions as the stopping test

Before we can optimize a subset of variables we need to know when to stop, both globally and on each subproblem. For a convex program the Karush-Kuhn-Tucker (KKT) conditions are necessary and sufficient for optimality, so they furnish an exact certificate. Written out for the SVM dual, they classify each training point by the value of its margin $y_i f(x_i)$ against the position of its coefficient in the box $[0, C]$.

Proposition (KKT conditions for the SVM dual)

A feasible α (satisfying $\sum_i \alpha_i y_i = 0$ and $0 \leq \alpha_i \leq C$) is optimal if and only if, with $f(x_i) = \sum_j \alpha_j y_j K_{ij} + b$ for the corresponding threshold b ,

$$\alpha_i = 0 \Rightarrow y_i f(x_i) \geq 1, \quad 0 < \alpha_i < C \Rightarrow y_i f(x_i) = 1, \quad \alpha_i = C \Rightarrow y_i f(x_i) \leq 1.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The reading is geometric. A point with $\alpha_i = 0$ sits outside the margin and does not participate; a non-bound point with $0 < \alpha_i < C$ sits exactly on the margin, $y_i f(x_i) = 1$; a point at the upper bound $\alpha_i = C$ has been pushed inside the margin or misclassified. A point that violates its condition is one whose coefficient is not yet where optimality demands, and the size of the violation measures how far the current solution is from the optimum.

This is more than a yes-or-no test. Schölkopf and Smola (2002, Proposition 10.1) show that the aggregate violation, the KKT gap, upper-bounds the suboptimality of the regularized risk itself, so one can bound the distance to the optimal objective without knowing that optimum. Crucially the gap is computable in $O(n)$ time once the function values $f(x_i)$ are known, which are maintained anyway during training, so checking convergence is almost free relative to the cost of the iterations. The gap does the double duty promised in the introduction: a global stopping rule for the whole solver, and, restricted to a working set, the score that tells decomposition which variables to work on next.

It is worth stressing why the stopping test lives in the margins $y_i f(x_i)$ rather than in the coefficients α_i themselves. What the user ultimately wants is the function f , and if the dual objective has a flat direction, a whole subspace of coefficient vectors yields the identical f . An algorithm that waited for the coefficients to converge might wait forever inside that subspace, wasting effort on differences that never reach the output. Proximity in the solution, measured by the KKT gap, is the right currency; proximity in the parameters is not.

10.3 Decomposition and chunking

The decomposition idea starts from a structural fact about the solution: it is sparse. Only the support vectors, the points with $\alpha_i > 0$, enter the final expansion, and in many problems these are a small fraction of the data. If we already knew the support set, we could throw away every other point, solve the reduced dual on the support vectors alone, and recover exactly the same machine (Vapnik 1982). We do not know the support set in advance, but we can hunt for it. This is chunking.

Chunking begins with an arbitrary subset, or chunk, small enough to fit in memory, and solves the SVM on it with any base optimizer. It keeps the support vectors it finds, discards the rest of the chunk, and refills the chunk with points that the current machine gets wrong, that is, points violating their KKT condition. Retraining on this new chunk and iterating, the working set gradually accretes the true support vectors and sheds the inactive points, until no point in the whole dataset violates the KKT conditions and the global optimum has been reached.

Osuna, Freund, and Girosi (1997) turned this heuristic into a convergent algorithm with a fixed-size working set. Rather than let the chunk grow, they fix its size q and, at each round, swap out a variable that satisfies its KKT condition for one that violates it. Because every optimized subproblem strictly decreases the objective and the violating variable is guaranteed to improve it, the procedure converges to the global optimum while never handling more than q variables at once. Joachims (1999), in the SVM^{light} solver, refined the choice of which variables enter the working set, added the caching and shrinking heuristics of the next section, and made large-scale training routine.

Algorithm (decomposition, outer loop)

Input kernel K , data $(x_i, y_i)_{i=1}^n$, penalty C , working-set size q , tolerance τ .

Output dual coefficients α and threshold b .

1. Initialize $\alpha \leftarrow 0$, $b \leftarrow 0$; all function values $f(x_i) \leftarrow 0$.
2. Compute the KKT violation of every variable from the current $f(x_i)$.
3. Select a working set S of q variables with the largest violations (respecting the equality constraint; see the selection rule below).
4. Solve the SVM dual exactly over $\{\alpha_i : i \in S\}$, holding the remaining coefficients fixed.
5. Update the stored $f(x_i)$ and the threshold b from the changed coefficients.
6. Repeat from step 2 until no KKT violation exceeds τ .

The inner solve in step 4 is a smaller quadratic program of the same shape, and any of the standard methods handles it. The art of decomposition is in step 3, the working-set selection, because the wrong choice can stall progress even though every individual subproblem is solved exactly.

10.3.1 Working-set selection

Which variables should the working set contain? The KKT gap points the way. After optimizing over a working set S , the KKT-gap terms belonging to S vanish, so the natural greedy rule is to fill S with the variables whose current contribution to the gap is largest: these are the most-violating points, and clearing their violation buys the biggest guaranteed reduction. Schölkopf and Smola (2002) catalogue three closely related scores, based respectively on the KKT gap, the gradient of the objective, and the mismatch of the Lagrange multipliers; the gap score is preferred because it uses the tightest bound.

The equality constraint $\sum_i \alpha_i y_i = 0$ complicates the naive greedy rule and must be honored. Moving a single variable off its current value would break the constraint, so a feasible search direction must keep $\sum_i y_i \delta \alpha_i = 0$. In practice this means the selected variables must include both signs of y_i , so that the changes can cancel; Joachims (1999) selects the q variables by sorting a per-variable score derived from the gradient and taking the top violators of each sign. Fan, Chen, and Lin (2005) sharpen this to a second-order rule, choosing the pair whose exchange yields the largest predicted decrease of the objective rather than the largest first-order violation, which is the working-set selection used in the LIBSVM solver.

Algorithm (working-set selection by maximal KKT violation)

Input current coefficients α , labels y , penalty C , stored scores $F_i = \sum_j \alpha_j y_j K_{ij}$.

Output the most-violating pair $(i_{\text{low}}, i_{\text{up}})$, or a certificate of optimality.

1. Form the index sets of variables free to increase and to decrease the objective along the constraint:
 $I_{\text{up}} = \{i : \alpha_i < C, y_i = +1\} \cup \{i : \alpha_i > 0, y_i = -1\}$,
 $I_{\text{low}} = \{i : \alpha_i < C, y_i = -1\} \cup \{i : \alpha_i > 0, y_i = +1\}$.
2. Compute $b_{\text{up}} = \min_{i \in I_{\text{up}}} F_i$ and $b_{\text{low}} = \max_{i \in I_{\text{low}}} F_i$.
3. If $b_{\text{low}} \leq b_{\text{up}} + 2\tau$, report optimality and stop.
4. Otherwise return $i_{\text{low}} = \arg \max_{i \in I_{\text{low}}} F_i$ and $i_{\text{up}} = \arg \min_{i \in I_{\text{up}}} F_i$.

The quantities b_{up} and b_{low} are the tightest thresholds consistent with the upper and lower margin sets; at optimality the interval $[b_{\text{low}}, b_{\text{up}}]$ is non-empty and any b in it satisfies every KKT condition, while a positive gap $b_{\text{low}} - b_{\text{up}}$ exhibits a violating pair. This is exactly the selection rule of Keerthi, Shevade, Bhattacharyya, and Murthy (2001), and the two indices it returns are the smallest possible working set. We now make that minimal case the whole algorithm.

Example (a KKT-violation check and pair selection)

Four points in $\mathbb{R}^2$ with a linear kernel $K(x, x') = x^\top x'$: $x_1 = (0, 0)$, $x_2 = (2, 0)$, $x_3 = (1, 2)$, $x_4 = (3, 2)$, labels $y = (-1, -1, +1, +1)$, penalty $C = 1$. Test the feasible dual point $\alpha = (0.5, 0, 0.5, 0)$.

1. Confirm feasibility. $\sum_i \alpha_i y_i = -0.5 - 0 + 0.5 + 0 = 0$ and every $\alpha_i \in [0, 1]$, so α is feasible.
2. Score each point. With $F_i = \sum_j \alpha_j y_j K_{ij}$ the coefficient vector $\alpha_j y_j = (-0.5, 0, 0.5, 0)$ gives $F = (0, 1, 2.5, 3.5)$.

3. Split into up and low sets. $I_{\text{up}} = \{1, 3, 4\}$ and $I_{\text{low}} = \{1, 2, 3\}$.
4. Bracket the threshold. $b_{\text{up}} = \min(F_1, F_3, F_4) = 0$ at point 1; $b_{\text{low}} = \max(F_1, F_2, F_3) = 2.5$ at point 3.
5. Read the violation. Since $b_{\text{low}} - b_{\text{up}} = 2.5 > 0$, the KKT test fails: the point is not optimal. The most-violating pair is $(i_{\text{low}}, i_{\text{up}}) = (3, 1)$.

Reading. The optimal-threshold interval $[b_{\text{low}}, b_{\text{up}}] = [2.5, 0]$ is empty, which is precisely the signature of a KKT violation, and the gap 2.5 is the amount of suboptimality the next step must remove. Points 3 and 1 are the pair whose coefficients should move.

Verification artifact. checks/example-ch-solve-example-9-1.json records the example source hash and verification scope.

10.4 Sequential minimal optimization

Decomposition still needs an inner solver for the subproblem in step 4. Platt (1998) observed that if the working set is made as small as it can possibly be, the subproblem becomes trivial: it can be solved with a formula, and no inner optimizer is required at all. The question is how small the working set can be while remaining useful, and the answer is dictated by the equality constraint.

Lemma (the smallest useful working set has size two)

Under the constraint $\sum_i \alpha_i y_i = 0$, fixing all but one coefficient pins the remaining one: it cannot change without violating the constraint. Hence at least two coefficients must be free to move, and a working set of size two is the smallest that permits any progress.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Suppose α_j is the only free variable and the others are held at their current values. The constraint reads $\alpha_j y_j = -\sum_{k \neq j} \alpha_k y_k$, whose right side is a fixed number. Since $y_j \in \{-1, +1\}$ is nonzero, α_j is forced to the single value $-y_j \sum_{k \neq j} \alpha_k y_k$, leaving no freedom to decrease the objective. With two free variables α_i, α_j the constraint becomes $y_i \alpha_i + y_j \alpha_j = \text{const}$, a line in the (α_i, α_j) plane along which both may vary. $\square$

10.4.1 The analytic two-variable step

Fix a pair (i, j) and hold every other coefficient constant. Write $s = y_i y_j$. Multiplying the equality constraint $y_i \alpha_i + y_j \alpha_j = \text{const}$ by y_i gives $\alpha_i + s \alpha_j = \text{const}$, so along the feasible line α_i is an affine function of α_j . Substituting this into the dual objective $W(\alpha)$ and discarding terms that do not involve α_j leaves a quadratic in the single variable α_j . Its second derivative is

$$\frac{\partial^2 W}{\partial \alpha_j^2} = -(K_{ii} + K_{jj} - 2K_{ij}) = -\eta, \quad \eta := K_{ii} + K_{jj} - 2K_{ij}.$$

The quantity η is nonnegative for a positive definite kernel, being $\|\Phi(x_i) - \Phi(x_j)\|^2$ in feature space, and it is strictly positive unless the two feature vectors coincide. Because W is concave in α_j , setting the first derivative to zero gives the unconstrained maximizer. Writing $E_k = f(x_k) - y_k$ for the prediction error at point k , the maximizer is

$$\alpha_j^{\text{new,unc}} = \alpha_j^{\text{old}} + \frac{y_j (E_i - E_j)}{\eta}.$$

This unconstrained value ignores the box $0 \leq \alpha_j \leq C$, so it must be clipped back onto the feasible line. The endpoints L and H of the allowed interval for α_j come from intersecting the line $\alpha_i + s \alpha_j = \text{const}$ with the square $[0, C]^2$, and they depend on whether the two labels agree.

Proposition (the clipping box)

If $y_i \neq y_j$ (so $s = -1$, and $\alpha_i - \alpha_j$ is fixed),

$$L = \max(0, \alpha_j^{\text{old}} - \alpha_i^{\text{old}}), \quad H = \min(C, C + \alpha_j^{\text{old}} - \alpha_i^{\text{old}}).$$

If $y_i = y_j$ (so $s = +1$, and $\alpha_i + \alpha_j$ is fixed),

$$L = \max(0, \alpha_i^{\text{old}} + \alpha_j^{\text{old}} - C), \quad H = \min(C, \alpha_i^{\text{old}} + \alpha_j^{\text{old}}).$$

The clipped update is $\alpha_j^{\text{new}} = \min(\max(\alpha_j^{\text{new,unc}}, L), H)$, and the partner follows from the constraint, $\alpha_i^{\text{new}} = \alpha_i^{\text{old}} + s(\alpha_j^{\text{old}} - \alpha_j^{\text{new}})$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

These are exactly the endpoints where the feasible line exits the box. When the labels differ, the line has slope +1 and slides along a diagonal; when they agree, slope -1 and slides along the anti-diagonal, and the two cases produce the two formulas above. Clipping is not an approximation: since the objective is a concave parabola in α_j , its constrained maximizer on an interval is the unconstrained vertex projected onto that interval, so the single min-max is exact.

Finally the threshold b is refreshed so that the KKT conditions hold at the freshly non-bound points. Requiring $f(x_i) = y_i$ after the update, that is $E_i = 0$, yields a candidate b_1 ; requiring $f(x_j) = y_j$ yields a candidate b_2 :

$$b_1 = b - E_i - y_i(\alpha_i^{\text{new}} - \alpha_i^{\text{old}})K_{ii} - y_j(\alpha_j^{\text{new}} - \alpha_j^{\text{old}})K_{ij},$$

$$b_2 = b - E_j - y_i(\alpha_i^{\text{new}} - \alpha_i^{\text{old}})K_{ij} - y_j(\alpha_j^{\text{new}} - \alpha_j^{\text{old}})K_{jj}.$$

If α_i^{new} is non-bound (strictly inside $(0, C)$) then b_1 makes point i satisfy its condition exactly, so $b = b_1$; symmetrically $b = b_2$ if α_j^{new} is non-bound; and when both new coefficients are at a bound, every b in the interval $[b_1, b_2]$ is KKT-consistent and one takes the midpoint $b = \frac{1}{2}(b_1 + b_2)$ (Keerthi et al. 2001).

Algorithm (SMO, one two-variable step)

Input pair (i, j) , coefficients α , threshold b , errors E_i, E_j , kernel entries K_{ii}, K_{jj}, K_{ij} , penalty C .

Output updated α_i, α_j and threshold b .

1. Set $s = y_i y_j$ and the curvature $\eta = K_{ii} + K_{jj} - 2K_{ij}$; if $\eta \leq 0$ fall back to evaluating the objective at the box ends.
2. Compute the box $[L, H]$ from the Proposition; if $L = H$ the pair cannot move, return unchanged.
3. Take the unconstrained step $\alpha_j^{\text{new,unc}} = \alpha_j + y_j(E_i - E_j)/\eta$.
4. Clip: $\alpha_j^{\text{new}} = \min(\max(\alpha_j^{\text{new,unc}}, L), H)$, then $\alpha_i^{\text{new}} = \alpha_i + s(\alpha_j - \alpha_j^{\text{new}})$.
5. Update the threshold from b_1, b_2 by the non-bound rule above, and refresh the stored errors E_k .
6. Repeat over selected pairs until no KKT violation exceeds τ .

The full SMO solver wraps this step in the selection rule of the previous section: an outer loop sweeps the data for a KKT-violating point i , and an inner heuristic chooses the partner j that maximizes $|E_i - E_j|$, the numerator of the step, so that the pair moves as far as possible. Platt's second-choice hierarchy tries the non-bound points first, then all points, before giving up on i . Because each step touches only two rows of the kernel, SMO never stores the Gram matrix, and its memory footprint is linear in n .

Example (a full analytic SMO step)

Two points on the line with a linear kernel: $x_1 = (1, 0)$, $x_2 = (2, 0)$, labels $y_1 = +1$, $y_2 = -1$, penalty $C = 1$. Start from $\alpha = (0.1, 0.1)$, $b = 0$, and take one step on the pair $(1, 2)$. Here $K_{11} = 1$, $K_{12} = 2$, $K_{22} = 4$.

1. Evaluate errors. $f(x_1) = \alpha_1 y_1 K_{11} + \alpha_2 y_2 K_{12} = 0.1 - 0.2 = -0.1$, so $E_1 = -0.1 - 1 = -1.1$; $f(x_2) = \alpha_1 y_1 K_{12} + \alpha_2 y_2 K_{22} = 0.2 - 0.4 = -0.2$, so $E_2 = -0.2 - (-1) = 0.8$.
2. Compute the curvature. $s = y_1 y_2 = -1$ and $\eta = K_{11} + K_{22} - 2K_{12} = 1 + 4 - 4 = 1$.
3. Take the unconstrained step. $\alpha_2^{\text{new,unc}} = 0.1 + y_2(E_1 - E_2)/\eta = 0.1 + (-1)(-1.1 - 0.8)/1 = 0.1 + 1.9 = 2.0$.
4. Clip to the box. Labels differ, so $L = \max(0, 0.1 - 0.1) = 0$ and $H = \min(1, 1 + 0.1 - 0.1) = 1$; the value 2.0 is clipped to $\alpha_2^{\text{new}} = 1$.
5. Update the partner. $\alpha_1^{\text{new}} = 0.1 + s(0.1 - 1) = 0.1 + (-1)(-0.9) = 1.0$; the constraint holds, $y_1 \alpha_1 + y_2 \alpha_2 = 1 - 1 = 0$, as before.
6. Refresh the threshold. $b_1 = 0 - (-1.1) - 1(0.9)(1) - (-1)(0.9)(2) = 2.0$ and $b_2 = 0 - 0.8 - 1(0.9)(2) - (-1)(0.9)(4) = 1.0$. Both new coefficients sit at the bound $C = 1$, so $b = \frac{1}{2}(b_1 + b_2) = 1.5$.

Reading. A single analytic step drives both coefficients from 0.1 to the upper bound $C = 1$: the two points become bound support vectors, and the misclassified point 2 (whose error $E_2 = 0.8$ flagged the violation) is corrected. No inner solver was called; the entire move is the four formulas for η , the clipped α_2 , the partner α_1 , and b .

Verification artifact. checks/example-ch-solve-example-9-2.json records the example source hash and verification scope.

10.5 Caching and shrinking

Two engineering ideas make decomposition and SMO fast in practice, and both exploit the same sparsity that motivated chunking. The first is caching. Since the Gram matrix cannot be stored, its entries are recomputed on demand, but the non-bound support vectors are revisited many times over the course of training, so their kernel rows are worth keeping. A row cache with a least-recently-used replacement policy, sized at perhaps ten percent of the full matrix, achieves an eighty to ninety percent hit rate and eliminates most kernel recomputation (Joachims 1999). The benefit is largest exactly when the number of non-bound coefficients is small, which is the common case.

The second idea is shrinking. As training proceeds, most coefficients settle onto a bound, $\alpha_i = 0$ or $\alpha_i = C$, and stay there. Shrinking detects such variables, temporarily removes them from the problem, and continues on the shrunken set, which can be an order of magnitude smaller. Once the reduced problem is solved, the full KKT conditions are checked on all variables to confirm that the removed ones were indeed at their optimal bounds, and any that were not are reinstated. A related restarting heuristic reuses the solution for one value of C or one kernel width as the warm start for a nearby value, since the optimizer is a smooth function of these parameters; starting from a large C and decreasing it keeps most variables unconstrained and can speed training substantially.

10.6 Interior-point methods, the alternative

Decomposition sidesteps the size of the Gram matrix by never forming it. The complementary strategy keeps the matrix but changes the algorithm: interior-point methods solve the quadratic program by following a smooth path to the optimum, and for moderate problem sizes they return solutions of very high accuracy. They are

the recommended choice whenever the kernel matrix fits in memory (Schölkopf and Smola 2002; Boyd and Vandenberghe 2004).

The idea is to satisfy the KKT conditions directly by Newton's method. The awkward part of those conditions is complementarity, the requirement that a coefficient and its associated slack cannot both be positive, which is combinatorial. An interior-point method relaxes it: it replaces the hard complementarity $\alpha_i s_i = 0$ with a softened $\alpha_i s_i = \mu$ for a parameter $\mu > 0$, solves the resulting smooth system of equations for the primal and dual variables, then drives $\mu \rightarrow 0$. Each value of μ defines a point on the central path, and the iterates track that path to the true KKT point as μ vanishes. The heavy step is solving the linearized, reduced KKT system at each iteration, which after eliminating the slack and box-multiplier increments reduces to a symmetric system whose leading block is Q plus a positive diagonal. This is solved by a Cholesky factorization, so the per-iteration cost is that of factorizing an $n \times n$ matrix; a predictor-corrector strategy keeps the number of iterations small and roughly independent of n .

The strength of interior-point methods is accuracy and reliability on moderate problems; their weakness is precisely the Cholesky factorization, whose $O(n^3)$ cost and $O(n^2)$ memory return us to the wall of the opening section. This is why the decision, sketched by Schölkopf and Smola (2002), is a size threshold: below a few thousand points use an interior-point code, above it use decomposition or SMO, and when even the reduced problem is too large, approximate the kernel matrix by a low-rank surrogate as in the large-scale-kernels chapter. A direct implementation also exploits the special structure of Q , for instance the block redundancy that appears in regression (below), which off-the-shelf solvers cannot see.

10.7 SMO for regression

The same machinery solves the support vector regression dual, with one bookkeeping complication. In ε -insensitive regression each training point carries two Lagrange multipliers, α_i for the upper side of the tube and α_i^* for the lower, with the equality constraint $\sum_i (\alpha_i - \alpha_i^*) = 0$. A working set of two patterns therefore involves four multipliers $\alpha_i, \alpha_i^*, \alpha_j, \alpha_j^*$, and the restriction that a point cannot be on both sides of the tube at once (α_i and α_i^* are never both nonzero) leaves up to three candidate sign combinations to try (Schölkopf and Smola 2002).

Within each combination the subproblem is again a one-dimensional concave quadratic in the difference $\delta = (\alpha_i - \alpha_i^*) - (\alpha_i^{\text{old}} - \alpha_i^{*\text{old}})$, and its curvature is the same $\eta = K_{ii} + K_{jj} - 2K_{ij}$ as in classification. The unconstrained step moves δ proportionally to the difference of the regression residuals $(f(x_i) - y_i) - (f(x_j) - y_j)$, which plays the role that $E_i - E_j$ played for classification, and the result is clipped to a box $[L, H]$ obtained from the constraints on all four multipliers. The threshold b is recovered from the non-bound multipliers exactly as before, using the condition $f(x_i) - y_i = \pm \varepsilon$ that holds at the edges of the tube. The pair selection again maximizes the residual discrepancy, so that the chosen patterns permit the largest step. The upshot is that regression reuses the classification step almost verbatim, at the cost of iterating over the handful of sign cases.

10.8 Summary

Solving the SVM is an exercise in respecting the $n \times n$ Gram matrix, which is too large to store or factorize once n passes a few thousand. The KKT conditions supply an exact, $O(n)$ stopping test and, through the KKT gap, a score for how far each variable is from optimality. Decomposition exploits the sparsity of the solution by optimizing a small working set at a time and swapping in the worst KKT violators, and its extreme case, sequential minimal optimization, shrinks the working set to the two variables that the equality constraint makes the minimum useful size. That minimality is the payoff: the two-variable subproblem is a clipped parabola with a closed-form maximizer, so SMO needs no inner solver and only linear memory. Caching the rows of active support vectors and shrinking away the bound variables make the sweeps cheap, while interior-point methods offer high-accuracy solutions when the matrix does fit. The regression variant carries four multipliers per pair but reuses the same analytic step. Between these tools and the low-rank approximations of the next scaling route, the convex program that defines the support vector machine becomes solvable at the scale real data demand; the online kernel methods push the same ideas to the streaming setting.

10.9 Common mistakes and practical implications

For **Solving the SVM: Decomposition and SMO**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

10.10 Summary and further reading

This chapter established explain the central definitions and claims in Solving the SVM: Decomposition and SMO; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Platt 1998), (Joachims 1999), (Osuna and Girosi 1997).

10.11 Exercises

1. warm-up State the three KKT cases for the SVM dual in words, and for each say where the point sits relative to the margin. A point has $\alpha_i = C$ but $y_i f(x_i) = 1.4 > 1$. Is its KKT condition satisfied? What if instead $0 < \alpha_i < C$ and $y_i f(x_i) = 1.4$?
2. computation Reproduce the worked SMO step by hand. With $K_{11} = 1, K_{12} = 2, K_{22} = 4$, labels $y_1 = +1, y_2 = -1, C = 1$, starting $\alpha = (0.1, 0.1), b = 0$, verify $\eta = 1$, the errors $E_1 = -1.1, E_2 = 0.8$, the unclipped $\alpha_2 = 2.0$, the box $[0, 1]$, and the final $\alpha = (1, 1), b = 1.5$. Then redo the step with $C = 0.4$ and report the new box $[L, H]$ and the resulting α_2^{new} .
3. proof Prove the Lemma that the smallest useful working set has size two. Show that under $\sum_i \alpha_i y_i = 0$ a single free variable is pinned, and that with two free variables the feasible set is a line segment. Explain why this equality constraint, and not the box constraints, is what forbids single-variable updates.
4. proof Derive the unconstrained two-variable optimum. Starting from $W(\alpha)$, eliminate α_i using $\alpha_i + s\alpha_j = \text{const}$, show that the resulting function of α_j has second derivative $-\eta$ with $\eta = K_{ii} + K_{jj} - 2K_{ij}$, and conclude that its maximizer is $\alpha_j^{\text{new,unc}} = \alpha_j^{\text{old}} + y_j(E_i - E_j)/\eta$ with $E_k = f(x_k) - y_k$. Hint
Write $\alpha_i = \gamma - s\alpha_j$ for a constant γ , substitute into the quadratic and linear parts of W , and collect the coefficient of α_j^2 and of α_j . The gradient of W with respect to α_j , before elimination, is $1 - y_j \sum_k \alpha_k y_k K_{jk}$, and $\sum_k \alpha_k y_k K_{jk} = f(x_j) - b$.
5. proof Derive the clipping box for the same-label case $y_i = y_j$. Along the line $\alpha_i + \alpha_j = \text{const} = k$, intersect with $0 \leq \alpha_i \leq C$ and $0 \leq \alpha_j \leq C$ to show $\alpha_j \in [\max(0, k - C), \min(C, k)]$, and confirm this matches the Proposition with $k = \alpha_i^{\text{old}} + \alpha_j^{\text{old}}$.
6. computation Take the four-point KKT example: $x_1 = (0, 0), x_2 = (2, 0), x_3 = (1, 2), x_4 = (3, 2), y = (-1, -1, +1, +1)$, linear kernel, $C = 1$. Verify the scores $F = (0, 1, 2.5, 3.5)$, the sets $I_{\text{up}} = \{1, 3, 4\}$ and $I_{\text{low}} = \{1, 2, 3\}$, and the selected pair $(3, 1)$. Now change α to $(1, 0, 1, 0)$ (both support vectors at the bound C) and recompute the sets: which points leave I_{up} or I_{low} , and does the pair change?
7. exploration Explain, in terms of caching and shrinking, why SMO speeds up as it approaches the optimum even though the number of KKT checks per sweep does not fall. Relate your answer to the observation that most coefficients settle onto a bound early and that the non-bound support vectors are the ones revisited many times.

8. challenge Contrast the two solvers on cost. For n points, give the per-iteration and memory cost of an interior-point method (Cholesky of an $n \times n$ system) and of one SMO step (two kernel rows). Explain why interior-point methods win on accuracy for n in the low thousands but lose to decomposition as n grows, and identify the exact operation that forces the crossover. Hint

The interior-point iteration factorizes Q plus a diagonal at cost $O(n^3)$ time and $O(n^2)$ memory, but needs only $O(\log(1/\text{tol}))$ iterations. An SMO step costs $O(n)$ time (two kernel rows and the error update) and $O(n)$ memory, but may need many sweeps. The crossover is set by whether Q can be stored and factorized at all.

11 Online Kernel Learning

Every kernel machine we have built so far is a batch machine: it reads the whole training set, assembles the Gram matrix, and solves one optimization problem to convergence. That is a luxury the world does not always grant. Data may arrive as a stream, one example at a time, faster than any solver can re-fit; the target may drift while we watch; or the sample may simply be too large for its Gram matrix to fit in memory. This chapter develops the kernel machines that learn from a stream, updating a little on each example and never forming the full Gram matrix. The two classical engines are the kernel perceptron, whose mistake-driven update is one line and whose convergence is governed by the margin through Novikoff's beautiful two-sided bound, and the kernel adatron, which turns the support vector dual into a coordinate-wise gradient ascent that climbs to the very same max-margin solution the batch support vector machine finds. We close with the price of streaming, the budget problem: an online kernel expansion tends to grow one term per mistake without limit, and taming it is what separates a toy from a deployable system.

11.1 The online learning setting

In the batch setting a learner is handed a fixed sample $S = \{(x_1, y_1), \dots, (x_\ell, y_\ell)\}$ and produces a single hypothesis. Online learning replaces this with a game played over time. At each round t the learner holds a current hypothesis f_t ; it receives an input x_t , predicts $\hat{y}_t = \text{sgn } f_t(x_t)$, then sees the true label y_t and suffers a loss. Only after that does it update, producing f_{t+1} . The data may be an unending stream, the distribution generating (x_t, y_t) may change over time, and so the goal is not to minimize an expected risk against one fixed distribution but to predict well, round by round, as the stream unfolds (Kivinen, Smola, and Williamson 2004).

Three constraints shape everything that follows. First, the learner processes one example at a time and cannot revisit the past at will. Second, and decisively for us, it never assembles the full $\ell \times \ell$ Gram matrix: a kernel evaluation $K(x_j, x_t)$ is computed on demand against the examples the learner has chosen to remember, and nothing more. Third, updates should be cheap, ideally constant work per round beyond the kernel evaluations. The methods below meet all three, and the reason a kernel can appear at all is the dual representation that has run through this whole book: the hypothesis is stored not as a weight vector in a feature space we cannot touch, but as a weighted sum of kernel functions centered on remembered examples.

11.2 The kernel perceptron

The oldest online learner is the perceptron of Rosenblatt (1958). In its primal form it maintains a weight vector w , predicts $\text{sgn}\langle w, x \rangle$, and on a mistake nudges w toward the offending example: $w \leftarrow w + y_t x_t$. The nudge is exactly right in direction, since adding $y_t x_t$ raises $y_t \langle w, x_t \rangle$ by $\|x_t\|^2$, pushing the example toward the correct side. Aizerman, Braverman, and Rozonoer (1964) observed that this algorithm never needs w itself: because w starts at zero and only ever accumulates multiples of training inputs, it always lives in their span, and every quantity the algorithm touches is an inner product. Replacing that inner product by a kernel gives the kernel perceptron, one of the earliest instances of the kernel trick.

11.2.1 The dual update and the kernel expansion

Work in the feature space of a kernel K with map φ , and let the weight vector be $w = \sum_j \alpha_j y_j \varphi(x_j)$ for nonnegative integer counts α_j . The counts start at zero, so $w_0 = 0$. The prediction on any point is then a pure kernel expansion,

$$f(x) = \langle w, \varphi(x) \rangle = \sum_j \alpha_j y_j K(x_j, x),$$

and the primal update $w \leftarrow w + y_t \varphi(x_t)$, triggered when $\text{sgn } f(x_t) \neq y_t$, becomes the trivially simple dual update

$$\alpha_t \leftarrow \alpha_t + 1.$$

Each mistake on example t increments that example's own coefficient by one and adds nothing anywhere else. The count α_j is literally the number of times the algorithm has blundered on x_j , and an example that is never a mistake keeps $\alpha_j = 0$ and drops out of the expansion entirely. This is where the sparsity and the streaming friendliness come from at once: the hypothesis is supported only on the mistaken examples, so the learner remembers only those, and each round costs one kernel evaluation against each remembered point. Compare this with the batch dual solver, which weighs every example against every other.

Algorithm (kernel perceptron)

Input stream $(x_1, y_1), (x_2, y_2), \dots$ with $y_t \in \{-1, +1\}$; kernel K .

Output dual counts α and the hypothesis $f(x) = \sum_j \alpha_j y_j K(x_j, x)$.

1. Initialize $\alpha \leftarrow 0$ and the support set $\text{SV} \leftarrow \emptyset$.
2. For each incoming example (x_t, y_t) :
3. compute the prediction $f(x_t) = \sum_{j \in \text{SV}} \alpha_j y_j K(x_j, x_t)$;
4. if $\text{sgn } f(x_t) \neq y_t$ (treating $\text{sgn } 0 = 0$ as an error), set $\alpha_t \leftarrow \alpha_t + 1$ and add t to SV ;
5. otherwise leave α unchanged and discard x_t .

Run as a batch procedure, the algorithm sweeps the training set repeatedly, updating on each mistake, and stops once a full sweep passes with no mistake at all, at which point the current α classifies every training point correctly. The following worked example runs exactly this loop on a set that no straight line through the origin can separate, and watches the quadratic kernel resolve it.

Example (kernel perceptron on a non-separable set)

Four points on the line, $x = -2, -1, +1, +2$, with the outer pair labelled $+1$ and the inner pair -1 . No bias-free line $\text{sgn}(wx)$ can separate outer from inner. Use the quadratic kernel $K(x, z) = (1 + xz)^2$, whose feature map contains x^2 ; there the outer points sit at $x^2 = 4$ and the inner at $x^2 = 1$, on opposite sides of a threshold. The Gram matrix is

$$K = \begin{pmatrix} 25 & 9 & 1 & 9 \\ 9 & 4 & 0 & 1 \\ 1 & 0 & 4 & 9 \\ 9 & 1 & 9 & 25 \end{pmatrix}.$$

Process the four points in the fixed order shown, pass after pass, with $\alpha = 0$ initially.

1. Sweep pass 1. With $\alpha = 0$ every prediction is $f = 0$, and $\text{sgn } 0 = 0$ counts as a mistake, so as the pass proceeds the predictions read $f = [+0, +9, +1, -1]$; all four signs disagree with the labels, so every coefficient is incremented: $\alpha = (1, 1, 1, 1)$, 4 mistakes.
2. Sweep pass 2. Now $f = [+24, +6, +6, +14]$. The two positives are already correct, but both inner points still read positive when they should be negative, so α_2 and α_3 each rise: $\alpha = (1, 2, 2, 1)$, 2 mistakes.
3. Sweep pass 3. Predictions $f = [+14, +2, +2, +4]$; the inner points are still on the wrong side, so again $\alpha = (1, 3, 3, 1)$, 2 mistakes.
4. Sweep pass 4. Predictions $f = [+4, -2, -2, +4]$: every sign now matches its label. A clean pass, no update, and the algorithm halts with $\alpha = (1, 3, 3, 1)$.

Reading. The perceptron made 8 updates in all and converged after four sweeps to $\alpha = (1, 3, 3, 1)$, a hypothesis that classifies all four points correctly even though the raw inputs are not linearly separable. The kernel did the lifting; the update rule never changed. Notice the coefficients are larger on the two inner points, which straddle the decision region and cost the learner more mistakes.

Verification artifact. checks/example-ch-online-example-10-1.json records the example source hash and verification scope.

11.3 Novikoff's mistake bound

The kernel perceptron never explicitly tries to make few mistakes, yet on a separable stream the number it can make is sharply limited. The classical guarantee is due to Novikoff (1962), and it is a statement purely about the margin: the harder the data are to separate, the smaller the margin, and the more mistakes the perceptron is entitled to make, with the count scaling as the inverse square of the margin. The proof is a small marvel, squeezing the number of updates between a lower and an upper bound on the length of the running weight vector.

Theorem (Novikoff, 1962)

Let the examples $(x_1, y_1), (x_2, y_2), \dots$ all satisfy $\|\varphi(x_t)\| \leq R$, so that their feature images lie in a ball of radius R about the origin. Suppose there is a unit weight vector w^* , $\|w^*\| = 1$, and a margin $\gamma > 0$ with $y_t \langle w^*, \varphi(x_t) \rangle \geq \gamma$ for every t . Then the kernel perceptron, started from $w_0 = 0$, makes at most

$$\left(\frac{R}{\gamma}\right)^2$$

updates before it ceases to err on the stream.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Index the mistakes $t = 1, 2, \dots$ and let w_t be the weight vector after the t -th update. Each update is triggered by a misclassified example, which we call $(x_{(t)}, y_{(t)})$, and performs $w_t = w_{t-1} + y_{(t)}\varphi(x_{(t)})$, which in dual terms is the increment $\alpha_{(t)} \leftarrow \alpha_{(t)} + 1$.

Lower bound: the projection onto w^* grows linearly. Project each update onto the margin vector:

$$\langle w_t, w^* \rangle = \langle w_{t-1}, w^* \rangle + y_{(t)} \langle \varphi(x_{(t)}), w^* \rangle \geq \langle w_{t-1}, w^* \rangle + \gamma,$$

using the margin hypothesis for the last inequality. Since $\langle w_0, w^* \rangle = 0$, induction gives $\langle w_t, w^* \rangle \geq t\gamma$.

Upper bound: the squared length grows at most linearly. Expand

$$\|w_t\|^2 = \|w_{t-1}\|^2 + 2y_{(t)} \langle w_{t-1}, \varphi(x_{(t)}) \rangle + \|\varphi(x_{(t)})\|^2.$$

The example was a mistake, meaning $y_{(t)} \langle w_{t-1}, \varphi(x_{(t)}) \rangle \leq 0$, so that middle term is not positive; and $\|\varphi(x_{(t)})\|^2 \leq R^2$. Hence $\|w_t\|^2 \leq \|w_{t-1}\|^2 + R^2$, and by induction from $w_0 = 0$, $\|w_t\|^2 \leq tR^2$.

Squeeze. Combine the two bounds through Cauchy-Schwarz, $\langle w_t, w^* \rangle \leq \|w_t\| \|w^*\| = \|w_t\|$:

$$t\gamma \leq \langle w_t, w^* \rangle \leq \|w_t\| \leq \sqrt{t} R.$$

Dividing by $\sqrt{t}$ gives $\sqrt{t} \gamma \leq R$, that is $t \leq R^2/\gamma^2$. The number of updates can therefore never exceed $(R/\gamma)^2$. $\square$

Two features of this argument deserve emphasis. It is entirely dimension-free: neither R nor γ refers to the dimension of the feature space, so the bound holds verbatim in the infinite-dimensional space of a Gaussian kernel. And it makes no probabilistic assumption at all, holding for any ordering of any separable stream. The margin γ here is exactly the geometric margin of the hard-margin support vector machine with no bias, which ties the perceptron's mistake count directly to the object the batch SVM maximizes. Shawe-Taylor and Cristianini (2004) build on precisely this to convert the update count into a generalization bound, since a hypothesis reconstructed from few mistakes is compressible.

Example (evaluating Novikoff's bound)

Six points in $\mathbb{R}^2$, separable through the origin by the vertical axis $x_1 = 0$: the three with $x_1 > 0$ are positive, the three with $x_1 < 0$ negative.

x_1	x_2	y
1	3	+1
1	-3	+1
5	0	+1
-1	3	-1
-1	-3	-1
-5	0	-1

Two points sit at $x_1 = \pm 1$, close to the boundary; two sit far out at $(\pm 5, 0)$. Use the linear kernel.

1. Measure the radius. The farthest points from the origin are $(\pm 5, 0)$, so $R = \sqrt{25} = 5$ and $R^2 = 25$.
2. Solve for the margin. The hard-margin, no-bias solution minimizes $\|w\|^2$ subject to $y_i \langle w, x_i \rangle \geq 1$; it comes out at $w^* \propto (1, 0)$, scaled so the closest points $x_1 = \pm 1$ sit at functional margin 1. The geometric margin is $\gamma = 1/\|w\| = 1$, so $\gamma^2 = 1$.
3. Form the bound. Novikoff promises at most $R^2/\gamma^2 = 25/1 = 25$ updates.
4. Run the perceptron. Sweeping the six points in order with the linear kernel, the algorithm updates only on the two boundary points and then passes cleanly: 2 updates in total, with final counts nonzero only at $(1, 3)$ and $(-1, 3)$.

Reading. The guarantee, 25 mistakes, is honored with room to spare by the actual count of 2. The gap is instructive: the two far-flung points at $(\pm 5, 0)$ inflate R and hence the bound, yet the perceptron never errs on them, because they lie deep inside their half-spaces. Novikoff's theorem bounds the worst case over all orderings, not the typical run; it is an upper bound, and a loose one whenever the geometry is benign.

Verification artifact. `checks/example-ch-online-example-10-2.json` records the example source hash and verification scope.

11.3.1 Voted and averaged perceptrons

The plain perceptron keeps only its final weight vector, and on noisy or non-separable data that last vector may reflect whatever example happened to arrive most recently. Freund and Schapire (1999) proposed a simple and effective fix that costs nothing extra to train. Record every intermediate hypothesis $w_1, w_2, \dots$ together with its survival time, the number of consecutive examples it classified correctly before the next update. At prediction time, let each stored hypothesis vote, weighted by its survival time, and take the sign of the weighted vote: this is the voted perceptron. A cheaper approximation replaces the vote by the survival-weighted average weight vector, the averaged perceptron, which predicts with a single $\bar{w} = \sum_t c_t w_t / \sum_t c_t$. Both remain pure kernel expansions, since each w_t is one, and both markedly improve robustness on data the perceptron cannot perfectly separate, while inheriting the mistake-bound analysis of the underlying algorithm.

11.4 The kernel adatron

The perceptron finds some separator; it does not seek the best one. The kernel adatron of Frieß, Cristianini, and Campbell (1998) closes that gap while keeping the online, one-coordinate-at-a-time character. Its idea is to run gradient ascent directly on the support vector dual objective, coordinate by coordinate, so that the online process converges not merely to a separator but to the maximum-margin separator itself.

Recall the hard-margin dual (with no bias, so no equality constraint survives),

$$W(\alpha) = \sum_{i=1}^{\ell} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{\ell} \alpha_i \alpha_j y_i y_j K(x_i, x_j), \quad \alpha_i \geq 0.$$

Its partial derivative in one coordinate is remarkably clean:

$$\frac{\partial W}{\partial \alpha_i} = 1 - y_i \sum_{j=1}^{\ell} \alpha_j y_j K(x_j, x_i) = 1 - y_i f(x_i),$$

where $f(x_i) = \sum_j \alpha_j y_j K(x_j, x_i)$ is the current prediction. A gradient-ascent step in coordinate i is therefore

$$\alpha_i \leftarrow \alpha_i + \eta(1 - y_i f(x_i)).$$

This has an appealing reading. When the example is classified with functional margin below one, $y_i f(x_i) < 1$, the derivative is positive and the coefficient grows, pulling the boundary toward the example; when it sits comfortably beyond the margin, $y_i f(x_i) > 1$, the coefficient shrinks. At a fixed point every in-play example has $y_i f(x_i) = 1$, which is exactly the support vector condition. The perceptron's mistake-driven increment $\alpha_i \leftarrow \alpha_i + 1$ is the crude ancestor of this rule (Schölkopf and Smola 2002); the adatron replaces the binary "did we err" by the graded "by how much is the margin violated."

The one subtlety is the constraint. The unconstrained step may drive α_i negative, or, in the soft-margin case, above the box ceiling C . Because the feasible set is the box $0 \leq \alpha_i \leq C$, the correct move is projected gradient ascent: take the gradient step, then project back onto the box by clipping,

$$\alpha_i \leftarrow \min(C, \max(0, \alpha_i + \eta(1 - y_i f(x_i)))).$$

For the hard-margin problem $C = \infty$ and the ceiling is inactive, leaving the single floor $\alpha_i \geq 0$. Projected coordinate ascent on a concave quadratic over a box converges, and its limit is the constrained maximizer, so the adatron converges to the max-margin dual solution (Frieß, Cristianini, and Campbell 1998). This is the same box constraint that the soft-margin SVM imposes, arrived at here through the projection rather than through a quadratic-program solver.

Algorithm (kernel adatron)

Input examples $(x_1, y_1), \dots, (x_{\ell}, y_{\ell})$; kernel K ; learning rate $\eta > 0$; box ceiling C (use $C = \infty$ for hard margin).

Output dual variables α approximating the max-margin solution, and $f(x) = \sum_j \alpha_j y_j K(x_j, x)$.

1. Initialize $\alpha \leftarrow 0$.
2. Repeat until every $|1 - y_i f(x_i)|$ with $0 < \alpha_i < C$ is below a tolerance τ :
3. pick a coordinate i (cyclically or by largest KKT violation) and evaluate $f(x_i) = \sum_j \alpha_j y_j K(x_j, x_i)$;
4. form the gradient $g_i = 1 - y_i f(x_i)$ and step $\alpha_i \leftarrow \alpha_i + \eta g_i$;
5. project onto the box: $\alpha_i \leftarrow \min(C, \max(0, \alpha_i))$.

11 Online Kernel Learning

Example (kernel adatron reaching the max-margin dual)

Four points, linear kernel, no bias, hard margin.

	x_1	x_2	y
p_1	1	2	+1
p_2	4	4	+1
p_3	-2	-1	-1
p_4	-4	-4	-1

$$K = \begin{pmatrix} 5 & 12 & -4 & -12 \\ 12 & 32 & -12 & -32 \\ -4 & -12 & 5 & 12 \\ -12 & -32 & 12 & 32 \end{pmatrix}.$$

Take $\eta = 0.05$, $\alpha = 0$. The reference max-margin dual, from the quadratic program, is $\alpha^* = (\frac{1}{9}, 0, \frac{1}{9}, 0)$ with margin $\gamma = 1/\|w\| = 2.1213$ and optimum $W^* = \frac{1}{9}$.

1. Sweep the coordinates. One full pass of the projected update over $i = 1, \dots, 4$ already lifts the objective from 0 to $W(\alpha) = 0.0591$, with $\alpha \approx (0.050, 0.020, 0.028, 0)$; the two outer points p_2, p_4 get clipped back toward zero as the closer points take over.
2. Watch the objective climb. The dual value increases monotonically: after 2 sweeps $W = 0.0906$, after 5 sweeps $W = 0.1101$, after 20 sweeps $W = 0.11109$. Projected coordinate ascent never lowers a concave objective, so each sweep is an improvement.
3. Converge. By 100 sweeps $W = 0.111111$ to six places and $\alpha = (0.1112, 0, 0.1111, 0)$; by 1000 sweeps $\alpha = (\frac{1}{9}, 0, \frac{1}{9}, 0)$ exactly to the printed precision.
4. Read off the machine. The converged weights give $w = (\frac{1}{3}, \frac{1}{3})$, hence margin $\gamma = 1/\|w\| = 2.1213$, matching the support vector machine's own margin.

Reading. The adatron, driven only by local gradient steps and a clip to the box, lands on $\alpha^* = (\frac{1}{9}, 0, \frac{1}{9}, 0)$: the two margin points p_1, p_3 become support vectors, the two interior points p_2, p_4 are switched off. The dual value climbs to $W^* = \frac{1}{9}$ and the induced margin equals the batch SVM's exactly. The online process and the batch quadratic program reach the same optimum; only the route differs.

Verification artifact. checks/example-ch-online-example-10-3.json records the example source hash and verification scope.

11.5 Online support vector regression

Nothing about the online recipe is special to classification. The same stochastic-gradient view that produced the adatron produces an online regressor once we swap the loss. Kivinen, Smola, and Williamson (2004) formulate this directly in the reproducing kernel Hilbert space: minimize the regularized risk $\frac{\lambda}{2} \|f\|_{\mathcal{H}}^2 + \mathbb{E} c(x, y, f(x))$ by stochastic gradient descent, using at round t the single-example estimate $\frac{\lambda}{2} \|f\|_{\mathcal{H}}^2 + c(x_t, y_t, f(x_t))$. The gradient of $\|f\|_{\mathcal{H}}^2$ is $2f$, and the gradient of the loss term, by the reproducing property, is $c'(x_t, y_t, f(x_t)) K(x_t, \cdot)$. The update is therefore

$$f_{t+1} = (1 - \eta\lambda) f_t - \eta c'(x_t, y_t, f_t(x_t)) K(x_t, \cdot),$$

a two-part move: shrink the whole current expansion by the factor $1 - \eta\lambda$, then append one new kernel term centered on the fresh example, weighted by the loss derivative.

For regression with the ε -insensitive loss $c = \max(0, |y - f(x)| - \varepsilon)$, the derivative is $-\text{sgn}(y_t - f_t(x_t))$ when the residual exceeds ε and zero inside the tube. So a new support vector is created, with coefficient $\pm\eta$, exactly when the prediction lands more than ε from the target; examples the current f already fits to within ε leave the expansion untouched. This reproduces the sparsity of batch support vector regression, now generated one example at a time: the tube decides membership. Setting $\varepsilon = 0$ recovers an online least-absolute-deviation regressor, and swapping the hinge loss back in recovers the regularized kernel perceptron, so a single update template covers classification, regression, and novelty detection by changing only the loss derivative c' .

The shrink factor $1 - \eta\lambda$ is the quiet workhorse. It is the regularization acting online, and it means the coefficient attached to an example decays geometrically with every subsequent round. An example seen long ago contributes $(1 - \eta\lambda)^k$ times its original weight after k further rounds, so its influence fades. This built-in forgetting is what makes the method track a drifting target, and, as we now see, it is also the lever for controlling memory.

11.6 NORMA and passive-aggressive updates

The shrink-and-append rule above is the **NORMA** template of Kivinen, Smola, and Williamson (2004). Its decisive feature is regularized stochastic gradient: the learning rate and λ jointly control both the newest coefficient and the decay of every older one. For a convex loss and a bounded kernel, standard stochastic-approximation guarantees require a step schedule whose sum diverges while the sum of squared steps converges, or a carefully stated constant-step tracking objective. A constant step on a drifting stream is useful, but it is not the same theorem as convergence to a fixed population minimizer.

Passive-aggressive learning starts from a different local question: what is the smallest RKHS change that makes the current example satisfy a unit-margin constraint? For classification it solves

$$\min_{f, \xi \geq 0} \frac{1}{2} \|f - f_t\|_{\mathcal{H}}^2 + C\xi \quad \text{subject to} \quad y_t f(x_t) \geq 1 - \xi.$$

The solution is

$$f_{t+1} = f_t + \tau_t y_t K(x_t, \cdot), \quad \tau_t = \min\left(C, \frac{\max(0, 1 - y_t f_t(x_t))}{K(x_t, x_t)}\right).$$

It is passive when the constraint already holds and otherwise makes the smallest capped correction. The regression version replaces hinge loss by the ε -insensitive violation and uses the residual sign. Unlike NORMA, the displayed update has no global shrinkage, so regularization comes from the conservative projection, the cap C , and any separate budget policy (Crammer et al. 2006).

Two implementation details matter. First, normalize the kernel or guard against tiny $K(x_t, x_t)$, which otherwise creates an enormous step. Second, store a global decay multiplier for NORMA instead of rescaling every coefficient on every round; periodically fold that multiplier into the coefficients to avoid underflow. Both methods cost one kernel evaluation per retained support vector, so their time and memory remain governed by the budget problem.

11.7 The budget problem

There is a catch that the mistake bound hides. Novikoff's theorem caps the number of updates only when the stream is separable with a positive margin. On a noisy or non-separable stream, or one whose target drifts, the perceptron and the online SVR keep erring, and every error appends a term to the kernel expansion. The support set grows without bound, so the memory footprint and the per-round prediction cost both climb linearly with time. One can show that under nonzero minimal risk the number of support vectors indeed grows linearly with the number of examples seen (Schölkopf and Smola 2002). An online learner that must run indefinitely cannot afford an unbounded model; this is the budget problem, and it is the central practical obstacle to deploying online kernel methods.

Two broad families of remedy exist, and the discussion here is deliberately at the level of strategy rather than specific update formulas. The first is forgetting through decay. The regularized update already multiplies every coefficient by $1 - \eta\lambda$ each round, so old coefficients shrink geometrically; once a coefficient falls below a threshold its term contributes negligibly and can be pruned. Truncating the expansion to a fixed time horizon, keeping only the most

recent terms whose decayed weight is still material, bounds the memory at the cost of a controlled approximation, and the learning rate trades the size of the model against how far into the past the learner remembers (Kivinen, Smola, and Williamson 2004). This is well suited to non-stationary streams, where forgetting the distant past is a feature.

The second family fixes a hard budget B on the number of support vectors and maintains it actively. When a new example would push the support set past B , the learner must make room, and three moves are standard. Removal discards the least useful current support vector, for instance the one of smallest coefficient or smallest contribution to the margin. Projection is subtler: rather than deleting the chosen term outright, it redistributes that term's contribution onto the remaining support vectors by projecting it onto their span in feature space, so the hypothesis changes as little as possible in RKHS norm. Merging replaces two nearby support vectors by a single synthetic one that approximates their combined effect. All three keep the model size at B forever, differing in how much accuracy they sacrifice to do so; projection is the most faithful and the most expensive, removal the cheapest and the crudest. The unifying principle is the one that has organized this whole chapter: because the hypothesis is a kernel expansion, budget maintenance is geometry in feature space, choosing which directions to keep and how to account for the ones we drop, and it connects directly to the low-rank and sparse-approximation techniques used for large-scale kernel machines.

11.8 Summary

Online kernel learning trades the single batch optimization for a stream of cheap, local updates, storing the hypothesis as a dual kernel expansion and never forming the full Gram matrix. The kernel perceptron makes this vivid: its update is the one-line increment $\alpha_t \leftarrow \alpha_t + 1$ on a mistake, its prediction is a kernel expansion over the mistaken examples, and Novikoff's two-sided argument bounds its total mistakes by $(R/\gamma)^2$, tying the mistake count to the very margin the support vector machine maximizes and to the generalization theory of margins. The kernel adatron upgrades the crude mistake increment to projected gradient ascent on the support vector dual, converging to the exact max-margin solution while respecting the box constraint through a clip. The same stochastic-gradient template, with the loss derivative swapped, gives online support vector regression and novelty detection, all sharing the shrink-and-append update whose decay factor performs regularization online. The price of streaming is the budget problem, the unbounded growth of the support set on hard streams, and controlling it, by forgetting through decay or by maintaining a hard budget through removal, projection, or merging, is what turns these elegant updates into systems that can run forever. The next chapters put the same dual machinery to work on ranking and on the approximations that make large-scale kernels tractable.

11.9 Common mistakes and practical implications

For **Online Kernel Learning**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

11.10 Summary and further reading

This chapter established explain the central definitions and claims in Online Kernel Learning; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Rosenblatt 1958), (Novikoff 1962), (Aizerman and Rozonoer 1964).

11.11 Exercises

1. warm-up Show that the kernel perceptron's dual update follows from its primal update. Starting from $w = \sum_j \alpha_j y_j \varphi(x_j)$ with $w_0 = 0$, verify that the primal move $w \leftarrow w + y_t \varphi(x_t)$ on a mistake is identical to $\alpha_t \leftarrow \alpha_t + 1$, and that the prediction $\langle w, \varphi(x) \rangle$ equals $\sum_j \alpha_j y_j K(x_j, x)$. Explain why an example never misclassified keeps $\alpha_j = 0$.
2. computation Repeat the worked kernel-perceptron example with the homogeneous quadratic kernel $K(x, z) = (xz)^2$ instead of $(1 + xz)^2$, on the same four points $x = -2, -1, 1, 2$ with outer label $+1$ and inner -1 . Write out the 4×4 Gram matrix and run the mistake-driven sweeps by hand until a clean pass. Does the algorithm still converge, and to what α ? Compare the number of updates with the $(1 + xz)^2$ run.
3. proof In Novikoff's proof, identify precisely where each of the two hypotheses is used: the margin condition $y_t \langle w^*, \varphi(x_t) \rangle \geq \gamma$ and the radius condition $\|\varphi(x_t)\| \leq R$. Then show that the middle-term inequality $y_{(t)} \langle w_{t-1}, \varphi(x_{(t)}) \rangle \leq 0$ is exactly the statement that the update was triggered by a mistake, and explain why the bound would fail if updates were made on correctly classified points. Hint

The lower bound uses only the margin condition; the upper bound uses only the mistake condition and the radius bound. A correctly classified example has $y_{(t)} \langle w_{t-1}, \varphi(x_{(t)}) \rangle > 0$, which would make the squared-length recursion grow faster than tR^2 .

4. computation Evaluate Novikoff's bound on the two-point set $x_1 = (1, 0)$, $y_1 = +1$ and $x_2 = (-1, 0)$, $y_2 = -1$, with the linear kernel and no bias. Compute R , the max-margin γ , and the bound R^2/γ^2 , then run the perceptron and count the actual updates. Now scale both points by a factor $a > 0$; show that R , γ , and the bound all respond so that R^2/γ^2 is unchanged, and explain why the mistake count cannot depend on the overall scale.
5. proof Derive the adatron gradient. Starting from $W(\alpha) = \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$, compute $\partial W / \partial \alpha_i$ and confirm it equals $1 - y_i f(x_i)$ with $f(x_i) = \sum_j \alpha_j y_j K(x_j, x_i)$. Then argue that at any interior fixed point of the projected update (one with $0 < \alpha_i < C$) the support vector condition $y_i f(x_i) = 1$ holds. Hint

Differentiate term by term; the quadratic contributes $-y_i \sum_j \alpha_j y_j K(x_i, x_j)$. At an interior fixed point the clip is inactive, so the gradient must vanish, giving $1 - y_i f(x_i) = 0$.

6. proof Show that the projected adatron update never decreases W along a single coordinate for a small enough step. Treating W as a function of α_i with the other coordinates fixed, note it is a concave quadratic $W(\alpha_i) = \alpha_i(1 - y_i \tilde{f}) - \frac{1}{2} K_{ii} \alpha_i^2 + \text{const}$, where $\tilde{f}$ collects the other terms. Find the unconstrained maximizer in α_i , and explain why clipping it to $[0, C]$ still yields the constrained maximizer along that coordinate. Hint

The one-variable maximizer is $\alpha_i^* = \alpha_i + (1 - y_i f(x_i))/K_{ii}$. A concave function on an interval attains its constrained maximum either at the unconstrained optimum or at the nearest endpoint, which is exactly what the clip returns; the choice $\eta = 1/K_{ii}$ makes one adatron step land on it.

7. computation Consider the online SVR update $f_{t+1} = (1 - \eta\lambda)f_t - \eta c'(x_t, y_t, f_t(x_t))K(x_t, \cdot)$ with the ε -insensitive loss. Suppose at rounds 1, 2, 3 the residuals $y_t - f_t(x_t)$ are $+2, 0, -2$ and $\varepsilon = 1, \eta = 0.5, \lambda = 0.1$. Track which rounds create a new support vector and write the coefficient attached to the round-1 term after rounds 2 and 3, given the geometric decay factor $1 - \eta\lambda$.
8. challenge The budget problem asks us to remove one support vector from an expansion $f = \sum_{j=1}^{B+1} \beta_j K(x_j, \cdot)$ with least damage. Formalize the projection strategy: to drop index r , choose new coefficients $\{\beta'_j\}_{j \neq r}$ minimizing $\left\| \sum_{j \neq r} \beta'_j K(x_j, \cdot) - f \right\|_{\mathcal{H}}^2$. Write this as a linear least-squares problem in the remaining coefficients, identify the normal equations in terms of the Gram matrix of the kept points, and explain why removal (setting $\beta'_j = \beta_j$ for $j \neq r$ and dropping β_r) is the special case that ignores the coupling. Hint

Expand the squared RKHS norm using $\langle K(x_i, \cdot), K(x_j, \cdot) \rangle_{\mathcal{H}} = K(x_i, x_j)$. The minimizer solves $K_{\text{keep}} \beta' = K_{\text{keep,all}} \beta$, a projection of the full expansion onto the span of the retained kernel functions; plain removal keeps the old coefficients and so pays the full norm of the dropped term.

12 Learning Theory in RKHS Balls

The previous chapters replaced the intractable 0/1 classification error with a convex surrogate, the φ -risk, and replaced arbitrary measurable functions with functions in a reproducing kernel Hilbert space. This chapter justifies both moves. First we prove a calibration theorem of Bartlett, Jordan, and McAuliffe (2006): minimizing a well-chosen convex φ -risk automatically minimizes the classification error. Then we develop the statistical side: Rademacher complexity as a measure of the capacity of a function class, a basic learning bound for empirical risk minimization with Lipschitz losses, and an explicit capacity bound for balls of an RKHS whose proof is three lines of Hilbert space geometry. The chapter closes the circle by showing, via Lagrangian duality, that empirical risk minimization over an RKHS ball is exactly the penalized problem $\min_f \frac{1}{n} \sum_i \varphi(y_i f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2$ that our algorithms actually solve. A self-contained interlude on convex duality supplies the tools, the same ones used to derive the support vector machine in Chapter 5, Support Vector Machines.

12.1 From the 0/1 loss to convex surrogates

We work in the standard binary classification setting. A pair (X, Y) is drawn from an unknown probability distribution P on $\mathcal{X} \times \{-1, +1\}$, and we observe a training sample $S_n = ((X_1, Y_1), \dots, (X_n, Y_n))$ of n independent copies of (X, Y) . A real-valued function $f : \mathcal{X} \rightarrow \mathbb{R}$ classifies a point x by the sign of $f(x)$. Its quality is measured by the 0/1 risk.

Definition (0/1 risk, Bayes risk)

The 0/1 risk (or classification error) of a measurable function $f : \mathcal{X} \rightarrow \mathbb{R}$ is

$$R(f) = P(\text{sign } f(X) \neq Y),$$

and the Bayes risk is $R^* = \min_{g \text{ measurable}} R(g)$. We write

$$\eta(x) = P(Y = 1 \mid X = x)$$

for the regression function of the labels.

Conditioning on $X = x$, a function that predicts +1 errs with probability $1 - \eta(x)$ and a function that predicts -1 errs with probability $\eta(x)$. The conditional error of any g is therefore at least $\min(\eta(x), 1 - \eta(x))$, and this floor is attained by the Bayes classifier $\text{sign}(2\eta(x) - 1)$. Integrating over x ,

$$R^* = \mathbb{E}[\min(\eta(X), 1 - \eta(X))].$$

Minimizing the empirical 0/1 risk directly is computationally hopeless: the loss $\mathbf{1}_{\{yf(x) \leq 0\}}$ is a discontinuous, non-convex function of f . The way out, already used for logistic regression and large-margin classifiers, is to replace the step function by a convex upper envelope of it.

Definition (φ -risk)

Let $\varphi : \mathbb{R} \rightarrow \mathbb{R}_+$ be a loss function. The φ -risk of a measurable $f : \mathcal{X} \rightarrow \mathbb{R}$ is

$$R_\varphi(f) = \mathbb{E}[\varphi(Yf(X))],$$

its minimal value over measurable functions is $R_\varphi^* = \min_{g \text{ measurable}} R_\varphi(g)$, and the *empirical φ -risk* on the sample S_n is

$$R_\varphi^n(f) = \frac{1}{n} \sum_{i=1}^n \varphi(Y_i f(X_i)).$$

The argument $u = yf(x)$ is the *margin*: it is positive when the classification is correct, and its magnitude measures confidence. Typical convex surrogates are decreasing functions of the margin, such as the hinge loss $\varphi(u) = \max(1 - u, 0)$, the logistic loss $\varphi(u) = \log(1 + e^{-u})$, and the squared loss $\varphi(u) = (1 - u)^2$. All of them dominate (up to scaling) the 0/1 loss, so a small φ -risk certainly bounds the classification error from above. But the real question is sharper: if we push the φ -risk all the way down to its minimum, do we land on a function whose *classification* error is also minimal? Nothing in the definition guarantees it; a badly chosen surrogate could prefer functions whose sign is wrong on a region where being wrong is cheap for φ but costly for the 0/1 loss.

12.2 Calibration: a small φ -risk ensures a small 0/1 risk

The following theorem answers the question in the affirmative for a large family of convex losses. The hypotheses are exactly designed to make the loss point in the right direction at the decision boundary: at zero margin φ must have a strictly negative slope, $\varphi'(0) < 0$, so it still rewards moving toward the correct sign there.

Theorem (Bartlett, Jordan, and McAuliffe, 2006)

Let $\varphi : \mathbb{R} \rightarrow \mathbb{R}_+$ be convex, differentiable at 0 with $\varphi'(0) < 0$. Let $f : \mathcal{X} \rightarrow \mathbb{R}$ be measurable such that

$$R_\varphi(f) = \min_{g \text{ measurable}} R_\varphi(g) = R_\varphi^*.$$

Then

$$R(f) = \min_{g \text{ measurable}} R(g) = R^*.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Read plainly, the theorem promises that the surrogate does not merely sit above the 0/1 loss, it points in the same direction. At every input the best possible response under φ already commits to the label the Bayes rule would choose, so driving the φ -risk down to its floor cannot strand us on a function whose classification is far from optimal. The entire burden rests on one hypothesis: that φ is still strictly decreasing as the margin crosses zero, $\varphi'(0) < 0$. Zero margin is the point of indecision, where the prediction is on the fence and the 0/1 loss most needs a committed sign. A loss that were flat or rising there could be driven to its minimum by a function that hedges at zero, or even leans the wrong way, precisely where doing so is most expensive for the classification error. Convexity then guarantees that this local preference set at the origin is never reversed further out, so a single condition at one point controls the sign everywhere.

The proof strategy is to show that a minimizer of the φ -risk is necessarily *sign-consistent* with η : where $\eta(x) > \frac{1}{2}$ the function must be positive, and where $\eta(x) < \frac{1}{2}$ it must be negative. Since these are precisely the signs of the Bayes classifier, f attains the Bayes risk. The key computation conditions on $X = x$ and compares f with $-f$:

$$R_\varphi(f \mid X = x) = \mathbb{E}[\varphi(Yf(X)) \mid X = x] = \eta(x) \varphi(f(x)) + (1 - \eta(x)) \varphi(-f(x)),$$

$$R_\varphi(-f \mid X = x) = \mathbb{E}[\varphi(-Yf(X)) \mid X = x] = \eta(x) \varphi(-f(x)) + (1 - \eta(x)) \varphi(f(x)),$$

so that

$$R_\varphi(f \mid X = x) - R_\varphi(-f \mid X = x) = [2\eta(x) - 1] \times [\varphi(f(x)) - \varphi(-f(x))].$$

Since f minimizes the φ -risk, we have $R_\varphi(f) \leq R_\varphi(-f)$, and localizing this comparison (replacing f by $-f$ only on measurable sets) forces the displayed difference to be non-positive almost surely: where $\eta(x) > \frac{1}{2}$ it gives $\varphi(f(x)) \leq \varphi(-f(x))$, and where $\eta(x) < \frac{1}{2}$ it gives $\varphi(f(x)) \geq \varphi(-f(x))$. Turning these comparisons of loss values into a statement about the sign of $f(x)$ is exactly where convexity and $\varphi'(0) < 0$ do their work; the complete argument is written out below.

Proof

Step 1: the conditional φ -risk. For $\eta \in [0, 1]$ define the conditional risk function

$$C_\eta(\alpha) = \eta \varphi(\alpha) + (1 - \eta) \varphi(-\alpha), \quad \alpha \in \mathbb{R},$$

so that, conditioning on X ,

$$R_\varphi(g) = \mathbb{E}[C_{\eta(X)}(g(X))] \quad \text{for every measurable } g.$$

Each C_η is convex on $\mathbb{R}$, as a convex combination of $\varphi(\alpha)$ and $\varphi(-\alpha)$, both convex in α ; being finite and convex it is also continuous. Since φ is differentiable at 0, so is C_η , with

$$C'_\eta(0) = \eta \varphi'(0) - (1 - \eta) \varphi'(0) = (2\eta - 1) \varphi'(0).$$

Write $H(\eta) = \inf_{\alpha \in \mathbb{R}} C_\eta(\alpha)$ for the optimal conditional risk.

Step 2: a global minimizer is a pointwise minimizer. We first check that

$$R_\varphi^* = \mathbb{E}[H(\eta(X))].$$

The inequality $R_\varphi(g) \geq \mathbb{E}[H(\eta(X))]$ holds for every measurable g , since $C_{\eta(x)}(g(x)) \geq H(\eta(x))$ pointwise. For the converse, let $Q_k = \{j/2^k : j \in \mathbb{Z}, |j| \leq 4^k\}$ be a nested sequence of finite grids whose union is dense in $\mathbb{R}$, and let $g_k(x)$ be the smallest element of Q_k minimizing $C_{\eta(x)}$ over Q_k ; each g_k is measurable because it is built from the finitely many measurable functions $x \mapsto C_{\eta(x)}(\alpha)$, $\alpha \in Q_k$. By continuity of C_η and density of $\bigcup_k Q_k$, the values $C_{\eta(x)}(g_k(x))$ decrease to $H(\eta(x))$ for every x ; they are bounded above by $C_{\eta(x)}(0) = \varphi(0)$ and below by 0, so monotone convergence gives $R_\varphi(g_k) \rightarrow \mathbb{E}[H(\eta(X))]$, proving the claim.

Now let f attain R_φ^* , and note that $R_\varphi(f) = R_\varphi^* \leq R_\varphi(0) = \varphi(0) < \infty$. The function $x \mapsto C_{\eta(x)}(f(x)) - H(\eta(x))$ is non-negative and its integral equals $R_\varphi(f) - \mathbb{E}[H(\eta(X))] = 0$, hence

$$C_{\eta(x)}(f(x)) = H(\eta(x)) \quad \text{for } P\text{-almost every } x.$$

Step 3: minimizers of C_η have the Bayes sign. We claim that if $\eta > \frac{1}{2}$, then every $\alpha \leq 0$ satisfies $C_\eta(\alpha) > H(\eta)$; symmetrically, if $\eta < \frac{1}{2}$, then every $\alpha \geq 0$ satisfies $C_\eta(\alpha) > H(\eta)$.

Suppose $\eta > \frac{1}{2}$. Then $C'_\eta(0) = (2\eta - 1)\varphi'(0) < 0$, and two consequences follow from convexity.

First, $C_\eta(0) > H(\eta)$: since $C'_\eta(0) < 0$, there exists $\beta > 0$ with $C_\eta(\beta) < C_\eta(0)$, so $C_\eta(0)$ is not the infimum.

Second, for every $\alpha < 0$ we have $C_\eta(\alpha) > C_\eta(0)$: convexity makes difference quotients non-decreasing, so

$$\frac{C_\eta(0) - C_\eta(\alpha)}{0 - \alpha} \leq C'_\eta(0) < 0,$$

and since $0 - \alpha > 0$ this forces $C_\eta(0) < C_\eta(\alpha)$.

Combining, every $\alpha \leq 0$ has $C_\eta(\alpha) \geq C_\eta(0) > H(\eta)$, which proves the claim. The case $\eta < \frac{1}{2}$ is identical after replacing α by $-\alpha$, because $C_\eta(\alpha) = C_{1-\eta}(-\alpha)$.

(This is the strictness the sketch above only gestured at: comparing f with $-f$ yields weak inequalities between loss values, which can be saturated on flat pieces of φ ; the assumption $\varphi'(0) < 0$, through convexity, is exactly what upgrades them to the strict sign statements needed here.)

Step 4: conclusion. By Steps 2 and 3, for almost every x with $\eta(x) > \frac{1}{2}$ we have $f(x) > 0$, and for almost every x with $\eta(x) < \frac{1}{2}$ we have $f(x) < 0$. The conditional 0/1 risk of f at x is

$$P(\text{sign } f(X) \neq Y \mid X = x) = \eta(x) \mathbf{1}\{f(x) \leq 0\} + (1 - \eta(x)) \mathbf{1}\{f(x) > 0\},$$

which equals $\min(\eta(x), 1 - \eta(x))$ on $\{\eta \neq \frac{1}{2}\}$ by the sign consistency just established, and equals $\frac{1}{2} = \min(\eta(x), 1 - \eta(x))$ automatically on $\{\eta = \frac{1}{2}\}$, whatever the value of f there. Integrating,

$$R(f) = \mathbb{E}[\min(\eta(X), 1 - \eta(X))] = R^*.$$

□

Remarks

- The theorem tells us that, if we knew P , minimizing the φ -risk would be a sound strategy even though our real target is the classification error. It licenses the surrogate.
- The assumptions on φ can be relaxed: the result holds for the broader class of *classification-calibrated* loss functions characterized by Bartlett, Jordan, and McAuliffe (2006). For convex losses, calibration turns out to be equivalent to differentiability at 0 with $\varphi'(0) < 0$.
- More is true, and more is needed in practice, since no estimator reaches R_φ^* exactly: Bartlett, Jordan, and McAuliffe (2006) prove a quantitative version, showing that if the excess φ -risk $R_\varphi(f) - R_\varphi^*$ is small, then the excess 0/1 risk $R(f) - R^*$ is small too. The remainder of the chapter therefore aims at controlling the excess φ -risk of estimators computed from a finite sample.

12.3 Empirical risk minimization

We do not know P , only the sample S_n , so we cannot minimize R_φ directly. To find a function with a small φ -risk, the natural candidate replaces the true risk by its empirical average and minimizes that over a class of functions $\mathcal{F}$ that we choose. This is the empirical risk minimization principle of Vapnik (1995), and the choice of $\mathcal{F}$ is where all the tension lives.

Definition (ERM estimator)

The *empirical risk minimization* (ERM) estimator on a functional class $\mathcal{F}$ is the solution (when it exists) of

$$\hat{f}_n = \arg \min_{f \in \mathcal{F}} R_\varphi^n(f).$$

Two questions decide whether this is a good idea.

- Is $R_\varphi^n(f)$ a good estimate of the true risk $R_\varphi(f)$? For a single fixed f the law of large numbers says yes, but $\hat{f}_n$ is selected using the data, so we need the estimate to be reliable uniformly over $\mathcal{F}$.
- Is $R_\varphi(\hat{f}_n)$ small? Not merely compared with the best of $\mathcal{F}$, but compared with R_φ^* ?

The second question splits into two parts according to the following decomposition, which organizes everything that follows:

$$R_\varphi(\hat{f}_n) - R_\varphi^* = \underbrace{R_\varphi(\hat{f}_n) - \inf_{f \in \mathcal{F}} R_\varphi(f)}_{\text{estimation error}} + \underbrace{\inf_{f \in \mathcal{F}} R_\varphi(f) - R_\varphi^*}_{\text{approximation error}}.$$

The approximation error is deterministic: it measures how well the class $\mathcal{F}$ can approach the best measurable function, and it can only decrease as $\mathcal{F}$ grows. The estimation error is statistical: it measures how much the finite sample misleads the minimizer within $\mathcal{F}$, and it grows with the size of $\mathcal{F}$. Choosing $\mathcal{F}$ is negotiating this trade-off.

12.4 Rademacher complexity

The ERM principle gives a good solution when $R_\varphi(\hat{f}_n)$ is close to the minimum achievable risk $\inf_{f \in \mathcal{F}} R_\varphi(f)$, and this can be ensured when $\mathcal{F}$ is not "too large". To turn that intuition into a theorem we need a quantitative measure of the "capacity" of $\mathcal{F}$. The measure used here asks: how well can the class correlate with pure noise?

Definition (Rademacher complexity)

The *Rademacher complexity* of a class of functions $\mathcal{F}$ is

$$\text{Rad}_n(\mathcal{F}) = \mathbb{E}_{X, \sigma} \left[\sup_{f \in \mathcal{F}} \frac{2}{n} \sum_{i=1}^n \sigma_i f(X_i) \right],$$

where the expectation is over the sample $(X_i)_{i=1, \dots, n}$ and over independent uniform $\{\pm 1\}$ -valued (Rademacher) random variables $(\sigma_i)_{i=1, \dots, n}$.

The random signs σ_i are pure noise, independent of the data and of each other. If $\mathcal{F}$ is rich enough to find, for every realization of the signs, a function whose values $f(X_i)$ track them, then $\text{Rad}_n(\mathcal{F})$ stays large: such a class can fit noise, and empirical risks computed over it cannot be trusted. If instead $\mathcal{F}$ is small, no single f can chase arbitrary sign patterns, the supremum stays close to zero, and empirical averages concentrate around their expectations uniformly over the class. Rademacher complexity is the right currency in which to price this uniformity, as the next theorem shows.

It helps to read the definition in units. The inner quantity $\frac{2}{n} \sum_i \sigma_i f(X_i)$ is, up to the factor of two, the empirical correlation between the function's values and a random relabeling of the sample. Fixing the signs, the supremum selects the member of $\mathcal{F}$ that aligns best with that particular pattern of noise; averaging over the signs then asks how well the class can pull off this alignment on average, across all 2^n possible labelings. A class that contains, for essentially every sign pattern, some function threading through it will score near the top; a rigid class able to produce only a few shapes is defeated by most patterns and scores near zero. Because the labels are random, a large value is not evidence of signal, it is evidence that the class can manufacture the appearance of signal from nothing, which is exactly the capacity we must charge for. This data-dependent measure refines the older combinatorial capacity notions of Vapnik (1995); the particular form used here, and the learning bound that follows, are due to Bartlett and Mendelson (2002).

It is worth pausing on why searching a class is what creates the danger, an intuition that Shawe-Taylor and Cristianini (2004) place at the center of their account of learning as the *detection of stable patterns*. A pattern is worth reporting only when it reflects the source that generated the data rather than an accident of the particular sample. A single fixed function is easy to trust, because the law of large numbers already concentrates its empirical

average around the truth; a function chosen for its fit to the sample is not, because the very act of choosing can seize on coincidence. Their sharpest illustration is the birthday paradox. Among twenty-eight people the probability that some pair shares a birthday exceeds one half, yet the probability that two of them share one *pre-specified* day is below one percent. Fixing the day only after inspecting the group, that is, searching over all 365 possible days for a match, turns a rare event into a likely one. Rademacher complexity is the price of exactly this freedom: it measures how much a class can exploit the license to pick its function after seeing the sample, which is why the same quantity that certifies a stable pattern also caps how badly the search can fool us.

12.5 A basic learning bound for ERM

Theorem (learning bound for Lipschitz losses)

Suppose φ is Lipschitz with constant L_φ :

$$\forall u, u' \in \mathbb{R}, \quad |\varphi(u) - \varphi(u')| \leq L_\varphi |u - u'|.$$

Then the φ -risk of the ERM estimator satisfies, on average over the sampling of the training set,

$$\underbrace{\mathbb{E}_{S_n} R_\varphi(\hat{f}_n) - R_\varphi^*}_{\text{excess } \varphi\text{-risk}} \leq \underbrace{4L_\varphi \text{Rad}_n(\mathcal{F})}_{\text{estimation error}} + \underbrace{\inf_{f \in \mathcal{F}} R_\varphi(f) - R_\varphi^*}_{\text{approximation error}}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Remark (where the bound comes from)

We state this result without proof; the standard argument has three moves. First, since $\hat{f}_n$ minimizes the empirical risk, its estimation error is at most twice the uniform deviation $\mathbb{E}_{S_n} \sup_{f \in \mathcal{F}} (R_\varphi(f) - R_\varphi^n(f))$. Second, the *symmetrization* device introduces an independent ghost sample and random signs, bounding this uniform deviation by the Rademacher complexity of the loss class $\{(x, y) \mapsto \varphi(yf(x)) : f \in \mathcal{F}\}$. Third, the *contraction principle* of Ledoux and Talagrand (1991) strips the L_φ -Lipschitz function φ at the price of the constant L_φ , leaving the Rademacher complexity of $\mathcal{F}$ itself. Collecting the constants yields the factor $4L_\varphi$. The symmetrization and concentration steps are by now standard machinery; Boucheron, Bousquet, and Lugosi (2005) survey them in a form tailored to classification.

The concentration step deserves to be named, since it is the engine that turns a statement about averages into a statement holding for the worst function in the class at once. The tool is a bounded-differences inequality, which Shawe-Taylor and Cristianini (2004) take as the analytic backbone of their treatment.

Theorem (McDiarmid's bounded-differences inequality)

Let $X_1, \dots, X_n$ be independent random variables taking values in a set $\mathcal{A}$, and let $F : \mathcal{A}^n \rightarrow \mathbb{R}$ change by a bounded amount whenever a single coordinate is altered:

$$\sup_{x_1, \dots, x_n, \hat{x}_i} |F(x_1, \dots, x_n) - F(x_1, \dots, \hat{x}_i, \dots, x_n)| \leq c_i, \quad 1 \leq i \leq n.$$

Then for every $\varepsilon > 0$,

$$P\{F(X_1, \dots, X_n) - \mathbb{E} F(X_1, \dots, X_n) \geq \varepsilon\} \leq \exp\left(\frac{-2\varepsilon^2}{\sum_{i=1}^n c_i^2}\right).$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Hoeffding's inequality is the special case $F(X_1, \dots, X_n) = \sum_i X_i$, and the deviation bound quoted for a single fixed function is nothing more than that case. The reason McDiarmid's form is the right one here is that the object we must control, the uniform deviation $\sup_{f \in \mathcal{F}} (R_\varphi(f) - R_\varphi^n(f))$, is a function of the whole sample that no single example can move by much: swapping one training point shifts each empirical average, and hence their supremum, by at most a term of order $1/n$. Feeding c_i of order $1/n$ into the inequality shows this supremum concentrates around its expectation at the rate $1/\sqrt{n}$, and that expectation is exactly what symmetrization rewrites as a Rademacher complexity. This is the source of the distribution-free $1/\sqrt{n}$ tail on the estimation error.

The theorem quantifies the trade-off announced above:

- $\mathcal{F}$ "large" means overfitting: the approximation error is small, but $\text{Rad}_n(\mathcal{F})$, and hence the estimation error, is large.
- $\mathcal{F}$ "small" means underfitting: the estimation error is small, but the approximation error is large.

Remark (from expectation to high probability, and a data-dependent capacity)

The bound above controls the excess risk on average over the draw of the training set. Shawe-Taylor and Cristianini (2004) give the companion statement that holds with high probability for *every* function in the class at once: fixing $\delta \in (0, 1)$, with probability at least $1 - \delta$ over the sample, every $f \in \mathcal{F}$ satisfies

$$R_\varphi(f) \leq R_\varphi^n(f) + \text{Rad}_n(\mathcal{F}) + \sqrt{\frac{\ln(2/\delta)}{2n}}.$$

Its proof is the recipe just described: bound the uniform deviation by its expectation with McDiarmid's inequality, then bound that expectation by the Rademacher complexity through symmetrization. The Rademacher term can itself be replaced by its *empirical* version

$$\widehat{\text{Rad}}_n(\mathcal{F}) = \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{F}} \frac{2}{n} \sum_{i=1}^n \sigma_i f(X_i) \mid X_1, \dots, X_n \right],$$

which conditions on the observed inputs and averages over the signs alone, at the cost of enlarging the confidence term to $3\sqrt{\ln(2/\delta)/2n}$. This empirical complexity is a quantity we can estimate directly from data by sampling sign patterns, so the capacity of the class becomes something we measure rather than postulate.

To use the bound we need classes $\mathcal{F}$ that are flexible enough to keep the approximation error moderate, yet whose Rademacher complexity we can actually compute. Balls of an RKHS are exactly such classes.

12.6 ERM in balls of an RKHS

Assume now that $\mathcal{X}$ is endowed with a positive definite kernel K , with RKHS $\mathcal{H}$ and feature map $x \mapsto K_x = K(x, \cdot)$, so that the reproducing property $f(x) = \langle f, K_x \rangle_{\mathcal{H}}$ holds for all $f \in \mathcal{H}$. As function class for the ERM we take the ball of radius B in the RKHS:

$$\mathcal{F}_B = \{f \in \mathcal{H} : \|f\|_{\mathcal{H}} \leq B\}.$$

The radius B is our knob on capacity. The following theorem shows that the Rademacher complexity of $\mathcal{F}_B$ is controlled by B and by the kernel alone, with no combinatorial argument anywhere: the reproducing property turns the supremum over the ball into a norm, and independence does the rest.

Before the proof, it is worth seeing why bounding a single number, the norm $\|f\|_{\mathcal{H}}$, should tame a whole infinite-dimensional class at all. The reproducing property answers this at every point at once. Writing $f(x) = \langle f, K_x \rangle_{\mathcal{H}}$ and applying Cauchy-Schwarz,

$$|f(x)| = |\langle f, K_x \rangle_{\mathcal{H}}| \leq \|f\|_{\mathcal{H}} \|K_x\|_{\mathcal{H}} = \|f\|_{\mathcal{H}} \sqrt{K(x, x)},$$

so a bound on the norm caps the height of every function in the ball, uniformly, by $B\sqrt{K(x, x)}$. The same inequality applied to the difference $f(x) - f(x') = \langle f, K_x - K_{x'} \rangle_{\mathcal{H}}$ controls how fast f can vary: two inputs whose feature vectors K_x and $K_{x'}$ lie close together must receive nearby values, with the allowed gap scaled by $\|f\|_{\mathcal{H}}$. The RKHS norm is therefore a smoothness budget, and the radius B is how much of it we permit any function to spend. A large ball buys wiggly functions that can lunge at individual data points; a small ball admits only gentle, slowly varying ones, which is precisely the noise-resistant regime the Rademacher bound rewards. The theorem below turns this qualitative picture into a number.

Theorem (capacity control of RKHS balls)

$$\text{Rad}_n(\mathcal{F}_B) \leq \frac{2B\sqrt{\mathbb{E} K(X, X)}}{\sqrt{n}}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Using the reproducing property and linearity of the inner product,

$$\text{Rad}_n(\mathcal{F}_B) = \mathbb{E}_{X, \sigma} \left[\sup_{f \in \mathcal{F}_B} \frac{2}{n} \sum_{i=1}^n \sigma_i f(X_i) \right] = \mathbb{E}_{X, \sigma} \left[\sup_{f \in \mathcal{F}_B} \left\langle f, \frac{2}{n} \sum_{i=1}^n \sigma_i K_{X_i} \right\rangle_{\mathcal{H}} \right].$$

For a fixed vector $g \in \mathcal{H}$, the Cauchy-Schwarz inequality gives $\langle f, g \rangle_{\mathcal{H}} \leq B\|g\|_{\mathcal{H}}$ for all $f \in \mathcal{F}_B$, with equality at $f = Bg/\|g\|_{\mathcal{H}}$ (and the supremum is trivially 0 = $B\|g\|_{\mathcal{H}}$ when $g = 0$). Hence

$$\text{Rad}_n(\mathcal{F}_B) = \mathbb{E}_{X, \sigma} \left[B \left\| \frac{2}{n} \sum_{i=1}^n \sigma_i K_{X_i} \right\|_{\mathcal{H}} \right] = \frac{2B}{n} \mathbb{E}_{X, \sigma} \left[\sqrt{\left\| \sum_{i=1}^n \sigma_i K_{X_i} \right\|_{\mathcal{H}}^2} \right].$$

Expanding the squared norm with $\langle K_{X_i}, K_{X_j} \rangle_{\mathcal{H}} = K(X_i, X_j)$, and applying Jensen's inequality to the concave square root,

$$\text{Rad}_n(\mathcal{F}_B) \leq \frac{2B}{n} \sqrt{\mathbb{E}_{X, \sigma} \left[\sum_{i,j=1}^n \sigma_i \sigma_j K(X_i, X_j) \right]}.$$

But $\mathbb{E}_{\sigma}[\sigma_i \sigma_j]$ is 1 if $i = j$, 0 otherwise, and the σ_i are independent of the X_i . Therefore

$$\text{Rad}_n(\mathcal{F}_B) \leq \frac{2B}{n} \sqrt{\mathbb{E}_X \left[\sum_{i,j=1}^n \mathbb{E}_{\sigma}[\sigma_i \sigma_j] K(X_i, X_j) \right]} = \frac{2B}{n} \sqrt{\mathbb{E}_X \sum_{i=1}^n K(X_i, X_i)} = \frac{2B\sqrt{\mathbb{E}_X K(X, X)}}{\sqrt{n}}.$$

□

The decay in $1/\sqrt{n}$ is the familiar parametric rate, yet $\mathcal{F}_B$ may be a ball in an infinite-dimensional space. What replaces dimension is the average diagonal of the kernel, $\mathbb{E} K(X, X)$, that is, the expected squared norm of the feature vector. In words, the capacity of the ball is fixed by just two quantities and nothing else: how large we let the functions grow, through the radius B , and how large the feature vectors are, through $\sqrt{\mathbb{E} K(X, X)}$, the

whole divided by $\sqrt{n}$ because averaging n independent sign terms shrinks their aligned part at exactly that rate. Dimension, the usual culprit behind overfitting, never appears. A function can inhabit an infinite-dimensional space and still be easy to learn, provided its norm is kept on a budget. Plugging the theorem into the learning bound gives the central statistical result of the chapter.

Corollary (basic learning bound in RKHS balls)

Suppose $K(X, X) \leq \kappa^2$ almost surely (for instance the Gaussian kernel, with $\kappa = 1$). Then the ERM estimator in $\mathcal{F}_B$ with an L_φ -Lipschitz loss satisfies

$$\mathbb{E} R_\varphi(\hat{f}_n) - R_\varphi^* \leq \frac{8L_\varphi \kappa B}{\sqrt{n}} + \left(\inf_{f \in \mathcal{F}_B} R_\varphi(f) - R_\varphi^* \right).$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Remarks

- B controls the trade-off between approximation and estimation error: enlarging the ball shrinks the approximation term and inflates the estimation term.
- The bound on the estimation error, $8L_\varphi \kappa B / \sqrt{n}$, is independent of the distribution P and decreases with n : whatever the problem, the statistical part of the risk is under uniform control.
- The approximation error is harder to analyze in general; it depends on how well the ball can approach a minimizer of the φ -risk, hence on the interplay between the kernel and P .
- In practice B , or the regularization parameter λ introduced next, is tuned by cross-validation.

Remark (the empirical version, and its combinatorial ancestor)

The theorem bounds the Rademacher complexity in expectation over the inputs. Conditioning instead on the observed sample, the identical argument, with the sample now held fixed, yields the fully empirical bound of Shawe-Taylor and Cristianini (2004),

$$\widehat{\text{Rad}}_n(\mathcal{F}_B) \leq \frac{2B}{n} \sqrt{\sum_{i=1}^n K(X_i, X_i)} = \frac{2B}{n} \sqrt{\text{tr } \mathbf{K}},$$

where $\mathbf{K} = (K(X_i, X_j))_{i,j}$ is the kernel matrix. This is computable from the data alone: it replaces the population diagonal $\mathbb{E} K(X, X)$ by the empirical trace of the Gram matrix, and it exposes the operational meaning of regularization. A learned function $f = \sum_i \alpha_i K_{X_i}$ has norm $\|f\|_{\mathcal{H}} = \sqrt{\alpha^\top \mathbf{K} \alpha}$, so shrinking $\alpha^\top \mathbf{K} \alpha$ literally shrinks the radius B of the ball the solution occupies, and with it the capacity term.

It is instructive that the pre-Rademacher theory reached the same verdict by a wholly combinatorial route. Vapnik controlled capacity through the VC dimension, the largest number of points a class can label in all possible ways, and for large-margin hyperplanes he proved a bound that echoes ours exactly. In the account of Schölkopf and Smola (2002), the canonical hyperplanes $x \mapsto \langle w, x \rangle$ with $\|w\| \leq \Lambda$, acting on data contained in a ball of radius R about the origin, have VC dimension

$$h \leq R^2 \Lambda^2.$$

The message is the one the RKHS-ball bound delivers: capacity is governed not by the ambient dimension, which may be infinite, but by the product of a size of the data, R^2 or $\mathbb{E} K(X, X)$, and a budget on the norm, Λ^2 or B^2 . Constraining $\|w\|$, equivalently $\|f\|_{\mathcal{H}}$, is what makes an infinite-dimensional class learnable, and it is why the support vector machine introduced in Chapter 5, Support Vector Machines minimizes the norm at all.

12.7 Fast rates: local complexity and stability

The corollary just proved is reassuring but pessimistic. It charges the estimation error at the rate $B/\sqrt{n}$, the same $1/\sqrt{n}$ that a single coin-flip average obeys and that the worst-case analyses of Chapter 12, VC Theory and Generalization also reach, no matter how favourable the problem. Yet the empirical risk minimizer does not roam the whole ball $\mathcal{F}_B$: it settles near the in-class risk minimizer, and functions near that optimum have small variance, so the sample pins them down far more reliably than it does a worst-case member of the ball. Two refinements exploit exactly this, and both can accelerate the rate from $n^{-1/2}$ toward n^{-1} . The first, *local Rademacher complexity*, measures capacity only inside a small-variance neighbourhood of the optimum and pays off under a variance condition on the loss. The second, *algorithmic stability*, abandons uniform convergence altogether and bounds the generalization gap of one specific estimator, which for the regularized kernel methods of Chapter 4, Kernel Ridge Regression and Smooth Losses is immediate. This section develops both, following Bartlett, Bousquet, and Mendelson (2005) and Bousquet and Elisseeff (2002).

12.7.1 Localization and the variance condition

Return to the symmetrization heuristic behind the basic bound. The global complexity $\text{Rad}_n(\mathcal{F})$ prices the ability of the class to chase noise across all of $\mathcal{F}$ at once. But the excess risk of the ERM is driven by the deviation of a single function, $\hat{f}_n$, whose excess loss is small in mean and, if the loss is well behaved, small in variance too. Concentration is sharper for small-variance quantities: the Bernstein and Talagrand inequalities replace the range by the variance in their leading term, so a function of variance v deviates from its mean by $O(\sqrt{v/n})$ rather than $O(\sqrt{1/n})$. Restricting the complexity to a variance ball captures precisely this gain.

Definition (local Rademacher complexity)

For a class $\mathcal{G}$ of functions and a radius $r > 0$, the *local Rademacher complexity* keeps only the members whose second moment is at most r :

$$\text{Rad}_n(\mathcal{G}; r) = \mathbb{E}_{X, \sigma} \left[\sup_{g \in \mathcal{G}, \mathbb{E} g^2 \leq r} \frac{2}{n} \sum_{i=1}^n \sigma_i g(X_i) \right].$$

The global complexity is the case $r = \infty$; shrinking r can only shrink the supremum, so $\text{Rad}_n(\mathcal{G}; r) \leq \text{Rad}_n(\mathcal{G})$.

Localization helps only if a small excess risk genuinely forces a small variance, so that the excess loss of the ERM lives in a variance ball of small r . That link is a hypothesis on the problem, not a theorem, and it is the exact condition that unlocks fast rates.

Definition (Bernstein variance condition)

Let $f^\star = \arg \min_{f \in \mathcal{F}} R_\varphi(f)$ be the in-class risk minimizer and let $g_f(x, y) = \varphi(yf(x)) - \varphi(yf^\star(x))$ be the excess loss of f . The class satisfies the (B, θ) -Bernstein condition, for constants $B > 0$ and $\theta \in (0, 1]$, if

$$\mathbb{E}[g_f^2] \leq B (\mathbb{E} g_f)^\theta = B (R_\varphi(f) - R_\varphi(f^\star))^\theta \quad \text{for every } f \in \mathcal{F}.$$

The strongest case $\theta = 1$ reads: the variance of the excess loss is at most B times the excess risk itself.

Read plainly, whenever a function is close to optimal in mean (small $\mathbb{E} g_f$) it is automatically close in mean square (small $\mathbb{E} g_f^2$), so the near-minimizers cluster in a thin variance shell that localization can exploit. The condition is not exotic. For the squared loss on a bounded convex class it holds with $\theta = 1$, because the excess risk equals $\|f - f^\star\|_{L_2}^2$, which also controls the variance of the excess loss. In binary classification it follows from the low-noise (margin) conditions on η taken up in Chapter 12, VC Theory and Generalization, and the calibration theory of the previous sections, due to Bartlett, Jordan, and McAuliffe (2006), converts such a margin condition into exactly this Bernstein bound on the excess φ -risk. When $\theta \rightarrow 0$ the condition is vacuous and no acceleration is possible; $\theta = 1$ buys the fastest rate.

12.7.2 The sub-root fixed point that sets the rate

With the variance condition in hand the rate is decided by a single scalar: the radius at which the local complexity stops shrinking faster than the radius itself. To state it we need the shape that local Rademacher complexities always take as a function of r .

Definition (sub-root function, critical radius)

A function $\psi : [0, \infty) \rightarrow [0, \infty)$ is *sub-root* if it is nondecreasing, not identically zero, and $r \mapsto \psi(r)/\sqrt{r}$ is nonincreasing. A sub-root function has a unique positive fixed point $r^\star$, the *critical radius*, solving

$$\psi(r^\star) = r^\star.$$

The map $r \mapsto \text{Rad}_n(\mathcal{G}; r)$, applied to the star-hull of the class so that the ratio condition holds, is sub-root: the supremum grows with r , but its $\sqrt{r}$ -normalized version falls, because widening the variance ball admits functions of larger norm only at a diminishing marginal rate. The theorem of the localized theory says the excess risk is set by where this sub-root bound meets the diagonal.

Theorem (fast rate from localization; Bartlett, Bousquet, and Mendelson, 2005)

Suppose the excess-loss class satisfies the $(B, 1)$ -Bernstein condition and that ψ is a sub-root upper bound on its local Rademacher complexity, $\text{Rad}_n(\mathcal{G}; r) \leq \psi(r)$, with critical radius $r^\star$. Then there are constants c_1, c_2 depending only on B and the range of the loss such that, with probability at least $1 - \delta$, the empirical risk minimizer obeys

$$R_\varphi(\hat{f}_n) - \inf_{f \in \mathcal{F}} R_\varphi(f) \leq c_1 r^\star + \frac{c_2 \ln(1/\delta)}{n}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The estimation error is therefore $O(r^\star)$, the critical radius, plus a benign $1/n$ term. Everything now rests on how the critical radius behaves, and for a kernel class it is read straight off the spectrum. The localized companion of the trace bound proved above, also due to Bartlett, Bousquet, and Mendelson (2005), controls the local complexity of the unit RKHS ball by the truncated eigenvalue sum of the kernel integral operator,

$$\text{Rad}_n(\mathcal{F}_1; r) \leq \sqrt{\frac{2}{n} \sum_{i=1}^{\infty} \min(\lambda_i, r)},$$

where $\lambda_1 \geq \lambda_2 \geq \dots$ are the Mercer eigenvalues of Chapter 18, Mercer's Theorem, Spectra, and Rates. The right-hand side is sub-root in r , so the rate is fixed by the fixed point of

$$r = \sqrt{\frac{1}{n} \sum_{i=1}^{\infty} \min(\lambda_i, r)},$$

where the constant has been dropped, since it only rescales $r^\star$. The truncation $\min(\lambda_i, r)$ is the whole mechanism: an eigen-direction with $\lambda_i > r$ is "large" and contributes its full r to the budget, while the tail directions with $\lambda_i < r$ contribute only their own small energy. The count of large directions, $d(r) = \#\{i : \lambda_i > r\}$, is the *effective dimension* at scale r , and the fixed point balances this growing dimension against the shrinking tail. When the spectrum decays polynomially that balance has a closed-form exponent, and it is genuinely faster than $n^{-1/2}$.

Example (critical radius for polynomial eigen-decay)

Kernel with Mercer eigenvalues $\lambda_i = i^{-2b}$, decay exponent $b = 1$ (so $\lambda_i = i^{-2}$, total trace $\sum_i \lambda_i = \zeta(2) = 1.6449$). Solve the fixed point $r = \sqrt{\frac{1}{n} \sum_i \min(\lambda_i, r)}$ for $n = 100, 1000, 10000$, and compare its decay with the slow rate $n^{-1/2}$ and the predicted fast rate $n^{-2b/(2b+1)} = n^{-2/3}$. All numbers from `checks/ch04-ex1.py`.

1. Fix the sub-root target. The right-hand side $\psi(r) = \sqrt{\frac{1}{n} \sum_i \min(i^{-2}, r)}$ is increasing in r ; the curve ψ starts above the diagonal near 0 and crosses it once, at $r^\star$, which bisection isolates.
2. Solve at three sample sizes. The fixed point $r^\star$ and the effective dimension $d(r^\star) = \#\{i : \lambda_i > r^\star\}$ are

n	$r^\star$	$d(r^\star)$	$n^{-1/2}$ (slow)
100	0.07035	3	0.1000
1000	0.01555	8	0.03162
10000	0.003387	17	0.01000

3. Read the empirical exponent. The log-log slope $-\Delta \ln r^\star / \Delta \ln n$ between successive rows is 0.655 then 0.662, climbing toward the predicted fast exponent $2b/(2b+1) = 0.6667$, and sitting well above the slow value 0.5.
4. Confirm against the reference curves. The ratio $r^\star/n^{-2/3}$ is 1.52, 1.56, 1.57, nearly constant, so $r^\star \propto n^{-2/3}$; meanwhile $r^\star/n^{-1/2}$ falls, $0.70 \rightarrow 0.49 \rightarrow 0.34$, so $r^\star$ decays strictly faster than the slow rate.
5. Steepen the spectrum. Repeating with $b = 2$ ($\lambda_i = i^{-4}$, trace $\zeta(4) = 1.0823$) lifts the measured slope to 0.771 then 0.785, tracking the higher target $2b/(2b+1) = 0.8$ and pushing toward the parametric ceiling n^{-1} .

Reading. The critical radius, and with it the excess risk, decays like $n^{-2/3}$ for this kernel, not $n^{-1/2}$: localization has turned the slow global bound into a genuinely fast rate. The engine is the effective dimension $d(r^\star)$, which grows only like $n^{1/3}$ (here 3, 8, 17), so the sample faces an effectively low-dimensional problem even though the RKHS is infinite-dimensional. The smoother the kernel (the larger b), the fewer the large eigen-directions and the faster the rate.

Verification artifact. `checks/example-ch04-example-11-1.json` records the example source hash and verification scope.

Remark (the rate, in one line)

For $\lambda_i \asymp i^{-2b}$ with $b > \frac{1}{2}$, splitting the sum at the crossover index $i \asymp r^{-1/(2b)}$ gives $\sum_i \min(\lambda_i, r) \asymp r^{1-1/(2b)}$, so the fixed-point equation $r^2 \asymp n^{-1} r^{1-1/(2b)}$ collapses to

$$r^\star \asymp n^{-\frac{2b}{2b+1}}.$$

As $b \rightarrow \frac{1}{2}^+$ this degrades to the slow $n^{-1/2}$; as $b \rightarrow \infty$, the smoother the kernel, the closer the rate approaches the parametric ceiling n^{-1} . Koltchinskii (2006) reaches the same critical-radius calculus through an iterative localization scheme, and Steinwart and Christmann (2008) carry it through to end-to-end oracle inequalities for the support vector machine.

12.7.3 Generalization from algorithmic stability

Localization still argues through uniform convergence over a class. A completely different route asks nothing about classes at all: it looks only at how much the *output* of a fixed learning algorithm moves when one training point is perturbed. If the algorithm is insensitive to any single example, then its empirical risk cannot sit far from its true risk, because no example was able to bend the fit toward itself. This is the stability programme of Bousquet and Elisseeff (2002), and it is tailor-made for the regularized kernel estimators of Chapter 4, Kernel Ridge Regression and Smooth Losses.

Definition (uniform stability)

Let A map a training sample $S = (z_1, \dots, z_n)$ to a predictor A_S , and let $S^{\setminus i}$ be S with its i -th example removed. The algorithm has *uniform stability* β_n with respect to a loss ℓ if

$$\sup_z |\ell(A_S, z) - \ell(A_{S^{\setminus i}}, z)| \leq \beta_n \quad \text{for all samples } S \text{ and all } i.$$

Theorem (stability implies generalization; Bousquet and Elisseeff, 2002)

If A has uniform stability β_n and the loss is bounded, $0 \leq \ell \leq M$, then the generalization gap is controlled in expectation,

$$|\mathbb{E}_S[R(A_S) - R^n(A_S)]| \leq 2\beta_n,$$

and, with probability at least $1 - \delta$ over the sample,

$$R(A_S) \leq R^n(A_S) + 2\beta_n + (4n\beta_n + M)\sqrt{\frac{\ln(1/\delta)}{2n}}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

For the bound to say anything, the stability must vanish faster than $1/\sqrt{n}$: a stability of order $1/n$ makes the expected gap $O(1/n)$, a fast rate, and keeps the high-probability term at $O(1/\sqrt{n})$. The decisive fact is that regularization in an RKHS delivers exactly $\beta_n = O(1/n)$, and the reason is the strong convexity that the norm penalty injects into the objective.

Proposition (stability of regularized kernel methods; Bousquet and Elisseeff, 2002)

Let $\ell(f, z)$ be convex and L -Lipschitz in the prediction $f(x)$, and let $K(x, x) \leq \kappa^2$. The regularized estimator

$$\hat{f}_S = \arg \min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \ell(f, z_i) + \lambda \|f\|_{\mathcal{H}}^2$$

has uniform stability

$$\beta_n \leq \frac{L^2 \kappa^2}{2\lambda n}.$$

In particular the support vector machine (hinge loss, $L = 1$) and kernel ridge regression (squared loss on a bounded range) are uniformly stable at the rate $1/n$ for fixed λ .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Write $R_\lambda(f) = \frac{1}{n} \sum_i \ell(f, z_i) + \lambda \|f\|_{\mathcal{H}}^2$ for the objective on S and $R_\lambda^{\setminus k}$ for the objective on $S^{\setminus k}$ (both empirical averages normalized by n), with minimizers $\hat{f} = \hat{f}_S$ and $\hat{g} = \hat{f}_{S^{\setminus k}}$. The penalty makes each objective 2λ -strongly convex in $\|\cdot\|_{\mathcal{H}}$, so at the minimizer $\hat{f}$,

$$R_\lambda(\hat{g}) - R_\lambda(\hat{f}) \geq \lambda \|\hat{g} - \hat{f}\|_{\mathcal{H}}^2,$$

and symmetrically $R_\lambda^{\setminus k}(\hat{f}) - R_\lambda^{\setminus k}(\hat{g}) \geq \lambda \|\hat{f} - \hat{g}\|_{\mathcal{H}}^2$. Adding the two inequalities, the regularization terms cancel and the empirical averages differ only in the single example z_k , leaving

$$2\lambda \|\hat{f} - \hat{g}\|_{\mathcal{H}}^2 \leq \frac{1}{n} |\ell(\hat{f}, z_k) - \ell(\hat{g}, z_k)| \leq \frac{L}{n} |\hat{f}(x_k) - \hat{g}(x_k)|.$$

The reproducing property and Cauchy-Schwarz bound the prediction gap by the norm gap, $|\hat{f}(x_k) - \hat{g}(x_k)| = |\langle \hat{f} - \hat{g}, K_{x_k} \rangle_{\mathcal{H}}| \leq \kappa \|\hat{f} - \hat{g}\|_{\mathcal{H}}$. Substituting and dividing by $\|\hat{f} - \hat{g}\|_{\mathcal{H}}$ gives

$$\|\hat{f} - \hat{g}\|_{\mathcal{H}} \leq \frac{L\kappa}{2\lambda n}.$$

Finally the loss is L -Lipschitz in the prediction and the prediction is κ -Lipschitz in the norm, so for any test point z , $|\ell(\hat{f}, z) - \ell(\hat{g}, z)| \leq L\kappa \|\hat{f} - \hat{g}\|_{\mathcal{H}} \leq L^2\kappa^2/(2\lambda n)$, which is the claimed β_n . $\square$

Two conclusions close the loop with the localized theory. The expected generalization gap of the support vector machine and of kernel ridge regression is at most $2\beta_n \leq L^2\kappa^2/(\lambda n)$, a clean $1/n$ bound obtained without a single covering number or Rademacher average, purely from the geometry of the regularized objective. And the appearance of λ in the denominator is the same trade-off seen throughout the chapter, now read through stability: a larger penalty shrinks the ball, stiffens the solution against perturbation, and improves the gap, at the price of the approximation error a smaller ball incurs. Balancing β_n against that approximation error, exactly as the radius B was tuned in the corollary, recovers the fast rates of the localized analysis from the algorithmic side, which is why Steinwart and Christmann (2008) treat stability and local complexity as two windows onto the same oracle inequality.

12.8 From constrained ERM to penalized ERM: regularization

The theory above analyzes ERM over the ball $\mathcal{F}_B$, which is the constrained minimization problem

$$\begin{cases} \min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \varphi(y_i f(x_i)) \\ \text{subject to } \|f\|_{\mathcal{H}} \leq B. \end{cases}$$

The algorithms of Chapter 4, Kernel Ridge Regression and Smooth Losses solve something else: an unconstrained problem with a norm penalty. The bridge between the two is convex duality. To make the problem practical we assume that φ is convex. The objective is then convex in f , since each map $f \mapsto \varphi(y_i \langle f, K_{x_i} \rangle_{\mathcal{H}})$ composes the convex φ with a continuous linear functional of f , and the constraint set is a convex ball. The problem is thus a convex problem in f for which strong duality holds (the interlude below develops the machinery; Slater's condition is trivially satisfied here since the constraint can be made strict). In particular, f solves the constrained problem if and only if it solves, for some dual parameter $\lambda \geq 0$, the unconstrained problem

$$\min_{f \in \mathcal{H}} \left\{ \frac{1}{n} \sum_{i=1}^n \varphi(y_i f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2 \right\}.$$

The penalized formulation is the one used in practice: it is unconstrained, the representer theorem reduces it to a finite-dimensional convex program in the coefficients of $f = \sum_i \alpha_i K_{x_i}$, and the parameter λ plays the role of $1/B$: larger λ means a smaller effective ball. The learning bounds for $\mathcal{F}_B$ therefore transfer to the penalized estimators our algorithms compute, and they explain *why* regularization works: the penalty is not a numerical convenience but a capacity constraint, and the corollary above prices exactly how much statistical safety each unit of $\|f\|_{\mathcal{H}}$ costs.

The dial is easiest to feel by turning it. In the interactive fit below the regularization parameter λ sets the effective norm budget, hence the radius of the RKHS ball the solution is allowed to live in: slide λ down to enlarge the ball

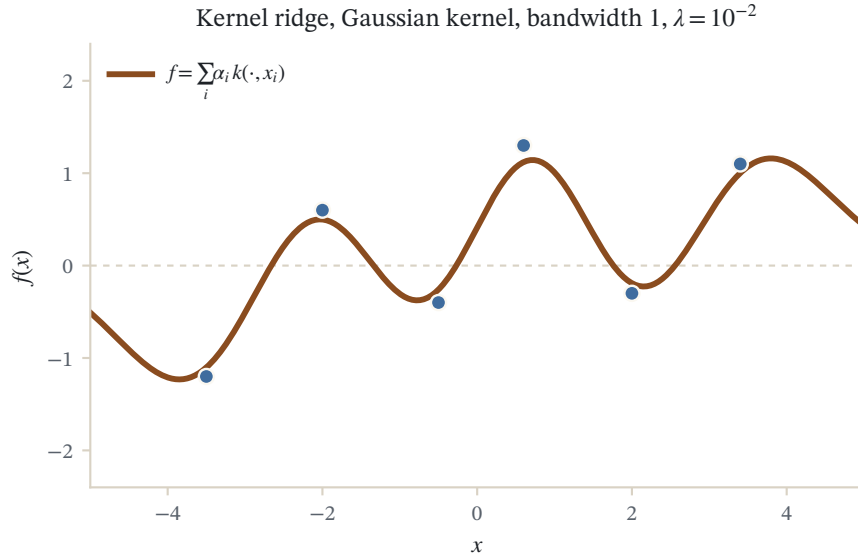


Figure 12.1: Kernel ridge regression on six points with a Gaussian kernel (bandwidth 1, ridge $\lambda = 10^{-2}$). The curve is the exact solution $f = \sum_i \alpha_i k(\cdot, x_i)$ with $\alpha = (K + \lambda n I)^{-1} y$.

and the curve grows wiggly, chasing every point, while sliding λ up shrinks the ball and smooths the curve out, which is the very capacity trade-off the corollary above quantifies.

This completes the design of large-margin classifiers, which is worth recording as a summary of the whole program:

$$\min_{f \in \mathcal{H}} \left\{ \frac{1}{n} \sum_{i=1}^n \varphi(y_i f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2 \right\}$$

- φ calibrated (for example decreasing with $\varphi'(0) < 0$) implies the φ -risk is a good proxy for the classification error, by the calibration theorem;
- φ convex, combined with the representer theorem, implies efficient algorithms.

12.9 Interlude: convex optimization and Lagrangian duality

The equivalence between constrained and penalized ERM invoked strong duality, and the derivation of the support vector machine in Chapter 5, Support Vector Machines leaned on it even harder. This interlude develops the needed facts from scratch, for a general constrained problem; no convexity is assumed until it is needed. The treatment follows the standard development in Boyd and Vandenberghe (2004).

12.9.1 Primal, dual, and the duality gap

The picture to keep in mind is two curves: the primal objective $f(x)$ is a curve whose minimum $f(x^*)$ we want; duality constructs a second function $q(\nu)$, the dual, every value of which sits below every value of f on the feasible set. *Strong duality* means the two meet:

$$\max_{\nu} q(\nu) = \min_x f(x).$$

Strong duality holds in most "reasonable cases" of convex optimization, in a sense made precise below. Two caveats accompany the picture. First, the relation between the primal optimum $x^\star$ and the dual optimum $\nu^\star$ is not always known a priori. Second, and more usefully, even without solving anything to optimality, any candidate pair $(\tilde{x}, \tilde{\nu})$ certifies its own quality through the *duality gap* $\delta(\tilde{x}, \tilde{\nu}) = f(\tilde{x}) - q(\tilde{\nu})$, since

$$0 \leq f(\tilde{x}) - f(x^\star) \leq \delta(\tilde{x}, \tilde{\nu}).$$

This makes duality a practical stopping criterion, not only a theoretical device. Dual problems are typically obtained by Lagrangian or Fenchel duality; we develop the Lagrangian route.

12.9.2 The Lagrangian and the dual function

Consider an equality and inequality constrained optimization problem over a variable $x \in \mathcal{X}$:

$$\begin{aligned} & \text{minimize} && f(x) \\ & \text{subject to} && h_i(x) = 0, \quad i = 1, \dots, m, \\ & && g_j(x) \leq 0, \quad j = 1, \dots, r, \end{aligned}$$

making no assumption on f, g and h . Denote by $f^\star$ the optimal value of the decision function under the constraints, that is, $f^\star = f(x^\star)$ if the minimum is reached at a global minimum $x^\star$.

Definition (Lagrangian, dual function)

The *Lagrangian* of this problem is the function $L : \mathcal{X} \times \mathbb{R}^m \times \mathbb{R}^r \rightarrow \mathbb{R}$ defined by

$$L(x, \lambda, \mu) = f(x) + \sum_{i=1}^m \lambda_i h_i(x) + \sum_{j=1}^r \mu_j g_j(x).$$

The *Lagrange dual function* $q : \mathbb{R}^m \times \mathbb{R}^r \rightarrow \mathbb{R} \cup \{-\infty\}$ is

$$q(\lambda, \mu) = \inf_{x \in \mathcal{X}} L(x, \lambda, \mu) = \inf_{x \in \mathcal{X}} \left(f(x) + \sum_{i=1}^m \lambda_i h_i(x) + \sum_{j=1}^r \mu_j g_j(x) \right).$$

The multipliers λ_i and μ_j price the constraints: the Lagrangian relaxes the hard constraints into linear charges on their violation. For the primal problem above, the *Lagrange dual problem* is

$$\begin{aligned} & \text{maximize} && q(\lambda, \mu) \\ & \text{subject to} && \mu \geq 0. \end{aligned}$$

Proposition

- q is concave in (λ, μ) , even if the original problem is not convex.
- The dual function yields lower bounds on the optimal value $f^\star$ of the original problem when μ is non-negative:

$$q(\lambda, \mu) \leq f^\star, \quad \forall \lambda \in \mathbb{R}^m, \forall \mu \in \mathbb{R}^r, \mu \geq 0.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Recall that

$$L(x, \lambda, \mu) = f(x) + \sum_{i=1}^m \lambda_i h_i(x) + \sum_{j=1}^r \mu_j g_j(x).$$

For each fixed x , the function $(\lambda, \mu) \mapsto L(x, \lambda, \mu)$ is affine, and therefore both convex and concave in (λ, μ) . The pointwise infimum of a family of concave functions is concave, therefore q is concave.

Let $\bar{x}$ be any feasible point, that is, $h(\bar{x}) = 0$ and $g(\bar{x}) \leq 0$. Then we have, for any λ and any $\mu \geq 0$,

$$\sum_{i=1}^m \lambda_i h_i(\bar{x}) + \sum_{j=1}^r \mu_j g_j(\bar{x}) \leq 0,$$

hence

$$L(\bar{x}, \lambda, \mu) = f(\bar{x}) + \sum_{i=1}^m \lambda_i h_i(\bar{x}) + \sum_{j=1}^r \mu_j g_j(\bar{x}) \leq f(\bar{x}),$$

and therefore

$$q(\lambda, \mu) = \inf_x L(x, \lambda, \mu) \leq L(\bar{x}, \lambda, \mu) \leq f(\bar{x})$$

for every feasible $\bar{x}$. Taking the infimum over feasible $\bar{x}$ gives $q(\lambda, \mu) \leq f^*$. $\square$

12.9.3 Weak and strong duality

Let q^* be the optimal value of the Lagrange dual problem. Each $q(\lambda, \mu)$ with $\mu \geq 0$ is a lower bound for f^* , and by definition q^* is the best lower bound so obtained. The following *weak duality* inequality therefore always holds:

$$q^* \leq f^*.$$

This inequality holds even when q^* or f^* is infinite. The difference $f^* - q^* \geq 0$ is called the *optimal duality gap* of the original problem.

We say that *strong duality* holds if the optimal duality gap is zero, that is,

$$q^* = f^*.$$

If strong duality holds, then the best lower bound obtainable from the Lagrange dual function is tight. Strong duality does not hold for general nonlinear problems, but it usually holds for convex problems; conditions that ensure it for convex problems are called *constraint qualifications*. When it holds, we have for all feasible primal and dual points $x, (\lambda, \mu)$,

$$q(\lambda, \mu) \leq q(\lambda^*, \mu^*) = L(x^*, \lambda^*, \mu^*) = f(x^*) \leq f(x),$$

so any feasible dual point brackets the optimum from below and any feasible primal point brackets it from above.

Theorem (Slater's constraint qualification)

Strong duality holds for a convex problem

$$\begin{aligned} & \text{minimize} && f(x) \\ & \text{subject to} && g_j(x) \leq 0, \quad j = 1, \dots, r, \\ & && Ax = b, \end{aligned}$$

if it is *strictly feasible*, that is, if there exists at least one feasible point satisfying

$$g_j(x) < 0, \quad j = 1, \dots, r, \quad Ax = b.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Remarks

- Slater's condition also ensures that the maximum q^* , when $q^* > -\infty$, is *attained*: there exists a point (λ^*, μ^*) with $q(\lambda^*, \mu^*) = q^* = f^*$.
- The condition can be sharpened: for example, strict feasibility is not required for affine constraints.
- Many other types of constraint qualifications exist.

12.9.4 Dual optimal pairs and complementary slackness

Suppose that strong duality holds, x^* is primal optimal, and (λ^*, μ^*) is dual optimal. Then we have

$$\begin{aligned} f(x^*) = q(\lambda^*, \mu^*) &= \inf_x \left(f(x) + \sum_{i=1}^m \lambda_i^* h_i(x) + \sum_{j=1}^r \mu_j^* g_j(x) \right) \\ &\leq f(x^*) + \sum_{i=1}^m \lambda_i^* h_i(x^*) + \sum_{j=1}^r \mu_j^* g_j(x^*) \\ &\leq f(x^*), \end{aligned}$$

where the last step uses $h_i(x^*) = 0$, $\mu_j^* \geq 0$ and $g_j(x^*) \leq 0$. Hence both inequalities are in fact equalities, and each carries information.

The first equality, read at the point x^* , shows that

$$L(x^*, \lambda^*, \mu^*) = \inf_x L(x, \lambda^*, \mu^*),$$

that is, x^* minimizes the Lagrangian at (λ^*, μ^*) : once the optimal multipliers are known, the constrained primal problem collapses to an unconstrained minimization. This is precisely the mechanism that converted ERM over an RKHS ball into penalized ERM, with λ the optimal multiplier of the norm constraint. The second equality gives the following important property.

Proposition (complementary slackness)

Each optimal Lagrange multiplier is zero unless the corresponding constraint is active at the optimum:

$$\mu_j^* g_j(x^*) = 0, \quad j = 1, \dots, r.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The second equality in the chain above states that $\sum_{i=1}^m \lambda_i^* h_i(x^*) + \sum_{j=1}^r \mu_j^* g_j(x^*) = 0$, and the equality-constraint terms vanish since $h_i(x^*) = 0$, leaving $\sum_{j=1}^r \mu_j^* g_j(x^*) = 0$. Each summand satisfies $\mu_j^* g_j(x^*) \leq 0$, being the product of a non-negative and a non-positive number; a sum of non-positive terms equals zero only if every term is zero. $\square$

Complementary slackness says that inactive constraints, those satisfied with room to spare at the optimum, cost nothing: their prices μ_j^* vanish. Only constraints that actually bind can carry positive multipliers. In kernel methods this is the source of *sparsity*: solutions supported on the few data points whose constraints are active.

12.10 The support vector machine in this light

This machinery is exactly what underlies the support vector machine (SVM) introduced in Chapter 5, Support Vector Machines. That classifier is the penalized ERM problem of this chapter with one particular choice of calibrated convex loss function, the hinge loss $\varphi(u) = \max(1 - u, 0)$. Historically it was the first “kernel method” for pattern recognition, and it remains the most popular, often state-of-the-art in performance. Two features distinguish it, and both are consequences of the duality theory developed here. The hinge loss leads to a *sparse* solution: by complementary slackness, not all points are involved in the decomposition $f = \sum_i \alpha_i K_{x_i}$, a form of compression that keeps only the support vectors. And the structure of the dual problem enables a particular algorithm for fast optimization, decomposition by chunking methods, which scales the approach to large training sets. Seen from the vantage of this chapter, the calibration, capacity, and duality results are precisely what those properties of Chapter 5, Support Vector Machines rest on.

A modern caveat: interpolation

The bounds of this chapter share a classical premise: capacity must be controlled, so a predictor that fits the training data exactly should generalize badly. Modern practice complicates that picture. Belkin, Hsu, Ma, and Mandal (2019) showed that test error, plotted against model size, can descend, rise to a peak at the point where the model just interpolates the data, and then descend a second time, the *double descent* curve; and kernel “ridgeless” regression, exact interpolation with no penalty, can generalize when the kernel’s spectrum decays at the right rate (Liang and Rakhlin, 2020). The RKHS-norm capacity here is not wrong, but it is only part of the story: *which* interpolant a method selects, and how the eigenvalue tail absorbs noise, matters as much as whether it interpolates. This is taken up in *Kernels in the Modern World*.

Further reading

The statistical learning theory behind these bounds is developed in Schölkopf and Smola (2002), *Learning with Kernels*, chapter 5, and in Shawe-Taylor and Cristianini (2004), *Kernel Methods for Pattern Analysis*, chapter 4. For the Rademacher-complexity machinery and its use for kernel classes see Bartlett and Mendelson (2002), and for the surrogate-loss calibration theory Bartlett, Jordan, and McAuliffe (2006). A comprehensive account is Steinwart and Christmann (2008), *Support Vector Machines*.

12.11 Common mistakes and practical implications

For **Learning Theory in RKHS Balls**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel

baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

12.12 Summary and further reading

This chapter established explain the central definitions and claims in Learning Theory in RKHS Balls; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Bartlett and McAuliffe 2006), (Vapnik 1995), (Bartlett and Mendelson 2002).

12.13 Exercises

1. warm-up The calibration theorem asks that φ be convex, differentiable at 0, with $\varphi'(0) < 0$. For each of the four standard surrogates, the hinge loss $\varphi(u) = \max(1 - u, 0)$, the logistic loss $\varphi(u) = \log(1 + e^{-u})$, the squared loss $\varphi(u) = (1 - u)^2$, and the exponential loss $\varphi(u) = e^{-u}$, compute $\varphi'(0)$ and confirm that each hypothesis holds. Then consider the one-sided loss $\varphi_0(u) = \max(-u, 0)$, which charges nothing once the margin is positive. Show that its right derivative at 0 is 0, so it fails the hypothesis $\varphi'(0) < 0$, and explain in one sentence, using the conditional risk $C_\eta(\alpha) = \eta \varphi(\alpha) + (1 - \eta) \varphi(-\alpha)$, why a function that hedges to $\alpha = 0$ at an input with $\eta(x) \neq \frac{1}{2}$ is then not penalized for choosing the wrong sign.
2. warm-up Rademacher complexity measures how well a class can correlate with pure noise. (a) Explain why the inner quantity $\frac{2}{n} \sum_{i=1}^n \sigma_i f(X_i)$ is, up to the factor of two, the empirical correlation between the values of f and a random relabeling of the sample. (b) For the single-function class $\mathcal{F} = \{f\}$, show directly from the definition that $\text{Rad}_n(\mathcal{F}) = 0$, using only $\mathbb{E}[\sigma_i] = 0$ and the independence of the signs from the data; interpret this as "a fixed pattern is free to trust." (c) In one paragraph, connect the birthday paradox of Shawe-Taylor and Cristianini (2004) to this quantity: why does searching a class over the 2^n sign patterns turn a coincidence into an expectation, and why is that search exactly what Rad_n prices?
3. proof This is the geometric heart of the RKHS-ball capacity bound. Fix a vector $g \in \mathcal{H}$ and let $\mathcal{F}_B = \{f \in \mathcal{H} : \|f\|_{\mathcal{H}} \leq B\}$. Prove that

$$\sup_{f \in \mathcal{F}_B} \langle f, g \rangle_{\mathcal{H}} = B \|g\|_{\mathcal{H}},$$

by showing $\leq$ with the Cauchy-Schwarz inequality and $\geq$ by exhibiting a specific $f \in \mathcal{F}_B$ that attains the value (treat $g = 0$ separately). Then explain in one line how, applied with $g = \frac{2}{n} \sum_{i=1}^n \sigma_i K_{X_i}$, this identity converts the supremum over the ball inside $\text{Rad}_n(\mathcal{F}_B)$ into the norm $\frac{2B}{n} \left\| \sum_i \sigma_i K_{X_i} \right\|_{\mathcal{H}}$, the step that removes f from the problem entirely. Hint

For the reverse inequality, when $g \neq 0$ take $f = B g / \|g\|_{\mathcal{H}}$; check that $\|f\|_{\mathcal{H}} = B$ so $f \in \mathcal{F}_B$, and compute $\langle f, g \rangle_{\mathcal{H}} = B \|g\|_{\mathcal{H}}$. When $g = 0$ both sides are 0.

4. proof Show that bounding a single number, the norm, controls a whole infinite-dimensional class pointwise. Using the reproducing property $f(x) = \langle f, K_x \rangle_{\mathcal{H}}$ and Cauchy-Schwarz, prove that every $f \in \mathcal{F}_B$ satisfies

$$|f(x)| \leq B \sqrt{K(x, x)} \quad \text{and} \quad |f(x) - f(x')| \leq B \|K_x - K_{x'}\|_{\mathcal{H}}$$

for all $x, x' \in \mathcal{X}$. Conclude that if $K(x, x) \leq \kappa^2$ everywhere then the whole ball is uniformly bounded by $B\kappa$, and interpret the second inequality as a smoothness budget: two inputs with nearby feature vectors must

receive nearby values, with the allowed gap scaled by the radius B . Finally, expand $\|K_x - K_{x'}\|_{\mathcal{H}}^2$ in terms of K to express the variation bound using kernel evaluations alone. Hint

For the second inequality write $f(x) - f(x') = \langle f, K_x - K_{x'} \rangle_{\mathcal{H}}$ and apply Cauchy-Schwarz. Then $\|K_x - K_{x'}\|_{\mathcal{H}}^2 = \langle K_x - K_{x'}, K_x - K_{x'} \rangle_{\mathcal{H}} = K(x, x) - 2K(x, x') + K(x', x')$.

5. proof The concentration step of the learning bound rests on McDiarmid's inequality applied to the uniform deviation. Assume the loss takes values in $[0, M]$, and define, on the sample $S_n = (Z_1, \dots, Z_n)$ with $Z_i = (X_i, Y_i)$,

$$F(Z_1, \dots, Z_n) = \sup_{f \in \mathcal{F}} \left(R_{\varphi}(f) - \frac{1}{n} \sum_{i=1}^n \varphi(Y_i f(X_i)) \right).$$

(a) Show that replacing a single example Z_i by any $\hat{Z}_i$ changes F by at most $c_i = M/n$; that is, F has the bounded-differences property with these constants. (b) Feed $c_i = M/n$ into McDiarmid's inequality and conclude that, for every $\varepsilon > 0$,

$$P\{F - \mathbb{E} F \geq \varepsilon\} \leq \exp\left(\frac{-2n\varepsilon^2}{M^2}\right),$$

so that the uniform deviation concentrates around its expectation at the rate $1/\sqrt{n}$. (c) State which later step of the argument rewrites $\mathbb{E} F$ as a Rademacher complexity. Hint

For (a), use that for any two functions $a(f), b(f)$, one has $|\sup_f a(f) - \sup_f b(f)| \leq \sup_f |a(f) - b(f)|$. Only the single term $\frac{1}{n} \varphi(Y_i f(X_i))$ changes, and it lies in $[0, M/n]$, so the empirical average moves by at most M/n . For (b), $\sum_i c_i^2 = n(M/n)^2 = M^2/n$. The expectation is bounded by the Rademacher complexity through the symmetrization device.

6. exploration Open the interactive kernel fit for this chapter (the λ slider on the figure in the regularization section, data-widget "kernel-lab"). The penalized problem it solves has λ playing the role of $1/B$: larger λ means a smaller effective RKHS ball. (a) Slide λ toward 0 and describe how the fitted curve behaves; slide it up and describe the change. Which regime corresponds to a large radius B , and which to a small one? (b) The corollary bounds the estimation error by $8L_{\varphi} \kappa B / \sqrt{n}$. Predict, before moving the slider, what should happen to the gap between training fit and out-of-sample behavior as you shrink the ball, and check it against the figure. (c) Using the empirical norm identity $\|f\|_{\mathcal{H}} = \sqrt{\alpha^\top \mathbf{K} \alpha}$ for $f = \sum_i \alpha_i K_{x_i}$, explain why increasing λ literally shrinks the radius of the ball the solution occupies.
7. challenge The theory analyzes constrained ERM over $\mathcal{F}_B$, but the algorithms solve the penalized problem. Using the Lagrangian duality interlude, make the reduction precise. (a) Write the constrained problem "minimize $\frac{1}{n} \sum_i \varphi(y_i f(x_i))$ subject to $\|f\|_{\mathcal{H}}^2 \leq B^2$ " with its single inequality constraint $g(f) = \|f\|_{\mathcal{H}}^2 - B^2 \leq 0$, form the Lagrangian, and identify the penalized objective $\frac{1}{n} \sum_i \varphi(y_i f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2$ as the minimization of $L(f, \lambda)$ over f at a fixed multiplier $\lambda \geq 0$. (b) Verify that Slater's condition holds, so strong duality gives an optimal $\lambda^* \geq 0$ for which the constrained and penalized minimizers coincide. (c) Invoke complementary slackness $\lambda^* g(f^*) = 0$ to show that either the norm constraint is active ($\|f^*\|_{\mathcal{H}} = B$) or $\lambda^* = 0$; interpret the second case as the unregularized minimizer already lying inside the ball. (d) Finally, relate this norm-budget picture to the margin-VC ancestor $h \leq R^2 \Lambda^2$ of Schölkopf and Smola (2002): argue that constraining $\|f\|_{\mathcal{H}} \leq B$ plays the role of the bound $\|w\| \leq \Lambda$, and $\mathbb{E} K(X, X)$ plays the role of the data radius R^2 , so both theories charge capacity as (data size) $\times$ (norm budget), never as ambient dimension. Hint

The Lagrangian is $L(f, \lambda) = \frac{1}{n} \sum_i \varphi(y_i f(x_i)) + \lambda (\|f\|_{\mathcal{H}}^2 - B^2)$; the constant $-\lambda B^2$ does not affect the minimizer over f , so $\arg \min_f L(f, \lambda)$ is exactly the penalized solution. For Slater, any f with $\|f\|_{\mathcal{H}} < B$ (for instance $f = 0$) is strictly feasible. For (c), the product of the non-negative λ^* and the non-positive $g(f^*)$ can vanish only if one factor is zero.

8. warm-up The fast-rate section trades the global rate $n^{-1/2}$ for the critical radius $r^\star$, the fixed point of $\psi(r) = \sqrt{\frac{1}{n} \sum_i \min(\lambda_i, r)}$. (a) Verify that ψ is sub-root: it is nondecreasing, and $\psi(r)/\sqrt{r}$ is nonincreasing, so the fixed point $r^\star$ is unique. (b) For a finite-rank kernel with exactly m nonzero eigenvalues, all equal to 1, and the rest zero, solve the fixed-point equation in closed form and show $r^\star = m/n$, the parametric rate of an m -dimensional model. (c) For the polynomial decay $\lambda_i = i^{-2b}$ with $b > \frac{1}{2}$, reproduce the scaling $r^\star \asymp n^{-2b/(2b+1)}$ by splitting the sum at the crossover index $i \asymp r^{-1/(2b)}$, and check that this exponent exceeds $\frac{1}{2}$ for every such b , so the localized rate always beats the global one. Hint

For (a), fix i and note that $r \mapsto \min(\lambda_i, r)/r = \min(\lambda_i/r, 1)$ is nonincreasing; summing preserves it, and $\psi(r)/\sqrt{r} = \sqrt{\frac{1}{nr} \sum_i \min(\lambda_i, r)}$. For (b), take $r \leq 1$ so that $\min(1, r) = r$ and $\sum_i \min(\lambda_i, r) = mr$; the equation $r^2 = mr/n$ gives $r^\star = m/n$. For (c), the head contributes $\asymp r \cdot r^{-1/(2b)}$ and the tail $\asymp r^{1-1/(2b)}$, both of order $r^{1-1/(2b)}$.

9. proof Complete the stability calculation for regularized kernel methods. (a) Show that the objective $R_\lambda(f) = \frac{1}{n} \sum_i \ell(f, z_i) + \lambda \|f\|_{\mathcal{H}}^2$ is 2λ -strongly convex in $\|\cdot\|_{\mathcal{H}}$ whenever ℓ is convex in f , and deduce that for its minimizer $\hat{f}$, $R_\lambda(g) - R_\lambda(\hat{f}) \geq \lambda \|g - \hat{f}\|_{\mathcal{H}}^2$ for every g . (b) Applying this to $\hat{f} = \hat{f}_S$ and its leave-one-out twin $\hat{g} = \hat{f}_{S \setminus k}$ and adding the two inequalities, derive $\|\hat{f} - \hat{g}\|_{\mathcal{H}} \leq L\kappa/(2\lambda n)$, using L -Lipschitzness of ℓ and $|f(x)| \leq \kappa \|f\|_{\mathcal{H}}$. (c) Conclude the uniform stability $\beta_n \leq L^2 \kappa^2 / (2\lambda n)$ and, via the Bousquet and Elisseeff (2002) theorem, that the expected generalization gap of the support vector machine is $O(1/(\lambda n))$. (d) In one sentence, explain why increasing λ improves the stability but need not improve the test risk. Hint

For (a), the map $f \mapsto \lambda \|f\|_{\mathcal{H}}^2$ has strong-convexity modulus 2λ , and a convex ℓ only adds nonnegative curvature; for the minimizer use $\langle \nabla R_\lambda(\hat{f}), g - \hat{f} \rangle \geq 0$. For (d), a larger λ shrinks the ball and stiffens the fit, lowering the estimation error but raising the approximation error.

13 VC Theory and Generalization

A learning machine only ever sees a finite sample, yet we ask it to perform on data it has never met. Why should low error on the training set say anything about error on the population? This chapter answers that question in its oldest and most influential form, the Vapnik-Chervonenkis theory. We begin with the gap between the risk we can measure and the risk we actually care about, show that a single function is tamed by the law of large numbers but a whole class of functions is not, and then build the machinery that closes the gap: consistency, uniform convergence, the growth function and VC dimension, and finally a genuine generalization bound assembled from symmetrization, a union bound, and the growth function. Along the way we collect the concentration inequalities the argument rests on and show that the capacity of a large-margin classifier is controlled by its margin, not by the dimension of the space it lives in. This is the classical companion to the modern Rademacher and margin route; the two chapters prove the same kind of statement with different tools, and it is worth seeing both.

13.1 The risk we want and the risk we can measure

Fix an input space $\mathcal{X}$, a label space $\mathcal{Y}$, and a distribution P on $\mathcal{X} \times \mathcal{Y}$ from which every example is drawn independently. A learning machine chooses a function f from some class $\mathcal{F}$, and we grade it with a loss, for pattern recognition the misclassification loss $\ell(f(x), y) = \frac{1}{2}|f(x) - y| \in \{0, 1\}$. The quantity we truly care about is the *expected risk*, the average loss over the whole population,

$$R[f] = \int \ell(f(x), y) dP(x, y) = \mathbb{E}_{(x,y) \sim P} \ell(f(x), y).$$

We cannot compute it, because P is unknown. All we hold is a sample $(x_1, y_1), \dots, (x_m, y_m)$, and the only risk we can evaluate is the *empirical risk*, the average loss on that sample,

$$R_{\text{emp}}[f] = \frac{1}{m} \sum_{i=1}^m \ell(f(x_i), y_i).$$

The principle of *empirical risk minimization* (ERM) is to return the $f_m \in \mathcal{F}$ that minimizes R_{emp} , in the hope that it also nearly minimizes R . Whether that hope is justified is the whole subject. A cautionary example shows it can fail completely. If $\mathcal{F}$ contains *all* functions from $\mathcal{X}$ to $\{\pm 1\}$, then on a continuous domain we may take f to reproduce the training labels exactly and predict -1 everywhere else. Its empirical risk is zero, yet a fresh test point almost never coincides with a training input, so the machine predicts -1 on it and does no better than chance. The values of f at the sample carry no information about its values elsewhere. This is the essence of the no-free-lunch theorem (Schölkopf and Smola 2002): with no restriction on $\mathcal{F}$, a small empirical risk is worthless. Learning is possible only when the class $\mathcal{F}$ is restricted, and the theory that follows is a precise account of how much restriction is enough and what a suitable measure of "how large is $\mathcal{F}$ " should be.

13.2 Concentration and the law of large numbers

Start with the easy case: one function f , fixed before we see any data. Then the per-example losses $\ell(f(x_i), y_i)$ are independent and identically distributed with mean $R[f]$, and their average $R_{\text{emp}}[f]$ is exactly the kind of quantity the law of large numbers governs. What we need is not just that the average converges but that it does so fast, with a probability of deviation that shrinks exponentially in the sample size. That is the content of a *concentration*

inequality. The most flexible one, and the parent of all the others we use, controls any function of independent inputs that does not depend too heavily on any single coordinate.

Theorem (McDiarmid, 1989)

Let $X_1, \dots, X_n$ be independent random variables taking values in a set A , and let $g : A^n \rightarrow \mathbb{R}$ have bounded differences: for each i there is a constant c_i with

$$\sup_{x_1, \dots, x_n, \hat{x}_i} |g(x_1, \dots, x_i, \dots, x_n) - g(x_1, \dots, \hat{x}_i, \dots, x_n)| \leq c_i.$$

Then for every $\epsilon > 0$,

$$\mathbb{P}\{g(X_1, \dots, X_n) - \mathbb{E} g(X_1, \dots, X_n) \geq \epsilon\} \leq \exp\left(\frac{-2\epsilon^2}{\sum_{i=1}^n c_i^2}\right).$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The theorem says a quantity that no single observation can move by much is sharply concentrated about its mean, and the certificate is just the vector $(c_1, \dots, c_n)$ of per-coordinate sensitivities. The classical inequality for sums drops out immediately, which is why we treat it as the special case rather than a separate result (Hoeffding 1963; Boucheron, Bousquet and Lugosi 2005).

Corollary (Hoeffding's inequality)

Let $Z_1, \dots, Z_m$ be independent with $Z_i \in [a_i, b_i]$, and let $\bar{Z} = \frac{1}{m} \sum_i Z_i$. Then for every $\epsilon > 0$,

$$\mathbb{P}\{\bar{Z} - \mathbb{E} \bar{Z} \geq \epsilon\} \leq \exp\left(\frac{-2m^2\epsilon^2}{\sum_{i=1}^m (b_i - a_i)^2}\right).$$

In particular, if every $Z_i \in [0, 1]$, then $\mathbb{P}\{|\bar{Z} - \mathbb{E} \bar{Z}| \geq \epsilon\} \leq 2e^{-2m\epsilon^2}$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Apply McDiarmid to $g(z_1, \dots, z_m) = \frac{1}{m} \sum_i z_i$. Replacing one coordinate $z_i \in [a_i, b_i]$ by another value in the same interval changes the average by at most $(b_i - a_i)/m$, so the bounded-difference constant is $c_i = (b_i - a_i)/m$ and $\sum_i c_i^2 = \frac{1}{m^2} \sum_i (b_i - a_i)^2$. Substituting into McDiarmid gives the one-sided bound. When every $Z_i \in [0, 1]$ each $c_i = 1/m$, so $\sum_i c_i^2 = 1/m$ and the exponent is $-2m\epsilon^2$; applying the same bound to $-g$ and adding the two tails yields the two-sided factor of 2. $\square$

Read with $Z_i = \ell(f(x_i), y_i) \in [0, 1]$ and $\mathbb{E} \bar{Z} = R[f]$, Hoeffding says that for any fixed f ,

$$\mathbb{P}\{|R_{\text{emp}}[f] - R[f]| \geq \epsilon\} \leq 2e^{-2m\epsilon^2}.$$

For one function the training error is an excellent estimate of the true error, and it converges exponentially fast. So where is the difficulty? The bound is fatally attached to a function chosen *before* the sample. The machine does the opposite: it inspects the sample and then chooses f_m to make $R_{\text{emp}}[f_m]$ small. That selected function is not fixed in advance, and among the many functions the class can implement there may well be one whose empirical risk happens to fall far below its true risk purely by chance. Concentration of each function separately does not rule out that one of them has gone badly wrong exactly where we looked.

13.3 Consistency and uniform convergence

To state precisely what we want, let f_{opt} minimize the true risk over $\mathcal{F}$ and let f_m minimize the empirical risk. We call ERM *consistent* if the risk of its output approaches the best available risk as the sample grows, $R[f_m] \rightarrow R[f_{\text{opt}}]$ in probability. Consistency is what "learning works" means: with enough data the machine does essentially as well as the best function in its repertoire. The two defining inequalities $R[f_{\text{opt}}] \leq R[f_m]$ (optimality of f_{opt}) and $R_{\text{emp}}[f_m] \leq R_{\text{emp}}[f_{\text{opt}}]$ (ERM picked f_m) show that the gap $R[f_m] - R[f_{\text{opt}}]$ is controlled once R_{emp} is close to R simultaneously at f_m and f_{opt} . Since we cannot know in advance which functions those will be, we are forced to ask for closeness at every function at once.

Theorem (Vapnik and Chervonenkis, 1971)

One-sided uniform convergence in probability,

$$\lim_{m \rightarrow \infty} \mathbb{P} \left\{ \sup_{f \in \mathcal{F}} (R[f] - R_{\text{emp}}[f]) > \epsilon \right\} = 0 \quad \text{for all } \epsilon > 0,$$

is a necessary and sufficient condition for the nontrivial consistency of empirical risk minimization.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

This is the pivot of the whole theory. It converts a question about an algorithm (does ERM learn?) into a question about a class of functions (does the empirical mean converge to the true mean uniformly over $\mathcal{F}$?). The dependence on $\mathcal{F}$ that we saw in the no-free-lunch example returns here in exact form: uniform convergence can hold for a small class and fail for a large one, and the rest of the chapter is about characterizing which classes are small enough. The abstract convergence statement is not something we want to verify by hand for each new machine, so we seek a combinatorial property of $\mathcal{F}$ that guarantees it.

13.4 The growth function and the VC dimension

The key realization is that on a finite sample a class of binary functions is itself effectively finite: only the pattern of outputs on the sample points can differ, and there are only so many patterns. This turns "how large is $\mathcal{F}$ " into a counting question.

Definition (shattering coefficient and growth function)

For a class $\mathcal{F}$ of functions into $\{\pm 1\}$ and a sample $x_1, \dots, x_m$, let $\mathcal{N}_{\mathcal{F}}(x_1, \dots, x_m)$ be the number of distinct label vectors $(f(x_1), \dots, f(x_m))$ realized as f ranges over $\mathcal{F}$. The *shattering coefficient* is the worst case over samples,

$$S_{\mathcal{F}}(m) = \max_{x_1, \dots, x_m} \mathcal{N}_{\mathcal{F}}(x_1, \dots, x_m),$$

and the *growth function* is its logarithm, $G_{\mathcal{F}}(m) = \ln S_{\mathcal{F}}(m)$.

Since each of m points carries one of two labels, $S_{\mathcal{F}}(m) \leq 2^m$ always. When $S_{\mathcal{F}}(m) = 2^m$ the class realizes every possible labeling of some m -point set, and we say $\mathcal{F}$ *shatters* that set. Shattering is the extreme of richness: the class can fit any labels whatsoever, so on such a set it has learned nothing beyond memorizing. The largest sample size at which this can still happen is the single most important capacity measure of the theory.

Definition (VC dimension)

The *VC dimension* h of $\mathcal{F}$ is the largest m for which $S_{\mathcal{F}}(m) = 2^m$, that is, the size of the largest set that $\mathcal{F}$ shatters; it is ∞ if arbitrarily large sets can be shattered.

Note carefully what shattering requires: there must *exist* some set of m points labeled in all 2^m ways, not that *every* m -point set can be. The value of the definition is that above the VC dimension the growth function stops being exponential. This is the polynomial bound of Vapnik and Chervonenkis (1971), also known as Sauer's lemma (Sauer 1972): for a class of finite VC dimension h ,

$$S_{\mathcal{F}}(m) \leq \sum_{i=0}^h \binom{m}{i} \leq \left(\frac{em}{h}\right)^h \quad \text{for } m \geq h,$$

so $G_{\mathcal{F}}(m)$ grows only logarithmically, like $h \ln(em/h)$, once m exceeds h . A finite VC dimension is therefore exactly the sublinear growth of $G_{\mathcal{F}}(m)$ that we will need to make the uniform-convergence probability vanish. Two small classes make the counting concrete.

Example (intervals on the line)

Let $\mathcal{F}$ label a point +1 when it lies inside a chosen interval $[a, b] \subset \mathbb{R}$ and -1 otherwise. On m ordered points an interval selects a contiguous block, so we count contiguous blocks (including the empty one). Take $m \in \{1, 2, 3, 4, 5\}$.

1. Count realizable labelings. The number of nonempty contiguous blocks of m ordered points is $\frac{m(m+1)}{2}$, and adding the empty block gives $S_{\mathcal{F}}(m) = \frac{m(m+1)}{2} + 1$. Brute-force enumeration confirms $S_{\mathcal{F}}(1..5) = 2, 4, 7, 11, 16$.
2. Test shattering at $m = 2$. Here $S_{\mathcal{F}}(2) = 4 = 2^2$: all four labelings of two points are realizable, so a two-point set is shattered.
3. Test shattering at $m = 3$. Now $S_{\mathcal{F}}(3) = 7 < 8 = 2^3$. The one missing labeling is $(+1, -1, +1)$: no single interval can include the outer two points while excluding the middle one. Three points are never shattered.
4. Read off the VC dimension. Two points can be shattered, three cannot, so $h = 2$.

Reading. The growth function is the exact polynomial $S_{\mathcal{F}}(m) = \frac{m(m+1)}{2} + 1$, quadratic rather than exponential, and the VC dimension is 2. The single un-realizable pattern $(+1, -1, +1)$ is the entire reason the class is learnable at all.

Verification artifact. checks/example-ch-vc-example-12-1.json records the example source hash and verification scope.

Example (halfplanes in the plane)

Let $\mathcal{F} = \{x \mapsto \text{sgn}(w^\top x + b)\}$ be the oriented affine halfplanes in $\mathbb{R}^2$. A labeling is realizable exactly when it is linearly separable, which we test by the feasibility of $y_i(w^\top x_i + b) \geq 1$. We probe three triangle points, four square corners, and four points with one inside the others.

1. Shatter a triangle. For three points in general position all $2^3 = 8$ labelings are separable, so a triangle is shattered.
2. Fail on four corners. For the square $(0, 0), (1, 0), (1, 1), (0, 1)$ only 14 of the 16 labelings are separable. The exception is the XOR pattern $(+1, -1, +1, -1)$, whose classes are the two diagonals, and two crossing diagonals cannot be split by a line.
3. Fail on a point inside a triangle. Placing the fourth point inside the triangle of the other three again realizes only 14 of 16 labelings. No four-point configuration is shattered.
4. Read off the VC dimension. Some three points are shattered, no four points are, so $h = 3$, matching the general formula $h = N + 1$ for hyperplanes in $\mathbb{R}^N$.

Reading. Halfplanes in $\mathbb{R}^2$ have VC dimension 3, one more than the input dimension. Both failure configurations, convex and non-convex, block the fourth point, which is why the count stops at three.

Verification artifact. checks/example-ch-vc-example-12-2.json records the example source hash and verification scope.

13.5 Deriving a VC bound

We can now assemble the generalization bound. We want to control the uniform deviation $\mathbb{P}\{\sup_f (R[f] - R_{\text{emp}}[f]) \geq \epsilon\}$, and the obstacle is that the supremum ranges over a possibly infinite class. Two devices remove the obstacle: symmetrization, which replaces the unknown true risk by a second empirical risk on a fictitious sample and thereby confines everything to $2m$ points, and the union bound, which handles the now finitely many labelings those points admit. We follow the account of Schölkopf and Smola (2002) and Vapnik (1998), keeping to pattern recognition with $\{\pm 1\}$ outputs.

The union bound is the trivial half. If $\mathcal{F} = \{f_1, \dots, f_N\}$ is finite, then the event that *some* f_k has a large deviation is contained in the union of the individual events, so its probability is at most the sum,

$$\mathbb{P}\left\{\max_{k \leq N} (R[f_k] - R_{\text{emp}}[f_k]) \geq \epsilon\right\} \leq \sum_{k=1}^N \mathbb{P}\{R[f_k] - R_{\text{emp}}[f_k] \geq \epsilon\} \leq N \cdot 2e^{-2m\epsilon^2},$$

each summand bounded by Hoeffding. The price of moving from one function to N is the factor N . The whole art is to make N small, and symmetrization is what makes it finite in the first place.

Lemma (symmetrization by a ghost sample)

Let $R'_{\text{emp}}[f]$ be the empirical risk on a second independent sample Z' of size m , the ghost sample. Provided $m\epsilon^2 \geq 2$,

$$\mathbb{P}\left\{\sup_{f \in \mathcal{F}} (R[f] - R_{\text{emp}}[f]) \geq \epsilon\right\} \leq 2 \mathbb{P}\left\{\sup_{f \in \mathcal{F}} (R'_{\text{emp}}[f] - R_{\text{emp}}[f]) \geq \frac{\epsilon}{2}\right\}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof (idea)

The true risk $R[f]$ is unknown but its natural stand-in is the ghost empirical risk, since $\mathbb{E}[R'_{\text{emp}}[f]] = R[f]$. If for the function achieving the supremum the deviation $R[f] - R_{\text{emp}}[f]$ exceeds ϵ , then with probability at least $\frac{1}{2}$ the ghost risk $R'_{\text{emp}}[f]$ lands within $\frac{\epsilon}{2}$ of $R[f]$, by Chebyshev applied to the single function f (this is where $m\epsilon^2 \geq 2$ is used), and then $R'_{\text{emp}}[f] - R_{\text{emp}}[f] \geq \frac{\epsilon}{2}$. Comparing the two-sample event to the one-sample event costs the factor 2. The gain is decisive: the right-hand side involves only the $2m$ points of $Z \cup Z'$, on which $\mathcal{F}$ realizes at most $S_{\mathcal{F}}(2m)$ distinct label vectors. $\square$

With the problem confined to $2m$ points, the class is effectively finite, with at most $N = \mathcal{N}_{\mathcal{F}}(Z \cup Z') \leq S_{\mathcal{F}}(2m)$ elements, and the union bound applies. Conditioning on the $2m$ points and averaging over random reassignments of which half is training and which is ghost (a permutation, or equivalently Rademacher, argument) produces a Hoeffding factor $e^{-m\epsilon^2/8}$ per surviving labeling. Combining the pieces gives the Vapnik-Chervonenkis bound.

Derivation (symmetrization, union bound, growth function)

Step 1: symmetrize. By the lemma, for $m\epsilon^2 \geq 2$,

$$\mathbb{P}\left\{\sup_f (R[f] - R_{\text{emp}}[f]) \geq \epsilon\right\} \leq 2 \mathbb{P}\left\{\sup_f (R'_{\text{emp}}[f] - R_{\text{emp}}[f]) \geq \frac{\epsilon}{2}\right\}.$$

Step 2: reduce to finitely many labelings. Condition on the pooled sample $Z \cup Z'$ of $2m$ points. The difference $R'_{\text{emp}}[f] - R_{\text{emp}}[f]$ depends on f only through its label vector on these $2m$ points, and there are at most $\mathcal{N}_{\mathcal{F}}(Z \cup Z') \leq S_{\mathcal{F}}(2m)$ such vectors. The supremum over $\mathcal{F}$ is thus a maximum over at most $S_{\mathcal{F}}(2m)$ effectively different functions.

Step 3: union bound plus Hoeffding. Randomly swapping each point between the training and ghost halves leaves the pooled sample fixed while turning $R'_{\text{emp}} - R_{\text{emp}}$ into an average of signed, bounded terms. For each fixed labeling, Hoeffding on these signs gives a tail $\leq 2e^{-m\epsilon^2/8}$, and the union bound over the $S_{\mathcal{F}}(2m)$ labelings multiplies it,

$$\mathbb{P}\left\{\sup_f (R'_{\text{emp}}[f] - R_{\text{emp}}[f]) \geq \frac{\epsilon}{2} \mid Z \cup Z'\right\} \leq 2 S_{\mathcal{F}}(2m) e^{-m\epsilon^2/8}.$$

Step 4: average out. Taking the expectation over the draw of $Z \cup Z'$ and folding in the factor 2 from Step 1,

$$\mathbb{P}\left\{\sup_f (R[f] - R_{\text{emp}}[f]) \geq \epsilon\right\} \leq 4 \mathbb{E}[N_{\mathcal{F}}(Z \cup Z')] e^{-m\epsilon^2/8} \leq 4 S_{\mathcal{F}}(2m) e^{-m\epsilon^2/8}.$$

□

The exponential factor $e^{-m\epsilon^2/8}$ fights the capacity factor $S_{\mathcal{F}}(2m)$. The bound is nontrivial exactly when $\ln S_{\mathcal{F}}(2m)$, the growth function, grows sublinearly in m , for then the exponential wins and the probability vanishes. By Sauer's lemma this is guaranteed whenever the VC dimension is finite, which is the promised link between capacity and learnability. Solving for the confidence level makes the statement usable: setting the right-hand side equal to δ and inverting gives a bound that holds with high probability for *every* function in the class at once.

Theorem (VC generalization bound)

For a class $\mathcal{F}$ of $\{\pm 1\}$ -valued functions, with probability at least $1 - \delta$ over the sample, every $f \in \mathcal{F}$ satisfies

$$R[f] \leq R_{\text{emp}}[f] + \sqrt{\frac{8}{m} \left(\ln S_{\mathcal{F}}(2m) + \ln \frac{4}{\delta} \right)}.$$

If $\mathcal{F}$ has VC dimension h , Sauer's lemma gives $\ln S_{\mathcal{F}}(2m) \leq h \ln(2em/h)$, so the confidence term is $O(\sqrt{(h \ln(m/h) + \ln(1/\delta))/m})$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The bound holds uniformly, so in particular at the empirical minimizer f_m , even though f_m was chosen after seeing the data. That universality is both its strength and its weakness: it applies to any function the machine could have returned, but because it ignores which one it actually returned, it is typically loose. It is instructive to put real numbers to it.

Example (the confidence term at work)

Take the interval class above, VC dimension $h = 2$, with a separable training set so $R_{\text{emp}}[f_m] = 0$. Use $m = 1000$ examples and confidence $1 - \delta = 0.95$, and compare against the halfplane class with $h = 3$.

1. Evaluate the exact growth function. For intervals, $S_{\mathcal{F}}(2m) = \frac{2m(2m+1)}{2} + 1 = 2001001$ at $m = 1000$, so $\ln S_{\mathcal{F}}(2m) = 14.5092$.
2. Add the confidence budget. The term $\ln(4/\delta) = \ln 80 = 4.382$ accounts for demanding that the bound hold with probability 0.95.
3. Assemble the interval bound. The confidence term is $\sqrt{\frac{8}{1000}(14.5092 + 4.382)} = \sqrt{0.15113} = 0.3888$, so with 95% confidence $R[f_m] \leq 0.389$.
4. Compare the halfplane class. With $h = 3$, Sauer gives $S_{\mathcal{F}}(2m) \leq \sum_{i=0}^3 \binom{2000}{i} = 1,333,335,001$, so $\ln S_{\mathcal{F}}(2m) \leq 21.0109$ and the term is $\sqrt{\frac{8}{1000}(21.0109 + 4.382)} = 0.4507$.

Reading. A zero-error classifier of VC dimension 2 still only certifies a true error below 0.389 at a thousand examples, and raising the capacity to $h = 3$ loosens the certificate to 0.451. The bound is honest but conservative: it is a worst-case guarantee over all distributions, and one more VC dimension costs real confidence. The dependence $\sqrt{h/m}$ is what makes small capacity valuable.

Verification artifact. checks/example-ch-vc-example-12-3.json records the example source hash and verification scope.

13.6 Structural risk minimization

The VC bound splits the true risk into two competing pieces: the empirical risk, which a richer class can always drive down, and the confidence term, which a richer class inflates through its larger growth function. Minimizing the empirical risk alone ignores the second piece and courts overfitting; minimizing the whole right-hand side balances them. But the capacity term is a property of the class, not of any individual function, so it cannot be minimized by moving f within a fixed $\mathcal{F}$. The remedy of Vapnik and Chervonenkis is to introduce a *structure*: a nested family of classes of increasing capacity,

$$\mathcal{F}_1 \subset \mathcal{F}_2 \subset \dots, \quad h_1 \leq h_2 \leq \dots,$$

and to minimize the bound jointly over the class index and the function within it. This is *structural risk minimization* (SRM).

Algorithm (structural risk minimization)

Input Nested classes $\mathcal{F}_1 \subset \mathcal{F}_2 \subset \dots$ with VC dimensions $h_1 \leq h_2 \leq \dots$; sample of size m ; confidence $1 - \delta$.

Output A class index $k^\star$ and a function $f^\star \in \mathcal{F}_{k^\star}$.

1. For each k , run ERM inside $\mathcal{F}_k$ to get $f_k = \arg \min_{f \in \mathcal{F}_k} R_{\text{emp}}[f]$.
2. For each k , evaluate the risk bound $B_k = R_{\text{emp}}[f_k] + \sqrt{\frac{8}{m}(h_k \ln(2em/h_k) + \ln(4/\delta))}$.
3. Return $k^\star = \arg \min_k B_k$ and $f^\star = f_{k^\star}$.

SRM turns the abstract trade-off into a concrete procedure: walk up the structure, and stop when the shrinking empirical risk no longer pays for the growing capacity term. A worked instance from Schölkopf and Smola (2002) selects a polynomial-kernel degree for a handwritten-character task (the USPS digits). Because the problem is essentially separable, the empirical risk term is negligible, and one chooses the kernel by minimizing the capacity term alone, using the margin-based VC bound of the next section as the capacity measure. Plotting that bound against the kernel degree tracks the measured test error closely, which is the payoff: a quantity computed from the training set alone predicts which model will generalize best. The bound need not be tight in absolute terms for this to work, only monotone in the right direction.

13.7 The margin controls capacity

A puzzle now presents itself. A support vector machine (see the SVM chapter) is a hyperplane in a feature space whose dimension may be enormous or infinite, and the VC dimension of hyperplanes in $\mathbb{R}^N$ is $N + 1$. Taken literally, the VC bound would then be vacuous for kernel machines. The resolution is that the SVM does not use arbitrary hyperplanes; it uses *large-margin* hyperplanes, and once a margin is imposed the capacity is governed by that margin and the radius of the data, with no reference to the dimension of the space.

Theorem (Vapnik, 1995)

Consider hyperplanes $\langle w, x \rangle = 0$ in canonical form with respect to a point set $X^\star = \{x_1, \dots, x_r\}$, meaning $\min_i |\langle w, x_i \rangle| = 1$. Let R be the radius of the smallest sphere centered at the origin containing $X^\star$. The decision functions $f_w(x) = \text{sgn}\langle w, x \rangle$ satisfying $\|w\| \leq \Lambda$ have VC dimension bounded by

$$h \leq R^2 \Lambda^2.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Suppose $x_1, \dots, x_r$ are shattered by canonical hyperplanes with $\|w\| \leq \Lambda$. Then for every labeling $(y_1, \dots, y_r) \in \{\pm 1\}^r$ there is a w with $\|w\| \leq \Lambda$ and $y_i \langle w, x_i \rangle \geq 1$ for all i . Summing these r inequalities,

$$r \leq \sum_{i=1}^r y_i \langle w, x_i \rangle = \left\langle w, \sum_{i=1}^r y_i x_i \right\rangle \leq \|w\| \left\| \sum_{i=1}^r y_i x_i \right\| \leq \Lambda \left\| \sum_{i=1}^r y_i x_i \right\|,$$

by Cauchy-Schwarz. Hence $\left\| \sum_{i=1}^r y_i x_i \right\| \geq r/\Lambda$ for every labeling. Now average the squared norm over labels drawn as independent, mean-zero signs:

$$\mathbb{E}_y \left\| \sum_{i=1}^r y_i x_i \right\|^2 = \sum_{i,j} \mathbb{E}[y_i y_j] \langle x_i, x_j \rangle = \sum_{i=1}^r \|x_i\|^2 \leq rR^2,$$

since $\mathbb{E}[y_i y_j] = \delta_{ij}$ and $\|x_i\| \leq R$. A quantity bounded below by $(r/\Lambda)^2$ for every labeling is in particular at most its own average, so $(r/\Lambda)^2 \leq rR^2$, giving $r \leq R^2 \Lambda^2$. Since h is the largest such r , $h \leq R^2 \Lambda^2$. $\square$

The dimension of the ambient space has vanished from the conclusion entirely. What remains is the product of the data radius and the weight-norm budget, and since the geometric margin of a canonical hyperplane is $1/\|w\|$, a bound $\|w\| \leq \Lambda$ is a lower bound on the margin. Small $\|w\|$, large margin, small VC dimension: this is the capacity argument that motivates maximizing the margin, and it is why kernel machines generalize despite living in vast feature spaces. One caveat is that the budget Λ must be fixed a priori for the theorem to apply as stated, whereas the SVM chooses $\|w\|$ from the data; making the argument rigorous for the data-dependent margin is the business of the fat-shattering and Rademacher refinements developed in the learning-theory chapter (Bartlett and Mendelson 2002; Shawe-Taylor and Cristianini 2004).

That refinement is the natural next step and the reason for the two-chapter split. The VC route we have followed bounds capacity by a single worst-case combinatorial number, the growth function, and pays for it with loose, distribution-free constants. The Rademacher route measures capacity by how well the class correlates with random noise on the actual sample, a data-dependent quantity that is both tighter and, as the birthday-paradox intuition of Shawe-Taylor and Cristianini (2004) makes vivid, an empirical stand-in for the VC dimension. The classical theory tells us *that* finite capacity suffices and names the mechanism; the modern theory measures capacity where it matters. Both rest on the same foundation laid here: a uniform law of large numbers, proved by concentration, over a class whose richness must be controlled. For the smoothness classes and approximation rates that decide how quickly the empirical risk itself can be driven down, see the chapter on Mercer kernels and rates.

13.8 Common mistakes and practical implications

For **VC Theory and Generalization**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

13.9 Summary and further reading

This chapter established explain the central definitions and claims in VC Theory and Generalization; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Vapnik and Chervonenkis 1971), (Vapnik 1995), (Vapnik 1998).

13.10 Exercises

Exercise 1 (warm-up)

Consider threshold functions on the line, $f_t(x) = \text{sgn}(x - t)$. Show that $S_{\mathcal{F}}(m) = m + 1$, that the class shatters one point but not two, and conclude that its VC dimension is 1.

Verification artifact. checks/example-ch-vc-example-ch-vc-4.json records the example source hash and verification scope.

Exercise 2 (warm-up)

Using the two-sided Hoeffding inequality for a loss in $[0, 1]$, how large must m be so that for a single fixed f , $|R_{\text{emp}}[f] - R[f]| \leq 0.05$ with probability at least 0.99? Explain why the answer does not involve any capacity measure.

Verification artifact. checks/example-ch-vc-example-ch-vc-5.json records the example source hash and verification scope.

Exercise 3 (medium)

Axis-aligned rectangles in $\mathbb{R}^2$ label a point +1 inside the rectangle. Show that this class shatters four points but not five, so its VC dimension is 4. *Hint: for the positive part, place four points as the extreme north, south, east, and west of a small cross; for the negative part, given any five points, one is never extreme in any of the four directions and so cannot be excluded on its own.*

Verification artifact. checks/example-ch-vc-example-ch-vc-6.json records the example source hash and verification scope.

Exercise 4 (medium)

Rederive the confidence interval of the VC generalization bound from $\mathbb{P}\{\sup_f (R[f] - R_{\text{emp}}[f]) \geq \epsilon\} \leq 4 S_{\mathcal{F}}(2m) e^{-m\epsilon^2/8}$ by setting the right-hand side equal to δ and solving for ϵ . Confirm you recover $\epsilon = \sqrt{\frac{8}{m} (\ln S_{\mathcal{F}}(2m) + \ln(4/\delta))}$.

Verification artifact. checks/example-ch-vc-example-ch-vc-7.json records the example source hash and verification scope.

Exercise 5 (medium)

Derive the two-sided bound $\mathbb{P}\{|\bar{Z} - \mathbb{E}\bar{Z}| \geq \epsilon\} \leq 2e^{-2m\epsilon^2}$ directly from the one-sided McDiarmid bound applied to $g(z) = \frac{1}{m} \sum_i z_i$ and to $-g$. Identify precisely where independence of the Z_i enters the argument.

Verification artifact. checks/example-ch-vc-example-ch-vc-8.json records the example source hash and verification scope.

Exercise 6 (hard)

In the margin bound $h \leq R^2 \Lambda^2$, suppose the data are rescaled by a factor $\alpha > 0$, $x_i \mapsto \alpha x_i$. Show that the canonical-form constraint forces $\|w\|$ to rescale as $w \mapsto w/\alpha$, so that $R \mapsto \alpha R$, $\Lambda \mapsto \Lambda/\alpha$, and the product $R^2 \Lambda^2$ is

invariant. Interpret this scale invariance: why should a capacity measure not change when we merely change units? *Hint: track what $\min_i |\langle w, x_i \rangle| = 1$ does to w under the rescaling.*

Verification artifact. checks/example-ch-vc-example-ch-vc-9.json records the example source hash and verification scope.

Exercise 7 (hard)

A practitioner trains n classifiers with random hyperparameters and reports the one with the best error on a held-out test set of size t . Model each test error as a mean of t independent Bernoulli losses. Use the union bound to show the reported error can underestimate the true error by roughly $\sqrt{\ln(n)/(2t)}$. Explain how this is the same phenomenon as overfitting the training set, now transferred to the test set. *Hint: this is the union bound of the chapter with $N = n$ fixed functions, and it is why tuning on the test set inflates optimism logarithmically in the number of trials.*

Verification artifact. checks/example-ch-vc-example-ch-vc-10.json records the example source hash and verification scope.

Exercise 8 (hard)

The VC entropy is $H_{\mathcal{F}}(m) = \mathbb{E} \ln \mathcal{N}_{\mathcal{F}}(x_1, \dots, x_m)$ and the annealed entropy is $H_{\mathcal{F}}^{\text{ann}}(m) = \ln \mathbb{E} \mathcal{N}_{\mathcal{F}}(x_1, \dots, x_m)$. Using Jensen's inequality, show $H_{\mathcal{F}}(m) \leq H_{\mathcal{F}}^{\text{ann}}(m) \leq G_{\mathcal{F}}(m)$, so that the growth-function condition $G_{\mathcal{F}}(m)/m \rightarrow 0$ is the strongest of the three sublinear-growth conditions and implies the others. *Hint: the logarithm is concave, and the maximum over samples dominates any average.*

Verification artifact. checks/example-ch-vc-example-ch-vc-11.json records the example source hash and verification scope.

14 Kernel PCA and Denoising

Much of data analysis happens before any label is available: we want to see the shape of a dataset, compress it to a few informative coordinates, or strip away the noise that obscures its structure. Principal component analysis answers all three at once by finding the directions of greatest variation, and replaying its derivation in a reproducing-kernel Hilbert space turns it into a nonlinear method. This chapter builds kernel PCA as a word-for-word translation of the classical algorithm, reads its spectrum as both the variance it captures and the reconstruction error it leaves behind, and uses that second reading to denoise by projecting away the low-variance directions in which noise tends to live.

14.1 Principal component analysis

We start from the classical algorithm, because kernel PCA is best understood as a word-for-word translation of it. Let $S = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ be a set of vectors in $\mathbb{R}^d$. PCA is a classical algorithm of multivariate statistics that finds a set of orthogonal directions capturing the maximum variance of the data. Its uses are ubiquitous: producing low-dimensional representations of high-dimensional points, visualizing a dataset by projecting it onto its first two principal directions, denoising, and preprocessing for other learning algorithms.

14.1.1 Formalization

Assume the data are centered; if they are not, we center them as a preprocessing step:

$$\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i = 0.$$

The object PCA optimizes over is a direction, but it is fruitful to think of a direction as the function it induces. The orthogonal projection onto a direction $\mathbf{w} \in \mathbb{R}^d$ is the function $h_{\mathbf{w}} : \mathbb{R}^d \rightarrow \mathbb{R}$ defined by

$$h_{\mathbf{w}}(\mathbf{x}) = \frac{\mathbf{x}^\top \mathbf{w}}{\|\mathbf{w}\|}.$$

Because the data are centered, the values $h_{\mathbf{w}}(\mathbf{x}_1), \dots, h_{\mathbf{w}}(\mathbf{x}_n)$ have zero mean, and the empirical variance captured by $h_{\mathbf{w}}$ is simply the mean of their squares:

$$\widehat{\text{var}}(h_{\mathbf{w}}) := \frac{1}{n} \sum_{i=1}^n h_{\mathbf{w}}(\mathbf{x}_i)^2 = \frac{1}{n} \sum_{i=1}^n \frac{(\mathbf{x}_i^\top \mathbf{w})^2}{\|\mathbf{w}\|^2}.$$

Definition (principal directions)

The i -th principal direction $\mathbf{w}_i$, for $i = 1, \dots, d$, is defined recursively by

$$\mathbf{w}_i = \arg \max_{\mathbf{w} \perp \{\mathbf{w}_1, \dots, \mathbf{w}_{i-1}\}} \widehat{\text{var}}(h_{\mathbf{w}}) \quad \text{s.t.} \quad \|\mathbf{w}\| = 1.$$

Each direction captures as much variance as possible while being orthogonal to all the previous ones.

14.1.2 Solution

The maximization has a classical closed-form answer. Let X be the $n \times d$ data matrix whose rows are the vectors $\mathbf{x}_1, \dots, \mathbf{x}_n$. Stacking the projections, we can write the variance as a Rayleigh quotient:

$$\widehat{\text{var}}(h_{\mathbf{w}}) = \frac{1}{n} \sum_{i=1}^n \frac{(\mathbf{x}_i^\top \mathbf{w})^2}{\|\mathbf{w}\|^2} = \frac{1}{n} \frac{\mathbf{w}^\top X^\top X \mathbf{w}}{\mathbf{w}^\top \mathbf{w}}.$$

Proposition (PCA as an eigenproblem)

The solutions of

$$\mathbf{w}_i = \arg \max_{\mathbf{w} \perp \{\mathbf{w}_1, \dots, \mathbf{w}_{i-1}\}} \mathbf{w}^\top X^\top X \mathbf{w} \quad \text{s.t.} \quad \|\mathbf{w}\| = 1$$

are the successive eigenvectors of $X^\top X$, ranked by decreasing eigenvalues.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

This is the standard variational characterization of the eigenvectors of a symmetric positive semidefinite matrix: the unit vector maximizing the quadratic form is the leading eigenvector, and constraining orthogonality to the previous maximizers moves us down the spectrum one eigenvector at a time. Note the matrix being diagonalized, $X^\top X$, is n times the empirical covariance matrix of the (centered) data. Everything PCA needs is therefore contained in second-order statistics, and, crucially for what follows, those statistics are built entirely from inner products between data points.

14.2 Kernel PCA

Classical PCA can only find linear structure, and it only applies to data living in $\mathbb{R}^d$. Both restrictions disappear if we replay the derivation in an RKHS, the observation that turned PCA into a nonlinear method (Schölkopf, Smola, and Müller, 1998). Let $\mathbf{x}_1, \dots, \mathbf{x}_n$ be data points in a set $\mathcal{X}$, let $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ be a positive definite kernel with RKHS $\mathcal{H}$, and let $\varphi : \mathcal{X} \rightarrow \mathcal{H}$ denote the corresponding embedding $\varphi(\mathbf{x}) = K(\mathbf{x}, \cdot)$.

14.2.1 Formalization in the RKHS

Each ingredient of PCA translates term by term. Centering the data becomes centering the embedded data: we assume

$$\frac{1}{n} \sum_{i=1}^n \varphi(\mathbf{x}_i) = 0,$$

which, as we will see below, can be enforced by manipulating the kernel matrix rather than the points themselves. A direction is now an element $f \in \mathcal{H}$, and the orthogonal projection onto it is the function $h_f : \mathcal{X} \rightarrow \mathbb{R}$ defined by

$$h_f(\mathbf{x}) = \frac{\langle \varphi(\mathbf{x}), f \rangle_{\mathcal{H}}}{\|f\|_{\mathcal{H}}},$$

the exact analogue of $h_{\mathbf{w}}(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} / \|\mathbf{w}\|$. The empirical variance captured by h_f is

$$\widehat{\text{var}}(h_f) := \frac{1}{n} \sum_{i=1}^n \frac{\langle \varphi(\mathbf{x}_i), f \rangle_{\mathcal{H}}^2}{\|f\|_{\mathcal{H}}^2}.$$

And now the reproducing property pays off: $\langle \varphi(\mathbf{x}_i), f \rangle_{\mathcal{H}} = f(\mathbf{x}_i)$. The abstract inner product with a direction in a possibly infinite-dimensional space is just the evaluation of the function f at the data point, so

$$\widehat{\text{var}}(h_f) = \frac{1}{n} \sum_{i=1}^n \frac{f(\mathbf{x}_i)^2}{\|f\|_{\mathcal{H}}^2}.$$

Definition (kernel principal directions)

The i -th principal direction f_i , for $i = 1, \dots, n$, is defined by

$$f_i = \arg \max_{f \perp \{f_1, \dots, f_{i-1}\}} \sum_{i=1}^n f(\mathbf{x}_i)^2 \quad \text{s.t.} \quad \|f\|_{\mathcal{H}} = 1,$$

where orthogonality is in the RKHS inner product.

14.2.2 Sanity check: kernel PCA with the linear kernel is PCA

Before solving the problem, let us verify that we have not changed it. Take $K(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{y}$, the linear kernel on $\mathbb{R}^d$. Its RKHS $\mathcal{H}$ is the set of linear functions

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x},$$

endowed with the norm $\|f_{\mathbf{w}}\|_{\mathcal{H}} = \|\mathbf{w}\|_{\mathbb{R}^d}$. Therefore

$$\widehat{\text{var}}(h_{\mathbf{w}}) = \frac{1}{n} \sum_{i=1}^n \frac{(\mathbf{x}_i^\top \mathbf{w})^2}{\|\mathbf{w}\|^2} = \frac{1}{n \|\mathbf{w}\|^2} \sum_{i=1}^n f_{\mathbf{w}}(\mathbf{x}_i)^2,$$

which is precisely the kernel PCA objective. Moreover $\mathbf{w} \perp \mathbf{w}'$ if and only if $f_{\mathbf{w}} \perp f_{\mathbf{w}'}$ in $\mathcal{H}$, so the orthogonality constraints coincide too. Kernel PCA with the linear kernel recovers classical PCA exactly.

14.2.3 The representer argument

The optimization over $f \in \mathcal{H}$ looks infinite-dimensional, but the representer theorem rescues us, just as it did in the supervised setting. The objective $\sum_i f(\mathbf{x}_i)^2$ depends on f only through its values at the data points, and the constraint involves only the RKHS norm; the same is true of the orthogonality constraints once the previous f_j are known to be spanned by the $\varphi(\mathbf{x}_j)$. Checking that the representer theorem indeed applies to this constrained maximization is a worthwhile exercise, and we leave it as one: any component of f orthogonal to the span of $\{\varphi(\mathbf{x}_1), \dots, \varphi(\mathbf{x}_n)\}$ does not change any $f(\mathbf{x}_i)$ and can only waste norm, so at the optimum it vanishes. Hence for each $i = 1, \dots, n$ we may write

$$\forall \mathbf{x} \in \mathcal{X}, \quad f_i(\mathbf{x}) = \sum_{j=1}^n \alpha_{i,j} K(\mathbf{x}_j, \mathbf{x}),$$

with a coefficient vector $\alpha_i = (\alpha_{i,1}, \dots, \alpha_{i,n})^\top \in \mathbb{R}^n$.

14 Kernel PCA and Denoising

Every quantity in the problem now becomes a quadratic form in α_i involving the Gram matrix K with entries $[K]_{kl} = K(\mathbf{x}_k, \mathbf{x}_l)$. For the norm,

$$\|f_i\|_{\mathcal{H}}^2 = \sum_{k,l=1}^n \alpha_{i,k} \alpha_{i,l} K(\mathbf{x}_k, \mathbf{x}_l) = \alpha_i^\top K \alpha_i.$$

For the variance term, $f_i(\mathbf{x}_k) = [K\alpha_i]_k$, so

$$\sum_{k=1}^n f_i(\mathbf{x}_k)^2 = \alpha_i^\top K^2 \alpha_i.$$

And for the orthogonality constraints,

$$\langle f_i, f_j \rangle_{\mathcal{H}} = \alpha_i^\top K \alpha_j.$$

14.2.4 Solving the finite-dimensional problem

Kernel PCA has become a sequence of maximizations in $\mathbb{R}^n$:

$$\alpha_i = \arg \max_{\alpha \in \mathbb{R}^n} \alpha^\top K^2 \alpha \quad \text{s.t.} \quad \alpha_i^\top K \alpha_j = 0 \text{ for } j = 1, \dots, i-1, \quad \alpha_i^\top K \alpha_i = 1.$$

This is almost a standard eigenproblem, except that the constraints use the K -weighted inner product rather than the Euclidean one. A change of variable straightens this out. Compute the eigenvalue decomposition of the kernel matrix,

$$K = U \Delta U^\top, \quad \Delta_1 \geq \dots \geq \Delta_n \geq 0,$$

where the columns $\mathbf{u}_1, \dots, \mathbf{u}_n$ of U are orthonormal eigenvectors, and define $K^{1/2} = U \Delta^{1/2} U^\top$. Setting $\beta = K^{1/2} \alpha$ turns $\alpha^\top K^2 \alpha$ into $\beta^\top K \beta$, and the constraints $\alpha^\top K \alpha'$ into plain Euclidean inner products $\beta^\top \beta'$. The problem becomes

$$\beta_i = \arg \max_{\beta \in \mathbb{R}^n} \beta^\top K \beta \quad \text{s.t.} \quad \beta_i^\top \beta_j = 0 \text{ for } j = 1, \dots, i-1, \quad \beta_i^\top \beta_i = 1,$$

which is exactly the variational characterization of the eigenvectors of K we met for classical PCA. Thus $\beta_i = \mathbf{u}_i$, the i -th eigenvector, is a solution. Undoing the change of variable, and using $K^{1/2} \mathbf{u}_i = \Delta_i^{1/2} \mathbf{u}_i$, we obtain

$$\alpha_i = \frac{1}{\sqrt{\Delta_i}} \mathbf{u}_i.$$

Theorem (kernel PCA)

Let $K = U \Delta U^\top$ be the eigendecomposition of the centered Gram matrix, with eigenvalues $\Delta_1 \geq \dots \geq \Delta_n \geq 0$ and orthonormal eigenvectors $\mathbf{u}_1, \dots, \mathbf{u}_n$. Then the i -th kernel principal direction is

$$f_i(\mathbf{x}) = \sum_{j=1}^n \alpha_{i,j} K(\mathbf{x}_j, \mathbf{x}), \quad \alpha_i = \frac{\mathbf{u}_i}{\sqrt{\Delta_i}},$$

and the vector of projections of the n data points onto the i -th direction is $K \alpha_i = \sqrt{\Delta_i} \mathbf{u}_i$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

In summary, the algorithm is short. It never forms a feature vector: every step is a manipulation of the $n \times n$ kernel matrix, and the only place a new point $\mathbf{x}$ enters is through its kernel values against the training set.

Algorithm (kernel PCA)

Input Kernel K , data $\mathbf{x}_1, \dots, \mathbf{x}_n$, number of components k , and a point $\mathbf{x}$ to project.

Output Coefficient vectors $\alpha_1, \dots, \alpha_k$ and the coordinates of $\mathbf{x}$ on the top k kernel principal components.

1. Form the Gram matrix $[K]_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$.
2. Center it: $K_c = JKJ$ with $J = I - \frac{1}{n}\mathbf{1}\mathbf{1}^T$.
3. Eigendecompose $K_c = U\Delta U^T$ with $\Delta_1 \geq \dots \geq \Delta_n \geq 0$ and orthonormal columns $\mathbf{u}_1, \dots, \mathbf{u}_n$.
4. Normalize $\alpha_i = \mathbf{u}_i / \sqrt{\Delta_i}$ for $i = 1, \dots, k$, so that each direction f_i has unit RKHS norm.
5. Read the training coordinates on component i as $K_c \alpha_i = \sqrt{\Delta_i} \mathbf{u}_i$.
6. Project $\mathbf{x}$: center its kernel vector, $\tilde{k}_j = K(\mathbf{x}, \mathbf{x}_j) - \frac{1}{n} \sum_l K(\mathbf{x}, \mathbf{x}_l) - \frac{1}{n} \sum_m K(\mathbf{x}_m, \mathbf{x}_j) + \frac{1}{n^2} \sum_{m,l} K(\mathbf{x}_m, \mathbf{x}_l)$, and return the coordinate $\alpha_i^T \tilde{\mathbf{k}}$ on each component $i = 1, \dots, k$.

It is worth pausing on what this spectrum means, because it is the whole content of the method. The centered Gram matrix records, in its entry (i, j) , how similar points i and j look through the kernel, and kernel PCA does nothing more than diagonalize it. Its leading eigenvector is the pattern the kernel's similarities most agree on, the weighting of the points whose mutual similarities reinforce one another most strongly, which is exactly the direction of greatest spread in feature space. Successive eigenvectors pick up progressively weaker patterns, each orthogonal to the ones before, and the eigenvalue Δ_i reports how much variance the i -th component captures. The two facts that make the method usable both live in this matrix. The coefficients $\alpha_i = \mathbf{u}_i / \sqrt{\Delta_i}$ rescale each eigenvector so that the corresponding direction f_i has unit RKHS norm, and the projections $K\alpha_i = \sqrt{\Delta_i} \mathbf{u}_i$ are just the eigenvector again, stretched by $\sqrt{\Delta_i}$: plotting the top two of them against each other is the kernel-PCA scatter that reveals nonlinear structure a linear projection would miss.

A three-point cartoon makes the eigenproblem tangible. Suppose the kernel reports that points 1 and 2 look nearly identical while point 3 sits apart, so that after centering the Gram matrix has a large positive coupling between 1 and 2 and negative entries linking each of them to 3. The leading eigenvector of that matrix then places weight of one sign on points 1 and 2 and the opposite sign on point 3: reading off its entries already sorts the sample into the two groups the kernel perceives, and the eigenvalue Δ_1 records how sharply. This is the same eigenvector-as-grouping that spectral clustering exploits, and the connection is made precise in Chapter 14, Kernel Clustering and Spectral Methods. Centering is what makes this reading clean. Before centering, the flat all-ones pattern that weights every point equally is a spurious would-be top direction reporting only where the cloud sits, not how it is shaped; subtracting the feature-space mean annihilates that common component, so the surviving eigenvectors describe genuine spread. Kernel PCA is nothing more than this reading, carried out for a whole dataset at once and for as many components as we choose to keep.

A worked example turns that cartoon into numbers. Take five points in the plane, split into a tight cluster near the origin and a separated pair off to the right, and run kernel PCA with the Gaussian kernel. The leading component should separate the two groups and the second should resolve structure within the cluster; the arithmetic below confirms it, and shows how a fresh point is projected onto the components once they are found.

Example (kernel PCA on five points, Gaussian kernel)

Five points $\mathbf{x}_1 = (0, 0)$, $\mathbf{x}_2 = (1, 0)$, $\mathbf{x}_3 = (0, 1)$ form an origin cluster, and $\mathbf{x}_4 = (5, 4)$, $\mathbf{x}_5 = (6, 4)$ form a separated pair. The kernel is Gaussian, $K(\mathbf{x}, \mathbf{y}) = \exp(-\|\mathbf{x} - \mathbf{y}\|^2 / 2\sigma^2)$ with $\sigma = 2$.

1. Build the Gram matrix. Kernel values fall off with distance, so the matrix is block structured: near 1 within each group, near 0 between them,

$$K = \begin{pmatrix} 1 & 0.8825 & 0.8825 & 0.0059 & 0.0015 \\ 0.8825 & 1 & 0.7788 & 0.0183 & 0.0059 \\ 0.8825 & 0.7788 & 1 & 0.0143 & 0.0036 \\ 0.0059 & 0.0183 & 0.0143 & 1 & 0.8825 \\ 0.0015 & 0.0059 & 0.0036 & 0.8825 & 1 \end{pmatrix}.$$

2. Center it. With $J = I - \frac{1}{5}\mathbf{1}\mathbf{1}^\top$, double-centering $K_c = JKJ$ subtracts the feature-space mean and gives a matrix with rows and columns summing to zero,

$$K_c = \begin{pmatrix} 0.3691 & 0.2690 & 0.2702 & -0.4547 & -0.4536 \\ 0.2690 & 0.4038 & 0.1839 & -0.4249 & -0.4318 \\ 0.2702 & 0.1839 & 0.4064 & -0.4277 & -0.4329 \\ -0.4547 & -0.4249 & -0.4277 & 0.7097 & 0.5977 \\ -0.4536 & -0.4318 & -0.4329 & 0.5977 & 0.7206 \end{pmatrix}.$$

The large negative blocks between the two groups are the signature of the split.

3. Eigendecompose. The eigenvalues are $\Delta = (2.1887, 0.2212, 0.1180, 0.0818, 0)$; the last vanishes because centering removes one degree of freedom. The first component alone captures $2.1887/2.6097 = 83.9\%$ of the variance and the second a further 8.5%. The leading two eigenvectors are

$$\mathbf{u}_1 = (-0.3794, -0.3570, -0.3588, 0.5452, 0.5500), \quad \mathbf{u}_2 = (0.0005, 0.7075, -0.7067, 0.0052, -0.0066).$$

$\mathbf{u}_1$ carries one sign on the cluster and the other on the pair; $\mathbf{u}_2$ contrasts $\mathbf{x}_2$ against $\mathbf{x}_3$ inside the cluster.

4. Normalize the coefficients. Dividing by $\sqrt{\Delta_i}$ gives the expansion vectors with unit RKHS norm,

$$\alpha_1 = (-0.2565, -0.2413, -0.2425, 0.3685, 0.3718), \quad \alpha_2 = (0.0011, 1.5043, -1.5025, 0.0111, -0.0140),$$

and one checks $\alpha_1^\top K_c \alpha_1 = \alpha_2^\top K_c \alpha_2 = 1$.

5. Read the projections. The training coordinates on the first component, $K_c \alpha_1 = \sqrt{\Delta_1} \mathbf{u}_1$, are $(-0.561, -0.528, -0.531, 0.807, 0.814)$: the origin cluster lands together on the negative side, the pair on the positive side. On the second component the coordinates are $(0.000, 0.333, -0.332, 0.003, -0.003)$, splitting $\mathbf{x}_2$ from $\mathbf{x}_3$ while leaving the far pair near zero.
6. Project a fresh point. For a test point $\mathbf{z} = (0.5, 0.5)$, the kernel vector $K(\mathbf{z}, \mathbf{x}_j) = (0.9394, 0.9394, 0.9394, 0.0172, 0.0049)$ is centered against the training statistics into $\tilde{\mathbf{k}} = (0.2949, 0.3123, 0.3136, -0.4570, -0.4638)$. Its coordinates are $\alpha_1^\top \tilde{\mathbf{k}} = -0.568$ on the first component and $\alpha_2^\top \tilde{\mathbf{k}} = 0.000$ on the second.

Reading. The first component is a group detector: it captures most of the variance and assigns the two clusters opposite signs, so the test point sitting among the origin cluster inherits its sign (-0.568). The second component captures within-cluster structure that a single coordinate cannot. This is exactly what discarding the flat all-ones direction by centering buys: eigenvectors that describe genuine spread rather than where the cloud sits.

Verification artifact. checks/example-ch06-example-13-1.json records the example source hash and verification scope.

The kernel's bandwidth controls this entire picture. Widen it and distant points begin to look alike, the matrix fills in, and the variance concentrates into a few dominant components; narrow it and every point resembles only itself, the matrix drifts toward the identity, and the spectrum flattens until no small set of components can summarize the data. The heatmap below is exactly the matrix kernel PCA diagonalizes, and it lets you watch the bandwidth reshape the spectrum that the eigenproblem then reads off.

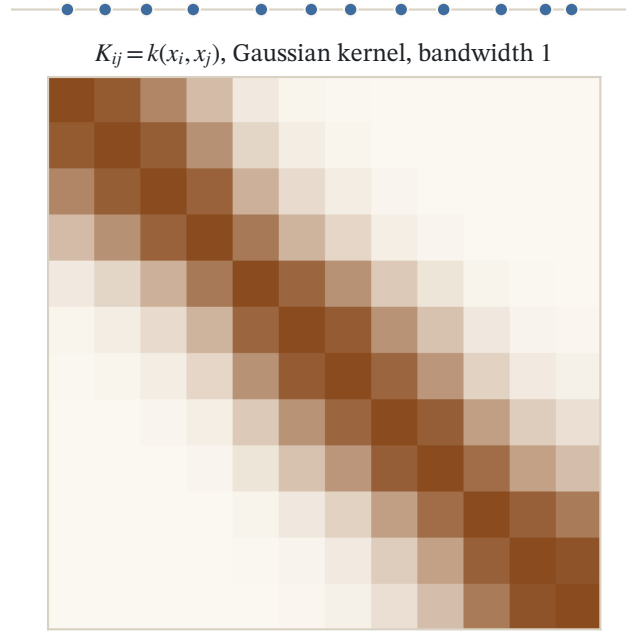


Figure 14.1: Twelve points on a line and their Gram matrix $K_{ij} = k(x_i, x_j)$ for a Gaussian kernel at bandwidth 1. A kernel is positive definite exactly when this matrix is positive semidefinite for every sample; narrowing the bandwidth drives it toward the identity.

Remark (centering the Gram matrix)

The derivation assumed $\frac{1}{n} \sum_i \varphi(\mathbf{x}_i) = 0$. We cannot center the embedded points directly, since we never compute them, but we can center the kernel matrix: replacing each $\varphi(\mathbf{x}_i)$ by $\varphi(\mathbf{x}_i) - \frac{1}{n} \sum_j \varphi(\mathbf{x}_j)$ changes the Gram matrix into

$$K_c = \left(I - \frac{1}{n} \mathbf{1}\mathbf{1}^\top\right) K \left(I - \frac{1}{n} \mathbf{1}\mathbf{1}^\top\right),$$

where $\mathbf{1} \in \mathbb{R}^n$ is the all-ones vector, an operation that again only requires kernel evaluations. It is K_c that step 1 of the algorithm produces and that is diagonalized in step 2.

This centering deserves a derivation rather than an assertion, because it is the one step where we manipulate the kernel matrix in place of points we never see, and the whole method rests on it being correct. Write $\mathbf{1}_n = \frac{1}{n} \mathbf{1}\mathbf{1}^\top$ for the averaging matrix, whose action on any vector replaces every entry by the mean of the entries.

Proposition (double-centered Gram matrix)

Let $\tilde{\varphi}(\mathbf{x}_i) = \varphi(\mathbf{x}_i) - \frac{1}{n} \sum_{a=1}^n \varphi(\mathbf{x}_a)$ be the embeddings recentered to have zero mean, and let $[\tilde{K}]_{ij} = \langle \tilde{\varphi}(\mathbf{x}_i), \tilde{\varphi}(\mathbf{x}_j) \rangle_{\mathcal{H}}$ be their Gram matrix. Then $\tilde{K}$ is computed from the original kernel values alone,

$$[\tilde{K}]_{ij} = K_{ij} - \frac{1}{n} \sum_{a=1}^n K_{aj} - \frac{1}{n} \sum_{b=1}^n K_{ib} + \frac{1}{n^2} \sum_{a,b=1}^n K_{ab},$$

which in matrix form is $\tilde{K} = K - \mathbf{1}_n K - K \mathbf{1}_n + \mathbf{1}_n K \mathbf{1}_n = (I - \mathbf{1}_n) K (I - \mathbf{1}_n)$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Expand the inner product by bilinearity, using $\langle \varphi(\mathbf{x}_i), \varphi(\mathbf{x}_j) \rangle_{\mathcal{H}} = K_{ij}$ throughout:

$$[\tilde{K}]_{ij} = \left\langle \varphi(\mathbf{x}_i) - \frac{1}{n} \sum_b \varphi(\mathbf{x}_b), \varphi(\mathbf{x}_j) - \frac{1}{n} \sum_a \varphi(\mathbf{x}_a) \right\rangle_{\mathcal{H}}.$$

The four resulting terms are K_{ij} , then $-\frac{1}{n} \sum_a K_{ia}$ from pairing $\varphi(\mathbf{x}_i)$ with the mean, then $-\frac{1}{n} \sum_b K_{bj}$ from pairing the mean with $\varphi(\mathbf{x}_j)$, and finally $+\frac{1}{n^2} \sum_{a,b} K_{ba}$ from the two means against each other. Renaming summation indices and using $K_{ia} = K_{ai}$ gives the displayed entrywise formula. To read it as a matrix identity, note that $\mathbf{1}_n K$ has (i, j) entry $\frac{1}{n} \sum_b K_{bj}$, the column mean, and $K \mathbf{1}_n$ has entry $\frac{1}{n} \sum_a K_{ia}$, the row mean, while $\mathbf{1}_n K \mathbf{1}_n$ has the constant entry $\frac{1}{n^2} \sum_{a,b} K_{ab}$, the grand mean. Subtracting the first two and adding the third is exactly the four-term formula, so $\tilde{K} = (I - \mathbf{1}_n)K(I - \mathbf{1}_n)$. Since $\mathbf{1}_n$ is symmetric and idempotent ($\mathbf{1}_n^2 = \mathbf{1}_n$, because averaging an already-averaged vector changes nothing), $I - \mathbf{1}_n$ is the orthogonal projection onto the zero-sum subspace, and no embedded point is ever formed. $\square$

The projection reading also explains why kernel PCA diagonalizes this matrix and not K . The nonzero eigenvalues of $\tilde{K}$ coincide with those of the centered covariance operator $C = \frac{1}{n} \sum_i \tilde{\varphi}(\mathbf{x}_i) \tilde{\varphi}(\mathbf{x}_i)^\top$ in feature space, up to the factor n : if C has eigenvector $\mathbf{v} = \sum_j \alpha_j \tilde{\varphi}(\mathbf{x}_j)$ with eigenvalue λ , then applying C and matching coefficients yields $\tilde{K}\alpha = n\lambda\alpha$, and conversely. Diagonalizing the $n \times n$ Gram matrix is therefore diagonalizing the feature-space covariance without ever writing it down, which is what makes the eigenvectors of $\tilde{K}$ the principal directions in $\mathcal{H}$.

Remark (Gram matrix versus covariance matrix)

In this formulation we diagonalize the $n \times n$ centered kernel Gram matrix, instead of the $d \times d$ covariance matrix of the classical setting. For the linear kernel these are $XX^\top$ and $X^\top X$ respectively, and it is a good exercise to check that $X^\top X$ and $XX^\top$ have the same spectrum, up to zero eigenvalues, with eigenvectors related by a simple relationship: if $X^\top X \mathbf{w} = \lambda \mathbf{w}$ with $\lambda \neq 0$ then $\mathbf{u} = X \mathbf{w} / \|X \mathbf{w}\|$ satisfies $XX^\top \mathbf{u} = \lambda \mathbf{u}$, and conversely. The Gram formulation is the one that survives kernelization: it remains valid for any p.d. kernel, and this is kernel PCA.

Remark (more components than input dimensions)

Diagonalizing the $n \times n$ Gram matrix rather than a $d \times d$ covariance matrix has a consequence classical PCA cannot match. Linear PCA on data in $\mathbb{R}^d$ has at most d nonzero eigenvalues, so it extracts at most d components no matter how many points it is given. Kernel PCA can produce up to n nonzero eigenvalues, and so may extract far more components than the input has coordinates, because the feature map opens up many more directions of variation than there are input dimensions (there are many more monomials than pixels, for a polynomial kernel on images). Schölkopf and Smola (2002) note that this is exactly why kernel PCA cannot be carried out on the covariance matrix directly: the informative components can lie outside the d -dimensional input span entirely.

The applications are exactly what the generality suggests: nonlinear PCA of vector data with nonlinear kernels, and PCA of non-vector objects such as strings or graphs using the structured kernels of later chapters (Chapter 21, Kernels for Sequences, Chapter 24, Kernels for and on Graphs). A classical illustration comes from computational biology: a set of 74 human tRNA sequences, analyzed with a kernel for sequences (a second-order marginalized kernel based on stochastic context-free grammars), projected onto its first two kernel principal components. The set contains three classes of tRNAs, called Ala-AGC, Asn-GTT, and Cys-GCA, and the three groups separate visibly in the plane of the first two kernel principal components (Tsuda et al., 2003). No feature vector was ever formed; the kernel did all the work.

14.2.5 Reconstruction error: the second face of the spectrum

We motivated the principal directions as the directions of greatest variance, but there is a complementary reading that Shawe-Taylor and Cristianini (2004) put at the center of their eigen-decomposition account, and it is the reading that makes denoising precise. Fix a dimension k and ask not which k -dimensional subspace of $\mathcal{H}$ captures the most variance, but which one reconstructs the embedded data best: which subspace U minimizes the average

squared residual left after projecting each point onto it. Writing P_U for the orthogonal projection onto U and $P_U^\perp = I - P_U$ for its complement, the quantity to minimize is

$$J^\perp(U) = \sum_{i=1}^n \|P_U^\perp \varphi(\mathbf{x}_i)\|_{\mathcal{H}}^2, \quad \dim U = k.$$

Proposition (principal subspaces minimize reconstruction error)

The minimizer of $J^\perp$ over k -dimensional subspaces is the span U_k of the first k kernel principal directions, and the minimum value is the sum of the discarded eigenvalues,

$$J^\perp(U_k) = \sum_{i=k+1}^n \Delta_i.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The proof is Pythagoras: for any subspace the squared norm splits as $\|\varphi(\mathbf{x}_i)\|_{\mathcal{H}}^2 = \|P_U \varphi(\mathbf{x}_i)\|_{\mathcal{H}}^2 + \|P_U^\perp \varphi(\mathbf{x}_i)\|_{\mathcal{H}}^2$, so minimizing the total residual is the same as maximizing the total captured norm $\sum_i \|P_U \varphi(\mathbf{x}_i)\|_{\mathcal{H}}^2$, which is again the trace maximization solved by the top eigenvectors; the leftover is exactly the mass $\sum_{i>k} \Delta_i$ sitting in the directions we dropped. The two views are therefore one theorem read from opposite ends: the top k eigenvectors both hold the most variance and leave the least reconstruction error, and the tail of the spectrum measures precisely what a k -dimensional summary throws away. This is the exact sense in which discarding low-variance components denoises: if the true signal lives in a few dominant directions and the noise is spread thinly across the rest, then $\sum_{i>k} \Delta_i$ is mostly noise energy, and projecting it away improves the representation (Shawe-Taylor and Cristianini, 2004).

Remark (when the projection is trustworthy)

Reading the residual off the training sample only helps if it also predicts the residual on unseen points. A stability analysis of kernel PCA shows that it does, provided the spectrum cooperates: with high probability the expected squared residual of a fresh point projected onto U_k is close to the training average $\frac{1}{n} \sum_{i>k} \Delta_i$, up to a term that shrinks like $\sqrt{(k+1)/n}$ (Shawe-Taylor and Cristianini, 2004). The practical reading is a rule of thumb: kernel PCA is reliable exactly when the eigenvalues fall off early, so that a small k already leaves a small tail. If the spectrum is flat, no low-dimensional projection is stable, and the leading components are as likely to reflect the particular sample as any structure in the distribution. How fast the eigenvalues of a kernel decay is governed by the kernel's smoothness, the theme of Chapter 18, Mercer's Theorem, Spectra, and Rates, so the reliability of kernel PCA is ultimately a statement about the kernel one chose.

The same projections are what one plots to see the data. Classical multidimensional scaling, which embeds objects into two or three dimensions so as to preserve their pairwise distances, reduces in a kernel-defined feature space to computing the first two or three kernel principal components: once dissimilarities are read as inner products, the elaborate scaling machinery collapses onto the eigendecomposition we already have (Williams, 2002; Shawe-Taylor and Cristianini, 2004), a correspondence developed in Chapter 17, Data Visualization and Kernel MDS. Kernel PCA is thus simultaneously a feature extractor, a denoiser, and the standard tool for laying a nonlinear dataset out on the page.

14.2.6 Denoising: projecting away the noise directions

The reconstruction reading is what makes denoising more than a slogan. Model an observed point as signal plus noise, and suppose the signal, being the structured part that many examples share, concentrates its feature-space energy in a few dominant directions, while the noise, being idiosyncratic to each example, scatters thinly and roughly evenly across the many remaining directions. Kernel PCA has already named those two families: the

signal lives near the top principal subspace U_k , and the noise lives mostly in its complement, the low-variance tail whose total energy is exactly the residual $J^\perp(U_k) = \sum_{i>k} \Delta_i$ from the reconstruction proposition. This is why the tail is dominated by noise whenever the spectrum falls off early: the signal has spent itself in the first few components, so what remains beyond k is mostly the scattered noise mass.

Denoising is then a single geometric move: replace each embedded point by its orthogonal projection onto the top components,

$$P_k \varphi(\mathbf{x}) = \sum_{i=1}^k \langle \varphi(\mathbf{x}), f_i \rangle_{\mathcal{H}} f_i,$$

which keeps the signal-bearing coordinates and zeroes out the directions where the noise sat. Because those discarded directions carried little of the true structure, the projection is close to the clean point and far from the corrupted one; the cleaner the separation between the dominant spectrum and its tail, the more thoroughly the projection strips the noise. The choice of k is the usual dial: too small and the projection erases genuine structure along with the noise, too large and it retains the very directions it was meant to discard, so k is set where the eigenvalues drop from the signal plateau into the noise floor.

The projection $P_k \varphi(\mathbf{x})$, however, is a point in the feature space $\mathcal{H}$, and for denoising to mean anything we must exhibit it as an actual input: an image, a signal, a vector in $\mathcal{X}$. That return trip is the pre-image problem of the next subsection, whose fixed-point iteration is treated in full in Chapter 20, Invariances and the Pre-Image Problem. Denoising is therefore a two-stage recipe: project onto the top k kernel principal components to clean the feature vector, then seek the input whose embedding best matches the cleaned vector. Applied to images of handwritten digits corrupted by noise, the two stages together return visibly cleaner digits, the original demonstration that discarding low-variance feature-space directions denoises in input space (Mika et al., 1999).

14.2.7 The pre-image problem

There is a subtlety we glossed over, and it is worth naming because it is the price of never forming the feature vectors. Kernel PCA gives us, for each point, its projections onto the leading directions, and that is all we usually want for visualization or as features for a downstream algorithm. But some applications, denoising being the classic one, want to go back: to take the projection of $\varphi(\mathbf{x})$ onto the top few components, a point

$$P_k \varphi(\mathbf{x}) = \sum_{i=1}^k \langle \varphi(\mathbf{x}), f_i \rangle_{\mathcal{H}} f_i \in \mathcal{H},$$

and read off an input $\mathbf{z} \in \mathcal{X}$ that this reconstruction represents. The reason one would want this is that the projection cleans the point up: discarding the low-variance components throws away the directions in which noise tends to live, so $P_k \varphi(\mathbf{x})$ is a denoised version of $\varphi(\mathbf{x})$, and its pre-image is the actual input, the image or the signal, that this cleaned-up feature vector stands for. The trouble is that $P_k \varphi(\mathbf{x})$ is a general element of $\mathcal{H}$, and there is in general no $\mathbf{z}$ at all whose embedding equals it: the set of genuine embeddings $\{\varphi(\mathbf{z}) : \mathbf{z} \in \mathcal{X}\}$ is a thin, curved surface inside the ambient space $\mathcal{H}$, and projecting onto a linear subspace almost always lands just off it. An exact pre-image need not exist.

What one does instead is settle for an approximate pre-image, the input whose embedding is closest to the target reconstruction:

$$\mathbf{z}^\star = \arg \min_{\mathbf{z} \in \mathcal{X}} \left\| \varphi(\mathbf{z}) - P_k \varphi(\mathbf{x}) \right\|_{\mathcal{H}}^2.$$

The objective expands into kernel values, like everything else in this chapter, but it is no longer convex in $\mathbf{z}$, so this is a genuine nonlinear optimization that may have several local minima. For the Gaussian kernel it reduces to a fixed-point iteration on $\mathbf{z}$, derived and analyzed in Chapter 20, Invariances and the Pre-Image Problem; in general

it is solved numerically and only approximately (Mika et al., 1999). The moral is that the kernel trick buys entry into $\mathcal{H}$ cheaply but charges for the return trip: mapping forward through φ is free, mapping back out is a problem in its own right.

The Gaussian kernel makes the denoising picture concrete. There each step of the fixed-point iteration writes the pre-image as a similarity-weighted average of the data,

$$\mathbf{z} \leftarrow \frac{\sum_{i=1}^n \gamma_i e^{-\|\mathbf{z}-\mathbf{x}_i\|^2/2\sigma^2} \mathbf{x}_i}{\sum_{i=1}^n \gamma_i e^{-\|\mathbf{z}-\mathbf{x}_i\|^2/2\sigma^2}},$$

with weights γ_i read off from the projection $P_k \varphi(\mathbf{x})$. Each reconstructed input is thus pulled toward the region where the data are dense and away from the isolated excursions in which noise tends to live, so the pre-image comes out smoother than the point it started from. Applied to images of handwritten digits corrupted by noise, keeping only the top kernel components and taking pre-images returns visibly cleaner digits, the standard demonstration that discarding low-variance directions in feature space denoises in input space (Mika et al., 1999).

14.3 Out-of-sample extension and the Nyström view

Everything derived so far produces coordinates for the n points we trained on: the eigenvectors of the centered Gram matrix are, after scaling, exactly the training projections. A working method, though, must place points it never saw. Denoising takes in a fresh corrupted image, projects it onto the top components, and returns it to input space; visualization drops a new query onto an existing map; a downstream classifier wants the kernel-PCA coordinates of every test point. Classical PCA does this without a thought, because a principal direction is a vector $\mathbf{w}$ and the coordinate of a new point is the dot product $\mathbf{w}^\top \mathbf{x}$. Kernel PCA never formed its directions f_i as vectors; it holds only their expansion coefficients α_i against the training embeddings. The question of this section is whether a new point can be placed using nothing but its kernel values to the training set. The answer is yes, the same coefficients α_i do the job, and the resulting formula turns out to be an instance of a single principle, the Nyström extension, that reaches every spectral embedding method. Step 6 of the kernel-PCA algorithm already wrote this projection down; we now derive it, pin down the one delicate point of how the test point is centered, and read it as a Nyström extension. Returning the placed point to input space, the last leg of the denoising pipeline, is the separate pre-image problem of Chapter 20, Invariances and the Pre-Image Problem.

14.3.1 Projecting a new point

Fix a fitted kernel PCA and a new point $\mathbf{x} \in \mathcal{X}$. Its coordinate on the i -th unit-norm component is the projection $h_{f_i}(\mathbf{x}) = \langle \varphi(\mathbf{x}), f_i \rangle_{\mathcal{H}}$, and two moves collapse this abstract inner product into kernel values. First, the direction is a combination of the centered training embeddings, $f_i = \sum_{j=1}^n \alpha_{i,j} \tilde{\varphi}(\mathbf{x}_j)$ with $\tilde{\varphi}(\mathbf{x}_j) = \varphi(\mathbf{x}_j) - \mu$ and $\mu = \frac{1}{n} \sum_{a=1}^n \varphi(\mathbf{x}_a)$ the training mean. Second, and this is the step that is easy to get wrong, the new point must be measured against the same origin: we center its embedding by the very same training mean, $\tilde{\varphi}(\mathbf{x}) = \varphi(\mathbf{x}) - \mu$. With both embeddings centered by μ ,

$$\langle \tilde{\varphi}(\mathbf{x}), f_i \rangle_{\mathcal{H}} = \sum_{j=1}^n \alpha_{i,j} \langle \tilde{\varphi}(\mathbf{x}), \tilde{\varphi}(\mathbf{x}_j) \rangle_{\mathcal{H}} = \alpha_i^\top \tilde{\mathbf{k}}(\mathbf{x}),$$

where the centered kernel vector $\tilde{\mathbf{k}}(\mathbf{x}) \in \mathbb{R}^n$ has entries obtained by expanding $\langle \varphi(\mathbf{x}) - \mu, \varphi(\mathbf{x}_j) - \mu \rangle_{\mathcal{H}}$ by bilinearity, exactly as in the double-centering proposition but with one argument now off the training set.

Proposition (out-of-sample projection)

Fit kernel PCA on $\mathbf{x}_1, \dots, \mathbf{x}_n$, with centered Gram matrix K_c , eigenpairs $(\Delta_i, \mathbf{u}_i)$, and coefficients $\alpha_i = \mathbf{u}_i / \sqrt{\Delta_i}$. The coordinate of any $\mathbf{x} \in \mathcal{X}$ on the i -th kernel principal component is

$$\pi_i(\mathbf{x}) = \alpha_i^\top \tilde{\mathbf{k}}(\mathbf{x}) = \frac{1}{\sqrt{\Delta_i}} \mathbf{u}_i^\top \tilde{\mathbf{k}}(\mathbf{x}),$$

with the centered kernel vector

$$\tilde{\mathbf{k}}_j(\mathbf{x}) = K(\mathbf{x}, \mathbf{x}_j) - \frac{1}{n} \sum_{a=1}^n K(\mathbf{x}, \mathbf{x}_a) - \frac{1}{n} \sum_{b=1}^n K(\mathbf{x}_b, \mathbf{x}_j) + \frac{1}{n^2} \sum_{a,b=1}^n K(\mathbf{x}_a, \mathbf{x}_b).$$

For $\mathbf{x} = \mathbf{x}_m$ the vector $\tilde{\mathbf{k}}(\mathbf{x}_m)$ is the m -th column of K_c , so $\pi_i(\mathbf{x}_m) = \sqrt{\Delta_i} (\mathbf{u}_i)_m$ recovers the training coordinate of the kernel-PCA theorem: the map extends kernel PCA from the sample to all of $\mathcal{X}$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The four terms deserve a slow look, because their asymmetry is the whole subtlety of the extension.

Remark (centering the test kernel vector)

Only the first two terms of $\tilde{\mathbf{k}}_j(\mathbf{x})$ involve the new point. The second, $\frac{1}{n} \sum_a K(\mathbf{x}, \mathbf{x}_a)$, is the test point's own mean similarity to the training set, the single number that recenters $\varphi(\mathbf{x})$. The third and fourth terms, the training column mean $\frac{1}{n} \sum_b K(\mathbf{x}_b, \mathbf{x}_j)$ and the training grand mean $\frac{1}{n^2} \sum_{a,b} K(\mathbf{x}_a, \mathbf{x}_b)$, carry no $\mathbf{x}$ at all: they are frozen statistics of the training Gram matrix, computed once and reused for every test point. This is what it means to center by the training mean μ rather than by some new mean: the new point is never folded into the averages. In particular the grand mean is not recomputed over the augmented set, and $\mathbf{x}$ is not centered against its own kernel row. Contrast the symmetric training rule $K_c = (I - \mathbf{1}_n)K(I - \mathbf{1}_n)$, where a single n -point mean centers both arguments; here one argument is on the training set and one is not, so the expression loses its symmetry while keeping the same μ . Getting this wrong, by refreshing the column or grand means with $\mathbf{x}$ included, silently biases every out-of-sample coordinate and breaks the exact agreement with kernel PCA on the training points.

That exact agreement is worth stating as a test one can run: because $\tilde{\mathbf{k}}(\mathbf{x}_m)$ is literally the m -th column of K_c , feeding a training point through the out-of-sample formula returns its kernel-PCA coordinate to machine precision, while feeding a genuinely new point returns a value that only approximates what a refit would give. The worked example checks both faces, the exact one and the approximate one, against direct computation.

Example (out-of-sample projection and the refit check)

Five training points on a shallow arc, $\mathbf{x}_1 = (-2, 0)$, $\mathbf{x}_2 = (-1, 1)$, $\mathbf{x}_3 = (0, 0)$, $\mathbf{x}_4 = (1, 1)$, $\mathbf{x}_5 = (2, 0)$, with the Gaussian kernel $K(\mathbf{x}, \mathbf{y}) = \exp(-\|\mathbf{x} - \mathbf{y}\|^2 / 2\sigma^2)$, $\sigma = 1.5$. We fit kernel PCA, then place the fresh point $\mathbf{z} = (-0.5, 0.5)$ out of sample.

1. Build and center the Gram matrix. The kernel falls off along the arc, giving

$$K = \begin{pmatrix} 1 & 0.6412 & 0.4111 & 0.1084 & 0.0286 \\ 0.6412 & 1 & 0.6412 & 0.4111 & 0.1084 \\ 0.4111 & 0.6412 & 1 & 0.6412 & 0.4111 \\ 0.1084 & 0.4111 & 0.6412 & 1 & 0.6412 \\ 0.0286 & 0.1084 & 0.4111 & 0.6412 & 1 \end{pmatrix},$$

and double-centering $K_c = JKJ$, $J = I - \frac{1}{5}\mathbf{1}\mathbf{1}^\top$, gives a matrix with rows summing to zero.

2. Eigendecompose. The eigenvalues are $\Delta = (1.3463, 0.5452, 0.2772, 0.2141, 0)$, so the first component captures $1.3463/2.3827 = 56.5\%$ of the variance and the second 22.9% , for 79.4% together. The leading eigenvectors are

$$\mathbf{u}_1 = (-0.5783, -0.4069, 0, 0.4069, 0.5783), \quad \mathbf{u}_2 = (0.5426, -0.3063, -0.4728, -0.3063, 0.5426),$$

with $\mathbf{u}_1$ reading position along the arc and $\mathbf{u}_2$ its symmetric curvature.

3. Normalize. Dividing by $\sqrt{\Delta_i}$ gives unit-RKHS-norm coefficients $\alpha_1 = (-0.4984, -0.3507, 0, 0.3507, 0.4984)$ and $\alpha_2 = (0.7349, -0.4148, -0.6403, -0.4148, 0.7349)$, and the training coordinates on the first two components are $\sqrt{\Delta_1} \mathbf{u}_1 = (-0.671, -0.472, 0, 0.472, 0.671)$ and $\sqrt{\Delta_2} \mathbf{u}_2 = (0.401, -0.226, -0.349, -0.226, 0.401)$.
4. Center the test kernel vector. The raw kernel vector of $\mathbf{z}$ is $K(\mathbf{z}, \mathbf{x}_j) = (0.5738, 0.8948, 0.8948, 0.5738, 0.2359)$. Its own mean similarity is $\frac{1}{5} \sum_a K(\mathbf{z}, \mathbf{x}_a) = 0.6346$; the frozen training column means are $(0.4378, 0.5604, 0.6209, 0.5604, 0.4378)$ and the training grand mean is 0.5235 . Subtracting the test mean and the frozen training statistics,

$$\tilde{\mathbf{k}}(\mathbf{z}) = (0.0248, 0.2233, 0.1628, -0.0978, -0.3131).$$

5. Read the out-of-sample coordinates. Dotting with the coefficients, $\pi_1(\mathbf{z}) = \alpha_1^\top \tilde{\mathbf{k}}(\mathbf{z}) = -0.281$ and $\pi_2(\mathbf{z}) = \alpha_2^\top \tilde{\mathbf{k}}(\mathbf{z}) = -0.368$. The Nyström reading agrees: the extended eigenfunction $\psi_1(\mathbf{z}) = \frac{1}{\Delta_1} \mathbf{u}_1^\top \tilde{\mathbf{k}}(\mathbf{z}) = -0.2422$ gives $\sqrt{\Delta_1} \psi_1(\mathbf{z}) = -0.281$. As an exact check, running $\mathbf{x}_1$ through the same formula returns -0.671 , its training coordinate, to machine precision.
6. Verify against a refit. Adding $\mathbf{z}$ to the set and re-running kernel PCA on all six points gives leading eigenvalues $(1.424, 0.6547)$ and places $\mathbf{z}$ at -0.275 on the first component and -0.293 on the second. The out-of-sample values $(-0.281, -0.368)$ match the first to 0.006 , a 2.3% relative gap, and the second to 0.075 ; for scale, adding one point already shifts the five training coordinates by up to 0.110 on the first component and 0.144 on the second.

Reading. The out-of-sample formula places a new point from the frozen fit alone, at the cost of one centered kernel vector and two dot products, and it reproduces the training coordinates exactly. Against a full refit it agrees closely on the leading, well-separated component and more loosely on the second, both gaps staying within the perturbation that adding a single point induces on every coordinate. The leading component extends most reliably precisely because its eigenvalue is the best separated, the same spectral condition that governs when the projection is trustworthy at all.

Verification artifact. `checks/example-ch06-example-13-2.json` records the example source hash and verification scope.

14.3.2 The Nyström view and its cost

The extension formula is not an ad hoc trick; it is the Nyström method for approximating the eigenfunctions of an integral operator, applied to the centered kernel. Read the kernel as an operator on functions: under the data distribution ρ , the operator $(T_\rho g)(\mathbf{x}) = \int \tilde{K}(\mathbf{x}, \mathbf{y}) g(\mathbf{y}) d\rho(\mathbf{y})$ has eigenfunctions ψ_i whose values at the training points are what kernel PCA recovers as the eigenvectors $\mathbf{u}_i$. Given only the sample, we cannot integrate against ρ ; the Nyström method (Williams and Seeger, 2001) replaces ρ by the empirical measure on the n points, turning the integral into the average $(T_n g)(\mathbf{x}) = \frac{1}{n} \sum_j \tilde{K}(\mathbf{x}, \mathbf{x}_j) g(\mathbf{x}_j)$. Its eigenfunctions, evaluated at a new $\mathbf{x}$, are

$$\psi_i(\mathbf{x}) = \frac{1}{\Delta_i} \sum_{j=1}^n \tilde{k}_j(\mathbf{x}) (\mathbf{u}_i)_j = \frac{1}{\Delta_i} \mathbf{u}_i^\top \tilde{\mathbf{k}}(\mathbf{x}),$$

which at a training point returns $\psi_i(\mathbf{x}_m) = \frac{1}{\Delta_i}(K_c \mathbf{u}_i)_m = (\mathbf{u}_i)_m$, the sampled eigenvector entry. Comparing with the proposition, the kernel-PCA coordinate is exactly $\sqrt{\Delta_i}$ times this Nyström-extended eigenfunction, $\pi_i(\mathbf{x}) = \sqrt{\Delta_i} \psi_i(\mathbf{x})$: out-of-sample kernel PCA is the Nyström extension of the centered kernel operator, and the worked example confirms the two expressions agree to the digit.

The economy of the extension is its point. Fitting kernel PCA costs one eigendecomposition of the $n \times n$ matrix, $O(n^3)$ dense or $O(n^2 k)$ with an iterative solver for the top k components. Afterwards, each new point costs n kernel evaluations to form its vector, an $O(n)$ centering, and one length- n dot product per component, so $O(nk)$ with no further eigendecomposition. Fit once, extend a stream of points cheaply. The same identity, run in reverse, is how kernel methods are scaled to large n : rather than build and diagonalize the full Gram matrix, one deliberately fits on a small set of m landmark points and Nyström-extends the resulting eigenvectors to the remaining $n - m$, a low-rank approximation costing $O(nm^2)$ that is developed in Chapter 37, Large-Scale Kernel Machines. Out-of-sample projection and the Nyström low-rank approximation are one formula read in opposite directions: the first extends a fixed basis to new data, the second chooses a small basis so the extension carries the load.

14.3.3 The general Nyström-extension principle

Kernel PCA is one member of a family of spectral embeddings that share a single computational shape: build a data-dependent $n \times n$ symmetric matrix, take its leading eigenvectors, and read the coordinates off them. Metric multidimensional scaling, Isomap, locally linear embedding, Laplacian eigenmaps, and spectral clustering all fit this mold, and several of them were originally defined only on the training sample, with no recipe for a new point short of rebuilding and re-diagonalizing the whole matrix. Bengio et al. (2004) unified them by the observation that each method is generated by a symmetric data-dependent kernel $K_D(\mathbf{x}, \mathbf{y})$, whose Gram matrix on the sample is the matrix the method diagonalizes, and that once K_D is named, a new point is embedded by the very Nyström formula above,

$$f_i(\mathbf{x}) = \frac{1}{\sqrt{\lambda_i}} \sum_{j=1}^n (\mathbf{v}_i)_j K_D(\mathbf{x}, \mathbf{x}_j),$$

with $(\lambda_i, \mathbf{v}_i)$ the eigenpairs of the sample matrix: evaluate the data-dependent kernel between the new point and each training point, weight by the eigenvector, and rescale by the eigenvalue. The methods differ only in which K_D they use.

Method	Data-dependent kernel $K_D(\mathbf{x}, \mathbf{y})$	Embedding read off
Kernel PCA, metric MDS	centered kernel $\tilde{K}(\mathbf{x}, \mathbf{y}) = \langle \tilde{\varphi}(\mathbf{x}), \tilde{\varphi}(\mathbf{y}) \rangle_{\mathcal{H}}$	principal components
Isomap	$-\frac{1}{2}$ double-centered squared graph geodesics	isometric coordinates
Locally linear embedding	$c I - (I - W)^\top (I - W)$ from local weights W	local-linear coordinates
Laplacian eigenmaps, spectral clustering	normalized affinity $D^{-1/2} A D^{-1/2}$ of the neighborhood graph	Laplacian embedding

Read this way, the transductive methods become inductive at $O(n)$ per new point, which is exactly what visualization needs when a fresh query must be dropped onto an Isomap or MDS layout without recomputing the embedding of the whole corpus, the use developed in Chapter 17, Data Visualization and Kernel MDS. Bengio et al. (2004) further show the extension is consistent: as n grows, $f_i(\mathbf{x})$ converges to an eigenfunction of a limiting operator fixed by K_D and the data density, and it reproduces the training embedding at the training points, precisely as the kernel-PCA extension did. The one piece of care that transfers verbatim is the centering. For kernel PCA the data-dependent kernel $\tilde{K}$ centers both arguments against the training mean, which is the frozen-statistics rule above; each other method carries its own normalization, a double-centering for Isomap and MDS, a degree

normalization for the graph-Laplacian methods, that must likewise be applied to the new point with training-set quantities, never recomputed with $\mathbf{x}$ folded in.

14.3.4 Robust and sparse variants

Two structural weaknesses of plain kernel PCA have each grown a family of remedies, and both leave the out-of-sample machinery untouched. The first is fragility to outliers. The feature-space covariance is a sum of squared embeddings, so a few points with large embedded norm can tilt the top components toward themselves, and the double-centering then smears each outlier's pull across every entry of K_c . Robust kernel PCA replaces the squared reconstruction criterion with a slower-growing loss, or iteratively reweights the points by their residual, so that high-residual examples are discounted and the principal subspace tracks the bulk of the data rather than its excursions (Nguyen and De la Torre, 2009). The second is density of the components. Each direction $f_i = \sum_j \alpha_{i,j} \phi(\mathbf{x}_j)$ puts nonzero weight on every training point, so evaluating it at a new point costs the full n kernel evaluations of the out-of-sample formula. Sparse kernel PCA adds a cardinality or ℓ_1 penalty on α_i , seeking directions supported on a handful of training points; it gives up a little captured variance in exchange for components that are cheaper to extend and easier to read, since a sparse α_i names the few examples that define the direction (Tipping, 2001). In both cases what changes is which α_i one extends, not how the extension is computed: the centered kernel vector and its projection are the same.

14.4 Common mistakes and practical implications

For **Kernel PCA and Denoising**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

14.5 Summary and further reading

This chapter established explain the central definitions and claims in Kernel PCA and Denoising; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Schölkopf and Müller 1998), (Tsuda and Asai 2003), (Mika and Rätsch 1999).

14.6 Exercises

1. computation A centered Gram matrix has eigenvalues $\Delta = (10, 6, 3, 1, 0, 0, 0, 0)$ (the last four vanish). (a) What fraction of the total variance is captured by projecting onto the top two kernel principal components? (b) By the reconstruction-error proposition, what is the residual $J^\perp(U_2) = \sum_{i>2} \Delta_i$ left by that two-dimensional summary? (c) What is the smallest number of components k needed for the captured variance to reach at least ninety percent of the total? (d) The stability remark ties reliability to how early the spectrum falls off; comment on whether a two-dimensional projection of this dataset looks trustworthy.

2. proof Derive the centering formula. Starting from the recentered embeddings $\tilde{\varphi}(\mathbf{x}_i) = \varphi(\mathbf{x}_i) - \frac{1}{n} \sum_{a=1}^n \varphi(\mathbf{x}_a)$, compute the new Gram entry $[K_c]_{ij} = \langle \tilde{\varphi}(\mathbf{x}_i), \tilde{\varphi}(\mathbf{x}_j) \rangle_{\mathcal{H}}$ purely in terms of the original kernel values, and show it equals

$$[K_c]_{ij} = K_{ij} - \frac{1}{n} \sum_{a=1}^n K_{aj} - \frac{1}{n} \sum_{b=1}^n K_{ib} + \frac{1}{n^2} \sum_{a,b=1}^n K_{ab}.$$

Then verify that in matrix form this is exactly $K_c = (I - U)K(I - U)$ with $U = \frac{1}{n}\mathbf{1}\mathbf{1}^\top$, and note that only kernel evaluations, never the embedded points, are required. Hint

Expand the inner product by bilinearity into four sums. The matrix $U = \frac{1}{n}\mathbf{1}\mathbf{1}^\top$ acts on a vector by replacing every entry with the mean; convince yourself that left-multiplying K by $(I - U)$ subtracts each column's mean and right-multiplying subtracts each row's mean, and that U is symmetric and idempotent ($U^2 = U$).

3. proof Prove that the top- k principal subspace minimizes reconstruction error, with residual equal to the sum of the discarded eigenvalues. For a k -dimensional subspace $U \subset \mathcal{H}$ with orthogonal projection P_U , use the Pythagorean split $\|\varphi(\mathbf{x}_i)\|^2 = \|P_U \varphi(\mathbf{x}_i)\|^2 + \|P_U^\perp \varphi(\mathbf{x}_i)\|^2$ to show that minimizing $J^\perp(U) = \sum_i \|P_U^\perp \varphi(\mathbf{x}_i)\|^2$ is the same as maximizing the captured norm $\sum_i \|P_U \varphi(\mathbf{x}_i)\|^2$. Identify that captured norm as a trace maximization over rank- k projections, invoke the eigenvector characterization to conclude the maximizer is the span U_k of the first k kernel principal directions, and read off

$$J^\perp(U_k) = \sum_{i>k} \Delta_i.$$

Hint

Write $P_U = \sum_{m=1}^k \mathbf{e}_m \mathbf{e}_m^\top$ for an orthonormal basis $\{\mathbf{e}_m\}$ of U in feature space, so that $\sum_i \|P_U \varphi(\mathbf{x}_i)\|^2 = \sum_m \mathbf{e}_m^\top C \mathbf{e}_m = \text{trace}(P_U C)$, where $C = \sum_i \varphi(\mathbf{x}_i) \varphi(\mathbf{x}_i)^\top$ is the (uncentered-in-notation, centered-in-fact) covariance operator whose nonzero eigenvalues match the Δ_i . The trace is maximized by aligning U with the top- k eigenvectors; the leftover $\sum_i \|\varphi(\mathbf{x}_i)\|^2 = \sum_i \Delta_i$ minus the captured part is the tail.

4. exploration Open the interactive Gram-matrix heatmap earlier in this chapter; this is exactly the matrix kernel PCA diagonalizes. Widen the kernel bandwidth and watch the off-diagonal entries fill in, then narrow it toward zero and watch the matrix drift toward the identity. Describe how the spectrum you would read off changes between these regimes: for which bandwidth does the variance concentrate into a few dominant eigenvalues, and for which does it flatten so that no small set of components can summarize the data? Connect your observation to the stability rule of thumb that kernel PCA is trustworthy exactly when the eigenvalues fall off early.
5. computation Out-of-sample by hand. Using the fitted quantities from the out-of-sample worked example, place the same test point $\mathbf{z} = (-0.5, 0.5)$ yourself. Its raw kernel vector is $K(\mathbf{z}, \mathbf{x}_j) = (0.5738, 0.8948, 0.8948, 0.5738, 0.2359)$, the frozen training column means are $(0.4378, 0.5604, 0.6209, 0.5604, 0.4378)$, and the training grand mean is 0.5235. (a) Form the centered kernel vector $\tilde{\mathbf{k}}(\mathbf{z})$, identifying among its three centering terms the one that uses the test point's own mean similarity and the two that are frozen training statistics. (b) With $\alpha_1 = (-0.4984, -0.3507, 0, 0.3507, 0.4984)$, compute the first-component coordinate $\alpha_1^\top \tilde{\mathbf{k}}(\mathbf{z})$. (c) Explain why repeating the computation with the third column of K in place of $K(\mathbf{z}, \cdot)$ returns $\mathbf{x}_3$'s training coordinate exactly, whereas the coordinate of $\mathbf{z}$ only approximates the value a refit would give. Hint

The test mean similarity is $\frac{1}{5} \sum_a K(\mathbf{z}, \mathbf{x}_a) = 0.6346$; it enters the second term only. The third column of K is a training column, so its centering is a column of K_c , and $\alpha_1^\top (K_c)_{:,3} = (K_c \alpha_1)_3 = \sqrt{\Delta_1} (\mathbf{u}_1)_3$.

6. proof The extension agrees with kernel PCA on the sample. Let $K_c = (I - \mathbf{1}_n)K(I - \mathbf{1}_n)$ be the centered training Gram matrix with coefficients $\alpha_i = \mathbf{u}_i / \sqrt{\Delta_i}$. Show that for a training point $\mathbf{x}_m$ the centered out-of-sample kernel vector $\tilde{\mathbf{k}}(\mathbf{x}_m)$, with entries $\tilde{k}_j = K_{mj} - \frac{1}{n} \sum_a K_{ma} - \frac{1}{n} \sum_b K_{bj} + \frac{1}{n^2} \sum_{a,b} K_{ab}$, is exactly the m -th column of K_c . Conclude that $\pi_i(\mathbf{x}_m) = \alpha_i^\top \tilde{\mathbf{k}}(\mathbf{x}_m) = \sqrt{\Delta_i} (\mathbf{u}_i)_m$ is the training coordinate of the kernel-PCA

theorem. Then argue why the corresponding statement for a fresh point holds only approximately, and name the two quantities that change when the point is added to the reference set. Hint

Compare the four-term $\tilde{k}_j$ with the entrywise formula $[K_c]_{mj} = K_{mj} - \frac{1}{n} \sum_a K_{aj} - \frac{1}{n} \sum_b K_{mb} + \frac{1}{n^2} \sum_{a,b} K_{ab}$ from the double-centering proposition; they coincide term by term using $K_{ma} = K_{am}$. For a new point, both the training mean μ and the eigenpairs $(\Delta_i, \mathbf{u}_i)$ shift when it joins the sample.

15 Kernel Clustering and Spectral Methods

Clustering asks us to partition a dataset into a small number of coherent groups with no labels to guide us, judging coherence entirely by how similar the points look to one another. When that similarity is measured through a kernel, the flat cell boundaries of ordinary K-means bend into curved ones that follow the geometry the kernel encodes. This chapter carries the clustering objective into a reproducing-kernel Hilbert space, giving kernel K-means and, through a spectral relaxation of the same combinatorial cost, the family of spectral clustering algorithms.

15.1 The K-means algorithm

We turn from finding coordinates to finding groups. Clustering asks us to partition a dataset into a small number of coherent groups without any labels to guide us. K-means is probably the most popular clustering algorithm, and, exactly as with PCA, understanding its classical form is the whole battle: kernel K-means will be the same algorithm read through a feature map.

15.1.1 An optimization point of view

Given data points $\mathbf{x}_1, \dots, \mathbf{x}_n$ in $\mathbb{R}^p$ and a target number of clusters k , K-means represents each cluster by a centroid $\mu_j \in \mathbb{R}^p$ and assigns each point an index $s_i \in \{1, \dots, k\}$ naming its cluster. It seeks the assignments and centroids that minimize the total squared distance from each point to the centroid of its cluster:

$$\min_{\substack{\mu_j \in \mathbb{R}^p, j=1, \dots, k \\ s_i \in \{1, \dots, k\}, i=1, \dots, n}} \sum_{i=1}^n \|\mathbf{x}_i - \mu_{s_i}\|_2^2.$$

This objective couples two very different kinds of unknown: the continuous centroids and the discrete assignments. Optimizing both at once is hard, but optimizing either one with the other held fixed is easy, and that observation is the algorithm. K-means performs alternate minimization, cycling between two steps until the assignments stop changing.

The first step is *cluster assignment*. With the centroids $\mu_1, \dots, \mu_k$ fixed, the objective decouples across points, and each $\mathbf{x}_i$ is best assigned to its nearest centroid:

$$\forall i, \quad s_i \in \arg \min_{s \in \{1, \dots, k\}} \|\mathbf{x}_i - \mu_s\|_2^2.$$

The second step is the *centroid update*. With the assignments $s_1, \dots, s_n$ fixed, the objective decouples across clusters, and within cluster j we minimize $\sum_{i: s_i=j} \|\mathbf{x}_i - \mu\|_2^2$ over $\mu \in \mathbb{R}^p$:

$$\forall j, \quad \mu_j = \arg \min_{\mu \in \mathbb{R}^p} \sum_{i: s_i=j} \|\mathbf{x}_i - \mu\|_2^2.$$

Setting the gradient to zero, the minimizer is the mean of the points currently assigned to the cluster:

$$\forall j, \quad \mu_j = \frac{1}{|C_j|} \sum_{i \in C_j} \mathbf{x}_i, \quad C_j = \{i : s_i = j\}.$$

Each step can only decrease the objective, which is bounded below, so the alternation converges (to a local minimum; the objective is nonconvex and the result depends on initialization). The only two operations K-means performs on the data are computing squared distances to centroids and averaging points, and, as with PCA, this is exactly the structure that lets us kernelize.

15.2 The kernel K-means algorithm

To cluster data whose natural similarity is nonlinear, or data that are not vectors at all, we move the objective into an RKHS. Given points $\mathbf{x}_1, \dots, \mathbf{x}_n$ in a set $\mathcal{X}$ and a p.d. kernel K with RKHS $\mathcal{H}$ and feature map φ , the centroids now live in $\mathcal{H}$ and the objective becomes

$$\min_{\substack{\mu_j \in \mathcal{H}, j=1, \dots, k \\ s_i \in \{1, \dots, k\}, i=1, \dots, n}} \sum_{i=1}^n \|\varphi(\mathbf{x}_i) - \mu_{s_i}\|_{\mathcal{H}}^2.$$

It helps to picture what this buys us. Ordinary K-means splits $\mathbb{R}^p$ into cells with flat boundaries, the point being assigned to whichever centroid is nearest, so it can only recover clusters that are roughly convex blobs; two interlocking rings or a small cluster wrapped inside a larger one defeat it. Running the same algorithm on $\varphi(\mathbf{x})$ draws those same flat boundaries in feature space, but a flat boundary in $\mathcal{H}$ pulls back to a curved boundary in $\mathcal{X}$, bent to follow the geometry the kernel encodes. The nonlinearity comes for free from the feature map; the algorithm itself is unchanged.

The alternating scheme goes through unchanged, but we must check that both steps can be carried out using kernel values alone, since we never see the vectors $\varphi(\mathbf{x}_i)$. The centroid update is what makes this work, and it rests on a small fact about centers of mass.

Proposition (the center of mass minimizes total dispersion)

The center of mass $\bar{\varphi}_n = \frac{1}{n} \sum_{i=1}^n \varphi(\mathbf{x}_i)$ solves

$$\min_{\mu \in \mathcal{H}} \sum_{i=1}^n \|\varphi(\mathbf{x}_i) - \mu\|_{\mathcal{H}}^2.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Expand the averaged objective, using bilinearity of the inner product:

$$\frac{1}{n} \sum_{i=1}^n \|\varphi(\mathbf{x}_i) - \mu\|_{\mathcal{H}}^2 = \frac{1}{n} \sum_{i=1}^n \|\varphi(\mathbf{x}_i)\|_{\mathcal{H}}^2 - \left\langle \frac{2}{n} \sum_{i=1}^n \varphi(\mathbf{x}_i), \mu \right\rangle_{\mathcal{H}} + \|\mu\|_{\mathcal{H}}^2.$$

Recognizing $\frac{1}{n} \sum_i \varphi(\mathbf{x}_i) = \bar{\varphi}_n$ in the middle term and completing the square in μ ,

$$= \frac{1}{n} \sum_{i=1}^n \|\varphi(\mathbf{x}_i)\|_{\mathcal{H}}^2 - \|\bar{\varphi}_n\|_{\mathcal{H}}^2 + \|\bar{\varphi}_n - \mu\|_{\mathcal{H}}^2.$$

The first two terms do not depend on μ , and the last is minimized, at value zero, exactly when $\mu = \bar{\varphi}_n$. $\square$

15.2.1 The two steps, in kernel form

By the proposition, with assignments fixed, the optimal centroid of cluster j is the center of mass of its members,

$$\forall j, \quad \mu_j = \frac{1}{|C_j|} \sum_{i \in C_j} \varphi(\mathbf{x}_i), \quad C_j = \{i : s_i = j\}.$$

We cannot store this vector, but we do not need to: it enters the assignment step only through squared distances, and those we can expand into kernel values. For the assignment step, fixing the centroids and using $\mu_s = \frac{1}{|C_s|} \sum_{j \in C_s} \varphi(\mathbf{x}_j)$,

$$s_i \in \arg \min_{s \in \{1, \dots, k\}} \left\| \varphi(\mathbf{x}_i) - \frac{1}{|C_s|} \sum_{j \in C_s} \varphi(\mathbf{x}_j) \right\|_{\mathcal{H}}^2.$$

Expanding the squared norm through the reproducing kernel, $\langle \varphi(\mathbf{x}_a), \varphi(\mathbf{x}_b) \rangle_{\mathcal{H}} = K(\mathbf{x}_a, \mathbf{x}_b)$, gives a formula in kernel values only:

$$s_i \in \arg \min_{s \in \{1, \dots, k\}} \left[K(\mathbf{x}_i, \mathbf{x}_i) - \frac{2}{|C_s|} \sum_{j \in C_s} K(\mathbf{x}_i, \mathbf{x}_j) + \frac{1}{|C_s|^2} \sum_{j, l \in C_s} K(\mathbf{x}_j, \mathbf{x}_l) \right].$$

The three terms are the squared norm of the point, its average similarity to the cluster, and the internal cohesion of the cluster. Every operation is a kernel evaluation, so kernel K-means never touches $\mathcal{H}$ directly. It alternates the centroid step (bookkeeping the memberships C_j) and this assignment step until convergence, a greedy descent on the objective just like ordinary K-means.

A concrete case makes the three terms tangible. Take two concentric rings of points with a Gaussian kernel whose bandwidth is smaller than the gap between the rings, so that $K(\mathbf{x}_i, \mathbf{x}_j)$ is appreciable only when i and j sit on the same ring. Then a candidate cluster that mixes the two rings gives its members a small average-similarity term $\frac{2}{|C_s|} \sum_{j \in C_s} K(\mathbf{x}_i, \mathbf{x}_j)$ and is penalized, whereas the two single-ring clusters make that term large. The assignment step therefore pushes each point toward the ring it belongs to and recovers a grouping that ordinary K-means cannot: with flat cell boundaries the linear method can only cut the pair of rings with a straight line through both, splitting each ring in half.

15.2.2 An equivalent objective

Substituting the optimal centroids back into the cost turns kernel K-means into a pure assignment problem: minimize, over assignments alone,

$$\min_{\substack{s_i \in \{1, \dots, k\} \\ i=1, \dots, n}} \sum_{i=1}^n \left\| \varphi(\mathbf{x}_i) - \frac{1}{|C_{s_i}|} \sum_{j \in C_{s_i}} \varphi(\mathbf{x}_j) \right\|_{\mathcal{H}}^2,$$

equivalently, in kernel values,

$$\min_{\substack{s_i \in \{1, \dots, k\} \\ i=1, \dots, n}} \sum_{i=1}^n \left[K(\mathbf{x}_i, \mathbf{x}_i) - \frac{2}{|C_{s_i}|} \sum_{j \in C_{s_i}} K(\mathbf{x}_i, \mathbf{x}_j) + \frac{1}{|C_{s_i}|^2} \sum_{j, l \in C_{s_i}} K(\mathbf{x}_j, \mathbf{x}_l) \right].$$

The last two sums simplify when we group points by cluster. Summing the cross term over all i and reindexing by the cluster l that i belongs to,

$$\sum_{i=1}^n \frac{1}{|C_{s_i}|} \sum_{j \in C_{s_i}} K(\mathbf{x}_i, \mathbf{x}_j) = \sum_{l=1}^k \frac{1}{|C_l|} \sum_{i,j \in C_l} K(\mathbf{x}_i, \mathbf{x}_j),$$

and by the same reindexing the double-counted cohesion term collapses identically:

$$\sum_{i=1}^n \frac{1}{|C_{s_i}|^2} \sum_{j,l \in C_{s_i}} K(\mathbf{x}_j, \mathbf{x}_l) = \sum_{l=1}^k \frac{1}{|C_l|} \sum_{i,j \in C_l} K(\mathbf{x}_i, \mathbf{x}_j).$$

The two combine into a single term, the constant $\sum_i K(\mathbf{x}_i, \mathbf{x}_i)$ is fixed by the data and drops out of the minimization, and after flipping the sign the minimization becomes a maximization.

Proposition (equivalent kernel K-means objective)

Minimizing the kernel K-means cost is equivalent to

$$\max_{\substack{s_i \in \{1, \dots, k\} \\ i=1, \dots, n}} \sum_{l=1}^k \frac{1}{|C_l|} \sum_{i,j \in C_l} K(\mathbf{x}_i, \mathbf{x}_j).$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The objective is now transparent: reward partitions whose clusters have high total internal similarity, normalized by cluster size. This is a hard combinatorial optimization problem, since it ranges over all assignments of n points to k clusters. There are two standard ways to attack it. The first is the greedy alternating descent we just derived, kernel K-means itself. The second is to relax the combinatorial constraint into a continuous one that can be solved exactly, which leads to spectral clustering. That these are not two unrelated algorithms but two attacks on one objective, and that this same trace maximization underlies the classical graph-cut criteria as well, was made explicit by Dhillon, Guan, and Kulis (2004).

15.3 Spectral clustering

Rather than descend greedily and risk poor local minima, spectral clustering relaxes the assignment problem into an eigenvalue problem, solves that exactly, and then rounds the continuous solution back to a partition. The starting point is the maximization we just obtained,

$$\max_{\substack{s_i \in \{1, \dots, k\} \\ i=1, \dots, n}} \sum_{l=1}^k \frac{1}{|C_l|} \sum_{i,j \in C_l} K(\mathbf{x}_i, \mathbf{x}_j).$$

15.3.1 A matrix form of the objective

To expose the eigenstructure we encode the partition in matrices. Introduce the binary assignment matrix $A \in \{0, 1\}^{n \times k}$, where $[A]_{ij} = 1$ if point i belongs to cluster j and 0 otherwise; each row of A sums to one, since every point lands in exactly one cluster. Introduce also the diagonal rescaling matrix $D \in \mathbb{R}^{k \times k}$ whose entry $[D]_{jj} = (\sum_{i=1}^n [A]_{ij})^{-1}$ is the reciprocal of the size of cluster j . With these, the double sum for cluster l is $[A^\top K A]_{ll}$, and dividing by the cluster size and summing over l is exactly a trace:

$$\sum_{l=1}^k \frac{1}{|C_l|} \sum_{i,j \in C_l} K(\mathbf{x}_i, \mathbf{x}_j) = \text{trace}(D^{1/2} A^\top K A D^{1/2}).$$

So the objective is (the algebra is a short exercise)

$$\max_{A,D} \text{trace}(D^{1/2} A^\top K A D^{1/2}) \quad \text{s.t.} \quad (\star) \text{ and } (\star\star),$$

where $(\star)$ is the constraint that A is binary with unit row sums and $(\star\star)$ that D holds the inverse cluster cardinalities.

15.3.2 The relaxation

The key observation is that these constraints force an orthonormality relation. A direct computation shows

$$D^{1/2} A^\top A D^{1/2} = I,$$

because $A^\top A$ is the diagonal matrix of cluster sizes and D inverts exactly those sizes (again a short exercise). Now set $Z = A D^{1/2} \in \mathbb{R}^{n \times k}$. The objective is $\text{trace}(Z^\top K Z)$ and the identity above reads $Z^\top Z = I$. The *relaxation* consists of dropping the rigid constraints $(\star)$ and $(\star\star)$ on A and D and keeping only this orthonormality, letting Z range over all real matrices with orthonormal columns:

$$\max_{Z \in \mathbb{R}^{n \times k}} \text{trace}(Z^\top K Z) \quad \text{s.t.} \quad Z^\top Z = I.$$

The point of the move is a change in the kind of problem we face. The original maximization ranges over assignments, a discrete set of exponentially many partitions with no calculus to exploit and generally no efficient way to find the best member. The relaxed one ranges over a continuous set, the matrices with orthonormal columns, on which the objective is a smooth quadratic form with a known global maximizer. We trade the guarantee that the answer is a genuine partition for the ability to solve the resulting problem exactly, and we recover a partition at the end by rounding.

Proposition (solution of the relaxed problem)

A solution $Z^\star$ of

$$\max_{Z \in \mathbb{R}^{n \times k}} \text{trace}(Z^\top K Z) \quad \text{s.t.} \quad Z^\top Z = I$$

is given by the $n \times k$ matrix whose columns are the eigenvectors of K associated with its k largest eigenvalues.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

This is the Ky Fan characterization of the top eigenspace: among all sets of k orthonormal vectors, the trace of the quadratic form is maximized by the top k eigenvectors. Notice that this is precisely the computation kernel PCA performs; spectral clustering and kernel PCA share the same linear-algebra core, an eigendecomposition of the Gram matrix, and differ only in what they do with the eigenvectors. (When K is built as a similarity or affinity matrix on a graph, this eigenproblem is the one governed by the graph Laplacian, which is why the method is called spectral: the relevant directions are the extreme eigenvectors of a matrix attached to the data graph.)

An idealized case explains why one can hope to round the eigenvectors back into a partition at all. Suppose the data split into k groups so cleanly that the kernel is block diagonal, with $K(\mathbf{x}_i, \mathbf{x}_j) = 0$ whenever i and j lie in

different groups. Then K is a direct sum of k blocks, its top eigenvectors may be taken to vanish outside a single block, and each point's row of $Z^\star$ becomes a scaled one-hot vector naming its group: the two rows belonging to the same group are identical, and rows from different groups are orthogonal. Rounding is then trivial, since the rows already fall on k distinct points. Real data only approach this block structure, so in practice the rows land near, rather than exactly on, those k ideal points, which is precisely why the robust rounding runs a final K-means to snap them back together.

15.3.3 From eigenvectors back to a partition

The relaxed solution $Z^\star$ is a real matrix, not a valid assignment matrix, so we must round it back. There are several ways to do this, reflecting that the rounding is a design choice rather than a theorem.

A first, direct answer uses the shape the original constraints would have imposed: with $(\star)$ in force each row of A has a single nonzero entry, so we simply read off, for each row of $Z^\star$, the index of its largest entry and assign the point to that cluster. A second, more robust and by far the most common answer normalizes each row of $Z^\star$ to unit ℓ_2 norm and then runs ordinary K-means on the n rows, treating each row as a k -dimensional feature vector for its point. The reason this is more than a heuristic is that the top eigenvectors give each point a short coordinate, and when the data really do fall into k well-separated groups these coordinates are nearly constant within a group, so the rows collect into k tight clusters that even plain K-means resolves cleanly; this is the observation behind the algorithm of Ng, Jordan, and Weiss (2002). This last procedure, eigendecompose the (kernel or affinity) matrix, take the top k eigenvectors, normalize the rows, and cluster them, is what is usually meant by *spectral clustering*. Other variants differ in normalization and in how many eigenvectors are used, but all share the same skeleton: relax to an eigenproblem, solve it exactly, and round.

Remark (affinity matrices and the Laplacian)

The relaxation above diagonalizes the kernel matrix K itself, but many spectral methods instead build a rescaled matrix whose spectrum reflects cluster structure more evenly. Collecting the row sums into the diagonal degree matrix D with $[D]_{ii} = \sum_j K(\mathbf{x}_i, \mathbf{x}_j)$, useful partitions can be read from the eigenvectors of $D^{-1}K$, of the symmetrically normalized $D^{-1/2}KD^{-1/2}$, or of the graph Laplacian $L = D - K$ (Shawe-Taylor and Cristianini, 2004). The Laplacian is the cleanest to reason about: a short computation gives $\mathbf{v}^\top L \mathbf{v} = \frac{1}{2} \sum_{i,j} K(\mathbf{x}_i, \mathbf{x}_j) (v_i - v_j)^2$, so L is positive semidefinite whenever the affinities are nonnegative, and the all-ones vector is always an eigenvector with eigenvalue 0. Its smallest nontrivial eigenvectors vary slowly across strongly connected points and jump between weakly connected ones, which is why thresholding or clustering them separates the graph along its weakest cuts. The normalizations by D matter in practice because they keep a few high-degree points from dominating the leading eigenvectors, giving the more balanced partitions that normalized-cut criteria were designed to produce.

15.4 The graph-cut view: normalized cuts

The relaxation above reached spectral clustering by starting from kernel K-means and loosening its combinatorial constraint. The closing remark hinted that a normalized version of the affinity produces better balanced partitions, but left the choice of normalization looking like a practical trick. There is a second, entirely independent road to the same eigenvectors that explains the normalization instead of positing it. It reads the data as a weighted graph and asks for the cheapest way to cut that graph into pieces. This is the route of Shi and Malik (2000), and it is worth walking because it makes precise both what objective spectral clustering approximates and, as we will see, exactly what the approximation gives up.

Assemble the affinities into a weighted graph. The vertices are the data points, and between points $\mathbf{x}_i$ and $\mathbf{x}_j$ we place an edge of weight $w_{ij} = K(\mathbf{x}_i, \mathbf{x}_j) \geq 0$. Collected into a matrix these weights form the weighted adjacency W , which is nothing but the Gram matrix K read in its role as a graph; the degree matrix D with $[D]_{ii} = d_i = \sum_j w_{ij}$ and the Laplacian $L = D - W$ are exactly those of the remark above. Clustering the points now means severing the graph into groups along its lightest edges, an image that connects the affinity spectrum to the intrinsic geometry

of the data in the same way the graph Laplacian shadows the Laplace-Beltrami operator in Chapter 27, Geometric and Equivariant Kernels.

The weight of the edges broken by a partition into A and its complement $B = V \setminus A$ is the *cut*,

$$\text{cut}(A, B) = \sum_{i \in A, j \in B} w_{ij}.$$

Minimizing the cut outright is the classical min-cut problem, and it is the wrong objective for clustering: because the cut counts only the edges crossing the boundary, it is minimized by shaving off whatever single vertex is most weakly attached to the rest, returning a lopsided partition of one point against $n - 1$. Shi and Malik (2000) diagnosed this bias and corrected it by measuring each cut against the total connectivity of the sides it produces rather than in absolute terms.

Definition (normalized cut and normalized association)

Write the *volume* of a vertex set for the total degree it carries, $\text{vol}(A) = \sum_{i \in A} d_i$, and the internal *association* for the weight it keeps inside, $\text{assoc}(A, A) = \sum_{i, j \in A} w_{ij}$. The normalized cut and normalized association of a partition (A, B) are

$$\text{Ncut}(A, B) = \frac{\text{cut}(A, B)}{\text{vol}(A)} + \frac{\text{cut}(A, B)}{\text{vol}(B)}, \quad \text{Nassoc}(A, B) = \frac{\text{assoc}(A, A)}{\text{vol}(A)} + \frac{\text{assoc}(B, B)}{\text{vol}(B)}.$$

Dividing by the volume is what defeats the isolation pathology: peeling off a single low-degree vertex makes $\text{cut}(A, B)$ small but makes $\text{vol}(A)$ just as small, so the ratio stays large, and only a cut that is light relative to the mass on both sides scores well. The two criteria are in fact one criterion. Since every edge leaving A either stays inside A or crosses to B , the degree sum splits as $\text{vol}(A) = \text{assoc}(A, A) + \text{cut}(A, B)$, so $\text{cut}(A, B)/\text{vol}(A) = 1 - \text{assoc}(A, A)/\text{vol}(A)$, and adding the two sides gives the exact relation

$$\text{Ncut}(A, B) = 2 - \text{Nassoc}(A, B).$$

Cutting the graph as cheaply as possible is therefore the same as keeping as much weight as possible inside the clusters, normalized by their volumes. That second reading is precisely the normalized within-cluster similarity that the equivalent kernel K-means objective rewarded, the first sign that these two developments are describing one problem.

The difficulty is that this clean objective is intractable to optimize exactly. Ranging over all ways to split n vertices, the search is combinatorial, and minimizing the normalized cut over all partitions is NP-hard, even in the two-way case on general graphs (Shi and Malik, 2000; von Luxburg, 2007). The ratio cut of Hagen and Kahng (1992), which normalizes by the cardinalities $|A|, |B|$ rather than the volumes, is hard for the same reason. So we do for the normalized cut what we did for the kernel K-means cost: relax the discrete search to a continuous one that linear algebra can solve, and round afterward.

15.5 The relaxation to the Fiedler vector

To relax the normalized cut we first write it as a Rayleigh quotient of the Laplacian, so that the objective becomes a quadratic form and the constraints become linear. The whole reduction rests on encoding the partition in a single cleverly scaled indicator vector.

Lemma (the normalized cut as a Laplacian Rayleigh quotient)

Fix a partition (A, B) and define $f \in \mathbb{R}^n$ by

$$f_i = \begin{cases} +\sqrt{\text{vol}(B)/\text{vol}(A)} & i \in A, \\ -\sqrt{\text{vol}(A)/\text{vol}(B)} & i \in B. \end{cases}$$

Then f is D -orthogonal to the constant vector, $(Df)^\top \mathbf{1} = 0$; its D -norm is fixed, $f^\top Df = \text{vol}(V)$; and it carries the cut,

$$f^\top Lf = \text{vol}(V) \cdot \text{Ncut}(A, B).$$

Consequently $\text{Ncut}(A, B) = f^\top Lf / f^\top Df$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

For the first identity, group the sum $\sum_i d_i f_i$ by side:

$$(Df)^\top \mathbf{1} = \sum_i d_i f_i = \text{vol}(A) \sqrt{\frac{\text{vol}(B)}{\text{vol}(A)}} - \text{vol}(B) \sqrt{\frac{\text{vol}(A)}{\text{vol}(B)}} = \sqrt{\text{vol}(A) \text{vol}(B)} - \sqrt{\text{vol}(A) \text{vol}(B)} = 0.$$

The same grouping gives the D -norm, since f_i^2 equals $\text{vol}(B)/\text{vol}(A)$ on A and $\text{vol}(A)/\text{vol}(B)$ on B :

$$f^\top Df = \text{vol}(A) \cdot \frac{\text{vol}(B)}{\text{vol}(A)} + \text{vol}(B) \cdot \frac{\text{vol}(A)}{\text{vol}(B)} = \text{vol}(B) + \text{vol}(A) = \text{vol}(V).$$

For the third, use the quadratic-form identity $f^\top Lf = \frac{1}{2} \sum_{i,j} w_{ij} (f_i - f_j)^2$ established for the Laplacian in the remark above. Only cross-boundary pairs contribute, because f is constant on each side, and for $i \in A$, $j \in B$ the gap is $f_i - f_j = \sqrt{\text{vol}(B)/\text{vol}(A)} + \sqrt{\text{vol}(A)/\text{vol}(B)}$, whose square is $\text{vol}(B)/\text{vol}(A) + 2 + \text{vol}(A)/\text{vol}(B)$. Summing the symmetric double sum over the two orientations of each cut edge cancels the factor $\frac{1}{2}$ and leaves

$$f^\top Lf = \text{cut}(A, B) \left[\frac{\text{vol}(B)}{\text{vol}(A)} + 2 + \frac{\text{vol}(A)}{\text{vol}(B)} \right] = \text{cut}(A, B) \left[\frac{\text{vol}(V)}{\text{vol}(A)} + \frac{\text{vol}(V)}{\text{vol}(B)} \right] = \text{vol}(V) \cdot \text{Ncut}(A, B),$$

where the middle step collects $\text{vol}(B)/\text{vol}(A) + 1 = \text{vol}(V)/\text{vol}(A)$ and its mirror image. Dividing by $f^\top Df = \text{vol}(V)$ gives the Rayleigh-quotient form. $\square$

The lemma turns the normalized cut into an exact quadratic ratio, but only over the discrete set of vectors f of the special two-valued shape. Minimizing that ratio over those vectors is still the original NP-hard search dressed in new notation. The *relaxation* is to drop the requirement that f take only two values and let it range over all of $\mathbb{R}^n$, keeping just the linear constraint $Df \perp \mathbf{1}$ that the shape forced:

$$\min_{f \in \mathbb{R}^n} \frac{f^\top Lf}{f^\top Df} \quad \text{s.t.} \quad (Df)^\top \mathbf{1} = 0.$$

15.5.1 The normalized Laplacian and the Fiedler vector

This relaxed problem is a generalized Rayleigh quotient, and its minimizers are eigenvectors of a generalized eigenproblem. The substitution $g = D^{1/2}f$ makes this transparent: the objective becomes $g^\top L_{\text{sym}} g / g^\top g$ with the symmetrically normalized Laplacian

$$L_{\text{sym}} = D^{-1/2} L D^{-1/2} = I - D^{-1/2} W D^{-1/2},$$

and the constraint becomes $g \perp D^{1/2}\mathbf{1}$. Now L_{sym} is symmetric positive semidefinite, and its smallest eigenvalue is 0 with eigenvector $g_0 = D^{1/2}\mathbf{1}$, because $L\mathbf{1} = 0$ makes $L_{\text{sym}} D^{1/2}\mathbf{1} = D^{-1/2} L\mathbf{1} = 0$. The constraint asks exactly that g be orthogonal to this first eigenvector, so by the Courant-Fischer characterization of eigenvalues the constrained minimum of the quotient is the second-smallest eigenvalue of L_{sym} , attained at its second eigenvector.

Proposition (the relaxed indicator is the Fiedler vector)

The relaxed problem is solved by $f^\star = D^{-1/2}g_2$, where g_2 is the eigenvector of L_{sym} for its second-smallest eigenvalue λ_2 . Equivalently, $f^\star$ is the eigenvector for the second-smallest eigenvalue of the generalized problem

$$Lf = \lambda Df,$$

and its eigenvalue equals λ_2 . This $f^\star$ is the *Fiedler vector* of the graph (Fiedler, 1973), the discrete analogue of the first nonconstant vibration mode; it is at once the second eigenvector of the random-walk Laplacian $L_{\text{rw}} = D^{-1}L$, which has the same spectrum, and the $D^{-1/2}$ rescaling of the second eigenvector g_2 of L_{sym} .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The recipe is now fixed by the derivation rather than chosen: build the affinity, form the normalized Laplacian, and read the partition off the second eigenvector. The eigenvalue λ_2 , the graph's algebraic connectivity, measures how weak the best cut is; a small λ_2 certifies a nearly disconnected graph and a clean split. To turn the real vector $f^\star$ into an actual partition one rounds it, in the two-way case by the sign, sending $\{i : f_i^\star > 0\}$ to one cluster and $\{i : f_i^\star < 0\}$ to the other (or thresholding at the median for a balanced split). The Fiedler coordinate is also an embedding: it places each point on a line so that strongly connected points sit close together, the one-dimensional case of the spectral embeddings used for layout in Chapter 17, Data Visualization and Kernel MDS. The following worked example carries the whole pipeline through on a graph small enough to read every number.

Example (a Fiedler cut on two triangles)

Two triangles joined by one weak bridge. Cluster $A = \{0, 1, 2\}$ and cluster $B = \{3, 4, 5\}$ are each a complete triangle of unit-weight edges, and the only edge between them is the bridge $\{2, 3\}$ with weight $\varepsilon = 0.1$. So W is 6×6 , $D = \text{diag}(d_i)$, $L = D - W$, and $L_{\text{sym}} = D^{-1/2} L D^{-1/2}$. All numbers from `checks/ch-cluster-ex1.py`.

1. Read the degrees and the true cut. The interior points have degree 2 and the two bridge endpoints degree 2.1, so $d = (2, 2, 2.1, 2.1, 2, 2)$, giving $\text{vol}(A) = \text{vol}(B) = 6.1$ and $\text{vol}(V) = 12.2$. Only the bridge crosses, so $\text{cut}(A, B) = 0.1$ and

$$\text{Ncut}(A, B) = \frac{0.1}{6.1} + \frac{0.1}{6.1} = 0.032787.$$

2. Form the normalized Laplacian and diagonalize it. The eigenvalues of L_{sym} , in ascending order, are

$$(0, 0.0314, 1.4524, 1.5, 1.5, 1.5162),$$

and the generalized problem $Lf = \lambda Df$ has the identical spectrum. The first eigenvalue 0 belongs to the constant mode; the second, $\lambda_2 = 0.0314$, is small, certifying that the graph is nearly two pieces.

3. Round the Fiedler vector. The second generalized eigenvector, scaled so its largest entry is 1, is

$$f^\star = (1, 1, 0.9372, -0.9372, -1, -1).$$

Its sign splits the vertices as $(+, +, +, -, -, -)$, recovering exactly $A = \{0, 1, 2\}$ against $B = \{3, 4, 5\}$. The bridge endpoints 2 and 3 carry the smallest magnitudes, 0.9372, because each touches the far side and is pulled toward the boundary value 0.

4. Confirm the Rayleigh-quotient identity. Since $\text{vol}(A) = \text{vol}(B)$, the two-valued indicator of the lemma is $g = (1, 1, 1, -1, -1, -1)$. It checks out that $(Dg)^\top \mathbf{1} = 0$ and $g^\top Dg = 12.2 = \text{vol}(V)$, while

$$g^\top Lg = 0.4 = 12.2 \times 0.032787 = \text{vol}(V) \cdot \text{Ncut}(A, B).$$

The relaxed optimum $\lambda_2 = 0.0314$ sits just below the true $\text{Ncut} = 0.0328$: the eigenvalue lower-bounds the cut it approximates.

Reading. The normalized Laplacian is assembled from the affinity alone, its second eigenvalue is small exactly because a light cut exists, and the sign of the matching eigenvector reproduces the two triangles without ever enumerating partitions. The gap between $\lambda_2 = 0.0314$ and $\text{Ncut} = 0.0328$ is the price of relaxing a discrete indicator to a real eigenvector.

Verification artifact. `checks/example-ch-cluster-example-14-1.json` records the example source hash and verification scope.

15.5.2 What the relaxation gives up

It is important to be precise about the sense in which the eigenvector solves the clustering problem, because the relaxation is not free. Three things are lost, and the worked example shows each. First, the minimizer $f^\star$ is a real vector, not one of the two-valued indicators; it is not itself a partition, and the sign or threshold rounding that turns it into one lies outside the optimization and carries no guarantee. Second, because the relaxed feasible set strictly contains the discrete one, its minimum can only be lower, so the second eigenvalue is a *lower bound* on the optimal normalized cut,

$$\lambda_2 \leq \min_{(A,B)} \text{Ncut}(A, B),$$

and the gap, 0.0314 against 0.0328 above, is genuine relaxation error rather than numerical noise. Third, the bound can in the worst case be arbitrarily loose: there are graphs on which the rounded partition is far from the true optimum, so the eigenvector is a heuristic with no approximation ratio, not an exact solver (von Luxburg, 2007). What earns the method its place is that on data with real cluster structure the bound is tight, the Fiedler coordinate is nearly piecewise constant, and the rounding is unambiguous, exactly as in the example.

15.6 Normalized-cut spectral clustering

For more than two clusters the same logic runs with the top eigenvectors in place of the single Fiedler vector. To cut a graph into k pieces one uses the k smallest eigenvectors of the normalized Laplacian, which is the same as the k largest eigenvectors of the normalized affinity $D^{-1/2}WD^{-1/2} = I - L_{\text{sym}}$, stacks them as the columns of an $n \times k$ matrix, and reads each point's row as its coordinates in a k -dimensional spectral embedding. In the ideal case of k disconnected components these rows fall on k mutually orthogonal directions, and for data that merely approach that structure they concentrate near k tight clusters, so a final K-means on the rows recovers the partition. Ng, Jordan, and Weiss (2002) observed that normalizing each row to unit length before the K-means is

what sharpens this concentration, since it projects the embedded points onto a sphere where the ideal clusters become well-separated points, and their procedure is the one most often meant by spectral clustering.

Algorithm (normalized-cut spectral clustering; Ng, Jordan, Weiss, 2002)

Input Points $\mathbf{x}_1, \dots, \mathbf{x}_n$; p.d. kernel K ; number of clusters k .

Output A partition of the points into k clusters.

1. Build the affinity W with $W_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ for $i \neq j$ and $W_{ii} = 0$, and the degree matrix $[D]_{ii} = \sum_j W_{ij}$.
2. Form the normalized affinity $M = D^{-1/2} W D^{-1/2}$, equivalently the symmetric normalized Laplacian $L_{\text{sym}} = I - M$.
3. Compute the k largest eigenvectors of M (the k smallest of L_{sym}) and place them in the columns of $X \in \mathbb{R}^{n \times k}$.
4. Normalize each row of X to unit ℓ_2 norm, $Y_{ij} = X_{ij} / (\sum_l X_{il}^2)^{1/2}$.
5. Cluster the n rows of Y with ordinary K-means into k groups.
6. Assign point i to the cluster that received row i of Y .

On the two-triangle graph the algorithm is easy to trace. The two largest eigenvalues of $M = D^{-1/2} W D^{-1/2}$ are 1 and 0.9686, the second being close to 1 precisely because the bridge is weak, and the row-normalized embedding sends every vertex of cluster A to about $(0.7071, -0.7069)$ and every vertex of B to about $(0.7071, 0.7069)$. The two clusters land on two nearly antipodal points of the unit circle, and K-means separates them at once. This is the multi-way face of the single-eigenvector cut of the previous section.

15.7 Kernel K-means and normalized cut are one objective

We have now reached the normalized cut twice, once by relaxing kernel K-means and once by relaxing a graph cut, and arrived both times at the eigenvectors of a normalized affinity. That is not a coincidence. Dhillon, Guan, and Kulis (2004) proved that the normalized cut is itself a weighted kernel K-means objective, so the two algorithms optimize the same function and differ only in how they descend on it. The bridge is the trace form the chapter already built. Encode a k -way partition by the volume-scaled indicators $z_c \in \mathbb{R}^n$, with $z_c(i) = \text{vol}(A_c)^{-1/2}$ when $i \in A_c$ and 0 otherwise. A direct computation gives $z_c^\top W z_c = \text{assoc}(A_c, A_c) / \text{vol}(A_c)$ and $z_c^\top D z_c = 1$, so with $Z = [z_1, \dots, z_k]$ the normalized association is a trace and the indicators are D -orthonormal:

$$\text{Nassoc}(\{A_c\}) = \sum_{c=1}^k z_c^\top W z_c = \text{trace}(Z^\top W Z), \quad Z^\top D Z = I.$$

Substituting $Y = D^{1/2} Z$, which satisfies $Y^\top Y = I$, turns this into the familiar orthonormally constrained trace maximization, now with the *normalized* affinity in the middle:

$$\max_{Y^\top Y = I} \text{trace}(Y^\top D^{-1/2} W D^{-1/2} Y).$$

This is exactly the relaxation of the chapter's kernel PCA-style eigenproblem, with the kernel matrix taken to be the normalized affinity $D^{-1/2} W D^{-1/2}$; its solution is the top- k eigenvectors, the same ones the Ng-Jordan-Weiss algorithm computes, and since $\text{trace}(Y^\top (I - L_{\text{sym}}) Y) = k - \text{trace}(Y^\top L_{\text{sym}} Y)$, maximizing normalized association is minimizing $\text{trace}(Y^\top L_{\text{sym}} Y)$, the standard normalized-cut objective. The two-way Fiedler derivation is the $k = 2$ case of this.

The equivalence is more than an aesthetic tidiness, because it runs in the discrete direction too. Reading the same identity as a genuine (unrelaxed) clustering cost, Dhillon, Guan, and Kulis (2004) show that the normalized cut equals the weighted kernel K-means objective

$$\sum_{c=1}^k \sum_{i \in A_c} d_i \|\varphi(\mathbf{x}_i) - \mathbf{m}_c\|_{\mathcal{H}}^2, \quad \mathbf{m}_c = \frac{\sum_{i \in A_c} d_i \varphi(\mathbf{x}_i)}{\sum_{i \in A_c} d_i},$$

run with point weights equal to the degrees d_i and a kernel whose Gram matrix is the normalized affinity (shifted by a multiple of D^{-1} to keep it positive definite). This gives a second, eigenvector-free way to minimize the normalized cut: run the weighted version of the greedy assignment loop from the start of the chapter, which decreases the objective at every step and needs only the kernel values, never an $n \times n$ eigendecomposition. Spectral clustering and weighted kernel K-means are thus the exact relaxed and greedy attacks on a single normalized-cut cost, and one can trade the global reach of the eigenvectors against the memory economy of the descent as the problem size demands.

15.8 Out-of-sample extension

Spectral clustering embeds a point by the value at that point of an eigenvector of an $n \times n$ matrix built from the whole training set. A new point $\mathbf{x}$ that arrives after the eigendecomposition has no row in that matrix, so it has no coordinate, and recomputing the eigenvectors of an enlarged matrix for every query is out of the question. The Nystrom method supplies the missing coordinate by treating each eigenvector as the sampling, at the training points, of an underlying eigenfunction of a kernel operator, and then evaluating that eigenfunction at the new point (Williams and Seeger, 2001; Bengio et al., 2004).

Concretely, let $M = D^{-1/2} W D^{-1/2}$ be the normalized affinity with eigenpairs $M \mathbf{u}_m = \lambda_m \mathbf{u}_m$, and let $\tilde{k}(\mathbf{x}, \mathbf{x}_i)$ denote the same normalized affinity between the query $\mathbf{x}$ and a training point $\mathbf{x}_i$, computed from the training degrees. The Nystrom extension of the m -th eigenvector to the new point is

$$\phi_m(\mathbf{x}) = \frac{1}{\lambda_m} \sum_{i=1}^n u_m(i) \tilde{k}(\mathbf{x}, \mathbf{x}_i).$$

Evaluated at a training point $\mathbf{x}_j$ this returns $\frac{1}{\lambda_m} (M \mathbf{u}_m)_j = u_m(j)$, so the extension is consistent: it agrees with the eigenvector it extends on the data used to build it, and interpolates elsewhere. To place the new point one forms its spectral coordinates $(\phi_1(\mathbf{x}), \dots, \phi_k(\mathbf{x}))$, applies the same row normalization, and assigns it to the nearest of the K-means centroids found during training, all without recomputing a single eigenvector.

This is not a new mechanism but one already met under another name. The extension formula is, up to the eigenvalue scaling, the projection of $\varphi(\mathbf{x})$ onto the m -th kernel principal component from Chapter 13, Kernel PCA and Denoising: spectral embedding and kernel PCA compute the same eigenfunctions, a correspondence Bengio et al. (2004) make precise and which explains why the out-of-sample rule looks identical in both. The one caveat is that the query is assumed not to disturb the graph, since its own degree and its effect on the normalization are ignored; the extension is therefore trustworthy for test points drawn from the same distribution as the training data, and should be recomputed from scratch when enough new points arrive to change the affinity structure they are being measured against.

15.9 Common mistakes and practical implications

For **Kernel Clustering and Spectral Methods**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

15.10 Summary and further reading

This chapter established explain the central definitions and claims in Kernel Clustering and Spectral Methods; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Dhillon and Kulis 2004), (Ng and Weiss 2002), (Schölkopf and Smola 2002).

15.11 Exercises

1. warm-up In the kernel K-means assignment step, the squared distance from point $\mathbf{x}_i$ to the centroid of a candidate cluster s expands into three terms: the self-similarity $K(\mathbf{x}_i, \mathbf{x}_i)$, the average similarity $\frac{2}{|C_s|} \sum_{j \in C_s} K(\mathbf{x}_i, \mathbf{x}_j)$, and the internal cohesion $\frac{1}{|C_s|^2} \sum_{j, l \in C_s} K(\mathbf{x}_j, \mathbf{x}_l)$. Say which of the three does not depend on the candidate cluster s , and may therefore be dropped from the arg min over s ; which depends on s but not on the point i ; and which couples the point to the cluster. Conclude that assigning $\mathbf{x}_i$ amounts to comparing, across clusters, the cohesion minus twice the average similarity.
2. challenge The graph Laplacian $L = D - K$, with $[D]_{ii} = \sum_j K(\mathbf{x}_i, \mathbf{x}_j)$, sits at the heart of the spectral methods. (a) Prove the quadratic-form identity $\mathbf{v}^\top L \mathbf{v} = \frac{1}{2} \sum_{i,j} K(\mathbf{x}_i, \mathbf{x}_j) (v_i - v_j)^2$. (b) Deduce that L is positive semidefinite whenever the affinities $K(\mathbf{x}_i, \mathbf{x}_j)$ are nonnegative, and that the all-ones vector $\mathbf{1}$ is always an eigenvector with eigenvalue 0. (c) Argue from the identity why the smallest nontrivial eigenvector varies slowly across strongly connected points and jumps across weakly connected ones, so that thresholding it separates the graph along its weakest cut. Hint

For (a), expand $\frac{1}{2} \sum_{i,j} K_{ij} (v_i - v_j)^2 = \frac{1}{2} \sum_{i,j} K_{ij} (v_i^2 - 2v_i v_j + v_j^2)$ and recognize the degree matrix in the pure-square sums and K in the cross term. For (c), a small value of $\mathbf{v}^\top L \mathbf{v}$ under a norm constraint forces $(v_i - v_j)^2$ to be small wherever K_{ij} is large, so tightly coupled points get nearly equal coordinates.

3. challenge The relaxed normalized cut led to the generalized eigenproblem $Lf = \lambda Df$ and to two normalized Laplacians, $L_{\text{sym}} = D^{-1/2} L D^{-1/2}$ and $L_{\text{rw}} = D^{-1} L$. Make the claim that they share a spectrum precise. (a) Show that f solves $Lf = \lambda Df$ if and only if $g = D^{1/2} f$ solves $L_{\text{sym}} g = \lambda g$, so the generalized pair (L, D) and the symmetric matrix L_{sym} have the same eigenvalues, with eigenvectors related by $D^{1/2}$. (b) Show that f solves $Lf = \lambda Df$ if and only if $L_{\text{rw}} f = \lambda f$, so L_{rw} has the same eigenvalues again, sharing the generalized eigenvectors directly. (c) Deduce that 0 is always an eigenvalue, with eigenvector $\mathbf{1}$ for L_{rw} and $D^{1/2} \mathbf{1}$ for L_{sym} , and that on a connected graph this eigenvalue is simple, so the Fiedler vector for the second-smallest eigenvalue is well defined. Hint

For (a) substitute $f = D^{-1/2} g$ into $Lf = \lambda Df$ and left-multiply by $D^{-1/2}$. For (b) left-multiply $Lf = \lambda Df$ by D^{-1} . For (c) use $L\mathbf{1} = 0$, and recall that $\mathbf{v}^\top L \mathbf{v} = \frac{1}{2} \sum_{i,j} w_{ij} (v_i - v_j)^2 = 0$ forces $\mathbf{v}$ to be constant on each connected component.

4. challenge A new point $\mathbf{x}$ is embedded by the Nystrom extension $\phi_m(\mathbf{x}) = \frac{1}{\lambda_m} \sum_{i=1}^n u_m(i) \tilde{k}(\mathbf{x}, \mathbf{x}_i)$, where $(\lambda_m, \mathbf{u}_m)$ is an eigenpair of the normalized affinity M with $[M]_{ji} = \tilde{k}(\mathbf{x}_j, \mathbf{x}_i)$. (a) Verify the consistency property: evaluated at a training point $\mathbf{x}_j$, the extension returns $\phi_m(\mathbf{x}_j) = u_m(j)$, so it agrees with the eigenvector it extends. (b) Recalling the projection of a feature vector onto a kernel principal component from kernel PCA, explain why the extension formula is, up to the factor $1/\lambda_m$, exactly that projection, so spectral embedding and kernel PCA compute one set of eigenfunctions. (c) Argue that the extension is trustworthy only when the query does not appreciably change the degrees and the normalization, and say what should be done when many new points arrive at once. Hint

For (a) write $\sum_i u_m(i) [M]_{ji} = (M \mathbf{u}_m)_j = \lambda_m u_m(j)$ and divide by λ_m . For (b) compare with the dual projection $\langle \phi(\mathbf{x}), \mathbf{v}_m \rangle = \frac{1}{\sqrt{\lambda_m}} \sum_i u_m(i) K(\mathbf{x}, \mathbf{x}_i)$; the two differ only by how the eigenvector is normalized. For

15 Kernel Clustering and Spectral Methods

(c) note that $\tilde{k}$ and the degrees are computed from the training set, so a query that would shift them is being measured against a stale graph.

16 Kernel CCA and Correlation Analysis

Often every object comes with two representations at once: an image and its caption, a gene's sequence and its expression profile, an audio clip and its transcript. Canonical correlation analysis looks for a direction in each view so that the two projected signals track each other as closely as possible, extracting the structure the two views share. This chapter kernelizes that search, meets an instructive failure along the way (without regularization the kernelized problem reports perfect correlation on any data), and shows how penalizing rough directions repairs it and ties correlation, covariance, and variance together as one eigen-decomposition family.

16.1 Kernel canonical correlation analysis

The last method in this chapter relates two views of the same objects rather than analyzing one. Suppose every object comes with two representations: an image and its caption, a gene's sequence and its expression profile, an audio clip and its transcript. Canonical correlation analysis (CCA) looks for a direction in each view such that the two projected signals are maximally correlated, extracting the shared structure between the views.

16.1.1 Classical CCA

Given two views collected as data matrices $X \in \mathbb{R}^{n \times p}$, whose rows are $\mathbf{x}_1^\top, \dots, \mathbf{x}_n^\top$, and $Y \in \mathbb{R}^{n \times d}$, whose rows are $\mathbf{y}_1^\top, \dots, \mathbf{y}_n^\top$, of the same n objects (so $\mathbf{x}_i \in \mathbb{R}^p$ and $\mathbf{y}_i \in \mathbb{R}^d$ are the two descriptions of object i), and assuming both views centered, CCA seeks directions $\mathbf{w}_a \in \mathbb{R}^p$ and $\mathbf{w}_b \in \mathbb{R}^d$ maximizing the empirical correlation of the projections $\mathbf{w}_a^\top \mathbf{x}_i$ and $\mathbf{w}_b^\top \mathbf{y}_i$:

$$\max_{\mathbf{w}_a \in \mathbb{R}^p, \mathbf{w}_b \in \mathbb{R}^d} \frac{\frac{1}{n} \sum_{i=1}^n \mathbf{w}_a^\top \mathbf{x}_i \mathbf{y}_i^\top \mathbf{w}_b}{\left(\frac{1}{n} \sum_{i=1}^n \mathbf{w}_a^\top \mathbf{x}_i \mathbf{x}_i^\top \mathbf{w}_a \right)^{1/2} \left(\frac{1}{n} \sum_{i=1}^n \mathbf{w}_b^\top \mathbf{y}_i \mathbf{y}_i^\top \mathbf{w}_b \right)^{1/2}}.$$

Interpreting the sums as sample covariance and variances, this is the population objective

$$\max_{\mathbf{w}_a, \mathbf{w}_b} \frac{\text{cov}(\mathbf{w}_a^\top X, \mathbf{w}_b^\top Y)}{\sqrt{\text{var}(\mathbf{w}_a^\top X)} \sqrt{\text{var}(\mathbf{w}_b^\top Y)}},$$

the correlation coefficient between the two one-dimensional projections. In matrix notation the objective is

$$\max_{\mathbf{w}_a, \mathbf{w}_b} \frac{\mathbf{w}_a^\top X^\top Y \mathbf{w}_b}{\left(\mathbf{w}_a^\top X^\top X \mathbf{w}_a \right)^{1/2} \left(\mathbf{w}_b^\top Y^\top Y \mathbf{w}_b \right)^{1/2}}.$$

This first optimum gives the first pair of canonical directions; subsequent pairs are found by solving the same problem under the constraint of orthogonality to the directions already found.

16.1.2 CCA is a generalized eigenvalue problem

The ratio is invariant to rescaling $\mathbf{w}_a$ and $\mathbf{w}_b$, so we may fix the scale by demanding unit projected variance and maximize only the numerator:

$$\max_{\mathbf{w}_a, \mathbf{w}_b} \mathbf{w}_a^\top X^\top Y \mathbf{w}_b \quad \text{s.t.} \quad \mathbf{w}_a^\top X^\top X \mathbf{w}_a = 1 \quad \text{and} \quad \mathbf{w}_b^\top Y^\top Y \mathbf{w}_b = 1.$$

Introducing multipliers λ_a, λ_b for the two constraints, the stationarity conditions of the Lagrangian

$$\min_{\mathbf{w}_a, \mathbf{w}_b} -\mathbf{w}_a^\top X^\top Y \mathbf{w}_b + \frac{\lambda_a}{2} (\mathbf{w}_a^\top X^\top X \mathbf{w}_a - 1) + \frac{\lambda_b}{2} (\mathbf{w}_b^\top Y^\top Y \mathbf{w}_b - 1)$$

are obtained by setting the gradients to zero:

$$-X^\top Y \mathbf{w}_b + \lambda_a X^\top X \mathbf{w}_a = 0, \quad -Y^\top X \mathbf{w}_a + \lambda_b Y^\top Y \mathbf{w}_b = 0.$$

Left-multiplying the first by $\mathbf{w}_a^\top$ and the second by $\mathbf{w}_b^\top$ and subtracting, the numerator terms cancel and the constraints give $\lambda_a \mathbf{w}_a^\top X^\top X \mathbf{w}_a = \lambda_b \mathbf{w}_b^\top Y^\top Y \mathbf{w}_b$, hence $\lambda_a = \lambda_b =: \lambda$. The two stationarity equations then assemble into a single generalized eigenvalue problem:

$$\begin{pmatrix} 0 & X^\top Y \\ Y^\top X & 0 \end{pmatrix} \begin{pmatrix} \mathbf{w}_a \\ \mathbf{w}_b \end{pmatrix} = \lambda \begin{pmatrix} X^\top X & 0 \\ 0 & Y^\top Y \end{pmatrix} \begin{pmatrix} \mathbf{w}_a \\ \mathbf{w}_b \end{pmatrix}.$$

Writing $\Sigma_A = \begin{pmatrix} 0 & X^\top Y \\ Y^\top X & 0 \end{pmatrix}$, $\Sigma_B = \begin{pmatrix} X^\top X & 0 \\ 0 & Y^\top Y \end{pmatrix}$, and $\mathbf{w} = \begin{pmatrix} \mathbf{w}_a \\ \mathbf{w}_b \end{pmatrix}$, this is $\Sigma_A \mathbf{w} = \lambda \Sigma_B \mathbf{w}$. If the block covariances are invertible, multiplying through and substituting $\mathbf{v} = \Sigma_B^{-1/2} \mathbf{w}$ turns it into the ordinary symmetric eigenproblem

$$\Sigma_B^{-1/2} \Sigma_A \Sigma_B^{-1/2} \mathbf{v} = \lambda \mathbf{v},$$

whose eigenvalues are the canonical correlations and whose eigenvectors, transformed back, give the canonical directions. The derivation follows Section 6.5 of Shawe-Taylor and Cristianini (2004).

16.1.3 Kernelizing CCA

Exactly as with PCA, we can replace each view by an RKHS embedding. Take two p.d. kernels $K_a, K_b : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$, with embeddings $\varphi_a : \mathcal{X} \rightarrow \mathcal{H}_a$ and $\varphi_b : \mathcal{X} \rightarrow \mathcal{H}_b$, giving two views $(\varphi_a(\mathbf{x}_i))_i$ and $(\varphi_b(\mathbf{x}_i))_i$ of the dataset. We look for directions $f_a \in \mathcal{H}_a$ and $f_b \in \mathcal{H}_b$ maximizing

$$\max_{f_a \in \mathcal{H}_a, f_b \in \mathcal{H}_b} \frac{\frac{1}{n} \sum_{i=1}^n \langle f_a, \varphi_a(\mathbf{x}_i) \rangle_{\mathcal{H}_a} \langle \varphi_b(\mathbf{x}_i), f_b \rangle_{\mathcal{H}_b}}{\left(\frac{1}{n} \sum_{i=1}^n \langle f_a, \varphi_a(\mathbf{x}_i) \rangle_{\mathcal{H}_a}^2 \right)^{1/2} \left(\frac{1}{n} \sum_{i=1}^n \langle \varphi_b(\mathbf{x}_i), f_b \rangle_{\mathcal{H}_b}^2 \right)^{1/2}},$$

and again the reproducing property replaces every inner product by a function evaluation, $\langle f_a, \varphi_a(\mathbf{x}_i) \rangle_{\mathcal{H}_a} = f_a(\mathbf{x}_i)$, giving

$$\max_{f_a \in \mathcal{H}_a, f_b \in \mathcal{H}_b} \frac{\frac{1}{n} \sum_{i=1}^n f_a(\mathbf{x}_i) f_b(\mathbf{x}_i)}{\left(\frac{1}{n} \sum_{i=1}^n f_a(\mathbf{x}_i)^2 \right)^{1/2} \left(\frac{1}{n} \sum_{i=1}^n f_b(\mathbf{x}_i)^2 \right)^{1/2}}.$$

Up to a few technical details (again left as an exercise), the representer theorem applies, and we seek solutions of the form $f_a(\cdot) = \sum_{i=1}^n \alpha_i K_a(\mathbf{x}_i, \cdot)$ and $f_b(\cdot) = \sum_{i=1}^n \beta_i K_b(\mathbf{x}_i, \cdot)$. Then $f_a(\mathbf{x}_i) = [K_a \alpha]_i$ and $f_b(\mathbf{x}_i) = [K_b \beta]_i$, and the objective becomes finite-dimensional:

$$\max_{\alpha, \beta \in \mathbb{R}^n} \frac{\alpha^\top K_a K_b \beta}{(\alpha^\top K_a^2 \alpha)^{1/2} (\beta^\top K_b^2 \beta)^{1/2}}.$$

Removing the scaling ambiguity as before, this is

$$\max_{\alpha, \beta \in \mathbb{R}^n} \alpha^\top K_a K_b \beta \quad \text{s.t.} \quad \alpha^\top K_a^2 \alpha = 1 \quad \text{and} \quad \beta^\top K_b^2 \beta = 1,$$

which, by the same Lagrangian argument as in the linear case, leads to a generalized eigenvalue problem. Subsequent canonical directions come from the same problem with added orthogonality constraints.

16.1.4 What goes wrong, and why we must regularize

There is a serious flaw hiding in the kernelized problem. Suppose K_a and K_b are invertible, and change variables to $\alpha' = K_a \alpha$ and $\beta' = K_b \beta$. The objective and constraints turn into

$$\max_{\alpha', \beta' \in \mathbb{R}^n} \alpha'^\top \beta' \quad \text{s.t.} \quad \alpha'^\top \alpha' = 1 \quad \text{and} \quad \beta'^\top \beta' = 1.$$

This is maximized, at value 1, by any $\alpha' = \beta'$ on the unit sphere. In other words the maximal correlation is always attainable and the solution is completely undetermined: the method reports perfect correlation between the two views no matter what data it is fed. This is not a numerical accident but the RKHS version of a familiar disease. In high or infinite dimension it is trivially easy to find directions that make any two finite samples look perfectly correlated, precisely the phenomenon of *spurious correlations* that populate collections like Tyler Vigen's gallery of nonsense coincidences. Unregularized kernel CCA overfits by construction.

It is worth saying plainly where the free lunch comes from. When K_a is invertible the map $\alpha \mapsto K_a \alpha$ is a bijection of $\mathbb{R}^n$, so the vector of projected values $(f_a(\mathbf{x}_1), \dots, f_a(\mathbf{x}_n))$ can be steered to any pattern we like by choosing α ; the same holds for the second view. Two arbitrary patterns can then be matched exactly, which is why the correlation saturates at its maximum of one. The kernel has handed each view as many adjustable directions as there are data points, and with that much freedom the two samples can always be fitted together, whether or not any real dependence links them. Regularization works by making some of those directions expensive: the added $\|f_a\|_{\mathcal{H}_a}^2$ term charges for wiggly, high-norm functions, so only shared structure smooth enough to be worth the price survives.

The failure has two faces. It is an overfitting problem, since the fit is perfect for reasons having nothing to do with genuine shared structure, and it is a numerical instability problem, since the derivation needed to invert the kernel matrices, which are typically ill-conditioned. Both are cured by the same remedy: regularize by preferring smooth directions, penalizing the RKHS norms $\|f_a\|_{\mathcal{H}_a}$ and $\|f_b\|_{\mathcal{H}_b}$. Concretely, recall $\|f_a\|_{\mathcal{H}_a}^2 = \alpha^\top K_a \alpha$, and replace the constraint $\alpha^\top K_a^2 \alpha = 1$ by the convex combination

$$(1 - \tau) \alpha^\top K_a^2 \alpha + \tau \underbrace{\alpha^\top K_a \alpha}_{\|f_a\|_{\mathcal{H}_a}^2} = 1,$$

and symmetrically for β with K_b . The parameter $\tau \in [0, 1]$ interpolates between the raw, degenerate problem at $\tau = 0$ and a pure norm penalty at $\tau = 1$. The added term makes the constraint matrices strictly positive definite, curing the numerical instability, and it forbids the wild, wiggly directions responsible for spurious correlations, curing the overfitting. Reading the penalty this way, as a preference for directions of small RKHS norm and

therefore for smooth, slowly varying functions, is what ties the fix back to the regularization that ran through the supervised chapters: there we penalized $\|f\|_{\mathcal{H}}$ to keep a fitted function from chasing noise, and here we do the same to keep a canonical direction from manufacturing correlation. The regularized problem is again a generalized eigenvalue problem, now well posed; this form of kernel CCA, together with its use as a kernel measure of dependence, is due to Bach and Jordan (2002).

Kernel CCA in this regularized form is a practical tool for learning shared representations across modalities. A representative application is finding a joint latent representation of images and their text tags, aligning the visual and textual views of a collection so that the two can be compared or retrieved against each other (Gong and Lazebnik, 2014).

16.1.5 Variance, covariance, and correlation as one family

It is worth stepping back to see that the unsupervised methods of this chapter are the same construction tuned by a single choice: what we ask of a pair of directions. Shawe-Taylor and Cristianini (2004) organize their whole treatment this way. Given one view, asking for the direction of maximal variance gives PCA, the eigenvectors of the covariance matrix. Given two paired views X and Y , we can instead ask for a pair of directions $(\mathbf{w}_a, \mathbf{w}_b)$ maximizing the raw covariance of the projections,

$$\max_{\|\mathbf{w}_a\|=\|\mathbf{w}_b\|=1} \mathbf{w}_a^\top C_{XY} \mathbf{w}_b, \quad C_{XY} = \frac{1}{n} X^\top Y,$$

whose solution is the leading pair of singular vectors of the cross-covariance matrix C_{XY} , the covariance itself being the top singular value, with subsequent pairs found by deflation. Or we can ask for the directions of maximal correlation, dividing that same covariance by the projected standard deviations, and we are back to CCA and its generalized eigenproblem. Variance is a self-covariance diagonalized, covariance is a cross-term factorized by singular value decomposition, and correlation is covariance normalized by variance; the eigen-decomposition machinery is identical, and only the matrix in the quadratic form changes.

This taxonomy also clarifies what regularization does to kernel CCA. The penalized constraint $(1 - \tau) \alpha^\top K_a^2 \alpha + \tau \alpha^\top K_a \alpha = 1$ does more than restore invertibility: as τ runs from 0 to 1 it slides the objective smoothly from pure correlation toward pure covariance (Shawe-Taylor and Cristianini, 2004). At $\tau = 0$ we have the degenerate correlation problem that reports a perfect match on any data; as τ grows, the projected-variance term is progressively replaced by a plain norm, and in the limit the method rewards directions of large covariance rather than large correlation, which is exactly the well-posed maximal-covariance problem above. The regularizer is thus not an ad hoc patch but a dial between two members of the same family, and a modest τ buys the numerical stability and the resistance to spurious correlation that pure correlation lacks.

16.2 Why regularization is mandatory

The last two sections diagnosed the degeneracy of unregularized kernel CCA and offered a dial, the parameter τ , for it. It is worth pinning down why that degeneracy is unavoidable rather than a corner case one might dodge with a lucky choice of kernel. The overfitting argument needed only one fact: that K_a and K_b are invertible, so that $\alpha \mapsto K_a \alpha$ is a bijection. Asking when that fact holds shows the trouble is forced by exactly the property that makes a kernel worth using.

16.2.1 Characteristic kernels force the saturation

The expressive kernels of the earlier chapters are the universal and characteristic ones: the Gaussian, the Laplace, and their relatives, whose RKHS is dense in the continuous functions and whose mean embedding separates probability measures. That defining richness casts a concrete finite-sample shadow. On any set of n distinct points a strictly positive definite kernel produces a Gram matrix of full rank n , hence invertible. So for precisely the kernels one reaches for, both K_a and K_b are invertible on any sample in general position, and the change of variables of the overfitting section applies verbatim.

Proposition (unregularized kernel CCA saturates)

Let K_a and K_b be strictly positive definite kernels, for instance any universal kernel such as the Gaussian, evaluated at n distinct points. Then the Gram matrices are invertible, and unregularized kernel CCA attains its maximum canonical correlation 1 on every such sample, whatever the pairing between the two views.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Strict positive definiteness makes K_a and K_b invertible, so $\alpha' = K_a \alpha$ and $\beta' = K_b \beta$ range over all of $\mathbb{R}^n$. Under this substitution the objective $\alpha^\top K_a K_b \beta$ with constraints $\alpha^\top K_a^2 \alpha = \beta^\top K_b^2 \beta = 1$ becomes $\max \alpha'^\top \beta'$ subject to $\|\alpha'\| = \|\beta'\| = 1$, whose value is 1 by Cauchy-Schwarz, attained at any $\alpha' = \beta'$ on the unit sphere. The pairing of the two views never entered the argument, so the reported correlation is 1 whether or not any real dependence links them. $\square$

This is the same disease that makes the regularized Fisher discriminant of Chapter 16, Kernel Discriminants and Projections replace its singular scatter matrix by $N + \mu I$: an expressive feature map hands each view as many free directions as there are points, and left unconstrained they will fit anything. Fukumizu, Bach, and Gretton (2007) make the statement asymptotic and precise. Working with the population cross-covariance operator between the two RKHSs, they show that the unregularized canonical correlation is degenerate, and that empirical kernel CCA is a consistent estimator of the true population dependence only when a regularizer κ_n is present and is sent to zero slowly as the sample grows. Regularization is therefore not a numerical convenience bolted on for conditioning; it is the condition under which kernel CCA estimates anything at all. Kernel CCA in this form was proposed by Lai and Fyfe (2000) and, as a kernel measure of dependence, by Bach and Jordan (2002), and given its standard overview by Haroon, Szedmak, and Shawe-Taylor (2004).

16.2.2 The shrinkage form of the regularized eigenproblem

We restored well-posedness earlier with the convex combination $(1 - \tau) \alpha^\top K_a^2 \alpha + \tau \alpha^\top K_a \alpha$. The parameterization used in most implementations, following Haroon, Szedmak, and Shawe-Taylor (2004), writes the same idea as a shrinkage of the kernel matrix itself: add κ to its diagonal before squaring, replacing K_a^2 by $(K_a + \kappa I)^2$. The two are the same penalty to first order, since

$$(K_a + \kappa I)^2 = K_a^2 + 2\kappa K_a + \kappa^2 I,$$

where the middle term $2\kappa K_a$ is exactly the RKHS-norm penalty $\alpha^\top K_a \alpha = \|f_a\|_{\mathcal{H}_a}^2$ that charges for wiggly directions, and the trailing $\kappa^2 I$ is a strict ridge that makes the constraint matrix positive definite even when K_a is only positive semidefinite. That last point matters for the non-universal kernels: a linear or low-degree polynomial kernel has rank $K_a < n$, so K_a^2 is singular and even the penalized matrix $K_a^2 + \tau K_a$ can stay singular, whereas $(K_a + \kappa I)^2$ is invertible for every $\kappa > 0$.

Definition (regularized kernel CCA)

For a shrinkage parameter $\kappa > 0$, regularized kernel CCA solves the generalized eigenvalue problem

$$\begin{pmatrix} 0 & K_a K_b \\ K_b K_a & 0 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \rho \begin{pmatrix} (K_a + \kappa I)^2 & 0 \\ 0 & (K_b + \kappa I)^2 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix},$$

whose largest eigenvalue $\rho \in [0, 1]$ is the leading canonical correlation and whose eigenvector supplies the dual weights (α, β) of the first canonical pair.

Because the right-hand matrix is block diagonal and positive definite, the substitution $\mathbf{u} = (K_a + \kappa I)\alpha$, $\mathbf{v} = (K_b + \kappa I)\beta$ reduces this to an ordinary symmetric eigenproblem exactly as in the linear derivation above: the canonical correlations are the eigenvalues of $(K_a + \kappa I)^{-1} K_a K_b (K_b + \kappa I)^{-1}$ and its transpose, so the shrinkage simply whitens each view through the well-conditioned $(K + \kappa I)^{-1}$ in place of the singular or near-singular K^{-1} . The consistency theory of Fukumizu, Bach, and Gretton (2007) is the statement that letting $\kappa = \kappa_n \rightarrow 0$ at a controlled rate recovers the population canonical correlations as $n \rightarrow \infty$; holding κ fixed trades a little bias for the stability that the next example makes numerical.

16.2.3 A worked example: saturation and its cure

A three-object sample makes both halves visible at once: the unregularized correlation is 1 by the proposition above, and the shrinkage dials it down to an honest, data-dependent value.

Example (kernel CCA on three paired points)

Three objects with scalar views $x^a = (-1, 0, 2)$ and $x^b = (0, 1, 1.5)$, each embedded with the Gaussian kernel $k(u, v) = e^{-(u-v)^2/2}$ (bandwidth $\sigma = 1$, universal). The Gram matrices are

$$K_a = \begin{pmatrix} 1 & 0.607 & 0.011 \\ 0.607 & 1 & 0.135 \\ 0.011 & 0.135 & 1 \end{pmatrix}, \quad K_b = \begin{pmatrix} 1 & 0.607 & 0.325 \\ 0.607 & 1 & 0.882 \\ 0.325 & 0.882 & 1 \end{pmatrix}.$$

1. Confirm both embeddings are full rank. The determinants $\det K_a = 0.6155$ and $\det K_b = 0.0955$ are nonzero, so K_a and K_b are invertible and the hypothesis of the saturation proposition holds.
2. Solve the unregularized eigenproblem. With the constraint matrix $\text{diag}(K_a^2, K_b^2)$ the top generalized eigenvalue is $\rho = 1.000$: the two views report a perfect canonical correlation, and the same 1 would appear for any other pairing of the six numbers.
3. Apply the $(K + \kappa I)$ shrinkage. Replacing each K^2 by $(K + \kappa I)^2$ and solving the regularized eigenproblem gives a leading correlation that falls monotonically as the shrinkage grows: $\rho = 0.896$ at $\kappa = 0.1$, then $\rho = 0.611$ at $\kappa = 0.5$, and $\rho = 0.413$ at $\kappa = 1$.
4. Read off the canonical directions. At $\kappa = 1$ the leading dual weights, normalized to unit length, are $\alpha = (0.519, 0.635, 0.572)$ and $\beta = (0.632, 0.627, 0.456)$, giving one concrete pair of canonical functions $f_a = \sum_i \alpha_i K_a(x_i^a, \cdot)$ and $f_b = \sum_i \beta_i K_b(x_i^b, \cdot)$.

Reading. The unregularized 1.000 is an artifact of an invertible, expressive kernel and carries no information about the data. The shrinkage turns κ into a dial that trades that illusory perfect fit for a genuine, data-dependent correlation, which is exactly the $(K + \kappa I)$ ridge whose vanishing rate the consistency theory prescribes.

Verification artifact. checks/example-ch-cca-example-15-1.json records the example source hash and verification scope.

16.3 Multi-view learning and deep CCA

Kernel CCA is one point in a wider space of multi-view methods, all of which read shared structure from paired data and differ only in how each view is represented before the correlations are taken. Linear CCA maps each view by a matrix, kernel CCA maps it by a fixed RKHS feature map, and one can instead learn the maps outright.

Method	View map	What is learned	Scaling in n
Linear CCA	linear $\mathbf{x} \mapsto W\mathbf{x}$	the two projection matrices	cheap, dimension bound
Kernel CCA	fixed feature map $\varphi(\mathbf{x})$	the dual weights α, β	$O(n^3)$ eigenproblem
Deep CCA	neural network $f_\theta(\mathbf{x})$	the network weights θ, ψ	minibatch, $O(n)$

Deep CCA (Andrew, Arora, Bilmes, and Livescu, 2013) keeps the correlation objective but replaces the two fixed feature maps by trainable neural networks f_θ and g_ψ , one per view. The networks are trained to maximize the total canonical correlation of their outputs, the sum of the top singular values of the normalized cross-covariance $\hat{\Sigma}_{ff}^{-1/2} \hat{\Sigma}_{fg} \hat{\Sigma}_{gg}^{-1/2}$, and the gradient of that spectral objective is backpropagated through both networks. This buys the nonlinearity of kernel CCA while escaping the $O(n^3)$ Gram-matrix eigenproblem, since the networks are optimized on minibatches rather than on the full sample. Crucially, the whitening matrices $\hat{\Sigma}_{ff}$ and $\hat{\Sigma}_{gg}$ are inverted with the very same $(\hat{\Sigma} + rI)$ shrinkage of the previous section: the expressive-map degeneracy does not vanish when the map is learned rather than fixed, so deep CCA regularizes its covariances for exactly the reason kernel CCA regularizes its Gram matrices.

The setting that has driven these methods most is a paired corpus. In the cross-lingual case each object is a document present in two languages, the two views are its representations under the per-language kernels of Chapter 23, Kernels for Text, and the leading canonical directions span a shared, language-independent semantic space in which a query in one language can be matched against candidates in the other; the same construction induces bilingual word and document embeddings from parallel text. The image-and-tags alignment mentioned earlier (Gong and Lazebnik, 2014) is the vision analogue, with one view a visual kernel and the other a text kernel over the tags. In every case the family relationship to the single-view spectral methods of Chapter 13, Kernel PCA and Denoising is the organizing thread: kernel PCA diagonalizes one view’s covariance, kernel CCA factorizes the cross-covariance of two, and deep CCA does the latter with learned features, but all three read geometry out of a spectral decomposition.

Further reading

For a book-length treatment of the methods in this chapter, Schölkopf and Smola (2002), *Learning with Kernels*, develops kernel PCA in its chapter 14 and takes up the related kernel feature-extraction methods in the neighbouring chapters, with the pre-image problem and kernel-PCA denoising worked out in detail. Shawe-Taylor and Cristianini (2004), *Kernel Methods for Pattern Analysis*, covers the same territory from the pattern-analysis viewpoint: its chapters 5 and 6 treat the spectral methods, kernel PCA, and canonical correlation analysis, and Section 6.5 is the source of the generalized-eigenvalue derivation of CCA used above. Both books also situate these unsupervised methods within the wider representer-theorem program that organizes these chapters.

16.4 Common mistakes and practical implications

For **Kernel CCA and Correlation Analysis**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a

non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

16.5 Summary and further reading

This chapter established explain the central definitions and claims in Kernel CCA and Correlation Analysis; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Andrew and Livescu 2013), (Bach and Jordan 2002), (Fukumizu and Gretton 2007).

16.6 Exercises

1. proof Show why unregularized kernel CCA saturates at correlation 1. Assume K_a and K_b are invertible and substitute $\alpha' = K_a \alpha$, $\beta' = K_b \beta$ into the finite-dimensional objective $\alpha'^T K_a K_b \beta$ with constraints $\alpha'^T K_a^2 \alpha = 1$ and $\beta'^T K_b^2 \beta = 1$. Show the problem becomes $\max \alpha'^T \beta'$ subject to $\|\alpha'\| = \|\beta'\| = 1$, whose maximum is 1, attained by any $\alpha' = \beta'$ on the unit sphere. Interpret the result: the map $\alpha \mapsto K_a \alpha$ being a bijection means the projected values $(f_a(\mathbf{x}_1), \dots, f_a(\mathbf{x}_n))$ can be steered to any pattern, so any two samples can be matched exactly. Then explain how replacing the constraint by $(1 - \tau) \alpha'^T K_a^2 \alpha + \tau \alpha'^T K_a \alpha = 1$ breaks the degeneracy. Hint

The Cauchy-Schwarz inequality bounds $\alpha'^T \beta' \leq \|\alpha'\| \|\beta'\| = 1$, with equality exactly when the two unit vectors coincide. For the regularization step, note that $\alpha'^T K_a \alpha = \|f_a\|_{\mathcal{H}_a}^2$ is the RKHS norm of the direction: adding $\tau > 0$ of it makes the constraint matrix strictly positive definite and charges for high-norm, wiggly directions, so the change of variable no longer decouples the problem.

2. challenge The chapter argues that variance, covariance, and correlation are one eigen-decomposition family tuned by a single choice. Fill in the family for a pair of centered views X, Y : (a) which matrix's eigenvectors solve the single-view maximal-variance problem (PCA); (b) which matrix's singular vectors solve the maximal-covariance problem $\max_{\|\mathbf{w}_a\|=\|\mathbf{w}_b\|=1} \mathbf{w}_a^T C_{XY} \mathbf{w}_b$; and (c) which generalized eigenproblem solves maximal correlation (CCA). Then explain, using the regularized constraint $(1 - \tau) \alpha'^T K_a^2 \alpha + \tau \alpha'^T K_a \alpha = 1$, why sending τ from 0 to 1 slides kernel CCA from pure correlation toward pure covariance, and why a modest τ is what buys resistance to spurious correlation.
3. computation Expand the shrinkage constraint matrix $(K + \kappa I)^2$ and name its three terms: the raw projected-variance matrix K^2 , the RKHS-norm penalty, and the strict ridge. Then take the linear kernel $k(u, v) = uv$ on the one-dimensional points $u \in \{1, 2, 3\}$, whose Gram matrix K has rank 1. Check that K^2 and the penalized matrix $K^2 + \tau K$ both remain singular, while $(K + \kappa I)^2$ is invertible for every $\kappa > 0$. Conclude why the shrinkage form is the safe default once the kernel is not universal. Hint

The rank of K^2 equals the rank of K , and adding τK cannot raise it because τK shares the null space of K ; only the $\kappa^2 I$ term fills that null space, lifting every eigenvalue to at least κ^2 .

4. challenge Reproduce the worked example's shrinkage sweep and confirm the leading canonical correlation $\rho(\kappa)$ falls monotonically from 1 as κ grows (the check script prints 1.000, 0.896, 0.611, 0.413 at $\kappa = 0, 0.1, 0.5, 1$). Then prove monotonicity in general. Writing the top correlation as the maximum of $\alpha'^T K_a K_b \beta$ over $\sqrt{\alpha'^T (K_a + \kappa I)^2 \alpha} \sqrt{\beta'^T (K_b + \kappa I)^2 \beta} = 1$, argue that enlarging the constraint matrices from K^2 to $(K + \kappa I)^2$ can only shrink the attainable maximum, so $\rho(\kappa)$ is nonincreasing. Hint

For a positive semidefinite K and $\kappa > 0$, $(K + \kappa I)^2 - K^2 = 2\kappa K + \kappa^2 I \geq 0$, so each denominator quadratic form only grows with κ while the numerator is untouched: every feasible ratio is bounded by its value at smaller κ .

5. warm-up Place kernel CCA in the multi-view family. (a) State precisely what deep CCA (Andrew et al., 2013) replaces relative to kernel CCA, and explain why it still needs the covariance shrinkage ($\hat{\Sigma} + rI$) of this chapter. (b) Describe the cross-lingual paired-corpus application: what are the two views, what does a leading canonical direction represent, and how is it used for retrieval across languages? (c) Relate the objective to kernel PCA of Chapter 13, Kernel PCA and Denoising: which covariance or cross-covariance does each of PCA, CCA, and deep CCA decompose?

17 Kernel Discriminants and Projections

The subspace methods of kernel PCA and kernel CCA read structure off the data without ever looking at a label. This chapter turns the same Gram matrix to supervised ends. When a label or a target is available, the question is no longer where the data spreads out the most, but which directions in feature space separate the classes or predict the response. We build three answers, all from one algebraic engine. Kernel Fisher discriminant analysis maximizes a ratio of between-class to within-class scatter and lands on a regularized generalized eigenproblem. Principal components regression keeps the eigenvectors of the kernel matrix but reweights them by how well each predicts the target. Kernel partial least squares abandons variance altogether and greedily extracts the directions of maximum covariance with the response, deflating the Gram matrix after each. Seen together, they are the supervised face of a single idea: optimize a Rayleigh quotient, or iterate a deflation, on the kernel matrix.

17.1 From variance to prediction

Unsupervised subspace analysis chooses directions by an internal criterion. Principal components analysis picks the directions of largest variance; it never asks whether those directions have anything to do with a label. That is often exactly the wrong instinct for a supervised task. As Shawe-Taylor and Cristianini (2004) put it plainly, what matters for prediction is not the size of the variance of the data but how well a direction can be used to predict the output, and it can happen that the high-variance directions found by PCA are uncorrelated with the target while a direction of relatively low variance carries all the predictive signal. A supervised subspace method must therefore let the label into the objective.

Remarkably, doing so does not change the shape of the computation. Almost every method in this chapter reduces to one of two canonical operations on symmetric matrices built from the Gram matrix: solving a generalized eigenvalue problem $Aw = \lambda Bw$, or iterating a deflation that strips off one direction and repeats. Fisher discriminant analysis is a generalized eigenproblem where A encodes class separation and B encodes within-class spread. Partial least squares is a deflation loop where the extracted direction maximizes covariance with the target. Principal components regression sits between them, an eigenproblem in the inputs followed by a target-driven reweighting. Because each operation touches the data only through inner products, all three pass into a kernel-defined feature space through the same dual representation used for kernel PCA. We begin with the shared engine.

17.2 The generalized eigenvalue problem and deflation

A Rayleigh quotient is a ratio of two quadratic forms whose maximizer is an eigenvector. We met the simplest version in PCA, where maximizing $w^T C w$ subject to $\|w\| = 1$ is the same as maximizing $\rho(w) = w^T C w / w^T w$, solved by the top eigenvector of the covariance C . The supervised methods below all optimize a two-matrix generalization.

Definition (generalized Rayleigh quotient)

Given symmetric matrices A and B with B positive definite, the generalized Rayleigh quotient is

$$\rho(w) = \frac{w^T A w}{w^T B w}.$$

Its stationary points are the solutions of the generalized eigenvalue problem $Aw = \lambda Bw$, and the maximum of ρ is the largest generalized eigenvalue λ_1 , attained at the corresponding eigenvector.

The claim is proved by turning the generalized problem into an ordinary one. Because B is positive definite it has a symmetric positive definite square root $B^{1/2}$ with $B^{1/2}B^{1/2} = B$, and $B^{1/2}$ is a bijection of $\mathbb{R}^\ell$. Substitute $v = B^{1/2}w$.

Reduction to a symmetric eigenproblem

The generalized problem $Aw = \lambda Bw$ is equivalent, under $w = B^{-1/2}v$, to the ordinary symmetric eigenvalue problem

$$B^{-1/2}AB^{-1/2}v = \lambda v,$$

and the quotient transforms as

$$\rho(w) = \frac{w^\top Aw}{w^\top Bw} = \frac{(B^{1/2}w)^\top B^{-1/2}AB^{-1/2}(B^{1/2}w)}{\|B^{1/2}w\|^2},$$

which is the ordinary Rayleigh quotient of the symmetric matrix $B^{-1/2}AB^{-1/2}$ in the variable v .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Premultiply $Aw = \lambda Bw$ by $B^{-1/2}$ and insert $B^{-1/2}B^{1/2} = I$ before w : $B^{-1/2}AB^{-1/2}(B^{1/2}w) = \lambda B^{1/2}w$, which is $B^{-1/2}AB^{-1/2}v = \lambda v$. The matrix $B^{-1/2}AB^{-1/2}$ is symmetric because A and $B^{-1/2}$ are, so its eigenvectors $v_1, \dots, v_\ell$ are orthonormal and its eigenvalues real. The quotient identity is a direct substitution. Since $B^{1/2}$ is a bijection, maximizing $\rho(w)$ over w is the same as maximizing the ordinary quotient over v , whose maximum is λ_1 at v_1 ; hence the top generalized eigenvector is $w_1 = B^{-1/2}v_1$ (Shawe-Taylor and Cristianini 2004). $\square$

The eigenvectors of the generalized problem are not orthogonal in the usual sense, but in the geometry set by A and B they are. This conjugacy is what lets us extract several directions without their interfering.

Generalized orthogonality

If the generalized eigenvalues are distinct, the eigenvectors w_i of $Aw = \lambda Bw$ satisfy

$$w_i^\top Bw_j = \delta_{ij}, \quad w_i^\top Aw_j = \delta_{ij}\lambda_i,$$

that is, they are orthonormal in the B -metric and orthogonal in the A -metric.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Once a leading eigenpair (λ, w) is found, the next is obtained by removing that direction and repeating. For the two-matrix problem the correct removal is a generalized deflation that respects the B -geometry.

Generalized deflation

After finding a nonzero generalized eigenpair λ, w of $Aw = \lambda Bw$, deflate the numerator matrix by

$$A \longleftarrow A - \lambda Bw(Bw)^\top,$$

leaving B unchanged. The deflated problem has the same remaining eigenpairs with the found direction removed.

Two facts about B drive everything that follows. First, if B is only positive semidefinite, its square root is not invertible and the reduction above breaks down: the quotient can be made arbitrarily large along the null space of B , so the problem is ill posed. This is exactly what happens in feature space, where the within-class scatter is singular, and it forces the regularization $B \leftarrow B + \mu I$ that appears in every algorithm below. Second, when A has rank one, the reduced matrix $B^{-1/2}AB^{-1/2}$ has a single nonzero eigenvalue, so only one eigenvector carries information and it can be read off by a single linear solve rather than a full eigendecomposition. Fisher discriminant analysis is precisely this rank-one case.

17.3 Kernel Fisher discriminant analysis

Fisher (1936) asked for the one-dimensional projection of a two-class dataset that best separates the classes. Good separation means two things at once: the projected class means should be far apart, and each class should be tightly concentrated around its own mean. Fisher captured both in a single ratio, and Mika et al. (1999) carried the construction into a kernel feature space, where the projection becomes nonlinear in the input.

17.3.1 Between-class and within-class scatter

Work in the feature space with map ϕ . Write the two classes as index sets of sizes ℓ_+ and ℓ_- , and let the class means in feature space be

$$\mu_c^\phi = \frac{1}{\ell_c} \sum_{i \in c} \phi(x_i), \quad c \in \{+, -\}.$$

A projection direction w sends a point $\phi(x)$ to the scalar $w^\top \phi(x)$. The projected class means are $w^\top \mu_c^\phi$, and the between-class scatter is the squared gap

$$(w^\top \mu_+^\phi - w^\top \mu_-^\phi)^2 = w^\top S_B w, \quad S_B = (\mu_+^\phi - \mu_-^\phi)(\mu_+^\phi - \mu_-^\phi)^\top.$$

The within-class scatter is the total spread of each class about its own projected mean,

$$w^\top S_W w, \quad S_W = \sum_{c \in \{+, -\}} \sum_{i \in c} (\phi(x_i) - \mu_c^\phi)(\phi(x_i) - \mu_c^\phi)^\top.$$

Fisher's criterion is the ratio of the two, a generalized Rayleigh quotient in w ,

$$J(w) = \frac{w^\top S_B w}{w^\top S_W w}.$$

Maximizing J is the generalized eigenproblem $S_B w = \lambda S_W w$. Because S_B has rank one, its range is spanned by $\mu_+^\phi - \mu_-^\phi$, and the optimal direction is $w \propto S_W^{-1}(\mu_+^\phi - \mu_-^\phi)$: a single linear solve, not a full eigendecomposition.

17.3.2 The dual, and why regularization is unavoidable

In a high-dimensional feature space we can neither form $\phi(x_i)$ nor S_W explicitly, so we pass to the dual. Any useful direction lies in the span of the data, $w = \sum_{i=1}^{\ell} \alpha_i \phi(x_i)$, and every quantity in J then depends on ϕ only through the kernel. Projecting the direction onto a class mean gives

$$w^\top \mu_c^\phi = \sum_{i=1}^{\ell} \alpha_i \frac{1}{\ell_c} \sum_{j \in c} \kappa(x_i, x_j) = \alpha^\top m_c, \quad (m_c)_i = \frac{1}{\ell_c} \sum_{j \in c} K_{ij},$$

so the between-class term becomes $w^\top S_B w = \alpha^\top M \alpha$ with $M = (m_+ - m_-)(m_+ - m_-)^\top$. A parallel computation writes the within-class term as $\alpha^\top N \alpha$ with

$$N = \sum_{c \in \{+, -\}} K_c \left(I_{\ell_c} - \frac{1}{\ell_c} \mathbf{1}\mathbf{1}^\top \right) K_c^\top,$$

where K_c is the $\ell \times \ell_c$ block of columns of the Gram matrix belonging to class c , and $I_{\ell_c} - \frac{1}{\ell_c} \mathbf{1}\mathbf{1}^\top$ is the centering projector for that class. The kernel Fisher objective is thus

$$J(\alpha) = \frac{\alpha^\top M \alpha}{\alpha^\top N \alpha}.$$

Now the singularity bites. The matrix N is built from ℓ feature vectors centered within their classes, so it is at most of rank $\ell - 2$; in a feature space of dimension larger than the sample it is always singular. Its null space contains directions along which $\alpha^\top N \alpha = 0$ while $\alpha^\top M \alpha > 0$, which would send J to infinity and pick out a meaningless direction that fits the training split perfectly and generalizes not at all. The cure, exactly the regularization of the generalized eigenproblem above, is to add a multiple of the identity, replacing N by $N + \mu I$. This penalizes the RKHS norm $\|w\|^2 = \alpha^\top K \alpha$ of the direction (a ridge on α in practice), restores invertibility, and stabilizes the solution. Since M is rank one, the maximizer is the single linear solve

$$\alpha \propto (N + \mu I)^{-1} (m_+ - m_-).$$

This is the kernel Fisher discriminant of Mika et al. (1999); the equivalent generalized discriminant analysis of Baudat and Anouar (2000) casts the same computation as an eigendecomposition of a kernel scatter matrix and extends it to more than two classes.

Algorithm (kernel Fisher discriminant)

Input Gram matrix $K \in \mathbb{R}^{\ell \times \ell}$, class index sets $+, -$ of sizes ℓ_+, ℓ_- , regularization $\mu > 0$.

Output dual direction α and threshold b ; projection $t(x) = \sum_i \alpha_i \kappa(x_i, x)$.

1. Form the class-mean vectors $(m_c)_i = \frac{1}{\ell_c} \sum_{j \in c} K_{ij}$ for $c \in \{+, -\}$.
2. Form the within-class matrix $N = \sum_c K_c (I_{\ell_c} - \frac{1}{\ell_c} \mathbf{1}\mathbf{1}^\top) K_c^\top$.
3. Solve the regularized system $(N + \mu I) \alpha = (m_+ - m_-)$; since the between-class matrix M has rank one, this single solve is the top generalized eigenvector.
4. Set the threshold midway between the projected class means, $b = \frac{1}{2} \alpha^\top (m_+ + m_-)$, and classify x by the sign of $t(x) - b$.

The same discriminant appears in the correlation chapter as a generalized eigenproblem in the input covariances. Writing y for the $\pm$ label vector and X for the (feature) data matrix, Shawe-Taylor and Cristianini (2004) show the regularized Fisher discriminant maximizes $w^\top E w / w^\top F w$ with $E = X^\top y y^\top X$ and $F = \lambda I + \frac{\ell}{2\ell_+ \ell_-} X^\top B X$; the numerator matrix $E = (X^\top y)(y^\top X)$ has rank one because $X^\top y$ is a single column, which is again why only the first eigenvector matters and why the solution reduces to a matrix inversion. The kernel form above is the dual of this statement.

Example (kernel Fisher on four points)

Two points per class in $\mathbb{R}^2$ with the linear kernel $\kappa(x, x') = \langle x, x' \rangle$: class + is $x_1 = (1, 0)$, $x_2 = (0, 1)$; class – is $x_3 = (-1, 0)$, $x_4 = (0, -1)$. Regularization $\mu = 1$. The Gram matrix is

$$K = \begin{pmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \\ -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{pmatrix}.$$

1. Average kernel columns within each class. The class-mean vectors are $m_+ = (0.5, 0.5, -0.5, -0.5)$ and $m_- = (-0.5, -0.5, 0.5, 0.5)$, so the between-class direction is $m_+ - m_- = (1, 1, -1, -1)$.
2. Assemble the within-class matrix. Centering each class block and summing gives

$$N = \begin{pmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix},$$

which is singular (rank one), confirming that without regularization the quotient is ill posed.

3. Solve the regularized system. Solving $(N + I)\alpha = m_+ - m_-$ yields $\alpha = (1, 1, -1, -1)$, that is $\alpha = (0.5, 0.5, -0.5, -0.5)$ after normalization to unit length.
4. Project the training points. The projections $t = K\alpha = (1, 1, -1, -1)$ place both + points at +1 and both – points at –1; the objective value is $\alpha^\top M\alpha / \alpha^\top (N + I)\alpha = 4$.

Reading. The discriminant sends the two classes to cleanly separated points ± 1 with the threshold at 0, and the whole computation used only the Gram matrix. The singular N makes concrete why the $+\mu I$ term is not a numerical convenience but a necessity.

Verification artifact. checks/example-ch-discriminant-example-16-1.json records the example source hash and verification scope.

17.4 Principal components regression

Suppose the target is real-valued and we want to regress. A natural pipeline is to run kernel PCA first, project the data onto its leading principal directions, and fit a linear regressor there. This is principal components regression (PCR), and its motivation is denoising: by discarding the low-variance directions, presumed to be noise, we reduce the variance of the regression estimate. Shawe-Taylor and Cristianini (2004) also flag its danger, the same warning that opened the chapter: PCA orders directions by variance, not by relevance to the target, so a low-variance direction that happens to predict the response well can be thrown away before the regression ever sees it.

The dual form is elegant. Let $K = \sum_j \lambda_j v_j v_j^\top$ be the eigendecomposition of the (centered) kernel matrix, eigenvalues in descending order. Keeping the first k components, the dual regression coefficients are a target-weighted sum of the leading eigenvectors,

$$\alpha = \sum_{j=1}^k \frac{1}{\lambda_j} (v_j^\top y) v_j,$$

and the fitted function is $f(x) = \sum_i \alpha_i \kappa(x_i, x)$. Each eigenvector contributes in proportion to its covariance $v_j^\top y$ with the target, tempered by the inverse eigenvalue $1/\lambda_j$. The construction is incremental: adding a component appends one term to α without disturbing the earlier ones, because the eigenvectors are orthogonal.

Algorithm (dual principal components regression)

Input centered Gram matrix $K \in \mathbb{R}^{\ell \times \ell}$, target vector $y \in \mathbb{R}^\ell$, number of components k .

Output dual coefficients α ; predictor $f(x) = \sum_i \alpha_i \kappa(x_i, x)$.

1. Eigendecompose $K = \sum_j \lambda_j v_j v_j^\top$ with $\lambda_1 \geq \lambda_2 \geq \dots$.
2. For each retained component compute the target covariance $v_j^\top y$.
3. Repeat for each component $j = 1, \dots, k$: accumulate $\alpha \leftarrow \alpha + \frac{1}{\lambda_j} (v_j^\top y) v_j$.
4. Predict new points via $f(x) = \sum_i \alpha_i \kappa(x_i, x)$, centering x against the training kernel as in kernel PCA.

Example (PCR and a low-variance predictive direction)

Three points in $\mathbb{R}^2$, linear kernel, targets $y = (1, -1, 0)$: $x_1 = (2, 0)$, $x_2 = (0, 1)$, $x_3 = (-2, -1)$. The Gram matrix is already centered,

$$K = \begin{pmatrix} 4 & 0 & -4 \\ 0 & 1 & -1 \\ -4 & -1 & 5 \end{pmatrix}, \quad y = (1, -1, 0).$$

1. Eigendecompose the centered kernel. The nonzero eigenvalues are $\lambda_1 = 5 + \sqrt{13} \approx 8.6056$ and $\lambda_2 = 5 - \sqrt{13} \approx 1.3944$, with a zero third eigenvalue (three centered points span a plane).
2. Measure covariance of each direction with the target. The projections are $v_1^\top y \approx -0.5537$ and $v_2^\top y \approx -1.3013$: the second, low-variance direction is the one more aligned with y .
3. Fit with only the top component. With $k = 1$, $\alpha \approx (0.0420, 0.0064, -0.0483)$ and the fitted values $K\alpha \approx (0.361, 0.055, -0.416)$ are far from y .
4. Add the second component. With $k = 2$, $\alpha = (0.5, -0.75, 0.25)$ and $K\alpha = (1, -1, 0)$ recovers the target exactly.

Reading. The predictive direction here is the one of smaller variance. Truncating PCR at the single leading component discards precisely the direction that carries the signal; only keeping the second eigenvector recovers the response. This is the concrete failure mode that motivates partial least squares.

Verification artifact. checks/example-ch-discriminant-example-16-2.json records the example source hash and verification scope.

17.5 Kernel partial least squares

Partial least squares fixes PCR's blind spot by choosing directions using the target from the start. Instead of the directions of maximum variance, it extracts the directions of maximum covariance with the response, then deflates and repeats. It originated in the chemometrics work of Wold (1975), where high-dimensional and highly correlated predictors are the norm, and it was kernelized by Rosipal and Trejo (2001). Because covariance couples inputs to outputs, PLS aligns its features with what actually predicts y .

17.5.1 Directions of maximum covariance

The building block is the direction pair that maximizes covariance between two views of the data. For paired data with cross-covariance C_{xy} , the problem

$$\max_{w_x, w_y} \frac{w_x^\top C_{xy} w_y}{\|w_x\| \|w_y\|}$$

is solved by the first singular vectors of $C_{xy} = U\Sigma V^\top$: $w_x = u_1$, $w_y = v_1$, with the covariance equal to the largest singular value σ_1 (Shawe-Taylor and Cristianini 2004). Unlike the Rayleigh quotient, C_{xy} is neither square nor symmetric, and we optimize over two vectors; but the mechanism to extract more than one direction is again deflation. After taking the leading pair, project both views onto the orthogonal complement,

$$X \leftarrow X(I - u_1 u_1^\top), \quad Y \leftarrow Y(I - v_1 v_1^\top),$$

and recompute. When the second view is the target, this is the engine of PLS regression.

17.5.2 Primal PLS and its deflation

Primal PLS runs the covariance step against the target and deflates the inputs. At stage j it takes u_j , the first singular vector of $X_j^\top Y$ (found by a short power iteration $u \leftarrow X_j^\top Y Y^\top X_j u$), forms the score $\tau_j = X_j u_j$, and then deflates. The key object is the loading

$$p_j = \frac{X_j^\top X_j u_j}{u_j^\top X_j^\top X_j u_j},$$

which is the direction the score τ_j explains in the input space. Deflation removes exactly that,

$$X_{j+1} = X_j(I - u_j p_j^\top).$$

This choice is what makes the extracted scores conjugate. One checks that $u_i^\top p_j = 0$ for $i < j$ and $u_j^\top p_j = 1$, so the successive scores $X_j u_j$ are mutually orthogonal, which in turn lets the regression coefficients be computed one component at a time without interaction (Shawe-Taylor and Cristianini 2004). Because PLS deflates only X and removes only the covariance already explained, deflating Y as well changes nothing, and the restriction $k \leq m$ that limits maximum-covariance features does not apply: PLS can extract as many components as it likes.

Example (one PLS deflation step)

Centered inputs and target,

$$X = \begin{pmatrix} 1 & 2 \\ 1 & -1 \\ -2 & -1 \end{pmatrix}, \quad y = (1, -1, 0),$$

with columns of X and entries of y each summing to zero.

1. Find the first covariance direction. $X^\top y = (0, 3)$, so the unit direction is $u_1 = (0, 1)$; the score is $\tau_1 = X u_1 = (2, -1, -1)$.
2. Form the loading. With $X^\top X = \begin{pmatrix} 6 & 3 \\ 3 & 6 \end{pmatrix}$ and $u_1^\top X^\top X u_1 = 6$, the loading is $p_1 = X^\top X u_1 / 6 = (0.5, 1)$, which differs from u_1 .

3. Read off the regression weight. $c_1 = y^\top Xu_1/6 = 0.5$, so this component's fitted contribution is $\tau_1 c_1 = (1, -0.5, -0.5)$.
4. Deflate the inputs. $X_2 = X(I - u_1 p_1^\top) = \begin{pmatrix} 0 & 0 \\ 1.5 & 0 \\ -1.5 & 0 \end{pmatrix}$; the check $X_2^\top \tau_1 = (0, 0)$ confirms the new residual is orthogonal to the extracted score.

Reading. One step selects the direction of maximum covariance with y , fits a regression weight along it, and subtracts what it explained. The residual X_2 is orthogonal to the score just used, so the next component is extracted from a genuinely new subspace: this orthogonality is what makes the deflation loop terminate cleanly and the coefficients accumulate independently.

Verification artifact. checks/example-ch-discriminant-example-16-3.json records the example source hash and verification scope.

17.5.3 The dual and kernel PLS

Everything above touches X only through inner products, so PLS kernelizes. In the dual we never form u_j ; we track the score τ_j directly. A power iteration on the kernel matrix, $\beta \leftarrow K_j Y_j Y_j^\top \beta$ (normalized), converges to a dual vector β_j , from which $\tau_j = K_j \beta_j$. The input deflation $X_{j+1} = (I - \tau_j \tau_j^\top / \tau_j^\top \tau_j) X_j$ becomes a two-sided deflation of the Gram matrix,

$$K_{j+1} = \left(I - \frac{\tau_j \tau_j^\top}{\tau_j^\top \tau_j}\right) K_j \left(I - \frac{\tau_j \tau_j^\top}{\tau_j^\top \tau_j}\right),$$

computable with no explicit feature vectors (Rosipal and Trejo 2001). After k steps, collecting the dual vectors as columns of B , the scores as columns of T , and the targets in Y , the dual regression coefficients are

$$\alpha = B(T^\top KB)^{-1} T^\top Y, \quad f_r(x) = \sum_{i=1}^{\ell} \alpha_i^r \kappa(x_i, x),$$

where the matrix $T^\top KB$ is upper triangular, so the inverse is cheap. This combines the two advantages the earlier methods split between them: like PLS it aligns features with the target through covariance, and like PCR it is not limited to $k \leq m$ components.

Algorithm (kernel PLS)

Input centered kernel matrix $K \in \mathbb{R}^{\ell \times \ell}$, target matrix $Y \in \mathbb{R}^{\ell \times m}$, number of components k .

Output dual regression coefficients α ; predictor $f_r(x) = \sum_i \alpha_i^r \kappa(x_i, x)$.

1. Initialize $K_1 = K$, $Y_1 = Y$.
2. Repeat for each component $j = 1, \dots, k$:
3. power-iterate $\beta \leftarrow K_j Y_j Y_j^\top \beta$, renormalizing, until β_j converges;
4. form the score $\tau_j = K_j \beta_j$ and the output weight $c_j = Y_j^\top \tau_j / (\tau_j^\top \tau_j)$;
5. deflate $K_{j+1} = (I - P_j) K_j (I - P_j)$ and $Y_{j+1} = Y_j - \tau_j c_j^\top$, where $P_j = \tau_j \tau_j^\top / \tau_j^\top \tau_j$.
6. Assemble $\alpha = B(T^\top KB)^{-1} T^\top Y$ from the stored $B = [\beta_1 \dots \beta_k]$ and $T = [\tau_1 \dots \tau_k]$.

17.6 One engine, three methods

The three algorithms differ only in which matrices they feed to the shared engine and whether they solve one eigenproblem or iterate a deflation. All read the data through the kernel matrix alone, and all can be viewed as choosing directions that optimize a quadratic criterion under a quadratic constraint. The table makes the parallel explicit.

Method	Criterion optimized	Engine	Uses target?
Kernel PCA	variance $w^T C w$	eigenproblem on K	no
Kernel Fisher	scatter ratio $\frac{\alpha^T M \alpha}{\alpha^T (N + \mu I) \alpha}$	regularized generalized eigenproblem (rank-one M : one solve)	yes (class labels)
PCR	variance, then target reweighting	eigenproblem on K plus $\frac{1}{\lambda_j} (v_j^T y)$	yes (in the weights)
Kernel PLS	covariance with y	deflation loop on K	yes (drives the direction)

Reading down the last two columns tells the whole story. PCA and PCR both diagonalize the kernel matrix, but PCR then lets the target choose how much of each eigenvector to keep. Fisher and PLS both let the target choose the direction itself, one through a generalized eigenproblem, the other through a deflation. The regularization that rescues Fisher from its singular scatter matrix is the same $+\mu I$ that the generalized eigenproblem needs whenever the denominator is only positive semidefinite, which in a rich feature space is always. What began as unsupervised subspace analysis becomes, with the label admitted into A or into the deflation target, a supervised one, at no change to the underlying linear algebra.

17.7 Summary

Supervised subspace methods choose directions in a kernel feature space by how well they separate classes or predict a target, and they all reduce to generalized eigenproblems or deflations on the Gram matrix. Kernel Fisher discriminant analysis maximizes the ratio of between-class to within-class scatter; because the within-class scatter is singular in feature space, it must be regularized, after which the rank-one numerator makes the solution a single linear solve. Principal components regression keeps the kernel eigenvectors but reweights them by their covariance with the target, and its worked failure mode, discarding a low-variance predictive direction, motivates the covariance-driven alternative. Kernel partial least squares extracts the directions of maximum covariance with the response and deflates the kernel matrix after each, combining target alignment with an unbounded number of components. The next parts put these projections to work alongside the supervised margin methods and revisit their statistical footing through Mercer's theorem and spectral rates.

17.8 Common mistakes and practical implications

For **Kernel Discriminants and Projections**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

17.9 Summary and further reading

This chapter established explain the central definitions and claims in Kernel Discriminants and Projections; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Shawe-Taylor and Cristianini 2004), (Fisher 1936), (Mika and Müller 1999).

17.10 Exercises

1. warm-up Explain in one or two sentences why maximizing the variance $w^\top C w$ subject to $\|w\| = 1$ has the same solution as maximizing the Rayleigh quotient $w^\top C w / w^\top w$ without any constraint. What role does the homogeneity of the quotient under rescaling $w \mapsto tw$ play?
2. computation Repeat the kernel Fisher worked example but with the polynomial kernel $\kappa(x, x') = (\langle x, x' \rangle + 1)^2$ on the same four points $x_1 = (1, 0)$, $x_2 = (0, 1)$, $x_3 = (-1, 0)$, $x_4 = (0, -1)$ and $\mu = 1$. Build K , the class means $m_\pm$, the matrices M and N , solve $(N + \mu I)\alpha = m_+ - m_-$, and report the four projections $t = K\alpha$. Are the classes still separated? Hint

Only the Gram matrix changes; the pipeline is identical to Example (kernel Fisher on four points). Compute $K_{ij} = (\langle x_i, x_j \rangle + 1)^2$ first, then reuse the steps. A short numpy script mirrors the check for the linear case.

3. proof Prove the reduction of the generalized eigenproblem: if B is symmetric positive definite with square root $B^{1/2}$, then $Aw = \lambda Bw$ is equivalent to $B^{-1/2}AB^{-1/2}v = \lambda v$ under $v = B^{1/2}w$, and conclude that the maximum of $w^\top Aw / w^\top Bw$ is the largest eigenvalue of $B^{-1/2}AB^{-1/2}$. Where in the argument does positive definiteness, rather than mere positive semidefiniteness, get used? Hint

Premultiply by $B^{-1/2}$ and insert $I = B^{-1/2}B^{1/2}$. Positive definiteness is what makes $B^{1/2}$ invertible and a bijection of $\mathbb{R}^d$, so the change of variables is one-to-one and no direction is lost.

4. proof Show that the between-class scatter matrix $S_B = (\mu_+^\phi - \mu_-^\phi)(\mu_+^\phi - \mu_-^\phi)^\top$ has rank one, and deduce that the Fisher direction is $w \propto S_W^{-1}(\mu_+^\phi - \mu_-^\phi)$ with no eigendecomposition required. Why does the rank-one structure mean that only one discriminant direction carries information in the two-class problem? Hint

An outer product $dd^\top$ has range $\text{span}\{d\}$, hence rank one. In $S_B w = \lambda S_W w$, the left side is always a multiple of $d = \mu_+^\phi - \mu_-^\phi$, so $w \propto S_W^{-1}d$; all other generalized eigenvalues are zero.

5. proof In primal PLS the loading is $p_j = X_j^\top X_j u_j / (u_j^\top X_j^\top X_j u_j)$. Verify the two conjugacy relations $u_j^\top p_j = 1$ and $u_i^\top p_j = 0$ for $i < j$, and explain why they imply that the scores $\tau_j = X_j u_j$ extracted at different stages are mutually orthogonal. Hint

The first is immediate from the definition of p_j . For the second, use that the deflation $X_{j+1} = X_j(I - u_j p_j^\top)$ makes columns of later X_i orthogonal to the already-removed loading directions; expand $u_i^\top p_j$ using the definition of p_j and the orthogonality of $X_i u_i$ to $X_j u_j$.

6. computation Take the PCR worked example and confirm the incremental property: compute α for $k = 1$, then show that adding the $j = 2$ term $\frac{1}{\lambda_2}(v_2^\top y)v_2$ gives the $k = 2$ coefficients without recomputing the first. Verify numerically that $K\alpha = (1, -1, 0)$ at $k = 2$, and explain in one sentence why the third (zero-eigenvalue) direction can never be used. Hint

Orthogonality of the v_j makes the sum $\alpha = \sum_j \frac{1}{\lambda_j}(v_j^\top y)v_j$ a running total. The $1/\lambda_j$ weight is undefined for $\lambda_3 = 0$; that direction has no variance and cannot enter a variance-based projection.

7. exploration Contrast PCR and kernel PLS on the situation of the PCR worked example, where the predictive direction has the smaller eigenvalue. Argue that PLS, extracting the direction of maximum covariance with y , would select that predictive direction first, whereas PCR ordered by variance selects it second. What does this say about when to prefer each method?
8. challenge The regularized kernel Fisher objective penalizes the RKHS norm $\|w\|^2 = \alpha^\top K \alpha$ of the discriminant direction, which is why N is replaced by $N + \mu K$ in some formulations and by $N + \mu I$ in others. Derive both variants from a penalized quotient $\alpha^\top M \alpha / (\alpha^\top N \alpha + \mu \Omega(\alpha))$, taking $\Omega(\alpha) = \alpha^\top K \alpha$ in one case and $\Omega(\alpha) = \alpha^\top \alpha$ in the other, and discuss which penalty corresponds to controlling the feature-space norm of w and which to a plain ridge on the dual coefficients. Hint

Since $w = \sum_i \alpha_i \phi(x_i)$, the feature-space norm is $\|w\|^2 = \sum_{i,j} \alpha_i \alpha_j \kappa(x_i, x_j) = \alpha^\top K \alpha$, giving the $N + \mu K$ form; the plain $\alpha^\top \alpha$ ridge gives $N + \mu I$. Both restore invertibility, but they penalize different geometries.

18 Data Visualization and Kernel MDS

A dataset that lives in a hundred dimensions, or in the infinite-dimensional feature space of a kernel, cannot be looked at. Yet looking is often the fastest way to understand: a scatter of points on a page can reveal clusters, outliers, curved structure, and the sheer separability of classes far quicker than any summary statistic. This chapter is about how to draw such a picture honestly. The problem is to place the points on a plane so that their apparent distances match their true distances as closely as possible, and the classical answer, multidimensional scaling, turns out to be nothing more than an eigenproblem on a cleverly transformed distance matrix. We derive that transformation, the double-centering identity, from scratch; we show that classical MDS is the same computation as principal component analysis and, once distances are read off a kernel, the same computation as kernel PCA from the kernel PCA chapter. The kernel is what fixes the geometry we are drawing, so visualization becomes a direct window onto the feature space a kernel builds.

18.1 The visualization problem

Suppose we are handed a sample $S = \{x_1, \dots, x_\ell\}$ together with a way of measuring how far apart any two of its members are. In the setting of this book that measure comes from a kernel: a feature map $\phi : \mathcal{X} \rightarrow F$ sends each point into a feature space, and the induced distance is

$$\|\phi(x_i) - \phi(x_j)\|^2 = \kappa(x_i, x_i) - 2\kappa(x_i, x_j) + \kappa(x_j, x_j).$$

The feature space F may have thousands of dimensions, or be infinite dimensional, so we cannot inspect the configuration $\{\phi(x_1), \dots, \phi(x_\ell)\}$ directly. What we can do is ask for a faithful shadow of it. Following Shawe-Taylor and Cristianini (2004), we state the goal as finding a map $\tau : \mathcal{X} \rightarrow \mathbb{R}^k$ into a low dimension $k \in \{2, 3\}$ such that the projected distances track the feature-space distances,

$$\|\tau(x_i) - \tau(x_j)\| \approx \|\phi(x_i) - \phi(x_j)\|, \quad i, j = 1, \dots, \ell.$$

A good τ turns an object we cannot see into one we can plot. This is more than cosmetic. Because the distances being preserved are the kernel's own distances, the picture we draw is a picture of the geometry the kernel imposes, and comparing the pictures produced by different kernels is one of the most direct tools available for kernel selection.

The information we are given may be even more meager than a feature map. In many applications what arrives is only a table of dissimilarities: how different two wines taste, how far apart two cities are by road, how dissimilar two questionnaire respondents were judged. There is no Gram matrix of inner products and no Euclidean embedding in sight, only a symmetric matrix D with D_{ij} the dissimilarity of items i and j . The historical name for the family of methods that build a map from such a table is multidimensional scaling, and its classical form, developed by Young and Householder (1938) and Torgerson (1952), is where we begin. The first task it faces is exactly the reverse of the computation above: to turn a matrix of distances back into a matrix of inner products.

18.2 From distances to inner products: classical MDS

An eigenproblem needs a matrix of inner products, not distances, so the crux of classical MDS is a formula that recovers the one from the other. At first sight this looks impossible: distances are invariant to translating and rotating the whole configuration, so they cannot determine absolute inner products, which depend on where the origin sits. The resolution is to fix the origin at the centroid of the points. Once we insist the configuration be centered, the inner products are pinned down, and a short calculation extracts them.

18.2.1 The double-centering identity

Write $d_{ij}^2 = \|x_i - x_j\|^2$ for the squared distances of an unknown configuration $x_1, \dots, x_n$, and let $\bar{x} = \frac{1}{n} \sum_i x_i$ be its centroid. We want the centered inner products $b_{ij} = \langle x_i - \bar{x}, x_j - \bar{x} \rangle$, the entries of the Gram matrix taken about the centroid. The identity that produces them from the d_{ij}^2 is the engine of the whole method.

Proposition (double-centering)

Let $x_1, \dots, x_n$ have squared distances d_{ij}^2 , and set the row, column, and grand means

$$d_{i\cdot}^2 = \frac{1}{n} \sum_{k=1}^n d_{ik}^2, \quad d_{\cdot j}^2 = \frac{1}{n} \sum_{k=1}^n d_{kj}^2, \quad d_{\cdot\cdot}^2 = \frac{1}{n^2} \sum_{k,l=1}^n d_{kl}^2.$$

Then the centered inner products are

$$b_{ij} = -\frac{1}{2}(d_{ij}^2 - d_{i\cdot}^2 - d_{\cdot j}^2 + d_{\cdot\cdot}^2).$$

Equivalently, with the centering matrix $J = I - \frac{1}{n} \mathbf{1}\mathbf{1}^\top$ and the squared-distance matrix $D^{(2)} = [d_{ij}^2]$,

$$B = -\frac{1}{2} J D^{(2)} J.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Distances are unchanged by translation, so we may assume the configuration is already centered, $\bar{x} = 0$; then $b_{ij} = \langle x_i, x_j \rangle =: g_{ij}$ are the ordinary inner products and $\sum_k g_{ik} = \langle x_i, \sum_k x_k \rangle = 0$ for every i . Expanding the square,

$$d_{ij}^2 = \|x_i\|^2 - 2\langle x_i, x_j \rangle + \|x_j\|^2 = g_{ii} - 2g_{ij} + g_{jj}.$$

Averaging over one index and writing $t = \frac{1}{n} \sum_k g_{kk} = \frac{1}{n} \text{tr } G$, the cross terms vanish by centering:

$$d_{i\cdot}^2 = g_{ii} + t, \quad d_{\cdot j}^2 = g_{jj} + t, \quad d_{\cdot\cdot}^2 = 2t.$$

Substituting,

$$-\frac{1}{2}(d_{ij}^2 - d_{i\cdot}^2 - d_{\cdot j}^2 + d_{\cdot\cdot}^2) = -\frac{1}{2}(g_{ii} - 2g_{ij} + g_{jj} - g_{ii} - t - g_{jj} - t + 2t) = g_{ij}.$$

For the matrix form, note that left-multiplying by J subtracts from each column its mean and right-multiplying by J subtracts from each row its mean, so $(JD^{(2)}J)_{ij} = d_{ij}^2 - d_{i\cdot}^2 - d_{\cdot j}^2 + d_{\cdot\cdot}^2$; the prefactor $-\frac{1}{2}$ completes the claim. $\square$

The name double-centering is now transparent: the operator J sweeps out the row means on one side and the column means on the other, and the two sweeps together erase all dependence on the unknown origin, leaving precisely the centered Gram matrix. Because $B = X_c X_c^\top$ for the centered coordinate matrix X_c (rows $x_i - \bar{x}$), it is symmetric and positive semidefinite, and its rank equals the dimension the points genuinely occupy.

18.2.2 The eigendecomposition and the embedding

Once B is in hand, recovering coordinates is the same factorization argument that underlies PCA. Diagonalize the symmetric matrix as $B = V\Lambda V^\top$ with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0$ down the diagonal of Λ and orthonormal eigenvectors in the columns of V . Splitting $\Lambda^{1/2}$ between the two factors,

$$B = (V\Lambda^{1/2})(V\Lambda^{1/2})^\top,$$

so the rows of $X = V\Lambda^{1/2}$ are a set of coordinates whose Gram matrix is exactly B , hence whose distances are exactly the given d_{ij} . To visualize we keep only the leading k columns, taking

$$X_k = V_k \Lambda_k^{1/2}, \quad V_k = [v_1, \dots, v_k], \quad \Lambda_k = \text{diag}(\lambda_1, \dots, \lambda_k),$$

and read off row i as the embedding $\tilde{x}_i \in \mathbb{R}^k$. This truncation is not arbitrary. The Gram matrix of X_k is the best rank- k approximation of B in Frobenius norm, by the Eckart-Young theorem, so among all k -dimensional configurations this one reproduces the inner products, and therefore the distances, as faithfully as possible. The discarded eigenvalues $\lambda_{k+1}, \dots, \lambda_n$ measure exactly the structure that a k -dimensional plot cannot show; when they are zero the embedding is exact. A negative eigenvalue, which can occur when D is not the distance matrix of any Euclidean configuration, signals that the dissimilarities are non-Euclidean, and classical MDS simply drops those directions.

Algorithm (classical MDS from a distance matrix)

Input Symmetric distance matrix $D \in \mathbb{R}^{n \times n}$, target dimension k .

Output Embedding coordinates $\tilde{x}_1, \dots, \tilde{x}_n \in \mathbb{R}^k$.

1. Square the distances entrywise, $D^{(2)} = [D_{ij}^2]$.
2. Double-center: $B = -\frac{1}{2} J D^{(2)} J$ with $J = I - \frac{1}{n} \mathbf{1}\mathbf{1}^\top$.
3. Eigendecompose $B = V\Lambda V^\top$, eigenvalues $\lambda_1 \geq \dots \geq \lambda_n$ in descending order.
4. Keep the top k nonnegative eigenpairs and form $X_k = V_k \Lambda_k^{1/2}$.
5. Return row i of X_k as the coordinates $\tilde{x}_i$.

The worked example runs the algorithm on a case small enough to check by hand and clean enough that the answer is recognizable on sight.

Example (recovering a rectangle from its distances)

Four points sit at the corners of a 4×3 rectangle, so their six pairwise distances are the integers of two 3-4-5 right triangles. We are told only the distance matrix, never the coordinates:

$$D = \begin{pmatrix} 0 & 4 & 5 & 3 \\ 4 & 0 & 3 & 5 \\ 5 & 3 & 0 & 4 \\ 3 & 5 & 4 & 0 \end{pmatrix}.$$

1. Square entrywise. $D^{(2)}$ has rows $(0, 16, 25, 9)$, $(16, 0, 9, 25)$, $(25, 9, 0, 16)$, $(9, 25, 16, 0)$.

2. Double-center with $J = I - \frac{1}{4}\mathbf{1}\mathbf{1}^\top$. Every diagonal of J is 0.75 and every off-diagonal is -0.25 ; the product $B = -\frac{1}{2}JD^{(2)}J$ is

$$B = \begin{pmatrix} 6.25 & -1.75 & -6.25 & 1.75 \\ -1.75 & 6.25 & 1.75 & -6.25 \\ -6.25 & 1.75 & 6.25 & -1.75 \\ 1.75 & -6.25 & -1.75 & 6.25 \end{pmatrix}.$$

3. Eigendecompose B . The eigenvalues come out as $(16, 9, 0, 0)$: two positive, two zero, so the data is exactly two-dimensional.
4. Take the top $k = 2$ pairs. Forming $X_2 = V_2\Lambda_2^{1/2}$ gives the coordinates

$$\tilde{x}_1 = (-2, 1.5), \quad \tilde{x}_2 = (2, 1.5), \quad \tilde{x}_3 = (2, -1.5), \quad \tilde{x}_4 = (-2, -1.5).$$

5. Check the distances. These four points span a rectangle of width 4 and height 3; recomputing all pairwise distances reproduces D exactly, with maximum error 0.

Reading. The two positive eigenvalues $16 = 4^2$ and $9 = 3^2$ are the squared side lengths, and the two zero eigenvalues certify that no third dimension is needed. Double-centering recovered the configuration up to the rotation and reflection that distances can never fix, which is exactly the freedom a visualization is allowed.

Verification artifact. `checks/example-ch-mds-example-17-1.json` records the example source hash and verification scope.

18.3 MDS is PCA is kernel PCA

The rectangle example already hints at the punchline: when the dissimilarities are honest Euclidean distances, the matrix B built by double-centering is a Gram matrix, and diagonalizing a Gram matrix is what PCA does. Making this precise ties classical MDS to the two chapters on either side of it.

Start from a genuine data matrix X whose rows are points $x_1, \dots, x_n \in \mathbb{R}^p$, and let $X_c = JX$ be its centered version. Principal component analysis diagonalizes the covariance $C = \frac{1}{n}X_c^\top X_c$, a $p \times p$ matrix, and projects the data onto its top eigenvectors. But the nonzero eigenvalues of $X_c^\top X_c$ and of the $n \times n$ Gram matrix $X_c X_c^\top$ coincide, and the projected coordinates, the principal component scores, are read directly off the eigenvectors of the Gram matrix as $V_k \Lambda_k^{1/2}$. Now compute what double-centering does to Euclidean distances. Since $d_{ij}^2 = \|x_i\|^2 - 2\langle x_i, x_j \rangle + \|x_j\|^2$, the proposition gives

$$B = -\frac{1}{2}JD^{(2)}J = J(XX^\top)J = (JX)(JX)^\top = X_c X_c^\top,$$

because the rank-one pieces built from $\|x_i\|^2$ and $\|x_j\|^2$ are annihilated by J , which kills any vector constant across a row or column. So the MDS matrix B is exactly the centered Gram matrix that PCA diagonalizes, and the two embeddings are identical. This is the classical equivalence: classical MDS on Euclidean distances returns the PCA scores.

Example (classical MDS equals PCA)

Five points in the plane, given as raw coordinates for this check:

$$X = \begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 3 & 0 \\ 0 & 2 \\ 1 & 3 \end{pmatrix}.$$

Route A runs PCA on the centered data; route B throws the coordinates away, keeps only the Euclidean distance matrix, and runs classical MDS.

1. Route A: center and diagonalize the Gram. Subtracting the column means gives X_c , and the centered Gram $G = X_c X_c^\top$ has eigenvalues (9.4928, 2.5072, 0, 0, 0); the top two eigenpairs give the PCA scores

$$Y^{\text{PCA}} : (-0.6923, 1.0586), (-0.5293, -0.3462), (-1.9341, -0.5092), (1.4963, 0.6008), (1.6594, -0.8040).$$

2. Route B: distances, then double-center. The Euclidean distance matrix has entries such as $D_{12} = \sqrt{2} \approx 1.4142$ and $D_{35} = \sqrt{13} \approx 3.6056$; double-centering gives $B = -\frac{1}{2}JD^{(2)}J$.
3. Compare the two matrices. B equals G entry for entry, $\max_{ij}|B_{ij} - G_{ij}| = 0$, so their eigenvalues match: (9.4928, 2.5072, 0, 0, 0).
4. Read off and align the MDS embedding. $Y^{\text{MDS}} = V_2 \Lambda_2^{1/2}$ is identical to Y^{PCA} ; the per-axis sign alignment is (+, +) and the maximum coordinate difference is 0.

Reading. The distance matrix carries the same information as the centered coordinates: run PCA on the points or MDS on their distances and the plot is the same, up to the sign of each axis that neither method can pin down. The two zero eigenvalues beyond the second again confirm the data was two-dimensional all along.

Verification artifact. checks/example-ch-mds-example-17-2.json records the example source hash and verification scope.

The step to kernels is now immediate and is the observation of Williams (2002). Replace the linear inner product $\langle x_i, x_j \rangle$ by a kernel value $\kappa(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$. The feature-space squared distances are $\|\phi(x_i) - \phi(x_j)\|^2 = \kappa(x_i, x_i) - 2\kappa(x_i, x_j) + \kappa(x_j, x_j)$, and running the same double-centering on them gives

$$B = -\frac{1}{2}JD^{(2)}J = JKJ, \quad K_{ij} = \kappa(x_i, x_j),$$

the centered kernel matrix. Diagonalizing JKJ and projecting onto its leading eigenvectors is precisely the kernel PCA of the kernel PCA chapter. So the three methods are one computation viewed through three lenses: MDS reads it as distance preservation, PCA as variance maximization, kernel PCA as principal components in feature space. Choosing the kernel κ chooses the geometry, and the metric MDS assumption, that the dissimilarities are Euclidean, is met automatically because a positive semidefinite kernel guarantees a Euclidean feature-space embedding by Mercer's theorem from the Mercer chapter.

18.4 Kernel MDS: visualizing the feature space

The equivalence pays off directly in practice. When the data already lives in a kernel-defined feature space we never need to build a distance matrix and convert it back to inner products, because the kernel hands us the inner products at the outset. The first stages of classical MDS, which existed only to manufacture a Gram matrix from dissimilarities, become unnecessary, and MDS collapses to computing the first two or three kernel PCA projections. Shawe-Taylor and Cristianini (2004) state the resulting procedure as follows.

Algorithm (kernel MDS for feature-space visualization)

Input Data $S = \{x_1, \dots, x_\ell\}$, kernel κ , display dimension $k \in \{2, 3\}$.

Output Low-dimensional coordinates $\tilde{x}_1, \dots, \tilde{x}_\ell$ for plotting.

1. Form the kernel matrix $K_{ij} = \kappa(x_i, x_j)$.
2. Center it: $\tilde{K} = JKJ$ with $J = I - \frac{1}{\ell} \mathbf{1}\mathbf{1}^T$, that is, subtract row, column, and grand means.
3. Eigendecompose $[V, \Lambda] = \text{eig}(\tilde{K})$, eigenvalues descending.
4. Set the dual projection vectors $\alpha^{(j)} = \frac{1}{\sqrt{\lambda_j}} v_j$ for $j = 1, \dots, k$.
5. Project each point onto axis j : $\tau_j(x) = \sum_{i=1}^{\ell} \alpha_i^{(j)} \kappa(x_i, x)$, and embed $\tilde{x}_i = (\tau_1(x_i), \dots, \tau_k(x_i))$.
6. Display the transformed sample $\tilde{S} = \{\tilde{x}_1, \dots, \tilde{x}_\ell\}$.

The normalization $\alpha^{(j)} = \lambda_j^{-1/2} v_j$ is what makes step five a projection onto a unit-norm feature-space direction, exactly as in kernel PCA; the coordinate $\tau_j(x_i)$ equals $\sqrt{\lambda_j} (v_j)_i$, so the plotted spread along axis j is governed by $\sqrt{\lambda_j}$. A new point outside the sample is placed by the same formula in step five, which is the out-of-sample extension that a purely distance-based MDS cannot offer without recomputation. Because the whole procedure sees the data only through κ , swapping the kernel redraws the picture: a linear kernel yields the ordinary PCA scatter, a Gaussian kernel unfolds the sample along the curved directions its feature map emphasizes, and a poorly matched kernel produces a visibly structureless blob. Reading these plots against each other is the practical value the introduction promised, a direct look at the geometry each candidate kernel would hand to a downstream clustering or classification stage such as those in the kernel clustering chapter.

18.5 Metric and non-metric MDS

Classical MDS makes a strong assumption, that the dissimilarities are, or are close to, Euclidean distances, so that double-centering yields a nearly positive semidefinite B whose top eigenvectors carry the signal. This is the metric branch of the subject: the numbers in D are taken at face value as distances to be reproduced. Metric MDS proper generalizes the eigenproblem to an explicit optimization, minimizing a stress functional

$$\text{stress}(\tau) = \sum_{i < j} (\|\tau(x_i) - \tau(x_j)\| - D_{ij})^2$$

over configurations τ , which coincides with the classical solution when D is Euclidean but degrades more gracefully when it is not. The kernel-based view sidesteps the worry entirely, because a positive semidefinite kernel supplies genuine Euclidean distances by construction, so the metric assumption always holds in feature space.

Non-metric MDS relaxes the assumption in the other direction, for the case where the dissimilarities are only ordinal: we trust the ranking of the D_{ij} , not their numerical values. It seeks an embedding whose distances have the same rank order as the input dissimilarities, minimizing a stress computed after passing the target distances through an arbitrary monotone increasing transformation, fit by isotonic regression. This is the appropriate tool when dissimilarities come from human judgments or ordinal scales, where "twice as dissimilar" has no meaning but "more dissimilar" does. Cox and Cox (2000) is the standard reference for the full metric and non-metric hierarchy; for the purposes of this book the essential point is that the kernel already commits to a metric, so the visualization we compute is a metric embedding of the feature-space geometry, and the choice of kernel is the choice of what "distance" the plot displays. The same double-centered kernel matrix reappears whenever two views of the data must be related, for instance in the correlation analysis of the kernel CCA chapter, underscoring that centering the Gram matrix is the common first move of the whole spectral toolbox.

18.6 Manifold learning as kernel PCA

Every embedding built so far has diagonalized a kernel matrix fixed in advance: choose κ , center it, and take the top eigenvectors. That linear-in-feature-space recipe is exactly what fails on data drawn from a curved manifold. Points sampled from a spiral or a rolled sheet can be near each other along the surface yet far apart across the ambient space, so a Gaussian or linear kernel, reading only ambient distance, folds the manifold onto itself and the plot becomes a tangle. The methods invented to cure this, Isomap, locally linear embedding, and Laplacian eigenmaps, all begin the same way: build a neighborhood graph on the sample and let the graph, rather than a formula, decide which points are close. The striking fact, established by Ham, Lee, Mika, and Schölkopf (2004), is that once the graph is built each of the three is again kernel PCA, run on a kernel matrix assembled from the graph. The spectral machinery of the kernel PCA chapter never changes, only the kernel does, and here the kernel is derived from the data instead of chosen. This section names the kernel behind each method, works two of them in numbers, and then draws the honest boundary at the neighbor-embedding methods t-SNE and UMAP, which leave the kernel-PCA family altogether.

18.6.1 Isomap: the centered geodesic kernel

The first idea, due to Tenenbaum, de Silva, and Langford (2000), keeps classical MDS intact and changes only the distances fed to it. On a curved manifold the honest measure of separation is the geodesic distance, the length of the shortest path that stays on the surface, and although we cannot see the surface we can approximate that path along the neighborhood graph. Isomap joins each point to its nearest neighbors, weights every edge by the Euclidean length of that short hop, and sets D_{ij}^{geo} to the shortest-path distance between i and j in the resulting graph. It then runs classical MDS on D^{geo} : square the geodesic distances, double-center, and diagonalize. By the double-centering identity that opened this chapter, this is diagonalizing the matrix

$$K_{\text{Iso}} = -\frac{1}{2} H D^{\text{geo}(2)} H, \quad H = I - \frac{1}{n} \mathbf{1}\mathbf{1}^T,$$

so Isomap is precisely kernel PCA with K_{Iso} as its kernel, the centered matrix of squared geodesic distances. When the sample is dense and the manifold unrolls isometrically onto a flat region, the geodesic distances are the ordinary distances of the unrolled coordinates, K_{Iso} is positive semidefinite, and its leading eigenvectors are those flat coordinates. When the geodesic metric is not exactly Euclidean, K_{Iso} can carry a few negative eigenvalues, and classical MDS simply drops those directions, the conditionally-Euclidean behavior discussed in the metric and non-metric section above.

Example (Isomap unrolls a curved arc)

Five points sit on a circular arc at angles $0^\circ, 40^\circ, 80^\circ, 120^\circ, 160^\circ$, a curved one-dimensional manifold in the plane. A symmetric 2-nearest-neighbor rule wires them into the path 1-2-3-4-5 that follows the arc, each edge weighted by its Euclidean length, the equal chord $c = 2 \sin 20^\circ \approx 0.6840$.

1. Take shortest paths for the geodesic distances. Along the path the graph distance between nodes i and j is $|i - j|c$, so the rows of D^{geo} are built from 0, 0.6840, 1.3681, 2.0521, 2.7362; the far endpoints 1 and 5 sit at geodesic distance 2.7362, against a straight Euclidean chord of only 1.9696.
2. Double-center the squared geodesic distances. With $H = I - \frac{1}{5} \mathbf{1}\mathbf{1}^T$, the Isomap kernel $K_{\text{Iso}} = -\frac{1}{2} H D^{\text{geo}(2)} H$ is

$$K_{\text{Iso}} = \begin{pmatrix} 1.8716 & 0.9358 & 0 & -0.9358 & -1.8716 \\ 0.9358 & 0.4679 & 0 & -0.4679 & -0.9358 \\ 0 & 0 & 0 & 0 & 0 \\ -0.9358 & -0.4679 & 0 & 0.4679 & 0.9358 \\ -1.8716 & -0.9358 & 0 & 0.9358 & 1.8716 \end{pmatrix}.$$

3. Diagonalize. The eigenvalues are (4.6791, 0, 0, 0, 0): a single positive value, so the geodesic geometry is exactly one-dimensional.
4. Read off the embedding. The top eigenpair gives coordinates $(-1.3681, -0.6840, 0, 0.6840, 1.3681)$, which are the centered arc-length positions $-2c, -c, 0, c, 2c$ exactly, error 0.
5. Compare with ordinary MDS. Double-centering the Euclidean chords instead gives eigenvalues (2.7660, 0.5758, 0, 0, 0): two positive values, so the raw distances force the plot into two dimensions, the bowed arc.

Reading. Swapping Euclidean distance for graph geodesic distance is the whole of Isomap, and it turns a two-dimensional bow into a one-dimensional line: the geodesic kernel K_{Iso} has rank one where the Euclidean double-centering has rank two. Isomap is classical MDS, hence kernel PCA, on the kernel that the neighborhood graph supplies.

Verification artifact. checks/example-ch-mds-example-17-3.json records the example source hash and verification scope.

18.6.2 Locally linear embedding: the max-eigenvalue-shift kernel

Roweis and Saul (2000) start from a different local invariant. Instead of distances they ask how each point sits inside its neighborhood: find weights W_{ij} , supported on the neighbors of i and summing to one along each row, that best reconstruct x_i as $\sum_j W_{ij}x_j$. Because the reconstruction error $\sum_i \|x_i - \sum_j W_{ij}x_j\|^2$ is invariant to rotating, translating, and rescaling each neighborhood, the weights record the local geometry in a form that survives the unrolling. The embedding is the low-dimensional configuration whose points obey the same reconstruction weights, and minimizing the embedding-space reconstruction error is a quadratic form in the matrix

$$M = (I - W)^\top (I - W).$$

Its minimizers are the eigenvectors of M with the smallest eigenvalues, discarding the constant eigenvector that sits in the null space with eigenvalue zero. This is the reverse of what kernel PCA does, which reads the largest eigenvalues, and the fix is a single shift: with $\lambda_{\max}$ the top eigenvalue of M , set

$$K_{\text{LLE}} = \lambda_{\max} I - M.$$

Subtracting M from $\lambda_{\max} I$ leaves the eigenvectors untouched and reverses their order, so the smallest eigenvectors of M become the largest eigenvectors of K_{LLE} , and kernel PCA on K_{LLE} returns the LLE embedding. Locally linear embedding is therefore kernel PCA with the shifted kernel K_{LLE} , a fact the exercises make precise.

18.6.3 Laplacian eigenmaps: the pseudo-inverse Laplacian kernel

The third method, from Belkin and Niyogi (2003), embeds the graph so that edges pull their endpoints together. Build the same neighborhood graph, with symmetric nonnegative weights W_{ij} , taken either as plain indicators of adjacency or as heat weights $W_{ij} = \exp(-\|x_i - x_j\|^2/t)$; let $D = \text{diag}(\sum_j W_{ij})$ be the degree matrix and $L = D - W$ the graph Laplacian. A one-dimensional embedding f that keeps neighbors close minimizes the Dirichlet energy

$$\sum_{i,j} W_{ij} (f_i - f_j)^2 = 2 f^\top L f$$

subject to a scale normalization, and the minimizers are the eigenvectors of L with the smallest nonzero eigenvalues, again dropping the constant eigenvector $\mathbf{1}$ that spans the null space of a connected graph. This Laplacian is the same combinatorial operator that the geometric-kernels chapter filters to build heat and Matern kernels on a graph,

and Belkin and Niyogi (2003) proved that on points sampled from a manifold it converges to the Laplace-Beltrami operator, which is why these graph eigenvectors track the manifold's own harmonics.

To read this as kernel PCA, take the unnormalized problem $Lf = \lambda f$ and invert the spectrum. The Laplacian is positive semidefinite with eigenvalues $0 = \lambda_1 < \lambda_2 \leq \dots \leq \lambda_n$ and eigenvectors $u_1 = \mathbf{1}/\sqrt{n}, u_2, \dots, u_n$, so its Moore-Penrose pseudo-inverse

$$L^\dagger = \sum_{i \geq 2} \frac{1}{\lambda_i} u_i u_i^\top$$

shares every eigenvector but replaces each nonzero eigenvalue by its reciprocal. The smallest nonzero eigenvalues of L become the largest eigenvalues of $L^\dagger$, so the Laplacian-eigenmap coordinates, the bottom nonzero eigenvectors of L , are exactly the top eigenvectors of $L^\dagger$. Laplacian eigenmaps is thus kernel PCA with the kernel $K = L^\dagger$, the pseudo-inverse graph Laplacian, which is the filter $\Phi(\lambda) = 1/\lambda$ named as a graph kernel in the geometric-kernels chapter and is precisely the commute-time (resistance-distance) kernel of the graph. The same eigenvectors reappear in the spectral clustering of the kernel clustering chapter, where the sign of the second eigenvector splits the graph: visualization and clustering read the identical spectrum for different ends.

Proposition (Laplacian eigenmaps is kernel PCA on $L^\dagger$)

Let L be a connected graph's Laplacian with spectrum $0 = \lambda_1 < \lambda_2 \leq \dots \leq \lambda_n$ and orthonormal eigenvectors $u_1, \dots, u_n$. The pseudo-inverse $L^\dagger$ has the same eigenvectors, with eigenvalue 0 on u_1 and $1/\lambda_i$ on u_i for $i \geq 2$, and it satisfies $HL^\dagger H = L^\dagger$ for the centering $H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$. Consequently the k -dimensional kernel PCA embedding of $L^\dagger$ uses the eigenvectors $u_2, \dots, u_{k+1}$, which are exactly the Laplacian-eigenmap coordinates.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Diagonalize $L = \sum_i \lambda_i u_i u_i^\top$. The pseudo-inverse of a symmetric matrix inverts the nonzero eigenvalues on their eigenspaces and is zero on the kernel, so $L^\dagger = \sum_{i \geq 2} \lambda_i^{-1} u_i u_i^\top$ with the stated spectrum. Since $u_1 = \mathbf{1}/\sqrt{n}$, every other eigenvector is orthogonal to $\mathbf{1}$, giving $Hu_i = u_i$ for $i \geq 2$ and $Hu_1 = 0$; hence $HL^\dagger H = \sum_{i \geq 2} \lambda_i^{-1} (Hu_i)(Hu_i)^\top = L^\dagger$, so $L^\dagger$ is already centered and kernel PCA may diagonalize it directly. Its eigenvalues in decreasing order are $1/\lambda_2 \geq \dots \geq 1/\lambda_n > 0$ followed by 0, so the top k eigenvectors are $u_2, \dots, u_{k+1}$. These are by definition the bottom k nonzero eigenvectors of L , the Laplacian-eigenmap embedding. $\square$

Example (Laplacian eigenmaps as kernel PCA on $L^\dagger$)

Five points form a lollipop graph: a triangle on nodes 1, 2, 3 with a two-edge tail 3-4-5, the shape a symmetric k -nearest-neighbor rule produces from a dense clump of three points and a short chain of two. With unit edge weights the degrees are (2, 2, 3, 2, 1) and the Laplacian $L = D - W$ is

$$L = \begin{pmatrix} 2 & -1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ -1 & -1 & 3 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 1 \end{pmatrix}.$$

1. Diagonalize the Laplacian. The eigenvalues are (0, 0.5188, 2.3111, 3, 4.1701); the zero belongs to the constant vector $\mathbf{1}$, and the Laplacian-eigenmap embedding keeps the two smallest nonzero eigenvectors, at $\lambda_2 = 0.5188$ and $\lambda_3 = 2.3111$.
2. Read the Fiedler vector. The eigenvector at λ_2 is $u_2 = (0.4193, 0.4193, 0.2018, -0.3380, -0.7024)$, decreasing monotonically from the triangle body through the hub to the tail tip: it orders the nodes along the graph.

3. Form the kernel $K = L^\dagger$. The pseudo-inverse is

$$L^\dagger = \begin{pmatrix} 0.5467 & 0.2133 & 0.08 & -0.32 & -0.52 \\ 0.2133 & 0.5467 & 0.08 & -0.32 & -0.52 \\ 0.08 & 0.08 & 0.28 & -0.12 & -0.32 \\ -0.32 & -0.32 & -0.12 & 0.48 & 0.28 \\ -0.52 & -0.52 & -0.32 & 0.28 & 1.08 \end{pmatrix},$$

whose rows sum to zero, so it is already centered, $HL^\dagger H = L^\dagger$, and kernel PCA can diagonalize it directly.

4. Diagonalize the kernel. Its eigenvalues in decreasing order are (1.9275, 0.4327, 0.3333, 0.2398, 0), which are exactly the reciprocals 1/0.5188, 1/2.3111, 1/3, 1/4.1701 of the nonzero Laplacian eigenvalues, with the zero landing on the constant vector.
5. Match the embeddings. The top two eigenvectors of K equal the two bottom nonzero eigenvectors of L to machine precision, maximum coordinate difference 0 after sign alignment. Kernel PCA scales axis j by $\sqrt{1/\lambda_{j+1}}$, here (1.3883, 0.6578), the only difference from the raw eigenmap.

Reading. Inverting the Laplacian's spectrum turns its smallest nonzero eigenvectors into a kernel's largest, so the Laplacian-eigenmap embedding and kernel PCA on $L^\dagger$ diagonalize the same operator and share their axes. The pseudo-inverse Laplacian is the kernel that was hiding inside Belkin and Niyogi's construction.

Verification artifact. checks/example-ch-mds-example-17-4.json records the example source hash and verification scope.

The three classical methods thus differ only in the kernel they hand to the same eigenproblem, and each kernel is manufactured from the neighborhood graph rather than chosen in advance.

Method	Data-derived kernel	Embedding vectors
Isomap (Tenenbaum et al. 2000)	$K_{\text{Iso}} = -\frac{1}{2}HD^{\text{geo}(2)}H$	top eigenvectors of K_{Iso}
LLE (Roweis and Saul 2000)	$K_{\text{LLE}} = \lambda_{\max} I - (I - W)^\top (I - W)$	top eigenvectors of K_{LLE} , the bottom of M
Laplacian eigenmaps (Belkin and Niyogi 2003)	$K = L^\dagger$	top eigenvectors of $L^\dagger$, the bottom nonzero of L

In every row the recipe is that of kernel PCA: center the kernel, diagonalize, keep the leading eigenvectors. The kernel view unifies what looked like three unrelated algorithms and ties them back to the double-centering that opened this chapter, since Isomap's kernel is literally the classical MDS matrix built from geodesic rather than Euclidean distances.

18.6.4 Where t-SNE and UMAP part company

Remark (neighbor embeddings are not kernel PCA)

The two most popular visualizers today, t-SNE (van der Maaten and Hinton 2008) and UMAP (McInnes, Healy, and Melville 2018), are not members of this family, and it is worth being exact about why. Both are neighbor-embedding methods. They convert distances into a probability or fuzzy weight on each neighbor pair, once in the data (Gaussian affinities for t-SNE, a smooth membership for UMAP) and once in the low-dimensional map (a Student- t kernel for t-SNE, a smooth membership for UMAP), and then move the embedding coordinates to minimize a divergence between the two weightings, the Kullback-Leibler divergence for t-SNE and a cross-entropy for UMAP. That objective is not a Rayleigh quotient: it is non-convex, the low-dimensional coordinates enter it nonlinearly through the map-space kernel, and it is minimized by gradient descent rather than by an eigendecomposition. There is no fixed $n \times n$ data kernel whose leading eigenvectors are the embedding, so no

analogue of the projection step of the kernel MDS algorithm above exists, and a new point cannot be placed without re-optimizing. The tradeoff is deliberate and often worth it: by giving up the global distance preservation that kernel PCA enforces, these methods render local cluster structure far more vividly, at the known cost of distorting large-scale geometry, so relative cluster sizes and separations on a t-SNE or UMAP plot are not to be read as distances. The classical spectral methods and the neighbor embeddings answer different questions, and only the former are kernel PCA.

18.7 Common mistakes and practical implications

For **Data Visualization and Kernel MDS**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

18.8 Summary and further reading

This chapter established explain the central definitions and claims in Data Visualization and Kernel MDS; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Shawe-Taylor and Cristianini 2004), (Young and Householder 1938), (Torgerson 1952).

18.9 Exercises

1. warm-up Show that the centering matrix $J = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$ is symmetric and idempotent, $J^2 = J$, and that $J\mathbf{1} = 0$. Conclude that $B = -\frac{1}{2}JD^{(2)}J$ always has $\mathbf{1}$ in its null space, so the all-ones vector is an eigenvector with eigenvalue 0.
2. warm-up For the two-point set with a single distance $D_{12} = d$, carry out classical MDS by hand: form $D^{(2)}$, double-center to get B , and verify that the one nonzero eigenvalue is $d^2/2$, giving an embedding of the two points at $\pm d/2$ on a line.
3. warm-up Take three points with all pairwise distances equal to 1 (an equilateral triangle). Compute B by double-centering and show its nonzero eigenvalues are both $1/2$. Explain why the embedding needs exactly two dimensions and why no single axis suffices.
4. computation Prove that classical MDS on Euclidean distances returns the PCA scores by filling in the identity $-\frac{1}{2}JD^{(2)}J = (JX)(JX)^\top$ for a data matrix X . Where exactly is the centering by J used to kill the $\|x_i\|^2$ and $\|x_j\|^2$ terms? Hint: write $D^{(2)} = \mathbf{u}\mathbf{1}^\top - 2XX^\top + \mathbf{1}\mathbf{u}^\top$ with $\mathbf{u}$ the vector of squared norms $\|x_i\|^2$, and use $J\mathbf{1} = 0$.
5. computation Let K be a kernel matrix and $\tilde{K} = JKJ$ its centering. Show that the squared feature-space distances $\|\phi(x_i) - \phi(x_j)\|^2 = K_{ii} - 2K_{ij} + K_{jj}$ satisfy $-\frac{1}{2}JD^{(2)}J = \tilde{K}$, so that kernel MDS and kernel PCA solve the same eigenproblem. Which property of K guarantees $\tilde{K}$ is positive semidefinite, so no negative eigenvalues appear?
6. computation A configuration is genuinely three-dimensional. Explain, in terms of the eigenvalues of B , what is lost when it is plotted in two dimensions, and give a quantity built from the discarded eigenvalues that measures the fraction of squared-distance structure the plot fails to show. Relate this to the Eckart-Young optimality of the truncation.

7. challenge Construct a symmetric dissimilarity matrix D (four points suffice) whose double-centered B has a strictly negative eigenvalue, so that D is not the distance matrix of any Euclidean configuration. Describe what classical MDS does with the negative direction and why the resulting plot can still be useful. Hint: violate the triangle inequality, e.g. make one distance much larger than the sum of two others, then double-center and inspect the spectrum.
8. challenge The out-of-sample problem. Given the kernel MDS embedding of a training sample, a new point x is placed by $\tau_j(x) = \sum_i \alpha_i^{(j)} \kappa(x_i, x)$. Show that this formula reproduces the training embedding when $x = x_m$ is one of the training points, and explain why a purely distance-based classical MDS has no comparable formula without re-solving the eigenproblem on the enlarged matrix. Hint: substitute $\kappa(x_i, x_m) = \tilde{K}_{im}$ after centering and use $\tilde{K}v_j = \lambda_j v_j$.
9. computation Locally linear embedding as kernel PCA. Let $M = (I - W)^\top (I - W)$ be the LLE cost matrix, symmetric positive semidefinite with eigenvalues $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n$ and the constant vector spanning the zero eigenspace. Show that $K_{\text{LLE}} = \mu_n I - M$ is positive semidefinite, has the same eigenvectors as M , and orders them in reverse, so the top nonconstant eigenvectors of K_{LLE} are the bottom nonconstant eigenvectors of M . Conclude that kernel PCA on K_{LLE} reproduces the LLE embedding. Hint: if $Mu = \mu u$ then $K_{\text{LLE}}u = (\mu_n - \mu)u$, so the shift only relabels eigenvalues.
10. challenge Why t-SNE is not kernel PCA. A kernel PCA embedding is the set of leading eigenvectors of one fixed symmetric matrix, so every embedding coordinate is a linear function of the columns of that matrix. Using the description of the t-SNE objective as a Kullback-Leibler divergence between a fixed data-space affinity and a Student- t affinity computed from the embedding itself, identify which of these two properties fails, and deduce that no fixed $n \times n$ kernel can have the t-SNE map as its top eigenvectors. Explain why this also blocks the out-of-sample projection formula that kernel MDS enjoys. Hint: the map-space affinities depend on the unknown embedding coordinates nonlinearly, so the matrix to be diagonalized would have to depend on its own eigenvectors.

19 Mercer's Theorem, Spectra, and Rates

- The kernel is the one place in a kernel method where prior knowledge about the problem enters: choosing a kernel means choosing which functions the RKHS norm considers simple. This chapter makes that choice quantitative. We first show, on a hand-computed example and then through Green functions, that RKHS norms are smoothness functionals. We then develop the spectral theory of kernels on compact spaces: the integral operator of a kernel, its diagonalization, Mercer's theorem of 1909 with its full proof, and the explicit construction of the RKHS from eigenfunctions and eigenvalues. Two worked examples, periodic kernels on the unit interval and dot-product kernels on the sphere, make the spectra concrete. The payoff comes at the end: the eigenvalue decay of the kernel governs the statistical performance of kernel ridge regression, and we prove convergence rates for polynomially decaying, exponentially decaying, and finite-rank spectra.
-

19.1 From smoothness functionals to kernels

Every learning algorithm we have built so far minimizes an empirical risk plus a multiple of the squared RKHS norm. The RKHS norm is therefore the regularizer, and the kernel determines what the regularizer penalizes. To use kernel methods intelligently we need intuition for the link between a kernel and the norm of its RKHS, and the natural language for that link is smoothness. Smoothness of a function is classically quantified by Sobolev-type norms, in particular L^2 norms of derivatives. A canonical example is spline regression on the real line:

$$\min_f \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \int f''(t)^2 dt,$$

which trades data fit against the integrated squared second derivative, and produces the familiar smooth curve threading through a noisy scatter of points. The question for us: is a penalty like $\int (f'')^2$ secretly an RKHS norm, so that spline-type problems are kernel ridge regressions in disguise? In this section we answer yes on a simple example, then in general through Green functions of differential operators.

19.1.1 A first example: the min kernel on $[0, 1]$

Consider the space of functions on the unit interval that vanish at the origin and have a square-integrable derivative:

$$\mathcal{H} = \{f : [0, 1] \rightarrow \mathbb{R} \mid f \text{ absolutely continuous, } f' \in L^2([0, 1]), f(0) = 0\},$$

endowed with the bilinear form

$$\forall f, g \in \mathcal{H}, \quad \langle f, g \rangle_{\mathcal{H}} = \int_0^1 f'(u) g'(u) du.$$

The associated quadratic form is exactly a smoothness functional: $\langle f, f \rangle_{\mathcal{H}} = \int_0^1 f'(u)^2 du = \|f'\|_{L^2([0,1])}^2$, the energy of the first derivative. Absolute continuity is the precise regularity needed for this to make sense; we recall the definition, since it recurs throughout the chapter.

Definition (absolute continuity)

Let $I \subset \mathbb{R}$ be an interval. A function $f : I \rightarrow \mathbb{R}$ is *absolutely continuous* (AC) on I if for any $\epsilon > 0$ there exists $\delta > 0$ such that for any finite sequence of pairwise disjoint sub-intervals $(u_k, v_k) \subset I$ with $\sum_k (v_k - u_k) < \delta$, it holds that $\sum_k |f(v_k) - f(u_k)| < \epsilon$.

Absolute continuity implies uniform continuity, hence continuity. The property that matters for us is the fundamental theorem of calculus: f is AC on $[a, b]$ if and only if f has a derivative f' almost everywhere, f' is Lebesgue integrable, and for all $x \in [a, b]$,

$$f(x) = f(a) + \int_a^x f'(t) dt.$$

Equivalently, there exists a Lebesgue integrable g on $[a, b]$ with $f(x) = f(a) + \int_a^x g(t) dt$ for all x , in which case $g = f'$ almost everywhere. Absolutely continuous functions are exactly the functions one can recover by integrating their derivative.

Theorem (the min kernel)

$\mathcal{H}$ is an RKHS with reproducing kernel

$$\forall (x, y) \in [0, 1]^2, \quad K(x, y) = \min(x, y).$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The RKHS norm of the min kernel is therefore precisely the smoothness functional we started from, $\|f\|_{\mathcal{H}} = \|f'\|_{L^2([0,1])}$. In particular, the first-derivative spline problem

$$\min_{f \in \mathcal{H}} \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \int_0^1 f'(t)^2 dt$$

is nothing but kernel ridge regression with kernel $K(x, y) = \min(x, y)$: replace the integral penalty by $\lambda \|f\|_{\mathcal{H}}^2$ and apply the representer theorem. A smoothing problem posed in analysis language has become a finite-dimensional linear-algebra problem posed in kernel language.

Proof

We must verify three things: that $\mathcal{H}$ is a Hilbert space of functions, that $K_x \in \mathcal{H}$ for every $x \in [0, 1]$, and that $\langle f, K_x \rangle_{\mathcal{H}} = f(x)$ for all x and all $f \in \mathcal{H}$.

Step 1: $\mathcal{H}$ is a pre-Hilbert space of functions. $\mathcal{H}$ is a vector space of functions and $\langle f, g \rangle_{\mathcal{H}}$ is a bilinear form with $\langle f, f \rangle_{\mathcal{H}} \geq 0$. Since f is absolutely continuous, it is differentiable almost everywhere and $f(x) = f(0) + \int_0^x f'(u) du$ for all $x \in [0, 1]$. Because $f(0) = 0$, the Cauchy-Schwarz inequality (applied to $f' \cdot \mathbf{1}_{[0,x]}$) gives

$$|f(x)| = \left| \int_0^x f'(u) du \right| \leq \sqrt{x} \left(\int_0^1 f'(u)^2 du \right)^{\frac{1}{2}} = \sqrt{x} \langle f, f \rangle_{\mathcal{H}}^{\frac{1}{2}}.$$

Hence $\langle f, f \rangle_{\mathcal{H}} = 0$ forces $f = 0$ everywhere, so the form is an inner product and $\mathcal{H}$ is a pre-Hilbert space. The same inequality shows that norm convergence in $\mathcal{H}$ controls pointwise values, which we use next.

Step 2: $\mathcal{H}$ is complete. Let $(f_n)_{n \in \mathbb{N}}$ be a Cauchy sequence in $\mathcal{H}$. Then $(f'_n)_{n \in \mathbb{N}}$ is a Cauchy sequence in $L^2([0, 1])$, hence converges to some $g \in L^2([0, 1])$. By the inequality of Step 1, $(f_n(x))_{n \in \mathbb{N}}$ is a Cauchy sequence of reals for each $x \in [0, 1]$, hence converges to a limit $f(x)$. Moreover

$$f(x) = \lim_n f_n(x) = \lim_n \int_0^x f'_n(u) du = \int_0^x g(u) du,$$

so f is absolutely continuous with $f' = g$ almost everywhere; in particular $f' \in L^2([0, 1])$. Finally $f(0) = \lim_n f_n(0) = 0$, so $f \in \mathcal{H}$, and $\|f_n - f\|_{\mathcal{H}} = \|f'_n - g\|_{L^2([0, 1])} \rightarrow 0$. Thus $\mathcal{H}$ is a Hilbert space.

Step 3: $K_x \in \mathcal{H}$ for all x . Fix x and let $K_x(y) = K(x, y) = \min(x, y)$ on $[0, 1]$. As a function of y , K_x rises with slope one until $y = x$ and is constant equal to x afterwards. It is differentiable except at the single point $y = x$, its derivative $\mathbf{1}_{[0, x]}$ is square integrable, and $K_x(0) = 0$. Hence $K_x \in \mathcal{H}$.

Step 4: the reproducing property. For any $x \in [0, 1]$ and $f \in \mathcal{H}$,

$$\langle f, K_x \rangle_{\mathcal{H}} = \int_0^1 f'(u) K'_x(u) du = \int_0^x f'(u) du = f(x).$$

This shows $\mathcal{H}$ is an RKHS with reproducing kernel K . $\square$

19.1.2 Green functions of differential operators

The min kernel is no accident. The computation above used only two structural facts: the inner product was $\langle Df, Dg \rangle_{L^2}$ for a differential operator D (here $Df = f'$), and the kernel section K_x was the function whose image under D^*D reproduces evaluation at x . That second object has a classical name.

Definition (Green function)

Consider the differential equation $f = Dg$ on a class of functions, where g is unknown. To solve it, one looks for g of the form

$$g(x) = \int_{\mathcal{X}} k(x, y) f(y) dy$$

for some function $k : \mathcal{X}^2 \rightarrow \mathbb{R}$. Applying D under the integral sign, k must satisfy, for all $x \in \mathcal{X}$,

$$f(x) = Dg(x) = \langle Dk_x, f \rangle_{L^2(\mathcal{X})}.$$

If such a k exists, it is called the *Green function* of the operator D .

Theorem (Green kernels)

Let $\mathcal{X} = \mathbb{R}^d$ and let D be a differential operator on a class of functions $\mathcal{H}$ such that, endowed with the inner product

$$\forall (f, g) \in \mathcal{H}^2, \quad \langle f, g \rangle_{\mathcal{H}} = \langle Df, Dg \rangle_{L^2(\mathcal{X})},$$

$\mathcal{H}$ is a Hilbert space. Then $\mathcal{H}$ is an RKHS whose reproducing kernel is the Green function of the operator D^*D , where D^* denotes the adjoint of D .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Let K be the Green function of D^*D . For every $x \in \mathcal{X}$, $K_x \in \mathcal{H}$, because

$$\langle DK_x, DK_x \rangle_{L^2(\mathcal{X})} = \langle D^* DK_x, K_x \rangle_{L^2(\mathcal{X})} = K_x(x) < \infty,$$

where the last equality is the Green function property applied to $f = K_x$. (Caveat: depending on how $\mathcal{H}$ is defined, membership may require further conditions, such as boundary conditions, to be checked case by case.) Moreover, for all $f \in \mathcal{H}$ and $x \in \mathcal{X}$,

$$f(x) = \langle D^* DK_x, f \rangle_{L^2(\mathcal{X})} = \langle DK_x, Df \rangle_{L^2(\mathcal{X})} = \langle K_x, f \rangle_{\mathcal{H}},$$

which is the reproducing property. Hence $\mathcal{H}$ is an RKHS with reproducing kernel K . $\square$

Example (recovering the min kernel)

Take $\mathcal{X} = [0, 1]$ and $Df(u) = f'(u)$, with the boundary condition $f(0) = 0$ defining $\mathcal{H}$ as before. To find the reproducing kernel we solve for k in

$$f(x) = \langle D^* Dk_x, f \rangle_{L^2([0,1])} = \langle Dk_x, Df \rangle_{L^2([0,1])} = \int_0^1 k'_x(u) f'(u) du.$$

Since $f(x) = \int_0^x f'(u) du$, the solution is $k'_x(u) = \mathbf{1}_{[0,x]}(u)$, which integrates (with $k_x(0) = 0$) to

$$k_x(u) = \begin{cases} u & \text{if } u \leq x, \\ x & \text{otherwise,} \end{cases}$$

that is, $k(x, x') = \min(x, x')$, exactly the kernel found by direct verification.

Verification artifact. checks/example-ch07-example-18-1.json records the example source hash and verification scope.

Green functions explain, on a case-by-case basis, why particular RKHS norms are Sobolev-type penalties. But we would like something more systematic: a decomposition that works for a whole class of kernels at once, exhibits an explicit feature space, and quantifies smoothness by a spectrum. That is Mercer's theory, to which the rest of this chapter is devoted.

19.2 Mercer kernels and their integral operator

19.2.1 Mercer kernels

When $\mathcal{X}$ is finite, a positive definite kernel is just a positive semidefinite matrix, and the spectral decomposition of that matrix produces an explicit feature map: $K(x, t) = \sum_k \lambda_k \psi_k(x) \psi_k(t)$ with $\lambda_k \geq 0$. Mercer's theory generalizes exactly this factorization from finite sets to compact metric spaces. Historically it also provided the first proof, due to Mercer (1909), that a positive definite kernel on a non-finite set is an inner product.

Definition (Mercer kernel)

A kernel K on a set $\mathcal{X}$ is called a *Mercer kernel* if:

1. $\mathcal{X}$ is a compact metric space (typically, a closed bounded subset of $\mathbb{R}^d$);
2. $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is a continuous positive definite kernel (with respect to the Borel topology).

The plan mirrors the finite case, one object at a time. The kernel matrix becomes a linear operator on a function space; positive semidefiniteness of the matrix becomes self-adjointness and positivity of the operator; diagonalization of the matrix becomes the spectral theorem for compact self-adjoint operators, which supplies a complete orthonormal basis of eigenfunctions with nonnegative eigenvalues; and the factorization of the matrix becomes the expansion

$$K(x, t) = \sum_{k=1}^{\infty} \lambda_k \psi_k(x) \psi_k(t).$$

Each step needs care, because we now work in infinite dimension. We begin with the operator vocabulary.

19.2.2 Compact, self-adjoint, positive operators

Definition (operator classes)

Let $\mathcal{H}$ be a Hilbert space.

- A *linear operator* is a continuous linear mapping from $\mathcal{H}$ to itself.
- A linear operator L is *compact* if, for any bounded sequence $\{f_n\}_{n=1}^{\infty}$, the sequence $\{Lf_n\}_{n=1}^{\infty}$ has a convergent subsequence.
- L is *self-adjoint* if $\langle f, Lg \rangle = \langle Lf, g \rangle$ for all $f, g \in \mathcal{H}$.
- L is *positive* if it is self-adjoint and $\langle f, Lf \rangle \geq 0$ for all $f \in \mathcal{H}$.

These three properties are the operator translations of three familiar facts about a positive semidefinite matrix, and it is worth reading them as such. Self-adjointness is symmetry, $\mathbf{K} = \mathbf{K}^T$; positivity is $\mathbf{f}^T \mathbf{K} \mathbf{f} \geq 0$; and compactness is the substitute, in infinite dimension, for the finite-dimensionality that let us diagonalize the matrix in the first place. In finite dimension every operator is automatically compact, so the notion is invisible; on an infinite-dimensional space it is precisely the extra hypothesis that rescues the spectral theorem, by guaranteeing a discrete spectrum accumulating only at zero rather than a continuous one. The work of the next subsection is to show that the integral operator of a Mercer kernel has all three properties, after which diagonalization proceeds exactly as for a symmetric matrix.

19.2.3 The integral operator of a kernel

Before writing any formula, it helps to be explicit about what we are trying to build and why it must be an operator on a function space. In the finite case the kernel is a matrix, a matrix acts on vectors, and diagonalizing it means finding the vectors it merely rescales. To repeat this program on an infinite $\mathcal{X}$ we need an object that acts on the right infinite-dimensional analogue of a vector, namely a function, and whose action we can diagonalize. The transform below is exactly that object: it consumes a function f and returns a new function whose value at x is a kernel-weighted average of f , just as the i -th entry of $\mathbf{K} \mathbf{f}$ is a kernel-weighted combination of the entries of $\mathbf{f}$. Choosing $L^2_{\nu}(\mathcal{X})$ as the home of these functions is what makes the diagonalization possible: it is a Hilbert space, so the spectral theorem is available, and its inner product is the continuous limit of the dot product that made the Gram matrix symmetric.

The operator that plays the role of the kernel matrix is the integral operator with kernel K . Let ν be a finite Borel measure on $\mathcal{X}$, and let $L^2_{\nu}(\mathcal{X})$ denote the Hilbert space of (equivalence classes of) square integrable functions on $\mathcal{X}$. For any function $K : \mathcal{X}^2 \rightarrow \mathbb{R}$, define the transform

$$\forall f \in L^2_{\nu}(\mathcal{X}), \quad (L_K f)(x) = \int_{\mathcal{X}} K(x, t) f(t) d\nu(t).$$

On a finite set with counting measure this is exactly multiplication by the kernel matrix: if $\mathcal{X} = \{t_1, \dots, t_m\}$ and ν is the counting measure, then $(L_K f)(x_i) = \sum_j K(x_i, t_j) f(t_j)$ is the i -th entry of the matrix-vector product

Kf. Reading the integral operator as the kernel matrix with the sum replaced by an integral is the right mental picture: the eigenfunctions of L_K are the continuous analogue of the eigenvectors of the Gram matrix, and the eigenvalues λ_k that Mercer's theorem is about are the continuous analogue of the Gram matrix eigenvalues. This correspondence between the finite matrix and the operator is the foundation of the operator-theoretic approach to learning theory (Cucker and Smale, 2002; Steinwart and Christmann, 2008).

Proposition (discrete Mercer: the Gram matrix eigendecomposition)

Let $\mathcal{X} = \{x_1, \dots, x_m\}$ be finite. The kernel is then the Gram matrix $\mathbf{K} = (K(x_i, x_j))_{ij}$, which is symmetric and positive semidefinite. The spectral theorem for symmetric matrices diagonalizes it as

$$\mathbf{K} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top = \sum_{k=1}^m \lambda_k u_k u_k^\top, \quad \lambda_1 \geq \dots \geq \lambda_m \geq 0,$$

with orthonormal eigenvectors $u_1, \dots, u_m$. Equivalently $\mathbf{K} = \mathbf{B}^\top \mathbf{B}$ with $\mathbf{B} = \mathbf{\Lambda}^{1/2} \mathbf{U}^\top$ (Shawe-Taylor and Cristianini, 2004), and reading off the columns of $\mathbf{B}$ exhibits an explicit feature map

$$\Phi(x_i) = \left(\sqrt{\lambda_k} [u_k]_i \right)_{k=1}^m, \quad K(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

This is nothing but Mercer's expansion with the sum truncated at m : each eigenvector coordinate $[u_k]_i$ plays the role of the eigenfunction value $\psi_k(x_i)$, so the eigenfunctions of L_K are the continuous limit of Gram-matrix eigenvectors sampled at the data (Schölkopf and Smola, 2002; Shawe-Taylor and Cristianini, 2004). The whole of Mercer's theory is the claim that this elementary finite factorization survives the passage to an infinite compact $\mathcal{X}$, once continuity and compactness supply the discreteness of the spectrum that finite dimension gave for free.

Before formalizing, it helps to look at the finite object the operator generalizes. The figure below assembles the Gram matrix $\mathbf{K} = (K(x_i, x_j))_{ij}$ of a handful of points and displays both its heatmap and its spectrum; the eigenvalues shown there are exactly the finite-sample stand-ins for the λ_k of L_K that the rest of the chapter studies, and widening or narrowing the kernel reshapes them just as it will reshape the operator's spectrum.

The following lemma transfers all the good properties of that matrix to L_K : symmetry of K becomes self-adjointness of the operator, positive definiteness becomes positivity, and, crucially, continuity of K on a compact space buys *compactness* of L_K , the property that will let us diagonalize it exactly as one diagonalizes a symmetric matrix.

Lemma (properties of L_K)

If K is a Mercer kernel and ν a finite Borel measure on $\mathcal{X}$, then L_K is a compact and bounded linear operator on $L^2_\nu(\mathcal{X})$, self-adjoint and positive.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Proof

Throughout, $C_K = \max_{x,t \in \mathcal{X}} |K(x, t)| < +\infty$, which exists because K is continuous and $\mathcal{X} \times \mathcal{X}$ is compact.

Step 1: L_K maps $L^2_\nu(\mathcal{X})$ into $L^2_\nu(\mathcal{X})$, and in fact into continuous functions. For any $f \in L^2_\nu(\mathcal{X})$ and $x_1, x_2 \in \mathcal{X}$,

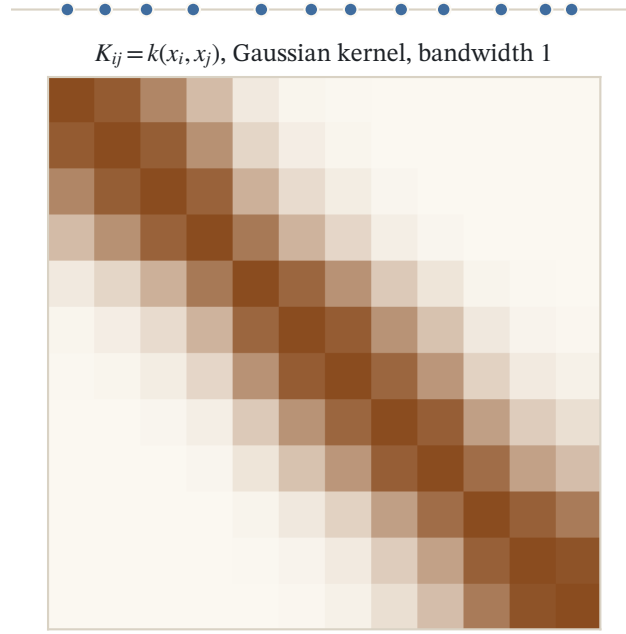


Figure 19.1: Twelve points on a line and their Gram matrix $K_{ij} = k(x_i, x_j)$ for a Gaussian kernel at bandwidth 1. A kernel is positive definite exactly when this matrix is positive semidefinite for every sample; narrowing the bandwidth drives it toward the identity.

$$\begin{aligned}
 |(L_K f)(x_1) - (L_K f)(x_2)| &= \left| \int (K(x_1, t) - K(x_2, t)) f(t) d\nu(t) \right| \\
 &= |\langle K_{x_1} - K_{x_2}, f \rangle_{L^2_\nu(\mathcal{X})}| \\
 &\leq \|K_{x_1} - K_{x_2}\|_{L^2_\nu(\mathcal{X})} \|f\|_{L^2_\nu(\mathcal{X})} \\
 &\leq \sqrt{\nu(\mathcal{X})} \max_{t \in \mathcal{X}} |K(x_1, t) - K(x_2, t)| \|f\|_{L^2_\nu(\mathcal{X})},
 \end{aligned}$$

where the first inequality is Cauchy-Schwarz. K being continuous on the compact $\mathcal{X} \times \mathcal{X}$ is uniformly continuous, so the right-hand side tends to zero as $x_2 \rightarrow x_1$, uniformly. Hence $L_K f$ is continuous, and in particular $L_K f \in L^2_\nu(\mathcal{X})$ (with the slight abuse of notation $C(\mathcal{X}) \subset L^2_\nu(\mathcal{X})$).

Step 2: L_K is linear and continuous. Linearity is immediate from the definition of L_K and linearity of the integral. For continuity, for all $f \in L^2_\nu(\mathcal{X})$ and $x \in \mathcal{X}$,

$$|(L_K f)(x)| = \left| \int K(x, t) f(t) d\nu(t) \right| \leq \sqrt{\nu(\mathcal{X})} \max_{t \in \mathcal{X}} |K(x, t)| \|f\|_{L^2_\nu(\mathcal{X})} \leq \sqrt{\nu(\mathcal{X})} C_K \|f\|_{L^2_\nu(\mathcal{X})},$$

and therefore

$$\|L_K f\|_{L^2_\nu(\mathcal{X})} = \left(\int (L_K f)(t)^2 d\nu(t) \right)^{\frac{1}{2}} \leq \nu(\mathcal{X}) C_K \|f\|_{L^2_\nu(\mathcal{X})}.$$

Step 3: a criterion for compactness. Let $C(\mathcal{X})$ denote the continuous functions on $\mathcal{X}$ with the uniform norm $\|f\|_\infty = \max_{x \in \mathcal{X}} |f(x)|$. A set of functions $G \subset C(\mathcal{X})$ is *equicontinuous* if

$$\forall \epsilon > 0, \exists \delta > 0, \forall (x, y) \in \mathcal{X}^2, \quad \|x - y\| < \delta \implies \forall g \in G, |g(x) - g(y)| < \epsilon.$$

The Ascoli theorem states that a subset $H \subset C(\mathcal{X})$ is relatively compact (i.e., its closure is compact) if and only if it is uniformly bounded and equicontinuous.

Step 4: L_K is compact. Let $(f_n)_{n \geq 0}$ be a bounded sequence in $L^2_\nu(\mathcal{X})$, say $\|f_n\|_{L^2_\nu(\mathcal{X})} < M$ for all n . By Step 1, $(L_K f_n)_{n \geq 0}$ is a sequence of continuous functions. It is uniformly bounded because, by Step 2,

$$\|L_K f_n\|_\infty \leq \sqrt{\nu(\mathcal{X})} C_K \|f_n\|_{L^2_\nu(\mathcal{X})} \leq \sqrt{\nu(\mathcal{X})} C_K M,$$

and it is equicontinuous because, by the bound of Step 1,

$$|L_K f_n(x_1) - L_K f_n(x_2)| \leq \sqrt{\nu(\mathcal{X})} \max_{t \in \mathcal{X}} |K(x_1, t) - K(x_2, t)| M,$$

with K uniformly continuous. By the Ascoli theorem we can extract a subsequence converging uniformly in $C(\mathcal{X})$, hence also in $L^2_\nu(\mathcal{X})$ since ν is finite. Thus L_K is compact.

Step 5: L_K is self-adjoint. K being symmetric, for all $f, g \in L^2_\nu(\mathcal{X})$,

$$\langle f, L_K g \rangle_{L^2_\nu(\mathcal{X})} = \int f(x) (L_K g)(x) d\nu(x) = \iint f(x) g(t) K(x, t) d\nu(x) d\nu(t) = \langle L_K f, g \rangle_{L^2_\nu(\mathcal{X})},$$

where the middle equality uses Fubini's theorem, justified since the integrand is integrable for the product measure (K bounded, f, g square integrable, ν finite).

Step 6: L_K is positive. Approximating the double integral by finite sums over points $x_1, \dots, x_k$,

$$\langle f, L_K f \rangle_{L^2_\nu(\mathcal{X})} = \iint f(x) f(t) K(x, t) \nu(dx) \nu(dt) = \lim_{k \rightarrow \infty} \frac{\nu(\mathcal{X})}{k^2} \sum_{i,j=1}^k K(x_i, x_j) f(x_i) f(x_j) \geq 0,$$

because each finite sum is nonnegative by positive definiteness of K . $\square$

19.2.4 Spectral decomposition of the operator

Compact self-adjoint operators are precisely the operators that can be diagonalized like symmetric matrices. We quote the general result (see, e.g., Debnath and Mikusinski, 2005, Section 4.10).

Theorem (spectral theorem)

Let L be a compact self-adjoint linear operator on a Hilbert space $\mathcal{H}$. Then there exists in $\mathcal{H}$ a complete orthonormal system $(\psi_1, \psi_2, \dots)$ of eigenvectors of L , with real eigenvalues $(\lambda_1, \lambda_2, \dots)$, which are nonnegative if L is positive.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Remark (continuity of eigenfunctions)

The spectral theorem applies to L_K by the lemma above. The eigenfunctions ψ_k associated with eigenvalues $\lambda_k \neq 0$ can be taken continuous, because

$$\psi_k = \frac{1}{\lambda_k} L_K \psi_k,$$

and L_K maps $L^2_\nu(\mathcal{X})$ into continuous functions (Step 1 of the proof). Each such ψ_k has a canonical continuous representative, which we use from now on.

19.3 Mercer's theorem

In plain terms, the theorem below says that a continuous positive definite kernel on a compact space can be written as an infinite sum of separable rank-one pieces $\lambda_k \psi_k(x) \psi_k(t)$, one per eigenvalue, and that this sum converges not merely in an averaged L^2 sense but pointwise and even uniformly. It is the exact infinite-dimensional counterpart of diagonalizing a symmetric positive semidefinite matrix as $\mathbf{K} = \sum_k \lambda_k u_k u_k^\top$: the eigenvectors u_k become eigenfunctions ψ_k , the finite sum becomes a series, and everything the kernel knows is stored in the two lists (λ_k) and (ψ_k) .

We can now state the main result. One hypothesis deserves emphasis: the measure ν must be *nondegenerate*, meaning $\nu(U) > 0$ for every nonempty open set $U \subset \mathcal{X}$. Nondegeneracy is what lets us upgrade an equality in L^2_ν to an equality at every point: a continuous function whose L^2_ν norm vanishes must vanish identically, since otherwise it would be bounded away from zero on some open set of positive measure.

Theorem (Mercer)

Let $\mathcal{X}$ be a compact metric space, ν a nondegenerate finite Borel measure on $\mathcal{X}$, and K a continuous positive definite kernel. Let $\lambda_1 \geq \lambda_2 \geq \dots \geq 0$ denote the nonnegative eigenvalues of L_K and $(\psi_1, \psi_2, \dots)$ the corresponding eigenfunctions. Then all functions ψ_k with $\lambda_k \neq 0$ are continuous, and for any $x, t \in \mathcal{X}$,

$$K(x, t) = \sum_{k=1}^{\infty} \lambda_k \psi_k(x) \psi_k(t),$$

where the convergence is absolute for each $x, t \in \mathcal{X}$, and uniform on $\mathcal{X} \times \mathcal{X}$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Two features of the statement carry the pedagogical weight. The first is that a single list of numbers (λ_k) and functions (ψ_k) simultaneously diagonalizes the operator L_K and reconstructs the kernel K itself: the spectrum is not an auxiliary computation but a complete description of the kernel. The second is the strength of the convergence. An equality that held only in L^2_ν would reconstruct K merely up to sets of measure zero, which is useless for a kernel we must evaluate at arbitrary individual points; uniform convergence of continuous partial sums guarantees instead that the series equals $K(x, t)$ at every single pair (x, t) , and, as the corollary below shows, that the induced feature map is continuous. This is the precise sense in which Mercer's theorem upgrades the abstract existence of a feature space to an explicit coordinate system valid point by point.

Remark (Mercer's theorem as the continuous face of positive definiteness)

Shawe-Taylor and Cristianini (2004) state the theorem in a slightly different but equivalent form worth recording, because it exposes what the hypothesis of positive definiteness is really doing. Let $\mathcal{X}$ be a compact subset of $\mathbb{R}^n$, let K be continuous and symmetric, and call its integral operator L_K *positive* when

$$\iint_{\mathcal{X} \times \mathcal{X}} K(x, z) f(x) f(z) dx dz \geq 0 \quad \text{for all } f \in L^2(\mathcal{X}).$$

Then K admits a uniformly convergent expansion $K(x, z) = \sum_j \phi_j(x) \phi_j(z)$ with $\langle \phi_i, \phi_j \rangle_{L^2} = \delta_{ij}$ and $\sum_j \|\phi_j\|_{L^2}^2 < \infty$; the $\phi_j = \sqrt{\lambda_j} \psi_j$ simply fold the eigenvalue into the eigenfunction, so this is exactly our expansion rewritten. The content of their proof is a single equivalence: positivity of the operator holds if and only if every finite Gram submatrix is positive semidefinite. One direction is the delicate one. If some submatrix on points $x_1, \dots, x_\ell$ had

$\sum_{i,j} K(x_i, x_j) \alpha_i \alpha_j = \varepsilon < 0$, place narrow Gaussian bumps of masses α_i at the x_i ; as their width shrinks to zero the double integral above converges to $\varepsilon < 0$, contradicting positivity. Mercer's theorem is therefore an equivalent reformulation of the finitely positive semidefinite property, the very property that defines a kernel: the uniform expansion and the positive Gram matrices are two faces of one condition.

The proof is constructive and follows Mercer's original 1909 argument. It repeatedly plays the two Hilbert spaces in the picture against each other: $L^2_\nu(\mathcal{X})$, where the spectral theorem lives, and the RKHS $\mathcal{H}$ of K , whose existence we know from the Aronszajn theory of Chapter 2, Positive Definite Kernels and RKHS, where evaluation functionals are continuous. Throughout, $C_K = \max_{x,t} |K(x, t)|$.

Proof (six steps)

Step 1: for any $k \geq 1$ with $\lambda_k > 0$, ψ_k belongs to the RKHS $\mathcal{H}$ of K . Since $\lambda_k > 0$, for all $x \in \mathcal{X}$,

$$\psi_k(x) = \frac{1}{\lambda_k} L_K \psi_k(x) = \frac{1}{\lambda_k} \int K(x, t) \psi_k(t) d\nu(t) = \lim_{n \rightarrow +\infty} \underbrace{\frac{\nu(\mathcal{X})}{\lambda_k n} \sum_{i=1}^n K(x, t_i) \psi_k(t_i)}_{h_n(x)}$$

for a conveniently chosen sequence of quadrature points $t_1, t_2, \dots$ (the integrand $t \mapsto K(x, t) \psi_k(t)$ is continuous, so the integral is a limit of such finite sums). Each h_n is a finite linear combination of the functions K_{t_i} , hence $h_n \in \mathcal{H}$, and for any $n, m \in \mathbb{N}$,

$$\langle h_n, h_m \rangle_{\mathcal{H}} = \frac{\nu(\mathcal{X})^2}{\lambda_k^2 nm} \sum_{i=1}^n \sum_{j=1}^m \psi_k(t_i) \psi_k(t_j) K(t_i, t_j),$$

using $\langle K_{t_i}, K_{t_j} \rangle_{\mathcal{H}} = K(t_i, t_j)$.

Step 2: the sequence (h_n) is Cauchy in $\mathcal{H}$ and converges to ψ_k . The double sums above are quadrature sums for a continuous integrand, so

$$\lim_{n, m \rightarrow +\infty} \langle h_n, h_m \rangle_{\mathcal{H}} = \frac{1}{\lambda_k^2} \iint K(t, t') \psi_k(t) \psi_k(t') d\nu(t) d\nu(t') := R,$$

and therefore

$$\|h_n - h_m\|_{\mathcal{H}}^2 = \langle h_n, h_n \rangle_{\mathcal{H}} + \langle h_m, h_m \rangle_{\mathcal{H}} - 2\langle h_n, h_m \rangle_{\mathcal{H}} \xrightarrow{n, m \rightarrow \infty} R + R - 2R = 0.$$

So $(h_n)_{n \in \mathbb{N}}$ is a Cauchy sequence in $\mathcal{H}$ and converges to some $h \in \mathcal{H}$. Convergence in the RKHS norm implies pointwise convergence (since $|g(x)| = |\langle g, K_x \rangle_{\mathcal{H}}| \leq \|g\|_{\mathcal{H}} \sqrt{K(x, x)}$), so for every $x \in \mathcal{X}$,

$$h(x) = \lim_{n \rightarrow +\infty} h_n(x) = \psi_k(x),$$

and finally $\psi_k = h \in \mathcal{H}$.

Step 3: $\{\sqrt{\lambda_k} \psi_k : \lambda_k > 0\}$ is an orthonormal system in $\mathcal{H}$. Let $i, j \geq 1$ with $\lambda_i, \lambda_j > 0$; then $\sqrt{\lambda_i} \psi_i, \sqrt{\lambda_j} \psi_j \in \mathcal{H}$ by Steps 1 and 2. Writing $\psi_i = \frac{1}{\lambda_i} \int K_t \psi_i(t) d\nu(t)$ as an $\mathcal{H}$ -valued integral (the limit of the h_n of Step 1), and using the reproducing property $\langle K_t, \psi_j \rangle_{\mathcal{H}} = \psi_j(t)$,

$$\begin{aligned}
\left\langle \sqrt{\lambda_i} \psi_i, \sqrt{\lambda_j} \psi_j \right\rangle_{\mathcal{H}} &= \frac{1}{\sqrt{\lambda_i}} \left\langle \int K_t \psi_i(t) d\nu(t), \sqrt{\lambda_j} \psi_j \right\rangle_{\mathcal{H}} \\
&= \sqrt{\frac{\lambda_j}{\lambda_i}} \int \langle K_t, \psi_j \rangle_{\mathcal{H}} \psi_i(t) d\nu(t) \\
&= \sqrt{\frac{\lambda_j}{\lambda_i}} \int \psi_j(t) \psi_i(t) d\nu(t) = \sqrt{\frac{\lambda_j}{\lambda_i}} \langle \psi_i, \psi_j \rangle_{L^2_\nu(\mathcal{X})} = \delta_{i,j},
\end{aligned}$$

since the ψ_k are orthonormal in $L^2_\nu(\mathcal{X})$ and the prefactor equals one when $i = j$.

Step 4: for any $x \in \mathcal{X}$, $\sum_{k:\lambda_k>0} \lambda_k \psi_k(x)^2 \leq C_K$. For any x , $K_x \in \mathcal{H}$ with $\|K_x\|_{\mathcal{H}}^2 = K(x, x) \leq C_K$. Since $\{\sqrt{\lambda_k} \psi_k : \lambda_k > 0\}$ is an orthonormal system in $\mathcal{H}$ and $\langle K_x, \sqrt{\lambda_k} \psi_k \rangle_{\mathcal{H}} = \sqrt{\lambda_k} \psi_k(x)$, Bessel's inequality gives

$$C_K \geq \|K_x\|_{\mathcal{H}}^2 \geq \sum_{k:\lambda_k>0} \left\langle K_x, \sqrt{\lambda_k} \psi_k \right\rangle_{\mathcal{H}}^2 = \sum_{k:\lambda_k>0} \lambda_k \psi_k(x)^2.$$

Step 5: for fixed x , the series $t \mapsto \sum_i \lambda_i \psi_i(x) \psi_i(t)$ converges uniformly in t to a continuous function g_x . Restricting sums to indices with $\lambda_i > 0$, Cauchy-Schwarz and Step 4 give, for any m, ℓ and any $t \in \mathcal{X}$,

$$\left| \sum_{i=m}^{m+\ell} \lambda_i \psi_i(x) \psi_i(t) \right| \leq \left(\sum_{i=m}^{m+\ell} \lambda_i \psi_i(x)^2 \right)^{\frac{1}{2}} \left(\sum_{i=m}^{m+\ell} \lambda_i \psi_i(t)^2 \right)^{\frac{1}{2}} \leq \sqrt{C_K} \left(\sum_{i=m}^{m+\ell} \lambda_i \psi_i(x)^2 \right)^{\frac{1}{2}},$$

and the right-hand side tends to zero as $m \rightarrow \infty$ (tail of a convergent series), uniformly in t . The series is therefore uniformly Cauchy in t ; its terms are continuous, so it converges uniformly to a continuous function g_x . The same bound shows the convergence is absolute at every pair (x, t) .

Step 6: $K_x = g_x$, first in L^2_ν , then everywhere. Expand K_x over the orthonormal basis $\{\psi_k, k \geq 1\}$ of $L^2_\nu(\mathcal{X})$, using $\langle K_x, \psi_k \rangle_{L^2_\nu(\mathcal{X})} = (L_K \psi_k)(x) = \lambda_k \psi_k(x)$:

$$K_x = \sum_{k \geq 1} \langle K_x, \psi_k \rangle_{L^2_\nu(\mathcal{X})} \psi_k = \sum_{k \geq 1} \lambda_k \psi_k(x) \psi_k = \sum_{k \geq 1: \lambda_k > 0} \lambda_k \psi_k(x) \psi_k,$$

so $K_x = g_x$ in $L^2_\nu(\mathcal{X})$, i.e., $\|K_x - g_x\|_{L^2(\nu)} = 0$. Since ν is nondegenerate and both K_x and g_x are continuous, this forces

$$\forall t \in \mathcal{X}, \quad K_x(t) = g_x(t) = \sum_i \lambda_i \psi_i(x) \psi_i(t).$$

It remains to upgrade to uniform convergence on $\mathcal{X} \times \mathcal{X}$, which is where continuity of K enters once more. Taking $t = x$ gives $\sum_i \lambda_i \psi_i(x)^2 = K(x, x)$ pointwise; the partial sums are continuous, nondecreasing in the number of terms, and converge pointwise to the continuous function $x \mapsto K(x, x)$ on the compact $\mathcal{X}$, so by Dini's theorem the convergence is uniform in x . The Cauchy-Schwarz bound of Step 5 then controls the tail at any pair (x, t) by the uniform tails on the diagonal, so the expansion of $K(x, t)$ converges uniformly on $\mathcal{X} \times \mathcal{X}$. $\square$

19.3.1 Mercer kernels as inner products in ℓ^2

The promised explicit feature space is the space of square-summable sequences. Let ℓ^2 denote the Hilbert space of real sequences $u = (u_k)_{k \in \mathbb{N}}$ with $\sum_{k \in \mathbb{N}} u_k^2 < +\infty$, endowed with $\langle u, v \rangle = \sum_{k \in \mathbb{N}} u_k v_k$.

Corollary (Mercer feature map)

The mapping

$$\Phi : \mathcal{X} \rightarrow \ell^2, \quad x \mapsto \left(\sqrt{\lambda_k} \psi_k(x) \right)_{k \in \mathbb{N}}$$

is well defined, continuous, and satisfies

$$K(x, t) = \langle \Phi(x), \Phi(t) \rangle_{\ell^2}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

By Mercer's theorem, for all $x \in \mathcal{X}$ the series $\sum_k \lambda_k \psi_k^2(x)$ converges to $K(x, x) < \infty$, so $\Phi(x) \in \ell^2$, and the identity $K(x, t) = \sum_k \lambda_k \psi_k(x) \psi_k(t) = \langle \Phi(x), \Phi(t) \rangle_{\ell^2}$ is the Mercer expansion. Continuity of Φ results from

$$\|\Phi(x) - \Phi(t)\|_{\ell^2}^2 = \sum_{k=1}^{\infty} \lambda_k (\psi_k(x) - \psi_k(t))^2 = K(x, x) + K(t, t) - 2K(x, t),$$

which tends to zero as $t \rightarrow x$ by continuity of K . $\square$

Remark (a Mercer kernel is trace class)

Integrating the diagonal identity $K(x, x) = \sum_k \lambda_k \psi_k(x)^2$ against ν and using that the ψ_k are L_ν^2 -normalized gives

$$\sum_{k=1}^{\infty} \lambda_k = \int_{\mathcal{X}} K(x, x) d\nu(x) \leq C_K \nu(\mathcal{X}) < \infty.$$

So the eigenvalues of a Mercer kernel are not merely nonnegative and summable-in-square: their sum itself is finite, which is the statement that L_K is *trace class*. This finite trace is the standing hypothesis behind the operator theory of this chapter, and it is exactly what will let us control tail sums $\sum_{i>J} \lambda_i$ in the rate analysis.

Remark (effective dimension: the mapped data lie in a shrinking box)

The feature map $\Phi(x) = (\sqrt{\lambda_k} \psi_k(x))_{k \in \mathbb{N}}$ confines the data to a very particular region of ℓ^2 . Its k -th coordinate never leaves the interval $[-\sqrt{l_k}, \sqrt{l_k}]$ with $l_k = \sup_{x \in \mathcal{X}} \lambda_k \psi_k(x)^2$, so $\Phi(\mathcal{X})$ sits inside an axis-parallel box with side lengths $2\sqrt{l_k}$. Whenever the eigenfunctions are uniformly bounded, $\sup_x \psi_k(x)^2 \leq C$, the sides shrink as fast as the spectrum, $l_k \leq C \lambda_k$ (Schölkopf and Smola, 2002). Although the feature space is infinite-dimensional, the mapped data therefore occupy only a thin, rapidly narrowing sliver of it.

This carries an immediate quantitative reading. Uniform convergence of the Mercer series means that for any $\varepsilon > 0$ there is a finite $n = n(\varepsilon)$ with

$$|K(x, t) - \langle \Phi_n(x), \Phi_n(t) \rangle| < \varepsilon, \quad \Phi_n(x) = \left(\sqrt{\lambda_1} \psi_1(x), \dots, \sqrt{\lambda_n} \psi_n(x) \right),$$

for all $x, t \in \mathcal{X}$: the feature space is finite-dimensional up to any prescribed accuracy, and the number of coordinates that genuinely matter, the *effective dimension*, is set by how fast $\lambda_k \rightarrow 0$. The same tail of the spectrum resurfaces at the end of the chapter as the quantity $\sum_{i>J} \lambda_i$ that controls the variance of kernel ridge regression.

19.3.2 Scope and limits of the construction

Three remarks put the theorem in perspective. First, this proof extends the proof valid when $\mathcal{X}$ is finite, and it is constructive: Mercer (1909) exhibits the feature map rather than merely proving its existence. Second, the eigensystem (the λ_k and ψ_k) depends on the choice of the measure $d\nu(x)$; different measures lead to different feature spaces for the same kernel on the same space $\mathcal{X}$. The kernel is intrinsic, its spectral coordinates are not. Third, compactness and continuity are genuinely required. For $\mathcal{X} = \mathbb{R}^d$, the eigenvalue problem

$$\int_{\mathcal{X}} K(x, t) \psi(t) dt = \lambda \psi(x)$$

need not have a countable family of eigenvalues, and Mercer's theorem does not hold. Other tools are then required, notably the Fourier transform for shift-invariant kernels, which is the subject of Chapter 19, Translation-Invariant, Semigroup, and Probabilistic Kernels.

19.4 Worked example 1: periodic kernels on $[0, 1]$

Our first worked example makes every object of Mercer's theorem computable. Take $\mathcal{X} = [0, 1]$ endowed with the Lebesgue measure $d\nu(x) = dx$, and consider a positive definite kernel of the convolutional form

$$K(x, t) = \kappa(x - t),$$

where $\kappa : \mathbb{R} \rightarrow \mathbb{R}$ is continuous and 1-periodic. To write Mercer's expansion we must find the eigenfunctions of L_K , i.e., solve

$$(L_K \psi)(x) = \int_0^1 \kappa(x - t) \psi(t) dt = \lambda \psi(x).$$

Convolution operators are diagonalized by Fourier modes, and that is exactly what happens.

19.4.1 Fourier eigenfunctions

Lemma (Fourier diagonalization)

Let $(\psi_n)_{n \in \mathbb{N}}$ be the Fourier orthonormal basis of $L^2([0, 1])$ given by $\psi_0(x) = 1$ and, for $n \geq 1$,

$$\psi_{2n-1}(x) = \sqrt{2} \sin(2\pi nx), \quad \psi_{2n}(x) = \sqrt{2} \cos(2\pi nx).$$

Let the Fourier expansion of κ be

$$\forall x \in [0, 1], \quad \kappa(x) = \sum_{n=0}^{\infty} \hat{\kappa}_{2n} \psi_{2n}(x),$$

which involves only cosine terms because K symmetric forces κ even, hence $\hat{\kappa}_{2n+1} = 0$ for all $n \in \mathbb{N}$. Then every ψ_n is an eigenfunction of L_K , with eigenvalue $\hat{\kappa}_0$ for ψ_0 and eigenvalue $\hat{\kappa}_{2n}/\sqrt{2}$ for both ψ_{2n-1} and ψ_{2n} .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof sketch

That $(\psi_n)_{n \in \mathbb{N}}$ is an orthonormal basis of $L^2([0, 1])$ follows from the direct computation $\int_0^1 \psi_i(x) \psi_j(x) dx = \delta_{ij}$ together with the classical completeness of the trigonometric system. The key structural identities come from the trigonometric expansions of $\sin(a + b)$ and $\cos(a + b)$:

$$\begin{cases} \psi_{2n}(x - t) = \frac{1}{\sqrt{2}} [\psi_{2n}(x) \psi_{2n}(t) + \psi_{2n-1}(x) \psi_{2n-1}(t)], \\ \psi_{2n-1}(x - t) = \frac{1}{\sqrt{2}} [\psi_{2n-1}(x) \psi_{2n}(t) - \psi_{2n}(x) \psi_{2n-1}(t)]. \end{cases}$$

Then a direct computation, term by term in the (uniformly convergent) expansion of κ , gives for instance

$$\begin{aligned} L_K \psi_{2n}(x) &= \sum_{\ell=0}^{\infty} \hat{\kappa}_{2\ell} \int_0^1 \psi_{2\ell}(x - t) \psi_{2n}(t) dt \\ &= \sum_{\ell=0}^{\infty} \frac{\hat{\kappa}_{2\ell}}{\sqrt{2}} \int_0^1 [\psi_{2\ell}(x) \psi_{2\ell}(t) + \psi_{2\ell-1}(x) \psi_{2\ell-1}(t)] \psi_{2n}(t) dt \\ &= \sum_{\ell=0}^{\infty} \frac{\hat{\kappa}_{2\ell}}{\sqrt{2}} \psi_{2\ell}(x) \delta_{n\ell} = \frac{\hat{\kappa}_{2n}}{\sqrt{2}} \psi_{2n}(x), \end{aligned}$$

and similarly for ψ_{2n-1} and ψ_0 . $\square$

Remark (Mercer's expansion, verified)

Mercer's theorem can be checked by hand here. All ψ_k are continuous, and for any $x, t \in [0, 1]$ the Mercer expansion of the kernel collapses back onto κ via the addition formulas:

$$K(x, t) = \hat{\kappa}_0 + \sum_{n=1}^{\infty} \frac{\hat{\kappa}_{2n}}{\sqrt{2}} [\psi_{2n-1}(x) \psi_{2n-1}(t) + \psi_{2n}(x) \psi_{2n}(t)] = \sum_{n=0}^{\infty} \hat{\kappa}_{2n} \psi_{2n}(x - t) = \kappa(x - t),$$

with absolute and uniform convergence, because κ is continuous.

19.4.2 Polynomial decay of eigenvalues: Bernoulli kernels

Example (polynomial decay)

For any integer $\beta \geq 1$, choose the Fourier coefficients

$$\hat{\kappa}_0 = 0, \quad \hat{\kappa}_{2n} = \sqrt{2} n^{-2\beta} \text{ for } n \geq 1,$$

so that the Mercer eigenvalues decay polynomially, $\hat{\kappa}_{2n}/\sqrt{2} = n^{-2\beta}$. Then the corresponding kernel has a closed form in terms of Bernoulli polynomials:

$$\forall x, t \in [0, 1], \quad K(x, t) = \frac{(-1)^{\beta-1}}{(2\beta)!} B_{2\beta}(x - t - [x - t]),$$

where $B_{2\beta}$ is the (2β) -th Bernoulli polynomial, e.g., $B_2(x) = x^2 - x + 1/6$ and $B_4(x) = x^4 - 2x^3 + x^2 - 1/30$. The sign factor $(-1)^{\beta-1}$ is essential: the Fourier expansion of $B_{2\beta}(\{u\})$ carries the alternating sign $(-1)^{\beta-1}$, so it is exactly this factor that makes all the Fourier coefficients $\hat{\kappa}_{2n}$ positive, hence the kernel positive definite. Dropping it would give negative eigenvalues for even β , and the resulting kernel would not be positive definite. The proof is left as an exercise: check the Fourier expansion of the Bernoulli polynomials.

Verification artifact. checks/example-ch07-example-18-2.json records the example source hash and verification scope.

19.4.3 Exponential decay of eigenvalues

Example (exponential decay)

For any $\rho > 0$, choose

$$\hat{\kappa}_0 = 0, \quad \hat{\kappa}_{2n} = e^{-\rho n} \text{ for } n \geq 1,$$

so the eigenvalues decay geometrically. The corresponding kernel again has a closed form:

$$\forall x, t \in [0, 1], \quad K(x, t) = \sqrt{2} \frac{e^\rho \cos(2\pi(x-t)) - 1}{e^{2\rho} - 2e^\rho \cos(2\pi(x-t)) + 1}.$$

The verification is left as an exercise, by summing the geometric series of cosines, or see Bach (2013, p. 21).

Verification artifact. checks/example-ch07-example-18-3.json records the example source hash and verification scope.

Keep these two families in mind. At the end of the chapter, polynomial versus exponential decay of the spectrum will translate directly into slow versus fast convergence rates for kernel ridge regression.

Remark (choosing a kernel is choosing a spectral filter)

The periodic examples make vivid a reading of Mercer's theorem due to Shawe-Taylor and Cristianini (2004). Here the Fourier coefficients $\hat{\kappa}_{2n}$ are, up to the factor $\sqrt{2}$, the eigenvalues, so prescribing the kernel is the same as prescribing a weight on each frequency pair ψ_{2n-1}, ψ_{2n} . A coefficient that vanishes deletes that frequency from the feature space outright; a small coefficient keeps the feature but gives it little say in the fitted function; a large coefficient lets that frequency dominate. Choosing a kernel is thus choosing a filter with a prescribed spectral characteristic, governing how much each frequency may influence the learned function, which is the same bookkeeping as the RKHS norm $\sum_k a_k^2 / \lambda_k$ taxing a coordinate in inverse proportion to its eigenvalue. The Bernoulli and geometric spectra above are two such filters: a gentle polynomial roll-off and a sharp exponential cutoff.

19.5 Worked example 2: dot-product kernels on the sphere S^{d-1}

Our second example replaces Fourier modes by their spherical cousins. Consider the unit sphere in $\mathbb{R}^d$,

$$\mathcal{X} = S^{d-1} = \{x \in \mathbb{R}^d : \|x\| = 1\},$$

with ν the Lebesgue (surface) measure on S^{d-1} , whose total mass is

$$\nu(S^{d-1}) = \frac{2\pi^{\frac{d}{2}}}{\Gamma(\frac{d}{2})}.$$

Consider a positive definite kernel of dot-product form,

$$K(x, t) = \varphi(x^\top t),$$

where $\varphi : [-1, 1] \rightarrow \mathbb{R}$ is continuous. To write Mercer's expansion we must solve

$$\int_{S^{d-1}} \varphi(x^\top t) \psi(t) d\nu(t) = \lambda \psi(x).$$

The eigenfunctions turn out to be polynomial: they come from the polynomials that solve the Laplace equation

$$\Delta f = \frac{\partial^2 f}{\partial x_1^2} + \cdots + \frac{\partial^2 f}{\partial x_d^2} = 0,$$

where Δ is the Laplacian operator on $\mathbb{R}^d$.

19.5.1 Spherical harmonics

Definition (spherical harmonics)

- A homogeneous polynomial of degree $k \geq 0$ in $\mathbb{R}^d$ whose Laplacian vanishes is called a *homogeneous harmonic* of order k .
- A *spherical harmonic* of order k is the restriction of a homogeneous harmonic of order k to the unit sphere S^{d-1} .
- The set $\mathcal{Y}_k(d)$ of spherical harmonics of order k is a vector space of dimension

$$N(d, k) = \dim(\mathcal{Y}_k(d)) = \frac{(2k + d - 2)(k + d - 3)!}{k!(d - 2)!}.$$

19.5.2 The Funk-Hecke theorem

Spherical harmonics are the Mercer eigenfunctions of every dot-product kernel at once, by the following classical result (see, e.g., Muller, 1998, p. 30).

Theorem (Funk-Hecke)

For any $x \in S^{d-1}$, any $Y_k \in \mathcal{Y}_k(d)$, and any $\varphi \in C([-1, 1])$,

$$\int_{S^{d-1}} \varphi(x^\top t) Y_k(t) d\nu(t) = \lambda_k Y_k(x),$$

where

$$\lambda_k = \nu(S^{d-2}) \int_{-1}^1 \varphi(t) P_k(d; t) (1 - t^2)^{\frac{d-3}{2}} dt,$$

and $P_k(d; t)$ is the Legendre polynomial of degree k in dimension d .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

When $\varphi \in C^k([-1, 1])$, the eigenvalues can also be computed by the Rodrigues rule (Muller, 1998, p. 23):

$$\lambda_k = \nu(S^{d-2}) \frac{\Gamma(\frac{d-1}{2})}{2^k \Gamma(k + \frac{d-1}{2})} \int_{-1}^1 \varphi^{(k)}(t) (1-t^2)^{k+\frac{d-3}{2}} dt.$$

For each $k \geq 0$, let $\{Y_{k,j}(d; x)\}_{j=1}^{N(d,k)}$ be an orthonormal basis of $\mathcal{Y}_k(d)$. The collection of all spherical harmonics, over all orders k , forms an orthonormal basis of $L^2(S^{d-1})$. Therefore, for any kernel $K(x, t) = \varphi(x^\top t)$ on S^{d-1} , the Mercer eigenvalues are exactly the Funk-Hecke numbers λ_k , each with multiplicity $N(d, k)$ and orthonormal eigenfunctions $\{Y_{k,j}(d; x)\}_{j=1}^{N(d,k)}$. Note the remarkable structure: the eigenfunctions are the same for every φ ; only the eigenvalues change with the kernel. Choosing a dot-product kernel on the sphere is choosing a decay profile over the fixed ladder of spherical harmonics.

19.5.3 A fully worked case: the quadratic kernel on the circle

Example ($d = 2$, quadratic kernel)

Take $d = 2$ and $K(x, t) = (1 + x^\top t)^2$ for $x, t \in S^1$. Applying the Rodrigues rule to $\varphi(u) = (1 + u)^2$ yields exactly three nonzero eigenvalues:

$$\lambda_0 = 3\pi, \quad \lambda_1 = 2\pi, \quad \lambda_2 = \frac{\pi}{2},$$

with multiplicities 1, 2, and 2. The corresponding orthonormal eigenfunctions are

$$\frac{1}{\sqrt{2\pi}}, \quad \frac{x_1}{\sqrt{\pi}}, \quad \frac{x_2}{\sqrt{\pi}}, \quad \frac{2x_1x_2}{\sqrt{\pi}}, \quad \frac{x_1^2 - x_2^2}{\sqrt{\pi}},$$

the degree-two pair being the spherical harmonics $\sin 2\theta$ and $\cos 2\theta$ in angular coordinates. The resulting Mercer feature map $\Phi(x) = (\sqrt{\lambda_k} \psi_k(x))_k$ is

$$\Phi(x) = \left(\sqrt{\frac{3}{2}}, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1x_2, \frac{x_1^2 - x_2^2}{\sqrt{2}} \right),$$

and one checks directly (exercise, using $x_1^2 + x_2^2 = 1$ on S^1) that $\Phi(x)^\top \Phi(t) = (1 + x^\top t)^2 = K(x, t)$: the abstract Mercer map reproduces the explicit polynomial feature map of the quadratic kernel.

Verification artifact. checks/example-ch07-example-18-4.json records the example source hash and verification scope.

19.6 The RKHS built from the spectrum

Mercer's theorem decomposes the kernel over the eigenfunctions of the integral operator, and in favorable cases the resulting feature space is intuitive. But the kernel also has an RKHS, like any positive definite kernel, and the RKHS norm is what our learning algorithms penalize. Can we describe the RKHS, and above all its norm, in terms of the eigenfunctions and eigenvalues of the operator? The answer is a complete and remarkably clean description: the RKHS consists of the eigenfunction expansions whose coefficients decay fast enough relative to the spectrum.

Throughout this section, $\mathcal{X}$ is a compact metric space, ν a nondegenerate finite Borel measure, and K a Mercer kernel with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots$ (in decreasing order) and eigenfunctions $(\psi_1, \psi_2, \dots)$, so that by Mercer's theorem

$$K(x, y) = \sum_{k=1}^{\infty} \lambda_k \psi_k(x) \psi_k(y) = \langle \Phi(x), \Phi(y) \rangle_{\ell^2}, \quad \Phi(x) = \left(\sqrt{\lambda_k} \psi_k(x) \right)_{k \in \mathbb{N}}.$$

Here is the statement in words before the symbols. Write a candidate function in the eigenfunction coordinates, $f = \sum_i a_i \psi_i$. Its RKHS norm is not the plain ℓ^2 norm of the coefficients; it is a *reweighted* norm, $\sum_k a_k^2 / \lambda_k$, that divides each coordinate by its eigenvalue. Directions with a large λ_k are cheap, directions with a small λ_k are expensive, and a direction with $\lambda_k = 0$ is forbidden outright. The RKHS is exactly the set of functions whose coordinates decay fast enough for this reweighted sum to stay finite. This is the weighted-ellipsoid picture promised earlier: the unit ball of $\mathcal{H}$ is an ellipsoid with axes ψ_k and semi-axes $\sqrt{\lambda_k}$, stretched wide where the kernel is smooth and pinched to nothing where it is not. The theorem makes this precise, and the geometry is unpacked right after.

Theorem (RKHS from eigenfunctions)

Assume all eigenvalues λ_k are positive. Then the RKHS of K is the Hilbert space

$$\mathcal{H} = \left\{ f = \sum_{i=1}^{\infty} a_i \psi_i : \sum_{k=1}^{\infty} \frac{a_k^2}{\lambda_k} < \infty \right\},$$

endowed with the inner product

$$\langle f, g \rangle_{\mathcal{H}} = \sum_{k=1}^{\infty} \frac{a_k b_k}{\lambda_k}, \quad \text{for } f = \sum_k a_k \psi_k, \quad g = \sum_k b_k \psi_k.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Remark (zero eigenvalues)

If some eigenvalues vanish, the result and its proof remain valid on the subspace spanned by the eigenfunctions with positive eigenvalues: the RKHS is then the set of expansions over those eigenfunctions only, with the same norm.

Proof

As always we must show three things: that $\mathcal{H}$ is a Hilbert space of functions from $\mathcal{X}$ to $\mathbb{R}$, that $K_x \in \mathcal{H}$ for any $x \in \mathcal{X}$, and that $f(x) = \langle f, K_x \rangle_{\mathcal{H}}$ for any $x \in \mathcal{X}$ and $f \in \mathcal{H}$.

Step 1: $\mathcal{H}$ is a Hilbert space. The map

$$L_K^{\frac{1}{2}} : L_{\nu}^2(\mathcal{X}) \rightarrow \mathcal{H}, \quad \sum_{i=1}^{\infty} a_i \psi_i \mapsto \sum_{i=1}^{\infty} a_i \sqrt{\lambda_i} \psi_i$$

is a linear bijection that carries the L^2_ν inner product to $\langle \cdot, \cdot \rangle_{\mathcal{H}}$: it is an isomorphism of inner-product spaces. $\mathcal{H}$ is therefore complete, hence a Hilbert space, like $L^2_\nu(\mathcal{X})$.

Step 2: elements of $\mathcal{H}$ are bounded pointwise by their norm. For any $f = \sum_{i=1}^\infty a_i \psi_i \in \mathcal{H}$ and $x \in \mathcal{X}$, whenever the series defining $f(x)$ makes sense,

$$|f(x)| = \left| \sum_{i=1}^\infty a_i \psi_i(x) \right| = \left| \sum_{i=1}^\infty \frac{a_i}{\sqrt{\lambda_i}} \sqrt{\lambda_i} \psi_i(x) \right| \leq \left(\sum_{i=1}^\infty \frac{a_i^2}{\lambda_i} \right)^{\frac{1}{2}} \left(\sum_{i=1}^\infty \lambda_i \psi_i(x)^2 \right)^{\frac{1}{2}} = \|f\|_{\mathcal{H}} K(x, x)^{\frac{1}{2}} \leq \|f\|_{\mathcal{H}} \sqrt{C_K},$$

by Cauchy-Schwarz and Mercer's theorem. In particular the series converges absolutely at every x (so $f(x)$ is well defined), and convergence in $\|\cdot\|_{\mathcal{H}}$ implies uniform convergence of functions.

Step 3: $\mathcal{H}$ is a space of continuous functions. Let $f_n = \sum_{i=1}^n a_i \psi_i$. Each ψ_i is continuous, hence each f_n is continuous, and $f_n \rightarrow f$ in $\mathcal{H}$, hence uniformly by Step 2, hence in the complete space of continuous functions with the uniform norm. Let f_c denote the continuous limit. Then $f_c \in L^2_\nu(\mathcal{X})$ and $\|f_n - f_c\|_{L^2_\nu(\mathcal{X})} \rightarrow 0$. On the other hand, comparing coefficients,

$$\|f - f_n\|_{L^2_\nu(\mathcal{X})} \leq \sqrt{\lambda_1} \|f - f_n\|_{\mathcal{H}} \xrightarrow{n \rightarrow \infty} 0,$$

since $\|g\|_{L^2_\nu}^2 = \sum_i b_i^2 \leq \lambda_1 \sum_i b_i^2 / \lambda_i = \lambda_1 \|g\|_{\mathcal{H}}^2$ for $g = \sum_i b_i \psi_i \in \mathcal{H}$. By uniqueness of limits in L^2_ν , $f = f_c$: every element of $\mathcal{H}$ is (the everywhere-defined version of) a continuous function.

Step 4: $K_x \in \mathcal{H}$. Fix $x \in \mathcal{X}$ and set $a_i = \lambda_i \psi_i(x)$ for all i . Then

$$\sum_{i=1}^\infty \frac{a_i^2}{\lambda_i} = \sum_{i=1}^\infty \lambda_i \psi_i(x)^2 = K(x, x) < \infty,$$

so $\varphi_x := \sum_{i=1}^\infty a_i \psi_i \in \mathcal{H}$. Since convergence in $\mathcal{H}$ implies pointwise convergence (Step 2), for any $t \in \mathcal{X}$,

$$\varphi_x(t) = \sum_{i=1}^\infty a_i \psi_i(t) = \sum_{i=1}^\infty \lambda_i \psi_i(x) \psi_i(t) = K(x, t),$$

by Mercer's theorem, so $\varphi_x = K_x \in \mathcal{H}$.

Step 5: the reproducing property. Let $f = \sum_{i=1}^\infty a_i \psi_i \in \mathcal{H}$ and $x \in \mathcal{X}$. By Step 4, $K_x = \sum_{i=1}^\infty \lambda_i \psi_i(x) \psi_i$, so

$$\langle f, K_x \rangle_{\mathcal{H}} = \sum_{i=1}^\infty \frac{a_i \cdot \lambda_i \psi_i(x)}{\lambda_i} = \sum_{i=1}^\infty a_i \psi_i(x) = f(x),$$

which concludes the proof. $\square$

19.6.1 The geometry of the RKHS

Several consequences deserve to be spelled out.

- Although $\mathcal{H}$ was built from the eigenfunctions of L_K , which depend on the choice of the measure $d\nu(x)$, we know by uniqueness of the RKHS of a given kernel that $\mathcal{H}$ itself is independent of ν and of L_K . Only the coordinate system changes with ν .
- Mercer's theorem thus provides a concrete recipe for building the RKHS: take linear combinations of the eigenfunctions of L_K , with adequately decaying weights.
- The eigenfunctions $(\psi_i)_{i \in \mathbb{N}}$ form an orthogonal basis of the RKHS, with

$$\langle \psi_i, \psi_j \rangle_{\mathcal{H}} = 0 \text{ for } i \neq j, \quad \|\psi_i\|_{\mathcal{H}} = \frac{1}{\sqrt{\lambda_i}}.$$

Directions with small eigenvalues are expensive: representing a unit of ψ_i costs $1/\sqrt{\lambda_i}$ in RKHS norm.

- Summarizing the picture that has emerged: the unit ball of $\mathcal{H}$, seen inside $L^2_{\nu}(\mathcal{X})$, is a well-defined ellipsoid whose principal axes are the eigenfunctions ψ_k , with semi-axis lengths $\sqrt{\lambda_k}$. A rapidly decaying spectrum means a thin, strongly anisotropic ellipsoid: the kernel deems only a few directions cheap, and that is exactly what regularization exploits.

It is worth pausing on why this reweighting deserves to be called a smoothness measure, since that closes the circle with the Green functions of the first section. In every example of the chapter the eigenfunctions ψ_k grow more oscillatory as k increases, sines and cosines of rising frequency on the interval, spherical harmonics of rising degree on the sphere, while the eigenvalues λ_k decrease. The RKHS norm $\sum_k a_k^2/\lambda_k$ therefore multiplies the energy a_k^2 that a function places on the k -th oscillation by the large weight $1/\lambda_k$: a function pays little for its slow components and dearly for its fast ones, so a small RKHS norm forces the fast components to be small, which is exactly what it means to be smooth. The rate at which $\lambda_k \rightarrow 0$ fixes the exchange rate between frequency and cost, and hence how strongly the norm insists on smoothness. A slowly decaying spectrum barely penalizes high frequencies and admits rough functions; a fast decay penalizes them heavily and admits only very smooth ones. This is the quantitative content of the slogan that opened the chapter, that choosing a kernel means choosing which functions the norm considers simple.

19.6.2 Example: Sobolev spaces of periodic functions

Applied to the Bernoulli kernels of the first worked example, the construction identifies the RKHS as a classical Sobolev space.

Corollary (periodic Sobolev space)

For $\beta \in \mathbb{N}^*$, consider the Mercer kernel with polynomially decaying eigenvalues

$$\forall x, t \in [0, 1], \quad K(x, t) = \frac{(-1)^{\beta-1}}{(2\beta)!} B_{2\beta}(x - t - [x - t]),$$

where $B_{2\beta}$ is the (2β) -th Bernoulli polynomial and the sign factor $(-1)^{\beta-1}$ keeps the eigenvalues $n^{-2\beta}$ positive, as noted above. Then the RKHS is the set of functions $f : [0, 1] \rightarrow \mathbb{R}$ whose Fourier coefficients satisfy

$$\|f\|_{\mathcal{H}}^2 := \sum_{n=1}^{\infty} (\hat{f}_{2n-1}^2 + \hat{f}_{2n}^2) n^{2\beta} < +\infty.$$

This is the Sobolev space of functions f such that $f^{(i)}$ is absolutely continuous and $f^{(i)}(0) = f^{(i)}(1)$ for $i = 0, \dots, \beta - 1$, with

$$\|f\|_{\mathcal{H}}^2 = \pi^{-2\beta} \int_0^1 (f^{(\beta)}(x))^2 dx.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof sketch

The characterization by Fourier coefficients is a direct application of the RKHS construction theorem: the Fourier basis is an orthonormal basis of eigenfunctions of L_K , the eigenvalues are $n^{-2\beta}$ (for both ψ_{2n-1} and ψ_{2n}), and the RKHS norm is $\sum_k a_k^2/\lambda_k$. For the identification with the Sobolev space, use the Parseval equality to rewrite the Sobolev norm as the ℓ^2 norm of the Fourier coefficients of $f^{(\beta)}$, which are (roughly) the Fourier coefficients of f multiplied by n^β ; the periodic boundary conditions are what make the term-by-term differentiation of the Fourier series legitimate. For details, see Tsybakov (2009, Proposition 1.14). $\square$

This closes the loop opened at the start of the chapter: smoothness penalties are RKHS norms (Green functions), and RKHS norms are weighted spectral norms (Mercer). The kernel's spectrum is a complete dial for the smoothness the method enforces. We now show that the same dial controls statistical performance.

19.7 Convergence rates of kernel ridge regression

19.7.1 The isometry between $\mathcal{H}$ and $L_v^2(\mathcal{X})$

The map used in Step 1 of the RKHS construction proof,

$$L_K^{\frac{1}{2}} : L_v^2(\mathcal{X}) \rightarrow \mathcal{H}, \quad \sum_{i=1}^{\infty} a_i \psi_i \mapsto \sum_{i=1}^{\infty} a_i \sqrt{\lambda_i} \psi_i,$$

is an isometric isomorphism between $L_v^2(\mathcal{X})$ and $\mathcal{H}$:

$$\forall f \in L_v^2(\mathcal{X}), \quad \|f\|_{L_v^2(\mathcal{X})} = \|L_K^{\frac{1}{2}} f\|_{\mathcal{H}}, \quad \text{and conversely} \quad \forall f \in \mathcal{H}, \quad \|f\|_{\mathcal{H}} = \|L_K^{-\frac{1}{2}} f\|_{L_v^2(\mathcal{X})}.$$

This is more than a formal remark: it lets us compute $L_v^2(\mathcal{X})$ norms, the norms in which prediction error is measured, using RKHS theory, where the estimator has a closed form. That is precisely how we will analyze the performance of kernel ridge regression (KRR).

19.7.2 KRR and the statistical model

Recall the KRR estimator. Given $(x_1, \dots, x_n) \in \mathcal{X}^n$, $(y_1, \dots, y_n) \in \mathbb{R}^n$, and $\lambda > 0$,

$$\hat{f}_\lambda = \arg \min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2,$$

whose solution, by the representer theorem, is

$$\hat{f}_\lambda(x) = \sum_{i=1}^n \alpha_i K(x_i, x), \quad \text{where} \quad \alpha = (\mathbf{K} + \lambda n I)^{-1} \mathbf{y},$$

with $\mathbf{K}$ the kernel Gram matrix. We adopt the following model; every hypothesis is used in the proofs.

Model (assumptions for the analysis)

- K is a Mercer kernel over the compact set $\mathcal{X}$, and ν is a nondegenerate probability measure on $\mathcal{X}$ (so $\nu(\mathcal{X}) = 1$).
- $\lambda_1 \geq \lambda_2 \geq \dots \geq 0$ are the eigenvalues of L_K and $\{\psi_i, i \geq 1\}$ the eigenfunctions; we assume the eigenvalues positive, so that $\{\varphi_i = \sqrt{\lambda_i} \psi_i, i \geq 1\}$ is an orthonormal basis of $\mathcal{H}$. (Beware the notational collision, standard in this literature: λ is the regularization parameter, λ_i the eigenvalues.)
- (X, Y) is a pair of random variables with distribution P , where $X \in \mathcal{X}$ has distribution ν and

$$Y = f^*(X) + \epsilon, \quad f^* \in \mathcal{H}, \quad \epsilon \sim \mathcal{N}(0, \sigma^2),$$

with ϵ independent of X .

- $(x_i, y_i)_{i=1, \dots, n}$ are i.i.d. realizations of (X, Y) .

We estimate the performance of KRR in terms of the mean squared error

$$\text{MSE}(\hat{f}_\lambda) = \mathbb{E} (Y - \hat{f}_\lambda(X))^2,$$

the expectation being over both the training sample and an independent test pair (X, Y) . Note that $\text{MSE}(f^*) = \sigma^2$ is the best achievable error, so the quantity to control is the excess $\text{MSE}(\hat{f}_\lambda) - \text{MSE}(f^*)$.

19.7.3 The bias-variance decomposition of the excess MSE

The analysis becomes transparent in spectral coordinates. Write $f^* = \sum_{i \geq 1} \beta_i^* \varphi_i$ with $\beta^* \in \ell^2$, let Φ_n be the $n \times \infty$ matrix

$$\Phi_n = \left(\varphi_j(x_i) \right)_{1 \leq i \leq n, 1 \leq j < +\infty},$$

and let $T : \ell^2 \rightarrow \ell^2$ be the diagonal operator $T(a_1, a_2, \dots) = (\lambda_1 a_1, \lambda_2 a_2, \dots)$.

Lemma (bias-variance decomposition)

Under the model above, for any $\lambda > 0$,

$$\text{MSE}(\hat{f}_\lambda) - \text{MSE}(f^*) = B_\lambda + V_\lambda,$$

where, with $\varepsilon = (\varepsilon_1, \dots, \varepsilon_n)^\top$,

$$B_\lambda = \mathbb{E} \left\| T^{\frac{1}{2}} \left(I - (\Phi_n^\top \Phi_n + \lambda n I)^{-1} \Phi_n^\top \Phi_n \right) \beta^* \right\|_{\ell^2}^2, \quad V_\lambda = \mathbb{E} \left\| T^{\frac{1}{2}} (\Phi_n^\top \Phi_n + \lambda n I)^{-1} \Phi_n^\top \varepsilon \right\|_{\ell^2}^2.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

This is the classical decomposition of the excess MSE as "bias plus variance": B_λ measures how much the regularization shrinks the true coefficient vector β^* , and increases with λ ; V_λ measures how much noise the estimator absorbs, and decreases with λ . The optimal λ balances the two, and the balance point depends on the spectrum through T .

Proof

Step 1: reduce to an RKHS norm. Since ϵ is independent of X and of $\hat{f}_\lambda$ (which depends only on the training sample), and $\mathbb{E}\epsilon = 0$,

$$\text{MSE}(\hat{f}_\lambda) = \mathbb{E} (f^*(X) - \hat{f}_\lambda(X) + \epsilon)^2 = \mathbb{E} (f^*(X) - \hat{f}_\lambda(X))^2 + \mathbb{E}\epsilon^2 = \mathbb{E} \|f^* - \hat{f}_\lambda\|_{L^2_{\nu}(X)}^2 + \text{MSE}(f^*),$$

using that $X \sim \nu$ is independent of the training sample. By the isometry between $L^2_{\nu}(X)$ and $\mathcal{H}$,

$$\text{MSE}(\hat{f}_\lambda) - \text{MSE}(f^*) = \mathbb{E} \left\| L_K^{\frac{1}{2}} (f^* - \hat{f}_\lambda) \right\|_{\mathcal{H}}^2.$$

Step 2: coordinates. Since $\{\varphi_i = \sqrt{\lambda_i} \psi_i, i \geq 1\}$ is an orthonormal basis of $\mathcal{H}$, define the linear isomorphism

$$e : \mathcal{H} \rightarrow \ell^2, \quad f = \sum_{i \geq 1} a_i \varphi_i \mapsto (a_1, a_2, \dots)^\top, \quad \text{i.e.} \quad e(f)_i = \langle f, \varphi_i \rangle_{\mathcal{H}}.$$

In particular, for any $x \in \mathcal{X}$, $e(K_x) = (\varphi_1(x), \varphi_2(x), \dots)^\top$, because $\langle K_x, \varphi_i \rangle_{\mathcal{H}} = \varphi_i(x)$. In this basis L_K is the diagonal operator $T = \text{diag}(\lambda_1, \lambda_2, \dots)$: for any $f = \sum_{i \geq 1} a_i \varphi_i \in \mathcal{H}$, $e(L_K f) = T e(f) = (\lambda_1 a_1, \lambda_2 a_2, \dots)^\top$.

Step 3: the estimator in coordinates. By construction $\Phi_n = (e(K_{x_1}), \dots, e(K_{x_n}))^\top$. The representer expansion $\hat{f}_\lambda = \sum_{i=1}^n \alpha_i K_{x_i}$ translates to

$$e(\hat{f}_\lambda) = \sum_{i=1}^n \alpha_i e(K_{x_i}) = \Phi_n^\top \alpha.$$

Notice that

$$[\Phi_n \Phi_n^\top]_{ij} = \langle e(K_{x_i}), e(K_{x_j}) \rangle_{\ell^2} = \langle K_{x_i}, K_{x_j} \rangle_{\mathcal{H}} = K(x_i, x_j),$$

so $\Phi_n \Phi_n^\top = \mathbf{K}$, and $\alpha = (\mathbf{K} + \lambda n I)^{-1} \mathbf{y}$ becomes $\alpha = (\Phi_n \Phi_n^\top + \lambda n I)^{-1} \mathbf{y}$. Putting these together and using the matrix inversion lemma, in the form $\Phi_n^\top (\Phi_n \Phi_n^\top + \lambda n I)^{-1} = (\Phi_n^\top \Phi_n + \lambda n I)^{-1} \Phi_n^\top$,

$$e(\hat{f}_\lambda) = \Phi_n^\top (\Phi_n \Phi_n^\top + \lambda n I)^{-1} \mathbf{y} = (\Phi_n^\top \Phi_n + \lambda n I)^{-1} \Phi_n^\top \mathbf{y}.$$

Step 4: the data in coordinates. Let $\beta^* = (\beta_1^*, \beta_2^*, \dots)^\top = e(f^*)$. For any $x \in \mathcal{X}$, $f^*(x) = \langle f^*, K_x \rangle_{\mathcal{H}} = \langle \beta^*, e(K_x) \rangle_{\ell^2}$, so the model $y_i = f^*(x_i) + \epsilon_i$, $i = 1, \dots, n$, translates to

$$\mathbf{y} = \Phi_n \beta^* + \epsilon, \quad \epsilon = (\epsilon_1, \dots, \epsilon_n)^\top.$$

Step 5: conclusion. Combining Steps 3 and 4,

$$e(f^* - \hat{f}_\lambda) = \beta^* - (\Phi_n^\top \Phi_n + \lambda n I)^{-1} \Phi_n^\top (\Phi_n \beta^* + \epsilon) = \left(I - (\Phi_n^\top \Phi_n + \lambda n I)^{-1} \Phi_n^\top \Phi_n \right) \beta^* - (\Phi_n^\top \Phi_n + \lambda n I)^{-1} \Phi_n^\top \epsilon.$$

Since e is an isometry and $e(L_K^{1/2} g) = T^{1/2} e(g)$,

$$\mathbb{E} \left\| L_K^{\frac{1}{2}} (f^* - \hat{f}_\lambda) \right\|_{\mathcal{H}}^2 = \mathbb{E} \left\| T^{\frac{1}{2}} e(f^* - \hat{f}_\lambda) \right\|_{\ell^2}^2,$$

and expanding the square, the cross term vanishes because ε is independent of Φ_n with mean zero. What remains is exactly $B_\lambda + V_\lambda$. $\square$

19.7.4 A simplification: replacing $\Phi_n^\top \Phi_n$ by nT

The bias and variance depend on the data only through the random operator $\Phi_n^\top \Phi_n : \ell^2 \rightarrow \ell^2$. For large n , the law of large numbers identifies its limit: for any $i, j \geq 1$,

$$[\Phi_n^\top \Phi_n]_{ij} = \sum_{k=1}^n \varphi_i(x_k) \varphi_j(x_k) \approx n \langle \varphi_i, \varphi_j \rangle_{L^2_\nu(x)} = n \sqrt{\lambda_i \lambda_j} \langle \psi_i, \psi_j \rangle_{L^2_\nu(x)} = n \lambda_i \delta_{ij},$$

so $\Phi_n^\top \Phi_n \approx nT$. We now study the bias and the variance under the substitution $\Phi_n^\top \Phi_n = nT$, and write $\tilde{B}_\lambda$ and $\tilde{V}_\lambda$ for the corresponding approximations. The difference between B_λ and $\tilde{B}_\lambda$ (respectively V_λ and $\tilde{V}_\lambda$) can be studied rigorously and does not change much the main results below; see Dicker et al. (2015) for details.

19.7.5 Upper bounds on bias and variance

Theorem (spectral bounds on $\tilde{B}_\lambda$ and $\tilde{V}_\lambda$)

For any integer $J \geq 1$,

$$\tilde{B}_\lambda \leq \left(\frac{\lambda^2}{\lambda_J} + \lambda_{J+1} \right) \|f^*\|_{\mathcal{H}}^2, \quad \tilde{V}_\lambda \leq \frac{\sigma^2}{n} \left[J + \frac{\sum_{i=J+1}^{+\infty} \lambda_i}{4\lambda} \right].$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The free integer J splits the spectrum into a head, where the regularization barely bites but each direction contributes noise, and a tail, where the eigenvalues themselves are small. Both J and λ will be optimized in the corollary below, depending on the assumptions made on f^* and on the decrease of the λ_i .

Remark (effective dimension)

The head-tail split is a discrete stand-in for a single continuous spectral quantity, the *effective dimension* at regularization level λ ,

$$N(\lambda) = \text{tr}(\mathbf{K}(\mathbf{K} + n\lambda I)^{-1}) = \sum_{i \geq 1} \frac{\lambda_i}{\lambda_i + \lambda},$$

the finite-sample trace on the left agreeing with the operator eigenvalue form on the right once $\mathbf{K} \approx nT$. Each term $\lambda_i/(\lambda_i + \lambda)$ is close to one when $\lambda_i \gg \lambda$ and close to zero when $\lambda_i \ll \lambda$, so $N(\lambda)$ counts, softly, the eigenvalues above the threshold λ : it is the number of directions the estimator can actually resolve. A fast-decaying spectrum has a small $N(\lambda)$, and it is $N(\lambda)$, rather than the ambient dimension, that sets the size of the variance and therefore the learning rate. The bound $\tilde{V}_\lambda \leq \frac{\sigma^2}{n} [J + \frac{1}{4\lambda} \sum_{i>J} \lambda_i]$ is one explicit way of writing $\tilde{V}_\lambda \lesssim \sigma^2 N(\lambda)/n$.

Proof

Bias. With $T = \text{diag}(\lambda_i; i \geq 1)$ and $\Phi_n^\top \Phi_n = nT$, the operator in $\tilde{B}_\lambda$ is diagonal:

$$T^{\frac{1}{2}} \left(I - (\Phi_n^\top \Phi_n + \lambda n I)^{-1} \Phi_n^\top \Phi_n \right) = T^{\frac{1}{2}} \left(I - (T + \lambda I)^{-1} T \right) = \text{diag} \left(\frac{\lambda \sqrt{\lambda_i}}{\lambda + \lambda_i}; i \geq 1 \right),$$

and therefore, splitting at an arbitrary $J \geq 1$,

$$\tilde{B}_\lambda = \sum_{i=1}^J \frac{\lambda^2 \lambda_i}{(\lambda + \lambda_i)^2} (\beta_i^*)^2 + \sum_{i \geq J+1} \frac{\lambda^2 \lambda_i}{(\lambda + \lambda_i)^2} (\beta_i^*)^2.$$

For the first term, use $\lambda_i^2/(\lambda + \lambda_i)^2 \leq 1$ and $\lambda_i \geq \lambda_J$ for $i \leq J$:

$$\sum_{i=1}^J \frac{\lambda^2 \lambda_i}{(\lambda + \lambda_i)^2} (\beta_i^*)^2 = \sum_{i=1}^J \frac{\lambda^2}{\lambda_i} \frac{\lambda_i^2}{(\lambda + \lambda_i)^2} (\beta_i^*)^2 \leq \frac{\lambda^2}{\lambda_J} \sum_{i=1}^J (\beta_i^*)^2 \leq \frac{\lambda^2}{\lambda_J} \|\beta^*\|_{\ell^2}^2.$$

For the second term, use $\lambda^2/(\lambda + \lambda_i)^2 \leq 1$ and $\lambda_i \leq \lambda_{J+1}$ for $i \geq J+1$:

$$\sum_{i \geq J+1} \frac{\lambda^2 \lambda_i}{(\lambda + \lambda_i)^2} (\beta_i^*)^2 \leq \lambda_{J+1} \sum_{i \geq J+1} (\beta_i^*)^2 \leq \lambda_{J+1} \|\beta^*\|_{\ell^2}^2.$$

Since e is an isometry, $\|\beta^*\|_{\ell^2} = \|f^*\|_{\mathcal{H}}$, and adding the two bounds gives the claim for $\tilde{B}_\lambda$.

Variance. Using $T = \text{diag}(\lambda_i; i \geq 1)$, $\Phi_n^\top \Phi_n = nT$, and $\mathbb{E} \varepsilon \varepsilon^\top = \sigma^2 I$,

$$\begin{aligned} \tilde{V}_\lambda &= \mathbb{E} \left\| T^{\frac{1}{2}} (\Phi_n^\top \Phi_n + \lambda n I)^{-1} \Phi_n^\top \varepsilon \right\|_{\ell^2}^2 = \frac{1}{n^2} \mathbb{E} \left\| T^{\frac{1}{2}} (T + \lambda I)^{-1} \Phi_n^\top \varepsilon \right\|_{\ell^2}^2 \\ &= \frac{1}{n^2} \text{Trace} \left[T^{\frac{1}{2}} (T + \lambda I)^{-1} \Phi_n^\top \mathbb{E}(\varepsilon \varepsilon^\top) \Phi_n (T + \lambda I)^{-1} T^{\frac{1}{2}} \right] \\ &= \frac{\sigma^2}{n^2} \text{Trace} \left[T^{\frac{1}{2}} (T + \lambda I)^{-1} \Phi_n^\top \Phi_n (T + \lambda I)^{-1} T^{\frac{1}{2}} \right] \\ &= \frac{\sigma^2}{n} \text{Trace} \left[T^{\frac{1}{2}} (T + \lambda I)^{-1} T (T + \lambda I)^{-1} T^{\frac{1}{2}} \right] = \frac{\sigma^2}{n} \left[\sum_{i=1}^J \frac{\lambda_i^2}{(\lambda_i + \lambda)^2} + \sum_{i=J+1}^{+\infty} \frac{\lambda_i^2}{(\lambda_i + \lambda)^2} \right], \end{aligned}$$

where we used successively the definition of the trace, the independence of ε and Φ_n , and $\Phi_n^\top \Phi_n = nT$ once more, which absorbs one factor of n . For the first term, $\lambda_i^2/(\lambda_i + \lambda)^2 \leq 1$ gives

$$\sum_{i=1}^J \frac{\lambda_i^2}{(\lambda_i + \lambda)^2} \leq J.$$

For the second term, the function $t \mapsto t/(t + \lambda)^2$ on $[0, \infty)$ reaches its maximum at $t = \lambda$, where it equals $1/(4\lambda)$, so

$$\sum_{i=J+1}^{+\infty} \frac{\lambda_i^2}{(\lambda_i + \lambda)^2} = \sum_{i=J+1}^{+\infty} \lambda_i \frac{\lambda_i}{(\lambda_i + \lambda)^2} \leq \frac{\sum_{i=J+1}^{+\infty} \lambda_i}{4\lambda}.$$

Combining both terms gives the claim for $\tilde{V}_\lambda$. $\square$

19.7.6 Rates of convergence

Optimizing J and λ against the decay profile of the spectrum yields the rates. Recall that the excess risk is $\tilde{B}_\lambda + \tilde{V}_\lambda$, so a bound on their sum is a bound on how fast KRR approaches the best possible MSE.

Corollary (convergence rates of KRR)

- **Polynomial-decay kernels.** Suppose there are constants $C > 0$ and $s > 1$ such that $0 < \lambda_i \leq Ci^{-s}$ for $i = 1, 2, \dots$. Take $\lambda = n^{-\frac{s}{s+1}}$. Then

$$\tilde{B}_\lambda + \tilde{V}_\lambda \leq O\left(\left(\|f^*\|_{\mathcal{H}}^2 + \sigma^2\right) n^{-\frac{s}{s+1}}\right).$$

- **Exponential-decay kernels.** Suppose there are constants $C > 0$ and $\alpha > 0$ such that $0 < \lambda_i \leq Ce^{-\alpha i}$ for $i = 1, 2, \dots$. Take $\lambda = n^{-1} \log(n)$. Then

$$\tilde{B}_\lambda + \tilde{V}_\lambda \leq O\left(\left(\|f^*\|_{\mathcal{H}}^2 + \sigma^2\right) \frac{\log(n)}{n}\right).$$

- **Finite-rank kernels.** Suppose there is $J \geq 1$ such that $\lambda_J = \lambda_{J+1} = \dots = 0$. Take $\lambda = n^{-1}$. Then

$$\tilde{B}_\lambda + \tilde{V}_\lambda \leq O\left(\frac{\|f^*\|_{\mathcal{H}}^2 + \sigma^2 J}{n}\right).$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The three cases are best read as a single story about the spectrum: faster eigenvalue decay buys a faster rate. Polynomially decaying eigenvalues, $\lambda_i \sim i^{-s}$, give the algebraic rate $n^{-s/(s+1)}$; a larger s , meaning a smoother kernel with fewer cheap directions, pushes the exponent toward the parametric n^{-1} but never reaches it, which is the familiar price of nonparametric estimation. Exponentially decaying eigenvalues collapse the spectrum so quickly that the rate improves to $\log(n)/n$, within a logarithm of the parametric rate: such a kernel behaves almost like a finite-dimensional model. Finite-rank kernels, whose spectrum stops after J terms, are genuinely finite-dimensional and attain the parametric rate J/n . In each case the regularization λ is tuned to place the head-tail split J of the previous theorem exactly where bias and variance balance, and that balance point is dictated entirely by how fast the λ_i fall. The eigenvalue decay that we read in the first section as smoothness, and in the second as the shape of the RKHS ellipsoid, is here revealed as the single quantity that governs how fast learning is possible.

Proof for polynomial-decay kernels

For n large enough that $\lambda \leq \lambda_1$, let J be such that $\lambda_{J+1} \leq \lambda \leq \lambda_J$.

Bias. By the choice of J , $\lambda^2/\lambda_J \leq \lambda$ and $\lambda_{J+1} \leq \lambda$, so the bias bound of the theorem gives immediately

$$\tilde{B}_\lambda \leq 2\lambda \|f^*\|_{\mathcal{H}}^2 = 2n^{-\frac{s}{s+1}} \|f^*\|_{\mathcal{H}}^2.$$

Variance: bounding J . Since $\lambda \leq \lambda_J \leq CJ^{-s}$,

$$J \leq C^{\frac{1}{s}} \lambda^{-\frac{1}{s}} = C^{\frac{1}{s}} n^{\frac{1}{s+1}}.$$

Variance: bounding the tail sum. Let $J_0 = \lceil C^{1/s} n^{1/(s+1)} \rceil + 1 \geq J$. Splitting the tail at J_0 and comparing the far tail with an integral,

$$\sum_{i=J+1}^{+\infty} \lambda_i = \sum_{i=J+1}^{J_0} \lambda_i + \sum_{i=J_0+1}^{+\infty} \lambda_i \leq J_0 \lambda + C \int_{J_0}^{+\infty} t^{-s} dt \leq J_0 n^{-\frac{s}{s+1}} + \frac{C}{s-1} J_0^{1-s},$$

using $\lambda_i \leq \lambda_{J+1} \leq \lambda$ for $i \geq J+1$. Since $J_0 \leq C^{1/s} n^{1/(s+1)} + 1$ and $1 \leq n^{1/(s+1)}$,

$$J_0 n^{-\frac{s}{s+1}} \leq \left(C^{\frac{1}{s}} + 1\right) n^{\frac{1-s}{s+1}},$$

and since $J_0 \geq C^{1/s} n^{1/(s+1)}$,

$$\frac{C}{s-1} J_0^{1-s} \leq \frac{C^{\frac{1}{s}} n^{\frac{1-s}{s+1}}}{s-1}.$$

Therefore the tail sum is upper bounded by

$$\sum_{i=J+1}^{+\infty} \lambda_i \leq \frac{s}{s-1} \left(C^{\frac{1}{s}} + 1\right) n^{\frac{1-s}{s+1}}.$$

Variance: conclusion. Plugging both bounds into the variance bound of the theorem, with $\lambda = n^{-s/(s+1)}$,

$$\tilde{V}_\lambda = \frac{\sigma^2}{n} \left[J + \frac{\sum_{i=J+1}^{+\infty} \lambda_i}{4 n^{-\frac{s}{s+1}}} \right] \leq \frac{\sigma^2}{n} \left[C^{\frac{1}{s}} n^{\frac{1}{s+1}} + \frac{1}{4} \frac{s}{s-1} \left(C^{\frac{1}{s}} + 1\right) n^{\frac{1-s}{s+1}} n^{\frac{s}{s+1}} \right] \leq \sigma^2 \left(C^{\frac{1}{s}} \left(1 + \frac{s}{4(s-1)}\right) + \frac{1}{4} \right) n^{-\frac{s}{s+1}}.$$

Adding $\tilde{B}_\lambda$ and $\tilde{V}_\lambda$ gives the announced rate. $\square$

Proof sketch for exponential-decay kernels

We proceed similarly, with $\lambda = n^{-1} \log(n)$ and J such that $\lambda_{J+1} \leq \lambda \leq \lambda_J$. From $\lambda \leq \lambda_J \leq C e^{-\alpha J}$ we deduce $J \leq O(\log n)$. Using the threshold $J_0 = \lfloor \alpha^{-1} \log(n) \rfloor + 1$ and summing the geometric tail, we deduce $\sum_{i \geq J+1} \lambda_i \leq O(\log(n)^2/n)$, hence $\tilde{V}_\lambda \leq O(\sigma^2 \log(n)/n)$, while the bias satisfies $\tilde{B}_\lambda \leq 2\lambda \|f^*\|_{\mathcal{H}}^2 = O(\|f^*\|_{\mathcal{H}}^2 \log(n)/n)$ as before. Details are left as an exercise; see Dicker et al. (2015, Corollary 2). $\square$

Proof for finite-rank kernels

Here we use a simpler bound on the bias: since $t/(t+\lambda)^2 \leq 1/(4\lambda)$ for any $t \geq 0$, every term of $\tilde{B}_\lambda = \sum_i \lambda^2 \lambda_i (\lambda + \lambda_i)^{-2} (\beta_i^*)^2$ is at most $\frac{\lambda}{4} (\beta_i^*)^2$, so

$$\tilde{B}_\lambda \leq \frac{\lambda}{4} \|f^*\|_{\mathcal{H}}^2.$$

For the variance, $\lambda_i = 0$ for $i \geq J$, so at most J terms survive in the trace and the bound simplifies to

$$\tilde{V}_\lambda \leq \frac{\sigma^2 J}{n}.$$

Taking $\lambda = n^{-1}$ and summing these inequalities gives the result. $\square$

19.7.7 Optimality and adaptivity

- The same rates hold for the exact quantities $B_\lambda + V_\lambda$, without the substitution $\Phi_n^\top \Phi_n = nT$; see Dicker et al. (2015, Corollaries 1 to 4), whose proof we followed and adapted. The constants hidden in the big- O notation depend only on the kernel K and the measure $d\nu(x)$.
- These rates are minimax optimal (Caponnetto and De Vito, 2007): no estimator can uniformly improve on them over the corresponding classes of target functions.
- The polynomial case connects back to classical nonparametric statistics. For $s = 2q$, the ball of the RKHS sits inside $L^2_\nu(\mathcal{X})$ as a Sobolev-type class of functions that are $q-1$ times differentiable with absolutely continuous derivatives and $\|f^{(q)}\|_{L^2_\nu(\mathcal{X})} < +\infty$, as the periodic Sobolev example made explicit with eigenvalues $n^{-2\beta}$ and $q = \beta$. The rate $n^{-\frac{s}{s+1}} = n^{-\frac{2q}{2q+1}}$ is then exactly the standard optimal convergence rate of nonparametric regression (Tsybakov, 2009).
- The rate depends not only on the spectrum but on how smooth the target f^* is *relative to the kernel*. This is made precise by a *source condition*: one assumes $f^* = T^r g$ for some $g \in L^2_\nu(\mathcal{X})$ and an exponent $r > 0$, where T is the integral operator. A larger r means f^* lives deeper inside the range of T , hence its coordinates β_i^* decay faster relative to λ_i , and the achievable rate improves accordingly; the case treated above corresponds to $r = \frac{1}{2}$, for which $f^* \in \mathcal{H}$. The source-condition exponent and the eigenvalue-decay exponent together fix the minimax rate (Caponnetto and De Vito, 2007).
- Under additional assumptions on f^* , for instance requiring not only $\sum_{i \geq 1} (\beta_i^*)^2 < \infty$ but also $\sum_{i \geq 1} i^\tau (\beta_i^*)^2 < \infty$ for some $\tau > 0$, or $\beta_i^* = 0$ for $i > J$, one obtains faster convergence rates, which are again minimax optimal for the class of functions considered. In this sense KRR is *adaptive*: the same estimator, with a properly tuned λ , achieves the optimal rate over a range of regularity classes (Caponnetto and De Vito, 2007; Dicker et al., 2015).

19.8 The integral-operator minimax framework: source and capacity

The rates corollary tied the speed of learning to a single number, the eigenvalue-decay exponent, under the standing assumption $f^* \in \mathcal{H}$. That assumption quietly hides a second dial. Two targets can both lie in $\mathcal{H}$ yet be learnable at very different speeds, because what governs the bias is not merely that f^* has finite RKHS norm but how deep inside the smoothing action of the kernel it sits. The modern account, the *integral-operator approach* to minimax learning rates, separates these two effects cleanly: a *source condition* measures the smoothness of the target relative to the kernel, a *capacity condition* measures the richness of the kernel through its spectrum, and a single exponent formula built from the two governs the optimal rate. The error-estimate technique was developed by Smale and Zhou (2007), and its optimal-rate form is due to Caponnetto and De Vito (2007); the framework has since been sharpened by Steinwart, Hush, and Scovel (2009), Fischer and Steinwart (2020), and Blanchard and Mücke (2018). This section names the framework, states its central theorem, and reads off the saturation phenomenon that specifically limits kernel ridge regression. Both conditions are phrased through the integral operator $T = L_K$, diagonal in the eigenbasis with $T\psi_i = \lambda_i\psi_i$, and its spectral powers $T^r(\sum_i c_i\psi_i) = \sum_i \lambda_i^r c_i\psi_i$.

19.8.1 The source condition

How smooth is the target relative to the kernel? The chapter's RKHS is the range of $T^{1/2}$, the functions whose coordinates decay one half-power of the spectrum faster than square-summable. Demanding that f^* sit even deeper inside the range of T , in the range of a higher power T^r , is exactly a quantitative smoothness assumption, and it is the hypothesis under which fast rates are proved.

Definition (source condition)

The target f^* satisfies a *source condition* of order $r > 0$ with radius R if it lies in the range of the r -th power of the integral operator,

$$f^* = T^r g, \quad \|g\|_{L^2_\nu(x)} \leq R,$$

where T^r is defined by the spectral calculus. Equivalently, writing $f^* = \sum_i c_i \psi_i$ in the L^2_ν eigenbasis, the coordinates obey

$$\sum_{i \geq 1} \frac{c_i^2}{\lambda_i^{2r}} \leq R^2.$$

The larger the exponent r , the faster the coordinates c_i must decay relative to λ_i^r , so the deeper f^* sits inside the smoothed range of T and the smoother it is relative to the kernel. The threshold $r = \frac{1}{2}$ is precisely membership in the RKHS: since $\|f^*\|_{\mathcal{H}}^2 = \sum_i c_i^2 / \lambda_i$, the source condition at $r = \frac{1}{2}$ reads $\sum_i c_i^2 / \lambda_i \leq R^2$, that is, $f^* \in \mathcal{H}$ with $\|f^*\|_{\mathcal{H}} \leq R$. So $r = \frac{1}{2}$ recovers exactly the standing hypothesis of the rate analysis above, as the optimality remarks already anticipated; $r > \frac{1}{2}$ asks for a genuinely smoother target, while $r \in (0, \frac{1}{2})$ admits targets rougher than a generic RKHS element, the misspecified regime handled by Steinwart, Hush, and Scovel (2009) and by Fischer and Steinwart (2020) through an additional embedding condition on the eigenfunctions.

19.8.2 The capacity condition and the effective dimension

The source condition speaks about the target; the second dial speaks about the kernel and the space it lives on. It asks how many directions the estimator is forced to fit, and the right measure is the effective dimension already met in the variance analysis, now promoted to a hypothesis in its own right.

Definition (capacity condition)

The kernel satisfies a *capacity condition* with exponent $p \in (0, 1]$ if the effective dimension

$$N(\lambda) = \text{tr}(T(T + \lambda I)^{-1}) = \sum_{i \geq 1} \frac{\lambda_i}{\lambda_i + \lambda}$$

obeys $N(\lambda) \leq C \lambda^{-p}$ for all $\lambda > 0$ and some constant C .

A polynomially decaying spectrum $\lambda_i \asymp i^{-1/p}$, equivalently $\lambda_i \asymp i^{-s}$ with $s = 1/p$, gives exactly $N(\lambda) \asymp \lambda^{-p}$. The borderline $p = 1$ is nothing more than the trace-class bound $\sum_i \lambda_i < \infty$ proved for every Mercer kernel, which always yields $N(\lambda) \leq \lambda^{-1} \sum_i \lambda_i$; a smaller p means a faster-decaying spectrum, a smaller effective dimension, and therefore fewer directions carrying variance. This is the same reading as the effective-dimension remark of the Mercer section: $N(\lambda)$ counts, softly, the eigenvalues above the threshold λ , and the capacity exponent p is the growth rate of that count as $\lambda \rightarrow 0$. It is this exponent, not the ambient dimension, that fixes the size of the variance.

19.8.3 The minimax rate and the saturation of KRR

With both dials in place the optimal rate is a single closed expression in the two exponents.

Theorem (Caponnetto and De Vito, 2007)

Suppose the kernel satisfies the capacity condition with exponent $p \in (0, 1]$ and the target the source condition with exponent $r \in [\frac{1}{2}, 1]$ and radius R . Then kernel ridge regression with regularization $\lambda_n \asymp n^{-1/(2r+p)}$ has excess risk

$$\mathbb{E} \left\| \hat{f}_{\lambda_n} - f^* \right\|_{L^2_\nu(x)}^2 \leq C (R^2 + \sigma^2) n^{-\frac{2r}{2r+p}},$$

and the exponent $\frac{2r}{2r+p}$ is minimax optimal: no estimator attains a uniformly faster rate over the class of distributions meeting these two conditions.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The formula reproduces every rate of the corollary above as a special case. At $r = \frac{1}{2}$, the case $f^* \in \mathcal{H}$, and capacity $p = 1/s$ from a spectrum $\lambda_i \asymp i^{-s}$, the exponent is $\frac{2r}{2r+p} = \frac{1}{1+p} = \frac{s}{s+1}$, exactly the polynomial-decay rate $n^{-s/(s+1)}$ obtained earlier by hand. The framework only makes explicit what the hand computation left implicit, that the rate also depends on the smoothness of the target, and it is the product structure $2r/(2r+p)$ that shows how the two dependencies interlock.

The intuition is exactly the bias-variance split already computed, now read through the operator calculus. The kernel ridge estimator smooths the target by $g_\lambda(T) = (T + \lambda I)^{-1}$, so its bias is the smoothing residual applied to f^* ,

$$\text{bias}(\lambda) = \left\| \lambda(T + \lambda I)^{-1} f^* \right\|_{L^2_\beta(x)} = \left\| \lambda(T + \lambda I)^{-1} T^r g \right\|_{L^2_\beta(x)} \leq \left(\sup_{0 \leq t \leq \|T\|} \frac{\lambda t^r}{t + \lambda} \right) R.$$

For $0 < r \leq 1$ the scalar supremum is of order λ^r , attained near $t \asymp \lambda$, so $\text{bias}(\lambda)^2 \lesssim \lambda^{2r} R^2$: a larger source exponent shrinks the bias faster as $\lambda \rightarrow 0$. The variance is controlled by capacity, since the spectral bound of the previous section reads $\tilde{V}_\lambda \lesssim \sigma^2 N(\lambda)/n \lesssim \sigma^2 \lambda^{-p}/n$. Balancing the two,

$$\lambda^{2r} \asymp \frac{\lambda^{-p}}{n} \implies \lambda_n \asymp n^{-\frac{1}{2r+p}}, \quad \text{bias}^2 + \text{variance} \asymp n^{-\frac{2r}{2r+p}}.$$

The source exponent governs how fast the bias falls, the capacity exponent how slowly the variance rises, and the rate is the crossing point of the two.

Remark (saturation of KRR at $r = 1$)

The bias estimate conceals a ceiling. The scalar function $t \mapsto \lambda t^r/(t + \lambda)$ has order λ^r only while $r \leq 1$; for $r > 1$ its supremum over $t \in [0, \|T\|]$ is attained at the top of the spectrum and has order λ , not λ^r . The kernel ridge bias therefore *saturates*: $\text{bias}(\lambda)^2 \asymp \lambda^2$ for every $r \geq 1$, so the best achievable rate freezes at $n^{-2/(2+p)}$, the value already reached at $r = 1$, no matter how much smoother f^* is. The minimax rate $n^{-2r/(2r+p)}$ keeps improving past $r = 1$, so on very smooth targets plain kernel ridge regression is provably suboptimal. Recovering the faster rate requires a higher-qualification spectral regularizer, such as iterated Tikhonov, spectral cut-off, or gradient-flow (Landweber) learning, analyzed in one inverse-problems framework by Blanchard and Mücke (2018). The gap between the two exponents past $r = 1$ is the price of the simplest regularizer.

The whole mechanism is easiest to trust in numbers: a concrete spectrum, its measured effective dimension, and the rate exponent seen to plateau in the source exponent.

Example (effective dimension, rate exponent, and saturation)

Take the polynomial spectrum $\lambda_i = i^{-2b}$ of the integral operator T with $b = 1$, that is $\lambda_i = i^{-2}$, so the decay exponent is $s = 2b = 2$. The theory predicts capacity exponent $p = 1/(2b) = \frac{1}{2}$. We read off the effective dimension, the predicted kernel ridge rate exponent as a function of the source exponent r , and the saturation at $r = 1$. The noise level σ^2 enters only through constants.

1. Confirm trace class. The eigenvalues sum to $\sum_{i \geq 1} i^{-2} = 1.644934$, the value $\pi^2/6$, which is finite, so T is trace class and the analysis applies. The capacity exponent $p = 1$ would already follow from this bound alone; the point is to do better.

2. Compute the effective dimension. Summing $N(\lambda) = \sum_i \lambda_i / (\lambda_i + \lambda)$ at three scales,

$$N(10^{-2}) = 15.208, \quad N(10^{-3}) = 49.173, \quad N(10^{-4}) = 156.580,$$

each within half a unit of the asymptote $(\pi/2) \lambda^{-1/2}$, namely 15.708, 49.673, 157.080. A log-log fit of $N(\lambda)$ across $\lambda \in [10^{-2}, 10^{-6}]$ has slope -0.5032 , so the measured capacity exponent is $p \approx 0.50$, matching $1/(2b)$.

3. Read the rate exponent. With $p = \frac{1}{2}$, the kernel ridge exponent $\rho_{\text{KRR}}(r) = 2 \min(r, 1) / (2 \min(r, 1) + p)$ and the minimax exponent $\rho_{\min}(r) = 2r / (2r + p)$ are

r	$\rho_{\text{KRR}}(r)$	$\rho_{\min}(r)$
0.5	0.6667	0.6667
0.75	0.7500	0.7500
1.0	0.8000	0.8000
1.5	0.8000	0.8571
2.0	0.8000	0.8889
3.0	0.8000	0.9231

At $r = \frac{1}{2}$ this reproduces the chapter's polynomial rate $n^{-s/(s+1)} = n^{-2/3}$ for $f^* \in \mathcal{H}$. The two columns agree up to $r = 1$, then ρ_{KRR} sticks at $2/(2 + p) = 0.8$ while $\rho_{\min}$ keeps climbing.

4. See the saturation in the balance. At $n = 10^4$, tuning $\lambda_* = n^{-1/(2 \min(r, 1) + p)}$ and reading $\text{bias}^2 = \lambda_*^{2 \min(r, 1)}$ and $\text{variance} = N(\lambda_*)/n$: for $r = \frac{1}{2}$, $\lambda_* = 2.154 \times 10^{-3}$, $\text{bias}^2 = 2.154 \times 10^{-3}$, $\text{variance} = 3.334 \times 10^{-3}$; for $r = 1$, $\lambda_* = 2.512 \times 10^{-2}$, $\text{bias}^2 = 6.310 \times 10^{-4}$, $\text{variance} = 9.411 \times 10^{-4}$. The $r = 2$ target produces the same three numbers as $r = 1$, to every printed digit: kernel ridge regression cannot tell a twice-smoother target from a once-smoother one.

Reading. Two exponents, not one, set the rate: the capacity $p = \frac{1}{2}$, fixed by the spectrum, and the source r , fixed by the target. They combine as $2 \min(r, 1) / (2 \min(r, 1) + p)$, and the $\min(r, 1)$ is the fingerprint of saturation. Past $r = 1$ the kernel ridge estimator leaves speed on the table that a higher-qualification method would collect.

Verification artifact. `checks/example-ch07-example-18-5.json` records the example source hash and verification scope.

19.9 Beyond compactness: an outlook

Everything in this chapter leaned on two hypotheses: $\mathcal{X}$ compact and K continuous. Suppose now that $\mathcal{X}$ is not compact, for example $\mathcal{X} = \mathbb{R}^d$. Then the eigenvalue problem $\int_{\mathcal{X}} K(x, t) \psi(t) d\nu(t) = \lambda \psi(x)$ need not have a countable family of eigenvalues, and Mercer's theorem does not hold. For the most important class of kernels on $\mathbb{R}^d$, the translation invariant (or shift-invariant) kernels $K(x, y) = \varphi(x - y)$, Fourier transforms provide a convenient extension, and harmonic analysis brings kernels well beyond vector spaces, to groups and semigroups.

The simplest instance already displays the whole mechanism. Call a kernel $K : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{R}$ translation invariant (or shift-invariant) if it depends only on the difference of its arguments, $K(x, y) = a_{x-y}$ for some sequence $(a_n)_{n \in \mathbb{Z}}$, the sequence being called positive definite when the kernel is. A classical theorem of Herglotz states that a sequence $(a_n)_{n \in \mathbb{Z}}$ is positive definite if and only if it is the Fourier-Stieltjes transform of a positive measure $\mu \in M(\mathbb{T})$, the set of finite Borel measures on the torus $[0, 2\pi]$ with 0 and 2π identified. The countable spectrum of eigenvalues has become a spectral measure. The next chapter develops this theme: Herglotz on $\mathbb{Z}$, Bochner's theorem on $\mathbb{R}^d$, the RKHS of shift-invariant kernels, and the generalization to semigroups.

Further reading

The spectral picture of this chapter is developed at book length in two standard references. Schölkopf and Smola (2002), *Learning with Kernels*, treat Mercer's theorem, the integral operator, and the eigenfunction feature map in their Chapter 2, with the functional-analytic prerequisites, the spectral theorem, and the proof of Mercer's expansion collected in the mathematical appendix; their exposition is the natural complement to the operator vocabulary introduced here. Shawe-Taylor and Cristianini (2004), *Kernel Methods for Pattern Analysis*, cover the same ground from the properties-of-kernels angle in their Chapter 3, including Mercer's theorem and the eigen-decomposition of the Gram matrix as the finite shadow of the integral operator, and they are especially good on the data-driven, empirical version of the spectrum that our figure previews.

For the statistical half of the chapter, the convergence rates of kernel ridge regression, the comprehensive reference is Steinwart and Christmann (2008), *Support Vector Machines*, whose treatment of integral operators, effective dimension, and the interplay between eigenvalue decay and learning rates places the results of the final section in their full generality. The reader who wants the sharpest and most complete rate theory should read it alongside the primary sources cited in the text, Cucker and Smale (2002), Caponnetto and De Vito (2007), Bach (2013), and Tsybakov (2009).

19.10 Common mistakes and practical implications

For **Mercer's Theorem, Spectra, and Rates**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

19.11 Summary and further reading

This chapter established explain the central definitions and claims in Mercer's Theorem, Spectra, and Rates; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Mercer 1909), (Aronszajn 1950), (Cucker and Smale 2002).

19.12 Exercises

1. warm-up Write a function in eigenfunction coordinates, $f = \sum_i a_i \psi_i$, so that its RKHS norm is $\|f\|_{\mathcal{H}}^2 = \sum_i a_i^2 / \lambda_i$. Suppose the spectrum is $\lambda_i = i^{-2}$. For each of the following coefficient sequences, decide whether the function lies in the RKHS, and rank those that do from smallest to largest norm: (a) $a_i = 1$ only for $i = 1$ and 0 otherwise; (b) $a_i = 1$ only for $i = 10$ and 0 otherwise; (c) $a_i = 1/i$ for all i ; (d) $a_i = 1/i^2$ for all i . Then explain in one sentence why, if the spectrum had a zero eigenvalue $\lambda_m = 0$, any function with $a_m \neq 0$ would be excluded from $\mathcal{H}$ outright.
2. computation Take the finite space $\mathcal{X} = \{x_1, x_2\}$ with Gram matrix

$$\mathbf{K} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

Compute its eigenvalues $\lambda_1 \geq \lambda_2$ and an orthonormal pair of eigenvectors u_1, u_2 , and write $\mathbf{K} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$. Then read off the discrete Mercer feature map $\Phi(x_i) = (\sqrt{\lambda_k} [u_k]_i)_{k=1}^2$ and verify by hand that $\langle \Phi(x_1), \Phi(x_2) \rangle = K(x_1, x_2)$ and $\|\Phi(x_1)\|^2 = K(x_1, x_1)$.

3. computation On $[0, 1]$ with the Lebesgue measure, take the Bernoulli kernel with $\beta = 1$, whose Fourier eigenfunctions $\psi_{2n-1} = \sqrt{2} \sin(2\pi nx)$ and $\psi_{2n} = \sqrt{2} \cos(2\pi nx)$ carry the common eigenvalue $\lambda_{2n-1} = \lambda_{2n} = n^{-2}$. Consider

$$f(x) = \sqrt{2} \sin(2\pi x) + \frac{1}{4} \sqrt{2} \cos(4\pi x).$$

Express f in the eigenfunction coordinates $f = \sum_i a_i \psi_i$, identify the two nonzero coefficients together with their eigenvalues, and compute $\|f\|_{\mathcal{H}}^2 = \sum_i a_i^2 / \lambda_i$. Which of the two frequency components contributes more to the norm, and why?

4. proof On the circle S^1 , the quadratic kernel $K(x, t) = (1 + x^\top t)^2$ has, by the Funk-Hecke computation of the chapter, the five-coordinate Mercer feature map

$$\Phi(x) = \left(\sqrt{\frac{3}{2}}, \sqrt{2} x_1, \sqrt{2} x_2, \sqrt{2} x_1 x_2, \frac{x_1^2 - x_2^2}{\sqrt{2}} \right).$$

Verify directly that $\langle \Phi(x), \Phi(t) \rangle = (1 + x^\top t)^2 = K(x, t)$ for all $x, t \in S^1$. This is an instance of Mercer's expansion checked by hand: the abstract eigenfunction map reproduces the explicit polynomial kernel. Hint

Expand $(1 + x^\top t)^2 = 1 + 2(x_1 t_1 + x_2 t_2) + (x_1 t_1 + x_2 t_2)^2$ and match term by term. The degree-two part needs the constraint $x_1^2 + x_2^2 = 1$ and $t_1^2 + t_2^2 = 1$: use it to rewrite $1 + x_1^2 t_1^2 + x_2^2 t_2^2$ as $\frac{3}{2} + \frac{1}{2}(x_1^2 - x_2^2)(t_1^2 - t_2^2)$.

5. proof Let $\mathcal{X} = \{x_1, \dots, x_m\}$ be finite, so the kernel is the symmetric positive semidefinite Gram matrix $\mathbf{K}$, diagonalized as $\mathbf{K} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ with orthonormal eigenvectors and $\lambda_1 \geq \dots \geq \lambda_m \geq 0$. Set $\mathbf{B} = \mathbf{\Lambda}^{1/2} \mathbf{U}^\top$. Prove that $\mathbf{K} = \mathbf{B}^\top \mathbf{B}$, and deduce that the columns of $\mathbf{B}$ form an explicit feature map $\Phi(x_i) = (\sqrt{\lambda_k} [u_k]_i)_{k=1}^m$ with $K(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$. Explain in one or two sentences why this is exactly Mercer's expansion $K(x, t) = \sum_k \lambda_k \psi_k(x) \psi_k(t)$ with the sum truncated at m , the coordinate $[u_k]_i$ standing in for the eigenfunction value $\psi_k(x_i)$. Hint

$\mathbf{B}^\top \mathbf{B} = \mathbf{U} \mathbf{\Lambda}^{1/2} \mathbf{\Lambda}^{1/2} \mathbf{U}^\top = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top$. The (i, j) entry of $\mathbf{B}^\top \mathbf{B}$ is the inner product of the i -th and j -th columns of $\mathbf{B}$; reading the k -th entry of column i gives $\sqrt{\lambda_k} [u_k]_i$.

6. proof Suppose the spectrum decays polynomially, $0 < \lambda_i \leq C i^{-s}$ with $s > 1$. Using the spectral bounds of the chapter, choose the head-tail split J so that $\lambda_{J+1} \leq \lambda \leq \lambda_J$, and show first that the bias obeys $\tilde{B}_\lambda \leq 2\lambda \|f^*\|_{\mathcal{H}}^2$ and that $J = O(\lambda^{-1/s})$. Then treat the excess risk, up to constants and the lower-order tail term, as the univariate expression

$$\lambda \|f^*\|_{\mathcal{H}}^2 + \frac{\sigma^2}{n} \lambda^{-1/s},$$

minimize it over $\lambda > 0$, and conclude that the optimal choice is $\lambda \asymp n^{-\frac{s}{s+1}}$, giving the rate $n^{-\frac{s}{s+1}}$. Comment on what happens to the exponent as $s \rightarrow \infty$. Hint

Differentiate $g(\lambda) = \lambda + c n^{-1} \lambda^{-1/s}$ and set $g'(\lambda) = 0$: this gives $\lambda^{1+1/s} \asymp n^{-1}$, i.e. $\lambda \asymp n^{-s/(s+1)}$. Substitute back; both terms are then of order $n^{-s/(s+1)}$. Larger s pushes $\frac{s}{s+1}$ toward 1, the parametric rate, but never reaches it.

7. exploration Open the interactive Gram-matrix figure of the chapter, which assembles $\mathbf{K} = (K(x_i, x_j))_{ij}$ for a handful of points and shows both its heatmap and its spectrum. This finite Gram matrix is precisely the discrete integral operator L_K of the chapter, with the integral replaced by a sum over the sampled points. Widen and then narrow the kernel bandwidth while watching the two panels. Describe how the off-diagonal structure of the heatmap changes, and how the eigenvalue spectrum reshapes: which direction of the bandwidth makes the spectrum decay fast (few cheap directions, a thin RKHS ellipsoid) and which makes it decay slowly. Relate what you see to the statement that the eigenvalues shown are the finite-sample stand-ins for the λ_k of L_K .
8. challenge The effective dimension of a Mercer kernel measures how many coordinates of the feature map $\Phi(x) = (\sqrt{\lambda_k} \psi_k(x))_k$ genuinely matter. Assume the eigenfunctions are uniformly bounded, $\sup_x \psi_k(x)^2 \leq C$. Show that the diagonal tail is controlled by the spectral tail,

$$0 \leq K(x, x) - \sum_{k=1}^n \lambda_k \psi_k(x)^2 \leq C \sum_{k>n} \lambda_k,$$

so that truncating Φ at n coordinates reproduces the kernel on the diagonal within $C \sum_{k>n} \lambda_k$. Then estimate the number of coordinates $n(\varepsilon)$ needed to make this tail at most ε , for (a) polynomial decay $\lambda_k \leq C_0 k^{-s}$ with $s > 1$, and (b) exponential decay $\lambda_k \leq C_0 e^{-\alpha k}$. Which spectrum yields a genuinely low effective dimension? Hint

The lower bound is nonnegativity of the omitted terms; the upper bound is $\sum_{k>n} \lambda_k \psi_k(x)^2 \leq C \sum_{k>n} \lambda_k$. For (a) compare the tail with $\int_n^\infty t^{-s} dt \asymp n^{1-s}$, so $n(\varepsilon) \asymp \varepsilon^{-1/(s-1)}$; for (b) sum the geometric tail, $\sum_{k>n} e^{-\alpha k} \asymp e^{-\alpha n}$, so $n(\varepsilon) \asymp \alpha^{-1} \log(1/\varepsilon)$.

9. computation Keep the spectrum $\lambda_i = i^{-2}$ of the integral-operator worked example, whose capacity exponent is $p = \frac{1}{2}$. Suppose the target has L_ν^2 eigen-coordinates $c_i = \langle f^*, \psi_i \rangle_{L_\nu^2(x)} = i^{-3}$. (a) Using the coordinate form $\sum_i c_i^2 / \lambda_i^{2r} < \infty$ of the source condition, show that $f^* = T^r g$ holds exactly for $r < \frac{5}{4}$. (b) Since the source exponent can be taken up to but not including $\frac{5}{4} > 1$, the KRR bias saturates: give the KRR rate exponent $2 \min(r, 1) / (2 \min(r, 1) + p)$ and the minimax exponent $2r / (2r + p)$ as $r \rightarrow \frac{5}{4}$, and state which one KRR attains. (c) In one sentence, name the kind of regularizer that would close the gap. Hint
- In (a), $c_i^2 / \lambda_i^{2r} = i^{-6} / i^{-4r} = i^{4r-6}$, and $\sum_i i^{4r-6}$ converges iff $4r - 6 < -1$. In (b), KRR uses $\min(r, 1) = 1$, giving $2 / (2 + \frac{1}{2}) = 0.8$, while the minimax exponent tends to $\frac{5}{6} \approx 0.833$; the difference is exactly saturation. In (c), any method of qualification above 1, such as spectral cut-off, iterated Tikhonov, or gradient-flow (Landweber) learning.
10. proof Model the excess risk of KRR under a source condition of order r and a capacity condition of exponent p by the univariate surrogate

$$E(\lambda) = \lambda^{2r} R^2 + \frac{c \sigma^2}{n} \lambda^{-p}, \quad \lambda > 0,$$

with the bias term λ^{2r} valid for $0 < r \leq 1$. (a) Minimize E over λ , and show the optimum is $\lambda_* \asymp n^{-1/(2r+p)}$ with $E(\lambda_*) \asymp n^{-2r/(2r+p)}$, recovering the rate of the theorem. (b) Recall that the true bias equals R times $\sup_{0 \leq t \leq \|T\|} \lambda t^r / (t + \lambda)$. Explain why, for $r > 1$, this supremum has order λ rather than λ^r , so the bias term should read λ^2 and the rate is capped at $n^{-2/(2+p)}$ however large r is. Hint

In (a), set $E'(\lambda) = 2r \lambda^{2r-1} - p c \sigma^2 n^{-1} \lambda^{-p-1} = 0$, giving $\lambda^{2r+p} \asymp n^{-1}$; substitute back and both terms have order $n^{-2r/(2r+p)}$. In (b), differentiate to find $\frac{d}{dt} [\lambda t^r / (t + \lambda)] \propto (r-1)t + r\lambda$, which is positive for all $t \geq 0$ when $r > 1$, so the ratio is increasing and maximized at $t = \|T\|$, where $\lambda t^r / (t + \lambda) = O(\lambda)$.

20 Translation-Invariant, Semigroup, and Probabilistic Kernels

The Green and Mercer constructions of the previous chapter build a kernel from a differential operator or from an eigenexpansion. This chapter follows three further roads into the kernel jungle. The first exploits symmetry: when a kernel depends only on the difference of its arguments, the Fourier transform diagonalizes everything, and positive definiteness becomes a statement about a positive spectral measure (Herglotz on the integers, Bochner on the line). The second road generalizes that algebra from groups to semigroups, where the exponential is replaced by a semicharacter and integral representations still hold. The third road builds kernels directly out of probabilistic models of the data: the Fisher kernel, the mutual information kernel, and the marginalized kernel. In each case the goal is the same, to encode prior knowledge about the structure of the data into an inner product, and by the end we will have a toolbox rather than a single best kernel.

20.1 Translation-invariant kernels on the integers

Many kernels used in practice never look at the absolute location of a point, only at how far apart two points are. On a periodic signal, on a time series sampled at integer instants, or on any stationary process, the natural symmetry is a shift. We formalize this by asking that the kernel be unchanged when both arguments are translated by the same amount.

Definition (translation-invariant kernel on $\mathbb{Z}$)

A kernel $K : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{R}$ is called *translation invariant*, or *shift invariant*, if it depends only on the difference of its arguments:

$$\forall x, y \in \mathbb{Z}, \quad K(x, y) = a_{x-y}$$

for some sequence $\{a_n\}_{n \in \mathbb{Z}}$. Such a sequence is called *positive definite* when the corresponding kernel K is positive definite.

The question is then purely about sequences: which $\{a_n\}$ give a positive definite kernel? Because a shift is a group operation, the eigenfunctions of translation are the characters $t \mapsto e^{int}$, and the answer is a Fourier statement. It is the oldest theorem of this kind.

Theorem (Herglotz)

A sequence $\{a_n\}_{n \in \mathbb{Z}}$ is positive definite if and only if it is the Fourier-Stieltjes transform of a positive measure $\mu \in M(\mathbb{T})$, the set of finite Borel measures on the torus $\mathbb{T} = [0, 2\pi]$ with 0 and 2π identified.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

20.1.1 The Fourier-Stieltjes transform on the torus

To read Herglotz's theorem we need the transform it refers to. Let $\mathbb{T}$ be the torus, let $C(\mathbb{T})$ be the continuous functions on it, and let $M(\mathbb{T})$ be the finite complex Borel measures (a measure defined on all open sets). The reason measures appear at all is a duality: $M(\mathbb{T})$ can be identified with the dual space $C(\mathbb{T})^*$. By the Riesz representation theorem, every bounded linear functional $\psi : C(\mathbb{T}) \rightarrow \mathbb{C}$ is integration against some measure,

$$\psi(f) = \frac{1}{2\pi} \int_{\mathbb{T}} f(t) d\mu(t).$$

This is the tool we will use in both proofs below: build a functional, bound it, and read off the measure it represents.

Definition (Fourier-Stieltjes coefficients)

For $\mu \in M(\mathbb{T})$, the Fourier-Stieltjes coefficients of μ form the sequence

$$\forall n \in \mathbb{Z}, \quad \hat{\mu}(n) = \frac{1}{2\pi} \int_{\mathbb{T}} e^{-int} d\mu(t).$$

This extends the ordinary Fourier transform of an integrable function: taking $d\mu(t) = f(t) dt$ recovers the classical Fourier coefficients of f . Herglotz's theorem thus says that positive definite sequences are exactly the Fourier coefficients of positive measures, a perfect analogue of the fact that a positive definite kernel has a nonnegative spectrum.

20.1.2 Two examples

Example (diagonal kernel)

Take $\mu = dt$, Lebesgue measure on the torus. Then

$$a_n = \hat{\mu}(n) = \frac{1}{2\pi} \int_{\mathbb{T}} e^{int} dt = \begin{cases} 1 & n = 0, \\ 0 & \text{otherwise.} \end{cases}$$

The resulting kernel is $K(x, t) = \mathbf{1}(x = t)$, the identity on $\ell^2(\mathbb{Z})$. A flat spectrum gives no correlation between distinct points.

Verification artifact. checks/example-ch08-example-19-1.json records the example source hash and verification scope.

Example (constant kernel)

Take $\mu = 2\pi C \delta_0$ for a constant $C \geq 0$, a point mass at the origin. Then

$$a_n = \hat{\mu}(n) = C \int_{\mathbb{T}} e^{int} \delta_0(t) = C,$$

so $K(x, t) = C$ for all x, t . A single spectral atom gives a perfectly correlated, rank-one kernel. Between these two extremes, the shape of μ tunes how strongly nearby points are tied together.

Verification artifact. checks/example-ch08-example-19-2.json records the example source hash and verification scope.

20.1.3 Proof of Herglotz's theorem

The easy direction shows that any Fourier coefficient sequence of a positive measure is positive definite; the transform makes the quadratic form a manifest integral of a square.

Proof ($\Leftarrow$)

Let $a_n = \hat{\mu}(n)$ for a positive $\mu \in M(\mathbb{T})$. For any $n \in \mathbb{N}$, points $x_1, \dots, x_n \in \mathbb{Z}$, and scalars $z_1, \dots, z_n \in \mathbb{C}$,

$$\sum_{i=1}^n \sum_{j=1}^n z_i \bar{z}_j a_{x_i - x_j} = \frac{1}{2\pi} \sum_{i,j} z_i \bar{z}_j \int_{\mathbb{T}} e^{-i(x_i - x_j)t} d\mu(t) = \frac{1}{2\pi} \int_{\mathbb{T}} \left| \sum_{j=1}^n z_j e^{-ix_j t} \right|^2 d\mu(t) \geq 0,$$

using $e^{-i(x_i - x_j)t} = e^{-ix_i t} \overline{e^{-ix_j t}}$ to complete the square. Since μ is positive the integral is nonnegative. $\square$

The converse is the substantive half. Given a positive definite sequence we must manufacture the measure. The idea is to smooth the sequence with a Fejér-type triangular window, which produces an explicitly positive trigonometric density, and then to pass to a weak limit.

Proof ($\Rightarrow$)

Step 1: a positive trigonometric polynomial. Fix $t \in \mathbb{R}$ and $N \in \mathbb{N}$, and choose the finitely supported sequence $z_n = e^{int}$ for $|n| \leq N$ and $z_n = 0$ otherwise. Positive definiteness gives

$$0 \leq \sum_{k=-N}^N \sum_{l=-N}^N a_{k-l} z_k \bar{z}_l = \sum_{k=-N}^N \sum_{l=-N}^N a_{k-l} e^{i(k-l)t}.$$

Collecting terms by the difference $m = k - l$ and counting multiplicities gives

$$= \sum_{m=-2N}^{2N} (2N + 1 - |m|) a_m e^{imt} = (2N + 1) \sum_{m \in \mathbb{Z}} \max\left(0, 1 - \frac{|m|}{2N+1}\right) a_m e^{imt} =: (2N + 1) \sigma_{2N}(t).$$

Thus $\sigma_N(t) := \sum_m \max\left(0, 1 - \frac{|m|}{N+1}\right) a_m e^{imt}$ is nonnegative for even N .

Step 2: the approximating measures. Set $d\mu_N = \sigma_N(t) dt$, a positive measure. Its coefficients are the windowed sequence,

$$\hat{\mu}_N(n) = \frac{1}{2\pi} \sum_{j=-N}^N a_j \left(1 - \frac{|j|}{N+1}\right) \int_{\mathbb{T}} e^{i(n-j)t} dt = a_n \max\left(0, 1 - \frac{|n|}{N+1}\right).$$

Because $\sigma_N \geq 0$, the total mass equals the value of the functional on $f \equiv 1$,

$$\|\mu_N\|_{M(\mathbb{T})} = \int_{\mathbb{T}} \sigma_N(t) dt = \sum_{n=-N}^N a_n \left(1 - \frac{|n|}{N+1}\right) \int_{\mathbb{T}} e^{int} dt = a_0,$$

so the family $\{\mu_N\}$ is uniformly bounded in total variation by a_0 .

Step 3: convergence on trigonometric polynomials. For a trigonometric polynomial $P(t) = \sum_{k=-K}^K b_k e^{ikt}$, with $\hat{P}(n) = b_n$,

$$\lim_{N \rightarrow \infty} \int_{\mathbb{T}} P(t) d\mu_N(t) = \lim_{N \rightarrow \infty} \sum_{k=-K}^K a_k b_k \left(1 - \frac{|k|}{N+1}\right) = \sum_{k=-K}^K a_k b_k = \sum_{k \in \mathbb{Z}} a_k \overline{\hat{P}(k)},$$

because the window tends to 1 pointwise on the fixed finite support of P .

Step 4: Riesz representation. The map $\Psi(P) = \sum_k a_k \hat{P}(k)$ is therefore a linear functional on trigonometric polynomials with norm at most a_0 . Trigonometric polynomials are dense in $C(\mathbb{T})$, so Ψ extends to all of $C(\mathbb{T})$. By the Riesz representation theorem there is a measure $\mu \in M(\mathbb{T})$ with $\|\mu\|_{M(\mathbb{T})} \leq a_0$ and

$$\forall f \in C(\mathbb{T}), \quad \Psi(f) = \int_{\mathbb{T}} f(t) d\mu(t).$$

Taking $f(t) = e^{int}$ gives $\hat{\mu}(n) = \Psi(e^{int}) = a_n$. Finally μ is positive: if $f \geq 0$, then $\Psi(f) = \lim_n \Psi(P_n) \geq 0$ as a limit of the nonnegative quantities $\int P_n d\mu_N$. $\square$

20.2 Translation-invariant kernels on $\mathbb{R}^d$

The continuous analogue replaces the integer lattice by $\mathbb{R}^d$ and the coefficient sequence by a function of the difference vector. Everything that held on $\mathbb{Z}$ has a counterpart, with the discrete torus replaced by the frequency space $\mathbb{R}^d$. The result carries the name of Bochner.

Definition (translation-invariant kernel on $\mathbb{R}^d$)

A kernel $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is translation invariant if

$$\forall x, y \in \mathbb{R}^d, \quad K(x, y) = \varphi(x - y)$$

for some function $\varphi : \mathbb{R}^d \rightarrow \mathbb{R}$. Such a φ is positive definite when K is.

Translation invariance means the kernel reads only the gap $x - y$, never the absolute positions, so on the line a single profile φ fixes the similarity of every pair of points. Before any spectral theory, it is worth seeing what positive definiteness of φ actually demands: that whatever finite set of points we sample, the Gram matrix $K_{ij} = \varphi(x_i - x_j)$ they produce be positive semidefinite. The following widget lets you watch that matrix form for the standard profiles on a line and see how the bandwidth controls the range over which points stay correlated.

Theorem (Bochner)

A continuous function $\varphi : \mathbb{R}^d \rightarrow \mathbb{R}$ is positive definite if and only if it is the Fourier-Stieltjes transform of a symmetric and positive finite Borel measure $\mu \in M(\mathbb{R}^d)$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

It is worth pausing on what this statement means, because it is the organizing idea of the whole subject. Read the transform in the other direction: Bochner says that every admissible φ can be written as

$$\varphi(x - y) = \int_{\mathbb{R}^d} e^{-i\omega^\top(x-y)} d\mu(\omega) = \int_{\mathbb{R}^d} \cos(\omega^\top(x - y)) d\mu(\omega),$$

a superposition of pure waves, one for each frequency ω , each weighted by the nonnegative amount $d\mu(\omega)$ of spectral mass sitting at that frequency. The kernel is a chord and the measure μ is the recipe saying how loud each note is played. Positive definiteness then has a strikingly simple reading: it is exactly the requirement that no note be played with negative volume. A single wave $\cos(\omega^\top(x - y))$ is already positive definite, since it factors as a real feature paired with itself, and a nonnegative mixture of positive definite functions is positive definite again; Bochner's content is that these mixtures exhaust the class. The spectral measure is the frequency content of the kernel, and choosing a kernel is choosing which frequencies to trust.

A concrete pass-fail test makes the criterion vivid. Take the single cosine $\varphi(t) = \cos(\omega_0 t)$: its spectral measure is the pair of atoms $\frac{1}{2}(\delta_{\omega_0} + \delta_{-\omega_0})$, both nonnegative, so φ is positive definite, as completing the square in the

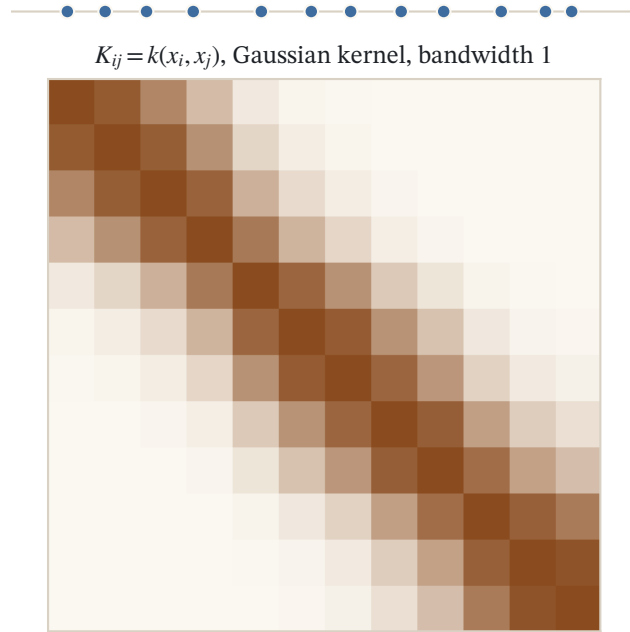


Figure 20.1: Twelve points on a line and their Gram matrix $K_{ij} = k(x_i, x_j)$ for a Gaussian kernel at bandwidth 1. A kernel is positive definite exactly when this matrix is positive semidefinite for every sample; narrowing the bandwidth drives it toward the identity.

sum $\sum_{i,j} z_i \bar{z}_j \cos(\omega_0(x_i - x_j))$ confirms directly. Now take the box $\varphi(t) = \mathbf{1}_{[-1,1]}(t)$, which looks harmless and even resembles a similarity that stays high for nearby points and drops for far ones. Its Fourier transform is $\hat{\varphi}(\omega) = 2 \sin(\omega)/\omega$, which is negative on $(\pi, 2\pi)$ and infinitely often beyond. Bochner's criterion is violated, so the box is not a kernel, and one can indeed exhibit a few points whose Gram matrix has a negative eigenvalue. The lesson is that the plausible-looking profile in the space variable is the wrong thing to inspect: the sign of its transform in the frequency variable is what decides admissibility.

This picture is not merely intuition, it is an algorithm. When μ is a probability measure the integral above is an expectation, so we can approximate it by drawing frequencies $\omega_1, \dots, \omega_D$ from μ and averaging the matching cosines: the Gaussian kernel appears as the average of a handful of random cosine features and sharpens toward the true kernel as D grows. That is precisely the random Fourier feature construction of Rahimi and Recht (2007), which is nothing but Monte Carlo sampling from the Bochner measure, and the widget below shows the average tightening onto the target as more features are drawn.

This single theorem underlies an enormous amount of practice. The Gaussian, Laplace, Matern, and sinc kernels are all Bochner transforms of familiar spectral densities, and, as just described, the random Fourier feature approximation samples the very measure μ that the theorem produces.

20.2.1 The Fourier-Stieltjes transform on $\mathbb{R}^d$

Let $C_0(\mathbb{R}^d)$ be the continuous functions vanishing at infinity and $M(\mathbb{R}^d)$ the finite complex Borel measures. As on the torus, Riesz identifies $M(\mathbb{R}^d)$ with the dual $C_0(\mathbb{R}^d)^*$: every bounded linear functional ψ is $\psi(f) = \int_{\mathbb{R}^d} f d\mu$.

Definition (Fourier-Stieltjes transform)

For $\mu \in M(\mathbb{R}^d)$, its Fourier-Stieltjes transform is

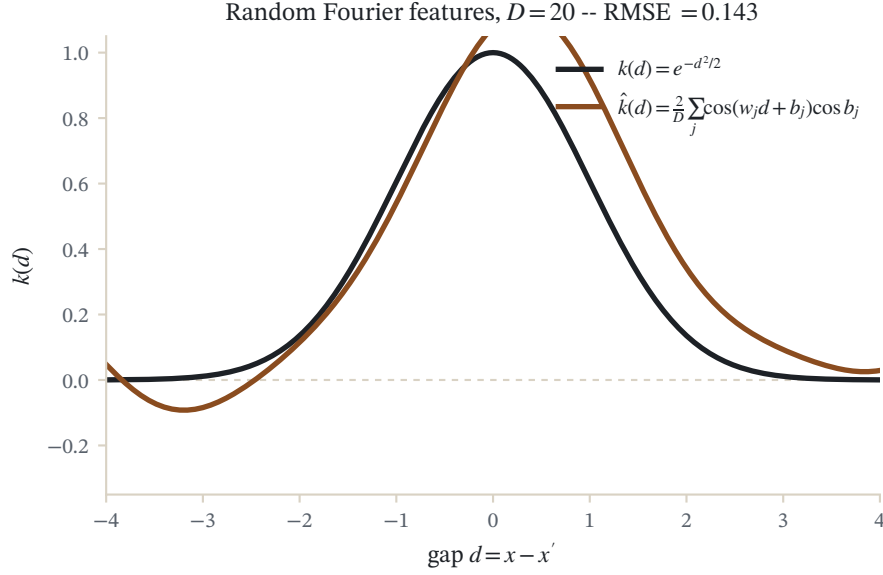


Figure 20.2: The Gaussian kernel $k(x, x') = e^{-(x-x')^2/2}$ (dark curve) against its $D = 20$ random Fourier feature estimate $\hat{k} = \frac{1}{D} \sum_j \cos(w_j x + b_j) \cos(w_j x' + b_j)$, as a function of the gap $x - x'$. Bochner's theorem makes the true kernel the average of these random features.

$$\forall \omega \in \mathbb{R}^d, \quad \hat{\mu}(\omega) = \int_{\mathbb{R}^d} e^{-i\omega^\top x} d\mu(x).$$

Taking $d\mu(x) = f(x) dx$ recovers the ordinary transform. A subtlety worth noting: for a general measure, $\hat{\mu}$ is uniformly continuous but need not vanish at infinity. For the Dirac $\mu = \delta_0$ we get $\hat{\mu}(\omega) = 1$ for all ω , which is exactly why the constant kernel reappears. The bridge between the space and frequency sides is Parseval's formula.

Parseval's formula

If $\mu \in M(\mathbb{R}^d)$ and both g and $\hat{g}$ lie in $L^1(\mathbb{R}^d)$, then

$$\int_{\mathbb{R}^d} g(x) d\mu(x) = \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} \hat{g}(\omega) \hat{\mu}(-\omega) d\omega.$$

In particular, if $g \in L^1(\mathbb{R}^d) \cap L^2(\mathbb{R}^d)$,

$$\int_{\mathbb{R}^d} g(x)^2 dx = \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} \hat{g}(\omega)^2 d\omega.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

20.2.2 Proof of Bochner's theorem

The easy direction is again a completion of the square inside the integral.

Proof ($\Leftarrow$)

If $\varphi = \hat{\mu}$ with μ positive, then for any points $x_1, \dots, x_n \in \mathbb{R}^d$ and scalars $z_1, \dots, z_n$,

$$\sum_{i,j} z_i \bar{z}_j \varphi(x_i - x_j) = \sum_{i,j} z_i \bar{z}_j \int_{\mathbb{R}^d} e^{-i(x_i - x_j)^\top t} d\mu(t) = \int_{\mathbb{R}^d} \left| \sum_j z_j e^{-ix_j^\top t} \right|^2 d\mu(t) \geq 0.$$

If μ is symmetric, then in addition φ is real-valued. $\square$

The converse reduces the continuous problem to the discrete one already solved: sample φ on a fine lattice $\lambda\mathbb{Z}$, apply Herglotz to the sampled sequence, and let the mesh shrink. The bookkeeping is done by a periodization of the test function and one application of Parseval.

Proof ($\Rightarrow$)

A representation lemma. Let $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ be continuous. If there is a constant $C \geq 0$ with

$$\frac{1}{2\pi} \int_{\mathbb{R}} \hat{g}(\xi) \varphi(-\xi) d\xi \leq C \sup_{x \in \mathbb{R}} |g(x)|$$

for every continuous $g \in L^1(\mathbb{R})$ whose transform $\hat{g}$ is continuous with compact support, then $\varphi = \hat{\mu}$ for some $\mu \in M(\mathbb{R})$. Indeed, let $\mathcal{G} \subset C_0(\mathbb{R})$ be the set of such g ; the map $\Psi(g) = \frac{1}{2\pi} \int \hat{g}(\xi) \varphi(-\xi) d\xi$ is linear and continuous on $\mathcal{G}$, extends to $C_0(\mathbb{R})$ by density, and Riesz produces $\mu \in M(\mathbb{R})$ with $\Psi(g) = \int g d\mu$. By Parseval this equals $\frac{1}{2\pi} \int \hat{g}(\xi) \hat{\mu}(-\xi) d\xi$ for all g , forcing $\varphi = \hat{\mu}$.

Reduction to $d = 1$. The argument for $d > 1$ is identical, so take $d = 1$. Let φ be continuous and positive definite. For any $\lambda > 0$ the sampled sequence $\{\varphi(n\lambda)\}_{n \in \mathbb{Z}}$ is positive definite, so Herglotz gives a positive $\mu_\lambda \in M(\mathbb{T})$ with $\varphi(\lambda n) = \hat{\mu}_\lambda(n)$ and $\|\mu_\lambda\|_{M(\mathbb{T})} = \hat{\mu}_\lambda(0) = \varphi(0)$.

Riemann sums. Fix $g \in \mathcal{G}$. For any $\epsilon > 0$ there is $\lambda > 0$ small enough that

$$\frac{1}{2\pi} \int_{\mathbb{R}} \hat{g}(\xi) \varphi(-\xi) d\xi < \frac{\lambda}{2\pi} \sum_{n \in \mathbb{Z}} \hat{g}(\lambda n) \varphi(-\lambda n) + \epsilon,$$

by approximating the integral with rectangles of width λ .

Periodization. Define on the torus $G_\lambda(t) = \sum_{m \in \mathbb{Z}} g\left(\frac{t+2\pi m}{\lambda}\right)$. By the decay of g , for small λ ,

$$\sup_{t \in \mathbb{T}} |G_\lambda(t)| \leq \sup_{x \in \mathbb{R}} |g(x)| + \epsilon.$$

A direct computation, unfolding the sum over m into one integral over $\mathbb{R}$, gives the sampled transform

$$\hat{G}_\lambda(n) = \frac{1}{2\pi} \int_{\mathbb{T}} e^{-int} G_\lambda(t) dt = \frac{\lambda}{2\pi} \int_{\mathbb{R}} e^{-in\lambda u} g(u) du = \frac{\lambda}{2\pi} \hat{g}(\lambda n).$$

Parseval on the torus. Therefore

$$\frac{\lambda}{2\pi} \sum_{n \in \mathbb{Z}} \hat{g}(\lambda n) \varphi(-\lambda n) = \sum_{n \in \mathbb{Z}} \hat{G}_\lambda(n) \hat{\mu}_\lambda(-n) = \frac{1}{2\pi} \int_{\mathbb{T}} G_\lambda(t) d\mu_\lambda(t) \leq \|\mu_\lambda\|_{M(\mathbb{T})} \sup_{t \in \mathbb{T}} |G_\lambda(t)| \leq C \sup_x |g(x)| + C\epsilon,$$

with $C = \varphi(0)$.

Conclusion. Combining the last two displays,

$$\frac{1}{2\pi} \int_{\mathbb{R}} \hat{g}(\xi) \varphi(-\xi) d\xi < C \sup_x |g(x)| + (C+1)\epsilon.$$

Letting $\epsilon \rightarrow 0$ gives the hypothesis of the lemma, so $\varphi = \hat{\mu}$ for some $\mu \in M(\mathbb{R})$. Moreover the identity $\frac{1}{2\pi} \int \hat{g} \varphi(-\cdot) = \int g d\mu$ is, for $g \geq 0$, approximated by $\frac{1}{2\pi} \int_{\mathbb{T}} G_{\lambda} d\mu_{\lambda} \geq 0$ since each μ_{λ} is positive and $G_{\lambda} \geq 0$; hence μ is a positive measure. $\square$

20.3 The RKHS of a translation-invariant kernel

Bochner tells us which φ are admissible; the next question is what function space the kernel generates. For a translation-invariant kernel the answer is beautifully explicit: the RKHS is a Fourier space in which each frequency is penalized by the reciprocal of the spectral density. Frequencies where $\hat{\varphi}$ is small are expensive, so the kernel enforces smoothness by starving the high frequencies.

Theorem (RKHS of a translation-invariant kernel)

Let $K(x, t) = \varphi(x - t)$ be a translation-invariant positive definite kernel with φ integrable on $\mathbb{R}^d$ together with its Fourier transform $\hat{\varphi}$. The subset $\mathcal{H}$ of $L^2(\mathbb{R}^d)$ consisting of the integrable and continuous functions f with

$$\|f\|_{\mathcal{H}}^2 := \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} \frac{\hat{f}(\omega)^2}{\hat{\varphi}(\omega)} d\omega < +\infty,$$

endowed with the inner product

$$\langle f, g \rangle_{\mathcal{H}} := \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} \frac{\hat{f}(\omega) \overline{\hat{g}(\omega)}}{\hat{\varphi}(\omega)} d\omega,$$

is a RKHS with K as reproducing kernel.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

That $\mathcal{H}$ is a Hilbert space is left as an exercise. Fix $x \in \mathbb{R}^d$ and consider the section $K_x(y) = K(x, y) = \varphi(x - y)$. Its transform is

$$\hat{K}_x(\omega) = \int e^{-i\omega^\top u} \varphi(u - x) du = e^{-i\omega^\top x} \hat{\varphi}(\omega).$$

Then $K_x \in \mathcal{H}$, because

$$\frac{1}{(2\pi)^d} \int \frac{|\hat{K}_x(\omega)|^2}{\hat{\varphi}(\omega)} d\omega = \frac{1}{(2\pi)^d} \int |\hat{\varphi}(\omega)| d\omega < \infty.$$

Finally, for $f \in \mathcal{H}$ and $x \in \mathbb{R}^d$, the reproducing property holds by inverse Fourier transform:

$$\langle f, K_x \rangle_{\mathcal{H}} = \frac{1}{(2\pi)^d} \int \frac{\hat{f}(\omega) \overline{\hat{K}_x(\omega)}}{\hat{\varphi}(\omega)} d\omega = \frac{1}{(2\pi)^d} \int \hat{f}(\omega) e^{i\omega^\top x} d\omega = f(x).$$

□

20.3.1 Reading off the smoothness class

The theorem turns kernel choice into spectral filter design. Three standard examples show the range.

Example (Gaussian kernel)

The kernel $K(x, y) = e^{-(x-y)^2/2\sigma^2}$ comes from $\varphi(t) = e^{-t^2/2\sigma^2}$ with transform $\hat{\varphi}(\omega) = e^{-\sigma^2\omega^2/2}$, so

$$\mathcal{H} = \left\{ f : \int \hat{f}(\omega)^2 e^{\sigma^2\omega^2/2} d\omega < \infty \right\}.$$

The penalty $e^{\sigma^2\omega^2/2}$ grows faster than any polynomial, so every function in the Gaussian RKHS is infinitely differentiable with all derivatives in L^2 . This is the analytic smoothness that makes the Gaussian kernel so strong a regularizer.

Verification artifact. checks/example-ch08-example-19-3.json records the example source hash and verification scope.

Example (Laplace kernel)

The kernel $K(x, y) = \frac{1}{2}e^{-\gamma|x-y|}$ comes from $\varphi(t) = \frac{1}{2}e^{-\gamma|t|}$ with $\hat{\varphi}(\omega) = \frac{\gamma}{\gamma^2 + \omega^2}$, giving

$$\mathcal{H} = \left\{ f : \int \hat{f}(\omega)^2 \frac{\gamma^2 + \omega^2}{\gamma} d\omega < \infty \right\}.$$

The penalty grows only quadratically, so this is a Sobolev norm: $\mathcal{H}$ is the space of L^2 functions with one derivative in L^2 . A much larger, rougher space than the Gaussian one.

Verification artifact. checks/example-ch08-example-19-4.json records the example source hash and verification scope.

Example (low-frequency filter)

The sinc kernel $K(x, y) = \frac{\sin(\Omega(x-y))}{\pi(x-y)}$ comes from $\varphi(t) = \frac{\sin(\Omega t)}{\pi t}$, whose transform is the indicator $\hat{\varphi}(\omega) = \mathbf{1}_{[-\Omega, \Omega]}(\omega)$. Here $\hat{\varphi}$ vanishes outside the band, so the norm forces $\hat{f}$ to vanish there too:

$$\mathcal{H} = \left\{ f : \int_{|\omega| > \Omega} \hat{f}(\omega)^2 d\omega = 0 \right\},$$

the space of functions whose spectrum is contained in $[-\Omega, \Omega]$. The kernel is a hard bandlimiting projector.

Verification artifact. checks/example-ch08-example-19-5.json records the example source hash and verification scope.

20.3.2 The Matern family

The Laplace and Gaussian kernels sit at opposite ends of a single knob. The Laplace RKHS is a first-order Sobolev space, rough and forgiving; the Gaussian RKHS is analytic, smooth to a fault. It is natural to want the whole range in between, and the Matern family supplies exactly that, with a smoothness parameter $\nu > 0$ that dials continuously from one extreme to the other. Writing $r = \|x - y\|$ for the distance and $\ell > 0$ for a length scale, the kernel is

$$k_\nu(r) = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} r}{\ell} \right)^\nu K_\nu \left(\frac{\sqrt{2\nu} r}{\ell} \right),$$

where K_ν is the modified Bessel function of the second kind. Its Bochner spectral density decays only polynomially,

$$\hat{\phi}(\omega) \propto \left(\frac{2\nu}{\ell^2} + \|\omega\|^2 \right)^{-(\nu+d/2)},$$

so the reciprocal-density penalty of the RKHS theorem grows like $\|\omega\|^{2\nu+d}$: the Matern RKHS is the Sobolev space of order $\nu + d/2$, and ν is literally the number of mean-square derivatives the kernel permits.

The two familiar kernels are the endpoints. At $\nu = \frac{1}{2}$ the Bessel function collapses and $k_{1/2}(r) = e^{-r/\ell}$, the exponential (Laplace) kernel with its single Sobolev derivative. As $\nu \rightarrow \infty$ the density tends to a Gaussian and $k_\nu(r) \rightarrow e^{-r^2/2\ell^2}$, recovering the analytic Gaussian kernel. The half-integer values $\nu = p + \frac{1}{2}$ are the ones used in practice, because there the Bessel function reduces to an exponential times a polynomial of degree p :

$$k_{3/2}(r) = \left(1 + \frac{\sqrt{3} r}{\ell} \right) e^{-\sqrt{3} r/\ell}, \quad k_{5/2}(r) = \left(1 + \frac{\sqrt{5} r}{\ell} + \frac{5r^2}{3\ell^2} \right) e^{-\sqrt{5} r/\ell}.$$

These closed forms make $\nu = \frac{3}{2}$ and $\nu = \frac{5}{2}$ the default choices when one wants a controlled amount of smoothness rather than the extremes. Stein (1999) argues from spatial-statistics practice that fixing the Gaussian's infinite smoothness in advance is rarely justified and that estimating ν is the safer default, and Rasmussen and Williams (2006) develop the family as the standard covariance for Gaussian-process regression.

20.3.3 A recap on the Green, Mercer, and Bochner families

The last three chapters have produced the same RKHS object from three different starting points. It is worth seeing them side by side, since each answers the question "what does $\|f\|_{\mathcal{H}}$ measure?" in its own vocabulary. Up to the technical assumptions specific to each row, we have the following correspondence.

Family	Kernel	RKHS $\mathcal{H}$	Squared norm $\ f\ _{\mathcal{H}}^2$
Green	Green function of D^*D	$L^2(\mathcal{X})$ with $\langle Df, Dg \rangle_{L^2(\mathcal{X})}$	$\ Df\ _{L^2(\mathcal{X})}^2$
Mercer	$\sum_{k=1}^{\infty} \lambda_k \psi_k(x) \psi_k(y)$	$\{f = \sum_k a_k \psi_k : \sum_k a_k^2 / \lambda_k < \infty\}$	$\sum_{k=1}^{\infty} a_k^2 / \lambda_k$
Fourier (Bochner)	$\kappa(x - y)$	continuous, integrable $f \in L^2(\mathbb{R}^d)$ with $\int \hat{f}(\omega) ^2 / \hat{\kappa}(\omega) d\omega < \infty$	$\frac{1}{(2\pi)^d} \int \frac{ \hat{f}(\omega) ^2}{\hat{\kappa}(\omega)} d\omega$

The pattern is identical in all three rows: a spectral quantity (the eigenvalue λ_k of an operator, or the spectral density $\hat{\kappa}(\omega)$) appears in the denominator, so that directions with little spectral energy are heavily penalized. Smoothness, in every guise, is small energy at high frequency.

20.4 Parametric spectral families: Matern and spectral mixtures

Bochner's theorem turns the design of a stationary kernel into the choice of a positive spectral density: decide how much mass to put at each frequency and the kernel follows. Two choices dominate practice, and they sit at opposite ends of a spectrum of ambition. The first fixes a rigid, single-peaked spectral shape with one tunable smoothness knob, the Matern family that the RKHS discussion above named as the bridge from Laplace to Gaussian; we now derive its spectrum straight from Bochner and pin down exactly how the knob controls smoothness. The second refuses to fix the shape at all and fits the spectral density itself as a Gaussian mixture, the spectral mixture kernel. A closing subsection then lets the spectral density vary from place to place, which breaks stationarity while keeping positive definiteness.

20.4.1 The Matern spectrum and mean-square smoothness

The brief treatment above located the Matern RKHS as a Sobolev space and named the two endpoints. Here we make the account systematic, starting from the object Bochner cares about, the spectral density, and reading the smoothness parameter off it. Writing $r = \|x - y\|$ for the distance and $\ell > 0$ for the length scale, the family is due to Matern (1960).

Definition (Matern kernel and spectrum, Matern 1960)

For smoothness $\nu > 0$ and length scale $\ell > 0$, the Matern kernel on $\mathbb{R}^d$ is the radial function

$$k_\nu(r) = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} r}{\ell} \right)^\nu K_\nu \left(\frac{\sqrt{2\nu} r}{\ell} \right),$$

with K_ν the modified Bessel function of the second kind, normalized so that $k_\nu(0) = 1$. At the half-integers $\nu = p + \frac{1}{2}$ it collapses to a degree- p polynomial times a decaying exponential; the first three are

$$k_{1/2}(r) = e^{-r/\ell}, \quad k_{3/2}(r) = \left(1 + \frac{\sqrt{3}r}{\ell}\right) e^{-\sqrt{3}r/\ell}, \quad k_{5/2}(r) = \left(1 + \frac{\sqrt{5}r}{\ell} + \frac{5r^2}{3\ell^2}\right) e^{-\sqrt{5}r/\ell}.$$

Its Bochner spectral density on $\mathbb{R}^d$ is, up to the positive constant fixed by $k_\nu(0) = 1$,

$$\hat{k}_\nu(\omega) \propto \left(\frac{2\nu}{\ell^2} + \|\omega\|^2 \right)^{-(\nu+d/2)},$$

a radial power law. Normalizing it to unit value at the origin strips the constant and exposes the exact shape,

$$\frac{\hat{k}_\nu(\omega)}{\hat{k}_\nu(0)} = \left(1 + \frac{\|\omega\|^2 \ell^2}{2\nu} \right)^{-(\nu+d/2)},$$

which is the density of a multivariate Student-t distribution with 2ν degrees of freedom.

The spectral density is not an afterthought to be verified: it is what constructs the kernel. Read Bochner backwards. Fix $\alpha = \sqrt{2\nu}/\ell$ and the exponent $s = \nu + d/2$, and start from the pure power law

$$S(\omega) = (\alpha^2 + \|\omega\|^2)^{-s}, \quad s > \frac{d}{2}.$$

This function is positive, symmetric under $\omega \mapsto -\omega$, and, because the radial integrand decays like $\|\omega\|^{-2s+d-1}$ with $2s > d$, integrable on $\mathbb{R}^d$. It is therefore a legitimate finite symmetric spectral measure, so Bochner promises that its inverse Fourier transform is a real positive definite stationary kernel with no further checking. That inverse transform is a classical Bessel-potential identity: for $\alpha > 0$ and $s > d/2$,

$$\int_{\mathbb{R}^d} (\alpha^2 + \|\omega\|^2)^{-s} e^{i\omega^\top \tau} d\omega = C_{d,s} \alpha^{d-2s} (\alpha\|\tau\|)^{s-d/2} K_{s-d/2}(\alpha\|\tau\|),$$

with $C_{d,s} = \pi^{d/2} 2^{1-s+d/2} / \Gamma(s) > 0$ a constant depending only on d and s . Substituting $s - d/2 = \nu$, the right-hand side is exactly $(\alpha\|\tau\|)^\nu K_\nu(\alpha\|\tau\|)$ up to a constant, which is the Matern function; requiring $k_\nu(0) = 1$ fixes the remaining scale. So the Matern kernel is not a shape someone guessed and then proved admissible, it is by definition the inverse Fourier transform of a power-law spectrum, and admissibility is built in.

The Student-t reading is the useful one. The tail index is ν : the density decays only as the power law $\|\omega\|^{-(2\nu+d)}$, far more slowly than the Gaussian's $e^{-\ell^2\|\omega\|^2/2}$. Heavy spectral tails mean the kernel keeps a nonvanishing amount of high-frequency content, and that surviving high frequency is exactly what makes Matern sample paths rough. As $\nu \rightarrow \infty$ the Student-t tends to a Gaussian, $(1 + \|\omega\|^2 \ell^2 / 2\nu)^{-(\nu+d/2)} \rightarrow e^{-\ell^2\|\omega\|^2/2}$, and the family collapses to the squared-exponential kernel; at the other extreme $\nu = \frac{1}{2}$ gives the Cauchy-Lorentzian spectrum $(2\nu/\ell^2 + \omega^2)^{-1}$ of the Laplace kernel. The single parameter ν slides the tail weight continuously between these two.

Because ν governs the tail, it governs smoothness in the precise sense of mean-square differentiability.

Proposition (the smoothness ladder)

A mean-square continuous stationary process with spectral density S is m -times mean-square differentiable if and only if its $2m$ -th spectral moment is finite, $\int_{\mathbb{R}^d} \|\omega\|^{2m} S(\omega) d\omega < \infty$. For the Matern density this holds exactly when $m < \nu$. Hence a Matern kernel of smoothness ν is $\lceil \nu \rceil - 1$ times mean-square differentiable: $\nu = \frac{1}{2}$ gives a continuous but nowhere-differentiable Ornstein-Uhlenbeck path, $\nu = \frac{3}{2}$ one derivative, $\nu = \frac{5}{2}$ two, and $\nu \rightarrow \infty$ the infinitely differentiable Gaussian limit.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Differentiating a stationary process m times multiplies its spectral density by $\|\omega\|^{2m}$, so the m -th derivative exists in mean square exactly when the variance of that derivative, namely $\int \|\omega\|^{2m} S(\omega) d\omega$, is finite. For the Matern density $S(\omega) \propto (\alpha^2 + \|\omega\|^2)^{-(\nu+d/2)}$, pass to polar coordinates: at large radius the integrand behaves like $\|\omega\|^{2m} \cdot \|\omega\|^{-(2\nu+d)} \cdot \|\omega\|^{d-1} = \|\omega\|^{2m-2\nu-1}$, whose radial integral converges at infinity if and only if $2m-2\nu-1 < -1$, that is $m < \nu$. The number of derivatives is then $\lceil \nu \rceil - 1$, the largest integer strictly below ν . $\square$

This is the sample-path face of a fact already met on the RKHS side. The reciprocal-density penalty $\int |\hat{f}(\omega)|^2 / \hat{k}_\nu(\omega) d\omega$ grows like $\|\omega\|^{2\nu+d}$, so the Matern RKHS is the Sobolev space of order $\nu + d/2$, the smoothness bookkeeping of Chapter 18, Mercer's Theorem, Spectra, and Rates. Committing to $\nu = \infty$ in advance, as the Gaussian kernel silently does, commits to analytic functions and can badly misjudge rougher data; Stein (1999) argues from kriging practice that ν should be estimated rather than assumed, and Rasmussen and Williams (2006) make the half-integer kernels $\nu = \frac{3}{2}$ and $\nu = \frac{5}{2}$ the workhorse covariances of Gaussian-process regression precisely because they buy a controlled, finite amount of smoothness in closed form.

20.4.2 Worked example: the smoothness ladder in numbers

Example (Matern kernel and spectrum across ν)

Length scale $\ell = 1$. We evaluate the Matern kernel at $\nu = \frac{1}{2}, \frac{3}{2}, \frac{5}{2}$ from the closed forms above and its Gaussian limit $e^{-r^2/2}$, then the normalized spectral density $(1 + \omega^2/2\nu)^{-(\nu+1/2)}$ in dimension $d = 1$. All numbers from checks/ch08-ex1.py.

1. Climb the ladder at a fixed distance. The kernel values are

r	$\nu = \frac{1}{2}$	$\nu = \frac{3}{2}$	$\nu = \frac{5}{2}$	Gaussian
0.5	0.606531	0.784888	0.828649	0.882497
1.0	0.367879	0.483358	0.523994	0.606531

At each distance the value increases with ν : a smoother kernel holds nearby points more strongly correlated.

2. Watch the Gaussian limit. Climbing the half-integer ladder, $k_\nu(1)$ rises through 0.367879, 0.483358, 0.523994, 0.566053, 0.58 at $\nu = \frac{1}{2}, \frac{3}{2}, \frac{5}{2}, \frac{7}{2}, \frac{9}{2}$, toward the limit $e^{-1/2} = 0.606531$. The largest gap $\max_r |k_\nu(r) - e^{-r^2/2}|$ on $[0, 3]$ shrinks as 0.0839, 0.0216, 0.0112 at $\nu = \frac{5}{2}, \frac{7}{2}, \frac{9}{2}$.
3. Read the heavy tail off the spectrum. The normalized spectral density is

ω	$\nu = \frac{1}{2}$	$\nu = \frac{3}{2}$	$\nu = \frac{5}{2}$	Gaussian
1	0.5000	0.5625	0.5787	0.6065
3	0.1000	0.0625	0.0456	0.0111
10	9.901×10^{-3}	8.483×10^{-4}	1.080×10^{-4}	1.929×10^{-22}

The Matern tails fall as the power law $\omega^{-(2\nu+1)}$, so even the smooth $\nu = \frac{5}{2}$ kernel retains, at $\omega = 10$, more than 10^{17} times the spectral mass the Gaussian keeps there.

Reading. One parameter ν slides continuously from the rough Laplace kernel to the analytic Gaussian, and the same ν that adds mean-square derivatives in the space domain lightens the power-law tail in the frequency domain. The Gaussian is the $\nu \rightarrow \infty$ corner of the Matern family, not a separate object.

Verification artifact. checks/example-ch08-example-19-6.json records the example source hash and verification scope.

20.4.3 Spectral mixture kernels

The Matern family still hard-wires the shape of the spectrum: a single bump anchored at zero frequency. A kernel that must model data with periodic or quasi-periodic structure needs spectral mass away from zero, and, more ambitiously, needs to learn where the mass sits rather than have it prescribed. Bochner licenses any placement of nonnegative symmetric spectral mass, so we may treat the spectral density itself as the model and fit it to data. Wilson and Adams (2013) take the most tractable rich family for this, a Gaussian mixture on the frequency axis.

Definition (spectral mixture kernel, Wilson and Adams 2013)

Place a symmetric mixture of Q Gaussians on the frequency axis as the spectral density, the q -th component carrying weight $w_q > 0$, center frequency $\mu_q \geq 0$, and spectral variance $v_q > 0$. In one dimension,

$$S(\omega) = \sum_{q=1}^Q \frac{w_q}{2} \left[\mathcal{N}(\omega; \mu_q, v_q) + \mathcal{N}(\omega; -\mu_q, v_q) \right],$$

and its inverse Fourier transform is the stationary spectral mixture kernel, with $\tau = x - x'$,

$$k(\tau) = \sum_{q=1}^Q w_q e^{-2\pi^2 \tau^2 v_q} \cos(2\pi \mu_q \tau).$$

Positive definiteness is free and needs no verification: S is a sum of Gaussians with positive weights, hence a nonnegative symmetric measure, so Bochner certifies k at once. The construction inverts the usual workflow. Rather than propose a kernel and then check admissibility, one writes down any nonnegative spectral density and

reads off a guaranteed-valid kernel. Each component contributes a Gaussian envelope $e^{-2\pi^2\tau^2v_q}$, whose width is set by the spectral variance v_q , modulating a cosine of period $1/\mu_q$. A single component with $\mu_1 = 0$ recovers the squared-exponential kernel; a component with $\mu_q \neq 0$ is a damped cosine, which lets the kernel represent correlations that oscillate with distance, dip below zero, and extrapolate periodic patterns that no single-bump kernel can capture. Because Gaussian mixtures are dense in the integrable spectral densities, the spectral mixture family is dense in the stationary kernels: with enough components it approximates any of them, which is why it serves as a flexible default covariance for Gaussian-process pattern discovery (Chapter 40, Gaussian Processes and the RVM).

Example (a spectral-mixture Gram is positive definite by construction)

A two-component kernel with weights $w = (1.0, 0.5)$, spectral variances $v = (0.30, 0.05)$, and center frequencies $\mu = (0.6, 1.4)$, evaluated on the five points $x = (0, 0.3, 0.9, 1.5, 2.2)$. All numbers from `checks/ch08-ex2.py`.

1. Assemble the Gram. With $K_{ij} = k(x_i - x_j)$,

$$K = \begin{pmatrix} 1.5000 & -0.1510 & -0.0221 & 0.0439 & 0.0037 \\ -0.1510 & 1.5000 & 0.1122 & -0.0514 & -0.0076 \\ -0.0221 & 0.1122 & 1.5000 & 0.1122 & 0.0402 \\ 0.0439 & -0.0514 & 0.1122 & 1.5000 & 0.2577 \\ 0.0037 & -0.0076 & 0.0402 & 0.2577 & 1.5000 \end{pmatrix}.$$

The diagonal is $k(0) = \sum_q w_q = 1.5000$, and several off-diagonals are negative ($K_{12} = -0.1510$, $K_{13} = -0.0221$), the signature of the cosine factor: correlation can pass below zero as distance grows.

2. Diagonalize. The eigenvalues are (1.2188, 1.3237, 1.4571, 1.6996, 1.8008), all strictly positive.
3. Certify. The smallest eigenvalue is $1.2188 > 0$, so $K > 0$. No positivity was imposed on the points or checked by hand: it is inherited from the nonnegative spectral density through Bochner.

Reading. Designing in the frequency domain makes validity automatic. Any weights and any placement of the Gaussian bumps yield a positive definite kernel, so the modeler is free to fit w_q, μ_q, v_q to data without ever leaving the admissible set.

Verification artifact. `checks/example-ch08-example-19-7.json` records the example source hash and verification scope.

20.4.4 Non-stationary kernels: varying length scales and warping

Every kernel so far is stationary: a single spectral density governs the whole space, so the kernel looks the same at every location. Many signals are not like that, being smooth in some regions and rough in others, or stretched along some directions and compressed along others. Two constructions leave stationarity behind while keeping positive definiteness, and both reuse the stationary kernels just built as raw material.

The first lets the length scale vary with position. Paciorek and Schervish (2004) attach to each input x a local covariance matrix $\Sigma(x) > 0$, the smoothing ellipse there, and couple two points through the average of their ellipses.

Proposition (non-stationary kernel, Paciorek and Schervish 2004)

Let k_0 be any isotropic positive definite function on $\mathbb{R}^d$, and let $x \mapsto \Sigma(x)$ assign a positive definite $d \times d$ matrix to each input. Then

$$k(x, x') = |\Sigma(x)|^{1/4} |\Sigma(x')|^{1/4} \left| \frac{\Sigma(x) + \Sigma(x')}{2} \right|^{-1/2} k_0(\sqrt{Q(x, x')}), \quad Q(x, x') = (x - x')^\top \left(\frac{\Sigma(x) + \Sigma(x')}{2} \right)^{-1} (x - x'),$$

is positive definite on $\mathbb{R}^d$. In one dimension, with scalar length scales $\ell(x)$, this reduces to

$$k(x, x') = \sqrt{\frac{2\ell(x)\ell(x')}{\ell(x)^2 + \ell(x')^2}} k_0\left(\frac{|x - x'|}{\sqrt{(\ell(x)^2 + \ell(x')^2)/2}}\right).$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The quadratic form Q is a Mahalanobis distance under the averaged ellipse, so where Σ is small the kernel resolves fine detail and where Σ is large it smooths aggressively. The determinant prefactor is exactly the normalizer that keeps the construction positive definite: Paciorek and Schervish obtain it by writing each local kernel as a Gaussian-shaped basis function of width $\Sigma(x)$ and convolving, and a convolution of positive kernels is positive again. Taking k_0 from the Matern family gives a non-stationary Matern kernel with a spatially adaptive length scale, a common default in geostatistics.

The second route is simpler still: pull a stationary kernel back through a nonlinear map. Given any warp $u : \mathcal{X} \rightarrow \mathbb{R}^m$ and a stationary kernel k_0 on $\mathbb{R}^m$, the composition

$$k(x, x') = k_0(u(x) - u(x'))$$

is positive definite, since it is k_0 evaluated on transformed inputs, and it is non-stationary whenever u stretches space unevenly. Warping is the manifold idea in flat disguise: bending the input space so that one stationary kernel fits varying smoothness is the Euclidean cousin of building a kernel from the geometry of a curved domain, developed for graphs and manifolds in Chapter 27, Geometric and Equivariant Kernels. Together, varying length scales and input warping show that the stationary spectral picture is a starting point rather than a ceiling: once a valid stationary kernel is in hand, position-dependent reshaping of it remain valid kernels.

20.5 Conditionally positive definite kernels

Bochner's theorem draws the boundary of the positive definite translation-invariant kernels, but that boundary is not the edge of the useful world. The Gaussian and Laplace profiles above are all nonnegative and decreasing, yet some of the most natural dissimilarity profiles on $\mathbb{R}^d$, the plain squared distance among them, fail positive definiteness while still generating a perfectly good geometry. The resolution, following Schölkopf and Smola (2002), is to relax the quadratic-form condition on one subspace and to read the kernel not as a dot product but as a squared distance.

Definition (conditionally positive definite kernel)

A symmetric kernel k is conditionally positive definite if

$$\sum_{i,j=1}^n c_i c_j k(x_i, x_j) \geq 0$$

for all $n \in \mathbb{N}$, all points $x_1, \dots, x_n$, and all real scalars $c_1, \dots, c_n$ subject to the single constraint $\sum_{i=1}^n c_i = 0$.

Every positive definite kernel is conditionally positive definite, since imposing the constraint only shrinks the set of test vectors that must pass; the containment is strict. The constraint $\sum_i c_i = 0$ is exactly an invariance that many kernel algorithms already enjoy. Shifting the origin in feature space, centering the data in kernel PCA, and the equality constraint $\sum_i \alpha_i y_i = 0$ in the support vector machine dual all project onto this zero-sum subspace, so they never distinguish k from k offset by a constant. Two closure facts drop out at once: sums of conditionally positive definite kernels are again conditionally positive definite, and adding any constant b preserves the property.

The reason this is the right class is geometric. A conditionally positive definite k with $k(x, x) = 0$ embeds as a squared distance: there is a feature map Φ into a Hilbert space with

$$\|\Phi(x) - \Phi(x')\|^2 = -k(x, x'),$$

so that positive definite kernels represent inner products while conditionally positive definite ones represent the distances arising from norms. The prototype is the negative squared Euclidean distance $k(x, x') = -\|x - x'\|^2$, which is conditionally positive definite but positive definite nowhere.

Theorem (positive definite exponentials)

A kernel k is conditionally positive definite if and only if $\exp(t k)$ is positive definite for every $t > 0$. The exponential half of this correspondence goes back to Schoenberg (1938), who showed that a translation-invariant k is negative definite exactly when $\exp(-t k)$ is positive definite for all $t > 0$; the conditional-positive-definiteness formulation used here is the account of Schölkopf and Smola (2002).

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

This one equivalence organizes a whole shelf of the kernels met above. Take $k(x, x') = -\|x - x'\|^2$, conditionally positive definite as just noted; the theorem makes $\exp(-t\|x - x'\|^2)$ positive definite for every $t > 0$, so the Gaussian kernel is simply the exponential of a conditionally positive definite generator, a cleaner certificate than evaluating Bochner's integral. The same reading recovers the exponential and Laplace kernels of the half-line as exponentials of conditionally positive definite generators, and it explains why these families are infinitely divisible, their n -th roots being kernels again.

Beyond the exponential family sit conditionally positive definite kernels of higher order, graded by how large a polynomial subspace the zero-sum constraint must annihilate. Three are standard (Schölkopf and Smola 2002): the inverse multiquadric $(\|x - x'\|^2 + c^2)^{-1/2}$, which is in fact already positive definite (order zero); the multiquadric $(\|x - x'\|^2 + c^2)^{1/2}$, conditionally positive definite of order one, which unlike a genuine similarity need not be positive anywhere; and the thin plate spline $\|x - x'\|^{2n} \ln \|x - x'\|$ of order n . A convenient test underlies these examples: $h(\|x - x'\|^2)$ is conditionally positive definite of order q exactly when h is completely monotonic of order q , meaning $(-1)^m h^{(m)} \geq 0$ for all $m \geq q$. These are the working kernels of scattered-data interpolation, where Micchelli (1986) established that conditionally positive definite radial functions make the interpolation matrix invertible on the zero-sum subspace, and they show that the admissible profiles reach well past the Bochner class once the origin is allowed to drift.

20.6 Generalization to semigroups

Herglotz and Bochner both rely on a group structure: differences $x - y$, inverses, characters $e^{i\omega t}$. Yet many data types support only a partial algebra. A bag of parts can be added to another bag, but it cannot be subtracted; a finite measure can be summed with another, but there is no inverse. The right abstraction is a semigroup, and the theory of Berg, Christensen, and Ressel (1984) extends the whole Fourier picture to that setting.

Definition (semigroup with involution)

A semigroup $(S, \circ)$ is a nonempty set S with an associative composition $\circ$ and a neutral element e . A semigroup with involution $(S, \circ, *)$ adds a map $*$: $S \rightarrow S$ satisfying

$$(s \circ t)^* = t^* \circ s^*, \quad (s^*)^* = s, \quad s, t \in S.$$

Two families of examples cover almost everything we need. Any group $(G, \circ)$ is a semigroup with involution under $s^* = s^{-1}$. And any abelian semigroup $(S, +)$ is a semigroup with involution under the identical involution $s^* = s$. The first recovers the group case; the second is the genuinely new one, where no inverse exists.

Definition (positive definite function on a semigroup)

Let $(S, \circ, *)$ be a semigroup with involution. A function $\varphi : S \rightarrow \mathbb{R}$ is positive definite if

$$K(s, t) = \varphi(s^* \circ t)$$

is a positive definite kernel on S .

Translation invariance is exactly this definition specialized to the group $(\mathbb{R}^d, +, -)$, which is abelian with involution $s^* = -s$: the function φ is positive definite in the semigroup sense precisely when $K(x, y) = \varphi(x - y)$ is a positive definite kernel. So the semigroup theory contains the shift-invariant theory as one instance.

20.6.1 Semicharacters

What plays the role of the character $e^{i\omega t}$? On a group a character is multiplicative and unimodular; on a semigroup we keep multiplicativity but drop the modulus constraint. The resulting objects are the elementary positive definite functions out of which all others are built.

Definition (semicharacter)

A function $\rho : S \rightarrow \mathbb{C}$ on an abelian semigroup with involution $(S, +, *)$ is a semicharacter if

$$\rho(0) = 1, \quad \rho(s + t) = \rho(s)\rho(t), \quad \rho(s^*) = \overline{\rho(s)}.$$

The set of semicharacters on S is denoted S^* .

Two remarks fix the picture. If the involution is the identity, then $\rho(s) = \overline{\rho(s)}$, so a semicharacter is automatically real-valued. If instead $(S, +)$ is an abelian group with $s^* = -s$, then $\rho(s)\rho(-s) = \rho(0) = 1$ and $\rho(-s) = \overline{\rho(s)}$ force $|\rho(s)| = 1$; the semicharacter takes values in the circle and is an ordinary group character. So semicharacters interpolate between real exponentials and complex characters.

The way to hold all of this in the head is that a semicharacter is a pure exponential mode adapted to the algebra at hand. On a group the modes are the oscillating characters $e^{i\omega t}$ that Fourier analysis is built from; on a semigroup with no inverses, such as the half-line, the modes are the decaying exponentials e^{-as} instead. In either case the mode is exactly the object that turns the semigroup operation into ordinary multiplication, $\rho(s + t) = \rho(s)\rho(t)$, and that multiplicativity is what lets the quadratic form factor into a perfect square, which is why a single semicharacter is automatically positive definite. The semicharacters are the atoms of the theory; the integral representation below then says that every positive definite function is built by mixing them, the exact semigroup analogue of Bochner's superposition of waves.

Lemma (semicharacters are positive definite)

Every semicharacter is positive definite, meaning $K(s, t) = K(t, s)$ and $\sum_{i,j=1}^n a_i a_j K(x_i, x_j) \geq 0$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The quadratic form factors: for the identity involution,

$$\sum_{i,j=1}^n a_i a_j \rho(x_i + x_j) = \sum_{i,j} a_i a_j \rho(x_i) \rho(x_j) = \left(\sum_i a_i \rho(x_i) \right)^2 \geq 0.$$

The general involution case is the same computation with a complex square. $\square$

Concretely, $\varphi(t) = e^{\beta t}$ is a semicharacter on $(\mathbb{R}, +, \text{Id})$, and $\varphi(t) = e^{i\omega t}$ is a semicharacter on the group $(\mathbb{R}, +, -)$. The first is a real exponential; the second is the Fourier character.

A discrete case pins the idea down further. On the additive semigroup of natural numbers $(\mathbb{N}_0, +, \text{Id})$, multiplicativity forces every semicharacter to be a geometric sequence $\rho(n) = \rho(1)^n = t^n$, and boundedness restricts the base to $t \in [-1, 1]$. The integral representation to come then reads: a bounded sequence $\{\varphi(n)\}$ is positive definite exactly when $\varphi(n) = \int_{-1}^1 t^n d\mu(t)$ for a positive measure μ , that is, exactly when it is a moment sequence. The abstract theorem thus specializes to the classical Hausdorff moment problem, one more instance of the same pattern: the semicharacters are the pure modes t^n , and the representing measure is the recipe that mixes them.

20.6.2 Integral representation of positive definite functions

The deep theorem is that, up to a growth condition, every positive definite function on an abelian semigroup is an integral mixture of semicharacters, exactly as Bochner writes φ as an integral of $e^{i\omega t}$ against the spectral measure. The growth condition is stated through an absolute value.

Definition (absolute value, exponential boundedness)

A function $\alpha : S \rightarrow \mathbb{R}$ on a semigroup with involution is an absolute value if $\alpha(e) = 1$, $\alpha(s \circ t) \leq \alpha(s)\alpha(t)$, and $\alpha(s^*) = \alpha(s)$. A function $f : S \rightarrow \mathbb{R}$ is exponentially bounded if $|f(s)| \leq C\alpha(s)$ for some absolute value α and constant $C > 0$.

Theorem (integral representation)

Let $(S, +, *)$ be an abelian semigroup with involution. A function $\varphi : S \rightarrow \mathbb{R}$ is positive definite and exponentially bounded (respectively bounded) if and only if it has the representation

$$\varphi(s) = \int_{S^*} \rho(s) d\mu(\rho),$$

where μ is a Radon measure with compact support on S^* (respectively on $\hat{S}$, the set of bounded semicharacters).

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof (sketch; full details in Berg, Christensen, and Ressel (1984), Theorem 4.2.5)

For a fixed absolute value α , the set P_1^α of α -bounded positive definite functions with $\varphi(0) = 1$ is a compact convex set, and its extreme points are exactly the α -bounded semicharacters. If φ is positive definite and exponentially bounded, then $\varphi(0)^{-1}\varphi \in P_1^\alpha$ for a suitable α . By the Krein-Milman theorem there is a Radon probability measure on P_1^α whose barycentre is $\varphi(0)^{-1}\varphi$, and this measure is supported on the extreme points, the semicharacters. $\square$

Two remarks. The result fails without the exponential-boundedness restriction on the semicharacters. And in the special case of an abelian group with $s^* = -s$ it reduces to Bochner's theorem for discrete abelian groups (Rudin 1962), confirming that this is the correct generalization.

20.6.3 Example: the half-line $(\mathbb{R}_+, +, \text{Id})$

The simplest new semigroup is the nonnegative reals under addition with the identity involution. Here positive definite functions are automatically nonnegative. The reason is that the semigroup operation is addition, so every $x \geq 0$ splits as $x = \frac{x}{2} + \frac{x}{2}$, and with the identity involution the one-point positive definiteness inequality gives $\varphi(x) = \varphi(\frac{x}{2} + \frac{x}{2}) = \varphi(\frac{x}{2})^2 \geq 0$. (Halving, not taking a square root, is the correct move, since $\sqrt{x} + \sqrt{x} = 2\sqrt{x} \neq x$.) The bounded semicharacters are exactly the decaying exponentials

$$s \in \mathbb{R}_+ \mapsto \rho_a(s) = e^{-as}, \quad a \in [0, +\infty]$$

(the endpoints included, with ρ_∞ the indicator of $s = 0$). Unbounded semicharacters are pathological: they include nonmeasurable solutions of the Cauchy equation $h(x + y) = h(x)h(y)$.

Feeding the bounded semicharacters into the integral representation theorem yields a clean characterization: $\varphi : \mathbb{R}_+ \rightarrow \mathbb{R}$ is positive definite and bounded if and only if

$$\varphi(s) = \int_0^\infty e^{-as} d\mu(a) + b\rho_\infty(s), \quad \mu \in \mathcal{M}_+^b(\mathbb{R}_+), b \geq 0.$$

The first term is the Laplace transform of μ . Continuity removes the atom at infinity, so φ is positive definite, bounded and continuous if and only if it is the Laplace transform of a finite positive measure. This is the Bochner theorem of the half-line, with the Laplace transform in place of the Fourier transform.

Two micro-examples show the criterion at work. The function $\varphi(s) = e^{-as}$ is positive definite for every $a \geq 0$, being a single semicharacter, and any nonnegative mixture of these is positive definite again: for instance $\varphi(s) = \frac{1}{1+s} = \int_0^\infty e^{-as} e^{-a} da$ is the Laplace transform of the finite measure $e^{-a} da$, hence a bona fide kernel on the half-line. By contrast $\varphi(s) = \cos(s)$, perfectly admissible on the group $(\mathbb{R}, +, -)$ where it is a Fourier mode, is not positive definite on $(\mathbb{R}_+, +, \text{Id})$: a positive definite function there must be nonnegative, as noted above, and the cosine is not. The involution, not merely the underlying set of points, is what decides which functions qualify.

20.6.4 Example: kernels for finite measures

The motivating application, following Cuturi, Fukumizu, and Vert (2005), is comparing bags of points: sets of points of a space $\mathcal{U}$, with repeats allowed. A finite-length string described by its k -mers is exactly such a bag. We want a positive definite kernel between two bags that depends only on their union, that is, that respects the additive structure of merging bags.

Represent a bag as a finite measure on $\mathcal{U}$,

$$x = \sum_i a_i \delta_{x_i},$$

where a_i is the multiplicity of x_i . Merging two bags corresponds to adding their measures, so the natural object is the semigroup $(\mathcal{M}_+^b(\mathcal{U}), +, \text{Id})$ of bounded Radon measures, and we seek positive definite functions φ on it.

Its continuous semicharacters have an explicit form. For any Borel measurable $f : \mathcal{U} \rightarrow \mathbb{R}$, the function

$$\rho_f(\mu) = e^{\mu[f]}, \quad \mu[f] = \int_{\mathcal{U}} f d\mu,$$

is a semicharacter on $(\mathcal{M}_+^b(\mathcal{U}), +)$, because $\mu[f]$ is additive in μ . Conversely, a semicharacter continuous for the weak convergence topology is of this form for a continuous f . (There is no such clean characterization for the non-continuous, even bounded, semicharacters.) Integrating over a measure on these test functions produces the general kernel.

Corollary (measure kernel)

Let $\mathcal{U}$ be a Hausdorff space. For any Radon measure $\mu \in \mathcal{M}_+^c(C(\mathcal{U}))$ with compact support on the space of continuous real functions on $\mathcal{U}$ (with the topology of pointwise convergence), the function

$$K(\mu, \nu) = \int_{C(\mathcal{U})} e^{\mu[f] + \nu[f]} d\mu(f)$$

is a continuous positive definite kernel on $\mathcal{M}_+^b(\mathcal{U})$ endowed with the weak convergence topology.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The converse fails: there are continuous positive definite kernels with no such integral representation, because the representing measure might need non-continuous semicharacters. Two concrete kernels of this form are especially useful.

Example (entropy kernel)

Let $\mathcal{X}$ be the set of probability densities on $\mathcal{U}$ (with respect to a reference measure) with finite entropy $h(x) = -\int_{\mathcal{U}} x \ln x$. Then for every $\beta > 0$,

$$K(x, x') = e^{-\beta h\left(\frac{x+x'}{2}\right)}$$

is a positive definite kernel on $\mathcal{X}$. It is valid only for densities, for example the density produced by a kernel density estimator from a bag of parts. Two bags are similar when their average is peaked (low entropy).

Verification artifact. checks/example-ch08-example-19-8.json records the example source hash and verification scope.

Example (inverse generalized variance kernel)

Let $\mathcal{U} = \mathbb{R}^d$ and let $\mathcal{M}_+^V(\mathcal{U})$ be the finite measures with a second-order moment and non-singular variance

$$\Sigma(\mu) = \mu[xx^T] - \mu[x]\mu[x]^T.$$

Then

$$K(\mu, \mu') = \frac{1}{\det \Sigma\left(\frac{\mu+\mu'}{2}\right)}$$

is a positive definite kernel on $\mathcal{M}_+^V(\mathcal{U})$, the inverse generalized variance (IGV) kernel. Two bags are similar when their pooled point cloud is tightly concentrated, so that the generalized variance $\det \Sigma$ is small.

Verification artifact. checks/example-ch08-example-19-9.json records the example source hash and verification scope.

Geometrically, the IGV kernel measures how the spread of the merged bag compares with the spreads of the individual bags: a weighted linear PCA of the two measures gives principal components whose captured variances multiply to $\det \Sigma$, and the kernel is the reciprocal of that product. When the two clouds align, merging does not inflate the variance and the kernel is large.

The plain IGV kernel has two weaknesses: a Gaussian second-moment summary may be a poor model, and $\det \Sigma$ degenerates in high dimension. Both are cured by the same two devices used throughout the book. Regularization replaces Σ by $\Sigma + \lambda I_d$,

$$K_\lambda(\mu, \mu') = \frac{1}{\det\left(\Sigma\left(\frac{\mu+\mu'}{2}\right) + \lambda I_d\right)},$$

and the kernel trick handles high dimension: since the nonzero eigenvalues of UU^T and $U^T U$ coincide, one replaces the covariance matrix by the centered Gram matrix of the points (technical details in Cuturi et al. 2005). The kernelized IGV kernel gave good results on character recognition from subsampled images.

The broader lesson is that semigroup kernels are a very general formalism for exploiting an algebraic structure in the data: whenever complex objects decompose into simple parts that can be pooled additively, one can build

a kernel on the pooled representation, and extensions to non-abelian groups (for instance permutations in the symmetric group) are natural next steps.

20.7 Kernels from probabilistic models

Kernel methods are sometimes faulted for their rigidity: a large effort goes into hand-designing the kernel. A powerful alternative is to let a generative probabilistic model of the data do the design. This is the idea behind kernels for generative models, which were state of the art for image and sequence representation for many years. Throughout, a parametric model is a family of distributions

$$\{P_\theta, \theta \in \Theta \subset \mathbb{R}^m\} \subseteq \mathcal{M}_+^1(\mathcal{X}),$$

and the kernel is read off from how the model explains each data point.

20.7.1 The Fisher kernel

The Fisher kernel of Jaakkola, Diekhans, and Haussler (2000) turns a fitted model into a feature map by asking, for each example, in which direction of parameter space the model would have to move to explain it better. That direction is the gradient of the log-likelihood.

Definition (Fisher kernel)

Fix a parameter $\theta_0 \in \Theta$, for instance the maximum-likelihood estimate on a training set. For each example x , compute the Fisher score vector

$$\Phi_{\theta_0}(x) = \nabla_\theta \log P_\theta(x) \big|_{\theta=\theta_0},$$

the local contribution of each parameter to generating x . The Fisher kernel is

$$K(x, x') = \Phi_{\theta_0}(x)^\top I(\theta_0)^{-1} \Phi_{\theta_0}(x'),$$

where $I(\theta_0) = \mathbb{E}[\Phi_{\theta_0}(x)\Phi_{\theta_0}(x)^\top]$ is the Fisher information matrix.

The picture behind this definition is worth drawing out. A fitted model P_{θ_0} is a single point in the space of distributions, and near it the parameters θ act as local coordinates. The score $\Phi_{\theta_0}(x) = \nabla_\theta \log P_\theta(x)$ records, for one example x , the direction in which we would have to tilt the parameters to raise its likelihood. Two examples that pull the model the same way, that would both be explained better by the same adjustment, get similar scores and hence a large kernel value; two examples that pull in opposite directions come out dissimilar. The model thus supplies the coordinate system, and each example is represented not by its raw features but by the fingerprint it leaves on the parameters, the gradient it induces on the log-likelihood.

Why divide by the information matrix at all? The raw score coordinates are measured in whatever units each parameter happens to use, and a parameter the data pin down tightly produces large scores while a slack one produces small scores, so comparing them directly would let the incidental scaling of the parametrization dominate the similarity. The whitening rests on a general identity for the score, not on any special property of the fit: under the model the expected score vanishes at every parameter value, $\mathbb{E}_{x \sim P_{\theta_0}} [\Phi_{\theta_0}(x)] = 0$, so the information matrix $I(\theta_0) = \mathbb{E}_{x \sim P_{\theta_0}} [\Phi_{\theta_0}(x)\Phi_{\theta_0}(x)^\top]$ is exactly the covariance of the score under the model. Multiplying by $I(\theta_0)^{-1}$ therefore whitens the scores, rotating and rescaling them into directions that are uncorrelated and of unit variance under the model. The Fisher kernel is therefore a Mahalanobis inner product between score vectors, and the reparametrization invariance proved below is the formal statement that this whitening erases the arbitrary choice of coordinates.

A one-parameter micro-example makes the score tangible before the fuller computations below. Take a Bernoulli model $P_\theta(x) = \theta^x(1 - \theta)^{1-x}$ for $x \in \{0, 1\}$. Its score is

$$\Phi_\theta(x) = \frac{\partial}{\partial \theta} [x \log \theta + (1 - x) \log(1 - \theta)] = \frac{x - \theta}{\theta(1 - \theta)},$$

and the Fisher information is $I(\theta) = 1/(\theta(1 - \theta))$, so the whitened Fisher vector collapses to the single number $\phi_\theta(x) = (x - \theta)/\sqrt{\theta(1 - \theta)}$. A success and a failure land on opposite sides of zero and so receive a negative kernel value, while two successes receive a positive one: the model's lone coordinate already sorts examples by whether they would push θ up or down. Notice too that at the maximum-likelihood estimate $\theta_0 = \hat{\theta}$ the scores average to zero over the training set. That is a distinct fact from the model identity above: the vanishing of the empirical mean of the scores is the first-order optimality condition that defines the MLE, whereas the vanishing of the model expectation $\mathbb{E}_{x \sim P_{\theta_0}} [\Phi_{\theta_0}(x)] = 0$, which is what makes $I(\theta_0)$ the covariance of the score, holds at every θ_0 regardless of the data.

Properties

The Fisher score describes how each parameter contributes to generating a particular example, which gives the kernel a clear interpretation. Two theoretical guarantees are worth stating. A kernel classifier using the Fisher kernel derived from a model that contains the label as a latent variable is asymptotically at least as good as the MAP labelling based on that model (Jaakkola and Haussler 1999). And a variant, the Tangent of Posterior kernel of Tsuda et al. (2002), can improve over the direct posterior classification by correcting for estimation error in the parameter.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Lemma (invariance under reparametrization)

The Fisher kernel is invariant under a change of parametrization.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Consider a new parametrization $\lambda = f(\theta)$ through a diffeomorphism f , with Jacobian $[J]_{ij} = \partial \theta_j / \partial \lambda_i$. By the chain rule the score transforms as

$$\Phi_{\lambda_0}(x) = \nabla_\lambda \log P_{\lambda_0}(x) = J \nabla_\theta \log P_{\theta_0}(x) = J \Phi_{\theta_0}(x),$$

and the information matrix as $I(\lambda_0) = \mathbb{E}[\Phi_{\lambda_0}(x)\Phi_{\lambda_0}(x)^\top] = J I(\theta_0) J^\top$. Hence $I(\lambda_0)^{-1} = J^{-\top} I(\theta_0)^{-1} J^{-1}$, and

$$\Phi_{\lambda_0}(x)^\top I(\lambda_0)^{-1} \Phi_{\lambda_0}(x') = \Phi_{\theta_0}(x)^\top J^\top J^{-\top} I(\theta_0)^{-1} J^{-1} J \Phi_{\theta_0}(x') = \Phi_{\theta_0}(x)^\top I(\theta_0)^{-1} \Phi_{\theta_0}(x').$$

The two Jacobians cancel, so the kernel is unchanged. $\square$

In practice $\Phi_{\theta_0}(x)$ can be computed explicitly for many models, notably hidden Markov models, after fitting the model to data. The information matrix $I(\theta_0)$ is often replaced by the identity for simplicity, and several models (different θ_0) can be trained and combined. Writing $\phi_{\theta_0}(x) = I(\theta_0)^{-1/2} \Phi_{\theta_0}(x)$ for the whitened score, called the Fisher vector, gives an explicit finite-dimensional embedding with

$$K(x, x') = \phi_{\theta_0}(x)^\top \phi_{\theta_0}(x').$$

This explicitness is what makes the Fisher kernel practical at scale: one computes and stores the vectors, no Gram matrix required.

20.7.2 A worked example: a Gaussian data model

Take a univariate normal $\mathcal{N}(\mu, \sigma^2)$ and write $\alpha = 1/\sigma^2$ for the precision, with $\theta = (\mu, \alpha)$. Then

$$\log P_\theta(x) = \frac{1}{2} \log \alpha - \frac{1}{2} \log(2\pi) - \frac{1}{2} \alpha (x - \mu)^2,$$

so the score components are

$$\frac{\partial \log P_\theta(x)}{\partial \mu} = \alpha(x - \mu), \quad \frac{\partial \log P_\theta(x)}{\partial \alpha} = \frac{1}{2} \left(\frac{1}{\alpha} - (x - \mu)^2 \right),$$

and the information matrix is diagonal,

$$I(\theta) = \begin{pmatrix} \alpha & 0 \\ 0 & \frac{1}{2} \alpha^{-2} \end{pmatrix}.$$

Whitening gives the Fisher vector

$$\phi_\theta(x) = \begin{pmatrix} (x - \mu)/\sigma \\ \frac{1}{\sqrt{2}} (1 - (x - \mu)^2/\sigma^2) \end{pmatrix},$$

whose two coordinates measure the standardized first moment and the deficit in second moment.

For an i.i.d. sample $x_1, \dots, x_n$ from $\mathcal{N}(\mu, \sigma^2)$, the log-likelihood is a sum, so the Fisher vector is the sum of the individual Fisher vectors:

$$\phi(x_1, \dots, x_n) = \sum_{i=1}^n \phi(x_i) = \begin{pmatrix} n(\hat{\mu} - \mu)/\sigma \\ n(\sigma^2 - \hat{\sigma}^2)/(\sqrt{2} \sigma^2) \end{pmatrix},$$

with $\hat{\mu} = \frac{1}{n} \sum_i x_i$ and $\hat{\sigma}^2 = \frac{1}{n} \sum_i (x_i - \mu)^2$. Both coordinates carry the factor n : summing n individual scores, the first component gives $\sum_i (x_i - \mu)/\sigma = n(\hat{\mu} - \mu)/\sigma$, and the second gives $\frac{1}{\sqrt{2}} (n - \sum_i (x_i - \mu)^2/\sigma^2) = n(\sigma^2 - \hat{\sigma}^2)/(\sqrt{2} \sigma^2)$. The representation of a whole set is thus the discrepancy between its empirical first and second moments and those of the model. This additivity over points is exactly the pooling that made semigroup kernels work, and it is the seed of the visual-words application.

20.7.3 Application: aggregation of visual words

Before deep learning, the dominant image representation was built from local descriptors, and the Fisher kernel provided the best way to pool them. The pipeline has three stages. In the patch extraction stage, an image is described as a set of patches $x_1, \dots, x_n$ at interest points, encoded by descriptors such as SIFT, HOG, LBP, color histograms, or later convolutional features. In the coding stage, the set of patches is mapped to a single high-dimensional vector $\phi(x_i)$. In the pooling stage, the codes are aggregated, for instance by sum pooling $\phi(x_1, \dots, x_n) = \sum_i \phi(x_i)$. Fisher vectors with a Gaussian mixture model (Perronnin and Dance 2007) are a simple and effective instance of this recipe.

Let $\theta = (\pi_j, \mu_j, \Sigma_j)_{j=1, \dots, k}$ parametrize a GMM with k Gaussian components,

$$P_\theta(x) = \sum_{j=1}^k \pi_j \mathcal{N}(x; \mu_j, \Sigma_j).$$

Each component is a visual word with a mean, a variance, and a mixing weight, so this is a much richer model than the classical hard-assignment bag of words. Diagonal covariances $\Sigma_j = \text{diag}(\sigma_j)$ are usually used, and the model is learned offline in advance.

After a lengthy but elementary computation, the Fisher vector concatenates per-component blocks,

$$\phi_\theta(x_1, \dots, x_n) = [\phi_{\pi_1}, \dots, \phi_{\pi_k}, \phi_{\mu_1}^\top, \dots, \phi_{\mu_k}^\top, \phi_{\sigma_1}^\top, \dots, \phi_{\sigma_k}^\top]^\top,$$

with the mean and variance blocks

$$\phi_{\mu_j}(X) = \frac{1}{n\sqrt{\pi_j}} \sum_{i=1}^n \gamma_{ij} (x_i - \mu_j)/\sigma_j, \quad \phi_{\sigma_j}(X) = \frac{1}{n\sqrt{2\pi_j}} \sum_{i=1}^n \gamma_{ij} ((x_i - \mu_j)^2/\sigma_j^2 - 1),$$

where the division of vectors is elementwise and the soft-assignment of patch i to component j is the posterior responsibility

$$\gamma_{ij} = \frac{\pi_j \mathcal{N}(x_i; \mu_j, \sigma_j)}{\sum_{l=1}^k \pi_l \mathcal{N}(x_i; \mu_l, \sigma_l)}.$$

Collecting responsibilities as $\gamma_j = \sum_i \gamma_{ij}$ and forming the soft-assigned moments $\hat{\mu}_j = \frac{1}{\gamma_j} \sum_i \gamma_{ij} x_i$ and $\hat{\sigma}_j^2 = \frac{1}{\gamma_j} \sum_i \gamma_{ij} (x_i - \mu_j)^2$, the blocks read

$$\phi_{\mu_j}(X) = \frac{\gamma_j}{\sqrt{\pi_j}} (\hat{\mu}_j - \mu_j)/\sigma_j, \quad \phi_{\sigma_j}(X) = \frac{\gamma_j}{\sqrt{2\pi_j}} (\hat{\sigma}_j^2 - \sigma_j^2)/\sigma_j^2.$$

Each block is precisely the first- and second-order statistics of the patches assigned to a word, measured against that word's mean and variance, the same discrepancy interpretation as the single-Gaussian example. The weight block $\phi_\pi(X)$ is usually dropped as negligible, leaving a representation of dimension $2kp$ with $p = \dim x_i$.

Fisher vectors were the state-of-the-art image representation before the revival of convolutional networks in 2012. They are an unsupervised, high-dimensional representation, and they remained competitive among unsupervised methods well afterwards (see the comparison in Bojanowski and Joulin 2017).

20.7.4 Relation to classification with generative models

The Fisher kernel is not just a heuristic feature map; when the model carries the label, it reconstructs the Bayes classifier. Suppose a generative model P_θ describes a pair (X, Y) with label $Y \in \{1, \dots, p\}$, and that the class priors are among the parameters, softmax-parametrized as

$$P_\theta(Y = k) = \pi_k = \frac{e^{\alpha_k}}{\sum_{k'=1}^p e^{\alpha_{k'}}}.$$

Classifying a new point by Bayes' rule means

$$\hat{y}(x) = \arg \max_k P_\theta(Y = k | x), \quad P_\theta(Y = k | x) = \frac{P_\theta(x | Y = k) \pi_k}{\sum_{k'} P_\theta(x | Y = k') \pi_{k'}}.$$

Now differentiate the marginal log-likelihood. Using $\nabla_\theta \log P_\theta(x) = \frac{1}{P_\theta(x)} \nabla_\theta P_\theta(x)$ and expanding $P_\theta(x) = \sum_k P_\theta(x, Y = k)$,

$$\nabla_{\theta} \log P_{\theta}(x) = \sum_{k=1}^p P_{\theta}(Y = k | x) [\nabla_{\theta} \log \pi_k + \nabla_{\theta} \log P_{\theta}(x | Y = k)].$$

Differentiating with respect to the softmax parameter α_k specifically gives the clean identity (exercise)

$$\frac{\partial \log P_{\theta}(x)}{\partial \alpha_k} = P_{\theta}(Y = k | x) - \pi_k.$$

So the first p coordinates of the Fisher score are the class posteriors minus the priors,

$$\phi_{\theta}(x) = [P_{\theta}(Y = 1 | x) - \pi_1, \dots, P_{\theta}(Y = p | x) - \pi_p, \dots].$$

Consider the linear classifier that, for class k , uses the weight vector w_k with a single 1 in the k -th position and the intercept $b_k = \pi_k$. Then

$$\hat{y}(x) = \arg \max_k \phi_{\theta}(x)^{\top} w_k + b_k = \arg \max_k (P_{\theta}(Y = k | x) - \pi_k) + \pi_k = \arg \max_k P_{\theta}(Y = k | x).$$

Bayes' rule is therefore implemented by a trivial linear classifier on the Fisher features, which is precisely why the Fisher kernel is asymptotically no worse than the generative model's own MAP rule.

20.7.5 Mutual information kernels

The Fisher kernel linearizes the model at a single point θ_0 . A complementary idea, due to Seeger (2002), keeps the whole parameter space and integrates the models against a prior, so that two examples are similar when many models assign high likelihood to both.

Definition (mutual information kernel)

Choose a prior $w(d\theta)$ on the measurable set Θ . The mutual information kernel is

$$K(x, x') = \int_{\theta \in \Theta} P_{\theta}(x) P_{\theta}(x') w(d\theta).$$

No finite-dimensional feature vector is computed. The kernel is an inner product in $L^2(w)$ between the infinite feature maps $\phi(x) = (P_{\theta}(x))_{\theta \in \Theta}$,

$$K(x, x') = \langle \phi(x), \phi(x') \rangle_{L^2(w)},$$

which makes positive definiteness immediate. This is the same construction as the mutual information kernels used for biological sequences, where Θ indexes a family of Markov models.

Example (coin toss)

Let $P_{\theta}(X = 1) = \theta$ and $P_{\theta}(X = 0) = 1 - \theta$ model a random coin, with $\theta \in [0, 1]$, and take $w = d\theta$, Lebesgue measure. For the strings $x = 001$ and $x' = 1010$,

$$P_{\theta}(x) = \theta(1 - \theta)^2, \quad P_{\theta}(x') = \theta^2(1 - \theta)^2,$$

so, using the Beta integral $\int_0^1 \theta^a (1 - \theta)^b d\theta = \frac{a! b!}{(a+b+1)!}$,

$$K(x, x') = \int_0^1 \theta^3 (1 - \theta)^4 d\theta = \frac{3! 4!}{8!} = \frac{1}{280}.$$

The kernel averages the joint likelihood of the two strings over all coin biases.

Verification artifact. checks/example-ch08-example-19-10.json records the example source hash and verification scope.

20.7.6 Marginalized kernels

Often the natural kernel lives not on the observed data but on a completed version of it that includes a hidden variable: an alignment, a state path, a parse tree. If we knew the hidden variable we would use a kernel on the complete data; since we do not, we average over its posterior. This is the marginalized kernel of Tsuda, Kin, and Asai (2002).

Definition (marginalized kernel)

For observed data $x \in \mathcal{X}$, let a latent variable $y \in \mathcal{Y}$ be attached through a conditional probability $P_x(dy)$. Let $K_{\mathcal{Z}}$ be a kernel on the complete data $z = (x, y)$. Then

$$K_{\mathcal{X}}(x, x') := \mathbb{E}_{P_x(dy) \times P_{x'}(dy')} K_{\mathcal{Z}}(z, z') = \iint K_{\mathcal{Z}}((x, y), (x', y')) P_x(dy) P_{x'}(dy')$$

is a valid kernel on $\mathcal{X}$, called a marginalized kernel.

Positive definiteness

If $K_{\mathcal{Z}}$ is positive definite, then so is $K_{\mathcal{X}}$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Since $K_{\mathcal{Z}}$ is positive definite on $\mathcal{Z}$, there is a Hilbert space $\mathcal{H}$ and a feature map $\Phi_{\mathcal{Z}} : \mathcal{Z} \rightarrow \mathcal{H}$ with $K_{\mathcal{Z}}(z, z') = \langle \Phi_{\mathcal{Z}}(z), \Phi_{\mathcal{Z}}(z') \rangle_{\mathcal{H}}$. Marginalizing and pulling the expectations through the inner product, which is linear in each argument,

$$K_{\mathcal{X}}(x, x') = \mathbb{E}_{P_x(dy) \times P_{x'}(dy')} \langle \Phi_{\mathcal{Z}}(z), \Phi_{\mathcal{Z}}(z') \rangle_{\mathcal{H}} = \left\langle \mathbb{E}_{P_x(dy)} \Phi_{\mathcal{Z}}(z), \mathbb{E}_{P_{x'}(dy')} \Phi_{\mathcal{Z}}(z') \right\rangle_{\mathcal{H}}.$$

The right-hand side is an inner product of the two averaged feature vectors $\bar{\Phi}(x) = \mathbb{E}_{P_x(dy)} \Phi_{\mathcal{Z}}(z)$, so $K_{\mathcal{X}}(x, x') = \langle \bar{\Phi}(x), \bar{\Phi}(x') \rangle_{\mathcal{H}}$ is positive definite on $\mathcal{X}$. (We assume throughout the regularity needed for every operation above to be valid and every quantity well defined.) $\square$

The marginalized kernel is the feature map of the complete-data kernel, averaged under the latent posterior: two objects are close when their expected complete-data representations are close. It closes the loop with the rest of the chapter, since the mutual information kernel and the pooled Fisher vector are both instances of averaging a simple feature map over a distribution, and it is the workhorse behind many kernels for sequences and graphs where the hidden variable is a path through a model.

20.8 The algebra of kernel construction

The three roads of this chapter each end at a family of kernels, but in practice a kernel is rarely used bare. More often it is assembled: two similarities are added, a similarity on one factor is multiplied by a similarity on another, a base kernel is normalized or reshaped to fit a domain. What makes this safe is that positive definiteness survives a short list of operations, so the admissible kernels form not a scattered collection but an algebra closed under the natural ways of combining them. We collect the operations that the constructions above have been quietly using, following Schölkopf and Smola (2002) and Shawe-Taylor and Cristianini (2004).

Closure properties of positive definite kernels

Let k_1, k_2 be positive definite kernels. Then each of the following is positive definite:

- (i) the nonnegative combination $\alpha_1 k_1 + \alpha_2 k_2$ for $\alpha_1, \alpha_2 \geq 0$, and any pointwise limit $k = \lim_n k_n$ of kernels, so the kernels form a convex cone closed under pointwise convergence;
- (ii) the pointwise product $(k_1 k_2)(x, x') = k_1(x, x') k_2(x, x')$, the Schur product;
- (iii) the tensor product $(k_1 \otimes k_2)((x_1, x_2), (x'_1, x'_2)) = k_1(x_1, x'_1) k_2(x_2, x'_2)$ on the product domain, and the direct sum, in which the two similarities are added rather than multiplied;
- (iv) the conformal transformation $k_f(x, x') = f(x) k(x, x') f(x')$ for any real-valued function f .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Part (i) is exactly what licenses the Bochner and semigroup integral representations: a spectral measure, or a mixture of semicharacters, is a limit of nonnegative combinations of the elementary kernels $\cos(\omega^\top(x - y))$ or $\rho(s)$, and the mixture inherits their positive definiteness. Part (ii), the pointwise product, is the least obvious entry on the list, and it has a memorable probabilistic proof.

Proof that the pointwise product is a kernel

It suffices to show that the entrywise (Hadamard) product of two positive semidefinite Gram matrices K_1, K_2 is positive semidefinite. Let $(V_1, \dots, V_m)$ and $(W_1, \dots, W_m)$ be independent mean-zero Gaussian random vectors with covariance matrices K_1 and K_2 . The product vector $(V_1 W_1, \dots, V_m W_m)$ then has covariance

$$\mathbb{E}[V_i W_i V_j W_j] = \mathbb{E}[V_i V_j] \mathbb{E}[W_i W_j] = (K_1)_{ij} (K_2)_{ij},$$

by independence of the two vectors. Hence the entrywise product $K_1 \circ K_2$ is itself a covariance matrix and therefore positive semidefinite (Schölkopf and Smola 2002). $\square$

20.8.1 Interpolating with ANOVA kernels, and normalization

The direct sum and the tensor product are the two extremes of how far a kernel lets features interact: the direct sum keeps each factor separate, while the tensor product couples every factor with every other. The ANOVA kernels fill the gap between them. Given base kernels $k^{(1)}, \dots, k^{(N)}$, one per attribute, the ANOVA kernel of order D sums the tensor products over all size- D subsets of distinct attributes,

$$k_D(x, x') = \sum_{1 \leq i_1 < i_2 < \dots < i_D \leq N} \prod_{j=1}^D k^{(i_j)}(x_{i_j}, x'_{i_j}),$$

so that $D = 1$ recovers the direct sum and $D = N$ the full tensor product, with intermediate D selecting exactly the interactions of order D (Schölkopf and Smola 2002). The name ANOVA, for analysis of variance, records the origin of the underlying decomposition. Because the sum runs over distinct indices, a naive evaluation would cost $\binom{N}{D}$ terms, but a dynamic-programming recursion in D and N brings it down to $O(ND)$ (Shawe-Taylor and

Cristianini 2004), and the whole construction is a special case of the convolution kernels for structured objects of Haussler (1999).

A final operation is cosmetic but ubiquitous: normalization. Given any kernel k , the normalized kernel

$$\tilde{k}(x, x') = \frac{k(x, x')}{\sqrt{k(x, x) k(x', x')}}}$$

is again positive definite and sends every point to the unit sphere of feature space, since $\tilde{k}(x, x) = 1$, so the similarity now reads as a cosine between feature vectors. The Gaussian kernel is already normalized in this sense, because $k(x, x) = 1$; for an unnormalized kernel such as a polynomial, the rescaling removes the dependence on the raw magnitudes of the feature vectors and often makes the kernel behave far better (Shawe-Taylor and Cristianini 2004).

Further reading

The constructions of this chapter are developed at length in two standard references. Schölkopf and Smola (2002), *Learning with Kernels*, devote their chapter 13 to the design of kernels and lay the groundwork for positive definite kernels and their properties in chapter 2. Shawe-Taylor and Cristianini (2004), *Kernel Methods for Pattern Analysis*, survey the basic kernels and the principles of kernel design in their chapter 9. Both give complementary derivations of the translation-invariant and probabilistic-model kernels discussed here, together with many further worked examples.

20.9 Summary

Three constructions, one theme. Translation invariance, through Herglotz and Bochner, makes positive definiteness equivalent to a positive spectral measure and turns the RKHS norm into a Fourier penalty that starves high frequencies; the Gaussian, Laplace, and sinc kernels are just different spectral filters. Semigroups extend this Fourier picture to data that can be pooled but not inverted, with semicharacters in place of exponentials and the Laplace transform in place of the Fourier transform, giving kernels such as the inverse generalized variance kernel for bags of points. Probabilistic-model kernels, the Fisher, mutual information, and marginalized kernels, let a generative model design the feature map, whether by differentiating the log-likelihood, integrating over parameters, or averaging over a latent variable. Many notions of smoothness and similarity become RKHS norms for a suitable kernel, and there is no uniformly best kernel: what we have is a toolbox for encoding the structure of the data, and the following chapters put it to work on sequences and graphs.

20.10 Common mistakes and practical implications

For **Translation-Invariant, Semigroup, and Probabilistic Kernels**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

20.11 Exercises

1. warm-up Read Bochner's theorem as a statement about spectral mass. For each of the following translation-invariant profiles on $\mathbb{R}$, name the spectral measure μ and say whether the profile is positive definite: (a) the constant $\varphi(t) = C$ with $C \geq 0$; (b) the single cosine $\varphi(t) = \cos(\omega_0 t)$; (c) the box $\varphi(t) = \mathbf{1}_{[-1,1]}(t)$. In one sentence, explain why inspecting the shape of φ in the space variable is the wrong test, and what one should inspect instead.
2. computation Let $P_\theta(x) = e^{-\theta} \theta^x / x!$ be the Poisson model on $x \in \mathbb{N}_0$, with scalar parameter $\theta > 0$. Compute the Fisher score $\Phi_\theta(x) = \partial_\theta \log P_\theta(x)$ and the Fisher information $I(\theta) = \mathbb{E}[\Phi_\theta(x)^2]$. Then write down the whitened Fisher vector $\phi_\theta(x) = I(\theta)^{-1/2} \Phi_\theta(x)$ and the resulting one-dimensional Fisher kernel $K(x, x') = \phi_\theta(x) \phi_\theta(x')$. Which pairs (x, x') receive a positive kernel value, and which a negative one?
3. proof Use Bochner's theorem to decide the positive definiteness of the triangle (tent) profile $\varphi(t) = \max(0, 1 - |t|)$ on $\mathbb{R}$. Rather than integrating directly, write φ as a self-convolution $\varphi = b * b$ of a box b , and use the convolution theorem to compute $\hat{\varphi}$. Conclude from the sign of $\hat{\varphi}$ that φ is a kernel, and contrast this with the box $\mathbf{1}_{[-1,1]}$ of the text, which is not. Hint

Take $b = \mathbf{1}_{[-1/2, 1/2]}$, so that $(b * b)(t) = \max(0, 1 - |t|)$. The convolution theorem gives $\hat{\varphi}(\omega) = \hat{b}(\omega)^2$, and $\hat{b}$ is real, so $\hat{\varphi} = \hat{b}^2 \geq 0$ everywhere. A nonnegative spectral density is exactly Bochner's admissibility criterion.

4. proof Prove the conformal-transformation closure property of Proposition (closure properties): if k is positive definite and $f : \mathcal{X} \rightarrow \mathbb{R}$ is any real-valued function, then $k_f(x, x') = f(x) k(x, x') f(x')$ is positive definite. Do it directly from the quadratic form, without invoking a feature map, and then identify the feature map of k_f in terms of the feature map Φ of k . Hint

In $\sum_{i,j} c_i c_j f(x_i) k(x_i, x_j) f(x_j)$, absorb $f(x_i)$ into the coefficient by setting $c'_i = c_i f(x_i)$; the sum becomes $\sum_{i,j} c'_i c'_j k(x_i, x_j) \geq 0$. The feature map is $\Phi_f(x) = f(x) \Phi(x)$.

5. proof Show that the negative squared Euclidean distance $k(x, x') = -\|x - x'\|^2$ on $\mathbb{R}^d$ is conditionally positive definite, that is, $\sum_{i,j} c_i c_j k(x_i, x_j) \geq 0$ whenever $\sum_i c_i = 0$. Deduce, via the Theorem (positive definite exponentials), that the Gaussian $\exp(-t\|x - x'\|^2)$ is positive definite for every $t > 0$, giving a Bochner-free certificate for the Gaussian kernel. Hint

Expand $\|x_i - x_j\|^2 = \|x_i\|^2 - 2\langle x_i, x_j \rangle + \|x_j\|^2$. Under $\sum_i c_i = 0$, the two pure terms $\sum_{i,j} c_i c_j \|x_i\|^2$ and $\sum_{i,j} c_i c_j \|x_j\|^2$ each vanish, leaving $2 \sum_{i,j} c_i c_j \langle x_i, x_j \rangle = 2 \left\| \sum_i c_i x_i \right\|^2 \geq 0$.

6. exploration Open the interactive figure above. It draws frequencies $\omega_1, \dots, \omega_D$ from the Gaussian's Bochner measure and averages the matching cosine features, which is exactly the random Fourier feature construction of Rahimi and Recht (2007). Increase the number of features D and watch the sampled kernel tighten onto the true Gaussian profile. Report roughly how large D must be before the approximation is visually indistinguishable from the target, and explain, using the fact that $\varphi(x - y) = \mathbb{E}_{\omega \sim \mu} \cos(\omega^\top (x - y))$, why the average sharpens as D grows. Hint

Each cosine feature is an unbiased sample of the expectation Bochner writes down, so the average is a Monte Carlo estimate of φ ; its fluctuation shrinks like $1/\sqrt{D}$.

7. proof In the generative-classification setting of Section (relation to classification), the class priors are softmax-parametrized, $\pi_k = e^{\alpha_k} / \sum_{k'} e^{\alpha_{k'}}$. Fill in the derivation the text leaves as an exercise: show that

$$\frac{\partial \log P_\theta(x)}{\partial \alpha_k} = P_\theta(Y = k \mid x) - \pi_k,$$

so that the corresponding Fisher score coordinates are the class posteriors minus the priors. Hint

Use $\partial_{\alpha_k} \log P_\theta(x) = P_\theta(x)^{-1} \partial_{\alpha_k} P_\theta(x)$ with $P_\theta(x) = \sum_{k'} P_\theta(x | Y = k') \pi_{k'}$, and the softmax derivative $\partial \pi_{k'} / \partial \alpha_k = \pi_{k'} (\delta_{k'k} - \pi_k)$. Recognize $P_\theta(x | Y = k') \pi_{k'} / P_\theta(x) = P_\theta(Y = k' | x)$, then use $\sum_{k'} P_\theta(Y = k' | x) = 1$.

8. challenge Work on the additive semigroup of natural numbers $(\mathbb{N}_0, +, \text{Id})$, whose bounded semicharacters are the geometric sequences $\rho_t(n) = t^n$ with $t \in [-1, 1]$. Show that the sequence $\varphi(n) = \frac{1}{n+1}$ is a positive definite function on this semigroup by exhibiting it as a moment sequence, $\varphi(n) = \int_{-1}^1 t^n d\mu(t)$, for an explicit positive measure μ . Then verify positive definiteness directly, without quoting the integral-representation theorem, and identify which classical moment problem this instance recovers. Hint

Note $\int_0^1 t^n dt = \frac{1}{n+1}$, so μ is Lebesgue measure on $[0, 1]$. For the direct check, $\sum_{i,j} a_i a_j \varphi(n_i + n_j) = \int_0^1 (\sum_i a_i t^{n_i})^2 dt \geq 0$. This is the Hausdorff moment problem, the specialization of the integral representation with pure modes t^n .

9. computation Work in $d = 1$ with $\ell = 1$, where the Matern spectral density is $\hat{k}_\nu(\omega) \propto (2\nu + \omega^2)^{-(\nu+1/2)}$. (a) Show that the $2m$ -th spectral moment $\int_{\mathbb{R}} \omega^{2m} \hat{k}_\nu(\omega) d\omega$ is finite if and only if $m < \nu$, and read off how many mean-square derivatives the $\nu = \frac{1}{2}, \frac{3}{2}, \frac{5}{2}$ kernels give. (b) Let $\nu \rightarrow \infty$ in the normalized spectrum $(1 + \omega^2/2\nu)^{-(\nu+1/2)}$ and show it tends to the Gaussian spectrum $e^{-\omega^2/2}$, so the family acquires finite spectral moments of every order only in the limit. (c) Reproduce the $r = 0.5$ column of the worked example from the half-integer closed forms, confirming $k_{1/2}(0.5) = e^{-1/2}$ and $k_{3/2}(0.5) = (1 + \frac{\sqrt{3}}{2})e^{-\sqrt{3}/2}$. Hint

For (a), at large ω the integrand behaves like $\omega^{2m-2\nu-1}$, integrable at infinity iff $2m - 2\nu - 1 < -1$. For (b), use $(1 + a/n)^n \rightarrow e^a$ with n of order ν : the factor $\omega^2/2\nu$ inside and the exponent $\nu + \frac{1}{2}$ conspire to $e^{-\omega^2/2}$.

10. proof The spectral mixture kernel of Wilson and Adams (2013) is $k(\tau) = \sum_{q=1}^Q w_q e^{-2\pi^2 \tau^2 v_q} \cos(2\pi \mu_q \tau)$. (a) Prove it is positive definite directly from Bochner by exhibiting its spectral density as a nonnegative symmetric measure, a mixture of Gaussian bumps at $\pm \mu_q$. (b) Show that a single component with $\mu_1 = 0$ is the squared-exponential kernel, and identify its length scale ℓ in terms of v_1 . (c) Explain why a component with $\mu_q \neq 0$ can drive $k(\tau)$ below zero for some τ , which no single Gaussian or Matern kernel can do, and what structure in the data this negative correlation lets the kernel capture. Hint

For (a), $\cos(2\pi \mu_q \tau)$ is the Fourier transform of $\frac{1}{2}(\delta_{\mu_q} + \delta_{-\mu_q})$, and multiplying by the Gaussian envelope convolves the spectrum, spreading each atom into a Gaussian bump; the weighted sum is a nonnegative symmetric density. For (b), at $\mu_1 = 0$ the cosine is 1 and $e^{-2\pi^2 \tau^2 v_1} = e^{-\tau^2/2\ell^2}$ forces $\ell^2 = 1/(4\pi^2 v_1)$.

21 Invariances and the Pre-Image Problem

The last chapters built kernels and learned with them; this chapter asks two questions that arise once a kernel machine is deployed on real data. The first is how to tell the machine what we already know: that a handwritten digit is still the same digit after a one-pixel shift or a small rotation. Encoding such prior knowledge about invariances is what turned support vector machines from an elegant theory into a benchmark-winning tool. The second question runs in the opposite direction to everything so far. Kernel algorithms deliver their answers as vectors in feature space, expansions $\sum_i \alpha_i \Phi(x_i)$; but a denoised digit, or a compressed classifier, is only useful as a point back in input space. Recovering that point is the pre-image problem, and it turns out to have no exact solution in general and a beautiful approximate one. The two themes meet at the same place: both are about the map Φ between input and feature space, one pushing prior structure forward through it, the other pulling solutions back.

21.1 Prior knowledge and invariance

In 1995 LeCun et al. (1995) compared learning algorithms on handwritten digits and remarked that the optimal margin classifier, the early support vector machine, was remarkable precisely because it used no prior knowledge at all: it would perform identically if the image pixels were scrambled by a fixed permutation. That is a striking property, and also a damning one. A method blind to the spatial layout of an image is throwing away most of what makes the problem solvable. Within a few years the same group had closed the gap, and the tool that closed it was the systematic incorporation of prior knowledge, following Schölkopf, Burges, and Vapnik (1996).

By prior knowledge we mean everything about the task that is available beyond the training examples themselves. Some of it is generic. Every kernel already encodes a smoothness assumption: using a kernel k as a regularizer penalizes $\|Yf\|^2$ for an operator Y whose Green function is k , so that patterns close in input space are pushed toward the same label. Some prior knowledge is more specific, such as knowing that in digit images the correlations between nearby pixels are more reliable than those between distant ones, which one builds in through kernels that emphasize local products of pixels. The sharpest and most useful form, and the subject of this chapter, is knowledge of a transformation that must not change the answer.

Definition (invariance under a transformation)

Let $\{L_t\}_t$ be a family of transformations of the input space $\mathcal{X}$, indexed by a parameter t with $L_0 = \text{id}$. A decision function $f : \mathcal{X} \rightarrow \mathbb{R}$ is *invariant* under $\{L_t\}$ if

$$f(L_t x) = f(x) \quad \text{for all } x \in \mathcal{X} \text{ and all admissible } t.$$

Common examples are the group of translations, rotations, or line-thickness changes of a digit image. When $\{L_t\}$ is a differentiable one-parameter group, the *tangent vector* at x is $\left. \frac{d}{dt} \right|_{t=0} L_t x$, the local direction in which the transformation moves the pattern.

Schölkopf, Burges, and Vapnik (1996) distinguish three ways to exploit such knowledge, and the rest of the first half of the chapter is an unpacking of these three. First, one can generate artificial training examples, virtual examples, by applying the transformations to the data and hoping the machine learns the invariance from the enlarged set. Second, one can change the learning algorithm itself, modifying the objective so the estimated function is forced to have small derivative along the tangent directions. Third, one can change the representation, mapping the data into a space where the invariance becomes trivial, for instance replacing each point by a feature that the transformation leaves fixed. The third is the most powerful when the required map can be found, but it is often unavailable or too global for a locally defined invariance, so in practice the first two dominate. A fourth, hybrid idea folds the transformation directly into the kernel.

21.2 The Virtual Support Vector method

Generating virtual examples from the entire training set is simple but expensive: multiplying a large database by the number of desired transforms produces an unwieldy training set, and much of the added data is redundant because it sits far from the decision boundary where it cannot affect the solution. The Virtual Support Vector (VSV) method of Schölkopf, Burges, and Vapnik (1996) exploits a structural fact about support vector machines to avoid that waste: the support vectors alone carry all the information the machine used, so a classifier retrained on just the support vector set matches one trained on the full data. If the support vectors contain the whole solution, then the transformations worth applying are the ones applied to them.

The logic is geometric. Small transformations move a support vector a short distance, and support vectors sit on or near the margin, so their transformed copies land near the decision surface too, exactly the region where new examples sharpen the boundary. Transformed copies of interior points, by contrast, stay interior and teach the machine nothing. This is why generating virtual examples from the full database, in the experiments of Schölkopf, Burges, and Vapnik (1996), gave no accuracy gain over generating them from the support vectors alone.

Algorithm (Virtual Support Vector method)

Input Training set $\{(x_i, y_i)\}_{i=1}^m$, kernel k , penalty C , a family of invariance transforms $\{L_t\}$ with chosen parameter values $t_1, \dots, t_r$.

Output An invariant SVM decision function f .

1. Train a standard SVM on $\{(x_i, y_i)\}$; extract the support vector set $S = \{x_i : \alpha_i > 0\}$.
2. For each support vector $x \in S$ and each transform $L_{t_1}, \dots, L_{t_r}$, form the virtual support vector $L_{t_j}x$ with the same label as x .
3. Collect the virtual support vectors together with the original support vectors into an enlarged set S' .
4. Train a second SVM on S' ; return its decision function.
5. Optionally iterate steps 1 to 4, with care: iterating a local invariance can accumulate into an unwanted global one, for instance rotating a 6 into a 9.

The one-pixel translation instance is the workhorse. On the USPS database, a degree-3 polynomial SVM had a test error of 4.0%; generating virtual support vectors by shifting each support vector one pixel in the four principal directions and retraining cut the error to 3.2%, a substantial gain from a nearly free preprocessing step. The table records how the support vector set changes.

Classifier trained on	Training size	Avg. no. of SVs	Test error
Full training set	7291	274	4.0%
Overall SV set	1677	268	4.1%
Virtual SV set	8385	686	3.2%
Virtual patterns from full DB	36455	719	3.4%

Two readings stand out. Training on the overall support vector set (row two) barely changes the error, confirming that the support vectors carry the solution. And generating virtual patterns from the whole database (row four) is far more expensive yet no better than the VSV row, confirming that transforms of non-support-vectors add little. On the larger MNIST database, where the method has more room to work, a degree-9 polynomial with twelve translated virtual examples per support vector reached 0.56% test error, the record for that benchmark at the time (DeCoste and Schölkopf 2002). The price is a larger final support vector set, hence slower evaluation, which is exactly the motivation for the reduced-set methods of the second half of this chapter. Because it manipulates only the data, the VSV method also handles discrete symmetries that derivative-based schemes cannot, such as the mirror reflection of a bilaterally symmetric object.

21.3 Tangent vectors, invariance kernels, and jittering

The second route modifies the algorithm rather than the data. Instead of asking the machine to infer the invariance from examples, we penalize any variation of the decision function along the tangent directions. Locally, invariance of g at a pattern x_j under the group $\{L_t\}$ is the statement $\frac{d}{dt}\big|_{t=0} g(L_t x_j) = 0$, and summing the squared derivative over the data gives a regularizer that pulls the solution toward invariant functions.

Carrying this through for a linear decision function $g(x) = \sum_i \alpha_i y_i \langle x, x_i \rangle + b$, following Schölkopf and Smola (2002), the tangent regularizer reduces to a modified quadratic form governed by the *tangent covariance matrix*

$$T = \frac{1}{m} \sum_{j=1}^m \left(\frac{d}{dt}\bigg|_0 L_t x_j \right) \left(\frac{d}{dt}\bigg|_0 L_t x_j \right)^\top,$$

the empirical covariance of the tangent vectors, which have zero mean. The whole invariant training problem then collapses back to a standard SVM once we replace the ordinary dot product by $\langle x, x' \rangle_A = x^\top A x'$ with $A = T_\gamma^{-1}$ and $T_\gamma = (1 - \gamma)T + \gamma I$. The trade-off parameter $\gamma \in (0, 1]$ interpolates between the plain SVM at $\gamma = 1$ and full invariance as $\gamma \rightarrow 0$; the regularized T_γ is strictly positive definite, hence invertible.

The preprocessing has a clean interpretation. Diagonalize $T_\gamma = UDU^\top$; then $A^{1/2} = UD^{-1/2}U^\top$ projects a pattern onto the eigenvectors of the tangent covariance and rescales each by the inverse square root of its eigenvalue. Directions in which the data vary a lot under the transformation, the large-eigenvalue directions of T , are scaled down, so the classifier leans on the features that the transformation leaves nearly fixed. This is a whitening of the tangent covariance, and it connects invariance to principal component analysis: for translations, it downweights the absolute positions of strokes and emphasizes the relative amounts of ink. Computing T from the support vectors alone turns it into a task-dependent covariance that concentrates invariance where it matters, near the boundary.

21.3.1 Tangent distance and translation-invariant kernels

The tangent-vector idea has an older cousin in the tangent distance of Simard et al. (1998). The set of all transforms $\{L_t x\}$ of a single pattern traces out a low-dimensional manifold in input space, the orbit of x under the invariance. Two patterns should be judged similar if their orbits come close, not if the raw points do, since a one-pixel shift can move a digit far in Euclidean distance while leaving its identity untouched. Computing the true distance between two curved manifolds is hard, so tangent distance replaces each orbit by its tangent plane at the pattern, spanned by the tangent vectors, and measures the distance between those planes. To first order in t this is exactly invariant under the transformation.

This is where the two halves of the book's kernel story touch. A translation-invariant kernel $k(x, x') = \varphi(x - x')$ is invariant by construction: it reads only the difference of its arguments, so translating both patterns together leaves it unchanged. Tangent distance builds the same idea locally and for a general transformation group, by making the metric, and hence any kernel built from it, insensitive to first-order motion along the orbit. A genuinely translation-invariant kernel is the global, exact version of what tangent distance achieves approximately and for one specific pair of patterns; both encode the invariance into the geometry itself rather than into extra training data. See Chapter 19, Translation-Invariant, Semigroup, and Probabilistic Kernels for the translation-invariant families and Chapter 39, Deep Learning from a Kernel Point of View for invariance as the prior that convolutional architectures hard-wire.

21.3.2 Jittering kernels

The hybrid method folds the transformation into the kernel evaluation itself, following DeCoste and Schölkopf (2002). Given any admissible kernel with Gram entries $K_{ij} = k(x_i, x_j)$, the jittering kernel considers all jittered forms of one argument, including the untransformed one, and keeps the closest match in feature space.

Definition (jittering kernel)

Let $\{x_i^{(1)}, \dots, x_i^{(J)}\}$ be the jittered forms of x_i (its transforms plus itself). The jittering kernel is

$$k_J(x_i, x_j) = K_{qj}, \quad q = \arg \min_{1 \leq p \leq J} (K_{ii}^{(p)} - 2K_{ij}^{(p)} + K_{jj}),$$

where the minimized quantity is the squared feature-space distance $\|\Phi(x_i^{(p)}) - \Phi(x_j)\|^2$ between the jittered pattern and x_j . For a normalized RBF kernel, where $k(x, x) = 1$, this reduces to selecting the jitter that maximizes $K_{ij}^{(p)}$.

Jittering trades space for time relative to VSV. The VSV method inflates the training set by the number of jitters J and pays the quadratic training cost of that larger set; the jittering kernel keeps the training set at its original size but makes each kernel evaluation J times more expensive, so training can scale only linearly in J , and kernel caching amortizes much of even that. The cost is that a jittering kernel need not be positive definite, and the induced distance can violate the triangle inequality: for the three single-pixel images $A = (1, 0, 0)$, $B = (0, 1, 0)$, $C = (0, 0, 1)$ under one-pixel translation, the jittered distances $d(A, B)$ and $d(B, C)$ are 0 while $d(A, C)$ is positive, so $d(A, C) > d(A, B) + d(B, C)$. In practice such violations are rare and SMO tolerates them, and a jittering kernel must keep jittering at test time, unlike VSV which compiles the invariance into its final support vectors.

21.4 The pre-image problem

We now turn the map around. A kernel algorithm produces a vector in feature space as an expansion in mapped inputs,

$$\Psi = \sum_{i=1}^m \alpha_i \Phi(x_i),$$

whether it is an SVM weight vector or a kernel PCA feature extractor (see Chapter 13, Kernel PCA and Denoising). For most purposes this is enough: taking a dot product with a test point $\Phi(x)$ turns into the kernel expansion $\sum_i \alpha_i k(x_i, x)$, computable even when Φ lands in an infinite-dimensional space. But sometimes we need the point in input space that Ψ represents: a denoised image reconstructed by kernel PCA, or a compact synthetic pattern that stands in for a whole expansion. A point z with $\Phi(z) = \Psi$ is a *pre-image* of Ψ .

When a pre-image exists and the kernel is an invertible function of the dot product, it is easy to recover, following Schölkopf and Smola (2002).

Exact pre-images (Schölkopf and Smola, 2002)

Let $\Psi = \sum_{j=1}^m \alpha_j \Phi(x_j)$ with $x_j \in \mathcal{X} \subseteq \mathbb{R}^N$, and suppose there is an invertible f_k with $k(x, x') = f_k(\langle x, x' \rangle)$. If a pre-image $z \in \mathbb{R}^N$ with $\Phi(z) = \Psi$ exists, then for any orthonormal basis $e_1, \dots, e_N$ of input space,

$$z = \sum_{i=1}^N f_k^{-1} \left(\sum_{j=1}^m \alpha_j k(x_j, e_i) \right) e_i.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Expand z in the orthonormal basis and read each coordinate as a dot product, which f_k^{-1} recovers from the kernel:

$$z = \sum_{i=1}^N \langle z, e_i \rangle e_i = \sum_{i=1}^N f_k^{-1}(k(z, e_i)) e_i = \sum_{i=1}^N f_k^{-1}\left(\sum_{j=1}^m \alpha_j k(x_j, e_i)\right) e_i,$$

where the last step uses $k(z, e_i) = \langle \Phi(z), \Phi(e_i) \rangle = \langle \Psi, \Phi(e_i) \rangle = \sum_j \alpha_j k(x_j, e_i)$, valid because $\Phi(z) = \Psi$. Invertible f_k covers polynomial kernels $(\langle x, x' \rangle + c)^d$ with $c \geq 0$ and d odd, and sigmoid kernels; a variant using polarization handles RBF kernels. $\square$

The catch is the hypothesis that a pre-image exists, and for the most-used kernel it usually does not. Under a Gaussian kernel Φ maps each input to a Gaussian bump centered on it, and no Gaussian is a linear combination of Gaussians centered elsewhere; so no expansion with more than one term has an exact pre-image. The problem as posed is unsolvable in general, which forces us to relax it.

We call z an *approximate pre-image* of Ψ when

$$\rho(z) = \|\Psi - \Phi(z)\|^2$$

is small. Good approximate pre-images do exist for the vectors we care about. Kernel PCA projects $\Phi(x)$ onto its top n eigenvectors, $P_n \Phi(x) = \sum_{j=1}^n \langle \Phi(x), v_j \rangle v_j$, the best rank- n approximation of the mapped data in the mean-square sense of Mika et al. (1999). If x is drawn from the same distribution as the training data, $P_n \Phi(x)$ stays close to the mapped-data manifold and has a good approximate pre-image; x itself is already one. This is the engine of kernel PCA *denoising*: map a noisy pattern x into feature space, discard the low-variance components that mostly capture noise, and take the pre-image of the projection as a cleaned-up z . On the USPS digits, kernel PCA denoising beats linear PCA once enough components are used, because the nonlinear map can devote more features to structure rather than noise (Mika et al. 1999).

21.5 Finding approximate pre-images

To minimize $\rho(z)$ we could differentiate $\|\Psi - \Phi(z)\|^2$ directly, but a lower-dimensional and better-scaled problem is to maximize the length of the projection of Ψ onto the ray through $\Phi(z)$,

$$\frac{\langle \Psi, \Phi(z) \rangle^2}{\langle \Phi(z), \Phi(z) \rangle},$$

after which the optimal scaling is set separately. For normalized RBF kernels, where $\langle \Phi(z), \Phi(z) \rangle = k(z, z) = 1$, this is simply the maximization of $\langle \Psi, \Phi(z) \rangle^2$, and its stationarity condition yields a fixed-point iteration.

Write $\Psi = \sum_i \alpha_i \Phi(x_i)$, so $\langle \Psi, \Phi(z) \rangle = \sum_i \alpha_i k(x_i, z)$. At an extremum the gradient in z vanishes,

$$0 = \nabla_z \langle \Psi, \Phi(z) \rangle = \sum_{i=1}^m \alpha_i \nabla_z k(x_i, z).$$

For a Gaussian kernel $k(x_i, z) = \exp(-\|x_i - z\|^2/2\sigma^2)$ we have $\nabla_z k(x_i, z) = k(x_i, z)(x_i - z)/\sigma^2$, so

$$0 = \sum_{i=1}^m \alpha_i \exp\left(-\frac{\|x_i - z\|^2}{2\sigma^2}\right)(x_i - z).$$

Solving the linear-in- z relation, $z \sum_i \alpha_i e_i(z) = \sum_i \alpha_i e_i(z) x_i$ with $e_i(z) = \exp(-\|x_i - z\|^2/2\sigma^2)$, gives z as a weighted average of the x_i , which we read as an iteration.

Algorithm (pre-image fixed-point iteration, Gaussian kernel)

Input Expansion $\Psi = \sum_{i=1}^m \alpha_i \Phi(x_i)$, Gaussian width σ , starting point z_0 , tolerance τ .

Output Approximate pre-image z with $\Phi(z) \approx \Psi$.

1. Set $n \leftarrow 0$; choose z_0 , for instance the weighted mean $\sum_i \alpha_i x_i / \sum_i \alpha_i$.
2. Compute the weights $w_i = \alpha_i \exp(-\|x_i - z_n\|^2 / 2\sigma^2)$.
3. Update $z_{n+1} = \frac{\sum_{i=1}^m w_i x_i}{\sum_{i=1}^m w_i}$.
4. Repeat steps 2 and 3 until $\|z_{n+1} - z_n\| < \tau$; return z_{n+1} .

The denominator equals $\langle \Psi, \Phi(z_n) \rangle$ up to normalization, so it is nonzero near a nontrivial extremum; instability only arises when the projection of Ψ onto the span of the mapped inputs is near zero, in which case there is nothing to approximate and one restarts from a different z_0 . The update has a vivid reading as clustering: with all $\alpha_i > 0$ it finds the center of a single Gaussian cluster that captures as much weight as possible, and when the α_i carry signs, as in an SVM where the sign is the label, it seeks a point where one class outweighs the other, estimating the difference of two densities rather than one.

Example (pre-image of a convex combination)

Three points in $\mathbb{R}^2$: $x_1 = (0, 0)$, $x_2 = (2, 0)$, $x_3 = (1, 1.5)$. Target $\Psi = \sum_i \beta_i \Phi(x_i)$ with $\beta = (0.5, 0.3, 0.2)$, Gaussian width $\sigma = 1$. We seek the approximate pre-image z minimizing $\|\Psi - \Phi(z)\|^2$, equivalently maximizing $J(z) = \langle \Psi, \Phi(z) \rangle = \sum_i \beta_i k(x_i, z)$.

1. Start at the weighted mean. $z_0 = \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 = (0.8, 0.3)$, with $J(z_0) = 0.5821$.
2. Reweight and average. The weights $w_i = \beta_i e^{-\|x_i - z_0\|^2 / 2}$ pull the estimate toward the heavier, nearer points, giving $z_1 = (0.6435, 0.2459)$, $J = 0.5959$.
3. Iterate. $z_2 = (0.5328, 0.2152)$, $z_3 = (0.4621, 0.1956)$, $z_4 = (0.4204, 0.1833)$, $z_5 = (0.3970, 0.1759)$, with J climbing 0.6024, 0.6050, 0.6059, 0.6062.
4. Converge. By $z_6 = (0.3842, 0.1717)$ the objective has settled at $J = 0.6062$; the iterates have stopped moving to four places.

Reading. The pre-image $z^* \approx (0.384, 0.172)$ lands between the three points but nearest x_1 , the one with the largest weight, exactly where a single Gaussian bump best matches the weighted mixture. The projection J increases at every step, confirming the iteration is climbing toward the extremum.

Verification artifact. checks/example-ch-invariance-example-20-1.json records the example source hash and verification scope.

21.6 Reduced set methods

The pre-image problem is the extreme case, one term, of a more general and more useful one. An SVM's decision function $\sum_i \alpha_i k(x_i, \cdot)$ costs one kernel evaluation per support vector, and after the VSV method the support vector set is large, so evaluation is slow, a real obstacle in tasks like face detection where a classifier is scanned over every image location. We would like to approximate the exact expansion $\Psi = \sum_{i=1}^{N_x} \alpha_i \Phi(x_i)$ by a shorter *reduced set* expansion

$$\Psi' = \sum_{i=1}^{N_z} \beta_i \Phi(z_i), \quad N_z \ll N_x,$$

minimizing $\|\Psi - \Psi'\|^2$, which is fully expressible in kernels. The problem splits into finding the reduced-set vectors z_i and finding the coefficients β_i ; the coefficients have a closed form.

Optimal expansion coefficients (Schölkopf and Smola, 2002)

Let $\Psi = \sum_{i=1}^{N_x} \alpha_i \Phi(x_i)$, and fix reduced-set vectors $z_1, \dots, z_{N_z}$ whose images $\Phi(z_i)$ are linearly independent. The coefficients $\beta = (\beta_1, \dots, \beta_{N_z})$ minimizing $\|\Psi - \sum_i \beta_i \Phi(z_i)\|^2$ are

$$\beta = (K^z)^{-1} K^{zx} \alpha,$$

where $K_{ij}^z = \langle \Phi(z_i), \Phi(z_j) \rangle$ and $K_{ij}^{zx} = \langle \Phi(z_i), \Phi(x_j) \rangle$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Setting the derivative of $\|\Psi - \sum_i \beta_i \Phi(z_i)\|^2$ with respect to β_j to zero gives, for every j ,

$$0 = \left\langle \Phi(z_j), \Psi - \sum_i \beta_i \Phi(z_i) \right\rangle = (K^{zx} \alpha)_j - (K^z \beta)_j.$$

In matrix form $K^z \beta = K^{zx} \alpha$, and since the $\Phi(z_i)$ are linearly independent K^z has full rank, so $\beta = (K^z)^{-1} K^{zx} \alpha$. No reduced-set method using the feature-space norm can beat this choice of coefficients for given z_i . $\square$

What differs between methods is how the z_i are chosen. *Selection* methods pick a subset of the original x_i ; this is worthwhile because an SVM expansion is not as sparse as it could be, its coefficients being pinned into $[-C, C]$ by the quadratic program. One can select via the near-null space of the Gram matrix, a computation closely tied to kernel PCA, or via an ℓ_1 penalty that shrinks coefficients to zero (Schölkopf and Smola 2002). On USPS, removing about 40% of the support vectors this way leaves the test error practically unchanged. *Construction* methods do better by allowing synthetic z_i that need not be training points, and they get them by iterating the pre-image algorithm.

Algorithm (reduced set construction, Burges 1996)

Input Expansion $\Psi = \Psi_1 = \sum_{i=1}^{N_x} \alpha_i \Phi(x_i)$, Gaussian kernel, target size N_z or residual threshold ϵ .

Output Reduced set $\{(z_i, \beta_i)\}_{i=1}^{N_z}$ with $\sum_i \beta_i \Phi(z_i) \approx \Psi$.

1. Set $m \leftarrow 1$ and let the current residual be Ψ_1 .
2. Find a one-term pre-image z_m of the residual $\Psi_m = \Psi - \sum_{i=1}^{m-1} \beta_i \Phi(z_i)$ by the fixed-point iteration above.
3. Recompute all coefficients $\beta_1, \dots, \beta_m$ optimally from Proposition (optimal expansion coefficients).
4. Increment m and repeat steps 2 and 3 until $m = N_z$ or $\|\Psi_m\| < \epsilon$.
5. Optionally run a phase II: jointly optimize all (z_i, β_i) by gradient descent, then recompute β once more.

On USPS, the construction method beats selection because it draws on vectors off the training set; a tenfold speedup (25 reduced-set vectors in place of 254 support vectors) costs only a rise from 4.4% to 5.1% error, competitive with the convolutional networks of the day, and the synthetic vectors even look like digits (Burges 1996; Schölkopf and Smola 2002). The next example shows the single-term version, the base case of the loop, in miniature.

Example (one-term reduced set of a three-term expansion)

An SVM-style expansion $\Psi = \sum_{i=1}^3 \alpha_i \Phi(x_i)$ with $x_1 = (0, 0)$, $x_2 = (1, 0)$, $x_3 = (0.5, 1)$, coefficients $\alpha = (1.0, 0.8, 0.6)$, Gaussian width $\sigma = 1$. Approximate it by a single term $\beta \Phi(z)$.

1. Measure the target. With Gram entries $k(x_1, x_2) = 0.6065$, $k(x_1, x_3) = k(x_2, x_3) = 0.5353$, the squared length is $\|\Psi\|^2 = \alpha^T K \alpha = 4.1266$, so $\|\Psi\| = 2.0314$.
2. Find the pre-image. Starting at the weighted mean $z_0 = (0.4583, 0.25)$, the fixed-point iteration converges in a handful of steps to $z^* = (0.4465, 0.2218)$.
3. Set the optimal coefficient. Since $k(z^*, z^*) = 1$, $\beta = \sum_i \alpha_i k(x_i, z^*) = 1.0 \cdot 0.8831 + 0.8 \cdot 0.8371 + 0.6 \cdot 0.7377 = 1.9955$.
4. Measure the error. With $\langle \Psi, \Phi(z^*) \rangle = \beta$, the residual is $\|\Psi - \beta \Phi(z^*)\|^2 = \|\Psi\|^2 - \beta^2 = 4.1266 - 3.9818 = 0.1448$, so $\|\Psi - \beta \Phi(z^*)\| = 0.3805$.

Reading. One synthetic term captures the three-term vector with a relative error of $0.3805/2.0314 = 18.7\%$. Collapsing three bumps into one loses a fifth of the vector's length; adding a second reduced-set vector on the residual, the next pass of the construction loop, would shrink it further. This is the speed-accuracy dial that reduced-set methods turn.

Verification artifact. `checks/example-ch-invariance-example-20-2.json` records the example source hash and verification scope.

21.6.1 Sequential evaluation and face detection

Because reduced-set vectors are found one at a time, in decreasing order of importance, a classifier can be evaluated *sequentially*: run the first reduced-set vector everywhere, discard the locations it confidently rejects, and only spend the second vector on what remains, and so on (Romdhani et al. 2001). In face detection, where a binary face-versus-nonface classifier is scanned over every patch of an image at many scales, most patches are obvious non-faces that the first one or two vectors already reject. Romdhani et al. (2001) reduced an SVM with 1742 support vectors to 60 reduced-set vectors and, evaluating them sequentially, spent on average fewer than three kernel evaluations per image location, turning a slow classifier into one competitive with the fastest detectors of the time. This is the payoff for the reduced-set machinery and the natural complement to the VSV method that made the expansion large in the first place; see Chapter 5, Support Vector Machines for the base classifier and Chapter 37, Large-Scale Kernel Machines for the broader problem of making kernel evaluation cheap.

21.7 Summary

Two problems, one map. Incorporating invariances pushes prior knowledge forward through Φ : the Virtual Support Vector method generates transformed copies of the support vectors and retrains, invariance kernels whiten the tangent covariance so the classifier ignores motion along the orbit, tangent distance and translation-invariant kernels build the invariance into the metric, and jittering kernels fold the transformation into each evaluation. The pre-image problem pulls solutions back through Φ : exact pre-images rarely exist for Gaussian kernels, so a fixed-point iteration finds an approximate one, powering kernel PCA denoising and, iterated, the reduced-set expansions that compress a slow kernel machine into a fast one. The through-line is that a kernel is not just a similarity but a map with two directions, and controlling both is what makes kernel methods practical. The Chapter 19, Translation-Invariant, Semigroup, and Probabilistic Kernels supply the invariant kernels this chapter assumed, and Chapter 39, Deep Learning from a Kernel Point of View returns to invariance as the prior knowledge that modern architectures learn or hard-wire.

21.8 Common mistakes and practical implications

For **Invariances and the Pre-Image Problem**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

21.9 Summary and further reading

This chapter established explain the central definitions and claims in Invariances and the Pre-Image Problem; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results

and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (LeCun and Vapnik 1995), (Schölkopf and Vapnik 1996), (Simard and Victorri 1998).

21.10 Exercises

1. warm-up Explain in two sentences why the Virtual SV method generates virtual examples only from the support vectors rather than from the full training set, citing the property of the SVM solution that makes this sound. What is the one experimental observation from the USPS table that confirms nothing is lost by this restriction?
2. computation Using the setup of the pre-image example (points $x_1 = (0, 0)$, $x_2 = (2, 0)$, $x_3 = (1, 1.5)$, weights $\beta = (0.5, 0.3, 0.2)$, $\sigma = 1$), carry out one step of the fixed-point iteration by hand starting from $z_0 = (0.8, 0.3)$: compute the three weights $w_i = \beta_i e^{-\|x_i - z_0\|^2/2}$ and the updated $z_1 = \sum_i w_i x_i / \sum_i w_i$. Check that you recover $z_1 \approx (0.644, 0.246)$.
3. proof Derive the Gaussian-kernel fixed-point update from scratch. Starting from the objective $J(z) = \langle \Psi, \Phi(z) \rangle = \sum_i \alpha_i \exp(-\|x_i - z\|^2/2\sigma^2)$, compute $\nabla_z J$, set it to zero, and solve the resulting equation for z to obtain the weighted-average form. Where in the derivation do you use that the kernel depends on z only through $\|x_i - z\|^2$? Hint

Use $\nabla_z \exp(-\|x_i - z\|^2/2\sigma^2) = \exp(-\|x_i - z\|^2/2\sigma^2) (x_i - z)/\sigma^2$. The common factor $1/\sigma^2$ drops out of the equation $\sum_i \alpha_i e_i(z)(x_i - z) = 0$, which rearranges to $z = \sum_i \alpha_i e_i(z) x_i / \sum_i \alpha_i e_i(z)$.

4. proof Prove the optimal-coefficient formula $\beta = (K^z)^{-1} K^{zx} \alpha$ of Proposition (optimal expansion coefficients) by expanding $\|\Psi - \sum_i \beta_i \Phi(z_i)\|^2$ as a quadratic in β and minimizing. Then verify that in the one-term case $N_z = 1$ with a normalized kernel, this reduces to $\beta = \sum_i \alpha_i k(x_i, z)$, the value used in the reduced-set worked example. Hint

The objective is $\alpha^\top K^x \alpha - 2\beta^\top K^{zx} \alpha + \beta^\top K^z \beta$; its gradient in β is $-2K^{zx} \alpha + 2K^z \beta$. For $N_z = 1$, $K^z = k(z, z) = 1$.

5. proof Show that a Gaussian kernel expansion $\Psi = \sum_{j=1}^m \alpha_j \Phi(x_j)$ with at least two distinct x_j and all $\alpha_j \neq 0$ has no exact pre-image. Use the fact that distinct Gaussians are linearly independent as functions, so $\Phi(z) = \Psi$ would express one Gaussian as a nontrivial combination of Gaussians centered elsewhere. Hint

If $\Phi(z) = k(z, \cdot)$ equalled $\sum_j \alpha_j k(x_j, \cdot)$ as functions, then the linear independence of Gaussians centered at distinct points forces a contradiction unless the expansion has a single term.

6. computation Take the three single-pixel images $A = (1, 0, 0)$, $B = (0, 1, 0)$, $C = (0, 0, 1)$ and a linear kernel. Under one-pixel translation the jittered distances give $d(A, B) = d(B, C) = 0$. Compute $d(A, C)^2 = \|A\|^2 - 2\langle A, C \rangle + \|C\|^2$ directly and confirm the triangle inequality $d(A, C) \leq d(A, B) + d(B, C)$ fails. What does this tell you about whether a jittering kernel is positive definite?
7. exploration The invariance-kernel preprocessing replaces the dot product by $\langle x, x' \rangle_{T_\gamma^{-1}}$ with $T_\gamma = (1 - \gamma)T + \gamma I$ and T the tangent covariance. Explain what the two limits $\gamma \rightarrow 1$ and $\gamma \rightarrow 0$ do, and why diagonalizing T_γ and rescaling by inverse square-root eigenvalues amounts to downweighting the directions in which the data vary most under the transformation. Relate this to whitening in principal component analysis. Hint

At $\gamma = 1$, $T_\gamma = I$ and the ordinary SVM is recovered. As $\gamma \rightarrow 0$, $T_\gamma \rightarrow T$ and the large-eigenvalue (high tangent-variance) directions are most strongly suppressed by $T^{-1/2}$.

8. challenge In the reduced-set construction algorithm, argue that iterating one-term pre-images greedily need not reach the globally optimal N_z -term reduced set, which motivates the phase-II joint optimization. Then explain why, for classification specifically, it is acceptable that the residual $\|\Psi - \Psi'\|$ does not fall to zero, by contrasting the quantity the reduced set minimizes with the quantity that actually governs test error. Hint

21 Invariances and the Pre-Image Problem

Greedy pre-imaging fixes earlier z_i before later ones are chosen, so it optimizes a restricted problem. For classification the object of interest is $\text{sgn}(\sum_j \beta_j k(z_j, x) + \tilde{b})$ integrated against the data distribution, not $\|\Psi - \Psi'\|$; re-optimizing the threshold $\tilde{b}$ recovers much of the accuracy even when the feature-space residual is nonzero.

22 Kernels for Sequences

Much of the data that kernel methods were built to attack does not live in $\mathbb{R}^d$. A protein is a string over a twenty-letter alphabet; a gene is a string over four; a document is a string over the characters of a language. This chapter builds kernels directly on such strings. We start from the biology that made the problem urgent, then develop three families: kernels from explicit feature maps indexed by short substrings (the spectrum, mismatch, and subsequence kernels, with their string-algorithm implementations), kernels derived from generative models of sequences (context-tree models, hidden Markov models, and stochastic context-free grammars for RNA), and finally the local alignment kernel, whose positive definiteness we prove in full by a convolution argument. Throughout, the recurring theme is that a good sequence kernel must be mathematically valid, fast to compute by dynamic programming, and adapted to the biology of the task.

22.1 Why sequences, and why kernels

The central dogma of molecular biology reads a one-dimensional string in two stages: DNA, a string over the four bases $\{A, C, G, T\}$, is transcribed into RNA and then translated into protein, a string over the twenty-letter alphabet of amino acids. A protein is therefore literally a variable-length word. Human insulin, for instance, is the chain

FVNQHLCGSHLVEALYLVCGERGFFYTPKA

The Human Genome Project set out to sequence all three billion bases of the human genome. It ran as a consortium of some twenty laboratories across six countries and cost between half a billion and one billion US dollars, and it closed the genomics era in 2003 with a set of surprises: only about 25,000 genes, representing a mere 1.2% of the genome, with the remaining 97% long dismissed as “junk DNA” but now understood to superimpose many biological signals still being discovered. Automatic gene finding was already being done with graphical models. Since then the cost of sequencing has collapsed far faster than Moore’s law, so sequences are produced at a rate that no human can annotate. The bottleneck moved from generating strings to comparing, classifying, and analyzing them.

A representative task makes the setting concrete.

Example (secreted protein prediction)

We are given a training set of protein sequences, each labelled secreted or non-secreted, for example the secreted MASKATLLLAFTLLFATCIARHQRRQQQNCQLQNIEA . . . against the non-secreted MAPPSVFAEVPQAQPVLVFKLIADFREDPDPRKVN LGVG . and we want a classifier that predicts whether a new protein is secreted. The label depends on subtle regularities of the amino-acid string, not on any obvious numeric feature.

Verification artifact. checks/example-ch09-example-21-1.json records the example source hash and verification scope.

Kernel methods are a natural fit. Since the seminal paper of Jaakkola, Diekhans, and Haussler (2000), the entire kernel toolbox, from the support vector machine to kernel PCA, becomes available on strings the moment we exhibit a valid kernel $K(x, x')$ between two sequences. The design question is therefore the only question, and a good string kernel must satisfy three demands at once:

- it must be *mathematically valid*, that is symmetric and positive definite (or conditionally positive definite);
- it must be *fast to compute*, because sequences are long and numerous;
- it must be *adapted to the problem*, encoding the right notion of biological similarity so that it actually performs.

There is no single best answer. Kernel engineering for sequences has produced three broad strategies, which organize the rest of this chapter: define an explicit, possibly very high dimensional, feature space (physico-chemical kernels; the spectrum, mismatch, and subsequence kernels; motif and pairwise kernels); derive a kernel from a generative model (the Fisher kernel, mutual information kernels, and marginalized kernels); or derive a kernel from a similarity measure (the local alignment kernel).

22.2 Supervised classification by vector embedding

The simplest idea is the most direct: map each string $x \in \mathcal{X}$ to a vector $\Phi(x) \in \mathcal{F}$ in some feature space, and then train any vectorial classifier, a nearest-neighbour rule, a perceptron, logistic regression, or a support vector machine, on the images $\Phi(x_1), \dots, \Phi(x_n)$ of the training strings. Every choice of embedding Φ yields a kernel $K(x, x') = \langle \Phi(x), \Phi(x') \rangle_{\mathcal{F}}$, and conversely we will often specify Φ only implicitly through K . Two questions remain: what should the coordinates of $\Phi(x)$ measure, and can the inner product be computed without ever forming $\Phi(x)$ explicitly?

22.2.1 Physico-chemical embeddings

One family of coordinates comes from chemistry. Each amino acid carries numerical properties (polarity, hydrophobicity, and so on), so a sequence $x = x_1 \dots x_n$ induces a numerical time series $h_1, \dots, h_n$, where h_i is the chosen property of residue x_i . We then summarize this series by a fixed-length vector of features: its length, its Fourier coefficients (Wang et al., 2004), or its autocorrelation coefficients (Zhang et al., 2003)

$$r_j = \frac{1}{n-j} \sum_{i=1}^{n-j} h_i h_{i+j}, \quad j = 1, 2, \dots$$

Such physico-chemical kernels are cheap and interpretable, but they discard the identity of the residues in favour of a coarse numerical profile. The kernels that follow keep the letters.

22.2.2 The vector space model and text kernels

The oldest string data is human text, and its kernel is the ancestor of every count kernel below. Information retrieval represents a document as a *bag of words*: the vector $\Phi(d) = (\text{tf}(t, d))_{t \in \mathcal{D}}$ indexed by a dictionary $\mathcal{D}$ of terms, whose coordinate for term t is the frequency $\text{tf}(t, d)$ with which t occurs in d . Word order and punctuation are discarded, so a phrase such as "bread winner" breaks into its constituents. This is the vector space model of Salton et al. (1975), and as Shawe-Taylor and Cristianini (2004) observe, reading it as a feature map turns the whole information-retrieval toolbox into a kernel method.

Definition (bag-of-words kernel)

For documents d, d' over a dictionary $\mathcal{D}$,

$$K(d, d') = \langle \Phi(d), \Phi(d') \rangle = \sum_{t \in \mathcal{D}} \text{tf}(t, d) \text{tf}(t, d').$$

Being an explicit inner product it is positive definite, and, exactly like the spectrum kernel, it is sparse: only terms present in both documents contribute.

The dictionary can hold hundreds of thousands of terms, so $\Phi(d)$ is never formed: each document is tokenized once into a sorted list of (term, count) pairs, and the kernel is read off by merging the two lists, multiplying counts whenever the term numbers coincide, in time $O(|d| + |d'|)$, the text analogue of the spectrum kernel's pre-indexation.

Raw counts weight every word equally, which is wasteful: "the" carries no topic while "homology" carries a great deal. The remedy is *tf-idf* weighting, a diagonal reweighting $R_{tt} = w(t)$ of the feature space with $w(t) = \log(\ell/\text{df}(t))$, where $\text{df}(t)$ is the number of documents containing t and ℓ the size of the corpus. Rare, informative terms are amplified and ubiquitous ones suppressed. The reweighted kernel is

$$\tilde{K}(d, d') = \sum_{t \in \mathcal{D}} w(t)^2 \text{tf}(t, d) \text{tf}(t, d'),$$

and because the weights use no labels they can be estimated from any large background corpus. Dividing by $\sqrt{K(d, d) K(d', d')}$ normalizes away document length, the cosine similarity of retrieval.

Both kernels still call two documents unrelated when they share no terms, even if one says "physician" where the other says "doctor". Semantic similarity has to be injected through a term proximity matrix P with $P_{tt'} > 0$ when t, t' are related, giving $\tilde{K}(d, d') = \Phi(d) P P^T \Phi(d')^T$, which represents each document by the less sparse vector $\Phi(d)P$ that fires on every term close to those it contains. The most useful P is learned: *latent semantic indexing* (Deerwester et al., 1990) takes P from the leading singular vectors of the term-document matrix, so terms that co-occur across the corpus collapse onto shared latent concepts. Shawe-Taylor and Cristianini (2004) note that this is precisely kernel PCA in the bag-of-words feature space, and its dual form, the *latent semantic kernel*, evaluates the concept projection from document inner products alone, so it layers on top of any base text kernel.

22.3 Substring indexation and the spectrum kernel

The second and more powerful family indexes the feature space by short strings. Fix an integer k and let $\mathcal{A}$ be the alphabet. For each word $u \in \mathcal{A}^k$ of length k , let $\Phi_u(x)$ be some measure of how u appears in x , and set

$$\Phi(x) = (\Phi_u(x))_{u \in \mathcal{A}^k}, \quad K(x, x') = \sum_{u \in \mathcal{A}^k} \Phi_u(x) \Phi_u(x').$$

Different choices of Φ_u give different kernels, all sharing the same $|\mathcal{A}|^k$ -dimensional index set:

- the number of occurrences of u in x , without gaps, gives the *spectrum kernel* (Leslie et al., 2002);
- the number of occurrences of u up to m mismatches, without gaps, gives the *mismatch kernel* (Leslie et al., 2004);
- the number of occurrences of u allowing gaps, each occurrence weighted by an exponentially decaying factor of the gap length, gives the *subsequence kernel* (Lodhi et al., 2002).

22.3.1 The spectrum kernel

The k -spectrum of a string lists all of its contiguous length- k windows. For $x = \text{CGGSLIAMMWFGV}$ the 3-spectrum is

$$(\text{CGG}, \text{GGS}, \text{GSL}, \text{SLI}, \text{LIA}, \text{IAM}, \text{AMM}, \text{MMW}, \text{MWF}, \text{WFG}, \text{FGV}).$$

Definition (spectrum kernel)

Let $\Phi_u(x)$ be the number of occurrences of the k -mer u as a contiguous substring of x . The k -spectrum kernel is

$$K(x, x') = \sum_{u \in \mathcal{A}^k} \Phi_u(x) \Phi_u(x').$$

Being an inner product of explicit feature vectors, it is positive definite by construction.

The intuition is compositional. Each coordinate of $\Phi(x)$ reports how often one particular short word occurs, so $\Phi(x)$ is a histogram of the local k -letter content of the string, blind to where in the string each word sits. The kernel is large exactly when two strings are built from the same short pieces in similar proportions, which is a sensible first notion of biological relatedness: conserved motifs survive as shared k -mers even after the surrounding sequence has drifted. Raising k sharpens the features, since longer words are rarer and more specific, but also makes them scarcer, so the choice of k trades specificity against the number of coincidences the kernel has to work with.

Naively the sum has $|\mathcal{A}|^k$ terms, an astronomically large number. The saving grace is sparsity: a string of length $|x|$ contains at most $|x| - k + 1$ distinct windows, so $\Phi(x)$ has at most $|x| - k + 1$ non-zero coordinates. The kernel is a sparse dot product, and everything hinges on how we align the non-zero coordinates of the two strings.

It helps to watch the sparse inner product on a tiny case before worrying about speed.

Example (a 2-spectrum inner product)

Take $k = 2$ and the DNA strings $x = \text{AGGA}$ and $x' = \text{AGGT}$. Their contiguous 2-mers are

$$x : (\text{AG}, \text{GG}, \text{GA}), \quad x' : (\text{AG}, \text{GG}, \text{GT}),$$

so the only nonzero coordinates of $\Phi(x)$ are $\Phi_{\text{AG}} = \Phi_{\text{GG}} = \Phi_{\text{GA}} = 1$, and similarly for x' . The kernel keeps only the coordinates the two strings share,

$$K(x, x') = \underbrace{1 \cdot 1}_{\text{AG}} + \underbrace{1 \cdot 1}_{\text{GG}} = 2,$$

since GA and GT each occur in one string only. The index set has $4^2 = 16$ entries, but we ever touched three per string: the work tracks the strings, not the alphabet.

Verification artifact. checks/example-ch09-example-21-2.json records the example source hash and verification scope.

22.3.1.1 Computing the spectrum kernel

With *pre-indexation*, we walk once through each string, hashing every length- k window to an integer and tallying counts, then intersect the two tallies. Both the counting and the intersection are linear, so

$$K(x, x') \text{ costs } O(|x| + |x'|).$$

Classification of a new sequence is just as fast. If $f(x) = w \cdot \Phi(x)$ is a learned linear score, then

$$f(x) = \sum_u w_u \Phi_u(x) = \sum_{i=1}^{|x|-k+1} w_{x_i \dots x_{i+k-1}},$$

a single sweep summing one weight per window, in $O(|x|)$. The spectrum kernel therefore works on any strings at all, natural language, time series, log files, and is a fast, scalable default for string classification; its mismatch variants extend it to approximate matching.

When we cannot afford to pre-index (for instance because the vocabulary is built on the fly), classical string data structures recover almost the same speed. A *retrieval tree* or *trie* stores every observed k -mer along a root-to-leaf path of length k ; for the sequence ACGTTTAACGTAC the depth-one nodes are A, C, G, T, the depth-two nodes are AA, AC, CG, GT, TA, TT, and so on down to the k -mers at the leaves. Traversing both strings through a shared trie computes $K(x, x')$ in $O(k(|x| + |x'|))$. A *suffix tree* compresses this further and restores the linear $O(|x| + |x'|)$ bound, at the price of a larger constant.

The (k, m) -mismatch kernel reuses this tree but is more expensive, because each observed k -mer now matches not one leaf but its entire *mismatch neighbourhood*, the set of words within m substitutions. The mismatch-tree algorithm of Leslie and Kuang (2004) enumerates that neighbourhood by depth-first traversal of a trie of depth k , allowing up to m mismatched branches along each root-to-leaf path; the number of feasible paths is $O(k^m |\mathcal{A}|^m)$, and processing the $O(|x| + |x'|)$ input k -mers against them gives the standard bound

$$O(k^{m+1} |\mathcal{A}|^m (|x| + |x'|)),$$

which reduces to the spectrum cost $O(k(|x| + |x'|))$ when $m = 0$. The table below summarizes the trade-offs.

Kernel	Data structure	Time and memory
spectrum	pre-indexation or suffix tree	$O(x + x')$
spectrum	trie	$O(k(x + x'))$
(k, m) -mismatch	mismatch tree (trie)	$O(k^{m+1} \mathcal{A} ^m (x + x'))$
subsequence (gapped)	dynamic programming	$O(k x x')$

22.4 The subsequence kernel and its dynamic program

The spectrum kernel demands that a k -mer appear as an *uninterrupted* block. Biological similarity is more forgiving: the same motif may recur with insertions between its letters. The subsequence kernel rewards such gapped matches, but penalizes gaps by an exponential decay, so that a spread-out match counts for less than a tight one.

22.4.1 Definition

We first name the gapped occurrences. For $1 \leq k \leq n$, let $\mathcal{J}(k, n)$ be the set of strictly increasing index sequences $\mathbf{i} = (i_1, \dots, i_k)$ with $1 \leq i_1 < i_2 < \dots < i_k \leq n$. For a string $x = x_1 \dots x_n$ and $\mathbf{i} \in \mathcal{J}(k, |x|)$, the selected subsequence and its span are

$$x(\mathbf{i}) = x_{i_1} x_{i_2} \dots x_{i_k}, \quad l(\mathbf{i}) = i_k - i_1 + 1.$$

The span $l(\mathbf{i})$ is the width of the window actually used, so it exceeds k exactly by the number of gapped positions.

Example

In $x = \text{ABRACADABRA}$, the index sequence $\mathbf{i} = (3, 4, 7, 8, 10)$ selects $x(\mathbf{i}) = \text{RADAR}$ with span $l(\mathbf{i}) = 10 - 3 + 1 = 8$, so three positions were skipped.

Verification artifact. checks/example-ch09-example-21-3.json records the example source hash and verification scope.

Definition (subsequence kernel)

Fix $k \in \mathbb{N}$ and a decay $\lambda \in (0, 1]$. For every $u \in \mathcal{A}^k$ define the feature

$$\Phi_u(x) = \sum_{\mathbf{i} \in \mathcal{J}(k, |x|) : x(\mathbf{i})=u} \lambda^{l(\mathbf{i})},$$

so each gapped occurrence of u contributes λ^{span} . The subsequence kernel is

$$K_{k,\lambda}(x, x') = \sum_{u \in \mathcal{A}^k} \Phi_u(x) \Phi_u(x').$$

Example ($k = 2$)

With the four strings cat, car, bat, bar and the 2-mers $u \in \{ca, ct, at, ba, bt, cr, ar, br\}$, the non-zero features are

$$\Phi(\text{cat}) = (\lambda^2, \lambda^3, \lambda^2, 0, 0, 0, 0, 0), \quad \Phi(\text{car}) = (\lambda^2, 0, 0, 0, 0, \lambda^3, \lambda^2, 0),$$

(the ct in cat is gapped, hence λ^3). Reading off inner products,

$$K(\text{cat}, \text{cat}) = K(\text{car}, \text{car}) = 2\lambda^4 + \lambda^6, \quad K(\text{cat}, \text{car}) = \lambda^4, \quad K(\text{cat}, \text{bar}) = 0.$$

The shared ca alone links cat and car, while cat and bar have no common 2-mer.

Verification artifact. checks/example-ch09-example-21-4.json records the example source hash and verification scope.

22.4.2 Computing the kernel by dynamic programming

Expanding the definition,

$$K_{k,\lambda}(x, x') = \sum_{u \in \mathcal{A}^k} \sum_{i: x(i)=u} \sum_{i': x'(i')=u} \lambda^{l(i)+l(i')}.$$

The idea that rescues us is recursion on the letters. A gapped occurrence of a length- k word is a gapped occurrence of its length- $(k-1)$ prefix, followed somewhere to the right by one more matched letter, and the decay weight λ^{span} factorizes across that boundary into the prefix's weight times the decay paid to reach the new letter. So if we scan a string one letter at a time, the length- k bookkeeping can be updated from the length- $(k-1)$ bookkeeping we already hold. The rest of this subsection turns that single observation into a table.

Enumerating index sequences is hopeless: there are on the order of $|x|^k$ of them. The way out, due to Lodhi et al. (2002), is to introduce an auxiliary feature in which the span is measured not to the last selected letter but to the *end of the string*. Alongside

$$\Phi_u(x) = \sum_{i: x(i)=u} \lambda^{i_k - i_1 + 1} \quad \text{define} \quad \Psi_u(x) = \sum_{i: x(i)=u} \lambda^{|x| - i_1 + 1}.$$

The auxiliary Ψ keeps a "gap account open" to the right end of the string, which is exactly what makes it compose cleanly when we append a letter. Writing $u = va$ with $a \in \mathcal{A}$ its last letter and $x_{[1,j]} = x_1 \dots x_j$, a direct rewriting gives

$$\Phi_{va}(x) = \sum_{j: x_j=a} \Psi_v(x_{[1,j-1]}) \lambda, \quad \Psi_{va}(x) = \sum_{j: x_j=a} \Psi_v(x_{[1,j-1]}) \lambda^{|x| - j + 1}.$$

From these follow the incremental update rules when a single letter a is appended to a string x , forming xa . If the last letter of u is *not* a , appending a cannot end a new occurrence of u , so Φ is unchanged while Ψ simply pays one more unit of decay:

$$\Phi_u(xa) = \Phi_u(x), \quad \Psi_u(xa) = \lambda \Psi_u(x).$$

If instead $u = va$ ends in a , the new letter can close an occurrence whose prefix v sat in x :

$$\Phi_{va}(xa) = \Phi_{va}(x) + \lambda \Psi_v(x), \quad \Psi_{va}(xa) = \lambda \Psi_{va}(x) + \lambda \Psi_v(x).$$

The two update rules become concrete once we grow a short string letter by letter and watch the auxiliary features accumulate.

Example (building Ψ one letter at a time)

Take single-letter words $v \in \{c, a\}$ and use the convention $\Psi_{\emptyset}(x) = 1$ for the empty word. Start from the empty string, where every Ψ is 0, and append c then a to reach ca . Appending c to the empty string closes a fresh occurrence of c :

$$\Psi_c(c) = \lambda \Psi_c(\emptyset) + \lambda \Psi_{\emptyset}(\emptyset) = \lambda \cdot 0 + \lambda \cdot 1 = \lambda.$$

Now append a . The word c does not end in a , so it just pays one more unit of decay, while a closes a new occurrence:

$$\Psi_c(ca) = \lambda \Psi_c(c) = \lambda^2, \quad \Psi_a(ca) = \lambda \Psi_a(c) + \lambda \Psi_{\emptyset}(c) = 0 + \lambda = \lambda.$$

Both agree with the closed form $\Psi_b(ca) = \lambda^{|x|-i+1}$ read off directly: c sits at position 1, giving $\lambda^{2-1+1} = \lambda^2$, and a sits at position 2, giving $\lambda^{2-2+1} = \lambda$. The "gap account to the right end" is precisely the exponent $|x| - i + 1$, and appending a letter is what advances it.

Verification artifact. checks/example-ch09-example-21-5.json records the example source hash and verification scope.

We now bundle these features into two kernels indexed by the common length k ,

$$B_k(x, x') = \sum_{u \in \mathcal{A}^k} \Psi_u(x) \Psi_u(x'), \quad K_k(x, x') = \sum_{u \in \mathcal{A}^k} \Phi_u(x) \Phi_u(x'),$$

so B_k is the auxiliary companion of the target $K_k = K_{k,\lambda}$. Their boundary conditions are

$$B_0(x, x') = K_0(x, x') = 1 \text{ for all } x, x', \quad B_k(x, x') = K_k(x, x') = 0 \text{ if } \min(|x|, |x'|) < k.$$

Summing the update rules over u turns them into recursions on the strings. Appending a to the first argument,

$$B_k(xa, x') = \lambda B_k(x, x') + \sum_{j: x'_j=a} B_{k-1}(x, x'_{[1,j-1]}) \lambda^{|x'|-j+2}.$$

To remove the inner sum over j , we append a letter to the second argument as well. Writing the second string as $x'b$ and using the identity above once in each argument yields the fully incremental form

$$B_k(xa, x'b) = \lambda B_k(x, x'b) + \lambda B_k(xa, x') - \lambda^2 B_k(x, x') + \delta_{a=b} \lambda^2 B_{k-1}(x, x'),$$

where $\delta_{a=b}$ is 1 when the two appended letters agree and 0 otherwise. It is worth reading this recursion as an inclusion-exclusion. The first term $\lambda B_k(x, x'b)$ collects the occurrences that ignore the newly appended a on the left while paying it one unit of decay, and the second term $\lambda B_k(xa, x')$ does the same for b on the right; but both terms count the occurrences that ignore *both* new letters, so those are added twice and must be removed once, which is the correction $-\lambda^2 B_k(x, x')$. Finally, an occurrence that genuinely uses the two new positions can exist only when $a = b$, and it extends a length- $(k - 1)$ match of the old prefixes, which is the term $\delta_{a=b} \lambda^2 B_{k-1}(x, x')$. Each entry $B_k(xa, x'b)$ now depends only on three neighbours in the same table plus one entry of the table for

$k - 1$, so it is a genuine two-dimensional dynamic program. The target kernel is recovered by a companion recursion that reuses B_{k-1} ,

$$K_k(xa, x') = K_k(x, x') + \lambda^2 \sum_{j: x'_j=a} B_{k-1}(x, x'_{[1,j-1]}),$$

and $K_{k,\lambda}(x, x') = K_k(x, x')$ once the tables are complete. Filling the B tables for orders $1, \dots, k - 1$ and then the K table costs $O(k |x| |x'|)$ time and memory. In pseudocode:

```

Input: strings x, x', order k, decay lambda
# B[p] is the (|x|+1) x (|x'|+1) table for order p
B[0][i][j] = 1 for all 0 <= i <= |x|, 0 <= j <= |x'|
for p = 1.. k-1:
  B[p][i][j] = 0 whenever i < p or j < p # base case
  for i = p.. |x|:
    for j = p.. |x'|:
      a = x[i]; b = x'[j]
      B[p][i][j] = lambda*B[p][i-1][j]
      + lambda*B[p][i][j-1]
      - lambda*lambda*B[p][i-1][j-1]
      + (a==b ? lambda*lambda*B[p-1][i-1][j-1] : 0)

K[i][j] = 0 for all i, j
for i = k.. |x|:
  for j = 1.. |x'|:
    a = x[i]
    s = 0
    for m = 1.. j: # sum over positions of x' equal to a
      if x'[m] == a: s += B[k-1][i-1][m-1]
    K[i][j] = K[i-1][j] + lambda*lambda*s
return K[|x|][|x'|]
```

(The innermost sum over m can itself be maintained incrementally, which is what keeps the overall cost at $O(k |x| |x'|)$ rather than a factor of $|x'|$ more.) The feature space has dimension $|\mathcal{A}|^k$, so downstream learning must be regularized; a support vector machine on this kernel is the standard choice.

Example (tracing the tables on *ca* against *ca*)

Take $k = 2$ and compute $K_{2,\lambda}(ca, ca)$, writing $B_p(i, j)$ and $K(i, j)$ for the prefixes $x_{[1,i]}$ and $x'_{[1,j]}$. The order-zero table is $B_0(i, j) = 1$ everywhere, and any table entry with an empty argument is 0. Filling B_1 by the incremental rule,

$$B_1(1, 1) = \lambda^2, \quad B_1(1, 2) = B_1(2, 1) = \lambda^3, \quad B_1(2, 2) = \lambda^4 + \lambda^4 - \lambda^4 + \lambda^2 = \lambda^4 + \lambda^2,$$

where the corner $B_1(2, 2)$ shows the inclusion-exclusion at work: the two extension terms each contribute λ^4 , one copy is subtracted, and the matched pair $a = a$ adds $\lambda^2 B_0(1, 1)$. This agrees with the direct reading $B_1(ca, ca) = \Psi_c^2 + \Psi_a^2 = (\lambda^2)^2 + \lambda^2$. Now the companion recursion for K fires only once, at $i = j = 2$ with appended letter a :

$$K(2, 2) = K(1, 2) + \lambda^2 \sum_{j: x'_j=a} B_1(x_{[1,1]}, x'_{[1,j-1]}) = 0 + \lambda^2 B_1(c, c) = \lambda^2 \cdot \lambda^2 = \lambda^4.$$

The single shared gap-free 2-mer *ca* has span 2, so $\Phi_{ca}(ca) = \lambda^2$ and the kernel is $(\lambda^2)^2 = \lambda^4$, exactly what the tables return.

Verification artifact. checks/example-ch09-example-21-6.json records the example source hash and verification scope.

Shawe-Taylor and Cristianini (2004) note one alternative worth keeping in mind: when only a bounded number m of gaps is permitted, the kernel can be evaluated with a *trie* instead of the dynamic program, at cost $O((|x| + |x'|)(k + m)^m)$, which wins over the $O(k |x| |x'|)$ table when the alphabet is large or few gaps are allowed.

22.4.3 Dictionary-based indexation

A third way to build an explicit embedding replaces "index by all k -mers" with "index by a chosen dictionary". Pick a dictionary of reference sequences $D = (x_1, \dots, x_n)$ and a similarity measure $s(x, x')$, and set

$$\Phi_D(x) = (s(x, x_i))_{x_i \in D}.$$

Two instances are common. In *motif kernels* (Logan et al., 2001) the dictionary is a library of known biological motifs and s is a matching function, so each coordinate reports how strongly x hits one motif. In the *pairwise kernel* (Liao and Noble, 2003) the dictionary is the training set itself and s is a classical sequence-similarity score; the embedding of x is its vector of similarities to all training sequences. Both trade a canonical feature space for one tailored by prior knowledge.

22.5 Kernels from generative models

Probabilistic modeling of biological sequences predates kernel design by decades, and it is a shame to waste it. The models come as parametric families $\{P_\theta : \theta \in \Theta \subset \mathbb{R}^m\}$ of distributions over sequences: hidden Markov models for proteins, stochastic context-free grammars for RNA. This section turns such models into kernels, so that the biological structure they encode transfers to the kernel machine.

22.5.1 P-kernels and conditional independence

Before the individual constructions, it pays to name the umbrella they live under. Haussler (1999) asked when a probability distribution is itself a kernel: if a symmetric joint distribution $P(x, z)$ over pairs from $\mathcal{X}$ has a positive semidefinite matrix $(P(x_i, x_j))_{i,j}$ on every finite sample, then $K(x, z) = P(x, z)$ is a valid kernel.

Definition (P-kernel, Haussler 1999)

A symmetric joint distribution $P(x, z)$ on $\mathcal{X} \times \mathcal{X}$ that satisfies the finitely positive semidefinite property is a *P-kernel* on $\mathcal{X}$. Any kernel with finite total mass $\sum_{x,z} K(x, z) = M < \infty$ rescales to one, since $P = K/M$ is such a distribution.

Not every distribution qualifies: on $\mathcal{X} = \{x_1, x_2\}$ the symmetric law that puts all its mass off the diagonal, $P(x_1, x_2) = P(x_2, x_1) = \frac{1}{2}$ and $P(x_1, x_1) = P(x_2, x_2) = 0$, has Gram matrix with eigenvalues $\pm \frac{1}{2}$, a negative direction. What rescues the construction in general is a hidden variable.

Proposition (conditional independence suffices)

Let m index a family of models with prior $P_M(m)$, and suppose x and z are conditionally independent given m ,

$$P(x, z \mid m) = P(x \mid m) P(z \mid m).$$

Then the marginal

$$P(x, z) = \sum_m P(x \mid m) P(z \mid m) P_M(m)$$

is a P-kernel.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

For each fixed m the term $P(x \mid m)P(z \mid m)$ is a rank-one product of a function of x with the same function of z , hence positive definite, and the marginal is a non-negative combination of these, again p.d. $\square$

Conditional independence is sufficient but not necessary, and it is exactly the engine behind the marginalized and generative kernels below: the model index m plays the role of the latent variable, and integrating a valid complete-data kernel over independent draws of the hidden layer can never destroy positive definiteness. Watkins (2000) developed the same conditional-independence idea independently, including its application to hidden Markov models.

22.5.2 Context-tree models and the context-tree kernel

A context-tree model is a Markov chain of variable memory: how many previous symbols the next symbol depends on is itself a function of those symbols. Formally

$$P_{D,\theta}(x) = P_{D,\theta}(x_1 \dots x_D) \prod_{i=D+1}^n P_{D,\theta}(x_i \mid x_{i-D} \dots x_{i-1}),$$

where D is a suffix tree telling us, for each context, how far back to look, and $\theta \in \Sigma_D$ is a collection of conditional multinomials. The tree lets short, predictive contexts stay short and only deepens where the data demand it.

Example

For a suitable tree,

$$P(\text{AABACBACC}) = P(\text{AAB}) \theta_{AB}(A) \theta_A(C) \theta_C(B) \theta_{ACB}(A) \theta_A(C) \theta_C(C),$$

each factor conditioning on just enough preceding context (here AB , A , C , ACB , and so on) to predict the next letter.

Verification artifact. checks/example-ch09-example-21-7.json records the example source hash and verification scope.

Rather than fit one tree and one θ , we average over all of them under a prior, which yields a kernel.

Theorem (context-tree kernel, Cuturi et al., 2005)

For particular choices of priors $w(d\theta \mid D)$ and $\pi(D)$, the context-tree kernel

$$K(x, x') = \sum_D \int_{\theta \in \Sigma_D} P_{D,\theta}(x) P_{D,\theta}(x') w(d\theta \mid D) \pi(D)$$

can be computed in $O(|x| + |x'|)$ by a variant of the Context-Tree Weighting algorithm. It is a valid mutual-information kernel, and its value is tied to an information-theoretic measure of the mutual information between the two strings.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

22.5.3 Marginalized kernels

The context-tree kernel is one instance of a general and useful construction. Suppose each observed sequence x hides a latent labelling y , coupled to it by a conditional $P_x(dy)$, and suppose we know a kernel K_Z on the complete data $z = (x, y)$. We cannot see y , so we average K_Z over the posterior of the hidden variables. The move is worth dwelling on: had we observed the latent y , comparison would be easy, because the complete data carries the biological annotation (which hidden state generated each residue) that we actually care about. Since y is hidden, the honest thing is to compare the two strings under every explanation of the hidden layer at once, weighting each explanation by how plausible the generative model finds it. Averaging a valid kernel over independent draws of the latent variables keeps the result valid, so the whole apparatus of the generative model, fitted with decades of biological knowledge, is smuggled into a legitimate kernel at no cost to positive definiteness.

Definition (marginalized kernel, Tsuda et al., 2002)

$$K_{\mathcal{X}}(x, x') = \mathbb{E}_{P_x(dy) \times P_{x'}(dy')} K_Z(z, z') = \int \int K_Z((x, y), (x', y')) P_x(dy) P_{x'}(dy').$$

Being an expectation of a kernel over independent latent draws, $K_{\mathcal{X}}$ is again a valid kernel on $\mathcal{X}$.

Concretely, take a hidden Markov model whose hidden states are a normal coin N and a biased coin B , emitting bits with $\pi(0 | N) = 0.5$ and $\pi(0 | B) = 0.2$. A realization is a pair $z = (x, y)$ of an emitted bit string x and its hidden state string y , for example the states $y = \text{NNNNNB}BB \dots$ driving the bits $x = 10010111 \dots$. If we could see z , a natural complete-data kernel is the 1-spectrum on the joint alphabet $\mathcal{A} \times \mathcal{S}$ of (bit, state) pairs,

$$K_Z(z, z') = \sum_{(a,s) \in \mathcal{A} \times \mathcal{S}} n_{a,s}(z) n_{a,s}(z'),$$

where $n_{a,s}(x, y)$ counts the positions whose hidden state is s and whose emitted symbol is a . Marginalizing over the unseen states gives a kernel on the observed bits alone,

$$K_{\mathcal{X}}(x, x') = \sum_{y, y'} K_Z((x, y), (x', y')) P(y | x) P(y' | x') = \sum_{(a,s)} \Phi_{a,s}(x) \Phi_{a,s}(x'),$$

with the marginalized feature

$$\Phi_{a,s}(x) = \sum_y P(y | x) n_{a,s}(x, y).$$

The last object looks like an intractable sum over all hidden paths, but the count $n_{a,s}$ is additive over positions, so the sum collapses:

$$\Phi_{a,s}(x) = \sum_y P(y | x) \sum_{i=1}^n \delta(x_i, a) \delta(y_i, s) = \sum_{i=1}^n \delta(x_i, a) \sum_y P(y | x) \delta(y_i, s) = \sum_{i=1}^n \delta(x_i, a) P(y_i = s | x).$$

The final expression has a clean reading. The hard count $n_{a,s}$, which would tally the positions where symbol a is emitted from state s , is replaced by a soft count: at each position that emits a , we add not one but the posterior probability $P(y_i = s | x)$ that the hidden coin was in state s there. The feature is the expected number of (a, s) events under the model's own beliefs about the hidden path, so a position the model is unsure about contributes fractionally to several states rather than committing to one. Each posterior marginal $P(y_i = s | x)$ is computed by the standard forward-backward algorithm, so the whole feature vector, and hence the kernel, is obtained in time linear in the sequence length times the number of hidden states. This same recipe underlies the spectrum kernel on the hidden states of a protein HMM (Tsuda et al., 2002), the marginalized kernels on labeled sequences and graphs of Kashima, Tsuda, and Inokuchi (2003), and phylogenetic kernels on multiple alignments (Vert, 2002).

22.5.4 Fixed-length and pair hidden Markov model kernels

The marginalized construction above softens a complete-data kernel over a posterior. A complementary route marginalizes the *joint generation probability* of the two strings directly: if a hidden model can emit both s and t , then

$$K(s, t) = \sum_h P(s | h) P(t | h) P(h)$$

is a P-kernel by conditional independence, evaluated by dynamic programming. Shawe-Taylor and Cristianini (2004) isolate two cases worth stating.

First take s and t of equal length n , emitted symbol by symbol from a hidden state path $h = h_1 \dots h_n$ that follows a first-order Markov chain, $P(h) = P_M(h_1) \prod_{i \geq 2} P_M(h_i | h_{i-1})$. Grouping the sum by the final state, let $\kappa_{k,a}(s, t)$ collect the paths of length k ending in state a . Then $\kappa_{1,a}(s, t) = P(s_1 | a) P(t_1 | a) P_M(a)$, and appending one position gives the recursion

$$\kappa_{k+1,a}(s, t) = P(s_{k+1} | a) P(t_{k+1} | a) \sum_{b \in A} P_M(a | b) \kappa_{k,b}(s, t),$$

with $K(s, t) = \sum_{a \in A} \kappa_{n,a}(s, t)$. Filling one table per state costs $O(n |A|^2)$. This *fixed-length HMM kernel* compares two aligned strings under every hidden annotation of their shared positions at once, and fits tasks with substitutions but no insertions or deletions.

Sequences in biology rarely share a length. The *pair HMM kernel* restores insertions and deletions by augmenting the state alphabet with the empty symbol ε , so a state may emit a letter to one string, to both, or to neither, exactly the moves of a pairwise alignment. Indexing the subkernels by the positions reached in each string, $\kappa_{i,j,a}(s, t)$, and starting from $\kappa_{0,0,a_I} = 1$, the recursion splits into the three ways a state can consume input:

$$\kappa_{i,j,a}(s, t) = \sum_{b \in A} P_M(a | b) \left[P((s_i, \varepsilon) | a) \kappa_{i-1,j,b} + P((\varepsilon, t_j) | a) \kappa_{i,j-1,b} + P((s_i, t_j) | a) \kappa_{i-1,j-1,b} \right],$$

and $K(s, t) = \sum_{a \in A} \kappa_{|s|,|t|,a}(s, t)$. This is the Durbin et al. (1998) forward algorithm read as a kernel, the probabilistic sibling of the local alignment kernel of the next section: both sum a score over all alignments, but the pair HMM sums a genuine probability and so is a P-kernel, positive definite with no convolution argument needed.

22.5.5 Stochastic context-free grammars for RNA

RNA folds back on itself: distant bases pair, so the relevant dependencies are nested rather than sequential, and a Markov chain is the wrong model. The right generative model is a stochastic context-free grammar, whose productions can generate matched pairs. A minimal grammar for base-paired RNA has the rules

$$S \rightarrow SS, \quad S \rightarrow aSa, \quad S \rightarrow aS, \quad S \rightarrow a,$$

where the rule $S \rightarrow aSa$ emits two paired bases wrapping a nested structure. Marginalizing a kernel over the hidden parse of such a grammar is the RNA analogue of the HMM construction above. The features count occurrences of each (base, grammar-state) combination, and the marginalization is carried out by the classical inside-outside algorithm, the context-free counterpart of forward-backward (Kin et al., 2002).

Example (tRNA clustering)

A set of 74 human tRNA sequences, spanning three classes (Ala-AGC, Asn-GTT, and Cys-GCA), was analyzed with the second-order marginalized kernel built on an SCFG (Tsuda et al., 2002). Projecting the kernel onto its

first two principal components separates the three classes, evidence that the grammar-based kernel captures the folded structure that defines the families.

Verification artifact. checks/example-ch09-example-21-8.json records the example source hash and verification scope.

22.5.6 The Fisher kernel for sequences

The marginalized and HMM kernels average over a discrete set of hidden paths. The Fisher kernel of Jaakkola and Haussler (1999) exploits the model’s *smooth* parameters instead, and it is the last strategy named in the chapter’s opening that we still owe a treatment. Its general construction is developed in Chapter 19, Translation-Invariant, Semigroup, and Probabilistic Kernels: fit a parametric model P_θ to the data at θ_0 , take the *Fisher score* $g(\theta_0, x) = \nabla_\theta \log P_\theta(x)|_{\theta=\theta_0}$, one coordinate per parameter, and whiten it by the Fisher information $I = \mathbb{E}[g(\theta_0, x)g(\theta_0, x)^\top]$ to obtain $K(x, z) = g(\theta_0, x)^\top I^{-1}g(\theta_0, z)$. We recall only what carries over here: the feature map $\Phi(x) = g(\theta_0, x)$ has fixed dimension whatever the length of x , and in practice I is replaced by the identity or by the empirical covariance of the scores. Two sequences are alike, in this view, when they stretch the model the same way: a prototypical sequence needs little adjustment, an unusual one pulls hard on the parameters that explain its oddity. What is specific to sequences is which generative model supplies the score, and what the resulting coordinates count.

Take the generative model to be a Markov chain over k -mers, with parameters $a_{u \rightarrow \sigma}$ governing the transition from a length- k context u to the next symbol σ . Shawe-Taylor and Cristianini (2004) compute the Fisher score at the flat setting where every transition is equally likely, and find that the coordinate for $u \rightarrow \sigma$ is

$$g(\theta_0, s)_{u \rightarrow \sigma} = |\{(v_1, v_2) : s = v_1 u \sigma v_2\}| - |\Sigma|^{-1} |\{(v_1, v_2) : s = v_1 u v_2\}|.$$

The first term counts the occurrences of the $(k+1)$ -mer $u\sigma$ in s : up to the centering correction of the second term, the Fisher kernel of this elementary sequence model is the $(k+1)$ -spectrum kernel, so the two families that open and close this chapter meet at this identity. For a genuine hidden Markov model of a protein family the score is taken with respect to the emission and transition parameters, and its hard k -mer counts soften into expected state-occupancy and transition counts, each supplied in one pass by the forward-backward algorithm. This is the SVM-Fisher baseline that the remote homology benchmark below pits against the local alignment kernel.

22.6 The local alignment kernel

The third strategy starts not from a feature map or a generative model but from the biologist's own notion of similarity: alignment. To compare

```
x1 = CGGSLIAMMWFVG
x2 = CLIVMMNRLMWFGV
```

a biologist looks for a good alignment, matching conserved residues and paying for the insertions and deletions in between:

CGGSLIAMM-----WFGV
|...|||||...|||
C-----LIVMMNBLMWFGV

An alignment π is scored with a substitution matrix $S \in \mathbb{R}^{\mathcal{A} \times \mathcal{A}}$, rewarding aligned residue pairs, and a gap penalty function $g : \mathbb{N} \rightarrow \mathbb{R}$, charging for each run of gaps:

$$s_{S,g}(\pi) = \sum_{\text{aligned } (a,b)} S(a,b) - \sum_{\text{gaps of length } \ell} g(\ell).$$

Reading the alignment above off column by column makes the score concrete. Ten columns pair two residues, and two runs of gaps, one of length three and one of length four, separate them:

$$s_{S,g}(\pi) = S(C,C) + S(L,L) + S(I,I) + S(A,V) + 2S(M,M) + S(W,W) + S(F,F) + S(G,G) + S(V,V) - g(3) - g(4).$$

Most columns pair a residue with itself and collect a large positive substitution score; the one substitution $S(A,V)$ pairs two different but chemically similar residues and scores positive but smaller; the two gap runs are charged once each. A high total means the two proteins can be made to line up cheaply, which is what a biologist means by similar.

22.6.1 From Smith-Waterman to a kernel

The classical Smith-Waterman score keeps only the best alignment (Smith and Waterman, 1981),

$$\text{SW}_{S,g}(x,y) = \max_{\pi \in \Pi(x,y)} s_{S,g}(\pi).$$

It is symmetric, and it is the workhorse of sequence comparison, but a maximum over alignments is *not* positive definite, so it cannot be used as a kernel. The fix of Saigo et al. (2004) is to replace the maximum by a Boltzmann-weighted sum over *all* alignments, softening the hard optimum into a smooth aggregate.

Definition (local alignment kernel)

For an inverse temperature $\beta > 0$,

$$K_{\text{LA}}^{(\beta)}(x,y) = \sum_{\pi \in \Pi(x,y)} \exp(\beta s_{S,g}(x,y,\pi)).$$

As $\beta \rightarrow \infty$ the sum is dominated by the best alignment and recovers Smith-Waterman up to the exponential, but for finite β the kernel is symmetric and, as we now prove, positive definite.

The parameter β is an inverse temperature, and the sum is a softened maximum. At large β the single best alignment overwhelms the rest, so the kernel is essentially Smith-Waterman; at small β every alignment, good or bad, contributes comparably, and the kernel measures the total mass of ways the two strings can be aligned rather than the single best way. The reason this rescues positive definiteness is structural, not a coincidence of the exponential: a maximum cannot in general be written as an inner product, whereas a sum over configurations of a product of local, individually valid weights can, which is exactly the convolution structure we exploit in the proof below. Aggregating over the whole ensemble of alignments is also often the more robust comparison, since two proteins may be related through many mediocre alignments rather than one exceptional one.

22.6.2 The local alignment kernel is positive definite

The proof is a beautiful application of kernel closure properties: we build the LA kernel out of trivially valid pieces using operations that preserve positive definiteness. We assemble the tools in order.

Lemma (closure properties)

If K_1 and K_2 are positive definite kernels, then so are $K_1 + K_2$, the product $K_1 K_2$, and cK_1 for any $c \geq 0$. Moreover, if a sequence $(K_i)_{i \geq 1}$ of p.d. kernels converges pointwise to a function K ,

$$K(x, x') = \lim_{i \rightarrow \infty} K_i(x, x') \quad \text{for all } x, x',$$

then K is also positive definite.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Sums and non-negative multiples are immediate from the definition, since a non-negative combination of positive semidefinite Gram matrices is positive semidefinite; and a pointwise limit of positive semidefinite matrices is positive semidefinite because the quadratic form $\sum_{i,j} \alpha_i \alpha_j K_i(x_i, x_j) \geq 0$ passes to the limit. Only the product needs work, and it is the Schur (Hadamard) product theorem. Let A and B be $n \times n$ positive semidefinite matrices. Diagonalizing A ,

$$A_{i,j} = \sum_{p=1}^n f_p(i) f_p(j)$$

for some vectors $f_1, \dots, f_n$. Then for any $\alpha \in \mathbb{R}^n$,

$$\sum_{i,j=1}^n \alpha_i \alpha_j A_{i,j} B_{i,j} = \sum_{p=1}^n \sum_{i,j=1}^n (\alpha_i f_p(i)) (\alpha_j f_p(j)) B_{i,j} \geq 0,$$

each inner double sum being non-negative because B is positive semidefinite. Hence the entrywise product $C_{i,j} = A_{i,j} B_{i,j}$ is positive semidefinite. $\square$

The next two lemmas let us combine kernels living on different spaces and kernels of sets.

Lemma (direct sum and product)

Let $\mathcal{X} = \mathcal{X}_1 \times \mathcal{X}_2$, with K_1 a p.d. kernel on $\mathcal{X}_1$ and K_2 a p.d. kernel on $\mathcal{X}_2$. Then both the direct sum

$$K((x_1, x_2), (y_1, y_2)) = K_1(x_1, y_1) + K_2(x_2, y_2)$$

and the direct product

$$K((x_1, x_2), (y_1, y_2)) = K_1(x_1, y_1) K_2(x_2, y_2)$$

are p.d. kernels on $\mathcal{X}$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Let $\Phi_1 : \mathcal{X}_1 \rightarrow \mathcal{H}$ satisfy $K_1(x_1, y_1) = \langle \Phi_1(x_1), \Phi_1(y_1) \rangle_{\mathcal{H}}$. Define $\Phi : \mathcal{X}_1 \times \mathcal{X}_2 \rightarrow \mathcal{H}$ by $\Phi((x_1, x_2)) = \Phi_1(x_1)$, ignoring the second coordinate. Then

$$\langle \Phi((x_1, x_2)), \Phi((y_1, y_2)) \rangle_{\mathcal{H}} = K_1(x_1, y_1),$$

so $(x, y) \mapsto K_1(x_1, y_1)$ is p.d. on the product space, and symmetrically for K_2 . The sum and product statements now follow from the closure lemma. $\square$

Lemma (kernel for sets)

Let K be a p.d. kernel on $\mathcal{X}$ and let $\mathcal{P}(\mathcal{X})$ be the set of finite subsets of $\mathcal{X}$. Then

$$K_{\mathcal{P}}(A, B) = \sum_{x \in A} \sum_{y \in B} K(x, y)$$

is a p.d. kernel on $\mathcal{P}(\mathcal{X}) \times \mathcal{P}(\mathcal{X})$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

With $K(x, y) = \langle \Phi(x), \Phi(y) \rangle_{\mathcal{H}}$, linearity of the inner product gives

$$K_{\mathcal{P}}(A, B) = \sum_{x \in A} \sum_{y \in B} \langle \Phi(x), \Phi(y) \rangle_{\mathcal{H}} = \left\langle \sum_{x \in A} \Phi(x), \sum_{y \in B} \Phi(y) \right\rangle_{\mathcal{H}} = \langle \Phi_{\mathcal{P}}(A), \Phi_{\mathcal{P}}(B) \rangle_{\mathcal{H}},$$

where $\Phi_{\mathcal{P}}(A) = \sum_{x \in A} \Phi(x)$. Being an inner product, it is p.d. $\square$

The decisive tool is Haussler's convolution, which builds a kernel on strings by splitting each string into two pieces in every possible way and combining sub-kernels on the pieces.

Definition (convolution kernel, Haussler 1999)

For p.d. string kernels K_1 and K_2 , their convolution $K_1 \star K_2$ is

$$K_1 \star K_2(x, y) = \sum_{x_1 x_2 = x, y_1 y_2 = y} K_1(x_1, y_1) K_2(x_2, y_2),$$

the sum running over all ways of writing x as a concatenation $x_1 x_2$ and y as $y_1 y_2$.

Lemma

If K_1 and K_2 are p.d., then $K_1 \star K_2$ is p.d.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

For a string x let $R(x) = \{(x_1, x_2) : x = x_1 x_2\} \subset \mathcal{X} \times \mathcal{X}$ be the finite set of its splits. Then

$$K_1 \star K_2(x, y) = \sum_{(x_1, x_2) \in R(x)} \sum_{(y_1, y_2) \in R(y)} K_1(x_1, y_1) K_2(x_2, y_2).$$

The integrand $(x_1, x_2), (y_1, y_2) \mapsto K_1(x_1, y_1) K_2(x_2, y_2)$ is a direct product kernel on $\mathcal{X} \times \mathcal{X}$, hence p.d.; summing it over the finite sets $R(x)$ and $R(y)$ is exactly the kernel-for-sets construction, hence p.d. $\square$

We are ready to name the three atomic string kernels from which the LA kernel is assembled.

Three basic string kernels

- the constant kernel $K_0(x, y) := 1$;
- a kernel for single letters,

$$K_a^{(\beta)}(x, y) := \begin{cases} 0 & \text{if } |x| \neq 1 \text{ or } |y| \neq 1, \\ \exp(\beta S(x, y)) & \text{otherwise;} \end{cases}$$

- a kernel for gaps,

$$K_g^{(\beta)}(x, y) = \exp[\beta (g(|x|) + g(|y|))].$$

Remark (validity of the atoms)

Here $S : \mathcal{A}^2 \rightarrow \mathbb{R}$ is the substitution score. The letter kernel $K_a^{(\beta)}$ is p.d. exactly when the matrix $(\exp(\beta S(a, b)))_{(a,b) \in \mathcal{A}^2}$ is positive semidefinite, which holds for every $\beta > 0$ when S is conditionally positive definite. The gap kernel is p.d. with no restriction on g , because it factorizes,

$$K_g^{(\beta)}(x, y) = \exp(\beta g(|x|)) \times \exp(\beta g(|y|)),$$

which is a product of a function of x with the same function of y , the canonical form of a p.d. kernel of rank one.

Finally, an alignment is a run of matched-letter blocks separated by gaps, so summing the alignment score over all alignments is precisely a sum of convolutions of the atoms.

Theorem (LA kernel is p.d.)

The local alignment kernel is a pointwise limit of convolution kernels,

$$K_{\text{LA}}^{(\beta)} = \sum_{n=0}^{\infty} K_0 \star \left(K_a^{(\beta)} \star K_g^{(\beta)} \right)^{\star(n-1)} \star K_a^{(\beta)} \star K_0,$$

and is therefore positive definite.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Proof

The expansion assembles the atoms in exactly the pattern of an alignment. Read one term of the sum from left to right: the outer K_0 factors fix the empty boundaries, and the repeated block $K_a^{(\beta)} \star K_g^{(\beta)}$ alternates one matched letter with one gap, so the n -th term lays down n aligned residues interleaved with gaps and reproduces the weight $\exp(\beta s_{S,g}(\pi))$ of every alignment with n aligned residue blocks, summed over those alignments. Because each atom is p.d. and convolution preserves positive definiteness, every term is a p.d. kernel by the convolution lemma; the partial sums are p.d. by the sum-closure lemma; and the whole series, being their pointwise limit, is p.d. by the limit-closure lemma. The induction on n that matches the n -th term to the n -block alignments is straightforward but long to write out in full; see Vert et al. (2004) for the details. $\square$

22.6.3 Computing the LA kernel by dynamic programming

The infinite sum over alignments is, like Smith-Waterman itself, computed by a dynamic program. Assume the standard *affine* gap penalty, which charges an opening cost d and a smaller extension cost e per additional gapped position:

$$g(0) = 0, \quad g(n) = d + e(n-1) \text{ for } n \geq 1.$$

The kernel decomposes into five interlocking tables. For $0 \leq i \leq |x|$ and $0 \leq j \leq |y|$, let $M(i, j)$ accumulate alignments of the prefixes $x_{[1,i]}$ and $y_{[1,j]}$ that end in a matched pair, let $X(i, j)$ and $Y(i, j)$ accumulate those ending in a gap in x or in y respectively, and let X_2, Y_2 carry the boundary contributions so that the final answer collects every alignment. Then

$$K_{LA}^{(\beta)}(x, y) = 1 + X_2(|x|, |y|) + Y_2(|x|, |y|) + M(|x|, |y|).$$

The tables are initialized to zero along the top and left borders,

$$M(i, 0) = M(0, j) = X(i, 0) = X(0, j) = Y(i, 0) = Y(0, j) = X_2(i, 0) = X_2(0, j) = Y_2(i, 0) = Y_2(0, j) = 0,$$

and filled for $i = 1, \dots, |x|$ and $j = 1, \dots, |y|$ by the recursion

$$M(i, j) = \exp(\beta S(x_i, y_j)) [1 + X(i-1, j-1) + Y(i-1, j-1) + M(i-1, j-1)],$$

$$X(i, j) = \exp(\beta d) M(i-1, j) + \exp(\beta e) X(i-1, j),$$

$$Y(i, j) = \exp(\beta d) [M(i, j-1) + X(i, j-1)] + \exp(\beta e) Y(i, j-1),$$

$$X_2(i, j) = M(i-1, j) + X_2(i-1, j),$$

$$Y_2(i, j) = M(i, j-1) + X_2(i, j-1) + Y_2(i, j-1).$$

Each table sums the weights of a family of partial alignments, classified by how they end. In $M(i, j)$, aligning x_i to y_j multiplies in the match weight $\exp(\beta S(x_i, y_j))$ and continues any alignment of the shorter prefixes that ended in a match, an x-gap, or a y-gap (or starts a fresh one, the "1"); in $X(i, j)$, the factor $\exp(\beta d)$ opens a gap out of a preceding match while $\exp(\beta e)$ extends a gap already open, and symmetrically for $Y(i, j)$. The two remaining tables X_2 and Y_2 handle the boundary: a local alignment may end with unaligned residues trailing off either string, and X_2, Y_2 accumulate exactly those terminal overhangs so that the final combination $1 + X_2(|x|, |y|) + Y_2(|x|, |y|) + M(|x|, |y|)$ counts every alignment, whether it closes on a matched pair or runs off the end. The ordering of the recurrences matters: M feeds X and Y , which in turn feed the boundary tables, so one left-to-right, top-to-bottom sweep fills all five. The whole table is filled in $O(|x| |y|)$ time, the same order as ordinary Smith-Waterman. In pseudocode:

```
Input: x, y, substitution S, gap-open d, gap-extend e, beta
initialize M, X, Y, X2, Y2 to 0 on row 0 and column 0
for i = 1.. |x|:
  for j = 1.. |y|:
    w = exp(beta * S(x[i], y[j]))
    M[i][j] = w * (1 + X[i-1][j-1] + Y[i-1][j-1] + M[i-1][j-1])
    X[i][j] = exp(beta*d)*M[i-1][j] + exp(beta*e)*X[i-1][j]
    Y[i][j] = exp(beta*d)*(M[i][j-1] + X[i][j-1]) + exp(beta*e)*Y[i][j-1]
    X2[i][j] = M[i-1][j] + X2[i-1][j]
    Y2[i][j] = M[i][j-1] + X2[i][j-1] + Y2[i][j-1]
return 1 + X2[|x|][|y|] + Y2[|x|][|y|] + M[|x|][|y|]
```

Equivalently, the recursion is exactly the computation performed by a finite-state transducer with match, gap, and boundary states, running in $O(|x| \times |x'|)$. One practical caveat: because scores are exponentiated, the table entries grow exponentially and overflow. Taking the logarithm of the LA kernel keeps the numbers in range and is common in practice, but the logarithm is no longer positive definite, so one trades validity for numerical safety.

22.7 Application: remote homology detection

The payoff for all this machinery is detecting *remote homology*, common evolutionary ancestry between proteins whose sequences have diverged so far that direct comparison fails. Homologous proteins share structure and function long after their sequences have drifted apart, so on a similarity axis they occupy a "twilight zone": close homologs are easy, non-homologs are clearly unrelated, and the remote homologs in between cannot be found by

naive sequence matching. The SCOP database organizes proteins hierarchically into families (close homologs), superfamilies (remote homologs), and folds, giving a ground truth.

The benchmark of Saigo et al. (2004) asks a classifier to recognize the correct superfamily: for a target, positive training examples are drawn from the same superfamily but different families, negatives from other superfamilies, and the test measures whether new sequences are assigned the right superfamily, a direct test of remote-homology detection. Support vector machines were compared across kernels using the ROC_{50} score. The local alignment kernel (SVM-LA) dominated, outperforming the pairwise kernel (SVM-pairwise), the mismatch kernel (SVM-mismatch), and the Fisher kernel (SVM-Fisher). Aligning the whole soft ensemble of local alignments, it turns out, sees homology that k -mer counts and generative-model gradients miss.

22.8 Summary

A single alphabet, three routes to a kernel. From explicit feature maps we obtained the spectrum kernel, computable in $O(|x| + |x'|)$ with suffix trees or $O(k(|x| + |x'|))$ with tries, its (k, m) -mismatch variant, computable in $O(k^{m+1}|\mathcal{A}|^m(|x| + |x'|))$ by a mismatch-tree traversal, and the gapped subsequence kernel, computable in $O(k|x||x'|)$ by a two-table dynamic program. From generative models we obtained context-tree kernels and, through the marginalized-kernel construction with forward-backward and inside-outside, kernels that inherit the biology of hidden Markov models and RNA grammars. From the biologist's alignment score we obtained the local alignment kernel, whose positive definiteness we proved by writing it as a limit of Haussler convolutions of three atomic kernels, and which wins on remote homology detection.

Several lessons generalize well beyond biology. Good kernel design is specific to the data and the task, and raw performance is not the only criterion: speed, interpretability, and validity all matter. String kernels admit fast implementations through classical string algorithms, and while the design is still something of an art, principled routes (explicit features, generative models, similarity scores) have emerged. None of the constructions in this chapter is confined to proteins; the same kernels classify natural language, time series, and any other data that arrives as a string.

Further reading

The textbook treatment closest to this chapter is Shawe-Taylor and Cristianini (2004), *Kernel Methods for Pattern Analysis*, whose chapter 11 develops kernels for text and strings, including the spectrum and gap-weighted subsequence kernels and their dynamic programs, in complementary detail. Scholkopf and Smola (2002), *Learning with Kernels*, is the standard reference for the general theory of kernels and support vector machines used throughout, though it predates most of the string-kernel work surveyed here and so says little about sequences specifically. For the individual constructions, the original papers remain the best sources: Leslie, Eskin, and Noble (2002) for the spectrum kernel, Lodhi et al. (2002) for the subsequence kernel, Haussler (1999) for convolution kernels, Jaakkola, Diekhans, and Haussler (2000) for the Fisher kernel on proteins, Cuturi et al. (2005) for context-tree and mutual-information kernels, and Saigo et al. (2004) for the local alignment kernel.

22.9 Common mistakes and practical implications

For **Kernels for Sequences**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

22.10 Exercises

1. warm-up Take the DNA string $x = \text{AGGAG}$ and $k = 2$. List its contiguous 2-mers, write down the nonzero coordinates of the spectrum feature vector $\Phi(x)$, and compute the self-kernel $K(x, x)$. Then explain in one sentence why $K(x, x)$ always equals $\sum_u \Phi_u(x)^2$, the sum of squared k -mer counts, and why raising k toward $|x|$ drives this value down.
2. computation Let $x = \text{CACAC}$ and $x' = \text{ACACA}$, both over $\{A, C\}$, with $k = 2$. By hand, list the contiguous 2-mers of each string, form the two count vectors, and compute $K(x, x')$, $K(x, x)$, and $K(x', x')$ for the spectrum kernel. Evaluate the normalized kernel

$$\hat{K}(x, x') = \frac{K(x, x')}{\sqrt{K(x, x) K(x', x')}},$$

and comment on the value you get. What does it say about the ability of the spectrum kernel to tell x and x' apart, and which structural feature of the two strings is responsible?

3. computation Compute the gap-weighted subsequence kernel $K_{2,\lambda}(x, x')$ for $x = \text{AAB}$ and $x' = \text{AB}$.
 - a. Directly: enumerate every ordered index pair $\mathbf{i} = (i_1, i_2)$ with $i_1 < i_2$ in each string, record the selected 2-mer and its span $l(\mathbf{i}) = i_2 - i_1 + 1$, collect the features Φ_u , and read off the inner product. You should find $K_{2,\lambda}(x, x') = \lambda^5 + \lambda^4$.
 - b. By the dynamic program: fill the order-1 auxiliary table $B_1(i, j)$ using

$$B_k(xa, x'b) = \lambda B_k(x, x'b) + \lambda B_k(xa, x') - \lambda^2 B_k(x, x') + \delta_{a=b} \lambda^2 B_{k-1}(x, x'),$$

with $B_0 \equiv 1$, then run the companion recursion for K_2 and confirm it returns the same value as part (a).

Hint

In (a), the only 2-mer shared by both strings is AB; in x it occurs both gap-free (span 2) and gapped (span 3), so $\Phi_{AB}(x) = \lambda^2 + \lambda^3$, while $\Phi_{AB}(x') = \lambda^2$. In (b) you need B_1 at the entries $(i-1, m-1)$ fed into the K_2 sum; the companion recursion only fires when the appended letter of x equals a letter of x' .

4. proof Prove directly from the definition that the k -spectrum kernel is positive definite: for any strings $x_1, \dots, x_n$ and any reals $c_1, \dots, c_n$, show that $\sum_{i,j} c_i c_j K(x_i, x_j) \geq 0$. Identify the vector whose squared norm this quadratic form equals. Explain why the mismatch and gap-weighted subsequence kernels inherit positive definiteness by the very same one-line argument, and why that argument breaks for the Smith-Waterman score $\text{SW}_{S,g}(x, y)$.
5. proof This exercise works through the P-kernel framework of Haussler (1999).
 - a. On $\mathcal{X} = \{x_1, x_2\}$, take the symmetric distribution with $P(x_1, x_2) = P(x_2, x_1) = \frac{1}{2}$ and $P(x_1, x_1) = P(x_2, x_2) = 0$. Write out its 2×2 Gram matrix, find its eigenvalues, and conclude that not every symmetric joint distribution is a valid kernel.
 - b. Now prove the rescue. Suppose a latent model index m has prior $P_M(m)$ and that x and z are conditionally independent given m , so that $P(x, z \mid m) = P(x \mid m) P(z \mid m)$. Show that the marginal $P(x, z) = \sum_m P(x \mid m) P(z \mid m) P_M(m)$ is positive definite.

Hint

In (a) the matrix is $\begin{pmatrix} 0 & 1/2 \\ 1/2 & 0 \end{pmatrix}$; its eigenvalues are $\pm \frac{1}{2}$, and the negative one exhibits a direction of negative quadratic form. In (b), for each fixed m the term $P(x \mid m) P(z \mid m)$ is a rank-one product $f(x)f(z)$ with $f(\cdot) = P(\cdot \mid m)$, hence p.d.; a non-negative combination of p.d. kernels is p.d.

6. proof Two steps of the local-alignment-kernel positive-definiteness argument.

- a. Show that the gap kernel $K_g^{(\beta)}(x, y) = \exp[\beta(g(|x|) + g(|y|))]$ is positive definite for *every* gap function g and every $\beta > 0$, with no condition on g . Contrast this with the letter kernel $K_a^{(\beta)}$, which requires the substitution matrix S to make $(\exp(\beta S(a, b)))_{a,b}$ positive semidefinite: what is the structural difference that frees the gap kernel of any condition?
- b. Assuming the direct-product lemma and the kernel-for-sets lemma proved in the chapter, prove the convolution lemma: if K_1 and K_2 are p.d., then so is $K_1 \star K_2$.

Hint

In (a), $K_g^{(\beta)}$ factorizes as $h(x)h(y)$ with $h(x) = \exp(\beta g(|x|))$, the canonical rank-one form, so it is p.d. regardless of g ; the letter kernel is a genuine two-argument matrix indexed by (a, b) , so its positivity is a real constraint on S . In (b), write $R(x) = \{(x_1, x_2) : x = x_1 x_2\}$ for the finite set of splits of x ; the summand $K_1(x_1, y_1)K_2(x_2, y_2)$ is a direct-product kernel on $\mathcal{X} \times \mathcal{X}$, and summing it over $R(x)$ and $R(y)$ is exactly the kernel-for-sets construction.

7. computation A corpus has $\ell = 4$ documents. The term `the` appears in all four, `kernel` in two, and `gene` in one. Using the idf weight $w(t) = \log(\ell/\text{df}(t))$, compute $w(\text{the})$, $w(\text{kernel})$, and $w(\text{gene})$. Two documents have term-frequency vectors

$$d : \text{tf}(\text{the}) = 3, \text{tf}(\text{gene}) = 1, \text{tf}(\text{kernel}) = 1, \quad d' : \text{tf}(\text{the}) = 2, \text{tf}(\text{gene}) = 0, \text{tf}(\text{kernel}) = 2.$$

Evaluate the tf-idf kernel $\tilde{K}(d, d') = \sum_t w(t)^2 \text{tf}(t, d) \text{tf}(t, d')$, and explain why the word `the`, despite being shared and by far the most frequent, makes no contribution at all.

8. challenge Take the generative model to be a Markov chain over k -mers, with a parameter $a_{u \rightarrow \sigma}$ for the transition from a length- k context u to the next symbol σ , and fit it at the flat setting θ_0 where every $a_{u \rightarrow \sigma} = |\Sigma|^{-1}$. Working from the log-likelihood of a sequence s and respecting the multinomial constraint $\sum_{\sigma} a_{u \rightarrow \sigma} = 1$, derive the Fisher score coordinate

$$g(\theta_0, s)_{u \rightarrow \sigma} = |\{(v_1, v_2) : s = v_1 u \sigma v_2\}| - |\Sigma|^{-1} |\{(v_1, v_2) : s = v_1 u v_2\}|,$$

and conclude that, up to the centering term, the Fisher kernel of this model is the $(k+1)$ -spectrum kernel. Comment on what the centering term does and why the two families that open and close the chapter meet here. Hint

Write $\log P_{\theta}(s)$ as a sum over positions of $\log a_{u_i \rightarrow \sigma_i}$; the derivative with respect to $a_{u \rightarrow \sigma}$ counts how often that exact transition, that is the $(k+1)$ -mer $u\sigma$, is used. Enforcing $\sum_{\sigma} a_{u \rightarrow \sigma} = 1$ by a Lagrange multiplier (or by differentiating along the simplex) produces the second term, which subtracts the expected usage of context u spread uniformly over the $|\Sigma|$ possible next symbols. The first term is exactly the $(k+1)$ -spectrum feature.

23 Efficient String and Tree Kernels

The companion chapter on kernels for sequences defined what a string kernel measures: the spectrum kernel counts shared contiguous substrings, the mismatch kernel tolerates errors, and the gap-weighted kernel rewards shared subsequences in proportion to how tightly packed they are. Every one of these feature maps lives in a space with a coordinate for each of exponentially many patterns, so writing the feature vectors down is out of the question. This chapter is about the computation: how a recursion over prefixes turns each of these kernels into a small dynamic-programming table, how a data structure called a trie makes the spectrum and mismatch kernels almost linear in the input length, and how the same recursive idea, applied bottom up from the leaves, gives kernels between trees. The recurrences and their costs are the whole point, so we give each one with its derivation, a worked table, and its complexity, and we close by placing all of them inside Haussler's convolution-kernel framework, the common ancestor of string, tree, and graph kernels alike.

23.1 From feature counts to dynamic programming

Fix a finite alphabet Σ . For a string s we write $|s|$ for its length, $s(i:j)$ for the substring $s_i \cdots s_j$, and $s(\mathbf{i})$ for the subsequence picked out by an index tuple $\mathbf{i} = (i_1 < \cdots < i_k)$; the span of that subsequence is $l(\mathbf{i}) = i_k - i_1 + 1$, the number of characters of s it reaches across. The feature maps of the previous chapter are all of the form "count, or weight, the occurrences of a pattern u ", and the kernel is the inner product

$$\kappa(s, t) = \sum_u \varphi_u(s) \varphi_u(t),$$

a sum over a pattern set that is $|\Sigma|^p$ for length- p patterns and infinite for the all-subsequences kernel. The reason any of this is tractable is that the sum telescopes: a common pattern in s and t either avoids the last character of s or ends on it, and this two-case split, applied to every prefix, replaces the exponential sum by a table with one cell per pair of prefixes. Dynamic programming is the systematic filling of that table, and the rest of the chapter is a catalogue of the recurrences it uses. Following Watkins (2000) and Lodhi et al. (2002), we start with the most permissive feature set, all subsequences at once, because its recurrence is the cleanest instance of the telescoping idea.

23.2 The all-subsequences kernel

The richest subsequence feature map indexes coordinates by *every* finite string and counts occurrences as a gapped subsequence.

Definition (all-subsequences kernel)

The feature space is indexed by $u \in \Sigma^*$, the set of all finite strings including the empty string ε , with embedding

$$\varphi_u(s) = |\{\mathbf{i} : u = s(\mathbf{i})\}|,$$

the number of ways u occurs as a subsequence of s . The kernel is $\kappa(s, t) = \sum_{u \in \Sigma^*} \varphi_u(s) \varphi_u(t)$, the number of common subsequences of s and t , counted with multiplicity and including ε .

To evaluate it, consider extending s by one symbol a to form sa . A common subsequence of sa and t either does not use the appended a , in which case it is already a common subsequence of s and t , or it ends in a matched to some occurrence of a in t , say at position k , and the part before it is a common subsequence of s and the prefix $t(1:k-1)$. Summing over all occurrences of a in t gives the recurrence

$$\kappa(\varepsilon, t) = 1, \quad \kappa(sa, t) = \kappa(s, t) + \sum_{k: t_k=a} \kappa(s, t(1:k-1)).$$

Writing $D[i][j] = \kappa(s(1:i), t(1:j))$ turns this into a table filled row by row. The inner sum over matching positions can be precomputed into a running array so that each row costs $O(|t|)$, giving the algorithm of Shawe-Taylor and Cristianini (2004).

Algorithm (all-subsequences kernel)

Input strings s of length n , t of length m .

Output $\kappa(s, t)$ = number of common subsequences = $D[n][m]$.

1. Initialise $D[0][j] = 1$ for $j = 0, \dots, m$ and $D[i][0] = 1$ for $i = 0, \dots, n$ (only ε matches an empty prefix).
2. For each $i = 1, \dots, n$: scan t once to build $P[k] = \sum_{k' \leq k, t_{k'}=s_i} D[i-1][k'-1]$, the cumulative contribution of matches of s_i .
3. For each $k = 1, \dots, m$: set $D[i][k] = D[i-1][k] + P[k]$.
4. Return $D[n][m]$.

Because the array P collapses the inner sum, each of the n rows costs $O(m)$, so the whole kernel is computed in $O(|s| |t|)$ time and $O(|t|)$ working memory, an exponential saving over enumerating subsequences. The following table makes the recurrence concrete.

Example (all-subsequences kernel on two short strings)

Take $s = \text{"gatta"}$ and $t = \text{"cata"}$. Cells where $s_i = t_j$ are shaded; these are the positions that trigger the inner sum. Base row and column are all 1.

D	ε	c	a	t	a
ε	1	1	1	1	1
g	1	1	1	1	1
a	1	1	2	2	3
t	1	1	2	4	5
t	1	1	2	6	7
a	1	1	3	7	14

1. Fill the base. The empty subsequence is common to every pair of prefixes, so the ε row and column are 1. The row for "g" stays 1, since "g" never appears in t and contributes no inner-sum term.
2. Trigger a match. At row "a", column "a" ($j = 2$), the recurrence gives $D = D_{\text{above}} + D[i-1][1] = 1 + 1 = 2$; at the second "a" ($j = 4$) both earlier occurrences contribute, $1 + 1 + 1 = 3$.
3. Read the intermediate value. After four rows, $D[4][4] = 7$, so $\kappa(\text{"gatt"}, \text{"cata"}) = 7$: the seven common subsequences are ε , "a", "t", "t", "at", "at", "att" counted with multiplicity.
4. Append the last character. Extending s to "gatta" adds row "a", and the appended "a" matched against the two "a"s of t lifts the corner to $D[5][4] = 7 + 1 + 6 = 14$.

Reading. The kernel value $\kappa(\text{"gatta"}, \text{"cata"}) = 14$ is read straight off the bottom-right cell, and the whole table cost 5×4 constant-time updates, matching the $O(|s| |t|)$ bound.

Verification artifact. checks/example-ch-strings2-example-22-1.json records the example source hash and verification scope.

23.3 The fixed-length subsequences kernel

The all-subsequences kernel mixes patterns of every length, and long ones can swamp short ones. Restricting to subsequences of a fixed length p gives a more controllable feature map, at the cost of an extra recursion. The feature map is $\varphi_u(s) = |\{\mathbf{i} : u = s(\mathbf{i})\}|$ for $u \in \Sigma^p$, and repeating the last-character split now has to track the remaining length as well, since matching the final symbol consumes one of the p positions. This produces a recursion over both prefixes and lengths (Shawe-Taylor and Cristianini 2004).

Algorithm (fixed-length subsequences kernel)

Input strings s, t , target length p .

Output $\kappa_p(s, t)$, the inner product of the length- p subsequence counts.

1. Base cases: $\kappa_0(s, t) = 1$ for all s, t , and $\kappa_q(s, \varepsilon) = 0$ for $q > 0$.
2. For each level $q = 1, \dots, p$ and each pair of prefixes, apply

$$\kappa_q(sa, t) = \kappa_q(s, t) + \sum_{k: t_k=a} \kappa_{q-1}(s, t(1:k-1)),$$

storing the previous level's table κ_{q-1} as it goes.

3. Return $\kappa_p(s, t)$.

The double loop over lengths and prefixes evaluates p tables of size $|s| \times |t|$, so the kernel costs $O(p|s||t|)$. A weighted blend $\sum_{l=1}^p a_l \kappa_l(s, t)$ with weights $a_l \geq 0$ is obtained at no extra asymptotic cost by accumulating the intermediate levels, since every κ_l is computed on the way to κ_p . This fixed-length recursion is the scaffold on which the gap-weighted kernel is built next, the only change being that positions now carry a decay weight.

23.4 The gap-weighted subsequences kernel

The kernel most often meant by "the string kernel", after Lodhi et al. (2002), weights each occurrence of a length- p subsequence by $\lambda^{l(\mathbf{i})}$, where $0 < \lambda \leq 1$ is a decay factor and $l(\mathbf{i})$ is the span. Tightly packed occurrences (few gaps) are rewarded, spread-out ones are penalised, and as $\lambda \rightarrow 0$ the kernel degenerates to the p -spectrum kernel of contiguous substrings. The feature map is

$$\varphi_u(s) = \sum_{\mathbf{i}: u=s(\mathbf{i})} \lambda^{l(\mathbf{i})}, \quad u \in \Sigma^p.$$

A direct evaluation via the fixed-length recursion would repeatedly recompute gap weights and cost $O(p|s|^2|t|^2)$. The efficient route, following the previous chapter's suffix idea, introduces an auxiliary *suffix kernel* that only counts subsequences whose final matched symbol is the last character of each string.

Definition (gap-weighted suffix kernel)

For length p , the suffix kernel $\kappa_p^S(s, t)$ sums $\lambda^{l(\mathbf{i})+l(\mathbf{j})}$ over pairs of length- p index tuples with $s(\mathbf{i}) = t(\mathbf{j})$ whose last indices are $|s|$ and $|t|$ respectively. Then the full kernel recovers every occurrence by letting the endpoints range over all prefixes,

$$\kappa_p(s, t) = \sum_{i=1}^{|s|} \sum_{j=1}^{|t|} \kappa_p^S(s(1:i), t(1:j)).$$

The suffix kernel is nonzero only when the two strings end in the same symbol. Splitting the pairs of subsequences by where their penultimate symbols fall leads, for $a = b$, to

$$\kappa_p^S(sa, tb) = \lambda^2 \sum_{i=1}^{|s|} \sum_{j=1}^{|t|} \lambda^{|s|-i+|t|-j} \kappa_{p-1}^S(s(1:i), t(1:j)),$$

and 0 when $a \neq b$. The nested geometric sum is exactly what makes a naive pass quadratic in each length. The trick of Lodhi et al. (2002) is to precompute it as an intermediate table

$$\text{DP}_p(k, l) = \sum_{i=1}^k \sum_{j=1}^l \lambda^{k-i+l-j} \kappa_{p-1}^S(s(1:i), t(1:j)),$$

which satisfies a two-dimensional inclusion-exclusion recurrence that adds one row and column at a time. Working it out, the overlap of the shifted sums is corrected by a single subtracted term.

Algorithm (gap-weighted subsequences kernel, dynamic program)

Input strings s, t , length p , decay $\lambda \in (0, 1]$.

Output $\kappa_p(s, t)$, the gap-weighted subsequence kernel.

1. Set $\kappa_1^S(s(1:i), t(1:j)) = [s_i = t_j] \lambda^2$ for all i, j , the level-1 suffix table.
2. For each level $q = 2, \dots, p$, fill the auxiliary table by

$$\text{DP}_q(k, l) = \kappa_{q-1}^S(k, l) + \lambda \text{DP}_q(k-1, l) + \lambda \text{DP}_q(k, l-1) - \lambda^2 \text{DP}_q(k-1, l-1),$$

with $\text{DP}_q(0, l) = \text{DP}_q(k, 0) = 0$.

3. Recover the next suffix table: $\kappa_q^S(k, l) = [s_k = t_l] \lambda^2 \text{DP}_q(k-1, l-1)$.
4. Return $\kappa_p(s, t) = \sum_{k,l} \kappa_p^S(k, l)$.

Each level fills one $|s| \times |t|$ table with constant work per cell, so the kernel costs $O(p |s| |t|)$ time, the headline result of the section. The subtracted term $-\lambda^2 \text{DP}_q(k-1, l-1)$ is pure inclusion-exclusion: the two shifted tables $\lambda \text{DP}_q(k-1, l)$ and $\lambda \text{DP}_q(k, l-1)$ both include the block up to $(k-1, l-1)$, so it is counted twice and must be removed once. We fill the tables by hand for a two-letter difference.

Example (gap-weighted DP table for "cat" and "car", $p = 2$)

Length $p = 2$, decay λ . The level-1 suffix table has λ^2 at the two matches (c, c) and (a, a) and 0 elsewhere. The auxiliary table $\text{DP}_2(k, l)$ below is built from it; shaded cells are those where $s_k = t_l$.

DP_2	c	a	r
c	λ^2	λ^3	λ^4
a	λ^3	$\lambda^2 + \lambda^4$	$\lambda^3 + \lambda^5$
t	λ^4	$\lambda^3 + \lambda^5$	$\lambda^4 + \lambda^6$

1. Seed the corner. $\text{DP}_2(1, 1) = \kappa_1^S(1, 1) = \lambda^2$, since "c" matches "c"; the first row and column then propagate it by the λ -shift, giving λ^3, λ^4 and λ^3, λ^4 .
2. Apply inclusion-exclusion. At (a, a), $\text{DP}_2 = \lambda^2 + \lambda \cdot \lambda^3 + \lambda \cdot \lambda^3 - \lambda^2 \cdot \lambda^2 = \lambda^2 + \lambda^4$; the doubled block λ^4 is removed exactly once.
3. Extract the suffix kernel. Only (a, a) is a match with a nonzero predecessor: $\kappa_2^S(2, 2) = \lambda^2 \text{DP}_2(1, 1) = \lambda^4$. Every other matched cell has $\text{DP}_2(k-1, l-1) = 0$.
4. Sum the suffix table. $\kappa_2(\text{"cat"}, \text{"car"}) = \sum_{k,l} \kappa_2^S(k, l) = \lambda^4$: the lone shared length-2 subsequence is "ca", spanning two characters in each word.

Gap-weighted subsequence DP: $s = \text{cat}$, $t = \text{car}$ ($\lambda = 0.5$, $p = 2$)

	DP ₁					DP ₂			
	ϵ	c	a	r		ϵ	c	a	r
ϵ	0	0	0	0	ϵ	0	0	0	0
c	0	0.25	0.125	0.0625	c	0	0	0	0
a	0	0.125	0.3125	0.1562	a	0	0	0.0625	0.03125
t	0	0.0625	0.1562	0.07812	t	0	0	0.03125	0.01562

$$K_2(\text{cat}, \text{car}) = \lambda^4 = 0.0625 \quad \text{normalized} = K_2 / \sqrt{K_2(s, s) K_2(t, t)} = 0.4444$$

Figure 23.1: The dynamic-programming table for the gap-weighted subsequence kernel on the words *cat* and *car* ($\lambda = 0.5$, $p = 2$): each cell combines its neighbors with the stated λ weights, giving $K_2 = \lambda^4 = 0.0625$ and normalized similarity 0.4444.

Reading. The self-kernels are $\kappa_2(\text{"cat"}, \text{"cat"}) = \kappa_2(\text{"car"}, \text{"car"}) = 2\lambda^4 + \lambda^6$, so the normalised kernel is $\hat{\kappa} = \lambda^4 / (2\lambda^4 + \lambda^6) = (2 + \lambda^2)^{-1}$. At $\lambda = \frac{1}{2}$ this is $0.0625 / 0.140625 = 0.4444$, independent of any long-range structure the two three-letter words do not have.

Verification artifact. checks/example-ch-strings2-example-22-2.json records the example source hash and verification scope.

23.4.1 Variants: character weightings, gap counts, and soft matching

The DP recurrence is a template, and small edits to its lines produce a family of related kernels without changing the $O(p|s||t|)$ cost (Shawe-Taylor and Cristianini 2004). A *character-weighting* kernel lets each skipped or matched symbol u carry its own decay λ_u , replacing the uniform λ by symbol-dependent factors in the recurrence

$$\text{DP}_p(k, l) = \kappa_{p-1}^S(k, l) + \lambda_{t_l} \text{DP}_p(k, l-1) + \lambda_{s_k} \text{DP}_p(k-1, l) - \lambda_{s_k} \lambda_{t_l} \text{DP}_p(k-1, l-1),$$

useful when some symbols, such as low-information residues, should count for less. A *gap-number* weighting charges only for the gaps rather than the whole span, obtained by setting $\lambda = 1$ on the matched positions and adjusting the correction term. Most importantly, a *soft-matching* or substitution kernel replaces the hard test $[s_k = t_l]$ by an entry $A_{s_k t_l}$ of a symbol similarity matrix A , so that near-synonymous symbols (mutating base pairs in DNA, confusable characters) contribute partial matches. For this to define a valid kernel A must itself be positive semidefinite, since it is exactly the Gram matrix of the single-character strings at $p = 1$; with that assumption the suffix recurrence becomes

$$\kappa_p^S(sa, tb) = \lambda^2 A_{ab} \sum_{i,j} \lambda^{|s|-i+|t|-j} \kappa_{p-1}^S(s(1:i), t(1:j)),$$

and the same dynamic program runs unchanged. Numeric sequences fall in scope too: with a soft-matching A that compares reals, the gap-weighted kernel applies to time series and not just to symbolic text.

23.5 Beyond dynamic programming: trie-based kernels

Dynamic programming pays $O(|s| |t|)$ per kernel evaluation, which is expensive when one string is compared against a whole database. For the spectrum and mismatch kernels, whose features are contiguous substrings, a different data structure wins: the trie, a "retrieval tree" whose root-to-node paths spell out strings. A complete trie of depth p has one node per string of length up to p , and the idea is to push the length- p substrings of s and of t down the tree together, so that two substrings meet at a leaf exactly when they are equal. Only paths that both strings actually populate are ever created, and the cost is charged to those paths rather than to the product of the lengths (Leslie et al. 2002, Vishwanathan and Smola 2003).

Algorithm (trie-based p -spectrum kernel)

Input strings s, t , substring length p .

Output $\kappa_p(s, t)$, the p -spectrum kernel, in the global accumulator Kern.

1. Attach to the root the lists $L_s(\varepsilon) = \{(s(i:i+p-1), 0)\}$ and $L_t(\varepsilon)$ of all length- p substrings of each string, each tagged with a depth index 0; set Kern = 0 and call processnode(ε).
2. At a node v of depth = p : add $|L_s(v)| \cdot |L_t(v)|$ to Kern (all substrings reaching this leaf are equal).
3. Otherwise, if both lists are nonempty, extend each substring by its next symbol u_{i+1} , moving (u, i) into the child list $L_s(v u_{i+1})$, and likewise for t .
4. Recurse into processnode(va) for each symbol $a \in \Sigma$, then discard the subtree.

Each of the $|s| - p + 1$ substrings of s descends one level per recursion step and is touched $O(p)$ times, and the leaf products accumulate the shared-substring counts. Evaluating a full row of the kernel matrix of s against strings $t^{(1)}, \dots, t^{(\ell)}$ costs

$$O\left(p \ell \sum_{i=1}^{\ell} |t^{(i)}|\right),$$

because the trie for s is built once and reused. Since only $p |\Sigma|$ nodes are ever held in memory at a time (a path plus the children being expanded), the method is far lighter than the $|s| |t|$ dynamic-programming table, and as a by-product it yields every lower-order spectrum along the way, so a blended kernel $\sum_i a_i \kappa_i$ comes for free by indexing Kern by depth (Vishwanathan and Smola 2003).

The same trie carries the *mismatch* kernel of Leslie et al. (2002, 2004). Now each substring descends not only along its own symbols but also along symbols that differ, provided the running number of mismatches, stored as a third component of each list entry, does not exceed a budget m . A length- p substring can therefore end up in as many as $\binom{p}{m}(|\Sigma| - 1)^m$ leaf lists, one for each allowed error pattern, and the leaf products count (p, m) -neighbouring substrings as matches. Weighting the mismatches by a cost matrix and thresholding the total cost gives a further generalisation. A *restricted gap-weighted* kernel lives on the same trie: allowing up to m gaps, each of the $|s| - p - m + 1$ substrings spawns $\binom{p+m-1}{m-1}$ leaf entries, and the leaf computation reweights them by their recorded gap counts, at overall cost

$$O((|s| + |t|)(p + m) \binom{p+m}{m}).$$

When m is small this beats the full dynamic program and approximates it well for small λ ; when m grows large the dynamic program is again preferable, so the two computational styles are complementary rather than competing.

23.6 Kernels for trees

The dynamic-programming idea is not tied to the linear order of a string. Any object built by combining simpler objects of the same type, above all a tree, admits a kernel evaluated by a recursion that proceeds bottom up from the leaves. Trees arise from parsing natural language, from biological taxonomy, and from XML and program syntax, and a kernel between them lets the same classifiers operate on parse forests (Collins and Duffy 2002). We take a tree to be a directed acyclic graph in which every node but the root has in-degree one; edges point away from the root, $d^+(v)$ is the out-degree of v , and $\tau(v)$ is the *complete subtree* rooted at v , all of v 's descendants. Two notions of "subtree" give two kernels.

Definition (co-rooted and general subtrees)

A *co-rooted subtree* of T is obtained by deleting, from some internal nodes, all the complete subtrees hanging off their children; it keeps the root of T and, whenever it keeps a node, keeps all that node's siblings. A *general subtree* is any co-rooted subtree of some complete subtree $\tau(v)$. A tree is *proper* if it has at least one edge.

Before the kernel, a simpler count fixes the recursive style. The number $N(T)$ of proper co-rooted subtrees satisfies $N(\tau(v)) = 0$ at a leaf, because there are no proper subtrees, and at an internal node each child v_i offers either one of its $N(\tau(v_i))$ co-rooted subtrees or the option of being left as a leaf, so

$$N(T) = \prod_{i=1}^{d^+(r(T))} (N(\tau(\text{ch}_i(r(T)))) + 1),$$

with the "+1" for the leaf option. This same product, with a matched pair of trees, is the co-rooted subtree kernel.

Definition (co-rooted subtree kernel)

The feature space is indexed by all proper trees with $\phi_S^r(T) = 1$ if S is a co-rooted subtree of T and 0 otherwise, so $\kappa_r(T_1, T_2) = \sum_S \phi_S^r(T_1) \phi_S^r(T_2)$ counts the co-rooted subtrees common to T_1 and T_2 .

If the two roots have different out-degrees, or either tree is a single node, no proper co-rooted subtree is shared and $\kappa_r = 0$. Otherwise a common co-rooted subtree is any independent choice, at each of the d^+ matched children, of a shared co-rooted subtree of that child or the leaf option, giving

$$\kappa_r(T_1, T_2) = \prod_{i=1}^{d^+(r(T_1))} (\kappa_r(\tau(\text{ch}_i(r(T_1))), \tau(\text{ch}_i(r(T_2)))) + 1).$$

The recursion visits each node at most once, so κ_r costs $O(\min(|T_1|, |T_2|))$; a mismatch in degree or, for labelled trees, in label prunes the subtree at once and only speeds it up.

Example (co-rooted and all-subtree counts on two tiny trees)

Let S be a root with two children, a leaf and a "cherry" (a node with two leaf children), so $|S| = 5$. Let T be a root with two leaf children, $|T| = 3$. Both roots have out-degree 2.

node pair	degrees	κ_r
leaf vs leaf	0, 0	0
cherry vs T	2, 2	1
S vs T	2, 2	1

1. Match the roots. Both roots have out-degree 2, so $\kappa_r(S, T) = (\kappa_r(\text{leaf}, \text{leaf}) + 1)(\kappa_r(\text{cherry}, \text{leaf}) + 1)$.
2. Evaluate the children. The first factor is $(0 + 1) = 1$; the second pairs the cherry (degree 2) against a leaf (degree 0), degrees differ, so $\kappa_r = 0$ and the factor is $(0 + 1) = 1$.
3. Take the product. $\kappa_r(S, T) = 1 \cdot 1 = 1$: the single shared co-rooted subtree is the root with two leaf children.

4. Count self-similarities. $\kappa_r(S, S) = N(S) = (0 + 1)(N(\text{cherry}) + 1) = (1)(1 + 1) = 2$, while $\kappa_r(T, T) = N(T) = 1$.

Reading. Summing the co-rooted kernel over all node pairs gives the all-subtree kernel $\kappa(S, T) = \kappa_r(S, T) + \kappa_r(\text{cherry}, T) = 1 + 1 = 2$: the cherry pattern is shared once as the whole of T matched to S 's root and once as T matched to S 's internal cherry.

Verification artifact. checks/example-ch-strings2-example-22-3.json records the example source hash and verification scope.

The all-subtree kernel promotes the feature set from co-rooted subtrees to all subtrees, and its value can be assembled from the co-rooted kernel by the identity $\kappa(T_1, T_2) = \sum_{v_1 \in T_1, v_2 \in T_2} \kappa_r(\tau(v_1), \tau(v_2))$, since any subtree is co-rooted in the complete subtree at some node. A direct recursion is cheaper than that double sum. Partitioning the subtrees into those co-rooted with both roots and those sitting inside a child, with an inclusion-exclusion correction for the subtrees counted in two children, gives the dynamic program of Shawe-Taylor and Cristianini (2004).

Algorithm (all-subtree kernel)

Input unlabelled trees T_1, T_2 , nodes ordered so each parent follows its children.

Output $\kappa(T_1, T_2) = \text{DP}(n_1, n_2)$, the all-subtree kernel.

1. Fill the co-rooted table: $\text{DP}_r(i_1, i_2) = 0$ if the out-degrees differ or either node is a leaf, else $\prod_k (\text{DP}_r(\text{ch}_k(i_1), \text{ch}_k(i_2)) + 1)$.
2. Fill the all-subtree table: at internal matched pairs set $\text{DP}(i_1, i_2) = \text{DP}_r(i_1, i_2)$, then add $\sum_{j_1} \text{DP}(\text{ch}_{j_1}(i_1), i_2)$ and $\sum_{j_2} \text{DP}(i_1, \text{ch}_{j_2}(i_2))$, and subtract $\sum_{j_1, j_2} \text{DP}(\text{ch}_{j_1}(i_1), \text{ch}_{j_2}(i_2))$.
3. Return $\text{DP}(n_1, n_2)$.

The two tables are indexed by pairs of nodes and each cell does work proportional to the product of the out-degrees, so the kernel costs $O(|T_1| |T_2| d_{\max}^2)$ with $d_{\max}$ the largest out-degree. Labelled trees are handled by adding a label-equality test to the degree test, which for sparse labellings can be exploited to build small per-label tables and sum them, avoiding the full quadratic table.

23.7 The unifying view: convolution kernels

Every kernel in this chapter has the same shape: decompose each object into parts, compare the parts with a base kernel, and sum over the allowed decompositions. Haussler (1999) abstracted this into the *convolution* or R -kernel. A decomposition structure R relates an object x to the tuples $((x_1, \kappa_1), \dots, (x_d, \kappa_d))$ of parts, each carrying its own kernel, into which x can be split; the associated kernel is

$$\kappa_R(x, z) = \sum_{\bar{x} \in R^{-1}(x)} \sum_{\bar{z} \in R^{-1}(z)} [T(\bar{x}) = T(\bar{z})] \prod_{i=1}^{|T(\bar{x})|} \kappa_i(x_i, z_i),$$

where the type match $[T(\bar{x}) = T(\bar{z})]$ ensures only decompositions of the same shape are compared. This is a genuine kernel, an inner product in a suitable feature space (Shawe-Taylor and Cristianini 2004), and it specialises to everything above. Taking the parts to be co-rooted subtrees recovers the co-rooted subtree kernel and taking them to be all subtrees recovers the all-subtree kernel; taking the parts to be the length- p subsequences with weight $\lambda^{(i)} \kappa_0$ recovers the gap-weighted kernel; taking them to be distinct attribute subsets recovers the ANOVA kernel of the text-kernel chapter; and indexing the sub-kernels by the edges of a graph along paths recovers the walk-based graph kernels. The framework thus stitches the string and tree kernels here to the ANOVA and graph kernels elsewhere, and to the marginalization view of generative kernels, all as one family of sums of products over substructures; the open question it leaves is when such a sum, though exponentially large, admits an efficient dynamic program, which is exactly what the earlier sections answer case by case.

23.8 Summary

The feature spaces of string and tree kernels are astronomically large, but their inner products are cheap because a common pattern either avoids or ends on the last symbol, which telescopes the exponential sum into a table over prefixes. The all-subsequences kernel costs $O(|s| |t|)$, the fixed-length and gap-weighted kernels $O(p |s| |t|)$ through a suffix recursion with an inclusion-exclusion auxiliary table, and small edits to that table yield character-weighted, gap-counting, and soft-matching variants at the same cost. For contiguous-substring features the trie beats dynamic programming, evaluating the spectrum and mismatch kernels in time governed by the populated paths rather than the product of the lengths. Trees inherit the same recursive style bottom up: the co-rooted subtree kernel is a product over matched children in $O(\min(|T_1|, |T_2|))$, and the all-subtree kernel a dynamic program in $O(|T_1| |T_2| d_{\max}^2)$. Haussler’s convolution kernels reveal all of these, together with the ANOVA and graph kernels, as one construction: sum a product of part comparisons over the decompositions of a structured object. The following chapters push the same recursive computation onto graphs and onto kernels read from generative models.

23.9 Common mistakes and practical implications

For **Efficient String and Tree Kernels**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

23.10 Summary and further reading

This chapter established explain the central definitions and claims in Efficient String and Tree Kernels; Apply the chapter’s principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Watkins 2000), (Lodhi and Watkins 2002), (Shawe-Taylor and Cristianini 2004).

23.11 Exercises

1. warm-up Using the all-subsequences recurrence $\kappa(sa, t) = \kappa(s, t) + \sum_{k: t_k=a} \kappa(s, t(1:k-1))$, fill the table for $s = \text{"bar"}$ and $t = \text{"bat"}$ by hand and read off $\kappa(\text{"bar"}, \text{"bat"})$. List the common subsequences explicitly and check that their count, empty subsequence included, matches the corner cell.
2. computation For the gap-weighted kernel of length $p = 2$, fill the auxiliary table DP_2 for $s = \text{"bat"}$ and $t = \text{"bar"}$ and show that $\kappa_2(\text{"bat"}, \text{"bar"}) = \lambda^4$, the shared subsequence being "ba". Then compute the normalised kernel and confirm it equals $(2 + \lambda^2)^{-1}$, the same value the text finds for "cat" and "car". Hint
The level-1 suffix table has λ^2 at (b, b) and (a, a). Only the match at (a, a) has a nonzero predecessor $\text{DP}_2(1, 1) = \lambda^2$, so $\kappa_2^S = \lambda^2 \cdot \lambda^2 = \lambda^4$; the self-kernel is $2\lambda^4 + \lambda^6$.
3. computation A complete trie of depth p over an alphabet of size $|\Sigma|$ is used for the p -spectrum kernel. Explain why at most $p |\Sigma|$ nodes need be held in memory at any moment, and why the leaf products $|L_s(v)| \cdot |L_t(v)|$ sum to the correct spectrum inner product. How does the cost of evaluating one string against ℓ database strings compare with ℓ independent dynamic-programming evaluations?

4. proof Derive the inclusion-exclusion recurrence for $DP_p(k, l)$ from its definition $DP_p(k, l) = \sum_{i \leq k, j \leq l} \lambda^{k-i+l-j} \kappa_{p-1}^S(s(1:i), t(1:j))$. Show that the two shifted tables $\lambda DP_p(k-1, l)$ and $\lambda DP_p(k, l-1)$ overlap on the block indexed up to $(k-1, l-1)$, so that the term $-\lambda^2 DP_p(k-1, l-1)$ corrects the double count, leaving the fresh row-and-column contribution $\kappa_{p-1}^S(k, l)$. Hint

Write $DP_p(k, l)$ as the sum over the last row $i = k$, the last column $j = l$, and the interior block $i < k, j < l$. The interior block is $\lambda^2 DP_p(k-1, l-1)$; the last row and column, minus their shared corner, give the two λ -shifted terms.

5. proof Prove the co-rooted recursion $\kappa_r(T_1, T_2) = \prod_i (\kappa_r(\tau(\text{ch}_i(r(T_1))), \tau(\text{ch}_i(r(T_2)))) + 1)$ when the two roots share out-degree d^+ . Argue that a co-rooted subtree common to both trees is exactly an independent choice at each matched child of a common co-rooted subtree of that child or the leaf option, and that this is why the empty (improper) choice contributes the +1. Hint

The feature φ_S^r factors over the children: S is a co-rooted subtree of T iff each child slot of S is either a leaf or a co-rooted subtree of the matching child of T . Summing the product of features over all such S turns into a product of sums, one per child.

6. computation For the trees S (root with a leaf and a cherry) and T (a cherry) of the worked example, verify by direct enumeration that the all-subtree kernel $\kappa(S, T) = 2$ using the identity $\kappa(T_1, T_2) = \sum_{v_1, v_2} \kappa_r(\tau(v_1), \tau(v_2))$. List every node pair whose co-rooted kernel is nonzero and confirm the sum. Then compute $\kappa(S, S)$. Hint

Only pairs of proper complete subtrees with equal root degrees contribute. In S the proper complete subtrees are S itself and the cherry; in T only T . Both pairs give $\kappa_r = 1$, so $\kappa(S, T) = 2$; for $\kappa(S, S)$ enumerate all pairs of S 's proper complete subtrees.

7. exploration The gap-weighted kernel interpolates between two limits as λ varies. Argue that as $\lambda \rightarrow 0$ the length- p gap-weighted kernel tends to the p -spectrum kernel of contiguous substrings, since a subsequence with any gap carries a strictly higher power of λ and vanishes faster. What does the kernel tend to, up to weighting, as $\lambda \rightarrow 1$? Relate your answer to the fixed-length subsequences kernel of the text.
8. challenge Cast the gap-weighted k -subsequences kernel as a Haussler convolution kernel: exhibit the decomposition structure R whose parts are the pairs $(u, \lambda^{l(i)} \kappa_0)$ over index tuples $\mathbf{i} \in I_k$ with $u = s(\mathbf{i})$, where κ_0 returns 1 on identical substrings and 0 otherwise. Verify that the associated R -kernel $\kappa_R(s, t)$ reproduces $\sum_u \varphi_u(s) \varphi_u(t)$ with $\varphi_u(s) = \sum_{\mathbf{i}: u=s(\mathbf{i})} \lambda^{l(i)}$, and explain why the type-match indicator in the R -kernel is what enforces $u = u'$ in the product. Hint

Each decomposition of s selects one occurrence $\mathbf{i}$ and emits the single part $(s(\mathbf{i}), \lambda^{l(i)} \kappa_0)$. The R -kernel sums $\lambda^{l(i)} \lambda^{l(j)} \kappa_0(s(\mathbf{i}), t(\mathbf{j}))$ over all pairs, and κ_0 is nonzero only when the substrings coincide, giving $\sum_u \varphi_u(s) \varphi_u(t)$.

24 Kernels for Text

Natural language text is, after tabular data, the most common thing we ask a machine to analyse, and it arrives in a form that no dot product will accept: a stream of words. This chapter builds the bridge that the information retrieval community discovered decades ago and that turns out to be a kernel all along. We start from the bag of words and the vector space model, weight the coordinates with tf-idf so that informative terms count and function words do not, and read the resulting document similarity as a kernel. We then confront the model's central weakness, that documents sharing no words are declared perfectly dissimilar even when they discuss the same topic, and repair it three ways: a hand-built proximity matrix, the generalised vector space model that learns term relatedness from co-occurrence, and the latent semantic kernel that finds concepts by a singular value decomposition. Throughout, one theme recurs, the duality between representing a document by its terms and representing a term by the documents it lives in, and it is exactly the primal-dual duality of kernel methods seen through a new window. The whole development is concrete: a corpus of four tiny documents carries every idea, and every number is computed.

24.1 From documents to vectors: the bag of words

To measure how similar two documents are we first need to place them in a space where similarity means something. The simplest and still the most widely used device for this is the vector space model, introduced for retrieval by Salton, Wong, and Yang (1975) and developed at length by Salton (1988). Its founding simplification is to forget word order and keep only word counts: a document becomes a *bag of words*, a set that remembers multiplicities but not sequence. The phrase "dog bites man" and "man bites dog" collapse to the same bag, and the meaning of "bread winner" is lost when it splits into "bread" and "winner". This throws away real linguistic structure, yet for judging what a document is *about* it keeps enough, and it buys us a vector.

Definition (vector space model)

A *term* is a word: a maximal string of letters between separators. The set of all terms occurring in the collection of documents, the *corpus*, is the *dictionary*, of size T . Each document d is mapped to the feature vector

$$\phi(d) = (\text{tf}(t_1, d), \text{tf}(t_2, d), \dots, \text{tf}(t_T, d)) \in \mathbb{R}^T,$$

where $\text{tf}(t, d)$ is the *term frequency*, the number of times term t appears in d . The coordinate axes are the terms; a document is a point whose coordinates count its words.

Stacking these row vectors for a corpus of N documents gives the *document-term matrix* $D \in \mathbb{R}^{N \times T}$, with $D_{it} = \text{tf}(t, d_i)$. This single object supports the duality that organizes the whole chapter. Read across a row and you have a document described by its terms; read down a column and you have a term described by the documents it occurs in. A method stated in the intuitive term representation can therefore be dualised into a document-based implementation, exactly as a primal weight vector becomes a dual expansion in kernel methods.

The similarity of two documents is their inner product in this space, and that inner product is our first kernel.

Definition (vector space kernel)

The *vector space kernel* is the dot product of document vectors,

$$\kappa(d_1, d_2) = \langle \phi(d_1), \phi(d_2) \rangle = \sum_t \text{tf}(t, d_1) \text{tf}(t, d_2).$$

Its value is large when the two documents share many terms with high frequency, and zero when they share none.

Because a document uses only a handful of the many thousands of terms in the dictionary, $\phi(d)$ is extremely sparse. We never store it explicitly. Tokenisation converts each document into a sorted list $L(d)$ of (term-number, frequency) pairs, and the kernel is evaluated by a merge that walks the two lists in step and multiplies frequencies whenever the term numbers coincide. The cost is proportional to the document lengths,

$$\kappa(d_1, d_2) = A(L(d_1), L(d_2)), \quad \text{time } O(|d_1| + |d_2|),$$

never to the dimension T of the space we are notionally working in. This is the kernel trick wearing an information-retrieval hat: we compute an inner product in a huge space without ever forming a vector in it. Joachims (1998) made exactly this observation to run support vector machines on text, taking the vector space kernel as the base similarity.

Two normalisations are almost always applied. Long documents have larger term counts and hence larger norms, which conflates length with content; if length is irrelevant to the task we divide it out, using the cosine normalisation of any kernel,

$$\hat{\kappa}(d_1, d_2) = \frac{\kappa(d_1, d_2)}{\sqrt{\kappa(d_1, d_1) \kappa(d_2, d_2)}},$$

so that $\hat{\kappa}(d, d) = 1$ and similarity becomes the cosine of the angle between document vectors. And uninformative words such as "the", "of", "and", the *stop words*, are dropped from the dictionary altogether, which is the same as giving them weight zero. Weighting the remaining terms is the subject of the next section.

24.2 Term weighting and tf-idf

Not all terms deserve equal say. A term that occurs in almost every document, even after stop words are removed, cannot help tell documents apart, while a term confined to a few documents is highly diagnostic of their topic. We would like the coordinate for a term to be scaled up when the term is rare across the corpus and down when it is common. The classical device, standard since Salton (1988), is the *inverse document frequency*.

Let $\text{df}(t)$ be the number of documents containing term t , the document frequency, and let N be the corpus size. The idf weight is

$$w(t) = \ln \frac{N}{\text{df}(t)}.$$

A term in every document has $\text{df} = N$ and weight $\ln 1 = 0$; a term in a single document has the maximal weight $\ln N$. The logarithm is what keeps the scale gentle: without it a term appearing in one document out of a million would swamp everything, whereas $\ln$ compresses the range so that a term occurring in roughly half the documents is only a constant factor less important than the rarest. Combining this corpus-level weight with the document-level term frequency gives the *tf-idf* weight $w(t) \text{tf}(t, d)$ for term t in document d .

In kernel form the weighting is a diagonal linear map. Let R be the diagonal matrix with $R_{tt} = w(t)$, so that the reweighted embedding is $\phi(d)R$. The associated kernel is

$$\tilde{\kappa}(d_1, d_2) = \phi(d_1) R R^\top \phi(d_2)^\top = \sum_t w(t)^2 \text{tf}(t, d_1) \text{tf}(t, d_2),$$

the vector space kernel with each term's contribution scaled by $w(t)^2$. It is computed by the same list merge as before, now carrying the squared weights, so the tf-idf kernel costs no more than the plain one. Because idf uses

no label information, it can even be estimated from a large external unlabelled corpus that supplies background knowledge about which terms are rare.

Algorithm (tf-idf document-term matrix and vector space kernel)

Input corpus $d_1, \dots, d_N$; a stop list.

Output tf-idf Gram matrix $K \in \mathbb{R}^{N \times N}$ with $K_{ij} = \tilde{\kappa}(d_i, d_j)$.

1. Tokenise each d_i into terms, discard stop words, and assign every distinct term a number; build the dictionary of size T .
2. Form the sorted list $L(d_i)$ of (term-number, tf) pairs, equivalently the row $D_{i\cdot}$ of the document-term matrix.
3. Compute document frequencies $\text{df}(t) = |\{i : D_{it} > 0\}|$ and weights $w(t) = \ln(N/\text{df}(t))$.
4. For each pair (i, j) , merge $L(d_i)$ and $L(d_j)$, accumulating $\sum_t w(t)^2 \text{tf}(t, d_i) \text{tf}(t, d_j)$ over matching term numbers; store as K_{ij} .
5. Optionally cosine-normalise: $K_{ij} \leftarrow K_{ij} / \sqrt{K_{ii}K_{jj}}$.

We now make all of this concrete on a corpus we will reuse for the rest of the chapter.

Example (tf-idf and the vector space kernel on a tiny corpus)

Four documents over the six-term dictionary {cat, kitten, dog, puppy, car, engine}:

d_1 = "cat kitten", d_2 = "cat kitten kitten dog dog puppy", d_3 = "dog puppy", d_4 = "car car engine".

Document-term matrix D of raw term frequencies:

	cat	kitten	dog	puppy	car	engine
d_1	1	1	0	0	0	0
d_2	1	2	2	1	0	0
d_3	0	0	1	1	0	0
d_4	0	0	0	0	2	1

1. Count document frequencies. Each pet term appears in exactly two documents and each vehicle term in one, so $\text{df} = (2, 2, 2, 2, 1, 1)$ across the dictionary, with $N = 4$.
2. Weight by idf. $w(t) = \ln(4/\text{df}(t))$ gives $\ln 2 = 0.693$ for the four pet terms and $\ln 4 = 1.386$ for "car" and "engine". Rare vehicle terms are worth twice as much as the shared pet terms.
3. Form the tf-idf Gram matrix. With $\tilde{\kappa}(d_i, d_j) = \sum_t w(t)^2 \text{tf}(t, d_i) \text{tf}(t, d_j)$,

K	d_1	d_2	d_3	d_4
d_1	0.961	1.441	0	0
d_2	1.441	4.805	1.441	0
d_3	0	1.441	0.961	0
d_4	0	0	0	9.609

For instance $\tilde{\kappa}(d_1, d_2) = w(\text{cat})^2(1)(1) + w(\text{kitten})^2(1)(2) = 0.693^2(1 + 2) = 1.441$.

4. Normalise to cosines. Dividing by $\sqrt{K_{ii}K_{jj}}$ gives $\hat{\kappa}(d_1, d_2) = 1.441/\sqrt{0.961 \cdot 4.805} = 0.671$, and likewise $\hat{\kappa}(d_2, d_3) = 0.671$.

Reading. The kernel sees d_1 and d_2 as related and d_1 and d_3 as completely unrelated, because $K_{13} = 0$: the two documents share no term. Yet both are about pets. The vector space kernel cannot possibly know this, and d_4 sits in its own orthogonal corner. Fixing the $K_{13} = 0$ blind spot is the task of the rest of the chapter.

Verification artifact. checks/example-ch-text-example-23-1.json records the example source hash and verification scope.

24.3 Semantic smoothing with a proximity matrix

The example exposes the flaw plainly. Synonyms and topically related words occupy distinct, orthogonal coordinates, so two documents that say the same thing in different words are declared dissimilar. The vector space model retains no notion that "cat" and "kitten" are related. The only way to inject such a notion is to make the coordinate axes themselves non-orthogonal, so that placing mass on one term spills a little onto its relatives. This is done with a *proximity matrix* $P \in \mathbb{R}^{T \times T}$, whose off-diagonal entry $P_{ij} > 0$ records the semantic relatedness of terms i and j . The document is re-embedded as $\phi(d)P$, a less sparse vector that now has mass on every term related to one the document actually uses, and the kernel becomes

$$\tilde{\kappa}(d_1, d_2) = \phi(d_1) P P^\top \phi(d_2)^\top.$$

The matrix $Q = P P^\top$ is a symmetric term-term similarity: Q_{ij} says how much credit a document earns for containing term i when compared against a document containing term j . Because it factors as $P P^\top$, the kernel $\tilde{\kappa}$ is positive semidefinite for every choice of P , with feature map $\phi(d)P$; it is a bona fide Mercer kernel in the sense of Chapter 18, Mercer's Theorem, Spectra, and Rates, so all the generalization theory applies unchanged. Two documents can now be judged similar even with no shared term, provided their terms are related through P . Note that idf weighting is the special case $P = R$ diagonal, and the plain vector space kernel is $P = I$.

Where does P come from? One route is external knowledge. A semantic network such as WordNet arranges the dictionary in a hierarchy, with general terms above the specific ones they subsume, so that "spouse" sits above "husband" and "wife". The length of the shortest path between two terms in this tree measures their dissimilarity, and setting P_{ij} to the inverse of that path length turns the hierarchy into a proximity matrix. This hand-crafting works but demands a curated resource for every domain and language. The more autonomous route, and the one that dominates modern practice, is to let the corpus itself reveal which terms are related, through the statistics of their co-occurrence. That is the generalised vector space model.

24.4 The generalised vector space model

The guiding intuition is a second application of the same duality. We already regard two documents as similar when they share terms; dually, we may regard two terms as similar when they co-occur in many documents. The generalised vector space model (GVSM) builds a proximity matrix from exactly this. Two terms are related to the extent that they appear together across the corpus, and the natural bookkeeping of co-occurrence is the matrix $D^\top D$, whose (i, j) entry counts the documents in which terms i and j both occur (weighted by their frequencies). Taking the proximity matrix to be $P = D^\top D$ re-embeds a document by its vector of similarities to every document in the corpus,

$$\tilde{\phi}(d) = \phi(d) D^\top,$$

and the induced kernel is an inner product in this co-occurrence space,

$$\kappa_{\text{GVSM}}(d_1, d_2) = \phi(d_1) D^\top D \phi(d_2)^\top.$$

To see why this repairs the blind spot, unfold the product. The middle matrix $G = D^\top D$ is the term-term co-occurrence matrix, and G_{ij} is nonzero whenever some document uses both term i and term j . A document d_1 that never uses term j still contributes to $\kappa_{\text{GVSM}}(d_1, d_2)$ through terms i it does use that co-occur, elsewhere in the corpus, with the terms j that d_2 uses. The co-occurrences act as bridges: term relatedness is inferred, not supplied. Equivalently, since $\kappa_{\text{GVSM}}(d_1, d_2) = \langle \phi(d_1) D^\top, \phi(d_2) D^\top \rangle$, a document is represented by which corpus documents it resembles, and two documents are similar when they resemble the same third documents.

Example (co-occurrence bridges two disjoint documents)

Same corpus as before. The term-term co-occurrence matrix $G = D^\top D$ has the pet block

$$G_{\text{pets}} = \begin{pmatrix} 2 & 3 & 2 & 1 \\ 3 & 5 & 4 & 2 \\ 2 & 4 & 5 & 3 \\ 1 & 2 & 3 & 2 \end{pmatrix}$$

on {cat, kitten, dog, puppy}, and a separate 2×2 vehicle block; the two blocks do not interact.

1. Locate the bridge. The hub document d_2 contains all four pet terms, so every pair among them co-occurs: $G_{\text{cat,dog}} = 2$ even though "cat" and "dog" never share a document other than d_2 . The pet block is fully connected off the diagonal.
2. Form the GVSM Gram matrix. Computing $K^{\text{GVSM}} = D G D^\top$ gives

d_1	d_2	d_3	d_4	
d_1	13	36	9	0
d_2	36	118	36	0
d_3	9	36	13	0
d_4	0	0	0	25
3.	Read the repaired entry. Where the plain kernel had $K_{13} = 0$, the GVSM kernel has $K_{13}^{\text{GVSM}} = 9$, a cosine similarity of $9/\sqrt{13 \cdot 13} = 0.692$. The vehicle document d_4 still has zero similarity to every pet document, correctly, because no term of d_4 co-occurs with any pet term.			

Reading. Documents d_1 and d_3 share not one word, yet the corpus statistics make them 0.692-similar, because the hub document taught the model that their vocabularies belong together. Meanwhile pets and vehicles stay apart. Co-occurrence has manufactured exactly the semantic link the raw kernel was missing.

Verification artifact. checks/example-ch-text-example-23-2.json records the example source hash and verification scope.

24.5 Latent semantic kernels

The GVSM uses co-occurrence, but bluntly: $G = D^\top D$ keeps every fluctuation, including the noise of which particular documents happened to pair which particular words. We would prefer to extract only the strong, repeated patterns of co-occurrence, the handful of underlying *concepts* that the vocabulary is really tracking, and to measure documents in that low-dimensional concept space. This is latent semantic indexing (LSI), introduced by Deerwester et al. (1990), and cast as a kernel by Cristianini, Shawe-Taylor, and Lodhi (2002). The engine is the singular value decomposition of the document-term matrix.

Write the thin SVD

$$D = U \Sigma V^\top, \quad U \in \mathbb{R}^{N \times r}, \Sigma = \text{diag}(\sigma_1 \geq \dots \geq \sigma_r), V \in \mathbb{R}^{T \times r}.$$

The columns of V , the right singular vectors, are directions in term space, each a weighted combination of terms; these are the concepts. The corresponding singular value σ_i reports how much of the corpus's variation that concept explains. Because the SVD merges highly correlated coordinates, terms that co-occur frequently line up in the same singular vector, so a concept is precisely a bundle of terms the corpus treats as interchangeable. Keeping only the top k concepts, the columns V_k of V , and projecting documents onto them defines the latent semantic kernel.

Definition (latent semantic kernel)

Fix a rank $k \leq r$ and let $V_k \in \mathbb{R}^{T \times k}$ hold the top k right singular vectors of D . The *latent semantic kernel* projects each document onto the concept space before taking the inner product:

$$\tilde{\kappa}(d_1, d_2) = \phi(d_1) V_k V_k^\top \phi(d_2)^\top = \langle \phi(d_1) V_k, \phi(d_2) V_k \rangle.$$

It is the vector space kernel with proximity matrix $P = V_k$, a projection onto the leading directions of term co-occurrence.

This form makes the relationship to two earlier objects exact. Setting $P = V_k V_k^\top$ exhibits LSI as semantic smoothing whose proximity matrix is learned, not hand-built. And projecting onto the top singular directions of the data is precisely principal component analysis in feature space, so the latent semantic kernel is kernel PCA (see Chapter 13, Kernel PCA and Denoising) applied to the bag-of-words embedding. Comparing with the GVSM sharpens the point. The GVSM kernel is $\phi(d_1) D^\top D \phi(d_2)^\top = \phi(d_1) V \Sigma^2 V^\top \phi(d_2)^\top$, which keeps every concept and weights it by σ_i^2 . LSI makes two changes: it truncates to the top k concepts, discarding the noise directions, and it drops the Σ^2 factor, so the retained concepts are treated on an equal, orthonormal footing rather than being dominated by the loudest.

Everything so far is stated in the primal, term representation, and for a dictionary of hundreds of thousands of terms the matrix V_k is enormous. Duality rescues the computation. The projection can be evaluated entirely from the base kernel matrix $K = DD^\top$, whose eigenvalue-eigenvector pairs (λ_i, v_i) are related to the SVD by $\lambda_i = \sigma_i^2$. The i th concept coordinate of a new document d is then

$$(\phi(d) V_k)_i = \lambda_i^{-1/2} \sum_{j=1}^N (v_i)_j \kappa(d_j, d),$$

a formula that never touches term space: it needs only base-kernel evaluations $\kappa(d_j, d)$ against the training documents. This is why the construction is called a latent semantic *kernel*: the base κ may itself be the tf-idf kernel, or a polynomial or Gaussian kernel over bags of words, and LSI wraps a concept projection around whatever base similarity we started from.

Algorithm (latent semantic kernel)

Input document-term matrix D (or base Gram matrix $K = DD^\top$); rank k .

Output latent kernel value $\tilde{\kappa}(d, d')$ for documents d, d' .

1. Compute the SVD $D = U \Sigma V^\top$; equivalently eigendecompose $K = DD^\top$ as (λ_i, v_i) with $\sigma_i = \sqrt{\lambda_i}$.
2. Keep the top k right singular vectors V_k (the concepts), discarding the rest as noise.
3. Project each document onto concept space: $\phi(d) \mapsto \phi(d) V_k$, or dually $(\lambda_i^{-1/2} \sum_j (v_i)_j \kappa(d_j, d))_{i=1}^k$.
4. Return the inner product $\tilde{\kappa}(d, d') = \langle \phi(d) V_k, \phi(d') V_k \rangle$.
5. Optionally cosine-normalise the result.

Example (rank-2 LSI links documents with no shared terms)

Same document-term matrix D as above. Its singular values, from `numpy.linalg.svd`, are

$$\sigma = (3.440, 2.236, 1.414, 0.411).$$

The top two right singular vectors are the concept directions

$$v_1 = (0.348, 0.615, 0.615, 0.348, 0, 0), \quad v_2 = (0, 0, 0, 0, 0.894, 0.447).$$

1. Name the concepts. The first concept v_1 loads positively and only on the four pet terms: it is the "pet" concept. The second v_2 loads only on "car" and "engine": the "vehicle" concept. The discarded third direction $(\frac{1}{2}, \frac{1}{2}, -\frac{1}{2}, -\frac{1}{2}, 0, 0)$ merely splits cats from dogs, corpus-specific noise we drop.
2. Project onto the two concepts. The concept coordinates DV_2 of the documents are

	pet	vehicle
d_1	0.964	0
d_2	3.158	0
d_3	0.964	0
d_4	0	2.236

Documents d_1 and d_3 land on the *same* point (0.964, 0): in a world of two concepts each is a pure pet document.

3. Form the latent kernel. $\tilde{\kappa}(d_i, d_j) = \langle (DV_2)_i, (DV_2)_j \rangle$ gives $\tilde{\kappa}(d_1, d_3) = 0.964 \times 0.964 = 0.929$, whereas the raw kernel had $\kappa(d_1, d_3) = 0$.
4. Normalise. The cosine latent similarity is $0.929/\sqrt{0.929 \cdot 0.929} = 1.000$ between d_1 and d_3 , and 0 between d_1 and d_4 .

Reading. Two documents with disjoint vocabularies come out perfectly aligned, because once the corpus is summarised by its two real concepts both documents are seen to be entirely about pets. The vehicle document remains orthogonal. The rank-2 projection has extracted the meaning the surface words hid: this is the payoff that a hand-built proximity matrix promised and that the SVD delivers automatically.

Verification artifact. checks/example-ch-text-example-23-3.json records the example source hash and verification scope.

24.6 Semantic diffusion kernels

The GVSM linked documents through one step of co-occurrence, and LSI distilled the strong co-occurrence directions. A further idea pushes the dual reasoning to its limit: if documents that share terms are similar, and terms that co-occur are similar, then documents that share terms which merely co-occur should also count as somewhat similar, and so on through ever longer chains. Iterating this interaction leads to a recurrence. Writing $K = DD^\top$ for the document kernel and $G = D^\top D$ for the term co-occurrence matrix, define refined similarities by

$$\hat{K} = \mu D\hat{G}D^\top + K, \quad \hat{G} = \mu D^\top \hat{K}D + G,$$

so that each similarity is augmented by the indirect similarity routed through the other. The factor $\mu < \|K\|^{-1}$ makes the longer-range contributions decay geometrically, so the sum converges. The solution has a closed form.

Proposition (diffusion solution)

Provided $\mu < \|K\|^{-1} = \|G\|^{-1}$, the document similarity solving the recurrence is

$$\hat{K} = K(I - \mu G)^{-1} \text{ (the von Neumann kernel over base } K), \quad \hat{G} = G(I - \mu G)^{-1}.$$

Moreover $\hat{K}$ is itself a vector space kernel, with proximity matrix $P = \sqrt{\mu\hat{G} + I}$, since $D\hat{P}\hat{P}^\top D^\top = D(\mu\hat{G} + I)D^\top = \mu D\hat{G}D^\top + K = \hat{K}$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

This is the von Neumann diffusion kernel of the string and graph chapters reappearing on text: $(I - \mu G)^{-1} = \sum_{n \geq 0} \mu^n G^n$ accumulates co-occurrence paths of every length, weighted down by μ^n . Small μ recovers the plain kernel; larger μ trusts longer semantic chains. Such semantic spaces are especially powerful across languages: aligning a bilingual corpus and extracting shared concepts by correlation, using the kernel canonical correlation analysis of Chapter 15, Kernel CCA and Correlation Analysis, yields a language-independent concept representation for cross-lingual retrieval, where a query in one language finds documents in another.

24.7 Neural embeddings and optimal transport

Every repair so far keeps the coordinate axes pinned to the dictionary and asks a matrix, hand-built P , the co-occurrence $D^\top D$, or the truncated projection $V_k V_k^\top$, to lend one term a little of another's mass. The approach that now dominates practice changes the axes themselves. Instead of a term being an orthogonal basis vector, each term is assigned a short dense vector, its *embedding*, learned so that words used in similar contexts land near one another. Two threads grow from this idea, and they fall on opposite sides of the line this book keeps drawing between kernels and distances. Reducing a document to a single point and taking inner products gives, once again, a positive definite kernel, the direct successor of the latent semantic kernel of the previous sections. Reducing a document instead to a whole cloud of word-points, and asking the least work to morph one cloud into the other, gives the Word Mover's Distance, a genuine metric that, as we will see, is not a positive definite kernel at all. Keeping the two straight is the point of this section.

24.7.1 Word and document embeddings as a linear kernel

A word embedding maps every term t of the dictionary to a dense vector $e_t \in \mathbb{R}^m$, with m small, a few hundred, and independent of the dictionary size T . The two standard constructions learn these vectors from the statistics of a huge corpus: word2vec (Mikolov et al. 2013) trains a shallow predictor to reconstruct a word from its neighbours, or the neighbours from the word, and GloVe (Pennington, Socher, and Manning 2014) factorises a log co-occurrence matrix. Both end with a table $E \in \mathbb{R}^{T \times m}$ whose row t is e_t . The geometry is the payoff: because "cat" and "kitten" occur in overlapping contexts, their rows come out nearly parallel, whereas "car" points elsewhere. On the vectors themselves the natural similarity is the plain *linear kernel* $\kappa(t, t') = \langle e_t, e_{t'} \rangle$, a bona fide positive definite kernel because it is an inner product in $\mathbb{R}^m$.

To compare documents rather than single words we must turn each cloud of word-vectors into something comparable. The simplest and most common choice represents a document by the frequency-weighted mean of the embeddings of its words,

$$\mu(d) = \sum_t p_t(d) e_t = \phi_{\text{nbow}}(d) E, \quad p_t(d) = \frac{\text{tf}(t, d)}{\sum_s \text{tf}(s, d)},$$

where $\phi_{\text{nbow}}(d)$ is the normalised bag of words, a probability vector over the dictionary. This $\mu(d)$ is exactly the empirical *kernel mean embedding* of the mean-embedding chapter, the average feature under the map $t \mapsto e_t$ of the word distribution $p(d)$. The induced document kernel is the linear kernel of these means,

$$\kappa_{\text{emb}}(d_1, d_2) = \langle \mu(d_1), \mu(d_2) \rangle = \phi_{\text{nbow}}(d_1) E E^\top \phi_{\text{nbow}}(d_2)^\top,$$

and its cosine normalisation is the number practitioners usually report. It is positive definite for the same reason the vector space kernel was, it is an inner product, now in the m -dimensional embedding space rather than the T -dimensional term space.

Read the middle expression again and the connection to everything before it snaps into place. It is the semantic-smoothing kernel $\phi(d_1)PP^\top\phi(d_2)^\top$ of the proximity-matrix section with proximity matrix $P = E$: the term-term similarity $Q = EE^\top$ is now the Gram matrix of learned word vectors, in place of a hand-built semantic network or a raw corpus co-occurrence count. The latent semantic kernel built its proximity from one corpus by an SVD, taking $P = V_k$; word embeddings build a far richer $P = E$ from a corpus of billions of words. In this precise sense the embedding kernel is the modern successor of the latent semantic kernel: the same kernel skeleton $\phi PP^\top\phi^\top$, with a better-learned P .

24.7.2 The Word Mover's Distance

Collapsing a document to a single mean vector throws away everything except the centroid of its word cloud. A second idea keeps the whole cloud. Regard a document as the distribution that places mass $p_i(d)$ at the point $e_i \in \mathbb{R}^m$, one atom per distinct word, and compare two documents by the least total work needed to carry the first pile of mass onto the second, where moving a unit of mass from e_i to e_j costs the Euclidean distance $\|e_i - e_j\|$. This is precisely the optimal transport problem of the transport chapter, and its value is the 1-Wasserstein distance between the two word distributions. Under the name *Word Mover's Distance* (WMD) it was introduced for text by Kusner, Sun, Kolkin, and Weinberger (2015):

$$\text{WMD}(d_1, d_2) = \min_{T \geq 0} \sum_{i,j} T_{ij} \|e_i - e_j\| \quad \text{subject to} \quad \sum_j T_{ij} = p_i(d_1), \quad \sum_i T_{ij} = p_j(d_2).$$

The coupling T_{ij} records how much of word i 's mass in the first document is routed to word j in the second. Because moving "cat" onto "dog" is cheap when their vectors are close, two documents that share no term can still be a short move apart, which is exactly the blind spot the whole chapter has been chasing, cured here by the ground metric of the embedding space rather than by a proximity matrix.

WMD captures semantics, but it is a *distance*, not a kernel, and the difference is not cosmetic. It inherits the metric axioms of the 1-Wasserstein distance, so it is nonnegative, symmetric, zero only between documents with the same word distribution, and obeys the triangle inequality; that is what makes it a sound input to a distance-based classifier such as k -nearest-neighbours, which is how Kusner et al. (2015) used it. But turning a distance into a similarity by negating or exponentiating it does not, in general, yield a positive definite kernel. The 1-Wasserstein distance with Euclidean ground cost is not of negative type in dimension $m \geq 2$, so neither $-\text{WMD}$ nor $\exp(-\gamma \text{WMD})$ is guaranteed positive semidefinite, and on real corpora it is not. WMD therefore belongs with the indefinite kernels of the Krein-space chapter: to place it inside a kernel machine one either works in the Krein space that its indefinite Gram matrix defines, or replaces it with a positive definite surrogate such as the embedding linear kernel above. The worked example makes both the semantic gain and the loss of positive definiteness concrete.

Example (embedding kernel and Word Mover's Distance beat the bag of words)

Five words carry a toy 2-D embedding, the pets clustered and a vehicle set apart:

$$e_{\text{cat}} = (1, 3), e_{\text{kitten}} = (1, 4), e_{\text{dog}} = (2, 3), e_{\text{puppy}} = (2, 4), e_{\text{car}} = (8, 0).$$

Three short documents, no two of which share a word:

$A = \text{"cat kitten kitten"}$, $B = \text{"dog puppy"}$, $C = \text{"car"}$, with normalised bags $p(A) = (\frac{1}{3}, \frac{2}{3})$ on {cat, kitten}, $p(B) = (\frac{1}{2}, \frac{1}{2})$ on {dog, puppy}, and $p(C) = 1$ on {car}.

1. Ask the bag of words. The raw count vectors are orthogonal, so the bag-of-words kernel returns $\kappa(A, B) = 0$ and $\kappa(A, C) = 0$. It sees A as equally unrelated to the pet document B and to the vehicle document C , which is exactly wrong.
2. Average the embeddings. The frequency-weighted means are $\mu(A) = (1, 3.667)$, $\mu(B) = (2, 3.5)$, $\mu(C) = (8, 0)$. Their linear kernels are $\langle \mu(A), \mu(B) \rangle = 14.833$ and $\langle \mu(A), \mu(C) \rangle = 8$; as cosines these are 0.968 for A, B and 0.263 for A, C . The embedding kernel places A firmly beside B and far from C .

3. Move the words. To transport $p(A)$ onto $p(B)$ the ground costs are $\|e_{\text{cat}} - e_{\text{dog}}\| = 1$, $\|e_{\text{cat}} - e_{\text{puppy}}\| = \sqrt{2}$, $\|e_{\text{kitten}} - e_{\text{dog}}\| = \sqrt{2}$, and $\|e_{\text{kitten}} - e_{\text{puppy}}\| = 1$. The optimal plan sends cat's $\frac{1}{3}$ to dog, kitten's $\frac{1}{2}$ to puppy, and kitten's remaining $\frac{1}{6}$ to dog, for $\text{WMD}(A, B) = \frac{1}{3}(1) + \frac{1}{6}\sqrt{2} + \frac{1}{2}(1) = 1.069$.
4. Compare the distances. Transporting A onto the far vehicle document costs $\text{WMD}(A, C) = \frac{1}{3}\|e_{\text{cat}} - e_{\text{car}}\| + \frac{2}{3}\|e_{\text{kitten}} - e_{\text{car}}\| = 7.913$, more than seven times the pet-to-pet distance 1.069. The move again records A as close to B and far from C .

Reading. Documents A and B share no word, yet both the embedding cosine kernel (0.968) and the Word Mover's Distance (1.069) recognise them as near, while placing the vehicle document C far away (cosine 0.263, distance 7.913). The bag-of-words kernel returned 0 for both pairs and could not tell the two topics apart. Learned word geometry supplies the semantic link that orthogonal term axes destroyed.

Verification artifact. checks/example-ch-text-example-23-4.json records the example source hash and verification scope.

Remark (WMD is a distance, not a positive definite kernel)

Gather the three worked-example documents into the negated similarity matrix $K = -\text{WMD}$, with entries $K_{ij} = -\text{WMD}(d_i, d_j)$ and zero diagonal. Its eigenvalues are $\{-11.083, 1.060, 10.023\}$: they sum to zero, as those of any zero-diagonal symmetric matrix must, and their signs are mixed, so K has a negative eigenvalue and is not positive semidefinite. This is the small print behind the previous paragraph. WMD is a perfectly good metric, but negating it, or exponentiating it, does not manufacture a positive definite kernel, and to use it one must pass to the Krein-space methods of Chapter 28, Indefinite and Krein-Space Kernels or fall back on the embedding linear kernel, which is positive definite by construction.

24.7.3 Contextual and sentence embeddings as a cosine kernel

Word embeddings give every occurrence of a word the same vector, so "bank" by a river and "bank" holding money collapse together, and a document still has to be assembled from its words by hand, whether by averaging or by transport. Contextual models remove both limitations. A pretrained transformer reads a whole sentence or short document at once and emits a single dense vector $s(d) \in \mathbb{R}^m$ for it, computed so that the vector reflects each word in the context of its neighbours. Sentence-BERT (Reimers and Gurevych 2019) is the standard such encoder tuned for comparison: it is trained so that the cosine of two sentence vectors tracks their semantic similarity. The similarity practitioners then use is the *cosine kernel*

$$\kappa_{\cos}(d_1, d_2) = \frac{\langle s(d_1), s(d_2) \rangle}{\|s(d_1)\| \|s(d_2)\|},$$

which is positive definite, being the plain linear kernel of the unit-normalised vectors $s(d)/\|s(d)\|$. It is the very cosine normalisation of the bag-of-words section, applied now to a dense learned representation instead of a sparse tf-idf vector.

Seen together, these embedding kernels are the latent semantic kernel grown up. LSI learned a linear concept map V_k from a single document-term matrix by one SVD and compared documents by the inner product of their projections; word2vec, GloVe, and sentence encoders learn a much richer, nonlinear map from enormous corpora, but the final comparison is again an inner product, or its cosine, in the learned space. The kernel skeleton, an inner product after a semantic re-embedding, is unchanged from the first section of this chapter to the last; only the map that builds the embedding has moved from a hand-set proximity matrix, through a corpus SVD, to a trained neural network. The Word Mover's Distance is the one genuinely new object here, and the price it pays for keeping the whole word cloud, rather than a single vector, is that it leaves the positive definite world for the metric one.

24.8 String and word-sequence kernels

Every kernel above discards word order the moment it forms the bag of words. Sometimes order carries the meaning: "the dog bit the man" and "the man bit the dog" have identical bags. String kernels restore order by comparing documents through the substrings, or subsequences, they share, and they apply to text at the character or the word level. A word-sequence kernel treats a document as a string whose alphabet is the dictionary and counts the (possibly gapped) word n -grams two documents have in common, with longer gaps penalised. Because the number of such subsequences is astronomically large, they are never enumerated; a dynamic program evaluates the kernel directly, in time polynomial in the document lengths and the subsequence length, exactly as for character strings. The construction, its recurrences, and its efficient evaluation belong to the string-kernel development of Chapter 21, Kernels for Sequences and Chapter 22, Efficient String and Tree Kernels; here we note only that they slot into the same framework, another choice of feature map over text, and that they can be combined with the vector space kernels, for example by a polynomial or Gaussian kernel over the normalised bag of words, when both content and a little structure matter.

24.9 Summary

Text becomes geometry through the bag of words, and the vector space kernel is the inner product of documents so embedded, computed in time linear in document length without ever forming the huge sparse vectors. tf-idf weights the axes so that rare, informative terms speak loudest. The model's one serious defect, blindness to documents that share meaning but not words, is cured by a proximity matrix: hand-built from a semantic network, learned from co-occurrence in the GVSM, or distilled into concepts by the SVD in the latent semantic kernel, which is kernel PCA on bags of words. Diffusion kernels iterate the co-occurrence reasoning to convergence, and string kernels restore the word order the bag threw away. Running through all of it is the primal-dual duality between describing a document by its terms and a term by its documents, the same duality that lets every one of these kernels be computed without visiting the space it lives in.

24.10 Common mistakes and practical implications

For **Kernels for Text**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

24.11 Summary and further reading

This chapter established explain the central definitions and claims in Kernels for Text; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Salton and Yang 1975), (Salton 1988), (Joachims 1998).

24.12 Exercises

1. warm-up Using the corpus of the worked examples, compute by hand the plain (unweighted) vector space Gram matrix DD^T and confirm that $\kappa(d_1, d_3) = 0$ while $\kappa(d_2, d_2) = 10$.
2. warm-up A term appears in every document of a corpus. Show from $w(t) = \ln(N/\text{df}(t))$ that its tf-idf weight is zero, and explain why this makes stop-word removal a special case of tf-idf weighting.
3. warm-up Show that the cosine-normalised kernel $\hat{\kappa}(d_1, d_2) = \kappa(d_1, d_2)/\sqrt{\kappa(d_1, d_1)\kappa(d_2, d_2)}$ is itself a valid kernel, by exhibiting its feature map $\phi(d)/\|\phi(d)\|$. Why does normalisation remove the effect of document length?
4. computation Let P be any $T \times T$ proximity matrix and $\tilde{\kappa}(d_1, d_2) = \phi(d_1)PP^T\phi(d_2)^T$. Prove that $\tilde{\kappa}$ is positive semidefinite for every choice of P , and identify the feature map. Which choices of P recover (a) the plain vector space kernel, (b) the tf-idf kernel, (c) the latent semantic kernel?
5. computation Verify the GVSM identity $\kappa_{\text{GVSM}}(d_1, d_2) = \phi(d_1)D^TD\phi(d_2)^T = \langle \phi(d_1)D^T, \phi(d_2)D^T \rangle$, and interpret the vector $\phi(d)D^T$ in words. Using the term-term matrix G from the co-occurrence example, recompute $K_{13}^{\text{GVSM}} = 9$. Hint: $\phi(d)D^T$ has one coordinate per corpus document; the coordinate for d_j is the plain kernel $\kappa(d, d_j)$.
6. computation Starting from the GVSM kernel $\phi(d_1)D^TD\phi(d_2)^T$, use the SVD $D = U\Sigma V^T$ to write it as $\phi(d_1)V\Sigma^2V^T\phi(d_2)^T$. State precisely the two modifications LSI makes to this expression and explain what each one buys. Hint: one modification changes the sum's range, the other changes the weights σ_i^2 .
7. challenge Show that the latent semantic kernel is kernel PCA on the bag-of-words embedding, by proving the dual projection formula $(\phi(d)V_k)_i = \lambda_i^{-1/2} \sum_j (v_i)_j \kappa(d_j, d)$, where (λ_i, v_i) are the eigenpairs of $K = DD^T$.
Hint: with $D = U\Sigma V^T$, the left singular vectors u_i are the eigenvectors v_i of K ; the term-space concept is $V_{\cdot i} = \sigma_i^{-1}D^Tu_i$, and $\phi(d)V_{\cdot i} = \sigma_i^{-1} \sum_j (u_i)_j \kappa(d_j, d)$.
8. challenge Prove the diffusion identity: the recurrences $\hat{K} = \mu D\hat{G}D^T + K$ and $\hat{G} = \mu D^T\hat{K}D + G$, with $K = DD^T$ and $G = D^TD$, are solved by $\hat{K} = K(I - \mu G)^{-1}$ and $\hat{G} = G(I - \mu K)^{-1}$, provided $\mu < \|G\|^{-1}$. Hint: substitute the second recurrence into the first, use $D^TD = G$ and $DD^T = K$, and sum the resulting Neumann series $\sum_{n \geq 0} \mu^n G^n = (I - \mu G)^{-1}$, which converges because $\mu\|G\| < 1$.
9. computation Let $E \in \mathbb{R}^{T \times m}$ be a word-embedding table and represent a document by its normalised bag of words $\phi_{\text{nbow}}(d)$. Show that the embedding document kernel $\kappa_{\text{emb}}(d_1, d_2) = \phi_{\text{nbow}}(d_1)EE^T\phi_{\text{nbow}}(d_2)^T$ is positive semidefinite by exhibiting its feature map, and identify it as a semantic-smoothing kernel with proximity matrix $P = E$. Which latent kernel of this chapter is the special case $P = V_k$? Hint: the feature map is $d \mapsto \mu(d) = \phi_{\text{nbow}}(d)E \in \mathbb{R}^m$, the mean word embedding; the latent semantic kernel is $P = V_k$.
10. challenge Using the toy embedding of the neural-embedding worked example, confirm the Word Mover's Distances $\text{WMD}(A, B) = 1.069$, $\text{WMD}(A, C) = 7.913$, and $\text{WMD}(B, C) = 6.960$, then assemble $K = -\text{WMD}$ on $\{A, B, C\}$ and show it has a negative eigenvalue, so it is not a positive definite kernel. Explain why a symmetric matrix with zero diagonal and any nonzero off-diagonal entry can never be positive semidefinite, and name the two ways this section offers to use WMD inside a kernel method regardless. Hint: the trace is zero, so the eigenvalues sum to zero; a positive semidefinite matrix with $K_{ii} = 0$ must have its whole i th row zero. Either work in the Krein space of Chapter 28, Indefinite and Krein-Space Kernels or replace WMD by the positive definite embedding kernel.

25 Kernels for and on Graphs

Graphs appear in machine learning in two very different roles, and the word alone hides the difference. Sometimes each data point is a graph: a molecule, an image parsed into regions, a program's call structure, and we want a kernel that compares two whole graphs. Sometimes there is a single fixed graph, an interaction network or a discretized domain, and each data point is one of its nodes: we want a kernel that compares two vertices of the same graph. These are genuinely separate problems with separate mathematics, so this chapter keeps them apart. The first half builds kernels between graphs and runs headlong into computational hardness: the most expressive comparisons are NP-hard, and the art is to trade a little expressiveness for tractability by counting walks and subtrees instead of subgraphs. The second half builds kernels on a graph from the graph Laplacian, and finds a clean answer, the diffusion kernel, that is the discrete cousin of the Gaussian and that we will read off directly from a discrete harmonic analysis.

25.1 Part A: Kernels between graphs

The template is the one we have used throughout. To classify or regress on objects living in a space $\mathcal{X}$, we map each object x to a vector $\Phi(x)$ in a Hilbert space $\mathcal{H}$, either explicitly or implicitly through a kernel

$$K(x, x') = \langle \Phi(x), \Phi(x') \rangle_{\mathcal{H}} = \Phi(x)^{\top} \Phi(x'),$$

and then run a linear method in $\mathcal{H}$. What is new is that now $\mathcal{X}$ is a set of graphs. The motivating applications are concrete. In virtual screening for drug discovery, each molecule is a labeled graph (atoms as vertices, bonds as edges) and we want to predict biological activity, for instance whether a compound is active against a target, from public assays such as the NCI AIDS screen. In image retrieval and classification (Harchaoui and Bach, 2007) an image is segmented into regions and encoded as a graph of adjacent regions. In both cases the learning problem is standard once we have a kernel; the whole difficulty is building a good kernel between graphs.

25.1.1 Explicit features: indexing by substructures

The most transparent approach is to represent each graph *explicitly* by a fixed-dimensional vector $\Phi(x) \in \mathbb{R}^p$ and then run any regression or pattern-recognition algorithm (SVM, kNN, PLS, decision trees) on those vectors. In chemoinformatics this is the classical idea of a *structural key* or *2D fingerprint*: fix a limited list of predefined substructures (particular rings, functional groups, fragments) and index a molecule by a binary vector recording, for each substructure, whether it is present. The premise is that the presence or absence of particular substructures carries much of the predictive signal, so counting them should retain the information that matters.

This raises the design question that governs everything that follows: which descriptors, that is, which substructures, do we count? Three demands pull against each other.

- **Expressiveness.** The features should retain as much information about the graph as possible.
- **Computation.** The features should be fast to compute.
- **Dimension.** A very large feature vector costs memory and speed, and creates statistical problems (many weakly populated coordinates).

The natural way to be expressive is to index by *all* substructures of a given kind and let the learning algorithm decide which matter. The trouble is that detecting the presence of a substructure can be computationally hard, and this is not a minor caveat: it is the central obstruction of the whole subject.

Definition (subgraph)

A *subgraph* of a graph (V, E) is a graph (V', E') with $V' \subset V$ and $E' \subset E$.

Indexing a graph by the counts of all its connected subgraphs would be maximally expressive. It is also intractable.

Theorem (all-subgraph indexing is hard)

Computing the vector of occurrences of all subgraphs is NP-hard.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The *linear graph* (path graph) on n vertices is a subgraph of a graph X with n vertices if and only if X has a Hamiltonian path, that is, a path visiting each vertex exactly once. So if we could read off, from the subgraph-occurrence vector, whether the length- n linear graph occurs, we would decide the Hamiltonian-path problem, which is NP-complete. Hence computing all subgraph occurrences is NP-hard. $\square$

One might hope that restricting to a simpler family of substructures escapes the difficulty. The most natural restriction is to paths.

Definition (path)

A *path* of a graph (V, E) is a sequence of *distinct* vertices $v_1, \dots, v_n \in V$ (so $i \neq j \Rightarrow v_i \neq v_j$) with $(v_i, v_{i+1}) \in E$ for $i = 1, \dots, n-1$. Equivalently, the paths are exactly the linear subgraphs.

Indexing by all paths, that is, recording for each possible label sequence how many paths carry it, is no easier.

Theorem (all-path indexing is hard)

Computing the vector of occurrences of all paths is NP-hard.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Identical to the subgraph case: because paths are exactly the linear subgraphs, deciding whether the length- n path occurs already decides the Hamiltonian-path problem. $\square$

The escape is not to change the type of substructure but to *limit the set* of substructures we insist on counting. Many computationally efficient choices have been proposed:

- substructures selected by domain knowledge (the classic MDL fingerprints);
- all paths up to a fixed length k (the Openeye fingerprint, Nicholls 2005);
- all shortest-path lengths (Borgwardt and Kriegel, 2005);
- all subgraphs up to k vertices, the graphlet kernel (Shervashidze et al., 2009);
- all subgraphs that are *frequent* in the database (Helma et al., 2004).

Two of these deserve their complexity accounting, because they show that limiting the family really does buy tractability.

Example (shortest-path indexing)

Index a graph by the counts of its shortest paths, keyed by the path length together with the labels of the two endpoints; a coordinate might read "there are two shortest paths of length 1 between an A and an A ." A graph on n vertices has $O(n^2)$ shortest paths (one per ordered pair), and the whole count vector is computed in $O(n^3)$ by the Floyd-Warshall all-pairs shortest-path algorithm (Borgwardt and Kriegel, 2005).

Verification artifact. checks/example-ch10-example-24-1.json records the example source hash and verification scope.

Example (graphlet counting)

Index a graph by the counts of all its subgraphs on up to k vertices. Naive enumeration scales as $O(n^k)$. For bounded-degree graphs one does much better: connected graphlets can be enumerated in $O(n d^{k-1})$ for graphs of maximum degree d and $k \leq 5$ (Shervashidze et al., 2009). When even that is infeasible, one samples subgraphs at random and estimates the counts.

Verification artifact. checks/example-ch10-example-24-2.json records the example source hash and verification scope.

Summary of the explicit approach

Explicit enumeration of substructure occurrences can be prohibitive: all subgraphs and all paths are both NP-hard. Restricting to a smaller, cleverly chosen family (short paths, shortest-path lengths, small graphlets, frequent patterns) restores polynomial time. And the NP-hardness is a worst case: for the graphs that occur in practice, often sparse and of small degree, the exact strategy may be perfectly affordable, so the right method depends on the data.

25.1.2 The expressiveness-complexity tradeoff

Instead of committing to an explicit feature vector, we can represent each graph *implicitly* through a kernel $K(G_1, G_2) = \langle \Phi(G_1), \Phi(G_2) \rangle$ and run a kernel method. This is attractive because it lets the feature space $\mathcal{H}$ be enormous, even infinite-dimensional, as long as the inner product is cheap. But before chasing cheap kernels we should ask what a good graph kernel could possibly look like, and here a sharp tension appears.

Definition (complete graph kernel)

A graph kernel K is *complete* if it distinguishes non-isomorphic graphs:

$$\forall G_1, G_2 \in \mathcal{X}, \quad d_K(G_1, G_2) = 0 \implies G_1 \simeq G_2,$$

where d_K is the Hilbert distance induced by K . Equivalently, $\Phi(G_1) \neq \Phi(G_2)$ whenever G_1 and G_2 are not isomorphic.

Completeness is exactly the property we want for expressiveness. If a kernel is *not* complete, then two non-isomorphic graphs collapse to the same point of $\mathcal{H}$, and no function built from the kernel can ever tell them apart: there is no hope of learning all functions over $\mathcal{X}$. On the other hand, we need the kernel to be tractable, computable in polynomial time with a small degree. Can a kernel be both tractable and complete? The following proposition says the two goals collide.

Proposition (Gärtner et al., 2003)

Computing any complete graph kernel is at least as hard as the graph isomorphism problem.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

For any kernel K , computing the distance d_K is no harder than computing K , since

$$d_K(G_1, G_2)^2 = K(G_1, G_1) + K(G_2, G_2) - 2K(G_1, G_2)$$

uses only three kernel evaluations. If K is complete, then by definition $d_K(G_1, G_2) = 0$ if and only if $G_1 \simeq G_2$, so computing d_K decides graph isomorphism. Hence computing K is at least as hard as graph isomorphism. $\square$

Graph isomorphism is not known to be polynomial (nor known to be NP-complete), so a complete kernel is at best of uncertain complexity. It gets worse if we try to build the feature map from subgraph counts directly.

Definition (subgraph kernel)

Let $(\lambda_H)_{H \in \mathcal{X}}$ be nonnegative weights. For a graph G and a connected graph H , let $\Phi_H(G)$ be the number of subgraphs of G isomorphic to H ,

$$\Phi_H(G) = \#\{G' \text{ subgraph of } G : G' \simeq H\}.$$

The *subgraph kernel* is

$$K_{\text{subgraph}}(G_1, G_2) = \sum_{\substack{H \in \mathcal{X} \\ H \text{ connected}}} \lambda_H \Phi_H(G_1) \Phi_H(G_2).$$

Proposition (Gärtner et al., 2003)

Computing the subgraph kernel is NP-hard.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Let P_n denote the path graph on n vertices, and write e_H for the feature-space basis vector indexing subgraph pattern H . The subgraphs of P_n are themselves path graphs, and one checks that its feature vector is

$$\Phi(P_n) = n e_{P_1} + (n-1) e_{P_2} + \cdots + e_{P_n},$$

counting n single vertices, $n-1$ edges, and so on down to one copy of P_n itself. The vectors $\Phi(P_1), \dots, \Phi(P_n)$ are linearly independent (they form a triangular system in the basis $e_{P_1}, \dots, e_{P_n}$), so the pure indicator e_{P_n} can be written as a combination

$$e_{P_n} = \sum_{i=1}^n \alpha_i \Phi(P_i),$$

whose coefficients α_i are found in polynomial time by solving an $n \times n$ triangular system. Now let G be any graph on n vertices. It has a Hamiltonian path, a path visiting each vertex exactly once, if and only if $\Phi(G)^\top e_{P_n} > 0$, that is, if and only if

$$\Phi(G)^\top \left(\sum_{i=1}^n \alpha_i \Phi(P_i) \right) = \sum_{i=1}^n \alpha_i K_{\text{subgraph}}(G, P_i) > 0.$$

So n evaluations of the subgraph kernel decide the Hamiltonian-path problem, which is NP-complete. Hence the subgraph kernel is NP-hard. $\square$

As with the explicit case, restricting to paths does not save us.

Definition (path kernel)

The *path kernel* is the subgraph kernel restricted to path patterns,

$$K_{\text{path}}(G_1, G_2) = \sum_{H \in \mathcal{P}} \lambda_H \Phi_H(G_1) \Phi_H(G_2),$$

where $\mathcal{P} \subset \mathcal{X}$ is the set of path graphs.

Proposition (Gärtner et al., 2003)

Computing the path kernel is NP-hard.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Where this leaves us

Completeness is intractable; the subgraph kernel is intractable; and restricting to linear subgraphs, the path kernel, is intractable too. The way out is to relax what a "substructure" means. Instead of counting subgraphs, we count objects that are only *homomorphic* to subgraphs, most usefully *walks*: sequences of adjacent vertices that, unlike paths, are allowed to revisit vertices. Walks are cheap to enumerate implicitly, and, as we now show, the resulting kernel is polynomial.

25.1.3 Walk kernels

The whole point of a walk is that it drops the distinctness requirement of a path. This one relaxation is what turns an NP-hard count into a linear-algebra computation.

To see why distinctness is the villain, imagine building a substructure one vertex at a time. To extend a path we must check that the next vertex has not already been used, so the bookkeeping has to carry the entire set of vertices visited so far, and there are exponentially many such sets: that memory is exactly what makes path enumeration combinatorial. A walk carries no such memory. Its next step depends only on where it stands now, not on where it has been, so extending walks is a Markov process, and counting them reduces to multiplying by the adjacency matrix once per step. Trading the path's global no-repeat constraint for the walk's purely local rule is the whole trick, and it is the concrete meaning of the earlier remark about counting objects that are only homomorphic to subgraphs.

Definition (walk)

A *walk* of a graph (V, E) is a sequence $v_1, \dots, v_n \in V$ with $(v_i, v_{i+1}) \in E$ for $i = 1, \dots, n-1$. Vertices may repeat. Write $\mathcal{W}_n(G)$ for the set of walks on n vertices and $\mathcal{W}(G) = \bigcup_{n \geq 1} \mathcal{W}_n(G)$ for all walks.

Every path is a walk, but a walk that steps back and forth, or loops around a cycle, is not a path. There are infinitely many walks in any graph with a cycle, which is precisely why we can afford to weight them and sum implicitly.

It is worth being honest about what the relaxation costs. A walk is a strictly weaker witness of structure than a path: because it may revisit vertices, the same short neighborhood can generate many walks, and two graphs that no path could confuse may still share large families of walks. So the walk kernel is not complete, and by the earlier proposition it could not be, if it is to stay polynomial. That is the tradeoff in its sharpest form. Paths sit at the expressive end and are NP-hard; walks sit at the tractable end and are provably incomplete; and the whole design space between them, non-tottering walks, subtrees, bounded-length paths, is the search for a point that keeps enough of the path's discriminating power while inheriting the walk's linear-algebraic tractability. Every refinement in the next section is an attempt to climb back toward paths without paying their price.

Definition (walk kernel)

Let $\mathcal{S}_n$ be the set of all possible label sequences of walks on n vertices (including both vertex and edge labels), and $\mathcal{S} = \bigcup_{n \geq 1} \mathcal{S}_n$. For each graph G assign a weight $\lambda_G(w) \geq 0$ to every walk $w \in \mathcal{W}(G)$. Define the feature vector $\Phi(G) = (\Phi_s(G))_{s \in \mathcal{S}}$ by

$$\Phi_s(G) = \sum_{w \in \mathcal{W}(G)} \lambda_G(w) \mathbf{1}(s \text{ is the label sequence of } w).$$

The *walk kernel* is

$$K_{\text{walk}}(G_1, G_2) = \sum_{s \in \mathcal{S}} \Phi_s(G_1) \Phi_s(G_2).$$

The coordinate $\Phi_s(G)$ is the total weight of walks in G whose labels spell out s ; the kernel then matches the two graphs through their *common labeled walks*. Different choices of the weights λ_G give qualitatively different kernels.

Examples of walk kernels

- The **n -th-order walk kernel** takes $\lambda_G(w) = 1$ if w has length n and 0 otherwise. It compares two graphs through their common walks of length exactly n .
- The **random walk kernel** takes $\lambda_G(w) = P_G(w)$, the probability of the walk under a Markov random walk on G . Then

$$K(G_1, G_2) = P(\text{label}(W_1) = \text{label}(W_2)),$$

the probability that two *independent* random walks W_1 on G_1 and W_2 on G_2 produce the same label sequence (Kashima et al., 2003).

- The **geometric walk kernel** takes $\lambda_G(w) = \beta^{\text{length}(w)}$ for some $\beta > 0$, when the resulting series converges. Because walks of every length contribute, the feature space is infinite-dimensional (Gärtner et al., 2003).

Verification artifact. checks/example-ch10-example-24-3.json records the example source hash and verification scope.

Proposition (tractability)

The n -th-order, random, and geometric walk kernels can all be computed in polynomial time.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The proof, and the reason walks succeed where paths fail, is a single beautiful construction: the product graph.

25.1.4 The product graph

The idea is to package “a pair of walks with the same labels, one in each graph” as “a single walk in one auxiliary graph.” That auxiliary graph is the product graph.

Definition (product graph)

Let $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ be graphs with labeled vertices. The *product graph* $G = G_1 \times G_2$ has

$$V = \{(v_1, v_2) \in V_1 \times V_2 : v_1 \text{ and } v_2 \text{ have the same label}\},$$

$$E = \{((v_1, v_2), (v'_1, v'_2)) \in V \times V : (v_1, v'_1) \in E_1 \text{ and } (v_2, v'_2) \in E_2\}.$$

A vertex of $G_1 \times G_2$ is a pair of identically labeled vertices, one from each graph, and an edge exists only when both graphs can take the corresponding step simultaneously. Walking on the product graph therefore means walking on both graphs in lockstep along matching labels, which is exactly the coincidence the kernel sums over.

This is the conceptual heart of the whole construction, so it is worth saying plainly why it helps. The walk kernel is defined as a sum over the label alphabet $\mathcal{S}$, which is unbounded, and each term asks us to compare the walk populations of two separate graphs, an awkward object to compute. The product graph removes the alphabet

entirely: instead of enumerating label sequences and then matching them, we enumerate the pairs of matched walks directly, as single walks in one graph whose very existence certifies that the labels agree. A structure we could only describe as a coincidence between two graphs becomes an ordinary combinatorial quantity, the number (or weight) of walks, in a graph we can build. And counting walks in a graph is not hard: it is what powers of the adjacency matrix do, because $[A^n]_{ij}$ is by definition the number of length- n walks from i to j . The product graph is precisely the device that lets us pay for "same-label walk pairs" with a matrix power.

Lemma (walk bijection)

There is a bijection between

1. the pairs of walks $w_1 \in \mathcal{W}_n(G_1)$ and $w_2 \in \mathcal{W}_n(G_2)$ with the same label sequence, and
2. the walks $w \in \mathcal{W}_n(G_1 \times G_2)$ on the product graph.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Corollary (walk kernel on the product graph)

Assigning the product weight $\lambda_{G_1 \times G_2}(w) = \lambda_{G_1}(w_1) \lambda_{G_2}(w_2)$ to the walk w corresponding to the pair (w_1, w_2) ,

$$K_{\text{walk}}(G_1, G_2) = \sum_{s \in \mathcal{S}} \Phi_s(G_1) \Phi_s(G_2) = \sum_{\substack{(w_1, w_2) \in \\ \mathcal{W}(G_1) \times \mathcal{W}(G_2)}} \lambda_{G_1}(w_1) \lambda_{G_2}(w_2) \mathbf{1}(l(w_1) = l(w_2)) = \sum_{w \in \mathcal{W}(G_1 \times G_2)} \lambda_{G_1 \times G_2}(w).$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The kernel has become a single weighted sum over walks in one graph, and such sums are what adjacency matrices compute. Take first the n -th-order kernel, where $\lambda_{G_1 \times G_2}(w) = 1$ exactly on walks of length n .

Computation of the n -th-order walk kernel

Let A be the adjacency matrix of $G_1 \times G_2$. The number of walks of length n between vertices i and j is $[A^n]_{i,j}$, so

$$K_{n\text{-th order}}(G_1, G_2) = \sum_{w \in \mathcal{W}_n(G_1 \times G_2)} 1 = \sum_{i,j} [A^n]_{i,j} = \mathbf{1}^\top A^n \mathbf{1}.$$

This costs $O(n |V_1| |V_2| d_1 d_2)$, where d_i is the maximum degree of G_i , because the product graph has $O(|V_1| |V_2|)$ vertices and $O(|V_1| |V_2| d_1 d_2)$ edges, and each of the n matrix-vector products costs one pass over the edges.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

For the random and geometric kernels the weight of a walk factorizes into an initial weight and a product of transition weights: for $w = v_1 \dots v_n$,

$$\lambda_G(v_1 \dots v_n) = \lambda_i(v_1) \prod_{k=2}^n \lambda_t(v_{k-1}, v_k).$$

Collect the initial weights into a vector Λ_i (entries $\lambda_i(v)$) and the transition weights into a matrix Λ_t (entries $\lambda_t(v, v')$) on the product graph. Summing over all walk lengths gives a geometric series in Λ_t :

$$K_{\text{walk}}(G_1, G_2) = \sum_{n=1}^{\infty} \sum_{w \in \mathcal{W}_n(G_1 \times G_2)} \lambda_i(v_1) \prod_{k=2}^n \lambda_t(v_{k-1}, v_k) = \sum_{n=0}^{\infty} \Lambda_i \Lambda_t^n \mathbf{1} = \Lambda_i (I - \Lambda_t)^{-1} \mathbf{1},$$

valid whenever the series converges (spectral radius of Λ_t below 1, which is what "when it converges" meant for the geometric kernel). The identity $\sum_{n \geq 0} \Lambda_t^n = (I - \Lambda_t)^{-1}$ is the matrix version of the scalar geometric series $\sum_n x^n = 1/(1-x)$, and it is doing real work here: the left side is an explicit sum over walks of every length n , an infinite-dimensional feature space, while the right side is a single matrix inverse. So the same trick that made the feature space infinite, weighting walks of unbounded length, is what collapses the computation to one linear solve. The cost is dominated by inverting a matrix of size $|V_1||V_2|$, so $O(|V_1|^3|V_2|^3)$. Either way, walks give a polynomial-time graph kernel, the goal we could not reach with paths.

25.1.5 Extensions: labels, tottering, and subtrees

The basic walk kernel is a starting point that several refinements improve, all still reducible to the product graph.

Label enrichment (Mahé et al., 2004). A walk kernel only matches walks with identical label sequences, so enriching the vertex labels makes matching stricter and more informative. The *Morgan index* relabels each atom by an integer computed iteratively: at order 0 every atom has index 1; at each order the new index of a vertex is the sum of its neighbors' current indices. Order- k indices thus encode the size of a growing neighborhood, giving a compromise between structureless fingerprints and rigid structural keys. Enrichment is also a speed trick: richer labels mean fewer coincidences, hence a smaller product graph and faster computation. Many relabeling schemes are possible.

Non-tottering walks (Mahé et al., 2005). A walk that steps from a vertex to a neighbor and immediately back, $v_i = v_{i+2}$, is said to *totter*. Tottering walks inflate the kernel with contributions that seem irrelevant for chemistry and other applications, since they describe no real substructure. Forbidding them moves the walk kernel closer to the (intractable) path kernel; on trees the non-tottering walk kernel and the path kernel actually coincide. Computationally, a second-order Markov random walk suppresses tottering, and it can be rewritten as an ordinary first-order random walk on an augmented (directed) graph, so we recover a plain walk kernel on that augmented graph.

Subtree kernels (Ramon and Gärtner, 2003; Mahé and Vert, 2009). Walks are one-dimensional; branching substructures carry more information. Here a "subtree" means a *tree-like* pattern that may reuse vertices and edges of the graph, the branching analogue of a walk. Like the walk kernel, the subtree kernel amounts to a weighted count of subtrees in the product graph, and it obeys a clean recursion. If $T(v, n)$ is the weighted number of subtrees of depth n rooted at vertex v , then

$$T(v, n+1) = \sum_{R \subseteq N(v)} \prod_{v' \in R} \lambda_t(v, v') T(v', n),$$

where $N(v)$ is the set of neighbors of v and the sum runs over all subsets R of neighbors used to extend the tree by one level. Combining this with the non-tottering transformation as preprocessing yields the non-tottering subtree kernel.

There is a pretty link back to label enrichment. The Morgan index of order k at a vertex v equals the number of leaves of the full subtree pattern of order k rooted at v , so the enrichment scheme was implicitly counting subtrees all along.

The Weisfeiler-Lehman subtree kernel (Shervashidze and Borgwardt, 2009). A more systematic enrichment uses the Weisfeiler-Lehman relabeling (Weisfeiler and Lehman, 1968), a classical graph-isomorphism heuristic. Each iteration performs three steps: (1) for every vertex, gather the multiset of its neighbors' current labels and sort it; (2) compress each distinct sorted multiset into a fresh symbol; (3) relabel every vertex by that symbol. A compressed label after k iterations encodes the full subtree pattern of depth k around a vertex, so the histogram of labels over iterations $0, 1, \dots, k$ is a subtree feature vector, and the WL subtree kernel is the inner product of

these histograms. The features up to order k are computed in $O(|E|k)$, essentially linear in the graph. Like the Morgan index, WL relabeling can be layered on top of *any* kernel that reads categorical node labels to make it more expressive (Shervashidze et al., 2011).

Weisfeiler-Lehman kernels and graph neural networks

The WL subtree kernel does not use a single iteration but combines all of them: it concatenates the per-iteration label histograms, so the final kernel is a weighted sum of the base kernels at each depth,

$$K_{\text{WL}}(G_1, G_2) = \sum_{h=0}^k w_h \langle \phi^{(h)}(G_1), \phi^{(h)}(G_2) \rangle,$$

where $\phi^{(h)}$ is the compressed-label histogram after h iterations. The plain kernel weights every iteration equally ($w_h = 1$), but the w_h can be tuned to emphasize coarse or fine neighborhoods. The same neighborhood-aggregation loop drives message-passing graph neural networks, which update each node's feature vector from the multiset of its neighbors' vectors exactly as WL updates labels. Xu, Hu, Leskovec, and Jegelka (2019) make the comparison precise: a message-passing GNN is at most as powerful as the 1-WL test at distinguishing non-isomorphic graphs, and it reaches that ceiling only when its aggregation is injective on multisets. The WL subtree kernel is thus the kernel counterpart of a maximally expressive message-passing network.

25.1.6 The Weisfeiler-Leman hierarchy

The relabeling loop inside the WL subtree kernel is, read as a decision procedure, a classical test for graph isomorphism: the one-dimensional Weisfeiler-Leman test (Weisfeiler and Leman, 1968), also transliterated Weisfeiler-Lehman as in the kernel just built. It colors the vertices, refines the coloring until it settles, and compares two graphs through their color histograms. To say exactly how powerful the WL subtree kernel and its message-passing cousins are, we first have to say exactly what this coloring can and cannot see.

Algorithm (1-WL color refinement)

Input a graph $G = (V, E)$; an initial coloring c_0 , constant on unlabeled graphs and equal to the vertex labels otherwise.

Output for each round t , the histogram $\llbracket c_t(v) : v \in V \rrbracket$ of vertex colors, where double braces denote a multiset that keeps repetitions.

1. Give every vertex its initial color $c_0(v)$.
2. Refine all colors at once: $c_{t+1}(v) = \text{HASH}(c_t(v), \llbracket c_t(u) : u \sim v \rrbracket)$, assigning a fresh symbol to each distinct pair of a current color and a sorted neighbor multiset.
3. Repeat until the partition of V into color classes stops changing.
4. To test two graphs, run the loop with one shared HASH (equivalently, on their disjoint union) and call them non-isomorphic as soon as some round's two histograms differ.

Two facts fix the reach of this test, one encouraging and one sobering. It is *sound* in one direction: because the update reads only the graph structure, isomorphic graphs always receive identical colorings, so a difference between two histograms is a genuine certificate of non-isomorphism. It is *incomplete* in the other: identical colorings do not certify isomorphism. What 1-WL actually computes is the *coarsest equitable partition* of the vertices, the coarsest coloring in which every two vertices of one color have the same number of neighbors of each color, and any two graphs sharing these partition statistics are invisible to it whether or not they are truly the same.

The cleanest blind spot is the regular graphs. On a d -regular graph every vertex begins with one color and forever sees a multiset of d identical colors, so refinement never splits the single class: every d -regular graph on m vertices yields the same trivial coloring, and 1-WL cannot tell any two of them apart. The worked example makes this concrete and contrasts it with a pair that refinement does separate.

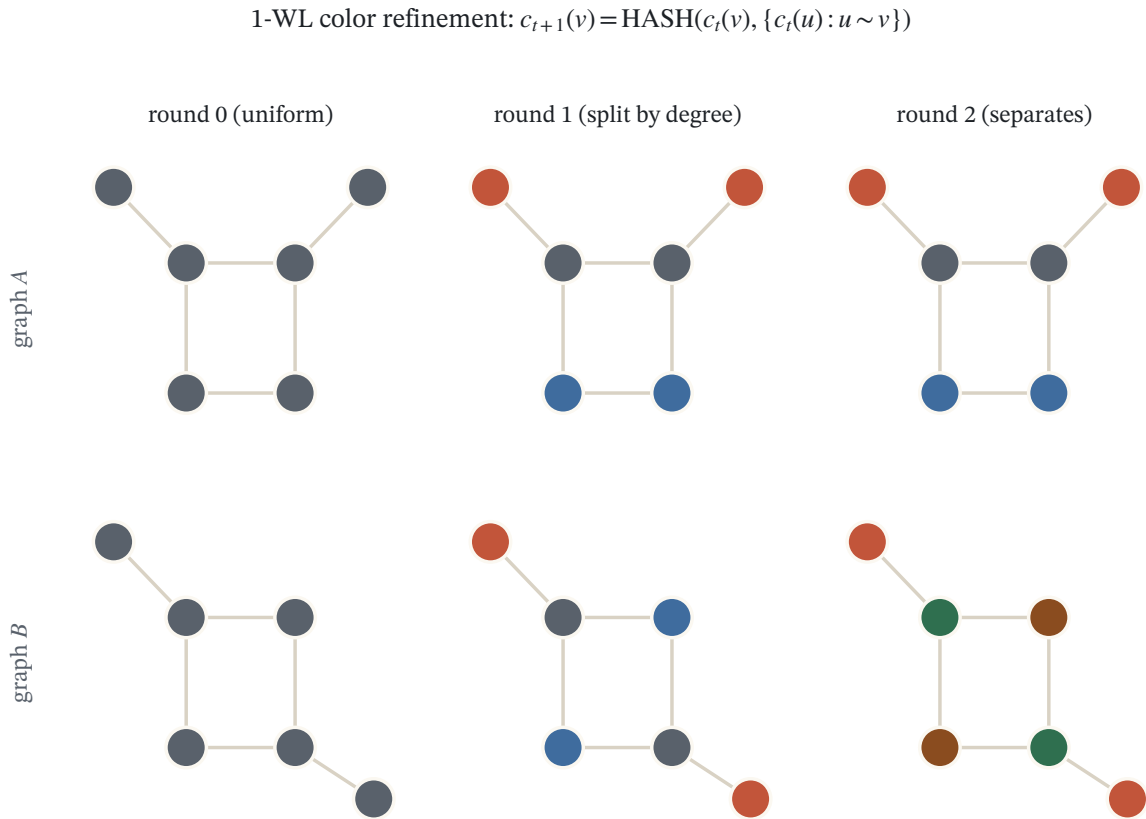


Figure 25.1: Weisfeiler–Lehman color refinement: at each round every node is relabeled by the multiset of its neighbors' colors, so structurally distinct nodes split into new classes. These refined colors are the features of the WL graph kernel.

Example (1-WL on four 6-vertex graphs)

Two 2-regular graphs 1-WL cannot separate: the 6-cycle C_6 (edges 0-1-2-3-4-5-0) and the disjoint pair of triangles $2C_3$ (edges 0-1-2-0 and 3-4-5-3); both have degree sequence (2, 2, 2, 2, 2, 2). And two it can separate: a 4-cycle 0-1-2-3-0 carrying two pendant vertices, attached at *adjacent* cycle vertices in A (pendant edges 0-4, 1-5) and at *opposite* ones in B (pendant edges 0-4, 2-5); both have degree sequence (3, 3, 2, 2, 1, 1). All colorings start from the uniform $c_0 \equiv 0$.

1. Refine the two regular graphs. Every vertex of C_6 and of $2C_3$ sees the neighbor multiset $\{0, 0\}$, so all six collapse to one new color, and the same happens at every later round. At rounds 0, 1, and 2 the histogram is therefore a single color class holding all six vertices, identical for the two graphs.
2. Take the WL inner product. With equal feature vectors over rounds 0 through 3, the WL subtree kernel gives $K(C_6, C_6) = K(2C_3, 2C_3) = K(C_6, 2C_3) = 144$, so the Hilbert distance is $d_K^2 = 144 + 144 - 2 \cdot 144 = 0$. The kernel, the 1-WL test, and, by the theorem below, every message-passing GNN are all blind to the difference, even though C_6 is connected and $2C_3$ is not.
3. Refine the pendant-cycle pair. Round 1 splits the vertices by degree into three classes of sizes (2, 2, 2), the same histogram for A and B . Round 2 refines the degree-3 color by its neighbors' round-1 colors: in A the two degree-3 hubs are adjacent, so each sees the other's hub color; in B they are not, so neither does. The two round-2 histograms now differ, sharing only the pendant color.
4. Take the inner product again. Now $K(A, A) = K(B, B) = 72$ while $K(A, B) = 52$, so $d_K^2 = 72 + 72 - 2 \cdot 52 = 40 > 0$, and the kernel separates A from B .

Reading. 1-WL sees a vertex only through the growing multiset of colors around it. It goes blind exactly when those multisets coincide, which they always do for regular graphs of a given degree, and it succeeds the instant a local asymmetry appears, here whether the two hubs are neighbors. The WL subtree kernel and a maximally expressive GNN inherit both the reach and the blind spot, down to the number.

Verification artifact. checks/example-ch10-example-24-4.json records the example source hash and verification scope.

The escape from the blind spot is to color *tuples* rather than single vertices. The k -dimensional Weisfeiler-Leman test (k -WL) attaches a color to every ordered k -tuple of vertices and refines it by aggregating over the tuples that differ from a given one in a single coordinate, so it registers joint relationships that no per-vertex coloring records. This buys a genuine ladder of tests: 1-WL and 2-WL have identical power, and for every $k \geq 2$ the $(k + 1)$ -WL test is strictly stronger than k -WL. No rung is complete, though. For every fixed k there are non-isomorphic graphs that k -WL fails to separate (Cai, Fürer, and Immerman, 1992), so no constant-order test decides isomorphism, exactly as the Gärtner et al. (2003) bound demanded of any tractable complete kernel. And the k -tuple coloring costs $O(n^k)$, so the ladder trades expressiveness against computation, the tension the whole chapter turns on.

25.1.7 Message-passing GNNs and the 1-WL ceiling

The remark that opened this thread noted that a message-passing graph neural network runs the same neighborhood-aggregation loop as WL relabeling, updating each vertex's feature vector from the multiset of its neighbors' vectors,

$$h_v^{(t+1)} = \text{UPDATE}\left(h_v^{(t)}, \text{AGG} \left\{ h_u^{(t)} : u \sim v \right\}\right),$$

and reading out a whole-graph vector by a permutation-invariant pooling of the final h_v . Because it is the same loop, it can be no sharper than the test.

Theorem (Xu et al., 2019; Morris et al., 2019)

For any choice of AGG, UPDATE, and readout, and any number of layers t , if 1-WL gives G_1 and G_2 the same color histogram after t rounds then the network gives them the same multiset of node features, hence the same graph

embedding. A message-passing GNN is therefore at most as powerful as 1-WL at distinguishing non-isomorphic graphs: no width, depth, or amount of training can separate a pair that 1-WL cannot.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The reason is an induction on layers: if two vertices carry equal features at round t and their neighbor multisets of features agree, the update returns equal features at round $t + 1$, which is exactly the WL invariant. So the network’s node features can only ever coarsen the WL colors, never refine them. In particular no message-passing GNN, whatever its parameters, tells the 6-cycle from the two triangles.

When is the bound tight? Exactly when the network never discards information that 1-WL keeps, that is, when AGG, UPDATE, and the readout are all *injective on multisets*. Xu, Hu, Leskovec, and Jegelka (2019) build such a network, the Graph Isomorphism Network (GIN), with a summation aggregator followed by a learned multilayer perceptron,

$$h_v^{(t+1)} = \text{MLP}^{(t)}\left((1 + \epsilon_t) h_v^{(t)} + \sum_{u \sim v} h_u^{(t)}\right).$$

Over a countable feature alphabet a sum is injective on multisets because it preserves multiplicities, whereas the popular alternatives are not: a mean forgets how many neighbors carry a label, unable to tell one neighbor colored a from two, and a max keeps only the set of distinct labels. Mean- and max-pooling networks therefore sit strictly below the 1-WL ceiling, while GIN reaches it.

Three cautions keep the equivalence honest. It bounds distinguishing power, the ability to separate non-isomorphic graphs, and says nothing on its own about how well a network generalizes or approximates a target once the graphs are separable. It assumes a countable feature alphabet for exact injectivity of the sum; with real-valued attributes injectivity holds only up to numerical resolution. And the ceiling it names is 1-WL, which is blind to the regular graphs of the worked example, so GIN inherits that blind spot exactly, since reaching the ceiling is not climbing above it. To climb, one must aggregate over tuples, just as k -WL does. The higher-order GNNs of Morris et al. (2019) pass messages between k -vertex subsets and match the power of k -WL, at the $O(n^k)$ cost the tuple coloring already warned of. What one gains is precisely the discrimination of the matching WL rung, paid for at the matching price, and the relationship between these networks and the kernels here is the subject of the deep-learning chapter.

25.1.8 Kernelized 1-WL: from histograms to optimal transport

The WL subtree kernel is the explicit-feature face of this same test. Its feature map $\phi^{(h)}(G)$ is the histogram of 1-WL colors after h rounds, and $K_{\text{WL}} = \sum_h \langle \phi^{(h)}(G_1), \phi^{(h)}(G_2) \rangle$ is a plain inner product of those histograms. So the kernel sees neither more nor less than the test: two graphs have equal WL feature vectors if and only if 1-WL fails to separate them, and then $K_{\text{WL}}(G_1, G_1) = K_{\text{WL}}(G_1, G_2) = K_{\text{WL}}(G_2, G_2)$ and the Hilbert distance collapses to zero. The 6-cycle and two-triangle graph are the smallest witness, at distance 0; the pendant-cycle pair, at squared distance 40, shows the kernel working when the test does.

The per-round count vector $\phi^{(h)}$ has a direct reading in the network picture. It is the multiset of depth- h subtree patterns rooted at the vertices, exactly the information a maximally expressive GIN stores in its layer- h node embeddings before the readout sums them. The WL subtree kernel and a top-power message-passing GNN thus record the same per-depth statistics and share the 1-WL ceiling; they part only in what they do with those statistics. The kernel applies a fixed, exact injective hash and equal per-depth weights and compares by a linear inner product; the network learns continuous, differentiable approximations to the hash and a task-trained readout. One is closed-form and data-independent, the other adaptive and data-driven, a contrast the deep-learning chapter develops in full.

The histogram inner product carries a built-in limitation: it compares two graphs only through *coincidences of identical* substructures and ignores how near two non-identical ones are. Two families of kernels widen the

view. The shortest-path kernel (Borgwardt and Kriegel, 2005) indexes a graph by the multiset of its shortest-path lengths keyed by endpoint labels, injecting the global distance information WL’s purely local neighborhoods never reach; it stays positive definite because it is still a histogram inner product, only over a richer feature. Optimal-assignment kernels go the other way: instead of summing a base kernel over *all* pairs of parts, as a convolution kernel does, they match each part of one graph to its single best counterpart in the other through an optimal bipartite assignment and sum only the matched similarities. This is often more discriminative and more interpretable, but the maximum over matchings breaks positive definiteness in general, so a naive assignment similarity is not a valid kernel; it is p.d. only for special hierarchically structured base kernels, whose constructions Kriege, Johansson, and Morris (2020) survey.

Carrying the matching idea into the WL embedding gives the last kernel of the chapter. The Wasserstein Weisfeiler-Leman kernel (Togninalli et al., 2019) embeds each vertex by its whole sequence of WL colors over rounds 0 through H , turning a graph into a cloud of node vectors, and compares two graphs by the Wasserstein (optimal-transport) distance between their clouds under a ground metric on the WL embeddings, rather than by an inner product of pooled histograms. Because a transport plan is a soft assignment, near-but-unequal subtrees now contribute what the histogram discarded, lifting the kernel off the exact-coincidence floor even while its embeddings remain 1-WL colors. The price is a familiar one: a Wasserstein distance is not in general the metric of a Hilbert space, so the Laplacian-type kernel $\exp(-\lambda d_W)$ it induces need not be positive definite. Togninalli et al. (2019) show it is p.d. for categorical WL labels and otherwise treat it as an indefinite kernel, the optimal-transport-meets-kernels theme that the frontier chapter pursues. Read as an averaging over label sequences, this whole WL-and-transport family is close kin to the marginalized graph kernels of the generative-kernels chapter.

25.1.9 The convolution kernel: one framework behind them all

Step back from the graph for a moment. The walk kernel, the subtree kernel, and the string and subsequence kernels of an earlier chapter all share a single shape. Each picks a way to break an object into parts, compares a part of one object with a part of the other through a base kernel, and sums the products of those comparisons over all admissible ways of decomposing the two objects. Haussler (1999) isolated this shape and named it the *convolution kernel*, or *R-kernel*; Shawe-Taylor and Cristianini (2004) develop it as the common framework for kernels on structured data. Reading our graph kernels as instances of it explains why they were built the way they were, and shows exactly where the design freedom lives.

Call a datatype *structured* when each object can be decomposed into a tuple of smaller parts. A single object usually admits many decompositions, and each part carries its own base kernel, so the bookkeeping needs both the parts and the kernels that compare them.

Definition (decomposition structure and convolution kernel)

A *decomposition structure* R on a set $\mathcal{X}$ associates to each object x a set $R^{-1}(x)$ of admissible decompositions, each a tuple $\vec{x} = ((x_1, \kappa_1), \dots, (x_d, \kappa_d))$ of parts x_i with attached base kernels κ_i ; its *type* $T(\vec{x}) = (\kappa_1, \dots, \kappa_d)$ records the list of base kernels used. The associated *convolution kernel* (or R-kernel) is

$$K_R(x, z) = \sum_{\vec{x} \in R^{-1}(x)} \sum_{\vec{z} \in R^{-1}(z)} \mathbf{1}(T(\vec{x}) = T(\vec{z})) \prod_{i=1}^{|\vec{x}|} \kappa_i(x_i, z_i),$$

where the indicator keeps only pairs of decompositions of matching type, so that comparable parts alone are ever multiplied together.

The abstraction earns its keep because it is closed: whatever decomposition we invent, provided the base kernels are valid, the result is a valid kernel.

Proposition (Haussler, 1999)

For any decomposition structure R whose base kernels are positive semi-definite, the convolution kernel K_R is positive semi-definite. It is the inner product $\langle \Phi(x), \Phi(z) \rangle$ of the feature map that sends each object to the sum, over its decompositions, of the tensor products of its part-features.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Our graph kernels are this very construction with particular choices of "part."

Walk and subtree kernels as convolution kernels

- The **walk kernel** decomposes a graph into its walks and takes the base kernel to be 1 when two walks carry the same label sequence and 0 otherwise. Summing the products over all pairs of walks is exactly the middle expression of the walk-kernel corollary, $\sum_{(w_1, w_2)} \lambda_{G_1}(w_1) \lambda_{G_2}(w_2) \mathbf{1}(l(w_1) = l(w_2))$. The product graph is then nothing but the efficient machinery for evaluating this one R-kernel.
- The **subtree kernel** is the same construction with subtree patterns in place of walks, and the **path kernel** with paths, so the whole walk-path-subtree family is a single convolution kernel seen through different decompositions.

Verification artifact. checks/example-ch10-example-24-5.json records the example source hash and verification scope.

The reach is wider still: the ANOVA and Gaussian kernels of an earlier chapter are convolution kernels too, the first decomposing a vector into its coordinate subsets and the second into its individual coordinates, each with per-coordinate base kernels (Shawe-Taylor and Cristianini, 2004). So the choice that defines a structured kernel is always the same pair of decisions: which parts to decompose into, and how to match a part with a part. Expressiveness, computation, and dimension, the three competing demands from the opening of the chapter, are all settled by that pair.

The framework is more than a re-description; it comes with concrete dynamic programs whenever the parts nest recursively. The clearest example is a kernel between objects that *are* trees, such as parse trees of sentences, XML documents, or fragment hierarchies of molecules.

Kernels between trees, by recursion

Index the features by subtrees. The *co-rooted subtree kernel* counts shared subtrees that contain the root; writing $c_i(T)$ for the subtree hanging from the i -th child of the root of T , it obeys the recursion

$$\kappa_r(T_1, T_2) = \prod_{i=1}^d (\kappa_r(c_i(T_1), c_i(T_2)) + 1)$$

when the two roots have the same out-degree d (and, for labeled trees, the same label), and $\kappa_r(T_1, T_2) = 0$ otherwise. The more permissive *all-subtree kernel*, whose features are all subtrees rather than only root-anchored ones, is recovered by summing the co-rooted kernel over every pair of nodes,

$$\kappa(T_1, T_2) = \sum_{v_1 \in T_1} \sum_{v_2 \in T_2} \kappa_r(\tau(v_1), \tau(v_2)),$$

where $\tau(v)$ is the complete subtree rooted at v ; a table indexed by pairs of nodes computes it bottom-up in $O(|T_1| |T_2| d_{\max}^2)$, with $d_{\max}$ the largest out-degree (Shawe-Taylor and Cristianini, 2004). Both kernels are convolution kernels, the decomposition sending a tree to its co-rooted subtrees or to all of its subtrees; the parse-tree version drives kernel methods for natural language (Collins and Duffy, 2002). This is the branching, tree-to-tree analogue of the walk-versus-subtree distinction we drew inside a single graph.

Verification artifact. checks/example-ch10-example-24-6.json records the example source hash and verification scope.

Kernels as prior knowledge

The convolution kernel is a disciplined way to build one kind of prior into a kernel between graphs: a declaration of which substructures are allowed to count as evidence of similarity. Schölkopf and Smola (2002) make the complementary point from the other side, that choosing a kernel is at once choosing a similarity measure, a feature space, and a regularizer, so that there is no free lunch in kernel choice. Part B takes exactly the regularizer view: rather than naming the parts, it fixes the prior directly as smoothness on a single fixed graph, and reads the kernel off from it.

25.1.10 Applications and summary

These kernels work well in practice. On the MUTAG dataset (188 aromatic and heteroaromatic compounds, 125 mutagenic and 63 not, assayed in *Salmonella typhimurium*), a 2D walk kernel reaches 91.2% ten-fold cross-validation accuracy against 81.4% for the earlier Progol rule learner (Mahé et al., 2005). Across a screen of inhibitors for 60 cancer cell lines, subtree kernels generally edge out walk kernels in AUC, confirming that branching structure helps. A broad comparison of graph feature methods on chemo- and bioinformatics classification (Shervashidze et al., 2011) is summarized below (ten-fold cross-validation accuracy, standard deviation in parentheses).

Method	NCI1	NCI109	ENZYMES
WL subtree	82.19 (± 0.18)	82.46 (± 0.24)	52.22 (± 1.26)
WL shortest path	84.55 (± 0.36)	83.53 (± 0.30)	59.05 (± 1.05)
Ramon & Gärtner	61.86 (± 0.27)	61.67 (± 0.21)	13.35 (± 0.87)
Geometric p -walk	58.66 (± 0.28)	58.36 (± 0.94)	27.67 (± 0.95)
Geometric walk	64.34 (± 0.27)	63.51 (± 0.18)	21.68 (± 0.94)
Graphlet count	66.00 (± 0.07)	66.59 (± 0.08)	32.70 (± 1.20)
Shortest path	73.47 (± 0.11)	73.07 (± 0.11)	41.68 (± 1.79)

The Weisfeiler-Lehman variants dominate, and shortest-path features are strong; pure geometric walk kernels are the weakest, which is the empirical face of the walk-vs-subtree expressiveness point. On the COREL14 image benchmark (1400 natural images in 14 classes), Harchaoui and Bach (2007) find that walk and subtree kernels, and especially a weighted subtree kernel and a combined kernel, reduce test error relative to a plain histogram kernel.

Summary: kernels between graphs

Kernels do not repeal the NP-hardness of subgraph and path patterns. What they buy is the ability to work with *approximate* substructures, walks and subtrees, in an infinite-dimensional feature space, at polynomial cost, through the product-graph trick. The price is interpretability: after learning in the implicit feature space it is hard to recover which patterns drove the decision.

25.2 Part B: Kernels on a graph

Now the graph is fixed and singular, and the data points are its nodes. We want a kernel $K(x, x')$ between two *vertices* of one graph. Such data arise for several reasons: by definition (an interaction network, the web, a social network), by discretizing or sampling a continuous domain, or simply by convenience when all we are given is a similarity between objects. A running example is predicting protein function from high-throughput protein-protein interaction data, where the graph is the interaction network and each node is a protein.

Since the vertex set $\mathcal{X} = \{x_1, \dots, x_m\}$ is finite, a kernel is just an $m \times m$ symmetric positive semi-definite matrix K ; every such matrix is a valid kernel. The design question is how to translate the *topology* of the graph into that matrix. Three strategies suggest themselves, and they turn out to converge:

- **Geometric.** Make $K_{i,j}$ large when x_i and x_j are close on the graph.
- **Functional.** Make the RKHS norm $\|f\|_K$ small when f is smooth on the graph, that is, varies little across edges.
- **Discrete/continuous link.** Look for the graph analogue of the continuous Gaussian kernel, ideally as a limit of fine discretizations.

25.2.1 Graph distance and conditionally p.d. kernels

The most direct geometric route is to build a kernel from the graph distance. To see whether that is even possible we need the notion of conditional positive definiteness, which is the exact condition for a distance to come from a Hilbert-space embedding.

Hilbert distance and c.p.d.

Any p.d. kernel is an inner product in a Hilbert space, $K(x, x') = \langle \Phi(x), \Phi(x') \rangle_{\mathcal{H}}$, and induces the Hilbert distance

$$d_K(x, x')^2 = K(x, x) + K(x', x') - 2K(x, x').$$

The negative squared distance $-d_K^2$ is *conditionally positive definite* (c.p.d.): in particular $\exp(-t d_K(x, x')^2)$ is p.d. for every $t > 0$.

The Euclidean prototype

On $\mathcal{X} = \mathbb{R}^n$ with $K(x, x') = x^\top x'$, the Hilbert distance is the Euclidean distance,

$$d_K(x, x')^2 = x^\top x + x'^\top x' - 2x^\top x' = \|x - x'\|^2,$$

and $-\|x - x'\|^2$ is c.p.d., which is exactly why the Gaussian $\exp(-t\|x - x'\|^2)$ is a valid kernel for all $t > 0$.

Verification artifact. checks/example-ch10-example-24-7.json records the example source hash and verification scope.

Transporting this to a graph, the natural distance between vertices is the *graph distance* $d_G(x, x')$, the length of the shortest path between them. We say $G = (V, E)$ embeds exactly in a Hilbert space if $-d_G$ is c.p.d., which would give us $\exp(-t d_G(x, x'))$ as a valid kernel for all $t > 0$. Unfortunately this fails in general.

Lemma (embeddability)

In general, graphs cannot be embedded exactly in a Hilbert space, i.e. $-d_G$ is not c.p.d. In some special cases exact embeddings do exist; in particular, *trees* embed exactly, and *closed chains* (cycles) embed exactly.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

A non-c.p.d. graph distance

For a small five-vertex graph one finds the distance matrix

$$d_G = \begin{pmatrix} 0 & 1 & 1 & 1 & 2 \\ 1 & 0 & 2 & 2 & 1 \\ 1 & 2 & 0 & 2 & 1 \\ 1 & 2 & 2 & 0 & 1 \\ 2 & 1 & 1 & 1 & 0 \end{pmatrix},$$

and the matrix with entries $\exp(-0.2 d_G(i, j))$ has smallest eigenvalue $\lambda_{\min} \approx -0.028 < 0$. Being indefinite, it is not a valid kernel, so this distance does *not* come from a Hilbert embedding.

Verification artifact. checks/example-ch10-example-24-8.json records the example source hash and verification scope.

Proof that tree distances are c.p.d.

Let $G = (V, E)$ be a tree and fix a root $x_0 \in V$. Represent each vertex x by the vector $\Phi(x) \in \mathbb{R}^{|E|}$ whose i -th coordinate is 1 if the i -th edge lies on the unique path from x to x_0 , and 0 otherwise. For two vertices, the edges on their two root-paths that are *not* shared are exactly the edges of the path between them (the shared prefix cancels), so

$$d_G(x, x')^2 = \|\Phi(x) - \Phi(x')\|^2.$$

Hence the graph distance is a squared Euclidean distance, $-d_G$ is c.p.d., and $\exp(-t d_G(x, x'))$ is p.d. for all $t > 0$. $\square$

Proof that closed-chain distances are c.p.d.

Let $G = C_m$ be the cycle on m vertices, so the graph distance is $d_G(i, j) = \min(|i - j|, m - |i - j|)$. Because the cycle is vertex-transitive, its distance matrix is *circulant*, hence diagonalized by the discrete Fourier basis, with eigenvalues

$$\mu_\nu = \sum_{k=0}^{m-1} d_G(0, k) \cos \frac{2\pi\nu k}{m}, \quad \nu = 0, \dots, m-1.$$

The constant mode $\nu = 0$ gives $\mu_0 = \sum_k d_G(0, k) > 0$, while one checks that every other eigenvalue satisfies $\mu_\nu \leq 0$. So the distance matrix is negative semi-definite on the space orthogonal to $\mathbf{1}$, which is exactly the statement that $-d_G$ is c.p.d. This holds for *every* m , even or odd, so no parity restriction is needed. When $m = 2p$ is even the embedding is transparent: number the $2p$ edges in order around the cycle and label them $e_1, \dots, e_p, -e_1, \dots, -e_p$ in $\mathbb{R}^p$, so opposite edges carry opposite labels; letting $\Phi(v)$ be the sum of the labels on the shortest path from a fixed root to v , one checks that $\|\Phi(v) - \Phi(v')\|^2 = d_G(v, v')$, an explicit isometric witness. $\square$

The Cayley graph of S_4

Let S_n be the symmetric group of permutations of n items, and form the Cayley graph G by connecting two permutations when they differ by one adjacent transposition. Here the graph distance (the number of adjacent transpositions needed, the Kendall-tau distance) can be computed in $O(n \log n)$, and it turns out to be c.p.d., so $\exp(-t d_G)$ is a valid kernel on permutations (Jiao and Vert, 2017).

Verification artifact. checks/example-ch10-example-24-9.json records the example source hash and verification scope.

Summary on graph distance

Some graph distances are c.p.d. and some are not, and there is a large mathematical literature on embedding graphs approximately. But the graph distance has two flaws for machine learning: it is only sometimes a valid kernel, and it is very sensitive to noise in the edges (a single spurious edge can create a shortcut and collapse many distances). We want a construction that yields a p.d. kernel for *every* graph and degrades gracefully under edge noise. That is the functional route, through the Laplacian.

25.2.2 Construction by regularization: the graph Laplacian

Rather than starting from a distance and hoping it is c.p.d., we start from the *functional* requirement, that smooth functions should have small norm, and let it define the RKHS. Designing a kernel is equivalent to defining an RKHS, and on a graph there is an intuitive notion of smoothness: a function is smooth if it changes little across edges. So the plan is to fix a smoothness functional on functions $f : \mathcal{X} \rightarrow \mathbb{R}$, show it is a squared RKHS norm, and read off the kernel.

Set up notation. The vertex set $\mathcal{X} = \{x_1, \dots, x_m\}$ is finite; write $x \sim x'$ when there is an edge between x and x' ; assume no self-loops and a single connected component. The adjacency matrix $A \in \mathbb{R}^{m \times m}$ has $A_{i,j} = 1$ if $i \sim j$ and 0 otherwise, and D is the diagonal degree matrix with $D_{i,i} = \sum_j A_{i,j}$, the number of neighbors of x_i .

What should "smooth on the graph" mean concretely? A function f assigns a number to each vertex, and the only structure we have to judge it by is the edge set, so the natural verdict is that f is smooth when it changes little along edges. The simplest functional expressing this is the sum of squared differences across edges, $\sum_{i \sim j} (f(x_i) - f(x_j))^2$, which is zero for a constant function and grows as f disagrees with itself across neighbors. This is a quadratic form in f , so it must be $f^\top M f$ for some symmetric matrix M , and the point of the coming lemma is that this matrix has a clean closed form: it is $D - A$. That matrix earns its own name.

Definition (graph Laplacian)

The *Laplacian* of the graph is $L = D - A$.

The Laplacian is the discrete object that measures roughness, as the following lemma makes precise.

Lemma (properties of L)

Let $L = D - A$ be the Laplacian of a connected graph. Then:

1. For any $f : \mathcal{X} \rightarrow \mathbb{R}$, the roughness functional

$$\Omega(f) := \sum_{i \sim j} (f(x_i) - f(x_j))^2 = f^\top L f,$$

where the sum runs over edges.

2. L is symmetric positive semi-definite.
3. 0 is an eigenvalue of multiplicity 1, with eigenvector the constant vector $\mathbf{1} = (1, \dots, 1)$.
4. The image of L is $\text{Im}(L) = \{f \in \mathbb{R}^m : \sum_{i=1}^m f_i = 0\}$, the mean-zero functions.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

For the first claim, expand

$$\Omega(f) = \sum_{i \sim j} (f(x_i) - f(x_j))^2 = \sum_{i \sim j} (f(x_i)^2 + f(x_j)^2 - 2f(x_i)f(x_j)).$$

Each vertex i contributes $f(x_i)^2$ once per incident edge, that is, $D_{i,i}$ times, so the squared terms sum to $\sum_i D_{i,i} f(x_i)^2 = f^\top D f$; the cross terms give $-2 \sum_{i \sim j} f(x_i)f(x_j) = -f^\top A f$. Hence $\Omega(f) = f^\top D f - f^\top A f = f^\top L f$. Symmetry of L is immediate from symmetry of A and D . Since $f^\top L f = \Omega(f) \geq 0$ for all f , the (real) eigenvalues of L are nonnegative, so L is positive semi-definite. Next, f is an eigenvector for eigenvalue 0 iff $f^\top L f = 0$ iff $\sum_{i \sim j} (f(x_i) - f(x_j))^2 = 0$ iff $f(x_i) = f(x_j)$ whenever $i \sim j$; because the graph is connected this forces f constant, so the kernel of L is one-dimensional, spanned by $\mathbf{1}$. Finally, L symmetric implies $\text{Im}(L)$ is the orthogonal complement of $\text{Ker}(L) = \text{span}(\mathbf{1})$, namely the functions orthogonal to $\mathbf{1}$, i.e. the mean-zero functions. $\square$

So $\Omega(f) = f^\top L f$ is exactly the smoothness penalty we wanted, and it is a positive semi-definite quadratic form. The only obstruction to using it as a squared norm is its one-dimensional kernel: constants are penalized 0. We therefore work on the mean-zero functions, where Ω is strictly positive.

The link between a penalty and a kernel is worth stating as a principle, since it governs this construction and the diffusion kernel to come. Whenever an RKHS norm is a quadratic form, $\|f\|^2 = f^\top M f$ for a positive definite M , the reproducing kernel is the *inverse* of that form, $K = M^{-1}$: penalizing directions where M is large is the same as making the kernel small there, so the kernel and the penalty are inverse views of each other. Here $M = L$, which is only positive semi-definite, so its inverse must be taken in the generalized sense on the subspace where it is invertible. That generalized inverse is the pseudo-inverse L^* , and it is exactly the kernel the theorem produces.

Theorem (the Laplacian kernel)

The space $\mathcal{H} = \{f \in \mathbb{R}^m : \sum_{i=1}^m f_i = 0\}$, endowed with the norm $\Omega(f) = \sum_{i \sim j} (f(x_i) - f(x_j))^2$, is an RKHS whose reproducing kernel is L^* , the Moore-Penrose pseudo-inverse of the graph Laplacian.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Recall: the pseudo-inverse of L

The pseudo-inverse L^* is the linear map that is 0 on $\text{Ker}(L)$ and equal to L^{-1} on $\text{Im}(L)$. Concretely, from the eigendecomposition $L = \sum_{i=1}^m \lambda_i u_i u_i^\top$,

$$L^* = \sum_{\lambda_i \neq 0} \frac{1}{\lambda_i} u_i u_i^\top.$$

It satisfies $L^* L = L L^* = \Pi_{\mathcal{H}}$, the orthogonal projection onto $\text{Im}(L) = \mathcal{H}$.

Proof of the theorem

Restricted to $\mathcal{H} = \text{Im}(L)$, the symmetric bilinear form $\langle f, g \rangle = f^\top L g$ is positive definite, because L is positive semi-definite and its only null direction, $\mathbf{1}$, has been removed. It is therefore a genuine inner product, making $\mathcal{H}$ a (finite-dimensional, hence complete) Hilbert space, with norm

$$\|f\|^2 = \langle f, f \rangle = f^\top L f = \Omega(f).$$

To identify the reproducing kernel we must exhibit K with $K_x \in \mathcal{H}$ for every x and $\langle f, K_x \rangle = f(x)$ for every $f \in \mathcal{H}$ and x . Take $K = L^*$. Since $\text{Ker}(L^*) = \text{Ker}(L) = \text{span}(\mathbf{1})$, we have $K\mathbf{1} = 0$, so every column of K is mean-zero, i.e. $K_x \in \mathcal{H}$. For the reproducing property, let $f \in \mathcal{H}$ and set $g_i = \langle K_{(i,\cdot)}, f \rangle$. In matrix form, using the inner product $\langle \cdot, \cdot \rangle = (\cdot)^\top L(\cdot)$,

$$g = K L f = L^* L f = \Pi_{\mathcal{H}}(f) = f,$$

because f already lies in $\mathcal{H}$. Thus $\langle K_{(i,\cdot)}, f \rangle = f_i = f(x_i)$, and $K = L^*$ is the reproducing kernel of $\mathcal{H}$. $\square$

This is our first genuine kernel on a graph: valid for *any* connected graph, and by construction it makes smooth functions cheap. Two limits make clear why L deserves the name "Laplacian" and why Ω is the right penalty.

Interpretation of L : a discrete second derivative

On a one-dimensional chain with spacing dx , the continuous Laplacian $\Delta f = f''$ is approximated by a symmetric second difference:

$$\Delta f(x) \approx \frac{f'(x + dx/2) - f'(x - dx/2)}{dx} \approx \frac{f_{i+1} - f_i - (f_i - f_{i-1})}{dx^2} = \frac{f_{i-1} + f_{i+1} - 2f_i}{dx^2} = -\frac{(Lf)_i}{dx^2}.$$

So the graph Laplacian is (minus) a discrete Laplace operator: $\Delta \approx -L/dx^2$.

Interpretation of the regularizer: a discrete Dirichlet energy

For $f : [0, 1] \rightarrow \mathbb{R}$ sampled at $x_i = i/m$ along a chain,

$$\Omega(f) = \sum_{i=1}^m \left(f\left(\frac{i+1}{m}\right) - f\left(\frac{i}{m}\right) \right)^2 \approx \sum_{i=1}^m \frac{1}{m^2} f'\left(\frac{i}{m}\right)^2 = \frac{1}{m} \cdot \frac{1}{m} \sum_{i=1}^m f'\left(\frac{i}{m}\right)^2 \approx \frac{1}{m} \int_0^1 f'(t)^2 dt.$$

Penalizing $f^\top L f$ on the graph is the discrete analogue of penalizing the Dirichlet energy $\int (f')^2$, the standard smoothness functional on the continuum.

25.2.3 The diffusion kernel

The Laplacian kernel L^* is one member of a richer family, and to find the family we chase the third strategy from the start: the graph analogue of the Gaussian. The continuous normalized Gaussian on $\mathbb{R}^d$,

$$K_t(x, x') = \frac{1}{(4\pi t)^{d/2}} \exp\left(-\frac{\|x - x'\|^2}{4t}\right),$$

cannot be moved to a graph by simply substituting the shortest-path distance for $\|x - x'\|$: as we saw, that need not even be p.d. Instead we characterize the Gaussian by a property that *does* transfer, namely that it solves a differential equation built from the Laplacian.

Lemma (the Gaussian is the heat kernel)

For any $x_0 \in \mathbb{R}^d$, the function $K_{x_0}(x, t) = K_t(x_0, x)$ solves the *diffusion equation*

$$\frac{\partial}{\partial t} K_{x_0}(x, t) = \Delta K_{x_0}(x, t), \quad K_{x_0}(x, 0) = \delta_{x_0}(x),$$

by direct computation. The Gaussian is the heat spreading out from a point source at x_0 .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

This description transfers verbatim. Replace the continuous Laplacian Δ by $-L$ (recall $\Delta \approx -L/dx^2$), so that for a finite-dimensional $f_t \in \mathbb{R}^m$ the discrete diffusion equation reads

$$\frac{\partial}{\partial t} f_t = -L f_t, \quad \text{with solution} \quad f_t = e^{-tL} f_0,$$

where the matrix exponential is defined by its series

$$e^{-tL} = I - tL + \frac{t^2}{2!} L^2 - \frac{t^3}{3!} L^3 + \dots$$

The intuition behind the substitution $\Delta \rightarrow -L$ is worth pausing on, because it is what makes the diffusion kernel more than a formal analogy. Applying $-L$ to f replaces each vertex value by the average pull of its neighbors: $(-L f)_i = \sum_{j \sim i} (f_j - f_i)$ is positive where a vertex sits below its neighbors and negative where it sits above, so the update $\dot{f} = -L f$ pushes every value toward the local average, which is precisely how heat flows from hot spots to cold ones. Starting from a unit spike at one vertex, the solution $e^{-tL} \delta_{x_0}$ is the graph's heat spreading outward from x_0 , and its value at another vertex measures how much of that heat has reached there by time t . That is a

sensible notion of similarity between two vertices: they are close if heat injected at one arrives quickly at the other, a quantity that pools over all paths between them rather than trusting a single shortest path, which is exactly the robustness to edge noise the graph distance lacked.

Just as the Gaussian was the operator carrying the initial condition to the solution at time t , the operator e^{-tL} carries f_0 to f_t , and it is a valid kernel.

Diffusion kernel (Kondor and Lafferty, 2002)

The *diffusion kernel* (or *heat kernel*) on the graph is

$$K = e^{-tL}.$$

It is symmetric positive semi-definite: writing $L = \sum_{i=1}^m \lambda_i u_i u_i^\top$ with $\lambda_i \geq 0$,

$$K = e^{-tL} = \sum_{i=1}^m e^{-t\lambda_i} u_i u_i^\top,$$

whose eigenvalues $e^{-t\lambda_i} > 0$ are strictly positive. The parameter t sets the diffusion time: larger t spreads similarity further across the graph.

The two extremes fix intuition for t . At $t = 0$ we have $K = e^0 = I$: no time has passed, so each vertex is similar only to itself and the kernel sees no graph structure at all. As $t \rightarrow \infty$ the heat equilibrates, $e^{-tL} \rightarrow \frac{1}{m} \mathbf{1}\mathbf{1}^\top$ on a connected graph (every eigenvalue $\lambda_i > 0$ is killed by $e^{-t\lambda_i} \rightarrow 0$, leaving only the constant mode $\lambda_1 = 0$), so every pair of vertices looks maximally and equally similar and the graph structure is again washed out. Useful values of t live in between, and t plays exactly the role that the bandwidth plays for the Gaussian on $\mathbb{R}^d$: it is the length scale over which the kernel considers two vertices related, and it is chosen by cross-validation like any other kernel hyperparameter.

Complete graph

On the complete graph on m vertices the diffusion kernel is

$$K_{i,j} = \begin{cases} \frac{1 + (m-1)e^{-tm}}{m} & i = j, \\ \frac{1 - e^{-tm}}{m} & i \neq j. \end{cases}$$

Verification artifact. checks/example-ch10-example-24-10.json records the example source hash and verification scope.

Closed chain

On the cycle of length m , using the discrete Fourier eigenvectors,

$$K_{i,j} = \frac{1}{m} \sum_{\nu=0}^{m-1} \exp\left(-2t\left(1 - \cos \frac{2\pi\nu}{m}\right)\right) \cos \frac{2\pi\nu(i-j)}{m}.$$

Verification artifact. checks/example-ch10-example-24-11.json records the example source hash and verification scope.

25.2.4 Harmonic analysis on graphs

The eigenexpansions above are not incidental: they say that both the Laplacian kernel and the diffusion kernel act by *reshaping the spectrum* of L . Making this explicit puts kernels on graphs into the frequency domain, exactly the harmonic-analysis picture we used for semigroup and translation-invariant kernels, and it produces a whole family of kernels at once.

Order the Laplacian eigenvalues $0 = \lambda_1 < \lambda_2 \leq \dots \leq \lambda_m$ (the strict first gap uses connectedness), with orthonormal eigenvectors $u_1, \dots, u_m$. The diffusion kernel $K_t = \sum_i e^{-t\lambda_i} u_i u_i^\top$ has strictly positive eigenvalues, hence is *invertible*, and its inverse lets us compute the RKHS norm directly.

Any $f \in \mathbb{R}^m$ can be written $f = K(K^{-1}f)$, so its norm in the diffusion RKHS is

$$\|f\|_{K_t}^2 = (K^{-1}f)^\top K(K^{-1}f) = f^\top K^{-1}f.$$

Introduce the *discrete Fourier transform* of f , the projections onto the eigenbasis,

$$\hat{f}_i = u_i^\top f, \quad i = 1, \dots, m.$$

Then, since $K^{-1} = \sum_i e^{t\lambda_i} u_i u_i^\top$,

$$\|f\|_{K_t}^2 = f^\top K^{-1}f = \sum_{i=1}^m e^{t\lambda_i} \hat{f}_i^2.$$

This is precisely the graph version of the continuous penalty $\int |\hat{f}(\omega)|^2 e^{\sigma^2 \omega^2} d\omega$ that characterizes the Gaussian RKHS: high-frequency components are penalized exponentially.

The Laplacian eigenbasis is a Fourier basis

The vector $\hat{f} = (\hat{f}_1, \dots, \hat{f}_m)^\top$ is the discrete Fourier transform of f . The eigenvectors u_i of the Laplacian play the role of the sine/cosine basis on $\mathbb{R}^n$, and the eigenvalues λ_i play the role of squared frequencies ω^2 . As λ_i grows, the eigenvector u_i oscillates more rapidly across the graph, so summing $e^{t\lambda_i} \hat{f}_i^2$ charges rough (high-frequency) functions the most.

Nothing in the argument required the specific weights $e^{t\lambda_i}$. Any reweighting of the spectrum defines a kernel.

Spectral family of graph kernels

For a non-increasing function $r : \mathbb{R}_+ \rightarrow \mathbb{R}_+^*$, define

$$K_r = \sum_{i=1}^m r(\lambda_i) u_i u_i^\top, \quad \text{with RKHS norm} \quad \|f\|_{K_r}^2 = \sum_{i=1}^m \frac{\hat{f}_i^2}{r(\lambda_i)}.$$

Choosing $r(\lambda) = e^{-t\lambda}$ recovers the diffusion kernel; $r(\lambda) = 1/\lambda$ (on $\text{Im}(L)$) recovers the Laplacian pseudo-inverse. Requiring r non-increasing makes high-frequency modes expensive, i.e. enforces smoothness.

Regularized Laplacian

Take $r(\lambda) = \frac{1}{\lambda + \epsilon}$ with $\epsilon > 0$. Then

$$K = \sum_{i=1}^m \frac{1}{\lambda_i + \epsilon} u_i u_i^\top = (L + \epsilon I)^{-1},$$

a full-rank kernel (no need to work on the mean-zero subspace, since ϵ shifts the zero eigenvalue). Its norm reads

$$\|f\|_K^2 = f^\top K^{-1} f = f^\top (L + \epsilon I) f = \sum_{i \sim j} (f(x_i) - f(x_j))^2 + \epsilon \sum_{i=1}^m f(x_i)^2,$$

the graph roughness $\Omega(f)$ plus a ridge term $\epsilon \|f\|^2$ that also penalizes large values. This is the smoothing-plus-shrinkage regularizer used in practice.

Verification artifact. checks/example-ch10-example-24-12.json records the example source hash and verification scope.

25.2.5 Applications

With a kernel on the graph in hand, standard kernel methods immediately do useful things on networks.

Graph partitioning. A classical relaxation of graph partitioning minimizes edge-cut energy subject to a normalization and to orthogonality with the constant vector,

$$\min_{f \in \mathbb{R}^m} \sum_{i \sim j} (f_i - f_j)^2 \quad \text{s.t.} \quad \sum_i f_i^2 = 1 \quad \text{and} \quad f \perp \mathbf{1}.$$

The constraint $f \perp \mathbf{1}$ (equivalently $\sum_i f_i = 0$) is essential: without it the trivial constant solution $f \propto \mathbf{1}$ wins, achieving zero edge-cut but assigning every vertex the same value and so partitioning nothing. With that constant mode removed, the minimizer is the *Fiedler vector*, the eigenvector of the second-smallest Laplacian eigenvalue λ_2 , whose sign pattern gives the partition. Using $\sum_{i \sim j} (f_i - f_j)^2 = \|f\|_{\mathcal{H}}^2$, the same problem reads

$$\max_f \sum_i f_i^2 \quad \text{s.t.} \quad \|f\|_{\mathcal{H}} \leq 1 \quad \text{and} \quad f \perp \mathbf{1}.$$

Maximizing the ordinary variance under an RKHS-norm constraint is exactly kernel PCA in the Laplacian RKHS, restricted to the mean-zero subspace, so the second-smallest spectral component partitions the graph.

Search on a graph. Given a query set of nodes $x_1, \dots, x_q$, rank the other nodes by similarity by solving

$$\min_f \|f\|_{\mathcal{H}} \quad \text{s.t.} \quad f(x_i) \geq 1 \quad \text{for } i = 1, \dots, q,$$

the smoothest function that is high on the query. Its values on the remaining nodes give a ranking that diffuses the query outward along the graph.

Semi-supervised learning. When only a few nodes are labeled, fitting a function that agrees with the labels while being smooth in the graph norm propagates the labels through dense regions of the graph, the kernel version of label propagation.

Tumor classification from microarray data (Rapaport et al., 2006). Here the graph is a biological network and the "nodes" are genes. One measures expression of more than 10^4 genes on fewer than 100 samples of two or more tumor classes, and wants a classifier

$$f(x) = \sum_{i=1}^p \beta_i x_i + \beta_0$$

that both separates the classes and is biologically interpretable through its weights β_i . With roughly 10^5 features and 100 samples no robust estimate exists without prior knowledge. The usual remedies each have drawbacks: constraining $\|\beta\|_2$ (ridge, SVM) or $\|\beta\|_1$ (lasso) classifies well but the small or sparse weights carry no biological

reading, and hard feature selection down to a handful of genes is not robust and is easily misinterpreted because neighboring genes are highly correlated.

The pathway insight is that biological function lives in *pathways*, groups of interacting genes, not in single genes, and many pathways are already mapped. Rapaport et al. (2006) therefore constrain the *diffusion RKHS norm* of β with respect to the gene network, which is appropriate exactly when the true decision function is smooth along the network. Projecting the weights of an ordinary SVM onto the complete known metabolic network of budding yeast (from the KEGG database) gives good accuracy but a scattered, uninterpretable weight pattern; projecting the weights of a *spectral* SVM, one regularized by the graph, gives comparable accuracy while the weights now vary smoothly over pathways and can be read biologically. Smoothness on the network buys interpretation at no cost in accuracy, which is the whole promise of a well-chosen kernel on a graph.

Further reading

The two standard kernel-methods textbooks both develop kernels for structured objects: Schölkopf and Smola (2002), and Shawe-Taylor and Cristianini (2004), whose chapters 10 and 11 build up string, tree, and general convolution kernels for structured data in detail. Both books, however, predate much of the graph-kernel literature this chapter draws on, so for the specific results here the primary papers are the place to go. On kernels between graphs, the hardness results and the walk-kernel program are due to Gärtner et al. (2003); the marginalized random-walk kernel to Kashima et al. (2003); the shortest-path kernel to Borgwardt and Kriegel (2005); the subtree kernel to Ramon and Gärtner (2003); and the Weisfeiler-Lehman subtree kernel and its survey comparison to Shervashidze et al. (2011). On kernels on a graph, the diffusion kernel is due to Kondor and Lafferty (2002), and the spectral family of graph kernels, together with the regularization reading of the Laplacian, to Smola and Kondor (2003). The survey of Vishwanathan, Schraudolph, Kondor, and Borgwardt (2010) casts the random-walk kernels in a single framework and shows how to compute them efficiently, and remains the standard overview of the graph-kernel literature as a whole.

25.3 Common mistakes and practical implications

For **Kernels for and on Graphs**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

25.4 Summary and further reading

This chapter established explain the central definitions and claims in Kernels for and on Graphs; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Gärtner and Wrobel 2003), (Kashima and Inokuchi 2003), (Borgwardt and Kriegel 2005).

25.5 Exercises

1. warm-up The chapter keeps two problems apart: kernels *between* graphs (each data point is a whole graph) and kernels *on* a graph (each data point is one node of a single fixed graph). For each task below, say which of the two you are facing, and name the object the kernel would be built from (a product graph, or a Laplacian):
(a) predicting whether each molecule in a chemical library is active against a target; (b) predicting the

function of each protein in one protein-protein interaction network; (c) classifying segmented images, each encoded as a graph of adjacent regions; (d) ranking web pages by relevance to a small set of query pages inside a single hyperlink graph.

2. warm-up Consider the graph on vertices $\{1, 2, 3, 4\}$ with edges 1-2, 2-3, 3-4, and 2-4. (a) For each of the sequences (1, 2, 3, 4), (1, 2, 4, 2), (1, 2, 3, 2), decide whether it is a path, a walk that is not a path, or neither. (b) Which of them *totters*, in the sense $v_i = v_{i+2}$ for some i ? (c) In one sentence, explain why counting all paths in a graph is NP-hard while counting all walks is not, referring to the distinctness constraint that a path carries and a walk drops.
3. computation Let G_1 be the labeled path on vertices (1, 2, 3) with labels (X, Y, X) and edges 1-2, 2-3; let G_2 be the single labeled edge on vertices (a, b) with labels (X, Y) and edge a-b. (a) Construct the product graph $G_1 \times G_2$: list its vertices (pairs of identically labeled vertices) and its edges, and draw it. (b) Write its adjacency matrix A . (c) Using the identity that the n -th-order walk kernel equals $\mathbf{1}^\top A^n \mathbf{1}$, compute the number of common labeled walks of length 1 and of length 2 shared by G_1 and G_2 . Hint

A vertex of the product graph pairs an X with an X or a Y with a Y; an edge exists only when both graphs can take the step at once. You should find three product vertices forming a path, so A is a 3×3 matrix. Recall $[A^n]_{i,j}$ counts length- n walks from i to j , and summing all entries is $\mathbf{1}^\top A^n \mathbf{1}$.

4. computation Take the graph on vertices $\{1, 2, 3, 4\}$ with edges 1-2, 2-3, 3-4, 2-4. (a) Write the adjacency matrix A , the degree matrix D , and the Laplacian $L = D - A$. (b) Verify directly that $L\mathbf{1} = 0$. (c) For $f = (2, 1, 0, 1)^\top$, compute the roughness $\Omega(f) = \sum_{i \sim j} (f_i - f_j)^2$ by summing over the four edges, and independently compute $f^\top L f$; confirm the two agree.
5. computation Let G be the triangle, the complete graph on three vertices, so $L = 3I - J$ with J the all-ones matrix. (a) Give the eigenvalues of L and a corresponding orthonormal eigenbasis. (b) Using $e^{-tL} = \sum_i e^{-t\lambda_i} u_i u_i^\top$, show that the diffusion kernel is

$$K_{i,j} = \begin{cases} \frac{1}{3} + \frac{2}{3}e^{-3t}, & i = j, \\ \frac{1}{3} - \frac{1}{3}e^{-3t}, & i \neq j, \end{cases}$$

and evaluate K at $t = \frac{1}{3} \ln 2$. (c) State $\lim_{t \rightarrow 0} K$ and $\lim_{t \rightarrow \infty} K$ and say in one line what each limit means about how much graph structure the kernel sees. Hint

Since $L = 3I - J$, the constant vector $\mathbf{1}$ is an eigenvector with eigenvalue 0, and every vector orthogonal to $\mathbf{1}$ is an eigenvector with eigenvalue 3. Write $e^{-tL} = e^0 \frac{1}{3} \mathbf{1} \mathbf{1}^\top + e^{-3t} (I - \frac{1}{3} \mathbf{1} \mathbf{1}^\top)$. At $t = \frac{1}{3} \ln 2$ you get $e^{-3t} = \frac{1}{2}$.

6. proof Let A be the adjacency matrix of an undirected graph with no self-loops, D the degree matrix, and $L = D - A$. Prove the quadratic-form identity

$$v^\top L v = \frac{1}{2} \sum_{i,j} A_{i,j} (v_i - v_j)^2 \quad \text{for all } v \in \mathbb{R}^m,$$

where the double sum runs over all ordered pairs (i, j) . Then explain why the right-hand side equals $\sum_{i \sim j} (v_i - v_j)^2$, the sum over undirected edges, and deduce that L is positive semi-definite. Hint

Expand $(v_i - v_j)^2 = v_i^2 - 2v_i v_j + v_j^2$ inside the double sum. Use $\sum_j A_{i,j} = D_{i,i}$ for the v_i^2 terms and the symmetry $A_{i,j} = A_{j,i}$. The factor $\frac{1}{2}$ appears because each undirected edge $\{i, j\}$ is counted once as (i, j) and once as (j, i) .

7. proof Let L be a real symmetric matrix with eigendecomposition $L = \sum_{i=1}^m \lambda_i u_i u_i^\top$. (a) Show that for every $t > 0$ the matrix exponential $e^{-tL} = \sum_{i=1}^m e^{-t\lambda_i} u_i u_i^\top$ is symmetric with strictly positive eigenvalues, hence is a valid (positive definite) kernel. (b) Explain why this argument makes no use of the sign of the λ_i , so that e^{-tL} is p.d. for *any* symmetric L , whereas substituting the graph distance into $\exp(-t d_G)$ can fail to be p.d. (recall the five-vertex counterexample of the chapter). Hint

The exponential of a symmetric matrix shares its eigenvectors and exponentiates its eigenvalues; $e^{-t\lambda_i} > 0$ no matter what λ_i is. Positive eigenvalues of a symmetric matrix are exactly the condition for a valid kernel. The diffusion kernel is built from the operator L , not from the distance, which is why it never runs into the c.p.d. obstruction.

8. challenge This exercise makes the expressiveness-complexity tradeoff precise from both ends. (a) For the geometric walk kernel, weights factor as an initial weight $\lambda_i(v_1)$ times transition weights $\lambda_t(v_{k-1}, v_k)$ along the walk. Collecting these into Λ_i and Λ_t on the product graph, derive the closed form

$$K_{\text{walk}}(G_1, G_2) = \sum_{n=0}^{\infty} \Lambda_i \Lambda_t^n \mathbf{1} = \Lambda_i (I - \Lambda_t)^{-1} \mathbf{1},$$

and state the exact condition on Λ_t under which the infinite sum converges. (b) Argue that once this condition holds, the kernel is computable in polynomial time, giving the size of the matrix that must be inverted in terms of $|V_1|$ and $|V_2|$. (c) Now the other end: using $d_K(G_1, G_2)^2 = K(G_1, G_1) + K(G_2, G_2) - 2K(G_1, G_2)$, explain why a *complete* graph kernel would let three kernel evaluations decide graph isomorphism, and conclude why the polynomial-time walk kernel of part (a) cannot be complete. What single feature of a walk, absent from a path, is responsible for both its tractability and its incompleteness? Hint

For (a) the matrix geometric series $\sum_{n \geq 0} \Lambda_t^n = (I - \Lambda_t)^{-1}$ converges exactly when the spectral radius of Λ_t is below 1; it is the matrix version of $\sum_{n \geq 0} x^n = 1/(1 - x)$. For (b) the product graph has $O(|V_1||V_2|)$ vertices. For (c) a complete kernel has $d_K(G_1, G_2) = 0$ iff $G_1 \simeq G_2$, so computing it is at least as hard as graph isomorphism; the culprit in both directions is that a walk may revisit vertices, dropping the path's global no-repeat memory.

9. computation Run 1-WL color refinement, from the uniform coloring $c_0 \equiv 0$, on two four-vertex graphs: the path P_4 with edges 1-2, 2-3, 3-4, and the star $K_{1,3}$ with edges 1-2, 1-3, 1-4. (a) After one refinement round, write each vertex's new color as the pair $(c_0(v), \{c_0(u) : u \sim v\})$, and give the two color histograms. (b) State whether 1-WL already separates the two graphs, and which vertex invariant its round-1 colors encode. (c) Contrast with the 6-cycle versus two-triangle pair of the worked example, which 1-WL never separates: what feature of those graphs defeats the test? Hint

After one round a vertex's color is fixed by its degree, since all neighbors still share color 0. The star has a degree-3 vertex the path lacks, so the histograms differ at round 1. The regular graphs of the worked example have no degree variation at all, so refinement has nothing to split.

10. proof A message-passing GNN updates $h_v \leftarrow \text{UPDATE}(h_v, \text{AGG} \{h_u : u \sim v\})$ and reads out a graph vector by a permutation-invariant pooling. (a) Using the theorem that such a network is at most as powerful as 1-WL, explain why no choice of parameters, depth, or training lets it distinguish the 6-cycle from the two disjoint triangles. (b) Show the mean aggregator is not injective on multisets by evaluating it on $\{a\}$ and $\{a, a\}$, then do the same for the sum to show it separates them; conclude that a mean-pooling network is strictly weaker than 1-WL while a sum-based one can reach it. (c) State the condition on AGG, UPDATE, and the readout under which the network attains the 1-WL ceiling. Hint

For (a) both graphs are 2-regular, so 1-WL gives them the same coloring and the theorem forces the network to give them the same embedding. For (b), $\text{mean}\{a\} = \text{mean}\{a, a\} = a$ but $\text{sum}\{a\} = a \neq 2a = \text{sum}\{a, a\}$ for $a \neq 0$. For (c), all three maps must be injective on multisets.

26 Kernels from Generative Models

We usually know something about where our data came from. DNA sequences descend from ancestors by a chain of mutations, a protein family is generated by a hidden Markov model, an RNA molecule folds under a grammar of base pairings, and a document is emitted by a source of words that reflects its topic. A generative model packages that knowledge as a probability distribution, and this chapter turns such a distribution into a kernel so that the structure it encodes transfers to any kernel machine. Two constructions organize the field. The first, the P-kernel and its marginalization form, reads a joint probability as an inner product and sums it over hidden states by dynamic programming: this gives the fixed-length and pair hidden Markov model kernels for biological sequences and the hidden tree kernel for phylogenetic profiles. The second, the Fisher kernel, differentiates the log-likelihood of a single fitted model and compares examples by the direction in which each would nudge the parameters. The general Fisher construction is set up in the kernel-families chapter and its sequence specialization worked out in the string-kernels chapter; here we place it beside the marginalized kernels it complements, and give every recursion in full.

26.1 P-kernels: a probability as an inner product

The most direct way to let a probabilistic model design a kernel is to let the model's own joint probability be the kernel. If we have a symmetric distribution over pairs of objects that assigns high probability to similar pairs, that number already behaves like a similarity, and the only question is whether it is a legitimate inner product. When it is, nothing further need be built.

Definition (P-kernel, Haussler 1999)

Let $\mathcal{X}$ be a finite or countable set. A symmetric joint distribution $P(x, z)$ on the product $\mathcal{X} \times \mathcal{X}$, meaning $P(x, z) = P(z, x) \geq 0$ and $\sum_{x, z} P(x, z) = 1$, is a *P-kernel* on $\mathcal{X}$ if it satisfies the finitely positive semidefinite property: for every finite sample $x_1, \dots, x_n$ the matrix $(P(x_i, x_j))_{i, j}$ is positive semidefinite. One then sets $\kappa(x, z) = P(x, z)$.

Any kernel with finite total mass rescales to a P-kernel. If $\kappa \geq 0$ and $M = \sum_{x, z \in \mathcal{X}} \kappa(x, z) < \infty$, then $P = \kappa/M$ is a symmetric distribution with the same Gram matrices up to the positive factor $1/M$, hence a P-kernel. So the class is at least as rich as the nonnegative summable kernels. The converse fails, and it fails for the simplest possible reason: not every symmetric distribution is positive semidefinite.

Example (a distribution that is not a kernel)

On the two-element set $\mathcal{X} = \{x_1, x_2\}$ put all the mass off the diagonal, $P(x_1, x_2) = P(x_2, x_1) = \frac{1}{2}$ and $P(x_1, x_1) = P(x_2, x_2) = 0$. The Gram matrix is

$$\begin{pmatrix} 0 & 0.5 \\ 0.5 & 0 \end{pmatrix},$$

whose eigenvalues are ± 0.5 . The negative eigenvalue means P is not positive semidefinite, so this perfectly good probability distribution is not a P-kernel. A distribution that rewards pairs for being *different* cannot be an inner product, which always rewards an object for agreeing with itself.

Verification artifact. checks/example-ch-generative-example-25-1.json records the example source hash and verification scope.

What rescues the construction in general is a hidden variable, and the mechanism is worth isolating because it recurs in every kernel of this chapter.

26.1.1 Conditional independence and marginalization

Suppose the pair (x, z) is not generated directly but through a latent model m , drawn from a set M with prior $P_M(m)$, and that given m the two objects are produced independently. Then the joint distribution is obtained by summing, that is marginalizing, over the unobserved m :

$$P(x, z) = \sum_{m \in M} P(x, z \mid m) P_M(m) = \sum_{m \in M} P(x \mid m) P(z \mid m) P_M(m).$$

Read $P(x, z \mid m) P_M(m)$ as a distribution over triples (x, z, m) ; then $P(x, z)$ is its marginal, integrating out m . This is why kernels of this form are called *marginalization kernels*. The variable m can be a discrete index, a hidden state path, a parse tree, or an evolutionary ancestor; the same algebra covers all of them.

Proposition (conditional independence gives a P-kernel)

If x and z are conditionally independent given m , so that $P(x, z \mid m) = P(x \mid m)P(z \mid m)$, then the marginal $P(x, z) = \sum_m P(x \mid m)P(z \mid m)P_M(m)$ is a P-kernel. Conditional independence is sufficient, though not necessary, for the finitely positive semidefinite property.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Fix a single model m and define the feature $\phi_m(x) = P(x \mid m)$, a real function of x . The term $P(x \mid m)P(z \mid m) = \phi_m(x)\phi_m(z)$ is then a rank-one product of a function with itself, so for any finite sample and scalars $c_1, \dots, c_n$,

$$\sum_{i,j} c_i c_j \phi_m(x_i) \phi_m(x_j) = \left(\sum_i c_i \phi_m(x_i) \right)^2 \geq 0,$$

which shows each m -term is positive semidefinite. The full kernel is the nonnegative combination $\sum_m P_M(m) \phi_m(x) \phi_m(z)$ of these terms with weights $P_M(m) \geq 0$, and a nonnegative combination of positive semidefinite kernels is positive semidefinite. Hence $P(x, z)$ is a P-kernel. $\square$

The proof exhibits the feature map explicitly: stacking the values across models, $\Phi(x) = (\sqrt{P_M(m)} P(x \mid m))_{m \in M}$, gives $P(x, z) = \langle \Phi(x), \Phi(z) \rangle$. The coordinates of Φ are indexed by the latent models, and two objects are similar when the models that explain one tend to explain the other. This is precisely the conditional-independence, or R-convolution, structure that Haussler (1999) identified as the backbone of kernels on discrete structures: build a kernel on a completed object, then average over the ways of completing it. Watkins (2000) arrived at the same conditional-independence principle independently, together with its use for hidden Markov models. The construction is the probabilistic face of the convolution kernels of the efficient string and tree kernels chapter, which realize the same "sum a product over decompositions" pattern deterministically.

The remaining sections instantiate the latent variable m as ever richer generative processes: an independent hidden label at each position, a Markov chain of hidden states, an alignment with gaps, and a tree of evolutionary ancestors. In each case the sum over m is astronomically large, and in each case a dynamic program collapses it.

26.2 The fixed-length marginalization kernel

Start with the simplest hidden model for strings. Fix a length n and an alphabet Σ , and suppose each symbol s_i of a string $s = s_1 \dots s_n$ is emitted independently from a hidden state h_i drawn from a state set A . If the hidden states are themselves independent across positions, $P(h) = \prod_{i=1}^n P(h_i)$, then the marginalization kernel sums the joint emission probability over all hidden strings $h \in A^n$:

$$\kappa(s, t) = \sum_{h \in A^n} P(s | h) P(t | h) P(h) = \sum_{h \in A^n} \prod_{i=1}^n P(s_i | h_i) P(t_i | h_i) P(h_i).$$

By the proposition this is a valid kernel: the hidden string h is the latent model m , and s, t are conditionally independent given it. What makes it cheap is that the joint quantity factors across positions, and the distributive law turns the sum of products into a product of sums:

$$\kappa(s, t) = \prod_{i=1}^n \left(\sum_{a \in A} P(s_i | a) P(t_i | a) P(a) \right).$$

Each factor is a single position's contribution, so the exponential sum over $|A|^n$ hidden strings costs only $O(n |A|)$. If $P(\cdot | a)$ is a genuine emission distribution the whole kernel is also normalized: summing $\kappa(s, t)$ over all pairs $(s, t) \in \Sigma^n \times \Sigma^n$ telescopes position by position to $\prod_{i=1}^n \left(\sum_{s_i, t_i} P(s_i, t_i) \right) = \prod_{i=1}^n 1 = 1$, confirming it is a P-kernel and not merely a positive semidefinite function. This model is unrealistic, since real symbols are correlated along the sequence, but it fixes the pattern of marginalizing over sums of products that the next model exploits with genuine structure.

26.3 The hidden Markov model kernel

The independence $P(h) = \prod_i P(h_i)$ throws away exactly the dependence that makes a sequence a sequence. The fixed-length hidden Markov model kernel keeps the emission model but gives the hidden states a first-order Markov structure, so the state at position i depends on the state at position $i - 1$:

$$P_M(h) = P_M(h_1) P_M(h_2 | h_1) \cdots P_M(h_n | h_{n-1}), \quad P(s, t | h) = \prod_{i=1}^n P(s_i | h_i) P(t_i | h_i).$$

The kernel is again $\kappa(s, t) = \sum_{h \in A^n} P(s | h) P(t | h) P_M(h)$, and it is a P-kernel for the same conditional-independence reason. Now the sum no longer factors across positions, because a hidden state is tied to its neighbor, but the Markov structure means the coupling reaches back only one step, which is exactly the situation dynamic programming was built for.

Group the hidden paths by their final state. For $k = 1, \dots, n$ and $a \in A$, let $A_a^k = \{h \in A^k : h_k = a\}$ be the state paths of length k ending in a , and define the subkernel

$$\kappa_{k,a}(s, t) = \sum_{h \in A_a^k} \prod_{i=1}^k P(s_i | h_i) P(t_i | h_i) P_M(h_i | h_{i-1}),$$

where the first factor uses the initial probability $P_M(h_1)$. At length one there is a single path, so $\kappa_{1,a}(s, t) = P(s_1 | a) P(t_1 | a) P_M(a)$. To extend a path to length $k + 1$ ending in a , we append the state a to some path ending in b at length k , pay the transition $P_M(a | b)$ and the emission of position $k + 1$, and sum over the predecessor b :

$$\kappa_{k+1,a}(s, t) = P(s_{k+1} | a) P(t_{k+1} | a) \sum_{b \in A} P_M(a | b) \kappa_{k,b}(s, t).$$

The full kernel is recovered by summing the length- n subkernels over the final state, $\kappa(s, t) = \sum_{a \in A} \kappa_{n,a}(s, t)$. This is the forward recursion of a hidden Markov model, read as a kernel rather than a likelihood.

Algorithm (fixed-length HMM marginalization kernel)

Input Equal-length strings $s, t \in \Sigma^n$; state set A ; emission $P(\sigma | a)$; initial $P_M(a)$; transition $P_M(a | b)$.

Output The marginalization kernel $\kappa(s, t) = \sum_{h \in A^n} P(s | h) P(t | h) P_M(h)$.

1. For each $a \in A$, initialize $\kappa_{1,a} \leftarrow P(s_1 | a) P(t_1 | a) P_M(a)$.
2. For $k = 1, \dots, n-1$ and each $a \in A$, set $\kappa_{k+1,a} \leftarrow P(s_{k+1} | a) P(t_{k+1} | a) \sum_{b \in A} P_M(a | b) \kappa_{k,b}$, overwriting the length- k table.
3. Return $\kappa(s, t) = \sum_{a \in A} \kappa_{n,a}$.

The recursion keeps one table entry per state, updates it in $O(|A|)$ work per position, and so evaluates the kernel in $O(n|A|^2)$ time and $O(|A|)$ space, an exponential saving over the naive sum. It compares two aligned strings under every hidden annotation of their shared positions at once, and it fits tasks with substitutions but no insertions or deletions. The following worked example runs the recursion by hand on a two-state model and checks it against a brute-force enumeration of all four hidden paths.

Example (marginalization kernel under a 2-state HMM)

Alphabet $\Sigma = \{A, B\}$, hidden states $\{1, 2\}$. Emissions $P(A | 1) = 0.8$, $P(B | 1) = 0.2$, $P(A | 2) = 0.3$, $P(B | 2) = 0.7$. Initial $P_M(1) = 0.6$, $P_M(2) = 0.4$. Transitions $P_M(1 | 1) = 0.7$, $P_M(2 | 1) = 0.3$, $P_M(1 | 2) = 0.4$, $P_M(2 | 2) = 0.6$. Compare $s = AB$ and $t = AA$, so $n = 2$.

1. Enumerate the four hidden paths. Each term is $P(s | h)P(t | h)P_M(h)$ with $P_M(h) = P_M(h_1)P_M(h_2 | h_1)$. Path (1, 1): $0.16 \cdot 0.64 \cdot 0.42 = 0.043008$. Path (1, 2): $0.56 \cdot 0.24 \cdot 0.18 = 0.024192$. Path (2, 1): $0.06 \cdot 0.24 \cdot 0.16 = 0.002304$. Path (2, 2): $0.21 \cdot 0.09 \cdot 0.24 = 0.004536$.
2. Sum the joint probabilities. $\kappa(s, t) = 0.043008 + 0.024192 + 0.002304 + 0.004536 = 0.074040$.
3. Base of the recursion. With $s_1 = t_1 = A$: $\kappa_{1,1} = 0.8 \cdot 0.8 \cdot 0.6 = 0.384000$ and $\kappa_{1,2} = 0.3 \cdot 0.3 \cdot 0.4 = 0.036000$.
4. Advance one position. With $s_2 = B$, $t_2 = A$: $\kappa_{2,1} = P(B | 1)P(A | 1)[P_M(1 | 1)\kappa_{1,1} + P_M(1 | 2)\kappa_{1,2}] = 0.16(0.2688 + 0.0144) = 0.045312$, and $\kappa_{2,2} = P(B | 2)P(A | 2)[P_M(2 | 1)\kappa_{1,1} + P_M(2 | 2)\kappa_{1,2}] = 0.21(0.1152 + 0.0216) = 0.028728$.
5. Sum over the final state. $\kappa(s, t) = \kappa_{2,1} + \kappa_{2,2} = 0.045312 + 0.028728 = 0.074040$.

Reading. The direct enumeration and the forward recursion return the identical value 0.074040, the recursion doing in two length-two updates what the enumeration does by touching all $|A|^n = 4$ paths. On a realistic protein model with dozens of states and sequences of length hundreds, the enumeration is impossible and the recursion is routine.

Verification artifact. checks/example-ch-generative-example-25-2.json records the example source hash and verification scope.

26.4 The pair hidden Markov model kernel

Sequences in biology rarely share a length. Two homologous genes differ not only by substitutions but by insertions and deletions, the gaps of a pairwise alignment, so a kernel that insists on equal lengths and a shared position index cannot compare them. The remedy is to let the hidden model emit to the two strings at different rates. Augment the alphabet with the empty symbol ε ; a hidden state may then emit a real symbol to one string while emitting ε , that is nothing, to the other. A state that emits (s_i, ε) advances only the first string, modeling a deletion; (ε, t_j) advances only the second, an insertion; and (s_i, t_j) advances both, a match or substitution. Because emissions no longer line up with positions, the states form a general Markov process, a probabilistic finite-state automaton with transition matrix $M_{ab} = P_M(b | a)$ and a distinguished initial state a_I .

Definition (pair hidden Markov model, Durbin et al. 1998)

A *pair data model* is a model whose states emit ordered pairs of data items: each state h carries a distribution $P_h(x_1, x_2) = P((x_1, x_2) | h)$ over pairs. A *pair hidden Markov model* is a hidden Markov model whose states emit ordered pairs. Its trajectories $h = (h_0 = a_I, h_1, \dots, h_N)$ begin at the initial state, and the joint emission of two strings is

$$P(s, t) = \sum_h P(s, t | h) P_M(h), \quad P_M(h) = \prod_i P_M(h_i | h_{i-1}).$$

After a standard preprocessing step that removes states emitting the doubly empty pair $(\varepsilon, \varepsilon)$, which would consume no input and stall the recursion, the model can be marginalized by dynamic programming over the two independent position indices, exactly as the string kernels of the string-kernels chapter carry a table indexed by prefixes of each string. Let $A_a = \{h \in \hat{M} : h_{|h|} = a, h_0 = a_I\}$ be the trajectories ending in state a , and define the subkernel over the prefixes $s(1:i)$ and $t(1:j)$:

$$\kappa_{i,j,a}(s, t) = \sum_{h \in A_a} \hat{P}(s(1:i), t(1:j) \mid h) P_M(h).$$

The base case reflects that before any symbol is read we sit in the initial state: $\kappa_{0,0,a_I}(s, t) = 1$, and $\kappa_{0,0,a}(s, t) = 0$ for every other a . Each state reaches (i, j) in one of three ways, according to what it just emitted, and each way is preceded by summing over the previous state b :

$$\kappa_{i,j,a}(s, t) = \sum_{b \in A} P_M(a \mid b) \left[P((s_i, \varepsilon) \mid a) \kappa_{i-1,j,b} + P((\varepsilon, t_j) \mid a) \kappa_{i,j-1,b} + P((s_i, t_j) \mid a) \kappa_{i-1,j-1,b} \right].$$

The three terms are the deletion step, the insertion step, and the match step. The kernel is the sum of the corner subkernels over the final state, $\kappa(s, t) = \sum_{a \in A} \kappa_{|s|,|t|,a}(s, t)$.

Algorithm (pair HMM marginalization kernel)

Input Strings s, t of lengths $|s|, |t|$; state set A with initial a_I ; transitions $P_M(a \mid b)$; pair emissions $P((s_i, \varepsilon) \mid a)$, $P((\varepsilon, t_j) \mid a)$, $P((s_i, t_j) \mid a)$.

Output The pair HMM kernel $\kappa(s, t) = \sum_h P(s \mid h) P(t \mid h) P_M(h)$.

1. Initialize $\kappa_{0,0,a_I} \leftarrow 1$ and $\kappa_{0,0,a} \leftarrow 0$ for $a \neq a_I$.
2. For $i = 0, \dots, |s|$ and $j = 0, \dots, |t|$ in order (skipping $i = j = 0$) and each $a \in A$, set $\kappa_{i,j,a} \leftarrow \sum_b P_M(a \mid b) [P((s_i, \varepsilon) \mid a) \kappa_{i-1,j,b} + P((\varepsilon, t_j) \mid a) \kappa_{i,j-1,b} + P((s_i, t_j) \mid a) \kappa_{i-1,j-1,b}]$, treating out-of-range subkernels as 0.
3. Return $\kappa(s, t) = \sum_{a \in A} \kappa_{|s|,|t|,a}$.

The table has $(|s|+1)(|t|+1)$ cells, each holding one value per state and updated in $O(|A|^2)$ work, so the kernel costs $O(|s| |t| |A|^2)$, the same order as the Needleman-Wunsch alignment it probabilistically mirrors. This is the Durbin et al. (1998) forward algorithm read as a kernel. Unlike a raw alignment score, it sums a genuine probability over all alignments, so by the conditional-independence proposition it is a P-kernel: positive definite with no separate argument needed. It is the probabilistic sibling of the local alignment kernel, which sums an exponentiated score over alignments and needs a convolution argument to certify positive definiteness. Transitions between two-symbol emitting states are independent of whether the state emits to one string or the other, which is what lets the three cases share a single transition factor.

26.5 The hidden tree model kernel

The pair HMM handles unequal lengths by threading the two strings through a linear chain of states. A different generative story fits data whose positions are related not sequentially but by shared ancestry. In a phylogenetic profile, the n symbols of an aligned sequence sit at the n leaves of a fixed tree; each internal node is an extinct ancestor, and the state of a node is drawn conditionally on the state of its parent, modeling the inheritance of a gene down the tree of life. The observed sequence is the vector of leaf states. This is a probabilistic graphical model over the tree, and marginalizing a complete-data kernel over the internal states gives, once more, a P-kernel:

$$P_M(s, t) = \sum_{h \in M} P(s \mid h) P(t \mid h) P_M(h),$$

where h now ranges over labelings of the internal nodes $v_0, \dots, v_N$ with states from A , the root v_0 drawn from a prior $P_M(a)$, and each child $\text{ch}_j(v)$ in state b with probability $P_j(b | a)$ given its parent's state a . The leaves are pinned to the observed symbols.

The sum over all internal labelings is again exponential, and again a recursion collapses it, this time message passing from the leaves toward the root. Store at each node one value per state,

$$\kappa_{v,a}(s, t) = \sum_{h \in T_a(v)} P(s | h) P(t | h) P_M(h),$$

where $T_a(v)$ ranges over labelings of the subtree rooted at v with v fixed in state a . Because the subtrees hanging off the children of v are conditionally independent given v 's state, the recursion factorizes over children and the distributive law pulls the sum inside each factor:

$$\kappa_{v,a}(s, t) = \prod_{j=1}^{\text{child}(a)} \left(\sum_{b \in A} P_j(b | a) \kappa_{\text{ch}_j(v),b}(s, t) \right).$$

At a leaf l_i the value is the indicator that both strings carry the same fixed symbol there, $\kappa_{l_i,a}(s, t) = [s_i = t_i = a]$ read off the observed positions, and the kernel is assembled at the root by weighting with the root prior,

$$\kappa(s, t) = \sum_{a \in A} P_M(a) \kappa_{v_0,a}(s, t).$$

Each internal node receives $|A|$ messages from each child and combines them, so the whole kernel costs one bottom-up sweep of the tree.

Example (binary tree, binary states)

Take a fixed binary tree with binary states and symbols, and assume both children share the same conditional table, leaving two mutation parameters for the internal edges,

$$P(0 | 1) = \epsilon, \quad P(1 | 1) = 1 - \epsilon, \quad P(1 | 0) = \delta, \quad P(0 | 0) = 1 - \delta,$$

and one root parameter $P(h(v_0) = 1) = \tau$. At an internal node v the two child messages $\kappa_{\text{ch}_1(v),b}$ and $\kappa_{\text{ch}_2(v),b}$ for $b \in \{0, 1\}$ are combined with these four probabilities into the two values $\kappa_{v,0}$ and $\kappa_{v,1}$, which are then passed to the parent. The root combines its two messages with τ and $1 - \tau$ to produce the kernel. As with the linear HMM, the exponential sum over internal labelings becomes a linear sweep because the tree's conditional independences let each node summarize its entire subtree in $|A|$ numbers.

Verification artifact. checks/example-ch-generative-example-25-3.json records the example source hash and verification scope.

This hidden tree kernel is the phylogenetic instance of the marginalization framework: it compares two genes by their entire modeled evolutionary history rather than by a bare count of the organisms in which they co-occur. It underlies the kernels on phylogenetic profiles used to predict gene function, and its message passing is the same sum-product computation that runs the HMM recursions above, now on a tree instead of a chain. The construction extends from trees toward certain graphs, where the general machinery of probabilistic graphical models takes over, a theme picked up by the graph-kernels chapter.

26.6 The Fisher kernel

Every kernel so far indexes the latent model m by a discrete object, a state path, an alignment, a tree labeling, and marginalizes over it while treating the model's numerical parameters as fixed. The Fisher kernel takes the complementary view. It fixes a single fitted model and asks how its *smooth* parameters would have to move to explain each example better, comparing two examples by the direction in which each stretches the parameters. Where marginalization sums over a discrete latent set, the Fisher kernel differentiates with respect to a continuous one, and the two are complementary tools for the same generative model.

The intuition is a gradient one. Suppose we hold a fitted model P_θ and consider adapting its parameters to raise the likelihood of a new point x . A point already typical of the model needs almost no adjustment, so its likelihood gradient is near zero; an unusual point pulls hard on whichever parameters are responsible for its oddity, giving a large gradient in those directions. The gradient of the log-likelihood is therefore a fingerprint of x relative to the model, of fixed dimension no matter the size of x , and comparing two fingerprints compares how the two points would deform the model.

Definition (Fisher score and Fisher information, Jaakkola and Haussler 1999)

Let $\{P_\theta : \theta \in \mathbb{R}^N\}$ be a smooth parametric model and fix a setting θ_0 , for instance the maximum-likelihood estimate on a training set. The *Fisher score* of an example x is the gradient of the log-likelihood,

$$g(\theta_0, x) = \nabla_\theta \log L_\theta(x) \Big|_{\theta=\theta_0} = \left(\frac{\partial}{\partial \theta_i} \log L_\theta(x) \right)_{i=1}^N,$$

one coordinate per parameter. The *Fisher information matrix* is the covariance of the score under the model,

$$I_M = \mathbb{E}[g(\theta_0, x) g(\theta_0, x)^\top],$$

the expectation taken over x generated by P_{θ_0} . The *Fisher kernel* is the whitened inner product of scores,

$$K(x, z) = g(\theta_0, x)^\top I_M^{-1} g(\theta_0, z).$$

The whitening by I_M^{-1} is not cosmetic. Under the model the expected score vanishes at every parameter setting, $\mathbb{E}_{x \sim P_{\theta_0}}[g(\theta_0, x)] = 0$, so I_M is exactly the covariance of the score, and multiplying by I_M^{-1} rotates and rescales the score coordinates into directions that are uncorrelated and of unit variance. This makes the kernel a Mahalanobis inner product that does not depend on how the model happens to be parametrized: the Fisher kernel is invariant under smooth reparametrization, which the general construction in the kernel-families chapter proves. In practice I_M is costly and often nearly singular, so it is routinely replaced by the identity, in which case the Fisher kernel is the plain dot product of scores and the feature map $\Phi(x) = g(\theta_0, x)$ is stored directly as a fixed-length vector, the *Fisher vector*. The following example computes such a score vector by hand for the smallest nontrivial model.

Example (Fisher score for a 2-symbol iid model)

Model: symbols drawn iid from $\Sigma = \{A, B\}$ with unnormalized weights $\theta = (\theta_A, \theta_B)$ and $P(\sigma | \theta) = \theta_\sigma / (\theta_A + \theta_B)$. For a string s of length n with counts (n_A, n_B) , the log-likelihood is $\log L(s) = \sum_\sigma n_\sigma \log \theta_\sigma - n \log(\theta_A + \theta_B)$. Evaluate at the uniform setting $\theta_A = \theta_B = 1$ (so $Z = 2$, $P = \frac{1}{2}$). Compare $s = AAB$, $t = ABB$, and $u = AAA$, all of length 3.

1. Differentiate the log-likelihood. $\frac{\partial}{\partial \theta_\sigma} \log L = \frac{n_\sigma}{\theta_\sigma} - \frac{n}{Z}$. At $\theta_A = \theta_B = 1$, $Z = 2$ this is $g(s)_\sigma = n_\sigma - \frac{n}{2}$, so the score vector is $g(s) = (n_A - \frac{n}{2}, n_B - \frac{n}{2})$.
2. Evaluate the score vectors. $g(s) = (2 - 1.5, 1 - 1.5) = (0.5, -0.5)$; $g(t) = (1 - 1.5, 2 - 1.5) = (-0.5, 0.5)$; $g(u) = (3 - 1.5, 0 - 1.5) = (1.5, -1.5)$. Each obeys $g_A + g_B = 0$, the centering that projects onto the direction perpendicular to the all-ones vector.

3. Form the Fisher kernel with $I = \text{Id}$. $K(s, t) = g(s) \cdot g(t) = (0.5)(-0.5) + (-0.5)(0.5) = -0.5$, while $K(s, s) = 0.25 + 0.25 = 0.5$ and $K(t, t) = 0.5$. The mismatched pair (s, t) scores negative and each self-pair scores positive.
4. Compare with an extreme. $K(s, u) = (0.5)(1.5) + (-0.5)(-1.5) = 1.5$: the A -heavy strings s and u align strongly, since both would push the model toward emitting more A .
5. Inspect the information matrix. The single-symbol Fisher information under the uniform model is $I_1 = \begin{pmatrix} 0.25 & -0.25 \\ -0.25 & 0.25 \end{pmatrix}$, with $\det I_1 = 0$. It is singular because the two counts satisfy $n_A + n_B = n$, so one score coordinate is redundant, which is exactly why I is set to the identity here.

Reading. The score turns each string into the vector of how much it would push the emission probability of each symbol up or down, and the Fisher kernel compares those pushes: s and t disagree ($K = -0.5$), while s and u agree strongly ($K = 1.5$). Up to the centering term, this two-symbol model's Fisher kernel is the 1-spectrum kernel, the count of shared symbols; enlarging the model to k -mer transitions recovers the $(k+1)$ -spectrum kernel, the sequence Fisher kernel derived in full in the string-kernels chapter.

Verification artifact. `checks/example-ch-generative-example-25-4.json` records the example source hash and verification scope.

Two further points connect the example to practice. First, the true information matrix is usually replaced by its empirical version $\hat{I}_M = \frac{1}{\ell} \sum_{i=1}^{\ell} g(\theta_0, x_i) g(\theta_0, x_i)^T$, the sample covariance of the scores, so whitening by $\hat{I}_M^{-1}$ is exactly whitening the Fisher vectors; the invariance this buys can come at the cost of amplifying noise along parameters the data barely constrain. Second, for a hidden Markov model the score with respect to the emission and transition parameters is not a hard k -mer count but an expected state-occupancy and transition count, and each such expectation is delivered in one pass of the forward-backward algorithm, the same dynamic program that ran the marginalization kernels above. The Fisher kernel thereby reuses the entire generative apparatus, differentiating where the marginalized kernels summed. When the generative model additionally carries the class label as a latent variable, the leading Fisher coordinates become the class posteriors minus the priors, so a linear classifier on the Fisher features reconstructs the Bayes rule, which is why the Fisher kernel is asymptotically no worse than the model's own MAP labeling (Jaakkola and Haussler 1999).

26.7 Further reading

The P-kernel and the conditional-independence route to positive definiteness are due to Haussler (1999), whose convolution kernels on discrete structures are the deterministic backbone of the whole construction; Watkins (2000) developed the conditional-independence idea and the hidden Markov model case independently. The formal pair HMM kernel here follows the forward algorithm of Durbin et al. (1998), the standard reference for probabilistic sequence models. The marginalized kernel as a general recipe, averaging a complete-data kernel over a latent posterior computed by forward-backward, is due to Tsuda, Kin, and Asai (2002), and was carried to labeled graphs by Kashima, Tsuda, and Inokuchi (2003). The hidden tree kernel for phylogenetic profiles is due to Vert (2002). The Fisher kernel was introduced by Jaakkola and Haussler (1999) and applied to remote protein homology by Jaakkola, Diekhans, and Haussler (2000); its treatment as the general derivative-of-log-likelihood construction, and the identity that reduces the sequence Fisher kernel to the spectrum kernel, are in Shawe-Taylor and Cristianini (2004), whose chapter 12 this chapter follows, with the broader kernel-design context in Schölkopf and Smola (2002). The next chapters put these generative kernels to work on sequences, on graphs, and, lifting the idea from a point to a whole distribution, in the mean-embedding chapter.

26.8 Common mistakes and practical implications

For **Kernels from Generative Models**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report

preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

26.9 Summary and further reading

This chapter established explain the central definitions and claims in Kernels from Generative Models; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Haussler 1999), (Watkins 2000), (Durbin and Mitchison 1998).

26.10 Exercises

1. computation Verify the two-element counterexample of the opening section: for $P(x_1, x_2) = P(x_2, x_1) = \frac{1}{2}$ and zeros on the diagonal, write down the 2×2 Gram matrix and confirm its eigenvalues are $\pm \frac{1}{2}$. Then show that if instead $P(x_1, x_1) = P(x_2, x_2) = \frac{1}{2}$ and the off-diagonal is zero, the distribution is a P-kernel.
2. computation In the 2-state HMM of the worked example, recompute $\kappa(s, t)$ for $s = AA$ and $t = AA$ both by enumerating the four hidden paths and by the forward recursion, and confirm the two agree. Is $\kappa(AA, AA)$ larger or smaller than $\kappa(AB, AA)$, and why?
3. proof Show that the fixed-length marginalization kernel with independent hidden states, $\kappa(s, t) = \sum_{h \in A^n} \prod_i P(s_i | h_i) P(t_i | h_i) P(h_i)$, factorizes as $\prod_{i=1}^n (\sum_a P(s_i | a) P(t_i | a) P(a))$. Hint: apply the distributive law one position at a time, or argue by induction on n .
4. proof Prove that the sum $\sum_{s, t \in \Sigma^n} \kappa(s, t)$ of the fixed-length kernel over all string pairs equals 1. Hint: exchange the order of summation so the sum over (s_i, t_i) sits inside the product, and use that the emission normalization collapses each position's factor to 1.
5. computation Take the pair HMM recursion with a single emitting state a beyond a_t , self-transition probability 1, and emissions $P((\sigma, \sigma) | a) = m$ for a match, $P((\sigma, \sigma') | a) = \mu$ for a mismatch, and $P((\sigma, \varepsilon) | a) = P((\varepsilon, \sigma) | a) = d$ for a gap. Fill the $\kappa_{i,j}$ table by hand for $s = AB$, $t = A$ and read off $\kappa(s, t)$ in terms of m, μ, d .
6. computation For the binary hidden tree with a single internal root and two leaves, parameters ϵ, δ, τ as in the example, write $\kappa(s, t)$ explicitly for the leaf pair $s_1 = t_1 = 1$ on the first leaf and $s_2 = t_2 = 1$ on the second, by carrying out the root sum $\sum_a P_M(a) \prod_j \sum_b P(b | a) \kappa_{ch_j, b}$. Hint: the leaf messages are indicators $[s_i = t_i = b]$.
7. computation Let $P_\theta(x) = e^{-\theta} \theta^x / x!$ be the Poisson model on $x \in \mathbb{N}_0$ with scalar $\theta > 0$. Compute the Fisher score $g(\theta, x) = \partial_\theta \log P_\theta(x)$, the Fisher information $I(\theta) = \mathbb{E}[g(\theta, x)^2]$, and the whitened one-dimensional Fisher kernel $K(x, x') = g(\theta, x) I(\theta)^{-1} g(\theta, x')$. Which pairs (x, x') receive a positive value at $\theta = 2$?
8. challenge For the 2-symbol iid model of the worked example, take the generative model to be a first-order Markov chain over k -mers with transition weights $a_{u \rightarrow \sigma}$ normalized to $p_{u \rightarrow \sigma} = a_{u \rightarrow \sigma} / \sum_{\sigma'} a_{u \rightarrow \sigma'}$. Differentiate the log-likelihood at the flat setting $a_{u \rightarrow \sigma} = |\Sigma|^{-1}$ to show the score coordinate for $u \rightarrow \sigma$ is $|\{(v_1, v_2) : s = v_1 u \sigma v_2\}| - |\Sigma|^{-1} |\{(v_1, v_2) : s = v_1 u v_2\}|$, and conclude that up to the centering term the Fisher kernel is the $(k+1)$ -spectrum kernel. Hint: the log-likelihood is a sum over positions of $\log p_{s(j:j+k-1) \rightarrow s_{j+k}}$; see the string-kernels chapter.

27 Signature and Sequence-Path Kernels

The string-kernel chapter built similarities for symbolic sequences by counting shared substrings; the efficient-kernels chapter showed how a recursion over prefixes turns those exponential feature counts into a small dynamic-programming table. This chapter carries the same programme to sequences whose entries are real vectors: time series, sensor streams, pen strokes, financial paths. The clean object here is not a discrete string but a continuous path, and its canonical feature map is the path signature, the collection of all iterated integrals of the path. We motivate the signature as a feature map, prove the three structural facts that make it useful (it ignores reparametrization, it multiplies under concatenation by Chen’s identity, and its coordinates satisfy the shuffle relations), compute a truncated signature by hand, and read off the truncated signature kernel. We then meet the untruncated signature kernel of Salvi and coauthors, which sidesteps the infinite feature vector by solving a Goursat partial differential equation. Finally we turn to an older idea, warping-based similarity: plain dynamic time warping is not positive definite, and we see exactly why, then repair it with Cuturi’s global alignment kernel, whose sum over alignments is a dynamic program of the same shape as the string kernels. A closing note places random and reservoir features for sequences alongside the random Fourier features of stationary kernels.

27.1 From points to paths

A supervised learner with a fixed kernel needs one thing: a positive definite similarity between whole examples. When an example is a single vector in $\mathbb{R}^d$, the kernels of the earlier chapters apply directly. When an example is a sequence $x = (x_0, x_1, \dots, x_N)$ of vectors in $\mathbb{R}^d$, sampled from some underlying process, the naive move of stacking the entries into one long vector fails on contact with reality: two recordings of the same gesture at different speeds, or sampled at different rates, produce different stacked vectors, yet they should be similar. What we want is a feature map on the sequence that reads its shape and the order of its events, not the incidental clock that timestamped them.

The device that makes this precise is to regard the sequence as a path. Interpolate the points linearly (or with any continuous interpolation) to get a continuous map $X : [0, T] \rightarrow \mathbb{R}^d$, a curve traced out in time. A time series is then a sampled path, and a kernel on sequences is a kernel on paths. The order in which the curve visits its points is intrinsic; the speed at which a pen draws a letter is not. A good path feature map should therefore be invariant to the speed, sensitive to the order, and rich enough to separate genuinely different shapes. The path signature, imported into statistics from the rough-path analysis of Lyons (1998), is exactly such a map, and it is in a strong sense the canonical one.

27.2 The path signature

The building block is the iterated integral. For a path X with coordinates $X_t = (X_t^1, \dots, X_t^d)$ and a word $i_1 \dots i_k$ over the alphabet $\{1, \dots, d\}$, integrate the coordinate increments in time order.

Definition (path signature)

The depth- k signature coordinate of X over $[a, b]$ indexed by the word $i_1 \dots i_k$ is the iterated integral

$$S(X)_{a,b}^{i_1 \dots i_k} = \int_{a < t_1 < t_2 < \dots < t_k < b} \dots \int dX_{t_1}^{i_1} dX_{t_2}^{i_2} \dots dX_{t_k}^{i_k}.$$

The signature is the whole collection, graded by level,

$$S(X)_{a,b} = (1, \mathbf{S}^1, \mathbf{S}^2, \mathbf{S}^3, \dots), \quad \mathbf{S}^k \in (\mathbb{R}^d)^{\otimes k},$$

where $\mathbf{S}^k$ is the level- k tensor collecting the d^k coordinates $S(X)^{i_1 \dots i_k}$, and the level-0 entry is the constant 1. The *depth- m* (truncated) signature $S^{\leq m}(X)$ keeps levels 0 through m .

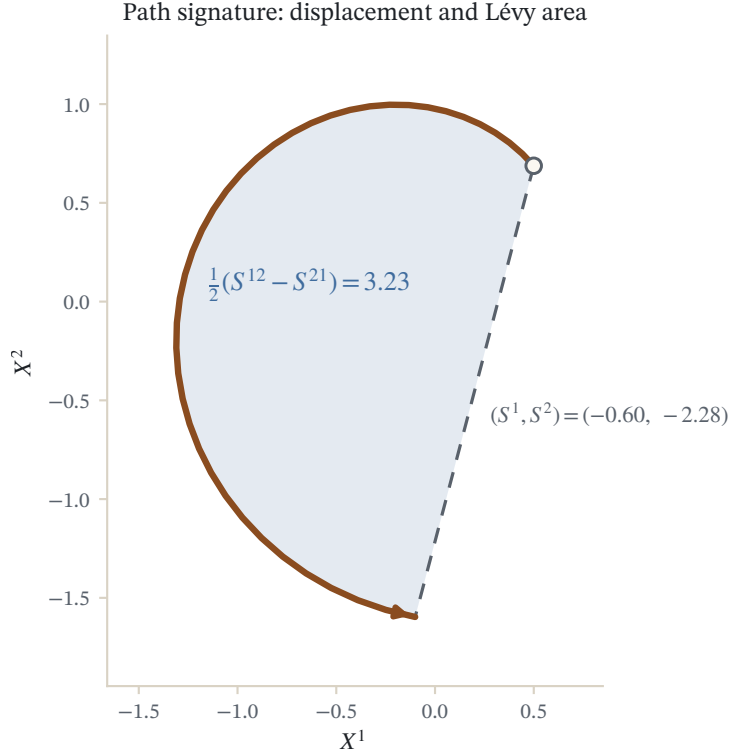


Figure 27.1: The depth-2 path signature of a stroke: level 1 is the displacement, and the genuinely new level-2 coordinate is the Lévy area $\frac{1}{2}(S^{12} - S^{21})$, the signed area (shaded) between the path and its chord. The signature depends only on the trace, not its parametrization.

The lowest levels already have plain meaning. Level 0 is always 1. Level 1 is the total increment,

$$S(X)_{a,b}^i = \int_a^b dX_t^i = X_b^i - X_a^i,$$

so the first level of the signature is just the displacement from start to end, the feature a stacked-difference model would use. Level 2 is where the signature starts to see structure a static summary cannot,

$$S(X)_{a,b}^{ij} = \int_a^b (X_t^i - X_a^i) dX_t^j,$$

the integral of one coordinate against the increments of another. Its antisymmetric part $\frac{1}{2}(S^{ij} - S^{ji})$ is the signed *Lévy area* swept between coordinates i and j , a measure of how the two channels lead and lag each other. Two paths with the same endpoints but opposite circulation are indistinguishable at level 1 and separated at level 2.

The truncated signature is a genuine finite-dimensional feature vector. Its dimension is

$$1 + d + d^2 + \cdots + d^m = \frac{d^{m+1} - 1}{d - 1},$$

which grows fast in the depth m but is fixed once m is chosen, so a learner can form it explicitly and hand it to any linear method. The remaining question is how to compute it, and the answer runs through the algebra of the signature.

27.2.1 One straight segment: the tensor exponential

The signature is easy to evaluate on a straight line, and every piecewise-linear path is a concatenation of straight lines, so the segment case is the whole computation in miniature. On a segment with constant increment Δ , parametrize $X_t = x_0 + t\Delta$ for $t \in [0, 1]$, so $dX_t = \Delta dt$. The level- k tensor pulls the constant $\Delta^{\otimes k}$ out of the integral and leaves the volume of the ordered simplex,

$$\mathbf{s}^k = \Delta^{\otimes k} \int_{0 < t_1 < \cdots < t_k < 1} \cdots \int dt_1 \cdots dt_k = \frac{\Delta^{\otimes k}}{k!}.$$

Summing over levels, the signature of a single segment is the *tensor exponential* of its increment,

$$S(\text{segment}) = \exp^{\otimes}(\Delta) := \sum_{k=0}^{\infty} \frac{\Delta^{\otimes k}}{k!} = \left(1, \Delta, \frac{\Delta^{\otimes 2}}{2}, \frac{\Delta^{\otimes 3}}{6}, \dots\right).$$

This closed form, together with the concatenation rule of the next section, is all we need to compute the signature of any discrete path.

27.3 Three structural properties

The signature earns its place as the canonical path feature map through three algebraic facts. Each is worth proving, because each corresponds to a modelling decision a practitioner cares about.

27.3.1 Invariance to reparametrization

The first property is the invariance we asked for at the outset: the signature does not depend on the speed at which the path is traversed.

Proposition (reparametrization invariance)

Let $\psi : [a, b] \rightarrow [a, b]$ be continuous, nondecreasing and surjective (a reparametrization), and let $\tilde{X}_t = X_{\psi(t)}$ be the reparametrized path. Then $S(\tilde{X})_{a,b} = S(X)_{a,b}$ at every level.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Fix a word $i_1 \cdots i_k$ and substitute $s_l = \psi(t_l)$ in each variable of the iterated integral for $\tilde{X}$. Because $\tilde{X}_t = X_{\psi(t)}$, the Riemann-Stieltjes increment satisfies $d\tilde{X}_t^i = dX_{\psi(t)}^i$, and because ψ is nondecreasing the ordered region $a < t_1 < \cdots < t_k < b$ maps onto the ordered region $a < s_1 < \cdots < s_k < b$. Hence

$$S(\tilde{X})^{i_1 \cdots i_k} = \int_{a < t_1 < \cdots < t_k < b} \cdots \int d\tilde{X}_{t_1}^{i_1} \cdots d\tilde{X}_{t_k}^{i_k} = \int_{a < s_1 < \cdots < s_k < b} \cdots \int dX_{s_1}^{i_1} \cdots dX_{s_k}^{i_k} = S(X)^{i_1 \cdots i_k}.$$

The change of variables introduces no Jacobian factor because the integrator is dX , not dt ; only the ordering of the sample times matters, and reparametrization preserves it. $\square$

The practical reading is that the signature sees the path as an oriented image with an order, not as a timetable. A sensor stream resampled onto a different grid, or a gesture performed slowly and then quickly, produces the same signature, so no explicit alignment or resampling step is needed before comparison. This is the continuous analogue of the string kernels' indifference to where in a document a pattern occurs.

27.3.2 Chen's identity: concatenation multiplies

The second property is what makes the signature computable. Splitting a path at an intermediate time turns the signature of the whole into a product of the signatures of the pieces, in the tensor algebra where multiplication is the tensor product $\otimes$.

Theorem (Chen's identity)

Let X run on $[a, b]$ and Y run on $[b, c]$ with $X_b = Y_b$, and let $X \star Y$ be their concatenation. Then

$$S(X \star Y)_{a,c} = S(X)_{a,b} \otimes S(Y)_{b,c},$$

that is, coordinate by coordinate,

$$S(X \star Y)_{a,c}^{i_1 \cdots i_n} = \sum_{k=0}^n S(X)_{a,b}^{i_1 \cdots i_k} S(Y)_{b,c}^{i_{k+1} \cdots i_n}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The level- n coordinate integrates over the ordered simplex $a < t_1 < \cdots < t_n < c$. Split that simplex by the number k of times that fall in the first piece: for some $0 \leq k \leq n$, the times $t_1, \dots, t_k$ lie in $[a, b]$ and $t_{k+1}, \dots, t_n$ lie in $[b, c]$. On the concatenation the integrator dX_t comes from X when $t \leq b$ and from Y when $t > b$, so the integral over the region with split point k factors as

$$\left(\int_{a < t_1 < \cdots < t_k < b} \cdots \int dX_{t_1}^{i_1} \cdots dX_{t_k}^{i_k} \right) \left(\int_{b < t_{k+1} < \cdots < t_n < c} \cdots \int dY_{t_{k+1}}^{i_{k+1}} \cdots dY_{t_n}^{i_n} \right) = S(X)^{i_1 \cdots i_k} S(Y)^{i_{k+1} \cdots i_n}.$$

Summing over the split point k gives the stated formula, which is precisely the level- n part of the tensor product $S(X) \otimes S(Y)$. $\square$

Chen's identity is the engine of every signature computation. A discrete path $x = (x_0, \dots, x_N)$, read as a piecewise-linear curve, is the concatenation of N straight segments with increments $\Delta_p = x_p - x_{p-1}$. Applying the theorem $N - 1$ times and the segment formula on each piece gives the signature as a product of tensor exponentials,

$$S(x) = \exp^{\otimes}(\Delta_1) \otimes \exp^{\otimes}(\Delta_2) \otimes \cdots \otimes \exp^{\otimes}(\Delta_N).$$

Truncating every factor and every product at depth m yields the truncated signature in a fixed number of tensor operations.

27.3.3 The shuffle identity

The third property constrains how signature coordinates multiply, and it is the reason linear models on signatures are expressive. The pointwise product of two signature coordinates is again a linear combination of signature coordinates, indexed by the shuffles of the two words.

Proposition (shuffle identity)

For words I, J , the product of signature coordinates satisfies

$$S^I S^J = \sum_{K \in \text{Sh}(I, J)} S^K,$$

where $\text{Sh}(I, J)$ is the multiset of interleavings of I and J that preserve the internal order of each. In particular, at level one, $S^i S^j = S^{ij} + S^{ji}$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof (level-one case)

Write $\Delta^i = X_b^i - X_a^i$ for the increments, so $S^i = \Delta^i$. By the product rule for the Stieltjes integral, $d[(X^i - X_a^i)(X^j - X_a^j)] = (X^i - X_a^i) dX^j + (X^j - X_a^j) dX^i$. Integrating from a to b , the left side telescopes to $(X_b^i - X_a^i)(X_b^j - X_a^j) = \Delta^i \Delta^j$, while the right side is exactly $S^{ij} + S^{ji}$. Hence $S^i S^j = S^{ij} + S^{ji}$, the shuffle of the two one-letter words. The general case iterates this integration by parts one letter at a time. $\square$

The shuffle identity says the linear span of signature coordinates is closed under multiplication: it is an algebra. Combined with reparametrization invariance and a point-separation argument, this makes linear functionals of the signature dense in the continuous functions on (unparametrized, tree-reduced) paths, a Stone-Weierstrass statement. The upshot for learning is that a *linear* model on a high-enough truncated signature can approximate any continuous function of the path, so the nonlinearity of the problem has been moved entirely into the fixed feature map, which is the same bargain the kernel trick strikes elsewhere in the book.

27.3.4 Uniqueness up to tree-like equivalence

How much of the path does the signature remember? Not the parametrization, by design. It also forgets any excursion that retraces itself, a segment walked out and immediately back, because the two directions of travel cancel in the integrals. These retracings are called tree-like pieces, and they are the only ambiguity.

Theorem (Hambly and Lyons, 2010)

Two paths of bounded variation have the same signature if and only if they are equal up to reparametrization and the insertion or deletion of tree-like (backtracking) pieces. Consequently the signature is a faithful invariant of the tree-reduced, unparametrized path.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The tree-like ambiguity is easy to remove in practice by making the path strictly monotone in one coordinate, which no backtracking can undo. The standard device is *time augmentation*: replace X_t by $\hat{X}_t = (t, X_t) \in \mathbb{R}^{d+1}$, appending the clock as an extra channel. The augmented path never retraces itself in the time coordinate, so its signature determines it exactly, while the added channel also lets the signature record at what time each event occurred, information a purely geometric feature map would discard. Chevyrev and Kormilitzin (2016) survey this and the other preprocessing choices (basepoint, lead-lag, cumulative sums) that turn a raw time series into a path whose signature is informative.

27.4 Computing the truncated signature

The segment formula and Chen's identity combine into a direct algorithm. Each segment contributes a tensor exponential, truncated at depth m ; the running product accumulates them in the truncated tensor algebra, where the level- n part of a product is the convolution $\sum_k (\cdot)_k \otimes (\cdot)_{n-k}$.

Algorithm (truncated signature of a discrete path)

Input points $x_0, \dots, x_N \in \mathbb{R}^d$; depth m .

Output truncated signature $S^{\leq m} = (1, \mathbf{S}^1, \dots, \mathbf{S}^m)$, with $\mathbf{S}^k \in (\mathbb{R}^d)^{\otimes k}$.

1. Initialise the running signature to the unit $A = (1, 0, \dots, 0)$.
2. For each segment $p = 1, \dots, N$: form the increment $\Delta_p = x_p - x_{p-1}$ and its truncated tensor exponential $E = (1, \Delta_p, \frac{\Delta_p^{\otimes 2}}{2}, \dots, \frac{\Delta_p^{\otimes m}}{m!})$.
3. Update by the truncated Chen product: for $n = m, m-1, \dots, 1$ set $A_n \leftarrow \sum_{k=0}^n A_k \otimes E_{n-k}$ (descending n allows the update in place).
4. Return A .

Each segment costs one tensor exponential and one Chen product, both $O(d^m)$ work at depth m , so the whole signature is linear in the number of points and polynomial in the dimension at fixed depth. The next worked example runs this by hand at depth two, the level where the signature first outperforms a static summary.

Example (depth-2 signature of two tiny paths, and their inner product)

Two paths in $\mathbb{R}^2$, each three points. $X : (0, 0) \rightarrow (1, 0) \rightarrow (1, 1)$, "right then up", increments $\Delta_1 = (1, 0)$, $\Delta_2 = (0, 1)$. $Y : (0, 0) \rightarrow (0, 1) \rightarrow (1, 1)$, "up then right", increments $\Gamma_1 = (0, 1)$, $\Gamma_2 = (1, 0)$. Both end at $(1, 1)$.

1. Level one is the displacement. Summing increments, $\mathbf{S}^1(X) = \Delta_1 + \Delta_2 = (1, 1)$ and $\mathbf{S}^1(Y) = \Gamma_1 + \Gamma_2 = (1, 1)$. At level one the two paths are identical.
2. Level two via Chen. The level-2 tensor is $\sum_p \frac{1}{2} \Delta_p^{\otimes 2} + \sum_{p < q} \Delta_p \otimes \Delta_q$. For X : $\frac{1}{2}(1, 0)^{\otimes 2} + \frac{1}{2}(0, 1)^{\otimes 2} + (1, 0) \otimes (0, 1)$, giving the matrix $\mathbf{S}^2(X) = \begin{pmatrix} \frac{1}{2} & 1 \\ 0 & \frac{1}{2} \end{pmatrix}$. For Y : $\mathbf{S}^2(Y) = \begin{pmatrix} \frac{1}{2} & 0 \\ 1 & \frac{1}{2} \end{pmatrix}$.
3. Read the Lévy area. The antisymmetric part $\frac{1}{2}(S^{12} - S^{21})$ is $+\frac{1}{2}$ for X and $-\frac{1}{2}$ for Y : the two paths circulate in opposite senses, and this is the only thing that distinguishes them below infinite depth.
4. Form the depth-2 kernel. Sum the level inner products: level 0 gives 1; level 1 gives $\langle (1, 1), (1, 1) \rangle = 2$; level 2 gives $\langle \mathbf{S}^2(X), \mathbf{S}^2(Y) \rangle = \frac{1}{2} \cdot \frac{1}{2} + 1 \cdot 0 + 0 \cdot 1 + \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{2}$. So $k^{\leq 2}(X, Y) = 1 + 2 + \frac{1}{2} = 3.5$.
5. Normalise. The self-kernels are $k^{\leq 2}(X, X) = k^{\leq 2}(Y, Y) = 1 + 2 + (\frac{1}{4} + 1 + 0 + \frac{1}{4}) = 4.5$, so the normalised kernel is $3.5/4.5 = 7/9 \approx 0.7778$.

Reading. The two paths agree in displacement (level-1 term 2, the full self-value) but the opposite areas pull the level-2 term down to $\frac{1}{2}$ instead of $\frac{3}{2}$, and the similarity drops from 1 to 7/9. A depth-1 signature kernel would have called them identical; depth 2 is the first that feels the orientation. The reproducing script also sums the full untruncated kernel, $\langle S(X), S(Y) \rangle \approx 3.5592$, whose higher levels add only 0.06 on top of the depth-2 value.

Verification artifact. checks/example-ch-signature-example-26-1.json records the example source hash and verification scope.

27.5 The truncated signature kernel

With an explicit feature map in hand, the kernel writes itself. The truncated signature kernel is the Euclidean inner product of the truncated signatures, level by level.

Definition (truncated signature kernel)

At depth m , the signature kernel of two paths is

$$k_{\text{sig}}^{\leq m}(X, Y) = \langle S^{\leq m}(X), S^{\leq m}(Y) \rangle = \sum_{k=0}^m \sum_{i_1, \dots, i_k=1}^d S(X)^{i_1 \dots i_k} S(Y)^{i_1 \dots i_k}.$$

Positive definiteness is immediate and needs no spectral argument: $S^{\leq m}$ is an explicit finite-dimensional feature map into $\mathbb{R}^{(d^{m+1}-1)/(d-1)}$, and $k_{\text{sig}}^{\leq m}$ is the ordinary dot product of feature vectors, hence positive definite by construction. This is the kernel for sequential data of Kiraly and Oberhauser (2019), who also show how to compute it without ever forming the exponentially long feature vector, by a Horner-type recursion over the sample points that costs $O(d m N_X N_Y)$ rather than $O(d^m)$. One can also raise the base inner product on $\mathbb{R}^d$ to any positive definite kernel κ before integrating, which replaces the coordinate products $dX^i dY^j$ by κ -inner-products and yields signatures of paths in a feature space, exactly the lift used to handle high-dimensional or structured channels. Toth and Oberhauser (2020) build Gaussian-process priors on sequences from this covariance, learning the depth and the per-level weights as kernel hyperparameters.

Truncation is a modelling knob with a cost. Deeper signatures separate finer differences in path shape, but the feature dimension grows geometrically, and for long paths the highest levels contribute little because the $1/k!$ in the segment exponential damps them. The natural question is whether the depth can be taken to infinity without paying the geometric price. It can, and the answer is a differential equation.

27.6 The untruncated signature kernel as a Goursat PDE

Send the depth to infinity and the signature kernel becomes an inner product of two infinite feature vectors,

$$k_{\text{sig}}(X, Y) = \langle S(X), S(Y) \rangle = \sum_{k=0}^{\infty} \langle \mathbf{s}^k(X), \mathbf{s}^k(Y) \rangle,$$

which converges for bounded-variation paths because the level- k term is bounded by $(L_X L_Y)^k / (k!)^2$ in the path lengths. Summing it term by term is hopeless, but Salvi, Cass, Foster, Lyons and Yang (2021) discovered that the sum is the value of a solution to a linear hyperbolic partial differential equation, so it can be obtained by a PDE solver rather than by enumerating tensor levels.

Theorem (Salvi, Cass, Foster, Lyons and Yang, 2021)

Let $X : [0, S] \rightarrow \mathbb{R}^d$ and $Y : [0, T] \rightarrow \mathbb{R}^d$ be continuously differentiable paths, and define the partial-kernel surface

$$U(s, t) = \langle S(X)_{0,s}, S(Y)_{0,t} \rangle.$$

Then U solves the Goursat problem

$$\frac{\partial^2 U}{\partial s \partial t} = \langle \dot{X}_s, \dot{Y}_t \rangle U(s, t), \quad U(s, 0) = U(0, t) = 1,$$

and the signature kernel is the corner value $k_{\text{sig}}(X, Y) = U(S, T)$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The mechanism is worth a sentence. Differentiating the partial signature $S(X)_{0,s}$ in s peels off its last integration and brings down a factor $\dot{X}_s$; doing the same in t for Y brings down $\dot{Y}_t$; and the two loose ends can only pair through

the inner product $\langle \dot{X}_s, \dot{Y}_t \rangle$, which multiplies the whole surface U . The boundary conditions record that an empty path has signature $(1, 0, 0, \dots)$, whose inner product with anything is the level-0 term 1. The equation is linear in U with a variable coefficient that is a simple inner product of velocities, so it is cheap to solve numerically.

For piecewise-linear paths the coefficient is piecewise constant. On the grid cell where X is on its p -th segment and Y on its q -th, the velocity inner product is the constant $\langle \Delta_p^X, \Delta_q^Y \rangle$, and a finite-difference scheme advances the surface cell by cell. Integrating the equation over one grid cell and approximating the right-hand side by the average of U at the four corners gives a second-order update.

Algorithm (finite-difference solver for the signature-kernel PDE)

Input increments $\Delta_1^X, \dots, \Delta_P^X$ and $\Delta_1^Y, \dots, \Delta_Q^Y$; refinement r (sub-steps per segment).

Output $k_{\text{sig}}(X, Y) \approx U(S, T)$.

1. Lay a grid of $Pr \times Qr$ cells with steps $h = 1/r$; set $U_{0,\cdot} = U_{\cdot,0} = 1$.
2. For each cell (i, j) , with segment indices $p = \lfloor i/r \rfloor$, $q = \lfloor j/r \rfloor$, set the local coefficient $a = \frac{1}{4} h^2 \langle \Delta_p^X, \Delta_q^Y \rangle$.
3. Advance the surface: $U_{i+1,j+1} = \frac{(1+a)(U_{i+1,j} + U_{i,j+1}) - (1-a)U_{i,j}}{1-a}$.
4. Return the corner $U_{Pr,Qr}$.

On the two tiny paths of the worked example above, refining this solver reproduces the full signature kernel to five digits: at $r = 128$ sub-steps per segment it returns $U(S, T) = 3.55917$, matching the deep-truncation value $\langle S(X), S(Y) \rangle = 3.55917$ computed by summing ten signature levels. The PDE has thus done the whole infinite sum without ever forming a tensor. On real problems the same solver runs on a coarse grid at each pair of sample times, so the kernel matrix of a dataset of long series costs a batch of small PDE solves, and Salvi and coauthors give the boundary-value tricks and GPU-parallel sweeps that make it scale.

27.7 Alignment kernels: warping done positively

The signature comes from analysis. A completely different and older tradition compares sequences by warping one onto the other, stretching and compressing the time axis to line up their features. This is the world of dynamic time warping (DTW), long the default similarity for speech and gesture. It is intuitive and often accurate, but as a kernel it has a fatal flaw, and understanding the flaw motivates the repair.

27.7.1 Why dynamic time warping is not a kernel

An *alignment* of two sequences $x = (x_1, \dots, x_n)$ and $y = (y_1, \dots, y_m)$ is a monotone path of matched index pairs from $(1, 1)$ to (n, m) , moving at each step down, right, or diagonally, so every entry of each sequence is matched to at least one entry of the other. Dynamic time warping picks the single cheapest alignment under a local cost, usually the squared distance,

$$\text{DTW}(x, y) = \min_{\pi \in \mathcal{A}(n, m)} \sum_{(i, j) \in \pi} \|x_i - y_j\|^2,$$

computed by the min-plus recursion $D_{ij} = \|x_i - y_j\|^2 + \min(D_{i-1,j}, D_{i,j-1}, D_{i,j-1})$. The value is a sensible discrepancy, and it is tempting to turn it into a similarity by $k(x, y) = e^{-\gamma \text{DTW}(x, y)}$. The trouble is that this is not positive definite, and the reason is structural, not numerical.

Dynamic time warping is not even a metric: the warping can align a repeated value to many, so two genuinely different sequences can sit at zero distance, and the triangle inequality fails. That failure is exactly what breaks positive definiteness, as a tiny computed example makes concrete.

A computed witness of indefiniteness

Take the length-3 sequences $s_1 = (0, 0, 3)$ and $s_3 = (0, 3, 3)$. They are different, yet dynamic time warping aligns the flat runs and reports $\text{DTW}(s_1, s_3) = 0$. Their distances to a third sequence $s_2 = (3, 0, 0)$ nevertheless disagree: $\text{DTW}(s_1, s_2) = 18$ while $\text{DTW}(s_3, s_2) = 27$. Form the DTW-kernel Gram matrix $G = e^{-\gamma \text{DTW}}$ on these three sequences at $\gamma = 0.1$:

$$G = \begin{pmatrix} 1 & 1 & 0.1653 \\ 1 & 1 & 0.0672 \\ 0.1653 & 0.0672 & 1 \end{pmatrix}, \quad \det G = -(0.1653 - 0.0672)^2 = -0.0096 < 0.$$

A symmetric 3×3 matrix with negative determinant has an odd number of negative eigenvalues, so G is indefinite; its spectrum is $\{-0.005, 0.978, 2.026\}$. Two rows are equal (both "identical" points at distance 0) while their third entries differ, which no genuine Gram matrix of feature vectors can do. On a five-sequence set the most negative eigenvalue reaches -0.098 . The similarity $e^{-\gamma \text{DTW}}$ is therefore not a kernel, and plugging it into an SVM voids the convexity and the RKHS geometry the earlier chapters rely on.

27.7.2 The global alignment kernel

The defect is the min. A minimum keeps only the single best alignment and discards the rest, and there is no feature map whose inner product is a minimum. Cuturi (2011) makes one change: replace the minimum over alignments by a sum, and replace the additive cost by a multiplicative similarity. Where DTW asks how good the best alignment is, the global alignment kernel asks how many good alignments there are.

Definition (global alignment kernel)

Fix a local similarity κ on $\mathbb{R}^d$, for instance $\kappa(u, v) = e^{-\phi(u, v)}$ with ϕ a divergence. The global alignment kernel sums, over every alignment, the product of the local similarities it matches:

$$k_{\text{GA}}(x, y) = \sum_{\pi \in \mathcal{A}(n, m)} \prod_{(i, j) \in \pi} \kappa(x_i, y_j).$$

The exponential sum over alignments collapses to a dynamic program of the very same shape as DTW, with the $(\min, +)$ semiring swapped for the $(+, \times)$ one. This is the soft, positively-weighted counterpart of the warping recursion, and it connects to the marginalization kernels, since summing a product over all latent alignments is exactly averaging a complete-data kernel over a hidden variable.

Algorithm (global alignment kernel, forward recursion)

Input sequences x of length n , y of length m ; local similarity κ .

Output $k_{\text{GA}}(x, y) = M_{n, m}$.

1. Initialise $M_{0,0} = 1$ and $M_{i,0} = M_{0,j} = 0$ for $i, j > 0$.
2. For $i = 1, \dots, n$ and $j = 1, \dots, m$, accumulate the three predecessors and weight by the local similarity:

$$M_{i,j} = \kappa(x_i, y_j) (M_{i-1,j} + M_{i-1,j-1} + M_{i,j-1}).$$

3. Return $M_{n,m}$.

Each cell does constant work, so the kernel costs $O(nm)$, the same as one DTW evaluation. Setting $\kappa \equiv 1$ makes $M_{n,m}$ count the alignments, which grows like a Delannoy number, so the recursion is genuinely summing an exponential family in polynomial time, the same telescoping that powered the string-kernel dynamic programs. We fill the table on a tiny instance.

Example (global alignment DP table for two short series)

Real sequences $x = (1, 2, 3)$ and $y = (1, 3)$, local similarity $\kappa(a, b) = e^{-(a-b)^2/2}$. The matrix of local similarities is $\kappa(1, 1) = 1$, $\kappa(1, 3) = e^{-2} = 0.1353$, $\kappa(2, 1) = \kappa(2, 3) = e^{-1/2} = 0.6065$, $\kappa(3, 1) = e^{-2} = 0.1353$, $\kappa(3, 3) = 1$. Shaded cells are the exact matches $x_i = y_j$.

M	ϵ	$y_1=1$	$y_2=3$
ϵ	1	0	0
$x_1=1$	0	1	0.135335
$x_2=2$	0	0.606531	1.056495
$x_3=3$	0	0.082085	1.745111

1. Seed the corner. $M_{1,1} = \kappa(1, 1)(M_{0,1} + M_{0,0} + M_{1,0}) = 1 \cdot (0 + 1 + 0) = 1$, the single alignment matching the two first entries.
2. Propagate the border. $M_{1,2} = \kappa(1, 3)M_{1,1} = e^{-2} = 0.1353$ and $M_{2,1} = \kappa(2, 1)M_{1,1} = e^{-1/2} = 0.6065$; each has only one reachable predecessor.
3. Sum three predecessors. $M_{2,2} = \kappa(2, 3)(M_{1,2} + M_{1,1} + M_{2,1}) = 0.6065(0.1353 + 1 + 0.6065) = 1.0565$, the soft sum over the three ways into cell $(2, 2)$.
4. Read the corner. $M_{3,2} = \kappa(3, 3)(M_{2,2} + M_{2,1} + M_{3,1}) = 1 \cdot (1.0565 + 0.6065 + 0.0821) = 1.745111 = k_{\text{GA}}(x, y)$.

Reading. The corner 1.7451 is the sum of the products of local similarities over all five alignments from $(1, 1)$ to $(3, 2)$; enumerating those five alignments and adding their products reproduces the same number, confirming that the recursion is a soft aggregation over alignments rather than the single best one. With $\kappa \equiv 1$ the corner would read 5, the count of alignments.

Verification artifact. checks/example-ch-signature-example-26-2.json records the example source hash and verification scope.

27.7.3 Why the global alignment kernel is positive definite

Turning the minimum into a sum is not just a smoothing; it is what restores positive definiteness. The GA kernel is a sum, over the discrete index set of alignments, of products of the local similarity, and both operations preserve positive definiteness when applied to positive definite building blocks. This is precisely the convolution-kernel pattern of Haussler (1999): decompose each object (here into an alignment of matched pairs), compare the parts with a base kernel, and sum the products of part comparisons over decompositions of the same shape. The sum-over-alignments in the GA definition is exactly such a convolution, so it inherits positive definiteness from the local kernel.

Theorem (Cuturi, 2011)

If the local similarity κ is such that $\frac{\kappa}{1 + \kappa}$ is a positive definite kernel on $\mathbb{R}^d$, then k_{GA} is a positive definite kernel on sequences.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The condition is met by a clean construction. Start from a Gaussian $g(u, v) = e^{-\|u-v\|^2/2\sigma^2}$, which is positive definite, and choose the local similarity

$$\kappa(u, v) = \frac{g(u, v)}{2 - g(u, v)}, \quad \text{so that} \quad \frac{\kappa}{1 + \kappa} = \frac{g}{2},$$

a scaled Gaussian, hence positive definite. Equivalently $\kappa = e^{-\phi_\sigma}$ with $\phi_\sigma(u, v) = \frac{1}{2\sigma^2}\|u-v\|^2 + \log(2 - e^{-\|u-v\|^2/2\sigma^2})$, the halved-Gaussian divergence Cuturi recommends. On the five-sequence set that made the DTW kernel

indefinite, the GA Gram built from this κ has smallest eigenvalue $+1.02$, positive definite as promised. The extra log term is the price of admissibility: it bends the plain Gaussian just enough that the geometric-series expansion $\kappa/(1+\kappa) = \kappa - \kappa^2 + \kappa^3 - \dots$ has the sign pattern a Gram matrix needs. The lesson generalises the DTW post-mortem: aggregate over structure with a sum of products of positive definite pieces, never with a minimum, and positive definiteness survives.

27.8 Reservoir and random-feature sequence kernels

The truncated signature is an explicit but geometrically large feature map, and the signature PDE avoids the map at the cost of a solve per pair. Between these lies the same Monte-Carlo idea that turned the Bochner integral into random Fourier features: approximate the expensive feature map by a handful of cheap random ones. Two related constructions do this for sequences.

The first is *random signature features*. Rather than store all d^m coordinates of the depth- m signature, project them onto a few random directions, or replace the tensor products by random tensor sketches; the resulting low-dimensional random feature has inner product close to the true signature kernel in expectation, and Toth and Oberhauser (2020) use exactly such random Fourier signature features to scale Gaussian processes with signature covariances to long series. The second is *reservoir computing*. Drive a fixed random recurrent dynamical system, a reservoir, with the input sequence and read out its state; because the state of a generic controlled system is a rich nonlinear functional of the driving path, its coordinates span a space close to that of the signature, and a linear readout on the reservoir state approximates a linear model on signature features. These randomized-signature and reservoir maps trade the exactness of the truncated kernel for a fixed, tunable feature budget, and they are the practical bridge from the elegant but expensive exact kernels to streaming, real-time sequence models. As always, the random feature is unbiased and its fluctuation shrinks like $1/\sqrt{D}$ in the number D of features, the same rate the random Fourier construction enjoys.

27.9 Summary

A sequence of vectors is best read as a sampled path, and the canonical feature map of a path is its signature, the graded collection of iterated integrals. Three facts make the signature the right object: it is invariant to reparametrization, so it compares shapes and not clocks; it multiplies under concatenation by Chen’s identity, so it is computed segment by segment as a product of tensor exponentials; and its coordinates satisfy the shuffle relations, so linear models on signatures are universal. The truncated signature kernel is the plain dot product of these finite feature vectors, positive definite by construction, and its infinite-depth limit is not a hopeless sum but the corner value of a Goursat PDE, solvable by a small finite-difference sweep that we checked reproduces the exact kernel. Warping-based similarity tells a cautionary tale: dynamic time warping takes a minimum over alignments, is not a metric, and yields indefinite Gram matrices, but Cuturi’s global alignment kernel replaces the minimum by a sum of products of a local kernel, a Haussler convolution kernel that is positive definite and computed by a DTW-shaped dynamic program. Random signature and reservoir features supply the scalable middle ground, the sequence-domain echo of random Fourier features. The next chapters carry these ideas onto generative models of sequences and onto data with richer geometry, always with the same reflex learned here and in the foundational chapters: build the similarity from positive definite parts, and the geometry takes care of itself.

27.10 Common mistakes and practical implications

For **Signature and Sequence-Path Kernels**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational,

report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

27.11 Summary and further reading

This chapter established explain the central definitions and claims in Signature and Sequence-Path Kernels; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Lyons 1998), (Chen 1958), (Hambly and Lyons 2010).

27.12 Exercises

1. warm-up A path in $\mathbb{R}^2$ visits $(0, 0) \rightarrow (2, 0) \rightarrow (2, 3)$. Compute its level-1 signature (the displacement) and its level-2 tensor $\mathbf{S}^2$ using $\sum_p \frac{1}{2} \Delta_p^{\otimes 2} + \sum_{p < q} \Delta_p \otimes \Delta_q$. Read off the Lévy area $\frac{1}{2}(S^{12} - S^{21})$ and check the shuffle relation $S^{12} + S^{21} = S^1 S^2$.

2. computation Reparametrize the path of Exercise 1 so it dwells three times as long on the first segment (walk $(0, 0) \rightarrow (2, 0)$ over $[0, 3]$, then $(2, 0) \rightarrow (2, 3)$ over $[3, 4]$). Recompute the depth-2 signature and confirm every coordinate is unchanged. In one sentence, say which feature of the raw time series this invariance throws away, and why that is usually the intended behaviour.

3. proof Prove Chen's identity at level two: for the concatenation $X \star Y$, show $S(X \star Y)^{ij} = S(X)^{ij} + S(X)^i S(Y)^j + S(Y)^{ij}$. Interpret the three terms as the three ways an ordered pair of times can fall relative to the join point. Hint

Split the region $\{t_1 < t_2\}$ by whether each of t_1, t_2 lies before or after the join b . The impossible case $t_1 > b > t_2$ is excluded by $t_1 < t_2$, leaving three cases matching $S(X)^{ij}$ (both before), $S(X)^i S(Y)^j$ (one each), and $S(Y)^{ij}$ (both after).

4. computation For the global alignment kernel with $\kappa(a, b) = e^{-(a-b)^2/2}$, fill the DP table for $x = (0, 1)$ and $y = (0, 0, 1)$ and read off $k_{\text{GA}}(x, y)$. Then set $\kappa \equiv 1$ and recompute the corner to count the alignments between a length-2 and a length-3 sequence. Hint

Use $M_{i,j} = \kappa(x_i, y_j)(M_{i-1,j} + M_{i-1,j-1} + M_{i,j-1})$ with $M_{0,0} = 1$ and zero borders. The local similarities you need are $\kappa(0, 0) = 1$, $\kappa(0, 1) = \kappa(1, 0) = e^{-1/2}$, $\kappa(1, 1) = 1$.

5. proof Show that the DTW recursion and the GA recursion are the same recursion evaluated in two different semirings: DTW uses $(\min, +)$ on costs, GA uses $(+, \times)$ on similarities with $\kappa = e^{-\text{cost}}$. Then argue that no similarity of the form "min over alignments" can be an inner product, and connect this to the indefiniteness witness in the text. Hint

Under $\kappa = e^{-c}$, a product of local similarities is $e^{-\sum c}$, so max of products corresponds to min of costs; the GA sum is the "soft-min" $-\log \sum e^{-c_\pi}$ up to the log. An inner product $\langle \phi(x), \phi(y) \rangle$ is bilinear and symmetric positive definite, none of which a pointwise minimum respects; the text's three sequences give an explicit indefinite 3×3 Gram.

6. proof Prove the global alignment kernel is positive definite under Cuturi's condition that $\kappa/(1 + \kappa)$ is positive definite, by exhibiting the resulting expansion as a Haussler convolution kernel. Identify the decomposition structure R (each object splits into an alignment of matched single-point parts) and the type match that forces two decompositions to have the same shape before their local kernels are multiplied. Hint

Recall from the convolution-kernel section that $\kappa_R(x, z) = \sum_{\bar{x}, \bar{z}} [T(\bar{x}) = T(\bar{z})] \prod_i \kappa_i(x_i, z_i)$ is positive definite whenever each κ_i is. Take the parts to be the matched pairs along an alignment and $\kappa_i = \kappa$; the sum over equal-shape decompositions is exactly k_{GA} . Cuturi's weaker condition on $\kappa/(1 + \kappa)$ refines this to allow local similarities that are not themselves positive definite.

7. exploration The signature PDE solver in the text converges to the exact kernel as the grid refines. Using the reproducing script, tabulate the finite-difference corner value at refinements $r = 1, 2, 4, \dots$ and confirm it approaches the deep-truncation value 3.55917 at second-order rate (the error roughly quartering when r doubles). For this particular path pair, verify separately that the truncated signature kernel approaches the same limit from below as depth grows. Explain why monotonicity is automatic for self-kernels but is not guaranteed for a general cross-kernel, whose per-level inner products can have either sign.
8. challenge Derive the signature-kernel Goursat PDE. Writing $U(s, t) = \langle S(X)_{0,s}, S(Y)_{0,t} \rangle$, differentiate under the sum over levels and use that $\partial_s S(X)_{0,s}^{i_1 \dots i_k} = S(X)_{0,s}^{i_1 \dots i_{k-1}} \dot{X}_s^{i_k}$, the fundamental theorem applied to the outermost integral. Show the mixed derivative $\partial_s \partial_t U$ reassembles into $\langle \dot{X}_s, \dot{Y}_t \rangle U(s, t)$, and identify the boundary values from the signature of the empty path. Hint

Differentiating in s drops the level of the X factor by one and attaches $\dot{X}_s^{i_k}$; differentiating that in t does the same to Y and attaches $\dot{Y}_t^{i_k}$. Summing over the shared last index i_k contracts the two velocities into $\langle \dot{X}_s, \dot{Y}_t \rangle$, while the remaining sum over lower levels rebuilds $U(s, t)$. The empty path has signature $(1, 0, 0, \dots)$, giving $U(s, 0) = U(0, t) = 1$.

28 Geometric and Equivariant Kernels

Every kernel built so far has assumed its data live in $\mathbb{R}^d$ or in a plain discrete set, and its similarity has read differences $x - y$ or Euclidean distances $\|x - y\|$. But much data is born on a curved or symmetric domain: wind directions on a sphere, molecular orientations in the rotation group, gene activity on a biological network, or the pixels of an image that no rotation should relabel. On such domains the flat notions of distance and shift are simply wrong, and substituting a geodesic distance into a Gaussian usually fails to be a kernel at all. This chapter builds kernels that respect the geometry. Two ideas carry the whole story. The first replaces the Fourier basis of the translation-invariant families by the eigenbasis of the domain's own Laplace operator, so that the heat kernel and the Matern family become solutions of a stochastic differential equation on a manifold or a graph. The second averages an ordinary kernel over the orbit of a group action, turning a symmetry into a hard constraint that every function in the resulting space must obey. Positive definiteness, in both constructions, is a statement about a positive spectrum.

28.1 Why the domain's geometry matters

The kernels of the earlier chapters encode a prior about smoothness through the frequencies they trust: Bochner's theorem writes a stationary kernel on $\mathbb{R}^d$ as a positive mixture of plane waves $\cos(\omega^\top(x - y))$, and the reciprocal-density penalty of the spectral view starves the high frequencies. None of this survives a naive move to a curved domain. There are no global plane waves on a sphere, no difference vector $x - y$ between two vertices of a graph, and the shortest-path distance on a graph need not even embed in a Hilbert space, so $\exp(-t d_{\text{graph}})$ can have a negative eigenvalue and fail to be a kernel, as the chapter on graphs shows by example.

What does transfer is the Laplacian. Every Riemannian manifold carries a canonical second-order operator, the Laplace-Beltrami operator, and every weighted graph carries its combinatorial Laplacian; both measure how much a function bends, both are self-adjoint and positive semidefinite, and both come with an orthonormal eigenbasis that plays the role of sines and cosines on the domain. Kernels built as spectral filters of that operator inherit positive definiteness for free and read smoothness in exactly the way the flat kernels did. That is the first half of the chapter. The second half takes the complementary route: rather than change the domain, it makes an ordinary kernel blind to a group of transformations by averaging over the group, a construction that connects directly to the invariances of the pre-image chapter and to the equivariant architectures of modern deep learning.

28.2 The Matern family as a stochastic differential equation

To carry the Matern kernel of Chapter 19, Translation-Invariant, Semigroup, and Probabilistic Kernels onto a manifold we need a description of it that mentions no coordinates. Its distance formula, an awkward product of a power of $r = \|x - y\|$ with a Bessel function, does not generalize, and its Bochner spectral density $(2\nu/\kappa^2 + \|\omega\|^2)^{-(\nu+d/2)}$ names frequencies ω that a manifold does not possess. The description that does transfer is a differential equation. Whittle (1954) observed, and Lindgren, Rue, and Lindstrom (2011) turned into a computational program, that the Matern field is the stationary solution of a stochastic partial differential equation driven by white noise.

Definition (Matern SPDE on $\mathbb{R}^d$)

Let W be Gaussian white noise on $\mathbb{R}^d$, let Δ be the Laplacian, and fix a smoothness $\nu > 0$ and length scale $\kappa > 0$. The Matern field is the stationary solution u of

$$\left(\frac{2\nu}{\kappa^2} - \Delta\right)^{(\nu+d/2)/2} u = W.$$

The reason this recovers the Matern kernel is a one-line Fourier computation. The operator $2\nu/\kappa^2 - \Delta$ acts on the plane wave $e^{i\omega^\top x}$ by multiplication by its symbol $2\nu/\kappa^2 + \|\omega\|^2$, since $-\Delta$ has symbol $\|\omega\|^2$. Raising the operator to the power $(\nu + d/2)/2$ raises the symbol to that power, so inverting it to solve for $u = (2\nu/\kappa^2 - \Delta)^{-(\nu+d/2)/2} W$ multiplies the flat spectrum of the white noise by $(2\nu/\kappa^2 + \|\omega\|^2)^{-(\nu+d/2)/2}$. The covariance of u is the squared modulus of that filter, namely $(2\nu/\kappa^2 + \|\omega\|^2)^{-(\nu+d/2)}$, which is precisely the Matern spectral density. The kernel is thus the covariance operator $(2\nu/\kappa^2 - \Delta)^{-(\nu+d/2)}$, an object assembled entirely from the Laplacian.

This is the pivot of the whole first half. The Laplacian is not special to $\mathbb{R}^d$: a Riemannian manifold has the Laplace-Beltrami operator, and a graph has its Laplacian. Replacing Δ by the domain's own operator transports the Matern kernel, and with it the heat kernel obtained in the smooth limit, to any geometry that has a Laplacian. Borovitskiy, Terenin, Mostowsky, and Deisenroth (2020) carried this out on manifolds, and Borovitskiy et al. (2021) on graphs.

28.3 Kernels on a Riemannian manifold

28.3.1 The Laplace-Beltrami operator and its spectrum

On a compact Riemannian manifold $\mathcal{M}$ the Laplace-Beltrami operator $\Delta_{\mathcal{M}}$ generalizes the flat Δ : it is the divergence of the gradient measured in the manifold's metric. It is self-adjoint and negative semidefinite on $L^2(\mathcal{M})$, and on a compact manifold its spectrum is discrete, a sequence of eigenpairs

$$-\Delta_{\mathcal{M}} f_n = \lambda_n f_n, \quad 0 = \lambda_0 \leq \lambda_1 \leq \lambda_2 \leq \dots \rightarrow \infty,$$

with the eigenfunctions $\{f_n\}$ forming an orthonormal basis of $L^2(\mathcal{M})$. These eigenfunctions are the manifold's Fourier basis, and the eigenvalues λ_n are its squared frequencies: a large λ_n means f_n oscillates rapidly across $\mathcal{M}$. On the circle S^1 they are the familiar $e^{in\theta}$ with $\lambda_n = n^2$; on the sphere S^2 they are the spherical harmonics with $\lambda_\ell = \ell(\ell + 1)$.

The discrete Laplacian converges to $\Delta_{\mathcal{M}}$

The connection to graphs is not an analogy but a limit. Belkin and Niyogi (2003) showed that if points are sampled from a manifold and joined into a weighted neighborhood graph with Gaussian edge weights, then the graph Laplacian converges, as the sample grows and the neighborhood shrinks, to the Laplace-Beltrami operator, and the graph's eigenvectors converge to the eigenfunctions f_n . This is the theoretical footing of Laplacian eigenmaps and of the spectral embeddings in the visualization chapter, and it is why the graph construction below is the faithful discrete shadow of the manifold one.

28.3.2 The manifold Matern and heat kernels

With the eigenpairs in hand the transported kernel writes itself. The covariance operator $(2\nu/\kappa^2 - \Delta_{\mathcal{M}})^{-(\nu+d/2)}$ is diagonal in the eigenbasis, acting on f_n by the scalar $(2\nu/\kappa^2 + \lambda_n)^{-(\nu+d/2)}$, so its kernel is the Mercer expansion of those eigenfunctions weighted by that spectral filter.

Definition (Matern and heat kernels on a manifold, Borovitskiy et al. 2020)

On a compact manifold $\mathcal{M}$ of dimension d , with Laplace-Beltrami eigenpairs (λ_n, f_n) , the Matern kernel of smoothness ν and length scale κ is

$$k_\nu(x, y) = \frac{1}{C_\nu} \sum_{n=0}^{\infty} \left(\frac{2\nu}{\kappa^2} + \lambda_n \right)^{-(\nu+d/2)} f_n(x) f_n(y),$$

and the heat (diffusion) kernel is its smooth limit

$$k_\infty(x, y) = \frac{1}{C_\infty} \sum_{n=0}^{\infty} e^{-\kappa^2 \lambda_n / 2} f_n(x) f_n(y).$$

The constant $C_\nu = \frac{1}{|\mathcal{M}|} \sum_n \Phi_\nu(\lambda_n)$ normalizes the average marginal variance to one. In practice the sum is truncated to the first J eigenpairs.

Positive definiteness is immediate from the form of the expansion, and the argument is worth stating because it recurs throughout the chapter: a spectral filter with nonnegative values produces a positive definite kernel.

Positive definiteness of a spectral filter

Let (λ_n, f_n) be the eigenpairs of a self-adjoint operator and let $\Phi(\lambda) \geq 0$ on the spectrum. Then $k(x, y) = \sum_n \Phi(\lambda_n) f_n(x) f_n(y)$ is a positive definite kernel, and so is every finite truncation.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Each term $f_n(x)f_n(y)$ is the rank-one kernel of the feature map $x \mapsto f_n(x)$, hence positive definite, since for any points $x_1, \dots, x_m$ and scalars $c_1, \dots, c_m$, $\sum_{i,j} c_i c_j f_n(x_i) f_n(x_j) = \left(\sum_i c_i f_n(x_i) \right)^2 \geq 0$. The kernel k is the combination $\sum_n \Phi(\lambda_n) f_n(x) f_n(y)$ with nonnegative coefficients $\Phi(\lambda_n) \geq 0$, and a nonnegative combination of positive definite kernels is positive definite, as is its pointwise limit. A truncation keeps a subset of the same nonnegative terms, so it is positive definite too. $\square$

Because the Matern filter $\Phi_\nu(\lambda) = (2\nu/\kappa^2 + \lambda)^{-(\nu+d/2)}$ and the heat filter $\Phi_\infty(\lambda) = e^{-\kappa^2 \lambda / 2}$ are strictly positive, both kernels are positive definite on any compact manifold. The heat kernel is genuinely the limit of the Matern family: writing the normalized Matern filter as $(1 + \frac{\kappa^2}{2\nu} \lambda)^{-\nu}$ and letting $\nu \rightarrow \infty$ with κ fixed gives $e^{-\kappa^2 \lambda / 2}$, so the smoothness parameter ν dials continuously from a rough field to the infinitely smooth diffusion field, exactly as it interpolated Laplace and Gaussian on $\mathbb{R}^d$.

28.4 Kernels on a weighted graph

A weighted graph is the manifold's discrete sibling, and the construction is identical with the graph Laplacian in place of $-\Delta_{\mathcal{M}}$. Let G have N nodes, symmetric nonnegative edge weights W_{ij} , degree matrix $D = \text{diag}(\sum_j W_{ij})$, and Laplacian $L = D - W$. As the graph chapter establishes, L is symmetric positive semidefinite, its eigenpairs (λ_i, u_i) are the graph's Fourier modes, and the constant vector spans the null space of a connected graph. We build kernels as spectral filters of L , and because a graph has no ambient dimension d , the exponent that was $\nu + d/2$ on a manifold is taken to be the free smoothness ν , the convention of Borovitskiy et al. (2021) and of the frontier chapter.

Definition (graph Matern and diffusion kernels)

With graph Laplacian $L = \sum_i \lambda_i u_i u_i^\top$, the graph Matern kernel of smoothness ν and length scale κ is

$$K_\nu = \frac{1}{C_\nu} \left(\frac{2\nu}{\kappa^2} I + L \right)^{-\nu} = \frac{1}{C_\nu} \sum_i \left(\frac{2\nu}{\kappa^2} + \lambda_i \right)^{-\nu} u_i u_i^\top,$$

and the diffusion (heat) kernel is

$$K_\infty = \frac{1}{C_\infty} \exp\left(-\frac{\kappa^2}{2} L\right) = \frac{1}{C_\infty} \sum_i e^{-\kappa^2 \lambda_i / 2} u_i u_i^\top.$$

The diffusion kernel here is exactly the kernel of Kondor and Lafferty (2002), which Chapter 24, Kernels for and on Graphs derived by solving the graph heat equation $\dot{f} = -Lf$; we have now recovered it a second way, as the $\nu \rightarrow \infty$ endpoint of a family with a tunable number of derivatives. Its positive definiteness deserves a direct statement, since it is the workhorse fact of the discrete theory.

The heat kernel is positive definite

For every $t > 0$ the matrix $\exp(-tL)$ is symmetric positive definite.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Diagonalize the symmetric Laplacian, $L = \sum_i \lambda_i u_i u_i^\top$ with an orthonormal eigenbasis and $\lambda_i \geq 0$ by positive semidefiniteness. The matrix exponential acts on each eigenvector by the scalar exponential, so

$$\exp(-tL) = \sum_i e^{-t\lambda_i} u_i u_i^\top,$$

which is symmetric and has eigenvalues $e^{-t\lambda_i} > 0$. A symmetric matrix with strictly positive eigenvalues is positive definite. The same computation with any strictly positive filter $\Phi(\lambda_i) > 0$ in place of $e^{-t\lambda_i}$, in particular $(2\nu/\kappa^2 + \lambda_i)^{-\nu}$, proves the graph Matern kernel positive definite as well. $\square$

These kernels are the tunable-smoothness completion of the family in Chapter 24, Kernels for and on Graphs. The Laplacian pseudo-inverse $L^\dagger$ is the filter $\Phi(\lambda) = 1/\lambda$ on the nonzero spectrum, and the regularized Laplacian $(L + \epsilon I)^{-1}$ is $\Phi(\lambda) = 1/(\lambda + \epsilon)$; the Matern filter $(2\nu/\kappa^2 + \lambda)^{-\nu}$ generalizes both, with ν setting how fast high-frequency modes are suppressed. The reading is identical to the reciprocal-density penalty of Chapter 19, Translation-Invariant, Semigroup, and Probabilistic Kernels: the RKHS norm charges a mode of frequency λ_i by $1/\Phi(\lambda_i)$, so a small filter value at a rough mode makes that mode expensive, and the kernel enforces smoothness on the graph exactly as its flat cousin did in the Fourier domain. Feeding these kernels into the Gaussian-process machinery of Chapter 40, Gaussian Processes and the RVM gives calibrated regression on manifolds and networks.

Algorithm (graph Matern / diffusion kernel from the Laplacian)

Input Weighted adjacency W ; smoothness $\nu \in (0, \infty]$; length scale κ ; (optional) truncation $J \leq N$.

Output Gram matrix K of a positive definite kernel on the graph's nodes.

1. Form the Laplacian $L = D - W$ with $D = \text{diag}(W\mathbf{1})$.
2. Eigendecompose $L = \sum_i \lambda_i u_i u_i^\top$ (a symmetric eigensolver; keep the J smallest λ_i if truncating).
3. Choose the spectral filter: $\Phi(\lambda) = (2\nu/\kappa^2 + \lambda)^{-\nu}$ for Matern, or $\Phi(\lambda) = e^{-\kappa^2 \lambda/2}$ for diffusion.
4. Assemble $K = \sum_i \Phi(\lambda_i) u_i u_i^\top$; every eigenvalue $\Phi(\lambda_i) > 0$, so K is positive definite.
5. Normalize by $C = \frac{1}{N} \sum_i \Phi(\lambda_i)$ so the average diagonal is one, giving unit marginal variance.

Example (Matern and diffusion kernels on the 5-cycle)

The graph is the cycle C_5 on nodes 0, 1, 2, 3, 4, the discrete circle. Laplacian $L = 2I - A$ with A the cycle adjacency. Matern uses $\nu = 1$, $\kappa^2 = 1$, so the filter is $(2 + \lambda)^{-1}$; diffusion uses $t = \kappa^2/2 = 1$. All numbers from checks/ch-geom-ex1.py.

1. Read off the spectrum. The Laplacian eigenvalues are the discrete Fourier frequencies of the cycle, $\lambda = (0, 1.382, 1.382, 3.618, 3.618)$, with $1.382 = 2 - 2 \cos 72^\circ$ and $3.618 = 2 - 2 \cos 144^\circ$.
2. Apply the Matern filter. With $\nu = 1, \kappa^2 = 1$, $K_1 = (2I + L)^{-1}$ is circulant (the kernel is stationary on the cycle), each row a shift of

$$(0.2895, 0.0789, 0.0263, 0.0263, 0.0789),$$

so similarity decays with graph distance. Normalizing to unit diagonal gives $(1, 0.2727, 0.0909, 0.0909, 0.2727)$.

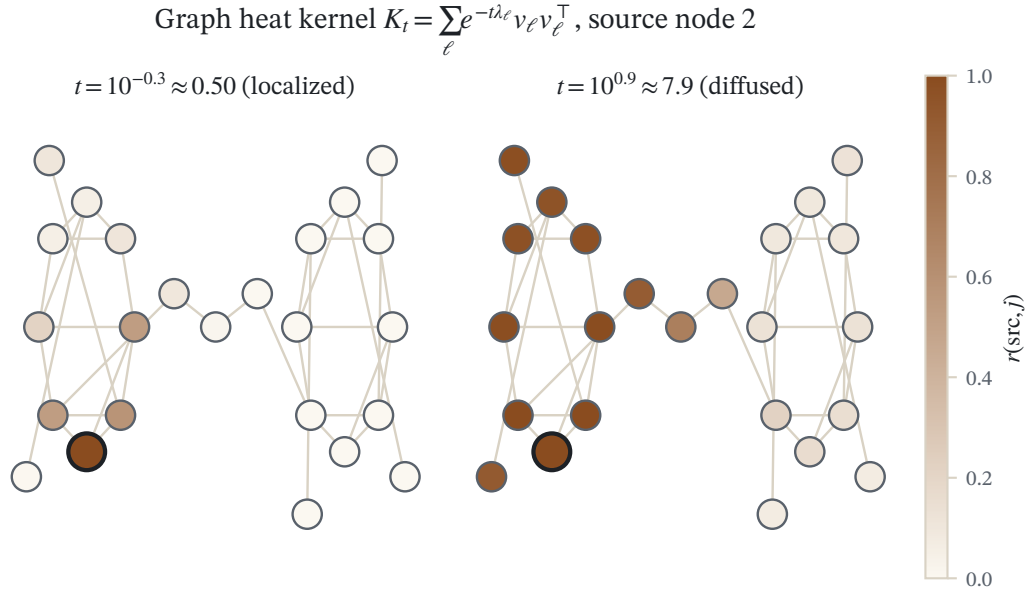


Figure 28.1: Node colors are the graph heat kernel $r(i, j)$ built from the eigenpairs of the Laplacian, for a fixed source node. A small diffusion time keeps similarity inside the source’s cluster; a larger time spreads it across the bridge.

3. Check positive-definiteness. Its eigenvalues are $1/(2 + \lambda_i) = (0.5, 0.2957, 0.2957, 0.178, 0.178)$, all strictly positive, so $K_1 > 0$.
4. Apply the diffusion filter. $K_{\infty} = e^{-L}$ is again circulant with first row $(0.3112, 0.2224, 0.1221, 0.1221, 0.2224)$ and eigenvalues $e^{-\lambda_i} = (1, 0.2511, 0.2511, 0.0268, 0.0268)$, all positive.
5. Watch the limit. The normalized Matern filter $(1 + \frac{1}{\nu}\lambda)^{-\nu}$ (taking $\kappa^2 = 2$ so that $\kappa^2/2 = 1 = t$) approaches $e^{-\lambda}$: the largest entrywise gap $\max|K_{\nu} - e^{-L}|$ falls as 0.0794 at $\nu = 2$, 0.0167 at $\nu = 10$, 0.0033 at $\nu = 50$.

Reading. Both kernels come straight from the Laplacian spectrum, are stationary on the discrete circle because C_5 is vertex-transitive, and are positive definite for free because their spectral filters are positive. The diffusion kernel is visibly the smooth $\nu \rightarrow \infty$ endpoint of the Matern family.

Verification artifact. checks/example-ch-geom-example-27-1.json records the example source hash and verification scope.

28.5 Lie groups and homogeneous spaces

Some domains are not just curved but are themselves groups, or quotients of groups. The rotations of three-dimensional space form the Lie group $SO(3)$, a natural home for orientations and poses; the sphere S^2 is the homogeneous space $SO(3)/SO(2)$, the orbit of a point under rotation. On such domains the spectral construction still applies, and representation theory hands us the eigenpairs in closed form. The Peter-Weyl theorem decomposes L^2 of a compact group into the matrix coefficients of its irreducible representations, and these matrix coefficients are exactly the eigenfunctions of the Laplace-Beltrami operator, whose eigenvalues are read off the Casimir element of each representation. Azangulov et al. (2022) work this out for compact Lie groups and their homogeneous spaces, and Azangulov et al. (2023) extend it to non-compact symmetric spaces such as the hyperbolic plane, where the discrete sum over eigenfunctions becomes an integral over a continuous spectrum.

The payoff is conceptual as well as computational. A Matern kernel built from the group’s own Laplacian is automatically stationary in the group’s sense: it is invariant under the group acting on itself, so $k(gx, gy) = k(x, y)$

for every group element g . Stationarity on a symmetric space is thus the same statement as invariance under the symmetry group, which is the bridge to the second half of the chapter. There, instead of assuming the domain is a group, we take an arbitrary kernel on an arbitrary space and force a chosen invariance by hand.

28.6 Group-invariant kernels by averaging

A recurring wish in the invariance chapter was for a classifier that ignores a known symmetry: a digit is the same digit after a small rotation, an object the same object after a translation. That chapter met the wish softly, by generating transformed training examples or by penalizing variation along tangent directions. Here we meet it as a hard constraint on the kernel itself. If a group G acts on the input space $\mathcal{X}$ and we want a kernel that cannot tell x from any transform gx , we take an ordinary base kernel and average it over the group orbit. The construction is due to Haasdonk and Burkhart (2007).

Let G be a compact group acting on $\mathcal{X}$, with normalized Haar measure dg , the unique probability measure that is invariant under the group's own left and right multiplication (a finite group simply averages uniformly over its elements). Given a base kernel k with feature map Φ , define the group-averaged feature map by integrating the feature over the orbit,

$$\bar{\Phi}(x) = \int_G \Phi(gx) dg,$$

a Bochner integral in the feature space. Its inner product is the averaged kernel.

Definition (group-averaged kernel)

The kernel obtained by averaging k over the group action is

$$k_G(x, y) = \langle \bar{\Phi}(x), \bar{\Phi}(y) \rangle = \int_G \int_G k(gx, g'y) dg dg'.$$

When the base kernel is already jointly invariant, $k(gx, gy) = k(x, y)$ for all g , the double average collapses to a single orbit average,

$$k_G(x, y) = \int_G k(x, gy) dg.$$

The collapse is a change of variables: substituting $g' \mapsto gg''$ in the inner integral and using joint invariance and the invariance of Haar measure turns $\int \int k(gx, g'y) dg dg'$ into $\int k(x, g''y) dg''$. The single-average form is the one to use in practice, since it evaluates $|G|$ base kernels rather than $|G|^2$. Two properties make the construction exactly what we wanted.

Averaging preserves positive definiteness

If k is positive definite, then k_G is positive definite.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

By the definition through the averaged feature map, for any points $x_1, \dots, x_m$ and scalars $c_1, \dots, c_m$,

$$\sum_{i,j} c_i c_j k_G(x_i, x_j) = \sum_{i,j} c_i c_j \langle \bar{\Phi}(x_i), \bar{\Phi}(x_j) \rangle = \left\| \sum_i c_i \bar{\Phi}(x_i) \right\|^2 \geq 0.$$

Equivalently, $\sum_{i,j} c_i c_j k_G(x_i, x_j) = \int_G \int_G (\sum_{i,j} c_i c_j k(gx_i, g'x_j)) dg dg'$, an average of the nonnegative quadratic forms of the positive definite k , hence nonnegative. $\square$

The averaged kernel is invariant

For every $h \in G$, $k_G(hx, y) = k_G(x, y)$ and $k_G(hx, h'y) = k_G(x, y)$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The averaged feature map is constant on orbits. Using the invariance of Haar measure under the substitution $g \mapsto gh^{-1}$,

$$\bar{\Phi}(hx) = \int_G \Phi(g hx) dg = \int_G \Phi(g'' x) dg'' = \bar{\Phi}(x).$$

Therefore $k_G(hx, y) = \langle \bar{\Phi}(hx), \bar{\Phi}(y) \rangle = \langle \bar{\Phi}(x), \bar{\Phi}(y) \rangle = k_G(x, y)$, and applying the same fact in the second argument gives joint invariance $k_G(hx, h'y) = k_G(x, y)$. $\square$

The invariance is a hard constraint, not a preference, and this is the essential difference from data augmentation and tangent penalties. Because $\bar{\Phi}(gx) = \bar{\Phi}(x)$, the feature map factors through the orbit space $\mathcal{X}/G$, so every function in the RKHS of k_G is exactly G -invariant: for f in the space, $f(gx) = \langle f, \bar{\Phi}(gx) \rangle = \langle f, \bar{\Phi}(x) \rangle = f(x)$. The hypothesis space contains no function that violates the symmetry, whereas augmentation only nudges the fitted function toward invariance and leaves violating functions in reach. This is the kernel counterpart of the weight sharing that hard-wires equivariance into convolutional and group-equivariant networks (Cohen and Welling 2016): a convolution is equivariant by construction, and a global pooling over the group is precisely the orbit average that turns an equivariant representation into an invariant one, the same integral $\int_G dg$ that defines k_G . The stability-to-deformation analysis of Bietti and Mairal (2019) makes the connection between such invariant kernels and deep architectures precise, and the deep-learning chapter returns to it.

Algorithm (group-averaging a kernel to enforce invariance)

Input Base positive definite kernel k ; group G (finite, or a Haar sample $g_1, \dots, g_S$); points $x_1, \dots, x_m$.

Output Invariant positive definite Gram matrix K_G .

1. For each ordered pair (i, j) and each group element $g \in G$, evaluate the transformed base kernel $k(x_i, g x_j)$.
2. Average over the group: $(K_G)_{ij} = \frac{1}{|G|} \sum_{g \in G} k(x_i, g x_j)$ (for a continuous group, average over the Haar sample).
3. If the base kernel is not jointly invariant, use the double average $\frac{1}{|G|^2} \sum_{g, g'} k(gx_i, g'x_j)$, equivalently build $\bar{\Phi}(x_i) = \frac{1}{|G|} \sum_g \Phi(gx_i)$ and take inner products.
4. Return K_G ; it is symmetric, positive definite, and invariant to the action of G .

Example (an RBF kernel made invariant to $\mathbb{Z}_4$ rotations)

The group $G = \mathbb{Z}_4$ acts on $\mathbb{R}^2$ by rotations of 0, 90, 180, 270 degrees, matrices $R_0, \dots, R_3$. Base kernel is the RBF $k(x, y) = e^{-\|x-y\|^2/2}$, which is jointly invariant since rotations preserve distance. Points $x_1 = (1, 0.4)$, $x_2 = (0.3, 0.6)$, $x_3 = (-0.7, 0.5)$. All numbers from checks/ch-geom-ex2.py.

1. Average over the orbit. For the pair (x_1, x_2) the four transformed base kernels are $k(x_1, R_j x_2) = (0.7672, 0.2767, 0.2605, 0.7225)$, whose mean is $k_G(x_1, x_2) = 0.5067$.

2. Assemble the invariant Gram. Repeating for every pair,

$$K_G = \begin{pmatrix} 0.4313 & 0.5067 & 0.4733 \\ 0.5067 & 0.6705 & 0.5987 \\ 0.4733 & 0.5987 & 0.5455 \end{pmatrix},$$

symmetric to machine precision.

3. Check positive-definiteness. Its eigenvalues are (0.0013, 0.0302, 1.6157), all nonnegative, so $K_G \geq 0$.
4. Verify invariance. Rotate x_1 by 90° . The raw RBF changes, $k(R_1 x_1, x_2) = 0.7225 \neq 0.7672 = k(x_1, x_2)$, but the averaged kernel does not: $k_G(R_1 x_1, x_2) = 0.506731 = k_G(x_1, x_2)$, and jointly $k_G(R_1 x_1, R_1 x_3) = 0.473287 = k_G(x_1, x_3)$, equal to twelve decimals.

Reading. Averaging the base kernel over the four-element orbit produces a positive definite kernel that is blind to the rotation, a symmetry the raw RBF plainly does not respect. The invariance is exact, not approximate, because it is built into the feature map rather than encouraged by extra data.

Verification artifact. `checks/example-ch-geom-example-27-2.json` records the example source hash and verification scope.

28.7 Summary

Two constructions carry a kernel onto a geometric domain. The spectral route reads the domain's Laplacian, the Laplace-Beltrami operator on a manifold or the combinatorial Laplacian on a graph, and builds the kernel as a positive spectral filter of its eigenpairs; the Matern family $(2\nu/\kappa^2 + \lambda)^{-\nu}$ and its smooth limit the heat kernel $e^{-\kappa^2 \lambda/2}$ are the leading examples, positive definite because their filters are positive, and they descend from a single stochastic differential equation in which the flat Laplacian is replaced by the geometry's own. On Lie groups and homogeneous spaces representation theory supplies the eigenpairs and stationarity becomes group invariance. The invariance route takes any base kernel and averages it over the orbit of a group action, producing a kernel that is positive definite by the averaged-feature-map argument and exactly invariant by the invariance of Haar measure, with every function in its RKHS constant on orbits. The first route changes the domain to fit the kernel; the second changes the kernel to fit a symmetry; both encode geometry as a hard property of the reproducing kernel Hilbert space rather than as a hint in the data, and both connect the kernel viewpoint to the manifold and equivariant methods at the frontier.

28.8 Common mistakes and practical implications

For **Geometric and Equivariant Kernels**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

28.9 Summary and further reading

This chapter established explain the central definitions and claims in Geometric and Equivariant Kernels; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Whittle 1954), (Lindgren and Lindström 2011), (Borovitskiy and Deisenroth 2020).

28.10 Exercises

1. warm-up A spectral filter Φ applied to a graph Laplacian $L = \sum_i \lambda_i u_i u_i^\top$ gives the kernel $K = \sum_i \Phi(\lambda_i) u_i u_i^\top$. For each of the following filters, say whether K is positive definite and name the kernel it produces: (a) $\Phi(\lambda) = e^{-t\lambda}$; (b) $\Phi(\lambda) = (2\nu/\kappa^2 + \lambda)^{-\nu}$; (c) $\Phi(\lambda) = 1/\lambda$ on the nonzero spectrum and 0 on the null space; (d) $\Phi(\lambda) = 1 - \lambda/\lambda_{\max}$. In one sentence, state the single condition on Φ that decides positive definiteness.
2. computation Work on the 4-cycle C_4 , whose Laplacian has eigenvalues 0, 2, 2, 4 with the discrete Fourier eigenvectors. Write the diffusion kernel e^{-tL} in the eigenbasis and evaluate, at $t = 1$, the diagonal entry K_{ii} and the nearest-neighbor entry $K_{i,i+1}$ as sums $\frac{1}{4} \sum_i e^{-t\lambda_i} \cos(\cdot)$. Confirm the diagonal exceeds the off-diagonals, and that all four eigenvalues $e^{-t\lambda_i}$ are positive.
3. proof Prove the spectral-filter criterion in full: if A is symmetric positive semidefinite with eigenpairs (λ_i, u_i) and $\Phi(\lambda) \geq 0$ on the spectrum of A , then $\Phi(A) = \sum_i \Phi(\lambda_i) u_i u_i^\top$ is positive semidefinite, and strictly positive definite when $\Phi > 0$. Deduce that $\exp(-tL)$ and $(2\nu/\kappa^2 I + L)^{-\nu}$ are positive definite for every $t > 0$ and $\nu > 0$.
4. proof Show, without assuming the base kernel is jointly invariant, that the double-averaged kernel $k_G(x, y) = \int_G \int_G k(gx, g'y) dg dg'$ is positive definite and satisfies $k_G(hx, y) = k_G(x, y)$ for all $h \in G$. Identify the feature map of k_G explicitly in terms of the feature map Φ of k . Hint

The feature map is the Bochner average $\bar{\Phi}(x) = \int_G \Phi(gx) dg$; positive definiteness is $\sum_{ij} c_i c_j k_G(x_i, x_j) = \|\sum_i c_i \bar{\Phi}(x_i)\|^2$, and invariance is $\bar{\Phi}(hx) = \bar{\Phi}(x)$ by the invariance of Haar measure.

5. proof Prove that every function in the RKHS $\mathcal{H}_{k_G}$ of a group-averaged kernel is G -invariant, that is $f(gx) = f(x)$ for all $g \in G$ and $f \in \mathcal{H}_{k_G}$. Explain why this makes the invariance a hard constraint on the hypothesis space, in contrast to the soft tangent-covariance penalty of Chapter 20, Invariances and the Pre-Image Problem. Hint

By the reproducing property $f(gx) = \langle f, k_G(\cdot, gx) \rangle_{\mathcal{H}} = \langle f, \bar{\Phi}(gx) \rangle$; now use $\bar{\Phi}(gx) = \bar{\Phi}(x)$ from the invariance proposition.

6. computation Let $G = \{+1, -1\}$ act on $\mathbb{R}$ by $x \mapsto \pm x$ (reflection), and take the base kernel $k(x, y) = (1 + xy)^2$. Compute the invariant kernel $k_G(x, y) = \frac{1}{2}(k(x, y) + k(x, -y))$ in closed form, and verify directly that $k_G(-x, y) = k_G(x, y)$. Which monomials in x, y survive the averaging, and which are annihilated?
7. challenge Establish the diffusion limit on a graph. Diagonalizing $K_\nu = \left(\frac{2\nu}{\kappa^2} I + L\right)^{-\nu}$, normalize the filter to $g_\nu(\lambda) = \left(1 + \frac{\kappa^2 \lambda}{2\nu}\right)^{-\nu}$ and show $g_\nu(\lambda) \rightarrow e^{-\kappa^2 \lambda/2}$ as $\nu \rightarrow \infty$ for each fixed λ . Conclude that the normalized graph Matern kernel converges entrywise to the diffusion kernel, and explain what happens to the $\lambda = 0$ mode. Hint

Take logarithms: $-\nu \ln(1 + \frac{\kappa^2 \lambda}{2\nu}) \rightarrow -\frac{\kappa^2 \lambda}{2}$ since $\ln(1 + u) \approx u$ for small u . The null mode $\lambda = 0$ has $g_\nu(0) = 1$ for every ν , so it is fixed and the constant eigenvector survives both kernels.

8. challenge On the circle S^1 , the Laplace-Beltrami eigenpairs are $f_n(\theta) = e^{in\theta}$ with $\lambda_n = -n^2$. Write the manifold Matern kernel $k_\nu(\theta, \theta') = \frac{1}{C_\nu} \sum_n (2\nu/\kappa^2 + n^2)^{-(\nu+1/2)} e^{in(\theta-\theta')}$ and show it is stationary, a function of $\theta - \theta'$ alone. Identify the sequence $a_n = (2\nu/\kappa^2 + n^2)^{-(\nu+1/2)}$ as its Fourier coefficients, and relate its nonnegativity to Herglotz's theorem for positive definite sequences from Chapter 19, Translation-Invariant, Semigroup, and Probabilistic Kernels. Hint

A sum $\sum_n a_n e^{in(\theta-\theta')}$ depends only on $\theta - \theta'$, so the kernel is stationary on the circle; the coefficients $a_n \geq 0$ are the spectral measure, and Herglotz says a sequence is a positive definite kernel on the integers exactly when its Fourier coefficients are nonnegative, which here is manifest.

29 Indefinite and Krein-Space Kernels

Everything in the RKHS chapter rests on one inequality: a valid kernel must be positive definite, so that its Gram matrix is positive semidefinite and secretly an inner product. Yet many of the most natural ways to compare real objects violate that inequality. The tanh similarity of a two-layer network, the tangent distance between images, the edit distance between strings, the dynamic time warping score between series, and a great many hand-built domain similarity matrices are all symmetric, all sensible, and all indefinite: their Gram matrices carry negative eigenvalues. This chapter asks what learning means when the similarity is not a kernel. The answer has two layers. The clean one is the theory of Krein spaces, where the feature space is a difference of two Hilbert spaces, the kernel splits as $k = k_+ - k_-$, and the representer theorem turns from a minimization into a saddle-point stabilization. The practical one is a toolbox of spectrum transformations, clip, flip, and shift, that force the Gram matrix back into the positive semidefinite cone at a cost we will measure. We keep the two honest against each other, and we are explicit throughout about what breaks when positive definiteness is gone.

29.1 When the similarity is not a kernel

The positive definite world of Chapter 1 and the conditionally positive definite extension of Chapter 8 both keep one foot inside a Hilbert space: a p.d. kernel is an inner product after an embedding, and a conditionally p.d. kernel is a squared distance in one. Indefinite similarities leave that world entirely. They arise for concrete reasons, not as pathologies to be avoided.

The sigmoid or "neural network" kernel $k(x, x') = \tanh(a \langle x, x' \rangle + c)$, proposed to make a support vector machine mimic a two-layer perceptron, is positive definite only for a narrow range of a and c , and indefinite almost everywhere else (Vapnik 1998; Lin and Lin 2003). The tangent distance built to be invariant to small image deformations is symmetric but not a metric, and its Gram matrix is indefinite. The edit distance between strings of the string-kernel chapter and the dynamic time warping alignment score between series of the time-series chapter both violate the triangle inequality, and the similarities read off from them are indefinite. Protein-alignment scores such as Smith-Waterman, human similarity judgments, and countless application-specific matrices behave the same way. Schleif and Tino (2015) survey this landscape under the name indefinite proximity learning and make the point that indefiniteness is the rule, not the exception, once similarities come from an algorithm rather than a dot product.

What exactly goes wrong is best seen on a small matrix. Take the sigmoid kernel with $a = \frac{1}{2}$, $c = \frac{1}{5}$ on the three one-dimensional points $x = (-1, 1, 3)$. Its Gram matrix and spectrum are

$$K = \begin{pmatrix} 0.6044 & -0.2913 & -0.8617 \\ -0.2913 & 0.6044 & 0.9354 \\ -0.8617 & 0.9354 & 0.9998 \end{pmatrix}, \quad \text{eig}(K) = (-0.3261, 0.3144, 2.2203).$$

One eigenvalue is negative. By Aronszajn's theorem this already forbids any feature map Φ with $K_{ij} = \langle \Phi(x_i), \Phi(x_j) \rangle$, because such a map would make K positive semidefinite. The failure is not abstract. If we try to read K as a set of inner products and form the induced squared distance $d^2(i, j) = K_{ii} + K_{jj} - 2K_{ij}$, the pair (x_2, x_3) returns

$$d^2(2, 3) = 0.6044 + 0.9998 - 2(0.9354) = -0.2666 < 0.$$

A negative squared distance cannot occur among points of any Euclidean space. The negative eigenvalue of K is exactly this impossibility made numerical, and it is what every method in this chapter must confront. The numbers here are reproduced in `checks/ch-krein-ex1.py`.

29.2 Krein spaces and the $k = k_+ - k_-$ decomposition

Before repairing the matrix, it pays to ask what geometry an indefinite kernel does describe, because it describes one perfectly well, just not a Hilbert geometry. The right setting was identified by Ong, Mary, Canu, and Smola (2004), building on the operator-theoretic reproducing kernel Krein spaces of Schwartz (1964) and Alpay (1991). The idea is to allow the feature space to have directions of negative squared length.

29.2.1 Indefinite inner products

A Hilbert space measures every vector with a nonnegative squared norm. A Krein space keeps the algebra of an inner product, bilinearity and symmetry, but drops the sign.

Definition (Krein space)

A vector space $\mathcal{K}$ with a symmetric bilinear form $\langle \cdot, \cdot \rangle_{\mathcal{K}}$ is a *Krein space* if it admits a *fundamental decomposition* into a direct sum of two Hilbert spaces,

$$\mathcal{K} = \mathcal{H}_+ \oplus \mathcal{H}_-, \quad f = f_+ + f_-,$$

on which the form acts as a difference of the two Hilbert inner products:

$$\langle f, g \rangle_{\mathcal{K}} = \langle f_+, g_+ \rangle_{\mathcal{H}_+} - \langle f_-, g_- \rangle_{\mathcal{H}_-}.$$

The operator $J = P_+ - P_-$, where $P_{\pm}$ project onto $\mathcal{H}_{\pm}$, is the *fundamental symmetry*; it satisfies $J^2 = \text{Id}$ and turns the indefinite form into a genuine inner product, $\langle f, g \rangle_{|\mathcal{K}|} := \langle f, Jg \rangle_{\mathcal{K}} = \langle f_+, g_+ \rangle_{\mathcal{H}_+} + \langle f_-, g_- \rangle_{\mathcal{H}_-}$.

The associated Hilbert space $|\mathcal{K}|$ with inner product $\langle \cdot, \cdot \rangle_{|\mathcal{K}|}$ supplies a norm and hence a topology; the fundamental decomposition is not unique, but every choice yields the same topology, so "continuous" is unambiguous. The one thing the Krein space lacks is a norm from its own form: the quantity

$$\langle f, f \rangle_{\mathcal{K}} = \|f_+\|_{\mathcal{H}_+}^2 - \|f_-\|_{\mathcal{H}_-}^2$$

can be positive, negative, or zero for a nonzero f . This single fact, that $\langle f, f \rangle_{\mathcal{K}}$ is not bounded below, is the source of everything that differs from the Hilbert case, and we will meet it again as the reason a minimization becomes a saddle point.

29.2.2 Reproducing kernel Krein spaces

A reproducing kernel Hilbert space was a Hilbert space of functions on which evaluation is continuous. The Krein analogue reads identically, with the associated Hilbert topology standing in for the norm.

Definition (reproducing kernel Krein space)

A Krein space $\mathcal{K}$ of real functions on $\mathcal{X}$ is a *reproducing kernel Krein space* (RKKS) if every evaluation functional $f \mapsto f(x)$ is continuous in the topology of $|\mathcal{K}|$. Then there is a unique symmetric $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ with $k(x, \cdot) \in \mathcal{K}$ and the reproducing property

$$\forall f \in \mathcal{K}, \forall x \in \mathcal{X}, \quad f(x) = \langle f, k(x, \cdot) \rangle_{\mathcal{K}}.$$

The function k is the reproducing kernel of $\mathcal{K}$.

The Moore-Aronszajn theorem of Chapter 1 paired every positive definite kernel with one RKHS. Its Krein counterpart replaces "positive definite kernel" by "difference of two positive definite kernels", and this is the structural theorem of the subject.

Theorem (Ong, Mary, Canu, and Smola, 2004)

A symmetric function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is the reproducing kernel of an RKKS if and only if it decomposes as a difference of two positive definite kernels,

$$k = k_+ - k_-,$$

where k_+ and k_- are the reproducing kernels of the Hilbert spaces $\mathcal{H}_+$ and $\mathcal{H}_-$ of a fundamental decomposition. The decomposition is not unique: adding any common positive definite kernel p to both, $k = (k_+ + p) - (k_- + p)$, leaves k unchanged, and among all decompositions there is a minimal one.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The contrast with the positive definite world is exact and worth stating in one line. In an RKHS the kernel is a single positive definite object and $\langle f, f \rangle_{\mathcal{H}} = \|f\|_{\mathcal{H}}^2 \geq 0$ is a true squared norm. In an RKKS the kernel is a difference $k_+ - k_-$, and $\langle f, f \rangle_{\mathcal{K}}$ is indefinite. The positive part k_+ carries the ordinary similarity; the negative part k_- is the correction that a plain inner product cannot express.

On a finite sample the decomposition is nothing more than the spectral split of the Gram matrix into its positive and negative eigenspaces, and it is completely explicit. Writing $K = U\Lambda U^T$ with $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$, collect the positive eigenpairs into K_+ and the negative ones, with their signs flipped, into K_- . Both are positive semidefinite by construction, and $K = K_+ - K_-$. This is the same eigendecomposition that produced the feature map in the finite proof of Aronszajn's theorem, with one change: the negative eigenvalues, which there could not occur, are now separated out rather than square-rooted, and the sign they carry is stored in the fundamental symmetry J . The finite statement is a short piece of linear algebra.

Lemma (finite indefinite kernels are differences of Gram matrices)

Every symmetric $K \in \mathbb{R}^{n \times n}$ decomposes as $K = K_+ - K_-$ with $K_+, K_- \geq 0$, built from the eigendecomposition $K = U\Lambda U^T$ by $K_+ = U\Lambda_+ U^T$ and $K_- = U\Lambda_- U^T$, where $(\Lambda_+)_{ii} = \max(\lambda_i, 0)$ and $(\Lambda_-)_{ii} = \max(-\lambda_i, 0)$. Moreover the feature map $\Phi(x_i)_\ell = \sqrt{|\lambda_\ell|} U_{i\ell}$ with signature $J = \text{diag}(\text{sign } \lambda_\ell)$ reproduces K as an indefinite inner product, $K_{ij} = \Phi(x_i)^T J \Phi(x_j)$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Entrywise $\Lambda = \Lambda_+ - \Lambda_-$, and U is orthogonal, so $K = U\Lambda U^\top = U\Lambda_+ U^\top - U\Lambda_- U^\top = K_+ - K_-$. Both Λ_+ and Λ_- carry only nonnegative entries, hence K_+ and K_- are positive semidefinite. For the feature map, set $|\Lambda| = \Lambda_+ + \Lambda_-$; since J and $|\Lambda|^{1/2}$ are diagonal they commute, and $J|\Lambda| = \text{diag}(\text{sign } \lambda_\ell |\lambda_\ell|) = \Lambda$, so $|\Lambda|^{1/2} J |\Lambda|^{1/2} = \Lambda$. Writing $\Phi = U|\Lambda|^{1/2}$ with row i equal to $\Phi(x_i)$,

$$\Phi J \Phi^\top = U|\Lambda|^{1/2} J |\Lambda|^{1/2} U^\top = U\Lambda U^\top = K.$$

The positive directions are those with $\lambda_\ell > 0$, the negative directions those with $\lambda_\ell < 0$; a Hilbert Gram matrix is the special case in which none of the latter occur. $\square$

Example (decomposing the sigmoid Gram matrix)

The running indefinite matrix, with its eigenvalues and signs, is

$$K = \begin{pmatrix} 0.6044 & -0.2913 & -0.8617 \\ -0.2913 & 0.6044 & 0.9354 \\ -0.8617 & 0.9354 & 0.9998 \end{pmatrix}, \quad \Lambda = \text{diag}(-0.3261, 0.3144, 2.2203), \quad \text{sign } \Lambda = (-1, +1, +1).$$

1. Split the spectrum. Build K_+ from the two positive eigenpairs and K_- from the single negative eigenpair with its sign flipped:

$$K_+ = \begin{pmatrix} 0.6761 & -0.3753 & -0.7559 \\ -0.3753 & 0.7026 & 0.8116 \\ -0.7559 & 0.8116 & 1.1560 \end{pmatrix}, \quad K_- = \begin{pmatrix} 0.0717 & -0.0839 & 0.1058 \\ -0.0839 & 0.0982 & -0.1238 \\ 0.1058 & -0.1238 & 0.1562 \end{pmatrix}.$$

2. Check the pieces are kernels. The smallest eigenvalue of K_+ is 0 and of K_- is 0, so both are positive semidefinite. Since only one eigenvalue was negative, K_- has rank 1: the correction is a single subtracted feature.
3. Recombine. Entry by entry $K_+ - K_- = K$, with maximum discrepancy 3×10^{-16} . The indefinite similarity is the difference of two ordinary ones.
4. Read off the Krein feature map. With $\Phi(x_i)_\ell = \sqrt{|\lambda_\ell|} U_{i\ell}$ and signature $J = \text{diag}(-1, +1, +1)$, the rows are $\Phi(x_1) = (0.2678, 0.4173, -0.7085)$, $\Phi(x_2) = (-0.3134, 0.3742, 0.7500)$, $\Phi(x_3) = (0.3952, 0.0139, 1.0751)$. The indefinite inner product $\Phi(x_i)^\top J \Phi(x_j)$ reproduces K exactly.

Reading. The first coordinate, weighted by -1 in J , is the $\mathcal{H}_-$ direction: it is where the geometry runs backward. The Krein squared norms $\langle \Phi(x_i), \Phi(x_i) \rangle_{\mathcal{H}}$ equal the diagonal $(0.6044, 0.6044, 0.9998)$, all positive here, yet the negative eigenvalue still forced the impossible distance $d^2(2, 3) < 0$ of the previous section, because that distance mixes the backward direction across two points. Numbers from `checks/ch-krein-ex2.py`.

Verification artifact. `checks/example-ch-krein-example-28-1.json` records the example source hash and verification scope.

29.3 The representer theorem becomes a stabilization

With the geometry understood, we can ask how to learn in it. Regularized risk minimization in an RKHS reads

$$\min_{f \in \mathcal{H}} \sum_{i=1}^n \ell(y_i, f(x_i)) + \lambda \langle f, f \rangle_{\mathcal{H}},$$

and it is well posed because the penalty $\langle f, f \rangle_{\mathcal{H}} = \|f\|_{\mathcal{H}}^2$ is convex and bounded below. Transport the same objective to an RKKS and the penalty becomes $\lambda \langle f, f \rangle_{\mathcal{H}} = \lambda (\|f_+\|^2 - \|f_-\|^2)$. Along the $\mathcal{H}_-$ directions this tends to $-\infty$: letting $\|f_-\| \rightarrow \infty$ drives the objective down without bound while the loss stays finite for the hinge and

other lower-bounded losses. There is no minimizer. Minimization is simply the wrong variational principle once the norm is indefinite.

Ong, Mary, Canu, and Smola (2004) replace it with the correct one. Rather than descend the penalty, one looks for a stationary point that descends along the positive directions and ascends along the negative ones: a saddle point.

Definition (stabilization)

Let $J : \mathcal{K} \rightarrow \mathbb{R}$ be a functional on a Krein space with fundamental decomposition $\mathcal{K} = \mathcal{H}_+ \oplus \mathcal{H}_-$. A point $f^\star = f_+^\star + f_-^\star$ is a *stabilizer* of J if it is a stationary point of J that minimizes J along $\mathcal{H}_+$ and maximizes it along $\mathcal{H}_-$:

$$f^\star = \arg \min_{f_+ \in \mathcal{H}_+} \arg \max_{f_- \in \mathcal{H}_-} J(f_+ + f_-).$$

Stabilization replaces minimization as the learning principle in an RKKS.

The reason this is the right notion is that the negative part of the penalty genuinely wants to grow, and the only stable configuration is the one where the loss gradient exactly balances that growth. What survives from the Hilbert theory is the representer theorem: the solution still lives in the finite span of the kernel sections at the data.

Theorem (representer theorem in an RKKS; Ong, Mary, Canu, and Smola, 2004)

Let k be the reproducing kernel of an RKKS $\mathcal{K}$, and let $J(f) = c(f(x_1), \dots, f(x_n)) + \Omega(\langle f, f \rangle_{\mathcal{K}})$ depend on f only through its values at the data and through $\langle f, f \rangle_{\mathcal{K}}$. Then any stabilizer of J admits an expansion

$$f^\star = \sum_{i=1}^n \alpha_i k(x_i, \cdot), \quad \alpha \in \mathbb{R}^n.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The proof is the Krein version of the RKHS argument: decompose $f = f_{\parallel} + f_{\perp}$ into its part in $\text{span}\{k(x_i, \cdot)\}$ and the orthogonal remainder in the associated Hilbert space. The values $f(x_i) = \langle f, k(x_i, \cdot) \rangle_{\mathcal{K}}$ depend only on $f_{\parallel}$, and $\langle f, f \rangle_{\mathcal{K}}$ splits so that $f_{\perp}$ enters only through its own indefinite square, which stationarity sends to zero. Substituting the expansion, $f^\star(x_i) = (K\alpha)_i$ and $\langle f^\star, f^\star \rangle_{\mathcal{K}} = \alpha^\top K \alpha$ with the indefinite Gram matrix K , so the infinite-dimensional saddle collapses to a finite one in α . Everything now depends on the sign structure of K , and that is where the practical trouble concentrates.

29.4 Spectrum transformations

The RKKS theory tells us the geometry is coherent and the solution is a finite expansion. It does not, by itself, make the standard convex solvers apply, because those were built for a positive semidefinite K . The most common response in practice is blunt and often effective: change the eigenvalues of K until it is positive semidefinite, then hand the repaired matrix to an ordinary kernel machine. Chen, Garcia, Gupta, Rahimi, and Cazzanti (2009) and Wu, Chang, and Zhang (2005) catalog the options; three act directly on the spectrum.

Algorithm (spectrum transformation of an indefinite Gram matrix)

Input Symmetric indefinite Gram matrix $K \in \mathbb{R}^{n \times n}$; choice of transform.

Output Positive semidefinite matrix $\tilde{K}$.

1. Eigendecompose $K = U \Lambda U^\top$, $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$.

2. Choose one branch. *Clip (denoise)*: set $\tilde{\lambda}_i = \max(\lambda_i, 0)$, giving $\tilde{K} = U \max(\Lambda, 0) U^\top = K_+$.
3. *Flip*: set $\tilde{\lambda}_i = |\lambda_i|$, giving $\tilde{K} = U |\Lambda| U^\top = K_+ + K_- = (K^2)^{1/2}$.
4. *Shift*: set $\tilde{\lambda}_i = \lambda_i - \lambda_{\min}$ with $\lambda_{\min} = \min_i \lambda_i < 0$, giving $\tilde{K} = K - \lambda_{\min} I$.
5. Return $\tilde{K} = U \text{diag}(\tilde{\lambda}) U^\top$.

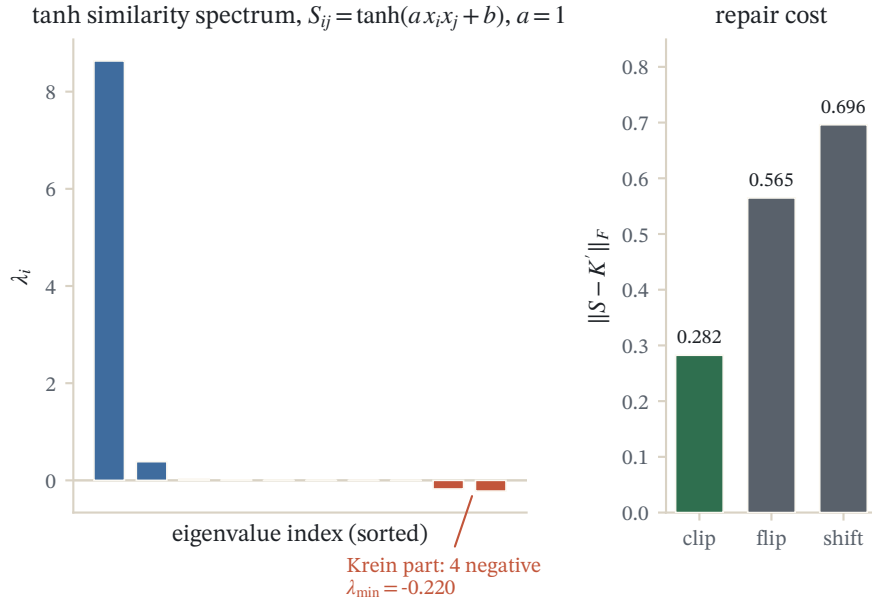


Figure 29.1: The exact spectrum of a tanh similarity matrix on ten points; the negative (Kreĭn) eigenvalues are drawn in red. Of the three PSD repairs, clipping the negative eigenvalues to zero changes the matrix least in Frobenius norm — the nearest-PSD solution.

Each transform embodies a different belief about the negative eigenvalues, and each pays a different price. Clip declares them noise and deletes them; it yields K_+ , which is exactly the nearest positive semidefinite matrix to K in Frobenius norm (Higham 1988), so it is the least violent repair, but it throws away whatever signal the negative part carried. Flip declares the negative directions informative and keeps their magnitude, $|\lambda_i|$, which preserves all spectral energy but reverses the geometry along those directions. Shift lifts the entire spectrum by $|\lambda_{\min}|$; it preserves every eigenvector and every off-diagonal similarity, touching only the diagonal, but it inflates each point's self-similarity and can be a large perturbation when $\lambda_{\min}$ is very negative.

Trans-form	Map on λ_i	Result	Keeps	Cost
Clip	$\max(\lambda_i, 0)$	K_+	nearest p.s.d. matrix (Frobenius)	discards negative-eigenvalue information
Flip	$ \lambda_i $	$K_+ + K_-$	all spectral energy	reverses geometry on negative directions
Shift	$\lambda_i - \lambda_{\min}$	$K - \lambda_{\min} I$	eigenvectors, off-diagonals	inflates self-similarity by $ \lambda_{\min} $
Square / proxy	λ_i^2	$K^2 = K^\top K$	extends to new points	replaces k by a second-order similarity

Example (clip, flip, and shift on the sigmoid matrix)

The running K has $\text{eig}(K) = (-0.3261, 0.3144, 2.2203)$ and the impossible squared distance $d^2(2, 3) = -0.2666$. We repair it three ways and compare, then fit a kernel ridge regressor with targets $y = (1, -1, 1)$ and ridge $\rho = 0.5$.

1. Clip. The spectrum becomes $(0, 0.3144, 2.2203)$ and

$$K_{\text{clip}} = \begin{pmatrix} 0.6761 & -0.3753 & -0.7559 \\ -0.3753 & 0.7026 & 0.8116 \\ -0.7559 & 0.8116 & 1.1560 \end{pmatrix}, \quad d^2(2, 3) = 0.2354.$$

The repair moved K by $\|K - K_{\text{clip}}\|_F = 0.3261$, exactly $|\lambda_{\min}|$, the least any positive semidefinite matrix can be.

2. Flip. The spectrum becomes $(0.3261, 0.3144, 2.2203)$ and the same pair opens to $d^2(2, 3) = 0.7375$, the widest of the three: flipping does not merely delete the negative direction, it pushes those two points apart.
3. Shift. Adding $|\lambda_{\min}| = 0.3261$ to the diagonal leaves every off-diagonal of K untouched and raises every squared distance by the same $2|\lambda_{\min}| = 0.6522$; thus $d^2(2, 3) = -0.2666 + 0.6522 = 0.3856$, and the relative geometry of distinct points is preserved exactly.
4. Compare the predictions. The fitted values $f = \tilde{K}(\tilde{K} + \rho I)^{-1}y$ are $(0.129, -0.080, -0.151)$ for clip, $(0.145, -0.070, -0.154)$ for shift, but $(0.446, -0.450, 0.317)$ for flip. Reading the classifier as $\text{sign } f$, clip and shift both label point 3 negative while flip labels it positive.

Reading. The three transforms are not cosmetic variants; they are three different models. Shift stays closest to the raw similarity because it changes only the diagonal, and clip stays closest in matrix norm, yet flip disagrees with both on the sign of a prediction. The choice of repair is a modeling decision that a validation set, not the algebra, must make. Numbers from `checks/ch-krein-ex1.py`.

Verification artifact. `checks/example-ch-krein-example-28-2.json` records the example source hash and verification scope.

Two caveats keep these heuristics honest. First, all four transforms are operations on the *training* Gram matrix. Extending them to a new test point is not automatic: the eigenvectors U were computed from the training data, so a new similarity vector must be projected through them and the same spectral map applied, an approximation that is exact only for shift, which never touches off-diagonals. The square transform sidesteps this because it has an explicit feature map: taking each row of K as the feature vector $\phi(x) = (k(x, x_1), \dots, k(x, x_n))$, the linear kernel of these vectors is $\langle \phi(x_i), \phi(x_j) \rangle = (K^\top K)_{ij} = (K^2)_{ij}$, always positive semidefinite and trivially evaluated on new points, which is why Chen et al. (2009) and Pekalska and Duin (2005) treat "similarities as features" as the safest embedding when out-of-sample consistency matters. Second, clipping is provably the closest positive semidefinite matrix, but "closest in Frobenius norm" is not the same as "best for the task": if the negative eigenvalues encode real structure, as they do for tangent and warping similarities, deleting them discards signal, and flip or an honest Krein method will do better.

29.5 Learning the SVM in a Krein space

The spectrum transforms treat the symptom. A method that treats the geometry as given must confront the indefinite K head on, and the cleanest place to see both the obstruction and its resolution is the support vector machine. Recall from the SVM chapter that the dual problem is

$$\max_{\alpha \in \mathbb{R}^n} \sum_{i=1}^n \alpha_i - \frac{1}{2} \alpha^\top G \alpha, \quad G_{ij} = y_i y_j K(x_i, x_j), \quad \text{s.t. } 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0.$$

Its good behavior rests entirely on G being positive semidefinite, which makes the objective concave and the problem a convex quadratic program with a unique solution. An indefinite kernel destroys exactly this.

Proposition (indefinite kernel gives a non-convex dual)

If K has a negative eigenvalue, then $G = \text{diag}(y) K \text{diag}(y)$ has one too, so the Hessian $-G$ of the dual objective is indefinite and the objective is not concave.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Let $D = \text{diag}(y)$. Because $y_i \in \{-1, +1\}$, we have $D^\top = D$ and $D^2 = I$, so D is orthogonal with $D^{-1} = D$. Then $G = DKD = DKD^{-1}$ is a similarity transform of K by an orthogonal matrix, and similar matrices have identical spectra, so $\text{eig}(G) = \text{eig}(K)$ and the negative eigenvalue of K is inherited by G . Concavity of $\alpha \mapsto -\frac{1}{2}\alpha^\top G\alpha$ is equivalent to $-G \leq 0$, that is to $G \geq 0$, which the inherited negative eigenvalue contradicts. For general nonzero weights D the same conclusion follows from Sylvester’s law of inertia, since $G = DKD^\top$ is a congruence and preserves the number of negative eigenvalues. $\square$

The consequence is not a mere inconvenience. An indefinite quadratic program is non-convex, has in general multiple local maxima, and is NP-hard to solve globally, so a sequential minimal optimization solver run on an indefinite K converges to a stationary point that need not be the global optimum (Lin and Lin 2003). Haasdonk (2005) gives this stationary point a precise geometric meaning: it minimizes the distance between reduced convex hulls in the pseudo-Euclidean feature space of the Krein embedding, so the machine is still separating the classes, just in an indefinite metric where “maximum margin” is no longer defined.

Example (the dual loses concavity)

Label the three running points $y = (1, -1, 1)$ and form $G = \text{diag}(y) K \text{diag}(y)$:

$$G = \begin{pmatrix} 0.6044 & 0.2913 & -0.8617 \\ 0.2913 & 0.6044 & -0.9354 \\ -0.8617 & -0.9354 & 0.9998 \end{pmatrix}.$$

1. Compare spectra. Numerically $\text{eig}(G) = (-0.3261, 0.3144, 2.2203) = \text{eig}(K)$: the label reweighting is an orthogonal similarity, so the negative eigenvalue survives intact.
2. Exhibit an ascent direction. The equality constraint asks $y^\top v = 0$. The direction $v = (1, 2, 1)$ satisfies $y^\top v = 1 - 2 + 1 = 0$ and gives $v^\top G v = -0.2782 < 0$.
3. Read the curvature. Along v the dual objective has curvature $-v^\top G v = +0.2782 > 0$: it curves upward inside the feasible set, so it is not concave there.

Reading. The box constraints keep the feasible set compact, so a maximum still exists, but the objective is a saddle-shaped quadratic and the maximizer can sit at any of several stationary points. The convexity that made the SVM tractable was exactly the positive definiteness of the kernel, and nothing less. Numbers from `checks/ch-krein-ex3.py`.

Verification artifact. `checks/example-ch-krein-example-28-3.json` records the example source hash and verification scope.

Loosli, Canu, and Ong (2016) resolve this without ever deleting the negative eigenvalues, by taking the stabilization principle seriously. Their Krein SVM stabilizes the regularized hinge risk directly in the RKKS,

$$\text{stab}_{f \in \mathcal{K}} \frac{1}{2} \langle f, f \rangle_{\mathcal{K}} + C \sum_{i=1}^n \max(0, 1 - y_i f(x_i)),$$

which by the representer theorem is a finite saddle problem in α with $\langle f, f \rangle_{\mathcal{K}} = \alpha^\top K \alpha$. They prove this stabilization is equivalent to a convex support vector machine after flipping the spectrum, and that the resulting classifier evaluates the *true* indefinite kernel at test time, so it uses the negative-eigenvalue geometry rather than discarding it.

Algorithm (Krein SVM by flip and stabilize; Loosli, Canu, and Ong, 2016)

Input Indefinite Gram matrix K , labels $y \in \{-1, +1\}^n$, penalty C .

Output Coefficients β and offset b of $f(x) = \sum_i \beta_i k(x_i, x) + b$.

1. Eigendecompose $K = U\Lambda U^\top$; form the flip $|K| = U|\Lambda|U^\top \succeq 0$ and the fundamental symmetry $S = U \text{sign}(\Lambda)U^\top$, so that $K = S|K|$ and $S^2 = I$.
2. Solve the ordinary convex C -SVM dual with the positive semidefinite matrix $|K|$, obtaining dual variables $\tilde{\alpha}$ and offset b .
3. Map the solution back to the indefinite geometry through the fundamental symmetry to obtain the Krein coefficients β (the exact correspondence, and the proof that β is a stabilizer, are in Loosli et al. 2016).
4. Classify a new point by sign $(\sum_i \beta_i k(x_i, x) + b)$, evaluated with the original indefinite kernel k .

The difference from clipping is now sharp. Clipping trains and predicts with K_+ , erasing k_- entirely; the Krein SVM uses $|K| = k_+ + k_-$ only as a computational device to convexify the stabilization, then predicts with $k = k_+ - k_-$. The negative part is carried through to the decision function rather than thrown away. Oglic and Gärtner (2018, 2019) push this further, formulating learning in the RKKS as a principled min-max problem with a regularizer on the negative component, and giving scalable solvers with convergence guarantees, so that the arbitrariness of choosing among clip, flip, and shift is replaced by a single well-posed objective.

29.6 Summary

Symmetric similarities that are not positive definite are common, not exotic: the sigmoid kernel, tangent distance, edit distance, dynamic time warping, and most hand-built domain matrices all produce indefinite Gram matrices, whose negative eigenvalues show up as impossible negative squared distances. The theory that makes sense of them is the Krein space, a difference of two Hilbert spaces, in which the kernel splits as $k = k_+ - k_-$, the inner product is indefinite, and regularized learning becomes a saddle-point stabilization rather than a minimization, with the representer theorem still confining the solution to the span of the data. In practice one either forces positive definiteness with a spectrum transformation, clip for the nearest matrix, flip to keep all the energy, shift to preserve the off-diagonals, each with a measurable cost and an out-of-sample caveat, or one keeps the indefinite geometry and stabilizes directly, as the Krein SVM of Loosli, Canu, and Ong does by flipping only to convexify and then predicting with the true kernel. The honest summary is that positive definiteness buys convexity and a genuine norm, and giving it up costs exactly those two things; the Krein framework is what one gets back when the similarity is worth keeping anyway.

29.7 Common mistakes and practical implications

For **Indefinite and Krein-Space Kernels**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

29.8 Summary and further reading

This chapter established explain the central definitions and claims in Indefinite and Krein-Space Kernels; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Ong and Smola 2004), (Haasdonk 2005), (Chen and Cazzanti 2009).

29.9 Exercises

1. warm-up Take the 2×2 similarity matrix $K = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$. Compute its eigenvalues and eigenvectors by hand, confirm it is indefinite, and write its decomposition $K = K_+ - K_-$ explicitly. Then compute the induced squared distance $d^2(1, 2) = K_{11} + K_{22} - 2K_{12}$ and explain in one sentence why its sign certifies that K is not a Gram matrix of any Euclidean configuration.
2. computation For the same $K = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$, apply each of the three spectrum transforms. Show that clip gives $K_{\text{clip}} = \begin{pmatrix} 1.5 & 1.5 \\ 1.5 & 1.5 \end{pmatrix}$, find the flip and shift matrices, and verify that shift changes only the diagonal while adding the same constant to $d^2(1, 2)$. Which transform leaves the off-diagonal entry $K_{12} = 2$ unchanged, and which two alter it?
3. proof Prove that clipping yields the nearest positive semidefinite matrix in Frobenius norm: for a symmetric $K = U\Lambda U^\top$, show $K_+ = U \max(\Lambda, 0) U^\top$ minimizes $\|K - M\|_F$ over all $M \geq 0$, and that the minimum equals $(\sum_{\lambda_i < 0} \lambda_i^2)^{1/2}$. Hint

Frobenius norm is unitarily invariant, so $\|K - M\|_F = \|\Lambda - U^\top M U\|_F$. Writing $N = U^\top M U \geq 0$, the diagonal of N must be nonnegative, so each diagonal term $(\lambda_i - N_{ii})^2$ is minimized at $N_{ii} = \max(\lambda_i, 0)$, and off-diagonal terms of N only add to the norm. This is Higham (1988).

4. proof The shift transform replaces K by $K + \eta I$. Show that this replaces $G = \text{diag}(y)K \text{diag}(y)$ by $G + \eta I$, so every eigenvalue of G rises by η , and conclude that the SVM dual becomes concave exactly when $\eta \geq |\lambda_{\min}(K)|$. Explain why this restores convexity only by changing the optimization problem, and connect the added η to the self-similarity inflation noted in the transform table. Hint
5. computation Verify the Krein feature map of the worked example. Using the eigenpairs of the running K , form $\Phi(x_i)_\ell = \sqrt{|\lambda_\ell|} U_{i\ell}$ and $J = \text{diag}(\text{sign } \lambda_\ell)$, and check that $\Phi(x_2)^\top J \Phi(x_2) = K_{22} = 0.6044$ while the plain dot product $\Phi(x_2)^\top \Phi(x_2)$ does not equal K_{22} . Which coordinate is responsible for the discrepancy, and what is its sign in J ?

6. proof The Lemma of Section (Krein spaces) shows any finite symmetric K is a difference of two positive semidefinite matrices. Complementing it, exhibit a small explicit example proving the difference of two positive definite kernels need not be positive definite, by giving two 2×2 positive definite matrices whose difference has a negative eigenvalue. Then explain why the finite Lemma does not by itself produce a kernel decomposition $k = k_+ - k_-$ on an infinite domain $\mathcal{X}$, and what additional property the negative part k_- must possess. Hint

For the example, $A = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}$ and $B = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$ are positive definite but $A - B$ has a negative eigenvalue. For the infinite case the difficulty is convergence: one needs k_- to be a bona fide positive definite kernel with a well-defined RKHS, which is where the RKKS existence theorem of Ong et al. (2004) does real work beyond linear algebra.

7. exploration The shift transform adds $\eta = |\lambda_{\min}|$ to every diagonal entry. Show that for kernel ridge regression the shifted dual coefficients $\alpha = (K + \eta I + \rho I)^{-1} y$ are exactly the raw-kernel coefficients with an enlarged ridge $\rho + \eta$. Argue from this that, for a regularized machine, shift adds no discriminative information beyond turning up the regularization, and contrast this with clip and flip, which alter the off-diagonal similarities. Hint

The linear system for α sees $K + \eta I$ and the ridge ρI only through their sum $K + (\eta + \rho)I$; the shift is absorbed into the regularizer. Clip and flip change $U\Lambda U^\top$ off the diagonal, so no such absorption is possible.

8. challenge Consider the stabilization $\text{stab}_{f \in \mathcal{X}} \frac{1}{2} \langle f, f \rangle_{\mathcal{X}} + C \sum_i \max(0, 1 - y_i f(x_i))$ with $\langle f, f \rangle_{\mathcal{X}} = \alpha^\top K \alpha$ after the representer expansion. Using $K = S|K|$ with S the fundamental symmetry, motivate why the change of variable that flips K to $|K|$ converts the indefinite quadratic $\alpha^\top K \alpha$ into a positive definite one, and hence why the saddle problem becomes a convex minimization. State precisely what the classifier evaluates at a new point, and why this differs from simply training an SVM on the clipped matrix K_+ . Hint

The indefinite square $\alpha^\top K \alpha = \alpha^\top S|K|\alpha$ becomes a definite square in the variable that absorbs S ; minimizing over the positive directions and maximizing over the negative ones is, after this change, an ordinary minimization against $|K|$. At test time the Krein classifier uses $k = k_+ - k_-$, whereas the clipped SVM uses only k_+ ; the two agree only when $k_- = 0$. Full details in Loosli, Canu, and Ong (2016).

30 Mean Embeddings, MMD, and Characteristic Kernels

Everything so far has used kernels to represent single points: a feature map sends each x to K_x in an RKHS, and the kernel measures the similarity between two points. This chapter lifts that idea one level up, from points to whole probability distributions. We attach to each distribution P a single vector in the RKHS, its mean embedding μ_P , and then compare distributions by comparing their embeddings. The distance between two embeddings is the Maximum Mean Discrepancy (MMD), a quantity we can estimate from samples alone, use as the statistic of a two-sample test, and even minimize to train a generative model. The whole construction rests on one delicate question: when does the embedding lose no information, so that $\mu_P = \mu_Q$ forces $P = Q$? Kernels with that property are called characteristic, and the last third of the chapter is devoted to identifying them.

30.1 Comparing two distributions from samples

Suppose we are handed two bags of data points, one drawn i.i.d. from an unknown distribution P and the other from an unknown distribution Q , and asked a single question: do P and Q differ? Concretely, one bag might be local field potential (LFP) recordings taken near a spike burst and the other recordings taken away from it, and we want to know whether the neural signal changes near the burst. Or one bag might be photographs of dogs and the other photographs of fish. We never see P or Q , only finite samples, and we want a principled, computable verdict.

The first idea anyone tries is to compare sample means. Given N points $x_1, \dots, x_N$ from P and M points $y_1, \dots, y_M$ from Q , form

$$\hat{\mu}_P = \frac{1}{N} \sum_{i=1}^N x_i, \quad \hat{\mu}_Q = \frac{1}{M} \sum_{j=1}^M y_j,$$

and ask whether they are far apart. If P and Q are two Gaussians with the same variance but different means, this works: the difference in raw means already separates them. But now take two Gaussians with the same mean and different variances. Their raw means coincide, and comparing $\hat{\mu}_P$ with $\hat{\mu}_Q$ sees nothing. The fix is to enrich the description: instead of comparing means of x , compare means of features $\varphi(x)$. With $\varphi(x) = (x, x^2)$, the second coordinate now records the variance, and the mean feature vectors differ. Push this further to a centered Gaussian versus a centered Laplace, which share mean and variance but differ in every higher moment. To tell them apart we need means of arbitrarily high order features $\varphi(x) = (x, x^2, x^3, \dots)$. That is exactly what an RKHS gives us for free: an implicit, possibly infinite feature map whose inner products we can evaluate through the kernel. Comparing distributions then becomes comparing the mean of the feature map under each distribution, and the kernel trick keeps the computation finite.

The move worth internalizing is that features can expose a difference the raw coordinates hide. Two samples can be indistinguishable in the statistic we happen to look at, here the raw mean, and yet be pried cleanly apart once every point is passed through a rich enough feature map. The next figure shows this in the simplest visual form: two point clouds that overlap completely in their original coordinates become separable after a nonlinear feature lift, which is exactly why we will compare means of features rather than means of raw points.

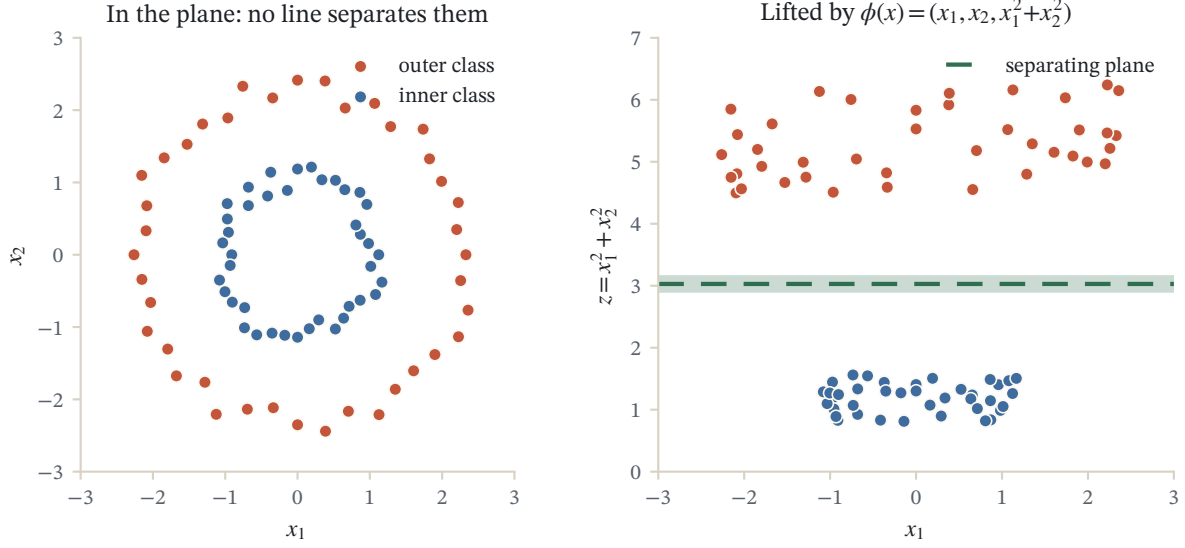


Figure 30.1: Two classes that no line separates in the plane become linearly separable under the lift $\phi(x) = (x_1, x_2, x_1^2 + x_2^2)$: a flat plane slices the inner ring off cleanly. The polynomial kernel computes $\langle \phi(x), \phi(x') \rangle$ without ever building the third coordinate.

30.2 The kernel mean embedding

Let K be a positive definite kernel on a topological set $\mathcal{X}$ with RKHS $\mathcal{H}$, and write $K_x = K(x, \cdot)$ for the canonical feature of the point x . We want the RKHS analogue of “the mean of the features,” an object that summarizes a whole distribution as one element of $\mathcal{H}$.

Definition (kernel mean embedding)

Let P be a Borel probability distribution on $\mathcal{X}$. Its *kernel mean embedding* is the function $\mu_P : \mathcal{X} \rightarrow \mathbb{R}$ defined by

$$\mu_P(y) := \mathbb{E}_{X \sim P}[K(X, y)], \quad y \in \mathcal{X}.$$

Equivalently, as an element of $\mathcal{H}$, $\mu_P = \mathbb{E}_{X \sim P}[K_X]$, the average of the feature map $x \mapsto K_x$ over P .

Two facts about the RKHS make this object usable. First, the reproducing kernel is an inner product of features,

$$K(x, x') = \langle K_x, K_{x'} \rangle_{\mathcal{H}},$$

and second, evaluation of any function is an inner product with a feature, the *kernel trick*,

$$f(x) = \langle f, K_x \rangle_{\mathcal{H}}, \quad f \in \mathcal{H}, \quad x \in \mathcal{X}.$$

Averaging the kernel trick over P suggests that expectations under P should collapse into a single inner product with μ_P . This is the *generalized kernel trick*:

$$\mathbb{E}_{X \sim P}[f(X)] = \langle f, \mu_P \rangle_{\mathcal{H}}, \quad f \in \mathcal{H}.$$

Read it slowly: the mean embedding μ_P is a single vector from which we can read off the expectation under P of every function in the RKHS, just by taking an inner product. In this precise sense μ_P summarizes P . The same

reasoning applied to two distributions gives the inner product between embeddings as a double expectation of the kernel,

$$\mathbb{E}_{(X,Y) \sim P \otimes Q}[K(X,Y)] = \langle \mu_P, \mu_Q \rangle_{\mathcal{H}},$$

which is what will let us compute distances between embeddings without ever forming them explicitly.

In practice we do not know P ; we know samples. The embedding is estimated by the empirical average of features,

$$\hat{\mu}_P(x) = \frac{1}{N} \sum_{i=1}^N K(X_i, x), \quad X_i \stackrel{\text{i.i.d.}}{\sim} P,$$

which is just the mean embedding of the empirical distribution $\frac{1}{N} \sum_i \delta_{X_i}$.

The cleanest sanity check is a single point. If $P = \delta_a$ is the point mass at a , then $\mu_P = \mathbb{E}_{X \sim \delta_a}[K_X] = K_a$: the embedding of a point mass is simply the feature of that point. The empirical embedding above is then a plain average of such single-point embeddings, $\hat{\mu}_P = \frac{1}{N} \sum_i K_{X_i}$, so building μ_P from data is nothing more exotic than averaging feature vectors, one per sample. A second micro-example shows how μ_P stores moments. With the linear kernel $K(x, y) = x^\top y$ on $\mathbb{R}^d$ we get $\mu_P(y) = \mathbb{E}_{X \sim P}[X]^\top y$, so the single vector μ_P encodes exactly the mean of P and nothing else; richer kernels store correspondingly more, which is the thread we pick up when we ask which kernels store everything.

It helps to picture this object concretely rather than as an abstract vector. Take a Gaussian kernel, where $K(X_i, \cdot)$ is a smooth bump centered at the sample point X_i . Then the empirical embedding $\hat{\mu}_P = \frac{1}{N} \sum_i K(X_i, \cdot)$ is nothing but the sum of one bump per data point, that is, a kernel-smoothed version of the sample, the same construction as a kernel density estimate. Its population counterpart μ_P is the smoothed density of P itself: a function whose height at y records how much mass P places near y , as seen through the kernel. This is the mental image to carry forward. The embedding is a fingerprint of the distribution, and two distributions will have the same fingerprint exactly when their kernel-smoothed densities agree everywhere. Whether that forces the distributions themselves to agree is the characteristic-kernel question we return to at the end.

30.2.1 Existence of the mean embedding

We have written μ_P as if it were guaranteed to be an element of $\mathcal{H}$, but that needs an argument: μ_P is defined by an integral, and there is no reason a priori that the integral of RKHS elements lands back inside $\mathcal{H}$, nor that the generalized kernel trick holds. A mild integrability condition settles the matter.

Proposition (existence and uniqueness of μ_P)

Let P be a Borel probability distribution on $\mathcal{X}$ with its Borel σ -algebra, and let K be a p.d. kernel on $\mathcal{X}$ with RKHS $\mathcal{H}$. Assume

$$\mathbb{E}_{X \sim P}[\sqrt{K(X, X)}] < \infty.$$

Then there exists a unique element $\mu_P \in \mathcal{H}$ such that

$$\mathbb{E}_{X \sim P}[f(X)] = \langle f, \mu_P \rangle_{\mathcal{H}}, \quad \forall f \in \mathcal{H}.$$

In particular, for every $y \in \mathcal{X}$,

$$\mu_P(y) = \langle K_y, \mu_P \rangle_{\mathcal{H}} = \mathbb{E}_{X \sim P}[K(X, y)].$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Consider the linear functional T_P on $\mathcal{H}$ defined by $T_P f = \mathbb{E}_{X \sim P}[f(X)]$. We show it is bounded. Using the kernel trick $f(X) = \langle f, K_X \rangle_{\mathcal{H}}$ and then Cauchy–Schwarz in $\mathcal{H}$,

$$|T_P f| \leq \mathbb{E}_{X \sim P}[|f(X)|] = \mathbb{E}_{X \sim P}[|\langle f, K_X \rangle_{\mathcal{H}}|] \leq \mathbb{E}_{X \sim P}[\sqrt{K(X, X)}] \|f\|_{\mathcal{H}},$$

where we used $\|K_X\|_{\mathcal{H}} = \sqrt{\langle K_X, K_X \rangle_{\mathcal{H}}} = \sqrt{K(X, X)}$. By the assumption the constant $\mathbb{E}_{X \sim P}[\sqrt{K(X, X)}]$ is finite, so T_P is a bounded linear functional on $\mathcal{H}$. By the Riesz representation theorem there is a unique $\mu_P \in \mathcal{H}$ with $T_P f = \langle f, \mu_P \rangle_{\mathcal{H}}$ for all f . Taking $f = K_y$ and applying the reproducing property gives $\mu_P(y) = \langle K_y, \mu_P \rangle_{\mathcal{H}} = \mathbb{E}_{X \sim P}[K(X, y)]$, matching the defining formula. $\square$

For a bounded kernel, say $K(x, x) \leq \kappa$ for all x , the condition is automatic, so every distribution has a mean embedding. From here on we assume it exists.

30.3 The Maximum Mean Discrepancy

Once each distribution is a point in $\mathcal{H}$, the natural way to compare P and Q is the distance between their embeddings. That distance is the Maximum Mean Discrepancy.

Definition (MMD)

The *maximum mean discrepancy* between P and Q is the RKHS distance between their mean embeddings,

$$\text{MMD}(P, Q) = \|\mu_P - \mu_Q\|_{\mathcal{H}}.$$

Read through the smoothed-density picture, this is exactly what it should be. The embeddings μ_P and μ_Q are the two kernel-smoothed densities; the MMD is the distance between them as functions in the RKHS. It is large when the two smoothed densities sit in different places and small when they overlap. The figure below lets you feel that distance directly: drag either sample cloud and watch the maximum mean discrepancy recomputed live from the points, growing as the clouds pull apart and falling toward zero as they come to coincide.

The name will be justified in the next section, where the same quantity reappears as a maximization over functions. First we turn the squared distance into something computable from the kernel alone. This expansion is the workhorse of the whole chapter, so we do it in full.

Proposition (MMD as expectations of the kernel)

For distributions P and Q with mean embeddings $\mu_P, \mu_Q \in \mathcal{H}$,

$$\text{MMD}^2(P, Q) = \mathbb{E}_{X, X' \sim P \otimes P}[k(X, X')] + \mathbb{E}_{Y, Y' \sim Q \otimes Q}[k(Y, Y')] - 2 \mathbb{E}_{X, Y \sim P \otimes Q}[k(X, Y)],$$

where X' is an independent copy of X and Y' an independent copy of Y .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Expand the squared norm of a difference using bilinearity of the inner product:

$$\text{MMD}^2(P, Q) = \|\mu_P - \mu_Q\|_{\mathcal{H}}^2 = \langle \mu_P, \mu_P \rangle_{\mathcal{H}} + \langle \mu_Q, \mu_Q \rangle_{\mathcal{H}} - 2 \langle \mu_P, \mu_Q \rangle_{\mathcal{H}}.$$

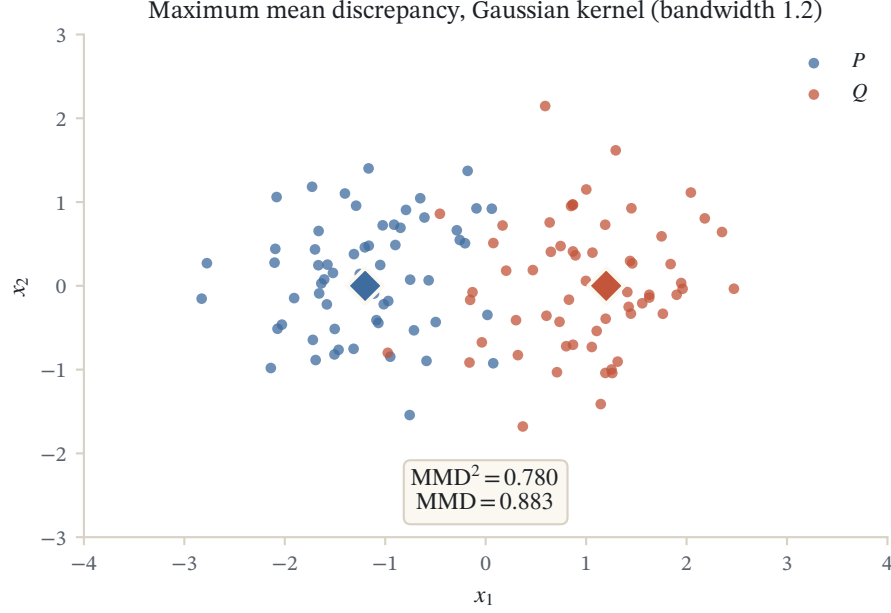


Figure 30.2: Samples from two distributions P and Q and their maximum mean discrepancy $\text{MMD}^2 = \mathbb{E} k(X, X') + \mathbb{E} k(Y, Y') - 2 \mathbb{E} k(X, Y)$, the RKHS distance between their kernel mean embeddings. With a characteristic kernel it vanishes only when $P = Q$.

Now use the identity $\langle \mu_P, \mu_Q \rangle_{\mathcal{H}} = \mathbb{E}_{(X,Y) \sim P \otimes Q} [K(X, Y)]$ established above, once for each term. Writing $\mu_P = \mathbb{E}_{X \sim P} [K_X]$ and $\mu_Q = \mathbb{E}_{Y \sim Q} [K_Y]$ and pulling the expectations out of the (continuous, bilinear) inner product,

$$\langle \mu_P, \mu_P \rangle_{\mathcal{H}} = \mathbb{E}_{X \sim P} \mathbb{E}_{X' \sim P} \langle K_X, K_{X'} \rangle_{\mathcal{H}} = \mathbb{E}_{X, X' \sim P \otimes P} [k(X, X')],$$

and identically $\langle \mu_Q, \mu_Q \rangle_{\mathcal{H}} = \mathbb{E}_{Y, Y' \sim Q \otimes Q} [k(Y, Y')]$ and $\langle \mu_P, \mu_Q \rangle_{\mathcal{H}} = \mathbb{E}_{X, Y \sim P \otimes Q} [k(X, Y)]$. Substituting the three expressions gives the claim. $\square$

The three terms have a clean reading. The first two, $\mathbb{E}_{X, X' \sim P \otimes P} [k(X, X')]$ and $\mathbb{E}_{Y, Y' \sim Q \otimes Q} [k(Y, Y')]$, are *intra-similarity* terms measuring how self-similar each distribution is under the kernel. The last, $\mathbb{E}_{X, Y \sim P \otimes Q} [k(X, Y)]$, is the *inter-similarity* between the two. The MMD is large when each distribution is internally coherent but cross-similarity is low.

There is an even simpler way to hear the three terms. The kernel measures how close two points are, so $\mathbb{E}[k(X, X')]$ is the average closeness of two independent draws from P , and likewise for Q ; the cross term is the average closeness of a draw from P to a draw from Q . If P and Q are the same distribution, a within- P pair and a cross pair are statistically identical, all three averages agree, and the combination collapses to zero. If P and Q live in different regions, points within each cloud are on average closer to each other than to points in the other cloud, the cross term drops, and the MMD grows. So MMD^2 is precisely the amount by which within-group similarity exceeds between-group similarity, a single number reading off how much the two smoothed densities fail to overlap.

A two-point micro-example makes the arithmetic tangible. Take $P = \delta_a$ and $Q = \delta_b$ with a normalized kernel, $K(x, x) = 1$ for all x . Each intra-similarity term is $K(a, a) = K(b, b) = 1$ and the cross term is $K(a, b)$, so the expansion gives $\text{MMD}^2(\delta_a, \delta_b) = 1 + 1 - 2K(a, b) = 2(1 - K(a, b))$. This is exactly $\|K_a - K_b\|_{\mathcal{H}}^2$, and it behaves as it must: it is 0 when $a = b$ (the kernel reports full similarity, $K(a, b) = 1$) and climbs toward 2 as the two points become dissimilar, $K(a, b) \rightarrow 0$. The same skeleton, one within-term per group minus twice the cross-term, is all that the general formula does, only averaged over the two clouds instead of evaluated at two single points.

One caution. In general the MMD is only a *pseudometric*: it satisfies non-negativity, symmetry, and the triangle inequality, but $\text{MMD}(P, Q) = 0$ does not by itself force $P = Q$. If the kernel is too poor, distinct distributions can share an embedding and the MMD collapses to zero. Kernels for which $\text{MMD}(P, Q) = 0 \iff P = Q$ are called

characteristic, and for them MMD is a genuine metric on distributions. We characterize such kernels in the final sections. Until then, we assume the kernel is characteristic so that MMD metrizes equality of distributions.

30.3.1 Estimating the MMD from samples

Given samples $x_1, \dots, x_N \sim P$ and $y_1, \dots, y_M \sim Q$, the obvious estimator plugs the empirical distributions into the expansion. Replacing each expectation by an average over all pairs gives the *biased* estimator

$$\widehat{\text{MMD}}_b^2(P, Q) = \frac{1}{N^2} \sum_{i,j} K(x_i, x_j) + \frac{1}{M^2} \sum_{i,j} K(y_i, y_j) - \frac{2}{NM} \sum_{i,j} K(x_i, y_j).$$

It is biased because the diagonal terms $K(x_i, x_i)$ and $K(y_i, y_i)$ are included: they estimate $\mathbb{E}[k(X, X')]$ with pairs that are not independent, inflating the intra-similarity terms. Dropping the diagonal, that is, averaging only over distinct indices $i \neq j$, removes the bias:

$$\widehat{\text{MMD}}_u^2(P, Q) = \frac{1}{N(N-1)} \sum_{i \neq j} K(x_i, x_j) + \frac{1}{M(M-1)} \sum_{i \neq j} K(y_i, y_j) - \frac{2}{NM} \sum_{i,j} K(x_i, y_j).$$

This *unbiased* estimator is the usual choice for testing. Both are simple sums of kernel evaluations, and both are differentiable in the sample points, a fact we exploit for generative modeling.

30.4 MMD as an integral probability metric

The distance-between-embeddings view is geometric. There is a second, dual view that explains the word "maximum" in the name and produces an interpretable object, the witness function. It fits MMD into a broad family of distances between distributions.

Definition (integral probability metric)

Let $\mathcal{F}$ be a set of measurable functions. The *integral probability metric* (IPM) associated to $\mathcal{F}$ is

$$D_{\mathcal{F}}(P, Q) := \sup_{f \in \mathcal{F}} (\mathbb{E}_{X \sim P}[f(X)] - \mathbb{E}_{Y \sim Q}[f(Y)]).$$

The idea behind an IPM is a courtroom of test functions. To decide whether P and Q differ, hand every function f in the class $\mathcal{F}$ the same challenge: report the gap between its average under P and its average under Q . A function that happens to be large exactly where P has more mass than Q will report a big positive gap; a function blind to the difference reports nothing. The IPM is the verdict of the most discriminating witness, the largest gap any allowed function can produce. This is where the word "maximum" in maximum mean discrepancy comes from. An IPM searches over a class of "test functions" $\mathcal{F}$ for the one whose average differs most between P and Q . If no function in the class can tell the two distributions apart, the supremum is zero. Different choices of $\mathcal{F}$ recover classical distances: functions bounded by 1 in sup norm, $\|f\|_{\infty} \leq 1$, give the total variation distance; bounded-Lipschitz functions, $\|f\|_{BL} \leq 1$, give the Dudley metric; and 1-Lipschitz functions, $\|f\|_{Lip} \leq 1$, give the 1-Wasserstein (Kantorovich) distance. The MMD is the IPM obtained by taking $\mathcal{F}$ to be the unit ball of the RKHS, $\|f\|_{\mathcal{H}} \leq 1$.

Proposition (MMD is the RKHS-ball IPM, with witness function)

Taking $\mathcal{F} = \{f \in \mathcal{H} : \|f\|_{\mathcal{H}} \leq 1\}$,

$$\sup_{\substack{f \in \mathcal{H} \\ \|f\|_{\mathcal{H}} \leq 1}} (\mathbb{E}_{X \sim P}[f(X)] - \mathbb{E}_{Y \sim Q}[f(Y)]) = \|\mu_P - \mu_Q\|_{\mathcal{H}} = \text{MMD}(P, Q),$$

and the supremum is attained at the *witness function*

$$f^\star = \frac{\mu_P - \mu_Q}{\|\mu_P - \mu_Q\|_{\mathcal{H}}}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Apply the generalized kernel trick to each expectation. For any $f \in \mathcal{H}$,

$$\mathbb{E}_{X \sim P}[f(X)] - \mathbb{E}_{Y \sim Q}[f(Y)] = \langle f, \mu_P \rangle_{\mathcal{H}} - \langle f, \mu_Q \rangle_{\mathcal{H}} = \langle f, \mu_P - \mu_Q \rangle_{\mathcal{H}}.$$

So the IPM is $\sup_{\|f\|_{\mathcal{H}} \leq 1} \langle f, \mu_P - \mu_Q \rangle_{\mathcal{H}}$. By Cauchy-Schwarz this inner product is at most $\|f\|_{\mathcal{H}} \|\mu_P - \mu_Q\|_{\mathcal{H}} \leq \|\mu_P - \mu_Q\|_{\mathcal{H}}$, with equality when f is the unit vector aligned with $\mu_P - \mu_Q$, namely $f^\star = (\mu_P - \mu_Q) / \|\mu_P - \mu_Q\|_{\mathcal{H}}$. Evaluating, $\langle f^\star, \mu_P - \mu_Q \rangle_{\mathcal{H}} = \|\mu_P - \mu_Q\|_{\mathcal{H}} = \text{MMD}(P, Q)$. $\square$

The witness function is worth dwelling on, because the proof hands us its shape for free. Cauchy-Schwarz is tight exactly when f points along $\mu_P - \mu_Q$, so the winning witness is the unit vector in that direction. Two things follow. First, the maximal gap it achieves is literally $\|\mu_P - \mu_Q\|_{\mathcal{H}}$, which is why the IPM over the RKHS ball equals the MMD; the geometric distance between embeddings and the dual “best test function” are two views of one number. Second, because $f^\star \propto \mu_P - \mu_Q$ and each embedding is a smoothed density, the witness is the difference of the two smoothed densities: $f^\star(t) \propto \mathbb{E}_{X \sim P}[K(X, t)] - \mathbb{E}_{Y \sim Q}[K(Y, t)]$, estimated from samples by $\frac{1}{N} \sum_i K(x_i, t) - \frac{1}{M} \sum_j K(y_j, t)$. It is therefore positive wherever P puts more mass than Q and negative wherever Q exceeds P . Plotting $f^\star$ does not just certify that the distributions differ; it points a finger at the regions of input space responsible for the difference, which is often more informative than the scalar MMD itself.

A concrete picture: let P be a cloud centered near the left and Q a cloud centered near the right, and use a Gaussian kernel. The smoothed density $\hat{\mu}_P$ is a bump over the left, $\hat{\mu}_Q$ a bump over the right, and their difference $f^\star \propto \hat{\mu}_P - \hat{\mu}_Q$ rises to a positive peak over the left cloud and dips to a negative trough over the right. Reading it as a test function, $f^\star$ is the recipe “score points on the left positively and points on the right negatively,” which is exactly the rule that most separates the two samples’ averages. The witness is thus both the geometric direction $\mu_P - \mu_Q$ in $\mathcal{H}$ and, plotted over $\mathcal{X}$, a readable map of where the distributions disagree.

Remark (energy distance is an MMD)

The *energy distance*, defined in statistics through pairwise Euclidean distances as $\mathcal{E}(P, Q) = 2 \mathbb{E}\|X - Y\| - \mathbb{E}\|X - X'\| - \mathbb{E}\|Y - Y'\|$ with $X, X' \sim P$ and $Y, Y' \sim Q$ independent, looks at first like a separate construction. Sejdinovic, Sriperumbudur, Gretton, and Fukumizu (2013) showed that it is exactly an MMD: for the distance-induced kernel $K(x, y) = \frac{1}{2}(\|x\|_0 + \|y\|_0 - \|x - y\|_0)$ built from a semimetric of negative type (here $\|\cdot\|_0$ a chosen distance from a base point), the MMD expansion reproduces $\mathcal{E}(P, Q)$ term for term. Energy distance and MMD are therefore two names for one family of discrepancies, and the choice of distance is just a choice of kernel.

30.5 A two-sample test using the MMD

We now make the informal question “do P and Q differ?” into a hypothesis test. Given samples $x_1, \dots, x_N \sim P$ and $y_1, \dots, y_N \sim Q$, the null and alternative hypotheses are

Hypotheses

$H_0 : P = Q$ versus $H_1 : P \neq Q$. Under H_0 the empirical $\widehat{\text{MMD}}^2$ should be close to zero; under H_1 it should be far from zero. The test statistic is the unbiased estimator

$$\widehat{\text{MMD}}^2(P, Q) = \frac{1}{N(N-1)} \sum_{i \neq j} K(x_i, x_j) + \frac{1}{N(N-1)} \sum_{i \neq j} K(y_i, y_j) - \frac{2}{N^2} \sum_{i,j} K(x_i, y_j).$$

To turn "close" and "far" into a decision we need the distribution of the statistic. It differs sharply between the two hypotheses, and this asymmetry is what makes the test work.

Under the alternative $P \neq Q$. The statistic is asymptotically normal (Gretton, Borgwardt, Rasch, Schölkopf, and Smola, 2006): with n the sample size and $V(P, Q)$ the asymptotic variance,

$$\sqrt{n} (\widehat{\text{MMD}}^2(P, Q) - \text{MMD}^2(P, Q)) \xrightarrow{d} \mathcal{N}(0, V(P, Q)).$$

It concentrates around the true positive value $\text{MMD}^2(P, Q) > 0$, so $n \widehat{\text{MMD}}^2$ grows linearly in n and runs off to $+\infty$.

Under the null $P = Q$. Here $\text{MMD}^2 = 0$, the leading term vanishes, and the rescaled statistic $n \widehat{\text{MMD}}^2$ has a nondegenerate limit that is *not* Gaussian (Gretton et al., 2006):

$$n \widehat{\text{MMD}}^2(P, Q) \xrightarrow{d} 2 \sum_{i=1}^{\infty} \lambda_i (z_i^2 - 1),$$

an infinite weighted sum of centered χ_1^2 variables. Here the $z_i \sim \mathcal{N}(0, 1)$ are i.i.d. standard Gaussians, and the weights λ_i are the eigenvalues of the integral operator $f \mapsto \mathbb{E}_{X \sim P}[\tilde{K}(X, X') f(X)]$ built from the *centered kernel*

$$\tilde{K}(x, x') = \langle K(x, \cdot) - \mu_P, K(x', \cdot) - \mu_P \rangle_{\mathcal{H}}.$$

The change of shape between the two regimes is not an accident, and it is worth seeing why. Under the alternative the statistic is estimating a nonzero number, so ordinary fluctuations around that number are governed by the central limit theorem and look Gaussian. Under the null the target is zero: the leading, first-order part of the statistic vanishes identically, and what remains is a second-order, quadratic fluctuation. A quadratic form in Gaussian noise is not Gaussian; it is a weighted sum of squared Gaussians, which is exactly the χ^2 mixture above. The weights λ_i are the eigenvalues of the centered kernel, so they encode how the kernel spreads its sensitivity across directions in the RKHS. This is the same degeneracy that makes a variance estimate behave differently at zero than away from it, and it is the reason the null distribution has to be handled with care rather than read off a normal table.

Collecting the two regimes, writing $T_0 := n \widehat{\text{MMD}}^2(P, Q)$,

$$T_0 \approx \begin{cases} n \text{MMD}^2(P, Q) + \sqrt{n} \mathcal{N}(0, V(P, Q)), & P \neq Q, \\ 2 \sum_{i=1}^{\infty} \lambda_i (z_i^2 - 1), & P = Q. \end{cases}$$

30.5.1 The threshold and the permutation test

The test controls the false-positive rate. Fix a significance level α (typically a small value such as 0.05) and choose the threshold c_α as the upper- α quantile of the null distribution,

$$\mathbb{P}(T_0 > c_\alpha \mid H_0) = \alpha.$$

If $T_0 \geq c_\alpha$ we reject the null, declaring $P = Q$ unlikely; otherwise we cannot reject and $P = Q$ remains plausible. The remaining difficulty is that the null distribution $2 \sum_i \lambda_i (z_i^2 - 1)$ depends on the unknown eigenvalues λ_i , so c_α is not available in closed form. We estimate the null by resampling.

Let T denote a draw from the null distribution. Equivalently to comparing T_0 with c_α , we compute the p -value $p := \mathbb{P}_T(T > T_0)$; then $T_0 \geq c_\alpha$ precisely when $p \leq \alpha$. To sample from the null, use a *permutation test*. Its logic is that under H_0 the two group labels carry no information: if $P = Q$, then which bag a point came from is arbitrary, so any reshuffling of the labels is as legitimate as the original. Pool the $2N$ points, and for each permutation j split them into two new groups $\tilde{P}, \tilde{Q}$ at random and recompute $T_j = n \widehat{\text{MMD}}^2(\tilde{P}, \tilde{Q})$. Since under H_0 all points are exchangeable, each T_j is a valid draw from the null distribution, and together they trace out the null empirically without our ever knowing the eigenvalues λ_i . With J permutations the p -value is approximated by the fraction of resampled statistics that exceed the observed one,

$$p \approx \frac{|\{j : T_j \geq T_0\}|}{J},$$

and we reject when this is at most α . The permutation test needs nothing but repeated evaluations of the kernel, so it is entirely practical.

A number makes the procedure concrete. Suppose we run $J = 1000$ permutations and only 7 of the reshuffled statistics land above the observed T_0 . Then $p \approx 7/1000 = 0.007$, comfortably below $\alpha = 0.05$, and we reject the null: a real difference between P and Q makes the true labels produce an unusually large discrepancy that random relabelings rarely match. Had the count instead been, say, 300, we would get $p \approx 0.30$, the observed statistic would sit squarely among the null draws, and we could not reject. The whole test reduces to asking where the observed T_0 falls in the histogram of permuted statistics.

30.6 Independence testing with HSIC

The same machinery answers a different question. Instead of asking whether two distributions differ, ask whether two random variables are dependent. Let (X, Y) be a pair with joint distribution P_{XY} on $\mathcal{X} \times \mathcal{Y}$ and marginals P_X and P_Y . Independence is exactly the statement $P_{XY} = P_X \otimes P_Y$, so a measure of dependence is just a discrepancy between the joint and the product of the marginals. Feeding those two distributions into the MMD gives the *Hilbert-Schmidt Independence Criterion* of Gretton, Bousquet, Smola, and Schölkopf (2005).

Definition (HSIC)

Let k be a kernel on $\mathcal{X}$ and ℓ a kernel on $\mathcal{Y}$, and let $K((x, y), (x', y')) = k(x, x') \ell(y, y')$ be their product kernel on $\mathcal{X} \times \mathcal{Y}$. The *Hilbert-Schmidt Independence Criterion* is the squared MMD between the joint and the product of the marginals,

$$\text{HSIC}(X, Y) = \text{MMD}^2(P_{XY}, P_X \otimes P_Y),$$

computed with the product kernel K .

Expanding the MMD with the product kernel gives a formula in the base kernels alone,

$$\text{HSIC}(X, Y) = \mathbb{E}_{XY} \mathbb{E}_{X'Y'} [k(X, X') \ell(Y, Y')] + \mathbb{E}_X \mathbb{E}_{X'} [k(X, X')] \mathbb{E}_Y \mathbb{E}_{Y'} [\ell(Y, Y')] - 2 \mathbb{E}_{XY} [\mathbb{E}_{X'} [k(X, X')] \mathbb{E}_{Y'} [\ell(Y, Y')]],$$

where (X', Y') is an independent copy of (X, Y) . When k and ℓ are characteristic, $\text{HSIC}(X, Y) = 0$ if and only if X and Y are independent, so HSIC detects dependence of every order, not just linear correlation. Given N paired samples, HSIC is estimated by $\frac{1}{N^2} \text{Tr}(KHLH)$ with K, L the two kernel matrices and $H = I - \frac{1}{N} \mathbf{1}\mathbf{1}^\top$ the centering matrix, and independence is tested by the same permutation calibration used above, now shuffling the pairing between the X and Y samples. This is the basis for the dependence detection, feature selection, and blind source separation applications noted in the summary.

30.7 Learning generative models with MMD

MMD is not only a diagnostic; because the empirical estimator is differentiable in the samples, we can *minimize* it to fit a model. The setting is generative modeling: given samples from a data distribution P over $\mathcal{X}$, we want a model that produces new samples from some $Q \approx P$.

Two broad families exist. An *explicit* generative model (EGM) specifies a density $q(y)$, has support on the whole space, is trained by maximum likelihood or score matching, and is sampled by MCMC. An *implicit* generative model (IGM) instead defines samples directly by a pushforward: draw latent noise $Z \sim \mu$ from a known simple distribution and set $Y = G(Z) \sim Q$. Its support can be a low-dimensional manifold (Arjovsky et al., 2017), it is trained by minimizing a well-chosen divergence $D(P, Q)$, and it is sampled just by pushing μ forward through G . IGMs power many striking applications: single-image super-resolution (Ledig et al., 2015), image-to-image translation (Isola et al., 2016), and text-to-image generation (Zhang et al., 2016).

30.7.1 The adversarial view and the GAN connection

Generative adversarial networks (Goodfellow et al., 2014) fit an IGM by an adversarial game against a divergence defined variationally. Given a class of critics $\mathcal{F}$,

$$D(P, Q) := \sup_{f \in \mathcal{F}} G(f, P, Q).$$

The *critic* maximizes $G(f, P, Q)$ over $f \in \mathcal{F}$ to find the best discriminating function f^* , and the *generator* minimizes $D(P, Q) = G(f^*, P, Q)$ over Q . The MMD is exactly this construction when $\mathcal{F}$ is the unit ball of an RKHS: the critic is then the witness function f^* , computed in closed form rather than by an inner optimization. In this sense an MMD-trained generator is a GAN whose discriminator is solved analytically by the kernel.

Concretely, we solve

$$\min_{\theta} \text{MMD}^2(P, Q_{\theta}),$$

where Q_{θ} is the pushforward of μ through a parametric generator G_{θ} . Each step of stochastic gradient descent is:

1. Sample a minibatch $X_1, \dots, X_B \sim P$ from the dataset.
2. Sample latent noise $Z_1, \dots, Z_B \sim \mu$.
3. Generate model samples $Y_b = G_{\theta}(Z_b) \sim Q_{\theta}$, $1 \leq b \leq B$.
4. Compute the empirical loss $L(\theta) = \text{MMD}^2_{\text{hat}}(P, Q_{\theta})$, which is differentiable in θ through the Y_b .
5. Update $\theta \leftarrow \theta - \gamma \cdot \text{grad } L(\theta)$.

This construction, an implicit generator trained by minimizing the MMD to the data, is the *generative moment-matching network* introduced independently by Li, Swersky, and Zemel (2015) and Dziugaite, Roy, and Ghahramani (2015). Trained this way with an RBF kernel, an IGM already generates reasonable MNIST digits. But the choice of kernel is crucial, and hand-designing a good kernel for high-dimensional data such as natural images is hard. The remedy is to learn the kernel too. Optimize over a family $\mathcal{K}$ of kernels,

$$\min_{\theta} \sup_{k \in \mathcal{K}} \text{MMD}_k^2(P, Q_{\theta}),$$

where a typical family is parameterized by a neural network, $k(x, y) = h(\varphi(x), \varphi(y))$ with φ a network and h a fixed p.d. kernel. The inner supremum adaptively selects the MMD that best discriminates the data from the

current model, exactly the adversarial critic again, and in practice one alternates gradient steps on k and on θ . Such adversarially trained MMD models produce competitive samples on datasets like MNIST and CelebA (Arbel, Sutherland, Bińkowski, and Gretton, 2018); the full story adds regularization, optimization stability, and evaluation, which we do not pursue here.

30.8 Characteristic kernels

Everything above quietly assumed the embedding loses no information: that $\mu_P = \mu_Q$ implies $P = Q$. Without it, MMD is only a pseudometric, a two-sample test can be blind to real differences, and a generative model can converge to the wrong distribution. So we must ask directly: can the mean embedding characterize the distribution?

The answer depends entirely on the kernel, as three examples show. Take the *linear* kernel $K(x, x') = x^\top x'$. Then $\mu_P(x) = \mathbb{E}_{X \sim P}[X]^\top x$, so the embedding sees only the mean of P . Any two distributions with equal means, whatever their higher moments, have identical embeddings; the kernel is not characteristic. Take the *polynomial* kernel of order two, $K(x, x') = (x^\top x')^2$. Then $\mu_P(x) = \text{Tr}(\mathbb{E}_{X \sim P}[XX^\top] xx^\top)$, which sees the second moment as well, but still nothing beyond it: distributions matched on first and second moments collapse together. Each finite-order kernel captures only finitely many moments.

Contrast this with the *exponential* kernel $K(x, y) = \exp(x^\top y)$. Now

$$\mu_P(y) = \mathbb{E}_{X \sim P}[\exp(X^\top y)],$$

which is precisely the *moment generating function* of P , evaluated at y . A classical result says that if two distributions have the same moment generating function then they are equal, that is $\mathbb{E}_{X \sim P}[f(X)] = \mathbb{E}_{Y \sim Q}[f(Y)]$ for all $f \in C_b(\mathcal{X})$. Intuitively, the family of functions $\{K_y : x \mapsto \exp(x^\top y)\}_{y \in \mathcal{X}}$ is rich enough that matching $\mathbb{E}_P[K_y]$ with $\mathbb{E}_Q[K_y]$ for every y pins down the distribution. This richness is exactly what we formalize next.

Definition (characteristic kernel)

Let $\mathcal{X}$ be a topological set and $\mathcal{P}$ the set of Borel probability measures on $\mathcal{X}$. Let K be a bounded measurable p.d. kernel on $\mathcal{X}$ with RKHS $\mathcal{H}$. The kernel K is *characteristic* if the mean embedding map

$$\mathcal{P} \ni P \longmapsto \mu_P = \mathbb{E}_{X \sim P}[K_X] \in \mathcal{H}$$

is injective, that is,

$$\forall P, Q \in \mathcal{P} : \quad \mu_P = \mu_Q \implies P = Q.$$

Three equivalences organize the idea. Equality of embeddings is the same as equality of RKHS expectations,

$$\mu_P = \mu_Q \iff \mathbb{E}_{X \sim P}[f(X)] = \mathbb{E}_{Y \sim Q}[f(Y)] \quad \forall f \in \mathcal{H},$$

which is immediate from the generalized kernel trick. Equality of distributions, on the other hand, is equality of expectations of all bounded continuous functions,

$$P = Q \iff \mathbb{E}_{X \sim P}[f(X)] = \mathbb{E}_{Y \sim Q}[f(Y)] \quad \forall f \in C_b(\mathcal{X}).$$

A kernel is characteristic exactly when these two conditions coincide, and that happens when $\mathcal{H}$ is rich enough to stand in for $C_b(\mathcal{X})$. It is worth seeing what "rich enough" buys downstream. The three examples above are all failures of richness of increasing subtlety: the linear kernel's $\mathcal{H}$ contains only linear functions, so matching μ_P and μ_Q pins down just the mean; the order-two polynomial adds quadratics and so pins down the second moment

as well, but no more; only a kernel whose $\mathcal{H}$ reaches every continuous function forces $P = Q$. Concretely, if we ran the two-sample test of the previous section with a linear kernel, it would be structurally blind to any two distributions sharing a mean, no matter how many samples we drew, because the population MMD is already zero. Choosing a characteristic kernel is what guarantees the test can, in principle, see every difference. The rest of the chapter makes "rich enough" precise, first for compact $\mathcal{X}$ through universality, then for $\mathcal{X} = \mathbb{R}^d$ through the Fourier transform.

30.9 Characteristic kernels via universality

On a compact domain the right notion of richness is that the RKHS is dense in the continuous functions.

Definition (universal kernel)

Let K be a p.d. kernel with RKHS $\mathcal{H}$ on a compact set $\mathcal{X}$. The kernel K is *universal* if $y \mapsto K(x, y)$ is continuous for every $x \in \mathcal{X}$ and $\mathcal{H}$ is dense in $C(\mathcal{X})$ in the maximum norm $\|\cdot\|_\infty$.

Density in the sup norm means every continuous function can be approximated uniformly by RKHS elements. That is exactly the leverage needed to make embeddings characterize distributions.

Proposition (universal $\Rightarrow$ characteristic)

Assume $\mathcal{X}$ is compact. If K is universal, then K is characteristic.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Let P and Q satisfy $\mu_P = \mu_Q$. We must show $\mathbb{E}_{X \sim P}[f(X)] = \mathbb{E}_{Y \sim Q}[f(Y)]$ for all $f \in C(\mathcal{X})$, which on a compact space gives $P = Q$. Fix $f \in C(\mathcal{X})$ and $\epsilon > 0$. By universality $\mathcal{H}$ is dense in $C(\mathcal{X})$ for $\|\cdot\|_\infty$, so there is $g \in \mathcal{H}$ with $\|f - g\|_\infty \leq \epsilon$. Split the difference of expectations through g :

$$|\mathbb{E}_P[f] - \mathbb{E}_Q[f]| \leq |\mathbb{E}_P[f] - \mathbb{E}_P[g]| + |\mathbb{E}_Q[f] - \mathbb{E}_Q[g]| + |\mathbb{E}_P[g] - \mathbb{E}_Q[g]|.$$

The first two terms are each at most $\mathbb{E}[|f - g|] \leq \|f - g\|_\infty \leq \epsilon$ by the choice of g . The last term vanishes: since $g \in \mathcal{H}$, the generalized kernel trick gives $\mathbb{E}_P[g] - \mathbb{E}_Q[g] = \langle g, \mu_P - \mu_Q \rangle_{\mathcal{H}} = 0$ because $\mu_P = \mu_Q$. Hence $|\mathbb{E}_P[f] - \mathbb{E}_Q[f]| \leq 2\epsilon$ for every $\epsilon > 0$, so $\mathbb{E}_P[f] = \mathbb{E}_Q[f]$. As $f \in C(\mathcal{X})$ was arbitrary, $P = Q$. $\square$

This reduces the problem to producing universal kernels. We give the machinery, Stone–Weierstrass and a general criterion, and then two concrete constructions.

30.9.1 Stone–Weierstrass and a general criterion

Density in $C(\mathcal{X})$ is exactly what the Stone–Weierstrass theorem controls, so it is the engine behind every universality result here.

Theorem (Stone–Weierstrass)

Let $(\mathcal{X}, d)$ be a compact metric space and $\mathcal{A}$ a linear subspace of $C(\mathcal{X})$. Then $\mathcal{A}$ is dense in $C(\mathcal{X})$ if

- $\mathcal{A}$ is an *algebra* for the pointwise product of functions;
- $\mathcal{A}$ *does not vanish*: for every $x \in \mathcal{X}$ there is $f \in \mathcal{A}$ with $f(x) \neq 0$;
- $\mathcal{A}$ *separates points*: for all $x \neq y$ there is $f \in \mathcal{A}$ with $f(x) \neq f(y)$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Here an *algebra* is a vector space $\mathcal{A}$ equipped with a bilinear binary product $\times : \mathcal{A} \times \mathcal{A} \rightarrow \mathcal{A}$, meaning $z \times (x + y) = z \times x + z \times y$, $(x + y) \times z = x \times z + y \times z$, and $(ax) \times (by) = (ab)(x \times y)$ for all $x, y, z \in \mathcal{A}$ and scalars a, b . We apply this to the span of the feature coordinates.

Theorem (general criterion for universality, Steinwart 2001)

Let $\mathcal{X}$ be a compact metric space and k a continuous kernel on $\mathcal{X}$ with $k(x, x) > 0$. Suppose there is an injective feature map $\Phi(x) = \{\varphi_i(x)\}_{i \geq 0}$ with

$$k(x, y) = \sum_{i=0}^{\infty} \varphi_i(x) \varphi_i(y).$$

If $\mathcal{A} := \text{span}\{\varphi_i : i \geq 0\}$ is an algebra, then k is universal.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

We check the three Stone–Weierstrass hypotheses for $\mathcal{A} \subseteq C(\mathcal{X})$, then transfer density to the RKHS.

$\mathcal{A} \subseteq C(\mathcal{X})$. The coordinates φ_i are continuous because $x \mapsto \Phi(x)$ is continuous: indeed $\|\Phi(x) - \Phi(y)\|^2 = K(x, x) + K(y, y) - 2K(x, y) \rightarrow 0$ as $y \rightarrow x$ since K is continuous, so each φ_i and every element of their span is continuous.

Does not vanish. If some x had $\varphi_i(x) = 0$ for all i , then $K(x, x) = \sum_i \varphi_i(x)^2 = 0$, contradicting $k(x, x) > 0$.

Separates points. If $x \neq y$ had $\varphi_i(x) = \varphi_i(y)$ for all i , then $\Phi(x) = \Phi(y)$, contradicting injectivity of Φ .

By hypothesis $\mathcal{A}$ is an algebra, so Stone–Weierstrass gives that $\mathcal{A}$ is dense in $C(\mathcal{X})$. Finally we lift this to $\mathcal{H}$. Let $f \in C(\mathcal{X})$ and $\epsilon > 0$; pick $g \in \mathcal{A}$ with $\|f - g\|_{\infty} < \epsilon$. By definition of $\mathcal{A}$, $g(x) = \langle w, \Phi(x) \rangle_{\ell^2}$ for some $w = (w_i)_{i \geq 0}$ with $w_i = 0$ for all $i > N$, for some finite N . Such a finite combination of features belongs to the RKHS $\mathcal{H}$ of k . Hence $\mathcal{H}$ is dense in $C(\mathcal{X})$, so k is universal. $\square$

30.9.2 Universality from a Taylor series

The general criterion turns universality into a bookkeeping question about the feature coordinates. When the kernel is a power series of the inner product with positive coefficients, the features are monomials and their span is the polynomial algebra.

Proposition (Taylor criterion, Steinwart 2001)

Let $0 < r \leq \infty$ and let $f : (-r, r) \rightarrow \mathbb{R}$ be a C^∞ function with Taylor expansion $f(x) = \sum_{i=0}^{\infty} a_i x^i$ at 0. Let $\mathcal{X}$ be a compact subset of the open ball in $\mathbb{R}^d$ of radius $\sqrt{r}$ centered at the origin. If $a_i > 0$ for all $i \geq 0$, then $k(x, y) = f(\langle x, y \rangle)$ is a universal kernel on $\mathcal{X}$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Take $d = 1$ for clarity. The kernel is continuous and expands as

$$K(x, y) = \sum_{i=0}^{\infty} a_i x^i y^i = \langle \Phi(x), \Phi(y) \rangle_{\ell^2}, \quad \Phi(x) = (\sqrt{a_i} x^i)_{i \geq 0},$$

which is legitimate since $a_i > 0$. The map Φ is injective because its coordinates already contain x itself (up to the positive factor $\sqrt{a_1}$). Moreover $K(x, x) = \sum_i a_i x^{2i} > 0$ as all $a_i > 0$. Finally $\mathcal{A} = \text{span}\{\varphi_i : i \geq 0\}$ is the algebra of polynomials, closed under products. All hypotheses of the general criterion hold, so K is universal. $\square$

Two examples. The exponential kernel $K(x, y) = \exp\langle x, y \rangle$ on any compact $\mathcal{X}$ uses $f(x) = e^x = \sum_i \frac{1}{i!} x^i$, whose coefficients $a_i = 1/i! > 0$; it is therefore universal, confirming the moment-generating-function intuition from before. The Gaussian kernel *on the unit sphere*, $K(x, y) = \exp(-\frac{1}{2}\|x - y\|^2)$, factors on $\|x\| = \|y\| = 1$ as $e^{-1} \exp\langle x, y \rangle$ because $\|x - y\|^2 = 2 - 2\langle x, y \rangle$; with $f(x) = e^{-1} e^x = e^{-1} \sum_i \frac{1}{i!} x^i$ the coefficients are again all positive, so it too is universal.

30.9.3 Universality from a Fourier series

A parallel construction handles translation-structured kernels on a box, using a Fourier expansion in place of a Taylor expansion.

Proposition (Fourier criterion, Steinwart 2001)

Let $f : [0, 2\pi] \rightarrow \mathbb{R}$ be continuous with a pointwise absolutely convergent Fourier series $f(t) = \sum_{n=0}^{\infty} a_n \cos(nt)$. If $a_n > 0$ for all $n \geq 0$, then

$$K(x, y) := \prod_{i=1}^d f(|x_i - y_i|)$$

is a universal kernel on every compact subset of $[0, 2\pi]^d$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Take $d = 1$. Using $\cos(n|x - y|) = \cos(n(x - y)) = \cos(nx) \cos(ny) + \sin(nx) \sin(ny)$, the kernel is continuous and expands as

$$K(x, y) = a_0 + \sum_{n=1}^{\infty} a_n (\sin(nx) \sin(ny) + \cos(nx) \cos(ny)) = \langle \Phi(x), \Phi(y) \rangle_{\ell^2},$$

with coordinates $\varphi_0(x) = \sqrt{a_0}$, $\varphi_{2n-1}(x) = \sqrt{a_n} \sin(nx)$, and $\varphi_{2n}(x) = \sqrt{a_n} \cos(nx)$ for $n \geq 1$. This Φ is injective on $[0, 2\pi]$, and $K(x, x) = \sum_{n \geq 0} a_n > 0$ since every $a_n > 0$. The span $\mathcal{A} = \text{span}\{\varphi_n : n \geq 0\}$ is an algebra, because products of sines and cosines reduce to sines and cosines of summed frequencies by the trigonometric identities, staying inside the span. The general criterion applies, so K is universal. $\square$

An example in this family is the regularized Fourier kernel (Vapnik 1998, p.470),

$$k(x, y) = \frac{1 - q^2}{2 - 4q \cos(x - y) + 2q^2}, \quad 0 < q < 1,$$

whose cosine coefficients are all positive.

To summarize the compact case: on a compact metric space, a universal kernel is a continuous kernel whose RKHS is dense in $C(\mathcal{X})$ in the maximum norm, and every universal kernel is characteristic. A large class of universal kernels arises from Taylor or Fourier series with positive coefficients, both special cases of the general criterion, which is itself a consequence of the Stone–Weierstrass theorem. This leaves one gap: what if $\mathcal{X}$ is not compact, for example $\mathcal{X} = \mathbb{R}^d$?

30.10 Characteristic kernels via the Fourier transform

On $\mathbb{R}^d$ the compactness argument fails, but for translation-invariant kernels a spectral argument takes its place. Consider $K(x, y) = \kappa(x - y)$ with $\kappa : \mathbb{R}^d \rightarrow \mathbb{R}$. Bochner's theorem says such a κ is (the transform of) a finite non-negative Borel measure Λ on $\mathbb{R}^d$,

$$\kappa(z) = \int_{\mathbb{R}^d} e^{-iz^\top w} d\Lambda(w).$$

This spectral measure Λ exposes the kernel as an inner product of Fourier features in $L^2(\Lambda)$: with $w \mapsto \Phi(x)(w) = e^{-ix^\top w}$,

$$K(x, y) = \langle \Phi(x), \Phi(y) \rangle_{L^2(\Lambda)}.$$

The mean embedding inherits the same structure. Writing $\mathcal{F}(P) = \mathbb{E}_{X \sim P}[\Phi(X)]$ for the Fourier transform of P ,

$$\mu_P(y) = \mathbb{E}_{X \sim P}[\langle \Phi(X), \Phi(y) \rangle_{L^2(\Lambda)}] = \langle \mathcal{F}(P), \Phi(y) \rangle_{L^2(\Lambda)}.$$

So the embedding is the Fourier transform of P , read against the Fourier features and weighted by Λ . Whether the embedding is injective now becomes a question about whether Λ lets us recover $\mathcal{F}(P)$. Two facts combine. The Fourier inversion theorem (Dudley 2002, Theorem 9.5.4) says the Fourier transform already determines the distribution: if $\mathcal{F}(P) = \mathcal{F}(Q)$ then $P = Q$. The remaining issue is that Λ must not annihilate any part of $\mathcal{F}(P)$; it must "preserve the information" in the Fourier transform. It does so exactly when its support is all of $\mathbb{R}^d$.

Theorem (translation-invariant characteristic kernels, Sriperumbudur 2008)

Let K be a translation-invariant kernel on $\mathbb{R}^d$, $K(x, y) = \kappa(x - y)$ with $\kappa(z) = \int e^{-iz^\top w} d\Lambda(w)$ for a finite non-negative Borel measure Λ . Then K is characteristic if and only if

$$\text{supp}(\Lambda) = \mathbb{R}^d.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The condition is intuitive: if Λ put no mass on some frequency region, distributions differing only at those frequencies would share an embedding, and the kernel would be blind to them. A full-support spectral measure sees every frequency.

Two examples close the discussion. The Gaussian kernel $K(x, y) = e^{-\frac{\sigma^2}{2} \|x-y\|^2}$ has spectral measure Λ with density $w \mapsto (1/\sqrt{2\pi\sigma^2})^d e^{-\frac{1}{2\sigma^2} \|w\|^2}$, a Gaussian on $\mathbb{R}^d$. Its support is all of $\mathbb{R}^d$, so the Gaussian kernel is characteristic. By contrast, the sinc kernel $\kappa(z) = z^{-1} \sin(z)$ has spectral measure equal to the uniform distribution on $[-1, 1]$, whose support is bounded; because it misses the high frequencies, $K(x, y) = \kappa(x - y)$ is *not* characteristic.

Further reading

Unlike most of this book, the material in this chapter largely postdates the two standard textbooks, so the primary sources are journal articles rather than monograph chapters. The mean embedding and the maximum mean discrepancy were introduced by Smola, Gretton, Song, and Schölkopf (2007), and the two-sample test was introduced by Gretton, Borgwardt, Rasch, Schölkopf, and Smola (2006), whose definitive treatment, including the asymptotic null distribution and the permutation calibration used here, is Gretton, Borgwardt, Rasch, Schölkopf, and Smola (2012). The Hilbert-Schmidt Independence Criterion is due to Gretton, Bousquet, Smola, and Schölkopf (2005), and the identification of energy distance with MMD is due to Sejdinovic, Sriperumbudur, Gretton, and Fukumizu (2013). The theory of characteristic kernels and the equivalence with universality and full spectral

support is developed by Steinwart (2001) for universality and by Sriperumbudur, Gretton, Fukumizu, Schölkopf, and Lanckriet (2010) for the characteristic property and its metrics on probability measures; the dependence-detection applications mentioned in the summary trace to Fukumizu, Gretton, Sun, and Schölkopf (2008). For a single comprehensive entry point that unifies embeddings, MMD, characteristic kernels, and their applications, the survey by Muandet, Fukumizu, Sriperumbudur, and Schölkopf (2017), *Kernel Mean Embedding of Distributions: A Review and Beyond*, is the standard reference. The underlying RKHS machinery, reproducing property, feature maps, and the kernel trick that all of the above build on, is presented in the textbook of Shawe-Taylor and Cristianini (2004).

30.11 Summary

Probability distributions can be represented by kernels through their mean embeddings $\mu_P = \mathbb{E}_{X \sim P}[K_X]$, a single RKHS vector that stores the expectation under P of every function in $\mathcal{H}$. The maximum mean discrepancy, the RKHS distance $\|\mu_P - \mu_Q\|_{\mathcal{H}}$ between embeddings, compares distributions, expands into expectations of the kernel that are estimable from samples, and doubles as the integral probability metric over the RKHS unit ball with the witness function as its optimal test direction. MMD drives two applications developed here: two-sample tests, whose null distribution is an infinite χ^2 mixture calibrated by permutation, and implicit generative models trained by minimizing $\text{MMD}^2(P, Q_\theta)$, which are GANs with an analytically solved critic and can be sharpened by learning the kernel adversarially. Other applications include dependence detection, feature selection, and blind source separation such as ICA. All of this rests on the kernel being characteristic, so that the embedding discards no information about the distribution. On a compact domain characteristic follows from universality, obtainable from a Taylor or Fourier expansion with positive coefficients via the Stone–Weierstrass theorem. On $\mathbb{R}^d$, a translation-invariant kernel is characteristic if and only if its spectral measure has full support, which holds for the Gaussian kernel but not for the sinc kernel.

30.12 Common mistakes and practical implications

For **Mean Embeddings, MMD, and Characteristic Kernels**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

30.13 Exercises

1. warm-up Let P and Q be two Gaussians on $\mathbb{R}$ with the same mean but different variances. Explain why comparing the raw sample means $\hat{\mu}_P = \frac{1}{N} \sum_i x_i$ and $\hat{\mu}_Q = \frac{1}{M} \sum_j y_j$ is structurally unable to tell them apart, no matter how many samples are drawn. Then give one explicit feature coordinate $\varphi(x)$ whose mean under P differs from its mean under Q , and say in one sentence what an RKHS supplies that a fixed finite list of such features does not.
2. warm-up Take the point mass $P = \delta_a$, the distribution that places all its weight at a single point $a \in \mathcal{X}$. Show directly from the definition $\mu_P = \mathbb{E}_{X \sim P}[K_X]$ that $\mu_{\delta_a} = K_a$. Using this, argue that the empirical embedding $\hat{\mu}_P = \frac{1}{N} \sum_{i=1}^N K_{X_i}$ is exactly the mean embedding of the empirical distribution $\frac{1}{N} \sum_i \delta_{X_i}$, so that estimating μ_P from data is nothing more than averaging one feature vector per sample.

3. computation Work on $\mathcal{X} = \mathbb{R}$ with the Gaussian kernel $K(x, y) = \exp(-\frac{1}{2}(x - y)^2)$. Let the two samples be $x_1 = 0, x_2 = 1$ from P and $y_1 = 3, y_2 = 4$ from Q , so $N = M = 2$. Write out the biased estimator

$$\widehat{\text{MMD}}_b^2(P, Q) = \frac{1}{4} \sum_{i,j} K(x_i, x_j) + \frac{1}{4} \sum_{i,j} K(y_i, y_j) - \frac{2}{4} \sum_{i,j} K(x_i, y_j)$$

term by term as a sum of exponentials, then evaluate it numerically. Repeat with the unbiased estimator (drop the diagonal pairs $i = j$ from the two intra-similarity sums) and explain why the two numbers differ in the direction they do. Hint

Each intra sum has four terms, two of which are the diagonal $K(x_i, x_i) = 1$. The unbiased estimator removes exactly those inflated 1s, so it reports a smaller intra-similarity and hence a smaller discrepancy. For the samples stated here it is approximately 1.1341, still positive; unbiased MMD estimates can be negative on other finite samples even though population MMD^2 is nonnegative.

4. proof Starting from the definition $\text{MMD}(P, Q) = \|\mu_P - \mu_Q\|_{\mathcal{H}}$, derive the expansion

$$\text{MMD}^2(P, Q) = \mathbb{E}_{X, X'}[k(X, X')] + \mathbb{E}_{Y, Y'}[k(Y, Y')] - 2 \mathbb{E}_{X, Y}[k(X, Y)],$$

with $X, X' \sim P$ independent and $Y, Y' \sim Q$ independent. State clearly which two properties of the mean embedding you use, namely bilinearity of the RKHS inner product and the identity $\langle \mu_P, \mu_Q \rangle_{\mathcal{H}} = \mathbb{E}_{X, Y}[K(X, Y)]$, and where each enters. Hint

Expand $\|\mu_P - \mu_Q\|^2 = \langle \mu_P, \mu_P \rangle + \langle \mu_Q, \mu_Q \rangle - 2\langle \mu_P, \mu_Q \rangle$. Then write $\mu_P = \mathbb{E}_X[K_X]$, $\mu_Q = \mathbb{E}_Y[K_Y]$ and pull the expectations out of each inner product, using $\langle K_X, K_{X'} \rangle_{\mathcal{H}} = k(X, X')$.

5. proof Consider the linear kernel $K(x, x') = x^\top x'$ on $\mathbb{R}^d$. Show that its mean embedding is $\mu_P(x) = \mathbb{E}_{X \sim P}[X]^\top x$, so that μ_P depends on P only through its mean $m_P = \mathbb{E}_{X \sim P}[X]$. Conclude that

$$\text{MMD}^2(P, Q) = \|m_P - m_Q\|_2^2$$

for the linear kernel, and hence that any two distributions with equal means have MMD zero even when $P \neq Q$: the linear kernel is not characteristic. Then indicate what an order-two polynomial kernel $K(x, x') = (x^\top x')^2$ adds, and what it still cannot see. Hint

Plug $K(X, Y) = X^\top Y$ into the expansion of the previous exercise; each expectation factors as a dot product of the per-distribution means because the two draws in every pair are independent. For the polynomial kernel, μ_P reads off $\mathbb{E}[XX^\top]$, the second moment, and nothing of order three or higher.

6. proof Assume the kernel is characteristic, so that $\text{MMD}(P, Q) = 0$ forces $P = Q$. Prove that MMD is then a genuine metric on the set of Borel probability measures: verify non-negativity, symmetry $\text{MMD}(P, Q) = \text{MMD}(Q, P)$, the identity of indiscernibles $\text{MMD}(P, Q) = 0 \iff P = Q$, and the triangle inequality $\text{MMD}(P, R) \leq \text{MMD}(P, Q) + \text{MMD}(Q, R)$. Point out exactly which of these four fails, and which survive, when the kernel is not characteristic. Hint

All four properties are inherited from the norm $\|\cdot\|_{\mathcal{H}}$ applied to the embeddings, since $\text{MMD}(P, Q) = \|\mu_P - \mu_Q\|_{\mathcal{H}}$; the triangle inequality is the RKHS triangle inequality on $\mu_P - \mu_R = (\mu_P - \mu_Q) + (\mu_Q - \mu_R)$. Only the forward direction of the identity of indiscernibles uses the characteristic property; without it MMD is merely a pseudometric (the triangle inequality still holds; only the identity of indiscernibles fails).

7. exploration Open the interactive two-sample figure and drag one point cloud away from the other while watching the maximum mean discrepancy recomputed live from the points. Describe how the value behaves as the clouds separate and as you bring them back to coincide, and connect what you see to the three-term expansion: say which of the intra-similarity and inter-similarity averages moves, and in which direction, as you pull the clouds apart. Then place the two clouds on top of each other and explain why the reported discrepancy does not settle at exactly zero even though the underlying distributions now match. Hint

Separating the clouds leaves the two intra-similarity averages roughly fixed but drives the cross average $\mathbb{E}[k(X, Y)]$ down, so the discrepancy grows. When the clouds coincide, the leftover value is finite-sample noise: the estimator averages over specific points, not the true distributions, so it fluctuates around, rather than exactly reaching, zero.

8. challenge Two parts on the two-sample test. (a) The witness function is $f^\star = (\mu_P - \mu_Q)/\|\mu_P - \mu_Q\|_{\mathcal{H}}$. Using the empirical embeddings, write the estimated witness $\hat{f}^\star(t) \propto \frac{1}{N} \sum_i K(x_i, t) - \frac{1}{M} \sum_j K(y_j, t)$ and explain why, for a Gaussian kernel, it is positive where P has more mass than Q and negative where Q exceeds P ; relate its squared achievable gap to MMD^2 . (b) You run a permutation test with $J = 1000$ reshufflings of the pooled labels and find that 9 of the permuted statistics T_j exceed the observed $T_0 = n \widehat{\text{MMD}}^2$. Compute the approximate p -value and state the decision at level $\alpha = 0.05$. Finally, argue that if you had used a linear kernel and P, Q shared a mean, the test would have no population signal against that alternative and could reject only at its nominal false-positive rate, however many permutations were used. Hint

For (a), the witness is the difference of the two kernel-smoothed densities, and by Cauchy-Schwarz its maximal gap over the unit ball equals $\|\mu_P - \mu_Q\|_{\mathcal{H}} = \text{MMD}(P, Q)$. For (b), the common plus-one estimate is $p = (9 + 1)/(1000 + 1) \approx 0.010 \leq 0.05$, so reject the null. With a linear kernel the population MMD is $\|m_P - m_Q\|_2$, which is zero when the means agree. More permutations refine the Monte Carlo resolution but do not create power against a distributional difference invisible to the chosen statistic; a level- α test may still reject with probability α by chance.

31 Kernel Hypothesis Testing

The previous chapter on mean embeddings and the MMD built the object we compare distributions with, and it sketched the two-sample test: a null distribution that is an infinite mixture of chi-squares, calibrated by permutation. That sketch left the real machinery unbuilt: a test is not a statistic but a decision procedure with guarantees. This chapter turns the MMD into a proper hypothesis test. We separate the biased V-statistic from the unbiased U-statistic and prove the unbiasedness, we explain precisely why the null law degenerates into a chi-square mixture while the alternative is asymptotically normal, and we define Type-I error, Type-II error, and test power exactly. Power is the quantity we then optimize: we derive the signal-to-noise objective that governs it, examine the median heuristic and where it fails, and survey the modern answers, learned and deep kernels, linear-time estimators for scale, and aggregated tests that dodge data splitting. We close with independence and interaction testing through HSIC, and a pointer to goodness-of-fit against a fixed model through the kernel Stein discrepancy.

31.1 From a discrepancy to a decision

We are given samples $x_1, \dots, x_n \sim P$ and $y_1, \dots, y_m \sim Q$ and must decide between the null hypothesis $H_0 : P = Q$ and the alternative $H_1 : P \neq Q$. The MMD supplies a statistic that is zero exactly when a characteristic kernel cannot tell P from Q , and positive otherwise, but a positive estimate is not yet a verdict. A finite sample produces a positive $\widehat{\text{MMD}}^2$ even when $P = Q$, simply from sampling noise. The whole problem is to decide when the observed value is too large to be noise. The kernel two-sample test that solves it was introduced by Gretton et al. (2006); we build out its testing theory here.

A test is a rule that outputs reject or retain. It can be wrong in two ways, and the asymmetry between them is the foundation of the theory.

Definition (errors, level, and power)

A test rejecting H_0 when a statistic T exceeds a threshold c commits a *Type-I error* if it rejects while H_0 holds, and a *Type-II error* if it retains while H_1 holds. Their probabilities are

$$\alpha_T = \mathbb{P}(T > c \mid H_0), \quad \beta_T = \mathbb{P}(T \leq c \mid H_1).$$

The test has *level* α if $\alpha_T \leq \alpha$ for every $P = Q$, and its *power* against a specific alternative is

$$\Pi = 1 - \beta_T = \mathbb{P}(T > c \mid H_1).$$

The two errors are treated asymmetrically. We first fix the level, insisting $\alpha_T \leq \alpha$ (a small value such as 0.05) whatever $P = Q$ is, so that a false alarm is rare by construction; only then, among tests that respect the level, do we maximize power. This ordering dictates everything below: the threshold c is set from the *null* distribution to hit level α , and the kernel is chosen to push the statistic far into the tail under the *alternative*. A test that ignores the first job is worthless however powerful it looks; a valid test that ignores the second wastes data.

31.2 The V-statistic and the U-statistic

Two estimators of $\text{MMD}^2(P, Q)$ sit at the heart of the test, and their difference is not cosmetic. Recall from Chapter 29, Mean Embeddings, MMD, and Characteristic Kernels the population identity $\text{MMD}^2 = \mathbb{E}_{P \otimes P}[k(X, X')] + \mathbb{E}_{Q \otimes Q}[k(Y, Y')] - 2 \mathbb{E}_{P \otimes Q}[k(X, Y)]$. Replacing every expectation by an average over all pairs, including the diagonal ones, gives the *V-statistic*

$$\widehat{\text{MMD}}_V^2 = \frac{1}{n^2} \sum_{i,j} k(x_i, x_j) + \frac{1}{m^2} \sum_{i,j} k(y_i, y_j) - \frac{2}{nm} \sum_{i,j} k(x_i, y_j) = \|\hat{\mu}_P - \hat{\mu}_Q\|_{\mathcal{H}}^2,$$

which is manifestly nonnegative, being the squared RKHS norm of a difference of empirical embeddings. Its nonnegativity is bought at a price: the diagonal terms $k(x_i, x_i)$ pair a point with itself, which is not an unbiased estimate of $\mathbb{E}_{P \otimes P}[k(X, X')]$ over *independent* draws, so $\widehat{\text{MMD}}_V^2$ overestimates the truth by an $O(1/n)$ bias. Dropping the diagonal removes the bias and yields the *U-statistic*, for equal sample sizes $n = m$,

$$\widehat{\text{MMD}}_U^2 = \frac{1}{n(n-1)} \sum_{i \neq j} [k(x_i, x_j) + k(y_i, y_j) - k(x_i, y_j) - k(x_j, y_i)] = \frac{1}{n(n-1)} \sum_{i \neq j} h(z_i, z_j),$$

where we have paired the data into $z_i = (x_i, y_i)$ and collected the four kernel terms into a single symmetric *core* $h(z_i, z_j) = k(x_i, x_j) + k(y_i, y_j) - k(x_i, y_j) - k(x_j, y_i)$. Writing it this way exposes $\widehat{\text{MMD}}_U^2$ as a textbook second-order U-statistic, the average of a symmetric core over distinct pairs, which is exactly the structure whose sampling theory (Serfling, 1980) delivers both the unbiasedness and the null law.

Proposition (the U-statistic is unbiased)

For $n = m \geq 2$, $\mathbb{E}[\widehat{\text{MMD}}_U^2] = \text{MMD}^2(P, Q)$ exactly, at every sample size.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Because the pairs z_i are i.i.d. and h is symmetric, every one of the $n(n-1)$ ordered distinct pairs has the same expectation, so $\mathbb{E}[\widehat{\text{MMD}}_U^2] = \mathbb{E}[h(z_1, z_2)]$. Now $z_1 = (x_1, y_1)$ and $z_2 = (x_2, y_2)$ are independent, so the four cores separate:

$$\mathbb{E}[h(z_1, z_2)] = \mathbb{E}[k(x_1, x_2)] + \mathbb{E}[k(y_1, y_2)] - \mathbb{E}[k(x_1, y_2)] - \mathbb{E}[k(x_2, y_1)].$$

Here $x_1, x_2 \sim P$ independently, so $\mathbb{E}[k(x_1, x_2)] = \mathbb{E}_{P \otimes P}[k(X, X')]$; likewise $\mathbb{E}[k(y_1, y_2)] = \mathbb{E}_{Q \otimes Q}[k(Y, Y')]$; and each cross term is $\mathbb{E}_{P \otimes Q}[k(X, Y)]$ since the two arguments are an independent P -draw and Q -draw. Summing gives $\mathbb{E}_{P \otimes P}[k] + \mathbb{E}_{Q \otimes Q}[k] - 2 \mathbb{E}_{P \otimes Q}[k] = \text{MMD}^2(P, Q)$. No diagonal term ever appears, so the equality is exact and holds for all n . $\square$

Unbiasedness has a cost that will matter in a moment: $\widehat{\text{MMD}}_U^2$ can come out negative, because subtracting the cross terms from a diagonal-free intra-similarity leaves a quantity with no reason to stay positive on a finite sample. The V-statistic never does this but is biased. For testing we use the unbiased U-statistic, and its occasional negativity is a feature, a signal that the observed discrepancy is below what independent relabelings of the same points already produce.

31.3 Why the null law is a chi-square mixture

To set the threshold we need the distribution of $\widehat{\text{MMD}}_U^2$, and it looks completely different under the two hypotheses. The reason is a single structural fact about the core h : its behavior when we average out one argument.

Define the *first-order projection* $h_1(z) = \mathbb{E}_{z'}[h(z, z')] - \text{MMD}^2$. A short computation evaluates it. Fixing $z = (x, y)$ and averaging $z' = (X', Y')$,

$$\mathbb{E}_{z'}[h(z, z')] = \mu_P(x) + \mu_Q(y) - \mu_Q(x) - \mu_P(y) = g(x) - g(y), \quad g := \mu_P - \mu_Q,$$

using $\mathbb{E}_{X'}[k(x, X')] = \mu_P(x)$ and the like. So $h_1(z) = g(x) - g(y) - \text{MMD}^2$, governed entirely by the witness function $g = \mu_P - \mu_Q$ of Chapter 29, Mean Embeddings, MMD, and Characteristic Kernels. Under the alternative $g \neq 0$, the projection h_1 is a nonconstant function, the U-statistic is *nondegenerate*, and the classical central limit theorem for U-statistics applies (Gretton et al., 2012): with $V_{H_1} = 4 \text{Var}_z[h_1(z)]$,

$$\sqrt{n}(\widehat{\text{MMD}}_U^2 - \text{MMD}^2) \xrightarrow{d} \mathcal{N}(0, V_{H_1}).$$

Under the null $P = Q$, everything changes. Then $g \equiv 0$, so $h_1 \equiv 0$: the first-order projection vanishes identically and the U-statistic is *degenerate*. Its leading fluctuation is not first order but second order, and a quadratic form in Gaussian noise is not Gaussian. Spectral analysis of the core makes the limit explicit.

Theorem (null distribution, Gretton et al., 2012)

Under $H_0 : P = Q$, the rescaled U-statistic converges in distribution to an infinite weighted sum of centered chi-squares,

$$n \widehat{\text{MMD}}_U^2 \xrightarrow{d} 2 \sum_{l=1}^{\infty} \lambda_l (Z_l^2 - 1),$$

where the $Z_l \sim \mathcal{N}(0, 1)$ are i.i.d. and the weights λ_l are the eigenvalues of the centered kernel $\tilde{k}(x, x') = \langle K_x - \mu_P, K_{x'} - \mu_P \rangle_{\mathcal{H}}$ under P , that is, the solutions of $\int \tilde{k}(x, x') \psi_l(x') dP(x') = \lambda_l \psi_l(x)$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof (sketch)

Mercer-expand the centered kernel under P as $\tilde{k}(x, x') = \sum_l \lambda_l \psi_l(x) \psi_l(x')$, with $\{\psi_l\}$ orthonormal in $L^2(P)$ and each centered, $\mathbb{E}_P[\psi_l] = 0$. Under H_0 the U-statistic core is $\tilde{k}$ itself up to the centering, so

$$n \widehat{\text{MMD}}_U^2 \approx \sum_l \lambda_l \left[\left(\frac{1}{\sqrt{n}} \sum_{i=1}^n \psi_l(x_i) \right)^2 - \frac{1}{n} \sum_{i=1}^n \psi_l(x_i)^2 \right].$$

For each l , the normalized sum $\frac{1}{\sqrt{n}} \sum_i \psi_l(x_i)$ converges to a standard Gaussian Z_l by the central limit theorem, since ψ_l is centered with unit variance, and the Z_l are asymptotically independent across l by the orthonormality of the ψ_l . The second term is the diagonal correction that unbiasedness subtracts, and it converges to $\mathbb{E}_P[\psi_l^2] = 1$. Hence the l -th summand tends to $\lambda_l(Z_l^2 - 1)$, a centered chi-square with one degree of freedom, and the constant collects into the stated factor. $\square$

Two consequences follow. First, the null law has mean zero, matching $\mathbb{E}[\widehat{\text{MMD}}_U^2] = 0$ when $P = Q$, and it lives on a scale $O(1/n)$, whereas under the alternative $\widehat{\text{MMD}}_U^2 \rightarrow \text{MMD}^2 > 0$ on scale $O(1)$; the test works because these two scales pull apart as n grows. Second, the null distribution depends on the unknown eigenvalues λ_l , which depend on P and the kernel, so the threshold c_α is not available in closed form. We must estimate the null.

31.4 Calibrating the null by permutation

Rather than estimate the eigenvalues, we simulate the null directly, exploiting a symmetry that holds exactly under H_0 . If $P = Q$, the group label carried by each point is uninformative: which bag a point came from is arbitrary, so the pooled sample $\{x_1, \dots, x_n, y_1, \dots, y_n\}$ is *exchangeable*, and any reassignment of the $2n$ points into two groups of n is as legitimate as the observed one. Recomputing the statistic on many such reassignments traces out the null distribution without ever touching the eigenvalues.

Concretely, let $T_0 = \widehat{\text{MMD}}_U^2$ on the true labels, and for each of B random relabelings π compute T_π by splitting the pooled points into two new groups and re-evaluating the U-statistic. The p -value is the fraction of permuted statistics that match or exceed the observed one, with the observed labeling included for exact validity,

$$\hat{p} = \frac{1 + |\{\pi : T_\pi \geq T_0\}|}{1 + B}.$$

Including the identity in the count is what makes the test exactly level α : under H_0 the observed labeling is exchangeable with the permuted ones, so T_0 is equally likely to hold any rank among the $B + 1$ values, and $\mathbb{P}(\hat{p} \leq \alpha \mid H_0) \leq \alpha$ for every $P = Q$. This validity is nonasymptotic and holds for any kernel, which is why the permutation test is the default calibration. When n is tiny the pooled sample admits few distinct relabelings and one enumerates them all, giving an *exact* test; when n is large one draws B permutations at random.

Algorithm (permutation two-sample MMD test)

Input samples $x_{1:n} \sim P, y_{1:n} \sim Q$; kernel k ; level α ; permutation count B .

Output decision to reject $H_0 : P = Q$ or not, with p -value $\hat{p}$.

1. Compute the observed statistic $T_0 = \widehat{\text{MMD}}_U^2(x_{1:n}, y_{1:n})$.
2. Pool the $2n$ points into $z_{1:2n}$.
3. For $\pi = 1, \dots, B$: draw a random partition of $z_{1:2n}$ into two groups of n ; compute $T_\pi = \widehat{\text{MMD}}_U^2$ on that partition.
4. Form $\hat{p} = (1 + |\{\pi : T_\pi \geq T_0\}|)/(1 + B)$.
5. Reject H_0 if $\hat{p} \leq \alpha$.

The procedure needs nothing but repeated kernel evaluations, and it is easiest to trust on a case small enough to enumerate by hand.

Example (an exact permutation test on eight points)

Two samples on the line, $X = \{0, 1, 2, 3\} \sim P$ and $Y = \{7, 8, 9, 10\} \sim Q$, so $n = m = 4$. Gaussian kernel $k(x, x') = \exp(-(x - x')^2/(2\sigma^2))$, bandwidth by the median heuristic. Level $\alpha = 0.05$.

1. Set the bandwidth. The median of the 28 pooled pairwise distances $|z_i - z_j|$ is $\sigma = 5$, so the kernel uses $2\sigma^2 = 50$.
2. Assemble the block averages. The off-diagonal within- X average and within- Y average are equal by translation, 0.937016, and the cross average is 0.391622.
3. Form the two statistics. The biased V-statistic (diagonal kept) is $\widehat{\text{MMD}}_V^2 = 1.122280$; the unbiased U-statistic is $\widehat{\text{MMD}}_U^2 = 0.937016 + 0.937016 - 2(0.391622) = 1.090788$.
4. Enumerate the exact null. All $\binom{8}{4} = 70$ relabelings give a mean of exactly 0.000000 and standard deviation 0.253891; the largest null value is 1.090788, attained by the observed split and its mirror image (swapping which group is called P).
5. Read the p -value. Only those two relabelings reach T_0 , so $\hat{p} = 2/70 = 0.0286 \leq 0.05$: reject H_0 .

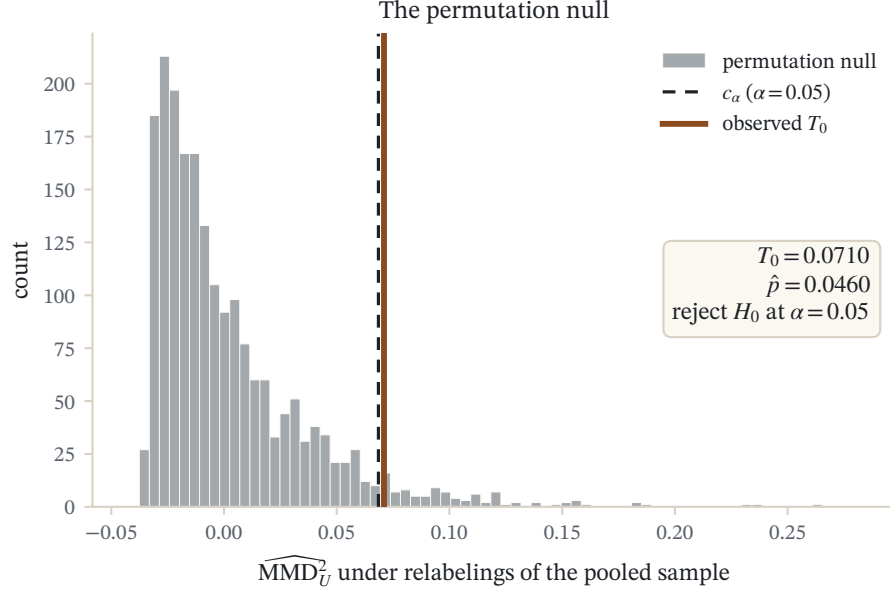


Figure 31.1: The permutation null distribution of the kernel two-sample statistic, built by repeatedly relabeling the pooled samples. The observed statistic (line) against the $\alpha = 0.05$ critical quantile gives the test's p-value.

Reading. The observed discrepancy is matched by only its own mirror among seventy relabelings, a decisive rejection. The permutation-null mean sitting at exactly 0 is the unbiasedness of the U-statistic made visible: averaging over all label assignments reproduces the population value $\text{MMD}^2 = 0$ that holds under H_0 . With four points per sample the smallest attainable p -value is $2/70 \approx 0.029$, already fine enough to clear 0.05.

Verification artifact. checks/example-ch-testing-example-30-1.json records the example source hash and verification scope.

An alternative to permutation is to bootstrap the chi-square mixture directly, estimating the eigenvalues λ_l from the spectrum of the centered Gram matrix, or to fit a two-parameter Gamma to the null by matching its first two moments (Gretton et al., 2012). These are faster when many tests are run, but the permutation test is exact and assumption-free, so we take it as the reference.

31.5 Test power and the objective for kernel choice

With the level pinned by permutation, the remaining freedom is the kernel, and it is decisive. Two characteristic kernels both give valid tests, yet one may reject at $n = 50$ where the other needs $n = 5000$. To choose well we need to know what power depends on. Combining the two regimes of the null and alternative laws yields a clean asymptotic answer.

Proposition (asymptotic power, Sutherland et al., 2017)

Fix a characteristic kernel and an alternative $P \neq Q$ with $\text{MMD}^2 > 0$ and first-order variance $V_{H_1} = 4 \text{Var}_z[h_1(z)] =: \sigma_{H_1}^2$. The level- α test that rejects when $\widehat{\text{MMD}}_U^2 > c_\alpha$ has power

$$\Pi \approx \Phi\left(\frac{\sqrt{n} \text{MMD}^2}{\sigma_{H_1}} - \frac{c_\alpha \sqrt{n}}{\sigma_{H_1}}\right),$$

where Φ is the standard normal CDF. Since the null concentrates on scale $O(1/n)$, the threshold satisfies $c_\alpha = O(1/n)$, so the second term is $O(1/\sqrt{n}) \rightarrow 0$ and, for large n ,

$$\Pi \approx \Phi\left(\sqrt{n} \frac{\text{MMD}^2}{\sigma_{H_1}}\right).$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The message is that, at fixed n and α , power is an increasing function of the single ratio

$$J(k) = \frac{\text{MMD}^2(P, Q)}{\sigma_{H_1}},$$

a signal-to-noise ratio: the squared discrepancy the kernel produces, divided by the sampling fluctuation of that discrepancy under the alternative. Maximizing power is therefore maximizing J over kernels. One caution comes with the objective: selecting the kernel to maximize the estimated $\hat{J}$ on the *same* data used for the test would reuse the sample twice and inflate the Type-I error, because a kernel tuned to the observed noise makes even $P = Q$ look discrepant. The fix is *data splitting*: choose the kernel on one half and test on the other, so the selection is independent of the test statistic and the level is preserved.

Algorithm (power-maximizing kernel selection by data splitting)

Input pooled samples from P and Q ; kernel family $\{k_\theta\}$; level α ; split fraction ρ .

Output a level- α decision using a kernel tuned for power.

1. Split each sample into a selection set (fraction ρ) and a test set (the rest).
2. On the selection set, form the estimator $\hat{J}(\theta) = \widehat{\text{MMD}}_U^2(\theta) / \hat{\sigma}_{H_1}(\theta)$ with a smooth, regularized estimate of σ_{H_1} .
3. Maximize $\hat{J}(\theta)$ over θ by gradient ascent to obtain $k_{\theta^\star}$.
4. On the test set, run the permutation test of the previous section with $k_{\theta^\star}$ at level α .

Because the selection step only ever inspects the selection set, the test set statistic keeps its permutation-calibrated null, and the test is exactly level α whatever the optimizer does. The price is that each half is smaller, a tension the aggregated tests below are designed to avoid.

31.6 The median heuristic and its limits

Before optimizing a kernel one needs a default, and the near-universal default for a Gaussian kernel is the *median heuristic*: set the bandwidth to the median of the pairwise distances of the pooled sample, $\sigma = \text{median}\{\|z_i - z_j\|\}$. Its logic is to place the kernel at the scale of the data. A bandwidth far below that scale sends every off-diagonal $k(z_i, z_j)$ to zero, so all points look mutually dissimilar and the discrepancy drowns in noise; a bandwidth far above it sends every $k(z_i, z_j)$ to one, so all points look identical and the discrepancy vanishes. The median sits between these dead zones, and it costs nothing to compute.

What the heuristic cannot do is notice *where* P and Q differ. It reads only the pooled inter-point distances, which reflect the overall spread of the data, not the discriminative scale of the difference between the distributions. When those two scales coincide, as they do for a plain location shift, the median heuristic is close to optimal; when they diverge, as when the difference lives in fine structure sitting on top of a broad common shape, the median heuristic picks a bandwidth blind to exactly the feature that separates the distributions, and its power collapses (Gretton et al., 2012b; Sutherland et al., 2017). The next example shows the sensitivity directly, and the same permutation test from before delivers opposite verdicts under two bandwidths on one dataset.

Example (a wider bandwidth flips the decision)

Overlapping samples $X = \{-3, -2, -1, 0, 1\} \sim P$ and $Y = \{0, 1, 2, 3, 4\} \sim Q$, a shift of 3 that overlaps on $[0, 1]$; $n = m = 5$. Gaussian kernel, exact permutation null over all $\binom{10}{5} = 252$ relabelings, level $\alpha = 0.05$. The power proxy is $t(\sigma) = \widehat{\text{MMD}}_U^2 / \text{std}_{\text{null}}$.

1. Fix the median-heuristic scale. The median of the 45 pooled pairwise distances is $\sigma = 2$.
2. Test with a narrow bandwidth. At $\sigma = 0.5$, $\widehat{\text{MMD}}_U^2 = -0.09495$ (negative, the estimate dips below zero), giving $t = -0.7565$ and $\hat{p} = 180/252 = 0.7143$: fail to reject.
3. Test with the median bandwidth. At $\sigma = 2$, $\widehat{\text{MMD}}_U^2 = 0.40469$, $t = 2.3304$, and $\hat{p} = 10/252 = 0.0397$: reject.
4. Sweep between. The exact p -value and power proxy move together as the bandwidth widens.

σ	0.5	1.0	1.5	2.0	3.0	5.0
$\widehat{\text{MMD}}_U^2$	-0.0950	0.1690	0.3351	0.4047	0.3822	0.2284
$t = \widehat{\text{MMD}}_U^2 / \text{std}$	-0.76	0.96	1.79	2.33	2.86	3.05
$\hat{p}$	0.714	0.190	0.087	0.040	0.040	0.040

Reading. The same data and the same test change verdict from retain to reject when the bandwidth widens from 0.5 to 2. The narrow kernel makes every distinct point look mutually dissimilar, collapsing the between-group signal into noise and even driving the unbiased estimate negative; the median heuristic lands squarely in the powerful regime, where the signal-to-noise proxy t has climbed above 2. Bandwidth is not cosmetic: it sets the power, and a poorly scaled kernel is blind by construction.

Verification artifact. `checks/example-ch-testing-example-30-2.json` records the example source hash and verification scope.

31.7 Learned and deep kernels

Optimizing a single bandwidth is the smallest version of the objective J ; the full version optimizes the entire kernel. Sutherland et al. (2017) make the criterion practical by giving a differentiable estimator of $J = \text{MMD}^2 / \sigma_{H_1}$, including a smooth regularized estimate of the variance σ_{H_1} , so that the bandwidth, or an anisotropic vector of bandwidths, can be tuned by gradient ascent on $\hat{J}$ over a selection split. For structured, high-dimensional data such as images, no fixed-shape kernel resolves the discriminative scale, and the remedy is to *learn the feature map itself*.

Definition (deep kernel, Liu et al., 2020)

Let ϕ_w be a neural network with parameters w , let k_a be a Gaussian kernel on its features, q a characteristic kernel on the raw inputs, and $\varepsilon \in (0, 1]$. The *deep kernel* is

$$k_\omega(x, x') = [(1 - \varepsilon) k_a(\phi_w(x), \phi_w(x')) + \varepsilon] q(x, x'),$$

with parameters $\omega = (w, a, \varepsilon)$. The factor q and the floor ε keep k_ω characteristic for any ϕ_w , so the learned representation carves the discriminative scale without sacrificing the injectivity of the embedding.

The parameters ω are trained on a split to maximize the same power proxy $\hat{J}(\omega)$, and the other split runs the permutation test with the fitted kernel, so validity is preserved exactly as in the data-splitting algorithm. The deep-kernel test of Liu et al. (2020) substantially outstrips fixed and single-bandwidth kernels on structured data, precisely because ϕ_w can place the kernel's sensitivity where P and Q actually diverge. A complementary route keeps the kernel fixed but replaces the full quadratic statistic by the witness evaluated at a few optimized

locations: the analytic mean-embedding and smooth-characteristic-function tests (Chwialkowski et al., 2015) and the interpretable features of Jitkrittum et al. (2016) select test locations to maximize power, yielding tests that run in linear time and, as a bonus, report the concrete points in input space where the distributions differ most.

31.8 Linear-time versus quadratic-time estimators

The U-statistic sums over all $O(n^2)$ pairs, and each of the B permutations repeats that cost, so the quadratic test scales poorly. When data are abundant but the budget is fixed, it is better to spend it on more data through a cheaper estimator. The *linear-time* MMD pairs the samples and averages the core over consecutive, non-overlapping pairs,

$$\widehat{\text{MMD}}_\ell^2 = \frac{2}{n} \sum_{i=1}^{n/2} h(z_{2i-1}, z_{2i}), \quad h(z, z') = k(x, x') + k(y, y') - k(x, y') - k(x', y),$$

a running average of $n/2$ independent terms that costs $O(n)$ time and $O(1)$ memory (Gretton et al., 2009; Gretton et al., 2012). Averaging independent terms has a decisive side effect: by the ordinary central limit theorem, $\widehat{\text{MMD}}_\ell^2$ is asymptotically *normal* under both hypotheses, including the null, where its mean is zero. There is no degenerate chi-square mixture and no permutation needed; the threshold is a normal quantile with a variance estimated online. The cost is statistical: discarding most pairs inflates the variance, so at a fixed n the linear-time test has lower power. For a fixed compute budget, however, it can process orders of magnitude more data and win, and block or incomplete U-statistics interpolate between the two extremes. The optimal kernel for the linear-time test, maximizing $\widehat{\text{MMD}}_\ell^2 / \hat{\sigma}_\ell$, can be chosen in closed form (Gretton et al., 2012b).

Estimator	Cost	Null law	Threshold	Power at fixed n
Quadratic U-statistic	$O(n^2)$	chi-square mixture	permutation / bootstrap	high
Linear-time $\widehat{\text{MMD}}_\ell^2$	$O(n)$	normal	normal quantile	lower, but scales

31.9 Aggregating over kernels: MMDAgg

Data splitting solves the double-use problem but wastes data, and a single selected bandwidth is fragile: guess the discriminative scale wrong and the test is weak. A cleaner idea is to test with a whole *collection* of bandwidths at once and aggregate the results, using all the data for both selection and testing while still controlling the level exactly. This is the MMD aggregated test, MMDAgg (Schrab et al., 2021).

Fix a collection of kernels $k_1, \dots, k_M$, for instance Gaussians on a geometric grid of bandwidths spanning the plausible scales, with nonnegative weights $w_1, \dots, w_M$ summing to one. Each kernel yields its own permutation test; the danger in simply reporting the smallest p -value is that taking a minimum over M tests inflates the Type-I error. MMDAgg corrects for this by calibrating the *aggregated* statistic with a single joint permutation scheme: the same relabelings drive all M statistics simultaneously, and the combined critical values are chosen so that the probability of any weighted component exceeding its threshold under H_0 is at most α .

Algorithm (MMDAgg, aggregated two-sample test)

Input samples from P and Q ; kernels $k_1, \dots, k_M$ with weights $w_{1:M}$; level α ; permutations B .

Output a level- α decision adaptive to the unknown discriminative scale.

1. For every kernel k_m , compute the observed statistic $T_0^{(m)}$ and, using one shared set of B relabelings, the permuted statistics $T_\pi^{(m)}$.

2. Find the smallest correction $u \in (0, \alpha)$ such that the aggregated test, which rejects when $T_0^{(m)}$ exceeds the $(1 - w_m u)$ -quantile of $\{T_\pi^{(m)}\}$ for some m , has joint permutation-level at most α .
3. Reject H_0 if any weighted component exceeds its corrected quantile at that u .

Because the correction is calibrated on the joint permutation distribution, the level is controlled exactly at α with no data splitting, and the test is adaptive: it is nearly as powerful as the best single kernel in the collection, and Schrab et al. (2021) prove it is minimax optimal over Sobolev balls up to an iterated-logarithm factor, meaning it attains the best achievable power rate uniformly over a whole scale of smoothness classes without knowing which one holds. Aggregation thus buys robustness to the very scale-guessing that defeats the median heuristic, at the modest cost of running M statistics through one shared permutation loop.

31.10 Independence and interaction testing

The same apparatus tests dependence rather than difference. As developed in Chapter 29, Mean Embeddings, MMD, and Characteristic Kernels, the Hilbert-Schmidt Independence Criterion is the squared MMD between the joint distribution P_{XY} and the product of marginals $P_X \otimes P_Y$, computed with a product kernel; it is zero exactly when X and Y are independent, for characteristic k and ℓ (Gretton et al., 2005). Turning it into a test needs a statistic and a null. The empirical criterion is $\widehat{\text{HSIC}} = \frac{1}{n^2} \text{Tr}(KHLH)$ with K, L the two kernel matrices and $H = I - \frac{1}{n} \mathbf{1}\mathbf{1}^\top$ the centering matrix, and its null distribution under independence is again an infinite chi-square mixture, now in the eigenvalues of the two centered kernels (Gretton et al., 2008). Calibration mirrors the two-sample case with one change: to simulate independence we break the pairing, permuting the y indices against fixed x indices, since under H_0 the pairing carries no information. A moment-matched Gamma approximation to the mixture gives a fast alternative when many independence tests are run (Gretton et al., 2008).

Independence between pairs is not the end of the story, because *pairwise* independence does not imply *joint* independence. Three variables can be independent in every pair yet dependent as a triple, a genuinely three-way interaction that no two-variable test can see. The kernel remedy embeds the *Lancaster interaction measure*, the signed combination

$$\Delta_L P_{XYZ} = P_{XYZ} - P_{XY}P_Z - P_{YZ}P_X - P_{XZ}P_Y + 2P_XP_YP_Z,$$

which vanishes precisely when the irreducible three-way dependence is absent, and tests whether its embedding is zero (Sejdinovic, Gretton, and Bergsma, 2013). This isolates the interaction that a factorized model would miss, and it complements the test of *total* independence $P_{XYZ} = P_XP_YP_Z$. Conditional independence, the workhorse of causal discovery, admits a related kernel treatment through conditional embeddings (Fukumizu et al., 2008), pursued in Chapter 33, Conditional Mean Embeddings and Kernel Bayes' Rule and Chapter 35, Causal Inference with Kernels.

31.11 Relative tests and goodness-of-fit

Two variations answer questions the plain two-sample test cannot. The first is model comparison: given a reference sample from P and two candidate models Q_1, Q_2 , we rarely want to know whether a model is exactly right, which it never is, but whether one model is significantly *closer* to the data than the other. The *relative similarity test* frames this as $H_0 : \text{MMD}(P, Q_1) \geq \text{MMD}(P, Q_2)$ against the alternative that Q_1 is nearer, using the joint asymptotic normality of the two dependent MMD estimates to get a threshold for their difference (Bounliphone et al., 2016). It is the natural tool for ranking generative models by fidelity.

The second variation removes the need to sample the alternative at all. All the tests above compare two samples, but goodness-of-fit asks whether one sample came from a *known* model Q , often specified only up to a normalizing constant, as an unnormalized density or an energy-based model. Sampling such a Q to run a two-sample test can be as hard as the original inference. The kernel Stein discrepancy sidesteps sampling entirely: it replaces the

mean-embedding difference by a Stein operator applied to the kernel, an object that depends on Q only through its score $\nabla \log q$, which is free of the normalizer. The resulting statistic is a U-statistic in a modified kernel with the same permutation-free and spectral machinery developed here, and it yields a goodness-of-fit test requiring only samples from the data and the score of the model (Liu et al., 2016; Chwialkowski et al., 2016). We develop that construction, and its connection to Stein’s method, in Chapter 34, Kernel Stein Discrepancy and Stein Methods. For discrepancies built on transport rather than embeddings, and their own tests, see Chapter 31, Optimal Transport and Kernels.

31.12 Block estimators and streaming tests

The quadratic-time U-statistic uses every cross-pair and is statistically efficient, but it requires quadratic kernel work. The linear estimator pairs observations and averages independent four-point contrasts. Between them lies a useful continuum: split the samples into blocks of size b , compute a quadratic MMD estimate within each block, and average across blocks. Memory is $O(b^2)$, work is $O(nb)$, and the number of approximately independent block summaries is n/b . Small blocks favor throughput and a simple Gaussian calibration; large blocks recover more of the full statistic’s power but make the null degeneracy visible again.

Streaming changes the null from a one-time decision into a monitoring problem. Repeatedly applying a fixed-level test inflates the probability of ever raising a false alarm. A valid monitor therefore needs an anytime-valid construction, an alpha-spending schedule, or a predeclared finite horizon. Window overlap also creates dependence. The window length, reference-update policy, alarm threshold, and reset rule are part of the statistical procedure and must be versioned with the kernel.

Algorithm (choose an MMD estimator under a compute budget)

1. State the smallest scientifically relevant discrepancy and the available memory and latency.
2. Use pilot data to estimate the variance of several block sizes without touching the final test split.
3. Allocate equal computation to each candidate and estimate power by simulation under declared alternatives.
4. Freeze the kernel, block size, calibration method, and random seed policy.
5. Run the final test once and report the effect estimate and uncertainty, not only the rejection bit.

31.13 Dependent observations and the wild bootstrap

Ordinary permutation is exact because labels are exchangeable under the null. Time series, spatial fields, clustered samples, and repeated measurements are not exchangeable observation by observation. Permuting individual time points destroys autocorrelation and generally gives the wrong null distribution. A block permutation preserves local dependence only approximately and requires a block-length choice. A wild bootstrap instead multiplies centered kernel contributions by dependent random weights designed to mimic the covariance of the process; kernel tests for random processes use this construction (Chwialkowski and Gretton 2015).

The assumption ledger must name the dependence regime. Mixing coefficients, stationarity, moment conditions, and bootstrap-weight bandwidth replace the iid assumption. A procedure proved for a stationary mixing sequence does not automatically cover a trending or seasonally changing process. For spatial data, permutation units should follow the sampling design, such as sites or clusters, rather than convenient rows in a table.

Proposition (what makes a permutation p-value exact)

Let a finite transformation group act on the pooled data. If the joint distribution under H_0 is invariant under that group and ties are handled by randomized or conservative ranking, the permutation p-value is super-uniform under H_0 . Arbitrary row permutations are justified only when row exchangeability supplies that invariance.

Assumptions. The stated group invariance holds exactly under the null; the statistic is recomputed under every sampled transformation; Monte Carlo permutations use a valid finite-sample correction. **Proof status.** Standard

randomization-test result; the chapter records the operative condition but does not reproduce the group-orbit proof.

31.14 Robustness, multiple testing, and selective kernel choice

MMD is an average embedding discrepancy, so a few high-leverage observations can dominate an unbounded kernel or a poorly scaled input. Robust alternatives include bounded kernels, robust feature scaling learned on training data, median-of-means aggregation across blocks, and explicit contamination models. Robustification changes both the estimand and the null calibration; clipping a statistic after seeing its value is not a valid robust test.

Testing many endpoints, layers, subgroups, times, or kernels creates a family of hypotheses. Bonferroni and Holm control family-wise error; false-discovery-rate procedures target a different error criterion. Dependence among kernel statistics matters to sharper procedures. MMDagg solves one specific multiplicity problem by calibrating an aggregate over a prespecified kernel family (Schrab and Gretton 2021); it does not license an unrestricted search over preprocessing pipelines or neural feature maps.

Data splitting is the clean default for learned kernels: use a training split to select features and bandwidths, then compute and calibrate the frozen statistic on an untouched test split. Reusing the same observations can be valid only with a theorem that accounts for selection. The report should name which choices were prespecified, which were learned, and which data informed each choice.

31.15 Conditional and localized two-sample questions

The ordinary two-sample null $P_X = Q_X$ can reject merely because two populations have different covariate mixes. A conditional question asks whether $P_{Y|X} = Q_{Y|X}$, or whether a deployment residual has the same conditional law after accounting for context. This is substantially harder because conditional distributions must be compared without pretending that continuous covariates form exact strata.

Practical routes include residualization with cross-fitting, conditional mean embeddings, and local kernel weighting. Each route needs overlap: if one population never visits a region of X , no test can distinguish conditional change there from missing support. Cross-fitting prevents the same residual noise from being used both to train a nuisance model and to certify its adequacy. A useful output is a localized witness function showing where the discrepancy lies, accompanied by uncertainty that accounts for the localization search.

31.16 Summary

A kernel two-sample test is the MMD plus a decision rule with guarantees. The unbiased U-statistic estimates MMD^2 exactly, as a second-order U-statistic whose first-order projection is the witness function; that projection vanishes under the null, which is why the null law degenerates into an infinite weighted mixture of chi-squares while the alternative is asymptotically normal. Since the mixture weights are unknown, the threshold is set by permutation, exploiting the exchangeability of the pooled sample to give an exact, assumption-free, level- α test. Power, defined as the probability of rejecting under the alternative, is governed asymptotically by the signal-to-noise ratio $\text{MMD}^2/\sigma_{H_1}$, which is the objective for choosing a kernel; the median heuristic is a cheap default that matches the data scale but is blind to the discriminative scale and can be badly suboptimal, as a bandwidth sweep that flips the decision on fixed data shows. The modern answers maximize power by tuning or learning the kernel with data splitting for validity, by learning deep feature maps, by aggregating over many bandwidths without splitting through MMDagg, and by trading the quadratic estimator for a linear-time normal-limit statistic when data are cheap and compute is dear. The identical machinery, with the pairing permuted, tests independence through HSIC and its three-variable interaction extensions, and, with the Stein operator in place of the embedding,

tests goodness-of-fit against an unnormalized model. A unified account of embeddings, discrepancies, and their tests is the survey of Muandet et al. (2017).

31.17 Common mistakes and practical implications

For **Kernel Hypothesis Testing**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

31.18 Summary and further reading

This chapter established explain the central definitions and claims in Kernel Hypothesis Testing; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Gretton and Smola 2006), (Gretton and Smola 2012), (Serfling 1980).

31.19 Exercises

1. warm-up State precisely the difference between a Type-I and a Type-II error for the two-sample test, and explain why the permutation calibration controls the Type-I error but says nothing directly about the Type-II error. Then explain in one sentence why a test that always retains H_0 has level 0 yet is useless, connecting this to the definition of power.
2. warm-up The biased V-statistic $\widehat{\text{MMD}}_V^2 = \|\hat{\mu}_P - \hat{\mu}_Q\|_{\mathcal{H}}^2$ is always nonnegative, while the unbiased U-statistic can be negative. Explain both facts from their definitions: why removing the diagonal terms can drive the estimate below zero, and why this is acceptable, indeed informative, for a test statistic. What does a negative $\widehat{\text{MMD}}_U^2$ tell you about the observed labeling relative to random relabelings?
3. computation Reproduce the first worked example by hand for the reduced samples $X = \{0, 1\}$ and $Y = \{4, 5\}$ with the Gaussian kernel of fixed bandwidth $\sigma = 1$ (so $2\sigma^2 = 2$). Write the four kernel values $k(x_i, y_j)$ as exponentials, compute the unbiased $\widehat{\text{MMD}}_U^2$ (each intra pair contributes its single off-diagonal term), and then enumerate the $\binom{4}{2} = 6$ relabelings to obtain the exact permutation p -value. State the decision at $\alpha = 0.05$ and comment on why such a small sample cannot reject at that level. Hint

With one point pair per group, $\widehat{\text{MMD}}_U^2 = k(x_1, x_2) + k(y_1, y_2) - k(x_1, y_1) - k(x_1, y_2) - k(x_2, y_1) - k(x_2, y_2)$ divided appropriately; here $k(x_1, x_2) = k(y_1, y_2) = e^{-1/2}$. Of the six relabelings the observed split and its mirror are the most extreme, so the smallest attainable p -value is $2/6 \approx 0.33$, above 0.05.

4. proof Prove the unbiasedness of the U-statistic, $\mathbb{E}[\widehat{\text{MMD}}_U^2] = \text{MMD}^2$, by expanding $\mathbb{E}[h(z_1, z_2)]$ for the core $h(z, z') = k(x, x') + k(y, y') - k(x, y') - k(x', y)$, being explicit about where the independence of z_1 and z_2 is used. Then show, by contrast, that the V-statistic has a positive bias of order $1/n$, by identifying the extra diagonal contribution it includes. Hint

For unbiasedness, each of the four expectations factorizes because the two paired draws are independent; there is no diagonal because $i \neq j$. For the bias, the V-statistic adds the n diagonal terms $k(x_i, x_i)$, each

contributing $\mathbb{E}[k(X, X)]$ rather than $\mathbb{E}_{P \otimes P}[k(X, X')]$, and the $1/n^2$ weighting of n such terms is the $O(1/n)$ inflation.

5. proof Show that under $H_0 : P = Q$ the first-order projection of the core vanishes, $h_1(z) = \mathbb{E}_{z'}[h(z, z')] - \text{MMD}^2 = 0$ for all z , and explain why this *degeneracy* is exactly what forces the null limit to be second order, hence a chi-square mixture rather than a Gaussian. Contrast with the alternative, where $h_1 \neq 0$ yields asymptotic normality. Hint

Compute $\mathbb{E}_{z'}[h(z, z')] = g(x) - g(y)$ with $g = \mu_P - \mu_Q$; under H_0 the witness g is identically zero. A U-statistic with zero first-order projection has its leading fluctuation from the second-order term, which is a quadratic form in Gaussian coordinates, and a quadratic form in Gaussians is a weighted sum of squared Gaussians.

6. computation Using the asymptotic power expression $\Pi \approx \Phi(\sqrt{n} \text{MMD}^2 / \sigma_{H_1})$, suppose a kernel gives $\text{MMD}^2 / \sigma_{H_1} = 0.2$. How large must n be for the power to reach 0.8, given $\Phi^{-1}(0.8) \approx 0.84$? Now suppose a better kernel doubles the ratio to 0.4; by what factor does the required n shrink? Explain why this quadratic dependence makes kernel choice worth optimizing. Hint

Set $\sqrt{n} \cdot 0.2 = 0.84$, so $\sqrt{n} = 4.2$ and $n \approx 18$; doubling the ratio halves $\sqrt{n}$, so n shrinks by a factor of 4. Power depends on n only through $\sqrt{n}$ times the ratio, so the sample size needed scales as the inverse square of the signal-to-noise ratio.

7. proof Explain why selecting the bandwidth to maximize the estimated signal-to-noise ratio $\hat{J}$ on the same sample used to compute the test statistic can inflate the Type-I error above α , and prove that data splitting restores exact level. Be precise about which independence is needed for the permutation null on the test half to remain valid. Hint

A bandwidth tuned to the observed sample can fit the finite-sample noise, so even under $P = Q$ the selected kernel produces an atypically large statistic, breaking exchangeability of the observed value with the permuted ones. If selection uses only the selection half, the chosen kernel is a fixed function independent of the test half, so conditional on that kernel the test half is still exchangeable under H_0 and the permutation p -value keeps its exact level.

8. challenge The median heuristic can fail when the discriminative scale differs from the data scale. Construct a one-dimensional thought experiment: let P and Q share the same two widely separated coarse clusters but differ only in the fine spread *within* each cluster. Argue that the median pairwise distance is dominated by the across-cluster gap, so the median-heuristic bandwidth is far too large to resolve the intra-cluster difference, and hence the test has low power. Explain how (a) a learned or deep kernel and (b) MMDAgg each recover power, and what each pays for it. Hint

With most pairwise distances set by the coarse gap, the median bandwidth saturates the kernel within each cluster, so $k \approx 1$ there and the fine difference is invisible. A deep or learned kernel tunes its features to the intra-cluster scale by maximizing $\hat{J}$, paying with a held-out selection split; MMDAgg includes a small bandwidth in its collection and calibrates jointly, paying with M statistics through one permutation loop but no data splitting and near-optimal adaptive power.

32 Optimal Transport and Kernels

The previous chapter on mean embeddings turned every distribution into a single RKHS vector and compared distributions by the distance between those vectors, the maximum mean discrepancy, an integral probability metric over the unit ball of the RKHS. That construction is blind to the geometry of the sample space in a precise way: it compares P and Q only through a fixed feature map, and the ground distance between points enters only as far as the kernel chooses to let it. This chapter develops the other great way to compare distributions, optimal transport, which measures the least work needed to reshape one pile of mass into the other and so is built directly on the geometry of the ground space. We meet the Kantorovich formulation and its dual, find that the 1-Wasserstein distance is again an integral probability metric but now over the 1-Lipschitz functions rather than an RKHS ball, and confront the price of that geometric fidelity: the empirical Wasserstein distance converges at the dimension-cursed rate $n^{-1/d}$, whereas the MMD converges at $n^{-1/2}$ in every dimension. Entropic regularization and the Sinkhorn algorithm make transport computable, and Sinkhorn divergences debias it into a one-parameter family that interpolates between optimal transport at one end and a kernel MMD at the other, closing the circle between the two halves of this book.

32.1 Two ways to measure the distance between distributions

Recall the integral probability metric of the previous chapter. Given a class $\mathcal{F}$ of test functions, the distance between P and Q is the largest gap any allowed function can open between their means,

$$D_{\mathcal{F}}(P, Q) = \sup_{f \in \mathcal{F}} \left(\mathbb{E}_{X \sim P}[f(X)] - \mathbb{E}_{Y \sim Q}[f(Y)] \right).$$

The MMD is this quantity for $\mathcal{F}$ the unit ball of an RKHS, and its power came from a fixed, rich feature map: two distributions are far apart when their kernel-smoothed densities sit in different places. But notice what the MMD never asks. It never asks how far mass would have to travel to turn P into Q . With a localized kernel such as a narrow Gaussian, two point masses δ_a and δ_b give $\text{MMD}^2 = 2(1 - K(a, b))$, which saturates at 2 once a and b are more than a few bandwidths apart and then stops growing, no matter how much further apart we push them. The discrepancy has gone deaf to distance.

Optimal transport is the answer to a different question. Picture P as a distribution of sand and Q as a set of holes to be filled. Moving a grain from x to y costs $c(x, y)$, typically a power of the distance. The transport distance is the cost of the cheapest plan that carries all of P onto Q . Now the ground geometry is the whole story: distant masses cost more to reconcile, and the distance grows without bound as the piles separate. Where the MMD reads off a fixed feature map, transport reads off the metric of the space itself. The rest of the chapter makes this precise, then measures what the geometric fidelity costs us in statistics and computation, and finally shows that the two viewpoints are the two ends of a single tunable object.

32.2 The Monge and Kantorovich problems

32.2.1 Monge's map

The original 1781 formulation of Monge asks for a transport *map*: a function $T : \mathcal{X} \rightarrow \mathcal{X}$ that sends each grain at x to a single destination $T(x)$, pushing P onto Q (written $T_{\#}P = Q$, meaning $Q(A) = P(T^{-1}(A))$ for every measurable A) at least total cost,

$$\inf_{T: T_{\#}P=Q} \int_{\mathcal{X}} c(x, T(x)) dP(x).$$

This is intuitive but brittle. If $P = \delta_0$ is a single grain and $Q = \frac{1}{2}\delta_{-1} + \frac{1}{2}\delta_1$ two half-holes, no function T can split the grain in two, and the feasible set is empty. A map cannot tear mass apart, so the Monge problem may have no solution at all.

32.2.2 Kantorovich's relaxation: transport plans

Kantorovich (1942) fixed this by allowing mass to split. Instead of a map he optimizes over a *coupling*, a joint distribution π on $\mathcal{X} \times \mathcal{X}$ whose marginals are P and Q ; the value $\pi(x, y)$ is the amount of mass shipped from x to y , and the marginal constraints say that everything leaving x sums to P and everything arriving at y sums to Q .

Definition (transport plan and optimal transport cost)

Let $\Pi(P, Q)$ be the set of couplings of P and Q , the Borel probability measures π on $\mathcal{X} \times \mathcal{X}$ with marginals P and Q :

$$\Pi(P, Q) = \left\{ \pi \geq 0 : \int_{\mathcal{X}} d\pi(\cdot, y) = P, \int_{\mathcal{X}} d\pi(x, \cdot) = Q \right\}.$$

For a cost $c : \mathcal{X} \times \mathcal{X} \rightarrow [0, \infty)$, the optimal transport cost is

$$\text{OT}_c(P, Q) = \min_{\pi \in \Pi(P, Q)} \int_{\mathcal{X} \times \mathcal{X}} c(x, y) d\pi(x, y).$$

When $c(x, y) = \|x - y\|^p$ for $p \geq 1$, the p -Wasserstein distance is $W_p(P, Q) = \text{OT}_c(P, Q)^{1/p}$.

Two features make this the right relaxation. First, the feasible set is never empty: the product coupling $\pi = P \otimes Q$, which ships independently of the source, always has the correct marginals, so $\Pi(P, Q)$ always contains at least one plan. Second, the objective is *linear* in π and the constraints are linear equalities, so this is a linear program (an infinite-dimensional one in general, a finite one for discrete measures). The minimum is attained because $\Pi(P, Q)$ is convex and weakly compact and the cost is lower semicontinuous (Villani 2009). The product coupling is almost never optimal, but its existence is what rescues the problem Monge left ill-posed. Where Monge sought a single-valued map, Kantorovich allows a grain at x to be dispersed across many destinations, and it is a theorem, not an assumption, that under mild conditions the optimum concentrates back onto a map anyway.

32.3 Kantorovich duality and the IPM connection

Every linear program has a dual, and the dual of optimal transport is what connects it to the integral probability metrics of the previous chapter. The primal ships mass; the dual sets prices.

Theorem (Kantorovich duality)

For lower semicontinuous cost $c \geq 0$,

$$\text{OT}_c(P, Q) = \sup_{(f, g) \in \Phi_c} \left(\mathbb{E}_{X \sim P}[f(X)] + \mathbb{E}_{Y \sim Q}[g(Y)] \right),$$

where the supremum is over the potentials

$$\Phi_c = \{(f, g) : f(x) + g(y) \leq c(x, y) \text{ for all } x, y\}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The story behind the dual is a shipping company. You own the sand and the holes; a contractor offers to run the whole move for you, charging $f(x)$ to load a unit at x and $g(y)$ to unload it at y . To keep your business the contractor's prices must never exceed the cost of doing it yourself, $f(x) + g(y) \leq c(x, y)$; subject to that, the contractor maximizes revenue $\mathbb{E}_P[f] + \mathbb{E}_Q[g]$. Duality says the best honest price the contractor can charge equals the least cost you could achieve on your own. The easy half of the equality, weak duality, is a one-line calculation that also fixes the direction of the inequality.

Proof (weak duality, $\sup \leq \min$)

Take any coupling $\pi \in \Pi(P, Q)$ and any feasible pair $(f, g) \in \Phi_c$. Because π has marginals P and Q , the potentials integrate against π as against their own marginals,

$$\mathbb{E}_P[f] + \mathbb{E}_Q[g] = \int (f(x) + g(y)) d\pi(x, y) \leq \int c(x, y) d\pi(x, y),$$

the inequality being the pointwise constraint $f(x) + g(y) \leq c(x, y)$ integrated against the nonnegative π . The left side does not depend on π and the right side does not depend on (f, g) , so taking the supremum over (f, g) and the infimum over π keeps the inequality: $\sup_{\Phi_c} (\mathbb{E}_P[f] + \mathbb{E}_Q[g]) \leq \min_{\Pi(P, Q)} \int c d\pi$. That the two are in fact equal, strong duality, holds for lower semicontinuous c and is proved by a Hahn-Banach or minimax argument (Villani 2009). $\square$

32.3.1 Kantorovich-Rubinstein: W_1 as an IPM over 1-Lipschitz functions

When the cost is a metric, $c(x, y) = \|x - y\|$, the dual collapses onto a single function and reveals W_1 as an integral probability metric. The reduction rests on the notion of the c -transform: given f , the best partner is $g(y) = \inf_x (c(x, y) - f(x))$, the largest unloading price still compatible with the loading prices. For a metric cost one shows the optimal pair satisfies $g = -f$ with f constrained to be 1-Lipschitz.

Theorem (Kantorovich-Rubinstein)

For the cost $c(x, y) = \|x - y\|$,

$$W_1(P, Q) = \sup_{\text{Lip}(f) \leq 1} (\mathbb{E}_{X \sim P}[f(X)] - \mathbb{E}_{Y \sim Q}[f(Y)]),$$

the supremum running over all 1-Lipschitz functions, $|f(x) - f(y)| \leq \|x - y\|$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The key step is that restricting the dual to the diagonal $g = -f$ loses nothing and turns the joint constraint into a Lipschitz condition. With $g = -f$, feasibility $f(x) + g(y) \leq c(x, y)$ reads $f(x) - f(y) \leq \|x - y\|$ for all x, y , which said in both directions is exactly $\text{Lip}(f) \leq 1$. Conversely a 1-Lipschitz f is its own optimal partner, $f^c = -f$, so no generality is lost (Villani 2009). The dual objective becomes $\mathbb{E}_P[f] - \mathbb{E}_Q[f]$, and we have landed precisely on the integral probability metric of the last chapter, now with a different courtroom of witnesses.

This is the clean contrast the chapter is built around. Both the MMD and W_1 are integral probability metrics; they differ only in the class $\mathcal{F}$ of test functions they optimize over.

MMD	1-Wasserstein W_1	
Function class $\mathcal{F}$	RKHS ball $\{\ f\ _{\mathcal{H}} \leq 1\}$	1-Lipschitz $\{\text{Lip}(f) \leq 1\}$
Optimal witness	$(\mu_P - \mu_Q)/\ \mu_P - \mu_Q\ _{\mathcal{H}}$, closed form	Kantorovich potential, a linear program
Ground geometry	only through the kernel	fully, through the cost
Estimation rate	$n^{-1/2}$, dimension-free	$n^{-1/d}$ for $d \geq 3$

The 1-Lipschitz class is enormous, far larger than an RKHS ball, and that difference in size is not cosmetic. It is exactly what makes W_1 geometry-aware, and, as the sample-complexity section will show, exactly what makes it expensive to estimate.

32.4 The one-dimensional case: transport by sorting

On the line, optimal transport has a closed form and needs no linear program. The reason is monotonicity: mass should never cross over itself. If a plan sent some mass from x_1 to y_2 and other mass from x_2 to y_1 with $x_1 < x_2$ but $y_1 < y_2$, uncrossing the two shipments to $x_1 \rightarrow y_1, x_2 \rightarrow y_2$ cannot increase a cost $|x - y|^p$, by a two-line rearrangement inequality. So the optimal coupling matches the quantiles of P and Q in order. In terms of the cumulative distribution functions F_P, F_Q and their quantile inverses,

$$W_p(P, Q)^p = \int_0^1 |F_P^{-1}(u) - F_Q^{-1}(u)|^p du, \quad W_1(P, Q) = \int_{\mathbb{R}} |F_P(t) - F_Q(t)| dt.$$

For two empirical measures with the same number n of equally weighted atoms, the quantile coupling is simply the sorted pairing: sort both samples and match them position by position, giving $W_p^P = \frac{1}{n} \sum_i |x_{(i)} - y_{(i)}|^p$.

Example (exact W_1 between two 1-D empirical measures)

Two three-point measures on the line, uniform weights 1/3:

$$P = \frac{1}{3}(\delta_0 + \delta_2 + \delta_5), \quad Q = \frac{1}{3}(\delta_1 + \delta_3 + \delta_4),$$

ground cost $c(x, y) = |x - y|$ for W_1 and $|x - y|^2$ for W_2 .

1. Sort both samples. Already sorted: $x_{(1,2,3)} = (0, 2, 5)$ and $y_{(1,2,3)} = (1, 3, 4)$.
2. Match in order and sum. The monotone coupling pairs $0 \rightarrow 1, 2 \rightarrow 3, 5 \rightarrow 4$, with distances $(1, 1, 1)$, so $W_1 = \frac{1}{3}(1 + 1 + 1) = 1.0$.
3. Check that crossing costs more. The reversed pairing $0 \rightarrow 4, 2 \rightarrow 3, 5 \rightarrow 1$ has distances $(4, 1, 4)$, giving $\frac{1}{3}(4 + 1 + 4) = 3.0$, three times the monotone cost.
4. Confirm through the CDF integral. Integrating the gap between the two step CDFs gives $\int_{\mathbb{R}} |F_P(t) - F_Q(t)| dt = 1.0$, matching step 2, and the squared cost gives $W_2 = 1.0$.
5. Contrast with a kernel discrepancy. The energy distance $\mathcal{E}(P, Q) = 2 \mathbb{E}|X - Y| - \mathbb{E}|X - X'| - \mathbb{E}|Y - Y'|$, which is an MMD with kernel $-|x - y|$ (Sejdinovic, Sriperumbudur, Gretton, and Fukumizu 2013), evaluates to $2(2.1111) - 2.2222 - 1.3333 = 0.6667$ on the same pair.

Reading. Sorting solves one-dimensional transport exactly, and the monotone pairing beats the crossed one by a factor of three here. The Wasserstein distance 1.0 and the energy-distance MMD 0.6667 are different numbers reading different things off the same two samples: one the least work to move the mass, the other a fixed-feature discrepancy. We will see them reappear as the two limits of a single object.

Verification artifact. checks/example-ch-ot-example-31-1.json records the example source hash and verification scope.

32.5 Sample complexity: the price of geometry

Geometry-awareness is not free. The cleanest way to see the cost is to ask how fast each distance, estimated from n samples, converges to its population value, and the answer separates the two IPMs sharply. The MMD is estimated by plugging empirical measures into its closed form; because the estimator is essentially an average of bounded kernel evaluations, its error concentrates at the parametric rate

$$|\widehat{\text{MMD}} - \text{MMD}| = O_P(n^{-1/2}),$$

with constants that do not grow with the dimension d (Gretton, Borgwardt, Rasch, Schölkopf, and Smola 2012; Sriperumbudur, Fukumizu, Gretton, Schölkopf, and Lanckriet 2012). The mean embedding lives in a fixed Hilbert space whose ball is small, so empirical fluctuations of the sup over that ball are dimension-free.

The empirical Wasserstein distance behaves very differently. For P with enough moments on $\mathbb{R}^d$,

$$\mathbb{E} W_1(\hat{P}_n, P) \asymp \begin{cases} n^{-1/2}, & d = 1, \\ n^{-1/2} \log n, & d = 2, \\ n^{-1/d}, & d \geq 3, \end{cases}$$

the sharp rates of Fournier and Guillin (2015), foreshadowed by Dudley (1969). In high dimension $n^{-1/d}$ is punishing: to halve the error in $d = 10$ one needs roughly 2^{10} times as many samples. The mechanism is exactly the IPM contrast of the previous section. The 1-Lipschitz class is vast, with metric entropy growing like ε^{-d} , so the supremum over it picks up empirical noise that the small RKHS ball never sees. What buys geometric fidelity, a rich enough class of witnesses to feel every direction of the ground space, is precisely what makes the empirical estimate noisy in high dimension.

This is the honest tension of the chapter. Transport respects the geometry of the data and produces meaningful comparisons and gradients even between distributions with disjoint supports, where a localized-kernel MMD has already saturated; but its sample complexity degrades exponentially in the dimension, while the MMD's does not. Neither distance dominates. The entropic regularization we turn to next was introduced for speed, but it also softens this statistical curse, and its debiased form will let us slide continuously between the two regimes.

32.6 Entropic regularization and the Sinkhorn algorithm

Solving the transport linear program exactly costs $O(n^3 \log n)$ for n atoms, too slow for the sample sizes of machine learning. Cuturi (2013) observed that adding a small entropy penalty transforms the problem into one solved by a handful of matrix-vector products; the monograph of Peyré and Cuturi (2019) is the standard reference for the computational theory. We work now with discrete measures: weight vectors $a \in \Delta_p$ and $b \in \Delta_m$ on the two supports, a cost matrix $C \in \mathbb{R}^{n \times m}$, and couplings $U(a, b) = \{\pi \in \mathbb{R}_+^{n \times m} : \pi \mathbf{1}_m = a, \pi^\top \mathbf{1}_n = b\}$.

Definition (entropic optimal transport)

For regularization strength $\varepsilon > 0$, the entropic optimal transport cost is

$$\text{OT}_\varepsilon(a, b) = \min_{\pi \in U(a, b)} \langle C, \pi \rangle + \varepsilon \text{KL}(\pi \| ab^\top), \quad \langle C, \pi \rangle = \sum_{i,j} C_{ij} \pi_{ij}.$$

Here $\text{KL}(\pi \| ab^\top) = \sum_{i,j} \pi_{ij} \log(\pi_{ij}/a_i b_j)$, with the usual zero-mass convention. This reference-measure convention makes the regularized cost nonnegative and is the convention used by the Sinkhorn divergence below.

The entropy term is strictly convex, so the minimizer is unique; as $\varepsilon \rightarrow 0$ it recovers the linear program, and as $\varepsilon \rightarrow \infty$ it pulls π toward the independent coupling $ab^\top$.

32.6.1 The scaling form of the solution

The entropy penalty does something remarkable to the shape of the solution: it forces the optimal plan to be a diagonal rescaling of a fixed matrix, and the rescaling is found by alternating normalizations. This is the Sinkhorn fixed point, and it falls straight out of the Lagrangian.

Proposition (Sinkhorn scaling form)

The solution of the entropic problem has the form $\pi^\star = \text{diag}(u) K \text{diag}(v)$ with the Gibbs kernel $K = e^{-C/\varepsilon}$ (entrywise) and positive vectors $u \in \mathbb{R}_+^n, v \in \mathbb{R}_+^m$ determined by the marginal fixed point

$$u = a \oslash (Kv), \quad v = b \oslash (K^\top u),$$

where $\oslash$ is entrywise division. Such u, v exist and are unique up to a scalar (Sinkhorn and Knopp 1967).

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Introduce multipliers $f \in \mathbb{R}^n$ and $g \in \mathbb{R}^m$ for the two marginal constraints $\pi \mathbf{1} = a$ and $\pi^\top \mathbf{1} = b$, and set the derivative of the Lagrangian in the entry π_{ij} to zero. The additive constant in the KL derivative is absorbed into the marginal multipliers, giving

$$C_{ij} + \varepsilon \log \frac{\pi_{ij}}{a_i b_j} - f_i - g_j = 0 \implies \pi_{ij} = a_i e^{f_i/\varepsilon} e^{-C_{ij}/\varepsilon} b_j e^{g_j/\varepsilon}.$$

Absorbing a_i and b_j into positive scalings u_i and v_j , and writing $K_{ij} = e^{-C_{ij}/\varepsilon}$, gives $\pi^\star = \text{diag}(u) K \text{diag}(v)$, positive entrywise. Imposing the row marginal, $\sum_j \pi_{ij}^\star = u_i \sum_j K_{ij} v_j = a_i$, is the vector equation $u \odot (Kv) = a$, that is $u = a \oslash (Kv)$; the column marginal gives $v = b \oslash (K^\top u)$ identically. Because K is strictly positive, Sinkhorn and Knopp (1967) guarantee a positive scaling pair, unique up to the exchange $(u, v) \mapsto (\lambda u, \lambda^{-1} v)$, and convergence of the alternating updates is linear in Hilbert's projective metric. $\square$

The fixed-point equations *are* the algorithm: hold v and set $u = a \oslash (Kv)$ to fix the row marginal, then hold u and set $v = b \oslash (K^\top u)$ to fix the column marginal, and repeat. Each half-step forces one marginal exactly right while nudging the other, and the two chase each other to the unique plan satisfying both.

Algorithm (Sinkhorn iterations)

Input marginals $a \in \Delta_n, b \in \Delta_m$; cost $C \in \mathbb{R}^{n \times m}$; regularization $\varepsilon > 0$; tolerance τ .

Output transport plan $\pi = \text{diag}(u) K \text{diag}(v)$ and cost $\langle C, \pi \rangle$.

1. Form the Gibbs kernel $K = e^{-C/\varepsilon}$ entrywise; initialize $v \leftarrow \mathbf{1}_m$.
2. Update the row scaling $u \leftarrow a \oslash (Kv)$.
3. Update the column scaling $v \leftarrow b \oslash (K^\top u)$.
4. Repeat steps 2 to 3 until the marginal violation $\|\pi \mathbf{1} - a\|_1 + \|\pi^\top \mathbf{1} - b\|_1$ falls below τ .

Example (Sinkhorn iterations on a tiny cost matrix)

Supports $x = (0, 1, 2)$ and $y = (0, 1, 2)$, squared-distance cost, and left- and right-heavy marginals:

$$C = \begin{pmatrix} 0 & 1 & 4 \\ 1 & 0 & 1 \\ 4 & 1 & 0 \end{pmatrix}, \quad a = (0.5, 0.2, 0.3), \quad b = (0.2, 0.3, 0.5), \quad \varepsilon = 1.$$

The Gibbs kernel is $K = e^{-C}$, with $e^{-1} = 0.3679$ and $e^{-4} = 0.0183$.



Figure 32.1: The entropic optimal-transport plan $\pi = \text{diag}(u) K \text{diag}(v)$ between a two-bump source and a wide target, converged by alternating Sinkhorn normalizations $u \leftarrow a \oslash (Kv)$, $v \leftarrow b \oslash (K^\top u)$.

1. Build the kernel. $K = \begin{pmatrix} 1 & 0.3679 & 0.0183 \\ 0.3679 & 1 & 0.3679 \\ 0.0183 & 0.3679 & 1 \end{pmatrix}$, and start from $v = \mathbf{1}$.
2. Run iteration 1. After one row and column update the plan is $\begin{pmatrix} 0.1772 & 0.1215 & 0.0124 \\ 0.0208 & 0.1055 & 0.0799 \\ 0.0019 & 0.0729 & 0.4077 \end{pmatrix}$; its column sums are exactly $b = (0.2, 0.3, 0.5)$, but its row sums $(0.3112, 0.2062, 0.4826)$ miss a . Marginal error 0.3776, transport cost $\langle C, \pi \rangle = 0.3527$.
3. Run iteration 2. Row sums move to $(0.3853, 0.2131, 0.4016)$; marginal error drops to 0.2294, cost rises to 0.4511.
4. Run iterations 3 and 4. Errors 0.1337 then 0.0751, roughly halving each pass; costs 0.5428 then 0.6081 as the plan sharpens.
5. Read off the limit. At convergence $\pi^\star = \begin{pmatrix} 0.1934 & 0.2292 & 0.0774 \\ 0.0063 & 0.0554 & 0.1383 \\ 0.0002 & 0.0154 & 0.2843 \end{pmatrix}$, with row sums exactly a , column sums exactly b , and cost 0.6997.

Reading. Each column update locks the column marginal onto b exactly while leaving the rows slightly off; the next row update repairs the rows, and the marginal error halves each pass. The converged plan carries mass rightward (the large entries sit above the diagonal), matching the left-heavy a to the right-heavy b as an entropically blurred version of the optimal map, all from repeated matrix-vector products.

Verification artifact. checks/example-ch-ot-example-31-2.json records the example source hash and verification scope.

32.7 Sinkhorn divergences: debiasing and the bridge to MMD

Entropic transport is fast, but it is subtly broken as a loss function. The entropy term rewards spreading mass, so it pays to blur the plan even when the two measures are identical, and $\text{OT}_\varepsilon(a, a) > 0$ in general. A distance to itself that is not zero is a poor training objective: minimizing $\text{OT}_\varepsilon(a, b)$ over a does not return b but a shrunk, over-smoothed version of it, the so-called entropic bias. The cure, due to Genevay, Peyré, and Cuturi (2018), is to subtract off the self-transport terms.

Definition (Sinkhorn divergence)

The Sinkhorn divergence is the debiased entropic cost

$$S_\varepsilon(a, b) = \text{OT}_\varepsilon(a, b) - \frac{1}{2} \text{OT}_\varepsilon(a, a) - \frac{1}{2} \text{OT}_\varepsilon(b, b).$$

The two correction terms exactly cancel the self-cost, so $S_\varepsilon(a, a) = 0$, and Feydy, Séjourné, Vialard, Amari, Trounev, and Peyré (2019) prove that for suitable costs S_ε is nonnegative, convex in each argument, and metrizes the convergence of measures. What makes the object central to this book is what it does at its two extremes.

Theorem (interpolation between OT and MMD, Feydy et al. 2019)

As the regularization vanishes and diverges, the Sinkhorn divergence recovers the two distances of this chapter:

$$S_\varepsilon(a, b) \xrightarrow{\varepsilon \rightarrow 0} \text{OT}_0(a, b) \quad (\text{unregularized transport}), \quad S_\varepsilon(a, b) \xrightarrow{\varepsilon \rightarrow \infty} \frac{1}{2} \|a - b\|_{-C}^2,$$

where $\|a - b\|_{-C}^2 = 2 \sum_{i,j} a_i b_j C_{ij} - \sum_{i,j} a_i a_j C_{ij} - \sum_{i,j} b_i b_j C_{ij}$ is the squared MMD with kernel $k = -C$, an energy distance when C is a distance matrix.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

So the single dial ε slides from pure optimal transport, geometry-driven and dimension-cursed, at $\varepsilon \rightarrow 0$, to a kernel MMD, dimension-free and geometry-blind, at $\varepsilon \rightarrow \infty$, with a debiased compromise for every value in between. On the pair from the worked example above the two endpoints are numbers we already have: at $\varepsilon \rightarrow 0$ the divergence tends to $W_1 = 1.0$, and at $\varepsilon \rightarrow \infty$ it tends to $\frac{1}{2}$ of the energy distance 0.6667, namely 0.3333. Computing S_ε is three Sinkhorn runs and a combination.

Algorithm (Sinkhorn divergence)

Input marginals a, b ; cost C (and the self-costs on the two supports); regularization ε ; tolerance τ .

Output Sinkhorn divergence $S_\varepsilon(a, b) \geq 0$, with $S_\varepsilon(a, a) = 0$.

1. Run Sinkhorn iterations for the cross term to get $OT_\varepsilon(a, b) = \langle C, \pi_{ab}^\star \rangle + \varepsilon KL(\pi_{ab}^\star \| ab^\top)$.
2. Run Sinkhorn for the self term $OT_\varepsilon(a, a)$ on support of a with both marginals a .
3. Run Sinkhorn for the self term $OT_\varepsilon(b, b)$ on support of b with both marginals b .
4. Return $S_\varepsilon(a, b) = OT_\varepsilon(a, b) - \frac{1}{2}OT_\varepsilon(a, a) - \frac{1}{2}OT_\varepsilon(b, b)$.

In practice the entropic costs are read from the converged dual potentials rather than by re-summing the plan, and the two self-terms use a faster symmetric fixed point; the debiasing is what makes the gradient of S_ε with respect to the sample points a sound training signal, which is why Sinkhorn divergences have become a standard loss for generative models and shape registration.

32.8 When to prefer optimal transport, when to prefer MMD

The interpolation theorem turns what could be a turf war into a design choice along one axis. The right end of the dial, the MMD, is the tool of the statistician: cheap at $O(n^2)$ in closed form, differentiable, and above all estimable at the dimension-free rate $n^{-1/2}$, which is why the MMD underwrites the two-sample and independence tests of kernel hypothesis testing and the quadrature rules of kernel quadrature and herding, where honest confidence intervals matter more than fidelity to the ground metric. The left end, optimal transport, is the tool of the geometer: it respects the metric of the sample space, yields interpretable transport plans and non-vanishing gradients even between distributions whose supports do not overlap, and so is the natural loss when the geometry of the space is the point, as in comparing shapes, color histograms, or word embeddings. Its cost is the curse of dimension in both computation and statistics.

Two rules of thumb follow. Prefer the MMD when the ambient dimension is high, samples are scarce, or a calibrated test statistic is needed, and a characteristic kernel already encodes the similarity you care about. Prefer optimal transport when the ground geometry carries the meaning, the dimension is modest or the data lie on a low-dimensional manifold, and you need a distance that keeps growing as supports separate. When unsure, the Sinkhorn divergence is the hedge: tune ε to trade geometric fidelity against statistical and computational economy, knowing the two familiar distances sit at its ends. The same geometry-versus-statistics choice recurs whenever distributions are the objects of learning, for instance in the distribution regression that takes measures as inputs.

32.9 Summary

Optimal transport measures the least work to reshape one distribution into another, and so, unlike the mean-embedding MMD, it is built on the geometry of the ground space. Kantorovich's relaxation from maps to couplings makes the problem a well-posed linear program whose dual, by Kantorovich-Rubinstein, exhibits the 1-Wasserstein distance as an integral probability metric over the 1-Lipschitz functions, the direct counterpart of the MMD's IPM over an RKHS ball. In one dimension transport reduces to sorting. The larger Lipschitz class is what makes W_1 geometry-aware and, at the same time, what curses its empirical estimate with the rate $n^{-1/d}$, against the MMD's dimension-free $n^{-1/2}$. Entropic regularization turns transport into the Sinkhorn algorithm, a matrix-scaling fixed point solved by alternating normalizations, and the debiased Sinkhorn divergence interpolates between unregularized optimal transport as $\varepsilon \rightarrow 0$ and a kernel MMD (an energy distance) as $\varepsilon \rightarrow \infty$. The choice between

transport and kernel discrepancies is thus not a dichotomy but a dial, trading geometric fidelity against statistical and computational cost, and it connects the distances built from kernels to the metric of the data itself.

32.10 Common mistakes and practical implications

For **Optimal Transport and Kernels**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

32.11 Summary and further reading

This chapter established explain the central definitions and claims in Optimal Transport and Kernels; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Kantorovich 1942), (Villani 2009), (Sejdinovic and Fukumizu 2013).

32.12 Exercises

1. warm-up Take $P = \frac{1}{3}(\delta_0 + \delta_1 + \delta_6)$ and $Q = \frac{1}{3}(\delta_2 + \delta_3 + \delta_4)$ on the line. Compute $W_1(P, Q)$ by sorting and matching the atoms in order, then verify the same value through the CDF integral $\int_{\mathbb{R}} |F_P(t) - F_Q(t)| dt$. Confirm that at least one crossed pairing costs strictly more.
2. computation With supports $\{0, 1\}$ and $\{0, 1\}$, cost $C = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, marginals $a = (0.6, 0.4)$, $b = (0.3, 0.7)$, and $\varepsilon = 1$, form $K = e^{-C}$ and run one Sinkhorn iteration from $v = \mathbf{1}$: update $u = a \oslash (Kv)$, then $v = b \oslash (K^T u)$. Write the plan $\text{diag}(u)K \text{diag}(v)$ and check that its column sums equal b exactly while its row sums do not yet equal a .

3. proof Prove weak duality for optimal transport directly: for any coupling $\pi \in \Pi(P, Q)$ and any potentials with $f(x) + g(y) \leq c(x, y)$, show $\mathbb{E}_P[f] + \mathbb{E}_Q[g] \leq \int c d\pi$, and conclude $\sup_{\Phi_c} (\mathbb{E}_P[f] + \mathbb{E}_Q[g]) \leq \text{OT}_c(P, Q)$. State exactly where the marginal property of π and the nonnegativity of π are each used. Hint

Integrate $f(x) + g(y)$ against π : the marginals turn $\int f(x) d\pi$ into $\mathbb{E}_P[f]$ and $\int g(y) d\pi$ into $\mathbb{E}_Q[g]$. Then the pointwise inequality integrated against the nonnegative measure π gives the bound.

4. proof Derive Kantorovich-Rubinstein from the general dual. Fix the cost $c(x, y) = \|x - y\|$, restrict the dual to pairs of the form $(f, -f)$, and show that feasibility $f(x) - f(y) \leq \|x - y\|$ for all x, y is equivalent to f being 1-Lipschitz. Conclude $W_1(P, Q) = \sup_{\text{Lip}(f) \leq 1} (\mathbb{E}_P[f] - \mathbb{E}_Q[f])$, assuming the standard fact that the optimum can be taken in this form. Hint

Feasibility of $(f, -f)$ reads $f(x) - f(y) \leq \|x - y\|$; applying it with x, y swapped gives $|f(x) - f(y)| \leq \|x - y\|$, which is $\text{Lip}(f) \leq 1$. Conversely a 1-Lipschitz f makes $(f, -f)$ feasible.

5. computation Show numerically that entropic transport is biased at zero. For $a = (0.5, 0.5)$ on support $\{0, 1\}$ with cost $C = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ and $\varepsilon = 1$, solve $\text{OT}_\varepsilon(a, a)$ (by symmetry the optimal plan is $\text{diag}(u)K \text{diag}(u)$ with a single scaling u) and check that the entropic cost is strictly positive, so $\text{OT}_\varepsilon(a, a) \neq 0$. Then confirm the debiasing identity $S_\varepsilon(a, a) = 0$ from the definition. Hint

With equal marginals the plan puts off-diagonal mass because K is not the identity, so $\langle C, \pi \rangle > 0$ and the entropy term is finite; the point is only that the total is not zero. The debiasing subtracts $\frac{1}{2}\text{OT}_\varepsilon(a, a) + \frac{1}{2}\text{OT}_\varepsilon(a, a) = \text{OT}_\varepsilon(a, a)$.

6. exploration On the worked-example pair $P = \frac{1}{3}(\delta_0 + \delta_2 + \delta_5)$, $Q = \frac{1}{3}(\delta_1 + \delta_3 + \delta_4)$ with cost $c(x, y) = |x - y|$, verify by hand that the energy distance $2\mathbb{E}|X - Y| - \mathbb{E}|X - X'| - \mathbb{E}|Y - Y'|$ equals 0.6667 and hence that the $\varepsilon \rightarrow \infty$ limit of the Sinkhorn divergence, $\frac{1}{2}$ of this, is 0.3333. Contrast this with the $\varepsilon \rightarrow 0$ limit $W_1 = 1.0$, and say in one sentence what each end of the dial is sensitive to that the other is not.
7. challenge Explain the sample-complexity gap through the size of the witness class. Argue informally that the empirical fluctuation of an IPM $\sup_{f \in \mathcal{F}} (\mathbb{E}_{\hat{P}_n}[f] - \mathbb{E}_P[f])$ is governed by the metric entropy of $\mathcal{F}$: the RKHS unit ball is effectively small (a Donsker class, fluctuation $n^{-1/2}$ independent of d), while the 1-Lipschitz ball on a bounded region of $\mathbb{R}^d$ has ε -covering number growing like $\exp(\varepsilon^{-d})$, producing the $n^{-1/d}$ Wasserstein rate. Conclude why entropic regularization, which effectively shrinks the class of achievable dual potentials, improves the statistics. Hint

A larger function class can align more closely with the finite-sample noise $\hat{P}_n - P$, inflating the supremum. Uniform-convergence bounds scale the fluctuation by $\sqrt{\log N(\varepsilon)/n}$ with N the covering number; a covering number $\exp(\varepsilon^{-d})$ is what turns the parametric $n^{-1/2}$ into $n^{-1/d}$. Entropic smoothing caps the Lipschitz constant of the achievable potentials, shrinking $\mathcal{F}$ toward the RKHS-like regime.

33 Kernel Quadrature and Herding

Almost every quantity we care about in probability is an integral: a mean, a variance, a marginal likelihood, a posterior prediction, an expected loss. When the integral has no closed form we fall back on a weighted sum $\sum_i w_i f(x_i)$, and the only question is where to place the nodes x_i and how to weight them. Plain Monte Carlo answers by sampling the nodes at random, and pays for that convenience with a slow $1/\sqrt{n}$ error. This chapter asks whether the RKHS geometry of the previous chapters can do better. The answer is clean and complete: the worst-case error of a quadrature rule over the RKHS unit ball is exactly the distance between the mean embedding of the target measure and the embedding of the weighted node set, a maximum mean discrepancy. Minimizing that error over the weights recovers Bayesian quadrature and its posterior variance; minimizing it greedily over the nodes is kernel herding, a way to manufacture deterministic super-samples. The same identity ties both to leverage scores, determinantal sampling, and coresets, so that numerical integration becomes one more thing the kernel does for free.

33.1 The integration problem, and why randomness is wasteful

Fix a probability measure P on a set $\mathcal{X}$ and a function $f : \mathcal{X} \rightarrow \mathbb{R}$. We want the integral

$$P[f] = \mathbb{E}_{X \sim P}[f(X)] = \int_{\mathcal{X}} f dP,$$

and we are allowed to evaluate f at finitely many nodes $x_1, \dots, x_n$. A *quadrature rule* is a choice of those nodes together with weights $w_1, \dots, w_n$, and it estimates the integral by

$$\widehat{P[f]} = \sum_{i=1}^n w_i f(x_i).$$

The classical answer is Monte Carlo: draw the nodes i.i.d. from P and weight them uniformly, $w_i = 1/n$. It is unbiased and needs nothing but a sampler, but its root-mean-square error decays like $1/\sqrt{n}$, so cutting the error in half costs four times the evaluations. That rate is the price of ignoring f : the nodes are scattered blindly, some landing almost on top of each other and wasting evaluations, others leaving whole regions unprobed. If we knew that f were smooth, we could place nodes deliberately, spread them out, and reweight them to cancel the redundancy. To make "smooth" precise and the optimization tractable, we assume f lives in the RKHS $\mathcal{H}$ of a positive definite kernel k , and we measure a rule by how badly it can do against any such f .

Definition (quadrature rule and its error)

A quadrature rule for P is a finite set of nodes $x_1, \dots, x_n \in \mathcal{X}$ with weights $w = (w_1, \dots, w_n) \in \mathbb{R}^n$. Its *error on f* is

$$\text{err}(f; w) = P[f] - \sum_{i=1}^n w_i f(x_i).$$

The weights are not required to be nonnegative, nor to sum to one.

Two remarks fix expectations. The weights are free real numbers: releasing them from the probability simplex is exactly what will let an optimal rule outperform any averaging scheme. And a single f tells us nothing, since a rule can be exact on one function by luck; the honest figure of merit is the largest error the rule can make over a whole class of integrands, which we take to be the RKHS unit ball.

33.2 Worst-case error is a maximum mean discrepancy

The right way to grade a rule is adversarial: how large can $\text{err}(f; w)$ be as f ranges over all integrands of unit RKHS norm? This worst case turns out to have a closed form, and it is precisely a distance between mean embeddings, which is why the machinery of mean embeddings and MMD transfers wholesale to integration.

Definition (worst-case error)

The *worst-case error* of the rule (x_i, w_i) over the unit ball of $\mathcal{H}$ is

$$e(w) = \sup_{\substack{f \in \mathcal{H} \\ \|f\|_{\mathcal{H}} \leq 1}} \left| P[f] - \sum_{i=1}^n w_i f(x_i) \right|.$$

Recall the two facts an RKHS gives us. Evaluation is an inner product with the canonical feature, $f(x) = \langle f, k(x, \cdot) \rangle_{\mathcal{H}}$, and integration against P is an inner product with the mean embedding $\mu_P = \mathbb{E}_{X \sim P}[k(X, \cdot)]$, the generalized kernel trick $P[f] = \langle f, \mu_P \rangle_{\mathcal{H}}$. Both the integral and the finite sum are therefore linear functionals of f , and their difference is a single inner product. That collapses the supremum by Cauchy-Schwarz.

Theorem (worst-case error is an embedding distance)

Let $\mu_P \in \mathcal{H}$ exist. For any nodes $x_1, \dots, x_n$ and weights w , the worst-case error equals the RKHS distance between the target embedding and the weighted node embedding,

$$e(w) = \left\| \mu_P - \sum_{i=1}^n w_i k(x_i, \cdot) \right\|_{\mathcal{H}} = \text{MMD}(P, Q_w), \quad Q_w := \sum_{i=1}^n w_i \delta_{x_i},$$

the maximum mean discrepancy between P and the (signed) weighted node measure Q_w .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Write the two functionals through their representers. For any $f \in \mathcal{H}$, the generalized kernel trick gives $P[f] = \langle f, \mu_P \rangle_{\mathcal{H}}$, and the reproducing property gives $f(x_i) = \langle f, k(x_i, \cdot) \rangle_{\mathcal{H}}$. Hence

$$P[f] - \sum_i w_i f(x_i) = \left\langle f, \mu_P - \sum_i w_i k(x_i, \cdot) \right\rangle_{\mathcal{H}}.$$

Taking the supremum of the absolute value over $\|f\|_{\mathcal{H}} \leq 1$, Cauchy-Schwarz bounds it by $\|\mu_P - \sum_i w_i k(x_i, \cdot)\|_{\mathcal{H}}$, and the bound is attained at the unit vector aligned with $\mu_P - \sum_i w_i k(x_i, \cdot)$. The right-hand side is $\|\mu_P - \mu_{Q_w}\|_{\mathcal{H}}$ because the embedding of $Q_w = \sum_i w_i \delta_{x_i}$ is $\mu_{Q_w} = \sum_i w_i k(x_i, \cdot)$, which is the definition of $\text{MMD}(P, Q_w)$. $\square$

The identity is the hinge of the chapter. Designing a quadrature rule is the same problem as approximating one point of $\mathcal{H}$, the target embedding μ_P , by a weighted combination of the feature vectors $k(x_i, \cdot)$ sitting at the nodes. Everything reduces to making that approximation good. Expanding the squared norm turns it into ordinary linear algebra in the Gram matrix, which is all any numerical routine ever touches.

Proposition (worst-case error in coordinates)

Let $K \in \mathbb{R}^{n \times n}$ be the Gram matrix $K_{ij} = k(x_i, x_j)$, let $z \in \mathbb{R}^n$ be the *kernel mean vector* $z_i = \mu_P(x_i) = \mathbb{E}_{X \sim P}[k(x_i, X)]$, and let $C = \mathbb{E}_{X, X' \sim P}[k(X, X')] = \|\mu_P\|_{\mathcal{H}}^2$. Then

$$e(w)^2 = C - 2w^\top z + w^\top K w.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Expand the squared norm by bilinearity:

$$\left\| \mu_P - \sum_i w_i k(x_i, \cdot) \right\|_{\mathcal{H}}^2 = \langle \mu_P, \mu_P \rangle_{\mathcal{H}} - 2 \sum_i w_i \langle \mu_P, k(x_i, \cdot) \rangle_{\mathcal{H}} + \sum_{i,j} w_i w_j \langle k(x_i, \cdot), k(x_j, \cdot) \rangle_{\mathcal{H}}.$$

The first term is $\|\mu_P\|_{\mathcal{H}}^2 = \mathbb{E}_{X, X' \sim P}[k(X, X')] = C$. In the second, $\langle \mu_P, k(x_i, \cdot) \rangle_{\mathcal{H}} = \mu_P(x_i) = z_i$ by the reproducing property. In the third, $\langle k(x_i, \cdot), k(x_j, \cdot) \rangle_{\mathcal{H}} = k(x_i, x_j) = K_{ij}$. Collecting the three pieces gives $C - 2w^\top z + w^\top K w$. $\square$

The three ingredients have plain meanings. The constant C is the self-similarity of P , fixed once the target and kernel are chosen. The vector z records how well each node "sees" the target, since $z_i = \mu_P(x_i)$ is the target embedding read off at x_i . The Gram matrix K records how redundant the nodes are with each other. A good rule wants nodes with large z (near the mass of P) but small off-diagonal K (not clustered), and the weights trade these off. Before optimizing, it is worth pinning down exactly what Monte Carlo achieves in this language, so we know the bar to beat.

Proposition (Monte Carlo error)

If the nodes are drawn i.i.d. from P and weighted uniformly, $w_i = 1/n$, then

$$\mathbb{E}[e(w)^2] = \frac{1}{n} \left(\mathbb{E}_{X \sim P}[k(X, X)] - \mathbb{E}_{X, X' \sim P}[k(X, X')] \right).$$

In particular $e(w) = O_P(1/\sqrt{n})$, and for a normalized kernel with $k(x, x) = 1$ the bracket is $1 - C$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

With uniform weights, $e(w)^2 = C - \frac{2}{n} \sum_i \mu_P(x_i) + \frac{1}{n^2} \sum_{i,j} k(x_i, x_j)$. Take the expectation over the i.i.d. draw. Each $\mathbb{E}[\mu_P(x_i)] = \mathbb{E}_{X, X' \sim P}[k(X, X')] = C$. In the double sum, the n diagonal terms average to $\mathbb{E}[k(X, X)]$ and the $n(n-1)$ off-diagonal terms, having independent arguments, average to C . Thus

$$\mathbb{E}[e(w)^2] = C - 2C + \frac{1}{n} \mathbb{E}[k(X, X)] + \frac{n-1}{n} C = \frac{1}{n} (\mathbb{E}[k(X, X)] - C).$$

$\square$

So Monte Carlo's expected squared worst-case error falls like $1/n$, and its worst-case error like $1/\sqrt{n}$. Two levers remain untouched in that bound: the weights are frozen at $1/n$, and the nodes are placed at random. The rest of the chapter pulls each lever. Optimizing the weights on fixed nodes gives Bayesian quadrature; optimizing the nodes greedily gives herding; optimizing both, or choosing the sampling law of the nodes cleverly, connects to leverage scores.

33.3 Optimally weighted quadrature

Hold the nodes fixed and minimize the worst-case error over the weights. Because $e(w)^2 = C - 2w^\top z + w^\top K w$ is a convex quadratic in w with Hessian $2K \geq 0$, the minimizer is found by setting the gradient to zero.

Proposition (optimal weights)

For fixed nodes with positive definite Gram matrix K , the worst-case error $e(w)^2$ is minimized by

$$w^\star = K^{-1}z,$$

and the minimal value is

$$e(w^\star)^2 = C - z^\top K^{-1}z.$$

The optimal weights need not be nonnegative and need not sum to one.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Differentiate $e(w)^2 = C - 2w^\top z + w^\top K w$ in w : the gradient is $-2z + 2Kw$, which vanishes at $w^\star = K^{-1}z$. Since the Hessian $2K$ is positive definite, $w^\star$ is the unique global minimum. Substituting, $e(w^\star)^2 = C - 2(K^{-1}z)^\top z + (K^{-1}z)^\top K(K^{-1}z) = C - 2z^\top K^{-1}z + z^\top K^{-1}z = C - z^\top K^{-1}z$. Nothing constrains the sign or the sum of the entries of $K^{-1}z$. $\square$

The quantity $z^\top K^{-1}z$ is the squared norm of the projection of μ_P onto the span of the node features, so $e(w^\star)^2 = C - z^\top K^{-1}z$ is the squared distance from μ_P to that span: the optimal rule is the orthogonal projection of the target embedding onto the nodes, and no reweighting can beat a projection. A small numeric instance shows the projection at work and how much it saves over uniform averaging.

Example (optimal weights beat uniform)

Target $P = \mathcal{N}(0, 1)$. Kernel $k(x, x') = \exp(-(x - x')^2/2)$, the Gaussian with lengthscale 1. Nodes $x = (-1, 0, 1)$. For this Gaussian-kernel, Gaussian-measure pair the two integrals are closed form: the kernel mean is $\mu_P(x) = \frac{1}{\sqrt{2}}e^{-x^2/4}$ and the self-similarity is $C = \mathbb{E}_{X, X'}[k(X, X')] = 1/\sqrt{3} = 0.577350$.

1. Assemble the Gram matrix. With $k(x_i, x_j) = e^{-(x_i - x_j)^2/2}$,

$$K = \begin{pmatrix} 1 & 0.606531 & 0.135335 \\ 0.606531 & 1 & 0.606531 \\ 0.135335 & 0.606531 & 1 \end{pmatrix}.$$

2. Read off the kernel mean vector. Evaluate $z_i = \mu_P(x_i) = \frac{1}{\sqrt{2}}e^{-x_i^2/4}$ at the three nodes: $z = (0.550695, 0.707107, 0.550695)$. The middle node, sitting on the mode of P , sees the most mass.
3. Score the uniform rule. With $w = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$, $e(w)^2 = C - 2w^\top z + w^\top K w = 0.004662$, so $e(w) = 0.068281$.
4. Solve for the optimal weights. $w^\star = K^{-1}z = (0.304856, 0.337297, 0.304856)$. These sum to 0.947010, not 1: the optimal rule deliberately undershoots the total mass to cancel the overlap between neighbouring nodes.
5. Score the optimal rule. $e(w^\star)^2 = C - z^\top K^{-1}z = 0.003079$, so $e(w^\star) = 0.055490$.

Reading. Reweighting the very same three nodes drops the squared worst-case error from 0.004662 to 0.003079, a fall of 0.001583, about 34%, at zero extra evaluations of f . The gain is pure geometry: the uniform rule places μ_{Q_w} somewhere in the node span, while $w^\star$ places it at the foot of the perpendicular from μ_P .

Verification artifact. checks/example-ch-quad-example-32-1.json records the example source hash and verification scope.

33.4 Bayesian quadrature

The optimal-weight rule can be derived a second way that supplies something the worst-case view does not: an honest posterior uncertainty about the integral. Instead of treating f as an unknown element of a norm ball, treat it as a random function with a Gaussian process prior whose covariance is the kernel k . Then the integral $P[f]$ is a random scalar, and conditioning on the observed evaluations gives a full posterior over it. This is Bayesian quadrature, introduced by O’Hagan (1991) as Bayes-Hermite quadrature and revived for machine learning as Bayesian Monte Carlo by Rasmussen and Ghahramani (2003). The construction belongs to the same circle of ideas as Gaussian process regression: a GP prior, linear observations, a Gaussian posterior.

Put $f \sim \mathcal{GP}(0, k)$. Integration against P is a linear functional, so the pair $(f(x_1), \dots, f(x_n), P[f])$ is jointly Gaussian. The node evaluations have covariance $K_{ij} = k(x_i, x_j)$; the cross-covariance between $P[f]$ and $f(x_i)$ is $\mathbb{E}[P[f]f(x_i)] = \int k(x, x_i) dP(x) = \mu_P(x_i) = z_i$; and the prior variance of $P[f]$ is $\mathbb{E}[P[f]^2] = \iint k(x, x') dP(x) dP(x') = C$. Conditioning the Gaussian on the observed values $f_i = f(x_i)$ yields a Gaussian posterior for the integral with the standard formulas.

Definition (Bayesian quadrature)

Under a $\mathcal{GP}(0, k)$ prior on f , given evaluations $f = (f_1, \dots, f_n)^\top$ at the nodes, the posterior over $P[f]$ is Gaussian with mean and variance

$$\mathbb{E}[P[f] \mid f] = z^\top K^{-1} f, \quad \text{Var}[P[f] \mid f] = C - z^\top K^{-1} z.$$

Two things deserve to be spelled out. First, the posterior mean is itself a quadrature rule: $z^\top K^{-1} f = \sum_i w_i^\star f_i$ with weights $w^\star = K^{-1} z$, the very weights of the optimal rule above. Bayesian quadrature and worst-case-optimal quadrature are the same estimator, reached from a probabilistic and a minimax door. Second, the posterior variance $C - z^\top K^{-1} z$ is precisely the minimal worst-case squared error $e(w^\star)^2$. The GP’s stated uncertainty about the integral coincides with the rule’s guaranteed error against the unit ball, so the error bar is not a heuristic but the exact worst case over the RKHS. This is why Example (optimal weights beat uniform) is simultaneously a Bayesian quadrature calculation: the number 0.003079 is at once the squared worst-case error and the posterior variance of the integral for those three nodes.

Algorithm (Bayesian quadrature weights)

Input Kernel k ; target P through its kernel mean $z_i = \mu_P(x_i)$ and self-similarity $C = \mathbb{E}_{P \otimes P}[k]$; nodes $x_1, \dots, x_n$; evaluations $f_i = f(x_i)$.

Output Estimate $\hat{Z}$ of $P[f]$ and its posterior variance V (= squared worst-case error).

1. Form the Gram matrix $K_{ij} = k(x_i, x_j)$ and the kernel mean vector $z = (\mu_P(x_i))_i$.
2. Solve the linear system $K w = z$ for the weights $w = K^{-1} z$.
3. Return the estimate $\hat{Z} = w^\top f = \sum_i w_i f_i$.
4. Return the variance $V = C - z^\top w = C - z^\top K^{-1} z$.

The bottleneck is the kernel mean vector z : it needs the integrals $\mu_P(x_i) = \int k(x_i, x) dP(x)$ in closed form or by a cheap sub-routine. These are tabulated for the pairs that matter in practice, Gaussian kernel against a Gaussian or mixture-of-Gaussians measure, and Bayesian quadrature is used exactly where evaluations of f are expensive enough that solving an $n \times n$ system to squeeze each one dry is worthwhile. Briol et al. (2019) survey this probabilistic view of integration, its convergence theory, and the calibration of the posterior variance.

33.5 Kernel herding

Bayesian quadrature optimizes the weights but leaves the nodes to us. The complementary move fixes the weights at uniform, $1/n$, so that the output is a plain sample we can hand to any downstream averaging code, and instead chooses the nodes to drive down the embedding error. Kernel herding, introduced by Welling (2009) and analyzed as a source of “super-samples” by Chen, Welling, and Smola (2010), does this greedily, one node at a time.

The objective after choosing t nodes is the squared worst-case error of the equally weighted rule,

$$E_t^2 = \left\| \mu_P - \frac{1}{t} \sum_{s=1}^t k(x_s, \cdot) \right\|_{\mathcal{H}}^2,$$

the squared MMD between P and the uniform empirical measure on the nodes. Herding adds the node that, given those already chosen, most reduces this quantity. To see which node that is, view the procedure through Frank-Wolfe, the observation of Bach, Lacoste-Julien, and Obozinski (2012). Minimizing $J(g) = \frac{1}{2} \|g - \mu_P\|_{\mathcal{H}}^2$ over the marginal polytope $\mathcal{M} = \text{conv}\{k(x, \cdot) : x \in \mathcal{X}\}$, the conditional-gradient step linearizes J at the current iterate g_{t-1} and picks the vertex minimizing $\langle \nabla J(g_{t-1}), k(x, \cdot) \rangle = \langle g_{t-1} - \mu_P, k(x, \cdot) \rangle$. By the reproducing property this inner product is $g_{t-1}(x) - \mu_P(x)$, so the step chooses

$$x_t = \arg \max_{x \in \mathcal{X}} [\mu_P(x) - g_{t-1}(x)], \quad g_{t-1}(x) = \frac{1}{t-1} \sum_{s=1}^{t-1} k(x_s, x),$$

with $x_1 = \arg \max_x \mu_P(x)$, and the step size $1/t$ makes the running iterate the uniform average $g_t = \frac{1}{t} \sum_{s \leq t} k(x_s, \cdot)$. The rule is intuitive on its own terms. The *acquisition* $\mu_P(x) - g_{t-1}(x)$ is the residual between the target smoothed density and the density already assembled from earlier nodes. Herding pushes the next node wherever P still has unrepresented mass, which forces the nodes apart: once a region is covered, its residual drops and the search moves on. This self-avoidance is the source of the improvement over random sampling, which happily revisits covered regions.

Algorithm (kernel herding, greedy selection)

Input Kernel k ; target P through its kernel mean $x \mapsto \mu_P(x) = \mathbb{E}_{X \sim P}[k(x, X)]$; candidate set $\mathcal{X}$; budget n .

Output Nodes $x_1, \dots, x_n$ carrying equal weights $1/n$.

1. Select the mode of the embedding: $x_1 \leftarrow \arg \max_{x \in \mathcal{X}} \mu_P(x)$.
2. For $t = 2, \dots, n$: form the running model $g_{t-1}(x) = \frac{1}{t-1} \sum_{s < t} k(x_s, x)$ and select $x_t \leftarrow \arg \max_{x \in \mathcal{X}} [\mu_P(x) - g_{t-1}(x)]$.
3. Return $\{x_1, \dots, x_n\}$ with uniform weights $1/n$.

The greedy criterion is cheap: each step is one pass over the candidates evaluating a scalar acquisition, with no linear solve. We trace three steps on the same target and kernel as before, so the two examples can be compared directly.

Example (three greedy herding steps)

Target $P = \mathcal{N}(0, 1)$, kernel $k(x, x') = e^{-(x-x')^2/2}$, so $\mu_P(x) = \frac{1}{\sqrt{2}} e^{-x^2/4}$ and $C = 1/\sqrt{3}$, as in the previous example. Candidate grid $\mathcal{X} = \{-1, -0.5, 0, 0.5, 1\}$. Ties in the $\arg \max$ are broken toward the smaller (leftmost) grid point.

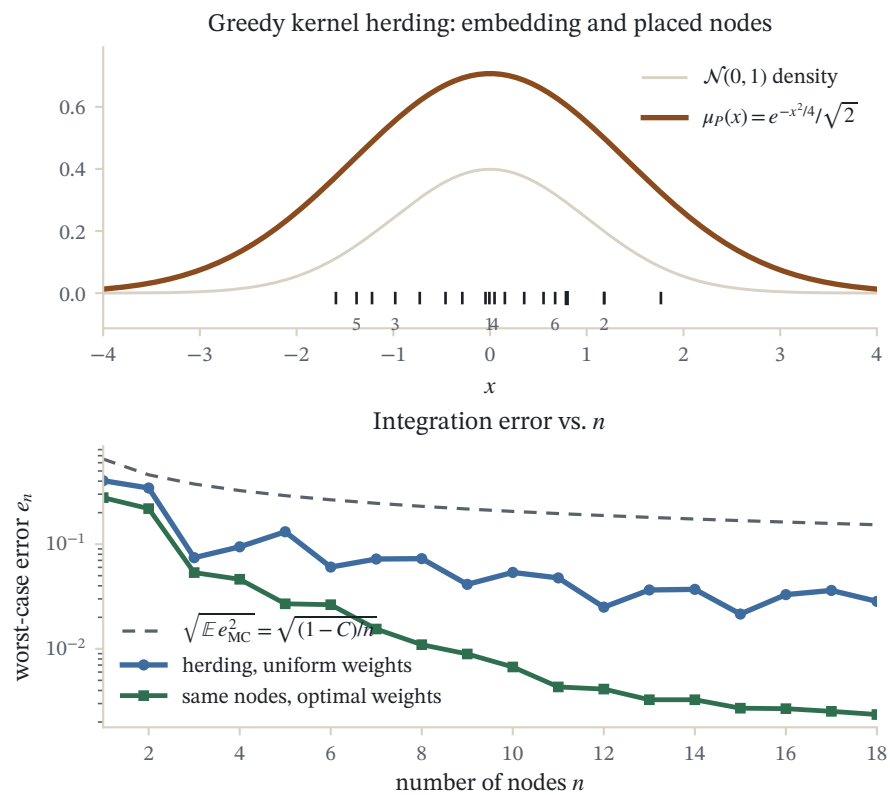


Figure 33.1: Kernel herding greedily places nodes at the argmax of the herding criterion for a standard-normal target (Gaussian kernel, closed-form embedding $\mu_P(x) = e^{-x^2/4}/\sqrt{2}$). The lower panel shows the worst-case integration error falling as nodes accrue, faster still when the nodes are optimally reweighted.

1. Pick the first node at the mode. The acquisition is μ_P itself: over the grid it is (0.550695, 0.664265, 0.707107, 0.664265, 0.550695) maximal at $x_1 = 0$. The one-node error is $E_1^2 = C - 2\mu_P(0) + 1 = 0.163137$, so $E_1 = 0.403902$.
2. Subtract the first node and pick again. With $g_1(x) = k(0, x)$, the acquisition $\mu_P(x) - k(0, x)$ over the grid is (−0.055835, −0.218232, −0.292893, −0.218232, −0.055835). It is most negative at the centre, where the first node already over-covers, and least negative, hence maximal, at the symmetric pair ± 1 ; the tie breaks to $x_2 = -1$. The two-node error falls to $E_2^2 = 0.122814$, $E_2 = 0.350448$.
3. Subtract both and pick the mirror point. With $g_2(x) = \frac{1}{2}(k(0, x) + k(-1, x))$, the acquisition $\mu_P(x) - g_2(x)$ is (−0.25257, −0.218232, −0.096159, 0.060691, 0.179762), maximal at $x_3 = 1$: having placed mass on the left, the residual now peaks on the right. The error drops sharply to $E_3^2 = 0.004662$, $E_3 = 0.068281$.

Reading. The errors decrease monotonically, $0.163137 \rightarrow 0.122814 \rightarrow 0.004662$, and the three greedy nodes are exactly $\{0, -1, 1\} = \{-1, 0, 1\}$, the node set of Example (optimal weights beat uniform). Herding chose that set from scratch with equal weights and worst-case squared error 0.004662; optimally reweighting it, that is, running Bayesian quadrature on the same nodes, presses the error further to 0.003079. For contrast, Proposition (Monte Carlo error) says a random 3-node rule has expected squared error $(1 - C)/3 = 0.140883$, some thirty times larger than herding’s deterministic 0.004662. Deliberate placement, not luck, is what buys the accuracy.

Verification artifact. checks/example-ch-quad-example-32-2.json records the example source hash and verification scope.

33.5.1 Super-samples and the rate, stated carefully

The name “super-samples” comes from a convergence claim that must be stated with its hypotheses attached, because the literature is easy to over-read. Chen, Welling, and Smola (2010) prove that when μ_P lies in the *interior* of the marginal polytope $\mathcal{M}$, with a ball of radius $r > 0$ around it inside $\mathcal{M}$, the herding nodes satisfy

$$E_n = \left\| \mu_P - \frac{1}{n} \sum_{s=1}^n k(x_s, \cdot) \right\|_{\mathcal{H}} = \mathcal{O}\left(\frac{1}{n}\right),$$

a quadratic improvement on Monte Carlo’s $O(1/\sqrt{n})$. Since n herding nodes then match the integration error of n^2 i.i.d. draws, they earn the name. The interior condition holds automatically when $\mathcal{H}$ is finite-dimensional, for instance for a polynomial kernel of bounded degree, and there the fast rate is real. Bach, Lacoste-Julien, and Obozinski (2012), by identifying herding with Frank-Wolfe, supply both the guarantee and its ceiling. The generic Frank-Wolfe analysis gives $E_n^2 = O(1/n)$, that is $E_n = O(1/\sqrt{n})$, with no assumption at all, matching Monte Carlo. The faster $O(1/n)$ on E_n needs the positive-radius interior condition, and that condition typically *fails* in an infinite-dimensional RKHS such as the Gaussian’s, where μ_P sits on the boundary of $\mathcal{M}$ and no interior ball exists. So the precise statement is: herding never does asymptotically worse than Monte Carlo, is provably a square faster when the embedding is interior (in particular in finite dimension), and in the infinite-dimensional characteristic-kernel case the proven worst-case guarantee is the same $O(1/\sqrt{n})$, though the error is deterministic and empirically often better. Bach, Lacoste-Julien, and Obozinski (2012) also show that fully-corrective and line-search Frank-Wolfe variants, which re-optimize the weights at every step, recover faster rates, which is the bridge to the next point.

33.5.2 Optimally weighted herding is Bayesian quadrature

Herding fixes the weights at $1/n$; Bayesian quadrature optimizes them. Composing the two is irresistible: pick the nodes greedily as herding does, then reweight them optimally as Bayesian quadrature does. Huszár and Duvenaud (2012) observed that this composition is not a heuristic patch but exactly Bayesian quadrature restricted to the herded nodes, and that reweighting can only reduce the worst-case error, since the uniform weights are a feasible point of the very minimization $w^\star$ solves. The improvement in the worked examples is the general phenomenon in miniature: the herded nodes $\{-1, 0, 1\}$ carry squared error 0.004662 at uniform weights and 0.003079 at the optimal weights $w^\star = K^{-1}z$. Uniform weights make herding’s output a genuine sample, usable wherever an

unweighted sample is expected; optimal weights make it a quadrature rule with the smallest error those nodes admit. The choice is exactly the trade between wanting super-samples and wanting a minimal-error estimate of one integral.

33.6 Optimal nodes, leverage scores, and determinantal sampling

Herding and Bayesian quadrature both take the node budget as given and work hard per node. A different question asks, before any greedy search, from what distribution the nodes should be *drawn* so that even random placement is near-optimal. Bach (2017) answers it by showing that kernel quadrature rules and random-feature expansions are two views of one approximation, and that the right sampling law is dictated by the spectrum of the kernel integral operator rather than by P alone. Sampling nodes proportionally to the *leverage function*, the analogue for integration of the ridge leverage scores that govern column subsampling in large-scale kernel machines, concentrates evaluations where the operator has energy that the current budget cannot yet resolve.

The governing quantity is the effective dimension, or number of degrees of freedom,

$$d(\lambda) = \text{tr}(\Sigma(\Sigma + \lambda I)^{-1}) = \sum_j \frac{\sigma_j}{\sigma_j + \lambda},$$

where σ_j are the eigenvalues of the integral operator of k under P , the same spectrum that sets learning rates in Mercer's theorem and rates. Bach (2017) shows that $O(d(\lambda) \log d(\lambda))$ nodes drawn from the leverage distribution suffice to integrate every unit-norm RKHS function to accuracy $\sqrt{\lambda}$, so the node count tracks the intrinsic complexity of the problem, not the ambient dimension. Independent leverage sampling still wastes budget on near-duplicate nodes, and the cure is to make the draw repulsive. A *determinantal point process* assigns a set of nodes probability proportional to $\det K_S$, the Gram determinant of the selected subset, which is the squared volume they span in feature space and is small whenever two nodes are similar. Determinantal sampling therefore builds the self-avoidance of herding into a single random draw, and it delivers quadrature rules whose error matches the optimal-weight bound in expectation, tying the random and greedy routes back together.

33.7 A note on coresets

Stepping back, every construction in this chapter produces a *coreset*: a small, possibly weighted set of points whose empirical measure stands in for P on a whole class of queries. Here the query class is integration of RKHS functions, and the approximation guarantee is exactly the worst-case error $\|\mu_P - \mu_{Q_w}\|_{\mathcal{H}} = \text{MMD}(P, Q_w)$: a set with small MMD to P answers every unit-norm RKHS integral to within that MMD, uniformly. Chen, Welling, and Smola's super-samples are an MMD coreset built greedily; Bayesian quadrature reweights a coreset to minimize its MMD; leverage and determinantal sampling draw one at random with the right repulsion. This reading also connects the material to optimal transport, since both the MMD and the Wasserstein distance are ways to measure how well a discrete measure represents a continuous one, and to the broader compression literature, where MMD-coresets summarize massive datasets for downstream averaging. The review of Muandet, Fukumizu, Sriperumbudur, and Schölkopf (2017) collects these threads, from herding and Bayesian quadrature to the sampling schemes, under the single banner of approximating a mean embedding.

33.8 Summary

Numerical integration in an RKHS is embedding approximation. The worst-case error of a quadrature rule over the unit ball equals $\|\mu_P - \sum_i w_i k(x_i, \cdot)\|_{\mathcal{H}}$, the MMD between the target and the weighted node measure, which in coordinates is $C - 2w^\top z + w^\top K w$. Minimizing over the weights gives $w^\star = K^{-1}z$ with error $C - z^\top K^{-1}z$; the

same estimator is the posterior mean of Bayesian quadrature under a Gaussian process prior, and its irreducible error is the posterior variance, so the RKHS worst case and the Bayesian error bar coincide. Minimizing over the nodes greedily, with weights frozen at $1/n$, is kernel herding, whose acquisition $\mu_P(x) - g_{t-1}(x)$ drives nodes into unrepresented mass and yields deterministic super-samples: provably a square faster than Monte Carlo when the embedding is interior to the marginal polytope, and never slower, with the fast rate contingent on that condition. Optimally reweighting herded nodes is Bayesian quadrature again and can only help. Choosing the nodes' sampling law by leverage scores or by a determinantal process makes even random placement track the effective dimension $d(\lambda)$, and every one of these rules is a coresets for integration, certified by its MMD to P . The next parts put the same embedding machinery to work on transport and on probabilistic prediction.

33.9 Common mistakes and practical implications

For **Kernel Quadrature and Herding**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

33.10 Summary and further reading

This chapter established explain the central definitions and claims in Kernel Quadrature and Herding; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (O'Hagan 1991), (Rasmussen and Ghahramani 2003), (Welling 2009).

33.11 Exercises

1. warm-up Show that a quadrature rule is exact on the constant function $f \equiv 1$ if and only if its weights sum to one. Then, using $e(w)^2 = C - 2w^\top z + w^\top K w$, explain in one sentence why the optimal weights $w^\star = K^{-1}z$ of Proposition (optimal weights) are generally *not* forced to sum to one, and what property of the kernel would be needed for the constant to lie in the RKHS.
2. computation Take $P = \frac{1}{2}\delta_a + \frac{1}{2}\delta_b$ with $a \neq b$, a two-point target, and any normalized kernel with $k(x, x) = 1$. Compute $C = \mathbb{E}_{X, X' \sim P}[k(X, X')]$ and the kernel mean $\mu_P(x) = \frac{1}{2}k(a, x) + \frac{1}{2}k(b, x)$. Using the single node $x_1 = a$ with the optimal weight from $w^\star = K^{-1}z$ (here a scalar), find the weight and the resulting worst-case squared error $C - z^\top K^{-1}z$. For $k(a, b) = \rho$, express both in terms of ρ .
3. proof Prove Proposition (Monte Carlo error) in the biased-estimator form: for i.i.d. nodes and uniform weights, show directly that $\mathbb{E}[e(w)^2] = \frac{1}{n}(\mathbb{E}[k(X, X)] - \mathbb{E}[k(X, X')])$, being careful to separate the n diagonal and $n(n-1)$ off-diagonal terms of $\frac{1}{n^2} \sum_{i,j} k(x_i, x_j)$. Conclude that the worst-case error is $O_P(1/\sqrt{n})$ and that it does not depend on the placement, only on the kernel and P .
4. computation Reproduce the first herding step of Example (three greedy herding steps) with a coarser grid $\mathcal{X} = \{-2, -1, 0, 1, 2\}$ and the same $P = \mathcal{N}(0, 1)$, $k(x, x') = e^{-(x-x')^2/2}$. Compute the step-2 acquisition $\mu_P(x) - k(0, x)$ on this grid, show that it is maximized at $x = \pm 2$, and verify numerically that adding $x_2 = -2$ with equal weights *raises* the error above E_1^2 . Explain why this does not contradict the convergence claim of Section (super-samples and the rate). Hint

Use $\mu_P(x) = \frac{1}{\sqrt{2}}e^{-x^2/4}$, $C = 1/\sqrt{3}$. The uniform-weight, fixed step $1/t$ update is Frank-Wolfe without line search, whose objective can rise on a single step even though the bound $E_n^2 = O(1/n)$ still holds; a line search, that is optimal reweighting, would forbid the increase.

5. proof Show that Bayesian quadrature reproduces optimal weighting. Starting from the jointly Gaussian vector $(f(x_1), \dots, f(x_n), P[f])$ with the covariances $K_{ij} = k(x_i, x_j)$, $\text{Cov}(P[f], f(x_i)) = z_i$, and $\text{Var}(P[f]) = C$, apply the Gaussian conditioning formula to derive the posterior mean $z^\top K^{-1}f$ and variance $C - z^\top K^{-1}z$ of Definition (Bayesian quadrature). Identify the posterior-mean weights with $w^\star = K^{-1}z$ and the posterior variance with $e(w^\star)^2$. Hint

For a jointly Gaussian (u, v) the conditional mean is $\mathbb{E}[v \mid u] = \Sigma_{vu}\Sigma_{uu}^{-1}u$ and variance $\Sigma_{vv} - \Sigma_{vu}\Sigma_{uu}^{-1}\Sigma_{uv}$. Here $u = f$, $v = P[f]$, $\Sigma_{uu} = K$, $\Sigma_{vu} = z^\top$, $\Sigma_{vv} = C$.

6. proof Derive the herding acquisition from the Frank-Wolfe step. For $J(g) = \frac{1}{2}\|g - \mu_P\|_{\mathcal{H}}^2$ on $\mathcal{M} = \overline{\text{conv}}\{k(x, \cdot)\}$, compute the gradient $\nabla J(g_{t-1}) = g_{t-1} - \mu_P$, and show that the conditional-gradient vertex $\arg \min_x \langle \nabla J(g_{t-1}), k(x, \cdot) \rangle$ equals $\arg \max_x [\mu_P(x) - g_{t-1}(x)]$. Then verify that the step $g_t = (1 - \frac{1}{t})g_{t-1} + \frac{1}{t}k(x_t, \cdot)$ unrolls to the uniform average $g_t = \frac{1}{t} \sum_{s \leq t} k(x_s, \cdot)$. Hint

Use the reproducing property $\langle h, k(x, \cdot) \rangle_{\mathcal{H}} = h(x)$ to turn the inner product into a pointwise value, and note that minimizing $g_{t-1}(x) - \mu_P(x)$ is maximizing $\mu_P(x) - g_{t-1}(x)$. Induct on t for the unrolling, with base case $g_1 = k(x_1, \cdot)$.

7. challenge A determinantal point process on a finite candidate set $\{x_1, \dots, x_m\}$ with kernel matrix K draws a subset S with probability proportional to $\det K_S$, the principal minor on S . For a size-2 draw, show that $\Pr(\{i, j\}) \propto k(x_i, x_i)k(x_j, x_j) - k(x_i, x_j)^2$, and deduce that for a normalized kernel the pair is chosen with probability proportional to $1 - k(x_i, x_j)^2$. Explain in one or two sentences how this realizes the same self-avoidance that herding produces greedily, and why it therefore tends to place quadrature nodes far apart in feature space. Hint

The 2×2 minor is $\det \begin{pmatrix} k_{ii} & k_{ij} \\ k_{ij} & k_{jj} \end{pmatrix} = k_{ii}k_{jj} - k_{ij}^2$. With $k_{ii} = k_{jj} = 1$ this is $1 - k_{ij}^2$, which vanishes as the two points coincide ($k_{ij} \rightarrow 1$) and is largest when they are dissimilar ($k_{ij} \rightarrow 0$).

34 Conditional Mean Embeddings and Kernel Bayes' Rule

The previous chapter, on mean embeddings, turned a whole probability distribution into a single vector in an RKHS and used the distance between such vectors to compare distributions. That construction embeds a marginal, an object that lives on its own. But most of probabilistic modeling is about conditioning: given that we observed $X = x$, what do we now believe about Y ? This chapter embeds the conditional distribution $P(Y | X = x)$, not as one vector but as a linear operator that turns the feature of the conditioning value x into the embedding of the resulting conditional. Once conditioning is an operator, the elementary calculus of probability, the sum rule, the chain rule, and Bayes' rule, can be rewritten as operator algebra on embeddings, giving a way to carry out probabilistic inference entirely inside the RKHS, with no densities, no parametric model, and no integration, only Gram matrices estimated from a joint sample. We build the operator, give its honest regularized estimate, recognize it as a vector-valued regression, and assemble the pieces into Kernel Bayes' Rule and, at a high level, a kernel filter for state-space models.

34.1 From marginal to conditional embeddings

Fix two spaces, an input space $\mathcal{X}$ with a positive definite kernel k and RKHS $\mathcal{H}_{\mathcal{X}}$, and an output space $\mathcal{Y}$ with kernel ℓ and RKHS $\mathcal{H}_{\mathcal{Y}}$. Write $\varphi(x) = k(x, \cdot) \in \mathcal{H}_{\mathcal{X}}$ and $\psi(y) = \ell(y, \cdot) \in \mathcal{H}_{\mathcal{Y}}$ for the two feature maps. The marginal embedding of a distribution P on $\mathcal{Y}$ was $\mu_P = \mathbb{E}_{Y \sim P}[\psi(Y)]$, the average feature. The natural conditional analogue simply averages the output feature under the conditional law.

Definition (conditional mean embedding)

Let (X, Y) be a random pair on $\mathcal{X} \times \mathcal{Y}$. For each $x \in \mathcal{X}$ at which $P(Y | X = x)$ is well defined, the *conditional mean embedding* of $P(Y | X = x)$ is

$$\mu_{Y|X=x} := \mathbb{E}[\psi(Y) | X = x] = \int_{\mathcal{Y}} \ell(y, \cdot) dP(y | x) \in \mathcal{H}_{\mathcal{Y}}.$$

This is not one embedding but a whole family, one element of $\mathcal{H}_{\mathcal{Y}}$ for every conditioning value x . What makes it useful is the same generalized reproducing property that made the marginal embedding useful, now conditional: averaging the kernel trick $g(Y) = \langle g, \psi(Y) \rangle_{\mathcal{H}_{\mathcal{Y}}}$ under the conditional law gives, for every $g \in \mathcal{H}_{\mathcal{Y}}$,

$$\langle g, \mu_{Y|X=x} \rangle_{\mathcal{H}_{\mathcal{Y}}} = \mathbb{E}[g(Y) | X = x].$$

Read it as the promise of the whole chapter: the single vector $\mu_{Y|X=x}$ stores the conditional expectation of every RKHS function g , recovered by one inner product. If we can produce these vectors from data, we can answer any question of the form "what is the expected value of $g(Y)$ once we learn $X = x$ " without ever writing down the conditional density. The reproducing-property background here is exactly that of the RKHS chapter; the new content is that conditioning turns out to be linear in the feature $\varphi(x)$.

34.2 The conditional mean embedding operator

The move introduced by Song, Huang, Smola, and Fukumizu (2009) is to seek a single linear operator that produces the entire family at once, mapping the input feature to the conditional embedding:

$$\mu_{Y|X=x} = \mathcal{C}_{Y|X} \varphi(x), \quad \mathcal{C}_{Y|X} : \mathcal{H}_X \rightarrow \mathcal{H}_Y.$$

If such an operator exists, conditioning is a matrix-vector product in feature space, and this is what will let the sum and Bayes rules become operator identities. To build it we need the second-order objects that summarize how the two feature maps covary, the covariance operators, studied for RKHS first by Baker (1973) and brought into learning by Fukumizu, Bach, and Jordan (2004).

Definition (covariance operators)

Assume $\mathbb{E}[k(X, X)] < \infty$ and $\mathbb{E}[\ell(Y, Y)] < \infty$. The (*uncentered*) covariance operator $\mathcal{C}_{XX} : \mathcal{H}_X \rightarrow \mathcal{H}_X$ and the cross-covariance operator $\mathcal{C}_{YX} : \mathcal{H}_X \rightarrow \mathcal{H}_Y$ are

$$\mathcal{C}_{XX} = \mathbb{E}[\varphi(X) \otimes \varphi(X)], \quad \mathcal{C}_{YX} = \mathbb{E}[\psi(Y) \otimes \varphi(X)],$$

where $(a \otimes b)c = \langle b, c \rangle a$. They are characterized by

$$\langle f, \mathcal{C}_{XX} f' \rangle_{\mathcal{H}_X} = \mathbb{E}[f(X) f'(X)], \quad \langle g, \mathcal{C}_{YX} f \rangle_{\mathcal{H}_Y} = \mathbb{E}[f(X) g(Y)],$$

for all $f, f' \in \mathcal{H}_X$ and $g \in \mathcal{H}_Y$.

These are the RKHS versions of a variance and a covariance matrix: $\mathcal{C}_{XX}$ records how input features co-vary with themselves, $\mathcal{C}_{YX}$ how output features co-vary with input features. With them the operator is forced.

Proposition (the conditional embedding operator, Song et al. 2009)

Suppose that for every $g \in \mathcal{H}_Y$ the conditional expectation $x \mapsto \mathbb{E}[g(Y) | X = x]$ belongs to $\mathcal{H}_X$. Then the operator $\mathcal{C}_{Y|X}$ satisfying $\mu_{Y|X=x} = \mathcal{C}_{Y|X} \varphi(x)$ for all x obeys

$$\mathcal{C}_{Y|X} \mathcal{C}_{XX} = \mathcal{C}_{YX}, \quad \text{so formally} \quad \mathcal{C}_{Y|X} = \mathcal{C}_{YX} \mathcal{C}_{XX}^{-1}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Write $h_g(x) := \mathbb{E}[g(Y) | X = x]$, which lies in $\mathcal{H}_X$ by assumption. The defining reproducing property says $\langle g, \mathcal{C}_{Y|X} \varphi(x) \rangle = \langle g, \mu_{Y|X=x} \rangle = h_g(x)$, while the reproducing property in $\mathcal{H}_X$ gives $h_g(x) = \langle h_g, \varphi(x) \rangle$. Since both hold for every x , the adjoint acts as $\mathcal{C}_{Y|X}^* g = h_g$. Now test $\mathcal{C}_{XX} \mathcal{C}_{Y|X}^* g$ against any $f \in \mathcal{H}_X$, using the tower property $\mathbb{E}[f(X)g(Y)] = \mathbb{E}[f(X) \mathbb{E}[g(Y) | X]]$:

$$\langle f, \mathcal{C}_{XX} \mathcal{C}_{Y|X}^* g \rangle = \langle f, \mathcal{C}_{XX} h_g \rangle = \mathbb{E}[f(X) h_g(X)] = \mathbb{E}[f(X) g(Y)] = \langle f, \mathcal{C}_{YX}^* g \rangle.$$

As f and g range over their spaces, $\mathcal{C}_{XX} \mathcal{C}_{Y|X}^* = \mathcal{C}_{YX}^*$; taking adjoints and using that $\mathcal{C}_{XX}$ is self-adjoint gives $\mathcal{C}_{Y|X} \mathcal{C}_{XX} = \mathcal{C}_{YX}$. Wherever $\mathcal{C}_{XX}^{-1}$ is meaningful this reads $\mathcal{C}_{Y|X} = \mathcal{C}_{YX} \mathcal{C}_{XX}^{-1}$. $\square$

The identity is clean, but the words "formally" and "wherever it is meaningful" are doing real work, and honesty requires unpacking them.

Remark (why the inverse is delicate)

Two subtleties sit inside $\mathcal{C}_{YX}\mathcal{C}_{XX}^{-1}$. First, for a bounded kernel $\mathcal{C}_{XX}$ is a compact (indeed trace-class) operator, so its eigenvalues accumulate at zero and its inverse $\mathcal{C}_{XX}^{-1}$ is *unbounded*: the product $\mathcal{C}_{YX}\mathcal{C}_{XX}^{-1}$ need not be a bounded operator on $\mathcal{H}_X$ at all. Second, the assumption that $\mathbb{E}[g(Y) | X = \cdot] \in \mathcal{H}_X$ for every $g \in \mathcal{H}_Y$ is genuinely restrictive: the conditional expectation can be smoother or rougher than the input RKHS contains, and then no exact operator $\mathcal{C}_{Y|X}$ reproduces the family. Both problems are met the same way, by regularizing: replace $\mathcal{C}_{XX}^{-1}$ by $(\mathcal{C}_{XX} + \lambda I)^{-1}$ for some $\lambda > 0$, which is bounded and always defined. The regression view of the next section removes the range assumption entirely, by never claiming an exact operator, only a best approximation.

34.3 The empirical estimate

In practice we never see $\mathcal{C}_{XX}$ or $\mathcal{C}_{YX}$; we see a joint sample $\{(x_i, y_i)\}_{i=1}^n$ drawn from $P(X, Y)$. Collect the features into the "feature matrices" $\Phi = (\varphi(x_1), \dots, \varphi(x_n))$ and $Y = (\psi(y_1), \dots, \psi(y_n))$, whose columns are RKHS elements, and form the two Gram matrices $K_{ij} = k(x_i, x_j)$ and $L_{ij} = \ell(y_i, y_j)$. The empirical covariance operators are $\hat{\mathcal{C}}_{XX} = \frac{1}{n}\Phi\Phi^*$ and $\hat{\mathcal{C}}_{YX} = \frac{1}{n}Y\Phi^*$, and the regularized conditional embedding operator is

$$\hat{\mathcal{C}}_{Y|X} = \hat{\mathcal{C}}_{YX}(\hat{\mathcal{C}}_{XX} + \lambda I)^{-1} = Y(K + n\lambda I)^{-1}\Phi^*,$$

where the second equality is the push-through identity $\Phi^*(\Phi\Phi^* + cI)^{-1} = (\Phi^*\Phi + cI)^{-1}\Phi^*$ with $c = n\lambda$. Applying it to the feature of a test input x gives the embedding as a weighted sum of the training output features,

$$\hat{\mu}_{Y|X=x} = \hat{\mathcal{C}}_{Y|X}\varphi(x) = \sum_{i=1}^n \beta_i(x) \psi(y_i), \quad \beta(x) = (K + n\lambda I)^{-1}\mathbf{k}_x,$$

with $\mathbf{k}_x = (k(x_1, x), \dots, k(x_n, x))^T$. The weight vector $\beta(x)$ is exactly a kernel ridge regression solution, the same object as the Gaussian-process posterior weights of the GP chapter, except that here the "targets" are the output features $\psi(y_i)$ rather than scalar labels. Reading off any conditional expectation is then a finite sum, since for $g \in \mathcal{H}_Y$,

$$\hat{\mathbb{E}}[g(Y) | X = x] = \langle g, \hat{\mu}_{Y|X=x} \rangle = \sum_{i=1}^n \beta_i(x) g(y_i).$$

Algorithm (empirical conditional mean embedding)

Input Joint sample $\{(x_i, y_i)\}_{i=1}^n$; input kernel k , output kernel ℓ ; regularization $\lambda > 0$; test input x ; query function $g \in \mathcal{H}_Y$.

Output Weights $\beta(x) \in \mathbb{R}^n$ with $\hat{\mu}_{Y|X=x} = \sum_i \beta_i(x)\psi(y_i)$, and the estimate $\hat{\mathbb{E}}[g(Y) | X = x]$.

1. Form the input Gram matrix $K \in \mathbb{R}^{n \times n}$, $K_{ij} = k(x_i, x_j)$.
2. Form the test vector $\mathbf{k}_x = (k(x_1, x), \dots, k(x_n, x))^T$.
3. Solve the ridge system $(K + n\lambda I)\beta(x) = \mathbf{k}_x$ for the weights $\beta(x)$.
4. Return $\hat{\mathbb{E}}[g(Y) | X = x] = \sum_{i=1}^n \beta_i(x) g(y_i)$, and, if the full embedding is needed, the weighted set $\{(\beta_i(x), y_i)\}$.

Regularization is not optional decoration here. The matrix K is often numerically singular (repeated or nearby inputs make its columns almost dependent), and even in the population the operator inverse is unbounded, so the raw $K^{-1}\mathbf{k}_x$ would be a wild, high-variance vector. The term $n\lambda I$ floors the spectrum of K at $n\lambda$, trading a little bias for a large reduction in variance, exactly the bias-variance dial of ridge regression.

Example (predicting a conditional expectation)

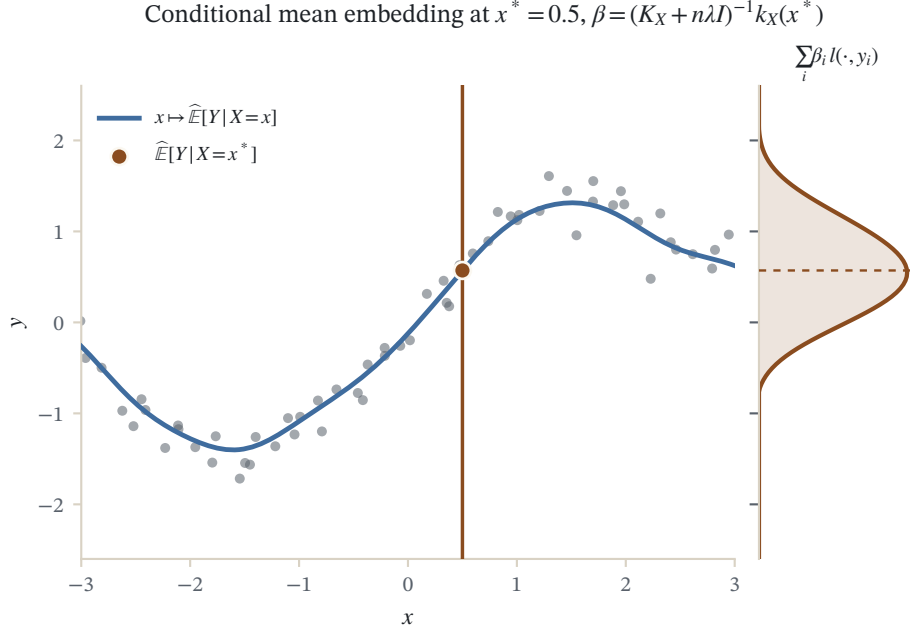


Figure 34.1: The conditional mean embedding $\beta(x^*) = (K_X + n\lambda I)^{-1} k_X(x^*)$ applied to sixty samples: the side profile is the embedded conditional $\sum_i \beta_i l(\cdot, y_i)$ and the dot marks its conditional mean at the conditioning point x^* .

Four joint pairs $(x_i, y_i) = (0, 0.0), (1, 1.2), (2, 1.8), (3, 3.0)$, so y roughly tracks x . Gaussian kernels $k(x, x') = e^{-(x-x')^2/2}$ and $\ell(y, y') = e^{-(y-y')^2/2}$ (both bandwidths 1). Regularization $\lambda = 0.125$, so $n\lambda = 0.5$. Test input $x_* = 1.5$. All numbers from `checks/ch-cme-ex1.py`.

1. Form the input Gram matrix. With $e^{-1/2} = 0.6065$, $e^{-2} = 0.1353$, $e^{-9/2} = 0.0111$,

$$K = \begin{pmatrix} 1 & 0.6065 & 0.1353 & 0.0111 \\ 0.6065 & 1 & 0.6065 & 0.1353 \\ 0.1353 & 0.6065 & 1 & 0.6065 \\ 0.0111 & 0.1353 & 0.6065 & 1 \end{pmatrix}.$$

2. Build the test vector. The distances from $x_* = 1.5$ to $(0, 1, 2, 3)$ give $\mathbf{k}_{x_*} = (0.3247, 0.8825, 0.8825, 0.3247)$.
3. Solve the ridge system. $\beta(x_*) = (K + 0.5 I)^{-1} \mathbf{k}_{x_*} = (0.0111, 0.4150, 0.4150, 0.0111)$, with $\sum_i \beta_i = 0.8522$.
4. Predict the conditional mean of the response. Using the plug-in $g(y) = y$, $\hat{\mathbb{E}}[Y | X = 1.5] = \sum_i \beta_i(x_*) y_i = 0.4150(1.2) + 0.4150(1.8) + 0.0111(0.0 + 3.0) = 1.2784$.
5. Predict a genuine RKHS query. For $g = \ell(2.0, \cdot) \in \mathcal{H}_y$, evaluate $g(y_i) = \ell(y_i, 2.0) = (0.1353, 0.7261, 0.9802, 0.6065)$, so $\hat{\mathbb{E}}[\ell(2.0, Y) | X = 1.5] = \sum_i \beta_i(x_*) g(y_i) = 0.7164$.

Reading. The two central training points, at $x = 1$ and $x = 2$, carry almost all the weight and receive equal shares because $x_* = 1.5$ sits symmetrically between them; the far points are all but ignored. The response estimate 1.28 is close to the sensible 1.5 but pulled down, because the weights sum to 0.85 rather than 1: regularization shrinks the embedding toward the origin, which biases the plug-in mean toward 0. Note also that the RKHS query 0.7164 is not equal to $\ell(2.0, \hat{\mathbb{E}}[Y | X = 1.5]) = 0.7708$: the embedding estimates $\mathbb{E}[g(Y) | X]$, the average of g over the conditional, not g evaluated at the conditional mean, and the two differ whenever g is nonlinear.

Verification artifact. `checks/example-ch-cme-example-33-1.json` records the example source hash and verification scope.

34.4 The regression view

The estimator $Y(K + n\lambda I)^{-1}\Phi^*$ was derived by regularizing an operator inverse, which left open both the range assumption and the question of what, precisely, it estimates when that assumption fails. Grünewälder, Lever, Baldassarre, Patterson, Gretton, and Pontil (2012) gave the reassuring answer: the empirical conditional mean embedding is the solution of a vector-valued ridge regression, and this holds with no assumption on the range of $\mathcal{C}_{XX}$.

The idea is to regress the output feature onto the input feature. Since $\mathbb{E}[\psi(Y) \mid X = x] = \mu_{Y|X=x}$, the feature $\psi(Y)$ is an unbiased, noisy observation of the target embedding $\mu_{Y|X}$, exactly the setup of least-squares regression, only with values in the Hilbert space $\mathcal{H}_Y$ instead of $\mathbb{R}$. The population target minimizes the surrogate risk $\mathcal{E}(C) = \mathbb{E} \|\psi(Y) - C\varphi(X)\|_{\mathcal{H}_Y}^2$ over operators C , whose minimizer over all measurable maps is the true conditional embedding; the empirical, regularized version is what we compute.

Proposition (CME as vector-valued regression, Grünewälder et al. 2012)

Among Hilbert-Schmidt operators $C : \mathcal{H}_X \rightarrow \mathcal{H}_Y$, the regularized empirical risk

$$\hat{\mathcal{E}}_\lambda(C) = \frac{1}{n} \sum_{i=1}^n \left\| \psi(y_i) - C\varphi(x_i) \right\|_{\mathcal{H}_Y}^2 + \lambda \|C\|_{\text{HS}}^2$$

is minimized by $\hat{\mathcal{C}}_{Y|X} = Y(K + n\lambda I)^{-1}\Phi^*$, the empirical conditional embedding operator.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Three payoffs follow from this identification. It is an honest definition: $\hat{\mathcal{C}}_{Y|X}$ is always the minimizer of a well-posed risk, whether or not the population range assumption holds, so we never rely on $\mathbb{E}[g(Y) \mid X = \cdot] \in \mathcal{H}_X$ to compute anything. It exposes the estimator as ridge regression, with $\beta(x)$ the ridge weights, making the whole toolbox of vector-valued RKHS regression available. And it delivers consistency: as $n \rightarrow \infty$ with $\lambda = \lambda_n \rightarrow 0$ at a suitable rate, $\|\hat{\mu}_{Y|X=x} - \mu_{Y|X=x}\|_{\mathcal{H}_Y} \rightarrow 0$ in probability, with rates that sharpen under smoothness (source) conditions on the target, in the same spirit as the learning rates of the rates chapter. Song et al. (2009) first proved consistency; the regression analysis of Grünewälder et al. (2012) tightened the rates, and the survey of Muandet, Fukumizu, Sriperumbudur, and Schölkopf (2017) collects the sharpest current statements.

34.5 Kernel probability rules

With a conditioning operator in hand we can lift the elementary rules of probability to embeddings. Throughout, a distribution is represented by its embedding and a conditional by an operator; the rules say how to combine them. Fix a *prior* π on $\mathcal{X}$ with embedding $\mu_X^\pi = \mathbb{E}_{X \sim \pi}[\varphi(X)]$, and read $\mathcal{C}_{Y|X}$ as the likelihood model $P(Y \mid X)$ it was estimated from.

34.5.1 The kernel sum rule

The sum rule of probability computes a marginal, $Q(y) = \int P(y \mid x) d\pi(x)$. Its embedding form is a single operator application. Draw $X \sim \pi$ and then $Y \sim P(Y \mid X)$; the embedding of the resulting Y -marginal is

$$\mu_Y^\pi = \mathbb{E}_{X \sim \pi}[\mathbb{E}[\psi(Y) \mid X]] = \mathbb{E}_{X \sim \pi}[\mathcal{C}_{Y|X}\varphi(X)] = \mathcal{C}_{Y|X} \mu_X^\pi,$$

the law of total expectation with the linear operator pulled outside the expectation. Empirically, if the prior embedding is carried by weights $\mathbf{m} \in \mathbb{R}^n$ on the training inputs, $\hat{\mu}_X^\pi = \sum_j m_j \varphi(x_j) = \Phi \mathbf{m}$, then

$$\hat{\mu}_Y^\pi = \hat{\mathcal{C}}_{Y|X} \Phi \mathbf{m} = Y (K + n\lambda I)^{-1} K \mathbf{m} = \sum_{i=1}^n \rho_i \psi(y_i), \quad \rho = (K + n\lambda I)^{-1} K \mathbf{m}.$$

The weights ρ are the sum-rule reweighting of the sample induced by the prior.

34.5.2 The kernel chain rule

The chain rule factors a joint as $Q(x, y) = P(y | x)\pi(x)$. In embedding language the joint is represented by an (uncentered) cross-covariance operator, and the chain rule says it is obtained by reweighting the empirical joint feature outer products with the same sum-rule weights ρ :

$$\mathcal{C}_{XY}^\pi = \sum_{i=1}^n \rho_i \varphi(x_i) \otimes \psi(y_i) = \Phi \text{diag}(\rho) Y^*, \quad \mathcal{C}_{YY}^\pi = \sum_{i=1}^n \rho_i \psi(y_i) \otimes \psi(y_i) = Y \text{diag}(\rho) Y^*.$$

These are the prior-weighted second-order embeddings that Bayes' rule will invert. Song, Fukumizu, and Gretton (2013) develop the sum and chain rules in this operator form as the building blocks of kernel probabilistic inference.

34.6 Kernel Bayes' Rule

Bayes' rule inverts the conditioning: from a prior $\pi(x)$ and a likelihood $P(y | x)$ it produces the posterior $Q(x | y) \propto P(y | x)\pi(x)$. We want its embedding, $\mu_{X|Y=y}^\pi$, computed entirely from the joint sample and the prior weights, with no densities anywhere. Fukumizu, Song, and Gretton (2013) obtained it by recognizing the posterior as a conditional embedding of X given Y under the prior-weighted model Q , and applying the conditional embedding operator built from the reweighted covariances:

$$\mu_{X|Y=y}^\pi = \mathcal{C}_{XY}^\pi (\mathcal{C}_{YY}^\pi)^{-1} \psi(y).$$

The catch is that $\mathcal{C}_{YY}^\pi = Y \text{diag}(\rho) Y^*$ is built from the signed weights ρ , which need not be nonnegative, so it is generally not a positive operator and a plain Tikhonov inverse $(\mathcal{C}_{YY}^\pi + \delta I)^{-1}$ can behave badly. Fukumizu et al. (2013) therefore invert the square, which is always positive, and use

$$\mu_{X|Y=y}^\pi = \mathcal{C}_{XY}^\pi ((\mathcal{C}_{YY}^\pi)^2 + \delta I)^{-1} \mathcal{C}_{YY}^\pi \psi(y).$$

Turning the operators into Gram matrices with the push-through identity gives the finite-sample rule below, where $D = \text{diag}(\rho)$ and $\ell_y = (\ell(y_1, y), \dots, \ell(y_n, y))^T$.

Algorithm (Kernel Bayes' Rule)

Input Joint sample $\{(x_i, y_i)\}_{i=1}^n$; kernels k, ℓ with Gram matrices K, L ; prior weights $\mathbf{m} \in \mathbb{R}^n$ (embedding $\sum_i m_i \varphi(x_i)$); regularizers $\varepsilon, \delta > 0$; observation y ; query $g \in \mathcal{H}_X$.

Output Posterior weights $\mathbf{w}(y) \in \mathbb{R}^n$ with $\hat{\mu}_{X|Y=y}^\pi = \sum_i w_i(y) \varphi(x_i)$, and $\hat{\mathbb{E}}[g(X) | Y = y]$.

1. Kernel sum rule: solve $(K + n\varepsilon I)\rho = K\mathbf{m}$ for the prior-weighted embedding weights ρ , and set $D = \text{diag}(\rho)$.
2. Form the observation vector $\ell_y = (\ell(y_1, y), \dots, \ell(y_n, y))^T$.
3. Kernel Bayes' rule: compute $\mathbf{w}(y) = DL((DL)^2 + \delta I)^{-1} D \ell_y$.
4. Return $\hat{\mathbb{E}}[g(X) | Y = y] = \sum_{i=1}^n w_i(y) g(x_i)$; if a distribution is wanted, project $\mathbf{w}$ onto the nonnegative simplex (or normalize).

Example (a Bayesian posterior update)

Five joint pairs $(x_i, y_i) = (0, 0), (1, 1), (2, 2), (3, 3), (4, 4)$, Gaussian kernels of bandwidth 1 on both sides. A non-uniform prior favoring large inputs, $\mathbf{m} = (0.05, 0.05, 0.10, 0.30, 0.50)$, so the prior mean of X is 3.15. Regularizers $\varepsilon = 0.1$ (so $n\varepsilon = 0.5$) and $\delta = 0.01$. We observe $y = 1.0$. All numbers from `checks/ch-cme-ex2.py`.

1. Apply the sum rule. $\rho = (K + 0.5 I)^{-1} K \mathbf{m} = (0.0363, 0.0436, 0.0995, 0.2620, 0.3488)$, inheriting the prior's tilt toward large x ; set $D = \text{diag}(\rho)$.
2. Encode the observation. $\ell_y = (\ell(y_i, 1.0)) = (0.6065, 1.0000, 0.6065, 0.1353, 0.0111)$, peaked at the training pair whose $y_i = 1$.
3. Apply Bayes' rule. $\mathbf{w}(y) = DL((DL)^2 + 0.01 I)^{-1} D \ell_y = (0.1100, 0.1847, 0.2679, 0.0513, -0.0721)$, with $\sum_i w_i = 0.542$.
4. Read off the posterior mean. Raw, $\sum_i w_i x_i = 0.5862$; after normalizing the weights to sum to one, the posterior mean of X is 1.0817.

Reading. The prior placed the mean of X at 3.15, betting on large inputs. Observing $y = 1$, which in this sample is the signature of $x = 1$, pulls the posterior mass down to concentrate near $x = 1$: the normalized posterior mean is 1.08, so the likelihood has correctly overwhelmed the prior. Two honesty notes. The posterior weights do not sum to one and one of them is negative (-0.072 at $x = 4$): the output of Kernel Bayes' Rule is an RKHS embedding, not a probability vector, so to read it as a distribution one normalizes or projects onto the simplex. And the answer depends on the two regularizers ε, δ , the price of the unbounded operator inverses that would otherwise be ill-defined.

Verification artifact. `checks/example-ch-cme-example-33-2.json` records the example source hash and verification scope.

Remark (consistency and its price)

Fukumizu, Song, and Gretton (2013) prove that the Kernel Bayes' Rule estimator is consistent: with the regularizers decayed at appropriate rates as $n \rightarrow \infty$, the estimated posterior embedding converges to the true one, and posterior expectations of RKHS functions converge to their correct values. The convergence is slower than for a single conditional embedding, because Bayes' rule chains two regularized inversions, the sum rule and the operator-square inverse, and each contributes error. This is the recurring bargain of the chapter: operator inverses that are unbounded in the population must be regularized in the sample, which buys well-posedness and consistency at the cost of a bias controlled by $\lambda, \varepsilon, \delta$ and of outputs that are embeddings rather than literal probabilities.

34.7 Kernelized inference and the kernel Bayes filter

The rules assembled above are exactly what a filtering algorithm needs. Consider a state-space model with a hidden state s_t that evolves through an unknown transition $s_t \rightarrow s_{t+1}$ and emits an observation o_t through an unknown observation model. Classical filters, the Kalman filter for the linear-Gaussian case and particle filters more generally, maintain the belief state $P(s_t | o_{1:t})$ and update it as each observation arrives. The *kernel Bayes filter* of Fukumizu, Song, and Gretton (2013) carries out the same recursion with the belief represented as a mean embedding $\mu_{s_t | o_{1:t}}$ and both models learned nonparametrically from training trajectories, alternating two steps.

Algorithm (kernel Bayes filter, one step)

Input Belief weights for the embedding $\mu_{s_t | o_{1:t}}$; a transition conditional embedding $\mathcal{C}_{s_{t+1} | s_t}$ and an observation model, both estimated from training trajectories; the new observation o_{t+1} .

Output Updated belief weights for the embedding $\mu_{s_{t+1} | o_{1:t+1}}$.

1. Prediction: push the current belief through the transition operator with the kernel sum rule, $\mu_{s_{t+1} | o_{1:t}} = \mathcal{C}_{s_{t+1} | s_t} \mu_{s_t | o_{1:t}}$.
2. Correction: fold in o_{t+1} with Kernel Bayes' Rule, using the predicted embedding as the prior, to obtain the filtered belief $\mu_{s_{t+1} | o_{1:t+1}}$.

Nothing in this recursion writes down a density or assumes linearity or Gaussianity; the dynamics and the observation model live entirely in Gram matrices built from data. That is what makes the kernel Bayes filter attractive when the state space is a complex object, when the transition and observation models are unknown but sampled trajectories are available, or when the true dynamics are strongly non-Gaussian. The same operator machinery also underpins kernel treatments of causal effect estimation, where a conditional embedding stands in for an interventional distribution, taken up in the causal inference chapter. The survey of Muandet et al. (2017) catalogues these inference applications and the assumptions each one needs.

34.8 Summary

Conditioning, the workhorse of probabilistic modeling, becomes a linear operator on embeddings. The conditional mean embedding $\mu_{Y|X=x} = \mathbb{E}[\psi(Y) | X = x]$ stores every conditional expectation $\mathbb{E}[g(Y) | X = x]$ as an inner product, and it is produced by a single operator $\mathcal{C}_{Y|X} = \mathcal{C}_{YX}\mathcal{C}_{XX}^{-1}$ applied to the input feature. That population identity hides two hazards, an unbounded covariance inverse and a restrictive range assumption, both cured by regularization, giving the empirical estimate $\hat{\mathcal{C}}_{Y|X} = Y(K + n\lambda I)^{-1}\Phi^*$, whose weights $\beta(x) = (K + n\lambda I)^{-1}\mathbf{k}_x$ are a kernel ridge regression. Grünewälder et al. (2012) made that precise: the conditional mean embedding is vector-valued regression of the output feature on the input feature, which frees it from the range assumption and delivers consistency. Lifting the sum rule, chain rule, and Bayes' rule to operators gives kernel probabilistic inference, culminating in Kernel Bayes' Rule, $\mathbf{w}(y) = DL((DL)^2 + \delta I)^{-1}D\ell_y$, which updates a prior embedding into a posterior embedding using only a joint sample. Chaining prediction by the sum rule with correction by Bayes' rule yields the kernel Bayes filter, a nonparametric recursion for state-space inference. The honest thread throughout is that these operator inverses are unbounded in the population and must be regularized in the sample, so the outputs are embeddings, consistent as the sample grows but biased at finite n and not literal probability distributions.

34.9 Common mistakes and practical implications

For **Conditional Mean Embeddings and Kernel Bayes' Rule**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

34.10 Summary and further reading

This chapter established explain the central definitions and claims in Conditional Mean Embeddings and Kernel Bayes' Rule; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Song and Fukumizu 2009), (Baker 1973), (Fukumizu and Jordan 2004).

34.11 Exercises

1. warm-up Starting from the definition $\mu_{Y|X=x} = \mathbb{E}[\psi(Y) | X = x]$ with $\psi(y) = \ell(y, \cdot)$, use the reproducing property of $\mathcal{H}_y$ to show the generalized conditional reproducing property $\langle g, \mu_{Y|X=x} \rangle = \mathbb{E}[g(Y) | X = x]$

for every $g \in \mathcal{H}_y$. Specialize to $g = \ell(y_0, \cdot)$ and read off what the number $\mu_{Y|X=x}(y_0)$ computes. In one sentence, say how this generalizes the marginal embedding of the mean-embedding chapter.

2. computation Use the setup of the worked conditional-embedding example (same four points, kernels, and $n\lambda = 0.5$), but move the test input to $x_* = 2.5$. Form $\mathbf{k}_{x_*}$, solve $(K + 0.5I)\beta = \mathbf{k}_{x_*}$, and report $\hat{\mathbb{E}}[Y | X = 2.5] = \sum_i \beta_i y_i$ together with $\sum_i \beta_i$. Explain, from where x_* now sits, which two training points dominate the weights, and why the weight sum is again below one.

3. proof Prove the operator identity $\mathcal{C}_{Y|X} \mathcal{C}_{XX} = \mathcal{C}_{YX}$ under the assumption that $h_g(x) := \mathbb{E}[g(Y) | X = x] \in \mathcal{H}_X$ for every $g \in \mathcal{H}_y$. State clearly where the tower property $\mathbb{E}[f(X)g(Y)] = \mathbb{E}[f(X)h_g(X)]$ enters, and explain in a sentence why dropping the assumption breaks the argument. Hint

The reproducing property forces the adjoint to act as $\mathcal{C}_{Y|X}^* g = h_g$. Test $\mathcal{C}_{XX} \mathcal{C}_{Y|X}^* g = \mathcal{C}_{XX} h_g$ against an arbitrary $f \in \mathcal{H}_X$: $\langle f, \mathcal{C}_{XX} h_g \rangle = \mathbb{E}[f(X)h_g(X)] = \mathbb{E}[f(X)g(Y)] = \langle f, \mathcal{C}_{YX}^* g \rangle$. Take adjoints, using $\mathcal{C}_{XX} = \mathcal{C}_{XX}^*$. Without $h_g \in \mathcal{H}_X$ the quantity $\mathcal{C}_{XX} h_g$ is not even defined, so no exact operator exists.

4. proof Establish the push-through identity used to pass from operators to Gram matrices: for a bounded operator Φ and any $c > 0$, $\Phi^*(\Phi\Phi^* + cI)^{-1} = (\Phi^*\Phi + cI)^{-1}\Phi^*$. Use it to show $\hat{\mathcal{C}}_{YX}(\hat{\mathcal{C}}_{XX} + \lambda I)^{-1} = Y(K + n\lambda I)^{-1}\Phi^*$, identifying $\Phi^*\Phi = K$. Hint

From the identity $\Phi^*(\Phi\Phi^* + cI) = (\Phi^*\Phi + cI)\Phi^*$, left-multiply by $(\Phi^*\Phi + cI)^{-1}$ and right-multiply by $(\Phi\Phi^* + cI)^{-1}$. Then $\hat{\mathcal{C}}_{YX} = \frac{1}{n}Y\Phi^*$ and $\hat{\mathcal{C}}_{XX} = \frac{1}{n}\Phi\Phi^*$, and the $\frac{1}{n}$ factors turn λ into $n\lambda$; the identity moves Φ^* across so the inverse acts on the $n \times n$ matrix $K = \Phi^*\Phi$.

5. computation Derive the finite-sample kernel sum rule. With a prior embedding $\hat{\mu}_X^\pi = \Phi\mathbf{m}$ carried by weights $\mathbf{m}$, show that $\hat{\mu}_Y^\pi = \hat{\mathcal{C}}_{Y|X} \hat{\mu}_X^\pi = \sum_i \rho_i \psi(y_i)$ with $\rho = (K + n\lambda I)^{-1}K\mathbf{m}$. Then, in the setup of the Kernel Bayes' Rule example, replace the tilted prior by the uniform prior $m_i = 1/5$, recompute ρ , and say which direction the sum-rule weights shift and why.

6. exploration Rerun the Kernel Bayes' Rule example (same sample, kernels, prior, and regularizers) but with the observation changed to $y = 3.0$. Compute the posterior weights $\mathbf{w}(y)$ and the normalized posterior mean of X . Compare with the prior mean 3.15 and with the $y = 1.0$ result, and explain how prior and likelihood now reinforce rather than oppose each other. Comment on any negative weight that appears. Hint

Now ℓ_y peaks at the pair with $y_i = 3$, so the likelihood points where the prior already leans; the posterior mean should land near 3, close to but not exactly the prior mean, since the likelihood is informative. Edit `yobs` in `checks/ch-cme-ex2.py` to reproduce every number.

7. challenge Derive the finite-sample Kernel Bayes' Rule weights from the operator form $\mu_{X|Y=y}^\pi = \mathcal{C}_{XY}^\pi((\mathcal{C}_{YY}^\pi)^2 + \delta I)^{-1}\mathcal{C}_{YY}^\pi \psi(y)$, where $\mathcal{C}_{XY}^\pi = \Phi D Y^*$ and $\mathcal{C}_{YY}^\pi = Y D Y^*$ with $D = \text{diag}(\rho)$. Show that the posterior embedding is $\Phi\mathbf{w}(y)$ with $\mathbf{w}(y) = DL((DL)^2 + \delta I)^{-1}D\ell_y$, where $\ell_y = Y^*\psi(y)$. Then explain why the square $(\mathcal{C}_{YY}^\pi)^2$ is inverted rather than $\mathcal{C}_{YY}^\pi$ itself. Hint

Seek the vector $z = ((\mathcal{C}_{YY}^\pi)^2 + \delta I)^{-1}\mathcal{C}_{YY}^\pi \psi(y)$ in the range of Y , writing $z = Y\mathbf{c}$; the component orthogonal to that range is killed by δI and must vanish. Substituting gives $((DL)^2 + \delta I)\mathbf{c} = D\ell_y$, and then $\mathcal{C}_{XY}^\pi z = \Phi D L \mathbf{c}$. The square is inverted because $\mathcal{C}_{YY}^\pi$ is built from the signed weights ρ and need not be positive, whereas $(\mathcal{C}_{YY}^\pi)^2 + \delta I$ is always positive definite.

35 Kernel Stein Discrepancy and Stein Methods

The mean embedding of the mean-embedding chapter compared two distributions from samples of each, and the conditional embedding carried that idea to operators. This chapter answers a harder question that both leave open. Very often we hold samples of one distribution q and, for the other, only a model p that we can evaluate up to an unknown constant: a Bayesian posterior known through its unnormalized density, an energy-based model, a Gibbs distribution. The maximum mean discrepancy cannot help, because it needs samples from p and we have none. The escape is a classical trick of Charles Stein: build an operator $\mathcal{A}_p$ that annihilates p in expectation and depends on p only through its score $\nabla \log p$, which is blind to the normalizer. Pairing that operator with an RKHS turns it into the kernel Stein discrepancy, a computable distance between a sample and an unnormalized model. The same operator, run in reverse, becomes Stein variational gradient descent, a deterministic particle sampler that transports points toward p . Both need only the score of p , never its normalizing constant.

35.1 Goodness of fit for a model you can only score

Fix a probability density p on $\mathbb{R}^d$ and a sample $x_1, \dots, x_n$ drawn from some unknown q . The goodness-of-fit question is whether the sample could have come from p : is $q = p$? When we can sample from p as well, this is just the two-sample problem, and the MMD two-sample test of the hypothesis-testing chapter settles it. The difficulty that defines this chapter is that in the most important cases we cannot sample from p , and we do not even know p exactly. We know it only in the form

$$p(x) = \frac{1}{Z} \tilde{p}(x), \quad Z = \int_{\mathbb{R}^d} \tilde{p}(x) dx,$$

where the unnormalized density $\tilde{p}$ is available in closed form but the constant Z is a high-dimensional integral we cannot evaluate. This is the rule, not the exception. A Bayesian posterior $p(\theta | \mathcal{D}) \propto p(\mathcal{D} | \theta) p(\theta)$ is known through its numerator, while the marginal likelihood $Z = p(\mathcal{D})$ is exactly the intractable integral that makes Bayesian computation hard, a point we return to when SVGD reappears as posterior sampling and which the Gaussian-process chapter meets from the marginal-likelihood side. An energy-based model $p(x) \propto e^{-E(x)}$ hides its normalizer inside the partition function. In every such case the density is a moving target only up to Z , and any discrepancy that requires Z , or requires sampling from p , is useless.

The one object that survives is the gradient of the log density. Because the logarithm turns the product $p = \tilde{p}/Z$ into a difference and Z is constant in x ,

$$\nabla_x \log p(x) = \nabla_x \log \tilde{p}(x) - \nabla_x \log Z = \nabla_x \log \tilde{p}(x).$$

The normalizer contributes nothing to the gradient. So if we can build a discrepancy that reads p only through $\nabla \log p$, it will cost us nothing that Z is unknown. That is precisely what the Stein operator delivers.

35.2 Stein's identity and the Stein operator

The gradient of the log density has a name.

Definition (score function)

The *score* of a differentiable density p on $\mathbb{R}^d$ is the vector field

$$s_p(x) := \nabla_x \log p(x) = \frac{\nabla_x p(x)}{p(x)} \in \mathbb{R}^d.$$

It is invariant to normalization: if $p \propto \tilde{p}$, then $s_p = \nabla_x \log \tilde{p}$, so the score can be computed from the unnormalized model alone.

Stein's method, introduced by Stein (1972) for bounding errors in the normal approximation, rests on a single observation: from the score one can manufacture a family of functions whose expectation under p is exactly zero. Averaging one of them against a sample therefore probes whether the sample really follows p . The construction is an operator.

Definition (Langevin Stein operator)

For a density p with score s_p , the *Stein operator* acts on a smooth vector field $f = (f_1, \dots, f_d) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ by

$$\mathcal{A}_p f(x) := s_p(x)^\top f(x) + \nabla \cdot f(x) = \sum_{i=1}^d (s_{p,i}(x) f_i(x) + \partial_{x_i} f_i(x)),$$

where $\nabla \cdot f = \sum_i \partial_{x_i} f_i$ is the divergence. In one dimension this is simply $\mathcal{A}_p f(x) = s_p(x)f(x) + f'(x)$.

The operator earns its keep through the following identity, whose proof is nothing but integration by parts arranged so the score appears.

Theorem (Stein's identity)

Let p be a differentiable density on $\mathbb{R}^d$ and let f belong to the Stein class of p , meaning f is differentiable, $\mathbb{E}_{X \sim p} |\mathcal{A}_p f(X)| < \infty$, and the flux of pf vanishes at infinity. Then

$$\mathbb{E}_{X \sim p} [\mathcal{A}_p f(X)] = 0.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Write the expectation as an integral and use $s_p(x) p(x) = \nabla p(x)$, the defining property of the score:

$$\mathbb{E}_{X \sim p} [\mathcal{A}_p f(X)] = \int_{\mathbb{R}^d} (s_p(x)^\top f(x) + \nabla \cdot f(x)) p(x) dx = \int_{\mathbb{R}^d} (\nabla p(x)^\top f(x) + p(x) \nabla \cdot f(x)) dx.$$

The integrand is exactly the divergence of the product field pf , since $\nabla \cdot (pf) = (\nabla p)^\top f + p \nabla \cdot f$ by the product rule. By the divergence theorem the integral equals the flux of pf through the sphere at infinity, which vanishes for f in the Stein class:

$$\int_{\mathbb{R}^d} \nabla \cdot (p(x)f(x)) dx = \lim_{R \rightarrow \infty} \oint_{\|x\|=R} p(x)f(x)^\top n(x) dS = 0.$$

In one dimension this is the fundamental theorem of calculus applied to $(pf)' = p'f + pf'$, giving $\int_{\mathbb{R}} (pf)' dx = [pf]_{-\infty}^{\infty} = 0$. $\square$

Read the identity as a test. If our sample truly came from p , then the sample average $\frac{1}{n} \sum_i \mathcal{A}_p f(x_i)$ should hover near zero for every admissible f , because its population value is zero. If the sample came from a different q , there is no reason for the average to vanish, and the amount by which it fails to vanish measures the mismatch. To see exactly what it measures, subtract the identity for q from the expectation under q . Since $\mathbb{E}_{X \sim q}[\mathcal{A}_q f(X)] = 0$ by the same theorem applied to q ,

$$\mathbb{E}_{X \sim q}[\mathcal{A}_p f(X)] = \mathbb{E}_{X \sim q}[\mathcal{A}_p f(X) - \mathcal{A}_q f(X)] = \mathbb{E}_{X \sim q}[(s_p(X) - s_q(X))^\top f(X)],$$

because the divergence terms cancel and only the two scores differ. The Stein operator of p , evaluated under q , sees precisely the gap between the two score fields, weighted by the test field f . It is zero for all f exactly when $s_p = s_q$ holds q -almost everywhere, which for regular densities forces $p = q$. The score difference $s_p - s_q$ is the raw material of every discrepancy in this chapter.

35.3 From the identity to a discrepancy

A single test field f can be fooled: a badly chosen f might average to zero under q even when $q \neq p$. The cure is the same one the MMD used for the mean embedding, namely to search over a whole class of test fields and report the worst case. This defines the Stein discrepancy.

Definition (Stein discrepancy)

For a class $\mathcal{F}$ of vector fields in the Stein class of p , the *Stein discrepancy* between q and p is

$$\mathbb{S}(q, p) := \sup_{f \in \mathcal{F}} \mathbb{E}_{X \sim q}[\mathcal{A}_p f(X)] = \sup_{f \in \mathcal{F}} \mathbb{E}_{X \sim q}[(s_p(X) - s_q(X))^\top f(X)].$$

Everything now hinges on the class $\mathcal{F}$. Too small and the supremum misses real differences; too large and it is neither finite nor computable. The choice made by Gorham and Mackey (2015) that opened the computable theory was a smoothness ball; the choice that closes it in one line, made independently by Liu, Lee, and Jordan (2016) and by Chwialkowski, Strathmann, and Gretton (2016), is the unit ball of a reproducing kernel Hilbert space. As with the MMD, the RKHS ball is the class rich enough to separate distributions yet structured enough that the supremum has a closed form.

35.4 The kernel Stein discrepancy

Let k be a positive definite kernel on $\mathbb{R}^d$ with RKHS $\mathcal{H}$, and let $\mathcal{H}^d = \mathcal{H} \times \cdots \times \mathcal{H}$ be the vector-valued RKHS of fields $f = (f_1, \dots, f_d)$ with $f_i \in \mathcal{H}$ and squared norm $\|f\|_{\mathcal{H}^d}^2 = \sum_i \|f_i\|_{\mathcal{H}}^2$. Taking $\mathcal{F}$ to be its unit ball gives the kernel Stein discrepancy.

Definition (kernel Stein discrepancy)

The *kernel Stein discrepancy* (KSD) between q and the target p is the Stein discrepancy over the unit ball of $\mathcal{H}^d$,

$$\text{KSD}(q, p) := \sup_{\|f\|_{\mathcal{H}^d} \leq 1} \mathbb{E}_{X \sim q}[\mathcal{A}_p f(X)].$$

The supremum collapses because, just as expectations of RKHS functions are inner products with the mean embedding, the whole functional $f \mapsto \mathbb{E}_q[\mathcal{A}_p f]$ is an inner product with a single element of $\mathcal{H}^d$. Recall from

the RKHS chapter the two reproducing identities $f_i(x) = \langle f_i, k(x, \cdot) \rangle_{\mathcal{H}}$ and, for a differentiable kernel, $\partial_{x_i} f_i(x) = \langle f_i, \partial_{x_i} k(x, \cdot) \rangle_{\mathcal{H}}$. Substituting them into $\mathcal{A}_p f$ and pulling the expectation inside the inner product,

$$\mathbb{E}_{X \sim q}[\mathcal{A}_p f(X)] = \sum_{i=1}^d \left\langle f_i, \mathbb{E}_{X \sim q}[s_{p,i}(X) k(X, \cdot) + \partial_{x_i} k(X, \cdot)] \right\rangle_{\mathcal{H}} = \langle f, \xi_{q,p} \rangle_{\mathcal{H}^d},$$

where the *Stein witness* $\xi_{q,p} \in \mathcal{H}^d$ has coordinates $\xi_{q,p,i} = \mathbb{E}_{X \sim q}[s_{p,i}(X) k(X, \cdot) + \partial_{x_i} k(X, \cdot)]$. This is the Stein-transformed analogue of the mean embedding: where $\mu_q = \mathbb{E}_q[k(X, \cdot)]$ averaged the plain feature, $\xi_{q,p}$ averages the feature after the Stein operator has acted on it. By Cauchy-Schwarz the supremum of $\langle f, \xi_{q,p} \rangle$ over the unit ball is $\|\xi_{q,p}\|_{\mathcal{H}^d}$, attained at $f^* = \xi_{q,p} / \|\xi_{q,p}\|_{\mathcal{H}^d}$. So the KSD is the RKHS norm of the Stein witness, exactly as the MMD was the RKHS norm of the difference of embeddings.

35.4.1 The Stein kernel and its closed form

To turn that norm into something computable, expand $\|\xi_{q,p}\|^2 = \langle \xi_{q,p}, \xi_{q,p} \rangle$ as a double expectation. Write the per-point Stein feature as $\beta_p(x, \cdot) = s_p(x)k(x, \cdot) + \nabla_x k(x, \cdot) \in \mathcal{H}^d$, so $\xi_{q,p} = \mathbb{E}_{X \sim q}[\beta_p(X, \cdot)]$. Then $\|\xi_{q,p}\|^2 = \mathbb{E}_{X, X' \sim q} \langle \beta_p(X, \cdot), \beta_p(X', \cdot) \rangle_{\mathcal{H}^d}$, and the inner product of two Stein features is a kernel in its own right.

Definition (Stein kernel)

The *Stein kernel* associated to the target p and base kernel k is

$$u_p(x, x') := \langle \beta_p(x, \cdot), \beta_p(x', \cdot) \rangle_{\mathcal{H}^d} = s_p(x)^\top s_p(x') k(x, x') + s_p(x)^\top \nabla_{x'} k(x, x') + \nabla_x k(x, x')^\top s_p(x') + \sum_{i=1}^d \partial_{x_i} \partial_{x'_i} k(x, x').$$

Equivalently $u_p(x, x') = \mathcal{A}_p^x \mathcal{A}_p^{x'} k(x, x')$, the Stein operator applied in the x slot and again in the x' slot of the kernel.

The four terms come from expanding the inner product coordinatewise with the four reproducing identities: $\langle k(x, \cdot), k(x', \cdot) \rangle = k(x, x')$ pairs the two score parts, the two cross identities $\langle k(x, \cdot), \partial_{x'_i} k(x', \cdot) \rangle = \partial_{x'_i} k(x, x')$ pair a score against a derivative, and $\langle \partial_{x_i} k(x, \cdot), \partial_{x'_i} k(x', \cdot) \rangle = \partial_{x_i} \partial_{x'_i} k(x, x')$ pairs the two derivatives. The decisive feature is written into the formula: u_p depends on p only through the score s_p , and on k through values and derivatives we can differentiate in closed form. Nowhere does the normalizer Z appear. This is what the whole construction was for.

Theorem (KSD as a double expectation, Liu, Lee, and Jordan, 2016)

With u_p the Stein kernel,

$$\text{KSD}^2(q, p) = \mathbb{E}_{X, X' \sim q}[u_p(X, X')],$$

where X, X' are independent draws from q . The quantity is non-negative, and if the base kernel k is sufficiently rich (C_0 -universal, integrally strictly positive definite) and mild integrability holds, then $\text{KSD}(q, p) = 0$ if and only if $q = p$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The witness expansion above gives $\text{KSD}(q, p) = \|\xi_{q,p}\|_{\mathcal{H}^d}$, so $\text{KSD}^2 = \langle \xi_{q,p}, \xi_{q,p} \rangle_{\mathcal{H}^d}$. Writing each copy of $\xi_{q,p}$ as an expectation of β_p and using bilinearity and continuity of the inner product to exchange it with the two independent expectations,

$$\text{KSD}^2(q, p) = \langle \mathbb{E}_X[\beta_p(X, \cdot)], \mathbb{E}_{X'}[\beta_p(X', \cdot)] \rangle_{\mathcal{H}^d} = \mathbb{E}_{X, X' \sim q} \langle \beta_p(X, \cdot), \beta_p(X', \cdot) \rangle_{\mathcal{H}^d} = \mathbb{E}_{X, X' \sim q} [u_p(X, X')].$$

Non-negativity is immediate, since it is a squared norm. As $\text{KSD}^2 = \|\xi_{q,p}\|^2$, it vanishes exactly when $\xi_{q,p} = 0$; a C_0 -universal kernel makes the map $q \mapsto \xi_{q,p}$ injective in the sense that $\xi_{q,p} = 0$ forces the score difference $s_p - s_q$ to vanish q -almost everywhere, hence $q = p$ (Chwialkowski, Strathmann, and Gretton, 2016; Gorham and Mackey, 2017). $\square$

Corollary (the Stein kernel is positive definite)

For fixed p and any base kernel k , the Stein kernel u_p is a positive definite kernel on $\mathbb{R}^d$, because $u_p(x, x') = \langle \beta_p(x, \cdot), \beta_p(x', \cdot) \rangle_{\mathcal{H}^d}$ is a Gram inner product of the feature map $x \mapsto \beta_p(x, \cdot)$. Consequently the empirical KSD is a genuine kernel statistic, and its double sum over any sample is non-negative.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Remark (only the score, and a mean-embedding reading)

The construction parallels the mean embedding term for term, with one change: the feature $k(x, \cdot)$ is replaced by the Stein feature $\beta_p(x, \cdot)$, which folds the target into the map before averaging. The KSD is then the MMD-style RKHS norm of a single embedded object $\xi_{q,p}$, and its double-expectation formula uses u_p exactly where the MMD used k . But whereas the MMD between q and p needed samples from both, the KSD needs samples from q alone and the score of p alone. This asymmetry is the entire point: it is what lets us test against a model we can only evaluate up to its normalizer.

35.4.2 Worked closed form: RBF kernel against a Gaussian target

Nothing here is abstract once the base kernel is fixed. Take the RBF kernel $k(x, x') = \exp(-\|x - x'\|^2/(2h^2))$ in one dimension, whose derivatives are $\partial_x k = -\frac{x-x'}{h^2} k$, $\partial_{x'} k = \frac{x-x'}{h^2} k$, and $\partial_x \partial_{x'} k = (\frac{1}{h^2} - \frac{(x-x')^2}{h^4}) k$. Take the target to be the Gaussian $p = \mathcal{N}(\mu, \sigma^2)$, whose score is the linear field $s_p(x) = -(x - \mu)/\sigma^2$, a formula that plainly never saw the normalizer $1/\sqrt{2\pi\sigma^2}$. Substituting into the four-term Stein kernel and collecting, with $g = x - x'$,

$$u_p(x, x') = \left[s_p(x)s_p(x') + \frac{g}{h^2}(s_p(x) - s_p(x')) + \frac{1}{h^2} - \frac{g^2}{h^4} \right] \exp\left(-\frac{g^2}{2h^2}\right).$$

Specializing further to the standard Gaussian $p = \mathcal{N}(0, 1)$ with $s_p(x) = -x$ and bandwidth $h = 1$, the middle terms telescope, $-xg + x'g = -g^2$, leaving the compact form

$$u_p(x, x') = [xx' + 1 - 2(x - x')^2] \exp\left(-\frac{1}{2}(x - x')^2\right).$$

This is the concrete object we now compute with. It is symmetric, and on the diagonal it reduces to $u_p(x, x) = x^2 + 1$, which grows as a point sits farther from the mean 0, already hinting that badly placed points inflate the discrepancy.

Example (empirical KSD, a fitting and a misfitting sample)

Target $p = \mathcal{N}(0, 1)$, score $s_p(x) = -x$, RBF kernel with $h = 1$, so $u_p(x, x') = [xx' + 1 - 2(x - x')^2] e^{-(x-x')^2/2}$. Two samples of size $n = 3$: sample A = $(-1, 0, 1)$, symmetric about the mean and consistent with p ; sample B = $(2, 3, 4)$, shifted well to the right. All numbers from `checks/ch-ksd-ex1.py`.

1. Build the Stein kernel matrix for sample A. Evaluating $u_p(x_i, x_j)$ on $(-1, 0, 1)$,

$$U^A = \begin{pmatrix} 2.0000 & -0.6065 & -1.0827 \\ -0.6065 & 1.0000 & -0.6065 \\ -1.0827 & -0.6065 & 2.0000 \end{pmatrix},$$

with diagonal $(2, 1, 2)$ equal to $x_i^2 + 1$ as predicted.

2. Average it two ways. The V-statistic averages all $n^2 = 9$ entries; the U-statistic averages only the 6 off-diagonal ones:

$$\widehat{\text{KSD}}_V^2(A) = \frac{1}{9} \sum_{i,j} U_{ij}^A = 0.0454, \quad \widehat{\text{KSD}}_U^2(A) = \frac{1}{6} \sum_{i \neq j} U_{ij}^A = -0.7652.$$

3. Build the Stein kernel matrix for sample B. On $(2, 3, 4)$ the score magnitudes $|s_p| = 2, 3, 4$ are large, so the entries swell,

$$U^B = \begin{pmatrix} 5.0000 & 3.0327 & 0.1353 \\ 3.0327 & 10.0000 & 6.6718 \\ 0.1353 & 6.6718 & 17.0000 \end{pmatrix},$$

with diagonal $(5, 10, 17)$.

4. Average it the same two ways.

$$\widehat{\text{KSD}}_V^2(B) = \frac{51.6797}{9} = 5.7422, \quad \widehat{\text{KSD}}_U^2(B) = \frac{19.6797}{6} = 3.2799.$$

Reading. The manifestly non-negative V-statistic reports 0.045 for the fitting sample against 5.74 for the shifted one, a hundredfold gap that flags B as inconsistent with p . The unbiased U-statistic sharpens the contrast, dipping slightly negative for A (its target is 0, so finite-sample noise puts it just below, exactly as the unbiased MMD does under its null) while staying firmly positive at 3.28 for B. Every entry was computed from the score $s_p(x) = -x$ alone, never from the constant $1/\sqrt{2\pi}$.

Verification artifact. `checks/example-ch-ksd-example-34-1.json` records the example source hash and verification scope.

35.5 Empirical KSD: U-statistics and V-statistics

The worked example already used the two estimators; here they are in general. Given a sample $x_1, \dots, x_n \sim q$, replace the double expectation $\mathbb{E}_{X, X' \sim q}[u_p(X, X')]$ by an average over pairs. Keeping all pairs, including $i = j$, gives the *V-statistic*

$$\widehat{\text{KSD}}_V^2 = \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n u_p(x_i, x_j),$$

which is biased upward because the diagonal terms $u_p(x_i, x_i)$ pair a point with itself, but is guaranteed non-negative since u_p is positive definite. Dropping the diagonal gives the *U-statistic*

$$\widehat{\text{KSD}}_U^2 = \frac{1}{n(n-1)} \sum_{i \neq j} u_p(x_i, x_j),$$

an unbiased estimator of $\text{KSD}^2(q, p)$, which is the usual choice for testing. Both are simple double sums over the $n \times n$ Stein kernel matrix, costing $O(n^2)$ evaluations, each of which needs one score $s_p(x_i)$ and the kernel derivatives. As with the MMD, the price of unbiasedness is that $\widehat{\text{KSD}}_U^2$ can fall below zero when $q = p$.

That sign behavior is the shadow of a deeper fact about the null distribution, and it is the same degeneracy the MMD showed. Under the alternative $q \neq p$ the statistic concentrates around the positive number $\text{KSD}^2(q, p)$ and is asymptotically normal. Under the null $q = p$ the population value is zero, the first-order part of the U-statistic vanishes, and the rescaled statistic has a non-Gaussian limit,

$$n \widehat{\text{KSD}}_U^2 \xrightarrow{d} \sum_{j=1}^{\infty} c_j (Z_j^2 - 1), \quad Z_j \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1),$$

an infinite weighted sum of centered χ_1^2 variables whose weights c_j are the eigenvalues of the Stein kernel u_p under p (Liu, Lee, and Jordan, 2016; Chwialkowski, Strathmann, and Gretton, 2016). A quadratic form in Gaussian noise is not Gaussian, which is why the null cannot be read off a normal table and must be calibrated by resampling.

35.6 A goodness-of-fit test for unnormalized models

We now turn the estimator into a test of $H_0 : q = p$ against $H_1 : q \neq p$. The unknown eigenvalues c_j put the null quantile out of closed-form reach, so we approximate the null by a *wild bootstrap*, following Chwialkowski, Strathmann, and Gretton (2016). The idea is to perturb the double sum by independent sign flips: multiplying the (i, j) term by $W_i W_j$ with $W_i \in \{-1, +1\}$ leaves the diagonal structure intact but randomizes the cross terms so that the resulting statistic mimics a draw from the degenerate null law, because $\mathbb{E}[W_i W_j] = 0$ for $i \neq j$. Repeating this many times traces out the null distribution without ever knowing the eigenvalues, precisely the role the permutation test played for the MMD in the testing chapter.

Algorithm (kernel Stein goodness-of-fit test)

Input Sample $x_1, \dots, x_n \sim q$; score function $s_p = \nabla \log \tilde{p}$ of the target, computed from the unnormalized model; base kernel k and its derivatives; bootstrap count B ; level α .

Output Reject or fail to reject $H_0 : q = p$.

1. Form the Stein kernel matrix $U_{ij} = u_p(x_i, x_j)$ from the four-term formula, using $s_p(x_i)$, $k(x_i, x_j)$, ∇k , and $\nabla \nabla' k$. No normalizer is required.
2. Compute the test statistic $T = \frac{1}{n(n-1)} \sum_{i \neq j} U_{ij}$, the unbiased U-statistic.
3. Draw i.i.d. Rademacher signs $W_1^{(b)}, \dots, W_n^{(b)} \in \{-1, +1\}$ and set the bootstrap statistic $T_b = \frac{1}{n(n-1)} \sum_{i \neq j} W_i^{(b)} W_j^{(b)} U_{ij}$.
4. Repeat step 3 for $b = 1, \dots, B$ to build the null sample $\{T_1, \dots, T_B\}$.
5. Return the p -value $\hat{p} = \frac{1}{B} |\{b : T_b \geq T\}|$; reject H_0 if $\hat{p} \leq \alpha$.

The test reuses the Stein kernel matrix U built once in step 1; every bootstrap replicate is just a re-weighted sum of its entries, so the whole procedure is one $O(n^2)$ matrix build plus B cheap passes. Because it reads p only through the score, it applies verbatim to energy-based models and unnormalized posteriors where a two-sample test is impossible. It comes with two cautions inherited from its ingredients. First, the alternative must register in the score: any q whose score matches p 's on the support it explores will pass, so the test detects departures in $\nabla \log q$, which for continuous densities is all departures but is worth stating. Second, the wild bootstrap, like the permutation calibration of the MMD test, assumes the multiplier signs reproduce the null dependence; the same recipe with autocorrelated multipliers extends the test to the Markov-chain samples of MCMC diagnostics, one of the primary uses of the KSD.

35.7 Stein variational gradient descent

The Stein operator was built to detect a mismatch between q and p . Run it in the other direction and it becomes a rule for removing the mismatch: instead of measuring how a sample fails to follow p , move the sample so that it does. This is Stein variational gradient descent (SVGD) of Liu and Wang (2016), a deterministic sampler that carries a set of particles toward p using nothing but the score.

Represent the current distribution by particles $x_1, \dots, x_n$, and consider nudging every particle by a small smooth map $T(x) = x + \epsilon \phi(x)$ with $\phi \in \mathcal{H}^d$. The pushforward $q_{[T]}$ of the particle distribution changes its KL divergence to p , and the first-order effect is governed exactly by the Stein operator.

Proposition (steepest descent of the KL is the Stein witness, Liu and Wang, 2016)

Let $q_{[T]}$ be the pushforward of q under $T(x) = x + \epsilon \phi(x)$. Then

$$\left. \frac{d}{d\epsilon} \text{KL}(q_{[T]} \parallel p) \right|_{\epsilon=0} = -\mathbb{E}_{X \sim q}[\mathcal{A}_p \phi(X)].$$

Maximizing the rate of decrease over the unit ball $\|\phi\|_{\mathcal{H}^d} \leq 1$ therefore yields the maximal value $\text{KSD}(q, p)$, attained at the normalized Stein witness $\phi^* = \xi_{q,p} / \|\xi_{q,p}\|_{\mathcal{H}^d}$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof (sketch; full derivation in Liu and Wang, 2016)

By the change-of-variables formula the pushforward density is $q_{[T]}(y) = q(T^{-1}(y)) |\det \nabla T^{-1}(y)|$. Differentiating $\text{KL}(q_{[T]} \parallel p) = \mathbb{E}_{q_{[T]}}[\log q_{[T]} - \log p]$ at $\epsilon = 0$, where T is the identity, the entropy term $\mathbb{E}[\log q_{[T]}]$ contributes $-\mathbb{E}_q[\nabla \cdot \phi]$ through the log-determinant, and the cross term contributes $-\mathbb{E}_q[s_p^\top \phi]$ through the shift of $\log p$. Adding them gives $-\mathbb{E}_q[s_p^\top \phi + \nabla \cdot \phi] = -\mathbb{E}_q[\mathcal{A}_p \phi]$. The supremum of this linear functional over the RKHS unit ball is, by the same Cauchy-Schwarz argument that defined the KSD, equal to $\|\xi_{q,p}\|_{\mathcal{H}^d} = \text{KSD}(q, p)$, attained at the aligned unit field. $\square$

The consequence is that the steepest descent direction on the KL divergence is the very witness whose norm is the KSD. So the discrepancy is not just a diagnostic; it is the length of the best move toward p , and following that move is an algorithm. Using the empirical witness, in which the expectation over q becomes an average over the current particles, the update reads off immediately.

Algorithm (SVGD update, one step)

Input Particles $x_1, \dots, x_n$; score $s_p = \nabla \log \tilde{p}$ of the target; base kernel k ; step size ϵ .

Output Updated particles $x_1, \dots, x_n$ that have moved toward p .

1. For each particle x_i , form the empirical perturbation

$$\hat{\phi}(x_i) = \frac{1}{n} \sum_{j=1}^n \left[\underbrace{k(x_j, x_i) s_p(x_j)}_{\text{driving force}} + \underbrace{\nabla_{x_j} k(x_j, x_i)}_{\text{repulsion}} \right].$$

2. Update every particle in parallel, $x_i \leftarrow x_i + \epsilon \hat{\phi}(x_i)$.
3. Repeat from step 1 until the particles stop moving, that is until the empirical KSD falls below a tolerance.

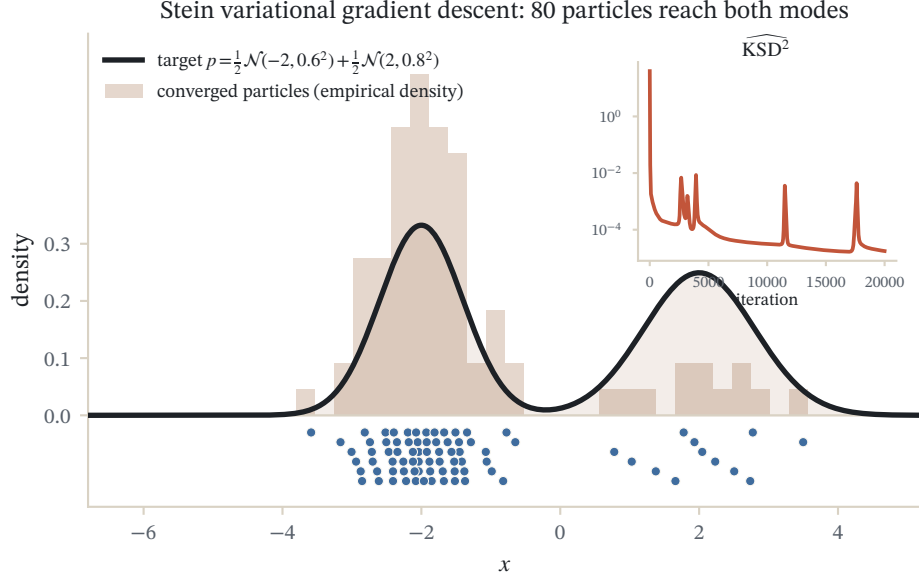


Figure 35.1: Stein variational gradient descent transports particles toward the two-mode target $p = \frac{1}{2}\mathcal{N}(-2, 0.6^2) + \frac{1}{2}\mathcal{N}(2, 0.8^2)$. Kernel repulsion spreads the swarm across the low-density valley so both modes are populated; the empirical kernel Stein discrepancy falls as the flow converges.

The two terms in the update have a clean mechanical reading. The driving force $k(x_j, x_i)s_p(x_j)$ is a kernel-weighted average of the score, pushing each particle toward regions where $\log p$ increases, that is toward the high-density part of the target; a lone particle would follow s_p and slide to a mode. The repulsion $\nabla_{x_j} k(x_j, x_i)$ points particles away from one another, and without it every particle would collapse onto the same mode. Their balance is what makes the particles spread out into the shape of p rather than piling up at its peak. Both terms use only s_p , so SVGD samples an unnormalized posterior with no MCMC and no normalizer, which is why it reappears in Bayesian inference as a fast deterministic alternative to sampling.

Example (one SVGD step toward a standard Gaussian)

Target $p = \mathcal{N}(0, 1)$, score $s_p(x) = -x$, RBF kernel with $h = 1$ so $\nabla_{x_j} k(x_j, x_i) = (x_i - x_j) k(x_j, x_i)$, step size $\epsilon = 0.1$. Three particles start at $x = (1, 2, 3)$, all to the right of the mean. All numbers from `checks/ch-ksd-ex2.py`.

1. Form the kernel matrix. With $K_{ji} = k(x_j, x_i) = e^{-(x_j - x_i)^2/2}$,

$$K = \begin{pmatrix} 1.0000 & 0.6065 & 0.1353 \\ 0.6065 & 1.0000 & 0.6065 \\ 0.1353 & 0.6065 & 1.0000 \end{pmatrix}.$$

2. Average the driving force. The score-weighted term $D_i = \frac{1}{n} \sum_j K_{ji} s_p(x_j)$ is negative for every particle, pulling all three toward the mean 0:

$$D = (-0.8730, -1.4754, -1.4495).$$

3. Average the repulsion. The term $R_i = \frac{1}{n} \sum_j (x_i - x_j) K_{ji}$ is antisymmetric: it pushes the left particle further left, the right particle further right, and leaves the middle one alone,

$$R = (-0.2924, 0.0000, +0.2924).$$

4. Combine into the step and move. With $\hat{\phi}(x_i) = D_i + R_i$,

$$\hat{\phi} = (-1.1654, -1.4754, -1.1571), \quad x^{\text{new}} = x + 0.1 \hat{\phi} = (0.8835, 1.8525, 2.8843).$$

Reading. Every particle moved left toward the target’s center: the cloud mean drops from 2.0000 to 1.8734 in a single step, and repeated steps would settle the three points into a spread that matches $\mathcal{N}(0, 1)$. The driving term did the transport while the repulsion kept the particles from converging to one point, splitting the outer two apart by equal and opposite amounts. As with the test, the update touched p only through the score $s_p(x) = -x$.

Verification artifact. checks/example-ch-ksd-example-34-2.json records the example source hash and verification scope.

35.8 Choosing the kernel, and a bridge to quadrature

The KSD is only as trustworthy as its base kernel, and here the story diverges from the compact-domain intuition of the MMD. On $\mathbb{R}^d$ a light-tailed base kernel can be fooled: Gorham and Mackey (2017) showed that with the Gaussian kernel the KSD can tend to zero along a sequence of q that does not converge to p , because the kernel decays faster than the score grows and stops seeing mass that escapes to infinity. Their fix is to require the KSD to control weak convergence, and they prove that the *inverse multiquadric* kernel $k(x, x') = (c^2 + \|x - x'\|^2)^{-\beta}$ with $\beta \in (0, 1)$ does so for a broad class of targets, its heavier tail keeping the discrepancy honest. The practical lesson is that for goodness-of-fit on unbounded domains the inverse multiquadric is the safer default, and a vanishing Gaussian-kernel KSD is not by itself proof of fit.

The same Stein-kernel machinery has a second life beyond testing and sampling. If u_p is a kernel whose functions integrate to a known value against p , one can build control variates that cancel the variance of a Monte Carlo estimator: the control functionals of Oates, Girolami, and Chopin (2017) fit an RKHS function in the range of the Stein operator to an integrand, subtract it, and are left with an estimator of far lower variance, sometimes converging faster than the Monte Carlo rate. It is the same object read a third way. The Stein operator that annihilates p in expectation gives, at once, a discrepancy for testing, a transport direction for sampling, and a zero-mean correction for integration.

Connections and further reading

The score that drives the KSD is the same quantity estimated by score matching (Hyvärinen, 2005), and the population KSD with a translation-invariant kernel is a smoothed version of the Fisher divergence $\mathbb{E}_q \|s_q - s_p\|^2$; the Stein operator is what lets us evaluate it without ever forming s_q . Like the mean-embedding chapter, this material postdates the standard textbooks, so the primary sources are articles: the kernel Stein discrepancy and its goodness-of-fit test are due independently to Liu, Lee, and Jordan (2016) and Chwialkowski, Strathmann, and Gretton (2016), building on the computable Stein discrepancies of Gorham and Mackey (2015); the convergence-control theory and the case for the inverse multiquadric kernel are from Gorham and Mackey (2017); Stein variational gradient descent is from Liu and Wang (2016); and the control-functional variance reduction is from Oates, Girolami, and Chopin (2017). The survey of Muandet, Fukumizu, Sriperumbudur, and Schölkopf (2017) places all of this within the wider theory of kernel mean embeddings.

35.9 Summary

When a target density p is known only up to its normalizer, the score $s_p = \nabla \log p$ is the one handle that survives, because the constant Z disappears under the log-gradient. Stein’s identity turns the score into an operator $\mathcal{A}_p f = s_p^\top f + \nabla \cdot f$ whose expectation under p is zero, so its expectation under any other q measures the score gap $s_p - s_q$. Maximizing that expectation over the unit ball of a vector RKHS gives the kernel Stein discrepancy, the RKHS norm of a Stein-transformed witness, with the closed-form double expectation $\text{KSD}^2(q, p) = \mathbb{E}_{X, X' \sim q} [u_p(X, X')]$ of the Stein kernel u_p , itself positive definite and built from four terms in s_p and the derivatives of the base kernel.

Estimated by its U-statistic or V-statistic over a sample, the KSD drives a goodness-of-fit test for unnormalized models, calibrated by a wild bootstrap because the null is a degenerate χ^2 mixture. The same witness is the steepest-descent direction of the KL divergence, so following it moves particles toward p : Stein variational gradient descent balances a score-driven attraction toward high density against a kernel repulsion that prevents collapse, and samples an unnormalized posterior with no MCMC. Every one of these uses of the Stein operator, testing, sampling, and control-variate integration, reads the target only through its score.

35.10 Common mistakes and practical implications

For **Kernel Stein Discrepancy and Stein Methods**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

35.11 Exercises

1. warm-up Let $p(x) = \tilde{p}(x)/Z$ be a density on $\mathbb{R}^d$ with unknown normalizer Z . Show that the score $s_p = \nabla_x \log p$ equals $\nabla_x \log \tilde{p}$, so that it can be computed from the unnormalized model alone. Then explain in one sentence why the maximum mean discrepancy of the mean-embedding chapter cannot be used to test $q = p$ in this setting, while the kernel Stein discrepancy can.
2. computation Work on $\mathbb{R}$ with target $p = \mathcal{N}(0, 1)$, score $s_p(x) = -x$, and RBF kernel $k(x, x') = e^{-(x-x')^2/2}$, so the Stein kernel is $u_p(x, x') = [xx' + 1 - 2(x - x')^2] e^{-(x-x')^2/2}$. For the two-point sample $x_1 = 0, x_2 = 1$, write out the 2×2 Stein kernel matrix entry by entry, then evaluate the V-statistic $\frac{1}{4} \sum_{i,j} u_p(x_i, x_j)$ and the U-statistic $\frac{1}{2} \sum_{i \neq j} u_p(x_i, x_j)$. Hint

The diagonal entries are $u_p(0, 0) = 0^2 + 1 = 1$ and $u_p(1, 1) = 1^2 + 1 = 2$. The off-diagonal entry is $u_p(0, 1) = [0 + 1 - 2] e^{-1/2} = -e^{-1/2} = -0.6065$, which is both cross terms. The V-statistic is $(1 + 2 - 2 \cdot 0.6065)/4$; the U-statistic averages only the two off-diagonal entries.

3. proof Prove Stein's identity in one dimension: for p differentiable with score $s_p = (\log p)' = p'/p$ and f in the Stein class of p , show $\mathbb{E}_{X \sim p}[s_p(X)f(X) + f'(X)] = 0$. State exactly where the boundary condition $\lim_{|x| \rightarrow \infty} p(x)f(x) = 0$ enters. Hint

Multiply through by p inside the integral: $s_p f p = p' f$, so the integrand becomes $p' f + p f' = (p f)'$. The integral of a derivative is the boundary difference $[p f]_{-\infty}^{\infty}$, which the Stein-class condition sends to zero.

4. proof Starting from $\text{KSD}(q, p) = \sup_{\|f\|_{\mathcal{H}^d} \leq 1} \mathbb{E}_{X \sim q}[\mathcal{A}_p f(X)]$, use the reproducing identities $f_i(x) = \langle f_i, k(x, \cdot) \rangle$ and $\partial_{x_i} f_i(x) = \langle f_i, \partial_{x_i} k(x, \cdot) \rangle$ to write the functional as $\langle f, \xi_{q,p} \rangle_{\mathcal{H}^d}$, identify the witness $\xi_{q,p}$, and conclude by Cauchy-Schwarz that $\text{KSD}(q, p) = \|\xi_{q,p}\|_{\mathcal{H}^d}$. Name the two properties of the RKHS you use. Hint

You need the reproducing property (to convert function and derivative evaluations into inner products with $k(x, \cdot)$ and $\partial_{x_i} k(x, \cdot)$) and continuity of the inner product (to pull $\mathbb{E}_q$ inside it). Cauchy-Schwarz on $\langle f, \xi_{q,p} \rangle$ over $\|f\| \leq 1$ gives the supremum $\|\xi_{q,p}\|$, attained at $f = \xi_{q,p}/\|\xi_{q,p}\|$.

5. computation Derive the four-term Stein kernel for a general one-dimensional target and RBF kernel with bandwidth h , obtaining

$$u_p(x, x') = \left[s_p(x)s_p(x') + \frac{g}{h^2}(s_p(x) - s_p(x')) + \frac{1}{h^2} - \frac{g^2}{h^4} \right] e^{-g^2/(2h^2)}, \quad g = x - x'.$$

Then set $p = \mathcal{N}(0, 1)$ and $h = 1$ and confirm it collapses to $[xx' + 1 - 2(x - x')^2] e^{-(x-x')^2/2}$. Hint

Use $\partial_x k = -\frac{g}{h^2}k$, $\partial_{x'} k = \frac{g}{h^2}k$, and $\partial_x \partial_{x'} k = (\frac{1}{h^2} - \frac{g^2}{h^4})k$. For the collapse, $s_p(x) = -x$ makes the middle term $\frac{g}{h^2}(-x + x') = -g^2$, which combines with $-g^2$ from the last term to give $-2g^2$.

6. proof Show that the Stein kernel u_p is positive definite for any base kernel k and target p , so that both the V-statistic and any single Gram matrix $[u_p(x_i, x_j)]$ are non-negative in the quadratic-form sense. Deduce that $\widehat{\text{KSD}}_V^2 \geq 0$ always, and explain why $\widehat{\text{KSD}}_U^2$ can nonetheless be negative. Hint

Write $u_p(x, x') = \langle \beta_p(x, \cdot), \beta_p(x', \cdot) \rangle_{\mathcal{H}^d}$ with the Stein feature $\beta_p(x, \cdot) = s_p(x)k(x, \cdot) + \nabla_x k(x, \cdot)$; any Gram matrix of a feature map is positive semidefinite, so $\sum_{ij} c_i c_j u_p(x_i, x_j) = \|\sum_i c_i \beta_p(x_i, \cdot)\|^2 \geq 0$. The V-statistic takes $c_i = 1/n$; the U-statistic removes the diagonal, which is not a quadratic form and can go negative.

7. exploration In the SVGD update $\hat{\phi}(x_i) = \frac{1}{n} \sum_j [k(x_j, x_i)s_p(x_j) + \nabla_{x_j} k(x_j, x_i)]$, consider the two limits of the base kernel bandwidth h . Argue what happens to the particle cloud as $h \rightarrow \infty$ (the kernel is nearly constant) and as $h \rightarrow 0$ (the kernel is nearly a spike), and say which term, driving or repulsion, dominates in each limit and why a moderate h is needed for the particles to approximate p . Hint

As $h \rightarrow \infty$, $k \rightarrow 1$ and $\nabla k \rightarrow 0$: the repulsion vanishes and every particle follows the same averaged score, so the cloud collapses toward a mode. As $h \rightarrow 0$, $k(x_j, x_i) \rightarrow 0$ for $i \neq j$: each particle sees only itself, follows its own score to the nearest mode, and the particles no longer coordinate to fill out p .

8. challenge Two parts on the goodness-of-fit test. (a) Explain why, under $H_0 : q = p$, the rescaled statistic $n \widehat{\text{KSD}}_U^2$ has the non-Gaussian limit $\sum_j c_j (Z_j^2 - 1)$ rather than a normal law, and connect this to the degeneracy that makes the first-order term of the U-statistic vanish. (b) Suppose the wild bootstrap of the test is run with $B = 500$ sign-flip replicates and 12 of the bootstrap statistics T_b exceed the observed T . Compute the approximate p -value and state the decision at level $\alpha = 0.05$. Then argue that a target whose score is wrong only on a region the sample never visits could still yield a small statistic, and say what that means for the power of the test. Hint

For (a), under the null the leading linear part of the U-statistic has mean zero and vanishes, leaving a quadratic form in the Gaussian fluctuations of the sample, whose law is a weighted sum of centered χ_1^2 variables with the eigenvalues of u_p under p as weights. For (b), $\hat{p} \approx 12/500 = 0.024 \leq 0.05$, so reject H_0 ; the test only senses the score difference $s_p - s_q$ where q places mass, so a mismatch confined to an unvisited region contributes little and the test loses power there.

36 Causal Inference with Kernels

The earlier chapters of this part embedded a distribution as a point in an RKHS through its mean embedding, and a conditional distribution as an operator through its conditional mean embedding. Those embeddings turn statistical questions into linear algebra. This chapter spends that machinery on the hardest such questions: which variables cause which, and what would happen if we intervened. We do two things. First we build a nonparametric test of conditional independence, the engine that constraint-based algorithms use to reconstruct a causal graph from observational data. Second we estimate the effect of an intervention when hidden confounders make ordinary regression lie, using instruments and proxies lifted into an RKHS. Throughout, the kernel buys freedom from assumptions about functional form, linear or Gaussian or otherwise, but it cannot buy the assumptions that make a causal quantity identifiable in the first place. A running theme is exactly which of those assumptions the data can check and which it cannot.

36.1 From dependence to intervention

Every method so far in this part of the book measures how two distributions differ or how two variables depend. Dependence, though, is silent about direction and about intervention. If ice-cream sales and drowning deaths rise together, an embedding reports their dependence faithfully and says nothing about whether banning ice cream would save swimmers. The gap between the statement that X and Y are dependent and the statement that X causes Y is filled by two extra ingredients, and this chapter supplies a kernel version of each.

The first ingredient is structure. A dependence between X and Z can arise because X causes Z , because Z causes X , or because a common cause Y drives both. These are told apart, when they can be at all, by patterns of conditional independence: in the chain $X \rightarrow Y \rightarrow Z$ and in the fork $X \leftarrow Y \rightarrow Z$, the variables X and Z are dependent but become independent once Y is held fixed, whereas in the collider $X \rightarrow Y \leftarrow Z$ the reverse happens. Constraint-based causal discovery (Spirtes, Glymour, and Scheines 2000; Pearl 2009) turns a list of such conditional-independence facts into the set of graphs compatible with them. It needs a test that can certify $X \perp Z \mid Y$ without assuming the variables are Gaussian or the dependence linear, and the next section builds one.

The second ingredient is intervention, which we must define before we can estimate it.

Definition (intervention and structural function)

In a structural causal model each variable is produced by a deterministic mechanism from its direct causes and an independent noise term. The intervention $\text{do}(X = x)$ replaces the mechanism for X by the constant x , leaving every other mechanism intact, and induces the interventional distribution $P(Y \mid \text{do}(X = x))$. When $Y = f(X) + e$ with e the aggregate noise, the *structural function* $f(x) = \mathbb{E}[Y \mid \text{do}(X = x)]$ is the object of interest, and it need not equal the observational regression $\mathbb{E}[Y \mid X = x]$.

The inequality $f(x) \neq \mathbb{E}[Y \mid X = x]$ is the entire difficulty of effect estimation. It fails exactly when a confounder makes X and the noise e dependent, so that the regression reads off a mixture of the causal response and the confounder's shadow. Sections on treatment effects below recover f despite this, first through instruments and then through proxies.

36.2 Dependence as a distance between embeddings

Before conditioning, recall the unconditional measure of dependence from the mean-embedding chapter. Independence of (X, Y) is the statement $P_{XY} = P_X \otimes P_Y$, an equality of distributions, so any distance between embeddings is automatically a dependence measure. Feeding the joint and the product of the marginals into the maximum mean discrepancy gives the Hilbert-Schmidt Independence Criterion of Gretton, Bousquet, Smola, and Schölkopf (2005).

Definition (HSIC, recap)

For a kernel k on $\mathcal{X}$ and a kernel ℓ on $\mathcal{Y}$, the Hilbert-Schmidt Independence Criterion is the squared MMD between the joint and the product of the marginals, $\text{HSIC}(X, Y) = \text{MMD}^2(P_{XY}, P_X \otimes P_Y)$. Given N paired samples it is estimated by

$$\widehat{\text{HSIC}} = \frac{1}{N^2} \text{Tr}(KHLH), \quad H = I - \frac{1}{N} \mathbf{1}\mathbf{1}^\top,$$

with K, L the two Gram matrices and H the centering matrix. When k and ℓ are characteristic (Sriperumbudur et al. 2010), $\text{HSIC}(X, Y) = 0$ if and only if $X \perp Y$.

HSIC is the right tool for marginal independence, and it already powers feature selection and independent-component analysis. But causal discovery asks a harder question. To decide whether Y screens off X from Z we must remove from X and Z everything that Y explains and test only what remains, and that is not a marginal operation. The next section builds the conditional version.

36.3 Conditional independence in an RKHS

The natural RKHS notion of "everything that Y explains" is a regression of feature maps onto the features of Y ; the residuals of that regression carry the conditional dependence. Fukumizu, Gretton, Sun, and Schölkopf (2008) make this precise through covariance operators, the RKHS analogues of covariance matrices.

Definition (cross-covariance and conditional cross-covariance operators)

Let ϕ, ψ, v be the feature maps of X, Y, Z . The cross-covariance operator $\Sigma_{XY} : \mathcal{H}_Y \rightarrow \mathcal{H}_X$ is the centered second moment satisfying $\langle g, \Sigma_{XY} h \rangle_{\mathcal{H}_X} = \text{Cov}(g(X), h(Y))$ for all $g \in \mathcal{H}_X$ and $h \in \mathcal{H}_Y$. The conditional cross-covariance operator partials out Z :

$$\Sigma_{XY|Z} = \Sigma_{XY} - \Sigma_{XZ} \Sigma_{ZZ}^{-1} \Sigma_{ZY},$$

the exact operator analogue of the partial covariance of Gaussian theory.

The hope is that $\Sigma_{XY|Z} = 0$ captures conditional independence. It very nearly does, with one repair.

Theorem (kernel characterization of conditional independence; Fukumizu et al., 2008)

Write $\tilde{X} = (X, Z)$ for X augmented by the conditioning variable, with feature map $\tilde{\phi}$. Under characteristic kernels and the regularity needed for the operators to be well defined,

$$\Sigma_{\tilde{X}Y|Z} = 0 \iff X \perp Y \mid Z.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The augmentation $\tilde{X} = (X, Z)$ is not cosmetic. Without it the operator equation $\Sigma_{XY|Z} = 0$ states only that the *expected* conditional covariance $\mathbb{E}_Z[\text{Cov}(g(X), h(Y) \mid Z)]$ vanishes, a strictly weaker condition than conditional independence, since positive and negative conditional dependence at different values of Z can cancel in the average. Building Z into the first argument removes that loophole and upgrades the characterization to the full statement (Fukumizu et al. 2008).

The empirical statistic of Zhang, Peters, Janzing, and Schölkopf (2011), called KCIT, estimates $\Sigma_{\tilde{X}Y|Z}$ by kernel ridge regression. Center every Gram matrix, $\tilde{K} = HKH$. Regressing a feature onto the span of the instruments' features $\{v(z_i)\}$ with ridge ε predicts through the smoother $\tilde{K}_Z(\tilde{K}_Z + \varepsilon I)^{-1}$, so the residual is applied by the operator

$$R_Z = I - \tilde{K}_Z(\tilde{K}_Z + \varepsilon I)^{-1} = \varepsilon(\tilde{K}_Z + \varepsilon I)^{-1},$$

the second equality because $(\tilde{K}_Z + \varepsilon I) - \tilde{K}_Z = \varepsilon I$. The residualized matrices $\tilde{K}_{\tilde{X}|Z} = R_Z \tilde{K}_{\tilde{X}} R_Z$ and $\tilde{K}_{Y|Z} = R_Z \tilde{K}_Y R_Z$ hold the parts of $\tilde{X}$ and Y that Z does not explain, and their normalized inner product is the test statistic

$$T_{\text{CI}} = \frac{1}{n} \text{Tr}(\tilde{K}_{\tilde{X}|Z} \tilde{K}_{Y|Z}).$$

Under $H_0 : X \perp Y \mid Z$ the statistic T_{CI} has no fixed null law: it converges to a weighted sum of independent χ_1^2 variables whose weights are the products $\lambda_i \mu_j$ of the eigenvalues of the two residual operators (Zhang et al. 2011), the same spectral shape that governs the two-sample MMD null (Gretton et al. 2012). In practice one either matches a two-parameter Gamma distribution to the first two moments of that mixture or samples it directly from the empirical eigenvalues. A permutation test, the standard calibration for marginal HSIC, is *not* valid here: shuffling the pairing destroys the dependence on Z along with any conditional dependence, so it does not draw from the conditional null. That conditional independence admits no simple resampling scheme is one honest reason nonparametric CI testing is genuinely hard.

Algorithm (kernel conditional-independence test, KCIT)

Input Samples $\{(x_i, y_i, z_i)\}_{i=1}^n$; kernels for X, Y, Z ; ridge $\varepsilon > 0$.

Output Statistic T_{CI} and an approximate p -value for $H_0 : X \perp Y \mid Z$.

1. Form Gram matrices $K_{\tilde{X}}$ on the augmented points $\tilde{x}_i = (x_i, z_i)$, K_Y on y_i , and K_Z on z_i ; center each as $\tilde{K} = HKH$.
2. Build the residual-maker $R_Z = \varepsilon(\tilde{K}_Z + \varepsilon I)^{-1}$.
3. Residualize: $\tilde{K}_{\tilde{X}|Z} = R_Z \tilde{K}_{\tilde{X}} R_Z$ and $\tilde{K}_{Y|Z} = R_Z \tilde{K}_Y R_Z$.
4. Compute $T_{\text{CI}} = \frac{1}{n} \text{Tr}(\tilde{K}_{\tilde{X}|Z} \tilde{K}_{Y|Z})$.
5. Approximate the null by a Gamma fit to its first two moments, or by Monte Carlo from the eigenvalues $\{\lambda_i\}, \{\mu_j\}$; report $p = \Pr[\text{null} \geq T_{\text{CI}}]$.

Example (a conditional-independence test on a chain)

A tiny sample of $n = 8$ from the chain $X \rightarrow Y \rightarrow Z$, designed so $X \perp Z \mid Y$ holds exactly: the conditioner Y has two strata, and inside each stratum the (X, Z) pairs form a balanced 2×2 grid, so X and Z are independent given Y while both drift upward with Y .

row	1	2	3	4	5	6	7	8
X	0	0	1	1	2	2	3	3
Y	0	0	0	0	1	1	1	1
Z	0	1	0	1	2	3	2	3

Gaussian kernels with bandwidth set by the median heuristic, which equals 1 for each of X, Y, Z here; ridge $\varepsilon = 10^{-3}$.

1. Measure marginal dependence with HSIC. The biased estimator $\frac{1}{n^2} \text{Tr}(\tilde{K}_A \tilde{K}_B)$ gives $\text{HSIC}(X, Z) = 0.0844$, $\text{HSIC}(X, Y) = 0.0572$, and $\text{HSIC}(Y, Z) = 0.0572$. All three are clearly positive: every adjacent pair is dependent, and so are the endpoints X and Z .
2. Build the residual-maker for Y . Center K_Y and form $R_Y = \varepsilon(\tilde{K}_Y + \varepsilon I)^{-1}$, the operator that strips out whatever the features of Y can predict.
3. Residualize and take the trace. With $\tilde{X} = (X, Y)$, compute $\tilde{K}_{\tilde{X}|Y} = R_Y \tilde{K}_{\tilde{X}} R_Y$ and $\tilde{K}_{Z|Y} = R_Y \tilde{K}_Z R_Y$. Their trace statistic is $T_{\text{CI}}(X, Z | Y) = 0.000000$, a ratio of 1.3×10^{-12} to the marginal $\text{HSIC}(X, Z)$: the dependence between X and Z is entirely explained by Y .
4. Confirm the statistic is not vacuous. Had Z instead tracked X inside each stratum, breaking conditional independence, the identical computation returns $T_{\text{CI}} = 0.097$, so the test does discriminate.

Reading. The endpoints of a chain are marginally dependent but conditionally independent given the middle variable, and the kernel statistic reports exactly that, dropping from 0.0844 to numerical zero once Y is partialled out. Note the honest limit: the fork $X \leftarrow Y \rightarrow Z$ and the reversed chain $Z \rightarrow Y \rightarrow X$ imply the same conditional independence, so this test constrains the graph without orienting every edge.

Verification artifact. `checks/example-ch-causal-example-35-1.json` records the example source hash and verification scope.

36.4 Estimating effects under confounding

Turn now from discovery to estimation. When every confounder of X and Y is observed, the back-door adjustment reduces the causal effect to a regression, and the conditional mean embedding estimates it directly. The interesting and common case is a confounder we cannot observe, where regression is biased and we need extra leverage from the data. Two kinds of leverage admit clean kernel treatments: an instrument that perturbs the treatment, and a pair of proxies that shadow the hidden confounder.

36.4.1 Kernel instrumental variables

Write the structural model $Y = f(X) + e$, where the noise e absorbs the unobserved confounder and is therefore correlated with the treatment X . Because $\mathbb{E}[e | X] \neq 0$, the regression of Y on X , kernel ridge included, estimates f plus the confounder's imprint, not f . An instrument breaks the deadlock.

Definition (instrument)

A variable Z is an *instrument* for the effect of treatment X on outcome Y in the model $Y = f(X) + e$ if it is (i) *relevant*, meaning Z is dependent on X ; (ii) *exogenous*, meaning $\mathbb{E}[e | Z] = 0$, so Z is unconfounded with the outcome noise; and (iii) subject to the *exclusion* restriction, meaning Z influences Y only through X .

Exogeneity converts the unobservable structural equation into an observable one. Taking the conditional expectation of $Y = f(X) + e$ given $Z = z$ and using $\mathbb{E}[e | Z] = 0$ lands on the crux identity of the whole method.

Proposition (the two-stage target)

If $Y = f(X) + e$ with $f \in \mathcal{H}_X$ and $\mathbb{E}[e | Z] = 0$, then for every z ,

$$\mathbb{E}[Y | Z = z] = \langle f, \mu_{X|Z=z} \rangle_{\mathcal{H}_X}, \quad \mu_{X|Z=z} = \mathbb{E}[\phi(X) | Z = z].$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

By the reproducing property, $f(X) = \langle f, \phi(X) \rangle_{\mathcal{H}_X}$. Since f is a fixed element, the inner product against it is a continuous linear functional and commutes with the (Bochner) conditional expectation, so

$$\mathbb{E}[f(X) | Z = z] = \langle f, \mathbb{E}[\phi(X) | Z = z] \rangle_{\mathcal{H}_X} = \langle f, \mu_{X|Z=z} \rangle_{\mathcal{H}_X}.$$

Adding $\mathbb{E}[e | Z = z] = 0$ gives the claim. $\square$

The unknown f is thus pinned down by an equation relating two observable objects, the outcome regression on the left and the conditional mean embedding of the treatment on the right. Solving it for f is a Fredholm integral equation of the first kind, and it is ill-posed: the conditional-expectation operator $z \mapsto \mu_{X|Z=z}$ is compact, so its inverse is unbounded and amplifies sampling noise without limit (Newey and Powell 2003; Darolles et al. 2011). Regularization is not a convenience here, it is a necessity.

Singh, Sahani, and Gretton (2019) solve the equation in two ridge-regularized stages, the RKHS lift of classical two-stage least squares. Stage one estimates the conditional mean embedding $\mu_{X|Z=z}$ from a first sample $\{(x_i, z_i)\}_{i=1}^n$ by kernel ridge regression, giving $\hat{\mu}(z) = \sum_i \beta_i(z) \phi(x_i)$ with weights $\beta(z) = (K_Z + n\lambda I)^{-1} k_Z(\cdot, z)$. Stage two regresses the outcome on those embeddings, using a second sample $\{(y_j, \tilde{z}_j)\}_{j=1}^m$ to fit

$$\hat{f} = \arg \min_{f \in \mathcal{H}_X} \frac{1}{m} \sum_{j=1}^m (y_j - \langle f, \hat{\mu}(\tilde{z}_j) \rangle)^2 + \xi \|f\|_{\mathcal{H}_X}^2.$$

By the representer theorem $\hat{f} = \sum_i \alpha_i \phi(x_i)$, and collecting the stage-one weights into a matrix $B \in \mathbb{R}^{n \times m}$ with $B_{ij} = \beta_i(\tilde{z}_j)$, the prediction at $\tilde{z}_j$ is $\langle \hat{f}, \hat{\mu}(\tilde{z}_j) \rangle = (B^\top K_X \alpha)_j$. The stage-two problem is then a ridge regression whose inputs are the estimated embeddings, with Gram matrix $G = B^\top K_X B$, solved in closed form by $c = (G + m\xi I)^{-1} y$ and $\alpha = Bc$.

Algorithm (kernel instrumental-variable regression, KIV)

Input Stage-1 sample $\{(x_i, z_i)\}_{i=1}^n$, stage-2 sample $\{(y_j, \tilde{z}_j)\}_{j=1}^m$; kernels k_X, k_Z ; ridges $\lambda, \xi > 0$.

Output Structural function $\hat{f}(x) = \sum_i \alpha_i k_X(x_i, x)$.

1. Stage 1: form K_Z on the stage-1 instruments and $\Gamma_{ij} = k_Z(z_i, \tilde{z}_j)$; solve $B = (K_Z + n\lambda I)^{-1} \Gamma$ for the conditional-mean-embedding weights.
2. Form the stage-1 treatment Gram K_X and the stage-2 embedding Gram $G = B^\top K_X B$.
3. Stage 2: solve $c = (G + m\xi I)^{-1} y$ and set $\alpha = Bc$.
4. Return $\hat{f}(\cdot) = \sum_i \alpha_i k_X(x_i, \cdot)$; the effect of moving X from x to x' is $\hat{f}(x') - \hat{f}(x)$.

Example (recovering a structural slope with an instrument)

A confounded linear model, all variables centered, $n = 6$. Instrument $z = (-1, -1, -1, 1, 1, 1)$; hidden confounder $u = (-1, 1, 0, -1, 1, 0)$, chosen sample-uncorrelated with z ; treatment $x = z + u$; outcome $y = \beta x + \gamma u$ with structural slope $\beta = 2$ and confounding strength $\gamma = 3$. The sample covariances are $\widehat{\text{Cov}}(z, u) = 0$ (valid instrument) and $\widehat{\text{Cov}}(x, u) = 0.667$ (the treatment is confounded). Linear kernels $k_X(x, x') = xx'$, $k_Z(z, z') = zz'$, for which KIV reduces to two-stage least squares.

1. See the bias in plain regression. The OLS slope is $\hat{\beta}_{\text{OLS}} = \widehat{\text{Cov}}(x, y) / \widehat{\text{Var}}(x) = 3.2$, far from $\beta = 2$. The gap is the confounding term $\gamma \widehat{\text{Cov}}(x, u) / \widehat{\text{Var}}(x) = 3 \times 0.667 / 1.667 = 1.2$.
2. Stage 1: regress treatment on instrument. The slope is $\hat{a} = \widehat{\text{Cov}}(z, x) / \widehat{\text{Var}}(z) = 1.0$, giving fitted treatments $\hat{x} = \hat{a} z = z$. This $\hat{x}$ is the conditional mean embedding in the linear case, and it is purged of u because z is uncorrelated with u .
3. Stage 2: regress outcome on the fitted treatment. The slope is $\hat{\beta}_{\text{2SLS}} = \widehat{\text{Cov}}(\hat{x}, y) / \widehat{\text{Var}}(\hat{x}) = 2.0$, recovering the structural slope exactly.
4. Confirm with the full KIV formula. Running the matrix algorithm with linear-kernel Grams and ridges $\lambda = \xi = 10^{-6}$ yields $\hat{\beta}_{\text{KIV}} = 2.0$, agreeing with the two-stage computation.

Reading. The confounder biases ordinary regression by 1.2, from the truth 2 up to 3.2. Routing the fit through the instrument, which is clean of the confounder, restores the causal slope. The kernel version replaces the two scalar regressions by two ridge regressions in feature space, so the same logic recovers a nonlinear structural function f .

Verification artifact. checks/example-ch-causal-example-35-2.json records the example source hash and verification scope.

The two-sample split, stage one on one half and stage two on the other, is what keeps the estimator honest, preventing the stage-two fit from chasing the stage-one estimation error. An alternative avoids the split entirely: Muandet, Mehrjou, Lee, and Raj (2020) rewrite instrumental-variable regression as a single convex-concave saddle-point problem (DualIV), replacing the nested regression by a minimax objective that is often easier to optimize and to analyze.

36.4.2 Proximal causal learning

Instruments are not always available. Proximal causal learning asks a different favor of the data: instead of a variable that pokes the treatment, it uses two *proxies* of the unmeasured confounder U , a treatment-side proxy W and an outcome-side proxy Z , which are noisy views of U satisfying conditional-independence restrictions (Miao, Geng, and Tchetgen Tchetgen 2018). Identification runs through a *bridge function* h that solves the conditional moment equation

$$\mathbb{E}[Y - h(W, X) \mid Z, X] = 0,$$

after which the interventional mean is recovered by averaging out the proxy, $\mathbb{E}[Y \mid \text{do}(X = x)] = \mathbb{E}_W[h(W, x)]$. This moment equation has the same first-kind, ill-posed structure as the IV equation, and Mastouri et al. (2021) solve it with the same kernel two-stage regression and maximum-moment-restriction machinery, producing a proximal estimator that inherits the RKHS consistency theory. Proximal learning thus extends kernel causal estimation to settings where the confounder is entirely hidden and only its shadows are observed.

36.5 Beyond the mean: distributional and counterfactual effects

A structural function returns the mean outcome under intervention, but a mean can hide the story: a treatment that helps half the population and harms the other half has zero average effect. Because the entire apparatus is built on embeddings, nothing forces us to stop at the mean. Replacing the scalar outcome Y by its feature map $\phi_Y(Y)$ and running the same estimators embeds the whole interventional distribution as a mean embedding $\mu_{Y|\text{do}(x)} = \mathbb{E}[\phi_Y(Y) \mid \text{do}(X = x)]$, the counterfactual mean embedding of Muandet, Kanagawa, Saengkyongam, and Marukatat (2021). From it one reads any smooth functional of the interventional law, its variance, its quantiles, or the probability of exceeding a threshold, by the ordinary embedding calculus of the mean-embedding chapter. When the treatment itself is a distribution or a function rather than a point, the same lift connects to distribution regression (Szabó et al. 2016), closing the loop with the rest of this part of the book.

36.6 What kernels buy, and what they do not

It is worth stating plainly what the kernel has and has not done. In every method of this chapter the RKHS bought exactly one thing: freedom from parametric form. KCIT tests conditional independence without assuming linearity or Gaussianity; KIV and the proximal estimator recover a structural function without assuming it is a straight line. That freedom is substantive, because a linearity assumption quietly built into a causal estimate is itself an untestable causal claim, and the nonparametric test avoids the false negatives that a linear correlation test suffers when the dependence is real but curved.

Identification, however, is a separate layer, and the kernel is silent there.

The assumptions the data cannot check

Constraint-based discovery reads structure from conditional independences only under *faithfulness*, the assumption that the sole independences present are those the graph forces; and even then it returns a Markov equivalence class, not a unique graph, so the chain $X \rightarrow Y \rightarrow Z$ and the fork $X \leftarrow Y \rightarrow Z$ stay indistinguishable by conditional-independence tests alone (Spirtes et al. 2000; Pearl 2009). Instrumental-variable estimation is consistent only if the instrument is valid, yet exogeneity and exclusion are statements about the unobserved noise that no amount of data can verify (Angrist, Imbens, and Rubin 1996). Proximal learning trades those for completeness conditions on the proxies, equally uncheckable. A perfect kernel fit to a misidentified estimand is a precise answer to the wrong question. The honest summary is that kernels supply consistent, assumption-lean estimators of causal quantities *once those quantities are identified*, and identification always rests on domain assumptions imported from outside the sample.

36.7 Summary

Causal inference splits into two problems, and embeddings answer a piece of each. For discovery, independence is a distance between a joint embedding and a product of marginals (HSIC), and conditional independence is the vanishing of a conditional cross-covariance operator, estimated by the residual-trace statistic of KCIT; this is the nonparametric engine of constraint-based structure learning. For effect estimation under hidden confounding, exogeneity turns the structural equation into an ill-posed integral equation whose solution is the two-stage kernel instrumental-variable regression, with proximal learning covering the case where only proxies of the confounder are seen, and counterfactual mean embeddings lifting the answer from the mean effect to the whole interventional distribution. In every case the kernel removes assumptions about functional form but not the identifying assumptions, faithfulness, instrument validity, proxy completeness, that make the causal question answerable at all. The next chapter turns from these inferential uses of embeddings to learning when the training example is itself a distribution.

36.8 Common mistakes and practical implications

For **Causal Inference with Kernels**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

36.9 Summary and further reading

This chapter established explain the central definitions and claims in Causal Inference with Kernels; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Gretton and Schölkopf 2005), (Fukumizu and Schölkopf 2008), (Sriperumbudur and Lanckriet 2010).

36.10 Exercises

1. warm-up The worked example certified $X \perp Z \mid Y$ for data generated by the chain $X \rightarrow Y \rightarrow Z$. Show that the fork $X \leftarrow Y \rightarrow Z$ and the reversed chain $Z \rightarrow Y \rightarrow X$ imply the same single conditional independence and no other. Conclude that a conditional-independence test alone cannot orient the edges among these three variables, and name one extra source of information, from the modeling assumptions or the data-collection design, that would break the tie.
2. computation Verify the residual-maker identity $I - \tilde{K}_Z(\tilde{K}_Z + \varepsilon I)^{-1} = \varepsilon(\tilde{K}_Z + \varepsilon I)^{-1}$ used in KCIT. Then describe the two limits of R_Z : what operator does it approach as $\varepsilon \rightarrow 0$, and what as $\varepsilon \rightarrow \infty$, and explain why each extreme destroys the test, one by removing too much and the other by partialling out nothing.
3. proof Show that with linear kernels $k_X(x, x') = xx'$ and $k_Z(z, z') = zz'$ on centered scalar data, the KIV algorithm reduces to two-stage least squares, $\hat{\beta} = \widehat{\text{Cov}}(\hat{X}, Y) / \widehat{\text{Var}}(\hat{X})$ with $\hat{X}$ the least-squares projection of X on Z . Hint

The linear-kernel Gram $K_Z = zz^\top$ is rank one, so the stage-1 embedding $\hat{\mu}(z)$ is proportional to the fitted value $\hat{x} = \hat{a}z$; as $\lambda, \xi \rightarrow 0$ the stage-2 ridge regression on that scalar embedding is ordinary least squares of Y on $\hat{X}$. Compare with Worked Example 2, where these steps give 1.0 then 2.0.

4. computation For the confounded model of Worked Example 2, derive the omitted-variable bias of ordinary least squares, $\hat{\beta}_{\text{OLS}} \rightarrow \beta + \gamma \widehat{\text{Cov}}(X, U) / \widehat{\text{Var}}(X)$, starting from $Y = \beta X + \gamma U$. Check it against the sample: with $\gamma = 3$, $\widehat{\text{Cov}}(X, U) = 0.667$, and $\widehat{\text{Var}}(X) = 1.667$, confirm the bias is 1.2 and hence $\hat{\beta}_{\text{OLS}} = 3.2$.
5. exploration Re-run the check script for Worked Example 1 after replacing Z by values that track X inside each stratum of Y (so the chain becomes a structure in which Y no longer screens off X from Z). Confirm that the KCIT statistic jumps from numerical zero to 0.097, and explain in one sentence why a causal-discovery algorithm needs exactly this behavior, a near-zero statistic under independence and a clearly positive one otherwise.
6. proof Interpret the KCIT statistic as a partial dependence. Dropping the augmentation, show that $\frac{1}{n} \text{Tr}(\tilde{K}_{X|Z} \tilde{K}_{Y|Z})$ is, up to normalization, the HSIC between the RKHS residuals of X and of Y after each is regressed on Z , and relate this to the partial-covariance reading of $\Sigma_{XY|Z} = \Sigma_{XY} - \Sigma_{XZ} \Sigma_{ZZ}^{-1} \Sigma_{ZY}$. Hint

The residual-maker R_Z applied to a centered feature matrix returns the regression residual in feature space; the trace of the product of two such residual Grams is the empirical HSIC of the residuals. The subtracted term $\Sigma_{XZ} \Sigma_{ZZ}^{-1} \Sigma_{ZY}$ is precisely the part of the cross-covariance mediated by Z .

7. challenge Suppose the exclusion restriction fails, so the instrument has a direct effect: $Y = \beta X + \delta Z + \gamma U$ with $\delta \neq 0$, while stage 1 is $X = aZ + U$ with $\text{Cov}(Z, U) = 0$. Show that two-stage least squares now estimates $\beta + \delta/a$ rather than β , so a small direct effect masquerades as a change in the structural slope. Discuss why no test on the observed (X, Y, Z) can detect this bias. Hint

Project Y on the fitted $\hat{X} = aZ$: the stage-2 slope is $\text{Cov}(aZ, \beta X + \delta Z + \gamma U) / \text{Var}(aZ)$. Use $\text{Cov}(Z, U) = 0$ and $\text{Cov}(Z, X) = a \text{Var}(Z)$ to get $(a\beta + \delta)/a$. The extra term involves only the instrument's observed relations to X and Y , which are equally consistent with a valid but stronger or weaker instrument.

37 Distribution Regression and Functional Data

Every chapter so far has fed the kernel a point: a vector, a string, a graph, some single object x . This chapter feeds it a whole probability distribution. The training example is no longer a vector but a distribution P , and we never see P directly, only a bag of samples drawn from it, with one label attached to the bag rather than to any single sample. The task is to learn a map from distributions to labels from a handful of such bags. The strategy is two stages, and it reuses everything from the mean-embedding chapter: first summarize each input distribution by its kernel mean embedding, a single point in an RKHS, then run kernel ridge regression on those points using a kernel built on the embeddings, a kernel of kernels. Two facts make this delicate and worth a chapter. The embedding kernel has to be positive definite in its own right, which we prove, and the estimator now carries two independent sources of error, one from having finitely many bags and one from each bag being finite. We build the estimator, decompose its risk along those two axes, and state the consistency theorem of Szabó, Sriperumbudur, Póczos, and Gretton (2016). The same machinery recovers set kernels and support measure machines, and it connects to functional data analysis, where the input is a function rather than a distribution.

37.1 When the input is a distribution

Ordinary supervised learning pairs a feature vector with a label. But a great deal of data does not arrive as one vector per example. A pathologist sees not a single cell but a slide of thousands, and the diagnosis labels the slide. An astronomer measures a cloud of photons and wants the redshift of the source that emitted them. A social scientist has a survey of many households in a county and wants to predict the county's median income or election result. In each case the natural input is a bag of points, and the label is attached to the whole bag, not to its members. Treating the bag as an unordered set, the object it represents is a probability distribution: the empirical distribution of its samples, standing in for the population the samples came from.

So we want to learn a function f whose argument is a probability distribution P and whose value $f(P)$ is a real label. The obstacle is that a distribution is an infinite-dimensional object and we never observe it exactly. We observe a finite bag $\{x^1, \dots, x^N\}$ of samples from P , and even the number of bags we are given is finite. Two approximations are therefore baked in from the start, and keeping them separate is the whole art of the subject. The first idea, and the one that organizes the chapter, is to compress each distribution into a single point of an RKHS using its mean embedding, turning the exotic input space of distributions into an ordinary Hilbert space on which we already know how to regress.

37.1.1 The two-stage sampled model

We make the sampling structure precise, because the guarantees depend on it. There is a meta-distribution, a distribution over distributions, from which the inputs are drawn.

Definition (distribution regression problem)

Let $\mathcal{X}$ be a space with a positive definite base kernel k , and let $\mathcal{P}$ be a set of Borel probability measures on $\mathcal{X}$. A *meta-distribution* $\mathfrak{M}$ generates labelled distributions (P, y) with $P \in \mathcal{P}$ and $y \in \mathbb{R}$. We do not observe the pairs (P_i, y_i) ; instead we observe, for $i = 1, \dots, \ell$, a bag of i.i.d. samples

$$X_i = \{x_i^1, \dots, x_i^{N_i}\}, \quad x_i^a \stackrel{\text{i.i.d.}}{\sim} P_i,$$

together with the label y_i . The goal is to learn a predictor $f : \mathcal{P} \rightarrow \mathbb{R}$ minimizing the risk $\mathcal{R}(f) = \mathbb{E}_{(P,y) \sim \mathfrak{M}}[(f(P) - y)^2]$.

The two integers ℓ and N_i name the two error sources. The number of bags ℓ is the ordinary sample size of a regression: with few bags we cannot pin down f , and this error vanishes only as $\ell \rightarrow \infty$. The bag size N_i is new: even with infinitely many bags, if each bag holds a handful of points we see each input distribution only blurrily, and this error vanishes only as $N_i \rightarrow \infty$. A one-stage regression has only the first source. Distribution regression has both, and its theory is the accounting that keeps them apart. From here on we write N for a common bag size to keep the notation light.

37.2 Representing each input by its mean embedding

The first stage borrows the central object of the mean-embedding chapter wholesale. Recall that a base kernel k on $\mathcal{X}$, with RKHS $\mathcal{H}$ and feature map $x \mapsto K_x = k(x, \cdot)$, embeds a distribution P as the average feature

$$\mu_P = \mathbb{E}_{X \sim P}[K_X] \in \mathcal{H}, \quad \mu_P(\cdot) = \mathbb{E}_{X \sim P}[k(X, \cdot)].$$

This single vector stores the expectation under P of every RKHS function, through the generalized kernel trick $\mathbb{E}_{X \sim P}[g(X)] = \langle g, \mu_P \rangle_{\mathcal{H}}$, and when k is characteristic the map $P \mapsto \mu_P$ is injective, so no information about P is lost. The embedding is exactly the linearizing device we need: it replaces each unwieldy input distribution P_i by a point μ_{P_i} in the fixed Hilbert space $\mathcal{H}$.

We do not have P_i , only its bag, so we use the empirical embedding, the mean embedding of the empirical distribution $\hat{P}_i = \frac{1}{N} \sum_a \delta_{x_i^a}$,

$$\hat{\mu}_i := \mu_{\hat{P}_i} = \frac{1}{N} \sum_{a=1}^N K_{x_i^a} = \frac{1}{N} \sum_{a=1}^N k(x_i^a, \cdot) \in \mathcal{H}.$$

This is nothing but the average of one feature per sample, the same kernel-smoothed fingerprint of the bag met before. Stage one, then, turns each bag into a point $\hat{\mu}_i \in \mathcal{H}$, and the gap $\|\hat{\mu}_i - \mu_{P_i}\|_{\mathcal{H}}$ between this point and the true one is precisely the finite-bag error, which we will bound in a moment.

37.3 A kernel on distributions: the kernel of kernels

Now that every input is a point μ_P in $\mathcal{H}$, stage two is a kernel method on those points, and it needs a kernel between them. This is a kernel whose two arguments are themselves distributions, evaluated through their embeddings, a kernel of kernels. Two choices span the range.

Definition (embedding kernels on distributions)

Let $\mu_P \in \mathcal{H}$ be the mean embedding of P under the base kernel k . The *linear embedding kernel* is the RKHS inner product of embeddings,

$$\mathcal{K}_{\text{lin}}(P, Q) = \langle \mu_P, \mu_Q \rangle_{\mathcal{H}} = \mathbb{E}_{X \sim P, Y \sim Q}[k(X, Y)].$$

For a scale $\gamma > 0$, the *Gaussian-on-MMD kernel* is a Gaussian in the RKHS distance between embeddings,

$$\mathcal{K}_{\gamma}(P, Q) = \exp\left(-\frac{\|\mu_P - \mu_Q\|_{\mathcal{H}}^2}{2\gamma^2}\right) = \exp\left(-\frac{\text{MMD}^2(P, Q)}{2\gamma^2}\right),$$

since $\|\mu_P - \mu_Q\|_{\mathcal{H}}$ is exactly the maximum mean discrepancy.

The linear kernel measures raw overlap of the smoothed densities and is a perfectly good similarity, but it can only ever see P through the single vector μ_P , so as a second-stage kernel it fits functions that are linear in the embedding. The Gaussian-on-MMD kernel restores nonlinearity: it turns the MMD, our metric on distributions, into a bump-shaped similarity, large when two distributions nearly coincide and decaying as they separate. Both are genuine kernels, which needs an argument, because positive definiteness on the strange domain of distributions is not automatic.

Proposition (the embedding kernels are positive definite)

For any base kernel k and any $\gamma > 0$, both $\mathcal{K}_{\text{lin}}$ and $\mathcal{K}_\gamma$ are positive definite kernels on $\mathcal{P}$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The map $\Phi : P \mapsto \mu_P$ is a feature map from $\mathcal{P}$ into the Hilbert space $\mathcal{H}$. For $\mathcal{K}_{\text{lin}}$, given distributions $P_1, \dots, P_m$ and scalars $c_1, \dots, c_m$,

$$\sum_{a,b} c_a c_b \langle \mu_{P_a}, \mu_{P_b} \rangle_{\mathcal{H}} = \left\| \sum_a c_a \mu_{P_a} \right\|_{\mathcal{H}}^2 \geq 0,$$

so $\mathcal{K}_{\text{lin}}$ is positive definite, being an inner product of feature vectors. For $\mathcal{K}_\gamma$, expand the squared distance as $\|\mu_P - \mu_Q\|^2 = \|\mu_P\|^2 + \|\mu_Q\|^2 - 2\langle \mu_P, \mu_Q \rangle$ and factor the exponential,

$$\mathcal{K}_\gamma(P, Q) = \underbrace{e^{-\|\mu_P\|^2/2\gamma^2} e^{-\|\mu_Q\|^2/2\gamma^2}}_{g(P)g(Q)} \cdot \exp\left(\frac{1}{\gamma^2} \langle \mu_P, \mu_Q \rangle\right).$$

The first factor is $g(P)g(Q)$ with $g(P) = e^{-\|\mu_P\|^2/2\gamma^2}$, a rank-one and hence positive definite kernel. The second is the exponential of $\frac{1}{\gamma^2} \mathcal{K}_{\text{lin}}$, which is positive definite because $\exp(t \mathcal{K}_{\text{lin}}) = \sum_{n \geq 0} \frac{t^n}{n!} \mathcal{K}_{\text{lin}}^n$ is a limit of nonnegative combinations of Schur powers of the positive definite kernel $\mathcal{K}_{\text{lin}}$, each power positive definite by the Schur product theorem. A pointwise product of positive definite kernels is positive definite, so the product of the two factors is $\mathcal{K}_\gamma$. Christmann and Steinwart (2010) and Muandet, Fukumizu, Sriperumbudur, and Schölkopf (2017) develop these embedding kernels in full. $\square$

The only property of $\mathcal{H}$ the proof used is that it is an inner product space, so the argument is the exact replay of "the Gaussian kernel on $\mathbb{R}^d$ is positive definite," now run in the embedding space. In practice we never form μ_P explicitly; every entry of the second-stage Gram matrix is computed from bag samples through the base kernel, which is the content of the next procedure.

Algorithm (kernel-on-embeddings Gram matrix from bags)

Input Bags $X_1, \dots, X_\ell$ with $X_i = \{x_i^1, \dots, x_i^N\}$; base kernel k ; scale γ .

Output Positive semidefinite matrix $\mathbf{K} \in \mathbb{R}^{\ell \times \ell}$, $\mathbf{K}_{ij} = \mathcal{K}_\gamma(P_i, P_j)$.

1. For each pair (i, j) , form the mean cross base-kernel $B_{ij} = \frac{1}{N^2} \sum_{a,b} k(x_i^a, x_j^b) = \langle \hat{\mu}_i, \hat{\mu}_j \rangle_{\mathcal{H}}$.
2. Form the squared empirical MMD $D_{ij} = B_{ii} + B_{jj} - 2B_{ij} = \|\hat{\mu}_i - \hat{\mu}_j\|_{\mathcal{H}}^2$.
3. Set $\mathbf{K}_{ij} = \exp(-D_{ij}/(2\gamma^2))$; for the linear kernel instead set $\mathbf{K}_{ij} = B_{ij}$.

The construction is nothing more than: average the base kernel across every pair of bags to get inner products, read off distances, and pass them through a Gaussian. The worked example carries three tiny bags plus a fourth all the way to the matrix.

Example (empirical embeddings and the Gaussian-on-MMD Gram matrix)

Four bags of three points each on $\mathbb{R}$, with clean generating means 0, 1, 2, 3 and varied spreads:

$$X_1 = \{-0.4, 0.1, 0.3\}, \quad X_2 = \{0.7, 1.0, 1.3\}, \quad X_3 = \{1.5, 2.1, 2.4\}, \quad X_4 = \{2.7, 3.0, 3.3\}.$$

Base kernel Gaussian with bandwidth $\sigma = 1$, that is $k(a, b) = e^{-(a-b)^2/2}$; embedding scale $\gamma = 1$.

1. Average the base kernel over each pair of bags. The matrix of embedding inner products $B_{ij} = \langle \hat{\mu}_i, \hat{\mu}_j \rangle$, each an average of 9 base-kernel values, is

$$B = \begin{pmatrix} 0.9212 & 0.6065 & 0.1783 & 0.0179 \\ 0.6065 & 0.9438 & 0.5993 & 0.1588 \\ 0.1783 & 0.5993 & 0.8796 & 0.6063 \\ 0.0179 & 0.1588 & 0.6063 & 0.9438 \end{pmatrix},$$

with self-similarities $\text{diag}(B) = (0.9212, 0.9438, 0.8796, 0.9438)$ measuring how tightly each bag is concentrated.

2. Read off the squared MMD distances. Using $D_{ij} = B_{ii} + B_{jj} - 2B_{ij}$,

$$D = \begin{pmatrix} 0 & 0.6520 & 1.4442 & 1.8293 \\ 0.6520 & 0 & 0.6249 & 1.5701 \\ 1.4442 & 0.6249 & 0 & 0.6109 \\ 1.8293 & 1.5701 & 0.6109 & 0 \end{pmatrix}.$$

Adjacent bags sit at squared distance about 0.62, the far corners at 1.83: the MMD grows with the gap between the generating means.

3. Pass distances through the Gaussian. With $\mathbf{K}_{ij} = e^{-D_{ij}/2}$,

$$\mathbf{K} = \begin{pmatrix} 1 & 0.7218 & 0.4857 & 0.4006 \\ 0.7218 & 1 & 0.7317 & 0.4561 \\ 0.4857 & 0.7317 & 1 & 0.7368 \\ 0.4006 & 0.4561 & 0.7368 & 1 \end{pmatrix}.$$

4. Confirm it is a valid kernel matrix. The eigenvalues of $\mathbf{K}$ are (0.1262, 0.3549, 0.7411, 2.7778), all strictly positive, so $\mathbf{K}$ is positive definite, exactly as the proposition promised.

Reading. Stage one has turned four bags of numbers into four points whose pairwise similarities form the matrix $\mathbf{K}$. The kernel behaves geometrically: it is 1 on the diagonal, decays monotonically as the generating means pull apart (from 0.7218 for neighbours to 0.4006 for the extremes), and its positive spectrum certifies that any downstream kernel machine may use it. Every number here came from bag samples through the base kernel alone; the embeddings $\hat{\mu}_i$ were never formed.

Verification artifact. checks/example-ch-distreg-example-36-1.json records the example source hash and verification scope.

37.4 The two-stage estimator

With a positive definite kernel on distributions in hand, the second stage is ordinary kernel ridge regression, run on the embedded inputs. This is the whole estimator: embed, then ridge.

Algorithm (two-stage distribution-regression estimator)

Input Labelled bags $(X_i, y_i)_{i=1}^\ell$; base kernel k ; embedding kernel $\mathcal{K}_\gamma$; ridge $\lambda > 0$.

Output Predictor $\hat{f}(P_*)$ for a new bag X_* .

1. Stage 1. Represent each bag by its empirical embedding $\hat{\mu}_i = \frac{1}{N} \sum_a k(x_i^a, \cdot)$, implicitly through k .
2. Build the $\ell \times \ell$ Gram matrix $\mathbf{K}_{ij} = \mathcal{K}_\gamma(P_i, P_j)$ by the previous algorithm.
3. Stage 2. Solve the ridge system $\alpha = (\mathbf{K} + \ell\lambda I)^{-1}\mathbf{y}$ for the dual weights.
4. For a new bag X_* , form the vector $(\mathbf{k}_*)_i = \mathcal{K}_\gamma(P_i, P_*)$ and return $\hat{f}(P_*) = \mathbf{k}_*^\top \alpha$.

The predictor is the familiar weighted sum of kernel similarities, $\hat{f}(P_*) = \sum_i \alpha_i \mathcal{K}_\gamma(P_i, P_*)$, only with the similarity now computed between bags. Nothing about the ridge solve knows it is acting on distributions; the distributional character lives entirely in how $\mathbf{K}$ and $\mathbf{k}_*$ were built. We complete the running example by attaching labels and predicting a held-out bag.

Example (predicting a label for a held-out bag)

The four bags of the previous example, with the label of each bag its generating mean. Train on bags 1, 2, 4 with labels $\mathbf{y} = (0, 1, 3)$; hold out bag 3, whose true label is 2.0. Reuse $\mathcal{K}_\gamma$ with $\gamma = 1$, and ridge $\lambda = 0.05$, so $\ell\lambda = 3(0.05) = 0.15$.

1. Extract the training Gram and the test column. From $\mathbf{K}$, the training block on $\{1, 2, 4\}$ and the similarities of the held-out bag to the training bags are

$$\mathbf{K}_{\text{tr}} = \begin{pmatrix} 1 & 0.7218 & 0.4006 \\ 0.7218 & 1 & 0.4561 \\ 0.4006 & 0.4561 & 1 \end{pmatrix}, \quad \mathbf{k}_* = \begin{pmatrix} 0.4857 \\ 0.7317 \\ 0.7368 \end{pmatrix}.$$

2. Solve the ridge system. With $\ell\lambda = 0.15$, $\alpha = (\mathbf{K}_{\text{tr}} + 0.15 I)^{-1}\mathbf{y} = (-1.3679, 0.5987, 2.8478)$.
3. Predict the held-out label. $\hat{f}(P_3) = \mathbf{k}_*^\top \alpha = 1.8719$, against the true value 2.0.
4. Compare to the naive baseline. Predicting the constant mean of the training labels, $(0 + 1 + 3)/3 = 1.3333$, has absolute error 0.6667; the two-stage prediction has absolute error 0.1281, five times smaller.

Reading. From three labelled bags the estimator predicts the held-out label as 1.87, close to the truth 2.0. It falls a little short for two reasons that are exactly the two error sources of the model: the ridge penalty shrinks the fit toward zero (a stage-two effect that shrinks with more bags and smaller λ), and each bag has only three points, so the embedding $\hat{\mu}_3$ is a noisy stand-in for μ_{P_3} (a stage-one effect that shrinks as the bags grow). The naive baseline, blind to which distribution each bag came from, does far worse.

Verification artifact. checks/example-ch-distreg-example-36-2.json records the example source hash and verification scope.

37.5 Consistency and the two error sources

The example's two apologies, ridge bias and tiny bags, are the whole story of the theory. We now make them quantitative. The clean way is to compare the estimator we can compute, which uses the empirical embeddings $\hat{\mu}_i$, against an oracle estimator that uses the true embeddings μ_{P_i} , and then against the target itself. Two short lemmas supply the pieces.

Lemma (finite-bag embedding error)

If the base kernel is bounded, $\sup_x k(x, x) \leq \kappa$, then for a bag of N i.i.d. draws from P ,

$$\mathbb{E} \left\| \hat{\mu}_P - \mu_P \right\|_{\mathcal{H}}^2 \leq \frac{\kappa}{N}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Write $\hat{\mu}_P - \mu_P = \frac{1}{N} \sum_{a=1}^N (K_{x^a} - \mu_P)$, a mean of N i.i.d. terms with mean zero in $\mathcal{H}$. By independence the variance of the mean is the single-term variance over N ,

$$\mathbb{E} \left\| \hat{\mu}_P - \mu_P \right\|^2 = \frac{1}{N} \mathbb{E} \left\| K_X - \mu_P \right\|^2 = \frac{1}{N} (\mathbb{E}[k(X, X)] - \|\mu_P\|^2) \leq \frac{\mathbb{E}[k(X, X)]}{N} \leq \frac{\kappa}{N}.$$

Hence $\left\| \hat{\mu}_P - \mu_P \right\|_{\mathcal{H}} = O_P(N^{-1/2})$. $\square$

So the stage-one error per bag decays like $N^{-1/2}$. It only helps if the second-stage kernel does not amplify it, which is what the next lemma guarantees: the Gaussian-on-MMD feature map is Lipschitz in the embedding, so a small perturbation of μ_P moves the second-stage feature by a controlled amount.

Lemma (the distribution kernel is Lipschitz in the embedding)

Let $\Psi(P) = \mathcal{K}_\gamma(P, \cdot) \in \mathcal{H}_{\mathcal{X}}$ be the canonical feature of the Gaussian-on-MMD kernel. Then

$$\left\| \Psi(P) - \Psi(P') \right\|_{\mathcal{H}_{\mathcal{X}}} \leq \frac{1}{\gamma} \left\| \mu_P - \mu_{P'} \right\|_{\mathcal{H}}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Since $\mathcal{K}_\gamma(P, P) = 1$, the reproducing property gives

$$\left\| \Psi(P) - \Psi(P') \right\|_{\mathcal{H}_{\mathcal{X}}}^2 = \mathcal{K}_\gamma(P, P) + \mathcal{K}_\gamma(P', P') - 2\mathcal{K}_\gamma(P, P') = 2(1 - e^{-\|\mu_P - \mu_{P'}\|^2 / 2\gamma^2}).$$

Apply the elementary bound $1 - e^{-u} \leq u$ for $u \geq 0$ with $u = \|\mu_P - \mu_{P'}\|^2 / (2\gamma^2)$:

$$2(1 - e^{-\|\mu_P - \mu_{P'}\|^2 / 2\gamma^2}) \leq 2 \cdot \frac{\|\mu_P - \mu_{P'}\|^2}{2\gamma^2} = \frac{\|\mu_P - \mu_{P'}\|^2}{\gamma^2}.$$

Taking square roots gives the claim. $\square$

The two lemmas chain: replacing each true embedding by its bag estimate perturbs each second-stage feature by at most $\frac{1}{\gamma} \|\hat{\mu}_i - \mu_{P_i}\| = O_P(N^{-1/2})$, and kernel ridge regression is stable to such feature perturbations. This is the mechanism behind the risk decomposition.

Proposition (two-stage risk decomposition)

Let $\hat{f}$ be the two-stage estimator built from the empirical embeddings, $\tilde{f}$ the oracle ridge estimator built from the same ℓ bags but with the true embeddings μ_{P_i} , and f_λ the population ridge solution. Then the excess risk splits as

$$\underbrace{\mathcal{R}(\hat{f}) - \mathcal{R}(f_\rho)}_{\text{total}} \lesssim \underbrace{\|\hat{f} - \tilde{f}\|^2}_{\text{stage 1: finite bags}} + \underbrace{\|\tilde{f} - f_\lambda\|^2}_{\text{stage 2: finite bags count } \ell} + \underbrace{\|f_\lambda - f_\rho\|^2}_{\text{approximation}}$$

where the stage-one term is $O_P(\lambda^{-2}N^{-1})$ by the two lemmas, the stage-two estimation term vanishes as $\ell \rightarrow \infty$, and the approximation term vanishes as $\lambda \rightarrow 0$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The triangle inequality for the risk, written through the RKHS norm that dominates it, gives $\|\hat{f} - f_\rho\| \leq \|\hat{f} - \tilde{f}\| + \|\tilde{f} - f_\lambda\| + \|f_\lambda - f_\rho\|$, which squares to the three displayed terms up to a constant. The middle and right terms are the standard bias-variance split of kernel ridge regression with ℓ exact examples: the estimation term $\|\tilde{f} - f_\lambda\|$ is the variance of a ridge fit from ℓ samples and tends to 0 as $\ell \rightarrow \infty$ for fixed λ , while the approximation term $\|f_\lambda - f_\rho\|$ is the ridge bias and tends to 0 as $\lambda \rightarrow 0$ when f_ρ lies in the closure of $\mathcal{H}_{\mathcal{K}}$. For the first term, $\hat{f}$ and $\tilde{f}$ are the same ridge map applied to feature vectors $\Psi(\hat{P}_i)$ and $\Psi(P_i)$; the ridge solution is Lipschitz in its inputs with constant $O(\lambda^{-1})$, and by the Lipschitz lemma each input differs by $\frac{1}{\gamma}\|\hat{\mu}_i - \mu_{P_i}\|$, whose square has expectation $O(N^{-1})$ by the finite-bag lemma. Hence $\|\hat{f} - \tilde{f}\|^2 = O_P(\lambda^{-2}N^{-1})$. $\square$

The decomposition already tells the qualitative story: drive $\lambda \rightarrow 0$ to kill the bias, take $\ell \rightarrow \infty$ to kill the estimation variance, and take $N \rightarrow \infty$ fast enough that $\lambda^{-2}N^{-1} \rightarrow 0$ despite the shrinking λ . The bag size must grow with the number of bags, but not arbitrarily fast. Szabó, Sriperumbudur, Póczos, and Gretton (2016) turned this heuristic into the first sharp analysis, under the standard smoothness (source) and effective-dimension (capacity) conditions of the rates chapter on the second-stage regression.

Theorem (consistency and optimal rates, Szabó, Sriperumbudur, Póczos, Gretton 2016)

Under source and capacity conditions on the second-stage kernel ridge problem and mild regularity of the embedding kernel, the two-stage estimator $\hat{f}$ with an appropriately chosen $\lambda = \lambda(\ell) \rightarrow 0$ is consistent, $\mathcal{R}(\hat{f}) - \mathcal{R}(f_\rho) \rightarrow 0$ in probability as $\ell \rightarrow \infty$, provided the bag size $N = N(\ell)$ grows at least polynomially in ℓ . Moreover, for a sufficient bag-growth rate the estimator attains the same minimax-optimal rate it would achieve if the true embeddings μ_{P_i} were observed: the finite-bag stage costs nothing asymptotically once the bags are large enough.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The headline is worth stating plainly. Distribution regression is consistent, and the price of never seeing the true input distributions, only bags of samples, is asymptotically nil as long as the bags grow with the number of bags. The delicate point their analysis settled is that both ℓ and N must diverge and that the required growth of N can be characterized; finite bags do not preclude learning, they only set how fast λ may shrink.

37.6 Set kernels and support measure machines

The two-stage recipe did not appear from nothing; its linear case predates the general theory and is worth naming. Take the linear embedding kernel and its empirical form: for two bags A and B ,

$$\mathcal{K}_{\text{lin}}(A, B) = \langle \hat{\mu}_A, \hat{\mu}_B \rangle_{\mathcal{H}} = \frac{1}{|A| |B|} \sum_{a \in A} \sum_{b \in B} k(a, b).$$

This is the *set kernel*, the average base kernel over all cross pairs, a special case of the convolution kernels for structured objects of Haussler (1999). It compares two bags without any distributional language at all, and plugging it into a support vector machine gives a classifier of sets. Muandet, Fukumizu, Dinuzzo, and Schölkopf (2012) generalized this from the linear kernel to the full nonlinear machinery, running kernel machines with the embedding kernels $\mathcal{K}_\gamma$ directly on probability measures.

Support measure machines

A *support measure machine* (Muandet et al. 2012) is a large-margin classifier whose training examples are probability measures $P_1, \dots, P_\ell$, each with a label, fitted by the SVM dual on the Gram matrix $\mathbf{K}_{ij} = \mathcal{K}_\gamma(P_i, P_j)$. It is exactly the SVM of the earlier chapters with points replaced by embeddings, so the representer theorem, the dual quadratic program, and the kernelization all carry over unchanged. When only samples are available, each $\mathcal{K}_\gamma(P_i, P_j)$ is estimated from the two bags as in the Gram-matrix algorithm, making the support measure machine the classification twin of the two-stage regressor.

For any of this to learn an arbitrary target, the embedding kernel must be rich enough on the space of measures, the distributional analogue of a characteristic or universal kernel. Christmann and Steinwart (2010) supplied the guarantee: Gaussian-type kernels built on the embeddings are universal on the space of probability measures, so kernel machines using them can approximate any continuous functional of the distribution, and are therefore consistent for classification and regression alike. A complementary line avoids the embedding entirely: Póczos, Rinaldo, Singh, and Wasserman (2013) estimate functionals of the input density directly, by nonparametric density or nearest-neighbour methods, giving a distribution-free route to distribution regression with its own consistency guarantees. The embedding approach and the density approach are the two poles of the field, trading the RKHS's smoothing for direct estimation.

37.7 Functional data: when the input is a function

A close cousin of a distribution-valued input is a function-valued one. In functional data analysis (Ramsay and Silverman 2005) each example is itself a function: a growth curve $x_i(t)$ of height against age, a spectrum of absorbance against wavelength, a daily temperature profile. As with bags, we never see the whole function, only its values on a finite grid, so the same finite-resolution error reappears, now as a finite grid rather than a finite bag. The objects live in L^2 , the space of square-integrable functions, with the inner product $\langle x, x' \rangle_{L^2} = \int x(t)x'(t) dt$.

Definition (functional linear model)

The functional linear regression model predicts a scalar label from a functional input $x \in L^2$ through

$$y = \alpha + \int x(t) \beta(t) dt + \varepsilon = \alpha + \langle x, \beta \rangle_{L^2} + \varepsilon,$$

with intercept $\alpha \in \mathbb{R}$ and coefficient function $\beta \in L^2$ to be estimated.

This is ordinary linear regression carried out in L^2 , the coefficient vector replaced by a coefficient function. To move beyond linearity, place a kernel on L^2 directly, for instance the Gaussian $\mathcal{K}(x, x') = \exp(-\|x - x'\|_{L^2}^2/2\gamma^2)$, and run kernel ridge regression, exactly stage two of the distribution-regression recipe with L^2 distance in place of the MMD. The two settings are in fact one. A mean embedding μ_P is an element of the RKHS $\mathcal{H}$, that is, a function, so distribution regression is functional regression on the embedding functions, and the MMD is the $\mathcal{H}$ -norm distance between them. Christmann and Steinwart's universality applies here too, so nonparametric functional regression with a Gaussian kernel on L^2 is consistent under the same kind of conditions. The distinctions that remain are practical: functional data are often smooth and are pre-smoothed by splines or a functional principal

component basis before the kernel sees them, whereas bags are rough empirical measures whose only smoothing is the base kernel itself.

37.8 Summary

When each training example is a probability distribution seen only through a bag of samples, learning proceeds in two stages. Stage one embeds every input distribution as a single RKHS point, its kernel mean embedding, estimated from the bag by the empirical embedding. Stage two runs kernel ridge regression on those points using a kernel on distributions, either the linear embedding kernel $\langle \mu_P, \mu_Q \rangle$, which is the set kernel and underlies support measure machines, or the Gaussian-on-MMD kernel $\exp(-\text{MMD}^2/2\gamma^2)$; both are positive definite, the second by a Schur-product argument that reruns "the Gaussian kernel is a kernel" inside the embedding space. The estimator carries two error sources, a finite number of bags ℓ and a finite bag size N , and its risk decomposes accordingly: the stage-two term vanishes as $\ell \rightarrow \infty$, and the stage-one term, controlled by the $N^{-1/2}$ embedding rate and the Lipschitz continuity of the distribution kernel, vanishes as $N \rightarrow \infty$. Szabó, Sriperumbudur, Póczos, and Gretton (2016) proved that with λ shrinking and the bags growing polynomially in ℓ the two-stage estimator is consistent and even minimax-optimal, so finite bags cost nothing asymptotically. The same construction reaches set kernels and support measure machines (Muandet et al. 2012), inherits universality on measures (Christmann and Steinwart 2010), has a density-based rival (Póczos et al. 2013), and merges with functional data analysis (Ramsay and Silverman 2005), where the input is a function in L^2 and the embedding view makes distribution regression a special case. The thread back to conditional mean embeddings and causal inference with kernels is the same one throughout: once a distribution is a point in a Hilbert space, every kernel method already built applies to it unchanged.

37.9 Common mistakes and practical implications

For **Distribution Regression and Functional Data**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

37.10 Summary and further reading

This chapter established explain the central definitions and claims in Distribution Regression and Functional Data; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Szabó and Gretton 2016), (Muandet and Schölkopf 2012), (Christmann and Steinwart 2010).

37.11 Exercises

1. warm-up Explain, in terms of the two-stage sampled model, why distribution regression has two sources of error while ordinary regression has one. Name the integer that controls each, say which limit $\ell \rightarrow \infty$ or $N \rightarrow \infty$ removes each error, and give a one-sentence example of a real task whose natural input is a bag of samples rather than a vector.

2. computation Take the two bags $A = \{0, 1\}$ and $B = \{2, 3\}$ on $\mathbb{R}$ with base kernel $k(a, b) = e^{-(a-b)^2/2}$. Compute the three empirical inner products $\langle \hat{\mu}_A, \hat{\mu}_A \rangle$, $\langle \hat{\mu}_B, \hat{\mu}_B \rangle$, and $\langle \hat{\mu}_A, \hat{\mu}_B \rangle$ as averages of four base-kernel values each, then form the squared empirical MMD $D_{AB} = \langle \hat{\mu}_A, \hat{\mu}_A \rangle + \langle \hat{\mu}_B, \hat{\mu}_B \rangle - 2\langle \hat{\mu}_A, \hat{\mu}_B \rangle$ and the Gaussian-on-MMD value $\mathcal{K}_1(A, B) = e^{-D_{AB}/2}$. Report each number.
3. computation Continue Example (predicting a label): keeping $\gamma = 1$ fixed, recompute the held-out prediction $\hat{f}(P_3) = \mathbf{k}_*^\top (\mathbf{K}_{\text{tr}} + \ell \lambda I)^{-1} \mathbf{y}$ for the two ridge values $\lambda = 0.2$ and $\lambda = 0.01$. Explain the direction each prediction moves relative to the value 1.8719 at $\lambda = 0.05$, and relate it to the bias term of the risk decomposition.
4. proof Prove that the linear embedding kernel $\mathcal{K}_{\text{lin}}(P, Q) = \langle \mu_P, \mu_Q \rangle_{\mathcal{H}}$ is positive definite directly from the quadratic form, and identify its feature map. Then show it equals $\mathbb{E}_{X \sim P, Y \sim Q}[k(X, Y)]$, so its empirical version is the set kernel $\frac{1}{|A||B|} \sum_{a,b} k(a, b)$. Hint

For positive definiteness, $\sum_{a,b} c_a c_b \langle \mu_{P_a}, \mu_{P_b} \rangle = \|\sum_a c_a \mu_{P_a}\|^2 \geq 0$; the feature map is $\Phi(P) = \mu_P$. For the expectation form, write $\mu_P = \mathbb{E}_X[K_X]$, $\mu_Q = \mathbb{E}_Y[K_Y]$, and pull both expectations out of the inner product using $\langle K_X, K_Y \rangle = k(X, Y)$.

5. proof Establish the finite-bag rate directly. For $\hat{\mu}_P = \frac{1}{N} \sum_{a=1}^N K_{x^a}$ with $x^a \stackrel{\text{i.i.d.}}{\sim} P$ and a bounded base kernel $k(x, x) \leq \kappa$, show $\mathbb{E} \|\hat{\mu}_P - \mu_P\|_{\mathcal{H}}^2 \leq \kappa/N$, hence $\|\hat{\mu}_P - \mu_P\|_{\mathcal{H}} = O_P(N^{-1/2})$. Then combine this with the Lipschitz lemma to bound $\|\Psi(\hat{P}) - \Psi(P)\|_{\mathcal{H}_{\mathcal{K}}}$ in expectation, showing the finite-bag error propagates to the second stage at the same $N^{-1/2}$ rate. Hint

The estimator is a mean of N i.i.d. zero-mean terms $K_{x^a} - \mu_P$, so its squared-norm expectation is $\frac{1}{N} (\mathbb{E}[k(X, X)] - \|\mu_P\|^2) \leq \kappa/N$. Then $\mathbb{E} \|\Psi(\hat{P}) - \Psi(P)\|^2 \leq \frac{1}{\gamma^2} \mathbb{E} \|\hat{\mu}_P - \mu_P\|^2 \leq \kappa/(\gamma^2 N)$ by the Lipschitz lemma.

6. proof Prove the Lipschitz lemma's key inequality without quoting it: for the Gaussian-on-MMD kernel, show $\|\Psi(P) - \Psi(P')\|_{\mathcal{H}_{\mathcal{K}}}^2 = 2(1 - e^{-\|\mu_P - \mu_{P'}\|^2/2\gamma^2}) \leq \gamma^{-2} \|\mu_P - \mu_{P'}\|^2$. Deduce that as $\gamma \rightarrow \infty$ the second-stage feature map becomes ever flatter in the embedding, and explain what that means for the bias-variance trade-off of the two-stage estimator. Hint

Use $\mathcal{K}_\gamma(P, P) = 1$ to get $\|\Psi(P) - \Psi(P')\|^2 = 2 - 2\mathcal{K}_\gamma(P, P')$, then the bound $1 - e^{-u} \leq u$ with $u = \|\mu_P - \mu_{P'}\|^2/2\gamma^2$. Large γ means a nearly constant kernel, low variance but high bias, the usual bandwidth trade-off now on distributions.

7. challenge Consider the linear embedding kernel with the linear base kernel $k(x, x') = x^\top x'$ on $\mathbb{R}^d$. Show that then $\mu_P = \mathbb{E}_{X \sim P}[X] = m_P$, the ordinary mean, so $\mathcal{K}_{\text{lin}}(P, Q) = m_P^\top m_Q$ and the two-stage regressor can only fit functions of the input mean. Conclude that this choice is blind to any feature of P beyond its mean, and state which property of the base kernel, in the language of the mean-embedding chapter, must hold for the two-stage estimator to be able to distinguish distributions with equal means. Finally, argue that using a characteristic base kernel is necessary but not sufficient for the two-stage estimator to be consistent, and name the extra ingredient the second stage requires. Hint

With the linear base kernel $\mu_P(x) = \mathbb{E}[X]^\top x$, so the embedding is the mean vector and the set kernel is $m_A^\top m_B$. Distinguishing equal-mean distributions requires a characteristic base kernel (injective embedding). But injectivity of stage one is not enough: the second-stage kernel on embeddings must also be universal on the space of measures, which is the Christmann and Steinwart (2010) condition.

38 Large-Scale Kernel Machines

Every kernel method we have built so far shares a hidden liability: it is expensive. The representer theorem turns a search over functions into a search over n coefficients, but the price of that convenience is the $n \times n$ Gram matrix. Forming it costs $O(n^2)$ kernel evaluations and $O(n^2)$ memory, and the linear systems at the heart of kernel ridge regression, Gaussian processes, and the SVM cost $O(n^3)$ to solve exactly. For n in the tens of thousands this is already uncomfortable; for n in the millions it is impossible. This chapter is about breaking that wall. We first take an interlude into optimization, because the escape route for very large problems is almost always to stop solving linear systems and start descending gradients, and the algorithms that do this well for large finite sums (stochastic gradient descent and its variance-reduced cousins) deserve to be understood on their own terms. We then return to kernels with two families of approximation, each of which removes a different part of the cost. The Nystrom method replaces the Gram matrix by a low-rank surrogate built from a small subset of points, killing the $O(n^2)$ storage and the $O(n^3)$ solve. Random Fourier features go further still: they build an explicit, finite-dimensional feature map whose inner products approximate the kernel, so that a kernel machine becomes an ordinary linear model that any large-scale linear solver can handle. Both turn an intractable kernel problem into a tractable linear one; they differ in how, and in what they give up.

38.1 The scaling problem

Recall the shape of a typical kernel learning problem. Given training data $(\mathbf{x}_i, y_i)_{i=1}^n$, a positive definite kernel K with RKHS $\mathcal{H}$, and a loss L , we solve

$$\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(\mathbf{x}_i)) + \lambda \|f\|_{\mathcal{H}}^2.$$

The representer theorem tells us that the minimizer has the form $f(\mathbf{x}) = \sum_{i=1}^n \alpha_i K(\mathbf{x}_i, \mathbf{x})$, and the problem reduces to a finite-dimensional one in the coefficient vector $\alpha \in \mathbb{R}^n$, written entirely in terms of the Gram matrix $K \in \mathbb{R}^{n \times n}$ with entries $[K]_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$. For the square loss this is the linear system $(K + n\lambda I)\alpha = \mathbf{y}$, and the solution costs $O(n^3)$ arithmetic and, before that, $O(n^2)$ kernel evaluations merely to build K , which then occupies $O(n^2)$ memory. The cubic term is the signature of an exact solve: eliminating n unknowns against n equations, as a Cholesky or Gaussian factorization does, spends work proportional to n^3 . One can dodge the cube with an iterative solver such as conjugate gradients, whose every step is a single matrix-vector product with K , but each such product still costs $O(n^2)$ and still needs the whole matrix at hand, so the memory wall stands even when the arithmetic wall is lowered. To break the problem open, one must attack the matrix itself.

These three numbers, $O(n^2)$ evaluations, $O(n^2)$ memory, and $O(n^3)$ solve, are the wall. It helps to attach units to them. At $n = 10^4$ the Gram matrix has 10^8 entries, roughly a gigabyte in double precision, and the cubic solve is 10^{12} operations: uncomfortable but survivable. At $n = 10^6$ the same matrix would need 10^{12} entries, on the order of eight terabytes, and the solve would be 10^{18} operations. The matrix no longer fits in memory, and even writing it down is out of reach, quite apart from inverting it. This is why a method that is elegant on a thousand points becomes hopeless on a million: the costs do not grow gently, they grow with the square and the cube. Every technique in this chapter is best read as an attack on one or more of these three numbers. It is useful to keep asking, of each approximation: which of the three costs does this remove, and what does it cost me in accuracy or generality in return?

The object at the centre of all three costs is the Gram matrix itself, so it is worth looking at directly before we start dismantling it. The next figure builds K live for a handful of points on a line: as you add points the matrix grows

in both directions at once, which is the $O(n^2)$ memory made visible, and the bandwidth of the kernel controls how quickly off-diagonal entries decay, which is what later lets a low-rank surrogate stand in for the whole thing.

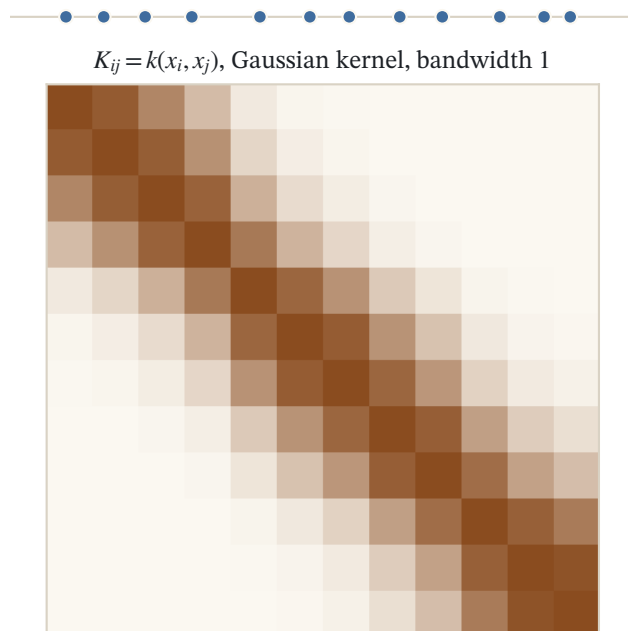


Figure 38.1: Twelve points on a line and their Gram matrix $K_{ij} = k(x_i, x_j)$ for a Gaussian kernel at bandwidth 1. A kernel is positive definite exactly when this matrix is positive semidefinite for every sample; narrowing the bandwidth drives it toward the identity.

The first idea does not touch the kernel at all. It observes that the finite-dimensional problem in α is, structurally, a large-scale *linear* learning problem, a sum of n terms to be minimized, and that the machine-learning community has excellent tools for exactly that. So we begin by studying those tools in the clean setting of linear models, and only afterwards ask how they, together with cheap feature maps, rescue kernels.

38.2 Interlude: large-scale learning with linear models

Strip away the kernel and consider the generic objective

$$\min_{\mathbf{w} \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{w}),$$

a large finite sum of functions of a parameter vector $\mathbf{w}$. Each f_i is typically the loss on the i -th example plus a share of the regularizer. When n is enormous, the deciding quantity is no longer the total number of arithmetic operations but the cost *per iteration*, and in particular whether that cost grows with n . We will meet three regimes: batch gradient descent, whose every step touches all n terms; stochastic gradient descent, whose every step touches one; and randomized incremental methods, which touch one term per step yet, remarkably, converge as fast as the batch method. To compare them honestly we first need a small kit of optimization principles.

38.2.1 Why convexity, and two inequalities

The reason convexity matters is epistemic: it lets local observations speak about the global optimum. For a differentiable convex function, the stationarity condition $\nabla f(\mathbf{w}) = 0$ is not merely necessary but sufficient for

global optimality, so a gradient that has vanished certifies that we have arrived. Convexity also makes it easy to bound the suboptimality $f(\mathbf{w}) - f^\star$, which is what a convergence proof must control. Two inequalities do almost all of the work.

The first expresses convexity itself. A differentiable function f is convex if and only if it lies above each of its tangents:

$$f(\mathbf{w}) \geq f(\mathbf{w}_0) + \nabla f(\mathbf{w}_0)^\top (\mathbf{w} - \mathbf{w}_0) \quad \text{for all } \mathbf{w}, \mathbf{w}_0.$$

The linear term on the right is the first-order approximation of f at $\mathbf{w}_0$; convexity says the true function never dips below it.

The second inequality bounds f from *above* and requires not convexity but smoothness. Say that ∇f is L -Lipschitz if $\|\nabla f(\mathbf{w}) - \nabla f(\mathbf{w}')\|_2 \leq L\|\mathbf{w} - \mathbf{w}'\|_2$ for all $\mathbf{w}, \mathbf{w}'$. Such an f is trapped beneath a quadratic that hugs its tangent.

Lemma (descent lemma, or the smoothness inequality)

If ∇f is L -Lipschitz continuous (f need not be convex), then for all $\mathbf{w}$ and $\mathbf{z}$,

$$f(\mathbf{w}) \leq g(\mathbf{w}) := f(\mathbf{z}) + \nabla f(\mathbf{z})^\top (\mathbf{w} - \mathbf{z}) + \frac{L}{2} \|\mathbf{w} - \mathbf{z}\|_2^2.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

By Taylor's theorem in integral form, writing the difference as a line integral of the gradient along the segment from $\mathbf{z}$ to $\mathbf{w}$,

$$f(\mathbf{w}) - f(\mathbf{z}) = \int_0^1 \nabla f(t\mathbf{w} + (1-t)\mathbf{z})^\top (\mathbf{w} - \mathbf{z}) dt.$$

Subtract the linear term $\nabla f(\mathbf{z})^\top (\mathbf{w} - \mathbf{z})$ from both sides and bound:

$$\begin{aligned} f(\mathbf{w}) - f(\mathbf{z}) - \nabla f(\mathbf{z})^\top (\mathbf{w} - \mathbf{z}) &= \int_0^1 (\nabla f(t\mathbf{w} + (1-t)\mathbf{z}) - \nabla f(\mathbf{z}))^\top (\mathbf{w} - \mathbf{z}) dt \\ &\leq \int_0^1 \|\nabla f(t\mathbf{w} + (1-t)\mathbf{z}) - \nabla f(\mathbf{z})\|_2 \|\mathbf{w} - \mathbf{z}\|_2 dt \\ &\leq \int_0^1 Lt \|\mathbf{w} - \mathbf{z}\|_2^2 dt = \frac{L}{2} \|\mathbf{w} - \mathbf{z}\|_2^2, \end{aligned}$$

where the first inequality is Cauchy-Schwarz and the second uses L -Lipschitzness with $\|(t\mathbf{w} + (1-t)\mathbf{z}) - \mathbf{z}\|_2 = t\|\mathbf{w} - \mathbf{z}\|_2$. Rearranging gives the claim. $\square$

The descent lemma is the engine of gradient descent. The upper bound g is a quadratic in $\mathbf{w}$ with curvature L , and completing the square shows it is minimized at

$$\mathbf{w}_1 = \mathbf{z} - \frac{1}{L} \nabla f(\mathbf{z}),$$

which is exactly a gradient step with step size $1/L$. So a gradient step does not just wander downhill; it jumps to the minimum of a global upper bound on f . Since $g(\mathbf{w}_1) \leq g(\mathbf{z}) = f(\mathbf{z})$, each step is guaranteed to decrease the objective.

38.2.2 Gradient descent and its rate

Theorem (gradient descent)

Assume f is convex and differentiable with L -Lipschitz gradient. The iterates

$$\mathbf{w}_t \leftarrow \mathbf{w}_{t-1} - \frac{1}{L} \nabla f(\mathbf{w}_{t-1})$$

satisfy

$$f(\mathbf{w}_t) - f^\star \leq \frac{L \|\mathbf{w}_0 - \mathbf{w}^\star\|_2^2}{2t}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Apply the descent lemma with $\mathbf{z} = \mathbf{w}_{t-1}$: for all $\mathbf{w}$,

$$f(\mathbf{w}) \leq g_t(\mathbf{w}) = f(\mathbf{w}_{t-1}) + \nabla f(\mathbf{w}_{t-1})^\top (\mathbf{w} - \mathbf{w}_{t-1}) + \frac{L}{2} \|\mathbf{w} - \mathbf{w}_{t-1}\|_2^2.$$

The quadratic g_t is minimized at $\mathbf{w}_t$, so it can be rewritten around its minimizer as $g_t(\mathbf{w}) = g_t(\mathbf{w}_t) + \frac{L}{2} \|\mathbf{w} - \mathbf{w}_t\|_2^2$. Evaluating at the optimum $\mathbf{w}^\star$,

$$\begin{aligned} f(\mathbf{w}_t) &\leq g_t(\mathbf{w}_t) = g_t(\mathbf{w}^\star) - \frac{L}{2} \|\mathbf{w}^\star - \mathbf{w}_t\|_2^2 \\ &= f(\mathbf{w}_{t-1}) + \nabla f(\mathbf{w}_{t-1})^\top (\mathbf{w}^\star - \mathbf{w}_{t-1}) + \frac{L}{2} \|\mathbf{w}^\star - \mathbf{w}_{t-1}\|_2^2 - \frac{L}{2} \|\mathbf{w}^\star - \mathbf{w}_t\|_2^2 \\ &\leq f^\star + \frac{L}{2} \|\mathbf{w}^\star - \mathbf{w}_{t-1}\|_2^2 - \frac{L}{2} \|\mathbf{w}^\star - \mathbf{w}_t\|_2^2, \end{aligned}$$

where the last step uses the convexity inequality $f(\mathbf{w}_{t-1}) + \nabla f(\mathbf{w}_{t-1})^\top (\mathbf{w}^\star - \mathbf{w}_{t-1}) \leq f(\mathbf{w}^\star) = f^\star$. Thus $f(\mathbf{w}_t) - f^\star \leq \frac{L}{2} \|\mathbf{w}^\star - \mathbf{w}_{t-1}\|_2^2 - \frac{L}{2} \|\mathbf{w}^\star - \mathbf{w}_t\|_2^2$. Summing from $t = 1$ to T , the right side telescopes, and because the sequence $f(\mathbf{w}_t)$ is nonincreasing the left side is at least $T(f(\mathbf{w}_T) - f^\star)$:

$$T(f(\mathbf{w}_T) - f^\star) \leq \sum_{t=1}^T (f(\mathbf{w}_t) - f^\star) \leq \frac{L}{2} \|\mathbf{w}^\star - \mathbf{w}_0\|_2^2 - \frac{L}{2} \|\mathbf{w}^\star - \mathbf{w}_T\|_2^2 \leq \frac{L}{2} \|\mathbf{w}^\star - \mathbf{w}_0\|_2^2.$$

Dividing by T gives the bound. $\square$

The rate $O(1/t)$ is the baseline for smooth convex problems. It improves under stronger assumptions, and some variants achieve $O(1/t^2)$ with an extrapolation trick (Nesterov, 2004). The most important improvement comes from strong convexity.

Say f is μ -strongly convex if it lies *above* a quadratic with curvature μ at each point:

$$f(\mathbf{w}) \geq f(\mathbf{w}_0) + \nabla f(\mathbf{w}_0)^\top (\mathbf{w} - \mathbf{w}_0) + \frac{\mu}{2} \|\mathbf{w} - \mathbf{w}_0\|_2^2.$$

Together with L -smoothness, this pins f between two quadratics of curvatures $\mu \leq L$, and the ratio L/μ (the condition number) governs the speed of descent.

Proposition (linear convergence under strong convexity)

If f is μ -strongly convex and differentiable with L -Lipschitz gradient, gradient descent with step size $1/L$ satisfies

$$f(\mathbf{w}_t) - f^\star \leq \left(1 - \frac{\mu}{L}\right)^t \frac{L \|\mathbf{w}_0 - \mathbf{w}^\star\|_2^2}{2}.$$

This geometric decay is called a linear convergence rate.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Start from the one-step inequality of the previous proof, but now use strong convexity to sharpen the lower bound on $f(\mathbf{w}_{t-1})$: since $f(\mathbf{w}_{t-1}) + \nabla f(\mathbf{w}_{t-1})^\top (\mathbf{w}^\star - \mathbf{w}_{t-1}) \leq f^\star - \frac{\mu}{2} \|\mathbf{w}^\star - \mathbf{w}_{t-1}\|_2^2$,

$$f(\mathbf{w}_t) \leq f^\star + \frac{L - \mu}{2} \|\mathbf{w}^\star - \mathbf{w}_{t-1}\|_2^2 - \frac{L}{2} \|\mathbf{w}^\star - \mathbf{w}_t\|_2^2.$$

On the other hand strong convexity applied at $\mathbf{w}^\star$ gives $f(\mathbf{w}_t) \geq f^\star + \frac{\mu}{2} \|\mathbf{w}_t - \mathbf{w}^\star\|_2^2$. Combining the two and simplifying,

$$\|\mathbf{w}^\star - \mathbf{w}_t\|_2^2 \leq \frac{L - \mu}{L + \mu} \|\mathbf{w}^\star - \mathbf{w}_{t-1}\|_2^2 \leq \left(1 - \frac{\mu}{L}\right) \|\mathbf{w}^\star - \mathbf{w}_{t-1}\|_2^2.$$

Iterating, $\|\mathbf{w}^\star - \mathbf{w}_t\|_2^2 \leq (1 - \mu/L)^t \|\mathbf{w}^\star - \mathbf{w}_0\|_2^2$, and finally $f(\mathbf{w}_t) - f^\star \leq \frac{L}{2} \|\mathbf{w}_t - \mathbf{w}^\star\|_2^2$ yields the claim. $\square$

The lesson is quantitative. To reach accuracy ε , the plain $O(1/t)$ rate needs $O(1/\varepsilon)$ iterations, while the linear rate needs only $O((L/\mu) \log(1/\varepsilon))$. But every one of those iterations, in the batch method, costs a full pass over the n terms of the sum. When n is large that per-iteration cost is the bottleneck, and the next algorithms attack it directly.

38.2.3 Stochastic gradient descent

Consider first the idealized problem of minimizing an expectation,

$$\min_{\mathbf{w} \in \mathbb{R}^p} f(\mathbf{w}) = \mathbb{E}_{\mathbf{x}}[\ell(\mathbf{x}, \mathbf{w})],$$

where $\mathbf{x} \mapsto \ell(\mathbf{x}, \mathbf{w})$ is differentiable (everything below extends to nonsmooth losses using subgradients). We cannot compute ∇f because we cannot integrate over the data distribution, but a single sample gives an unbiased estimate of it. This is the whole idea of stochastic gradient descent, which descends from the stochastic approximation method of Robbins and Monro (1951).

The finite sum $\frac{1}{n} \sum_i f_i(\mathbf{w})$ is the same object in disguise: it is the expectation of $f_I(\mathbf{w})$ when the index I is drawn uniformly from $\{1, \dots, n\}$, so its gradient is the average of the n per-example gradients. Batch gradient descent evaluates that average exactly, paying for all n terms to learn the single direction it will step in. The observation behind SGD is that we do not need the exact average to make progress, only its direction on average: a single randomly chosen $\nabla f_i(\mathbf{w})$ already points downhill in expectation, because its expectation over the random draw is precisely $\nabla f(\mathbf{w})$. We are trading an exact gradient for a cheap unbiased guess of it, and betting that many quick guesses outrun a few exact computations. The cost this removes is the per-iteration one: the price of a step drops from n gradient evaluations to a single one, and stops growing with the dataset entirely. There is a second reason the bet pays off, visible if we ask when the noise actually hurts. Far from the optimum, the per-example gradients largely agree on which way is downhill, so a single one is nearly as informative as their average, and the cheap

step loses almost nothing exactly when there is the most progress to be made. The disagreement between them, and hence the noise, becomes the limiting factor only near the optimum, which is precisely where the next family of methods will step in.

Algorithm (stochastic gradient descent)

At iteration t :

1. Draw one example $\mathbf{x}_t$ at random (from the data or the stream).
2. Update $\mathbf{w}_t \leftarrow \mathbf{w}_{t-1} - \eta_t * \text{grad_w ell}(\mathbf{x}_t, \mathbf{w}_{t-1})$.
3. (optional) Average the iterates:
 $\tilde{\mathbf{w}}_t \leftarrow (1 - \gamma_t) \tilde{\mathbf{w}}_{t-1} + \gamma_t \mathbf{w}_t$.

Because $\mathbb{E}[\nabla_{\mathbf{w}} \ell(\mathbf{x}_t, \mathbf{w}_{t-1}) \mid \mathbf{w}_{t-1}] = \nabla f(\mathbf{w}_{t-1})$, each step follows the true gradient in expectation while touching a single example. The step sizes η_t may be constant or decreasing, and the optional averaging of the iterates stabilizes the trajectory; the right choices depend on the problem. Under standard assumptions one obtains

$$f(\tilde{\mathbf{w}}_t) - f^* = O\left(\frac{1}{\sqrt{t}}\right) \text{ for convex } f, \quad f(\tilde{\mathbf{w}}_t) - f^* = O\left(\frac{1}{t}\right) \text{ for } \mu\text{-strongly convex } f.$$

These rates look worse than batch gradient descent's, and per accuracy they are: the noise in the one-sample gradient forces the step sizes down and slows the late-stage convergence. But the accounting is per iteration, and there each SGD step costs a single gradient evaluation against n for a batch step. When n is huge, many cheap noisy steps beat a few expensive exact ones. Moreover, when data are effectively infinite (a stream), SGD directly minimizes the expected risk, which is the quantity we actually care about, with no detour through an empirical average. Its main practical nuisance is that choosing the learning rate schedule well remains something of an open problem.

38.2.4 Randomized incremental methods for finite sums

It is worth being precise about why SGD's noise never goes away, because that is exactly what the next methods repair. Even when we sit at the optimum $\mathbf{w}^*$, the full gradient $\nabla f(\mathbf{w}^*)$ is zero only as an *average*: the individual gradients $\nabla f_i(\mathbf{w}^*)$ generally are not zero, they merely cancel in aggregate. A one-sample estimate picks out one of these nonzero vectors at random, so its variance does not shrink as we approach the solution. To keep this irreducible jitter from throwing the iterate back out, SGD must shrink its step size toward zero, and shrinking steps are what drag the rate down to $O(1/t)$. The escape is to build a gradient estimate whose variance *does* vanish as $\mathbf{w} \rightarrow \mathbf{w}^*$, so that constant step sizes become safe again and the fast geometric rate returns.

SGD paid for its cheap steps with a slow rate, because its gradient estimate, though unbiased, is noisy, and the noise never goes away. But the true objective of interest here is a *finite* sum,

$$\min_{\mathbf{w} \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{w}),$$

and a finite sum offers something a stream does not: we can remember. A family of randomized incremental methods exploits this to get the best of both worlds, a per-iteration cost like SGD's yet a linear convergence rate like the batch method's, for strongly convex problems. The prototype is the stochastic average gradient, SAG (Schmidt, Le Roux, and Bach, 2017).

SAG keeps in memory one stored gradient $\mathbf{y}_i$ per example and, at each step, refreshes exactly one of them while stepping along the average of all n stored gradients:

$$\mathbf{w}_t \leftarrow \mathbf{w}_{t-1} - \frac{\gamma}{Ln} \sum_{i=1}^n \mathbf{y}_i^t, \quad \mathbf{y}_i^t = \begin{cases} \nabla f_i(\mathbf{w}_{t-1}) & \text{if } i = i_t, \\ \mathbf{y}_i^{t-1} & \text{otherwise,} \end{cases}$$

where i_t is drawn at random. Only one new gradient is computed per iteration, and the running sum is maintained incrementally, so the per-iteration cost is that of a single ∇f_i . As the iterates settle, the stored gradients grow accurate and their average converges to the true full gradient, so the effective noise vanishes and fast convergence sets in. Related methods include SAGA (Defazio, Bach, and Lacoste-Julien, 2014), SVRG (Johnson and Zhang, 2013), SDCA (Shalev-Shwartz and Zhang, 2013), and MISO (Mairal, 2015). None of this is free: the variance is not so much removed as paid for with a different resource. SAG buys it with memory, storing one gradient vector per example at a cost of $O(np)$; SVRG (Johnson and Zhang, 2013) instead spends occasional extra computation, periodically recomputing a full-batch gradient to anchor its corrections and holding no per-example table. The choice between them is usually a choice between spare memory and spare compute.

Most of these can be read as SGD with a variance-reduced gradient estimator. They take a step $\mathbf{w}_t \leftarrow \mathbf{w}_{t-1} - \eta_t \mathbf{g}_t$ with $\mathbb{E}[\mathbf{g}_t | \mathbf{w}_{t-1}] = \nabla f(\mathbf{w}_{t-1})$, exactly as SGD does, but $\mathbf{g}_t$ has lower variance because it corrects the one-sample gradient using stored information (this is explicit in SVRG). The payoff is a rate of the form

$$f(\mathbf{w}_t) - f^\star = O\left(\left(1 - C \max\left\{\frac{1}{n}, \frac{\mu}{L}\right\}\right)^t\right),$$

which is linear (geometric) for strongly convex problems. The features that make these methods attractive at scale are worth stating plainly: their per-iteration complexity is independent of n ; unlike SGD they are often almost parameter-free, needing no delicate learning-rate schedule; and they can be further accelerated (Lin, Mairal, and Harchaoui, 2015).

The conclusion of the interlude is that huge-scale *linear* problems are, by now, a solved matter: we can minimize a sum of a million convex terms efficiently. The remaining question is how to make a kernel problem look like one, and that is what the rest of the chapter delivers.

38.3 Matrix-free computation and EigenPro

Iterative solvers do not require a stored Gram matrix; they require a function that computes $v \mapsto Kv$. Kernel blocks can be generated, multiplied, and discarded, reducing resident memory from $O(n^2)$ to the block size. The arithmetic remains $O(n^2)$ per exact product, but modern accelerators can batch those evaluations and structured kernels can provide faster products. A matrix-free claim is therefore incomplete unless it reports kernel-product time, peak memory, precision, stopping residual, and the number of products.

The spectrum controls iteration count. For gradient descent on kernel least squares, the largest eigenvalue limits the stable step size while the small eigenvalues set slow directions. **EigenPro** uses a randomized estimate of the leading eigenspace to build a preconditioner that flattens the largest eigenvalues toward a chosen threshold (Ma and Belkin, 2017). The transformed problem permits a larger stable step without changing the target solution. This is acceleration by spectral reshaping, not a low-rank replacement of the kernel.

Algorithm (EigenPro-style matrix-free solve)

1. Sample a manageable subset and estimate its top eigenpairs with a randomized eigensolver.
2. Construct the spectral correction that maps the leading eigenvalues toward the cutoff while leaving the remaining directions unchanged.
3. Run minibatch gradient iterations, applying the correction through top-eigenvector products and computing kernel blocks on demand.
4. Stop on a validation or residual rule fixed in advance; report both the original-system residual and the preconditioned iteration count.

5. Compare wall time and peak memory with unpreconditioned iteration and with a Nyström or random-feature baseline.

The approximate eigenvectors introduce setup cost and their quality limits acceleration. A cutoff that is too aggressive can amplify numerical error, while a cutoff that is too conservative buys little. Preconditioning also changes the path followed by early-stopped iteration, so if stopping is statistical regularization, the preconditioner is part of the estimator and must be recorded.

38.4 Before approximation: decomposition and interior-point solvers

The interlude solved the large-scale *linear* problem, but it is worth remembering how the kernel community first fought the wall, because the lineage of the approximations to come runs straight through it. Historically the pressure was felt most sharply in the support vector machine, whose training is not an unconstrained sum but a constrained convex quadratic program. In dual form it reads

$$\min_{\alpha \in \mathbb{R}^n} \frac{1}{2} \alpha^\top Q \alpha - \mathbf{1}^\top \alpha \quad \text{subject to} \quad \mathbf{y}^\top \alpha = 0, \quad 0 \leq \alpha_i \leq C,$$

with $[Q]_{ij} = y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$ (Scholkopf and Smola, 2002). The entire cost lives in Q , an $n \times n$ matrix as dense and as expensive as the Gram matrix itself, so the QP inherits exactly the three costs of the previous section. Two very different strategies grew up to attack it, and the choice between them is essentially a choice of problem size.

For problems small enough that Q fits in memory, the method of choice was, and for high-precision work still is, the interior-point solver. Its structure is instructive because it is the second-order counterpart of the first-order methods above. Rather than enforce the complementarity condition $\alpha_i s_i = 0$ between each multiplier and its constraint slack directly, an interior-point method relaxes it to $\alpha_i s_i = \mu$ for a positive barrier parameter μ , solves the resulting smooth system by a Newton predictor-corrector step, and drives $\mu \rightarrow 0$ along a central path (Scholkopf and Smola, 2002). Each Newton step reduces to a linear system in the n variables whose matrix is the bordered *reduced KKT system*, factorized once per iteration, and this is the $O(n^3)$ work that caps the method at moderate n . What the reader should carry away is its stopping rule, since it recurs everywhere: the difference between the primal and dual objective values, the duality gap $\sum_i \alpha_i c_i(\alpha)$, bounds the suboptimality with no knowledge of the optimum, so one can certify a solution to a fixed number of significant figures and stop.

For problems too large for that, the answer was to never look at the whole matrix at once. Decomposition rests on a structural fact about the SVM: only the support vectors, the points with $\alpha_i > 0$, enter the final predictor, and they are usually a small fraction of the data. If one knew the support set in advance, one could discard everything else and solve a small problem. Chunking approximates that oracle iteratively (Scholkopf and Smola, 2002): start from an arbitrary working subset that fits in memory, solve the QP restricted to it, keep the support vectors, discard the rest, and refill the working set with the points that most violate the optimality (KKT) conditions, repeating until no violations remain. Subset-selection methods generalize the idea, at each round optimizing only a small working set, chosen as the variables that contribute most to the duality gap.

Pushed to its limit, decomposition shrinks the working set to just two variables, which is Sequential Minimal Optimization (Platt, 1998). With two coupled multipliers and the single equality constraint $\mathbf{y}^\top \alpha = 0$, the restricted problem has one genuine degree of freedom and can be solved *analytically*, by clipping the unconstrained optimum to the box $[L, H]$, so no inner QP solver is needed at all. Each step touches only two columns of the kernel, and no part of Q need ever be stored, which is what made kernel SVMs trainable on large datasets in the years before the approximation methods of this chapter. Two practical devices, common to all these solvers, are worth naming: caching a fraction of the kernel rows (an LRU cache holding roughly a tenth of K already reaches an 80 to 90 percent hit rate, since the same non-bound multipliers are revisited many times), and warm-starting, the reuse of the current solution when a hyperparameter such as C or the kernel width changes only slightly (Scholkopf and Smola, 2002).

These methods share one commitment that the rest of the chapter abandons: they keep the kernel *exact*. Every entry they touch is a true $K(\mathbf{x}_i, \mathbf{x}_j)$, and the working matrices, though never all resident at once, are cut from the

same $O(n^2)$ cloth. They defeat the memory wall by refusing to build all of it simultaneously, not by making it smaller. The two families we turn to now take the opposite route: they replace K itself by a genuinely low-rank or explicitly finite-dimensional surrogate. The first of them descends, as the next section makes plain, directly from a reduced-set idea born in this same decomposition literature.

38.5 The Nystrom approximation

The first bridge back to kernels is the Nystrom method. Its aim is to remove the $O(n^2)$ storage and $O(n^3)$ solve by replacing the full kernel geometry with a cheap, finite-dimensional surrogate, while staying honestly inside the RKHS. It was introduced several times in the kernel literature (Williams and Seeger, 2001; Smola and Scholkopf, 2000; Fine and Scheinberg, 2001).

38.5.1 The principle: projection onto a small subspace

Let K be a p.d. kernel with RKHS $\mathcal{H}$ and feature map $\varphi : \mathcal{X} \rightarrow \mathcal{H}$, so $K(\mathbf{x}, \mathbf{x}') = \langle \varphi(\mathbf{x}), \varphi(\mathbf{x}') \rangle_{\mathcal{H}}$. The kernel matrix is expensive precisely because each point $\varphi(\mathbf{x})$ lives in the full, possibly infinite-dimensional $\mathcal{H}$, where every pair of points can be independently far apart. The Nystrom idea is to declare a small subspace and pretend all the data live in it. Fix $p \ll n$ anchor points $f_1, \dots, f_p \in \mathcal{H}$, let

$$\mathcal{F} := \text{Span}(f_1, \dots, f_p),$$

and replace each $\varphi(\mathbf{x})$ by its orthogonal projection onto $\mathcal{F}$,

$$\Pi_{\mathcal{F}}[\mathbf{x}] := \arg \min_{f \in \mathcal{F}} \|\varphi(\mathbf{x}) - f\|_{\mathcal{H}}^2.$$

Once every point sits in a p -dimensional subspace, all inner products, and hence the whole kernel matrix, are captured by p coordinates per point. The rank of the approximated geometry is at most p , which is what breaks the wall. Why should so drastic a collapse cost little? Because the Gram matrices of smooth kernels are, in practice, nearly low rank already: their eigenvalues decay quickly, so most of the geometry of the data lives in a subspace of modest dimension and only a thin tail is discarded when we project. Nystrom bets on that decay, and the right p is essentially the effective rank the spectrum reveals: enough anchors to span the directions that carry the mass, no more. When the eigenvalues decay quickly, a p far below n already captures nearly all of the kernel geometry, and the error paid for the collapse is only the discarded tail. The error incurred by approximating a Gram matrix this way, and how it scales with the number of anchors, was analyzed by Drineas and Mahoney (2005). Notice which of the three costs this removes. Instead of an $n \times n$ matrix we will store n vectors of length p , so the $O(n^2)$ memory becomes $O(pn)$; instead of an $O(n^3)$ dense solve we will solve a linear problem in p dimensions. The $O(n^2)$ kernel evaluations are also avoided, since we only ever evaluate the kernel between data points and the handful of anchors. It remains to compute the projection explicitly and to choose the anchors well.

Write a candidate element of $\mathcal{F}$ as $\sum_{j=1}^p \beta_j f_j$. The projection coefficients solve

$$\beta^{\star} \in \arg \min_{\beta \in \mathbb{R}^p} \left\| \varphi(\mathbf{x}) - \sum_{j=1}^p \beta_j f_j \right\|_{\mathcal{H}}^2.$$

Expanding the squared norm and dropping the constant $\|\varphi(\mathbf{x})\|_{\mathcal{H}}^2$,

$$\min_{\beta \in \mathbb{R}^p} -2 \sum_{j=1}^p \beta_j \langle f_j, \varphi(\mathbf{x}) \rangle_{\mathcal{H}} + \sum_{j,l=1}^p \beta_j \beta_l \langle f_j, f_l \rangle_{\mathcal{H}}.$$

Now introduce two objects that carry all the needed information: the $p \times p$ anchor Gram matrix $[K_f]_{jl} = \langle f_j, f_l \rangle_{\mathcal{H}}$, and the vector $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_p(\mathbf{x}))^\top \in \mathbb{R}^p$ of evaluations of the anchors at $\mathbf{x}$. Here we used the reproducing property in the form $\langle f_j, \varphi(\mathbf{x}) \rangle_{\mathcal{H}} = f_j(\mathbf{x})$ when $f_j \in \mathcal{H}$. The problem is the quadratic

$$\min_{\beta \in \mathbb{R}^p} -2\beta^\top \mathbf{f}(\mathbf{x}) + \beta^\top K_f \beta,$$

and, assuming K_f nonsingular for simplicity, its solution is

$$\beta^\star(\mathbf{x}) = K_f^{-1} \mathbf{f}(\mathbf{x}).$$

The projected point is $\varphi(\mathbf{x}) \approx \sum_{j=1}^p \beta_j^\star(\mathbf{x}) f_j$, and the approximate inner product between two projected points is

$$\langle \varphi(\mathbf{x}), \varphi(\mathbf{x}') \rangle_{\mathcal{H}} \approx \left\langle \sum_j \beta_j^\star(\mathbf{x}) f_j, \sum_l \beta_l^\star(\mathbf{x}') f_l \right\rangle_{\mathcal{H}} = \beta^\star(\mathbf{x})^\top K_f \beta^\star(\mathbf{x}').$$

This is already a finite-dimensional kernel, but we can make it look like a plain Euclidean dot product. Factor $K_f = K_f^{1/2} K_f^{1/2}$ and define the explicit feature map

$$\psi(\mathbf{x}) = K_f^{1/2} \beta^\star(\mathbf{x}) = K_f^{-1/2} \mathbf{f}(\mathbf{x}) \in \mathbb{R}^p.$$

Then $K(\mathbf{x}, \mathbf{x}') \approx \langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle_{\mathbb{R}^p}$. We have manufactured an explicit p -dimensional feature map whose dot products approximate the kernel. In matrix terms, for a training set $S = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, stacking $\psi(S) = [\psi(\mathbf{x}_1), \dots, \psi(\mathbf{x}_n)] \in \mathbb{R}^{p \times n}$ gives the low-rank approximation

$$K \approx \psi(S)^\top \psi(S),$$

a Gram matrix of rank at most p . Storing $\psi(S)$ costs $O(pn)$ rather than $O(n^2)$, and any kernel machine can now be solved as a linear model in $\mathbb{R}^p$. Because the approximation keeps the points inside $\mathcal{H}$ (they are genuine projections of feature vectors), the resulting objects are legitimate elements of the RKHS: translations, linear combinations, and further projections are all meaningful. What remains is the one open choice: the anchors f_j .

38.5.2 Choosing the anchors by kernel PCA

The most principled choice learns the anchors to minimize the total projection error over the training data. Given $\mathbf{x}_1, \dots, \mathbf{x}_n$, solve

$$\min_{\substack{f_1, \dots, f_p \in \mathcal{H} \\ \beta_{ij} \in \mathbb{R}}} \sum_{i=1}^n \left\| \varphi(\mathbf{x}_i) - \sum_{j=1}^p \beta_{ij} f_j \right\|_{\mathcal{H}}^2.$$

By the same expansion as above, and minimizing over the coefficients β_i with the anchors held fixed (which gives $\beta_i = K_f^{-1} \mathbf{f}(\mathbf{x}_i)$), the problem over the anchors becomes

$$\max_{f_1, \dots, f_p \in \mathcal{H}} \sum_{i=1}^n \mathbf{f}(\mathbf{x}_i)^\top K_f^{-1} \mathbf{f}(\mathbf{x}_i).$$

This objective is invariant under a change of basis of $\mathcal{F}$, so we may look for a particularly convenient optimal solution. Take any optimal $\mathbf{f}^\star$, diagonalize its anchor Gram matrix $K_{f^\star} = U \Delta U^\top$, and define new functions

$$\mathbf{g}^\star(\mathbf{x}) = (g_1^\star(\mathbf{x}), \dots, g_p^\star(\mathbf{x})) = \Delta^{-1/2} U^\top \mathbf{f}^\star(\mathbf{x}).$$

Each $g_j^\star$ is a linear combination of the $f_k^\star$, hence still an element of $\mathcal{H}$. A direct computation shows these functions are orthonormal in the RKHS: their Gram matrix is

$$[K_{g^\star}]_{jl} = \frac{1}{\sqrt{\Delta_{jj}}} \frac{1}{\sqrt{\Delta_{ll}}} \mathbf{u}_j^\top K_{f^\star} \mathbf{u}_l = \frac{1}{\sqrt{\Delta_{jj}}} \frac{1}{\sqrt{\Delta_{ll}}} \Delta_{ll} \mathbf{u}_j^\top \mathbf{u}_l = \delta_{j=l},$$

so $K_{g^\star} = I$. The change of basis leaves the objective unchanged, because

$$\mathbf{f}^\star(\mathbf{x}_i)^\top K_{f^\star}^{-1} \mathbf{f}^\star(\mathbf{x}_i) = \mathbf{f}^\star(\mathbf{x}_i)^\top U \Delta^{-1} U^\top \mathbf{f}^\star(\mathbf{x}_i) = \mathbf{g}^\star(\mathbf{x}_i)^\top \mathbf{g}^\star(\mathbf{x}_i) = \mathbf{g}^\star(\mathbf{x}_i)^\top K_{g^\star}^{-1} \mathbf{g}^\star(\mathbf{x}_i).$$

So $\mathbf{g}^\star$ is also optimal, and because $K_{g^\star} = I$ the objective for it reduces to $\sum_{j=1}^p \sum_{i=1}^n g_j^\star(\mathbf{x}_i)^2$ subject to $g_j \perp g_k$ for $j \neq k$ and $\|g_j\|_{\mathcal{H}} = 1$. That is exactly the kernel PCA problem: the optimal orthonormal anchors are the top p kernel principal directions of the data. Learning the Nystrom subspace and running kernel PCA are the same computation.

Recipe 1: Nystrom via kernel PCA

Given training points $\mathbf{x}_1, \dots, \mathbf{x}_n$:

1. Randomly choose a subset $Z = [\mathbf{x}_{\{z_1\}}, \dots, \mathbf{x}_{\{z_m\}}]$ of $m \leq n$ points.
2. Compute the $m \times m$ kernel matrix K_Z .
3. Run kernel PCA on K_Z : keep the $p \leq m$ leading principal directions, parametrized by vectors $\alpha_1, \dots, \alpha_p$ in $\mathbb{R}^m$.
4. Encode any point $\mathbf{x}$ by
 $\psi(\mathbf{x}) = (\sum_i \alpha_i K(\mathbf{x}_{\{z_i\}}, \mathbf{x}), \dots, \sum_i \alpha_p K(\mathbf{x}_{\{z_i\}}, \mathbf{x}))^\top$ in $\mathbb{R}^p$.

The coordinates of $\psi(\mathbf{x})$ are the coordinates of the projection of $\varphi(\mathbf{x})$ onto the orthogonal PCA basis. Restricting kernel PCA to a random landmark subset of size m is what keeps this affordable: training costs $O(m^3)$ for the eigendecomposition of K_Z plus $O(m^2)$ kernel evaluations, and encoding a new point costs $O(mp)$ for the matrix-vector product plus $O(m)$ kernel evaluations. The dominant recurring cost is encoding, which scales with the landmark count $m > p$.

38.5.3 Choosing the anchors by random sampling

The encoding cost above is annoying because it depends on m , the number of landmarks used inside the PCA, rather than on p , the final dimension. A cruder but faster choice removes the gap: take the anchors to be feature vectors of sampled training points themselves. Choose p indices $z_1, \dots, z_p$ and set

$$f_j = \varphi(\mathbf{x}_{z_j}), \quad \mathcal{F} = \text{Span}(\varphi(\mathbf{x}_{z_1}), \dots, \varphi(\mathbf{x}_{z_p})).$$

Now the anchor Gram matrix K_f is simply the $p \times p$ kernel matrix K_Z of the sampled points, and $\mathbf{f}(\mathbf{x}) = (K(\mathbf{x}_{z_1}, \mathbf{x}), \dots, K(\mathbf{x}_{z_p}, \mathbf{x}))^\top$ is a vector of kernel values. The general feature map specializes to

$$\psi(\mathbf{x}) = K_Z^{-1/2} \mathbf{f}_Z(\mathbf{x}) = K_Z^{-1/2} (K(\mathbf{x}_{z_1}, \mathbf{x}), \dots, K(\mathbf{x}_{z_p}, \mathbf{x}))^\top.$$

Recipe 2: Nystrom via random point sampling

1. Randomly choose a subset $Z = [\mathbf{x}_{\{z1\}}, \dots, \mathbf{x}_{\{zp\}}]$ of p points.
2. Compute the $p \times p$ kernel matrix K_Z and the factor $K_Z^{-1/2}$.
3. Encode any point $\mathbf{x}$ by
 $\psi(\mathbf{x}) = K_Z^{-1/2} (K(\mathbf{x}_{\{z1\}}, \mathbf{x}), \dots, K(\mathbf{x}_{\{zp\}}, \mathbf{x}))^T$.

Here landmarks and final dimension coincide, so the costs collapse onto p . Training is $O(p^3)$ for the eigendecomposition (or Cholesky) of K_Z plus $O(p^2)$ kernel evaluations; encoding one point is $O(p^2)$ for the matrix-vector product plus $O(p)$ kernel evaluations, so encoding the whole training set of n points is $O(p^2n)$. This is the $O(p^3) + O(p^2n)$ budget that makes Nystrom practical: with p fixed it is *linear* in n , a decisive improvement over $O(n^3)$. The price is that we lose the optimality of the PCA basis, and a purely random choice of anchors is not clever: it may sample redundant or uninformative points.

38.5.4 Greedy and K-means anchors

Between blind sampling and full kernel PCA lies a greedy selection that adds anchors one at a time, always taking the point currently worst approximated. Suppose $Z = \{\mathbf{x}_{z_1}, \dots, \mathbf{x}_{z_k}\}$ are the anchors so far. The squared residual of a point $\mathbf{x}$ projected onto their span is

$$\min_{\beta \in \mathbb{R}^k} \left\| \varphi(\mathbf{x}) - \sum_{j=1}^k \beta_j \varphi(\mathbf{x}_{z_j}) \right\|_{\mathcal{H}}^2 = \|\varphi(\mathbf{x})\|_{\mathcal{H}}^2 - \mathbf{f}_Z(\mathbf{x})^\top K_Z^{-1} \mathbf{f}_Z(\mathbf{x}),$$

with $\mathbf{f}_Z(\mathbf{x}) = (K(\mathbf{x}_{z_1}, \mathbf{x}), \dots, K(\mathbf{x}_{z_k}, \mathbf{x}))^\top$. Since $\|\varphi(\mathbf{x})\|_{\mathcal{H}}^2 = K(\mathbf{x}, \mathbf{x})$, the next anchor is the training point maximizing the residual $K(\mathbf{x}_i, \mathbf{x}_i) - \mathbf{f}_Z(\mathbf{x}_i)^\top K_Z^{-1} \mathbf{f}_Z(\mathbf{x}_i)$ (Bach and Jordan, 2002; Smola and Scholkopf, 2000; Fine and Scheinberg, 2001).

Recipe 3: Nystrom via greedy anchor selection

Initialize $Z = \{\}$. For $k = 1, \dots, p$:

1. Select $z_k = \operatorname{argmax}_i K(\mathbf{x}_i, \mathbf{x}_i) - \mathbf{f}_Z(\mathbf{x}_i)^\top K_Z^{-1} \mathbf{f}_Z(\mathbf{x}_i)$.
2. Update $Z \leftarrow Z + \{\mathbf{x}_{\{z_k\}}\}$.

A naive implementation recomputes K_Z^{-1} from scratch each round, costing $O(k^2n + k^3)$ per iteration. The fix is a rank-one update. When $Z' = Z \cup \{\mathbf{z}\}$, block-inverting the bordered matrix gives

$$K_{Z'}^{-1} = \begin{pmatrix} K_Z & \mathbf{f}_Z(\mathbf{z}) \\ \mathbf{f}_Z(\mathbf{z})^\top & K(\mathbf{z}, \mathbf{z}) \end{pmatrix}^{-1} = \begin{pmatrix} K_Z^{-1} + \frac{1}{s} \mathbf{b} \mathbf{b}^\top & -\frac{1}{s} \mathbf{b} \\ -\frac{1}{s} \mathbf{b}^\top & \frac{1}{s} \end{pmatrix},$$

where $\mathbf{b} = K_Z^{-1} \mathbf{f}_Z(\mathbf{z})$ and $s = K(\mathbf{z}, \mathbf{z}) - \mathbf{f}_Z(\mathbf{z})^\top K_Z^{-1} \mathbf{f}_Z(\mathbf{z})$ is the Schur complement, which is precisely the residual of the newly chosen point. So $K_{Z'}^{-1}$ follows from K_Z^{-1} in $O(k^2)$ operations, provided we keep the n vectors $\mathbf{f}_Z(\mathbf{x}_i)$ in memory; refreshing those after adding an anchor costs n kernel evaluations. The total training cost is $O(p^2n)$ float operations and $O(pn)$ kernel evaluations, matching random sampling's order while selecting far better anchors. This greedy rule is exactly an incomplete Cholesky factorization of the kernel matrix.

The classical ancestor: sparse greedy Gram approximation

The greedy rule above did not originate as a Nystrom variant. It is essentially the *sparse greedy matrix approximation* of Smola and Scholkopf (2000), one of the earliest systematic attacks on the $O(n^2)$ Gram matrix, and reading the two side by side shows how little the modern method had to add. That work starts from the same spectral premise we invoked for Nystrom: the kernel matrix typically has many small eigenvalues, so it can be replaced by the outer product $K \approx ZZ^\top$ of a tall, thin $Z \in \mathbb{R}^{n \times p}$ with $p \ll n$ at little cost (Scholkopf and Smola, 2002). It selects a small set of columns $K(\mathbf{x}_{z_1}, \cdot), \dots, K(\mathbf{x}_{z_p}, \cdot)$ and approximates every remaining $K(\mathbf{x}_i, \cdot)$ by its best

linear combination of them, with the error measured in RKHS distance, which is exactly the projection residual $K(\mathbf{x}_i, \mathbf{x}_i) - \mathbf{f}_Z(\mathbf{x}_i)^\top K_Z^{-1} \mathbf{f}_Z(\mathbf{x}_i)$ that the recipe above minimizes. Two of its devices survive intact in practice: the rank-one (bordered-inverse) update of K_Z^{-1} when a column is appended, and the trick of choosing the next column not from all n candidates but from a small random pool. A sampling argument keeps that pool startlingly small (Scholkopf and Smola, 2002): a random subset of only 59 candidates contains, with 95 percent confidence, an element better than 95 percent of all candidates, a guarantee independent of n . Nystrom's data-dependent anchor selection is thus the direct heir of this reduced-set method, incomplete-Cholesky update and all.

A last variant applies when $\mathcal{X} = \mathbb{R}^d$, where we may *synthesize* anchors rather than pick existing points. Running K-means on the training data yields p centroids $\mathbf{z}_1, \dots, \mathbf{z}_p$; setting $f_j = \varphi(\mathbf{z}_j)$ and applying the standard Nystrom construction gives the Clustered Nystrom approximation (Zhang, Tsang, and Kwok, 2008). Its cost matches random sampling apart from the K-means step, but because the anchors are placed where the data actually concentrate, it is data-dependent and can significantly outperform the other variants in practice.

Remark (what Nystrom is, in one line)

Every variant produces a low-rank factorization $K \approx LL^\top$ of the kernel matrix, with a geometric reading as orthogonal projection onto a p -dimensional subspace of $\mathcal{H}$. The greedy rule is an incomplete Cholesky factorization. Crucially, the approximation lives inside the RKHS: because $\psi(\mathbf{x})$ encodes genuine projections of feature vectors, RKHS operations on it (translations, linear combinations, projections) remain valid. This is exactly what the next method gives up.

38.6 Choosing landmarks by leverage scores

The recipes left the most consequential choice, which points to sample, in an unsatisfying state. Uniform sampling is the cheapest option and the blindest; greedy selection and kernel PCA see the data but pay extra for the privilege; K-means splits the difference when the inputs are Euclidean. What is still missing is a number attached to each individual point that measures how much the kernel geometry would lose without it: a quantity we could sample proportionally to, prove theorems about, and estimate cheaply. The intuition for what that number must express is already on the table. A point inside a tight cluster is exchangeable: any neighbor spans nearly the same direction of $\mathcal{H}$, so whatever importance the cluster has should be shared out among its members. An isolated point is the sole witness of its direction: drop it and an entire eigendirection of the Gram matrix vanishes from the landmark subspace. Importance is also relative to the accuracy actually being solved for. In ridge regression, eigendirections of K with eigenvalue far below the regularization level are crushed by the ridge whether or not the approximation keeps them, so a sensible score should discount them from the start. The quantity with exactly these properties is the ridge leverage score, put to work for kernel methods by El Alaoui and Mahoney (2015).

38.6.1 Ridge leverage scores and the effective dimension

Definition (ridge leverage scores, El Alaoui and Mahoney, 2015)

Let $K \in \mathbb{R}^{n \times n}$ be a kernel matrix and $\lambda > 0$ a regularization level (absorbing the sample-size factor, so the λ here plays the role of $n\lambda$ in the system of the opening section). The *ridge leverage score* of point i is

$$\ell_i(\lambda) = [K(K + \lambda I)^{-1}]_{ii},$$

and the *effective dimension* at level λ is their sum,

$$d_{\text{eff}}(\lambda) = \sum_{i=1}^n \ell_i(\lambda) = \text{tr}(K(K + \lambda I)^{-1}) = \sum_{k=1}^n \frac{\lambda_k}{\lambda_k + \lambda},$$

where $\lambda_1 \geq \dots \geq \lambda_n \geq 0$ are the eigenvalues of K .

The eigendecomposition makes the definition transparent. Writing $K = \sum_k \lambda_k \mathbf{u}_k \mathbf{u}_k^\top$ with orthonormal eigenvectors $\mathbf{u}_k$, the matrix $K(K + \lambda I)^{-1}$ has the same eigenvectors with eigenvalues $\lambda_k/(\lambda_k + \lambda)$, so the diagonal entry is

$$\ell_i(\lambda) = \sum_{k=1}^n \frac{\lambda_k}{\lambda_k + \lambda} u_{ki}^2,$$

where u_{ki} is the i -th coordinate of $\mathbf{u}_k$. Each eigendirection carries a survival weight $\lambda_k/(\lambda_k + \lambda)$, close to 1 when $\lambda_k \gg \lambda$ and close to 0 when $\lambda_k \ll \lambda$, and distributes that weight over the points in proportion to its eigenvector mass u_{ki}^2 . Because the rows of an orthogonal matrix have unit norm, $\sum_k u_{ki}^2 = 1$, and every weight lies in $(0, 1)$, so $0 \leq \ell_i(\lambda) < 1$: a leverage score is a fraction. A point that carries most of the mass of a strong or isolated eigendirection scores near 1, irreplaceable; the members of a redundant cluster split the weight of their shared direction, each scoring low. Summing over points, $d_{\text{eff}}(\lambda)$ counts the eigendirections that survive the ridge, a smooth version of the number of eigenvalues above λ ; how fast it grows as λ shrinks is exactly the spectral-decay information studied in the Mercer chapter. The score also has a direct statistical reading: $K(K + \lambda I)^{-1}$ is the smoother matrix of kernel ridge regression, the operator taking observations $\mathbf{y}$ to fitted values, so $\ell_i(\lambda)$ is the sensitivity of the i -th fitted value to its own observation, and $d_{\text{eff}}(\lambda)$ is the classical degrees of freedom of the fit.

This is the vocabulary in which uniform sampling's failure mode can be stated precisely. Uniform sampling of p landmarks catches any fixed point with probability p/n . An eigendirection whose mass u_{ki}^2 is concentrated on a handful of points, an isolated point, a small cluster, a rare pattern, enters the landmark subspace only if one of those few witnesses is drawn, and when the draw misses them all, the Nystrom projection loses the entire direction and pays its full eigenvalue in trace error. When leverage is flat across points, uniform sampling is essentially optimal, and nothing better can be done; when leverage is spiky, uniform sampling needs p large enough to catch the spikes by luck, which can be a large multiple of the $d_{\text{eff}}(\lambda)$ directions actually worth capturing. Sampling with probabilities proportional to $\ell_i(\lambda)$ puts the budget where the geometry lives by construction.

38.6.2 Sampling proportionally to leverage

Algorithm (Nystrom with ridge leverage sampling)

Input data $\mathbf{x}_1, \dots, \mathbf{x}_n$, kernel K , regularization λ , landmark budget p .

Output landmark set Z and Nystrom feature map ψ .

1. Compute or approximate the ridge leverage scores $\ell_i(\lambda)$, $i = 1, \dots, n$.
2. Draw p landmark indices independently with probabilities $q_i = \ell_i(\lambda)/d_{\text{eff}}(\lambda)$.
3. Build $\psi(\mathbf{x}) = K_Z^{-1/2} \mathbf{f}_Z(\mathbf{x})$ from the sampled landmarks as in Recipe 2.

The guarantee this buys is best stated as a spectral sandwich. Recall that any Nystrom approximation satisfies $\tilde{K} \preceq K$, because it is the Gram matrix of orthogonal projections of the feature vectors. Leverage sampling controls the other side.

Proposition (leverage-score Nystrom, El Alaoui and Mahoney, 2015)

Fix $\lambda > 0$, $\varepsilon \in (0, 1)$, and $\delta \in (0, 1)$, and sample p landmarks with probabilities proportional to the ridge leverage scores $\ell_i(\lambda)$. There is a constant C such that if

$$p \geq C \frac{d_{\text{eff}}(\lambda)}{\varepsilon^2} \log \frac{d_{\text{eff}}(\lambda)}{\delta},$$

then with probability at least $1 - \delta$ the Nystrom approximation $\tilde{K}$ built on those landmarks satisfies

$$0 \leq K - \tilde{K} \preceq \varepsilon(K + \lambda I).$$

In particular $(1 - \epsilon)(K + \lambda I) \preceq \tilde{K} + \lambda I \preceq K + \lambda I$, so ridge regression computed with $\tilde{K}$ in place of K at level λ is spectrally indistinguishable from the exact problem up to $(1 \pm \epsilon)$ factors, and its in-sample risk matches the exact estimator's up to a constant depending only on ϵ .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The reading of the sample size deserves emphasis, because it is the point of the whole construction: the number of landmarks that suffices is governed by $d_{\text{eff}}(\lambda)$, a property of the spectrum at the accuracy actually being solved for, not by n . For a smooth kernel on concentrated data the effective dimension grows very slowly as λ decreases, so a few dozen landmarks can faithfully replace a Gram matrix of any size. The one apparent circularity is that computing the scores exactly means forming $(K + \lambda I)^{-1}$, the very $O(n^3)$ object we are trying to avoid. The circularity is benign: the guarantee survives if the sampling probabilities use overestimates of the true scores, at the price of a proportionally larger p , and constant-factor overestimates can be obtained cheaply, for instance from a preliminary sketch built by uniform sampling, as El Alaoui and Mahoney (2015) show. In practice one bootstraps: a crude uniform sketch yields approximate scores, and the approximate scores yield a sharp landmark set.

Example (leverage scores find the irreplaceable landmarks)

Seven points on the line: a tight cluster of five, $x = 0, 0.15, 0.3, 0.45, 0.6$, and two isolated points at $x = 5$ and $x = 10$. Gaussian kernel with $\sigma = 1$, regularization $\lambda = 0.01$, landmark budget $p = 3$. All numbers from `checks/ch13-ex1.py`.

1. Read the spectrum. The eigenvalues of K are 4.7879, 1.0000, 1.0000, 0.2087, 0.0034, 0, 0: the cluster is essentially one strong direction plus small corrections, and each isolated point contributes its own eigenvalue of exactly 1, since it is nearly orthogonal to everything else. The best rank-3 approximation, by the top eigenvectors, has Frobenius error 0.2087 and trace error 0.2121: that is the floor no choice of three landmarks can beat.
2. Score the points. The ridge leverage scores are $\ell = (0.6415, 0.3217, 0.2794, 0.3217, 0.6415, 0.9901, 0.9901)$. The two isolated points score 0.9901, nearly the maximum 1: each is the sole witness of a surviving eigendirection. The five cluster members share their common directions, scoring between 0.28 and 0.64. The scores sum to the effective dimension $d_{\text{eff}}(\lambda) = 4.1861$: about four directions matter at this λ , one more than the budget $p = 3$ can hold.
3. Sample by leverage. The three highest-leverage points are the two isolated points plus one cluster representative, $Z = \{0, 5, 10\}$. The resulting Nystrom approximation has Frobenius error 0.5889 and trace error 0.594: above the rank-3 floor, as it must be with $p < d_{\text{eff}}$, but every surviving eigendirection is represented.
4. Sample uniformly. Averaged over all 35 subsets of size 3, uniform sampling incurs Frobenius error 1.0329 and trace error 1.21. Its worst draw is the all-cluster pick $Z = \{0, 0.15, 0.3\}$, with Frobenius error 1.4142 and trace error 2.001: both isolated directions are lost outright, and the trace error is, to three digits, the sum $1 + 1$ of their abandoned eigenvalues. Such a draw is no fluke: a uniform sample of three lands entirely inside the cluster with probability 0.2857.

Reading. Leverage sampling is nearly twice as accurate as the average uniform draw here, and its real advantage is the failure mode it deletes: more than a quarter of uniform draws miss both isolated points and pay their full eigenvalues in error, an event that sampling proportional to $\ell_i(\lambda)$ makes essentially impossible. The scores also announce, through $d_{\text{eff}} = 4.1861$, that a budget of three landmarks is one short of the geometry's demands: the gap between 0.5889 and the floor 0.2087 is the price of that missing landmark, not of the sampling rule.

Verification artifact. `checks/example-ch13-example-37-1.json` records the example source hash and verification scope.

38.7 FALKON and preconditioned solvers

Landmark selection settled, a gap remains in the accounting. Nystrom shrank the $n \times n$ kernel problem to a problem in p dimensions, and we have quietly assumed that the small problem is easy. At the scales that motivate this chapter it is not. The statistical theory below will mandate landmark counts like $\sqrt{n}$, which at $n = 10^8$ means 10^4 landmarks; solving the reduced problem by a direct factorization then costs $O(np^2) = O(n^2)$ arithmetic, and the wall we knocked down reappears one story lower. The escape is the one the interlude taught, iterate rather than factor, but applied with a twist that is the heart of this section: the reduced problem is small enough that a little exact linear algebra on it, two Cholesky factorizations that never touch n , can precondition the iteration into converging in a handful of steps. This combination, Nystrom plus preconditioned conjugate gradient, is the FALKON algorithm of Rudi, Carratino, and Rosasco (2017), and it is the closest thing large-scale kernel methods have to a reference solver.

38.7.1 The landmark system and its conditioning

Fix m landmarks $\mathbf{x}_{z_1}, \dots, \mathbf{x}_{z_m}$, sampled uniformly or by leverage (we write m for the landmark count in this section, as the solver literature does). Let $K_{mm} \in \mathbb{R}^{m \times m}$ be their kernel matrix and $K_{nm} \in \mathbb{R}^{n \times m}$ the matrix of kernel evaluations $[K_{nm}]_{ij} = K(\mathbf{x}_i, \mathbf{x}_{z_j})$ between data and landmarks. Restricting the representer expansion to the landmark span, which is exactly the projection step of the Nystrom method, means searching over predictors $f = \sum_{j=1}^m \beta_j K(\mathbf{x}_{z_j}, \cdot)$, for which $f(\mathbf{x}_i) = [K_{nm}\beta]_i$ and $\|f\|_{\mathcal{H}}^2 = \beta^\top K_{mm}\beta$. Kernel ridge regression over this span is

$$\min_{\beta \in \mathbb{R}^m} \frac{1}{n} \|K_{nm}\beta - \mathbf{y}\|_2^2 + \lambda \beta^\top K_{mm}\beta,$$

and setting the gradient to zero gives the $m \times m$ linear system

$$H\beta = \mathbf{v}, \quad H = \frac{1}{n} K_{nm}^\top K_{nm} + \lambda K_{mm}, \quad \mathbf{v} = \frac{1}{n} K_{nm}^\top \mathbf{y}.$$

Solving it directly costs $O(nm^2)$ to form $K_{nm}^\top K_{nm}$ plus $O(m^3)$ for the factorization, and the first term is the killer: at $m \sim \sqrt{n}$ it is $O(n^2)$. Conjugate gradient avoids forming H at all. Each iteration needs one product $H\mathbf{w}$, computed as $\frac{1}{n} K_{nm}^\top (K_{nm}\mathbf{w}) + \lambda K_{mm}\mathbf{w}$: two streaming passes over the data at $O(nm)$ arithmetic, with only $O(m^2)$ memory resident if K_{nm} is generated in blocks and discarded. The question becomes how many iterations are needed, and here the classical convergence theory of conjugate gradient gives the bound $O(\sqrt{\kappa(H)} \log(1/\varepsilon))$ iterations for relative accuracy ε , with $\kappa(H)$ the condition number. That number is bad by construction. The top of H 's spectrum is set by the data term $\frac{1}{n} K_{nm}^\top K_{nm}$; the bottom is set by λ times the tail of the landmark spectrum, and λ is statistically small, of order $1/\sqrt{n}$ or smaller, precisely because we want to fit finely. In the worked example below, a system with $\kappa(K_{mm}) = 2.26 \times 10^4$ turns into $\kappa(H) = 4.34 \times 10^8$ once the ridge drops to 10^{-5} . Plain conjugate gradient on such a system spends hundreds of $O(nm)$ passes, and the wall is rebuilt in iteration count.

38.7.2 A preconditioner from the landmarks alone

Preconditioning is the standard cure: instead of $H\beta = \mathbf{v}$, solve the transformed system

$$(B^\top H B) \mathbf{u} = B^\top \mathbf{v}, \quad \beta = B\mathbf{u},$$

for an invertible matrix B chosen to make $\kappa(B^\top H B)$ small. The perfect choice would come from factoring H itself, which is circular: it costs what the solve costs. FALKON's observation is that the landmarks already carry a faithful miniature of H . If the m landmarks *were* the entire dataset, the matrix H would be

$$H_{mm} = \frac{1}{m} K_{mm}^2 + \lambda K_{mm},$$

the same object with the data average $\frac{1}{n} K_{nm}^\top K_{nm}$ replaced by the landmark average $\frac{1}{m} K_{mm}^2$. This miniature can be factored exactly with two Cholesky decompositions of $m \times m$ matrices: writing $K_{mm} = T^\top T$ with T upper triangular, and then $\frac{1}{m} T T^\top + \lambda I = A^\top A$ with A upper triangular,

$$H_{mm} = T^\top \left(\frac{1}{m} T T^\top + \lambda I \right) T = (AT)^\top (AT),$$

so the choice

$$B = T^{-1} A^{-1}$$

gives $B^\top H_{mm} B = I$ exactly. Applied to the true H , the same B gives $B^\top H B = I + E$, where E is controlled by the discrepancy between the data average and the landmark average of the rank-one matrices $\mathbf{f}_Z(\mathbf{x})\mathbf{f}_Z(\mathbf{x})^\top$: a Monte Carlo error over the landmark draw, small as soon as m is modestly large, and entirely independent of how large n is. The preconditioner costs $O(m^3)$ once, never touches the data, and is applied inside each iteration by two triangular solves at $O(m^2)$.

Proposition (the preconditioned system is well conditioned, Rudi, Carratino, and Rosasco, 2017)

Let the landmarks be drawn uniformly from the data and let B be the FALKON preconditioner above. There are constants $c_0, c_1, c_2 > 0$, depending only on the kernel bound, such that for any $\delta \in (0, 1)$, if

$$m \geq c_0 \frac{1}{\lambda} \log \frac{n}{\delta},$$

then with probability at least $1 - \delta$ all eigenvalues of $B^\top H B$ lie in $[c_1, c_2]$. The condition number of the preconditioned system is then bounded by a constant independent of n and λ , and conjugate gradient reaches relative accuracy ε in $O(\log(1/\varepsilon))$ iterations. At the statistically motivated scaling $\lambda \sim n^{-1/2}$, the requirement reads $m \gtrsim \sqrt{n \log n}$, and refinements replace the $1/\lambda$ by the effective dimension $d_{\text{eff}}(\lambda)$ when the landmarks are drawn by leverage.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Algorithm (FALKON)

Input data $(\mathbf{x}_i, y_i)_{i=1}^n$, kernel K , landmark count m , ridge λ , iteration budget t .

Output coefficients $\beta \in \mathbb{R}^m$ of the predictor $f(\mathbf{x}) = \sum_{j=1}^m \beta_j K(\mathbf{x}_{z_j}, \mathbf{x})$.

1. Sample m landmarks uniformly from the data (or by ridge leverage scores).
2. Factor $K_{mm} = T^\top T$ and $\frac{1}{m} T T^\top + \lambda I = A^\top A$, two Cholesky factorizations of $m \times m$ matrices; set $B = T^{-1} A^{-1}$, applied only through triangular solves.
3. Run conjugate gradient on $B^\top H B \mathbf{u} = B^\top \mathbf{v}$, computing each product $H \mathbf{w}$ as $\frac{1}{n} K_{nm}^\top (K_{nm} \mathbf{w}) + \lambda K_{mm} \mathbf{w}$ over streaming blocks of K_{nm} , never storing the full matrix.
4. Stop after t iterations (or at a target residual); return $\beta = B \mathbf{u}$.

The budget now assembles into the headline. The preconditioner costs $O(m^3)$ once; each of the t iterations costs $O(nm)$ for the streamed kernel products plus $O(m^2)$ for the triangular solves; memory holds $O(m^2)$ floats. With $m \sim \sqrt{n}$ landmarks and $t = O(\log n)$ iterations, the total is $\tilde{O}(n\sqrt{n})$ time and $O(n)$ memory, against $O(n^3)$ and $O(n^2)$ for the exact solve, and against $O(n^2)$ time for Nystrom with a direct factorization. The iteration count is the quantity to watch, and the worked example makes it concrete.

Example (the FALKON preconditioner collapses the iteration count)

$n = 1000$ points drawn uniformly on $[0, 4]$, responses $y = \sin(3x)$ plus Gaussian noise of standard deviation 0.1; Laplacian kernel $K(x, x') = e^{-|x-x'|/\sigma}$ with $\sigma = 0.5$; $m = 100$ landmarks sampled uniformly from the data; ridge $\lambda = 10^{-5}$; conjugate gradient stopped at relative residual 10^{-6} . All numbers from `checks/ch13-ex2.py`.

1. Form the landmark system. $H = \frac{1}{n}K_{nm}^\top K_{nm} + \lambda K_{mm}$ is 100×100 . The landmark kernel matrix is tame, $\kappa(K_{mm}) = 2.26 \times 10^4$, but the small ridge exposes the decaying spectrum of the data term: $\kappa(H) = 4.34 \times 10^8$, four orders of magnitude worse.
2. Run plain conjugate gradient. It takes 751 iterations to reach the residual 9.4×10^{-7} , each iteration a full $O(nm)$ pass through the kernel products. In exact arithmetic CG on a 100-dimensional system must terminate within 100 steps; needing seven and a half times that is ill-conditioning made visible, round-off destroying the conjugacy the guarantee rests on.
3. Build the preconditioner. Two Cholesky factorizations of 100×100 matrices give T, A , and $B = T^{-1}A^{-1}$, at a cost independent of n . The transformed operator $M = B^\top HB$ has $\kappa(M) = 2.44 \times 10^1$: seven orders of magnitude below $\kappa(H)$.
4. Run preconditioned conjugate gradient. It reaches residual 5.6×10^{-7} in 25 iterations, a $30.0\times$ reduction, each iteration costing the same $O(nm)$ as before plus two cheap triangular solves. The two runs return the same predictor: their in-sample predictions agree to 7.2×10^{-4} in relative norm, and both fits have RMSE 0.033 against the noiseless $\sin(3x)$, a third of the noise level.

Reading. The preconditioner spends $O(m^3)$ arithmetic once, on matrices that never see n , to make every subsequent $O(nm)$ iteration count: the iteration budget now reflects $\kappa(M) = 24.4$, which the proposition above pins at a constant as n grows, rather than $\kappa(H)$, which deteriorates as the ridge shrinks. This is the entire engineering content of FALKON: iterate on the big system, but let the small system decide the geometry of the iteration.

Verification artifact. `checks/example-ch13-example-37-2.json` records the example source hash and verification scope.

38.7.3 Optimal rates: approximation as regularization

Speed would mean little if the landmark restriction degraded what can be learned. The statistical story, worked out two years before FALKON itself, says it does not, and the way it says so reframes approximation entirely. The question is how the excess risk of Nystrom kernel ridge regression depends on m , and the answer of Rudi, Camoriano, and Rosasco (2015) is that beyond a threshold, it does not.

Theorem (optimal-rate Nystrom, Rudi, Camoriano, and Rosasco, 2015)

In the standard learning setting for kernel ridge regression (bounded kernel, a target function in $\mathcal{H}$, the basic source condition), let the Nystrom subspace be built from m landmarks drawn uniformly from the data, and choose $\lambda \sim n^{-1/2}$. There is a constant c such that if

$$m \geq c \sqrt{n} \log n,$$

then with high probability the Nystrom estimator's excess risk is $O(n^{-1/2})$, the minimax-optimal rate attained by exact kernel ridge regression on this class. Under refined capacity assumptions the threshold improves to the order of the effective dimension $d_{\text{eff}}(\lambda)$, with leverage-score sampling reaching it.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The title of that paper, less is more, is the right slogan. Subsampling is not a corruption to be minimized but a form of regularization: below the threshold the approximation error contributes to the risk, above it the statistical error dominates and further landmarks buy nothing. The landmark count should track the effective dimension of the problem at the chosen λ , not the dataset size, and computation saved past the threshold is free in statistical terms, a phenomenon the authors call computational regularization. FALKON is what happens when this statistical threshold meets the solver of the previous subsection: with $m \sim \sqrt{n}$ landmarks mandated by the statistics and $O(\log n)$ iterations delivered by the preconditioner, the full pipeline achieves the optimal rate at a strictly subquadratic budget (Rudi, Carratino, and Rosasco, 2017).

Theorem (FALKON, Rudi, Carratino, and Rosasco, 2017)

In the same setting, run FALKON with $m = \Theta(\sqrt{n} \log n)$ uniform landmarks, $\lambda \sim n^{-1/2}$, and $t = O(\log n)$ iterations of preconditioned conjugate gradient. With high probability the returned predictor has excess risk $O(n^{-1/2})$, matching exact kernel ridge regression, at total cost

$$\tilde{O}(n\sqrt{n}) \text{ time} \quad \text{and} \quad \tilde{O}(n) \text{ memory.}$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The gap between this theory and hardware is closed by engineering, and it is worth recording how far that goes. Meanti, Carratino, Rosasco, and Rudi (2020) reimplemented FALKON for multi-GPU machines: the two Cholesky factorizations run out-of-core in blocks so that m can exceed single-device memory, the K_{nm} products are fused and streamed across GPUs, and most of the arithmetic runs in single precision with the factorizations kept in double where conditioning demands it. The resulting solver handles datasets with n in the billions on a single multi-GPU server, and put kernel machines back on the same large benchmarks as deep networks, at competitive accuracy and training time. The interlude taught that at scale one stops solving and starts iterating; FALKON sharpens the lesson: keep a solve, but make it $m \times m$, and let it steer the iteration.

38.8 Random Fourier features

Nystrom builds its feature map from the data. Random Fourier features (Rahimi and Recht, 2007) build one from the kernel alone, sampling it out of the kernel's own Fourier transform, independently of any training set. The construction applies to shift-invariant kernels and, like Nystrom, ends by turning the kernel machine into a linear model; but it removes the cost differently, and it leaves the RKHS behind.

38.8.1 From Bochner's theorem to a random feature

Assume the kernel is real-valued, continuous, positive definite, and translation invariant, $K(\mathbf{x}, \mathbf{y}) = \kappa(\mathbf{x} - \mathbf{y})$ with $\kappa : \mathbb{R}^d \rightarrow \mathbb{R}$, and normalized so $\kappa(0) = 1$. Bochner's theorem then says that κ is the characteristic function of a probability measure: there is a density q on $\mathbb{R}^d$ with

$$\kappa(\mathbf{z}) = \mathbb{E}_{\mathbf{w} \sim q} [e^{i\mathbf{w}^\top \mathbf{z}}].$$

Concretely, under mild assumptions κ is the inverse Fourier transform of its transform $\hat{\kappa}$,

$$\kappa(\mathbf{x} - \mathbf{y}) = \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} \hat{\kappa}(\mathbf{w}) e^{i\mathbf{w}^\top \mathbf{x}} e^{-i\mathbf{w}^\top \mathbf{y}} d\mathbf{w},$$

and the density is $q(\mathbf{w}) = \frac{1}{(2\pi)^d} \hat{\kappa}(\mathbf{w})$. It is a genuine probability density: nonnegative because K is p.d., real-valued because κ is even and real, and integrating to 1 because $\kappa(0) = 1$.

It is worth pausing on what Bochner's theorem has bought us, because the whole method rests on it. The kernel, an object that seemed to require comparing two points through an inner product in some vast feature space, is now written as a plain average over random frequencies $\mathbf{w}$: each frequency contributes a simple oscillation $e^{i\mathbf{w}^\top (\mathbf{x}-\mathbf{y})}$, and the kernel is what those oscillations average out to under q . The spread of q is set by the kernel's bandwidth: a wide, smooth kernel corresponds to low frequencies, a narrow, wiggly kernel to high ones. Approximating the kernel therefore reduces to a task we know how to cost: draw a manageable number of frequencies from q and average their contributions. That is the maneuver that will remove the $O(n^2)$ and $O(n^3)$ costs, and unlike Nystrom it reads the frequencies off the kernel itself, never touching the data.

This integral already exhibits the kernel as an expectation of a product, $e^{i\mathbf{w}^\top \mathbf{x}} \cdot \overline{e^{i\mathbf{w}^\top \mathbf{y}}}$, which is the seed of a feature map. To get a real feature we peel off the imaginary part. Because κ is real,

$$\begin{aligned} \kappa(\mathbf{x} - \mathbf{y}) &= \int_{\mathbb{R}^d} q(\mathbf{w}) \cos(\mathbf{w}^\top \mathbf{x} - \mathbf{w}^\top \mathbf{y}) d\mathbf{w} \\ &= \int_{\mathbb{R}^d} q(\mathbf{w}) [\cos(\mathbf{w}^\top \mathbf{x}) \cos(\mathbf{w}^\top \mathbf{y}) + \sin(\mathbf{w}^\top \mathbf{x}) \sin(\mathbf{w}^\top \mathbf{y})] d\mathbf{w} \\ &= \int_{\mathbb{R}^d} \int_0^{2\pi} q(\mathbf{w}) \frac{1}{2\pi} 2 \cos(\mathbf{w}^\top \mathbf{x} + b) \cos(\mathbf{w}^\top \mathbf{y} + b) d\mathbf{w} db \\ &= \mathbb{E}_{\mathbf{w} \sim q, b \sim \mathcal{U}[0, 2\pi]} [\sqrt{2} \cos(\mathbf{w}^\top \mathbf{x} + b) \cdot \sqrt{2} \cos(\mathbf{w}^\top \mathbf{y} + b)]. \end{aligned}$$

The step from the second to the third line uses the identity $2 \cos(\alpha + b) \cos(\gamma + b) = \cos(\alpha - \gamma) + \cos(\alpha + \gamma + 2b)$ and the fact that averaging the second term over $b \sim \mathcal{U}[0, 2\pi]$ kills it, leaving $\cos(\alpha - \gamma)$; this is a short exercise. The upshot is a clean statement: the kernel is the expected product of the scalar random feature $\sqrt{2} \cos(\mathbf{w}^\top \mathbf{x} + b)$ with $\mathbf{w} \sim q$ and b uniform. A single random draw $(\mathbf{w}, b)$ gives an unbiased, if noisy, estimate of $K(\mathbf{x}, \mathbf{y})$; averaging p independent draws concentrates it.

38.8.2 The random feature map

Recipe (random Fourier features)

1. Fourier-transform the kernel: $q(\mathbf{w}) = \text{kappa_hat}(\mathbf{w}) / (2\pi)^d$.
2. Draw $\mathbf{w}_1, \dots, \mathbf{w}_p$ i.i.d. from q ,
and $b_1, \dots, b_p$ i.i.d. from $\text{Uniform}[0, 2\pi]$.
3. Define the feature map
 $\psi(\mathbf{x}) = \sqrt{2/p} (\cos(\mathbf{w}_1^\top \mathbf{x} + b_1), \dots, \cos(\mathbf{w}_p^\top \mathbf{x} + b_p))^\top$ in $\mathbb{R}^p$.

Then $\kappa(\mathbf{x} - \mathbf{y}) \approx \langle \psi(\mathbf{x}), \psi(\mathbf{y}) \rangle_{\mathbb{R}^p}$, with equality in expectation.

The dot product of these features is

$$\langle \psi(\mathbf{x}), \psi(\mathbf{y}) \rangle_{\mathbb{R}^p} = \frac{2}{p} \sum_{j=1}^p \cos(\mathbf{w}_j^\top \mathbf{x} + b_j) \cos(\mathbf{w}_j^\top \mathbf{y} + b_j) = \frac{1}{p} \sum_{j=1}^p \sqrt{2} \cos(\mathbf{w}_j^\top \mathbf{x} + b_j) \sqrt{2} \cos(\mathbf{w}_j^\top \mathbf{y} + b_j),$$

which is the empirical average of p independent copies of the random feature whose expectation is exactly $\kappa(\mathbf{x} - \mathbf{y})$. So the feature dot product is an unbiased estimator of the kernel, and the only question is how fast it concentrates as p grows.

The figure below makes that concentration visible: it plots the Gaussian kernel against its random Fourier feature estimate as a function of the gap $\mathbf{x} - \mathbf{y}$, so you can raise the feature count D and watch the finite average settle onto the exact kernel that Bochner's theorem promises it targets, with the running RMSE to the true kernel reported as you go.

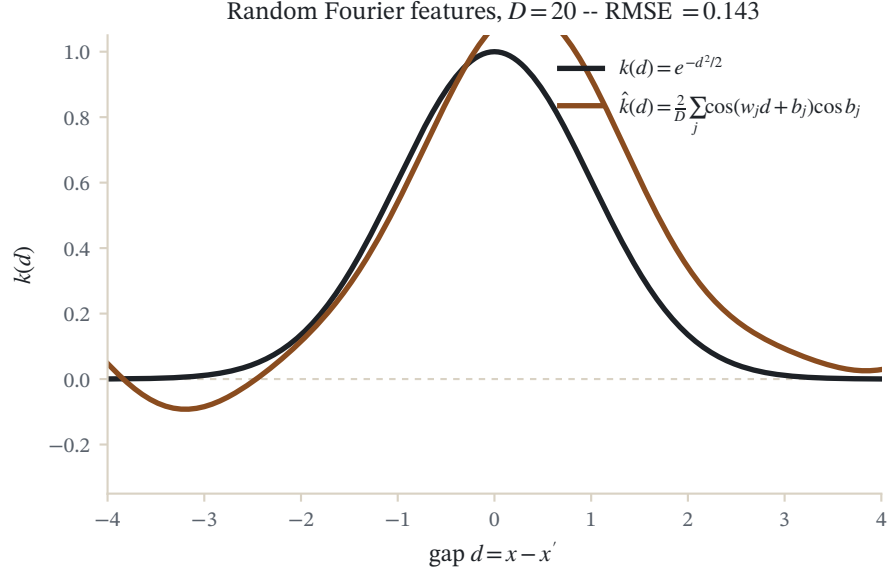


Figure 38.2: The Gaussian kernel $k(x, x') = e^{-(x-x')^2/2}$ (dark curve) against its $D = 20$ random Fourier feature estimate $\hat{k} = \frac{1}{D} \sum_j \cos(w_j x + b_j) \cos(w_j x' + b_j)$, as a function of the gap $x - x'$. Bochner's theorem makes the true kernel the average of these random features.

38.8.3 The approximation guarantee

Theorem (Rahimi and Recht, 2007)

Let $K(\mathbf{x}, \mathbf{y}) = \kappa(\mathbf{x} - \mathbf{y})$ be a shift-invariant kernel on $\mathbb{R}^d$ with $\kappa(0) = 1$, and let ψ be the random Fourier feature map above. On any compact set $\mathcal{X} \subset \mathbb{R}^d$, for every $\varepsilon > 0$,

$$\mathbb{P} \left[\sup_{\mathbf{x}, \mathbf{y} \in \mathcal{X}} |\kappa(\mathbf{x} - \mathbf{y}) - \langle \psi(\mathbf{x}), \psi(\mathbf{y}) \rangle_{\mathbb{R}^p}| \geq \varepsilon \right] \leq 2^8 \left(\frac{\sigma_q \text{diam}(\mathcal{X})}{\varepsilon} \right)^2 e^{-\frac{p\varepsilon^2}{4(d+2)}},$$

where $\sigma_q^2 = \mathbb{E}_{\mathbf{w} \sim q}[\mathbf{w}^\top \mathbf{w}]$ is the second moment of the kernel's Fourier transform.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Two features of this bound deserve emphasis. First, the guarantee is *uniform* over all pairs in $\mathcal{X}$ and is not data dependent: it holds for the kernel as a function, before any training data are seen, unlike Nyström whose quality depends on the sampled points. Second, it certifies convergence: choosing $\varepsilon_p = \sqrt{\frac{\log p}{p}} \sigma_q \text{diam}(\mathcal{X})$ makes the right-hand side tend to zero as $p \rightarrow \infty$, so $O(1/\varepsilon^2)$ features up to a log factor suffice for a uniform error ε . The one

thing lost is membership in the RKHS: a predictor built as a linear function of the random features is generally *not* an element of $\mathcal{H}$, so the RKHS operations that Nystrom preserved are no longer available.

Proof sketch

The argument is a covering-number bound with three moves. Fix a single pair $(\mathbf{x}, \mathbf{y})$ and let $f(\mathbf{x}, \mathbf{y}) = \kappa(\mathbf{x} - \mathbf{y}) - \langle \psi(\mathbf{x}), \psi(\mathbf{y}) \rangle$. Since $\langle \psi(\mathbf{x}), \psi(\mathbf{y}) \rangle$ is an average of p i.i.d. bounded terms with mean $\kappa(\mathbf{x} - \mathbf{y})$, Hoeffding's inequality gives the pointwise bound

$$\mathbb{P}[|f(\mathbf{x}, \mathbf{y})| \geq \varepsilon] \leq 2e^{-p\varepsilon^2/4}.$$

Second, because f depends only on $\mathbf{x} - \mathbf{y}$, cover the difference set $\mathcal{X}_\Delta = \{\mathbf{x} - \mathbf{y} : \mathbf{x}, \mathbf{y} \in \mathcal{X}\}$ with a net of at most $T = (4 \text{diam}(\mathcal{X})/r)^d$ balls of radius r , and apply Hoeffding at the centers together with a union bound:

$$\mathbb{P}\left[\sup_i f(\text{center}_i) \geq \frac{\varepsilon}{2}\right] \leq \sum_i \mathbb{P}\left[f(\text{center}_i) \geq \frac{\varepsilon}{2}\right] \leq 2Te^{-p\varepsilon^2/16}.$$

Third, control the variation of f inside each ball (its gradient has bounded second moment governed by σ_q) so that smallness at the centers forces smallness everywhere, and optimize the radius r to balance the covering count against the in-ball error. This last balancing is where σ_q , $\text{diam}(\mathcal{X})$, and the dimension factor $d + 2$ enter the stated constant. $\square$

38.8.4 Reduction to a linear model

The payoff mirrors Nystrom's, from a different starting point. Having built $\psi : \mathcal{X} \rightarrow \mathbb{R}^p$ with $K(\mathbf{x}, \mathbf{y}) \approx \langle \psi(\mathbf{x}), \psi(\mathbf{y}) \rangle$, any kernel machine becomes an ordinary linear model in the explicit features. Kernel ridge regression, for instance, turns into

$$\min_{\mathbf{w} \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n L(y_i, \mathbf{w}^\top \psi(\mathbf{x}_i)) + \lambda \|\mathbf{w}\|_2^2,$$

which is exactly the large finite sum of the interlude. We never form the $n \times n$ Gram matrix, never solve an $n \times n$ system: computing the features costs $O(npd)$, and fitting the linear model costs $O(np^2 + p^3)$ by a direct solve, or far less with the stochastic and incremental methods of the interlude, each of whose steps touches a single example at cost $O(p)$. With p fixed and $p \ll n$, the whole pipeline is linear in n . The $O(n^2)$ memory and $O(n^3)$ solve are gone, replaced by a controllable approximation error that shrinks as the number of random features grows. It is worth noting what sets p : the guarantee below ties it to the target kernel-approximation accuracy and the kernel's Fourier moment, not to the sample size, so a feature count chosen once can be reused as the dataset grows. This is the sharpest contrast with Nystrom, whose anchors are read off the data and may need to grow with the data's complexity.

Nystrom versus random Fourier features

Both replace the kernel by a p -dimensional feature map and reduce a kernel machine to a linear model, defeating the same $O(n^2)$ memory and $O(n^3)$ solve. They differ in where the features come from and what survives. Nystrom is data-dependent: it samples or synthesizes anchors from the data, gives a low-rank factorization $K \approx LL^\top$, and keeps the approximation inside the RKHS, so its points admit all RKHS operations. Random Fourier features are data-independent: they sample from the kernel's Fourier transform, come with a uniform, distribution-free guarantee that holds before any data are seen, and apply to shift-invariant kernels, but their predictors do not live in $\mathcal{H}$. Data-adaptive accuracy on one side, universal and label-free simplicity on the other.

38.9 Nystrom versus random features: pricing the choice

The remark above states the dichotomy; it does not yet price it. Given a fixed budget of p dimensions, which map serves the learning problem better? The honest answer depends on the spectrum of the problem, and on who is allowed to see it, and it is worth working out because the two families fail in different places.

The error currencies differ first. Nystrom's error after p landmarks is a projection residual, controlled by the tail of the Gram spectrum: with well-chosen landmarks it is roughly the eigenvalue mass beyond the top p directions, and with leverage sampling the budget that suffices for ridge regression at level λ is $d_{\text{eff}}(\lambda)$ up to logarithms. It is a data-dependent quantity: an easy dataset, one whose kernel geometry is nearly low rank, genuinely costs fewer landmarks. Random features pay in a different coin: Monte Carlo error, $O(1/\sqrt{p})$ uniformly over the domain, identical for an easy dataset and a hard one, because the construction never looks at the data at all. From this single difference the practical folklore follows. When the spectrum decays fast, a smooth kernel, inputs concentrated near a low-dimensional set, clustered data, Nystrom captures with tens of landmarks what a data-blind average still resolves noisily with thousands of features, and Nystrom wins outright. When the spectrum decays slowly, there is no thin tail for data-dependence to exploit, the two families need comparable dimensions, and the tie is broken by logistics rather than approximation: random features need no pass over the training data, can be regenerated on the fly from a random seed rather than stored, extend to a new point with no bookkeeping, and parallelize trivially, which is why they are the default in streaming and privacy-constrained settings. Nystrom, for its part, is the only option of the two for kernels with no Fourier transform to sample, such as the kernels on strings, trees, and graphs of the structured-data chapters, since it asks nothing of the kernel beyond positive definiteness.

The statistical theory refines this picture without overturning it. On the Nystrom side, the optimal-rate theorem of Rudi, Camoriano, and Rosasco (2015) puts the landmark threshold for minimax performance at $\tilde{O}(\sqrt{n})$; on the random-features side, Rudi and Rosasco (2017) prove the matching statement, that $\tilde{O}(\sqrt{n})$ random features suffice for the same $O(n^{-1/2})$ excess risk. At the level of worst-case rates the families tie. The separation is adaptivity: leverage-adapted Nystrom can ride a benign spectrum down to $d_{\text{eff}}(\lambda) \ll \sqrt{n}$ landmarks, while plain random features, blind to the data, cannot see a benign spectrum at all. The comparison in brief:

Nystrom	Random Fourier features	
Feature source	the data (sampled or learned anchors)	the kernel's Fourier transform
Kernels covered	any positive definite kernel	shift-invariant kernels
Error currency	spectral tail, $d_{\text{eff}}(\lambda)$ with leverage sampling	Monte Carlo, $O(1/\sqrt{p})$ uniformly
Guarantee	data-dependent, spectrum-adaptive	distribution-free, holds before any data
Predictor in $\mathcal{H}$	yes, a genuine projection	no
Features for optimal KRR rate	$\tilde{O}(\sqrt{n})$, down to $\tilde{O}(d_{\text{eff}})$ adaptive	$\tilde{O}(\sqrt{n})$
Wins when	spectrum decays fast; kernel non-spectral	streaming, no second pass; slow spectral decay

38.9.1 Structured and quasi-random features

Inside the random-features budget there is a second cost that the count p hides: the map itself. Vanilla random Fourier features store a dense Gaussian matrix $W \in \mathbb{R}^{p \times d}$ of frequencies and spend $O(pd)$ arithmetic per encoded point, and for high-dimensional inputs, images, spectra, text embeddings, with p in the tens of thousands, the matrix and the product dominate the pipeline. A line of work removes this cost by replacing the i.i.d. Gaussian matrix with structured randomness, and a parallel line improves the constant in the $O(1/\sqrt{p})$ error by making the frequencies better spread than chance. Both keep the Bochner integral as the target and change only how it is sampled.

Fastfood (Le, Sarlós, and Smola, 2013) replaces W by a product of diagonal and Walsh-Hadamard matrices, $V = \frac{1}{\sigma\sqrt{d}} SHG\Pi HB$, with Π a random permutation, B and S random diagonal matrices, G a diagonal of Gaussians, and H the Walsh-Hadamard transform. The point of the construction is that H needs no storage and multiplies a vector in $O(d \log d)$ by an FFT-style recursion, so the whole map costs $O(p \log d)$ time and $O(p)$ memory against $O(pd)$ and $O(pd)$: log-linear, as the name advertises. Each row of V is still marginally Gaussian, so the kernel estimate remains unbiased, and the rows' weak dependence costs only a modest factor in concentration. Orthogonal random features (Yu et al., 2016) keep the cost of the dense map but attack its variance: draw the frequency block as a uniformly random orthogonal matrix with rows rescaled by chi-distributed norms, so that each row is again exactly Gaussian, but distinct rows are exactly orthogonal rather than merely independent. The estimator of the Gaussian kernel stays unbiased, and the negative dependence between orthogonal directions cancels part of the Monte Carlo noise: the mean squared error is provably below that of i.i.d. features, with the largest gains at moderate distances $\|\mathbf{x} - \mathbf{y}\|$, and a Hadamard-based variant recovers the $O(p \log d)$ evaluation cost as well. Quasi-Monte Carlo features (Avron, Sindhvani, Yang, and Mahoney, 2016) go after the sampling itself: replace the i.i.d. draws from q by a deterministic low-discrepancy sequence, Halton or Sobol, pushed through the inverse distribution function of q , so the frequencies are equidistributed by design rather than on average. For integrands as smooth as the kernel's, the integration error improves from $O(p^{-1/2})$ toward $O(1/p)$ up to logarithmic factors, and the paper's box-discrepancy theory tailors the sequence to the kernel's spectral measure; the idea is the same trade of randomness for equidistribution that drives the kernel herding and quadrature rules of the quadrature chapter.

The pattern across all three is the pattern of the whole chapter in miniature. The Bochner integral is fixed; only our quadrature of it improves, by cheapening each sample (Fastfood), by decorrelating the samples (orthogonal features), or by spreading them evenly (quasi-Monte Carlo). That is precisely the ladder the Nystrom sections climbed on the other side, from uniform landmarks through greedy selection to leverage scores. In both families the kernel machine at the end is the same linear model; the craft, and the theory, live entirely in where the sampling budget is spent.

Further reading

The optimization machinery behind large-scale kernel machines, decomposition and working-set solvers for the support vector machine, and the day-to-day practice of training these models at scale, is developed at length by Scholkopf and Smola (2002) in *Learning with Kernels*, chapters 10 and 11, which remain a good entry point to the implementation and optimization side of the subject. That book predates the two approximation families that organize this chapter. The random-features idea was introduced later, by Rahimi and Recht (2007), and the modern statistical analysis of both Nystrom and random-feature approximations, which quantifies how few landmarks or features already match the generalization of the exact kernel estimator, belongs to a still later line of work exemplified by Rudi and Rosasco (2017). For those sharper guarantees one has to look past the textbook to the research literature.

38.10 Block Krylov methods and many right-hand sides

Conjugate gradients solves one symmetric positive-definite system without forming an inverse. Gaussian-process prediction, multi-output regression, cross-validation approximations, and uncertainty estimation often require many systems with the same matrix $A = K + n\lambda I$ but different right-hand sides. Solving each from scratch discards shared spectral information. Block conjugate gradients advances a matrix of residuals at once and builds a block Krylov space

$$\mathcal{K}_t(A, B) = \text{span}\{B, AB, \dots, A^{t-1}B\}.$$

This can amortize kernel products and exploit matrix-matrix hardware, but nearly dependent right-hand sides cause rank loss. Deflate converged directions and monitor the residual of every column. A block method's iteration count is not directly comparable to scalar CG unless kernel-product count, block width, memory traffic, and achieved accuracy are all reported.

Preconditioners must be applied consistently across right-hand sides. A Nystrom preconditioner captures the leading spectrum; diagonal or block-diagonal terms handle local scale; flexible Krylov variants are needed if the preconditioner itself changes during the solve.

38.11 Fast kernel products without global low rank

Low rank is not the only structure. Translation-invariant kernels on a grid admit convolution and FFT-based products. Separable kernels on Cartesian products yield Kronecker factors. Compactly supported kernels yield sparse Gram matrices. Hierarchical and fast-multipole constructions compress well-separated blocks while retaining fine local interactions. KeOps-style symbolic reductions stream pairwise computations without storing the full matrix.

The structure must match the data rather than the desired algorithm. Snapping irregular observations to a grid changes the model; forcing a globally low-rank approximation on a short-length-scale kernel can erase local behavior. Report approximation error at the level the downstream method uses: residual and prediction error for linear solves, log-determinant error for GP evidence, and decision disagreement for classifiers.

38.12 Mixed precision and reliable stopping

Kernel workloads are often bandwidth-bound, so half or single precision can substantially improve throughput. The danger is that ill-conditioning turns small arithmetic errors into large coefficient errors. A reliable pattern is mixed-precision iterative refinement: compute products and a provisional solve in lower precision, form the residual in higher precision, solve for a correction, and repeat. This succeeds only when the low-precision system is accurate enough for refinement to contract.

Never stop on iteration count alone. For $A\alpha = y$, report the relative residual $\|y - A\alpha\|_2 / \|y\|_2$, a preconditioned residual when appropriate, and a task-level quantity such as validation loss. A tiny residual does not guarantee a small prediction error under model misspecification, while a moderate coefficient error may be harmless if it lies in weak kernel directions.

Algorithm (precision-aware matrix-free KRR)

1. Estimate diagonal scale and condition difficulty using a small spectral probe.
2. Choose the kernel-product backend and preconditioner from data structure, not only sample count.
3. Run preconditioned CG in the fastest precision that passes a residual-contraction pilot.
4. Recompute residuals periodically in higher precision and restart if recursive and true residuals diverge.
5. Validate predictions against a small high-precision reference problem.
6. Record kernel products, bytes moved, wall time, peak memory, precision, residuals, and hardware.

38.13 Distributed, federated, and streaming kernels

Distributed Nystrom separates landmark selection, local cross-kernel computation, and a global reduced solve. Distributed random features broadcast the feature-map seed and aggregate sufficient statistics. Communication can dominate arithmetic, so scaling plots must include network volume and synchronization, not just per-worker compute. Reproducibility requires deterministic feature seeds and a stable reduction order when bitwise agreement matters.

Federated settings add privacy and heterogeneity. Sharing random-feature sufficient statistics can avoid centralizing raw examples, but those statistics can still leak information and differential-privacy noise changes both optimization and statistical error. Non-identically distributed clients also make a single global bandwidth questionable. Report client sampling, local preprocessing, privacy accounting, and per-client performance.

Streaming kernels face a growing dictionary. Budget maintenance may merge, project, or evict basis elements using novelty, leverage, age, or coefficient magnitude. The budget rule is part of the model: it determines which geometry survives concept drift. Evaluate prequential loss, update latency, memory, drift recovery, and retained-dictionary diversity. The online algorithms in Chapter 10, Online Kernel Learning supply the update rules; this chapter supplies the systems constraints that make them usable at scale.

38.14 Common mistakes and practical implications

For **Large-Scale Kernel Machines**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

38.15 Summary and further reading

This chapter established explain the central definitions and claims in Large-Scale Kernel Machines; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Rahimi and Recht 2007), (Williams and Seeger 2001), (Drineas and Mahoney 2005).

38.16 Exercises

1. warm-up The chapter isolates three separate costs of the exact kernel solve: $O(n^2)$ kernel evaluations to build the Gram matrix, $O(n^2)$ memory to store it, and $O(n^3)$ arithmetic to solve $(K + n\lambda I)\alpha = \mathbf{y}$. For each method below, say which of the three costs it removes and which it leaves standing: (a) replacing the direct solve by conjugate gradients while keeping K exact; (b) sequential minimal optimization and decomposition for the SVM; (c) Nystrom with $p \ll n$ sampled anchors; (d) random Fourier features with p frequencies. One sentence per method.
2. computation Work in double precision, 8 bytes per float. (a) At $n = 10^5$, how many entries does the Gram matrix K hold, and how many gigabytes does it occupy? (b) A dense Cholesky solve costs about $\frac{1}{3}n^3$ floating-point operations; evaluate that at $n = 10^5$. (c) Approximate instead by Nystrom with $p = 500$: give the size in gigabytes of the stored feature matrix $\psi(S) \in \mathbb{R}^{p \times n}$, and the order of magnitude of the training budget $O(p^3) + O(p^2n)$. (d) By what factor does the storage shrink, and at what value of n would K in double precision first exceed 8 gigabytes? Hint

The matrix has n^2 entries, so its size in bytes is $8n^2$; a gigabyte is 10^9 bytes. For (d) the storage ratio is $8n^2/(8pn) = n/p$, and solving $8n^2 > 8 \times 10^9$ gives the crossover n .

3. proof Show where the $O(n^3)$ comes from. Solve the symmetric positive definite system $A\alpha = \mathbf{y}$ with $A = K + n\lambda I$ by the Cholesky factorization $A = LL^\top$, L lower triangular. (a) From the defining relation $A_{ij} = \sum_{k=1}^j L_{ik}L_{jk}$ for $i \geq j$, show that computing L_{ij} requires a dot product of length j , so forming all of column j below the diagonal costs on the order of $(n-j)j$ multiply-add operations. (b) Sum over columns to get $\sum_{j=1}^n (n-j)j \sim n^3/6$, hence a $\Theta(n^3)$ factorization, and note that the two triangular solves that finish the job cost only $\Theta(n^2)$. Conclude that the cube is the price of eliminating n unknowns against n equations, not of the right-hand side. Hint

Approximate the sum by an integral: $\sum_{j=1}^n (n-j)j \approx \int_0^n (n-t)t dt = \frac{n^3}{2} - \frac{n^3}{3} = \frac{n^3}{6}$. An iterative solver such as conjugate gradients avoids the factorization but still spends $\Theta(n^2)$ per matrix-vector product, so it lowers the arithmetic wall without touching the memory wall.

4. proof Show that the random Fourier feature dot product is an unbiased estimator of the kernel. Let $\psi(\mathbf{x}) = \sqrt{2/p} (\cos(\mathbf{w}_1^\top \mathbf{x} + b_1), \dots, \cos(\mathbf{w}_p^\top \mathbf{x} + b_p))^\top$ with $\mathbf{w}_j \sim q$ i.i.d. and $b_j \sim \mathcal{U}[0, 2\pi]$ i.i.d. (a) Using the identity $2 \cos(\alpha + b) \cos(\gamma + b) = \cos(\alpha - \gamma) + \cos(\alpha + \gamma + 2b)$, show that averaging over $b \sim \mathcal{U}[0, 2\pi]$ annihilates the second term, so $\mathbb{E}_b[2 \cos(\mathbf{w}^\top \mathbf{x} + b) \cos(\mathbf{w}^\top \mathbf{y} + b)] = \cos(\mathbf{w}^\top (\mathbf{x} - \mathbf{y}))$. (b) Take the further expectation over $\mathbf{w} \sim q$ and invoke Bochner's theorem to conclude $\mathbb{E}[\langle \psi(\mathbf{x}), \psi(\mathbf{y}) \rangle] = \kappa(\mathbf{x} - \mathbf{y})$. (c) For the Gaussian kernel $\kappa(\mathbf{z}) = e^{-\|\mathbf{z}\|_2^2/(2\sigma^2)}$, name the sampling density q explicitly. Hint

For (a), $\int_0^{2\pi} \cos(\alpha + \gamma + 2b) \frac{db}{2\pi} = 0$. For (b), $\mathbb{E}_{\mathbf{w} \sim q}[\cos(\mathbf{w}^\top \mathbf{z})] = \text{Re } \mathbb{E}_{\mathbf{w}}[e^{i\mathbf{w}^\top \mathbf{z}}] = \kappa(\mathbf{z})$ since κ is real. Each summand of $\langle \psi(\mathbf{x}), \psi(\mathbf{y}) \rangle$ is an i.i.d. copy of the same unbiased term, so the average is unbiased too. For (c), the Fourier transform of a Gaussian is a Gaussian: $q = \mathcal{N}(\mathbf{0}, \sigma^{-2}I)$.

5. proof Show that the Nystrom low-rank error equals the discarded eigenvalue tail. Let the centered Gram matrix have eigendecomposition $K = \sum_{k=1}^n \lambda_k \mathbf{u}_k \mathbf{u}_k^\top$ with $\lambda_1 \geq \dots \geq \lambda_n \geq 0$, and let $\Pi_{\mathcal{F}}$ be the orthogonal projection onto the span of the top p kernel principal directions. (a) Show the total squared projection residual satisfies $\sum_{i=1}^n \|\varphi(\mathbf{x}_i) - \Pi_{\mathcal{F}} \varphi(\mathbf{x}_i)\|_{\mathcal{H}}^2 = \sum_{k=p+1}^n \lambda_k$. (b) Deduce that the approximation is exact when $\text{rank}(K) \leq p$, and that in general its error is controlled entirely by the tail $\sum_{k>p} \lambda_k$, which is small precisely when the spectrum decays fast, the premise the chapter invokes for Nystrom. Hint

The total feature energy is $\sum_i \|\varphi(\mathbf{x}_i)\|_{\mathcal{H}}^2 = \text{tr}(K) = \sum_k \lambda_k$. Projecting onto the top p eigendirections retains exactly $\sum_{k=1}^p \lambda_k$ of it, and since projection is orthogonal, Pythagoras gives residual = total – retained = $\sum_{k=p+1}^n \lambda_k$.

6. computation The sparse greedy Gram approximation of Smola and Scholkopf (2000) chooses the next anchor not from all n candidates but from a small random pool, and the chapter quotes the number 59. Justify it. Suppose a pool of r candidates is drawn uniformly at random. (a) Show the probability that none of the r lands in the top 5% of all candidates is $(0.95)^r$. (b) Solve $(0.95)^r \leq 0.05$ for the smallest integer r and confirm $r = 59$. (c) Explain why this guarantee is independent of n , and what that independence buys for the per-iteration cost of greedy selection.
7. exploration Open the interactive figure above in the random Fourier features section. It plots the Gaussian kernel $\kappa(\mathbf{x} - \mathbf{y})$ against its random-feature estimate as a function of the gap $\mathbf{x} - \mathbf{y}$, and reports the running RMSE to the true kernel. (a) Start at a small feature count D : the estimate is jagged and the RMSE is large. Raise D and watch the finite average settle onto the smooth exact kernel while the RMSE falls. (b) The estimate is an average of D i.i.d. bounded terms, so from that fact alone, what rate should the RMSE follow in D , and by what factor must you raise D to halve the error? (c) Redraw at fixed D : the curve wiggles differently because the frequencies are resampled. What does that resampling illustrate about the gap between the pointwise variance of a single estimate and the uniform, distribution-free guarantee of the Rahimi and Recht theorem?
8. challenge Trace the memory-versus-compute trade behind variance reduction. The chapter argues SGD's noise never dies. (a) At the optimum $\mathbf{w}^*$, the full gradient $\frac{1}{n} \sum_i \nabla f_i(\mathbf{w}^*) = 0$; construct an $n = 2$ example in which the individual gradients $\nabla f_i(\mathbf{w}^*)$ are nonzero yet cancel, and conclude that the one-sample estimator $\nabla f_I(\mathbf{w}^*)$ has strictly positive variance. (b) Explain why that irreducible variance forces SGD to send $\eta_t \rightarrow 0$, which is what drags its strongly convex rate down to $O(1/t)$, whereas a variance-reduced estimate whose variance vanishes as $\mathbf{w} \rightarrow \mathbf{w}^*$ permits a constant step and the geometric rate to return. (c) SAG restores that fast rate by storing one gradient per example, while SVRG spends periodic full-batch recomputation instead. State the resource each pays, and compute the size in gigabytes of SAG's gradient table for $n = 10^6$, $p = 10^3$, in double precision. Hint

For (a) take $f_1(w) = \frac{1}{2}(w-1)^2$, $f_2(w) = \frac{1}{2}(w+1)^2$; the average is minimized at $w^\star = 0$ where $\nabla f_1 = -1$, $\nabla f_2 = +1$, so a random draw returns ± 1 , variance $1 \neq 0$. For (c) SAG's table is $n \times p$ floats, so $10^6 \times 10^3 \times 8$ bytes; SVRG trades that memory for the compute of recomputing a full gradient every so often.

9. proof Establish the basic identities of ridge leverage scores. Let $K = \sum_{k=1}^n \lambda_k \mathbf{u}_k \mathbf{u}_k^\top$ with orthonormal eigenvectors and $\lambda > 0$. (a) Show that $\ell_i(\lambda) = \sum_k \frac{\lambda_k}{\lambda_k + \lambda} u_{ki}^2$, and deduce $0 \leq \ell_i(\lambda) < 1$ for every i . (b) Show that $\sum_i \ell_i(\lambda) = d_{\text{eff}}(\lambda) = \sum_k \frac{\lambda_k}{\lambda_k + \lambda}$. (c) Determine the limits of $d_{\text{eff}}(\lambda)$ as $\lambda \rightarrow 0^+$ and as $\lambda \rightarrow \infty$, and check both against the worked example, whose spectrum $(4.7879, 1, 1, 0.2087, 0.0034, 0, 0)$ has five nonzero eigenvalues and whose effective dimension at $\lambda = 0.01$ is 4.1861. Hint

For (a), $K(K + \lambda I)^{-1}$ has the same eigenvectors as K with eigenvalues $\lambda_k/(\lambda_k + \lambda) \in [0, 1]$, and the rows of an orthogonal matrix have unit norm, $\sum_k u_{ki}^2 = 1$. For (c), each term $\lambda_k/(\lambda_k + \lambda)$ tends to 1 if $\lambda_k > 0$ and to 0 always as $\lambda \rightarrow \infty$, so $d_{\text{eff}} \rightarrow \text{rank}(K) = 5$ and $d_{\text{eff}} \rightarrow 0$ respectively; 4.1861 at $\lambda = 0.01$ sits below 5 because the eigenvalue 0.0034 is already mostly crushed by the ridge, contributing only $0.0034/0.0134 \approx 0.25$.

10. proof Show that the FALKON preconditioner is exact when the landmarks are the whole dataset. Let $K_{mm} = T^\top T$ and $\frac{1}{m} T T^\top + \lambda I = A^\top A$ be the two Cholesky factorizations, and $B = T^{-1} A^{-1}$. (a) Verify the identity $\frac{1}{m} K_{mm}^2 + \lambda K_{mm} = (AT)^\top (AT)$. (b) Suppose $m = n$ and the landmark set is the entire dataset, so $K_{nm} = K_{mm}$. Show that $H = \frac{1}{n} K_{nm}^\top K_{nm} + \lambda K_{mm}$ equals $\frac{1}{m} K_{mm}^2 + \lambda K_{mm}$, and conclude $B^\top H B = I$, so preconditioned conjugate gradient converges in a single iteration. (c) For $m < n$, write $B^\top H B = I + E$ and identify the matrix whose deviation from its landmark average E measures. What kind of convergence, in m , controls the size of E ? Hint

For (a), factor $K_{mm} = T^\top T$ out on both sides: $\frac{1}{m} K_{mm}^2 + \lambda K_{mm} = T^\top (\frac{1}{m} T T^\top + \lambda I) T = T^\top A^\top A T$. For (b), substitute and cancel: $B^\top (AT)^\top (AT) B = A^{-\top} T^{-\top} T^\top A^\top A T T^{-1} A^{-1} = I$; CG on the identity solves in one step. For (c), $E = B^\top (\frac{1}{n} K_{nm}^\top K_{nm} - \frac{1}{m} K_{mm}^2) B$: the data average of the rank-one matrices $\mathbf{f}_Z(\mathbf{x}_i) \mathbf{f}_Z(\mathbf{x}_i)^\top$ minus their landmark average, a Monte Carlo deviation over the landmark draw that concentrates as m grows, independently of n .

11. computation Test the conjugate gradient iteration bound against the FALKON worked example. The classical bound says CG reaches relative accuracy ε within about $\frac{1}{2} \sqrt{\kappa} \ln(2/\varepsilon)$ iterations on a system with condition number κ . The worked example reports $\kappa(H) = 4.34 \times 10^8$, $\kappa(M) = 2.44 \times 10^1$, tolerance 10^{-6} , and measured iteration counts 751 and 25. (a) Evaluate the bound for the preconditioned system M and compare with the measured 25. (b) Evaluate it for H and observe that the prediction is around two hundred times the measured 751: explain the gap through the shape of the spectrum, recalling that CG adapts to all the eigenvalues, effectively deleting a few extreme ones after a few iterations, while κ sees only the two endpoints. (c) The measured 751 exceeds the dimension $m = 100$, yet exact-arithmetic CG must terminate in at most m steps. What does the overshoot demonstrate about finite-precision CG on ill-conditioned systems? Hint

With $\ln(2/\varepsilon) = \ln(2 \times 10^6) \approx 14.5$: for M , $\frac{1}{2} \sqrt{24.4 \times 14.5} \approx 36$, the right order against the measured 25; for H , $\frac{1}{2} \sqrt{4.34 \times 10^8 \times 14.5} \approx 1.5 \times 10^5$, wildly pessimistic because the bound is tight only for spectra spread evenly between the extremes. For (c), round-off destroys the exact conjugacy of the search directions, so the finite-termination property fails precisely when conditioning is poor; the preconditioned run, at $25 < 100$, sits safely inside the regime where the theory works.

39 Multiple Kernel Learning

Every method in this book so far has taken the kernel as given. We fixed a positive definite kernel K , formed its Gram matrix, and let a representer argument reduce learning to linear algebra on that matrix. But where does K come from? In practice we almost never have one obviously correct kernel. We have a menu: linear, polynomial, and Gaussian kernels with a range of bandwidths on the same feature vectors, or genuinely different kernels built from genuinely different views of the same objects, a protein described at once by its sequence, its three-dimensional structure, and its mass spectrometry profile. This chapter is about what to do with such a menu. The simplest answer, adding the kernels together, turns out to be exactly concatenating their feature spaces, and it already gives a principled way to fuse heterogeneous data. The more ambitious answer is to let the data choose. We will see that the value of the learning problem is a convex function of the kernel, so we can search a convex set of candidate kernels by convex optimization. Learning a convex combination $K_\eta = \sum_i \eta_i K_i$ jointly with the classifier is Multiple Kernel Learning (Lanckriet et al., 2004); the problem is jointly convex in the weights and the classifier, and, read correctly, it is a sparsity-inducing group lasso in disguise.

39.1 Choosing or combining kernels

We have seen how to build a learning algorithm once we are handed a kernel K on a data space $\mathcal{X}$. The kernel is the entire interface between the data and the algorithm: it is where our knowledge about similarity between objects lives. This makes the choice of kernel the single most consequential modeling decision, and it is rarely a decision with one clear answer.

Two situations recur. In the first, we have a single description of the data but a whole family of candidate kernels on it, obtained by varying the kernel type or its parameters: a linear kernel, polynomial kernels of several degrees, Gaussian kernels across a range of bandwidths. Which bandwidth is right is exactly the kind of question the data should answer, not the modeler. In the second situation we have several genuinely different descriptions of the same objects, different views. A protein can be characterized by its amino acid sequence, by its folded structure, by its mass spectrometry profile, by which other proteins it interacts with. Each view comes with its own natural kernel, and each captures partial, complementary information. The question in both cases is the same: how do we choose among, or integrate, several kernels in a single learning task?

It helps to feel the difficulty before proposing a cure. A single Gaussian kernel already forces one hard choice, its bandwidth, and that choice alone can swing a fit from useless to excellent. A narrow bandwidth makes every point its own island, so the fitted function spikes at the data and forgets it everywhere in between; a wide bandwidth blurs neighboring points together, so the fit goes flat and misses real structure. There is usually no single setting that is right across every region of every problem, which is exactly why we might want several bandwidths available at once rather than one chosen in advance. The figure below lets you move the bandwidth of a single kernel and watch the same data support wildly different curves; Multiple Kernel Learning is the formalization of the wish that follows, learning a convex combination $\sum_i \eta_i K_i$ of several such kernels at once and letting the data set the mixing weights.

39.1.1 The setting: learning with one kernel

To talk about combining kernels we first fix notation for learning with one. Throughout, for a function $f : \mathcal{X} \rightarrow \mathbb{R}$ and data points $\mathbf{x}_1, \dots, \mathbf{x}_n$, we write

$$\mathbf{f}_n = (f(\mathbf{x}_1), \dots, f(\mathbf{x}_n)) \in \mathbb{R}^n$$

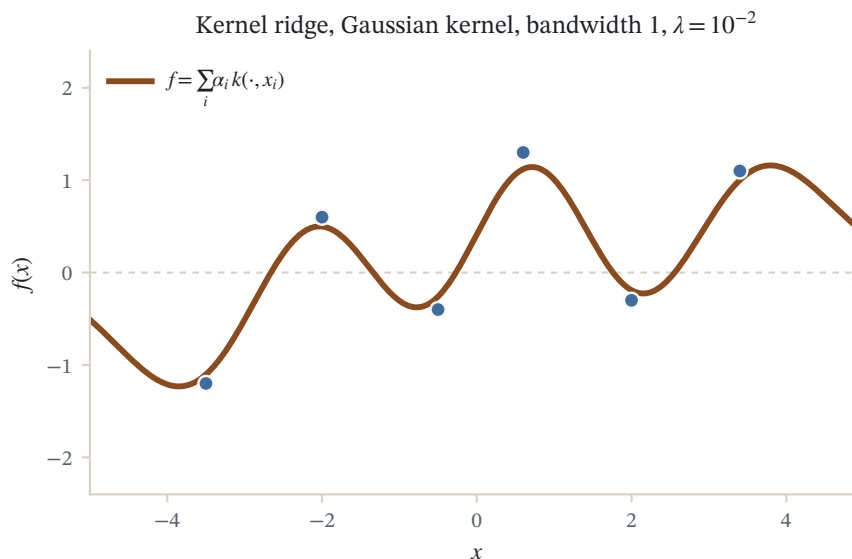


Figure 39.1: Kernel ridge regression on six points with a Gaussian kernel (bandwidth 1, ridge $\lambda = 10^{-2}$). The curve is the exact solution $f = \sum_i \alpha_i k(\cdot, x_i)$ with $\alpha = (K + \lambda nI)^{-1}y$.

for the vector of values of f on the training set. Given a positive definite kernel $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ with RKHS $\mathcal{H}$, we learn by solving the regularized empirical risk minimization problem

$$\min_{f \in \mathcal{H}} R(\mathbf{f}_n) + \lambda \|f\|_{\mathcal{H}}^2,$$

where $\lambda > 0$ and $R : \mathbb{R}^n \rightarrow \mathbb{R}$ is a closed convex empirical risk. Every method we have built fits this template through its choice of R :

Method	Empirical risk $R(\mathbf{u})$
Kernel ridge regression	$\frac{1}{n} \sum_{i=1}^n (u_i - y_i)^2$
Support vector machine	$\frac{1}{n} \sum_{i=1}^n \max(1 - y_i u_i, 0)$
Kernel logistic regression	$\frac{1}{n} \sum_{i=1}^n \log(1 + \exp(-y_i u_i))$

Recall that R is *closed* if for every $A \in \mathbb{R}$ the sublevel set $\{\mathbf{u} \in \mathbb{R}^n : R(\mathbf{u}) \leq A\}$ is closed; a continuous R , such as all three above, is closed. Closedness and convexity are exactly what we will need to invoke Fenchel duality later, so we carry them from the start. This single-kernel problem is the atom out of which everything in this chapter is assembled.

39.2 The sum kernel

The most elementary way to combine kernels is to add them. Since a sum of positive definite kernels is positive definite, the result is again a legitimate kernel, and it requires no new machinery to learn with. What makes the sum kernel more than a convenience is that it has a transparent meaning in feature space.

Definition (sum kernel)

Let $K_1, \dots, K_M$ be kernels on $\mathcal{X}$. The *sum kernel* K_S is the kernel on $\mathcal{X}$ defined by

$$\forall \mathbf{x}, \mathbf{x}' \in \mathcal{X}, \quad K_S(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^M K_i(\mathbf{x}, \mathbf{x}').$$

39.2.1 Summing kernels is concatenating features

Each kernel K_i comes with a feature map into its own space. Adding the kernels stacks those feature maps side by side into one long feature vector, so the sum kernel is the inner product in the direct sum of the individual feature spaces. This is precisely why the sum kernel is the natural tool for data integration: laying complementary views next to each other in one feature vector is the most literal notion of combining them.

Theorem (sum kernel and vector concatenation)

For $i = 1, \dots, M$, let $\Phi_i : \mathcal{X} \rightarrow \mathcal{H}_i$ be a feature map with

$$K_i(\mathbf{x}, \mathbf{x}') = \langle \Phi_i(\mathbf{x}), \Phi_i(\mathbf{x}') \rangle_{\mathcal{H}_i}.$$

Then $K_S = \sum_{i=1}^M K_i$ can be written as

$$K_S(\mathbf{x}, \mathbf{x}') = \langle \Phi_S(\mathbf{x}), \Phi_S(\mathbf{x}') \rangle_{\mathcal{H}_S},$$

where $\Phi_S : \mathcal{X} \rightarrow \mathcal{H}_S = \mathcal{H}_1 \oplus \dots \oplus \mathcal{H}_M$ is the concatenation of the feature maps,

$$\Phi_S(\mathbf{x}) = (\Phi_1(\mathbf{x}), \dots, \Phi_M(\mathbf{x}))^\top.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The inner product on the direct sum $\mathcal{H}_S = \mathcal{H}_1 \oplus \dots \oplus \mathcal{H}_M$ is the sum of the inner products on the summands. Hence

$$\langle \Phi_S(\mathbf{x}), \Phi_S(\mathbf{x}') \rangle_{\mathcal{H}_S} = \sum_{i=1}^M \langle \Phi_i(\mathbf{x}), \Phi_i(\mathbf{x}') \rangle_{\mathcal{H}_i} = \sum_{i=1}^M K_i(\mathbf{x}, \mathbf{x}') = K_S(\mathbf{x}, \mathbf{x}').$$

□

So summing kernels amounts to concatenating their feature space representations. This is a very natural way to integrate different features, and it is the mechanism behind the bioinformatics application below.

The picture to carry is a spreadsheet. Each view fills its own block of columns, and stacking the blocks side by side gives one wide table whose rows are the concatenated feature vectors $\Phi_S(\mathbf{x})$. A linear model on that wide table is automatically a sum of one linear model per block, and the similarity it uses, the inner product between two rows, is the sum of the per-block similarities, which is exactly K_S . Nothing is lost and nothing is entangled: the blocks sit next to each other untouched, so a view is added or removed by appending or deleting its columns. This is why the sum kernel is the honest default for data integration, the baseline to beat before reaching for anything more elaborate.

39.2.2 Example: data integration in bioinformatics

Yamanishi, Vert, and Kanehisa (2004), in their study of protein network inference from multiple genomic data, give a clean illustration of integration by the sum kernel. The goal is to reconstruct, for the yeast *Saccharomyces cerevisiae*, the network whose vertices are proteins and whose edges join proteins that interact, and to do so from heterogeneous genomic measurements. They cast the network inference as a supervised problem built on a variant of kernel canonical correlation analysis, using a gold-standard partial network as the target.

Four widely available data sources are each turned into a kernel: gene expression, represented by a Gaussian RBF kernel K_{exp} ; protein interactions measured by yeast two-hybrid systems, represented by a diffusion kernel K_{ppi} ; protein localization in the cell, a linear kernel K_{loc} ; and phylogenetic profiles, a linear kernel K_{phy} . All kernels are normalized to one on the diagonal and centered in feature space. The experiment compares learning from each source alone against learning from the integrated kernel

$$K_{\text{exp}} + K_{\text{ppi}} + K_{\text{loc}} + K_{\text{phy}}.$$

Assessed by cross-validated recovery of the gold-standard network, the integrated kernel outperforms any single source, confirming that fusing complementary genomic views is beneficial for prediction. The authors emphasize that new data sources can be folded in at any time, as long as a kernel can be derived from them, which is exactly the modularity the concatenation theorem promises: adding a source appends a block to the feature vector without disturbing the rest.

39.2.3 The sum kernel from the functional side

The feature space picture tells us what the sum kernel *is*; the next theorem tells us what learning with it *does*. It decomposes the single problem in $\mathcal{H}_{K_S}$ into a coupled problem over the individual RKHSs, where the coupling is only through the shared data-fitting term.

Theorem (learning with the sum kernel)

The solution $f^* \in \mathcal{H}_{K_S}$ of the problem learned with $K_S = \sum_{i=1}^M K_i$ equals

$$f^* = \sum_{i=1}^M f_i^*,$$

where $(f_1^*, \dots, f_M^*) \in \mathcal{H}_{K_1} \times \dots \times \mathcal{H}_{K_M}$ is the solution of

$$\min_{f_1, \dots, f_M} R\left(\sum_{i=1}^M (\mathbf{f}_i)_n\right) + \lambda \sum_{i=1}^M \|f_i\|_{\mathcal{H}_{K_i}}^2.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Reading the right-hand side back through the concatenation theorem: a function in $\mathcal{H}_{K_S}$ decomposes into one piece per feature block, the data-fitting risk sees only the sum of the pieces (because that is what the concatenated predictor computes), and the penalty is the total squared norm across blocks. We prove a more general version below and recover this one as the special case of equal weights.

39.3 The weighted sum kernel

Adding kernels treats every source as equally trustworthy. That is rarely appropriate: a noisy or irrelevant view should count for less than an informative one. The obvious fix is to attach a nonnegative weight to each kernel before summing, which leads to the weighted sum kernel and, once we decide to learn the weights, to Multiple Kernel Learning.

Theorem (learning with the weighted sum kernel)

Let $\eta_1, \dots, \eta_M \geq 0$ and set $K_\eta = \sum_{i=1}^M \eta_i K_i$. The solution f^* of the problem learned with K_η equals

$$f^* = \sum_{i=1}^M f_i^*,$$

where $(f_1^*, \dots, f_M^*) \in \mathcal{H}_{K_1} \times \dots \times \mathcal{H}_{K_M}$ is the solution of

$$\min_{f_1, \dots, f_M} R\left(\sum_{i=1}^M (\mathbf{f}_i)_n\right) + \lambda \sum_{i=1}^M \frac{\|f_i\|_{\mathcal{H}_{K_i}}^2}{\eta_i}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The weight η_i appears in the denominator of the penalty: a large η_i discounts the cost of using kernel i , letting f_i grow, while $\eta_i \rightarrow 0$ forbids kernel i altogether. Setting all $\eta_i = 1$ recovers the plain sum kernel of the previous section. The reciprocal is worth dwelling on, because that is where the intuition lives. Read η_i as a spending allowance for kernel i : the penalty charges $\|f_i\|^2/\eta_i$, so a generous allowance makes signal drawn from that kernel cheap and the optimizer will route the fit through it, while a stingy allowance taxes every use of the kernel and pushes the fit onto its competitors. At $\eta_i \rightarrow 0$ the tax is infinite and the kernel is switched off. Weighting the sources is thus the same as allocating a modeling budget across the available views, and learning the weights is learning that allocation instead of guessing it.

The proof is the heart of the chapter's machinery, so we work through it in full; it produces a dual problem that depends on the weights only through the combined Gram matrix $\sum_i \eta_i K_i$, and that observation is what makes learning the weights tractable.

Proof

Step 1: reduce to a finite-dimensional problem. Since R is convex, the objective is strictly convex in $(f_1, \dots, f_M)$ and has a unique solution $(f_1^*, \dots, f_M^*) \in \mathcal{H}_{K_1} \times \dots \times \mathcal{H}_{K_M}$. By the representer theorem applied in each $\mathcal{H}_{K_i}$, there exist $\alpha_1^*, \dots, \alpha_M^* \in \mathbb{R}^n$ with

$$f_i^*(\mathbf{x}) = \sum_{j=1}^n \alpha_{ij}^* K_i(\mathbf{x}_j, \mathbf{x}).$$

Writing K_i also for the i -th Gram matrix, so that $(\mathbf{f}_i)_n = K_i \alpha_i$ and $\|f_i\|_{\mathcal{H}_{K_i}}^2 = \alpha_i^\top K_i \alpha_i$, the coefficients solve

$$\min_{\alpha_1, \dots, \alpha_M \in \mathbb{R}^n} R\left(\sum_{i=1}^M K_i \alpha_i\right) + \lambda \sum_{i=1}^M \frac{\alpha_i^\top K_i \alpha_i}{\eta_i}.$$

Step 2: introduce the fitted values and dualize. Introduce a variable $\mathbf{u} = \sum_{i=1}^M K_i \alpha_i$ for the vector of fitted values, giving the equivalent constrained problem

$$\min_{\mathbf{u}, \alpha_1, \dots, \alpha_M} R(\mathbf{u}) + \lambda \sum_{i=1}^M \frac{\alpha_i^\top K_i \alpha_i}{\eta_i} \quad \text{s.t.} \quad \mathbf{u} = \sum_{i=1}^M K_i \alpha_i.$$

Attaching a multiplier $2\lambda\gamma$ to the equality constraint yields the saddle-point problem

$$\min_{\mathbf{u}, \alpha_1, \dots, \alpha_M} \max_{\gamma \in \mathbb{R}^n} R(\mathbf{u}) + \lambda \sum_{i=1}^M \frac{\alpha_i^\top K_i \alpha_i}{\eta_i} + 2\lambda \gamma^\top \left(\mathbf{u} - \sum_{i=1}^M K_i \alpha_i \right).$$

By Slater's condition strong duality holds, so we may swap the min and the max:

$$\max_{\gamma \in \mathbb{R}^n} \min_{\mathbf{u}, \alpha_1, \dots, \alpha_M} R(\mathbf{u}) + \lambda \sum_{i=1}^M \frac{\alpha_i^\top K_i \alpha_i}{\eta_i} + 2\lambda \gamma^\top \left(\mathbf{u} - \sum_{i=1}^M K_i \alpha_i \right).$$

Step 3: minimize over the primal variables. The inner minimization separates. In $\mathbf{u}$,

$$\min_{\mathbf{u}} R(\mathbf{u}) + 2\lambda \gamma^\top \mathbf{u} = -\max_{\mathbf{u}} \{ -2\lambda \gamma^\top \mathbf{u} - R(\mathbf{u}) \} = -R^*(-2\lambda\gamma),$$

where R^* is the Fenchel conjugate of R ,

$$R^*(\mathbf{v}) = \sup_{\mathbf{u} \in \mathbb{R}^n} \mathbf{u}^\top \mathbf{v} - R(\mathbf{u}).$$

In each α_i we minimize the quadratic

$$\min_{\alpha_i} \lambda \frac{\alpha_i^\top K_i \alpha_i}{\eta_i} - 2\lambda \gamma^\top K_i \alpha_i = -\lambda \eta_i \gamma^\top K_i \gamma,$$

the minimum being attained at $\alpha_i^* = \eta_i \gamma$.

Step 4: the dual and its consequences. Collecting the terms, the dual problem is

$$\max_{\gamma \in \mathbb{R}^n} \left\{ -R^*(-2\lambda\gamma) - \lambda \gamma^\top \left(\sum_{i=1}^M \eta_i K_i \right) \gamma \right\}.$$

This is exactly the dual we would have obtained by learning from the single kernel $K_\eta = \sum_i \eta_i K_i$, namely

$$\max_{\gamma \in \mathbb{R}^n} \{ -R^*(-2\lambda\gamma) - \lambda \gamma^\top K_\eta \gamma \}.$$

If γ^* solves the dual, then $\alpha_i^* = \eta_i \gamma^*$, so

$$f_i^*(\mathbf{x}) = \sum_{j=1}^n \alpha_{ij}^* K_i(\mathbf{x}_j, \mathbf{x}) = \sum_{j=1}^n \eta_i \gamma_j^* K_i(\mathbf{x}_j, \mathbf{x}),$$

and therefore $f^* = \sum_{i=1}^M f_i^*$ satisfies

$$f^*(\mathbf{x}) = \sum_{i=1}^M \sum_{j=1}^n \eta_i \gamma_j^* K_i(\mathbf{x}_j, \mathbf{x}) = \sum_{j=1}^n \gamma_j^* K_\eta(\mathbf{x}_j, \mathbf{x}),$$

which is the predictor obtained by learning directly with K_η . The two views agree. $\square$

Setting every $\eta_i = 1$ in this theorem recovers the sum kernel theorem of the previous section, so the plain sum kernel is the uniform-weight special case.

39.4 Learning the kernel

The weighted sum kernel is only useful if we know the weights. If we knew how much to trust each source we would simply set η accordingly and learn with K_η . Usually we do not. The natural idea is to make η part of the optimization: rather than fix the kernel and learn the predictor, learn the kernel and the predictor together. For this to be more than wishful thinking, the objective as a function of the kernel must be something we can actually minimize.

One might object that we already know how to set kernel parameters: sweep a grid of candidates and keep whatever cross-validates best. That works for one or two knobs, but the grid grows exponentially with the number of kernels, it returns a single winner rather than a blend, and it never even considers mixing genuinely different sources on a common footing. Promoting η to a continuous optimization variable addresses all three at once, and, as the next result shows, it does so over a landscape with no spurious local minima to trap the search. That last property is not automatic, and it is what separates a principled method from a heuristic, so we establish it first.

39.4.1 The value of the problem is convex in the kernel

Fix the data and the risk R , and record how good the best predictor is as a function of the kernel we are allowed to use.

Theorem (convexity in the kernel)

For any positive definite kernel K on $\mathcal{X}$, let

$$J(K) = \min_{f \in \mathcal{H}} \{R(\mathbf{f}_n) + \lambda \|f\|_{\mathcal{H}}^2\}.$$

Then the map $K \mapsto J(K)$ is convex.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

By the strong duality established in the previous proof (specialized to a single kernel),

$$J(K) = \max_{\gamma \in \mathbb{R}^n} \{-R^*(-2\lambda\gamma) - \lambda \gamma^\top K \gamma\}.$$

For each fixed γ , the term $-R^*(-2\lambda\gamma) - \lambda \gamma^\top K \gamma$ is an affine function of the matrix K , since $\gamma^\top K \gamma$ is linear in the entries of K ; an affine function is convex. The map $K \mapsto J(K)$ is a pointwise supremum over γ of these convex functions, and a supremum of convex functions is convex. $\square$

This is the key that unlocks kernel learning. It suggests a principled recipe: define a convex set of candidate kernels, and minimize the convex function $J(K)$ over it by convex optimization. Different choices of candidate set give different kernel-learning methods; the choice that defines Multiple Kernel Learning is the set of convex combinations of a fixed dictionary.

39.4.2 Multiple Kernel Learning

Lanckriet et al. (2004) take the candidate set to be the convex combinations of M given base kernels. Writing Σ_M for the probability simplex,

$$K_\eta = \sum_{i=1}^M \eta_i K_i, \quad \eta \in \Sigma_M = \left\{ \eta_i \geq 0, \sum_{i=1}^M \eta_i = 1 \right\},$$

and optimize the weights and the predictor together:

$$\min_{\eta \in \Sigma_M} J(K_\eta) = \min_{\eta \in \Sigma_M} \min_{f \in \mathcal{H}_{K_\eta}} \{R(\mathbf{f}_n) + \lambda \|f\|_{\mathcal{H}_{K_\eta}}^2\}.$$

Definition (Multiple Kernel Learning)

Given base kernels $K_1, \dots, K_M$, *Multiple Kernel Learning* (MKL) solves

$$\min_{\eta \in \Sigma_M} \min_{f \in \mathcal{H}_{K_\eta}} \{R(\mathbf{f}_n) + \lambda \|f\|_{\mathcal{H}_{K_\eta}}^2\},$$

returning both a weight vector $\eta \in \Sigma_M$ and the predictor trained with the kernel K_η .

Two facts make this practical. First, Σ_M is a convex set (a simplex), and we just proved J is convex in K_η , which is affine in η ; so the outer problem is a convex minimization in η . Second, in terms of the dual variable γ and the weights η , the joint problem is jointly convex in (η, α) , and can therefore be solved efficiently to a global optimum rather than a local one. Joint convexity is the reassurance that matters in practice: it means the weights the algorithm reports are the genuinely optimal weights, not an artifact of where the optimizer happened to start, so we can read them as evidence about which sources helped. The output is at once a set of weights, telling us how much each source contributed, and a predictor, the kernel method trained with K_η . Because Lanckriet et al. phrased the search over kernels through semidefinite programming, this line of work is where semidefinite programming entered kernel learning.

It is worth separating the two convexity statements, because they carry different guarantees. The theorem of the previous subsection says the optimized value $J(K_\eta)$ is a convex function of the weights η , which is what keeps the outer search over the simplex from stalling in a false minimum. The stronger claim is that the full problem, before the predictor is optimized away, is convex in the pair (η, α) jointly, so a single convex solver can descend on the weights and the coefficients at once rather than alternating between them. The joint statement rests on a small fact worth naming: each penalty term $\alpha_i^\top K_i \alpha_i / \eta_i$ is jointly convex in (α_i, η_i) for $\eta_i > 0$, the quadratic-over-linear perspective that appears throughout convex analysis. Either way the operational conclusion is the same, and it is the one that makes MKL trustworthy: there is a single basin, every downhill path reaches it, and the weights the solver reports are a property of the data and the dictionary rather than an accident of where the optimizer began.

39.4.3 Example: genomic data fusion for protein annotation

Lanckriet, De Bie, Cristianini, Jordan, and Noble (2004), in "A statistical framework for genomic data fusion," apply MKL to protein classification in yeast and give the canonical demonstration that learning the weights helps. The task is to recognize particular classes of proteins, namely membrane proteins and ribosomal proteins, from several complementary data types. Seven kernels are assembled, listed below.

Kernel	Data	Similarity measure
K_{SW}	protein sequences	Smith-Waterman
K_{B}	protein sequences	BLAST

Kernel	Data	Similarity measure
K_{Pfam}	protein sequences	Pfam HMM
K_{FFT}	hydropathy profile	FFT
K_{LI}	protein interactions	linear kernel
K_{D}	protein interactions	diffusion kernel
K_{E}	gene expression	radial basis kernel
K_{RND}	random numbers	linear kernel

The final kernel K_{RND} , built from random numbers, is a deliberate control: a well-behaved weighting scheme should learn to ignore it. Each kernel makes explicit one type of similarity, from sequence homology (BLAST, Smith-Waterman, Pfam HMMs), through the hydrophobicity periodicity that the FFT kernel detects and that is diagnostic of membrane-spanning proteins, to the interaction patterns that are informative for ribosomal proteins because ribosomal proteins interact with one another.

Trained by the semidefinite-programming MKL formulation over all sources at once, the SVM significantly outperforms the same algorithm trained on any single data type, measured by ROC score and by the true-positive rate at one percent false positives, over thirty random train/test splits. The learned weights are themselves interpretable: they concentrate on the informative kernels for each task and place little mass on the control kernel, so MKL performs a soft, data-driven selection among the sources.

39.4.4 Example: image classification

Harchaoui and Bach (2007) apply the same idea to natural image classification on the COREL14 dataset, 1400 natural images in 14 classes. Here the base kernels are different structured kernels on images represented as graphs or histograms: a kernel between color histograms (H), a walk kernel (W), a subtree kernel (TW), and a weighted subtree kernel (wTW). MKL (M) combines them. The combination attains the lowest test error of all, below every individual kernel, illustrating that even when the candidate kernels are elaborate structured objects rather than simple bandwidth variants, learning their combination pays off.

39.5 MKL is a group lasso

The name "Multiple Kernel Learning" and the picture of a weighted average of kernels suggest the method is essentially about averaging. That picture is misleading. Bach et al. (2004) show that eliminating the weights turns MKL into a penalized risk minimization with a sparsity-inducing penalty, which explains both why it selects among kernels and why it differs from the plain sum kernel.

Theorem (MKL as a mixed-norm problem)

With $K_\eta = \sum_{i=1}^M \eta_i K_i$ and $\eta \in \Sigma_M$, the solution f^* of

$$\min_{\eta \in \Sigma_M} \min_{f \in \mathcal{H}_{K_\eta}} \{R(\mathbf{f}_n) + \lambda \|f\|_{\mathcal{H}_{K_\eta}}^2\}$$

is $f^* = \sum_{i=1}^M f_i^*$, where $(f_1^*, \dots, f_M^*) \in \mathcal{H}_{K_1} \times \dots \times \mathcal{H}_{K_M}$ solves

$$\min_{f_1, \dots, f_M} \left\{ R\left(\sum_{i=1}^M \mathbf{f}_i\right)_n + \lambda \left(\sum_{i=1}^M \|f_i\|_{\mathcal{H}_{K_i}}\right)^2 \right\}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Start from the weighted sum kernel theorem, which rewrites the inner problem for fixed η as a problem over $(f_1, \dots, f_M)$ with penalty $\sum_i \|f_i\|_{\mathcal{H}_{K_i}}^2 / \eta_i$. The MKL objective is then

$$\min_{\eta \in \Sigma_M} \min_{f_1, \dots, f_M} \left\{ R\left(\sum_{i=1}^M (f_i)_n\right) + \lambda \sum_{i=1}^M \frac{\|f_i\|_{\mathcal{H}_{K_i}}^2}{\eta_i} \right\}.$$

The risk term does not involve η , so we may swap the two minimizations and carry out the minimization over η first:

$$= \min_{f_1, \dots, f_M} \left\{ R\left(\sum_{i=1}^M (f_i)_n\right) + \lambda \min_{\eta \in \Sigma_M} \sum_{i=1}^M \frac{\|f_i\|_{\mathcal{H}_{K_i}}^2}{\eta_i} \right\}.$$

It remains to evaluate the inner minimization over the simplex. For any nonnegative reals $a_1, \dots, a_M$,

$$\min_{\eta \in \Sigma_M} \sum_{i=1}^M \frac{a_i^2}{\eta_i} = \left(\sum_{i=1}^M a_i \right)^2,$$

which is a direct consequence of the Cauchy-Schwarz inequality: writing $a_i = \frac{a_i}{\sqrt{\eta_i}} \cdot \sqrt{\eta_i}$,

$$\sum_{i=1}^M a_i = \sum_{i=1}^M \frac{a_i}{\sqrt{\eta_i}} \sqrt{\eta_i} \leq \left(\sum_{i=1}^M \frac{a_i^2}{\eta_i} \right)^{1/2} \left(\sum_{i=1}^M \eta_i \right)^{1/2} = \left(\sum_{i=1}^M \frac{a_i^2}{\eta_i} \right)^{1/2},$$

using $\sum_i \eta_i = 1$. Squaring gives $(\sum_i a_i)^2 \leq \sum_i a_i^2 / \eta_i$ for every $\eta \in \Sigma_M$, so the minimum is at least $(\sum_i a_i)^2$; equality holds at $\eta_i \propto a_i$, which lies in Σ_M , so the minimum is attained and equals $(\sum_i a_i)^2$. Applying this with $a_i = \|f_i\|_{\mathcal{H}_{K_i}}$ yields the claimed objective. $\square$

The optimizing weights fall out of the same inequality. Equality in Cauchy-Schwarz forces $\eta_i \propto \|f_i\|_{\mathcal{H}_{K_i}}$, so at the optimum each kernel is handed budget in exact proportion to how much of the fitted function it carries. A block that ends up unused, with $\|f_i\|_{\mathcal{H}_{K_i}} = 0$, receives weight zero and drops out of the combined kernel altogether. This is the precise sense in which learning the mixing weights and selecting the kernels are one and the same act: the single minimization that fixes the coefficients also decides which sources survive.

It is worth stating the conclusion plainly. Although MKL can be pictured as optimizing a convex combination of kernels, it is more correct to view it as a penalized risk minimization estimator with a *group lasso* penalty. Define

$$\Omega(f) = \min_{f_1 + \dots + f_M = f} \sum_{i=1}^M \|f_i\|_{\mathcal{H}_{K_i}}.$$

This is a sum, over blocks, of RKHS norms, an ℓ_1 norm of the vector of per-kernel norms rather than the ℓ_2 sum of squared norms that the plain sum kernel penalizes. Just as the ℓ_1 penalty in the lasso drives individual coefficients to exactly zero, this mixed ℓ_1/ℓ_2 penalty drives entire blocks f_i to zero, which is to say it sets the corresponding weights η_i to zero and discards kernel i . This is why MKL selects kernels rather than merely averaging them, and it is what the protein experiment observed when the control kernel received essentially no weight.

It is worth asking what this sparsity actually buys, since that is the whole reason to prefer the group lasso over the plain sum. Three things. It buys interpretation: the surviving weights are a short list of the sources that mattered, which is why the protein study could read off that sequence homology drove one task and interaction patterns the other. It buys robustness to junk: an irrelevant or noisy kernel, like the deliberate random control, is not

merely down-weighted but zeroed out, so it cannot leak variance into the predictor. And it buys economy at test time, since a predictor that keeps three of twenty kernels evaluates only three Gram functions per new point. The squared- ℓ_2 penalty of the sum kernel offers none of these; it keeps every kernel slightly active and spreads the fit thinly across all of them, which is exactly right when every source is trustworthy and exactly wrong when the menu is padded with kernels that carry no signal. The choice between them is therefore a statement about the dictionary: sum when you believe all of it, MKL when you suspect only some of it.

39.5.1 Sum kernel versus MKL, side by side

The contrast is entirely in the exponent on the norms. Learning with the sum kernel, the uniform combination, solves

$$\min_{f_1, \dots, f_M} \left\{ R\left(\sum_{i=1}^M (\mathbf{f}_i)_n\right) + \lambda \sum_{i=1}^M \|f_i\|_{\mathcal{H}_{K_i}}^2 \right\},$$

while learning with MKL, the best convex combination, solves

$$\min_{f_1, \dots, f_M} \left\{ R\left(\sum_{i=1}^M (\mathbf{f}_i)_n\right) + \lambda \left(\sum_{i=1}^M \|f_i\|_{\mathcal{H}_{K_i}} \right)^2 \right\}.$$

The sum kernel sums squared norms (a squared ℓ_2 penalty across blocks) and spreads weight over all kernels; MKL squares a sum of norms (an ℓ_1 -type penalty across blocks) and concentrates weight on a few. The next example makes the difference concrete in the most familiar setting.

39.5.2 Example: ridge versus lasso regression

Take $\mathcal{X} = \mathbb{R}^d$, and for $\mathbf{x} = (x_1, \dots, x_d)^\top$ consider the d rank-one coordinate kernels

$$K_i(\mathbf{x}, \mathbf{x}') = x_i x'_i, \quad i = 1, \dots, d.$$

The RKHS of K_i consists of the functions $f_i(\mathbf{x}) = \beta_i x_i$ of the single coordinate x_i , with norm $\|f_i\|_{\mathcal{H}_{K_i}} = |\beta_i|$. The sum kernel is

$$K_S(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^d x_i x'_i = \mathbf{x}^\top \mathbf{x}',$$

the ordinary linear kernel, whose RKHS consists of the linear functions $f(\mathbf{x}) = \beta^\top \mathbf{x}$ with norm $\|f\|_{\mathcal{H}_{K_S}} = \|\beta\|_{\mathbb{R}^d}$. Now the two combination rules specialize to two textbook estimators. Learning with the sum kernel solves a ridge regression problem,

$$\min_{\beta \in \mathbb{R}^d} \left\{ R(X\beta) + \lambda \sum_{i=1}^d \beta_i^2 \right\},$$

while learning with MKL solves a lasso regression problem,

$$\min_{\beta \in \mathbb{R}^d} \left\{ R(X\beta) + \lambda \left(\sum_{i=1}^d |\beta_i| \right)^2 \right\}.$$

The squared ℓ_2 penalty of ridge regression shrinks all coordinates smoothly and keeps them nonzero; the ℓ_1 penalty of the lasso sets many coordinates to exactly zero and performs variable selection. Reading a coordinate as a one-dimensional kernel, selecting variables and selecting kernels are the same operation, and MKL is the kernel-space lasso.

39.6 Algorithm and extensions

39.6.1 The simpleMKL algorithm

The joint convexity guarantees a global optimum exists; simpleMKL (Rakotomamonjy et al., 2008) is a clean way to reach it that reuses any existing single-kernel solver as a black box. The idea is to optimize the weights in the outer loop and delegate the predictor to the inner loop. We want to minimize over $\eta \in \Sigma_M$,

$$\min_{\eta \in \Sigma_M} J(K_\eta) = \min_{\eta \in \Sigma_M} \max_{\gamma \in \mathbb{R}^n} \{ -R^*(-2\lambda\gamma) - \lambda \gamma^\top K_\eta \gamma \}.$$

For a fixed η , evaluating $J(K_\eta)$ is nothing but training a single kernel machine with the combined kernel K_η : a standard solver returns the maximizer γ^* , and

$$J(K_\eta) = -R^*(-2\lambda\gamma^*) - \lambda \gamma^{*\top} K_\eta \gamma^*.$$

The same γ^* hands us the gradient of J at almost no extra cost. By Danskin's theorem, differentiating through the inner maximum with γ^* held fixed,

$$\frac{\partial J(K_\eta)}{\partial \eta_i} = -\lambda \gamma^{*\top} K_i \gamma^*.$$

The reason we may hold γ^* fixed while differentiating deserves a word, because it is what makes the gradient nearly free. Since $J(K_\eta)$ is a maximum over γ , at the maximizer the derivative of the inner objective with respect to γ vanishes; so when we nudge η , the induced motion of γ^* contributes nothing to first order and only the explicit dependence on η survives. That explicit dependence is the lone term $-\lambda \gamma^\top K_\eta \gamma$, and differentiating it in η_i leaves $-\lambda \gamma^{*\top} K_i \gamma^*$. Read plainly: the very dual solution that trains the machine also scores each base kernel by how much of the current fit it accounts for, and that score is the gradient. Kernels that explain a lot of the fit get pushed up in the next weight update, kernels that explain little get pushed down.

Each partial derivative is a single quadratic form in the base Gram matrix K_i . With the value and gradient in hand, $J(K_\eta)$ is minimized over the simplex Σ_M by a projected gradient or reduced gradient algorithm. The result is an outer loop over η wrapped around an off-the-shelf single-kernel solver, calling it as a black box and never opening it up, which is what makes simpleMKL simple: any SVM or ridge code you already trust becomes an MKL solver at the cost of a gradient step on the simplex.

```
simpleMKL
  initialize eta in the simplex (e.g. uniform)
  repeat
    1. train a single-kernel machine with K_eta -> gamma*
    2. J = -R*(-2 lambda gamma*) - lambda gamma*' K_eta gamma*
       dJ/d eta_i = -lambda gamma*' K_i gamma*
    3. update eta by a projected/reduced gradient step on the simplex
  until convergence
```


Each pass does two honest things: it trains the best predictor for the current mixture, then rebalances the mixture toward whichever kernels that predictor leaned on, projecting back onto the simplex so the weights remain a valid combination. Because the outer objective is convex in η , this descent converges to the global optimum, and it does so while treating the single-kernel trainer as a sealed box, so any improvement to that solver carries over at no extra cost. The price of not knowing the kernel in advance is simply that we now solve a short sequence of single-kernel problems in place of one, each cheap, with the weights settling as the sequence proceeds.

39.6.2 Beyond the simplex: the ℓ_r extension

The simplex constraint $\sum_i \eta_i = 1$ is one choice among many, and it is the one that produces the sparse, group-lasso behavior. Micchelli and Pontil (2005) generalize it by constraining a different norm of the weight vector. For a parameter $r > 0$, take

$$K_\eta = \sum_{i=1}^M \eta_i K_i, \quad \eta \in \Sigma_M^r = \left\{ \eta_i \geq 0, \sum_{i=1}^M \eta_i^r = 1 \right\}.$$

Theorem (ℓ_r multiple kernel learning)

The solution f^* of

$$\min_{\eta \in \Sigma_M^r} \min_{f \in \mathcal{H}_{K_\eta}} \{R(\mathbf{f}_n) + \lambda \|f\|_{\mathcal{H}_{K_\eta}}^2\}$$

is $f^* = \sum_{i=1}^M f_i^*$, where $(f_1^*, \dots, f_M^*) \in \mathcal{H}_{K_1} \times \dots \times \mathcal{H}_{K_M}$ solves

$$\min_{f_1, \dots, f_M} \left\{ R\left(\sum_{i=1}^M (\mathbf{f}_i)_n\right) + \lambda \left(\sum_{i=1}^M \|f_i\|_{\mathcal{H}_{K_i}}^{\frac{2r}{r+1}}\right)^{\frac{r+1}{r}} \right\}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The penalty is now a mixed norm with exponent $\frac{2r}{r+1}$ on the per-block RKHS norms. The proof mirrors the group-lasso derivation, replacing the Cauchy-Schwarz step by the corresponding Hölder inequality on Σ_M^r . Setting $r = 1$ recovers the standard MKL problem with its ℓ_1 block penalty and its kernel selection; letting r grow relaxes the sparsity toward the dense, ridge-like behavior of the plain sum kernel. The parameter r is thus a dial between selecting a few kernels and averaging over all of them, and where to set it is itself a modeling choice, one more instance of the theme this chapter began with: the more of the kernel we are willing to learn, the fewer decisions we have to make by hand.

39.6.3 The ℓ_p -norm generalization

The most widely used form of this dial is the ℓ_p -norm MKL of Kloft, Brefeld, Sonnenburg, and Zien (2011), who replace the simplex by a constraint on an arbitrary ℓ_p norm of the weight vector, $\|\eta\|_p \leq 1$ with $p \geq 1$. At $p = 1$ this is the sparse, selecting MKL of the previous sections, whose group-lasso penalty zeroes out kernels. For $p > 1$ the constraint pushes the weights toward a smoother, non-sparse profile, so no kernel is discarded outright and every source keeps some share of the fit. Kloft and coauthors report that this non-sparse ℓ_p -norm MKL frequently outperforms the sparse ℓ_1 version, especially when many of the base kernels carry partial signal and hard selection throws useful information away. The exponent p then joins λ as a hyperparameter, interpolating between hard selection and soft averaging.

39.6.4 Does learning the kernel actually help?

Theory and practice part company here, and it is worth stating the gap directly. Cortes (2009), in "Can learning kernels help performance?", weighs the empirical record and finds that learned kernel weights often fail to beat two simple baselines: the uniform, unweighted sum of the base kernels, and a single kernel tuned by cross-validation. The convexity of $J(K_\eta)$ and the group-lasso selection are genuine, but the accuracy they buy over an honest uniform combination is in many benchmarks small or absent. The working lesson is to treat the uniform sum kernel as the baseline to beat and to report it alongside any learned combination, and to reach for MKL when its data-driven selection is wanted for interpretation or test-time economy as much as for raw predictive accuracy.

Further reading

Multiple Kernel Learning is younger than the two standard kernel-methods textbooks, so it appears in neither in the form given here. Schölkopf and Smola (2002) and Shawe-Taylor and Cristianini (2004) both predate the semidefinite-programming formulation of Lanckriet et al. (2004) and the group-lasso reading of Bach et al. (2004). Shawe-Taylor and Cristianini (2004) is nonetheless the right place to read the basics of combining and manipulating kernels, closure under sums and products and the feature-space view that this chapter builds on. For MKL proper, the standard entry point is the survey of Gönen and Alpaydın (2011), Multiple Kernel Learning Algorithms, which catalogues the formulations, the optimization strategies including simpleMKL, and the empirical record of dozens of methods on a common footing. The primary sources for the results proved above are Lanckriet et al. (2004) for the convexity of $J(K)$ and the semidefinite program, Bach et al. (2004) for the group-lasso equivalence, Rakotomamonjy et al. (2008) for simpleMKL, and Micchelli and Pontil (2005) for the ℓ_r generalization.

39.7 Centered kernel alignment as a selection criterion

Multiple Kernel Learning selects among kernels by solving a joint convex program that trains a kernel machine at every step of the search over η . Often we want something far cheaper: a score we can read straight off the Gram matrices, before any training, to rank candidate kernels or to fix the weights in one shot. Kernel alignment supplies exactly such a score. It measures the angle between a kernel's Gram matrix and the ideal kernel built from the labels, it comes with a concentration guarantee and an existence-of-a-good-predictor guarantee, and in its centered form it yields a two-stage alternative to the joint optimization: choose the weights by maximizing alignment, then train once. The one subtlety, which is most of what this section is about, is that the naive uncentered alignment is misleading and that centering is not optional. Alignment was introduced by Cristianini, Shawe-Taylor, Elisseeff, and Kandola (2002), and its indispensable centered form, together with the algorithms below, by Cortes, Mohri, and Rostamizadeh (2012).

39.7.1 Kernel alignment and the ideal kernel

An $n \times n$ Gram matrix is, forgetting its structure for a moment, just a vector of n^2 numbers. The natural similarity between two kernels restricted to a common sample is then the cosine of the angle between these two vectors, the normalized Frobenius inner product.

Definition (kernel alignment)

For two kernels with Gram matrices K_1, K_2 on the same sample of n points, their alignment is

$$A(K_1, K_2) = \frac{\langle K_1, K_2 \rangle_F}{\|K_1\|_F \|K_2\|_F}, \quad \langle K_1, K_2 \rangle_F = \sum_{i,j=1}^n (K_1)_{ij} (K_2)_{ij} = \text{tr}(K_1 K_2),$$

with $\|K\|_F = \sqrt{\langle K, K \rangle_F}$. It is the cosine of the angle between the two Gram matrices viewed as vectors, so $A(K_1, K_2) \in [-1, 1]$, and $A(K_1, K_2) \in [0, 1]$ when both matrices are positive semidefinite.

The point of the definition is to compare a candidate kernel against a target that encodes the labels. For binary labels $y \in \{-1, +1\}^n$, the rank-one matrix $yy^\top$ is the Gram matrix of the ideal kernel, the one that scores $+1$ for every same-label pair and -1 for every opposite-label pair. Its alignment with a candidate kernel, the target alignment $A(K, yy^\top)$, grades how well K 's geometry matches the labels, and it costs only a single quadratic form, since

$$\langle K, yy^\top \rangle_F = y^\top Ky.$$

No predictor is trained, no matrix is inverted, no fold is held out. This cheapness is the whole appeal: a proxy for kernel quality read directly off K and y .

39.7.2 Why centering is essential

The raw target alignment is easily fooled, and understanding why fixes the flaw. Both $\langle K, yy^\top \rangle_F$ and $\|K\|_F$ see the component of K along the constant direction $\mathbf{1}\mathbf{1}^\top$, yet that component says nothing about the label split. A kernel whose features carry a large mean, any un-centered representation, has a big constant component that inflates $\|K\|_F$ and, depending on the label balance, distorts the numerator, so the score can be dominated by a quantity irrelevant to classification. The cure is to center the kernel in feature space before measuring the angle, that is, to subtract the feature-space mean, which on the Gram matrix is conjugation by the centering matrix $H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$. Then $K^c = HKH$ is the Gram matrix of the centered features $\phi(x_i) - \frac{1}{n} \sum_j \phi(x_j)$.

Definition (centered alignment, Cortes, Mohri, Rostamizadeh, 2012)

The centered alignment of K_1, K_2 on a common sample is the alignment of their centered Gram matrices,

$$A_c(K_1, K_2) = \frac{\langle HK_1H, HK_2H \rangle_F}{\|HK_1H\|_F \|HK_2H\|_F}.$$

For the label target $L = yy^\top$, centering gives $HLH = \tilde{y}\tilde{y}^\top$ with the centered label vector $\tilde{y} = y - \bar{y}\mathbf{1}$.

Centering is essential, not cosmetic, and the reason is a single identity: $H\mathbf{1} = 0$, so $H(K + c\mathbf{1}\mathbf{1}^\top)H = HKH$ for every constant c . The centered alignment is therefore invariant to adding any constant offset to the kernel, which is exactly the nuisance that misleads the uncentered version. Cortes, Mohri, and Rostamizadeh (2012) show that the uncentered alignment can be pushed close to its constant-kernel value by a kernel that carries no discriminative structure at all, and conversely can be dragged toward zero for the ideal kernel by a large offset. The next example exhibits both failures at once, on numbers small enough to check by hand.

Example (centering flips the ranking)

Four points carry balanced labels $y = (+1, +1, -1, -1)$, so the ideal kernel is

$$L = yy^\top = \begin{pmatrix} 1 & 1 & -1 & -1 \\ 1 & 1 & -1 & -1 \\ -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 \end{pmatrix}.$$

Two candidates are the linear (rank-one) Gram matrices $K = \phi\phi^\top$ of two scalar features. The first feature is a perfect separator carrying a large constant offset, $\phi_1 = (3, 3, 1, 1) = (1, 1, -1, -1) + 2$; the second is an imperfect, already-centered separator, $\phi_2 = (1, 0.5, -0.5, -1)$:

$$K_1 = \begin{pmatrix} 9 & 9 & 3 & 3 \\ 9 & 9 & 3 & 3 \\ 3 & 3 & 1 & 1 \\ 3 & 3 & 1 & 1 \end{pmatrix}, \quad K_2 = \begin{pmatrix} 1 & 0.5 & -0.5 & -1 \\ 0.5 & 0.25 & -0.25 & -0.5 \\ -0.5 & -0.25 & 0.25 & 0.5 \\ -1 & -0.5 & 0.5 & 1 \end{pmatrix}.$$

Both are positive semidefinite, each being an outer product. Which kernel fits the labels better?

1. Score kernel one uncentered. Using $\langle K, yy^\top \rangle_F = y^\top Ky = (\phi \cdot y)^2$ with $\phi_1 \cdot y = 4$, the numerator is $\langle K_1, L \rangle_F = 16$, while $\|K_1\|_F = \|\phi_1\|^2 = 20$ and $\|L\|_F = 4$. Hence $A(K_1, L) = 16/(20 \cdot 4) = 0.2$.
2. Score kernel two uncentered. Here $\phi_2 \cdot y = 3$, so $\langle K_2, L \rangle_F = 9$, $\|K_2\|_F = \|\phi_2\|^2 = 2.5$, and $A(K_2, L) = 9/(2.5 \cdot 4) = 0.9$. The uncentered verdict is emphatic and wrong: K_2 at 0.9 dwarfs K_1 at 0.2.
3. Center the features. With $H = I - \frac{1}{4}\mathbf{1}\mathbf{1}^\top$ we get $H\phi_1 = (1, 1, -1, -1) = y$ exactly, so $K_1^c = HK_1H = yy^\top$: after centering, K_1 is the ideal kernel, and its Frobenius norm drops from 20 to $\|K_1^c\|_F = 4$. Since ϕ_2 already sums to zero, $H\phi_2 = \phi_2$ and $K_2^c = K_2$.
4. Score both centered. Now $A_c(K_1, L) = A(yy^\top, yy^\top) = 1.0$, while $A_c(K_2, L) = (\phi_2 \cdot y)^2/(\|\phi_2\|^2\|y\|^2) = 9/(2.5 \cdot 4) = 0.9$. The centered verdict is correct: K_1 at 1.0 edges out K_2 at 0.9.

Reading. The constant offset +2 hidden in ϕ_1 inflates $\|K_1\|_F$ from the discriminative value 4 all the way to 20, dragging the uncentered alignment down to 0.2 and ranking the perfect kernel below the imperfect one. Centering deletes the offset, exposes that K_1 equals the ideal kernel $yy^\top$, and restores the right order. Measure alignment after centering, never before.

Verification artifact. checks/example-ch12-example-38-1.json records the example source hash and verification scope.

39.7.3 What alignment guarantees

Two results turn alignment from a plausible heuristic into a justified criterion. The first says the score is stable across samples; the second says a high score is worth chasing.

Theorem (concentration of centered alignment, Cortes, Mohri, Rostamizadeh, 2012)

Suppose the kernels are bounded on the data. Let $\hat{\rho}(K, L)$ be the empirical centered alignment computed on an i.i.d. sample of size n , and let $\rho(K, L)$ be the corresponding population alignment of the kernel functions under the data distribution. There is a constant $c > 0$, depending only on the kernel bound, such that for every $\delta \in (0, 1)$, with probability at least $1 - \delta$ over the draw of the sample,

$$|\hat{\rho}(K, L) - \rho(K, L)| \leq \frac{c}{\sqrt{n}} \sqrt{\log \frac{2}{\delta}}.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof (sketch)

The three ingredients of the centered alignment, the Frobenius inner product $\langle HKH, HLH \rangle_F$ and the two norms $\|HKH\|_F$ and $\|HLH\|_F$, are each empirical averages of bounded functions of the sample, the empirical moments of the centered kernel values, so replacing any one of the n sample points changes each of them by $O(1/n)$. The alignment is a fixed smooth function of these three quantities, Lipschitz on the region where the denominator stays bounded away from zero, hence it too has bounded differences of order $O(1/n)$ under a single-point change. McDiarmid's bounded-difference inequality, the concentration engine of Chapter 11, Learning Theory in RKHS Balls, then bounds the deviation of $\hat{\rho}$ from its expectation by $O(1/\sqrt{n})$ with the stated confidence, and a separate estimate controls the small gap between that expectation and the population value ρ . $\square$

The second guarantee explains why a large alignment is a good thing to have. Cristianini, Shawe-Taylor, Elisseeff, and Kandola (2002) show that a high target alignment upper bounds the error of a simple classifier read straight off the kernel, the threshold rule $f(x) = \sum_i y_i k(x_i, x)$ that compares a new point's average similarity to the positive class against its average similarity to the negative class. The larger the alignment $A(K, yy^\top)$, the wider the margin this mean-difference rule enjoys and the smaller its error bound, so alignment does not merely correlate with accuracy, it certifies the existence of a good predictor, and that certificate feeds into the RKHS-ball risk bounds of Chapter 11, Learning Theory in RKHS Balls. The rule it controls is exactly the sample version of the difference of

class-conditional kernel mean embeddings of the mean-embedding chapter, comparing a test point to the two class means in feature space. Maximizing centered alignment is therefore maximizing the feature-space separation of the two class means, which is why it is a principled surrogate for the classification we actually want.

39.7.4 Two-stage alignment-based kernel weighting

The alignment is cheap enough to replace the joint optimization of the earlier sections with a two-stage recipe. Rather than train a kernel machine at every step of a search over η , as simpleMKL does, Cortes, Mohri, and Rostamizadeh (2012) choose the weights once, by maximizing the centered alignment of the combined kernel to the ideal kernel, and only then train a single machine with the resulting K_η . The weight-selection stage never opens the learner and costs one linear solve.

The derivation is short. Center each base kernel, $K_i^c = HK_iH$, and the target, $M_y = Hyy^\top H$. Because centering is linear, the centered combination is $(K_\eta)^c = \sum_i \eta_i K_i^c$, and its alignment to the ideal kernel is

$$A_c\left(\sum_i \eta_i K_i, yy^\top\right) = \frac{\langle \sum_i \eta_i K_i^c, M_y \rangle_F}{\left\| \sum_i \eta_i K_i^c \right\|_F \|M_y\|_F} = \frac{a^\top \eta}{\|M_y\|_F \sqrt{\eta^\top M \eta}},$$

where $a_i = \langle K_i^c, M_y \rangle_F$ collects each base kernel's own alignment with the labels and $M_{ij} = \langle K_i^c, K_j^c \rangle_F$ is the Gram matrix of the centered kernels, an $M \times M$ matrix of kernel-kernel inner products. Maximizing $a^\top \eta / \sqrt{\eta^\top M \eta}$ is maximizing a linear functional over an ellipsoid: the objective is invariant to the scale of η , so we may fix $\eta^\top M \eta = 1$ and maximize $a^\top \eta$ under that constraint by a Lagrange multiplier, which gives $a = 2\mu M\eta$ and hence

$$\eta^* \propto M^{-1}a.$$

Restoring the constraint $\eta \geq 0$ turns this into a small quadratic program, whose solution Cortes, Mohri, and Rostamizadeh (2012) give in closed form; $M^{-1}a$ projected onto the nonnegative orthant is the unconstrained starting point. The weight η_i^* rewards a base kernel for aligning with the labels on its own, through a large a_i , while the factor M^{-1} discounts kernels that merely duplicate others, decorrelating the dictionary.

Algorithm (alignment-based kernel weighting)

Input Base Gram matrices $K_1, \dots, K_M$ on the sample, labels $y \in \{-1, +1\}^n$.

Output Nonnegative weights η and the combined kernel $K_\eta = \sum_i \eta_i K_i$.

1. Form the centering matrix $H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$; center every base kernel, $K_i^c = HK_iH$, and the target, $M_y = Hyy^\top H$.
2. Compute the alignment vector $a_i = \langle K_i^c, M_y \rangle_F$.
3. Compute the kernel-kernel matrix $M_{ij} = \langle K_i^c, K_j^c \rangle_F$.
4. Solve for the weights $\eta \leftarrow M^{-1}a$, clip $\eta \leftarrow \max(\eta, 0)$, and normalize $\eta \leftarrow \eta / \|\eta\|$.
5. Train any single-kernel machine once with $K_\eta = \sum_i \eta_i K_i$.

The contrast with simpleMKL is the absence of any inner training loop. Where the joint method alternates between fitting a predictor and nudging the weights, here the weights are read off the Gram matrices and the labels in one solve, and the learner is called exactly once, at the end. This is a proxy method, since it maximizes alignment rather than the risk itself, but it is a proxy carrying the two guarantees just established: the concentration theorem says the alignment it maximizes on the sample tracks the population alignment, and the generalization result says a high alignment secures a good predictor. It also respects the honest baseline of the previous section. Because Cortes (2009) found that learned weights often fail to beat the plain uniform sum, a weighting scheme this cheap is attractive precisely because it can be computed and reported alongside the uniform combination at negligible extra cost.

39.8 Common mistakes and practical implications

For **Multiple Kernel Learning**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

39.9 Summary and further reading

This chapter established explain the central definitions and claims in Multiple Kernel Learning; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Lanckriet and Jordan 2004), (Lanckriet and Noble 2004), (Yamanishi and Kanehisa 2004).

39.10 Exercises

1. warm-up Let $K_1, \dots, K_M$ be positive definite kernels on $\mathcal{X}$ and let $\eta_1, \dots, \eta_M \geq 0$. Show that the weighted sum $K_\eta = \sum_{i=1}^M \eta_i K_i$ is again a positive definite kernel, so that every kernel in the MKL candidate set is legitimate. Then explain why the constraint $\sum_i \eta_i = 1$ is a normalization rather than a restriction: given any nonnegative weights, what does rescaling them do to the learned predictor if λ is rescaled to match?
2. warm-up The penalty in the weighted sum kernel theorem is $\lambda \sum_i \|f_i\|_{\mathcal{H}_{K_i}}^2 / \eta_i$, with η_i in the denominator. Explain in words why a large η_i encourages the fit to use kernel i heavily while $\eta_i \rightarrow 0$ switches the kernel off entirely. Reconcile this with the equally intuitive reading of $K_\eta = \sum_i \eta_i K_i$, where a large η_i makes kernel i count for more in the combined similarity: why do the two pictures, weight in the kernel and reciprocal weight in the penalty, agree on which kernel is emphasized?
3. proof Prove the sum-kernel concatenation theorem from the ground up. For $i = 1, \dots, M$ let $\Phi_i : \mathcal{X} \rightarrow \mathcal{H}_i$ satisfy $K_i(\mathbf{x}, \mathbf{x}') = \langle \Phi_i(\mathbf{x}), \Phi_i(\mathbf{x}') \rangle_{\mathcal{H}_i}$. Define the direct sum $\mathcal{H}_S = \mathcal{H}_1 \oplus \dots \oplus \mathcal{H}_M$ with inner product $\langle (u_1, \dots, u_M), (v_1, \dots, v_M) \rangle_{\mathcal{H}_S} = \sum_i \langle u_i, v_i \rangle_{\mathcal{H}_i}$, and set $\Phi_S(\mathbf{x}) = (\Phi_1(\mathbf{x}), \dots, \Phi_M(\mathbf{x}))$. Verify that $\langle \cdot, \cdot \rangle_{\mathcal{H}_S}$ is a genuine inner product, and show $\langle \Phi_S(\mathbf{x}), \Phi_S(\mathbf{x}') \rangle_{\mathcal{H}_S} = K_S(\mathbf{x}, \mathbf{x}')$. Then give the spreadsheet reading: if each $\mathcal{H}_i = \mathbb{R}^{d_i}$, describe the feature vector $\Phi_S(\mathbf{x})$ concretely and say what removing kernel j does to it.
4. proof Prove that $K \mapsto J(K)$ is convex, where $J(K) = \min_{f \in \mathcal{H}} \{R(\mathbf{f}_n) + \lambda \|f\|_{\mathcal{H}}^2\}$, directly from the dual representation

$$J(K) = \max_{\gamma \in \mathbb{R}^n} \{ -R^*(-2\lambda\gamma) - \lambda \gamma^\top K \gamma \}.$$

Show that for each fixed γ the bracketed expression is affine in the entries of the matrix K , and conclude convexity from the fact that a pointwise supremum of affine (hence convex) functions is convex. Where in the argument did you use that K ranges over symmetric matrices rather than the abstract kernel? Explain why this is exactly the property that makes the outer MKL search over η , which enters K_η affinely, a convex minimization. Hint

Affineness needs only that $\gamma^\top K \gamma = \sum_{j,k} \gamma_j \gamma_k K_{jk}$ is linear in the entries K_{jk} , with the constant term $-R^*(-2\lambda\gamma)$ not depending on K at all. Composition: $J(K_\eta)$ is a convex function of K_η and K_η is affine in η , and convex-after-affine is convex.

5. proof Derive the group-lasso weight relation. For fixed $(f_1, \dots, f_M)$ with $a_i = \|f_i\|_{\mathcal{H}_{K_i}} \geq 0$, show that

$$\min_{\eta \in \Sigma_M} \sum_{i=1}^M \frac{a_i^2}{\eta_i} = \left(\sum_{i=1}^M a_i \right)^2,$$

and that the minimizing weights are $\eta_i^* = a_i / \sum_j a_j$, that is $\eta_i^* \propto \|f_i\|_{\mathcal{H}_{K_i}}$. Give the argument two ways: first by the Cauchy-Schwarz step in the chapter, and second by a Lagrange multiplier on the constraint $\sum_i \eta_i = 1$. Then read off the selection consequence: what weight does a block with $\|f_i\|_{\mathcal{H}_{K_i}} = 0$ receive, and why does this mean learning the weights and selecting the kernels are the same act? Hint

For the Lagrangian, minimize $\sum_i a_i^2/\eta_i + \mu(\sum_i \eta_i - 1)$; stationarity gives $-a_i^2/\eta_i^2 + \mu = 0$, so $\eta_i \propto a_i$. Substitute back and use $\sum_i \eta_i = 1$ to fix the constant.

6. exploration Open the interactive single-kernel figure at the top of the chapter and sweep the bandwidth of the Gaussian kernel from very narrow to very wide while watching the fitted curve on the fixed data. Describe the two failure modes at the extremes: what does the fit do to the training points when the bandwidth is tiny, and what does it do to the between-point structure when the bandwidth is huge? Identify, if you can, a region of the input where a narrow setting fits well and another where a wide setting fits better. Then argue from that single observation why no one bandwidth is right everywhere, and how making several bandwidths available at once and learning a convex combination $\sum_i \eta_i K_i$ is the natural response. Which combination rule, the plain sum kernel or MKL, would you expect to keep only the bandwidths that a given dataset actually needs?
7. challenge Work the ridge-versus-lasso specialization and connect it to the algorithm. Take $\mathcal{X} = \mathbb{R}^d$ with the rank-one coordinate kernels $K_i(\mathbf{x}, \mathbf{x}') = x_i x'_i$, whose RKHS is $\{f_i(\mathbf{x}) = \beta_i x_i\}$ with $\|f_i\|_{\mathcal{H}_{K_i}} = |\beta_i|$. Show that learning with the sum kernel is ridge regression (penalty $\sum_i \beta_i^2$) and learning with MKL is lasso regression (penalty $(\sum_i |\beta_i|)^2$), and explain why the second sets coordinates to exactly zero while the first only shrinks them. Now run one simpleMKL step by hand on this problem: at a current weight vector η , the inner solver returns γ^* ; write the gradient component $\partial J(K_\eta)/\partial \eta_i = -\lambda \gamma^{*\top} K_i \gamma^*$ for the coordinate kernel K_i , interpret it as scoring coordinate i by how much of the current fit it explains, and say which coordinate the next projected-gradient update on the simplex will push up. Hint

For the selection contrast, recall that squaring a sum of absolute values keeps the ℓ_1 geometry: the sublevel sets have corners on the axes, so the optimum tends to land where some $\beta_i = 0$. For the gradient, with K_i the rank-one Gram matrix of the i -th coordinate, $\gamma^{*\top} K_i \gamma^* = (\sum_j \gamma_j^* x_{ji})^2 \geq 0$, largest for the coordinate most aligned with the current dual solution; the update raises the η_i with the most negative gradient, i.e. the largest such score.

8. computation Reproduce the centering example. Take $y = (+1, +1, -1, -1)$ and the two rank-one kernels $K_1 = \phi_1 \phi_1^\top$, $K_2 = \phi_2 \phi_2^\top$ from $\phi_1 = (3, 3, 1, 1)$ and $\phi_2 = (1, 0.5, -0.5, -1)$. First prove the identity $\langle K, yy^\top \rangle_F = y^\top K y$ that makes the target alignment a single quadratic form, and use it to recompute the uncentered alignments $A(K_1, yy^\top) = 0.2$ and $A(K_2, yy^\top) = 0.9$. Then apply $H = I - \frac{1}{4} \mathbf{1}\mathbf{1}^\top$, verify $H\phi_1 = y$ so that K_1 centered equals the ideal kernel $yy^\top$, and recompute the centered alignments $A_c(K_1, yy^\top) = 1.0$ and $A_c(K_2, yy^\top) = 0.9$. In one sentence, name the feature property of ϕ_1 that fools the uncentered score and say why centering removes it. Hint

$\langle K, yy^\top \rangle_F = \sum_{ij} K_{ij} y_i y_j = y^\top K y$, and for $K = \phi \phi^\top$ this is $(\phi \cdot y)^2$ while $\|\phi \phi^\top\|_F = \|\phi\|^2$. The offset $+2$ makes $\|\phi_1\|^2 = 20$ though the discriminative part contributes only 4; centering subtracts the feature mean, and $H\mathbf{1} = 0$ kills the offset outright.

9. proof Establish the two facts the two-stage method rests on. (a) Offset invariance: show that for any kernels K, L and any constant c , the centered alignment is unchanged under $K \mapsto K + c \mathbf{1}\mathbf{1}^\top$, that is $A_c(K + c \mathbf{1}\mathbf{1}^\top, L) = A_c(K, L)$; this is the precise sense in which centered alignment ignores the nuisance that misleads the uncentered one. (b) Alignment-maximizing weights: with $a_i = \langle K_i^c, M_y \rangle_F$ and $M_{ij} = \langle K_i^c, K_j^c \rangle_F$,

show that maximizing the centered alignment of $K_\eta = \sum_i \eta_i K_i$ to $yy^\top$ over $\eta \in \mathbb{R}^M$, with the nonnegativity constraint relaxed, is the same as maximizing $a^\top \eta / \sqrt{\eta^\top M \eta}$, and that its maximizer is $\eta^* \propto M^{-1}a$. Hint

For (a) use $H\mathbf{1} = 0$, so $H(K + c\mathbf{1}\mathbf{1}^\top)H = HKH$; both numerator and denominator of A_c depend on K only through HKH . For (b) the ratio is invariant to the scale of η , so fix $\eta^\top M \eta = 1$ and maximize the linear $a^\top \eta$ with a Lagrange multiplier: $\nabla(a^\top \eta - \mu \eta^\top M \eta) = 0$ gives $a = 2\mu M \eta$, hence $\eta \propto M^{-1}a$.

40 Deep Learning from a Kernel Point of View

Deep neural networks and kernel machines are usually told as rival stories: one learns a hierarchy of features from data, the other fixes an infinite-dimensional feature map in advance and learns only a linear function on top of it. This chapter argues that the two are far closer than the rivalry suggests. We will see that the infinite-width limit of a randomly initialized network is a kernel (the random-feature kernel of Neal), that the trajectory of gradient descent on a very wide network is described by another kernel (the neural tangent kernel of Jacot, Gabriel, and Hongler), and that a whole convolutional architecture, pooling and all, can be built as an explicit map into a reproducing kernel Hilbert space so that a convolutional network becomes a single linear function in that space. Once a deep network lives in an RKHS, the RKHS norm becomes a common currency: the very same quantity controls the smoothness of the representation, its stability to deformations of the input, and the generalization of the trained model. We develop this program on images through convolutional kernel networks, then transport it to graphs and to biological sequences, where the same three-move recipe (extract local patterns, embed them with a kernel, pool) yields models that are trained end to end yet retain the interpretability and the regularization theory of kernels.

40.1 What it means to do deep learning with kernels

Before any construction, it is worth naming the objects we will keep meeting, because each is a different bridge between the two worlds. The first is the family of *dot-product kernels*. On vectors these take the form

$$K(\mathbf{x}, \mathbf{x}') = \kappa(\mathbf{x}^\top \mathbf{x}') \quad \text{or, in homogeneous form,} \quad K(\mathbf{x}, \mathbf{x}') = \|\mathbf{x}\| \|\mathbf{x}'\| \kappa\left(\frac{\mathbf{x}^\top \mathbf{x}'}{\|\mathbf{x}\| \|\mathbf{x}'\|}\right).$$

They depend on the inputs only through angles and norms, exactly the quantities a single artificial neuron sees, which is why they are the natural kernels to place at the junction with neural networks. The second object is the *hierarchical composition of feature maps*: a deep kernel is one whose feature map is itself a composition,

$$K(\mathbf{x}, \mathbf{x}') = \langle \Phi(\mathbf{x}), \Phi(\mathbf{x}') \rangle \quad \text{with} \quad \Phi(\mathbf{x}) = \varphi_2(\varphi_1(\mathbf{x})),$$

mirroring the layer-by-layer structure of a network. The third is the *neural tangent kernel*, which captures the asymptotic behavior of overparametrized deep networks trained by gradient descent. The fourth is the family of *convolutional kernel networks* (CKNs), which combine convolutional and hierarchical kernel constructions with end-to-end learning.

That last phrase deserves a definition, since it is the one that sounds paradoxical. Kernels are supposed to be fixed in advance, so what could it mean to *learn* a kernel representation end to end from labels? The answer, which we will make precise through the Nyström method, is that we learn a finite-dimensional linear subspace of the RKHS onto which we project the data, and we choose that subspace by backpropagation to minimize the training loss. Doing deep learning with kernels means learning *where in the RKHS to look*, not abandoning the RKHS.

40.2 Deep kernel machines and dot-product kernels

The simplest way to obtain a deep kernel is to compose two feature maps, and the cleanest place to do it is on the sphere. If φ_1 sends inputs to the unit sphere of some Hilbert space and K is a dot-product kernel there, then

$$K(\mathbf{x}, \mathbf{x}') = \kappa_2(\langle \varphi_1(\mathbf{x}), \varphi_1(\mathbf{x}') \rangle) = \kappa_2(\kappa_1(\mathbf{x}^\top \mathbf{x}'))$$

is again a valid kernel (Cho and Saul, 2009b). Iterating this composition builds arbitrarily deep kernels, each layer a dot-product kernel applied to the output of the previous one. For this to make sense we need to know which functions κ turn an inner product into a positive definite kernel, and a classical theorem answers exactly that.

Theorem (Schoenberg, 1942)

Let $\mathcal{X} = \mathbb{S}$ be the unit sphere of a Hilbert space $\mathcal{H}_0$, and consider $K : \mathcal{X}^2 \rightarrow \mathbb{R}$ given by $K(\mathbf{x}, \mathbf{y}) = \kappa(\langle \mathbf{x}, \mathbf{y} \rangle_{\mathcal{H}_0})$. Then K is positive definite for every $\mathcal{H}_0$ if and only if κ is smooth and admits an expansion $\kappa(u) = \sum_i a_i u^i$ with non-negative coefficients $a_i \geq 0$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The condition is intuitive in light of earlier chapters: a Taylor series with non-negative coefficients is a non-negative combination of the powers $u \mapsto u^i$, and each power $u^i = (\mathbf{x}^\top \mathbf{y})^i$ is the kernel of a tensor-power feature map, hence positive definite; positive definiteness is preserved under non-negative sums and pointwise limits. Requiring it for all $\mathcal{H}_0$, including infinite-dimensional ones, is what forces the coefficients to be non-negative rather than merely making a specific Gram matrix work.

It is worth pausing on why the sphere is the right stage for this construction. A dot-product kernel reads only the angle between two normalized inputs, so composing kernels means composing maps between angles: the first layer turns the raw similarity $\mathbf{x}^\top \mathbf{x}'$ into a new one $\kappa_1(\mathbf{x}^\top \mathbf{x}')$, the second reshapes that similarity again through κ_2 , and so on. Each layer bends the geometry of the sphere, pulling some pairs of directions closer in feature space and pushing others apart, exactly as a layer of a neural network reshapes its representations. Depth, in this picture, is repeated reshaping of angular similarity, and Schoenberg's condition is the guarantee that no reshaping in the chain can ever destroy positive definiteness. This is the first appearance of the chapter's throughline: a deep architecture is a map into an RKHS, and here we are simply stacking such maps.

40.2.1 A catalog of dot-product kernels

Writing $\mathbf{z}, \mathbf{z}'$ for points on the sphere, the kernels that recur in deep-network analysis are the following.

Kernel	Expression $\kappa(\langle \mathbf{z}, \mathbf{z}' \rangle)$
linear	$\langle \mathbf{z}, \mathbf{z}' \rangle$
exponential	$e^{\alpha(\langle \mathbf{z}, \mathbf{z}' \rangle - 1)}$
inverse polynomial	$\frac{1}{2 - \langle \mathbf{z}, \mathbf{z}' \rangle}$
polynomial of degree p	$(c + \langle \mathbf{z}, \mathbf{z}' \rangle)^p$
arc-cosine of degree 1	$\frac{1}{\pi} (\sin \theta + (\pi - \theta) \cos \theta), \quad \theta = \arccos \langle \mathbf{z}, \mathbf{z}' \rangle$
Vovk's of degree 3	$\frac{1}{3} \frac{1 - \langle \mathbf{z}, \mathbf{z}' \rangle^3}{1 - \langle \mathbf{z}, \mathbf{z}' \rangle} = \frac{1}{3} (1 + \langle \mathbf{z}, \mathbf{z}' \rangle + \langle \mathbf{z}, \mathbf{z}' \rangle^2)$

It is worth reading this table as a menu of angular reshapings rather than a list of formulas. Every entry is a univariate function κ of a single number, the cosine of the angle between the two normalized inputs, and each

shapes that cosine differently: the polynomial kernels sharpen it by raising it to a power, so that only nearly aligned directions retain appreciable similarity; the exponential kernel decays smoothly as the angle opens; the arc-cosine kernels bend it into the particular curve a rectified neuron traces. Choosing a dot-product kernel is therefore choosing how aggressively the layer should contract angular similarity, and Schoenberg's condition is the single admissibility test every choice on the menu must pass. Each satisfies it. The exponential kernel is the bridge to the everyday Gaussian: if $\|\mathbf{z}\| = \|\mathbf{z}'\| = 1$, then

$$\kappa_{\text{exp}}(\langle \mathbf{z}, \mathbf{z}' \rangle) = e^{\alpha(\langle \mathbf{z}, \mathbf{z}' \rangle - 1)} = e^{-\frac{\alpha}{2} \|\mathbf{z} - \mathbf{z}'\|^2},$$

because $\|\mathbf{z} - \mathbf{z}'\|^2 = 2 - 2\langle \mathbf{z}, \mathbf{z}' \rangle$ on the sphere. So the Gaussian kernel is a dot-product kernel in disguise once inputs are normalized, and the arc-cosine kernel, as we will now see, is precisely what a layer of rectified neurons computes.

40.3 Random feature kernels: the infinite-width limit

Consider a one-hidden-layer network with m neurons,

$$f_{\theta}(\mathbf{x}) = \frac{1}{\sqrt{m}} \sum_{i=1}^m v_i \sigma(\mathbf{w}_i^{\top} \mathbf{x}),$$

and ask what happens as the width $m \rightarrow \infty$. There are two natural ways to let it grow, and each produces a kernel. In the first, due to Neal (1996) and revived by Rahimi and Recht (2007) under the name random features, we *freeze* the hidden weights at their random initialization $\mathbf{w}_i \sim \mathcal{N}(0, I)$ and train only the output weights $\theta = (v_i)_i$. The network is then linear in its parameters, a linear model over the random features $\mathbf{x} \mapsto \sigma(\mathbf{w}_i^{\top} \mathbf{x})$, and by the law of large numbers the associated kernel converges to an expectation over the weight distribution.

Definition (random feature kernel)

With hidden weights drawn as $\mathbf{w} \sim \mathcal{N}(0, I)$, the random feature kernel is

$$K_{\text{RF}}(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{\mathbf{w} \sim \mathcal{N}(0, I)} [\sigma(\mathbf{w}^{\top} \mathbf{x}) \sigma(\mathbf{w}^{\top} \mathbf{y})].$$

There is a third, older way to read this limit that explains where the kernel comes from in the first place. Fix the inputs and let the random weights vary: the output $f_{\theta}(\mathbf{x})$ of the untrained network is then a random variable, and as the width grows it is a sum of many independent bounded contributions, so by the central limit theorem it becomes Gaussian. A collection of outputs at several inputs becomes jointly Gaussian, which is to say the infinitely wide network at initialization is a Gaussian process, and its covariance function is exactly K_{RF} . This is Neal's (1996) observation, the historical root of the whole subject: a prior over network weights induces a prior over functions, and in the wide limit that prior is a Gaussian process with a kernel we can write down. Freezing the hidden layer and training only the output weights is then nothing but doing kernel regression with that kernel.

Two readings of the same object help here. Statistically, K_{RF} is an expectation, and a width- m network computes an empirical average of m independent draws of the feature product $\sigma(\mathbf{w}^{\top} \mathbf{x}) \sigma(\mathbf{w}^{\top} \mathbf{y})$. By the law of large numbers that average converges to $K_{\text{RF}}(\mathbf{x}, \mathbf{y})$ as m grows, with a fluctuation that shrinks like $1/\sqrt{m}$. Architecturally, this says that a wide random hidden layer is a Monte Carlo estimate of a kernel: every neuron contributes one sample, and widening the layer refines the estimate. This is the precise sense in which an infinitely wide random network and a kernel machine are the same object, and it is worth watching the estimate settle.

The figure plots a kernel against the estimate a finite bank of random features produces, so you can raise the number of features D and watch the Monte Carlo estimate converge onto its infinite-width limit.

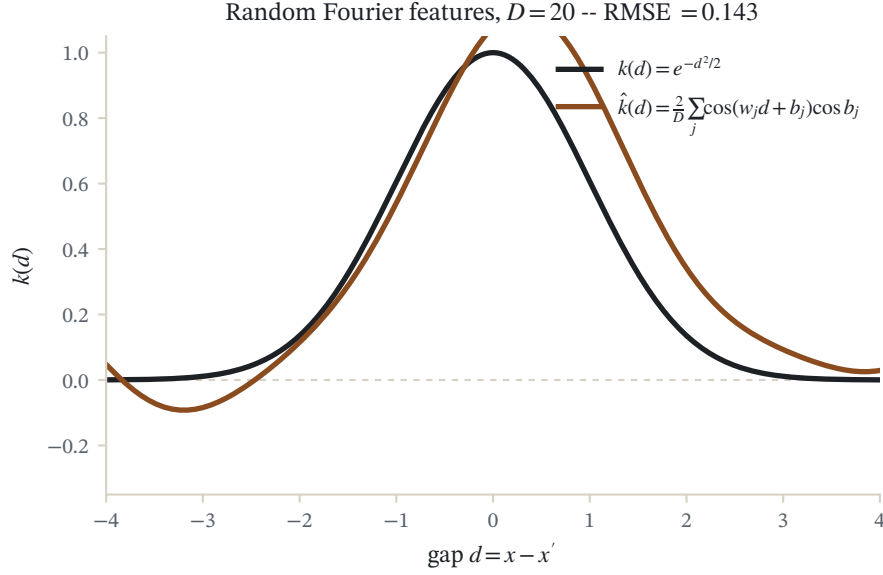


Figure 40.1: The Gaussian kernel $k(x, x') = e^{-(x-x')^2/2}$ (dark curve) against its $D = 20$ random Fourier feature estimate $\hat{k} = \frac{1}{D} \sum_j \cos(w_j x + b_j) \cos(w_j x' + b_j)$, as a function of the gap $x - x'$. Bochner's theorem makes the true kernel the average of these random features.

This is the population inner product of the random features, and the finite network approximates it with m Monte Carlo samples. Chapter 37, Large-Scale Kernel Machines met such integral representations for translation-invariant kernels, where the features were random Fourier modes. The point here is that they are available for many dot-product kernels too. Cho and Saul (2009b) give the family

$$k_n(\mathbf{x}, \mathbf{y}) = \frac{1}{\pi} \|\mathbf{x}\|^n \|\mathbf{y}\|^n J_n(\theta), \quad \theta = \cos^{-1} \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|},$$

the arc-cosine kernels, where the angular part is

$$J_n(\theta) = (-1)^n (\sin \theta)^{2n+1} \left(\frac{1}{\sin \theta} \frac{\partial}{\partial \theta} \right)^n \frac{\pi - \theta}{\sin \theta}.$$

The first three are

$$J_0(\theta) = \pi - \theta, \quad J_1(\theta) = \sin \theta + (\pi - \theta) \cos \theta, \quad J_2(\theta) = 3 \sin \theta \cos \theta + (\pi - \theta)(1 + 2 \cos^2 \theta).$$

Comparing J_1 to the table above shows that k_1 restricted to the sphere is exactly the arc-cosine kernel of degree 1. And these kernels are random feature kernels for a specific activation.

Theorem (Cho and Saul, 2009a)

For the rectified power activation $\sigma(u) = \frac{u^n}{\sqrt{2}} (1 + \text{sign}(u))$, that is $\sigma(u) = \sqrt{2} u^n$ for $u > 0$ and 0 otherwise, the arc-cosine kernel is a random feature kernel:

$$k_n(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{\mathbf{w} \sim \mathcal{N}(0, I)} [\sigma(\mathbf{w}^\top \mathbf{x}) \sigma(\mathbf{w}^\top \mathbf{y})].$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The theorem is not merely asserted here; both low-order cases can be derived by hand from a two-dimensional Gaussian integral, and it is worth doing so once, because the same integral is the engine of the depth recursion below.

Proof (arc-cosine k_0 and k_1)

Both kernels are expectations over $\mathbf{w} \sim \mathcal{N}(0, I)$ of a product of activations, and both depend on the inputs only through their norms and the angle θ , so fix the plane spanned by $\mathbf{x}$ and $\mathbf{y}$ and integrate the in-plane part of $\mathbf{w}$ there. Write it in polar form, radius r with density $r e^{-r^2/2}$ on $[0, \infty)$ and angle φ uniform on $[0, 2\pi)$, independent. Aligning $\mathbf{x}$ with $\varphi = 0$ and placing $\mathbf{y}$ at angle θ ,

$$\mathbf{w}^\top \mathbf{x} = r \|\mathbf{x}\| \cos \varphi, \quad \mathbf{w}^\top \mathbf{y} = r \|\mathbf{y}\| \cos(\varphi - \theta).$$

For degree zero, $\sigma(u) = \sqrt{2} \Theta(u)$ with Θ the Heaviside step, so the product $\sigma(\mathbf{w}^\top \mathbf{x}) \sigma(\mathbf{w}^\top \mathbf{y})$ equals 2 when both cosines are positive and 0 otherwise. Both are positive exactly for $\varphi \in (\theta - \frac{\pi}{2}, \frac{\pi}{2})$, an arc of length $\pi - \theta$, so with φ uniform

$$k_0(\mathbf{x}, \mathbf{y}) = 2 \mathbb{P}(\cos \varphi > 0, \cos(\varphi - \theta) > 0) = 2 \cdot \frac{\pi - \theta}{2\pi} = \frac{\pi - \theta}{\pi} = \frac{J_0(\theta)}{\pi}.$$

For degree one, $\sigma(u) = \sqrt{2} \max(0, u)$, and the radius and angle separate,

$$k_1(\mathbf{x}, \mathbf{y}) = 2 \mathbb{E}[r^2] \|\mathbf{x}\| \|\mathbf{y}\| \cdot \frac{1}{2\pi} \int_{\theta-\pi/2}^{\pi/2} \cos \varphi \cos(\varphi - \theta) d\varphi.$$

The radial moment is $\mathbb{E}[r^2] = \int_0^\infty r^2 \cdot r e^{-r^2/2} dr = 2$. Using $\cos \varphi \cos(\varphi - \theta) = \frac{1}{2}[\cos \theta + \cos(2\varphi - \theta)]$, the angular integral over the arc of length $\pi - \theta$ equals $\frac{1}{2}[\sin \theta + (\pi - \theta) \cos \theta] = \frac{1}{2}J_1(\theta)$. Collecting the constants $2 \cdot 2 \cdot \frac{1}{2\pi} \cdot \frac{1}{2} = \frac{1}{\pi}$,

$$k_1(\mathbf{x}, \mathbf{y}) = \frac{\|\mathbf{x}\| \|\mathbf{y}\|}{\pi} J_1(\theta),$$

which on the unit sphere is $k_1 = J_1(\theta)/\pi$. The same polar reduction with $\sigma(u) = \sqrt{2} u^n$ for $u > 0$ yields J_n for every n . $\square$

The case $n = 1$ is the one to remember, because there the rectified power is proportional to the rectified linear unit and

$$k_1(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{\mathbf{w} \sim \mathcal{N}(0, I)} [\text{ReLU}(\mathbf{w}^\top \mathbf{x}) \text{ReLU}(\mathbf{w}^\top \mathbf{y})].$$

A layer of infinitely many random ReLU neurons computes, in expectation, the degree-1 arc-cosine kernel. This identity is one of the fundamental tools for analyzing ReLU networks, and it makes the arc-cosine kernel the canonical dot-product kernel of the deep-learning era.

There is a vivid geometric reading behind these formulas. Draw a random hyperplane through the origin with normal $\mathbf{w} \sim \mathcal{N}(0, I)$. The degree-0 arc-cosine kernel measures the probability that $\mathbf{x}$ and $\mathbf{y}$ land on the same side of it, and this depends only on the angle θ between them: the closer their directions, the fewer random hyperplanes fall between them, and indeed $J_0(\theta) = \pi - \theta$ decreases linearly as the angle opens from 0 to π . Raising the degree to $n = 1$ keeps this same counting of sign agreements but weights each agreement by how strongly both points

activate the neuron, which is exactly what a random ReLU unit does. The angle, not the raw coordinates, is the only thing these kernels ever see, matching the invariance a rectified neuron has to rotations of its weight vector and giving a concrete geometric meaning to what it means to embed data by a random layer.

40.3.1 The deep limit: the NNGP covariance recursion

Neal's argument was for a single hidden layer, but the same central-limit mechanism applies at every layer of a deep network, and this is what turns one random-feature kernel into a recursion on depth (Lee et al., 2018; Matthews et al., 2018). Consider a fully connected network whose pre-activations at layer ℓ are $h^{(\ell)}$, with weights drawn as $\mathcal{N}(0, \sigma_w^2/n_{\ell-1})$ and biases as $\mathcal{N}(0, \sigma_b^2)$. As each hidden width runs to infinity, the pre-activations at layer ℓ , evaluated at any finite set of inputs, become jointly Gaussian, and their covariance is propagated from the layer below by a single scalar rule.

Proposition (NNGP covariance recursion)

The infinite-width pre-activation covariance obeys $K^{(0)}(\mathbf{x}, \mathbf{y}) = \sigma_w^2 \frac{\mathbf{x}^\top \mathbf{y}}{n_0} + \sigma_b^2$ and, for $\ell \geq 1$,

$$K^{(\ell)}(\mathbf{x}, \mathbf{y}) = \sigma_b^2 + \sigma_w^2 \mathbb{E}_{(u,v) \sim \mathcal{N}(0, \Lambda^{(\ell-1)})} [\phi(u) \phi(v)], \quad \Lambda^{(\ell-1)} = \begin{pmatrix} K^{(\ell-1)}(\mathbf{x}, \mathbf{x}) & K^{(\ell-1)}(\mathbf{x}, \mathbf{y}) \\ K^{(\ell-1)}(\mathbf{x}, \mathbf{y}) & K^{(\ell-1)}(\mathbf{y}, \mathbf{y}) \end{pmatrix}.$$

The map $K^{(\ell-1)} \mapsto K^{(\ell)}$ depends on the previous kernel only through the three covariances in $\Lambda^{(\ell-1)}$, and the expectation it applies is the *dual activation* of ϕ .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The recursion is Neal's central-limit argument repeated layer by layer. Condition on layer $\ell - 1$: its outputs $\phi(h^{(\ell-1)}(\mathbf{x}))$ are fixed vectors, and a unit at layer ℓ forms $h_i^{(\ell)}(\mathbf{x}) = \sum_j W_{ij} \phi(h_j^{(\ell-1)}(\mathbf{x})) + b_i$, a sum of many independent zero-mean terms, hence Gaussian as $n_{\ell-1} \rightarrow \infty$. Its covariance across two inputs is σ_w^2 times the empirical average of $\phi(h_j(\mathbf{x})) \phi(h_j(\mathbf{y}))$ over units j , which by the law of large numbers converges to the expectation under the layer's own Gaussian law $\mathcal{N}(0, \Lambda^{(\ell-1)})$, plus the bias variance. Because each layer hands the next one a Gaussian law, the argument iterates, and the covariance is carried through the whole network by the dual activation alone.

For the ReLU $\phi = \max(0, \cdot)$, the dual-activation expectation is exactly the arc-cosine kernel k_1 derived above, so the NNGP recursion is nothing but the arc-cosine composition of the opening section. On normalized inputs, where each layer is rescaled to keep $K^{(\ell)}(\mathbf{x}, \mathbf{x}) = 1$, it collapses to a one-dimensional map on the correlation $c = K(\mathbf{x}, \mathbf{y})$,

$$c^{(\ell)} = \frac{1}{\pi} \left(\sqrt{1 - (c^{(\ell-1)})^2} + (\pi - \arccos c^{(\ell-1)}) c^{(\ell-1)} \right) = \frac{J_1(\arccos c^{(\ell-1)})}{\pi}.$$

This is the precise sense in which a deep random ReLU network is a Gaussian process whose kernel is a fixed function of depth; the Gaussian-process viewpoint that makes such a prior over functions computable is developed in Chapter 40, Gaussian Processes and the RVM. The recursion is easy to run by hand.

Algorithm (NNGP / arc-cosine kernel by depth)

Input unit inputs $\mathbf{x}, \mathbf{y}$ on the sphere, depth L , ReLU dual activation.

Output NNGP correlation $c^{(L)}$ at depth L .

1. Set $c^{(0)} = \mathbf{x}^\top \mathbf{y}$.
2. For $\ell = 1, \dots, L$: let $\theta = \arccos c^{(\ell-1)}$ and set $c^{(\ell)} = \frac{1}{\pi} (\sin \theta + (\pi - \theta) \cos \theta)$.

3. Return $c^{(L)}$. For a general activation replace the update by $\mathbb{E}_{\mathcal{N}(0, \Lambda)}[\phi(u)\phi(v)]$ normalized by the running diagonal.

Example (arc-cosine kernel and its depth recursion)

Two unit vectors $\mathbf{z} = (1, 0)$ and $\mathbf{z}' = (\frac{1}{2}, \frac{\sqrt{3}}{2})$, so $\langle \mathbf{z}, \mathbf{z}' \rangle = 0.5$ and $\theta_0 = \arccos 0.5 = \pi/3 = 60^\circ$; activation ReLU, weights $\mathbf{w} \sim \mathcal{N}(0, I)$.

1. Evaluate the angular parts. At $\theta_0 = 60^\circ$, $J_0 = \pi - \theta_0 = 2.094395$ and $J_1 = \sin \theta_0 + (\pi - \theta_0) \cos \theta_0 = 0.866025 + 2.094395 \cdot 0.5 = 1.913223$.
2. Read off the two kernels. $k_0 = J_0/\pi = 0.666667$ and $k_1 = J_1/\pi = 0.608998$. The half-space overlap probability is $\mathbb{P}(\text{both} > 0) = (\pi - \theta_0)/(2\pi) = 0.333333$, and indeed $k_0 = 2 \cdot 0.333333$.
3. Cross-check against random features. With 4,000,000 draws $\mathbf{w} \sim \mathcal{N}(0, I)$: $\mathbb{E}[\text{ReLU} \cdot \text{ReLU}] \approx 0.3046$ against the closed $J_1/(2\pi) = 0.3045$, so $2\times$ that ≈ 0.6093 matches $k_1 = 0.609$; and the empirical $\mathbb{P}(\text{both} > 0) \approx 0.3335$ matches 0.3333 .
4. Propagate through depth. Apply $c \mapsto J_1(\arccos c)/\pi$: $c^{(0)} = 0.5 \rightarrow c^{(1)} = 0.608998 \rightarrow c^{(2)} = 0.683906 \rightarrow c^{(3)} = 0.738128$. The correlation climbs monotonically with depth.
5. Locate the fixed point. Iterating the map to convergence gives $c^* \approx 0.999034$, so depth drives every pair of distinct directions toward perfect correlation.

Reading. One layer of infinitely many random ReLUs turns $\cos 60^\circ = 0.5$ into 0.609, and the Monte Carlo layer of 4 million neurons reproduces the closed form to three decimals, the finite-width estimate of the exact kernel. Stacking layers pushes correlations toward 1: this is the degeneracy that makes very deep ReLU NNGP kernels lose discriminative power unless the scale is managed carefully.

Verification artifact. checks/example-ch14-example-39-1.json records the example source hash and verification scope.

40.4 Neural tangent kernels: the trajectory of gradient descent

Freezing the hidden layer is a strong assumption, and the more interesting question is what happens when we train *all* the parameters $\theta = (v_i, \mathbf{w}_i)_i$ by gradient descent from an initialization $\theta_0 \sim \mathcal{N}(0, I)$. The remarkable discovery of Jacot, Gabriel, and Hongler (2018) is that in the infinite-width limit this is again governed by a kernel, though a different one. The mechanism is what Chizat, Oyallon, and Bach (2019) call *lazy training*: when m is large, the parameters barely move, θ stays close to θ_0 throughout training, and the network is well approximated by its first-order Taylor expansion around initialization,

$$f_\theta(\mathbf{x}) \approx f_{\theta_0}(\mathbf{x}) + \langle \theta - \theta_0, \nabla_\theta f_{\theta_0}(\mathbf{x}) \rangle_{\theta=\theta_0}.$$

This linearized model is linear in the deviation $\theta - \theta_0$, with feature map the gradient $\nabla_\theta f_{\theta_0}(\mathbf{x})$. Training it by gradient descent is therefore linear regression in that fixed feature space, and its kernel is the inner product of the gradients.

It is worth being precise about why training should be lazy, because the phenomenon can look like magic otherwise. The $1/\sqrt{m}$ prefactor is doing the work. To change the output $f_\theta(\mathbf{x})$ by an order-one amount, gradient descent needs to move the whole population of m neurons only by an order-one amount collectively, which means each individual weight need only move by about $1/\sqrt{m}$. As the width grows, that per-parameter displacement shrinks to nothing, so every weight stays in a vanishing neighborhood of its initialization even though the function the network computes changes appreciably. Small displacements are exactly the regime in which the first-order Taylor expansion is accurate, so the nonlinear network behaves like its own linearization for the entire duration of training. Lazy training is thus not an assumption bolted on for convenience; it is a consequence of the scaling, and it is what freezes the feature map $\nabla_\theta f_{\theta_0}$ into a fixed object that gradient descent never gets to reshape.

Definition (neural tangent kernel)

$$K_{\text{NTK}}(\mathbf{x}, \mathbf{y}) = \lim_{m \rightarrow \infty} \langle \nabla_{\theta} f_{\theta_0}(\mathbf{x}), \nabla_{\theta} f_{\theta_0}(\mathbf{y}) \rangle.$$

Under the NTK parameterization and the regularity assumptions of Jacot, Gabriel, and Hongler (2018), the limit exists and is deterministic: as every hidden width grows, the random gradient inner product concentrates on a fixed kernel and stays constant over finite training time. The conclusion concerns this particular infinite-width, lazy-training regime; it is not a statement about arbitrary finite networks or parameterizations.

Proposition (NTK gradient flow; Jacot, Gabriel, and Hongler 2018)

For least-squares loss, under NTK scaling and in the infinite-width limit, the vector of training predictions follows the linear kernel-gradient-flow equation governed by K_{NTK} . Training to convergence with no weight decay gives the corresponding zero-ridge kernel interpolant, subject to the contribution of the initial function. Explicit weight decay yields a ridge-type solution, while stopping at finite time applies a time-dependent spectral filter rather than a single ridge penalty.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

For our two-layer network the gradient has two blocks, one from the output weights v_i and one from the hidden weights $\mathbf{w}_i$, and computing the inner product of each in the limit gives

$$K_{\text{NTK}}(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{\mathbf{w}} [\sigma(\mathbf{w}^T \mathbf{x}) \sigma(\mathbf{w}^T \mathbf{y}) + (\mathbf{x}^T \mathbf{y}) \sigma'(\mathbf{w}^T \mathbf{x}) \sigma'(\mathbf{w}^T \mathbf{y})].$$

The first term is exactly the random feature kernel from differentiating with respect to the output weights; the second, from differentiating with respect to the hidden weights, brings in the derivative σ' and an explicit $\mathbf{x}^T \mathbf{y}$ factor. With ReLU networks σ' is a step function, both expectations are functions of the angle alone, and K_{NTK} is again a dot-product kernel, assembled from the arc-cosine kernels of the previous section.

The decomposition also sharpens the difference between the two limits. Freezing the hidden layer keeps only the first term, the random feature kernel, because the hidden weights contribute no gradient; training all the parameters adds the second term, which is precisely the extra similarity carried by the hidden weights being allowed to move a little. That the two limits give different kernels is the whole point: the random feature kernel describes a network that never adapts its features, while the neural tangent kernel describes one whose features do move, but only infinitesimally, so that the movement is still captured by a fixed linear model. Neither limit performs genuine feature learning, in which the representation would reorganize itself around the data; that reorganization is the subject of Chapter 43, *The Frontier: Feature Learning and Beyond*. That is what makes both objects tractable and, at the same time, what marks the boundary of what these kernels can explain about finite networks, which do learn their features. The lesson to carry forward is that the very same wide network yields a kernel whether we freeze it or train it; only which kernel changes.

40.4.1 Depth: the neural tangent kernel recursion

The two-term formula was written for a single hidden layer, but for a deep network the tangent kernel obeys its own recursion, assembled from the same dual activations that drove the NNGP recursion (Jacot et al., 2018; Arora et al., 2019). Alongside the NNGP kernel $\Sigma^{(\ell)}$, which is the dual activation of σ , one needs the *derivative kernel* $\dot{\Sigma}^{(\ell)}$, the dual activation of σ' , because backpropagating a gradient through a layer multiplies by σ' .

Proposition (deep NTK recursion)

For an L -layer network with activation σ , let $\Sigma^{(\ell)}$ be the NNGP kernels of the previous recursion and define the derivative kernels

$$\dot{\Sigma}^{(\ell)}(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{(u,v) \sim \mathcal{N}(0, \Lambda^{(\ell-1)})} [\sigma'(u) \sigma'(v)], \quad \Lambda^{(\ell-1)} \text{ built from } \Sigma^{(\ell-1)}.$$

Then the neural tangent kernel is assembled by

$$\Theta^{(0)} = \Sigma^{(0)}, \quad \Theta^{(\ell)}(\mathbf{x}, \mathbf{y}) = \Sigma^{(\ell)}(\mathbf{x}, \mathbf{y}) + \Theta^{(\ell-1)}(\mathbf{x}, \mathbf{y}) \dot{\Sigma}^{(\ell)}(\mathbf{x}, \mathbf{y}).$$

For ReLU the two dual activations are the arc-cosine kernels: $\Sigma^{(\ell)}$ uses k_1 and $\dot{\Sigma}^{(\ell)}$ uses k_0 .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Each layer contributes one block to the parameter gradient. Differentiating the output with respect to the weights of layer ℓ produces the product of the representation feeding into that layer, whose inner product is the NNGP kernel $\Sigma^{(\ell-1)}$, with the Jacobian backpropagated from ℓ to the top, whose inner product is a running product of derivative kernels. Summing the per-layer contributions and grouping them telescopes into the recursion above: each deeper layer multiplies the accumulated tangent kernel by the local derivative kernel $\dot{\Sigma}^{(\ell)}$ and adds the fresh NNGP term $\Sigma^{(\ell)}$. The two-term formula for one hidden layer is exactly the case $L = 1$, whose summands $\Sigma^{(1)}$ and $\Theta^{(0)}\dot{\Sigma}^{(1)} = (\mathbf{x}^\top \mathbf{y}) \mathbb{E}[\sigma' \sigma']$ are the two terms above. Because $\sigma' = \Theta$ for ReLU, the derivative kernel is the degree-0 arc-cosine kernel and the NNGP kernel is the degree-1, so the whole deep NTK is built from the closed forms of the opening section; its spectral behavior, which governs what such a kernel can learn, is the concern of Chapter 18, Mercer's Theorem, Spectra, and Rates, and its finite approximation that of Chapter 37, Large-Scale Kernel Machines.

Example (deep ReLU NTK by recursion)

The same two unit vectors, $c^{(0)} = 0.5$, $\theta_0 = 60^\circ$. The normalization (a factor 2 in each dual activation) keeps the diagonal at 1, so on the sphere $\Sigma^{(\ell)} = J_1(\theta_{\ell-1})/\pi$ and $\dot{\Sigma}^{(\ell)} = (\pi - \theta_{\ell-1})/\pi$ with $\theta_{\ell-1} = \arccos c^{(\ell-1)}$. Depth $L = 3$.

1. Layer 1. From $c^{(0)} = 0.5$: $\Sigma^{(1)} = 0.608998$, $\dot{\Sigma}^{(1)} = 0.666667$, so $\Theta^{(1)} = \Sigma^{(1)} + \Theta^{(0)}\dot{\Sigma}^{(1)} = 0.608998 + 0.5 \cdot 0.666667 = 0.942331$.
2. Layer 2. Now $c^{(1)} = 0.608998$: $\Sigma^{(2)} = 0.683906$, $\dot{\Sigma}^{(2)} = 0.708428$, so $\Theta^{(2)} = 0.683906 + 0.942331 \cdot 0.708428 = 1.351480$.
3. Layer 3. Now $c^{(2)} = 0.683906$: $\Sigma^{(3)} = 0.738128$, $\dot{\Sigma}^{(3)} = 0.739720$, so $\Theta^{(3)} = 0.738128 + 1.351480 \cdot 0.739720 = 1.737845$.
4. Normalize. On the diagonal every dual activation is 1, so $\Theta^{(L)}(\mathbf{x}, \mathbf{x}) = L + 1 = 4$; the NTK correlation is $1.737845/4 = 0.434461$.

Reading. The tangent kernel is assembled from the same arc-cosine pieces as the NNGP kernel, but the accumulation $\Theta^{(\ell)} = \Sigma^{(\ell)} + \Theta^{(\ell-1)}\dot{\Sigma}^{(\ell)}$ makes it grow with depth. After three layers two inputs at 60° have NTK correlation 0.434, well below the NNGP correlation 0.738 at the same depth, because the tangent kernel keeps a memory of every layer's contribution rather than only the last one.

Verification artifact. checks/example-ch14-example-39-2.json records the example source hash and verification scope.

40.5 The Nyström bridge and end-to-end learning

The kernels above are infinite-dimensional, so to compute with them we need a finite approximation, and the approximation itself turns out to look like a neural-network layer. This is the technical heart of the chapter, so we develop it carefully. Given a kernel K with feature map φ into an RKHS $\mathcal{H}$, the Nyström method replaces any point $\varphi(\mathbf{x})$ by its orthogonal projection onto a finite-dimensional subspace spanned by the embeddings of p anchor points $Z = [\mathbf{z}_1, \dots, \mathbf{z}_p]$,

$$\mathcal{F} = \text{span}(\varphi(\mathbf{z}_1), \dots, \varphi(\mathbf{z}_p)).$$

The projection is the best approximation of $\varphi(\mathbf{x})$ inside $\mathcal{F}$,

$$\Pi_{\mathcal{F}}[\mathbf{x}] = \sum_{j=1}^p \beta_j^* \varphi(\mathbf{z}_j), \quad \beta^* \in \arg \min_{\beta \in \mathbb{R}^p} \left\| \varphi(\mathbf{x}) - \sum_{j=1}^p \beta_j \varphi(\mathbf{z}_j) \right\|_{\mathcal{H}}^2,$$

a least-squares problem solved by the normal equations. Writing everything through the kernel and, for a dot-product kernel $K(\mathbf{x}, \mathbf{y}) = \kappa(\langle \mathbf{x}, \mathbf{y} \rangle)$, approximating the kernel by the inner product of projections gives a finite-dimensional embedding ψ ,

$$K(\mathbf{x}, \mathbf{y}) \approx \langle \Pi_{\mathcal{F}}[\mathbf{x}], \Pi_{\mathcal{F}}[\mathbf{y}] \rangle_{\mathcal{H}} = \langle \psi(\mathbf{x}), \psi(\mathbf{y}) \rangle_{\mathbb{R}^p}, \quad \psi(\mathbf{x}) = \kappa(Z^{\top} Z)^{-1/2} \kappa(Z^{\top} \mathbf{x}),$$

where κ is applied entrywise. Read the right-hand side left to right: $Z^{\top} \mathbf{x}$ is a linear map, $\kappa(\cdot)$ is a pointwise nonlinearity, and multiplication by $\kappa(Z^{\top} Z)^{-1/2}$ is a second linear map. The Nyström embedding of a dot-product kernel is a small neural network layer: linear, nonlinearity, linear (Williams and Seeger, 2001; Smola and Schölkopf, 2000; Fine and Scheinberg, 2001).

This is what lets us learn kernels end to end. The encoding $\psi_Z(\mathbf{x}) = \kappa(Z^{\top} Z)^{-1/2} \kappa(Z^{\top} \mathbf{x})$ is parametrized by the anchors Z , and there are two ways to choose them. Unsupervised: run K-means on the data and take the centroids as anchors, so that $\mathcal{F}$ covers the region of the RKHS where data actually live. Supervised: treat Z as network parameters and optimize them by backpropagation jointly with a linear predictor,

$$\min_{\mathbf{w}, Z} \frac{1}{n} \sum_{i=1}^n L(y_i, \mathbf{w}^{\top} \psi_Z(\mathbf{x}_i)) + \lambda \|\mathbf{w}\|^2.$$

End-to-end learning with kernels, then, means learning a parametrized linear subspace $\mathcal{F}$ of the RKHS onto which the data are projected. The kernel supplies the geometry; learning chooses the subspace.

40.6 Convolutional kernel networks

We now scale this idea up from a single layer to a whole convolutional architecture. Mairal (2016) shows it is possible to design a functional space $\mathcal{H}$, an RKHS, in which deep convolutional networks live: for a network

$$f(\mathbf{x}) = \sigma_k(A_k \sigma_{k-1}(A_{k-1} \cdots \sigma_2(A_2 \sigma_1(A_1 \mathbf{x})) \cdots)) = \langle f, \Phi(\mathbf{x}) \rangle_{\mathcal{H}},$$

the entire nonlinear network becomes a single linear functional of a fixed feature map Φ . The construction is called a *convolutional kernel network*. The recipe has a theoretical and a practical half. For the theory, replace each layer's map $\mathbf{x} \mapsto \sigma(A\mathbf{x})$ by a kernel mapping $\mathbf{x} \mapsto \varphi(\mathbf{x})$ associated with a dot-product kernel. For practice, replace it by the Nyström embedding $\mathbf{x} \mapsto \kappa(Z^{\top} Z)^{-1/2} \kappa(Z^{\top} \mathbf{x})$, whose anchors Z are learned by K-means (unsupervised) or backpropagation (supervised), exactly as above.

Two features of this decomposition make it worth the effort. First, Φ depends only on the architecture and is *independent of the training data*, so it can be analyzed once and for all: we can ask whether it is stable to perturbations and whether it loses information about the signal. Second, f is a predictor whose behavior is controlled by its RKHS norm through the defining inequality of any RKHS,

$$|f(\mathbf{x}) - f(\mathbf{x}')| \leq \|f\|_{\mathcal{H}} \|\Phi(\mathbf{x}) - \Phi(\mathbf{x}')\|_{\mathcal{H}}.$$

The right-hand side factorizes cleanly. The geometry factor $\|\Phi(\mathbf{x}) - \Phi(\mathbf{x}')\|_{\mathcal{H}}$ is a property of the architecture alone; the complexity factor $\|f\|_{\mathcal{H}}$ is a property of the learned model. Controlling each separately is the program of the rest of the chapter.

This is the point at which the chapter's throughline stops being a slogan and becomes a construction. In the random-feature and neural-tangent stories the kernel appeared as a limit, something a network approaches as its width runs to infinity. Here the roles are reversed: we build the RKHS first, by hand, layer by layer, and then observe that an ordinary convolutional network sits inside it as one linear functional f of the feature map Φ . Nothing has been sent to a limit. A deep architecture is, quite literally, a map Φ into an RKHS, and training it is choosing the linear functional and, through the anchor points, the finite subspace of that RKHS on which the map lands. Because Φ is fixed once the architecture is fixed, every question about the representation becomes a question about the fixed geometry of $\mathcal{H}$, and every question about the trained predictor becomes a question about the single scalar $\|f\|_{\mathcal{H}}$. The two factors above are the formal expression of that separation.

40.7 Constructing the RKHS for continuous signals

To analyze stability to deformations honestly we must work with continuous signals, where a deformation is a smooth warp of the domain rather than a permutation of pixels. So we model an image as a map, not an array. The input is a continuous signal

$$\mathbf{x}_0 : \Omega \rightarrow \mathcal{H}_0, \quad \mathbf{x}_0 \in L^2(\Omega, \mathcal{H}_0),$$

where $\Omega = \mathbb{R}^d$ is the domain ($d = 2$ for images) and $\mathbf{x}_0(u) \in \mathcal{H}_0$ is the value at location u (with $\mathcal{H}_0 = \mathbb{R}^3$ for RGB). Each layer transforms a feature map $\mathbf{x}_{k-1} \in L^2(\Omega, \mathcal{H}_{k-1})$ into the next one $\mathbf{x}_k \in L^2(\Omega, \mathcal{H}_k)$ by composing three operators,

$$\mathbf{x}_k = A_k M_k P_k \mathbf{x}_{k-1}.$$

Each operator has one job, and it is the interplay of the three that makes the theory work.

40.7.1 Patch extraction P_k

The first operator gathers local context. For a patch shape $S_k \subset \Omega$ (a small box, say), P_k stacks the values of the feature map over a neighborhood of each point,

$$P_k \mathbf{x}_{k-1}(u) := (\mathbf{x}_{k-1}(u + v))_{v \in S_k} \in \mathcal{P}_k = \mathcal{H}_{k-1}^{S_k}.$$

So $P_k \mathbf{x}_{k-1}(u)$ is the patch of $\mathbf{x}_{k-1}$ centered at u , living in a product Hilbert space. This is a linear operator, and it is what will fail to commute with deformations at high frequencies, the technical obstacle we confront later.

40.7.2 Nonlinear mapping M_k

The second operator embeds each patch through the kernel. Pointwise in u ,

$$M_k P_k \mathbf{x}_{k-1}(u) := \varphi_k(P_k \mathbf{x}_{k-1}(u)) \in \mathcal{H}_k,$$

where φ_k is the feature map of a homogeneous dot-product kernel on patches,

$$K_k(\mathbf{z}, \mathbf{z}') = \|\mathbf{z}\| \|\mathbf{z}'\| \kappa_k\left(\frac{\langle \mathbf{z}, \mathbf{z}' \rangle}{\|\mathbf{z}\| \|\mathbf{z}'\|}\right) = \langle \varphi_k(\mathbf{z}), \varphi_k(\mathbf{z}') \rangle, \quad \kappa_k(u) = \sum_{j=0}^{\infty} b_j u^j, \quad b_j \geq 0, \quad \kappa_k(1) = 1.$$

The non-negative coefficients guarantee positive definiteness (Schoenberg), and $\kappa_k(1) = 1$ normalizes the map so that $\|\varphi_k(\mathbf{z})\| = \|\mathbf{z}\|$. Standard choices are the Gaussian kernel on the sphere $\kappa_{\text{exp}}(\langle \mathbf{z}, \mathbf{z}' \rangle) = e^{\langle \mathbf{z}, \mathbf{z}' \rangle - 1}$ and the inverse polynomial $\kappa_{\text{inv-poly}}(\langle \mathbf{z}, \mathbf{z}' \rangle) = 1/(2 - \langle \mathbf{z}, \mathbf{z}' \rangle)$. Being pointwise, M_k will commute with any warp of the domain, which is why it never causes trouble in the stability analysis.

40.7.3 Pooling A_k

The third operator blurs, providing local invariance. It is a linear convolution against a Gaussian pooling filter at scale σ_k ,

$$\mathbf{x}_k(u) = A_k M_k P_k \mathbf{x}_{k-1}(u) = \int_{\mathbb{R}^d} h_{\sigma_k}(u - v) M_k P_k \mathbf{x}_{k-1}(v) dv \in \mathcal{H}_k,$$

with $h_{\sigma_k}(u) = \sigma_k^{-d} h(u/\sigma_k)$ and h a Gaussian. Two properties matter. First, A_k is linear and non-expansive, $\|A_k\| \leq 1$, so it can never amplify a perturbation. Second, in practice pooling is followed by *discretization*: we subsample at resolution σ_k , which is what lets the feature maps shrink from layer to layer. The blur before the subsample is anti-aliasing, and information is preserved as long as the subsampling factor does not exceed the patch size, a claim we make precise in the discretization section.

40.7.4 The multilayer representation and the CKN kernel

Composing n layers gives the feature map of the whole network,

$$\Phi_n(\mathbf{x}) = A_n M_n P_n A_{n-1} M_{n-1} P_{n-1} \cdots A_1 M_1 P_1 \mathbf{x}_0 \in L^2(\Omega, \mathcal{H}_n).$$

In practice the patch sizes S_k and pooling scales σ_k grow exponentially with depth, because each layer is subsampled and so a fixed patch covers an exponentially larger region of the original image. A prediction is a linear functional of this representation, $f(\mathbf{x}) = \langle \mathbf{w}, \Phi_n(\mathbf{x}) \rangle$, and the corresponding kernel is the "linear kernel" on top of the feature maps,

$$K_{\text{CKN}}(\mathbf{x}, \mathbf{x}') = \langle \Phi_n(\mathbf{x}), \Phi_n(\mathbf{x}') \rangle_{L^2(\Omega)} = \int_{\Omega} \langle \mathbf{x}_n(u), \mathbf{x}'_n(u) \rangle du.$$

Finally, real inputs are discrete signals acquired by a physical device, which we model as $\mathbf{x}_0 = A_0 \mathbf{x}$, where $\mathbf{x}$ is the underlying continuous signal and A_0 is a local integrator at scale σ_0 performing anti-aliasing. The full representation is then $\Phi_n(A_0 \mathbf{x})$, and this is the object whose stability we study.

40.8 Convolutional kernel networks in practice

It helps to see exactly where a CKN and a CNN part ways, because they differ in one operator only. Given a patch $\mathbf{x}$, a convolutional network computes

$$\psi_{\text{CNN}}(\mathbf{x}) = \sigma(Z^T \mathbf{x}),$$

a linear map followed by a pointwise nonlinearity, with Z the filters. A convolutional kernel network computes the Nyström embedding of the patch kernel,

$$\psi_{\text{CKN}}(\mathbf{x}) = \|\mathbf{x}\| \kappa(Z^T Z)^{-1/2} \kappa(Z^T \mathbf{x} / \|\mathbf{x}\|).$$

The extra factor $\kappa(Z^\top Z)^{-1/2}$ is precisely the projection onto the subspace $\mathcal{F}$, the geometric content that distinguishes learning a subspace of an RKHS from applying an arbitrary nonlinearity. Everything downstream flows from this one change. We inherit a geometric interpretation of the filters as anchor points defining a subspace, an unsupervised training route through K-means, a still-feasible supervised route through backpropagation, and regularization mechanisms from the kernel viewpoint. Because the representation is a genuine kernel embedding, it can in principle be reused in other kernel machinery: statistical testing, kernel PCA, CCA, K-means.

On CIFAR-10 the construction is competitive rather than merely illustrative. A 14-layer supervised model with 256 anchor points per layer reaches about 92% accuracy with only mild data augmentation, vanilla SGD with momentum, and no batch normalization. A two-layer unsupervised model with a very large number of anchor points, from 1024 up to 16384, and no augmentation reaches about 86%, and its accuracy increases monotonically with the number of anchors, because more anchors mean a better kernel approximation. Computing the exact kernel is impractical but feasible on many CPUs, and a three-layer exact model reaches about 90% (unpublished, by A. Bietti), consistent with the findings of Shankar et al. (2020). Related lines of work connect kernels to deep learning through hierarchical kernel descriptors (Bo et al., 2011), other multilayer models (Bouvier et al., 2009; Montavon et al., 2011; Anselmi et al., 2015), deep Gaussian processes (Damianou and Lawrence, 2013), multilayer PCA (Schölkopf et al., 1998), old image kernels (Schölkopf, 1997) that amount to a one-layer CKN, and RBF networks (Broomhead and Lowe, 1988).

40.9 Deep learning and stability to deformations

We now cash in the promise that Φ can be analyzed independently of the data. The results are those of Bietti and Mairal (2019a): a multilayer construction of the RKHS $\mathcal{H}$ that contains convolutional networks with smooth homogeneous activations; conditions under which the feature map Φ preserves the signal, is stable to deformations, and is non-expansive; constructions achieving invariance to groups beyond translation; and bounds on the RKHS norm $\|\cdot\|_{\mathcal{H}}$ that control both stability and generalization of a learned model, through the same inequality

$$|f(\mathbf{x}) - f(\mathbf{x}')| \leq \|f\|_{\mathcal{H}} \|\Phi(\mathbf{x}) - \Phi(\mathbf{x}')\|_{\mathcal{H}}.$$

40.9.1 Smooth homogeneous activations

For a network to live inside this RKHS, its activation must be admissible, and the plain ReLU is not: it is not smooth at the origin. The fix is to homogenize. Instead of $\mathbf{z} \mapsto \text{ReLU}(\mathbf{w}^\top \mathbf{z})$, the kernel construction uses

$$\mathbf{z} \mapsto \|\mathbf{z}\| \sigma(\mathbf{w}^\top \mathbf{z} / \|\mathbf{z}\|),$$

where σ is a smooth surrogate of the ReLU (a "sReLU"). The factor $\|\mathbf{z}\|$ restores the positive homogeneity that the ReLU has and that a bare smooth function would lack, so that the map is homogeneous of degree one, like the original neuron, while being differentiable everywhere the analysis needs.

40.9.2 Deformations and the notion of stability

A deformation is a C^1 -diffeomorphism $\tau : \Omega \rightarrow \Omega$ acting on signals through

$$L_\tau \mathbf{x}(u) = \mathbf{x}(u - \tau(u)).$$

This is a far richer group of transformations than pure translations $L_c \mathbf{x}(u) = \mathbf{x}(u - c)$: patterns are not only shifted but locally warped. The right notion of robustness, following Mallat's (2012) analysis of the scattering transform (Bruna and Mallat, 2013), asks the representation to change little under small deformations.

Definition (stability, Mallat, 2012)

A representation Φ is stable to deformations if there are constants C_1, C_2 such that

$$\|\Phi(L_\tau \mathbf{x}) - \Phi(\mathbf{x})\| \leq (C_1 \|\nabla \tau\|_\infty + C_2 \|\tau\|_\infty) \|\mathbf{x}\|,$$

where $\|\nabla \tau\|_\infty = \sup_u \|\nabla \tau(u)\|$ measures the size of the deformation and $\|\tau\|_\infty = \sup_u |\tau(u)|$ measures the maximal displacement.

The second term is a translation term; driving $C_2 \rightarrow 0$ yields translation invariance. The goal is a representation for which C_1 is small (genuine deformation stability) and C_2 can be made small at will (tunable invariance).

40.9.3 Warmup: translation invariance

Pure translation is instructive because every operator in the network is *equivariant* to it: patch extraction, the pointwise nonlinear map, and pooling all commute with a shift, $\square L_c = L_c \square$. Therefore the shifted representation is the shift of the representation, and the whole error collapses onto the last pooling layer:

$$\|\Phi_n(L_c \mathbf{x}) - \Phi_n(\mathbf{x})\| = \|L_c \Phi_n(\mathbf{x}) - \Phi_n(\mathbf{x})\| \leq \|L_c A_n - A_n\| \cdot \|M_n P_n \Phi_{n-1}(\mathbf{x})\| \leq \|L_c A_n - A_n\| \|\mathbf{x}\|.$$

Mallat's estimate on how much a shift disturbs a Gaussian pooling of scale σ_n is $\|L_c A_n - A_n\| \leq (C_2/\sigma_n) c$. So the last layer's pooling scale σ_n directly controls translation invariance: the coarser the final pooling, the more shift-invariant the representation. This is the mechanism a global average-pooling layer exploits.

40.9.4 Stability to deformations

General deformations are harder because patch extraction P_k and pooling A_k do *not* commute with L_τ ; only the pointwise map M_k does. The disturbance is measured by commutators. For pooling, Mallat's bound gives

$$\|[A_k, L_\tau]\| = \|A_k L_\tau - L_\tau A_k\| \leq C_1 \|\nabla \tau\|_\infty,$$

which is well behaved. Patch extraction is the problem: $[P_k, L_\tau]$ is unstable at high frequencies, because a fixed patch operator resolves detail finer than the current feature map can safely carry under a warp. The remedy is to adapt the patch to the current resolution. If the patch of layer k is applied after the pooling of layer $k-1$, so that its size is controlled by σ_{k-1} , the commutator is tamed,

$$\|[P_k A_{k-1}, L_\tau]\| \leq C_{1,\kappa} \|\nabla \tau\|_\infty, \quad \sup_{u \in S_k} |u| \leq \kappa \sigma_{k-1},$$

where κ is the ratio of patch size to resolution. The constant grows as $C_{1,\kappa} = O(\kappa^{d+1})$, so small patches relative to the resolution are more stable. This is a first-principles reason to prefer the small 3-by-3 patches of VGG-style architectures. Assembling the layerwise bounds yields the main theorem.

Theorem (stability of CKNs, Bietti and Mairal, 2019a)

Let $\Phi_n(\mathbf{x}) = \Phi(A_0 \mathbf{x})$ and assume $\|\nabla \tau\|_\infty \leq 1/2$. Then

$$\|\Phi_n(L_\tau \mathbf{x}) - \Phi_n(\mathbf{x})\| \leq \left(C\beta (n+1) \|\nabla \tau\|_\infty + \frac{C}{\sigma_n} \|\tau\|_\infty \right) \|\mathbf{x}\|.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The bound reads as a design guide. Translation invariance comes from a large final scale σ_n , which shrinks the second term. Deformation stability comes from small patches, since β is the patch size and the deformation constant behaves as $C_\beta = O(\beta^3)$ for images. Signal preservation requires the subsampling factor to be roughly the patch size. Because each layer can only subsample by about β , reaching a final scale σ_n from an initial σ_0 needs several layers,

$$n = O\left(\frac{\log(\sigma_n/\sigma_0)}{\log \beta}\right).$$

Small patches force depth: the network must be deep precisely because we insisted on small, stable patches. The proof rests on controlling the commutator norm $\|[L_\tau, P_k A_{k-1}]\|$, extending Mallat's (2012) control of $\|[L_\tau, A]\|$, and requires patches S_k adapted to the resolution, $\text{diam } S_k \leq \beta \sigma_{k-1}$.

40.9.5 Invariance to other groups

Nothing above is special to translations. To build invariance to a locally compact group G (rotations, reflections), let the group act by $L_g \mathbf{x}(u) = \mathbf{x}(g^{-1}u)$ and define feature maps on G itself. The recipe is equivariant inner layers followed by a global pooling in the last layer. Patch extraction becomes $P\mathbf{x}(u) = (\mathbf{x}(uv))_{v \in S}$; the nonlinear map is equivariant automatically because it is pointwise; and pooling integrates against the left-invariant Haar measure μ ,

$$A\mathbf{x}(u) = \int_G \mathbf{x}(uv) h(v) d\mu(v) = \int_G \mathbf{x}(v) h(u^{-1}v) d\mu(v).$$

This recovers and generalizes group-equivariant constructions (Sifre and Mallat, 2013; Cohen and Welling, 2016; Raj et al., 2016). The same analysis extends to the neural tangent kernel of infinitely wide convolutional networks (Bietti and Mairal, 2019b), which is stable too, with the weaker rate

$$\|\Phi_n(L_\tau \mathbf{x}) - \Phi_n(\mathbf{x})\| \leq \left(C\beta n^{7/4} \|\nabla \tau\|_\infty^{1/2} + C'\beta n^2 \|\nabla \tau\|_\infty + \sqrt{n+1} \frac{C}{\sigma_n} \|\tau\|_\infty \right) \|\mathbf{x}\|.$$

The half-power on $\|\nabla \tau\|_\infty$ makes the convolutional NTK less deformation-stable than the CKN, a concrete difference between the lazy-training kernel and the architectural kernel.

40.10 Discretization and signal preservation

We promised that subsampling loses no information as long as its factor stays below the patch size, and we now show why in one dimension, because it explains what the "filters" of a CKN really encode. Write $\bar{\mathbf{x}}_k \in \ell^2(\mathbb{Z}, \mathcal{H}_k)$ for the discrete feature maps, obtained from the continuous $\mathbf{x}_k \in L^2(\mathbb{R}, \mathcal{H}_k)$ by subsampling with factor s_k after pooling at a matching scale $\sigma_k \approx s_k$,

$$\bar{\mathbf{x}}_k[n] = \bar{A}_k \bar{M}_k \bar{P}_k \bar{\mathbf{x}}_{k-1}[ns_k].$$

Claim (recovery)

If s_k is at most the patch size, then $\bar{\mathbf{x}}_{k-1}$ can be recovered from $\bar{\mathbf{x}}_k$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The mechanism is that the RKHS $\mathcal{H}_k$ contains the linear functionals of a patch, so we can read the patch back out of its embedding. For a linear function $f_{\mathbf{w}}$, evaluating it against the embedded patch recovers a linear measurement of the raw patch,

$$\langle f_{\mathbf{w}}, \bar{M}_k \bar{P}_k \bar{\mathbf{x}}_{k-1}(u) \rangle = f_{\mathbf{w}}(\bar{P}_k \bar{\mathbf{x}}_{k-1}(u)) = \langle \mathbf{w}, \bar{P}_k \bar{\mathbf{x}}_{k-1}(u) \rangle,$$

and running $\mathbf{w}$ over an orthonormal basis $\mathcal{B}$ of patch space reconstructs the whole patch,

$$\bar{P}_k \bar{\mathbf{x}}_{k-1}(u) = \sum_{\mathbf{w} \in \mathcal{B}} \langle f_{\mathbf{w}}, \bar{M}_k \bar{P}_k \bar{\mathbf{x}}_{k-1}(u) \rangle \mathbf{w}.$$

Given the patches at every location one can deconvolve the pooling and recover $\bar{\mathbf{x}}_{k-1}$, provided the subsampling was not so aggressive as to skip patches, hence the "factor at most patch size" condition. One warning: this is an information-theoretic statement, with no claim that the recovery is practical or numerically stable. It is the pre-image of the filters in principle, not a recommended algorithm.

40.11 What the patch RKHS contains, and CNNs inside it

Stability told us about the map Φ ; to talk about the learned model f we need to know which functions live in the RKHS and what their norms are. For the homogeneous patch kernel $K_k(\mathbf{z}, \mathbf{z}') = \|\mathbf{z}\| \|\mathbf{z}'\| \kappa(\langle \mathbf{z}, \mathbf{z}' \rangle / (\|\mathbf{z}\| \|\mathbf{z}'\|))$ with $\kappa(u) = \sum_{j \geq 0} b_j u^j$, the RKHS contains all homogeneous functions of the form

$$f : \mathbf{z} \mapsto \|\mathbf{z}\| \sigma(\langle \mathbf{g}, \mathbf{z} \rangle / \|\mathbf{z}\|),$$

for any smooth activation $\sigma(u) = \sum_{j \geq 0} a_j u^j$ with non-negative coefficients, exactly the homogenized neurons of the stability analysis. Their norm is bounded through the coefficients of σ and κ ,

$$\|f\|_{\mathcal{H}_k}^2 \leq C_\sigma^2(\|\mathbf{g}\|^2) = \sum_{j=0}^{\infty} \frac{a_j^2}{b_j} \|\mathbf{g}\|^{2j} < \infty,$$

a homogeneous version of the analysis of Zhang et al. (2016, 2017). The rate at which C_σ^2 grows measures how expensive the activation is to represent: $\sigma(u) = u$ (linear) gives $C_\sigma^2(\lambda^2) = O(\lambda^2)$; a degree- p polynomial gives $O(\lambda^{2p})$; and smoother saturating activations such as the sine, the sigmoid, or a smooth ReLU cost $O(e^{c\lambda^2})$, an exponential price for their smoothness.

This is the last ingredient needed for the linearization principle. A convolutional network with filters $W_k^{ij}(u)$ (layer k , output filter i , input channel j) and smooth homogeneous activations can be built hierarchically inside $\mathcal{H}_K$, so that

$$f(\mathbf{x}) = \sigma_k(A_k \sigma_{k-1}(A_{k-1} \cdots \sigma_1(A_1 \mathbf{x}) \cdots)) = \langle f, \Phi(\mathbf{x}) \rangle_{\mathcal{H}}.$$

The RKHS norm of the network is bounded by nesting the activation-cost function through the layers,

$$\|f_\sigma\|^2 \leq \|W_{n+1}\|_2^2 C_\sigma^2(\|W_n\|_2^2 C_\sigma^2(\|W_{n-1}\|_2^2 C_\sigma^2(\cdots))),$$

and in the purely linear case ($\sigma = \text{id}$) this collapses to the product of spectral norms of the layers,

$$\|f_\sigma\|^2 \leq \|W_{n+1}\|_2^2 \cdot \|W_n\|_2^2 \cdots \|W_1\|_2^2.$$

40.11.1 Link with generalization

Because the trained CNN is now a function in an RKHS ball, the classical generalization theory of Chapter 11, Learning Theory in RKHS Balls applies with no modification. For the ball $\mathcal{F}_B = \{f \in \mathcal{H}_K : \|f\| \leq B\}$, the Rademacher complexity over N samples of radius R is

$$\text{Rad}_N(\mathcal{F}_B) \leq O\left(\frac{BR}{\sqrt{N}}\right),$$

which yields a margin bound $O(\|\hat{f}_N\|R/(\gamma\sqrt{N}))$ for a learned CNN $\hat{f}_N$ with confidence margin $\gamma > 0$. Since $\|\hat{f}_N\|$ is controlled by the product of spectral norms, this recovers, from the kernel viewpoint, the spectral-norm generalization bounds for neural networks of Bartlett et al. (2017) and Neyshabur et al. (2018), and connects to the classical treatments of Boucheron et al. (2005) and Shalev-Shwartz and Ben-David (2014).

The conclusion of the convolutional story is a single unifying observation: the same quantity $\|f\|$ controls both stability and generalization. Deformation stability comes from small patches adapted to the resolution, signal preservation from subsampling below the patch size, and group invariance from the choice of patch extraction and pooling. The tension is real, since higher capacity, a larger $\|f\|$, is exactly what is needed to discriminate small deformations, which is why regularization is delicate. Open questions remain: what is the right regularizer, how does SGD implicitly control capacity in CNNs, and what happens in networks without pooling such as ResNets.

40.12 Application to graphs: graph convolutional kernel networks

Images are not the only data with local structure. Molecules, protein interaction maps, social networks, and chemical pathways are graphs, and the CKN recipe (extract local patterns, embed with a kernel, pool) transfers to them. The state of the art has three camps: deep learning for graphs, that is graph neural networks (GNNs); graph kernels, notably the Weisfeiler-Lehman (WL) subtree kernel; and hybrids such as the graph neural tangent kernel (GNTK). Graph convolutional kernel networks (GCKN) unify them: a multilayer graph kernel that is more expressive than WL, that yields easy-to-regularize and scalable unsupervised representations, and that can also be trained supervised like a GNN.

The reason the transfer works at all is that the CKN recipe never referred to the grid. Patch extraction only needed a notion of local neighborhood, kernel embedding only needed a similarity between local patterns, and pooling only needed a way to aggregate over positions. Images supply these through boxes of pixels, but a graph supplies them just as well through the paths leaving a node, and once the three moves are named abstractly the same map into an RKHS is available on either domain. So the throughline survives the change of data: a graph convolutional kernel network is again a feature map Φ sending a graph into an RKHS, and a prediction is again a linear functional of it. What changes is only the local pattern the first move extracts.

A graph is a triple $G = (\mathcal{V}, \mathcal{E}, a)$ with vertices $\mathcal{V}$, edges $\mathcal{E}$, and an attribute function $a : \mathcal{V} \rightarrow \mathbb{R}^d$ assigning a feature vector to each node. A large class of graph kernels represents a graph by summing a base embedding of a local pattern $\ell_G(u)$ over its nodes,

$$\Phi(G) = \sum_{u \in \mathcal{V}} \varphi_{\text{base}}(\ell_G(u)), \quad K(G, G') = \sum_{u \in \mathcal{V}} \sum_{u' \in \mathcal{V}'} \langle \varphi_{\text{base}}(\ell_G(u)), \varphi_{\text{base}}(\ell_{G'}(u')) \rangle = \sum_u \sum_{u'} \kappa_{\text{base}}(\ell_G(u), \ell_{G'}(u')),$$

comparing every local pattern of G with every one of G' (Shervashidze et al., 2011; Lei et al., 2017; Kriege et al., 2020).

40.12.1 From the path kernel to its relaxation

The local pattern we build on is the *path*. Paths are more expressive than walks, because a walk may repeat vertices while a path may not, though paths are costlier to enumerate. Let $\mathcal{P}_k(G, u)$ be the set of paths of length k starting at node u . The k -path embedding places a Dirac spike at the concatenated attributes $a(p)$ of each path,

$$\varphi_{\text{path}}(u) = \sum_{p \in \mathcal{P}_k(G, u)} \delta_{a(p)}(\cdot) \implies \Phi(G) = \sum_{u \in \mathcal{V}} \sum_{p \in \mathcal{P}_k(G, u)} \delta_{a(p)},$$

so $\Phi(G)$ is a histogram of path occurrences. This has two defects. The Dirac δ only makes hard, all-or-nothing comparisons, so it handles discrete attributes but not continuous ones; and it is not differentiable, so it cannot be optimized by backpropagation. Both defects vanish if we soften the spike into a Gaussian bump,

$$\varphi_{\text{path}}(u) = \sum_{p \in \mathcal{P}_k(G, u)} \delta_{a(p)}(\cdot) \longrightarrow \sum_{p \in \mathcal{P}_k(G, u)} e^{-\frac{\alpha}{2} \|a(p) - \cdot\|^2},$$

a sum of Gaussians centered at each path's attributes: soft, differentiable, and defined for continuous attributes.

40.12.2 One layer of GCKN

The one-layer GCKN is exactly this relaxed path kernel mapping,

$$\varphi_1(u) := \sum_{p \in \mathcal{P}_k(G, u)} e^{-\frac{\alpha_1}{2} \|a(p) - \cdot\|^2} = \sum_{p \in \mathcal{P}_k(G, u)} \varphi_{\text{RBF}}(a(p)) \in \mathcal{H}_1,$$

and it decomposes into the same three moves as a CKN layer. *Path extraction*: enumerate all paths $\mathcal{P}_k(G, u)$ from u , the graph analogue of patch extraction. *Kernel mapping*: embed each path's features through the Gaussian feature map φ_{RBF} . *Path aggregation*: sum the path embeddings, the graph analogue of pooling. The result is a new graph on the same topology but with richer node features,

$$(\mathcal{V}, \mathcal{E}, a) \xrightarrow{\varphi_{\text{path}}} (\mathcal{V}, \mathcal{E}, \varphi_1).$$

40.12.3 Stacking layers

Because the output is again an attributed graph, we can apply φ_{path} again,

$$(\mathcal{V}, \mathcal{E}, a) \xrightarrow{\varphi_{\text{path}}} (\mathcal{V}, \mathcal{E}, \varphi_1) \xrightarrow{\varphi_{\text{path}}} (\mathcal{V}, \mathcal{E}, \varphi_2) \xrightarrow{\varphi_{\text{path}}} \dots \xrightarrow{\varphi_{\text{path}}} (\mathcal{V}, \mathcal{E}, \varphi_j),$$

and read out the graph representation at layer j as $\Phi(G) = \sum_{u \in \mathcal{V}} \varphi_j(u)$. Depth buys structure: applying φ_{path} once captures paths (GCKN-path), applying it twice captures subtrees (GCKN-subtree), and applying it more may capture higher-order substructures. This is how the model reaches long-range structure without ever enumerating long paths, which would be computationally prohibitive; a stack of short-path layers captures long-range substructures that a single layer could not afford.

40.12.4 Scalable approximation

The Gaussian embedding $\varphi_{\text{RBF}}(a(p)) = e^{-\frac{\alpha}{2}\|a(p)-\cdot\|^2}$ is infinite-dimensional, so we apply Nyström exactly as for images. Project $\varphi_{\text{RBF}}(a(p))$ onto $\text{span}(\varphi_{\text{RBF}}(z_1), \dots, \varphi_{\text{RBF}}(z_q))$, a subspace parametrized by anchors $Z = \{z_1, \dots, z_q\}$, giving a finite embedding $\Psi(a(p)) \in \mathbb{R}^q$. Here the anchors $z_j \in \mathbb{R}^{d_k}$ have a concrete meaning: they are learned *path features*, prototypical local patterns. As before, Z can be fit unsupervised by K-means on the observed path features, or supervised by end-to-end backpropagation (Chen et al., 2019a,b; Williams and Seeger, 2001).

40.12.5 GCKN versus GNN

The comparison to a graph neural network is now transparent, since both sum a local aggregation over nodes,

$$f_{\text{GCKN}}(G) = \sum_{u \in G} \psi_k(u), \quad f_{\text{GNN}}(G) = \sum_{u \in G} f_k(u),$$

but they aggregate differently:

$$\psi_k(u) = \sum_{p \in \mathcal{P}_k(G, u)} \kappa(Z^\top Z)^{-1/2} \kappa(Z^\top \psi_{k-1}(p)), \quad f_k(u) = \sum_{v \in \mathcal{N}(u)} \text{ReLU}(Z^\top f_{k-1}(v)).$$

GCKN	GNN	
aggregation	local path aggregation	neighborhood aggregation
nonlinearity	projection in a known RKHS	ReLU, no RKHS interpretation
training	supervised and unsupervised	supervised only

The GCKN aggregates over paths rather than immediate neighbors, and its nonlinearity $\kappa(Z^\top Z)^{-1/2} \kappa(Z^\top \cdot)$ is a projection onto a known subspace of an RKHS, which is what gives it an unsupervised training mode and a built-in regularization theory that the GNN's bare ReLU lacks.

40.12.6 Experiments and interpretation

On benchmark graphs with discrete attributes (MUTAG, PROTEINS, PTC, NCI1, IMDB-B, IMDB-M, COLLAB), measured as accuracy improvement over the WL subtree kernel and against GNTK, GCN, and GIN, GCKN-path already outperforms the baselines, adding layers brings larger improvements, and supervised training does not raise accuracy over unsupervised but yields more compact representations. On graphs with continuous attributes (ENZYMES, PROTEINS, BZR, COX2), compared to the WWL kernel and GNTK, the picture is the same, suggesting path features are already predictive enough (Shervashidze et al., 2011; Du et al., 2019; Xu et al., 2019; Kipf and Welling, 2017; Togninalli et al., 2019).

Because a GCKN is a kernel model, it is also interpretable. For mutagenicity prediction one can ask for the minimal connected subgraph that preserves the model's prediction (in the spirit of Ying et al., 2019). On a mutagenic molecule this minimal component isolates the chemical substructure responsible for the classification, so the model does not merely predict mutagenicity but points at the fragment driving the prediction, closing the loop between accuracy and explanation that motivated the kernel viewpoint.

40.13 Application to biological sequences

The third domain is biological sequences, where the same recipe produces convolutional and recurrent kernel networks. The task is standard supervised learning: given sequences $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathcal{X}$ with labels $y_1, \dots, y_n$, learn a predictive and *interpretable* function by regularized risk minimization,

$$\min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(\mathbf{x}_i)) + \mu \Omega(f),$$

and the modeling question is how to choose $\mathcal{F}$. The classical answer for a sequence over an alphabet $\mathcal{A}$ is a string kernel comparing counts of k -mers,

$$K(\mathbf{x}, \mathbf{x}') = \sum_{u \in \mathcal{A}^k} \delta_u(\mathbf{x}) \delta_u(\mathbf{x}') = \langle \Phi(\mathbf{x}), \Phi(\mathbf{x}') \rangle,$$

where $\delta_u(\mathbf{x})$ counts occurrences of the k -mer u : exact occurrences (spectrum kernel, Leslie et al., 2002), occurrences up to m mismatches (mismatch kernel, Leslie and Kuang, 2004), or occurrences with gaps weighted by an exponential decay (substring kernel, Lodhi et al., 2002). The feature map $\Phi(\mathbf{x})$ is a histogram of pattern occurrences, and its discreteness is the same obstacle we met with paths.

40.13.1 A convolutional kernel network for sequences

We relax the mismatch kernel to a continuous, differentiable one (Chen et al., 2019a; Morrow et al., 2017) by comparing all pairs of k -mers through a smooth kernel K_0 rather than exact matching,

$$K_{\text{CKN}}(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^{|\mathbf{x}|-k+1} \sum_{j=1}^{|\mathbf{x}'|-k+1} K_0(\mathbf{x}[i:i+k], \mathbf{x}'[j:j+k]) = \left\langle \sum_i \varphi_0(\mathbf{x}[i:i+k]), \sum_j \varphi_0(\mathbf{x}'[j:j+k]) \right\rangle.$$

Each k -mer is one-hot encoded as a matrix in $\mathbb{R}^{k \times d}$, for instance the DNA window TTGAG over the alphabet $\{A, T, C, G\}$ becomes a 4×5 binary matrix with a single 1 per column, and K_0 is a Gaussian kernel over these encodings. The mapping $\Phi(\mathbf{x}) = \sum_i \varphi_0(\mathbf{x}[i:i+k])$ is a differentiable relaxation of the mismatch kernel.

To make it finite we again use Nyström, and here it is worth stating the general form. Projecting $\varphi_0(u)$ onto $E_0 = \text{span}(\varphi_0(z_1), \dots, \varphi_0(z_q))$ gives $\psi_0(u) \in \mathbb{R}^q$ with

$$\psi_0(u) = K_0(Z, Z)^{-1/2} K_0(Z, u),$$

and when $K_0(u, u') = \kappa(\langle u, u' \rangle)$ is a dot-product kernel this specializes to the by-now familiar layer

$$\psi_0(u) = \kappa(Z^\top Z)^{-1/2} \kappa(Z^\top u),$$

a linear operation, a pointwise nonlinearity κ , and a linear operation, with $\kappa(\beta) = e^{\beta-1}$, polynomial, inverse polynomial, or arc-cosine. A single-layer CKN for sequences extracts k -mers, applies ψ_0 , pools globally into $\Psi(\mathbf{x}) \in \mathbb{R}^q$, and predicts with $\langle \mathbf{w}, \Psi(\mathbf{x}) \rangle$; stacking an embedding layer, a convolutional kernel layer with a dot-product kernel, and pooling gives the multilayer version.

40.13.2 Gapped k-mers and recurrent kernel networks

Biological motifs tolerate insertions, so we want to compare k -mers *with gaps*. For a sequence $\mathbf{x} = x_1 \dots x_n$ and an ordered index tuple $\iota = (i_1, \dots, i_k)$, define the k -substring $\mathbf{x}[\iota] = x_{i_1} x_{i_2} \dots x_{i_k}$ and let $\text{gaps}(\iota)$ count the gaps it skips. For example, in $\mathbf{x} = \text{ABRACADABRA}$ the indices $\iota = (3, 4, 7, 8, 10)$ read off $\mathbf{x}[\iota] = \text{RADAR}$ with $\text{gaps}(\iota) = 3$. The recurrent kernel network compares all gapped k -mers, penalizing gaps geometrically with a factor $\lambda \in (0, 1)$,

$$K_{\text{RKN}}(\mathbf{x}, \mathbf{x}') = \sum_{\iota \in \mathcal{I}(k, |\mathbf{x}|)} \sum_{j \in \mathcal{I}(k, |\mathbf{x}'|)} \lambda^{\text{gaps}(\iota)} \lambda^{\text{gaps}(j)} K_0(\mathbf{x}[\iota], \mathbf{x}'[j]), \quad \Phi(\mathbf{x}) = \sum_{\iota \in \mathcal{I}(k, |\mathbf{x}|)} \lambda^{\text{gaps}(\iota)} \varphi_0(\mathbf{x}[\iota]),$$

a differentiable relaxation of the substring kernel. Enumerating all index tuples is exponentially costly, but the Nyström feature map

$$\Psi(\mathbf{x}) = \sum_{\iota \in \mathcal{I}(k, t)} \lambda^{\text{gaps}(\iota)} \psi_0(\mathbf{x}[\iota]) \in \mathbb{R}^q, \quad \psi_0(\mathbf{x}[\iota]) = \kappa(Z^\top Z)^{-1/2} \kappa(Z^\top \mathbf{x}[\iota]),$$

can be computed by dynamic programming (Lodhi et al., 2002), and the resulting recursion is precisely a particular recurrent neural network (Lei et al., 2017). When K_0 is Gaussian, the RKN feature map is a mixture of Gaussians centered at the substrings $\mathbf{x}[\iota]$ and weighted by $\lambda^{\text{gaps}(\iota)}$: the network is a soft, gap-tolerant substring counter that trains by gradient descent.

On protein fold classification (SCOP 2.06; Hou et al., 2017), using informative sequence features such as PSSM profiles, secondary structure, and solvent accessibility, these models are competitive with or better than deep baselines at a fraction of the parameters. A CKN with 512 filters (843k parameters) reaches about 84% top-1 and 94% top-5 accuracy, and the recurrent RKN with 512 filters matches its parameter budget while improving to about 85% top-1 and 95% top-5, with a marked gain on the hardest fold-level stratum, where gap tolerance matters most (Chen et al., 2019a).

Further reading

Almost everything in this chapter postdates the two standard kernel-methods textbooks, so it is not covered by either of them and the primary literature is the place to go. For the neural tangent kernel and the sense in which wide networks train like kernel machines, the original reference is Jacot, Gabriel, and Hongler (2018), with the lazy-training mechanism isolated by Chizat, Oyallon, and Bach (2019). For convolutional kernel networks and the idea of designing an RKHS in which a whole convolutional network lives as a single linear functional, learned end to end through the anchor points, see Mairal (2016). For the analysis of what that representation guarantees, the invariance and stability of deep convolutional representations to translations and to smooth deformations, the reference is Bietti and Mairal (2019). The random-feature side traces back to Neal (1996) for the Gaussian-process view and to Cho and Saul (2009) for the arc-cosine kernels, while the graph and sequence constructions are developed by Chen, Jacob, and Mairal (2020) and the works cited there.

40.14 Why a fixed kernel makes a poor hidden neuron

Everything above puts kernels around deep networks: as their infinite-width limit, as their training dynamics, as the RKHS that contains them. It is worth asking the blunter question the history of the field already answered the hard way. Why not use a kernel as the neuron itself, a hidden unit $x \mapsto k(x, c)$ with a trainable center c , stack those into layers, and train the whole thing by gradient descent the way we train a network of rectifiers? Radial basis function networks (Broomhead and Lowe, 1988) did exactly this with the Gaussian unit $\phi_c(x) = \exp(-\|x - c\|^2/2\sigma^2)$, and for a shallow width-one-layer model with a linear readout they work well. As a deep, end-to-end, gradient-trained building block the Gaussian neuron failed, and it failed for reasons that say something precise about what a hidden unit has to do.

40.14.1 The activation, and its gradient, vanish almost everywhere

A rectifier $\max(0, w^\top x + b)$ has a gradient that is either zero or one; on its active half-space the signal passes through undiminished no matter how far the input sits from the boundary. A Gaussian unit is the opposite. Away from its center it is not merely small, it is exponentially flat: both the activation and its derivatives decay like $\exp(-\|x - c\|^2/2\sigma^2)$. The gradient that would move the center toward the data,

$$\frac{\partial \phi_c(x)}{\partial c} = \frac{x - c}{\sigma^2} \phi_c(x),$$

carries the same vanishing factor $\phi_c(x)$. So a unit whose center starts far from a training point receives an almost-zero learning signal about that point, and in high dimensions *every* randomly placed center starts far from *every* point, because volume concentrates away from any fixed location. The network begins in a dead state and gradient descent has almost no gradient with which to leave it. A rectifier saturates on one side only; a Gaussian saturates on all sides at once.

The tell in the training recipe

Classical RBF networks were almost never trained end to end. The standard recipe placed the centers first by an unsupervised method, k-means over the inputs, and only then fitted the linear readout, which is a convex problem the representer theorem hands you for free. That two-stage split is precisely an admission that gradient descent on the centers does not work: the placement had to be solved by other means because the vanishing signal would not solve it.

40.14.2 Locality does not compose the way piecewise-linearity does

The deeper obstacle is representational. A Gaussian unit is a *local* feature: it responds to a bounded neighborhood and is blind everywhere else. A network of local features is a memory-based method in disguise, and to cover a region of a d -dimensional input space with bumps of width σ takes on the order of $(L/\sigma)^d$ of them. Bengio, Delalleau, and Le Roux (2006) made this exact, showing that local kernel machines need a number of examples, and of units, that grows with the number of variations of the target function, so a function that turns over many times cannot be represented compactly by any shallow bank of kernels. Depth is supposed to be the escape, but stacking Gaussian layers composes local bumps of local bumps and the result stays local, while the vanishing-gradient problem compounds at every layer. Compose piecewise-linear maps instead and the number of linear regions grows exponentially in depth (Montufar, Pascanu, Cho, and Bengio, 2014): the rectifier buys expressivity from depth precisely where the Gaussian does not.

40.14.3 Bandwidth is a knife-edge, and scale is unmanaged

There is also a conditioning problem that gradient descent handles badly. The bandwidth σ sits on a knife-edge: too small and every unit sees only its own center, the Gram matrix approaches the identity, and there is no signal to share between points; too large and all units respond almost identically, the Gram matrix collapses toward rank one, and there is nothing to discriminate. The usable band is narrow and data-dependent, and because each layer rescales distances, a single fixed σ cannot stay in that band through a deep stack. Rectifier networks have a matching answer for scale in their initialization, the variance-preserving schemes of Glorot and Bengio (2010) and their successors, which keep activations and gradients at unit scale across many layers. There is no comparable initialization that keeps a stack of Gaussian units alive at depth, because at any reasonable initial scale the high-dimensional centers are already out of range of the data.

40.14.4 A kernel is a readout, not a representation

These failures share one root, and it is the same duality this book has celebrated from the start. A kernel is a *fixed* similarity, an inner product in a feature space you do not get to reshape. That is exactly what you want in the *last* layer, the readout, where the representer theorem applies, the problem stays convex, and the fixed geometry is the source of the guarantees in Parts II through IV. A hidden layer has the opposite job: it must *learn a representation*, which means transporting its nonlinearity's sensitive region to wherever the data actually is. A rectifier's sensitive region is a hyperplane whose gradient is constant on the active side, so gradient descent moves it freely; a Gaussian's sensitive region is a small fixed ball whose gradient vanishes the moment the ball is in the wrong place, so gradient descent cannot move it there. The neural tangent kernel view of this chapter says the same thing from the other end: in the wide, lazy regime the features do not move at all, and for a Gaussian layer that immobility is not a limit reached only at infinite width but the operating condition at every width, imposed by the vanishing signal.

The modern reading

None of this says kernels and deep learning are incompatible; the whole chapter argues the reverse. It says that the naive substitution, a fixed local kernel dropped in where a rectifier goes, asks the kernel to do the one thing a fixed kernel cannot, learn its own geometry. The convolutional kernel networks of this chapter succeed as deep kernel machines precisely because they do not make that substitution: they keep the kernel as a fixed, well-conditioned local embedding and learn a small set of anchor points through the Nyström projection, so the representation is shaped by end-to-end training while the kernel supplies the geometry. The lesson is not that the kernel neuron was a bad idea but that a hidden unit must be able to move its own attention, and a fixed bump cannot. The theoretical statement of this gap, the mean-field limit in which hidden units do move and the larger-than-RKHS function space they explore, is the subject of the next chapter, *Kernels in the Modern World*.

40.15 Conclusion

The thread running through this chapter is a single idea seen from many sides: a deep architecture is a map into a reproducing kernel Hilbert space. Fixing the hidden layer sends the infinite-width network to the random-feature kernel; letting gradient descent run on a very wide network sends its trajectory to the neural tangent kernel; and building each layer as a Nyström-approximated kernel embedding places an entire convolutional, graph, or recurrent network inside an RKHS, where it becomes one linear functional of a data-independent feature map Φ . Once there, the RKHS norm becomes a single lever controlling smoothness, deformation stability, and generalization at once, the pieces we treated separately as the geometry factor and the complexity factor of the basic inequality $|f(\mathbf{x}) - f(\mathbf{x}')| \leq \|f\|_{\mathcal{H}} \|\Phi(\mathbf{x}) - \Phi(\mathbf{x}')\|_{\mathcal{H}}$. Kernels do not compete with deep learning; they supply the geometry within which deep learning can be understood, regularized, and, through the learned anchor points, trained end to end.

40.16 Common mistakes and practical implications

For **Deep Learning from a Kernel Point of View**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

40.17 Summary and further reading

This chapter established explain the central definitions and claims in Deep Learning from a Kernel Point of View; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Neal 1996), (Cho and Saul 2009), (Rahimi and Recht 2007).

40.18 Exercises

1. warm-up Schoenberg's theorem says that $K(\mathbf{x}, \mathbf{y}) = \kappa(\langle \mathbf{x}, \mathbf{y} \rangle)$ is positive definite on the sphere of every Hilbert space exactly when κ has a power series with non-negative coefficients. Decide which of the following univariate functions passes the test on $[-1, 1]$, and for each one that fails, name the offending coefficient: (a) the exponential $\kappa(u) = e^{\alpha(u-1)}$ with $\alpha > 0$; (b) the inverse polynomial $\kappa(u) = 1/(2-u)$; (c) the polynomial $\kappa(u) = (c+u)^p$ with $c \geq 0$ and integer $p \geq 1$; (d) $\kappa(u) = 1-u$; (e) $\kappa(u) = \cos u$. For (b), expand the geometric series $1/(2-u) = \frac{1}{2} \sum_{j \geq 0} (u/2)^j$ and read off the sign of every coefficient.
2. computation Take two unit vectors $\mathbf{z}, \mathbf{z}'$ on the sphere separated by angle $\theta = \pi/3$, so $\langle \mathbf{z}, \mathbf{z}' \rangle = \frac{1}{2}$. (a) Using the table of dot-product kernels, evaluate the degree-1 arc-cosine kernel $\frac{1}{\pi}(\sin \theta + (\pi - \theta) \cos \theta)$ and Vovk's degree-3 kernel $\frac{1}{3}(1+u+u^2)$ at this angle, to three decimals. (b) Confirm the identity $\|\mathbf{z} - \mathbf{z}'\|^2 = 2 - 2\langle \mathbf{z}, \mathbf{z}' \rangle$ here, and use it to check that the exponential kernel $e^{\alpha(\langle \mathbf{z}, \mathbf{z}' \rangle - 1)}$ equals the Gaussian $e^{-\frac{\alpha}{2}\|\mathbf{z} - \mathbf{z}'\|^2}$ at $\alpha = 2$. (c) Evaluate the degree-0 angular part $J_0(\theta) = \pi - \theta$ and confirm it decreases as the angle opens, consistent with its reading as a count of separating hyperplanes.
3. proof Derive the degree-0 arc-cosine kernel from a random layer of step neurons. Let σ be the Heaviside step, $\sigma(t) = 1$ if $t > 0$ and 0 otherwise, and draw $\mathbf{w} \sim \mathcal{N}(0, I)$. Show that

$$K_{\text{RF}}(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{\mathbf{w}}[\sigma(\mathbf{w}^\top \mathbf{x}) \sigma(\mathbf{w}^\top \mathbf{y})] = \mathbb{P}(\mathbf{w}^\top \mathbf{x} > 0 \text{ and } \mathbf{w}^\top \mathbf{y} > 0) = \frac{\pi - \theta}{2\pi},$$

where $\theta = \arccos(\mathbf{x}^\top \mathbf{y} / (\|\mathbf{x}\| \|\mathbf{y}\|))$. Conclude that this expectation depends on the inputs through the angle alone, and relate it to the Cho and Saul factor $J_0(\theta) = \pi - \theta$. Why does the answer not change if you replace $\mathbf{x}$ by $\lambda \mathbf{x}$ for any $\lambda > 0$? Hint

The pair $(\mathbf{w}^\top \mathbf{x}, \mathbf{w}^\top \mathbf{y})$ is jointly Gaussian and centered, so only the plane spanned by $\mathbf{x}$ and $\mathbf{y}$ matters; project $\mathbf{w}$ onto it. The direction of $\mathbf{w}$ in that plane is uniform on the circle by rotational invariance of $\mathcal{N}(0, I)$, so the probability that a uniformly random half-plane through the origin puts both $\mathbf{x}$ and $\mathbf{y}$ on its positive side is the fraction of directions lying within $\pi/2$ of both, which is $(\pi - \theta)/(2\pi)$. Positive scaling of $\mathbf{x}$ leaves the sign of $\mathbf{w}^\top \mathbf{x}$ untouched.

4. proof Make precise the claim that a wide random layer is a Monte Carlo estimate of K_{RF} . Fix inputs $\mathbf{x}, \mathbf{y}$, draw $\mathbf{w}_1, \dots, \mathbf{w}_m$ independently from $\mathcal{N}(0, I)$, and set the empirical kernel

$$\hat{K}_m(\mathbf{x}, \mathbf{y}) = \frac{1}{m} \sum_{i=1}^m \sigma(\mathbf{w}_i^\top \mathbf{x}) \sigma(\mathbf{w}_i^\top \mathbf{y}).$$

Assume the feature product $\xi_i = \sigma(\mathbf{w}_i^\top \mathbf{x}) \sigma(\mathbf{w}_i^\top \mathbf{y})$ has finite variance $v(\mathbf{x}, \mathbf{y}) < \infty$. Show that $\mathbb{E}[\hat{K}_m(\mathbf{x}, \mathbf{y})] = K_{\text{RF}}(\mathbf{x}, \mathbf{y})$ for every m (the estimate is unbiased), that $\text{Var}[\hat{K}_m(\mathbf{x}, \mathbf{y})] = v(\mathbf{x}, \mathbf{y})/m$, and hence that $\hat{K}_m(\mathbf{x}, \mathbf{y}) \rightarrow K_{\text{RF}}(\mathbf{x}, \mathbf{y})$ with a typical fluctuation of order $1/\sqrt{m}$. Explain in one sentence why this is exactly the statement that widening a frozen hidden layer refines a kernel estimate rather than changing the kernel it targets. Hint

The ξ_i are independent and identically distributed with common mean $K_{\text{RF}}(\mathbf{x}, \mathbf{y})$, so linearity of expectation gives the mean and independence gives $\text{Var}[\frac{1}{m} \sum_i \xi_i] = \frac{1}{m^2} \sum_i \text{Var}[\xi_i] = v/m$. The $1/\sqrt{m}$ rate is the standard deviation, the square root of the variance; the law of large numbers then forces convergence to the population mean, which is the kernel.

5. proof Derive the two-term neural tangent kernel for the two-layer network $f_\theta(\mathbf{x}) = \frac{1}{\sqrt{m}} \sum_{i=1}^m v_i \sigma(\mathbf{w}_i^\top \mathbf{x})$ with parameters $\theta = (v_i, \mathbf{w}_i)_i$. (a) Compute the two blocks of the gradient, $\partial f_\theta / \partial v_i$ and $\nabla_{\mathbf{w}_i} f_\theta$, and check that each carries a factor $1/\sqrt{m}$. (b) Form the gradient inner product $\langle \nabla_{\theta} f_{\theta_0}(\mathbf{x}), \nabla_{\theta} f_{\theta_0}(\mathbf{y}) \rangle$, summing both blocks over i . (c) With $\mathbf{w}_i, v_i$ drawn from $\mathcal{N}(0, I)$ and $\mathbb{E}[v_i^2] = 1$, take $m \rightarrow \infty$ and apply the law of large numbers to obtain

$$K_{\text{NTK}}(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{\mathbf{w}}[\sigma(\mathbf{w}^\top \mathbf{x}) \sigma(\mathbf{w}^\top \mathbf{y}) + (\mathbf{x}^\top \mathbf{y}) \sigma'(\mathbf{w}^\top \mathbf{x}) \sigma'(\mathbf{w}^\top \mathbf{y})].$$

Identify which term comes from the output weights and which from the hidden weights, and explain why freezing the hidden layer deletes the second term and leaves exactly K_{RF} . Hint

Differentiating gives $\partial f_\theta / \partial v_i = \frac{1}{\sqrt{m}} \sigma(\mathbf{w}_i^\top \mathbf{x})$ and $\nabla_{\mathbf{w}_i} f_\theta = \frac{1}{\sqrt{m}} v_i \sigma'(\mathbf{w}_i^\top \mathbf{x}) \mathbf{x}$. The v -block sum is $\frac{1}{m} \sum_i \sigma(\mathbf{w}_i^\top \mathbf{x}) \sigma(\mathbf{w}_i^\top \mathbf{y})$; the $\mathbf{w}$ -block sum is $\frac{1}{m} \sum_i v_i^2 \sigma'(\mathbf{w}_i^\top \mathbf{x}) \sigma'(\mathbf{w}_i^\top \mathbf{y}) (\mathbf{x}^\top \mathbf{y})$. Each empirical average converges to its expectation; use $\mathbb{E}[v_i^2] = 1$ and independence of v_i from $\mathbf{w}_i$.

6. exploration Open the interactive figure above in the section on random feature kernels. It plots a target dot-product kernel against the estimate produced by a finite bank of D random features, the same object a width- D hidden layer computes. Start at small D , where the estimate is visibly rough and rattles around the smooth target curve, then raise D and watch the estimate settle onto its infinite-width limit. Answer three questions from what you see: (a) roughly how does the size of the wiggle around the target shrink as you multiply D by four, and does that match the $1/\sqrt{m}$ rate you proved above? (b) the target curve itself never moves as D grows, only the estimate tightens onto it; which fact about frozen random features does that illustrate, as opposed to what would happen if the layer were being trained? (c) in the language of the chapter, in what precise sense is each additional random feature one more Monte Carlo sample of the population inner product $\mathbb{E}_{\mathbf{w}}[\sigma(\mathbf{w}^\top \mathbf{x}) \sigma(\mathbf{w}^\top \mathbf{y})]$?
7. challenge Explain, and then quantify, why the lazy regime performs no feature learning. Consider the linearization $f_\theta(\mathbf{x}) \approx f_{\theta_0}(\mathbf{x}) + \langle \theta - \theta_0, \nabla_{\theta} f_{\theta_0}(\mathbf{x}) \rangle$ around initialization. (a) Argue from the $1/\sqrt{m}$ prefactor of $f_\theta = \frac{1}{\sqrt{m}} \sum_i v_i \sigma(\mathbf{w}_i^\top \mathbf{x})$ that changing the output by an order-one amount requires each of the m hidden weights to move by only about $1/\sqrt{m}$, which vanishes as $m \rightarrow \infty$. (b) Conclude that the feature map $\nabla_{\theta} f_{\theta_0}$ is frozen throughout training, so the trained model is a linear predictor in a fixed feature space, that is kernel ridge regression with K_{NTK} . (c) Contrast this with genuine feature learning, in which the hidden representation reorganizes around the data, and state precisely what it is about the scaling, not any added assumption, that rules such reorganization out in the wide limit. Where in the two-term NTK of the previous exercise does the small but non-zero movement of the hidden weights leave its trace? Hint

Because the outputs of the m neurons are summed with weight $1/\sqrt{m}$, a collective order-one change in the sum needs only an order- $1/\sqrt{m}$ change per neuron, so the per-parameter displacement $\|\theta - \theta_0\|_{\text{per weight}}$ is $O(1/\sqrt{m})$. Small displacement is exactly the regime where the first-order Taylor expansion is accurate, so the nonlinear network tracks its own linearization; a fixed feature map means fixed features, hence no feature learning. The second term of K_{NTK} , carrying σ' and $\mathbf{x}^\top \mathbf{y}$, is the residue of that infinitesimal hidden-weight motion.

8. challenge Trace the Nyström bridge that turns an infinite-dimensional kernel into a finite neural-network layer, and follow it onto graphs. (a) For a dot-product kernel $K(\mathbf{x}, \mathbf{y}) = \kappa(\langle \mathbf{x}, \mathbf{y} \rangle)$ with feature map φ and anchors $Z = [\mathbf{z}_1, \dots, \mathbf{z}_p]$, the Nyström embedding is $\psi(\mathbf{x}) = \kappa(Z^\top Z)^{-1/2} \kappa(Z^\top \mathbf{x})$. Reading the three factors from right to left, identify which is a linear map, which is a pointwise nonlinearity, and which is a second linear map, and explain in what sense the factor $\kappa(Z^\top Z)^{-1/2}$ is what distinguishes projecting onto a subspace of the RKHS from applying an arbitrary nonlinearity as a plain CNN does. (b) Verify the normalization

role of $\kappa(Z^\top Z)^{-1/2}$ by showing that when the anchors are orthonormal in feature space, so $\kappa(Z^\top Z) = I$, the embedding reduces to $\psi(\mathbf{x}) = \kappa(Z^\top \mathbf{x})$. (c) A one-layer graph convolutional kernel network replaces patches by paths: it enumerates the paths $\mathcal{P}_k(G, u)$ from each node u , embeds each path's attributes through the same Nyström layer, and sums. Write the node update $\psi(u) = \sum_{p \in \mathcal{P}_k(G, u)} \kappa(Z^\top Z)^{-1/2} \kappa(Z^\top \psi_{\text{prev}}(p))$ and say which of the three CKN moves (extract local patterns, embed with a kernel, pool) each piece implements, and why the anchors Z are now interpretable as learned prototypical path features. Hint

In $\psi(\mathbf{x}) = \kappa(Z^\top Z)^{-1/2} \kappa(Z^\top \mathbf{x})$, the innermost $Z^\top \mathbf{x}$ is linear, the entrywise $\kappa(\cdot)$ is the nonlinearity, and left-multiplication by $\kappa(Z^\top Z)^{-1/2}$ is the second linear map; a CNN keeps the first two and drops the third, which is precisely the orthogonal projection onto $\mathcal{F} = \text{span}(\varphi(\mathbf{z}_1), \dots, \varphi(\mathbf{z}_p))$. For (c), path enumeration is patch extraction, the Nyström layer is the kernel embedding, and the sum over paths is pooling; the anchors sit in path-feature space, so each $\mathbf{z}_j$ is a template local pattern the layer scores against.

41 Gaussian Processes and the RVM

Every method so far has been built by writing down a loss, adding a regularizer, and minimizing. The Bayesian route reaches the same algorithms from the opposite side: it places a probability distribution over functions before any data arrive, then updates that belief with Bayes' rule once the data are in. The payoff is twofold. First, the answer is no longer a single curve but a whole posterior distribution, so every prediction comes with an honest error bar. Second, the regularizer that seemed like a free modelling choice turns into a prior, and the object doing the work is again a kernel, this time read as a covariance. The centerpiece of this chapter is a single identity: the Gaussian process posterior mean is exactly the kernel ridge regression fit of the ridge chapter, with the noise variance playing the role of the regularization strength. Around that identity we develop marginal-likelihood learning of hyperparameters, Gaussian process classification, and the sparse Bayesian prior behind the Relevance Vector Machine.

41.1 The Bayesian view: a prior over functions

The framework of risk minimization asks which function best fits the data under a complexity penalty. The Bayesian framework asks a different question: given a prior belief about which functions are plausible, and given a model of how observations are generated from a function, how plausible is each function after we see the data? The two ingredients are a likelihood and a prior, and the machinery that combines them is Bayes' rule. This account follows Schölkopf and Smola (2002) and MacKay (1992).

Suppose we observe inputs $X = (x_1, \dots, x_m)$ and targets $Y = (y_1, \dots, y_m)$, and we posit a hypothesis f together with a model $P(y | x, f(x))$ of how each label is produced from the underlying value $f(x)$. The prototypical case is additive noise, $y = f(x) + \xi$, so that $P(y | x, f(x)) = P(y - f(x))$. Under the usual independence assumption the likelihood factorizes,

$$P(Y | X, f) = \prod_{i=1}^m P(y_i | x_i, f(x_i)).$$

The likelihood tells us how probable the observed sample is if f is responsible for it. The prior $p(f)$ encodes what we believe before seeing data: a preference for smooth functions, for small values, or for a particular correlation structure among the values $f(x_i)$.

Bayes' rule turns the likelihood around. Since $p(Y | f, X)p(f) = p(f | X, Y) p(Y)$, and $p(Y)$ does not depend on f , the posterior over hypotheses is

$$p(f | X, Y) \propto p(Y | f, X) p(f).$$

To predict at a new location x we integrate the label model against the posterior,

$$p(y | X, Y, x) = \int p(y | f, x) p(f | X, Y) df,$$

which yields not a point estimate but a predictive distribution. When this integral is intractable, a common shortcut is the maximum a posteriori (MAP) estimate, the mode of the posterior,

$$f_{\text{MAP}} = \arg \min_f [-\ln p(Y | f, X) - \ln p(f)].$$

Read the two terms: the negative log likelihood is a data-fit term and the negative log prior is a penalty independent of the data. This is precisely the regularized risk functional $R_{\text{emp}}[f] + \Omega[f]$, with the loss identified as the negative log likelihood and the regularizer as the negative log prior. Squared loss corresponds to Gaussian noise, and a quadratic RKHS penalty corresponds to a Gaussian prior. The correspondence is exact at the level of the mode; the Bayesian side additionally offers the mean and the variance, which risk minimization has no counterpart for.

41.2 The Gaussian process

To put a prior on a function we cannot write down a density on an infinite-dimensional space directly. The trick, going back to Neal (1996) and Williams and Rasmussen (1996), is to specify the prior only through the joint distribution of the function values at any finite set of points, and to make that joint distribution Gaussian. A collection of function values that is jointly Gaussian for every finite subset of locations is a Gaussian process.

Definition (Gaussian process)

A stochastic process $t(x)$ indexed by $x \in \mathcal{X}$ is a *Gaussian process* if, for every finite set $x_1, \dots, x_m \in \mathcal{X}$, the vector $(t(x_1), \dots, t(x_m))$ is normally distributed. It is determined by a mean function $\mu(x) = \mathbb{E}[t(x)]$ and a covariance function

$$k(x, x') = \text{cov}(t(x), t(x')), \quad K_{ij} = k(x_i, x_j).$$

We write $(t(x_1), \dots, t(x_m)) \sim \mathcal{N}(\mu, K)$, and take $\mu \equiv 0$ unless stated otherwise.

The covariance function is not an arbitrary symmetric function. For any coefficients $c \in \mathbb{R}^m$ the variance of the linear combination $\sum_i c_i t(x_i)$ is nonnegative,

$$0 \leq \text{Var}\left(\sum_i c_i t(x_i)\right) = \sum_{i,j} c_i c_j \text{cov}(t(x_i), t(x_j)) = c^\top K c,$$

so K is positive semidefinite for every choice of points. In other words the covariance function of a Gaussian process is exactly a positive definite kernel in the sense of the kernel chapter, and every Mercer kernel is a legitimate covariance. This is the bridge: choosing a covariance is choosing a kernel, and the smoothness properties the kernel enforces in an RKHS reappear as the correlation structure of the prior. A zero-mean prior on the values $t = (t(x_1), \dots, t(x_m))$ reads

$$p(t) = (2\pi)^{-m/2} (\det K)^{-1/2} \exp\left(-\frac{1}{2} t^\top K^{-1} t\right),$$

and $-\ln p(t) = \frac{1}{2} t^\top K^{-1} t + \text{const}$ is the same quadratic form that the RKHS regularizer $\|f\|_{\mathcal{H}_K}^2$ produces (Schölkopf and Smola 2002). Kernels favoring slowly varying functions, such as the Gaussian covariance $k(x, x') = \exp(-\|x - x'\|^2 / 2\ell^2)$, give priors that assign high probability to smooth realizations, as one sees by expanding t in the eigenvectors of K : directions with large eigenvalue λ_i are cheap because $t^\top K^{-1} t$ weights them by $1/\lambda_i$, so the prior prefers the smooth, large-eigenvalue modes. The link back to the Mercer eigen-analysis is direct: the spectrum of K ranks hypotheses by prior plausibility.

41.3 Gaussian process regression

In regression we do not observe the latent function directly. We observe it through additive Gaussian noise, $y_i = t(x_i) + \xi_i$ with $\xi_i \sim \mathcal{N}(0, \sigma^2)$ independent. Because a sum of independent Gaussians is Gaussian, the observed vector y and any future latent value are jointly Gaussian, and everything we need follows from one fact about Gaussians: conditioning a joint Gaussian on part of its coordinates produces another Gaussian, with closed-form mean and covariance.

Write $k_* = (k(x_1, x_*), \dots, k(x_m, x_*))^\top$ for the vector of covariances between the training points and a test point x_* , and $k_{**} = k(x_*, x_*)$. Since $y_i = t(x_i) + \xi_i$, the training observations have covariance $K + \sigma^2 I$, while the latent test value $f_* = t(x_*)$ has covariance k_* with the training values and variance k_{**} with itself. The joint law is

$$\begin{pmatrix} y \\ f_* \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} K + \sigma^2 I & k_* \\ k_*^\top & k_{**} \end{pmatrix}\right).$$

The Gaussian conditioning formula, that $(a | b)$ has mean $\Sigma_{ab}\Sigma_{bb}^{-1}b$ and covariance $\Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$, gives the predictive distribution of f_* given the data at once.

Theorem (GP predictive equations)

Under a zero-mean Gaussian process prior with covariance k and additive Gaussian noise of variance σ^2 , the posterior over the latent value at a test point x_* is Gaussian, $f_* | X, Y, x_* \sim \mathcal{N}(\bar{m}(x_*), v(x_*))$, with

$$\bar{m}(x_*) = k_*^\top (K + \sigma^2 I)^{-1} y, \quad v(x_*) = k_{**} - k_*^\top (K + \sigma^2 I)^{-1} k_*.$$

The predictive distribution of a noisy observation y_* adds σ^2 to $v(x_*)$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Apply the conditioning identity with $a = f_*$ and $b = y$, reading the blocks off the joint covariance: $\Sigma_{bb} = K + \sigma^2 I$, $\Sigma_{ab} = k_*^\top$, $\Sigma_{aa} = k_{**}$. The conditional mean is $\Sigma_{ab}\Sigma_{bb}^{-1}b = k_*^\top (K + \sigma^2 I)^{-1}y$ and the conditional variance is $\Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba} = k_{**} - k_*^\top (K + \sigma^2 I)^{-1}k_*$. For a noisy observation $y_* = f_* + \xi_*$ with independent $\xi_* \sim \mathcal{N}(0, \sigma^2)$, the variance of f_* and ξ_* add, giving $v(x_*) + \sigma^2$. $\square$

Three features of these equations deserve emphasis. The mean is a linear combination of kernel functions centered at the training points, $\bar{m}(x_*) = \sum_i \alpha_i k(x_i, x_*)$ with $\alpha = (K + \sigma^2 I)^{-1}y$, so the posterior mean lives in the span of the training kernels exactly as the representer theorem predicts. The variance does not depend on the observed targets y at all, only on where the data were taken: it shrinks near training points and grows in the gaps, which is what a principled error bar should do. And the noise variance σ^2 is added to the diagonal of K , which is what keeps the inverse well conditioned. Rasmussen and Williams (2006) is the standard reference for this development.

Algorithm (GP regression)

Input Training data X, y ; covariance function k with hyperparameters θ ; noise variance σ^2 ; test point x_* .

Output Predictive mean $\bar{m}(x_*)$, variance $v(x_*)$, and (optionally) a hyperparameter gradient step.

1. Form the Gram matrix K with $K_{ij} = k(x_i, x_j)$ and set $A = K + \sigma^2 I$.
2. Compute the Cholesky factor $A = LL^\top$ (numerically stabler than inverting A).
3. Solve $A\alpha = y$ by two triangular solves against L .
4. Form $k_* = (k(x_1, x_*), \dots, k(x_m, x_*))^\top$; return $\bar{m}(x_*) = k_*^\top \alpha$.
5. Solve $Lw = k_*$; return $v(x_*) = k(x_*, x_*) - w^\top w$.
6. To learn hyperparameters, compute the log marginal likelihood $\mathcal{L}(\theta, \sigma^2)$ and its gradient $\partial \mathcal{L} / \partial \theta_j = \frac{1}{2} y^\top A^{-1} (\partial A / \partial \theta_j) A^{-1} y - \frac{1}{2} \text{tr}(A^{-1} \partial A / \partial \theta_j)$, take a gradient-ascent step, and repeat from step 1.

Example (GP posterior on three points)

Training inputs $x = (0, 1, 2)$, targets $y = (1, 0.5, -0.5)$, Gaussian kernel $k(x, x') = e^{-(x-x')^2/2}$ (length scale $\ell = 1$), noise variance $\sigma^2 = 0.1$. Predict at $x_* = 0.5$. All numbers from `checks/ch-gp-ex1.py`.

1. Form the Gram matrix. With $e^{-1/2} = 0.6065$ and $e^{-2} = 0.1353$,

$$K = \begin{pmatrix} 1 & 0.6065 & 0.1353 \\ 0.6065 & 1 & 0.6065 \\ 0.1353 & 0.6065 & 1 \end{pmatrix}, \quad A = K + 0.1 I.$$

2. Solve for the coefficients. $\alpha = A^{-1}y = (0.7321, 0.5046, -0.8228)$.
3. Evaluate the test covariances. $k_* = (e^{-1/8}, e^{-1/8}, e^{-9/8}) = (0.8825, 0.8825, 0.3247)$.
4. Read off the mean. $\bar{m}(x_*) = k_*^\top \alpha = 0.8242$.
5. Read off the variance. $v(x_*) = k_{**} - k_*^\top A^{-1} k_* = 1 - 0.9176 = 0.0824$, so the posterior standard deviation is 0.287.
6. Score the fit. The log marginal likelihood is $-\frac{1}{2}y^\top A^{-1}y - \frac{1}{2} \ln \det A - \frac{3}{2} \ln 2\pi = -3.2002$, using $y^\top A^{-1}y = 1.3958$ and $\ln \det A = -0.509$.

Reading. The prediction at $x_* = 0.5$ is 0.82 ± 0.29 . The error bar is tight here because x_* sits close to two training points; it would widen in a region with no data, since the variance ignores y and tracks only the geometry of the inputs.

Verification artifact. `checks/example-ch-gp-example-40-1.json` records the example source hash and verification scope.

41.4 Kernel ridge regression is the GP posterior mean

The coefficient vector $\alpha = (K + \sigma^2 I)^{-1}y$ should look familiar. In the ridge chapter the kernel ridge solution was $\alpha = (K + \lambda n I)^{-1}y$, where n is the number of training points and λ the regularization strength, and the fitted function was $\hat{f}(x_*) = \sum_i \alpha_i k(x_i, x_*) = k_*^\top \alpha$. The GP posterior mean is the identical expression with σ^2 in place of λn . This is not a coincidence; it is the MAP-equals-mean phenomenon made concrete.

Proposition (KRR / GP correspondence)

Kernel ridge regression with regularization parameter λ on n points produces the same predictor as the Gaussian process posterior mean with noise variance

$$\sigma^2 = \lambda n.$$

Concretely, $\hat{f}_{\text{KRR}}(x_*) = k_*^\top (K + \lambda n I)^{-1}y = k_*^\top (K + \sigma^2 I)^{-1}y = \bar{m}(x_*)$ for every x_* .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

With additive Gaussian noise the negative log posterior in coefficient space is, up to constants,

$$-\ln p(t \mid y) = \frac{1}{2\sigma^2} \|y - K\alpha\|^2 + \frac{1}{2} \alpha^\top K \alpha,$$

writing $t = K\alpha$. This is the regularized least-squares objective whose stationarity condition is $K[(K + \sigma^2 I)\alpha - y] = 0$, solved by $\alpha = (K + \sigma^2 I)^{-1}y$. The ridge objective $\frac{1}{n}\|K\alpha - y\|^2 + \lambda \alpha^\top K\alpha$ has stationarity condition $K[(K + \lambda n I)\alpha - y] = 0$, solved by $\alpha = (K + \lambda n I)^{-1}y$. The two coincide iff $\sigma^2 = \lambda n$, and since the predictor is $k_*^\top \alpha$ in both cases, the functions agree everywhere. Because the noise is Gaussian the posterior is Gaussian, so its mode and mean coincide, which is why the MAP fit (ridge) equals the posterior mean (GP). $\square$

Example (the two fits agree)

Same kernel $k(x, x') = e^{-(x-x')^2/2}$ on $x = (0, 1, 2)$, now with $y = (1, 0, -1)$. Ridge parameter $\lambda = 0.05$, $n = 3$, so $\lambda n = 0.15$; set the GP noise variance to the matched value $\sigma^2 = 0.15$. Numbers from `checks/ch-gp-ex2.py`.

1. Solve the ridge system. $\alpha_{\text{KRR}} = (K + 0.15 I)^{-1}y = (0.9855, 0, -0.9855)$. The middle coefficient vanishes because both y and the geometry are antisymmetric about $x = 1$.
2. Solve the GP system. $\alpha_{\text{GP}} = (K + 0.15 I)^{-1}y = (0.9855, 0, -0.9855)$; the maximum coefficient difference is exactly 0.
3. Predict at two test points. At $x_* = 0.5$: $\hat{f}_{\text{KRR}} = \bar{m} = 0.549782$. At $x_* = 1.5$: $\hat{f}_{\text{KRR}} = \bar{m} = -0.549782$. The differences are 0 to machine precision.

Reading. The two algorithms, derived from a loss-plus-penalty on one side and from Bayes' rule on the other, return the same numbers to the last digit. The Gaussian process adds only what ridge cannot supply: a variance at each test point.

Verification artifact. `checks/example-ch-gp-example-40-2.json` records the example source hash and verification scope.

41.5 The marginal likelihood and hyperparameter learning

A kernel comes with knobs: the length scale of a Gaussian covariance, the noise level σ^2 , an overall amplitude. In risk minimization these are set by cross-validation. The Bayesian framework offers an internal alternative. Collect the hyperparameters into θ and integrate the latent function out of the joint distribution; because the prior on t is Gaussian and the noise is Gaussian, the marginal distribution of the data is Gaussian too, $y \mid \theta \sim \mathcal{N}(0, K_\theta + \sigma^2 I)$. Its density, viewed as a function of θ , is the marginal likelihood, or evidence, and maximizing it is the type-II maximum likelihood estimate (MacKay 1992).

Definition (log marginal likelihood)

With $A = K_\theta + \sigma^2 I$, the log marginal likelihood is

$$\mathcal{L}(\theta) = \ln p(y \mid X, \theta) = -\frac{1}{2} y^\top A^{-1} y - \frac{1}{2} \ln \det A - \frac{n}{2} \ln(2\pi).$$

The three terms tell the whole story of why this works. The quadratic term $-\frac{1}{2} y^\top A^{-1} y$ rewards fitting the data. The log-determinant term $-\frac{1}{2} \ln \det A$ is a complexity penalty: a flexible covariance that could explain almost anything has a large determinant and is charged for it. Maximizing their sum trades fit against complexity automatically, an instance of Occam's razor built into the arithmetic, with no held-out set required. Differentiating with respect to a hyperparameter θ_j gives, by the standard identities $\partial \ln \det A = \text{tr}(A^{-1} \partial A)$ and $\partial A^{-1} = -A^{-1}(\partial A)A^{-1}$,

$$\frac{\partial \mathcal{L}}{\partial \theta_j} = \frac{1}{2} y^\top A^{-1} \frac{\partial A}{\partial \theta_j} A^{-1} y - \frac{1}{2} \text{tr}\left(A^{-1} \frac{\partial A}{\partial \theta_j}\right),$$

which is the gradient step in the GP regression algorithm above. Gradient ascent or a Newton method then finds a (local) evidence maximum. The cost is dominated by the $O(n^3)$ factorization of A , which is why scalable approximations, sparse and low-rank, are the subject of the large-scale chapter.

A refinement of this idea is automatic relevance determination. Give each input dimension its own length scale, $k(x, x') = \exp(-\sum_d (x_d - x'_d)^2 / 2\ell_d^2)$, and let the evidence choose them. A dimension that is irrelevant to the target drives its $1/\ell_d^2$ toward zero, effectively removing that input. The same principle, applied not to input dimensions but to basis functions, is what makes the Relevance Vector Machine sparse.

41.6 Gaussian process classification

For classification the target is a label $y \in \{-1, +1\}$, and the additive-Gaussian story breaks: a Gaussian latent value cannot be a Bernoulli label. We keep the Gaussian process prior on a latent function $t(x)$ but squash it through a non-Gaussian link, typically the logistic $P(y = 1 | t) = \sigma(t) = (1 + e^{-t})^{-1}$ or the probit $P(y = 1 | t) = \Phi(t)$. The posterior over the latent values is now

$$p(t | X, Y) \propto \left[\prod_{i=1}^m p(y_i | t(x_i)) \right] \exp\left(-\frac{1}{2} t^\top K^{-1} t\right),$$

which is no longer Gaussian, and the predictive integral has no closed form. Two approximations dominate.

The *Laplace approximation* (Williams and Rasmussen 1996) fits a Gaussian to the posterior at its mode. Writing the negative log posterior as $\Psi(t) = -\sum_i \ln p(y_i | t(x_i)) + \frac{1}{2} t^\top K^{-1} t$, one finds the mode by Newton's method: with c the gradient of the log likelihood and $C = -\nabla^2 \ln p(y | t)$ its (diagonal) negative Hessian, the update is

$$t_{\text{new}} = (K^{-1} + C)^{-1} (C t_{\text{old}} + c),$$

equivalent in coefficient form to $\alpha_{\text{new}} = (KC + I)^{-1} (KC \alpha_{\text{old}} + c)$. The curvature C at the mode then supplies the covariance of the Gaussian approximation, from which predictive probabilities follow. Because the logistic likelihood is log-concave, Ψ is convex and the mode is unique. A second option replaces the intractable posterior with the best matching Gaussian in a moment-matching sense rather than a curvature sense; expectation propagation is the standard such scheme, and it is generally more accurate than Laplace for the probit link, at comparable cost. A variational method of Jaakkola and Jordan, which sandwiches the logistic between tractable quadratic bounds, is a third route (Schölkopf and Smola 2002). In every case the approximation reduces classification to a sequence of the same linear-algebra operations that regression required, an $n \times n$ solve per iteration.

41.7 Laplace mechanics: from mode to prediction

The previous section stated the Laplace idea in one line, fit a Gaussian at the mode, and moved on. Since this approximation is the workhorse of Gaussian process classification, it deserves to be opened up: which Gaussian is being fitted, how the computation is organized so that it never forms K^{-1} , and what the classifier finally reports. The program was carried through for GP classifiers by Williams and Barber (1998), including the multiclass case with a softmax link; the numerically stable formulation below follows Rasmussen and Williams (2006).

Start from the negative log posterior $\Psi(t) = -\ln p(Y | t) + \frac{1}{2} t^\top K^{-1} t$, up to an additive constant, and let $\hat{t}$ be its minimizer, found by the Newton iteration of the previous section. Because the gradient vanishes at the minimizer, the Taylor expansion of Ψ around $\hat{t}$ has no linear term:

$$\Psi(t) = \Psi(\hat{t}) + \frac{1}{2} (t - \hat{t})^\top (K^{-1} + C) (t - \hat{t}) + O(\|t - \hat{t}\|^3),$$

where $C = -\nabla^2 \ln p(Y | \hat{t})$ is the likelihood curvature at the mode, diagonal because the likelihood factorizes over points. Dropping the cubic remainder and exponentiating leaves a Gaussian, and that Gaussian is the Laplace approximation:

$$p(t \mid X, Y) \approx q(t \mid X, Y) = \mathcal{N}(\hat{t}, (K^{-1} + C)^{-1}).$$

The covariance adds two curvatures: the prior contributes K^{-1} and each observation contributes C_{ii} on the diagonal. For the logistic link $C_{ii} = \pi_i(1 - \pi_i) \leq \frac{1}{4}$ with $\pi_i = \sigma(\hat{t}_i)$, so a single binary label is only mildly informative, and most informative exactly where the current prediction is undecided at $\pi_i = \frac{1}{2}$.

The stationarity condition $\nabla \Psi(\hat{t}) = 0$ reads $\hat{t} = K\hat{c}$ with $\hat{c} = \nabla \ln p(Y \mid \hat{t})$: the mode is once more a kernel expansion over the training points, the classification counterpart of $\alpha = (K + \sigma^2 I)^{-1}y$ in regression. The coefficients are readable. For the logistic link, $\hat{c}_i = y_i \sigma(-y_i \hat{t}_i)$, which is the probability the model currently assigns to the *wrong* class at point i , signed by the label. Confidently fitted points contribute almost nothing to the expansion; uncertain and misfitted points carry it. The Laplace classifier has soft support vectors: the hard sparsity of the SVM is replaced by an exponential taper of influence.

Numerics next. The matrix $K^{-1} + C$ should never be assembled from K^{-1} , whose computation is exactly what ill-conditioned kernels punish. The right object is

$$B = I + C^{1/2} K C^{1/2},$$

whose eigenvalues lie in $[1, 1 + \frac{1}{4}\lambda_{\max}(K)]$ for the logistic link: bounded below by one no matter how close to singular K is. By the Woodbury identity, $(K^{-1} + C)^{-1} = K - K C^{1/2} B^{-1} C^{1/2} K$, so every Newton step and every predictive quantity reduces to Cholesky solves against B , at $O(n^3)$ per iteration and only a handful of iterations, since the log-concave likelihood makes Ψ convex.

Prediction assembles the same two Gaussian-conditioning moves as regression, now under the approximate posterior. The latent value at x_* has approximate mean and variance

$$\mu_* = k_*^\top \hat{c}, \quad v_* = k_{**} - k_*^\top C^{1/2} B^{-1} C^{1/2} k_*,$$

the mean following from $\mathbb{E}_q[f_*] = k_*^\top K^{-1} \hat{t} = k_*^\top \hat{c}$, the variance from $(K + C^{-1})^{-1} = C^{1/2} B^{-1} C^{1/2}$. The reported class probability then averages the link over the latent uncertainty,

$$\bar{\pi}_* = \int \sigma(f) \mathcal{N}(f \mid \mu_*, v_*) df,$$

a one-dimensional integral: closed form $\bar{\pi}_* = \Phi(\mu_*/\sqrt{1 + v_*})$ for the probit link, a short quadrature for the logistic. The averaging is not decoration. Plugging μ_* straight into the sigmoid ignores v_* and overstates confidence far from the data; averaging pulls $\bar{\pi}_*$ toward $\frac{1}{2}$ exactly where the latent variance says the model does not know.

Finally, the same expansion prices the whole fit: integrating the Gaussian approximation gives an approximate evidence

$$\ln q(Y \mid X, \theta) = -\frac{1}{2} \hat{t}^\top \hat{c} + \ln p(Y \mid \hat{t}) - \frac{1}{2} \ln \det B,$$

using $\hat{t}^\top K^{-1} \hat{t} = \hat{t}^\top \hat{c}$ at the mode. The three terms replay the regression marginal likelihood: a prior charge, a data fit, and a determinant acting as the Occam factor. Ascending this quantity tunes length scales and amplitudes for classification exactly as the exact evidence did for regression; Williams and Barber (1998) ran this adaptation with the multiclass softmax, where C becomes block structured but the pipeline is unchanged. The same mechanics reappear wherever a Gaussian prior meets a factorized non-Gaussian likelihood, from robust regression to point-process intensity models.

Algorithm (Laplace GP classification)

Input Gram matrix K ; labels $y \in \{-1, +1\}^n$; link likelihood $p(y_i \mid t_i)$; test point x_* .

Output Predictive class probability $\bar{\pi}_*$; approximate evidence $\ln q(Y \mid X, \theta)$.

1. Initialize $t = 0$.
2. Compute the gradient $c = \nabla \ln p(Y | t)$ and curvature $C = -\nabla^2 \ln p(Y | t)$; form $B = I + C^{1/2} K C^{1/2}$ and its Cholesky factor.
3. Newton step: set $b = Ct + c$ and update $t \leftarrow K(b - C^{1/2} B^{-1} C^{1/2} K b)$.
4. Repeat steps 2 and 3 until Ψ stops decreasing; call the results $\hat{t}, \hat{c}, \hat{C}, \hat{B}$.
5. Predict: $\mu_* = k_*^\top \hat{c}$, $v_* = k_{**}^\top \hat{C}^{-1/2} \hat{B}^{-1} \hat{C}^{1/2} k_*$, and $\tilde{\pi}_* = \int \sigma(f) \mathcal{N}(f | \mu_*, v_*) df$ (probit: $\Phi(\mu_*/\sqrt{1 + v_*})$).
6. Report $\ln q(Y | X, \theta) = -\frac{1}{2} \hat{t}^\top \hat{c} + \ln p(Y | \hat{t}) - \frac{1}{2} \ln \det \hat{B}$ for hyperparameter ascent.

41.8 Sparse Bayesian priors and the Relevance Vector Machine

The Gaussian process prior spreads its belief smoothly, and its posterior mean uses every training point: $\alpha = (K + \sigma^2 I)^{-1} y$ is generically dense. Sometimes we want the opposite, a prediction supported on a handful of basis functions. Sparsity is a statement about the prior, and there are two ways to build it in.

The first keeps a single prior and changes its shape. A *Laplacian process* (Schölkopf and Smola 2002) places an independent Laplace prior on the expansion coefficients of $f(x) = \sum_i \alpha_i k(x_i, x)$, so that $-\ln p(\alpha) \propto \sum_i |\alpha_i|$. The MAP estimate then minimizes a data-fit term plus an ℓ_1 penalty, a linear or quadratic program whose solutions are genuinely sparse, in the spirit of basis pursuit and the LASSO. Because the prior depends on the data locations through the kernel expansion it is a data-dependent prior, a mild departure from the textbook Bayesian setup, but a fruitful one: it imports ℓ_1 sparsity into the kernel world while retaining error bars.

The second way, and the one we develop in detail, is the Relevance Vector Machine of Tipping (2001). Rather than fix the prior variance of the coefficients, give every coefficient its own variance, controlled by its own hyperparameter, and let the evidence decide which variances to shrink to zero.

Definition (RVM prior)

Model the target as $t = K\alpha$ with independent Gaussian noise of variance σ^2 . Place on each coefficient an independent zero-mean Gaussian prior with its own precision $s_i > 0$,

$$p(\alpha_i | s_i) = \sqrt{\frac{s_i}{2\pi}} \exp\left(-\frac{1}{2} s_i \alpha_i^2\right), \quad p(\alpha | s) = \mathcal{N}(0, S^{-1}), \quad S = \text{diag}(s_1, \dots, s_m),$$

with a flat hyperprior on $\ln s_i$ (or a broad Gamma prior). Each s_i is an automatic relevance determination hyperparameter for one basis function.

The precision s_i is the inverse prior variance of α_i : a large s_i means α_i is pinned tightly to zero, a small s_i lets it roam. With the noise variance σ^2 and Gaussian prior fixed, the posterior over α is Gaussian with covariance and mean

$$\Sigma = (\sigma^{-2} K^\top K + S)^{-1}, \quad \mu = \sigma^{-2} \Sigma K^\top y.$$

Learning the s_i proceeds by type-II maximum likelihood, exactly as for GP hyperparameters: the coefficients are integrated out to give the marginal likelihood $p(y | s, \sigma^2) = \mathcal{N}(0, \sigma^2 I + K S^{-1} K^\top)$, which is then maximized over s and σ^2 . Setting derivatives to zero yields fixed-point updates.

Algorithm (RVM update)

Input Design matrix K ($K_{ij} = k(x_i, x_j)$), targets y ; initial precisions s , noise σ^2 .

Output Sparse mean coefficients μ ; most $\mu_i = 0$.

1. Form the posterior covariance $\Sigma = (\sigma^{-2} K^\top K + S)^{-1}$ and mean $\mu = \sigma^{-2} \Sigma K^\top y$.
2. For each i , compute the well-determinedness $\gamma_i = 1 - s_i \Sigma_{ii} \in [0, 1]$.
3. Update each precision $s_i \leftarrow \frac{\gamma_i}{\mu_i^2} = \frac{1 - s_i \Sigma_{ii}}{\mu_i^2}$.

4. Update the noise $\sigma^2 \leftarrow \frac{\|y - K\mu\|^2}{m - \sum_i \gamma_i}$.
5. Whenever $s_i \rightarrow \infty$ (numerically, exceeds a large threshold), prune basis i : delete its row and column and continue. Repeat from step 1 until μ stabilizes.

Why does this produce sparsity? The update $s_i \leftarrow (1 - s_i \Sigma_{ii})/\mu_i^2$ is the mechanism. If a basis function contributes little to explaining y , its posterior mean μ_i is small while its posterior variance Σ_{ii} stays near the prior variance $1/s_i$, so $\gamma_i = 1 - s_i \Sigma_{ii}$ is near zero and the numerator collapses faster than the denominator: s_i is driven upward, which shrinks μ_i further, which pushes s_i higher still. The feedback runs away, $s_i \rightarrow \infty$, the prior variance $1/s_i \rightarrow 0$, and α_i is clamped exactly at zero and removed. Only the basis functions the data genuinely demand survive; Tipping (2001) calls the survivors the relevance vectors. An equivalent view integrates the hyperparameter out: with a Gamma hyperprior, $\int p(\alpha_i | s_i) p(s_i) ds_i$ is a Student- t distribution over α_i , sharply peaked at zero with heavy tails, and a MAP estimate under such a prior naturally sends most coefficients to zero while letting a few grow large. The effect is like the support vectors of an SVM, but the retained points need not lie near the decision boundary, and the solution is typically far sparser. In the large-sample limit the RVM converges to a Gaussian process with a data-dependent covariance, closing the loop with the start of the chapter (Schölkopf and Smola 2002).

The price of this elegance is that the marginal likelihood over the s_i is highly multimodal, so the fixed-point iteration finds a local optimum and the training is heavier than an SVM's on large data. The reward is a probabilistic, extremely sparse predictor that still carries the full apparatus of Bayesian error bars, and whose predictive mean and variance,

$$\bar{y}(x_*) = k_*^\top \mu, \quad v(x_*) = \sigma^2 + k_*^\top \Sigma k_*,$$

mirror the Gaussian process formulas of the regression section, now built on only the relevance vectors.

41.9 Sparse and variational Gaussian processes

The Relevance Vector Machine made its predictor sparse because the prior wanted few active basis functions. A second, blunter pressure pushes in the same direction: cost. Every exact GP quantity in this chapter, the posterior mean, the variance, the evidence and its gradient, passes through a factorization of the $n \times n$ matrix $K + \sigma^2 I$, at $O(n^3)$ time and $O(n^2)$ memory, repeated at every step of hyperparameter learning. Beyond a few tens of thousands of points the exact equations stop being computable. The large-scale chapter develops the generic remedies, random features and Nyström-type low-rank factorizations, which approximate the *matrix*. The Gaussian process literature developed a complementary line that approximates the *model*: replace the full process by one whose information about the data is carried by $m \ll n$ well-placed points. The prize for staying inside the probabilistic frame is that variances and evidences survive with their meaning intact, and the quality of the approximation itself becomes a measurable quantity. Throughout this section n counts training points and m counts inducing points.

Definition (inducing points and the Nyström surrogate)

Let $Z = (z_1, \dots, z_m)$ be *inducing inputs* in $\mathcal{X}$ and let $u = (t(z_1), \dots, t(z_m))$ be the latent process evaluated there. Write K_{uu}, K_{uf}, K_{ff} for the kernel matrices among inducing and training inputs (so $K_{ff} = K$), and define, for any two blocks of inputs a and b ,

$$Q_{ab} = K_{au} K_{uu}^{-1} K_{ub}.$$

Q_{ab} is the covariance that remains when all correlation between a and b is forced to pass through u ; as a matrix approximation it is exactly the Nyström approximation of K_{ab} built from the columns at Z .

Conditioning the joint Gaussian prior of (f, u) gives $p(f | u) = \mathcal{N}(K_{fu} K_{uu}^{-1} u, K_{ff} - Q_{ff})$, by the same block formula that produced the predictive equations. The conditional covariance $K_{ff} - Q_{ff}$ is a Schur complement,

hence positive semidefinite, and its i th diagonal entry vanishes whenever x_i belongs to Z : it measures exactly the part of the prior that the bottleneck u fails to carry. The classical sparse constructions, unified by Quiñero-Candela and Rasmussen (2005), all keep the exact $p(u) = \mathcal{N}(0, K_{uu})$ and tamper only with this conditional. The *subset of regressors* (SoR) approximation pretends the conditional is deterministic at training and test points alike, $f = K_{fu} K_{uu}^{-1} u$; the model degenerates to a rank- m GP with kernel Q , and its variances collapse wherever Q does. The *deterministic training conditional* (DTC) keeps the deterministic rule at the training points but restores the exact conditional at test points, repairing the predictive variance. The *fully independent training conditional* (FITC) of Snelson and Ghahramani (2006) is subtler: it keeps every training point's exact conditional variance and severs only their correlations, replacing $K_{ff} - Q_{ff}$ by its diagonal. Each variant is exact Gaussian inference in its modified model, and all share one predictive template. Setting

$$\Lambda_{\text{SoR}} = \Lambda_{\text{DTC}} = \sigma^2 I, \quad \Lambda_{\text{FITC}} = \text{diag}(K_{ff} - Q_{ff}) + \sigma^2 I,$$

and $\Sigma = (K_{uu} + K_{uf} \Lambda^{-1} K_{fu})^{-1}$, the predictive law at x_* is $\mathcal{N}(\mu_*, v_*)$ with

$$\mu_* = k_{u*}^\top \Sigma K_{uf} \Lambda^{-1} y, \quad v_* = k_{**} - Q_{**} + k_{u*}^\top \Sigma k_{u*},$$

where k_{u*} collects the covariances between Z and x_* and $Q_{**} = k_{u*}^\top K_{uu}^{-1} k_{u*}$. SoR alone replaces the honest k_{**} by Q_{**} , which is what deflates its error bars away from Z . The cost is $O(nm^2)$ once, then $O(m)$ per test mean and $O(m^2)$ per test variance: linear in n , with m ours to choose.

Algorithm (sparse GP prediction with inducing points)

Input Training data X, y (n points); inducing inputs Z ($m \ll n$); kernel k ; noise σ^2 ; family (DTC or FITC).

Output Predictive mean and variance at any x_* ; a training objective for θ, σ^2, Z .

1. Form K_{uu} ($m \times m$) and K_{uf} ($m \times n$); factor K_{uu} by Cholesky.
2. With k_{ui} the i th column of K_{uf} , compute the Nyström diagonal $q_{ii} = k_{ui}^\top K_{uu}^{-1} k_{ui}$ for all i in $O(nm^2)$; never form the full Q_{ff} .
3. Set $\Lambda = \sigma^2 I$ (DTC) or $\Lambda = \text{diag}(k(x_i, x_i) - q_{ii}) + \sigma^2 I$ (FITC); Λ is diagonal.
4. Factor $\Sigma^{-1} = K_{uu} + K_{uf} \Lambda^{-1} K_{fu}$ ($m \times m$) and cache the weights $w = \Sigma K_{uf} \Lambda^{-1} y$.
5. At a test point form k_{u*} and report $\mu_* = k_{u*}^\top w$, $v_* = k_{**} - k_{u*}^\top K_{uu}^{-1} k_{u*} + k_{u*}^\top \Sigma k_{u*}$.
6. To train, ascend the sparse evidence $\ln \mathcal{N}(y | 0, Q_{ff} + \Lambda)$ or the variational bound $\mathcal{L}_T$ below over θ, σ^2 , and the positions Z ; each gradient costs $O(nm^2)$.

41.9.1 The variational answer: sparsity without changing the model

DTC and FITC are honest algorithms with an awkward epistemology: each is exact inference in a model that is not the one we wrote down. Their evidences are evidences *of the surrogates*, so ascending them over Z rewards whichever surrogate most flatters the data, not the best approximation to the true posterior; the inducing positions act as extra kernel parameters, free to overfit. Titsias (2009) recast sparsity as approximate inference in the *exact* model. Choose a variational posterior of the constrained form $q(f, u) = p(f | u) q(u)$, which may reshape belief over the inducing values but must propagate it to f through the true conditional, and maximize a lower bound on the exact evidence. Maximizing over $q(u)$ in closed form collapses the bound to a formula.

Proposition (Titsias, 2009)

For any inducing inputs Z , the collapsed variational bound

$$\mathcal{L}_T = \ln \mathcal{N}(y | 0, Q_{ff} + \sigma^2 I) - \frac{1}{2\sigma^2} \text{tr}(K_{ff} - Q_{ff})$$

satisfies $\mathcal{L}_T \leq \ln p(y | X)$, with equality whenever $Q_{ff} = K_{ff}$, in particular when $Z = X$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Fix u and apply Jensen's inequality to the training conditional: $\ln p(y | u) = \ln \mathbb{E}_{p(f|u)}[p(y | f)] \geq \mathbb{E}_{p(f|u)}[\ln p(y | f)]$. With $p(f | u) = \mathcal{N}(a, K_{ff} - Q_{ff})$, $a = K_{fu}K_{uu}^{-1}u$, and $\ln p(y | f) = -\frac{1}{2\sigma^2}\|y - f\|^2 - \frac{n}{2}\ln(2\pi\sigma^2)$, the expectation uses $\mathbb{E}\|y - f\|^2 = \|y - a\|^2 + \text{tr}(K_{ff} - Q_{ff})$, giving the pointwise inequality

$$p(y | u) \geq \mathcal{N}(y | a, \sigma^2 I) \exp\left(-\frac{1}{2\sigma^2} \text{tr}(K_{ff} - Q_{ff})\right).$$

The trace factor does not depend on u . Averaging both sides over $p(u) = \mathcal{N}(0, K_{uu})$ and using the linear-Gaussian marginalization $\int \mathcal{N}(y | K_{fu}K_{uu}^{-1}u, \sigma^2 I) \mathcal{N}(u | 0, K_{uu}) du = \mathcal{N}(y | 0, Q_{ff} + \sigma^2 I)$ gives $p(y) \geq \mathcal{N}(y | 0, Q_{ff} + \sigma^2 I) e^{-\text{tr}(K_{ff} - Q_{ff})/(2\sigma^2)}$; take logarithms. If $Q_{ff} = K_{ff}$ the trace vanishes and the first term is the exact log marginal likelihood, so the inequality is tight; $Z = X$ gives $Q_{ff} = K_{ff}K_{ff}^{-1}K_{ff} = K_{ff}$. $\square$

Read $\mathcal{L}_T$ term by term. The first term is exactly the DTC evidence. The second charges it $\frac{1}{2\sigma^2} \sum_i \text{Var}[f_i | u]$, the total conditional variance the bottleneck throws away. That one trace changes the status of everything: the positions Z , and m itself, become variational parameters, and improving $\mathcal{L}_T$ provably tightens an approximation to one fixed model rather than drifting toward a more convenient model, with the bound only improving as inducing points are added (Titsias 2009). The optimal inducing posterior is available in closed form, $q^*(u) = \mathcal{N}(\sigma^{-2}K_{uu}\Sigma K_{uf}y, K_{uu}\Sigma K_{uu})$ with $\Sigma = (K_{uu} + \sigma^{-2}K_{uf}K_{fu})^{-1}$, and the prediction it induces coincides with the DTC formulas above. Variationally trained sparse GPs therefore predict like DTC; what changes is the objective that places Z and tunes the hyperparameters.

Example (two inducing points against the full GP)

Training inputs $x = (-2, -1, 0.5, 2)$, targets $y = (-1.7, -0.8, 0.7, 1.5)$, squared-exponential kernel $k(x, x') = e^{-(x-x')^2/8}$ (length scale $\ell = 2$), noise $\sigma^2 = 0.1$. Inducing inputs $Z = (-1, 0.5)$, the two middle training points, so $n = 4$ and $m = 2$. Predict at $x_* = 0.3$. All numbers from `checks/ch-gp-ex3.py`.

1. Fit the exact GP for reference.

$$K_{ff} = \begin{pmatrix} 1 & 0.8825 & 0.4578 & 0.1353 \\ 0.8825 & 1 & 0.7548 & 0.3247 \\ 0.4578 & 0.7548 & 1 & 0.7548 \\ 0.1353 & 0.3247 & 0.7548 & 1 \end{pmatrix},$$

and the full predictive equations give $\bar{m}(x_*) = 0.4672$, $v(x_*) = 0.0648$.

2. Assemble the Nyström surrogate. Since $Z = (x_2, x_3)$, $K_{uu} = \begin{pmatrix} 1 & 0.7548 \\ 0.7548 & 1 \end{pmatrix}$ is a principal submatrix of K_{ff} and K_{uf} is its rows 2 and 3. Then

$$Q_{ff} = K_{fu}K_{uu}^{-1}K_{uf} = \begin{pmatrix} 0.8797 & 0.8825 & 0.4578 & 0.0397 \\ 0.8825 & 1 & 0.7548 & 0.3247 \\ 0.4578 & 0.7548 & 1 & 0.7548 \\ 0.0397 & 0.3247 & 0.7548 & 0.7095 \end{pmatrix}.$$

Rows 2 and 3 reproduce K_{ff} exactly; the corners degrade, the x_1 - x_4 covariance dropping from 0.1353 to 0.0397 because it must be carried through u . The discarded diagonal is $\text{diag}(K_{ff} - Q_{ff}) = (0.1203, 0, 0, 0.2905)$, zero exactly at the inducing points, and $Q_{**} = 0.998$ against $k_{**} = 1$.

3. Predict with $\Lambda = \sigma^2 I$ (DTC, equally the Titsias posterior). The template gives $\mu_* = 0.5054$ and, keeping k_{**} , $v_* = 0.0443$. SoR substitutes Q_{**} and reports 0.0423.

4. Predict with FITC. $\Lambda = \text{diag}(0.2203, 0.1, 0.1, 0.3905)$: the noise floor 0.1 plus the discarded variance, inflated exactly at the two poorly covered inputs. The template gives $\mu_* = 0.4364$ and $v_* = 0.0608$.
5. Score the sparsification with the variational bound. The fit term is $\ln \mathcal{N}(y \mid 0, Q_{ff} + \sigma^2 I) = -4.982$ and the trace penalty $\text{tr}(K_{ff} - Q_{ff})/(2\sigma^2) = 2.0544$ (the trace itself is 0.4109), so $\mathcal{L}_T = -4.982 - 2.0544 = -7.0364$. The exact log marginal likelihood is -5.3762 : the bound holds with gap 1.6602, while the fit term alone, DTC's own evidence, overshoots the truth.

Reading. Half the data serving as inducing set moves the predictive means by a few hundredths, but the error bars tell the finer story: 0.0423 (SoR) < 0.0443 (DTC) < 0.0608 (FITC) against the exact 0.0648; the more of the discarded conditional variance a scheme keeps, the closer its uncertainty comes to the truth. The bound line is the operational one. $\mathcal{L}_T$ sits 1.6602 below the exact evidence, all of it charged by the trace over the two badly covered inputs x_1 and x_4 , so an optimizer ascending $\mathcal{L}_T$ over Z is pushed to cover exactly those points, and at $Z = X$ the bound closes.

Verification artifact. `checks/example-ch-gp-example-40-3.json` records the example source hash and verification scope.

41.9.2 Stochastic variational Gaussian processes

One computational ceiling remains: evaluating $\mathcal{L}_T$ or its gradient sweeps all n points through the $O(nm^2)$ sums, once per optimizer step. Hensman, Fusi, and Lawrence (2013) removed it by refusing to collapse. Keep $q(u) = \mathcal{N}(m_u, S)$ as explicit variational parameters; the uncollapsed evidence lower bound is

$$\mathcal{L}(m_u, S, Z, \theta) = \sum_{i=1}^n \mathbb{E}_{q(f_i)} [\ln p(y_i \mid f_i)] - \text{KL}(q(u) \parallel p(u)),$$

where $q(f_i)$ is Gaussian with mean $k_{ui}^\top K_{uu}^{-1} m_u$ and variance $k(x_i, x_i) - q_{ii} + k_{ui}^\top K_{uu}^{-1} S K_{uu}^{-1} k_{ui}$. For the Gaussian likelihood, maximizing over (m_u, S) at fixed Z recovers $\mathcal{L}_T$ exactly, so nothing is conceded. What is gained is the shape of the objective: the data enter only through a sum of independent per-point terms, so a random minibatch of size b , rescaled by n/b , gives an unbiased estimate of the sum, and stochastic gradient ascent applies jointly to m_u , S , Z , and the kernel hyperparameters. One step costs $O(bm^2 + m^3)$, with no dependence on n at all; the bound moreover fits the framework of stochastic variational inference, where natural-gradient steps in the exponential-family parameterization of $q(u)$ speed convergence. This is SVGP, demonstrated by Hensman, Fusi, and Lawrence (2013) on hundreds of thousands of flight records with a few hundred inducing points, and it is the template modern GP software implements for big data.

Two further dividends. Because each expectation in the sum is a one-dimensional Gaussian integral, any likelihood whose scalar expectations can be computed or quadratured slots straight in, so the same loop trains GP classifiers with the links of the classification section on data far beyond the Laplace approximation's reach. And the machinery composes: the sparse variational posterior is the substrate on which the deep constructions of the next section stand, and it is what makes Gaussian process surrogates affordable inside larger systems.

41.10 Deep, multi-output, and spectral-mixture processes

The prior we have used so far is restrictive in three separable ways: the kernel is picked from a small catalogue of fixed stationary forms, the process models a single scalar function, and the function is drawn in one shot rather than built out of simpler stages. Each restriction has a principled lift, and all three run on machinery this chapter has already built, evidence-driven hyperparameter learning and inducing-point inference.

41.10.1 Learning the kernel: spectral mixtures

Bochner’s theorem, developed in the kernel-families chapter, says a stationary kernel is the Fourier transform of a nonnegative spectral density: choosing a stationary covariance *is* choosing a distribution over frequencies. Seen through that lens the standard kernels are rigid commitments: the squared-exponential is a single Gaussian density centered at zero frequency, so it can say nothing beyond “smooth at scale ℓ ”, and automatic relevance determination tunes one number per dimension without ever changing the shape. Wilson and Adams (2013) put the flexibility where the theorem says it lives, modelling the spectral density as a mixture of Q Gaussians. In one dimension the resulting kernel is closed form,

$$k_{\text{SM}}(\tau) = \sum_{q=1}^Q w_q \exp(-2\pi^2 \sigma_q^2 \tau^2) \cos(2\pi \mu_q \tau), \quad \tau = x - x',$$

with weights w_q , center frequencies μ_q , and bandwidths σ_q . Each component is a squared-exponential envelope times a cosine: a quasi-periodic pattern at frequency μ_q that persists over a range of order $1/\sigma_q$; taking $Q = 1$ and $\mu_1 = 0$ recovers the squared-exponential itself. Since mixtures of Gaussians can approximate any spectral density, spectral-mixture kernels are dense among stationary covariances, and every parameter is learned by ascending the marginal likelihood: kernel learning reduced to the hyperparameter learning of this chapter. The practical payoff Wilson and Adams (2013) demonstrated is extrapolation, the evidence discovering periodic structure and continuing it beyond the data, where a fixed squared-exponential can only fall back to its mean.

41.10.2 Many outputs: coregionalization

Often the target is vector valued: p sensors on one machine, p related tasks, p outputs of a simulator. Modelling each coordinate with an independent GP wastes exactly what makes the problem interesting, that the outputs are correlated and data on one should sharpen predictions of another. A Gaussian prior over vector-valued functions needs a matrix-valued covariance, $\text{cov}(f_i(x), f_j(x'))$, positive semidefinite in the joint sense. The simplest separable choice, long used in geostatistics under the name co-kriging, is the *intrinsic coregionalization model*

$$\text{cov}(f_i(x), f_j(x')) = B_{ij} k(x, x'),$$

with k an ordinary kernel on inputs and B a positive semidefinite $p \times p$ *coregionalization matrix*; the joint Gram matrix is the Kronecker product $B \otimes K$, positive semidefinite by the product closure rules. Writing $B = AA^T$ with A of rank r exposes the generative reading: the p outputs are linear mixtures of r shared latent GPs, and the *linear model of coregionalization* sums several such terms with different base kernels. The evidence learns B , which is a task-similarity matrix read off from data, and transfer happens mechanically: an observation of output j enters the posterior of output i with weight proportional to B_{ij} . This is the construction behind multi-task surrogates in Bayesian optimization, where cheap evaluations of a related objective narrow the posterior over an expensive one, and Kronecker algebra plus the inducing-point machinery keeps the joint model tractable.

41.10.3 Depth: compositions of processes

Stationarity is a strong commitment: one length scale for the whole input space. A function that undulates in one region and is nearly flat in another fits no stationary prior well. The classical fix warps the inputs through a fixed map h and models $f(h(x))$; the radical fix is to learn the warp as a function too, and then there is no reason to stop at one stage. A *deep Gaussian process* (Damianou and Lawrence 2013) composes layers $f = f_L \circ \dots \circ f_1$, each layer a GP prior over a map between adjacent feature spaces. The composite is no longer a Gaussian process: its marginals are non-Gaussian, can be multimodal, and its effective smoothness varies with position, which is precisely the expressive gain. The price is inference, since each layer’s inputs are the previous layer’s random outputs; Damianou and Lawrence (2013) made it tractable by placing inducing points at every layer and nesting the variational bound of the previous section, and later stochastic variants scale the same construction with

minibatches. The contrast with Neal (1996) is worth savoring: widening a single hidden layer to infinity collapses a neural network onto one GP with a fixed kernel, while stacking finite GP layers keeps a learned, data-dependent composition. That dialogue, between kernels fixed by an architecture and representations learned through it, is where this book is heading; the frontier chapter takes it up in earnest through the infinite-width limits of deep networks and the kernels that grow depth.

41.11 Common mistakes and practical implications

For **Gaussian Processes and the RVM**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

41.12 Summary and further reading

This chapter established explain the central definitions and claims in Gaussian Processes and the RVM; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Rasmussen and Williams 2006), (Williams and Rasmussen 1996), (Neal 1996).

41.13 Exercises

1. **(Conditioning a bivariate normal.)** Let (f_1, f_2) be zero-mean Gaussian with covariance $K = \begin{pmatrix} 1 & 0.75 \\ 0.75 & 1 \end{pmatrix}$. Compute the conditional mean and variance of f_2 given $f_1 = 1$. Verify that observing f_1 reduces the variance of f_2 from 1 to $1 - 0.75^2$.
2. **(Noise-free limit.)** Show that as $\sigma^2 \rightarrow 0$ the GP posterior mean interpolates the data exactly, $\bar{m}(x_i) = y_i$ for every training point, and that $v(x_i) \rightarrow 0$ there. Why does this make σ^2 the analogue of the ridge parameter λn ?
3. **(KRR equals GP, symbolically.)** Starting from the ridge objective $\frac{1}{n} \|K\alpha - y\|^2 + \lambda \alpha^\top K \alpha$, derive the normal equations and confirm the solution $\alpha = (K + \lambda n I)^{-1} y$. State precisely why the GP posterior mean coincides with this fit but the GP additionally yields a variance. *Hint: identify the loss with the negative log Gaussian likelihood and the penalty with the negative log Gaussian prior.*
4. **(Marginal-likelihood gradient.)** With $A = K_\theta + \sigma^2 I$, verify the identities $\partial \ln \det A = \text{tr}(A^{-1} \partial A)$ and $\partial A^{-1} = -A^{-1}(\partial A)A^{-1}$, and use them to derive $\partial \mathcal{L} / \partial \theta_j$. Then specialize to $\theta_j = \sigma^2$, where $\partial A / \partial \sigma^2 = I$. *Hint: for the determinant identity differentiate $\ln \det A = \text{tr} \ln A$.*
5. **(Occam factor.)** Take a Gaussian covariance with amplitude a and length scale ℓ , so $K_\theta = a K_1(\ell)$. Explain qualitatively how the two terms of $\mathcal{L}$ move as ℓ shrinks toward zero (a very flexible prior) and as ℓ grows large (a very rigid one), and argue that the maximizer sits between the extremes. *Difficulty: medium.*
6. **(RVM fixed point.)** Show that the update $s_i \leftarrow (1 - s_i \Sigma_{ii}) / \mu_i^2$ has $s_i = \infty$, $\mu_i = 0$ as a fixed point, and argue that a basis function with small posterior mean and near-prior posterior variance is attracted to it. Interpret $\gamma_i = 1 - s_i \Sigma_{ii}$ as the number of well-determined parameters contributed by basis i . *Difficulty: hard. Hint: track how $\gamma_i \rightarrow 0$ and $\mu_i \rightarrow 0$ reinforce each other across iterations.*

7. **(Laplace vs. Gaussian prior.)** Contrast the MAP estimate under a Gaussian coefficient prior ($-\ln p \propto \sum_i \alpha_i^2$) with that under a Laplace prior ($-\ln p \propto \sum_i |\alpha_i|$). Explain why only the second yields exactly-zero coefficients, using the shape of the penalty near the origin. *Difficulty: medium.*
8. **(GP classification curvature.)** For the logistic likelihood $p(y | t) = \sigma(yt)$, compute $c_i = \partial_{t_i} \ln p(y_i | t_i)$ and $C_{ii} = -\partial_{t_i}^2 \ln p(y_i | t_i)$, and confirm $0 < C_{ii} \leq \frac{1}{4}$. Explain why log-concavity guarantees the Laplace-approximation mode is unique. *Difficulty: hard. Hint: $\sigma' = \sigma(1 - \sigma)$.*
9. **(The Nyström gap.)** Show that $K_{ff} - Q_{ff}$ is the covariance of the conditional $p(f | u)$, hence positive semidefinite, and that if a training input x_i belongs to the inducing set then the i th row and column of $K_{ff} - Q_{ff}$ vanish. Check both claims against the worked sparse example, where $Z = (x_2, x_3)$ gave $\text{diag}(K_{ff} - Q_{ff}) = (0.1203, 0, 0, 0.2905)$. Conclude that the FITC correction $\Lambda_{\text{FITC}} - \sigma^2 I$ never subtracts variance. *Difficulty: medium. Hint: a coordinate of u has zero variance given u , and a positive semidefinite matrix with a zero diagonal entry has a zero row.*
10. **(A bound and a non-bound.)** Show that at $Z = X$ one has $Q_{ff} = K_{ff}$, so both the DTC evidence $\ln \mathcal{N}(y | 0, Q_{ff} + \sigma^2 I)$ and the Titsias bound $\mathcal{L}_T$ equal the exact log marginal likelihood. Explain why for general Z only $\mathcal{L}_T$ is guaranteed to sit below the exact value, and confirm it on the worked example, where the DTC evidence -4.982 exceeds the exact -5.3762 while $\mathcal{L}_T = -7.0364$ respects the bound with gap 1.6602 . Which of the two is the safe objective for optimizing the positions Z , and why? *Difficulty: medium.*
11. **(Averaging the probit.)** Prove the closed form used in the Laplace predictive step: for $v \geq 0$,

$$\int \Phi(f) \mathcal{N}(f | \mu, v) df = \Phi\left(\frac{\mu}{\sqrt{1+v}}\right).$$

Conclude that latent uncertainty always pulls the reported class probability toward $\frac{1}{2}$ compared with plugging the mean into the link, and that the pull grows with v . *Difficulty: hard. Hint: write $\Phi(f) = P(Z \leq f)$ for $Z \sim \mathcal{N}(0, 1)$ independent of $f \sim \mathcal{N}(\mu, v)$, and read the left side as $P(Z - f \leq 0)$ with $Z - f \sim \mathcal{N}(-\mu, 1 + v)$.*

42 Kernelized Bandits and Bayesian Optimization

Every method so far assumed the data were given: a fixed sample arrives, and we fit. This chapter hands the learner a new power and a new cost. The power is to choose where to look next; the cost is that each look is expensive, so we get only a handful. Think of tuning the hyperparameters of a large model, where one evaluation is a full training run, or running a physical experiment, where one measurement takes a day in the lab. The objective f is unknown, we can query it only at points of our choosing, each query returns a noisy value, and we want to find its maximum in as few queries as possible. The engine that makes this tractable is the Gaussian process posterior of the Bayesian-view chapter, now used not to predict but to decide. We keep a running belief about f , score each candidate point by how promising it looks under that belief, query the winner, update, and repeat. The score is called an acquisition function, and the central tension it must resolve is between exploiting where the belief is already high and exploring where the belief is uncertain. The chapter builds the loop, derives the two workhorse acquisitions in closed form, and proves that the most analyzable of them, GP-UCB, is a no-regret algorithm whose regret is controlled by a single spectral quantity, the maximum information gain.

42.1 Sequential optimization of an unknown function

Fix a domain $\mathcal{X}$ and an unknown objective $f : \mathcal{X} \rightarrow \mathbb{R}$. At each round $t = 1, 2, \dots$ we choose a point $x_t \in \mathcal{X}$ and receive a noisy observation $y_t = f(x_t) + \varepsilon_t$, with ε_t zero-mean noise of variance σ^2 . We want the maximizer $x^\star = \arg \max_{x \in \mathcal{X}} f(x)$, but we never see f itself, only the values it returns where we probe. Two ingredients make the problem well posed. First, a regularity assumption: without one, an adversary could hide a sharp spike anywhere and no finite budget would find it. We assume either that f is a sample from a Gaussian process with a known covariance kernel k , the Bayesian setting, or that f lives in the reproducing kernel Hilbert space of k with bounded norm $\|f\|_k \leq B$, the agnostic setting. Both say the same thing in spirit: nearby points have similar values, at a rate the kernel fixes. Second, a way to score progress.

Definition (regret)

The *instantaneous regret* at round t is $r_t = f(x^\star) - f(x_t)$, the shortfall of the point we queried against the true optimum. The *cumulative regret* over horizon T is $R_T = \sum_{t=1}^T r_t$, and the *simple regret* is $\min_{t \leq T} r_t$, the shortfall of the best point found. A strategy is *no-regret* if $R_T/T \rightarrow 0$, so that its average per-round shortfall vanishes.

The two regrets fit two goals. Cumulative regret is the right cost when every query counts as a real action, as when a recommender must serve good content while it learns, the bandit framing. Simple regret is the right cost when only the final recommendation matters and the intermediate queries are cheap probes, as in hyperparameter tuning, the pure-optimization framing. They are linked by an inequality that we will lean on: since the minimum is below the average, $\min_{t \leq T} r_t \leq R_T/T$, so any cumulative-regret bound that grows slower than T forces the simple regret to zero and certifies that the search finds a near-optimal point. This is why controlling R_T is the mathematical heart of the subject even when optimization, not online reward, is the aim. The classical references are Shahriari, Swersky, Wang, Adams, and de Freitas (2016) for the review and Frazier (2018) for the tutorial.

42.2 The surrogate: a running Gaussian process belief

To decide where to look we need a belief about f at every point, not just where we have data, and it must come with a calibrated uncertainty so that "unexplored" is a quantity we can act on. The Gaussian process supplies exactly this. Place a zero-mean GP prior with covariance k on f ; after t rounds we hold the data $\mathcal{D}_t = \{(x_i, y_i)\}_{i=1}^t$, and the posterior is again a Gaussian process whose mean and variance we already derived in the Gaussian-process

chapter. Writing $k_t(x) = (k(x_1, x), \dots, k(x_t, x))^\top$ for the vector of covariances against the queried points and K_t for their Gram matrix, the posterior at any test point x is $\mathcal{N}(\mu_t(x), \sigma_t^2(x))$ with

$$\mu_t(x) = k_t(x)^\top (K_t + \sigma^2 I)^{-1} y_{1:t}, \quad \sigma_t^2(x) = k(x, x) - k_t(x)^\top (K_t + \sigma^2 I)^{-1} k_t(x).$$

These are the surrogate. The mean μ_t is the current best guess of f , and by the correspondence proved in the Bayesian chapter it is exactly the kernel ridge regression fit with regularizer σ^2 , so the agnostic RKHS analysis and the Bayesian analysis run on the same formulas. The variance σ_t^2 is the surrogate's honesty: it ignores the observed values y entirely and depends only on where we have queried, shrinking near data and swelling in the gaps. That is the object exploration will chase. Two practical notes carry over. The posterior costs an $n \times n$ solve that grows as we collect data, so the $O(n^3)$ factorization and its sparse and low-rank remedies from the large-scale chapter apply directly to long optimization runs. And the kernel's hyperparameters, the length scale and amplitude, are refit as data arrive by maximizing the marginal likelihood, the type-II route of the same chapter.

42.3 Acquisition functions and the exploration-exploitation tradeoff

The surrogate turns the decision into a one-dimensional judgement at each candidate: given a posterior mean $\mu_{t-1}(x)$ and standard deviation $\sigma_{t-1}(x)$, how much do we want to query x next? A good answer must reward two different things. High mean is good, because we are hunting a maximum and $\mu_{t-1}(x)$ is our guess of $f(x)$; this is exploitation. High variance is also good, because a point we know little about might hide the optimum, and querying it sharpens the belief everywhere nearby; this is exploration. An acquisition function $\alpha_t(x)$ is any rule that combines the two into a score, and the strategy is always the same: query $x_t = \arg \max_x \alpha_t(x)$. The art is in the combination, and three designs dominate.

42.3.1 GP-UCB: optimism in the face of uncertainty

The cleanest combination adds the two terms with a tunable weight. The upper confidence bound acquisition scores a point by an optimistic estimate of its value, the posterior mean plus a multiple of the posterior standard deviation,

$$\alpha_t^{\text{UCB}}(x) = \mu_{t-1}(x) + \beta_t^{1/2} \sigma_{t-1}(x),$$

and queries its maximizer. The name records the reading: $\alpha_t^{\text{UCB}}(x)$ is a high-probability upper bound on $f(x)$, and by always querying the point with the largest plausible value we are optimistic in the face of uncertainty. The weight $\beta_t^{1/2}$ is the exploration dial. At $\beta_t = 0$ the rule is greedy, chasing the current mean and never probing the unknown; as β_t grows it favours uncertain regions. Neither extreme works, and the regret theory below pins down the schedule β_t that balances them. This acquisition, and its analysis, are due to Srinivas, Krause, Kakade, and Seeger (2010), who named the algorithm GP-UCB.

Algorithm (GP-UCB)

Input decision set $D \subseteq \mathcal{X}$; kernel k ; noise variance σ^2 ; exploration schedule $\{\beta_t\}$; horizon T .

Output queried points $x_1, \dots, x_T$ and the best observed point.

1. Initialize $\mathcal{D}_0 = \emptyset$, so $\mu_0 \equiv 0$ and $\sigma_0^2(x) = k(x, x)$.
2. For $t = 1, \dots, T$: compute $\mu_{t-1}(x)$ and $\sigma_{t-1}(x)$ on D from $\mathcal{D}_{t-1}$ by the GP posterior equations.
3. Select $x_t = \arg \max_{x \in D} \mu_{t-1}(x) + \beta_t^{1/2} \sigma_{t-1}(x)$.
4. Query the objective, $y_t = f(x_t) + \varepsilon_t$, and set $\mathcal{D}_t = \mathcal{D}_{t-1} \cup \{(x_t, y_t)\}$.
5. Return $\arg \max_{t \leq T} y_t$ (or the posterior-mean maximizer $\arg \max_x \mu_T(x)$).

A tiny run makes the two forces visible. We take a five-point grid, a squared-exponential kernel, and watch the rule swing from exploration to exploitation across two rounds.

Example (two GP-UCB steps)

Decision set $D = \{0, 0.5, 1, 1.5, 2\}$; squared-exponential kernel $k(x, x') = e^{-(x-x')^2/(2\ell^2)}$ with length scale $\ell = 0.5$; noise variance $\sigma^2 = 0.01$; exploration weight fixed at $\beta_t^{1/2} = 2$ for illustration. The unknown objective is $f(x) = \sin(3x)$, whose grid maximum sits at $x^* = 0.5$ with $f(x^*) = 0.9975$. We start from two observations at the ends, $(0, 0)$ and $(2, -0.2794)$. All numbers from `checks/ch-bo-ex1.py`.

1. Fit the surrogate (round 1). With the two end points the posterior mean over the grid is $\mu = (-0, -0.003, -0.037, -0.168, -0.277)$ and the standard deviation is $\sigma = (0.099, 0.797, 0.982, 0.797, 0.099)$: near the data σ collapses, in the unexplored middle it is largest.
2. Score and query. The acquisition $\mu + 2\sigma = (0.199, 1.592, 1.926, 1.427, -0.078)$ peaks at $x = 1.0$, the most uncertain point, so round 1 *explores* the middle. Observe $y = f(1) = 0.1411$.
3. Refit (round 2). With three points the mean rises in the middle, $\mu = (0.000, 0.093, 0.139, -0.072, -0.276)$, and the uncertainty there drops, $\sigma = (0.099, 0.595, 0.099, 0.595, 0.099)$.
4. Score and query again. Now $\mu + 2\sigma = (0.199, 1.283, 0.338, 1.118, -0.077)$ peaks at $x = 0.5$: with the middle pinned down, the rule *exploits* the promising left shoulder. Observe $y = f(0.5) = 0.9975$, exactly the optimum.

Reading. Two queries suffice here: round 1 spends its budget reducing uncertainty where the belief is blank, round 2 cashes that in, and the simple regret drops to 0. The swing is entirely driven by the σ term shrinking after the first observation, which is the exploration-exploitation tradeoff playing out in numbers.

Verification artifact. `checks/example-ch-bo-example-41-1.json` records the example source hash and verification scope.

42.3.2 Expected improvement

UCB scores a point by an optimistic value. A different and older idea scores it by the improvement it is expected to bring over the best value seen so far. Let $f^+ = \max_{i \leq t} y_i$ be the current incumbent. If we query x and observe value v , the gain is the improvement $(v - f^+)^+ = \max(v - f^+, 0)$, zero if we do no better than the incumbent and the excess otherwise. Since v is unknown, we score x by the expectation of this gain under the posterior, which is the expected improvement of Mockus, Tiesis, and Žilinskas (1978), popularized as the efficient global optimization criterion by Jones, Schonlau, and Welch (1998). Its virtue is that the expectation has a closed form, so no sampling is needed.

Proposition (expected improvement, closed form)

Let the posterior at x be $f(x) \sim \mathcal{N}(\mu, \sigma^2)$ with $\sigma > 0$, write μ for $\mu_{t-1}(x)$ and σ for $\sigma_{t-1}(x)$, and let f^+ be the incumbent. With ϕ and Φ the standard-normal density and distribution function, the expected improvement is

$$\text{EI}(x) = \mathbb{E}[(f(x) - f^+)^+] = (\mu - f^+) \Phi(z) + \sigma \phi(z), \quad z = \frac{\mu - f^+}{\sigma}.$$

When $\sigma = 0$ the point is known and $\text{EI}(x) = (\mu - f^+)^+$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Write $f(x) = \mu + \sigma Z$ with $Z \sim \mathcal{N}(0, 1)$. The improvement $\mu - f^+ + \sigma Z$ is positive exactly when $Z > -(\mu - f^+)/\sigma = -z$, so

$$\text{EI}(x) = \int_{-z}^{\infty} (\mu - f^+ + \sigma u) \phi(u) du = (\mu - f^+) \int_{-z}^{\infty} \phi(u) du + \sigma \int_{-z}^{\infty} u \phi(u) du.$$

The first integral is $1 - \Phi(-z) = \Phi(z)$. For the second, note $\frac{d}{du}[-\phi(u)] = u \phi(u)$, so $\int_{-z}^{\infty} u \phi(u) du = [-\phi(u)]_{-z}^{\infty} = \phi(-z) = \phi(z)$, using that ϕ is even. Adding the two terms gives $(\mu - f^+)\Phi(z) + \sigma\phi(z)$. If $\sigma = 0$ there is no randomness and the improvement is the deterministic $(\mu - f^+)^+$. $\square$

The formula is worth reading termwise. The first term $(\mu - f^+)\Phi(z)$ is exploitation: it is large when the mean already beats the incumbent. The second term $\sigma\phi(z)$ is exploration: it is large when the uncertainty is high, and it rewards querying points whose mean is nowhere near the incumbent as long as they are uncertain enough that they *might* exceed it. Expected improvement therefore folds the tradeoff into a single expression with no free weight to tune, which is its main appeal over UCB, though a small exploration offset ξ is often subtracted, using $\mu - f^+ - \xi$ in place of $\mu - f^+$, to push it toward exploration. Notice also that when $\sigma = 0$ at an already-queried noiseless point, $\text{EI} = 0$, so the rule never wastes a query repeating itself.

Example (expected improvement at a candidate)

Same surrogate as the GP-UCB example, conditioned on the two initial observations, so the incumbent is $f^+ = \max(0, -0.2794) = 0$. We compute EI in full at the candidate $x = 1.0$, where the posterior is $\mu = -0.0374$, $\sigma = 0.9817$, then over the whole grid. Numbers from `checks/ch-bo-ex2.py`.

1. Standardize the gap. $z = (\mu - f^+)/\sigma = (-0.0374 - 0)/0.9817 = -0.0381$.
2. Evaluate the normal pieces. $\Phi(z) = 0.4848$ and $\phi(z) = 0.3987$.
3. Assemble the two terms. Exploitation $(\mu - f^+)\Phi(z) = -0.0181$, exploration $\sigma\phi(z) = 0.3914$, so $\text{EI}(1.0) = -0.0181 + 0.3914 = 0.3732$. The uncertainty term dominates: the mean is slightly below the incumbent, yet the point scores high.
4. Compare across the grid. $\text{EI} = (0.040, 0.317, 0.373, 0.241, 0.000)$ peaks at $x = 1.0$, so EI, like UCB, elects to explore the middle first.

Reading. At the left shoulder $x = 0.5$ the mean is higher but the uncertainty lower, giving $\text{EI} = 0.317$, just under the 0.373 at the more uncertain $x = 1.0$. Early on, when little is known, expected improvement is uncertainty-led, and only as σ shrinks do the mean-driven terms take over: the same arc the UCB run traced.

Verification artifact. `checks/example-ch-bo-example-41-2.json` records the example source hash and verification scope.

42.3.3 Thompson sampling and probability of improvement

A third design is probabilistic rather than deterministic. Thompson sampling, the oldest idea of all, going back to Thompson (1933), draws one sample function $\tilde{f}$ from the current GP posterior and queries its maximizer, $x_t = \arg \max_x \tilde{f}(x)$. Because the sample is random, the rule explores automatically: a point is queried with exactly the posterior probability that it is the true optimum, so uncertain regions get their fair share of queries without any explicit variance term. The GP version and its regret analysis are developed by Chowdhury and Gopalan (2017), and the general principle is surveyed by Russo, Van Roy, Kazerouni, Osband, and Wen (2018). A simpler and older sibling, the probability of improvement, scores a point by $\text{PI}(x) = \Phi(z)$, the posterior probability that it beats the incumbent, but it tends to under-explore because it ignores the size of the improvement, rewarding a near-certain tiny gain over an uncertain large one. Expected improvement fixes precisely that defect by weighting the probability by the magnitude.

These acquisitions are the interchangeable core of the Bayesian optimization loop. The loop itself is acquisition-agnostic: fit the surrogate, maximize the acquisition to pick the next query, evaluate, update, and repeat until the budget is spent.

Algorithm (Bayesian optimization loop)

Input domain $\mathcal{X}$; kernel k with hyperparameters θ ; acquisition α (for instance EI or UCB); initial design $\{x_i\}_{i=1}^{n_0}$; budget T .

Output the best evaluated point.

1. Evaluate the initial design, $y_i = f(x_i) + \varepsilon_i$, and fit the GP posterior (μ, σ) .
2. For $t = n_0 + 1, \dots, T$: re-estimate θ by maximizing the marginal likelihood, and refresh $(\mu_{t-1}, \sigma_{t-1})$.
3. Maximize the acquisition over the domain, $x_t = \arg \max_{x \in \mathcal{X}} \alpha_t(x)$, by a numerical inner optimizer (multi-start gradient ascent, or a fine grid in low dimension).
4. Query the objective, $y_t = f(x_t) + \varepsilon_t$, and append (x_t, y_t) to the data.
5. Return the incumbent $\arg \max_{t \leq T} y_t$, or the posterior-mean maximizer if the observations are noisy.

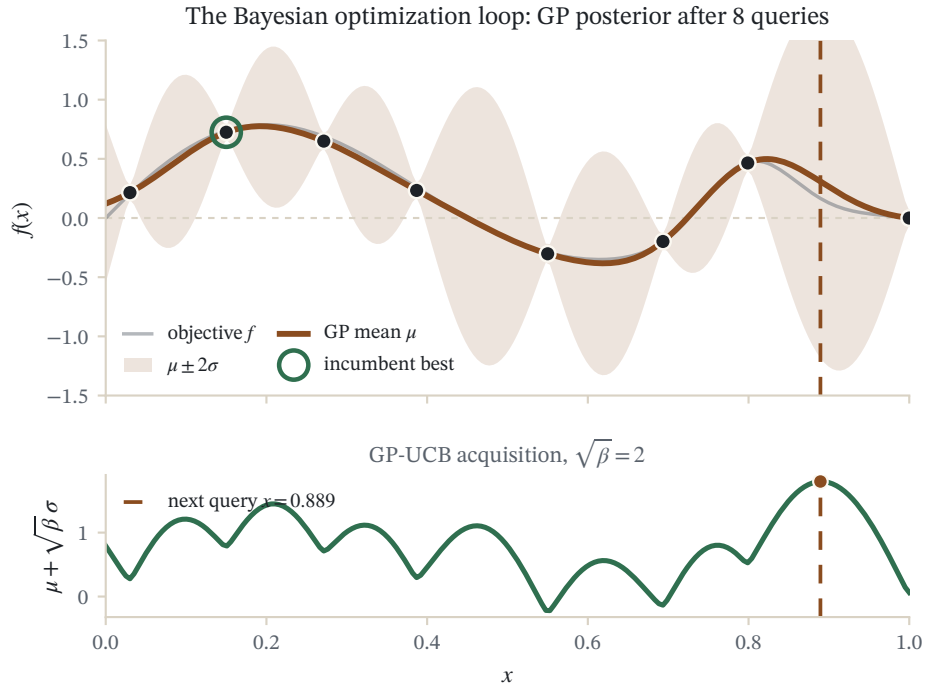


Figure 42.1: A Bayesian optimization loop on a hidden objective: the shaded band is the Gaussian-process posterior $\mu \pm 2\sigma$ refit after each query, and the lower strip is the acquisition function whose argmax (dashed line) chooses the next point.

Two costs deserve mention. The inner maximization in step 3 is itself a global optimization, but of the cheap acquisition rather than the expensive f , which is the whole point: we have traded queries of f for arithmetic on the surrogate. And re-fitting hyperparameters each round makes the prior data-dependent, so the clean Bayesian regret theory strictly applies to a fixed kernel; in practice the loop with adapted hyperparameters is what is run (Snoek, Larochelle, and Adams 2012).

42.4 Regret and the maximum information gain

We now make good on the promise that GP-UCB is a no-regret algorithm and identify what governs its rate. The remarkable fact, due to Srinivas, Krause, Kakade, and Seeger (2010), is that the cumulative regret is controlled by a single quantity that measures how much the queries can teach us about f : the maximum information gain. Its definition rests on the entropy of a Gaussian.

Definition (information gain and γ_T)

For a set of points $A = \{x_1, \dots, x_T\}$ with noisy observations y_A of the latent values f_A , the *information gain* is the mutual information $I(y_A; f_A) = H(y_A) - H(y_A | f_A)$. For a GP with Gaussian noise it has the closed form

$$I(y_A; f_A) = \frac{1}{2} \ln \det(I + \sigma^{-2} K_A),$$

with K_A the Gram matrix on A . The *maximum information gain* after T rounds is

$$\gamma_T = \max_{A \subseteq \mathcal{X}, |A|=T} I(y_A; f_A).$$

The closed form follows because $y_A = f_A + \varepsilon$ is Gaussian with covariance $K_A + \sigma^2 I$, so $H(y_A) = \frac{1}{2} \ln \det(2\pi e(K_A + \sigma^2 I))$ while $H(y_A | f_A) = \frac{1}{2} \ln \det(2\pi e \sigma^2 I)$, and the determinants divide. Read γ_T as the most any T queries could reduce our uncertainty about f , a purely kernel-and-domain quantity with no reference to the algorithm. It is small when the kernel is smooth, because then observations are highly redundant and few of them already say most of what there is to say, and it grows with roughness. Since $I(y_A; f_A)$ is a monotone submodular function of A , greedy selection of high-variance points is near-optimal for it, which is the bridge to the sequential algorithm. Crucially, γ_T is governed by the decay of the kernel's eigenvalues, the same spectrum that set approximation rates in the Mercer chapter, and the information gain is itself a divergence between the posterior and prior over f , a way of measuring distance between distributions that recurs through the RKHS embeddings of the mean-embedding chapter. The known bounds are as follows (Srinivas et al. 2010, with the polynomial rates later tightened by Vakili, Khezeli, and Picheny 2021).

Kernel on $[0, r]^d$	Maximum information gain γ_T
Linear	$O(d \ln T)$
Squared-exponential	$O((\ln T)^{d+1})$
Matern, smoothness ν	$O(T^{d(d+1)/(2\nu+d(d+1))} \ln T)$, refined toward $O(T^{d/(2\nu+d)} (\ln T)^{2\nu/(2\nu+d)})$

In every case γ_T is sublinear in T : polylogarithmic for the smooth kernels, and polynomial with an exponent below one for the Matern family once ν is large enough relative to d . That sublinearity is the whole game, because the regret bound is essentially $\sqrt{T\gamma_T}$ up to the exploration factor, and $\sqrt{T\gamma_T}/T \rightarrow 0$.

Theorem (GP-UCB regret, Srinivas, Krause, Kakade, and Seeger, 2010)

Let f be sampled from a zero-mean GP with covariance k satisfying $k(x, x) \leq 1$, on a finite decision set D , with Gaussian noise of variance σ^2 . Fix $\delta \in (0, 1)$ and run GP-UCB with the schedule $\beta_t = 2 \ln(|D| t^2 \pi^2 / (6\delta))$. Then with probability at least $1 - \delta$,

$$R_T \leq \sqrt{C_1 T \beta_T \gamma_T} \quad \text{for all } T \geq 1, \quad C_1 = \frac{8}{\ln(1 + \sigma^{-2})}.$$

A schedule with an extra logarithmic term in T gives the same rate on a continuous compact domain, and in the agnostic RKHS setting with $\|f\|_k \leq B$ the analogous bound is $R_T = O^*(\sqrt{T} (B\sqrt{\gamma_T} + \gamma_T))$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The proof factors into three clean pieces: a confidence bound that holds uniformly, an instantaneous-regret bound that turns optimism into a per-round guarantee, and a summation that ties the posterior variances to γ_T . We prove the two tractable middle pieces in full and cite the concentration and summation lemmas.

Lemma (uniform confidence bound)

Under the schedule above, with probability at least $1 - \delta$, every round t and every $x \in D$ satisfy $|f(x) - \mu_{t-1}(x)| \leq \beta_t^{1/2} \sigma_{t-1}(x)$. This is a union bound over Gaussian tails on the finite set D and the horizon; the schedule β_t is exactly the size needed for the total failure probability to sum to δ (Srinivas et al. 2010, Lemma 5.1).

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Lemma (optimism bounds instantaneous regret)

On the confidence event, the instantaneous regret obeys $r_t \leq 2 \beta_t^{1/2} \sigma_{t-1}(x_t)$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Apply the confidence bound at the optimum $x^\star$: $f(x^\star) \leq \mu_{t-1}(x^\star) + \beta_t^{1/2} \sigma_{t-1}(x^\star) = \alpha_t^{\text{UCB}}(x^\star)$. Since x_t maximizes the acquisition, $\alpha_t^{\text{UCB}}(x^\star) \leq \alpha_t^{\text{UCB}}(x_t) = \mu_{t-1}(x_t) + \beta_t^{1/2} \sigma_{t-1}(x_t)$. Apply the confidence bound again at x_t , now on the lower side: $f(x_t) \geq \mu_{t-1}(x_t) - \beta_t^{1/2} \sigma_{t-1}(x_t)$. Subtracting,

$$r_t = f(x^\star) - f(x_t) \leq [\mu_{t-1}(x_t) + \beta_t^{1/2} \sigma_{t-1}(x_t)] - [\mu_{t-1}(x_t) - \beta_t^{1/2} \sigma_{t-1}(x_t)] = 2 \beta_t^{1/2} \sigma_{t-1}(x_t).$$

The mean cancels, leaving twice the posterior uncertainty at the chosen point. This is the payoff of optimism: the very rule that could overreach instead guarantees that each round's shortfall is no larger than the uncertainty it was willing to bet on. $\square$

Lemma (variances sum to the information gain)

The posterior variances at the queried points satisfy $\sum_{t=1}^T \sigma_{t-1}^2(x_t) \leq \frac{2}{\ln(1 + \sigma^{-2})} \gamma_T$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The information gain from the sequential queries telescopes into per-round terms that use the variance just before each observation,

$$I(y_{1:T}; f_{1:T}) = \frac{1}{2} \sum_{t=1}^T \ln(1 + \sigma^{-2} \sigma_{t-1}^2(x_t)),$$

because conditioning on each new point multiplies the determinant by $1 + \sigma^{-2} \sigma_{t-1}^2(x_t)$ (Srinivas et al. 2010, Lemma 5.3). Now $k(x, x) \leq 1$ forces $\sigma_{t-1}^2(x_t) \leq 1$, so the argument $u = \sigma^{-2} \sigma_{t-1}^2(x_t)$ lies in $[0, \sigma^{-2}]$. On that interval the elementary inequality $u \leq \frac{\sigma^{-2}}{\ln(1 + \sigma^{-2})} \ln(1 + u)$ holds, since $u / \ln(1 + u)$ is increasing. Multiplying by σ^2 gives $\sigma_{t-1}^2(x_t) \leq \frac{1}{\ln(1 + \sigma^{-2})} \ln(1 + \sigma^{-2} \sigma_{t-1}^2(x_t))$. Summing and using the telescoping identity,

$$\sum_{t=1}^T \sigma_{t-1}^2(x_t) \leq \frac{2}{\ln(1 + \sigma^{-2})} I(y_{1:T}; f_{1:T}) \leq \frac{2}{\ln(1 + \sigma^{-2})} \gamma_T,$$

the last step because any actual selection has information gain at most the maximum γ_T . $\square$

Proof (of the theorem)

Work on the confidence event, which holds with probability at least $1 - \delta$. By the optimism lemma and $\beta_t \leq \beta_T$,

$$\sum_{t=1}^T r_t^2 \leq \sum_{t=1}^T 4\beta_t \sigma_{t-1}^2(x_t) \leq 4\beta_T \sum_{t=1}^T \sigma_{t-1}^2(x_t) \leq \frac{8\beta_T}{\ln(1 + \sigma^{-2})} \gamma_T = C_1 \beta_T \gamma_T,$$

using the variance-sum lemma. By Cauchy-Schwarz, $R_T = \sum_t r_t \leq \sqrt{T \sum_t r_t^2} \leq \sqrt{C_1 T \beta_T \gamma_T}$. $\square$

Since β_T grows only logarithmically and γ_T is sublinear, $R_T = O^*(\sqrt{T\gamma_T})$ up to log factors, and $R_T/T \rightarrow 0$: GP-UCB is no-regret, and by the simple-regret inequality it finds arbitrarily near-optimal points. The uniform confidence bound at the root of the argument is a concentration statement of exactly the flavour that drives the generalization bounds of the learning-theory chapter. To see γ_T as a genuine number rather than a symbol, we compute it for the run of the first example.

Example (information gain of a short run)

Take the four points GP-UCB acquired in the first example, in order $x = (0, 2, 1, 0.5)$, same kernel and $\sigma^2 = 0.01$. We compute the information gain $I = \frac{1}{2} \ln \det(I + \sigma^{-2}K)$, verify the telescoping identity, and check the variance-sum bound of the proof. Numbers from `checks/ch-bo-ex3.py`.

1. Batch information gain. With $\sigma^{-2} = 100$, the 4×4 Gram matrix gives $I = \frac{1}{2} \ln \det(I + 100K) = 8.7015$ nats. This is a lower bound on γ_4 , which maximizes over all four-point sets.
2. Telescoping check. The pre-observation variances at the four points are $v_t = (1, 1, 0.9637, 0.3539)$, and $\frac{1}{2} \sum_t \ln(1 + 100 v_t) = 8.7015$, matching I to four decimals: the identity in the variance-sum lemma holds numerically.
3. Variance-sum bound. The constant is $C = 2/\ln(1 + 100) = 0.4334$, so the lemma predicts $\sum_t v_t \leq C \cdot I = 0.4334 \times 8.7015 = 3.7709$. The actual sum is $\sum_t v_t = 3.3176$, comfortably under the bound.

Reading. The abstract chain of the proof becomes three concrete numbers: the queries earned 8.70 nats of information, the telescoping decomposition reproduces it exactly, and the cumulative posterior variance 3.32 sits below the certified ceiling 3.77. It is this ceiling, times the exploration factor, that the regret bound converts into a guarantee.

Verification artifact. `checks/example-ch-bo-example-41-3.json` records the example source hash and verification scope.

42.4.1 Sharper confidence: IGP-UCB

The Bayesian theorem assumes f is literally a GP sample. The agnostic setting, where f is only known to have bounded RKHS norm and the noise is sub-Gaussian, is harder, and Srinivas et al. reach it with an exploration schedule inflated by a $\ln^3 t$ factor, which is wasteful. Chowdhury and Gopalan (2017) remove that waste with a sharper self-normalized concentration inequality for the kernel ridge estimator, of the kind pioneered for linear bandits. Their bound, holding uniformly in t and x with probability at least $1 - \delta$, is

$$|f(x) - \mu_{t-1}(x)| \leq \left(B + R\sqrt{2(\gamma_{t-1} + 1 + \ln(1/\delta))} \right) \sigma_{t-1}(x),$$

where $B \geq \|f\|_k$ and R is the sub-Gaussian noise parameter. Reading off the bracket as $\beta_t^{1/2}$ and running the same GP-UCB rule, now called IGP-UCB, gives the regret bound $R_T = O(\sqrt{T\gamma_T} (B + \sqrt{\gamma_T + \ln(1/\delta)}))$, which drops the extra logarithmic factors of the earlier analysis and matches the Bayesian rate up to constants. The same paper analyzes GP Thompson sampling and proves a comparable bound, putting the two acquisition families on the same theoretical footing. The exploration weight is no longer a free dial: it is the width of a rigorous confidence tube, and γ_t inside it is once more the kernel's information budget doing the accounting.

42.5 Active learning and experimental design

Bayesian optimization selects points to improve the best objective value. Active learning selects points to improve a predictive model, and experimental design selects points to reduce uncertainty about parameters, functions, or decisions. They share the same GP posterior but optimize different utilities. Variance reduction, integrated posterior variance, entropy, mutual information, and expected value of information are not interchangeable with expected improvement.

For a finite candidate set, maximizing information gain about latent function values is a monotone submodular set problem. Greedy selection therefore has a constant-factor approximation guarantee under the usual Gaussian observation model (Krause et al. 2008). This explains why posterior-variance sampling often works for sensor placement, but it does not make variance sampling an optimizer: a design can learn the whole surface efficiently while spending few points near its maximum.

Goal	Appropriate utility	Required report
find a maximizer	EI, UCB, Thompson sampling	simple regret and incumbent
map a latent field	integrated variance or information	held-out calibration and coverage
estimate a parameter	D-, A-, or task-specific design	posterior sensitivity to the prior
classify near a boundary	predictive entropy or disagreement	classwise error and sampling bias
choose an intervention	expected decision value	utility model and safety constraints

Adaptive sampling changes the covariate distribution. A predictive model evaluated only on the adaptively acquired sample can look better than it is on the deployment population; use a representative evaluation set or importance-aware assessment.

42.6 Constrained and safe optimization

Suppose the objective $f(x)$ can be evaluated only where unknown constraints $c_j(x) \geq 0$ are satisfied. Constrained expected improvement multiplies improvement by posterior feasibility probabilities when objective and constraint models are conditionally independent (Gardner et al. 2014). Correlated outputs require a joint posterior calculation. A high probability of feasibility is not the same as a hard safety guarantee, especially under kernel misspecification.

Safe optimization starts from a known safe seed and expands only through points whose lower confidence bounds certify the constraint (Sui et al. 2015). This distinction is operationally critical: constrained BO tolerates occasional failed experiments as information, whereas safe BO treats a violation as unacceptable.

Algorithm (safe GP optimization)

1. Declare the safety threshold, trusted initial safe set, kernel, noise bound, and confidence schedule.
2. Fit separate or joint GP models for the objective and constraints.
3. Form the certified safe set from constraint lower confidence bounds.
4. Among safe candidates, balance objective improvement against points that could expand the certified set.
5. Query only a certified point, update all posteriors, and log near-violations and model checks.
6. Stop if the safe set cannot expand under the declared budget; do not silently relax the threshold.

SafeOpt-style guarantees inherit strong assumptions: the true constraint must obey the stated RKHS or GP model, confidence widths must be valid, and the safe region must be reachable from the seed through certifiable steps. If those assumptions are doubtful, add physical interlocks or a human approval layer rather than treating the posterior as a safety system.

42.7 Multi-fidelity and cost-aware search

Many experiments have a fidelity coordinate: small training subsets, coarse simulations, short horizons, low-resolution meshes, or inexpensive assays. Low fidelity is useful only to the extent that it predicts the high-fidelity target. A multi-output or product kernel over configuration and fidelity can learn that relationship; an acquisition then compares expected high-fidelity value against query cost. Continuous-fidelity methods choose both x and a fidelity level and can obtain guarantees when approximation bias is controlled (Kandasamy et al. 2017).

A robust implementation must learn or challenge cross-fidelity correlation rather than assume that cheap evaluations rank configurations correctly. Record wall time and resource cost, not just query count. The final recommendation must be evaluated at target fidelity, and the regret definition should use total cost when fidelities differ substantially.

Cost-aware acquisition divides a value measure by estimated cost or optimizes a utility with an explicit budget. The ratio heuristic can overfavor nearly free but uninformative queries, so impose a terminal-fidelity requirement and inspect the accumulated information about the target fidelity.

42.8 Batch, asynchronous, and high-dimensional Bayesian optimization

Two departures from the basic loop matter in practice. The first is parallelism. The loop as written is strictly sequential, one query per round, but experiments are often run in batches: an experimenter with sixteen machines wants sixteen points at once. The difficulty is that a naive batch would query the acquisition's maximizer sixteen times over, since the surrogate does not update within a batch. The fix is to encode the pending queries into the belief. GP-BUCB (Desautels, Krause, and Burdick 2014) selects a batch greedily, after each pick updating the posterior variance, which needs no observed value, so that later picks in the same batch avoid points made redundant by earlier ones, and it preserves the regret guarantees up to a bounded penalty. Local penalization (González, Dai, Hennig, and Lawrence 2016) instead multiplies the acquisition by a soft exclusion zone around each pending point, using a Lipschitz estimate to size it. Both turn the scalar loop into a batch loop while keeping the surrogate honest about what is already in flight.

Asynchronous workers finish at different times. The scheduler should condition on completed observations while retaining pending locations through fantasies, posterior integration, or penalization. It must also record failures and timeouts: deleting slow evaluations biases the data when runtime depends on the objective or constraint. Batch size, worker utilization, wall-clock regret, and duplicate-query rate belong in the evaluation.

The second departure is dimension. The information-gain bounds degrade quickly with d , and, more concretely, the acquisition's inner maximization and the space-filling role of exploration both suffer the curse of dimensionality: covering a high-dimensional box takes exponentially many points. Progress comes from structural assumptions that shrink the effective dimension. Additive models (Kandasamy, Schneider, and Póczos 2015) assume f decomposes as a sum of low-dimensional components on disjoint variable groups, so the surrogate and acquisition factor over the groups and the regret scales with the group size rather than d . Random embeddings (Wang, Hutter, Zoghi, Matheson, and de Freitas 2016) assume f varies only along an unknown low-dimensional subspace and optimize inside a random linear image of a low-dimensional box, provably hitting the active subspace with high probability. Both are instances of the same lesson that runs through the book: high-dimensional learning is only possible when the target has low intrinsic complexity, and the kernel, or here the search, must be built to exploit it.

42.9 Summary

Bayesian optimization is the Gaussian process turned from a predictor into a decision-maker. The posterior of the Bayesian chapter serves as a surrogate whose mean guesses the objective and whose variance measures ignorance, and an acquisition function scores each candidate by trading the two off. GP-UCB adds an optimistic multiple of the standard deviation to the mean; expected improvement, derived here in closed form as $(\mu - f^+) \Phi(z) + \sigma \phi(z)$,

scores the expected gain over the incumbent with no free weight; Thompson sampling explores by querying the maximizer of a posterior sample. The theory is unusually complete: GP-UCB is a no-regret algorithm whose cumulative regret is $O^*(\sqrt{T\beta_T\gamma_T})$, controlled by the maximum information gain γ_T , a spectral property of the kernel that is polylogarithmic for smooth kernels and sublinear for the Matern family. The proof reduces to optimism, which caps each round's regret at twice the posterior width, and a summation that ties those widths to γ_T ; Chowdhury and Gopalan (2017) sharpen the confidence tube to match. Batch and high-dimensional variants extend the loop by keeping the surrogate honest about pending queries and by exploiting low intrinsic dimension. What began as a covariance function ends as a principled recipe for spending a scarce experimental budget.

42.10 Common mistakes and practical implications

For **Kernelized Bandits and Bayesian Optimization**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

42.11 Summary and further reading

This chapter established explain the central definitions and claims in Kernelized Bandits and Bayesian Optimization; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Shahriari and Freitas 2016), (Frazier 2018), (Srinivas and Seeger 2010).

42.12 Exercises

1. warm-up In the first worked example, round 1 has posterior mean $\mu = (-0, -0.003, -0.037, -0.168, -0.277)$ on the grid. A purely greedy rule sets $\beta_t = 0$ and queries $\arg \max_x \mu_{t-1}(x)$. Which grid point does it choose, and why does that stall the search? Contrast with the GP-UCB choice $x = 1.0$, and explain in one sentence what the $\beta_t^{1/2}\sigma$ term buys.
2. computation Using the posterior in the expected-improvement example (incumbent $f^+ = 0$, the grid means and standard deviations given there), compute the probability of improvement $\text{PI}(x) = \Phi((\mu - f^+)/\sigma)$ at $x = 0.5$ and $x = 1.0$. Compare the two rankings, PI versus EI, and explain why PI can prefer a lower-variance point that EI passes over.
3. computation Verify the information-gain identity by hand on two points. Take the kernel and noise of the third worked example and the pair $A = \{0, 1\}$. Compute $I = \frac{1}{2} \ln \det(I + \sigma^{-2}K_A)$ directly, then via the telescoping form $\frac{1}{2} \ln(1 + \sigma^{-2}\sigma_0^2(x_1)) + \frac{1}{2} \ln(1 + \sigma^{-2}\sigma_1^2(x_2))$, and confirm the two agree.
4. proof Show that expected improvement is strictly increasing in the posterior standard deviation σ for fixed mean $\mu \neq f^+$. Conclude that, among points with equal mean, EI always prefers the more uncertain one, which is its exploration mechanism. Hint

Write $\text{EI} = \sigma(z\Phi(z) + \phi(z))$ with $z = (\mu - f^+)/\sigma$. Differentiate $g(z) = z\Phi(z) + \phi(z)$ to get $g'(z) = \Phi(z) > 0$, so g is increasing, and combine with $\partial z/\partial \sigma = -(\mu - f^+)/\sigma^2$; the product rule leaves $\partial \text{EI}/\partial \sigma = \phi(z) > 0$.

5. proof Prove the instantaneous-regret bound $r_t \leq 2\beta_t^{1/2}\sigma_{t-1}(x_t)$ of the optimism lemma from scratch, stating exactly where the confidence bound and the acquisition-maximizing choice of x_t each enter. Then explain why the argument fails for the greedy rule $\beta_t = 0$. Hint

Sandwich $f(x^\star)$ above by its UCB and $f(x_t)$ below by its lower confidence bound, then use $\alpha_t^{\text{UCB}}(x^\star) \leq \alpha_t^{\text{UCB}}(x_t)$. With $\beta_t = 0$ the two confidence bounds collapse to the mean and there is nothing to guarantee $f(x^\star) \leq \mu_{t-1}(x_t)$.

6. proof Establish the elementary inequality used in the variance-sum lemma: for $u \in [0, C_0]$, $u \leq \frac{C_0}{\ln(1+C_0)} \ln(1+u)$. Then apply it with $C_0 = \sigma^{-2}$ to complete the step $\sigma_{t-1}^2(x_t) \leq \frac{1}{\ln(1+\sigma^{-2})} \ln(1+\sigma^{-2}\sigma_{t-1}^2(x_t))$, noting where $k(x, x) \leq 1$ is used. Hint

The map $u \mapsto u/\ln(1+u)$ is increasing on $(0, \infty)$, so its value at any $u \leq C_0$ is at most its value at C_0 , giving $u/\ln(1+u) \leq C_0/\ln(1+C_0)$. The bound $k(x, x) \leq 1$ guarantees $\sigma_{t-1}^2(x_t) \leq 1$, hence $u = \sigma^{-2}\sigma_{t-1}^2(x_t) \leq \sigma^{-2} = C_0$.

7. challenge Deduce a simple-regret guarantee from the cumulative bound, then read it against the information-gain table. Using $\min_{t \leq T} r_t \leq R_T/T$ and $R_T \leq \sqrt{C_1 T \beta_T \gamma_T}$, write the simple-regret rate for the squared-exponential kernel (where $\gamma_T = O((\ln T)^{d+1})$) and for a Matern kernel with $\gamma_T = O(T^a)$, $0 < a < 1$. For which a does simple regret still vanish, and what does that say about the roughest objective GP-UCB can reliably optimize? Hint

$\min_{t \leq T} r_t \leq \sqrt{C_1 \beta_T \gamma_T / T}$, and $\beta_T = O(\ln T)$. For the squared-exponential this is $O^*(\sqrt{(\ln T)^{d+2}/T}) \rightarrow 0$; for the Matern case it is $O^*(\sqrt{T^{a-1} \ln T})$, which vanishes precisely when $a < 1$. The kernel's smoothness must be enough to keep the information-gain exponent below one.

43 Modern Generalization Theory

The two textbooks this book synthesizes were finished around 2004, and their account of generalization rests on a single intuition: a model with more capacity than it needs will fit the noise and generalize badly, so the art is to stop short of interpolation. Deep learning broke that intuition in plain sight, and kernels turned out to be the cleanest setting in which to understand why. This chapter derives the three results that rebuilt the theory around interpolation rather than against it: the double descent risk curve and its peak at the interpolation threshold, the benign overfitting condition under which a minimum-norm interpolant of noisy data still generalizes, and the spectral learning curves that fix the power-law rate at which a kernel method learns. The common thread is the kernel's spectrum: the eigenvalue decay that Chapter 18, Mercer's Theorem, Spectra, and Rates read as a smoothness scale is, in this modern light, also the dial that decides whether interpolation is safe and how fast learning proceeds. Feature learning, the neural tangent kernel, and attention, the other half of the modern story, are the subject of the closing chapter Chapter 43, The Frontier: Feature Learning and Beyond; here we stay with the fixed kernel and ask what it does when it interpolates.

43.1 The interpolation revolution

The most consequential modern result is that a central lesson of Parts II and IV is incomplete. The classical theory says a model that fits the training data exactly, an interpolant, must generalize badly, because it has spent all its capacity memorizing noise; the RKHS-ball bounds of Chapter 11, Learning Theory in RKHS Balls and the bias-variance decomposition of Chapter 18, Mercer's Theorem, Spectra, and Rates both rest on that reading. Modern practice contradicts it. Overparameterized networks, and kernel machines too, routinely drive the training error to zero on noisy labels and still predict well on unseen data. Belkin, Ma, and Mandal (2018) argued the point in their title, that to understand deep learning we need to understand kernel learning, precisely because kernel machines also interpolate noisy data and still generalize, so whatever explains the phenomenon must already be visible in the linear algebra of kernels.

The reframing is that overfitting is a question about *which* interpolant, not *whether* to interpolate. Once a model is expressive enough to fit the data exactly there are typically infinitely many ways to do so, and the learning algorithm silently selects one. Gradient descent from a small initialization, and the normal equations with a pseudoinverse, both select the interpolant of smallest norm. That choice is the whole story, so we begin by writing it down.

43.1.1 The minimum-norm interpolant

Fix a feature map $\varphi : \mathcal{X} \rightarrow \mathbb{R}^D$, which may be an explicit random-feature map or the (possibly infinite-dimensional) map of a kernel, and fit a linear predictor $f(x) = \varphi(x)^\top \beta$ to data $(x_i, y_i)_{i=1}^n$. Stack the features into the design matrix $\Phi \in \mathbb{R}^{n \times D}$ with rows $\varphi(x_i)^\top$. When $D < n$ the system $\Phi\beta = y$ is overdetermined and least squares returns a unique β ; when $D > n$ it is underdetermined and there is an affine space of exact interpolants. The learning algorithm picks the smallest one.

Definition (minimum-norm interpolant)

Given features $\Phi \in \mathbb{R}^{n \times D}$ and targets $y \in \mathbb{R}^n$, the *minimum-norm least-squares* solution is

$$\hat{\beta} = \arg \min \{ \|\beta\|_2 : \beta \text{ minimizes } \|\Phi\beta - y\|_2^2 \} = \Phi^+ y,$$

where Φ^+ is the Moore-Penrose pseudoinverse. When $D > n$ and Φ has full row rank this interpolates, $\Phi\hat{\beta} = y$, and equals $\hat{\beta} = \Phi^\top(\Phi\Phi^\top)^{-1}y$; when $D < n$ and Φ has full column rank it is the ordinary least-squares estimator $\hat{\beta} = (\Phi^\top\Phi)^{-1}\Phi^\top y$.

The overparameterized form $\hat{\beta} = \Phi^\top(\Phi\Phi^\top)^{-1}y$ is exactly kernel ridge regression at ridge zero, with Gram matrix $K = \Phi\Phi^\top$: the predictor is $f(x) = k(x)^\top K^{-1}y$, the ridgeless kernel interpolant of Chapter 4, Kernel Ridge Regression and Smooth Losses. So the question “does the minimum-norm interpolant generalize” is a question about kernels, and its answer is written in the spectrum of K . Liang and Rakhlin (2020) showed that kernel ridgeless regression can indeed generalize when the eigenvalues decay at the right rate, and the two theorems that follow make the mechanism precise.

43.2 Double descent

Plot test error against model size and the classical picture predicts a U: error falls as the model grows expressive enough to capture the signal, then rises as it grows expressive enough to fit the noise, with a sweet spot in between. Belkin, Hsu, Ma, and Mandal (2019) showed that the real curve does not stop at the U. Past the point where the model has just enough parameters to interpolate, the error descends a *second* time, so the full curve descends, rises to a peak, and descends again. The peak sits exactly at the **interpolation threshold** $D = n$, where the number of parameters first equals the number of samples.

The mechanism is visible in the minimum-norm interpolant. As $D \rightarrow n$ from either side the design matrix Φ becomes square and generically ill-conditioned: its smallest singular value approaches zero, so the interpolating coefficient vector $\hat{\beta} = \Phi^+ y$ has enormous norm and the predictor swings wildly between the data points. This is the peak. For $D > n$ there are many interpolants and the minimum-norm one need not be wild, because the extra directions give it room to stay small; as D grows further the solution smooths out and the error falls again. The cleanest place to see all of this with exact formulas is the isotropic linear model, where the risk can be computed in closed form.

43.2.1 The risk of ridgeless least squares

Theorem (double-descent risk of minimum-norm least squares; Belkin et al. 2019, Hastie et al. 2019, Mei and Montanari 2022)

Let $x \sim \mathcal{N}(0, I_p)$, $y = x^\top \beta + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ independent and $\|\beta\|^2 = r^2$. From n samples form the minimum-norm least-squares estimator $\hat{\beta} = X^+ y$, and measure the excess prediction risk $R(\hat{\beta}) = \mathbb{E}_x[(x^\top \hat{\beta} - x^\top \beta)^2] = \|\hat{\beta} - \beta\|^2$. Then its expectation over the training noise and design is

$$\mathbb{E} R(\hat{\beta}) = \begin{cases} \frac{\sigma^2 p}{n - p - 1}, & p < n - 1 \quad (\text{underparameterized}), \\ r^2 \frac{p - n}{p} + \frac{\sigma^2 n}{p - n - 1}, & p > n + 1 \quad (\text{overparameterized}). \end{cases}$$

Both branches diverge as $p \rightarrow n$, producing the double-descent peak at the interpolation threshold. As $n, p \rightarrow \infty$ with $p/n \rightarrow \gamma$ they converge to the Marchenko-Pastur limits $\sigma^2 \gamma / (1 - \gamma)$ for $\gamma < 1$ and $r^2(1 - 1/\gamma) + \sigma^2 / (\gamma - 1)$ for $\gamma > 1$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The underparameterized branch is pure variance: the estimator is unbiased, and every added feature is one more direction of noise to fit, so the variance climbs and blows up as $p \rightarrow n$. The overparameterized branch has two terms with opposite trends. The bias $r^2(p - n)/p$ is the signal energy lost to the $p - n$ directions the interpolant

cannot see; it shrinks as p grows, since a richer feature space captures more of β . The variance $\sigma^2 n/(p - n - 1)$ also shrinks as p grows, because the fixed budget of noise energy is spread over more directions. Their sum descends from the peak, reaches a minimum in the overparameterized regime, and rises slowly back toward r^2 as $p \rightarrow \infty$ and the interpolant loses ever more signal. The proof of both branches is one application of the inverse-Wishart mean.

Proof

Underparameterized case $p < n$. Here X has full column rank almost surely, so $\hat{\beta} = (X^\top X)^{-1} X^\top y$ and, writing $y = X\beta + \varepsilon$, $\hat{\beta} - \beta = (X^\top X)^{-1} X^\top \varepsilon$. The estimator is unbiased, and its risk is

$$\mathbb{E} \|\hat{\beta} - \beta\|^2 = \mathbb{E} [\varepsilon^\top X (X^\top X)^{-2} X^\top \varepsilon] = \sigma^2 \mathbb{E} \text{tr}((X^\top X)^{-2} X^\top X) = \sigma^2 \mathbb{E} \text{tr}((X^\top X)^{-1}),$$

taking the expectation over ε first. Now $X^\top X$ is a $p \times p$ Wishart matrix with n degrees of freedom and identity scale, whose inverse has the known mean $\mathbb{E}[(X^\top X)^{-1}] = I_p/(n - p - 1)$ for $n - p - 1 > 0$. Hence $\mathbb{E} \text{tr}((X^\top X)^{-1}) = p/(n - p - 1)$, giving the first branch.

Overparameterized case $p > n$. Here X has full row rank almost surely, $\hat{\beta} = X^\top (XX^\top)^{-1} y$, and with $\Pi = X^\top (XX^\top)^{-1} X$ the orthogonal projection onto the n -dimensional row space,

$$\hat{\beta} - \beta = (\Pi - I)\beta + X^\top (XX^\top)^{-1} \varepsilon.$$

The two terms are uncorrelated because ε is independent with mean zero, so the risk splits into bias and variance. For the bias, $\|(\Pi - I)\beta\|^2 = \beta^\top (I - \Pi)\beta$ since $I - \Pi$ is idempotent; by rotational invariance of the isotropic Gaussian design the row space is a uniformly random n -dimensional subspace, so $\mathbb{E}[I - \Pi] = \frac{p-n}{p}I$ and the bias equals $r^2(p - n)/p$. For the variance,

$$\mathbb{E} [\varepsilon^\top (XX^\top)^{-1} XX^\top (XX^\top)^{-1} \varepsilon] = \sigma^2 \mathbb{E} \text{tr}((XX^\top)^{-1}).$$

Now $XX^\top$ is an $n \times n$ Wishart matrix with p degrees of freedom, so $\mathbb{E}[(XX^\top)^{-1}] = I_n/(p - n - 1)$ and the trace is $n/(p - n - 1)$. Adding the two terms gives the second branch. $\square$

43.2.2 Why regularization flattens the peak

The divergence at $p = n$ is a property of the *ridgeless* minimum-norm solution, not of overparameterization itself. Replace the pseudoinverse by ridge regression, $\hat{\beta}_\lambda = (X^\top X + n\lambda I)^{-1} X^\top y$, and the near-zero singular values that blow up the interpolant are lifted away from zero by $n\lambda$. The peak is exactly the place where a good ridge helps most, and a well-chosen λ removes it.

Optimal ridge monotonizes the risk

Mei and Montanari (2022) computed the risk of ridge regression in the same proportional limit $p/n \rightarrow \gamma$ and showed that at the *optimal* ridge $\lambda^*(\gamma)$ the risk is a monotone function of the sample size: the double-descent peak disappears entirely, and the interpolation threshold loses its special status. Double descent is therefore the signature of under-regularization. It appears when one interpolates on purpose or lets an implicit bias interpolate for you, and it is invisible to a practitioner who tunes the ridge by cross-validation. The modern lesson is not that interpolation is good, but that interpolation is safe under conditions the next section makes precise, and that regularization interpolates between the two regimes.

Example (double descent in the isotropic model)

Isotropic model with $n = 40$, noise $\sigma^2 = 0.25$, signal $\|\beta\|^2 = r^2 = 1$. The minimum-norm least-squares risk is averaged over 200 deterministic seeds and compared to the exact formulas of the Theorem. Feature counts p sweep across the threshold $p = n = 40$.

1. Climb toward the peak. Underparameterized, the exact risk $\sigma^2 p/(n - p - 1)$ rises from 0.263 at $p = 20$ ($\gamma = 0.5$) to 3.00 at $p = 36$ ($\gamma = 0.9$); simulation gives 0.276 and 2.81.
2. Cross the threshold. Just past interpolation, $p = 44$ ($\gamma = 1.1$) has exact risk 3.42 and simulated 3.12; the risk is largest right at $p = n$, where both formulas diverge.
3. Descend a second time. Overparameterized, $r^2(p - n)/p + \sigma^2 n/(p - n - 1)$ falls to 0.756 at $p = 80$ ($\gamma = 2$), below the best underparameterized value; simulation gives 0.760.
4. Approach the ceiling. Far overparameterized, $p = 400$ ($\gamma = 10$) has risk 0.928 (both exact and simulated), climbing back toward $r^2 = 1$ as the interpolant sheds signal.

Reading. The curve is not a U. It descends, spikes at $p = n$, and descends again into the overparameterized regime, where the best test error (0.76 at $\gamma = 2$) beats every underparameterized model. Every simulated number matches the closed form to sampling error, and the spike is exactly the two Wishart denominators $n - p - 1$ and $p - n - 1$ passing through zero.

Verification artifact. checks/example-ch-modern-example-42-1.json records the example source hash and verification scope.

43.3 Benign overfitting

The isotropic model of the last section overfits harmfully: at any fixed $p > n$ the risk stays bounded away from zero, because isotropic features give the noise nowhere to hide. Real kernels are not isotropic. Their eigenvalues decay, so the feature space has a few high-variance directions carrying the signal and a long tail of low-variance directions. Bartlett, Long, Lugosi, and Tsigler (2020) showed that this tail is exactly what makes interpolation safe: it absorbs the label noise without distorting the signal, so the minimum-norm interpolant of noisy data can still generalize. Whether it does is decided by two notions of how many effective directions the covariance spectrum contains.

Definition (effective ranks)

Let Σ have eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots > 0$. For $k \geq 0$ define the two effective ranks of the tail beyond index k ,

$$r_k(\Sigma) = \frac{\sum_{i>k} \lambda_i}{\lambda_{k+1}}, \quad R_k(\Sigma) = \frac{(\sum_{i>k} \lambda_i)^2}{\sum_{i>k} \lambda_i^2}.$$

The first, r_k , is a count in units of the largest tail eigenvalue; the second, R_k , is the participation ratio, the number of tail directions of comparable size (largest when the tail is flat, since $R_k = m$ for a flat tail of m equal eigenvalues). The *critical index* is $k^\star = \min\{k \geq 0 : r_k(\Sigma) \geq bn\}$ for a fixed constant b .

The index $k^\star$ splits the spectrum into a head of at most $k^\star$ strong directions, where the signal lives and is estimated as in classical low-dimensional regression, and a tail whose job is to soak up noise. The theorem attaches a precise cost to each part.

Theorem (effective-rank characterization; Bartlett, Long, Lugosi, and Tsigler 2020)

Consider the well-specified linear model $y = x^\top \beta^\star + \varepsilon$, with independent noise of variance σ^2 , and let $\hat{\beta}$ be the minimum-Euclidean-norm interpolant of n independent samples. Under the paper's covariance regularity and sub-Gaussian design assumptions, there are universal constants $b, c > 0$ for which high-probability upper and lower bounds decompose the excess prediction risk into a signal-dependent bias term and a noise-dependent variance term of order

$$\text{Var} \asymp \sigma^2 \left(\frac{k^\star}{n} + \frac{n}{R_{k^\star}(\Sigma)} \right).$$

For a sequence of problems in which the paper’s bias condition also vanishes, its effective-rank conditions characterize when the variance vanishes:

$$\frac{r_0(\Sigma)}{n} \rightarrow \infty, \quad \frac{k^\star}{n} \rightarrow 0, \quad \frac{n}{R_{k^\star}(\Sigma)} \rightarrow 0.$$

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Scope of the characterization

The displayed conditions do not by themselves prove benign overfitting for an arbitrary nonlinear, misspecified, dependent, or heavy-tailed problem. They characterize the effective-rank mechanism within the stated linear model and must be combined with control of the signal-dependent bias. Later extensions change the assumptions and constants; this chapter uses the theorem only for the covariance mechanism it establishes.

Read the variance as a competition. The first piece, $\sigma^2 k^\star/n$, is the ordinary cost of fitting noise in the $k^\star$ strong directions with n samples: it is small only when the number of high-variance directions is small compared to the sample size, exactly the classical demand. The second piece, $\sigma^2 n/R_{k^\star}$, is the price of interpolating the noise using the tail. The interpolant must fit every label exactly, and it does so by spending the tail directions; because there are effectively $R_{k^\star}$ of them, each carries only a $1/R_{k^\star}$ share of the noise energy, and the total noise leaked back into the prediction is of order $n/R_{k^\star}$. When the tail is high-dimensional, $R_{k^\star} \gg n$, that leak is negligible: the noise is diluted across so many harmless directions that its footprint on any test point vanishes. The condition $r_0(\Sigma)/n \rightarrow \infty$ ensures the tail also has enough total variance to interpolate at all without inflating the strong directions, which would bias the signal. We can turn the whole mechanism into numbers.

Example (benign versus harmful interpolation)

Spiked covariance: $s = 5$ signal eigenvalues equal to 1, then a flat junk tail of m eigenvalues at level τ ; signal $\beta^\star$ lies in the spike with $\|\beta^\star\|_\Sigma^2 = 1$, noise $\sigma^2 = 0.25$, $n = 100$, constant $b = 1$. The min-norm interpolant’s excess risk is averaged over 120 seeds. Two tails are compared.

1. Build a wide, low-energy tail. Benign case: $m = 20,000$, $\tau = 0.001$, so the tail variance $\sum_{i>k^\star} \lambda_i = 20$ is well below n while its dimension is huge. The critical index is $k^\star = 5$, and $R_{k^\star} = 20,000$.
2. Read off the variance budget. $k^\star/n = 0.05$ and $n/R_{k^\star} = 0.005$, so the BLLT variance term is $\sigma^2(0.05 + 0.005) = 0.0138$. The full simulated excess risk is 0.040, against a null-predictor risk of 1.0: the interpolant fits all 100 noisy labels and still predicts well.
3. Narrow the tail. Harmful case: $m = 150$, $\tau = 0.5$, so the tail dimension is comparable to n . Now $R_{k^\star} = 150$, so $n/R_{k^\star} = 0.667$ and the variance term jumps to $\sigma^2(0.05 + 0.667) = 0.179$.
4. Watch overfitting turn harmful. The simulated excess risk is 0.608, most of the way to the null risk of 1.0: with only $\sim n$ tail directions, the noise cannot be diluted and floods the prediction.

Reading. The same interpolation rule, on the same signal, is harmless with a wide tail (risk 0.04) and ruinous with a narrow one (risk 0.61). The single quantity that flips is the tail’s effective rank $R_{k^\star}$: 20,000 versus 150. A high-dimensional spectral tail is not a nuisance to be regularized away; it is the mechanism that makes interpolation safe.

Verification artifact. `checks/example-ch-modern-example-42-2.json` records the example source hash and verification scope.

The connection to double descent is now exact. The isotropic model interpolates harmfully because $\Sigma = I$ has no tail: every direction is a strong direction, $k^\star$ grows with p , and there is no low-variance subspace to absorb the noise. A decaying kernel spectrum is the opposite, and Liang and Rakhlin’s (2020) result that kernel ridgeless regression can generalize is the infinite-dimensional face of the same theorem, with the effective ranks read off the Mercer eigenvalues of Chapter 18, Mercer’s Theorem, Spectra, and Rates.

43.4 Spectral learning curves and scaling laws

The spectrum that decides whether interpolation is safe also fixes the rate at which a kernel method learns. For kernel ridge regression the generalization error typically falls as a power law in the number of samples, $\varepsilon(n) \sim n^{-\beta}$, and the exponent β is set by two spectral quantities: how fast the kernel's eigenvalues decay, and how fast the target's coefficients in the kernel eigenbasis decay. A target aligned with the top eigenfunctions is learned quickly; a rough target that excites the tail is learned slowly. The learning curve is a joint statement about the kernel and the function it is asked to fit, and the tool that makes it exact is a per-mode accounting of bias and variance.

Set up the eigenbasis. By Mercer's theorem the kernel integral operator on $L^2(p)$ has orthonormal eigenfunctions ϕ_i with eigenvalues $\eta_1 \geq \eta_2 \geq \dots$, and any target expands as $f^\star = \sum_i \bar{a}_i \phi_i$. The decay of the coefficients $\bar{a}_i$ is the **source condition** of Caponnetto and De Vito (2007): writing $\bar{a}_i^2 \sim i^{-b}$ and $\eta_i \sim i^{-a}$, the pair (a, b) is what the rate depends on. Kernel ridge regression at ridge λ with n points then has a remarkably clean predicted error, derived by the replica method of statistical physics.

Replica prediction (Sollich 1999; Bordelon et al. 2020; Canatar et al. 2021)

Let $\kappa > 0$ be the unique solution of the self-consistent equation

$$\kappa = \lambda + \sum_i \frac{\eta_i \kappa}{n\eta_i + \kappa},$$

and define the per-mode *learnability* and the overfitting factor

$$\mathcal{L}_i = \frac{n\eta_i}{n\eta_i + \kappa} \in (0, 1), \quad \gamma = \frac{1}{n} \sum_i \mathcal{L}_i^2 \in (0, 1).$$

In the typical-case random-design setting analyzed by the cited work, the replica calculation predicts the dataset-averaged generalization error

$$E_g = \frac{1}{1-\gamma} \left[\sum_i (1-\mathcal{L}_i)^2 \bar{a}_i^2 + \sigma^2 \gamma \right], \quad 1-\mathcal{L}_i = \frac{\kappa}{n\eta_i + \kappa}.$$

Mode i is learned once $n\eta_i$ exceeds the effective ridge κ (then $\mathcal{L}_i \rightarrow 1$ and it drops out of the error) and unlearned while $n\eta_i \ll \kappa$ (then $\mathcal{L}_i \rightarrow 0$ and it contributes its full energy $\bar{a}_i^2$).

The self-consistent κ is an *effective ridge*: even at $\lambda = 0$, having only n samples acts like a positive regularizer, because the finite design cannot resolve modes below a resolution set by the spectrum. The equation is what fixes that resolution. To see the structure, derive the per-mode error from the ridge bias-variance decomposition.

Heuristic derivation (per-mode bias and variance)

Work in the eigenbasis, where the population problem is diagonal: mode i has feature variance η_i and true coefficient $\bar{a}_i$. Kernel ridge regression is ridge regression on these features, and with n samples the effective penalty on mode i is not λ but the self-consistent κ , which aggregates the interference from all other modes competing for the same n data points. The ridge estimate of a coefficient with signal-to-penalty balance $n\eta_i$ versus κ shrinks it by the factor $\mathcal{L}_i = n\eta_i/(n\eta_i + \kappa)$, leaving the residual coefficient $(1-\mathcal{L}_i)\bar{a}_i = \kappa \bar{a}_i/(n\eta_i + \kappa)$. Squaring and weighting by the mode's contribution to the L^2 error gives the per-mode bias $(1-\mathcal{L}_i)^2 \bar{a}_i^2$. Summing over modes and adding the noise, which the estimator fits with the same shrinkage and which contributes $\sigma^2 \gamma$, gives the bracket. The prefactor $1/(1-\gamma)$ is the variance amplification: $\gamma = \frac{1}{n} \sum_i \mathcal{L}_i^2$ is the fraction of the n degrees of freedom already spent on the learned modes, and as it approaches 1 the estimator runs out of data to fit anything new and the error diverges, the learning-curve echo of the interpolation peak. That κ makes this accounting self-consistent is the replica prediction; the argument explains the formula but is not a rigorous finite-sample proof.

The power law follows from the threshold. The learned modes are those with $n\eta_i \gtrsim \kappa$, i.e. $i \lesssim i^\star(n)$ where $\eta_{i^\star} \approx \kappa/n$. Feeding $\eta_i \sim i^{-a}$ into the self-consistent equation gives $i^\star$ growing proportionally to n , so the error is dominated by the still-unlearned tail,

$$E_g \approx \sum_{i>i^\star} \bar{a}_i^2 \sim \sum_{i>i^\star} i^{-b} \sim (i^\star)^{-(b-1)} \sim n^{-(b-1)},$$

a power law whose exponent is set jointly by the eigenvalue decay a (through how fast $i^\star$ advances) and the source decay b (through how much target energy the tail still holds). Spigler, Geiger, and Wyart (2020) and Cui et al. (2021) work out the exponent in each regime, including the crossover to the noise-limited rate once $\sigma^2 > 0$. The prediction is not merely asymptotic folklore; it tracks real kernel ridge regression mode by mode.

Example (learning curve of kernel ridge regression)

A discrete domain of $P = 1200$ points with the uniform measure carries a kernel with eigenvalues $\eta_i = i^{-1.5}$ and orthonormal eigenfunctions; the noiseless target has coefficients $\bar{a}_i^2 = i^{-2}$. Kernel ridge regression at $\lambda = 10^{-4}$ is run on n points drawn uniformly, averaged over 150 draws, and its test error is compared to the omniscient prediction.

1. Solve for the effective ridge. The self-consistent κ falls from 1.02 at $n = 10$ to 0.115 at $n = 320$: more data resolves finer modes, lowering the effective penalty.
2. Read the overfitting factor. γ rises from 0.32 to 0.49 as the sample budget fills, staying safely below 1, so the amplification $1/(1 - \gamma)$ stays modest.
3. Compare theory to simulation. The omniscient E_g reads 0.256, 0.133, 0.0665, 0.0323, 0.0150, 0.00643 at $n = 10, 20, 40, 80, 160, 320$; the measured KRR error reads 0.259, 0.137, 0.0700, 0.0339, 0.0166, 0.00815. Theory and experiment agree to a few percent at every sample size.
4. Fit the power law. The large- n points give a slope $\beta \approx 1.17$ on log-log axes, near the source-limited value $b - 1 = 1$ predicted by the tail estimate.

Reading. The generalization error is not a single opaque number but a sum over modes, each switched on when the sample size crosses its eigenvalue. The replica formula reproduces the whole KRR learning curve from the spectrum and the target alone, and its power-law tail is the kernel version of a neural scaling law.

Verification artifact. checks/example-ch-modern-example-42-3.json records the example source hash and verification scope.

These curves are the theoretical backbone of the empirical scaling laws seen in large models, and they close the loop with double descent. The near-singular direction that produces the interpolation peak is exactly the mode whose eigenvalue the sample size has just reached, where $n\eta_i \approx \kappa$ and $\mathcal{L}_i \approx \frac{1}{2}$: the peak and the power-law tail are two views of one spectral accounting. The exponent depends on the target, so the same kernel learns a smooth function fast and a rough one slowly, which is why the “right” kernel for a task is the one whose eigenfunctions align with the target, the alignment that Canatar, Bordelon, and Pehlevan (2021) quantify.

43.5 PAC-Bayes and data-dependent certificates

Uniform convergence asks for one event on which every hypothesis in a class behaves well. PAC-Bayes takes a different route. It places a prior distribution P over predictors before observing the sample, allows learning to return a posterior distribution Q , and charges the learner for moving from P to Q through $\text{KL}(Q\|P)$. The resulting certificate concerns the randomized predictor obtained by drawing $f \sim Q$. For a bounded loss, a representative bound has the shape

$$R(Q) \lesssim \hat{R}(Q) + \sqrt{\frac{\text{KL}(Q\|P) + \ln(1/\delta)}{n}},$$

with constants and lower-order terms depending on the chosen PAC-Bayes inequality. This is not a license to choose the prior after looking at all labels. A data-dependent prior needs either an independent data split, a hierarchical argument, or a theorem that explicitly pays for the dependence.

For a kernel machine, take Q to be a Gaussian perturbation around the learned coefficient vector or RKHS function. A flat minimum tolerates a broader perturbation without increasing empirical loss, while a sharp minimum does not. The KL term measures how far the learned center and covariance moved from the prior. This turns the informal claim that flat solutions generalize into an auditable optimization: choose the posterior scale that minimizes the sum of perturbed empirical risk and information cost. Spectrally normalized neural bounds use the same logic in parameter space (Neyshabur and Srebro 2018), while an RKHS version exposes the kernel eigenbasis directly.

Algorithm (a PAC-Bayes reporting protocol)

1. Declare the prior, loss range, confidence level, and the exact theorem before using validation or test labels.
2. Fit the predictor and define a posterior family around it.
3. estimate the empirical Gibbs risk using fresh perturbation draws with a fixed random seed.
4. Compute the KL term analytically where possible and optimize only parameters permitted by the theorem.
5. Report the certificate beside the ordinary test estimate, including Monte Carlo error and every data-dependent choice.

43.6 Compression and algorithm-dependent complexity

A second escape from worst-case class capacity is to describe the output of the learning algorithm rather than the entire class it could have returned. If a classifier can be reconstructed from a small subset of s training examples plus a short side message, sample-compression bounds scale with the information in that description rather than the nominal feature dimension. Sparse support-vector expansions make the connection immediate: the support vectors are a compression set, although their number can itself approach n and must not be treated as small without measurement.

Stability gives a complementary view. Replace one training example and ask how much the learned predictor or loss changes. Strong regularization and a well-conditioned Gram system make kernel ridge regression stable; ridgeless interpolation can be unstable near small empirical eigenvalues even when its average risk is benign. Compression, stability, PAC-Bayes, and spectral effective dimension therefore answer different questions:

View	Data-dependent object	Main failure mode
PAC-Bayes	posterior-to-prior information	invalid data-dependent prior
Compression	reconstruction description length	a large or unstable compression set
Stability	sensitivity to replacing one example	ill-conditioning or weak curvature
Spectral analysis	covariance and kernel eigenvalues	mismatched source or tail assumptions

Agreement among these views is useful evidence. Disagreement is diagnostic, not paradoxical: each theorem controls a different property under different assumptions.

43.7 Random-matrix equivalents and finite-sample diagnostics

Modern proportional asymptotics replace random traces of resolvents by deterministic equivalents. In ridge regression the key object is

$$m_n(-\lambda) = \frac{1}{n} \operatorname{tr}(K + n\lambda I)^{-1},$$

and derivatives of this resolvent trace control variance and effective degrees of freedom. When n and feature dimension grow together under a specified random-design model, m_n can converge to the solution of a fixed-point equation. The resulting formulas predict test risk, but they are not distribution-free finite-sample guarantees. Their validity depends on the design ensemble, aspect ratio, spectral convergence, noise model, and whether the target is deterministic or random.

The practical use is a three-way diagnostic. Compare the observed learning curve with the deterministic-equivalent prediction, a nonparametric bootstrap over examples, and held-out risk. If the first fails while the latter two agree, the asymptotic model is misspecified. If the bootstrap is erratic, influential observations or dependence may invalidate iid reasoning. If all three disagree, first audit leakage, preprocessing, and numerical conditioning before telling a new generalization story.

43.8 Limits, lower bounds, and what cannot be universal

No spectrum-only condition can characterize every regression problem. The same covariance eigenvalues can be paired with different target alignments, noise distributions, leverage profiles, or train-test shifts. Benign overfitting conditions must therefore separate signal assumptions from variance conditions, and sufficient conditions must not be restated as necessary ones. Likewise, a typical-case replica prediction is not a worst-case theorem, and a proportional limit can conceal slow convergence at the sample sizes used in practice.

Three negative checks should accompany any modern generalization claim:

1. **Null-target check.** Set the signal to zero and verify that the claimed mechanism still controls fitted noise.
2. **Adversarial-alignment check.** Move target energy into weak eigenfunctions and see which source condition breaks.
3. **Shift check.** Change the test covariate distribution while preserving training risk and document whether the guarantee survives.

These checks make the chapter's main lesson precise: interpolation is neither automatically harmful nor automatically benign. Its behavior is conditional on algorithm, spectrum, signal, noise, and evaluation distribution.

43.9 Where this connects

The three phenomena are one story told from three angles: interpolation is safe or not, learning is fast or slow, according to how the kernel's eigenvalues decay and how the target's energy is spread across the eigenbasis. That spectrum is the object Chapter 18, Mercer's Theorem, Spectra, and Rates built and Chapter 11, Learning Theory in RKHS Balls first used for bounds, now doing double duty as the arbiter of overfitting. The algorithms that make interpolation with a fixed kernel practical at scale are the Nystrom and random-feature methods of Chapter 37, Large-Scale Kernel Machines, and the Bayesian twin of every learning curve here is a Gaussian-process posterior, the subject of Chapter 40, Gaussian Processes and the RVM. What this chapter deliberately leaves out is the case where the kernel itself is learned: when features move, a network can beat its own tangent kernel, and the separation between the fixed-kernel world and feature learning is the closing story of Chapter 43, The Frontier: Feature Learning and Beyond.

43.10 Common mistakes and practical implications

For **Modern Generalization Theory**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel

baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

43.11 Summary and further reading

This chapter established explain the central definitions and claims in Modern Generalization Theory; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Belkin and Mandal 2019), (Belkin and Mandal 2018), (Hastie and Tibshirani 2019).

43.12 Exercises

1. warm-up In the isotropic double-descent theorem, take $\sigma^2 = 0$ (no label noise). Show the underparameterized risk is exactly zero and the overparameterized risk is pure bias $r^2(p - n)/p$. Explain why the peak at $p = n$ disappears, and what this says about the source of the double-descent spike.
2. computation Verify the underparameterized branch numerically-by-hand for $n = 10$, $p = 4$, $\sigma^2 = 1$: compute $\sigma^2 p/(n - p - 1)$ and compare with the asymptotic $\sigma^2 \gamma/(1 - \gamma)$ at $\gamma = 0.4$. Which is larger, and why does the finite-sample formula exceed the limit? Hint: compare $p/(n - p - 1)$ with $\gamma/(1 - \gamma) = p/(n - p)$; the extra -1 in the denominator is the finite-sample correction.
3. computation Show directly that ridge regression removes the divergence at $p = n$. For the estimator $\hat{\beta}_\lambda = (X^\top X + n\lambda I)^{-1} X^\top y$, argue that the smallest eigenvalue of $X^\top X + n\lambda I$ is at least $n\lambda$, so no factor $1/(n - p)$ can appear, and the variance stays finite through $p = n$. Hint: bound $\text{tr}((X^\top X + n\lambda I)^{-2} X^\top X)$ using the eigenvalues s_j of $X^\top X$ and the inequality $s_j/(s_j + n\lambda)^2 \leq 1/(4n\lambda)$.
4. computation For the benign-overfitting effective ranks, take the spiked spectrum $\lambda_1 = \dots = \lambda_s = 1$ and $\lambda_{s+1} = \dots = \lambda_{s+m} = \tau$. Compute r_k and R_k for $s \leq k < s + m$, and confirm that at the spike cutoff $R_s = m$ and $r_s = m\tau/\tau = m$. Deduce the condition on m and τ for benign overfitting with n samples. Hint: a flat tail of m equal eigenvalues has participation ratio exactly m ; benign needs $m \gg n$ and tail variance $m\tau$ neither vanishing nor swamping the signal.
5. challenge Prove the isotropic model is never benign. With $\Sigma = I_p$ and $p > n$, show $k^\star$ is forced to be of order p and $R_{k^\star}$ of order $p - k^\star$, so the variance term $k^\star/n + n/R_{k^\star}$ cannot tend to zero as p grows at fixed n . Relate this to the overparameterized branch $r^2(p - n)/p + \sigma^2 n/(p - n - 1)$ staying bounded away from zero. Hint: for a flat spectrum $r_k(\Sigma) = p - k$, so $r_k \geq bn$ fails only once k is within bn of p .
6. challenge In the omniscient learning curve, take the ridgeless limit $\lambda \rightarrow 0$ with a finite number M of nonzero modes and $n > M$. Show the self-consistent equation forces $\kappa \rightarrow 0$, hence $\mathcal{L}_i \rightarrow 1$ for every mode, $\gamma \rightarrow M/n$, and $E_g \rightarrow \sigma^2 \gamma/(1 - \gamma) = \sigma^2 M/(n - M)$. Interpret this as ordinary least squares with M features. Hint: divide the self-consistent equation by κ and let $\kappa \rightarrow 0$; the equation $1 = \sum_i \eta_i/(n\eta_i + \kappa)$ has no positive root when $n > M$.
7. challenge Show that when $n < M$ the ridgeless self-consistent equation $1 = \sum_{i=1}^M \eta_i/(n\eta_i + \kappa)$ has a unique positive root $\kappa > 0$, so the effective ridge is nonzero even at $\lambda = 0$. Explain why this is the learning-curve statement of the same implicit regularization that keeps the minimum-norm interpolant well-behaved. Hint: the right side is continuous, strictly decreasing in κ , equals $M/n > 1$ at $\kappa = 0^+$ when $M > n$, and tends to 0 as $\kappa \rightarrow \infty$, so it crosses 1 exactly once.
8. synthesis Explain in one paragraph, using the effective ranks and the per-mode learnability, why a kernel with a slowly decaying spectrum (large tail effective rank) both interpolates benignly and learns with a shallow power-law exponent, so that safety at interpolation and slowness of learning are two consequences of the same heavy tail. What does this trade-off imply for choosing a kernel's bandwidth?

44 The Frontier: Feature Learning and Beyond

*Every kernel in this book fixes its geometry before it sees a single label. The feature map Φ is chosen, the RKHS is determined, and learning happens inside a space whose shape is frozen. The defining move of modern deep learning is to refuse that separation and let the representation itself be learned. This closing chapter is about the seam between the two worlds. We show precisely when a neural network is secretly a fixed kernel machine (the lazy, neural tangent kernel regime), when it is not (the rich, feature learning regime), and what mathematics decides which. We then ask what survives of the kernel view once features move: attention turns out to be a kernel smoother, deep and convolutional kernels grow their own hierarchy, and kernels on manifolds and graphs carry the geometric torch. The companion survey *Modern Generalization Theory* traces the interpolation and double-descent story; here we derive the feature-learning boundary and state, honestly, where the fixed kernel cannot follow.*

44.1 Two limits of one wide network

Take a neural network and make it very wide. Two things can happen as the width grows, and they are genuinely different limits of the same object. In one, the network trains as if its internal features were frozen at initialization and only the last linear read-out effectively adapts: it behaves exactly like a fixed kernel machine. In the other, the hidden units migrate during training and the representation is learned. The whole frontier lives in the gap between these two limits, so we develop them carefully.

44.1.1 The lazy regime and the neural tangent kernel

Write a network as a function $f(x; \theta)$ of its parameters $\theta \in \mathbb{R}^p$. Gradient descent barely moves each individual weight when there are millions of them, so a first-order Taylor expansion about the initialization θ_0 is the natural thing to try:

$$f(x; \theta) \approx f(x; \theta_0) + \nabla_{\theta} f(x; \theta_0)^{\top} (\theta - \theta_0).$$

This is an affine function of θ , and it is a linear model in the fixed feature map $x \mapsto \nabla_{\theta} f(x; \theta_0)$. The inner product of two such feature vectors is a kernel.

Definition (neural tangent kernel)

For a parametric model $f(\cdot; \theta)$, the *empirical neural tangent kernel* at parameters θ is

$$\Theta_{\theta}(x, x') = \nabla_{\theta} f(x; \theta)^{\top} \nabla_{\theta} f(x'; \theta) = \sum_{r=1}^p \frac{\partial f(x; \theta)}{\partial \theta_r} \frac{\partial f(x'; \theta)}{\partial \theta_r}.$$

It is a positive definite kernel for every θ , being a Gram matrix of Jacobian rows.

The reason this kernel governs training, and not merely the linear approximation, is a differential identity. Under gradient flow $\dot{\theta} = -\nabla_{\theta} L$ on the squared loss $L(\theta) = \frac{1}{2} \sum_i (f(x_i; \theta) - y_i)^2$, the network output at any point moves according to the empirical NTK.

Proposition (output dynamics under gradient flow)

Let $u_i(t) = f(x_i; \theta_t)$ be the outputs on the training set and Θ_t the empirical NTK at time t . Then for any point x ,

$$\frac{d}{dt}f(x; \theta_t) = - \sum_i (u_i(t) - y_i) \Theta_t(x, x_i), \quad \dot{u} = -\Theta_t(u - y).$$

If $\Theta_t \equiv \Theta_0$ stays constant, the training outputs obey a linear ODE with solution $u(t) - y = e^{-\Theta_0 t}(u(0) - y)$, and the learned function converges to

$$f_\infty(x) = f(x; \theta_0) + \Theta_0(x, X) \Theta_0(X, X)^{-1}(y - u(0)),$$

the kernel (ridgeless) regression predictor with kernel Θ_0 on the residual targets.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

By the chain rule $\dot{f}(x) = \nabla_\theta f(x)^\top \dot{\theta} = -\nabla_\theta f(x)^\top \nabla_\theta L$. The loss gradient is $\nabla_\theta L = \sum_i (u_i - y_i) \nabla_\theta f(x_i)$, so $\dot{f}(x) = -\sum_i (u_i - y_i) \nabla_\theta f(x)^\top \nabla_\theta f(x_i) = -\sum_i (u_i - y_i) \Theta_t(x, x_i)$, which is the stated equation and, specialized to $x = x_j$, the vector form $\dot{u} = -\Theta_t(u - y)$. When $\Theta_t \equiv \Theta_0$ this is a constant-coefficient linear system whose solution is $u(t) - y = e^{-\Theta_0 t}(u(0) - y)$; since Θ_0 is positive definite, $u(t) \rightarrow y$. Integrating the equation for a general x and using $\int_0^\infty (u(s) - y) ds = -\Theta_0^{-1}(u(0) - y)$ (from the same exponential) gives $f_\infty(x) = f(x; \theta_0) - \Theta_0(x, X) \int_0^\infty (u(s) - y) ds = f(x; \theta_0) + \Theta_0(x, X) \Theta_0(X, X)^{-1}(y - u(0))$. $\square$

Everything hinges on the hypothesis that Θ_t does not move. That is exactly what infinite width buys. Jacot, Gabriel, and Hongler (2018) proved that for a suitably (NTK-)parametrized fully-connected network, as the hidden widths tend to infinity the empirical NTK at initialization converges in probability to a deterministic limit Θ_∞ that does not depend on the random weights, and moreover stays constant throughout gradient-descent training.

Theorem (NTK constancy, Jacot, Gabriel, and Hongler 2018)

For a fully-connected network under the NTK parametrization, in the limit of infinite hidden widths taken in sequence, the empirical NTK Θ_{θ_0} converges in probability to a deterministic kernel Θ_∞ determined only by the architecture and the nonlinearity, independent of the initialization. Along gradient descent on a fixed training set, $\Theta_{\theta_t} \rightarrow \Theta_\infty$ uniformly in t over any finite horizon. Consequently the trained infinitely-wide network computes the kernel regression predictor of the Proposition with kernel Θ_∞ .

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

This is the completion of the deep-learning bridge opened in Chapter 39, Deep Learning from a Kernel Point of View: in this limit a network is not merely related to a kernel machine, it *is* one, with a specific and computable kernel. The limit kernel obeys its own layerwise recursion, paired with the NNGP covariance of the next section, so the tangent kernel of a deep or convolutional network is as explicit as that of a two-layer net (Arora et al. 2019).

Algorithm (empirical NTK by Jacobian products)

Input network $f(\cdot; \theta)$, parameters θ , points $x_1, \dots, x_n$.

Output empirical NTK Gram $\Theta \in \mathbb{R}^{n \times n}$.

1. For each i , compute the Jacobian row $g_i = \nabla_\theta f(x_i; \theta) \in \mathbb{R}^p$ by backpropagation.
2. Form $\Theta_{ij} = g_i^\top g_j$ (a single matrix product $GG^\top$ with $G = [g_1; \dots; g_n]$).

The next worked example runs this by hand on a network small enough to hold in the head, and checks the Proposition’s central claim: kernel regression with the empirical NTK reproduces, to the digit, the prediction of the linearized network trained to convergence.

Example (tiny NTK equals its linearized network)

Two-hidden-unit ReLU network $f(x; \theta) = \sum_{j=1}^2 a_j \text{relu}(w_j x + b_j)$, scalar input, six parameters. Fix $a = (1, -1)$, $w = (1, 2)$, $b = (0.5, -1)$. Train inputs $X = (-0.2, 0.6, 1.2)$, targets $y = (0.4, -0.2, 1.0)$, test point $x_* = 0.9$.

1. Build the Jacobian rows. With $\partial f / \partial a_j = \text{relu}(w_j x + b_j)$, $\partial f / \partial w_j = a_j x \mathbf{1}[w_j x + b_j > 0]$, $\partial f / \partial b_j = a_j \mathbf{1}[\cdot > 0]$, the three rows of G are $(0.3, 0, -0.2, 0, 1, 0)$, $(1.1, 0.2, 0.6, -0.6, 1, -1)$, $(1.7, 1.4, 1.2, -1.2, 1, -1)$.
2. Form the empirical NTK. $\Theta = GG^\top = \begin{pmatrix} 1.13 & 1.21 & 1.27 \\ 1.21 & 3.97 & 5.59 \\ 1.27 & 5.59 & 9.73 \end{pmatrix}$, and the test-column $\Theta(x_*, X) = (1.24, 4.78, 7.66)$.
3. Read off the initial outputs. $f(X; \theta_0) = (0.3, 0.9, 0.3)$ and $f(x_*; \theta_0) = 0.6$, so the residual targets are $y - u(0) = (0.1, -1.1, 0.7)$.
4. Solve the kernel system. $\alpha = \Theta^{-1}(y - u(0)) = (1.4545, -2.9017, 1.5492)$, giving $f_\infty(x_*) = 0.6 + \Theta(x_*, X)^\top \alpha = 0.4000$.
5. Train the linearized model directly. The minimum-norm least-squares solution of $G\delta = y - u(0)$ is δ with $\|\delta\| = 2.1028$; the linearized prediction $f(x_*; \theta_0) + \nabla f(x_*; \theta_0)^\top \delta = 0.4000$.

Reading. The two routes agree to machine precision: gradient descent on the linearized network and kernel regression with its tangent kernel are the same computation. In the infinite-width limit the linearization stops being an approximation, which is why the wide network is a fixed kernel machine.

Verification artifact. checks/example-ch-frontier-example-43-1.json records the example source hash and verification scope.

44.1.2 Lazy training is about scale, not depth

It is tempting to read the NTK theorem as a fact about neural networks. Chizat, Oyallon, and Bach (2019) showed it is really a fact about *scale*. Consider any differentiable model and multiply its output by a constant α , fitting $x \mapsto \alpha h(x; \theta)$ with the target held fixed. As α grows the parameters need to move only by $O(1/\alpha)$ to change the (amplified) output by $O(1)$, so the model stays within its own linearization for the whole trajectory and the tangent kernel is frozen. They make this quantitative through a dimensionless ratio comparing the curvature of h to the size of its gradient over the loss scale; when that ratio is small, training is *lazy* and the model reduces to kernel regression regardless of whether it is a neural network. Large width is one way to enter this regime (the read-out scaling $1/\sqrt{\text{width}}$ is an implicit α), but so is simply turning up a multiplier. The lesson is sobering: lazy training is a degenerate corner in which the model does no representation learning at all, and reaching state-of-the-art accuracy generally means leaving it.

44.1.3 The parametrization decides the regime

Which limit a wide network falls into is not fixed by the architecture; it is fixed by how the weights and learning rates are scaled with width. Under the *NTK (standard) parametrization*, the natural width-scaling of initialization and step size sends the network into the lazy limit above: features do not move. Under the *mean-field parametrization* of a two-layer network, one instead writes the output as an average $f(x) = \frac{1}{m} \sum_{j=1}^m a_j \sigma(w_j^\top x)$ and lets the empirical distribution of the neurons (a_j, w_j) evolve. Mei, Montanari, and Nguyen (2018) and Chizat and Bach (2018) proved that as $m \rightarrow \infty$ this evolution is a Wasserstein gradient flow on the space of probability measures over neurons: the units genuinely move, and the representation is learned. Yang and Hu (2021) unified the picture with the *maximal-update parametrization* (μP), the unique width-scaling under which every layer’s features update by an $\Theta(1)$ amount in the infinite-width limit, so feature learning persists at any width. The abstract point is that the same network admits a one-parameter family of infinite-width limits, and only the endpoints are the fixed-kernel and the feature-learning worlds; the choice of parametrization selects among them.

44.2 Why a fixed kernel cannot learn features

The separation between the two regimes is not cosmetic. There are natural target functions that a fixed kernel provably learns slowly and that feature learning learns fast, and the reason is exactly that the kernel’s geometry is chosen before the data. This is the mathematical content of “the curse of the fixed representation.”

The clean testbed is a *single-index* target on the high-dimensional sphere: $f^\star(x) = g(\langle w^\star, x \rangle)$ depends on the input only through one unknown direction $w^\star \in \mathbb{R}^d$. A rotation-invariant kernel (the Gaussian kernel, or the NTK of a fully-connected net) has no way to prefer that direction; its eigenfunctions on the sphere are the spherical harmonics graded purely by degree, and a degree- k component of g sits in an eigenspace of dimension $\sim d^k$ with eigenvalue $\sim d^{-k}$. Fitting that component to constant error therefore needs the sample size to reach the eigenvalue’s inverse, which gives the following barrier.

Theorem (kernel sample barrier for single-index targets; Ghorbani, Mei, Misiakiewicz, and Montanari 2020)

Let $f^\star(x) = g(\langle w^\star, x \rangle)$ on the sphere in $\mathbb{R}^d$ whose expansion in Legendre/Hermite components has a nonzero degree- k part. For any rotation-invariant kernel, kernel ridge regression with n samples incurs test error bounded away from zero unless $n \gtrsim d^k$ (up to logarithmic factors). A two-layer network trained in the feature-learning regime recovers the direction $w^\star$ and then fits g with n of order d (up to logarithmic factors), an exponential-in- k sample saving.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

Abbe, Boix-Adsera, and Misiakiewicz (2022, 2023) sharpened the feature-learning side into a combinatorial law. For sparse targets that are polynomials in a few coordinates, gradient descent on a two-layer network learns them in a sequence of *staircase* steps, each new monomial becoming learnable only once the monomials it builds on have been picked up; the total sample complexity is governed by the largest single jump in this staircase (the *leap*), not by the polynomial degree. A pure staircase of increasing monomials is learned with $n \sim d$ samples, exactly the targets on which the fixed kernel pays d^k . The mechanism is that feature learning uses early gradient signal from the low-degree part to align a hidden unit with $w^\star$, after which the higher-degree part is easy; a frozen kernel gets no such alignment.

The function-space companion of this gap was set out by Bach (2017). The functions an infinitely-wide single hidden layer can represent with bounded weights form not an RKHS but the strictly larger *variation-norm* (Barron) space, whose norm is an ℓ^1 -type penalty over neurons rather than the ℓ^2 RKHS norm. That ℓ^1 geometry is what lets the space concentrate on a few important directions and achieve approximation rates independent of d for functions with bounded Barron norm, escaping the curse of dimensionality that the corresponding RKHS suffers. The size of the gap between the Barron space and the RKHS is, quite literally, the value of learning the representation.

44.3 Kernels that grow depth and learn parts

If a fixed shallow kernel is the limitation, one response stays inside the kernel world: give the kernel depth and internal components of its own. Two constructions do this.

44.3.1 The neural-network Gaussian process hierarchy

Before any training, a wide random network already defines a kernel. Neal (1996) observed that a one-hidden-layer network with independent random weights is, in the infinite-width limit, a Gaussian process by the central limit theorem; Lee et al. (2018) and Matthews et al. (2018) extended this to arbitrary depth. The covariance, the *NNGP kernel*, is built by a layerwise recursion in which each layer applies the nonlinearity inside a two-dimensional Gaussian expectation.

Proposition (NNGP covariance recursion)

For a network with pre-activations $h^{(\ell)}(x) = W^{(\ell)}\sigma(h^{(\ell-1)}(x)) + \beta^{(\ell)}$, weights i.i.d. $\mathcal{N}(0, \sigma_w^2/n_{\ell-1})$ and biases $\mathcal{N}(0, \sigma_b^2)$, the infinite-width covariance $K^{(\ell)}(x, x') = \mathbb{E}[h_i^{(\ell)}(x)h_i^{(\ell)}(x')]$ satisfies

$$K^{(\ell)}(x, x') = \sigma_b^2 + \sigma_w^2 \mathbb{E}_{(u,v) \sim \mathcal{N}(0, \Lambda^{(\ell-1)})}[\sigma(u)\sigma(v)], \quad \Lambda^{(\ell-1)} = \begin{pmatrix} K^{(\ell-1)}(x, x) & K^{(\ell-1)}(x, x') \\ K^{(\ell-1)}(x', x) & K^{(\ell-1)}(x', x') \end{pmatrix},$$

starting from the input kernel $K^{(0)}(x, x') = \sigma_b^2 + \sigma_w^2 x^\top x' / n_0$. For $\sigma = \text{relu}$ the expectation is the arc-cosine kernel of Cho and Saul, closed-form in the angle between the two inputs.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** No separate proof is attached to this result; independent technical review must determine whether the surrounding derivation is sufficient.

The NNGP describes the network at initialization; the tangent kernel of the previous section describes it during training, and it obeys the companion recursion $\Theta^{(\ell)} = K^{(\ell)} + \Theta^{(\ell-1)}\dot{K}^{(\ell)}$ with $\dot{K}^{(\ell)}$ the analogous Gaussian expectation of σ' . Both are deep kernels assembled from a shallow one, so depth enters the kernel explicitly rather than through learned weights. What they still lack is the second half of the phrase: the components are not learned.

44.3.2 Convolutional kernel networks

Mairal (2016) closed that gap with the *convolutional kernel network* (CKN), a kernel that is both deep and partly learned. One layer works on local patches: for two patches z, z' (say normalized image patches) it uses a Gaussian patch kernel $\kappa(z, z') = e^{-\|z-z'\|^2/2\sigma^2}$, whose feature map lives in an infinite-dimensional RKHS, and then pools the resulting feature maps spatially. Stacking such layers yields a hierarchical kernel over the whole signal, a genuine multilayer geometry. The learned part enters through the finite-dimensional approximation: each layer's RKHS is approximated by a Nyström projection onto a small set of *anchor points* (learnable filters), and these anchors are trained end-to-end by backpropagation to minimize the supervised loss. A CKN is thus a kernel whose depth is fixed by construction but whose internal representation, the anchor points at every layer, is learned from data, occupying a deliberate middle ground between a frozen kernel and a free neural network.

44.4 Attention is a kernel smoother

The architecture behind large language models turns out to compute a kernel average, and seeing it that way both explains its cost and points to the cure the kernel literature already owns. For a query q and keys and values $\{(k_j, v_j)\}$, an attention head returns

$$\text{Attn}(q) = \sum_j \frac{\exp(q^\top k_j / \sqrt{d})}{\sum_\ell \exp(q^\top k_\ell / \sqrt{d})} v_j.$$

Tsai et al. (2019) observed that this is exactly a Nadaraya-Watson kernel smoother with the *softmax kernel* $\kappa(q, k) = \exp(q^\top k / \sqrt{d})$: a weighted average of the values, with weights the kernel evaluations normalized to sum to one. The scaled dot product is the kernel, the softmax is the normalization, and the value is the regression target.

Reading attention as a kernel makes an identity visible: the softmax kernel is a Gaussian kernel in disguise, reweighted per point.

Proposition (softmax kernel is a rescaled Gaussian)

With $\tau = \sqrt{d}$,

$$\exp\left(\frac{q^\top k}{\tau}\right) = \underbrace{\exp\left(\frac{\|q\|^2}{2\tau}\right)}_{c(q)} \underbrace{\exp\left(\frac{\|k\|^2}{2\tau}\right)}_{m(k)} \exp\left(-\frac{\|q-k\|^2}{2\tau}\right).$$

Consequently the attention weights are those of a genuine Gaussian (RBF) kernel smoother with key-dependent masses $m(k_j)$; the query factor $c(q)$ cancels in the normalization.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

Expand $\|q-k\|^2 = \|q\|^2 + \|k\|^2 - 2q^\top k$, so $q^\top k = \frac{1}{2}(\|q\|^2 + \|k\|^2 - \|q-k\|^2)$. Dividing by τ and exponentiating gives the displayed factorization. In the attention weight $A_{ij} = \kappa(q_i, k_j) / \sum_\ell \kappa(q_i, k_\ell)$, substitute the factorization: the common factor $c(q_i)$ appears in numerator and denominator and cancels, leaving $A_{ij} = g_{ij}m(k_j) / \sum_\ell g_{i\ell}m(k_\ell)$ with $g_{ij} = \exp(-\|q_i - k_j\|^2/2\tau)$ an RBF kernel. $\square$

Example (attention as smoothing on three tokens)

Three tokens, self-attention, $d = 2$, $\tau = \sqrt{2}$. Queries equal keys $q_1 = k_1 = (1, 0)$, $q_2 = k_2 = (0, 1)$, $q_3 = k_3 = (1, 1)$; values $v_1 = (1, 0)$, $v_2 = (0, 2)$, $v_3 = (3, 1)$.

1. Score and exponentiate. $q^\top k/\tau$ has off-diagonal entries 0.7071 and 1.4142; the softmax kernel matrix is $\begin{pmatrix} 2.028 & 1.000 & 2.028 \\ 1.000 & 2.028 & 2.028 \\ 2.028 & 2.028 & 4.113 \end{pmatrix}$.
2. Normalize rows. Dividing by row sums gives the attention weights, e.g. row 1 is (0.401, 0.198, 0.401) and row 3 is (0.248, 0.248, 0.504); every row sums to 1.
3. Average the values. Attn = AV has rows (1.604, 0.797), (1.401, 1.203), (1.759, 1.000).
4. Check the Gaussian identity. Reconstructing the kernel matrix as $c(q_i)m(k_j) \exp(-\|q_i - k_j\|^2/2\tau)$ reproduces step 1 to $< 10^{-12}$: the softmax kernel is exactly the rescaled Gaussian.

Reading. A transformer head is a Nadaraya-Watson estimator; the quadratic cost of attention is the cost of evaluating a dense kernel matrix, and the identity says that matrix is an RBF Gram with per-token weights.

Verification artifact. checks/example-ch-frontier-example-43-2.json records the example source hash and verification scope.

Because the cost is the kernel matrix, the kernel cure applies. If the softmax kernel could be written as an inner product $\langle \varphi(q), \varphi(k) \rangle$ of an explicit finite feature map, the weighted sum would collapse into a running total and attention would cost linear rather than quadratic time in the sequence length. Katharopoulos et al. (2020) do this with a simple positive feature map; the *Performer* of Choromanski et al. (2021) does it with random features that are unbiased for the softmax kernel itself, using the following identity, which is the Gaussian integral read as a kernel factorization.

Proposition (positive random features for the softmax kernel; Choromanski et al. 2021)

For $\omega \sim \mathcal{N}(0, I_d)$ and the feature $\varphi_\omega(x) = \exp(\omega^\top x - \frac{1}{2}\|x\|^2)$,

$$\mathbb{E}_\omega[\varphi_\omega(q) \varphi_\omega(k)] = \exp(q^\top k).$$

Averaging φ over D sampled directions gives an unbiased, almost surely positive estimate of the (unscaled) softmax kernel, so attention with these features costs $O(nD)$ rather than $O(n^2)$.

Assumptions. All domains, quantifiers, regularity conditions, and parameter restrictions stated in the result are in force; no converse is implied. **Proof status.** Proved immediately below.

Proof

The Gaussian moment generating function gives $\mathbb{E}_\omega[\exp(\omega^\top a)] = \exp(\frac{1}{2}\|a\|^2)$ for any a . Take $a = q + k$:

$$\mathbb{E}_\omega[\varphi_\omega(q)\varphi_\omega(k)] = e^{-\frac{1}{2}\|q\|^2 - \frac{1}{2}\|k\|^2} \mathbb{E}_\omega[e^{\omega^\top(q+k)}] = e^{-\frac{1}{2}\|q\|^2 - \frac{1}{2}\|k\|^2} e^{\frac{1}{2}\|q+k\|^2} = e^{q^\top k},$$

since $\frac{1}{2}\|q+k\|^2 - \frac{1}{2}\|q\|^2 - \frac{1}{2}\|k\|^2 = q^\top k$. Positivity is immediate because $\varphi_\omega > 0$. $\square$

The check for this section confirms both identities numerically, including that the Monte-Carlo average of $\varphi_\omega(q)\varphi_\omega(k)$ over sampled directions converges to $\exp(q^\top k)$. This is precisely the random Fourier feature idea of Chapter 37, Large-Scale Kernel Machines transplanted into a sequence model: the scaling wall and the random-feature escape from it reappear at the center of modern architectures.

44.5 Kernels on curved and discrete domains

Feature learning is the frontier for the representation; the frontier for the *domain* is geometry. Kernel design, the subject of Part V, gained a principled way to build covariances on spaces that are not $\mathbb{R}^d$: Riemannian manifolds and graphs, where the notion of distance and smoothness must respect curvature or connectivity. Borovitskiy et al. (2020, 2021) construct Matern Gaussian processes on such domains by starting from the stochastic partial differential equation that defines the Matern process on $\mathbb{R}^d$ and replacing the Laplacian by the geometry's own Laplace operator.

The graph case is the most concrete. Let L be the graph Laplacian of a weighted graph on N nodes, a symmetric positive semidefinite matrix whose eigenpairs (λ_i, u_i) are the graph's Fourier modes. The Matern Gaussian process with smoothness ν and length scale κ is defined so that its covariance is a spectral filter of L .

Definition (Matern and diffusion kernels on a graph)

With graph Laplacian L , the graph Matern kernel is the matrix

$$K_\nu = \left(\frac{2\nu}{\kappa^2} I + L \right)^{-\nu},$$

that is, the filter $\Phi(\lambda) = (2\nu/\kappa^2 + \lambda)^{-\nu}$ applied to each Laplacian eigenvalue, up to a normalizing constant. In the limit $\nu \rightarrow \infty$ with fixed κ , this converges to the *diffusion (heat) kernel* $K_\infty = \exp(-\frac{\kappa^2}{2}L)$.

Both kernels are positive semidefinite by construction, since $\Phi(\lambda) > 0$ on the spectrum of L , and they penalize functions that vary sharply across edges exactly as the reciprocal-density penalty of a translation-invariant kernel penalizes high frequency (the parallel with the spectral view is exact, with the Laplacian eigenvalues playing the role of squared frequency). The diffusion limit is precisely the diffusion kernel of Chapter 24, Kernels for and on Graphs, now recovered as the smooth endpoint of a family with a tunable number of derivatives, and the manifold construction gives the same object on a curved surface with the Laplace-Beltrami operator in place of L . This is the geometric branch of kernel design maturing into a rigorous tool for structured domains, and it interoperates cleanly with the Gaussian-process machinery of Chapter 40, Gaussian Processes and the RVM.

44.6 Quantum feature maps and kernel estimation

A quantum feature map prepares a state $|\phi(x)\rangle = U(x)|0\rangle$ and defines a kernel from state overlap, for example $k(x, x') = |\langle\phi(x) | \phi(x')\rangle|^2$. The Gram matrix can be estimated on quantum hardware and passed to a classical SVM or ridge solver, as demonstrated by Havlíček et al. (Havlíček et al. 2019). Positive semidefiniteness is exact for the ideal overlap kernel but can be violated by finite-shot noise and device error, requiring a documented PSD repair whose effect is evaluated.

The central research question is not whether the feature space is exponentially large. Classical Gaussian kernels already have infinite-dimensional feature spaces. The question is whether the relevant Gram entries or learned decision rule can be obtained with an end-to-end advantage after state preparation, sampling noise, error mitigation,

kernel concentration, hyperparameter selection, and classical baselines are counted. A credible quantum-kernel experiment must report circuit family and depth, shot budget, device noise, PSD correction, wall time, and comparisons to classically simulated and ordinary kernels at matched tuning effort.

44.7 Foundation-model representations as learned kernels

Any frozen representation $h_\theta(x)$ induces the kernel $k_\theta(x, x') = \langle h_\theta(x), h_\theta(x') \rangle$. This turns a foundation model into a learned feature map followed by kernel ridge regression, an SVM, a GP approximation, or a distributional test. The construction is mathematically ordinary but practically powerful: it separates expensive representation learning from a convex, auditable readout and permits spectral, calibration, and influence diagnostics in the induced geometry.

It also imports every bias of the representation. Centering, normalization, layer choice, pooling, prompt template, and model version all change the kernel. Treat them as hyperparameters and keep selection away from the final test set. For multimodal representations, matched and unmatched pairs offer a direct alignment audit through HSIC or MMD. For retrieval, report neighborhood composition and hubness, not only average accuracy.

Some mechanistic accounts of in-context learning show transformers implementing gradient-like updates on stylized regression tasks (Von Oswald et al. 2023). This does not establish that general language-model inference is kernel regression. The useful boundary is empirical: compare predictions to a frozen-kernel or linearized surrogate, measure disagreement as context and scale vary, and state the task family for which the approximation holds.

44.8 Privacy, federated learning, and fairness

Kernel methods do not become private because only Gram matrices or support vectors are shared. Pairwise similarities can reveal membership and attributes. Differentially private regularized empirical-risk minimization perturbs objectives or outputs under explicit smoothness and convexity conditions and includes nonlinear-kernel constructions (Chaudhuri et al. 2011). Privacy accounting must include preprocessing, kernel tuning, repeated releases, and validation, not only the final solver.

Federated kernel learning can aggregate random-feature statistics or local coefficients, but client heterogeneity changes the target and communication can leak updates. Report per-client performance, participation, communication, secure-aggregation assumptions, and the complete privacy budget. A global average can conceal a model that fails on small clients.

Fairness is a property of a decision process and population, not of PSD geometry. Kernels can express independence penalties, match conditional distributions, or construct subgroup tests, but each encodes a normative choice about protected attributes, legitimate covariates, and error tradeoffs. Report subgroup sample sizes, uncertainty, intersectional results, and incompatibilities among fairness criteria. Never infer a sensitive attribute merely to claim fairness without documenting consent and governance.

44.9 Operator algebra and noncommutative data

The operator-valued kernels of Chapter 45, Vector- and Operator-Valued Kernels still index functions by ordinary inputs. Quantum states, covariance operators, and dynamical observables motivate similarities whose arguments or values are themselves operators and whose multiplication need not commute. Completely positive maps, trace kernels, and operator monotone functions become the analogue of scalar PSD constructions.

This direction connects kernel learning to quantum information and operator learning, but basic scalar shortcuts can fail: entrywise products, pointwise functional calculus, and simultaneous diagonalization require commutativity or stronger positivity notions. The chapter treats this as an open boundary, not a settled toolkit. Any proposed noncommutative kernel should state its domain, positivity notion, representation theorem, computational oracle, and what classical reduction it improves upon.

44.10 Frontier status and annual update policy

This chapter has a literature cutoff of **30 June 2026**. Each annual revision should verify links and maintenance status, distinguish peer-reviewed results from preprints, update negative as well as positive evidence, and record changed claims in the revision history. A frontier claim enters the stable core only after its assumptions and proof status can be stated independently of a single benchmark or implementation.

44.11 The synthesis: what the kernel view keeps, and where it stops

It is worth stating plainly what this chapter has and has not shown. The kernel view keeps three things at the frontier. First, *theory*: the sharpest available statements about when a wide network generalizes are kernel statements, because in the lazy limit the network is a kernel machine with a computable spectrum, and the learning-curve and interpolation analyses of Chapter 42, Modern Generalization Theory apply verbatim. Second, *uncertainty and calibration*: a kernel is a covariance, so the Gaussian-process posterior gives principled error bars that a plain network does not, and deep kernel learning keeps them while feeding a learned representation into the kernel. Third, *structure*: on manifolds and graphs the kernel is still the natural object, because it is the geometry, not a representation to be discovered.

The kernel view stops at exactly one place, and the stop is structural rather than a matter of engineering. A kernel fixes its feature map before it sees the labels, so it cannot learn the representation. The single-index and staircase separations of the middle of this chapter are not artifacts; they are the precise price of a frozen geometry, an unavoidable d^k sample gap on targets that hide their relevant structure in an unknown low-dimensional direction. No amount of kernel engineering closes it, because closing it is by definition leaving the kernel regime. The honest reading of the modern literature is therefore not that kernels were replaced, and not that networks are secretly kernels after all, but that the boundary between them is now drawn with a straightedge. We can say, more sharply than the classical theory ever could, exactly where the fixed geometry of a kernel suffices and exactly where a model must learn its own.

The open questions live right on that boundary and remain genuinely open. What is the correct function space of a network trained beyond the lazy regime at finite width, where the tangent kernel neither is constant nor has fully escaped to the mean-field limit? Can one characterize the effective, data-dependent kernel that a feature-learning network converges to, and would it carry the calibration guarantees the fixed kernel enjoys? How much of the staircase mechanism survives in deep and attention-based models, whose relevant directions are not single coordinates but learned subspaces? These are the questions where the kernel view meets its edge, and they are, fittingly for a closing chapter, still being written.

44.12 Common mistakes and practical implications

For **The Frontier: Feature Learning and Beyond**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

44.13 Summary and further reading

This chapter established explain the central definitions and claims in The Frontier: Feature Learning and Beyond; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Jacot and Hongler 2018), (Arora and Wang 2019), (Chizat and Bach 2019).

44.14 Exercises

1. warm-up Verify that the empirical NTK $\Theta_\theta(x, x') = \nabla_\theta f(x)^\top \nabla_\theta f(x')$ is a positive definite kernel for every fixed θ . Which property of the definition can fail if two distinct inputs produce identical Jacobian rows, and what does that mean for invertibility of $\Theta(X, X)$?
2. computation For the linear model $f(x; \theta) = \theta^\top x$, compute the empirical NTK and show it equals the linear kernel $x^\top x'$ for all θ . Conclude that a linear model is exactly (not just approximately) in the lazy regime, and identify its "learned function" from the Proposition on output dynamics.
3. computation Reproduce the tiny-NTK worked example with the alternative test point $x_* = -0.1$. Recompute the Jacobian row at x_* (mind which ReLUs are active), the test column $\Theta(x_*, X)$, and the prediction $f_\infty(x_*)$. Confirm it again matches the linearized-network value. Hint: at $x_* = -0.1$ the second unit $2x + b_2$ is inactive; its three Jacobian entries vanish.
4. computation Prove that the softmax kernel $\kappa(q, k) = \exp(q^\top k / \tau)$ is positive definite on any bounded set of vectors. Hint: use the Gaussian-rescaling Proposition together with the fact that the Gaussian kernel is positive definite and that multiplying a kernel by $c(q)c(k)$ for a fixed function c preserves positive definiteness (a rank-one rescaling).
5. challenge The Performer estimator averages D independent copies of $\varphi_\omega(q)\varphi_\omega(k)$. Using the Gaussian moment generating function, compute $\text{Var}_\omega[\varphi_\omega(q)\varphi_\omega(k)]$ in closed form and show it grows with $\|q + k\|$. Explain why softmax attention with large scores is the hard case for random-feature approximation. Hint: $\mathbb{E}[\varphi_\omega(q)^2 \varphi_\omega(k)^2] = \mathbb{E}[e^{2\omega^\top(q+k)}]e^{-\|q\|^2 - \|k\|^2}$; apply the MGF with $a = 2(q + k)$.
6. challenge On the graph Matern kernel $K_\nu = (2\nu/\kappa^2 I + L)^{-\nu}$, diagonalize with $L = \sum_i \lambda_i u_i u_i^\top$ and write K_ν in the eigenbasis. Take $\nu \rightarrow \infty$ with κ fixed and show each eigenvalue tends to $e^{-\kappa^2 \lambda_i / 2}$, recovering the diffusion kernel. Hint: $(1 + \frac{\kappa^2 \lambda_i}{2\nu})^{-\nu} \rightarrow e^{-\kappa^2 \lambda_i / 2}$; handle the normalizing constant so that the $\lambda_i = 0$ mode is fixed.
7. synthesis Argue, using the single-index sample barrier, why increasing the width of a network in the NTK regime does not help it learn $f^\star(x) = g(\langle w^\star, x \rangle)$ faster, whereas the mean-field parametrization does. Which quantity in the two limits differs, and what does the difference say about the role of the read-out layer versus the hidden layer?

45 Applications and Practice

Every chapter so far has built one piece of the kernel toolbox: a construction that turns structure in the data into an inner product, an algorithm that fits a predictor once the inner product is fixed, a theorem that says when the fit will generalize. This closing chapter puts the pieces together the way a practitioner does. We walk three classic applications end to end, naming the kernel, the pipeline, and the result that made each a landmark; we distill the choices that decide whether a kernel method works in practice, from picking a kernel for your data type to the model-selection loop, the normalization steps, and the spectrum diagnostics that tell you whether you are under or overfitting; and we close with a map of the software that puts all of this a few lines of code away. The aim is not new theory but judgment: given a problem, which of the book's kernels do you reach for, how do you tune it honestly, and when do you trade the exact solve for an approximation.

45.1 The anatomy of a kernel-method project

A kernel method has a rigid skeleton, and seeing it once makes every application below a variation on one theme. You choose a kernel k that encodes similarity for your data, form or implicitly access the Gram matrix $K_{ij} = k(x_i, x_j)$, feed it to a convex learner (a support vector machine, kernel ridge regression, a Gaussian process), and tune a few hyperparameters by cross-validation. The kernel is where domain knowledge enters, the learner is off the shelf, and the tuning is where honesty enters. Almost everything separating a method that works from one that does not lives in three decisions: whether the kernel matches the structure of the data, whether the data and kernel are normalized so the similarity means what you think, and whether the model's capacity is matched to the data, which is what the spectrum of K measures.

Two facts frame the enterprise. There is no universally best kernel: the representer theorem and the RKHS geometry hold for any positive definite k , so choosing k is choosing what "similar" means, a modeling decision no theorem makes for you. And once k is fixed the problem is convex, so the only real freedom left is the handful of hyperparameters. The art is front-loaded into kernel design and back-loaded into model selection, with a mechanical optimization between: the applications show the former, the recipe the latter.

45.2 Three applications, three kernels

The kernels for structured data earn their keep where the raw objects, protein sequences, documents, molecules, are not vectors at all. In each the kernel is the only bridge from the object to the RKHS machinery, and in each a specific kernel plus a plain support vector machine set the state of the art for years.

45.2.1 Remote protein homology with string and mismatch kernels

The task is detection of distant evolutionary relatives. Two proteins in the same structural superfamily can share as little as a fifth of their amino acids, so a raw identity count fails, and the goal is to decide, for a query sequence, whether it belongs to a given family despite that low overlap. The discriminative formulation, following Jaakkola, Diekhans, and Haussler (2000), trains a support vector machine to separate family members from non-members, which turns the problem into the design of a kernel between variable-length strings over the twenty-letter amino-acid alphabet.

The spectrum kernel of Leslie, Eskin, and Noble (2002) is the simplest such kernel and already very strong. For a fixed length k , its feature map counts the occurrences of every k -mer (contiguous substring of length k), so a protein becomes a vector of 20^k counts and the kernel is the inner product of two count vectors. The mismatch kernel of Leslie et al. (2004) relaxes the exact match: a k -mer contributes to every k -mer within m substitutions of it, giving the (k, m) -mismatch kernel, with $(5, 1)$ a standard choice. Both are string kernels from the chapter on sequence kernels, computed without materializing the 20^k -dimensional vector by the trie and suffix-tree techniques of the efficient string-kernel chapter in time linear in sequence length. Since a long protein has more k -mers than a short one, the raw kernel scales with length and is always cosine-normalized before use, the normalization step discussed below.

A complementary route lets a generative model design the features. The Fisher kernel of Jaakkola and Haussler (1999) fits a profile hidden Markov model to the positive family, represents each sequence by the gradient of its log-likelihood under that model, and feeds those Fisher scores to the SVM; this is the generative-model kernel of the chapter on kernels from generative models and the original SVM-Fisher method for the benchmark. The string and Fisher kernels bracket the two philosophies of the book: build similarity from the object, or read it off a fitted model.

The pipeline is identical in either case: build the Gram matrix over the training sequences, normalize it, train one SVM per family, score held-out sequences. On the standard SCOP benchmark, in which each of thirty-three families is held out in turn and performance is measured by the ROC and ROC50 scores (the area under the ROC curve, and under its first fifty false positives), the string kernels match or exceed the SVM-Fisher method while being computed directly from sequence with no model to fit (Leslie et al. 2004; Jaakkola et al. 2000). Alignment kernels scoring pairs by their optimal local alignment (Saigo et al. 2004) push detection further still, and all are convolution kernels in the sense of Haussler (1999). The lesson that carried into practice: a cheap, exact, sequence-derived kernel can beat an expensive model-based one, and the string kernels became the default for biological sequence classification.

45.2.2 Text categorization with the vector-space and string kernels

Assigning topic labels to documents was the problem on which support vector machines first proved themselves in the discrete world. The classical representation is the bag of words: fix a vocabulary and represent a document by its (TF-IDF weighted) term counts, ignoring word order. The kernel is the inner product in this space, which once the vectors are length-normalized is exactly the cosine similarity of the vector-space model. The SVM is the right learner because of the geometry the earlier chapters built: the feature space has tens of thousands of dimensions, almost all irrelevant to any given category, and the margin bounds of the support vector machine chapter guarantee that a large-margin separator there generalizes from few examples with no explicit feature selection.

Joachims (1998) established this on the Reuters-21578 benchmark, where the SVM was the best of the methods tried across essentially all categories. The reported microaveraged precision-recall breakeven points make the gap concrete.

Method	Microaveraged breakeven (%)
Naive Bayes	72.0
Rocchio	79.9
C4.5 decision tree	79.4
k -nearest neighbors	82.3
SVM (RBF kernel, best setting)	86.4

The numbers are those reported by Joachims (1998) on the Reuters-21578 "ModApte" split; the SVM improves on the strongest classical baseline, the nearest-neighbor rule, by about four points and on naive Bayes by more than fourteen. Word order can be recovered without leaving the framework: the string subsequence kernel of Lodhi et al. (2002) works directly on character sequences, scoring documents by their shared (possibly gapped) substrings,

competitive with the word kernels while needing no tokenization, and latent semantic kernels (Cristianini et al. 2002) fold a low-rank semantic smoothing into the same inner product. Both are developed in the chapter on kernels for text. The takeaway is that for text a normalized linear kernel on a good weighting is a strong baseline that is hard to beat, with the string kernels held in reserve for when subword structure matters.

45.2.3 Molecular property prediction with graph kernels

A small molecule is naturally a labeled graph: atoms are vertices labeled by element, bonds are edges labeled by type. Predicting a property (mutagenicity, toxicity, activity) from that graph is the cheminformatics problem, and graph kernels are the bridge to the SVM. The difficulty is fundamental: Gärtner et al. (2003) showed that a kernel able to separate any two non-isomorphic graphs is as hard to compute as graph isomorphism, so every practical graph kernel compares tractable substructures. The marginalized graph kernel counts label sequences along random walks, a marginalized kernel in the sense of the generative-kernels chapter; the shortest-path kernel of Borgwardt and Kriegel (2005) compares multisets of shortest-path lengths; and the Weisfeiler-Lehman subtree kernel of Shervashidze et al. (2011), comparing iteratively refined subtree patterns, is the scalable modern default. All belong to the chapter on kernels for and on graphs.

The application that named the field is Ralaivola et al. (2005), who built marginalized and fingerprint-based (Tanimoto and MinMax) graph kernels, paired them with an SVM, and reported performance competitive with the best hand-engineered molecular descriptors of the time on standard structure-activity benchmarks: mutagenicity, carcinogenicity, and the NCI anticancer screens. Because a random-walk kernel sums over walks of every length, its raw value grows with molecule size, so cosine normalization is again mandatory. This is what moved cheminformatics onto the kernel framework: the same SVM code that classifies proteins and documents classifies molecules, once a graph kernel supplies the inner product. The worked examples below are the small numeric surrogate for such a pipeline.

45.3 The practical recipe

The applications share a skeleton, and that skeleton is the recipe: choose a kernel, normalize the inputs and the kernel, select the hyperparameters by nested cross-validation, and diagnose the fit. We take these in turn and collect them into one algorithm.

45.3.1 Choosing a kernel for your data

The first decision is the least mechanical, and the book has been a catalog of answers to it. The following guide maps a data type to a sensible first kernel and to the chapter that builds it. It is a starting point, not a verdict: the honest test is always the cross-validated error of the section that follows.

Your data	A good first kernel	Where it is built
Fixed-length real vectors, smooth response	Gaussian (RBF); Matern if you want to tune smoothness	[[ch:kernel-families Kernel families]], fit with [[ch:support-vector-machines SVM]] / [[ch:kernel-ridge-and-friends KRR]] / [[ch:gaussian-processes-and-rvm GP]]
Vectors, interactions of bounded order	Polynomial or ANOVA kernel	[[ch:kernel-families Kernel families]]

Your data	A good first kernel	Where it is built
Biological sequences (variable length)	Spectrum, mismatch, or alignment string kernel	[[ch:string-kernels Sequence kernels]], [[ch:efficient-string-and-tree-kernels efficient evaluation]]
Free-text documents	Normalized bag-of-words linear kernel, or string subsequence kernel	[[ch:kernels-for-text Kernels for text]]
Molecules, parse trees, networks	Weisfeiler-Lehman, random-walk, or shortest-path graph kernel	[[ch:graph-kernels Graph kernels]]
Objects with a fitted generative model	Fisher or marginalized kernel	[[ch:generative-and-marginalization-kernels Generative-model kernels]]
Data with a known invariance	Invariant / jittered kernel, tangent distance	[[ch:invariances-and-pre-images Invariances]]
Sets, bags, or probability distributions	Mean-embedding or distribution kernel	[[ch:distribution-regression Distribution regression]]
Several heterogeneous sources	A learned combination of base kernels	[[ch:multiple-kernel-learning Multiple kernel learning]]

Two rules of thumb cut across the table. On vectors in doubt, start with the Gaussian kernel and one well-tuned bandwidth: it is universal, and its single length scale is easy to search. On structured data, spend your effort on the kernel, not the classifier, since the SVM downstream is interchangeable. And if you truly cannot choose, do not: multiple kernel learning turns the choice into a convex problem, as its chapter develops.

45.3.2 Normalization and centering

A kernel is a statement about similarity, and that statement is only as trustworthy as the units it is made in. Three normalizations recur, and skipping them is the most common way a sound method underperforms.

Feature standardization. For a distance-based kernel such as the Gaussian, a coordinate measured in large units dominates $\|x - x'\|^2$ and silently sets the similarity, so raw features are first standardized to zero mean and unit variance, $x \mapsto (x - \hat{\mu})/\hat{\sigma}$ coordinatewise. The estimates $\hat{\mu}, \hat{\sigma}$ must be computed on the training fold alone; computing them on the full data before splitting leaks information and biases the cross-validated error downward.

Kernel (cosine) normalization. Any kernel can be rescaled to unit diagonal by

$$\tilde{k}(x, x') = \frac{k(x, x')}{\sqrt{k(x, x)k(x', x')}},$$

which sends every point to the unit sphere of feature space, so $\tilde{k}(x, x') = \cos \angle(\Phi(x), \Phi(x'))$. This is a conformal transformation of the kernel and preserves positive definiteness, as the closure properties guarantee. It is cosmetic for the Gaussian, whose diagonal is already 1, but essential for string and graph kernels, whose raw magnitude tracks object size rather than content, and for polynomial kernels, whose magnitude tracks input scale.

Centering in feature space. Some methods need the data centered at the origin of feature space. The centered Gram matrix is

$$\tilde{K} = HKH, \quad H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top,$$

which subtracts the feature-space mean without ever computing it, and it is the first step of kernel PCA in its chapter. Supervised machines often skip it: the constraint $\sum_i \alpha_i y_i = 0$ in the SVM dual already makes the solution invariant to a constant offset of the kernel, the conditional-positive-definiteness invariance of the kernel-families chapter.

45.3.3 The model-selection loop

With the kernel and its normalizations fixed, a kernel machine still has a small grid of dials: the kernel hyperparameters (the Gaussian bandwidth σ , the polynomial degree) and the regularization (the SVM penalty C , the ridge λ). These are not learned by the convex fit but chosen by estimating out-of-sample error. The most common methodological error in applied kernel work is to let one cross-validation both choose the hyperparameters and estimate the final error: reusing the selection folds to report performance is optimistic, because the grid was tuned to those very folds. The fix is nested cross-validation, an inner loop that selects and an outer loop that scores, so no data point ever helps choose the model later graded on it. Chapelle et al. (2002) give gradient-based alternatives to the grid; the grid remains the robust default.

Algorithm (End-to-end kernel method)

Input data $\{(x_i, y_i)\}_{i=1}^n$; a kernel family k_θ with hyperparameter grid Θ ; a learner with regularization grid $\mathcal{C}$; outer/inner fold counts K_1, K_2 .

Output a deployed predictor and an unbiased estimate of its error.

1. Encode each object into the kernel's input form (standardize vectors; leave sequences, graphs, and sets as is).
2. Split the data into K_1 outer folds.
3. For each outer fold, holding out (test) and keeping (train):
4. fit the feature standardization on (train) only;
5. for each $(\theta, C) \in \Theta \times \mathcal{C}$, estimate the error on (train) by K_2 -fold inner cross-validation, rebuilding and cosine-normalizing the Gram matrix inside each inner fold;
6. pick $(\theta^\star, C^\star)$ minimizing the inner error, refit on all of (train), and record the error on (test).
7. Report the mean and spread of the outer-fold errors as the generalization estimate.
8. Refit once on all the data at the $(\theta^\star, C^\star)$ selected on the full set, and deploy that predictor.

The worked example makes the inner loop concrete on a dataset small enough to check every number by hand.

Example (a model-selection loop by hand)

Six points on a line, $x_{\text{raw}} = (0, 1, 2, 3, 4, 5)$, with regression target $y = \sin(x_{\text{raw}}) = (0, 0.8415, 0.9093, 0.1411, -0.7568, -0.9589)$. We standardize the inputs (mean 2.5, standard deviation 1.7078) to $x = (-1.4639, -0.8783, -0.2928, 0.2928, 0.8783, 1.4639)$, fit kernel ridge regression with the RBF kernel $k(x, x') = e^{-(x-x')^2/2\sigma^2}$ and fixed ridge $\lambda = 0.1$, and select the bandwidth by 2-fold cross-validation over $\sigma \in \{0.5, 2.0\}$. The interleaved folds are $A = \{0, 2, 4\}$ and $B = \{1, 3, 5\}$. All numbers from `checks/ch-apps-ex1.py`.

1. Score the narrow bandwidth. At $\sigma = 0.5$, training on A and testing on B gives held-out MSE 0.1786; training on B and testing on A gives 0.1686; the mean CV error is 0.1736.
2. Score the wide bandwidth. At $\sigma = 2.0$, the two folds give 0.0935 and 0.3724, averaging to 0.2330. The wide kernel is excellent on one split and poor on the other, a reminder that averaging folds, not trusting a single split, is what stabilizes the estimate.
3. Pick the winner. The narrow bandwidth has the smaller CV error, $0.1736 < 0.2330$, so $\sigma^\star = 0.5$.
4. Refit and read the gap. Refitting at $\sigma^\star = 0.5$ on all six points drives the training MSE to 0.0025, far below the cross-validated 0.1736. The near-interpolation of the training data alongside a much larger held-out error is the signature of a high-capacity fit, which is exactly what the spectrum will quantify next.

Reading. Cross-validation, not a smoothness heuristic, chooses the bandwidth: here it prefers the narrower, higher-capacity kernel because the target genuinely wiggles over the range, even though "wider is smoother is safer" would have guessed otherwise. The wide-bandwidth fold swing and the narrow-bandwidth train-versus-CV gap are the two things to watch, and both are visible in numbers on six points.

Verification artifact. `checks/example-ch-apps-example-44-1.json` records the example source hash and verification scope.

45.3.4 Diagnosing under and overfitting through the spectrum

When a fit disappoints, the eigenvalues of the Gram matrix say why. Mercer’s theorem in its chapter reads the spectrum of K as the energy the kernel places along each eigen-direction, and how fast the eigenvalues decay is the effective number of directions the model can use. A useful scalar summary at ridge level λ is the effective dimension

$$d_{\text{eff}}(\lambda) = \sum_{i=1}^n \frac{\lambda_i}{\lambda_i + \lambda},$$

which counts eigenvalues large compared to λ (each near 1) and discounts the small ones (near 0). It is the number of parameters the machine actually spends, running from 0 when λ swamps every eigenvalue to $\text{rank}(K)$ as $\lambda \rightarrow 0$, and it is the degrees of freedom of kernel ridge regression that govern the rates of the ridge chapter and the Gaussian-process chapter.

Example (reading the spectrum)

The same six standardized points. We form the RBF Gram matrix at the two bandwidths and read its eigenvalues and effective dimension at $\lambda = 0.1$. The trace is 6 in both cases (unit diagonal), so the spectra differ only in how that fixed energy is distributed. All numbers from `checks/ch-apps-ex2.py`.

1. Take the wide-bandwidth spectrum. At $\sigma = 2.0$ the eigenvalues are (4.8828, 1.0143, 0.0976, 0.0052, 0.0002, 0.0000): the top two already hold 98.3% of the trace, and the tail is numerically negligible. The kernel exposes very few directions.
2. Take the narrow-bandwidth spectrum. At $\sigma = 0.5$ the eigenvalues are (1.9971, 1.6210, 1.1422, 0.6959, 0.3666, 0.1773): the energy is spread across all six directions, with the top two holding only 60.3%.
3. Convert to effective dimension. At $\lambda = 0.1$ the wide kernel has $d_{\text{eff}} = 2.43$, the narrow one $d_{\text{eff}} = 5.11$. The wide kernel is a roughly two-parameter model; the narrow one spends five of its six possible parameters.
4. Match the diagnosis to the fit. The narrow kernel’s high effective dimension is why it nearly interpolated the six points in the previous example, and why its train error collapsed while its CV error did not. A wide kernel with $d_{\text{eff}} = 2.43$ would risk the opposite failure, too few directions to bend to the target, which is underfitting.

Reading. The spectrum turns a vague worry about capacity into a number. A fast-decaying spectrum (small d_{eff}) means a simple model prone to underfitting: narrow the bandwidth or lower the ridge to expose more directions. A slow-decaying spectrum (large d_{eff}) with a train-versus-validation gap means overfitting: widen the bandwidth or raise the ridge. Both knobs act on the same quantity, d_{eff} , and matching it to the data is the whole game.

Verification artifact. `checks/example-ch-apps-example-44-2.json` records the example source hash and verification scope.

45.3.5 Exact, Nystrom, or random features

Everything above assumes the $n \times n$ Gram matrix can be formed and factored, at $O(n^2)$ memory and $O(n^3)$ time. That is fine up to roughly $n \sim 10^4$; beyond it the exact solve, not the statistics, becomes the bottleneck. The large-scale chapter develops two escapes, and the choice follows the data type.

The Nystrom method (Williams and Seeger 2001; Drineas and Mahoney 2005) samples $m \ll n$ landmark points and approximates K by its best rank- m reconstruction from the landmark columns, a low-rank approximation of the top of the spectrum. It shines when the spectrum decays fast, that is, when d_{eff} is small, because then a few landmarks capture the signal; the effective dimension of the previous section is the right guide to how many landmarks suffice, and Rudi et al. (2015) show that m of order d_{eff} preserves the statistical rate. It works for any kernel, including string and graph kernels, needing only to evaluate k on sampled pairs. Random Fourier features (Rahimi and Recht 2007) instead sample the Bochner spectral measure of a translation-invariant kernel to build an explicit feature map whose inner product approximates k , and Fastfood (Le et al. 2013) accelerates the sampling;

this route is specific to shift-invariant kernels on vectors but feeds a fast linear solver directly. As a rule of thumb: exact solve up to $n \sim 10^4$; Nystrom for structured data or fast-decaying spectra; random features for the Gaussian and its relatives on vectors. For the largest problems the preconditioned Nystrom solver Falkon (Rudi et al. 2017; Meanti et al. 2020) fits kernel ridge regression on 10^7 to 10^9 points on a single machine.

45.4 Case study: spatial exposure mapping

Suppose monitoring stations measure an environmental exposure and the goal is a map with uncertainty. Random train-test splitting is optimistic because nearby stations are correlated. The unit of validation should be a held-out region or buffered spatial block. Start with a mean model for known covariates, then model residual dependence with the spatial kernels of Chapter 50, Spatial and Spatiotemporal Kernel Models. Compare a Matern GP, a compactly supported approximation, and a simple regional baseline.

The deliverable is not only RMSE. Report spatially blocked error, interval coverage by region, variogram diagnostics, sensitivity to coordinate system and range parameter, and maps of both posterior mean and standard deviation. State whether the map interpolates within the sampled domain or extrapolates beyond its support. Never present narrow posterior bands as total uncertainty when sensor bias, preferential site placement, or mean misspecification are omitted.

45.5 Case study: a scientific inverse problem

Consider inferring a forcing field from noisy measurements of a physical response. Encode the forward operator and boundary conditions before choosing an optimizer. The workflows in Chapter 47, Inverse Learning and Spectral Regularization and Chapter 53, Kernels for Scientific Computing and Operator Learning separate observation fit from the regularity and physics assumptions that identify the inverse.

Create synthetic recovery tests where the truth is known, including a truth outside the assumed RKHS. Plot recovered spectra, forward residuals, parameter error, and uncertainty calibration. Compare Tikhonov, early-stopped Krylov iteration, and a mesh-based baseline under equal compute. A low observation residual can coexist with a wrong forcing field; the report must expose null spaces and resolution limits.

45.6 Case study: dynamics and off-policy evaluation

For a controlled dynamical system, split by complete trajectories and time, never by individual transitions. A one-step kernel model can achieve excellent random-split error while compounding bias over a rollout. Evaluate one-step prediction, multi-step rollout, invariant or constraint violations, and performance under policies different from the behavior policy. Use Chapter 52, Kernels for Dynamical Systems, Control, and Reinforcement Learning to distinguish state-transition regression, Koopman approximation, value-function learning, and kernelized policy evaluation.

Coverage is the central diagnostic. Plot importance weights or a kernel discrepancy between behavior-policy and target-policy state-action distributions. If support overlap fails, report the target value as unidentified rather than stabilizing a meaningless estimate with stronger regularization.

45.7 Case study: distribution shift and calibrated decisions

Train a predictor on a source population and deploy it on a changing target population. First distinguish covariate, label, and concept shift; each permits different corrections. Use a classifier-based density-ratio diagnostic and witness functions to localize change, then compare unweighted training, clipped importance weighting, and a representation chosen without target labels. The framework in Chapter 51, Distribution Shift, Robustness, and Conformal Prediction supplies the assumptions and conformal layer.

Report source and target discrimination, effective sample size of weights, subgroup performance, and conformal coverage with interval width. Marginal coverage can hide subgroup failure, and online recalibration spends labels. State the label-acquisition delay and the exact exchangeability or weighted-exchangeability argument behind each interval.

45.8 Case study: censored and time-to-event outcomes

Time-to-event data add right censoring and competing risks. A kernel can enter through a nonlinear risk score, an accelerated failure-time regression, or a kernel on longitudinal histories. Splits must respect patient and calendar time. Censoring weights require a model for the censoring mechanism and can become unstable in the tail.

Report time-dependent discrimination, integrated Brier score, calibration at clinically meaningful horizons, and the number still at risk. Compare against regularized Cox and non-kernel survival baselines. Do not tune on a concordance measure and claim calibrated survival probabilities; ranking and calibration are separate targets.

45.9 Model cards and reproducibility packets

Every applied result should ship with a compact reproducibility packet:

- task definition, intended use, excluded use, and decision costs;
- dataset version, license, cohort construction, missingness, and split unit;
- preprocessing fit boundaries and leakage audit;
- kernel formula, all learned hyperparameters, regularization, solver, tolerance, and random seeds;
- compute environment, wall time, memory, and a competitive non-kernel baseline;
- uncertainty or calibration procedure and the assumptions it needs;
- subgroup, shift, and failure-mode analysis;
- links to the exact notebook, artifacts, and book sections.

This packet is the applied analogue of a theorem’s assumption block. It makes clear what was established, on which population, and what would have to remain true for the result to transfer.

45.10 A map of the software

None of this needs to be written from scratch. The table is a dated reproducibility record, not a timeless endorsement. Project links and status were checked on **2026-07-19**, matching this edition’s software cutoff; re-check the linked release notes before reproducing an environment.

Library	Verified role and maintenance note	Project
LIBSVM	Exact dual SVM and SVR via SMO; stable reference implementation for small-to-medium dense kernel problems	project

Library	Verified role and maintenance note	Project
LIBLIN-EAR	Linear SVM and logistic regression after an explicit feature map; maintained separately from kernel solvers	project
scikit-learn	Version 1.9.0 documents SVC/SVR, KernelRidge, KernelPCA, Nystroem, RBFSampler, pipelines, and model selection	stable API
GPy	Classical Python GP library with sparse models; maintenance and compatibility should be checked before starting a new project	repository
GPflow	Actively maintained TensorFlow GP library with variational and inducing-point models	project
GPyTorch	Version 1.15.2 documents exact, variational, multitask, deep-kernel, and matrix-multiply-based GPU inference	stable docs
KeOps	Symbolic and lazy kernel matrix-vector products with accelerator support, useful when a dense Gram matrix cannot be stored	docs
Falkon	Preconditioned Nystrom kernel ridge regression with accelerator support	project
SHOGUN	Broad historical kernel collection; the latest GitHub release shown at the cutoff is 6.1.4 from 2019, so treat it as a legacy option and verify toolchain compatibility	repository

For the companion environment, create a clean virtual environment and run `python -m pip install -r requirements-notebooks.txt`; the exact pins are versioned with the book. For a minimal classical workflow, `python -m pip install scikit-learn==1.9.0` matches the checked API. Accelerator libraries have platform-specific wheels and must follow their linked installation pages. Record Python, package, BLAS, accelerator, and driver versions with every benchmark.

A division of labor follows the table. Prototype classical estimators and approximations in scikit-learn; move to a linear solver once features are explicit, to KeOps or Falkon when a Gram matrix cannot be stored, and to GPflow or GPyTorch when predictive uncertainty is part of the task. For string and graph kernels, evaluate whether a maintained domain library or a small tested implementation is safer than adopting a legacy umbrella package.

45.11 Summary

A kernel method is a pipeline with a rigid shape and a few load-bearing choices. The kernel encodes what “similar” means, and the book has supplied one for nearly every data type: Gaussian and polynomial kernels for vectors, string and mismatch kernels for sequences, graph kernels for molecules and networks, Fisher and marginalized kernels when a generative model is at hand. The three applications showed one skeleton in three domains: choose the kernel, normalize it, hand the Gram matrix to a support vector machine, and the state of the art followed. The recipe made the tuning honest, through nested cross-validation, the normalizations that make similarity mean what it should, and the spectrum diagnostics that read under and overfitting off the eigenvalues of K as an effective dimension. When the exact solve runs out of room, the Nystrom method, random features, and preconditioned solvers of the large-scale chapter carry the same kernels to millions of points. The book’s through-line returns one last time: fix a positive definite kernel and a convex loss, and a great deal of learning is a linear-algebra problem in disguise. What remains is judgment, and judgment is what practice teaches.

45.12 Common mistakes and practical implications

For **Applications and Practice**, do not apply a displayed formula without checking its domain, statistical assumptions, and numerical conditioning. Avoid selecting kernels or hyperparameters on test data, and do not interpret an optimization residual as a generalization guarantee. When the method is computational, report preprocessing, kernel parameters, regularization, solver tolerance, condition diagnostics, runtime, and a non-kernel baseline. When the result is theoretical, distinguish sufficient conditions from necessary ones and finite-sample claims from asymptotic statements.

45.13 Summary and further reading

This chapter established explain the central definitions and claims in Applications and Practice; Apply the chapter's principal methods and interpret their outputs; State the assumptions behind formal results and connect them to earlier chapters. Revisit the assumptions attached to each formal result before transferring it to a new setting. For primary and extended treatments, consult (Leslie and Noble 2002), (Leslie and Noble 2004), (Jaakkola and Haussler 2000).

45.14 Exercises

1. warm-up Using the decision guide, name a sensible first kernel and the chapter that builds it for each: (a) a house price from ten standardized numeric features; (b) proteins classified into structural families from their sequences; (c) the toxicity of small molecules given as labeled graphs; (d) news articles categorized by topic; (e) two probability distributions given as samples. In one sentence, say why "there is no universally best kernel" makes this a modeling choice rather than a lookup.

2. computation Take the 3×3 Gram matrix $K = \begin{pmatrix} 4 & 2 & 0 \\ 2 & 9 & 3 \\ 0 & 3 & 2 \end{pmatrix}$. (a) Cosine-normalize it, $\tilde{K}_{ij} = K_{ij}/\sqrt{K_{ii}K_{jj}}$, and

verify the diagonal is all ones. (b) Double-center the original K with $H = I - \frac{1}{3}\mathbf{1}\mathbf{1}^\top$, forming HKH , and verify every row sum is 0. State in one line what each operation does to the feature-space picture.

3. proof Show that cosine normalization preserves positive definiteness: if k is a positive definite kernel with $k(x, x) > 0$ for all x , then $\tilde{k}(x, x') = k(x, x')/\sqrt{k(x, x)k(x', x')}$ is positive definite. Then identify the feature map of $\tilde{k}$ in terms of the feature map Φ of k . Hint

This is the conformal-transformation closure property with $f(x) = 1/\sqrt{k(x, x)}$: $\tilde{k}(x, x') = f(x)k(x, x')f(x')$. The feature map is $\tilde{\Phi}(x) = \Phi(x)/\|\Phi(x)\|$, the projection of $\Phi(x)$ onto the unit sphere.

4. exploration Rerun the model-selection example with a third bandwidth $\sigma = 1.0$ in the grid. Before computing, predict where its $d_{\text{eff}}(0.1)$ falls relative to 2.43 (at $\sigma = 2.0$) and 5.11 (at $\sigma = 0.5$), and whether its CV error beats the winner 0.1736; then check with a short numpy script and explain any surprise using the spectrum. Hint

The effective dimension is monotonic in the bandwidth: a smaller σ spreads the spectrum and raises d_{eff} , so $\sigma = 1.0$ should land between 2.43 and 5.11. Cross-validated error need not be monotonic, though, since it trades bias against variance; the point of the check is to see that the best bandwidth is an interior compromise, not an endpoint.

5. challenge Explain precisely why selecting the bandwidth and reporting the error on the same folds gives an optimistically biased estimate, and how nested cross-validation removes the bias. Then count the model fits: for outer K_1 folds, inner K_2 folds, and a grid of G settings, how many times is the base learner trained, and how does that compare with the biased single-loop count? Hint

The grid is chosen to minimize the very quantity you then report, so the minimum of G noisy estimates is biased below the truth, and the bias grows with G . Nested CV grades the selected model on outer folds it never saw during selection. The fit count is $K_1 \cdot G \cdot K_2$ for selection plus K_1 refits, versus $G \cdot K_2$ for the single loop; the extra factor K_1 is the price of an honest estimate.

6. exploration Study how $d_{\text{eff}}(\lambda) = \sum_i \lambda_i / (\lambda_i + \lambda)$ depends on the ridge λ for a fixed Gram matrix, and connect it to Nystrom. (a) Show $d_{\text{eff}}(\lambda) \rightarrow \text{rank}(K)$ as $\lambda \rightarrow 0^+$ and $\rightarrow 0$ as $\lambda \rightarrow \infty$, and argue it decreases in λ . (b) Explain why a Nystrom approximation with $m \approx d_{\text{eff}}(\lambda)$ landmarks loses little accuracy at ridge level λ . Hint
- Each term $\lambda_i / (\lambda_i + \lambda)$ is decreasing in λ , tends to $\mathbf{1}[\lambda_i > 0]$ as $\lambda \rightarrow 0$, and to 0 as $\lambda \rightarrow \infty$. Nystrom approximates the top of the spectrum; the directions it drops have $\lambda_i \ll \lambda$, so they already contribute almost nothing to d_{eff} and almost nothing to the ridge-regularized fit. This is the argument of Rudi et al. (2015).

46 Vector- and Operator-Valued Kernels

Scalar kernels predict one number at a time. Many scientific tasks instead return a vector field, a collection of related responses, or an entire function. Operator-valued kernels preserve the RKHS machinery while allowing the output coordinates to share information. This chapter develops the construction, its representer theorem, and the block systems used in practice.

46.1 From scalar to vector-valued reproduction

Let the output space $\mathcal{Y}$ be a real separable Hilbert space. It may be $\mathbb{R}^q$, an L^2 space of curves, or another function space. Write $\mathcal{L}(\mathcal{Y})$ for its bounded linear operators. An operator-valued kernel assigns an operator, rather than a scalar, to every input pair.

Definition (operator-valued positive definite kernel)

A map $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathcal{L}(\mathcal{Y})$ is Hermitian positive definite when $K(x, z) = K(z, x)^*$ and, for every finite collection (x_i, y_i) ,

$$\sum_{i,j=1}^n \langle y_i, K(x_i, x_j) y_j \rangle_{\mathcal{Y}} \geq 0.$$

Assumptions. The evaluation operators in the induced function space are bounded. **Proof status.** This is a definition; existence and uniqueness of the associated RKHS follow by the operator-valued Moore-Aronszajn construction (Micchelli and Pontil 2005).

The corresponding RKHS $\mathcal{H}_K$ consists of functions $f : \mathcal{X} \rightarrow \mathcal{Y}$. Its reproducing identity is

$$\langle f(x), y \rangle_{\mathcal{Y}} = \langle f, K(\cdot, x)y \rangle_{\mathcal{H}_K}.$$

This identity is the correct replacement for scalar evaluation. It says that every direction y of the output is represented by one section $K(\cdot, x)y$.

46.2 The vector-valued representer theorem

Theorem (finite representer form)

Consider an objective

$$\min_{f \in \mathcal{H}_K} \Phi(f(x_1), \dots, f(x_n)) + \lambda \|f\|_{\mathcal{H}_K}^2,$$

where $\lambda > 0$, Φ depends only on the displayed evaluations, and a minimizer exists. Every minimizer of minimum RKHS norm has the form

$$f(\cdot) = \sum_{i=1}^n K(\cdot, x_i) c_i, \quad c_i \in \mathcal{Y}.$$

Assumptions. K is operator-valued positive definite; point evaluation is bounded; the penalty is strictly increasing in the norm. **Proof status.** Proved by the orthogonal-decomposition argument: the component perpendicular to the span of kernel sections changes no training evaluation and can only increase the penalty (Micchelli and Pontil 2005).

For squared loss with finite-dimensional outputs, stack the c_i and responses into vectors in $\mathbb{R}^{nq}$. The block Gram operator $\mathbf{K}$ has block $\mathbf{K}_{ij} = K(x_i, x_j)$, and the coefficients solve

$$(\mathbf{K} + n\lambda I_{nq})c = y.$$

The same numerical warnings as scalar KRR apply, but the matrix is now nq by nq . A Cholesky factorization therefore costs $O(n^3 q^3)$ if structure is ignored. Matrix-free products, Kronecker identities, and output low rank are not optional when q is large.

46.3 Separable and nonseparable constructions

The simplest construction is separable:

$$K(x, z) = k(x, z)B,$$

where k is a scalar positive definite kernel and B is a positive semidefinite output operator. In $\mathbb{R}^q$, B is a task-similarity matrix. Its eigendirections determine which linear combinations of tasks borrow strength. The block Gram matrix is $K_X \otimes B$, enabling eigendecompositions and fast solves.

Example (two coupled tasks)

Take $B = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$ with $|\rho| \leq 1$. At $\rho = 0$, the two outputs use independent copies of the same scalar kernel. Positive ρ transfers evidence with the same sign; negative ρ transfers it with the opposite sign. As $|\rho|$ approaches one, B becomes ill-conditioned and a solver should use an eigenthreshold or output ridge term.

Verification artifact. checks/example-ch-operator-example-operator-two-task.json records the example source hash and verification scope.

Separable kernels impose the same input geometry on every output eigendirection. A sum

$$K(x, z) = \sum_{r=1}^R k_r(x, z)B_r$$

is more expressive and remains positive definite when every k_r and B_r is positive semidefinite. Nonseparable kernels go further: their output coupling can change with the input pair. They are useful for physical vector fields and structured outputs, but require a direct positivity argument rather than coordinatewise intuition (Álvarez et al. 2012).

46.4 Functional responses and structured outputs

When $\mathcal{Y} = L^2(T)$, the kernel value $K(x, z)$ is itself an integral operator. A common construction uses

$$[K(x, z)g](t) = k(x, z) \int_T b(t, s)g(s) ds.$$

In computation, a basis or discretization of T turns this into a matrix-valued problem. The discretization error and the statistical regularization error are separate quantities and should be reported separately. For structured outputs, operator-valued kernels are most direct when the prediction is a vector of compatible measurements. Combinatorial outputs often need a structured loss or decoding step in addition to the RKHS predictor.

46.5 Learning the output geometry

Fixing B in a separable kernel assumes that task relatedness is known. It can instead be estimated jointly with the predictor:

$$\min_{f, B \succeq 0} \sum_i \ell\{y_i, f(x_i)\} + \lambda \|f\|_{\mathcal{H}_{KB}}^2 + \rho \Omega(B).$$

The regularizer Ω prevents a scale non-identifiability: multiplying B by a constant and compensating elsewhere can leave predictions nearly unchanged while altering the norm. Trace, Frobenius, log-determinant, or low-rank penalties encode different beliefs about task sharing.

A safe parameterization writes $B = LL^\top + \delta I$. It guarantees positive semidefiniteness and exposes a low-rank latent task representation. The parameterization is nonconvex in L , even when optimization over B would be convex. Report the chosen rank, diagonal floor, and sensitivity to initialization.

Proposition (task rotation decouples a separable kernel)

Let $K_X = U\Lambda U^\top$ and $B = V\Sigma V^\top$. In the rotated coordinates $V^\top Y$, squared-loss regression with block kernel $K_X \otimes B$ decomposes into scalar spectral systems with denominators $\lambda_i \sigma_j + n\lambda$.

Assumptions. Finite-dimensional outputs, symmetric positive-semidefinite K_X and B , squared loss, and isotropic ridge regularization. **Proof status.** Proved by the Kronecker eigendecomposition $(U \otimes V)(\Lambda \otimes \Sigma)(U \otimes V)^\top$.

The formula shows when transfer occurs. Output directions with large σ_j are trusted and fit with weaker relative shrinkage. Directions in the null space of B cannot be learned unless a diagonal component or another kernel term supplies them.

46.6 Differentially constrained vector fields

Vector-valued kernels can encode conservation and potential structure. If a smooth scalar kernel is $\psi(x, z)$, differential operators applied to its arguments create matrix-valued kernels. In Euclidean space, Hessian-based constructions can produce curl-free fields, while a complementary projection produces divergence-free fields.

Validity follows by viewing differentiation as a bounded linear operator on the scalar RKHS and applying it to both kernel arguments. Smoothness must be sufficient for every derivative evaluation. Boundary conditions require additional projection or a domain-specific Green kernel; a whole-space divergence-free kernel does not automatically respect a wall boundary.

Physical constraints can be hard, built into the range of the kernel, or soft, added as residual penalties. Hard constraints reduce variance when correct and create structural bias when approximate physics, unresolved forcing,

or discretization violates them. The scientific workflow in Chapter 53, Kernels for Scientific Computing and Operator Learning separates these cases.

46.7 Structured prediction beyond vector regression

A structured output may be a sequence, tree, matching, segmentation, or ranking. A joint feature map $\Psi(x, y)$ defines a compatibility score

$$F_w(x, y) = \langle w, \Psi(x, y) \rangle, \quad \hat{y}(x) = \arg \max_{y \in \mathcal{Y}} F_w(x, y).$$

Kernelization can act on joint input-output pairs through

$$K\{(x, y), (x', y')\} = \langle \Psi(x, y), \Psi(x', y') \rangle.$$

The structured hinge loss compares the observed output with every alternative and adds a task loss $\Delta(y_i, y)$. The resulting optimization may contain exponentially many constraints, so training uses a loss-augmented inference oracle. Positive definiteness of the joint kernel solves representation, not decoding complexity.

Operator-valued regression is appropriate when outputs can be averaged in a Hilbert space. A structured max-margin method is more natural when averaging labels is meaningless and a decoder enforces combinatorial constraints. Hybrid models first predict a vector of sufficient scores and then decode; their consistency depends on both surrogate calibration and decoder correctness.

46.8 Functional responses and discretization

For a functional output, observations may be irregular samples rather than complete curves. Let $S_i : \mathcal{Y} \rightarrow \mathbb{R}^{m_i}$ be the sampling operator for response i . The empirical loss should use $S_i f(x_i)$, not an interpolated curve treated as truth. The representer then contains adjoints S_i^* , and uncertainty should include response-sampling error.

Basis truncation creates three resolutions:

- the input sample size;
- the number of output basis functions;
- the rank used for task coupling.

Increasing one while fixing the others can expose a different bottleneck. Convergence claims must state which limits are coupled and in which output norm error is measured.

46.9 Scalable block solvers

For $K_X \otimes B$, matrix-vector products can be computed as

$$(K_X \otimes B) \text{vec}(C) = \text{vec}(BCK_X^\top),$$

without forming the full block matrix. Eigen-rotations, Kronecker preconditioners, low-rank output factors, and conjugate gradients reduce memory and time. Sums of separable kernels lose a single Kronecker diagonalization but retain structured products.

Nonseparable kernels may need operator-valued random features or block Nyström approximations. An approximation must preserve positive semidefiniteness of the entire block kernel. Approximating entries independently can create a matrix that is symmetric-looking but invalid.

Algorithm (multi-output model-selection workflow)

Input. Multi-output observations, output scales and missingness, candidate scalar kernels, candidate output ranks, and a validation split.

Output. A valid operator-valued predictor with transfer diagnostics.

1. Standardize each output using training data and record missing-response sampling operators.
2. Fit independent scalar baselines before introducing task coupling.
3. Fit a separable kernel, constraining $B \geq 0$ and recording its spectrum.
4. Compare low-rank, diagonal-plus-low-rank, and sums-of-separable alternatives.
5. Evaluate per-task risk, joint risk, calibration where applicable, and negative transfer relative to independent baselines.
6. Report block residuals, preconditioned iteration counts, output rank, and sensitivity to task scaling.

Stop iterative solves by a block residual measured in the original system, not only by change in the low-rank factors.

46.10 Generalization and negative transfer

Capacity depends jointly on the input kernel and the output operator. Trace or operator-norm constraints on B control how much total task complexity is available, while low rank constrains the number of shared latent directions. Rates also depend on output noise covariance and whether tasks share the same design.

More tasks do not automatically improve a target task. Negative transfer occurs when the learned coupling pools incompatible functions or when a high-resource task dominates the loss scale. A credible multi-task result reports every task, compares independent fits, and includes a diagonal B ablation. Average improvement can conceal severe harm to a minority task.

46.11 Common mistakes and practical implications

- Checking each matrix entry of $K(x, z)$ as if it were a scalar kernel does not establish operator-valued positivity.
- A learned B must stay positive semidefinite; unconstrained optimization can silently invalidate the kernel.
- Similar output scales do not imply related tasks. Standardize outputs and validate transfer against independent fits.
- A separable model is a modeling assumption, not merely a computational shortcut.

In practice, begin with $k \otimes B$, inspect the spectrum of B , and compare against q independent scalar models. Move to sums of separable kernels only when held-out evidence supports input-dependent task sharing.

46.12 Summary and further reading

Operator-valued kernels replace scalar similarities by bounded output operators. Positive definiteness produces a vector-valued RKHS, and the representer theorem reduces regularized learning to a block system. Separable kernels expose useful Kronecker structure; nonseparable kernels offer richer coupling at greater verification and computational cost. The foundational treatment is (Micchelli and Pontil 2005), while (Álvarez et al. 2012) connects these kernels to multi-output Gaussian processes.

46.13 Exercises

1. warm-up Prove directly that $K(x, z) = k(x, z)B$ is positive definite when k and B are positive semidefinite.
2. computation For two inputs and two outputs, form the 4×4 block Gram matrix for the linear kernel and $B = \begin{pmatrix} 1 & 1/2 \\ 1/2 & 1 \end{pmatrix}$. Compute its eigenvalues from the Kronecker spectra.
3. proof Complete the orthogonal-decomposition proof of the vector-valued representer theorem, explicitly showing that the perpendicular component vanishes at every training input.
4. synthesis Design a kernel for simultaneously predicting velocity and pressure. State which couplings should be encoded, how positive definiteness will be guaranteed, and how you would detect harmful transfer.

47 Semi-Supervised and Manifold Regularization

Unlabeled observations reveal where probability mass lies, but not automatically where labels change. Manifold regularization makes a precise conditional bet: the target varies smoothly along high-density geometry. A graph Laplacian estimates that geometry, while an ambient RKHS supplies out-of-sample prediction.

47.1 What unlabeled data can and cannot identify

Let l examples carry labels and let u additional examples be unlabeled. The marginal distribution P_X is better observed than the conditional $P_{Y|X}$, but P_X alone cannot determine the labels. Semi-supervised learning therefore requires a compatibility assumption.

Definition (manifold or cluster assumption)

The prediction function is assumed to vary slowly along high-density directions of P_X , and decision boundaries are assumed to avoid those regions. This is a modeling assumption linking P_X to $P_{Y|X}$, not a consequence of having more data.

If two classes overlap throughout the same connected high-density region, unlabeled data can reinforce the wrong smoothness bias. A supervised baseline and a sensitivity analysis over graph construction are therefore essential.

47.2 Graph energy and label propagation

Build a weighted graph on all $n = l + u$ observations, with symmetric weights $W_{ij} \geq 0$, degree matrix $D_{ii} = \sum_j W_{ij}$, and Laplacian $L = D - W$. For the vector $f = (f(x_1), \dots, f(x_n))^T$,

$$f^T L f = \frac{1}{2} \sum_{i,j} W_{ij} (f_i - f_j)^2.$$

Proposition (Laplacian energy)

For symmetric nonnegative W , L is positive semidefinite and $f^T L f = 0$ exactly when f is constant on every connected component.

Assumptions. The graph is finite and undirected. **Proof status.** Proved by expanding the displayed sum of squares; the null-space claim follows edge by edge.

Label propagation minimizes this energy while fixing or penalizing the labeled values. Its output is naturally transductive: it labels the vertices already in the graph. Adding a new point requires rebuilding or extending the graph.

47.3 Manifold regularization

Manifold regularization adds both an ambient and an intrinsic penalty:

$$\min_{f \in \mathcal{H}_k} \frac{1}{l} \sum_{i=1}^l \ell(y_i, f(x_i)) + \gamma_A \|f\|_{\mathcal{H}_k}^2 + \frac{\gamma_I}{n^2} f_X^\top L f_X.$$

The ambient norm controls behavior away from the observed graph and makes evaluation at a new point well defined. The intrinsic term encourages smoothness over the sample geometry (Belkin et al. 2006).

Theorem (manifold representer form)

When $\gamma_A > 0$, every minimum-norm solution has the expansion

$$f(\cdot) = \sum_{i=1}^{l+u} \alpha_i k(x_i, \cdot).$$

Assumptions. k is positive definite, the loss depends on labeled evaluations, and a minimizer exists. **Proof status.** The standard representer proof applies because both penalties depend only on the sample evaluations and the RKHS norm (Belkin et al. 2006).

For squared loss, substituting the expansion yields a convex linear system. Solve it with a symmetric method and matrix-vector products when possible; forming dense K and L costs $O(n^2)$ memory. Sparse k -nearest-neighbor graphs keep the Laplacian product cheap, while Nyström or random features approximate the ambient kernel.

Algorithm (inductive manifold-regularized regression)

1. Standardize features using training data only.
2. Construct a symmetric neighbor graph and record its metric, neighbor count, and weight bandwidth.
3. Form L , choose an ambient kernel, and tune (γ_A, γ_I) using labeled validation data.
4. Solve the coefficient system with residual and condition-number diagnostics.
5. Predict a new x by $f(x) = \sum_i \alpha_i k(x_i, x)$.
6. Compare with the supervised model $\gamma_I = 0$ and report whether unlabeled data helped.

47.4 Consistency and graph choices

The graph Laplacian is not a fixed population operator. Its limit depends on neighbor radius, bandwidth, normalization, sampling density, and manifold regularity. Unnormalized, random-walk, and symmetric normalized Laplacians encode different geometries; they should not be exchanged without changing the theorem being invoked (Luxburg 2007). Consistency statements require coupled limits in which sample size grows and the graph scale shrinks at an admissible rate.

47.5 Harmonic extension and label propagation

Partition graph vertices into labeled and unlabeled sets and write the Laplacian in blocks:

$$L = \begin{pmatrix} L_{\ell\ell} & L_{\ell u} \\ L_{u\ell} & L_{uu} \end{pmatrix}.$$

Fixing labeled values $f_\ell = y$ and minimizing $f^\top L f$ over f_u gives

$$L_{uu}f_u = -L_{ue}y.$$

Proposition (graph harmonic extension)

If every connected component containing an unlabeled vertex also contains a labeled vertex, then L_{uu} is positive definite and the harmonic extension is unique. At each unlabeled vertex, its value is a weighted average of its neighbors.

Assumptions. A finite undirected graph with nonnegative weights and at least one labeled vertex in every relevant component. **Proof status.** Proved by the graph-energy identity and the null-space characterization of the Laplacian.

The averaging identity follows from $(Lf)_i = 0$. It gives a random-walk interpretation: under suitable absorbing-boundary conventions, the prediction is an average of labels weighted by hitting probabilities. A disconnected unlabeled component makes the solution unidentified, not merely uncertain.

Soft label propagation penalizes disagreement with labels rather than fixing them. Normalized variants change the averaging geometry and class-mass behavior. Their names are similar, but their fixed points and population limits differ.

47.6 Laplacian regularized least squares and SVMs

Substitute $f_X = K\alpha$ into the manifold objective. With squared loss on labeled vertices and selector matrix J , one obtains an objective proportional to

$$\|y - JK\alpha\|^2 + \gamma_A \alpha^\top K\alpha + \gamma_I \alpha^\top K L K \alpha.$$

Its normal equations contain the ambient Gram geometry and the graph geometry in different positions. Symmetrizing and preconditioning the system matters because KLK can be badly scaled relative to K .

For a hinge loss, the same intrinsic penalty augments an SVM and produces a Laplacian SVM. The dual is no longer the ordinary SVM dual with a new kernel substituted casually; the regularizer changes the effective inverse geometry. A robust implementation derives the finite problem from the stated primal and verifies the KKT residual.

47.7 When unlabeled data help or hurt

Unlabeled inputs estimate P_X , while prediction needs $P_{Y|X}$. Improvement is possible only through an assumption connecting them. Three common connections are:

- decision boundaries pass through low-density regions;
- labels are nearly constant on graph communities;
- the target is smooth in the intrinsic manifold metric.

Counterexamples are easy. If labels alternate rapidly along a well-sampled manifold, intrinsic smoothing erases signal. If a narrow bridge connects classes, graph construction can propagate labels across it. If sampling density varies for reasons unrelated to the outcome, an unnormalized Laplacian can impose density-dependent bias.

The correct baseline is $\gamma_I = 0$ with the same labeled data and ambient kernel. Report the distribution of improvements across label budgets and graph perturbations, not only the best graph.

47.8 Graph limits and density bias

For data sampled on a smooth manifold, a properly scaled graph Laplacian can converge to a differential operator. The limiting operator depends on the kernel bandwidth, graph normalization, sampling density, boundary, and manifold dimension. Shrinking bandwidth reduces geometric bias but increases variance and can disconnect the graph.

The order of limits matters. Keeping a fixed bandwidth as sample size grows estimates a nonlocal integral operator, not the Laplace-Beltrami operator. Shrinking it too quickly produces a noisy graph. Estimating intrinsic dimension and density adds further uncertainty.

Consistency of spectral clustering, label propagation, or manifold regularization needs more than pointwise convergence of graph weights. One must control eigenvectors or resolvents and then propagate that error through the learning algorithm (Luxburg 2007).

47.9 Out-of-sample extensions

Pure harmonic extension is transductive. Several inductive strategies are available:

1. Manifold regularization learns an ambient RKHS function over all labeled and unlabeled training points.
2. A Nyström extension connects a new point to the existing graph and extends graph eigenfunctions.
3. A parametric or neural map learns from graph coordinates to predictions.
4. A new graph is constructed for each deployment batch.

Each strategy defines a different estimator. Nyström extension assumes the new point lies within the sampled geometry. Rebuilding a graph changes old predictions and complicates reproducibility. An ambient RKHS extrapolates off the graph according to its own kernel, where the intrinsic penalty supplies no direct information.

47.10 Scaling sparse graphs

Sparse neighbor graphs allow Lf in time proportional to the number of edges. The ambient kernel may still be dense. Random features yield a primal problem with graph penalty $Z^T LZ$, while Nyström features yield a smaller landmark system. Approximate nearest-neighbor search changes the graph and therefore the estimator, not only runtime.

Algorithm (large-scale manifold regularization)

Input. Labeled and unlabeled inputs, graph and ambient-kernel budgets, validation labels, and approximation tolerances.

Output. An inductive predictor and graph-sensitivity report.

1. Build a sparse mutual-neighbor graph and record connectivity and approximate-neighbor recall.
2. Normalize features using training data and construct low-rank ambient features.
3. Form graph-feature products without materializing dense L or K .
4. Solve the convex system by preconditioned iteration and monitor the original residual.
5. Tune graph scale, ambient regularization, and intrinsic regularization on labeled validation data only.
6. Compare with supervised, transductive, and graph-free low-rank baselines.

Complexity is governed by graph edges, feature rank, and solver iterations. Stop when the system residual and held-out labeled error are stable across consecutive checkpoints.

47.11 Common mistakes and practical implications

- Selecting graph hyperparameters using all labels leaks validation information.
- A disconnected graph propagates no information between components.
- Very wide graphs collapse geometry toward a global smoothness penalty; very narrow graphs are unstable.
- Transductive performance does not establish accurate out-of-sample prediction.
- More unlabeled data can hurt when the manifold assumption is false.

Report graph connectivity, degree distribution, bandwidth, scaling method, and the supervised ablation. For large data, use sparse graphs and matrix-free solvers; do not materialize a dense Laplacian merely because the ambient kernel is dense.

47.12 Summary and further reading

Graph energy translates a geometry estimate into a smoothness penalty. Pure graph methods are transductive; manifold regularization combines that penalty with an RKHS norm to obtain an inductive function. Its gains are conditional on a relationship between marginal geometry and labels. The primary framework is (Belkin et al. 2006), and (Luxburg 2007) explains the spectral graph choices behind it.

47.13 Exercises

1. warm-up Show that the multiplicity of eigenvalue zero of L equals the number of connected components.
2. computation For a path of three vertices with endpoint labels $+1$ and -1 , minimize Laplacian energy over the middle value and interpret the result.
3. proof Derive the representer expansion by decomposing f into the span of $k(x_i, \cdot)$ and its orthogonal complement.
4. exploration Construct a two-moons experiment in which unlabeled data helps, then rotate the labeling rule so the same geometry becomes misleading. State a rubric based on supervised ablation, graph sensitivity, and held-out error.

48 Inverse Learning and Spectral Regularization

Kernel regression solves an inverse problem: recover a function after a compact operator has attenuated its high-frequency directions. Regularization is therefore best understood as a spectral filter. This viewpoint unifies ridge regression, truncated eigensolvers, gradient descent, and early stopping while making their limits explicit.

48.1 Learning as an inverse problem

For a distribution P_X , define the covariance operator $T : \mathcal{H}_k \rightarrow \mathcal{H}_k$ by

$$Tf = \int k_x \langle k_x, f \rangle_{\mathcal{H}_k} dP_X(x),$$

where $k_x = k(x, \cdot)$. The population normal equation is $Tf = g$. When T is compact, its eigenvalues μ_j approach zero. Direct inversion multiplies error in the j -th coordinate by $1/\mu_j$, so small empirical perturbations can dominate the solution (Rosasco et al. 2004).

Definition (spectral regularization family)

A family $g_\lambda : [0, \kappa^2] \rightarrow \mathbb{R}$ defines the estimator $f_\lambda = g_\lambda(T)g$. Its residual function is $r_\lambda(\mu) = 1 - \mu g_\lambda(\mu)$. A regularization family keeps g_λ bounded for fixed λ and makes $r_\lambda(\mu)$ tend to zero for every positive μ as λ tends to zero.

Tikhonov regularization uses $g_\lambda(\mu) = 1/(\mu + \lambda)$. Spectral cutoff uses $g_\lambda(\mu) = \mu^{-1} \mathbf{1}\{\mu \geq \lambda\}$. Both approach inversion, but one damps continuously and the other deletes weak directions.

48.2 Iterative regularization and early stopping

Gradient descent on squared loss, initialized at zero with step η , gives after t iterations

$$g_t(\mu) = \frac{1 - (1 - \eta\mu)^t}{\mu}, \quad r_t(\mu) = (1 - \eta\mu)^t.$$

Thus the iteration count is a regularization parameter. Large eigenvalues are learned first; small ones remain damped until later. This is iterative regularization, not explicit ridge. The two filters can have comparable effective scales, but they are not algebraically identical.

Proposition (Landweber stability)

If $0 < \eta \leq 1/\|T\|$, then $0 \leq r_t(\mu) \leq 1$ on the spectrum of T , and $r_t(\mu)$ decreases to zero for every $\mu > 0$.

Assumptions. T is bounded, self-adjoint, and positive. **Proof status.** Proved by scalar functional calculus: $1 - \eta\mu$ lies in $[0, 1]$.

Data-dependent stopping can balance bias and noise. A discrepancy rule stops when the residual reaches an estimate of the observation noise. Cross-validation is more robust when the noise model is unknown, but its search range and reuse of validation data must be reported.

48.3 Source conditions, qualification, and saturation

A source condition writes the target as

$$f_{\star} = T^r w, \quad \|w\| \leq R.$$

Larger r means the target is better aligned with stable, high-eigenvalue directions. The bias is controlled through $\mu^r r_{\lambda}(\mu)$. A filter has qualification q when this quantity is bounded at the expected regularization scale for source exponents up to q . Beyond that range, additional target smoothness may not improve the method's rate; this ceiling is called saturation.

Proof-status warning

No standalone convergence rate follows from a source condition. A valid rate also needs assumptions on eigenvalue or effective-dimension decay, noise, sampling, and the rule selecting λ or t . Statements may be upper bounds, lower bounds, or minimax equivalences, and those statuses must remain separate (Caponnetto and De Vito 2007).

48.4 A finite spectral diagnostic

Example (three spectral directions)

Suppose the Gram eigenvalues are 10, 1, 0.01, with ridge parameter $\lambda = 0.1$. The fitted-value filter $\mu/(\mu + \lambda)$ is approximately 0.990, 0.909, 0.091. The weak third direction is strongly suppressed. Direct inversion would instead amplify its coefficient by 100. A numerical report should show both the data-fit residual and this spectral amplification profile.

Verification artifact. `checks/example-ch-inverse-example-inverse-filter.json` records the example source hash and verification scope.

Algorithm (matrix-free early-stopped kernel regression)

1. Implement the product $v \mapsto Kv$ without requiring a stored dense matrix when possible.
2. Estimate an upper spectral bound and choose a stable step size.
3. Iterate the residual update from zero coefficients.
4. At checkpoints, record training residual, validation error, and coefficient norm.
5. Stop by a preregistered discrepancy or validation rule; retain the entire trace for reproducibility.
6. Verify the final linear-system residual and compare against a ridge baseline at matched effective degrees of freedom.

48.5 A catalog of spectral filters

Regularization methods can be compared by their fitted-value filter $q_{\lambda}(\mu) = \mu g_{\lambda}(\mu)$ and residual $r_{\lambda} = 1 - q_{\lambda}$.

Method	Inverse filter g	Main behavior
Tikhonov	$(\mu + \lambda)^{-1}$	smooth shrinkage
Iterated Tikhonov	repeated resolvent product	higher qualification
Spectral cutoff	$\mu^{-1} \mathbf{1}\{\mu \geq \lambda\}$	hard deletion
Landweber	$\{1 - (1 - \eta\mu)^t\}/\mu$	early-stopped iteration
Gradient flow	$\{1 - \exp(-t\mu)\}/\mu$	continuous-time shrinkage

Cutoff has excellent bias behavior on retained directions but is discontinuous in the estimated spectrum. Tikhonov is stable and convenient but saturates for sufficiently smooth source conditions. Iteration can increase qualification, although numerical and sampling errors accumulate.

Accelerated first-order methods produce polynomial filters. Their optimization acceleration does not automatically imply better statistical regularization: oscillating residual polynomials can amplify noisy directions. Every accelerated method needs a spectral stability analysis together with its stopping rule.

48.6 Conjugate gradients as regularization

Conjugate gradients applied to the normal equations chooses, at iteration t , the best solution in a data-dependent Krylov space. Its filter is a polynomial whose roots depend on the observed spectrum. Large, well-estimated eigendirections are often resolved early.

Unlike Landweber, conjugate-gradient filters are data dependent and not monotone direction by direction. The method can converge rapidly in linear-system residual while beginning to fit noise in weak directions. Early stopping therefore remains necessary.

Preconditioning changes the spectrum and the Krylov subspace. If stopping time is selected after preconditioning, the preconditioner is part of the statistical estimator. Matching two methods by iteration count is meaningless; match effective degrees of freedom, residual scale, or validation risk.

48.7 Choosing the regularization level

Parameter-choice rules use different information:

- **Discrepancy principle:** stop when the data residual reaches a known noise scale.
- **Hold-out or cross-validation:** select predictive error on untouched observations.
- **Generalized cross-validation:** exploit linear smoother structure and approximate leverage.
- **Balancing or Lepskii rules:** compare estimators across scales and stop when changes become compatible with noise.
- **Marginal likelihood:** choose a probabilistic variance ratio under a GP model.

No rule is universally adaptive. A discrepancy principle is sensitive to noise estimation and forward-model error. Random cross-validation can violate dependence. Marginal likelihood targets its model evidence, not an arbitrary downstream loss.

Proposition (monotone discrepancy crossing for Landweber)

For a positive self-adjoint empirical operator, stable Landweber step size, and zero initialization, the training residual norm is nonincreasing with iteration. Hence the first crossing of a fixed discrepancy threshold is well defined whenever the limiting residual lies below that threshold.

Assumptions. Exact arithmetic, a fixed positive self-adjoint operator, step size in the stable interval, and a reachable threshold. **Proof status.** Proved by diagonalizing the residual update and observing that each spectral residual magnitude is multiplied by a factor in the unit interval.

Model discrepancy can prevent the residual from reaching the nominal noise scale. The correct response is not unlimited iteration; it is to revise the noise or forward model.

48.8 Statistical rates and effective dimension

A source condition controls bias, while the effective dimension

$$\mathcal{N}(\lambda) = \text{tr}\{T(T + \lambda I)^{-1}\}$$

controls variance for Tikhonov-type methods. Under eigenvalue decay and noise assumptions, risk is bounded by a bias term depending on r and a variance term involving $\mathcal{N}(\lambda)/n$. Balancing them selects the statistical scale (Caponnetto and De Vito 2007).

Upper rates should state whether they are in prediction norm, RKHS norm, or another interpolation norm. A minimax claim additionally needs a matching lower bound over a specified source and capacity class. Adaptivity means achieving the rate without knowing those exponents; selecting the best rate after observing the test set is not adaptivity.

48.9 Stochastic and online iterative regularization

Stochastic gradient methods introduce an algorithmic noise source in addition to observation noise. Step size, mini-batch size, averaging, sampling with or without replacement, and number of passes jointly define the filter-like behavior. Early stopping can regularize, but there may be no single deterministic polynomial filter.

Multiple passes reduce optimization bias and can eventually fit observation noise. Tail averaging and decaying step sizes change both variance and implicit weighting of iterations. Compare stochastic and deterministic methods at matched kernel products or wall time, and record the full learning curve.

48.10 Beyond scalar regression

The same framework applies to:

- conditional mean embeddings, where a covariance operator is inverted;
- kernel instrumental variables and proximal causal equations;
- vector-valued regression with block covariance operators;
- scientific inverse problems with linearized forward maps;
- deconvolution and inverse source recovery.

In each case, compactness and partial identification determine which directions can be recovered. Regularization selects a stable solution but cannot manufacture information in the operator null space. The scientific treatment in Chapter 53, *Kernels for Scientific Computing and Operator Learning* distinguishes operator discretization from statistical noise.

Algorithm (spectral regularizer comparison)

Input. A matrix-free empirical operator, observations, a noise estimate or validation set, candidate filters, and a compute budget.

Output. A selected estimator with filter and stability diagnostics.

1. Estimate spectral bounds and verify self-adjoint positive products.
2. Run ridge, Landweber, and conjugate-gradient paths at matched operator-product checkpoints.
3. Record residual, validation loss, solution norm, effective degrees of freedom where available, and spectral amplification.
4. Apply a preregistered discrepancy or validation rule without test reuse.
5. Perturb observations at the estimated noise scale and measure solution sensitivity.

6. Report the selected filter, parameter or stopping time, preconditioner, arithmetic precision, and final residual.

The solve stops only when the selection rule is met or the compute budget is exhausted; either outcome is part of the result.

48.11 Common mistakes and practical implications

- Early stopping is a regularizer only together with an initialization, step-size schedule, and stopping rule.
- A small training residual does not certify a stable inverse.
- Source conditions constrain the target relative to T ; they are not generic smoothness labels.
- Qualification is filter-specific, and saturation is not universal across algorithms.
- Finite precision creates an additional cutoff near machine accuracy.

Matrix-free iteration can reduce memory and exploit fast kernel products. Preconditioning changes convergence across spectral directions, so its interaction with early stopping must be treated as part of the estimator rather than an invisible implementation detail.

48.12 Summary and further reading

Compact covariance operators turn learning into an ill-posed inverse problem. Spectral filters stabilize the inverse by damping weak directions. Ridge, cutoff, and iterative methods use different filters; source conditions and qualification explain their bias, while noise and effective dimension govern variance. See (Rosasco et al. 2004) for the learning-to-inverse-problem bridge, (Caponnetto and De Vito 2007) for statistical rates, and (Raskutti et al. 2014) for early stopping.

48.13 Exercises

1. warm-up Plot the ridge residual $r_\lambda(\mu)$ for three values of λ and explain which spectral directions remain biased.
2. computation Compute the first five Landweber fitted-value filters for $\mu \in \{1, 0.1, 0.01\}$ with $\eta = 1/2$.
3. proof Prove the Landweber stability proposition and show what can fail when $\eta > 2/\|T\|$.
4. challenge For a source condition $f_\star = T^r w$, derive a uniform ridge-bias bound from $\sup_\mu \mu^r \lambda / (\mu + \lambda)$. State the range of r for which your bound has the asserted power of λ .

49 Deep Kernel Learning

Deep kernel learning composes a trainable representation with a positive definite base kernel, then performs Gaussian-process or kernel inference on top. It can learn task-specific geometry while retaining predictive covariance, but joint optimization creates identifiability, calibration, and approximation failures that a fixed-kernel analysis does not expose.

49.1 The model and its distinction from NTK

Let $h_\theta : \mathcal{X} \rightarrow \mathbb{R}^p$ be a neural feature map and let k_ϕ be a positive definite base kernel. Define

$$k_{\theta,\phi}(x, z) = k_\phi(h_\theta(x), h_\theta(z)).$$

For every fixed (θ, ϕ) , this is positive definite by composition. Training changes θ , and therefore changes the geometry itself.

Definition (deep kernel learning)

Deep kernel learning jointly estimates a parametric feature map and the hyperparameters of a kernel or Gaussian-process readout, usually by optimizing marginal likelihood or a predictive objective (Wilson and Xing 2016).

This is not the neural tangent kernel limit. In standard NTK analysis, the tangent kernel becomes effectively fixed and training is described by kernel gradient flow. In DKL, movement of h_θ is the point of the method. Nor is DKL the same as a deep Gaussian process, which places stochastic processes at multiple layers and integrates over intermediate functions.

49.2 Marginal-likelihood training

For Gaussian regression, let $C = K_{\theta,\phi} + \sigma^2 I$. The negative log marginal likelihood, up to an additive constant, is

$$\mathcal{L}(\theta, \phi, \sigma) = \frac{1}{2} y^\top C^{-1} y + \frac{1}{2} \log \det C.$$

The first term rewards data fit; the log determinant penalizes covariance volume. Their competition is sometimes called an automatic Occam effect, but it does not guarantee calibrated predictions under misspecification or approximate inference.

Proposition (covariance-parameter gradient)

For any scalar parameter ψ entering C ,

$$\frac{\partial \mathcal{L}}{\partial \psi} = \frac{1}{2} \operatorname{tr} \left[(C^{-1} - \alpha \alpha^\top) \frac{\partial C}{\partial \psi} \right], \quad \alpha = C^{-1} y.$$

Assumptions. C is differentiable in ψ and strictly positive definite. **Proof status.** Proved using the derivative of a quadratic inverse and $\partial \log \det C = \operatorname{tr}(C^{-1} \partial C)$.

The representation can shrink pairwise distances while the base length scale expands, creating non-identifiability. Constrain or regularize feature scales and log both feature norms and kernel hyperparameters.

49.3 Exact and approximate computation

Exact training requires an $n \times n$ factorization, $O(n^3)$ time and $O(n^2)$ memory. Inducing-point methods replace the full process by m representative locations, often costing $O(nm^2 + m^3)$. Structured kernel interpolation places inducing points on a grid and interpolates feature-space covariances, enabling fast matrix-vector products when the learned representation and grid retain the required structure (Wilson and Nickisch 2015).

Algorithm (auditable DKL training)

1. Split data before fitting preprocessing, features, inducing locations, or kernel hyperparameters.
2. Initialize a feature map and a strictly positive observation-noise parameter.
3. Optimize marginal likelihood with jitter, gradient clipping, and logged condition estimates.
4. Evaluate predictive log density, calibration, and task loss on untouched data.
5. Compare against a fixed base-kernel GP and a neural point predictor of similar capacity.
6. Repeat over seeds and report approximation settings, wall time, peak memory, and failures.

49.4 Calibration and failure modes

Example (representation collapse)

If $h_\theta(x)$ maps many distinct inputs to nearly the same point, an RBF readout assigns them nearly identical covariance. Training likelihood can still improve through noise and mean parameters, while uncertainty away from the training set becomes misleading. Monitor pairwise feature distances and evaluate shifted inputs, not just interpolation error.

Verification artifact. checks/example-ch-dkl-example-dkl-collapse.json records the example source hash and verification scope.

Other failures include vanishing learned noise, unstable Cholesky factors, inducing points that cover only dense modes, and overconfident extrapolation. Approximate objectives can distort posterior variances even when point predictions look strong. Deep-kernel model selection should therefore include proper scoring rules and coverage curves rather than RMSE alone. Empirical work has shown that DKL uncertainty behavior is sensitive to architecture and training choices (Ober et al. 2021).

49.5 Non-Gaussian likelihoods

Classification, counts, censored responses, and ordinal outcomes replace the Gaussian likelihood by a nonconjugate one. The posterior over latent values is no longer Gaussian, so Laplace, expectation propagation, variational inference, or sampling is required. Feature learning and posterior approximation then interact: a representation can change to make the chosen approximation look favorable rather than to improve the exact predictive distribution.

For classification, a variational objective has the schematic form

$$\mathbb{E}_q \log p(y | f) - \text{KL}(q \| p_{\theta, \phi}),$$

where the prior depends on learned features. Mini-batch training is possible when the expected log likelihood decomposes, but inducing and feature parameters receive noisy coupled gradients. Report the likelihood, link, quadrature or Monte Carlo estimator, and variance-reduction method.

Class probabilities should be assessed with log loss, Brier score, reliability diagrams, and decision-specific utility. Accuracy alone cannot diagnose a collapsed posterior or a representation that makes all logits extreme.

49.6 Variational inducing-point DKL

Let inducing variables $u = f(Z)$ live at locations Z in learned feature space. A sparse variational GP optimizes a lower bound over $q(u)$, kernel parameters, inducing locations, and network parameters. The inducing set is not merely a computational cache: it determines which posterior directions can be represented.

Three design choices must be distinguished:

1. inducing inputs in original input space, transformed by h_θ ;
2. free inducing locations directly in latent feature space;
3. structured grids used for interpolation.

Free latent inducing points may not correspond to any realizable input. That is acceptable for algebra but complicates interpretation and shift diagnostics. A structured grid assumes learned features remain within and sufficiently cover the interpolation domain (Wilson and Nickisch 2015).

The evidence lower bound can improve while predictive variance deteriorates if the variational family is too restrictive. Increase inducing rank until proper scores and coverage stabilize, not only until training loss stops improving.

49.7 Identifiability and geometry control

For an RBF readout, multiplying all features by a and multiplying the length scale by the same factor leaves pairwise normalized distances unchanged. Neural weights, feature normalization, output scale, kernel amplitude, and observation noise create further symmetries or near-symmetries.

Proposition (feature-length-scale invariance)

For an RBF base kernel

$$k_\ell\{h(x), h(z)\} = \sigma_f^2 \exp \left\{ -\frac{\|h(x) - h(z)\|^2}{2\ell^2} \right\},$$

the simultaneous transformation $h \mapsto ah$, $\ell \mapsto |a|\ell$ leaves the covariance unchanged for nonzero a .

Assumptions. The base kernel is the displayed isotropic RBF and no other model component depends on the absolute feature scale. **Proof status.** Proved by direct substitution.

This flat direction harms optimization diagnostics and makes raw parameter values uninterpretable. Feature normalization, explicit priors, norm penalties, or fixed length scales can break it. The remedy changes the model and should be included in ablations.

49.8 Derivatives, invariances, and structured inputs

Because the composed kernel differentiates through h_θ , DKL can incorporate derivative observations when both network and base kernel are smooth enough. The derivative covariance includes Jacobians of the representation, so unstable network derivatives can create ill-conditioned blocks even when feature values look benign.

Domain structure can enter before or inside the learned map: convolution for images, message passing for graphs, equivariant layers for symmetry, and sequence encoders for text or paths. Positive definiteness of the readout remains automatic for fixed features, but data leakage, invariance errors, and representation collapse remain possible.

For multiple outputs, combine a learned input map with the operator-valued kernels in Chapter 45, Vector- and Operator-Valued Kernels. Learning both input and output geometry creates strong non-identifiability; independent-output and fixed-feature baselines are mandatory.

49.9 OOD uncertainty and conformal calibration

A stationary base kernel in learned feature space measures distance after the network transformation. Inputs far apart in raw space can map nearby, so GP variance need not increase under semantic or covariate shift. Training likelihood rarely constrains behavior in regions without data.

OOD evaluation should include:

- synthetic directions that move away from the training support;
- realistic source or time shifts;
- feature-space nearest-neighbor distances;
- predictive entropy and proper scores;
- interval or set coverage after calibration;
- comparison with a fixed kernel and a deterministic ensemble.

Conformal prediction from Chapter 51, Distribution Shift, Robustness, and Conformal Prediction can calibrate marginal coverage on representative data. It does not repair unsupported feature mappings, and its exchangeability assumptions can fail under the very shifts being tested.

49.10 Comparing DKL with neighboring models

DKL sits among several models:

- **fixed deep features plus GP**: isolates the value of joint feature learning;
- **NTK or NNGP**: uses a fixed infinite-width kernel and convex readout dynamics;
- **deep GP**: composes stochastic functions and integrates intermediate uncertainty;
- **neural ensemble**: learns features without a GP covariance interpretation;
- **neural operator**: maps functions to functions, often across discretizations;
- **ordinary GP**: tests whether learned geometry is necessary at all.

Match training data, validation budget, preprocessing, and compute. If DKL receives a pretrained network, report that data and cost. If baselines do not, the comparison measures pretraining plus DKL rather than the readout alone.

Algorithm (DKL stress-test protocol)

Input. In-distribution data, at least one declared shift, fixed and learned-feature baselines, and a compute budget.

Output. Accuracy, calibration, geometry, and failure diagnostics.

1. Fit preprocessing on training data and preserve untouched calibration and shifted test sets.
2. Train exact or variational DKL while logging feature norms, pairwise distances, noise, jitter, and condition estimates.
3. Increase inducing rank or interpolation resolution until predictive scores stabilize.
4. Compare joint training with frozen features, random features, a fixed GP, and a deterministic predictor.
5. Evaluate proper scores, coverage, width, accuracy, OOD ranking, seed variability, wall time, and memory.
6. Inspect collapsed or saturated feature directions and report failed factorizations and recovery actions.

Optimization stops on validation proper score and stable geometry diagnostics, not training marginal likelihood alone.

49.11 Common mistakes and practical implications

- Positive definiteness of the composed kernel does not imply a well-conditioned Gram matrix.
- Marginal likelihood is an objective, not proof that the model is correctly specified.
- Test inputs must not influence feature normalization or inducing-point selection.
- Comparing DKL only with a weak fixed kernel does not isolate the value of uncertainty modeling.
- A feature-learning model should not be described with fixed-NTK conclusions.

Use double precision for kernel linear algebra when conditioning is difficult, constrain noise away from zero, and record the jitter actually added. Evaluate both in-distribution and shifted data. If uncertainty is not a product requirement, compare the extra complexity against a deterministic representation plus calibrated residual model.

49.12 Summary and further reading

DKL learns a representation inside a valid kernel and uses probabilistic or kernel inference on the transformed inputs. Marginal likelihood balances fit and covariance complexity but introduces coupled optimization. Scalable approximations change both computation and uncertainty, so calibration, conditioning, and ablations are first-class checks. The original scalable construction is (Wilson and Xing 2016), with structured interpolation in (Wilson and Nickisch 2015) and a focused empirical analysis in (Ober et al. 2021).

49.13 Exercises

1. warm-up Prove that $k(h_\theta(x), h_\theta(z))$ is positive definite for fixed θ .
2. computation For a 2×2 covariance matrix, compute both marginal-likelihood terms and their change when observation noise doubles.
3. proof Derive the covariance-parameter gradient proposition from matrix differential identities.
4. synthesis Design an evaluation comparing DKL, a fixed-kernel GP, an NTK predictor, and a neural ensemble. Specify accuracy, calibration, shift, compute, and seed reporting criteria.

50 Smoothing Splines and Additive RKHS Models

Long before the phrase kernel machine became standard, statisticians estimated smooth functions by penalizing their derivatives. The solution was a spline, and its apparently special finite form was an instance of the representer theorem. This chapter develops that classical route, explains the unpenalized null space that ordinary kernel ridge regression often hides, and extends the construction to additive models with interpretable interactions.

50.1 From roughness to a finite estimator

Suppose observations satisfy $y_i = f^\star(x_i) + \varepsilon_i$ on an interval. Interpolation alone is unstable because a function can pass through every observation while oscillating violently between them. A smoothing spline balances fit against a differential measure of roughness:

$$\min_{f \in \mathcal{V}} \frac{1}{n} \sum_{i=1}^n \{y_i - f(x_i)\}^2 + \lambda J(f), \quad J(f) = \int \{D^m f(t)\}^2 dt.$$

The derivative order m specifies which behavior is considered rough. The tuning parameter λ controls the tradeoff: small λ follows the observations, while large λ drives the solution toward functions annihilated by D^m .

Definition (penalty null space)

The null space of a seminorm penalty J is

$$\mathcal{N}_J = \{f \in \mathcal{V} : J(f) = 0\}.$$

For the integrated squared m -th derivative, $\mathcal{N}_J$ is the space of polynomials of degree strictly smaller than m , subject to the chosen domain and boundary convention.

This null space is not a nuisance. It identifies the trends that should remain unpenalized. Penalizing an intercept or a linear trend merely because they appear in a coefficient vector changes the statistical model.

50.2 The spline representer theorem

Decompose the function space as $\mathcal{V} = \mathcal{N}_J \oplus \mathcal{H}_J$, where $\mathcal{H}_J$ is an RKHS on which $J(f) = \|f\|_{\mathcal{H}_J}^2$. Let $p_1, \dots, p_q$ span the null space and let R be the reproducing kernel of $\mathcal{H}_J$.

Theorem (smoothing-spline representer form)

Every minimizer of the squared-error and roughness objective has a representative of the form

$$\hat{f}(x) = \sum_{a=1}^q d_a p_a(x) + \sum_{i=1}^n c_i R(x_i, x).$$

When the loss is strictly convex on the observed values and the usual side constraints identify the null-space component, the fitted values are unique.

Assumptions. Evaluation is continuous on $\mathcal{H}_J$; $\mathcal{N}_J$ is finite dimensional; a minimizer exists; the design identifies the null-space basis. **Proof status.** Proved by decomposing the penalized component into the span of $R(x_i, \cdot)$ and its orthogonal complement. The complement changes no observation and only increases J (Kimeldorf and Wahba 1971; Wahba 1990).

With $K_{ij} = R(x_i, x_j)$ and $P_{ia} = p_a(x_i)$, the coefficients solve a saddle system such as

$$\begin{pmatrix} K + n\lambda I & P \\ P^\top & 0 \end{pmatrix} \begin{pmatrix} c \\ d \end{pmatrix} = \begin{pmatrix} y \\ 0 \end{pmatrix}.$$

The constraint $P^\top c = 0$ prevents the penalized kernel expansion from duplicating the unpenalized trend. A stable implementation should factor the null space first, solve in its orthogonal complement, and report the condition number after scaling the inputs.

50.3 Green functions, splines, and kernel ridge regression

The penalty operator and its boundary conditions determine a Green function. That Green function becomes the reproducing kernel of the penalized space. The construction in Mercer's theorem, spectra, and rates therefore has a differential counterpart: the eigenfunctions of the boundary-value problem define the smoothing directions, and the inverse eigenvalues determine how strongly each direction is penalized (Green 1984).

If the null space is absent or has already been projected out, the spline estimator is kernel ridge regression:

$$\hat{f}(x) = k_x^\top (K + n\lambda I)^{-1} y.$$

The distinction is conceptual rather than cosmetic. A generic positive-definite kernel penalizes every RKHS direction. A spline seminorm leaves selected low-order trends free. This is why the extrapolation of a natural spline can differ sharply from that of an RBF kernel even when both interpolate the observed region well.

Example (the null space controls extrapolation)

Consider two estimators with similar in-sample residuals. One uses a derivative seminorm whose null space contains affine functions; the other uses a strictly positive-definite stationary kernel with a zero mean. Beyond the observations, the first estimator tends toward its fitted affine component, while the second tends toward its prior mean as correlations decay. The difference is a consequence of the function-space decomposition, not merely a different bandwidth.

Verification artifact. checks/example-ch-splines-example-spline-nullspace.json records the example source hash and verification scope.

50.4 The smoother matrix and effective complexity

For fixed design and tuning parameters, a squared-loss spline is a linear smoother:

$$\hat{y} = S_\lambda y.$$

The eigenvalues of S_λ lie between zero and one under the standard construction. Directions with eigenvalues near one are retained; directions near zero are suppressed. The quantity

$$\text{df}(\lambda) = \text{tr}(S_\lambda)$$

is the effective degrees of freedom. It is generally not the number of nonzero coefficients. It measures how much the fitted vector responds, in total, to perturbations of the observations.

Proposition (leave-one-out residual identity)

For a linear smoother whose matrix does not depend on y , the leave-one-out residual at observation i is

$$\frac{y_i - \hat{y}_i}{1 - S_{\lambda,ii}},$$

whenever the denominator is nonzero.

Assumptions. The fitted values are linear in y , and deleting one observation leaves the same smoothing rule applied to the reduced problem. **Proof status.** Proved by the rank-one deletion identity; the result is the spline analogue of the KRR identity in Chapter 4, Kernel Ridge Regression and Smooth Losses (Craven and Wahba 1979).

Generalized cross-validation replaces the individual diagonal values by their average:

$$\text{GCV}(\lambda) = \frac{n^{-1} \|(I - S_\lambda)y\|^2}{\{1 - n^{-1} \text{tr}(S_\lambda)\}^2}.$$

GCV is rotation invariant in observation space and inexpensive once the trace is available. It is not immune to dependence, heteroscedasticity, or model selection bias. Those conditions require blocked validation, weighted criteria, or a likelihood-based alternative.

50.5 Smoothing-spline ANOVA

For multivariate inputs $x = (x_1, \dots, x_d)$, an additive decomposition separates main effects and interactions:

$$f(x) = \mu + \sum_j f_j(x_j) + \sum_{j < k} f_{jk}(x_j, x_k) + \dots$$

Each component belongs to an RKHS, and centering constraints make the decomposition identifiable. Tensor products construct interaction spaces. If $\mathcal{H}_j$ has kernel k_j , then a two-factor interaction has kernel $k_j k_k$. Expanding

$$\prod_j (1 + k_j)$$

reveals the constant, main-effect, and interaction subspaces. This is the functional counterpart of the ANOVA kernel algebra in Chapter 19, Translation-Invariant, Semigroup, and Probabilistic Kernels.

The model can use separate penalties $\lambda_r J_r(f_r)$, allowing different smoothness for different components. Selection then concerns both smoothing and structure. A hierarchy rule can require an interaction to enter only when its main effects are present, while sparsity penalties can suppress whole components.

Algorithm (auditable additive RKHS fit)

Input. Training observations, component spaces, null-space bases, candidate smoothing parameters, and a validation design.

Output. An identified additive predictor with componentwise uncertainty and complexity diagnostics.

1. Center covariates and construct each null-space and penalized basis using training data only.

2. Impose centering constraints on main effects and marginal constraints on interactions.
3. Solve the penalized system by a stable direct or matrix-free method.
4. Select smoothing parameters with nested validation, GCV under justified assumptions, or marginal likelihood.
5. Report residual diagnostics, componentwise effective degrees of freedom, condition estimates, and sensitivity to the interaction hierarchy.
6. Stop iterative solves when both the linear residual and held-out predictions stabilize.

Dense direct cost is cubic in the total basis size. Tensor structure, low-rank bases, and backfitting can lower cost, but their convergence tolerance becomes part of the estimator.

50.6 Bayesian interpretation and uncertainty

The Kimeldorf-Wahba correspondence reads the penalized component as a Gaussian process and the null-space component as an unpenalized or diffuse trend (Kimeldorf and Wahba 1971). Under Gaussian observation noise, the posterior mean equals the smoothing-spline estimator after matching the variance ratio to λ . This connection explains why GCV, empirical Bayes, and marginal likelihood can produce related but nonidentical smoothing choices.

Posterior intervals inherit the GP model assumptions. Frequentist confidence intervals inherit approximations about bias and smoothing-parameter selection. Neither interpretation justifies ignoring selection uncertainty. When coverage matters, the conformal methods in Chapter 51, Distribution Shift, Robustness, and Conformal Prediction provide a complementary layer with different assumptions.

50.7 Common mistakes and practical implications

- Treating a seminorm as a norm can accidentally penalize the intercept or leave the system unidentified.
- Using raw coordinates without scaling makes derivative penalties and tensor products difficult to interpret.
- Selecting λ and reporting error on the same validation data is optimistic.
- A large basis is not automatically a complex fit; the effective degrees of freedom depend on shrinkage.
- High-order interactions can dominate computation and destroy interpretability without hierarchy constraints.
- Pointwise intervals do not automatically provide simultaneous or post-selection coverage.

Use a spline when the roughness operator and its null space encode scientifically meaningful behavior. Use a generic kernel when similarity is easier to specify than derivatives. In either case, report the smoother spectrum rather than only the nominal number of coefficients.

50.8 Summary and further reading

Smoothing splines are kernel machines derived from differential roughness penalties. Their representer form contains both an unpenalized null space and a penalized RKHS expansion. The smoother matrix exposes effective complexity and fast cross-validation, while tensor products lead to additive and interaction models. The foundational correspondence is developed by (Kimeldorf and Wahba 1971) and (Wahba 1990), with generalized cross-validation in (Craven and Wahba 1979).

50.9 Exercises

1. warm-up For the integrated squared second-derivative penalty, identify the null space and explain why its basis must appear outside the penalized kernel expansion.
2. computation Given a symmetric smoother matrix and a residual vector, compute the leave-one-out residuals, effective degrees of freedom, and generalized cross-validation score.
3. proof Prove the smoothing-spline representer theorem by decomposing the penalized component into observed kernel sections and their orthogonal complement.
4. challenge Derive the identifiable kernel decomposition for a two-variable smoothing-spline ANOVA model containing both main effects and one interaction.
5. synthesis Design a comparison among a natural spline, an RBF KRR model, and a Matérn GP for data requiring extrapolation. Specify the null-space assumptions, selection protocol, uncertainty metrics, and conditioning diagnostics.

51 Spatial and Spatiotemporal Kernel Models

Spatial data violate the independent-sample picture: nearby observations share weather, geology, ecology, or measurement conditions, and prediction is often required at an unobserved location rather than for another exchangeable case. Covariance kernels provide the geometry, kriging supplies the predictor, and sparse stochastic differential constructions make large fields computationally accessible.

51.1 Random fields, covariance, and variograms

Let $Z(s)$ be a second-order random field indexed by a spatial location $s \in \mathcal{D}$. Write $m(s) = \mathbb{E}Z(s)$ and

$$C(s, t) = \text{Cov}\{Z(s), Z(t)\}.$$

The covariance function must be positive definite, exactly as in Chapter 2, Positive Definite Kernels and RKHS. A stationary model has $C(s, t) = c(s - t)$; an isotropic model further reduces this to a function of distance.

Definition (semivariogram)

For an intrinsically stationary field, the semivariogram is

$$\gamma(h) = \frac{1}{2} \text{Var}\{Z(s + h) - Z(s)\}.$$

When a stationary covariance exists, $\gamma(h) = C(0) - C(h)$. More generally, a valid variogram is conditionally negative definite rather than positive definite.

Variograms remove an unknown constant mean through increments and can remain meaningful when the field has no finite stationary variance. Confusing a variogram with a covariance reverses the geometry: small variogram means strong similarity, while large covariance means strong similarity.

51.2 Kriging as constrained kernel prediction

Suppose $y_i = Z(s_i) + \varepsilon_i$ with independent noise variance τ^2 . A linear predictor at s_0 has the form $\hat{Z}(s_0) = w^\top y$. Under a known constant mean, ordinary kriging minimizes prediction variance subject to $1^\top w = 1$.

Theorem (ordinary kriging system)

Let $K_{ij} = C(s_i, s_j) + \tau^2 \mathbf{1}\{i = j\}$ and $k_0 = (C(s_i, s_0))_i$. The minimum-variance unbiased linear predictor solves

$$\begin{pmatrix} K & 1 \\ 1^\top & 0 \end{pmatrix} \begin{pmatrix} w \\ \eta \end{pmatrix} = \begin{pmatrix} k_0 \\ 1 \end{pmatrix}.$$

Assumptions. The covariance and noise model are second-order valid; the displayed saddle system is nonsingular; the mean is an unknown constant. **Proof status.** Proved by minimizing the quadratic prediction variance with a Lagrange multiplier for unbiasedness (Stein 1999).

Universal kriging replaces the constant by a trend $m(s) = p(s)^\top \beta$ and imposes $P^\top w = p(s_0)$. This is the spatial analogue of the null-space treatment in Chapter 49, Smoothing Splines and Additive RKHS Models. With Gaussian

assumptions, the kriging predictor and variance are the Gaussian-process posterior mean and variance. Without Gaussianity, they remain best linear unbiased quantities, but not a complete posterior distribution.

Example (random validation can conceal spatial failure)

Suppose a monitoring network contains dense local clusters and sparse remote regions. A random train-test split places neighbors of most test sites in training, so interpolation appears excellent. Holding out entire spatial blocks instead measures transfer across distance and may reveal bias from a misspecified range or trend. The correct split follows the intended deployment geometry.

Verification artifact. checks/example-ch-spatial-example-spatial-validation.json records the example source hash and verification scope.

51.3 Anisotropy and nonstationarity

Geometric anisotropy replaces Euclidean distance by

$$r_A(s, t) = \|A(s - t)\|,$$

where A rotates and rescales directions. If $c(r)$ is a valid isotropic covariance, then $c(r_A)$ remains valid. This models direction-dependent ranges but not location-dependent behavior.

Nonstationary models may use spatially varying length scales, local mixtures, deformation maps, or process convolutions. A deformation $u : \mathcal{D} \rightarrow \mathbb{R}^p$ gives $C(s, t) = c\{u(s) - u(t)\}$, which is valid by composition. It can nevertheless fold distant locations together or become weakly identifiable. Maps and local range estimates should be inspected directly.

Compactly supported kernels from approximation theory create sparse covariance matrices when observation pairs beyond a range have zero covariance. Sparsity is valuable, but a hard support radius is a scientific assumption. Tapering an existing covariance alters both prediction and uncertainty and needs sensitivity analysis (Matérn 1960; Stein 1999).

51.4 Matérn fields and the SPDE connection

The Matérn covariance links smoothness, dimension, and differential operators. In Euclidean space, a Matérn field can be characterized formally through an SPDE of the form

$$(\kappa^2 - \Delta)^{\alpha/2} Z = \mathcal{W},$$

where $\mathcal{W}$ is spatial white noise and α controls smoothness relative to dimension. Discretizing the weak equation in a local finite-element basis produces a Gaussian Markov random field with a sparse precision matrix (Lindgren et al. 2011).

Proposition (local basis yields sparse precision)

When the weak SPDE is discretized in compactly supported basis functions, precision-matrix entries vanish for basis pairs whose supports do not interact through the local differential operator.

Assumptions. The operator is local, the weak formulation is well defined, and the chosen basis functions have compact support. **Proof status.** Follows by inspection of the mass and stiffness integrals; nonoverlapping local supports contribute zero (Lindgren et al. 2011).

This changes the computational object from a dense covariance to a sparse precision. It does not make inference exact automatically: mesh resolution, boundary treatment, numerical quadrature, and hyperparameter approximation introduce additional errors.

51.5 Space-time covariance design

A separable space-time covariance has

$$C\{(s, t), (s', t')\} = C_S(s, s')C_T(t, t').$$

It is valid and computationally convenient, especially on a complete grid where Kronecker algebra applies. It assumes that temporal correlation has the same shape at every spatial scale and that spatial correlation evolves identically at every lag. Those restrictions can be unrealistic.

Nonseparable constructions couple spatial and temporal lag while preserving positive definiteness. The Gneiting family is a standard example in which temporal separation modifies both spatial range and marginal scale (Gneiting 2002). Other constructions arise from advection-diffusion equations, latent dynamic factors, and state-space models.

Irregularly sampled time series need not form a rectangular grid. Matrix-free covariance products, inducing points, state-space representations, or sparse precision approximations can exploit structure without filling missing grid cells with invented observations.

51.6 Multivariate fields and change of support

Several physical quantities observed at the same locations require a matrix-valued covariance. Linear models of coregionalization use

$$K(s, t) = \sum_r c_r(s, t)B_r,$$

which is the spatial version of the operator-valued construction in Chapter 45, Vector- and Operator-Valued Kernels. Cross-covariances cannot be chosen independently; the entire block kernel must be positive definite.

Spatial observations may represent points, pixel averages, administrative regions, or sensor footprints. If an observation is an average over region A , its covariance with an average over B is obtained by integrating the point covariance over both supports. Treating region averages as point values at centroids understates this change-of-support uncertainty.

Algorithm (spatial kernel workflow)

Input. Georeferenced observations, support geometry, trend covariates, measurement-error information, and a deployment region.

Output. Predictions and uncertainty validated at the deployment scale.

1. Plot the sampling geometry, supports, missingness, and response against coordinates and time.
2. Fit the trend without using test regions, then inspect directional residual variograms.
3. Compare stationary, anisotropic, and scientifically motivated nonstationary kernels.
4. Estimate covariance parameters with likelihood or restricted likelihood, recording bounds and numerical jitter.
5. Validate with spatial or space-time blocks whose separation matches deployment.

6. Report predictive scores, interval coverage, residual spatial dependence, condition estimates, and sensitivity to mesh or inducing resolution.

Dense factorization is cubic in sample size. Sparse precision, compact support, Kronecker structure, and iterative solves can reduce cost, but each changes the diagnostics that must be reported.

51.7 Common mistakes and practical implications

- Random cross-validation usually overstates performance when nearby observations share latent conditions.
- A nugget can represent measurement error, microscale variation, or both; those interpretations are not interchangeable.
- Fitting a flexible mean and a long-range covariance simultaneously can be weakly identifiable.
- Latitude and longitude should not be treated as planar coordinates over large regions.
- A positive-definite spatial kernel need not remain valid after an arbitrary space-time modification.
- Posterior standard deviations omit covariance-selection and mesh uncertainty unless those are explicitly integrated.

Spatial kernels are most useful when geometry, support, and deployment are specified before model selection. Maps of uncertainty and residual dependence are primary diagnostics, not decorative outputs.

51.8 Summary and further reading

Spatial kernel models combine a trend, a valid covariance or variogram, and a validation design that respects dependence. Kriging is constrained kernel prediction and coincides with GP posterior prediction under Gaussian assumptions. Anisotropy, nonstationarity, space-time coupling, and change of support require explicit modeling. Matérn SPDE methods replace dense covariance algebra with sparse precision algebra. Classical treatments include (Matérn 1960) and (Stein 1999), with the SPDE construction in (Lindgren et al. 2011) and nonseparable space-time kernels in (Gneiting 2002).

51.9 Exercises

1. warm-up Explain the sign and definiteness difference between a covariance function and a semivariogram.
2. computation Solve an ordinary-kriging saddle system for a supplied covariance matrix and target covariance vector, then compute the prediction variance.
3. proof Prove that an anisotropic covariance obtained by applying a fixed linear map to spatial differences remains positive definite.
4. challenge Derive the covariance between two region-average observations from a point-level covariance and state a numerical quadrature strategy.
5. synthesis Design a blocked evaluation for a space-time environmental monitoring problem, including trend selection, covariance alternatives, support geometry, uncertainty scores, and scaling choices.

52 Distribution Shift, Robustness, and Conformal Prediction

A model can be accurate on held-out data and fail immediately after deployment because the held-out data came from the same sampling system as training. Kernel embeddings make distribution change measurable, importance weighting makes a restricted form of change correctable, and conformal prediction turns residual ranks into finite-sample prediction sets. Each tool has precise assumptions, and none converts arbitrary shift into a solved problem.

52.1 A taxonomy of distribution change

Let P denote the training distribution and Q the deployment distribution on (X, Y) .

Definition (common shift models)

- **Covariate shift:** $P_X \neq Q_X$ while $P_{Y|X} = Q_{Y|X}$.
- **Label shift:** $P_Y \neq Q_Y$ while $P_{X|Y} = Q_{X|Y}$.
- **Concept shift:** the conditional law of Y given X changes.
- **Support shift:** deployment places mass where training has little or no support.

These models are restrictions on the joint distributions. Observed marginal change alone does not identify which model holds.

Covariate shift is the most natural setting for reweighting. For a deployment risk

$$R_Q(f) = \mathbb{E}_Q \ell\{Y, f(X)\},$$

the conditional invariance assumption gives

$$R_Q(f) = \mathbb{E}_P [w(X) \ell\{Y, f(X)\}], \quad w(x) = \frac{dQ_X}{dP_X}(x),$$

provided Q_X is absolutely continuous with respect to P_X . No finite weight can repair a target region absent from training. This overlap condition is as important as the conditional invariance assumption.

52.2 Detecting shift with kernel embeddings

The MMD from Chapter 29, Mean Embeddings, MMD, and Characteristic Kernels compares the input marginals through

$$\text{MMD}_k(P_X, Q_X) = \|\mu_{P_X} - \mu_{Q_X}\|_{\mathcal{H}_k}.$$

A characteristic kernel makes zero population MMD equivalent to equality of distributions. The empirical test machinery in Chapter 30, Kernel Hypothesis Testing determines whether an observed discrepancy is distinguishable from sampling variation (Gretton and Smola 2012).

The witness function

$$g(x) = \langle k(x, \cdot), \mu_{P_X} - \mu_{Q_X} \rangle$$

localizes where the embedded distributions differ. It is useful for diagnosis, but its sign and scale depend on the kernel. A small MMD under one bandwidth is not proof that the deployment risk is unchanged.

Example (detection does not identify a correction)

Suppose an MMD test detects that deployment inputs differ from training inputs. The same observation is compatible with harmless covariate shift, a changed labeling mechanism, new unsupported regions, or a measurement-system change. Reweighting is justified only after defending conditional invariance and overlap. The test answers whether marginals differ, not why they differ.

Verification artifact. checks/example-ch-reliability-example-shift-detection-correction.json records the example source hash and verification scope.

52.3 Kernel mean matching

Kernel mean matching estimates weights without separately estimating two densities. Choose nonnegative weights β_i so the weighted training embedding approaches the deployment embedding:

$$\min_{\beta} \left\| \frac{1}{n} \sum_{i=1}^n \beta_i k(x_i, \cdot) - \frac{1}{m} \sum_{j=1}^m k(z_j, \cdot) \right\|_{\mathcal{H}_k}^2.$$

Expanding the norm yields a quadratic program in Gram matrices. Constraints such as $0 \leq \beta_i \leq B$ and an approximately unit average prevent a few samples from absorbing all mass.

Proposition (RKHS moment balance)

If the empirical kernel-mean-matching objective is at most ε^2 , then every $h \in \mathcal{H}_k$ satisfies

$$\left| \frac{1}{n} \sum_i \beta_i h(x_i) - \frac{1}{m} \sum_j h(z_j) \right| \leq \varepsilon \|h\|_{\mathcal{H}_k}.$$

Assumptions. Both empirical embeddings exist and h lies in the RKHS. **Proof status.** Proved by writing the difference as an RKHS inner product and applying Cauchy-Schwarz.

The bound says exactly what is balanced: functions in the chosen RKHS. It does not guarantee balance for arbitrary losses or hidden confounders. Modern rate analyses additionally track the severity of the density ratio, source smoothness, and effective dimension (Feng et al. 2023).

The weight effective sample size

$$n_{\text{eff}} = \frac{(\sum_i \beta_i)^2}{\sum_i \beta_i^2}$$

is an indispensable diagnostic. A weighted risk based on a tiny effective sample can have large variance even when empirical moments match well.

52.4 Robust losses and distributional neighborhoods

Robustness has several meanings. A robust loss reduces sensitivity to extreme residuals. A contamination model allows a fraction of observations to come from an arbitrary distribution. Distributionally robust optimization minimizes worst-case risk over a neighborhood of the empirical distribution. Adversarial robustness protects against input perturbations. These are different uncertainty sets and should not share a theorem by analogy.

An MMD ambiguity set takes

$$\mathcal{Q}_\rho = \{Q : \text{MMD}_k(Q, \hat{P}) \leq \rho\}.$$

For losses lying in or controlled by the RKHS, the worst-case expectation can be bounded by empirical expectation plus a radius times an RKHS norm. This connects distributional robustness to regularization. If the loss is not in the RKHS or the chosen kernel ignores the relevant perturbation, the bound can be vacuous (Sriperumbudur and Lanckriet 2012).

Robust model design should report the perturbation model, radius selection, clean-data cost, shifted-data benefit, and failure outside the uncertainty set. Merely replacing squared loss by Huber loss does not address covariate shift.

52.5 Conformal prediction from exchangeable scores

Conformal prediction wraps a fitted model with a rank calibration step. Split the data into a proper training set and a calibration set. Fit $\hat{f}$ on training data and compute nonconformity scores $r_i = s(X_i, Y_i; \hat{f})$ on calibration data. For a new input x , form

$$\mathcal{C}_\alpha(x) = \{y : s(x, y; \hat{f}) \leq q_{1-\alpha}\},$$

where $q_{1-\alpha}$ is the finite-sample corrected empirical score quantile.

Theorem (split-conformal marginal coverage)

If the calibration examples and the new example are exchangeable, and the fitted scoring rule is fixed with respect to calibration labels, then the split-conformal set satisfies

$$\mathbb{P}\{Y_{\text{new}} \in \mathcal{C}_\alpha(X_{\text{new}})\} \geq 1 - \alpha,$$

with the usual finite-sample quantile convention.

Assumptions. Exchangeability; a measurable score; no reuse of calibration labels in fitting or tuning the score; valid randomized or conservative handling of ties. **Proof status.** Proved by exchangeability of the calibration and test score ranks.

For regression, $s(x, y) = |y - \hat{f}(x)|$ gives a constant-width residual interval. Locally scaled or quantile-based scores adapt width to heteroscedasticity. For classification, scores based on class probabilities yield prediction sets. Conformalized ridge regression illustrates that validity can be added to a strong linear or kernel predictor with limited asymptotic efficiency loss under its working model (Burnaev and Vovk 2014).

Marginal coverage is not conditional coverage at every x . A method can attain the theorem while under-covering a small subgroup and over-covering elsewhere. Report coverage and width by scientifically relevant groups, regions, and difficulty strata.

52.6 Conformal prediction under shift

Ordinary conformal validity can fail when deployment examples are not exchangeable with calibration data. Under covariate shift with known or estimated density ratios, weighted conformal quantiles can target the deployment distribution. Their reliability depends on the same overlap and conditional invariance assumptions as importance weighting.

Under temporal dependence, block or sequential methods require assumptions adapted to the process. Under arbitrary concept shift, no calibration wrapper can guarantee future coverage without further information. Monitoring must therefore accompany prediction sets: detect changes, diagnose likely mechanisms, and trigger abstention or recalibration when assumptions become implausible.

Algorithm (shift-aware kernel prediction)

Input. Training data, untouched calibration data, deployment covariates, candidate kernels, and a declared shift model.

Output. Predictions, prediction sets, and an assumption audit.

1. Define the deployment unit and split data by time, site, group, or source accordingly.
2. Test and visualize input shift with several justified kernel scales.
3. Diagnose support overlap and estimate weights only if covariate-shift assumptions are defensible.
4. Fit weighted and unweighted kernel baselines, recording effective sample size and conditioning.
5. Calibrate conformal scores without reusing calibration labels for model selection.
6. Report risk, coverage, set width, groupwise diagnostics, abstention rate, and sensitivity to weight clipping.

Kernel mean matching requires a constrained quadratic solve; large problems need low-rank features or stochastic moment matching. Stop when both the optimization residual and the embedding discrepancy are stable.

52.7 Common mistakes and practical implications

- Detecting $P_X \neq Q_X$ does not prove covariate shift.
- Density-ratio weights cannot create labels in unsupported target regions.
- Weight clipping trades bias for variance and must be reported.
- Choosing a shift kernel after inspecting test labels leaks deployment information.
- Conformal marginal coverage is not automatic subgroup or conditional coverage.
- Calibration data cease to be calibration data when repeatedly used for tuning.
- Robustness to one uncertainty set says nothing about a different perturbation mechanism.

The practical goal is not a universal certificate. It is a chain of explicit assumptions, diagnostics, corrections, prediction sets, and monitoring rules that can fail visibly.

52.8 Summary and further reading

Kernel embeddings detect and localize marginal distribution change. Kernel mean matching balances an RKHS function class and can correct covariate shift when conditional invariance and overlap hold. Robust losses and distributional neighborhoods address different perturbation models. Conformal prediction adds finite-sample marginal coverage under exchangeability, with weighted variants requiring stronger shift assumptions. See (Feng et al. 2023) for kernel rates under shift and (Burnaev and Vovk 2014) for the ridge-regression connection.

52.9 Exercises

1. warm-up Give one data-generating example for each shift type and explain which observable marginals would change.
2. computation Expand the empirical kernel-mean-matching objective into its quadratic-program matrix and vector terms, including the normalization constraints.
3. proof Prove the RKHS moment-balance proposition and state why it does not control every bounded measurable loss.
4. proof Prove split-conformal marginal coverage from the exchangeability of score ranks, including the finite-sample quantile correction.
5. synthesis Design a deployment audit for a kernel classifier moving from one hospital or sensor network to another. Specify shift tests, overlap criteria, weighting, conformal scoring, subgroup diagnostics, and retraining triggers.

53 Kernels for Dynamical Systems, Control, and Reinforcement Learning

A dynamical system is not a bag of independent input-output pairs. Its observations are linked by evolution, its errors propagate through time, and control changes the distribution from which future data arrive. Kernels enter at three complementary levels: as priors for unknown transition functions, as feature spaces in which nonlinear evolution becomes a linear operator, and as regularizers for value functions and policies.

53.1 Sequential data and four sources of error

Consider a controlled Markov system

$$X_{t+1} = F(X_t, A_t) + \varepsilon_t,$$

or, more generally, a transition law $P(dx' \mid x, a)$. Learning from trajectories introduces distinctions absent from IID regression:

- **Observation error** corrupts measured states or rewards.
- **Process noise** is genuine randomness in the transition.
- **Sampling error** reflects finite, dependent trajectories and incomplete state-action coverage.
- **Projection error** appears when the chosen RKHS cannot represent the desired transition, eigenfunction, or value function.

Off-policy learning adds a distribution mismatch: data are generated by one behavior policy while the target quantity belongs to another. Long trajectories do not necessarily provide many independent observations; mixing time and coverage determine effective information.

53.2 Gaussian-process transition models

A GP dynamics model places a prior on each component of F or uses an operator-valued kernel for coupled components. Conditioning on transition pairs (x_t, a_t, x_{t+1}) gives a predictive distribution for the next state. Iterating that distribution is difficult because the input at the next step is itself uncertain.

Common approximations propagate moments, sample trajectories, or maintain a variational distribution over latent states. In a state-space model with noisy observations

$$X_{t+1} = F(X_t) + \varepsilon_t, \quad Y_t = H(X_t) + \eta_t,$$

system identification must infer both latent states and transition functions. Structured variational methods preserve temporal dependence better than treating inferred states as fixed regression inputs (Eleftheriadis et al. 2017).

Example (one-step accuracy need not imply rollout accuracy)

A transition model can have small held-out one-step error while its simulated trajectories drift rapidly. Small systematic bias is repeatedly fed back as a new input, and the rollout can enter regions unsupported by training data. Evaluation must therefore include multi-step trajectory error, uncertainty calibration, invariant behavior, and state-space coverage.

Verification artifact. checks/example-ch-dynamics-example-dynamics-rollout.json records the example source hash and verification scope.

For control, posterior variance is not a safety certificate. It measures uncertainty under the GP model and approximation. Safe deployment additionally requires constraint models, conservative reachable sets, fallback actions, and tests under model misspecification.

53.3 Koopman and transfer operators

For deterministic dynamics $x' = F(x)$, the Koopman operator acts on observables rather than states:

$$[\mathcal{K}g](x) = g\{F(x)\}.$$

Although F may be nonlinear, $\mathcal{K}$ is linear. Its eigenfunctions satisfy $\mathcal{K}\phi = \lambda\phi$ and reveal coordinates that evolve linearly. The Perron-Frobenius operator instead propagates distributions. They are dual under an appropriate pairing (Rowley et al. 2009).

Conditional mean embeddings provide a probabilistic transfer operator. If $\phi(X)$ is an RKHS feature map, then

$$\mathbb{E}\{\phi(X_{t+1}) \mid X_t = x\} = C_{X_{t+1}X_t}(C_{X_tX_t} + \lambda I)^{-1}\phi(x),$$

under the regularized empirical construction from Chapter 33, Conditional Mean Embeddings and Kernel Bayes' Rule (Song and Gretton 2013). This operator propagates embedded distributions and remains meaningful for stochastic dynamics.

53.4 Kernel EDMD

Extended dynamic mode decomposition approximates the Koopman operator in a finite dictionary. A kernel replaces an explicit large dictionary by its Gram matrix. Given paired states x_i and $x'_i = F(x_i)$, define

$$G_{ij} = k(x_i, x_j), \quad A_{ij} = k(x_i, x'_j).$$

A regularized empirical operator is represented through a system such as $(G + n\lambda I)^{-1}A$, with conventions depending on whether coefficients or feature-space operators are used.

Proposition (empirical Koopman eigenfunction)

If v solves the regularized generalized eigenproblem

$$Av = \xi(G + n\lambda I)v,$$

then the function $\hat{\phi}(x) = \sum_i v_i k(x_i, x)$ is an empirical eigenfunction for the projected Koopman action represented by the paired samples.

Assumptions. The Gram matrices follow the stated convention; the regularized system is nonsingular; evaluation lies in the empirical span. **Proof status.** Proved by substituting the representer expansion into the Galerkin equations. It is an eigenfunction of the empirical projected operator, not automatically of the population Koopman operator.

Small regularization can create unstable spectral estimates when Gram eigenvalues are weak. Report residuals of both the generalized eigenproblem and held-out temporal prediction. Complex eigenpairs require consistent

conjugate handling, and eigenspaces near repeated eigenvalues should be compared as subspaces rather than vector by vector.

53.5 Value functions and Bellman operators

For a fixed policy π , the discounted value function satisfies

$$V^\pi(x) = r^\pi(x) + \gamma \mathbb{E}\{V^\pi(X') \mid X = x, A \sim \pi\}.$$

The Bellman operator is a contraction in the sup norm when the discount lies strictly below one, but an RKHS projection changes the geometry. Kernel least-squares temporal-difference methods estimate a projected fixed point; kernel Bellman-residual methods minimize a sampled residual. The two objectives are not equivalent under finite data or function approximation.

Definition (regularized RKHS policy evaluation)

A regularized policy-evaluation estimator solves either a projected moment equation or an empirical objective of the form

$$\min_{V \in \mathcal{H}_k} \frac{1}{n} \sum_t \{V(X_t) - R_t - \gamma V(X_{t+1})\}^2 + \lambda \|V\|_{\mathcal{H}_k}^2,$$

with sampling and instrumental-variable choices made explicit.

Naively minimizing a squared temporal-difference residual can be biased because the same next-state randomness appears inside the target and residual. Projected equations or double sampling address different aspects of this problem. Regularized policy iteration alternates RKHS policy evaluation with policy improvement and admits statistical analyses involving RKHS capacity, transition properties, and error propagation (Farahmand et al. 2016).

53.6 Off-policy evaluation and coverage

Data from a behavior policy μ may poorly cover actions selected by a target policy π . Importance ratios can correct expectations under absolute continuity, but products of ratios across long horizons have extreme variance. Marginal or stationary density-ratio estimation reduces some variance while introducing its own inverse problem.

Kernel moment matching can balance state-action distributions, and MMD can diagnose mismatch, but neither creates coverage. A policy should not be evaluated in regions where the behavior data provide no comparable transitions. Conservative estimates, partial identification, or explicit abstention are more honest than a precise unsupported value.

53.7 Control with learned kernel models

Model-predictive control with a GP transition model repeatedly optimizes actions under a posterior predictive model. Kernel models are attractive in data-scarce regimes because they provide smoothness and uncertainty. They also create computational pressure: prediction occurs inside an optimization loop, often over many candidate trajectories.

Sparse GPs, random features, local experts, and online dictionaries reduce cost. Approximation error can alter constraint satisfaction, so solver tolerance, posterior approximation, and controller discretization must be included

in safety margins. Data collection changes the closed-loop state distribution, making periodic recalibration and shift monitoring necessary.

Algorithm (auditable sequential kernel model)

Input. Trajectories with policy metadata, a state-action kernel, discount or prediction horizon, and deployment constraints.

Output. A dynamics, operator, or value estimator with sequential diagnostics.

1. Split by complete trajectories or contiguous time blocks, never by individual transitions alone.
2. Quantify state-action coverage and dependence under behavior and target policies.
3. Fit one-step models or projected operators with regularization selected on held-out trajectories.
4. Evaluate multi-step rollouts, spectral residuals, or Bellman residuals as appropriate.
5. Compare against linear dynamics, tabular or linear value approximation, and a no-control baseline.
6. Report conditioning, dictionary growth, effective sample size, constraint violations, and performance under perturbed dynamics.

Iterative linear solves stop on a declared residual; policy iteration stops only when policy change and held-out value diagnostics stabilize.

53.8 Common mistakes and practical implications

- Randomly splitting transitions leaks adjacent states across train and test sets.
- Treating process noise as observation noise produces overconfident rollouts.
- An empirical Koopman eigenfunction is not automatically a population invariant coordinate.
- Small Bellman residual under the behavior distribution need not imply low target-policy error.
- Importance weighting without overlap can report a finite number with no reliable meaning.
- GP uncertainty is conditional on the kernel, likelihood, and approximation.
- A controller that explores changes the data distribution used to fit its model.

Sequential kernel methods should be evaluated as closed-loop systems, not only as regressors. Stability, coverage, and compounding error are first-class results.

53.9 Summary and further reading

Kernels model dynamics directly with Gaussian processes, linearize evolution through Koopman and transfer operators, and regularize value functions in reinforcement learning. Each view solves a different problem and has distinct error sources. Kernel EDMD approximates a projected operator; GP state-space models infer uncertain latent dynamics; RKHS policy iteration controls value-function complexity. See (Rowley et al. 2009), (Eleftheriadis et al. 2017), and (Farahmand et al. 2016).

53.10 Exercises

1. warm-up Distinguish observation noise, process noise, projection error, and off-policy distribution mismatch in one controlled Markov example.
2. computation Given paired-state Gram matrices, solve a regularized kernel-EDMD generalized eigenproblem and evaluate one empirical eigenfunction at a new state.
3. proof Prove that the Koopman operator is linear even when the underlying state dynamics are nonlinear.
4. challenge Derive the finite representer system for regularized kernel Bellman-residual minimization and identify the source of double-sampling bias.

5. synthesis Design a safe model-based control study comparing a GP transition model, a random-feature approximation, and a linear baseline. Include trajectory splits, coverage, rollout calibration, constraints, compute, and fallback behavior.

54 Kernels for Scientific Computing and Operator Learning

Scientific learning asks more than prediction at another row of a data table. It may require a function satisfying a differential equation, an inverse coefficient compatible with boundary measurements, or an operator mapping one field to another across discretizations. Kernels handle these tasks by representing continuous functions, evaluating linear physics operators through differentiated kernels, and quantifying numerical uncertainty when a probabilistic model is justified.

54.1 Linear information about an unknown function

Let $u \in \mathcal{H}_k$ be an unknown function and suppose the available information consists of bounded linear functionals

$$y_i = L_i u + \varepsilon_i.$$

Point evaluation, derivatives, integrals, boundary traces, and differential-equation residuals all fit this form when they are bounded on the chosen RKHS. By the Riesz representation theorem, each functional has a representer $r_i \in \mathcal{H}_k$ satisfying $L_i u = \langle u, r_i \rangle_{\mathcal{H}_k}$.

Theorem (representer theorem for linear scientific information)

The minimum-norm regularized estimator

$$\min_{u \in \mathcal{H}_k} \sum_i \ell_i(y_i, L_i u) + \lambda \|u\|_{\mathcal{H}_k}^2$$

has a solution in the span of the functional representer:

$$\hat{u} = \sum_i c_i r_i.$$

For a sufficiently smooth scalar kernel, $r_i(x) = L_i^{(z)} k(x, z)$, with the functional applied to the indicated kernel argument.

Assumptions. Each L_i is bounded on $\mathcal{H}_k$; the regularized problem has a minimizer; the penalty is strictly increasing in the norm. **Proof status.** Proved by orthogonally decomposing u into the representer span and its complement.

The Gram matrix becomes

$$G_{ij} = L_i^{(x)} L_j^{(z)} k(x, z).$$

Positive semidefiniteness follows because G is the Gram matrix of the Riesz representer. Differentiating a kernel formula is legitimate only when the required derivatives exist and evaluation remains bounded.

54.2 Symmetric kernel collocation

Consider a linear boundary-value problem

$$\mathcal{A}u = f \quad \text{in } \Omega, \quad \mathcal{B}u = g \quad \text{on } \partial\Omega.$$

Choose interior points and boundary points, then create functionals from $\mathcal{A}$ and $\mathcal{B}$ evaluated at those locations. Solving the functional Gram system produces a function that matches the equation and boundary information at the collocation sites.

Proposition (validity of the symmetric collocation matrix)

If all collocation functionals are bounded on $\mathcal{H}_k$, the matrix obtained by applying each pair of functionals to the two kernel arguments is positive semidefinite.

Assumptions. The kernel is differentiable to the order required by the differential and boundary operators; the functionals are bounded. **Proof status.** Proved by identifying the matrix with the Gram matrix of functional representer.

Unsymmetric collocation applies the operator to only one argument and can be cheaper, but it loses this direct symmetric positive-definite structure. Least-squares collocation uses more residual points than basis centers and can improve robustness to noise and inconsistent information.

Native-space error estimates relate fill distance, kernel smoothness, and target regularity. A very smooth kernel is not automatically better: it can be badly conditioned and can impose regularity the true solution does not possess (Wendland 2005). Shape-parameter selection must balance approximation and numerical stability.

Example (boundary constraints change the admissible space)

Two kernel solvers may use the same interior differential residuals but different boundary functionals. Their solutions can agree at every interior collocation site and still represent different boundary-value problems. Boundary conditions are part of the operator and function space, not an afterthought to be patched onto an unconstrained regression fit.

Verification artifact. checks/example-ch-scientific-example-scientific-boundary.json records the example source hash and verification scope.

54.3 Green kernels and physics-informed covariance

When a Green function for $\mathcal{A}$ and the boundary conditions is available, it can define a kernel whose sections already respect the operator geometry. This mirrors the spline construction in Chapter 49, Smoothing Splines and Additive RKHS Models. Physics can also be encoded by applying projection operators to a base kernel, yielding divergence-free, curl-free, or conservation-compatible vector fields.

A hard constraint restricts the hypothesis space. A soft residual penalty allows model discrepancy:

$$\sum_i \{y_i - u(x_i)\}^2 + \rho \sum_j \{\mathcal{A}u(z_j) - f(z_j)\}^2 + \lambda \|u\|_{\mathcal{H}_k}^2.$$

The parameter ρ expresses trust in the equation relative to observations. Sending it to infinity is justified only when the numerical operator and physical model are exact at the required scale.

54.4 Gaussian processes with differential observations

If $u \sim \mathcal{GP}(m, k)$, bounded linear transformations remain jointly Gaussian. For linear functionals L_i and M_j ,

$$\text{Cov}(L_i u, M_j u) = L_i^{(x)} M_j^{(z)} k(x, z).$$

This permits conditioning on values, derivatives, fluxes, integrals, and linear PDE residuals in one Gaussian system. Unknown forcing or coefficients make the problem nonlinear and may require approximation, hierarchical priors, or alternating inference (Raissi and Karniadakis 2017).

The posterior covariance measures uncertainty under the GP prior, observation model, and exact functional evaluations. Numerical differentiation, operator discretization, boundary approximation, and kernel selection are additional sources of uncertainty. A narrow posterior obtained with a misspecified operator is not a calibrated numerical guarantee.

54.5 Probabilistic numerical methods

A probabilistic numerical method places a probability model on an unknown mathematical object and conditions on information produced by a numerical procedure. Bayesian quadrature in Chapter 32, Kernel Quadrature and Herding is one example; differential-equation solvers are another (Cockayne et al. 2019).

Three questions keep the interpretation honest:

1. What is random: epistemic uncertainty about a fixed function, a genuinely random physical field, or a device for numerical error representation?
2. Which information operator was observed, and was it evaluated exactly?
3. Is posterior contraction calibrated to actual numerical error under a stated class of problems?

Posterior variance can guide adaptive mesh refinement by selecting information where uncertainty or an error indicator is large. The selection policy then becomes part of the method and must be tested against deterministic residual-based refinement.

54.6 Inverse scientific problems

In an inverse problem, observations depend on an unknown coefficient through a forward solution operator:

$$y = \mathcal{G}(a) + \varepsilon.$$

Kernel regularization may act on a , on the state u , or on both. Linear inverse problems connect directly to Chapter 47, Inverse Learning and Spectral Regularization. Nonlinear forward maps require local linearization, adjoints, sampling, or surrogate models.

Identifiability precedes optimization. If distinct coefficients produce indistinguishable observations, a smooth kernel selects one representative but does not recover information absent from the experiment. Report the observation operator, sensitivity spectrum, prior or RKHS penalty, and uncertainty conditional on nonidentifiable directions.

54.7 Learning operators between function spaces

Many scientific tasks observe pairs of functions (a_i, u_i) and seek an operator $\mathcal{G} : a \mapsto u$. A scalar kernel on input functions combined with an operator-valued output kernel yields

$$\hat{\mathcal{G}}(a) = \sum_i K(a, a_i) c_i,$$

where each coefficient may itself be a function. Basis discretization reduces the problem to the vector-valued framework of Chapter 45, Vector- and Operator-Valued Kernels.

Resolution transfer requires more than evaluating a matrix on a finer grid. The input and output spaces, sampling operators, and norms must be defined independently of discretization. Training and testing on different meshes is a necessary diagnostic.

Neural operators learn nonlinear maps between function spaces. Fourier neural operators parameterize an integral kernel in spectral coordinates and have become an important comparator for parametric PDE families (Li and Anandkumar 2020). Kernel operator regression offers convex fitting for fixed representations and transparent regularization; neural operators offer learned representations and often greater expressivity. Comparisons should match training trajectories, meshes, parameter counts, error norms, and rollout horizons.

Algorithm (auditable scientific kernel solver)

Input. A domain, operators and boundary conditions, observations or simulation pairs, a kernel family, and target error norms.

Output. A continuous estimator or learned operator with decomposed error diagnostics.

1. State the continuous problem, units, boundary conditions, and observation functionals before discretization.
2. Choose a kernel whose native space supports every required functional and plausible solution regularity.
3. Separate training sites, validation functions or parameter settings, and test meshes.
4. Solve the regularized functional Gram system with scaling, preconditioning, and residual monitoring.
5. Measure data error, equation residual, boundary residual, discretization sensitivity, and conditioning separately.
6. Compare against an established numerical method and, for operator learning, a resolution-independent neural baseline.

Stop adaptive refinement when the target norm and independent residual indicator stabilize, not merely when posterior variance becomes small.

54.8 Common mistakes and practical implications

- Differentiating a nonsmooth kernel beyond its supported order invalidates the functional Gram matrix.
- Small collocation residual at training sites does not guarantee small solution error between sites.
- A physics penalty does not compensate for wrong boundary conditions.
- Treating discretized vectors as functions without specifying sampling and norms makes resolution claims meaningless.
- Posterior numerical uncertainty can omit model and discretization error.
- Comparing a kernel solver with a neural operator on different meshes or training budgets is inconclusive.
- Ill-conditioning can dominate approximation error for flat or excessively smooth kernels.

Scientific kernel models are most compelling when the continuous information operators are explicit and when numerical error is decomposed rather than hidden inside a single test metric.

54.9 Summary and further reading

Bounded linear scientific information has RKHS representers obtained by applying functionals to kernel arguments. This yields symmetric collocation, derivative-observation GPs, and constrained vector fields. Probabilistic numerical methods attach uncertainty to numerical information under an explicit model. Operator regression extends the same ideas to maps between functions and provides a principled comparator to neural operators. See (Wendland 2005), (Cockayne et al. 2019), (Raissi and Karniadakis 2017), and (Li and Anandkumar 2020).

54.10 Exercises

1. warm-up State the smoothness needed to use point, first-derivative, and second-derivative observations with an RKHS kernel.
2. computation Form the functional Gram matrix for value and derivative observations of a one-dimensional RBF kernel and verify its symmetry.
3. proof Prove positive semidefiniteness of the symmetric collocation matrix using Riesz representers.
4. challenge Derive a kernel formulation that enforces a linear boundary condition exactly while penalizing an interior differential residual softly.
5. synthesis Design a resolution-transfer benchmark comparing kernel operator regression and a Fourier neural operator on a parametric PDE family. Specify meshes, norms, boundary handling, compute, uncertainty, and failure diagnostics.

55 Reproducing-Kernel Banach and Variation Spaces

RKHS geometry makes evaluation linear, regularization quadratic, and representer theorems almost effortless. The same geometry also favors dense squared norms. Banach and variation spaces keep bounded evaluation while replacing the inner product by duality and nonsmooth norms, allowing sparse atoms and finite-width feature representations to emerge from infinite-dimensional optimization.

55.1 What the Hilbert structure was buying

Three familiar RKHS moves depend on the inner product:

1. Every bounded functional has a unique representer in the same space.
2. Orthogonal projection separates observed kernel sections from an irrelevant complement.
3. The squared norm has the linear gradient $f \mapsto f$ after identifying the space with its dual.

In a Banach space there may be no orthogonality, no symmetric inner product, and no canonical identification with the dual. Bounded evaluation can still hold, but the reproducing object and the representer theorem must be expressed through duality.

Definition (reproducing-kernel Banach space)

A Banach space $\mathcal{B}$ of functions on $\mathcal{X}$ is a reproducing-kernel Banach space when every point evaluation $\delta_x : f \mapsto f(x)$ is a bounded linear functional and the resulting evaluation functionals admit a kernel representation through an appropriate dual pairing.

The exact kernel may be scalar, two-sided, operator valued, or induced by a semi-inner product, depending on the geometry of $\mathcal{B}$.

Unlike the Hilbert case, a positive-definite symmetric scalar kernel is not by itself a complete description of an arbitrary RKBS. Smoothness, strict convexity, reflexivity, and a chosen duality map determine how primal and dual representations relate (Zhang and Zhang 2009).

55.2 Semi-inner products and duality mappings

In a smooth Banach space, the normalized duality map $J : \mathcal{B} \rightarrow \mathcal{B}^*$ satisfies

$$\langle f, J(f) \rangle = \|f\|_{\mathcal{B}}^2, \quad \|J(f)\|_{\mathcal{B}^*} = \|f\|_{\mathcal{B}}.$$

The induced semi-inner product can be written $[g, f] = \langle g, J(f) \rangle$. It is linear in its first argument but need not be symmetric. Reproduction then takes a form such as

$$f(x) = [f, K_x]$$

for a suitable section K_x . Because the second slot is nonlinear through J , expansions that are linear in the dual variable need not be linear in the primal function.

For ℓ_p -type geometry with $p \neq 2$, the duality map raises coordinate magnitudes to a power and preserves their signs. This changes the shrinkage path and the meaning of similarity. It also explains why carrying an RKHS coefficient formula into a Banach norm by substitution can be wrong.

55.3 Banach-space representer theorems

Consider

$$\min_{f \in \mathcal{B}} \Phi\{f(x_1), \dots, f(x_n)\} + \lambda \Omega(\|f\|_{\mathcal{B}}).$$

In smooth strictly convex settings, optimality often yields a finite representation in the dual:

$$J(\hat{f}) \in \text{span}\{\delta_{x_1}, \dots, \delta_{x_n}\}.$$

Recovering $\hat{f}$ requires applying the inverse duality map, which can be nonlinear.

Theorem (dual representer principle)

Suppose $\mathcal{B}$ is reflexive, strictly convex, and smooth; evaluations are bounded; Ω is strictly increasing; and a minimizer exists. Then a minimum-norm solution can be chosen so that its norming functional lies in the span of the observed evaluation functionals.

Assumptions. Reflexivity, strict convexity, smoothness, bounded evaluations, existence, and the stated monotonicity. **Proof status.** Cited at the level of a general RKBS representer principle; different Banach constructions require different technical forms (Zhang and Zhang 2009).

The theorem is finite dimensional in the dual, not necessarily a linear primal kernel expansion. Algorithms may solve for dual coefficients and repeatedly evaluate J^{-1} . Conditioning now involves both a kernel matrix and the nonlinear geometry of the norm.

55.4 Nonsmooth norms and sparse atoms

Nonsmooth spaces such as ℓ_1 , spaces of finite measures, and total-variation spaces do not have a single smooth duality map. Optimality uses subgradients and extreme points. The representer can become sparse because an optimizer of a convex problem over a measure ball may be supported on finitely many atoms.

Proposition (finite atomic extreme solutions)

For a convex data-fit problem depending on finitely many bounded linear measurements and regularized by a measure or atomic norm, an extreme minimizer can be chosen with finite atomic support, under the relevant compactness and existence conditions.

Assumptions. Convex lower-semicontinuous loss, bounded measurements, existence of a minimizer, and an atomic norm whose unit ball has the required compactness. **Proof status.** This is an abstract convex-geometric principle; precise support bounds and topologies depend on the space (Wang et al. 2024).

This sparsity differs from support-vector sparsity. In an SVM, selected atoms are kernel sections centered at observed inputs. In a variation space, atom parameters may be learned continuously and need not coincide with training locations.

Example (quadratic and atomic regularization select different geometry)

An RKHS squared norm spreads a solution across correlated directions because energy adds quadratically. An atomic norm pays for the total magnitude of selected atoms and can prefer a small set of extreme directions. Both

estimators can interpolate the same observations, yet one expresses smooth distributed energy while the other expresses sparse learned features.

Verification artifact. `checks/example-ch-rkbs-example-rkbs-dense-sparse.json` records the example source hash and verification scope.

55.5 Variation spaces and ridge splines

A shallow ridge-function model has

$$f(x) = \int a(w, b) \sigma(w^\top x - b) d\nu(w, b).$$

Treating the coefficient as a signed measure and penalizing its total variation produces a Banach variation space. Under suitable activations and side conditions, extreme solutions are finite sums of ridge atoms, hence finite-width neural networks. The width bound comes from the number of measurements and convex geometry rather than being fixed before optimization (Parhi and Nowak 2021).

For truncated-power activations, these functions are multivariate ridge splines. The connection extends the spline story in Chapter 49, Smoothing Splines and Additive RKHS Models: classical splines place knots in the input domain, while ridge splines learn oriented hyperplane knots.

This is a bridge to feature learning. A fixed RKHS optimizes coefficients in a fixed feature geometry. A variation norm can optimize the locations and directions of atoms while retaining a convex infinite-dimensional formulation. The finite neural network is a representer of that variational problem.

55.6 Algorithms: from duality to column generation

Finite dictionaries reduce atomic regularization to lasso-like optimization. Continuous dictionaries require finding an atom most correlated with a residual or dual certificate. Conditional-gradient and exchange methods alternate between a linear minimization oracle and coefficient refitting.

Algorithm (atomic column generation)

Input. Measurements, a loss, an atomic family, a regularization level, and an oracle tolerance.

Output. A finite atomic predictor and a dual-gap certificate.

1. Initialize with an empty or small atom set.
2. Solve the restricted convex coefficient problem on active atoms.
3. Form the dual residual and find an atom maximizing its absolute correlation.
4. Add the atom, optionally remove inactive atoms, and refit coefficients.
5. Stop when the oracle value satisfies the dual optimality tolerance and the held-out objective is stable.
6. Report active atoms, primal and dual objectives, oracle accuracy, and sensitivity to initialization.

The outer convergence guarantee assumes an exact or controlled linear oracle. For neural ridge atoms that oracle can itself be nonconvex, so its approximation error must be distinguished from the convex master problem.

55.7 Statistical and computational tradeoffs

Changing the norm changes the capacity measure. RKHS analysis naturally uses eigenvalues, effective dimension, and Rademacher complexity of Hilbert balls. Atomic and variation spaces use dual norms, covering numbers, sparsity, or path-like complexity. A sparse representation can reduce evaluation cost without improving statistical error if the atom search overfits.

Banach methods also lose some computational conveniences. Gram matrices may no longer capture the entire problem, duality maps can be nonlinear, and nonreflexive spaces require weak-star arguments. The gain is a geometry aligned with robustness, sparsity, or learned features.

55.8 Common mistakes and practical implications

- A Banach norm cannot generally be inserted into an RKHS derivation while keeping the same linear representer.
- Bounded evaluation must be verified for the chosen space and topology.
- Dual sparsity and primal sparsity are not identical.
- A finite atomic representer theorem guarantees existence, not that atom search is easy.
- Approximate linear oracles can invalidate claimed dual certificates.
- Learned atoms need independent regularization and validation even when the master problem is convex.
- A neural-network representation does not imply that ordinary nonconvex network training solves the variation-space problem.

Use RKBS or variation geometry when the desired inductive bias is genuinely nonquadratic. The extra functional analysis is justified only if it changes sparsity, robustness, or feature learning in a measurable way.

55.9 Summary and further reading

RKBSs preserve bounded evaluation while replacing Hilbert inner products by duality. Smooth Banach spaces yield representer principles in the dual; nonsmooth measure and atomic norms yield finite sparse extreme solutions. Variation spaces connect continuous-domain regularization to finite-width neural ridge splines. Foundations appear in (Zhang and Zhang 2009), sparse representers in (Wang et al. 2024), and the neural ridge-spline bridge in (Parhi and Nowak 2021).

55.10 Exercises

1. warm-up Identify the exact steps of the standard RKHS representer proof that fail without an inner product.
2. computation Compute the normalized duality map and its inverse for a finite-dimensional ℓ_p space at a supplied vector.
3. proof Derive the first-order optimality condition showing that the norming functional of a smooth RKBS solution lies in the span of evaluation functionals.
4. challenge Prove a finite-support bound for an extreme solution of an atomic-norm interpolation problem with finitely many scalar measurements.
5. synthesis Compare an RKHS estimator, a finite-dictionary lasso, and a continuous variation-space model on the same regression task. Specify capacity control, optimization certificates, sparsity, compute, and extrapolation diagnostics.

56 Accountable Kernels: Uncertainty, Explanation, and Audit

A prediction that will steer a spacecraft, propose a molecule for synthesis, or drive a satellite crop-yield estimate is not judged by its accuracy alone. Someone will ask how sure the model is, which training data the decision rests on, whether the inputs still resemble what the model was trained on, and whether the whole thing can be reproduced and documented for a review board. For most modern models these are afterthoughts, bolted on with a separate uncertainty head, a post-hoc saliency map, or a monitoring dashboard that lives outside the model. This chapter argues that for a kernel machine they are not afterthoughts at all: they fall out of the same geometry the book has been building. The representer theorem makes every prediction an explicit, signed sum over named training points, so attribution is exact and free. The fit is a deterministic convex solve, so it is reproducible bit for bit. The Gaussian-process posterior returns a calibrated variance as a first-class output, and where that calibration is not to be trusted, a distribution-free conformal wrapper restores a finite-sample guarantee. The audit tools, drift tests and independence tests, are themselves kernel statistics. Accountability, in short, is a structural property of a kernel model rather than a wrapper around it, and the next chapter shows the payoff in the sciences that demand it.

56.1 Accountability is a property of the model, not a wrapper

Three facts, each proved earlier in the book, do all the work of this chapter, and it is worth stating them together before we develop any one of them, because their conjunction is the argument.

The first is the representer theorem of the kernel-tricks chapter, in the general form of Schölkopf, Herbrich, and Smola (2001): the minimizer of any regularized empirical risk over an RKHS lies in the span of the training data mapped into feature space, so the fitted predictor is

$$f(x) = \sum_{i=1}^n \alpha_i k(x_i, x).$$

Every value the model ever outputs is a weighted vote over the training examples, with the weight on example i equal to $\alpha_i k(x_i, x)$. This is not an approximation to an explanation; it is the model. Ask why the prediction at x came out as it did and the answer is a list of past cases and their signed similarities to x .

The second is that fitting is a deterministic convex program. Kernel ridge regression and the Gaussian-process posterior mean solve the linear system $(K + \lambda I)\alpha = y$; the support vector machine of its chapter solves a convex quadratic program with a unique optimum (Cortes and Vapnik 1995). There is no random initialization, no mini-batch order, no early-stopping seed. Given the data, the kernel, and the hyperparameters, refitting reproduces the same solution to the last bit that the arithmetic allows. A regulator who asks for a reproducible training procedure is handed a short, closed specification rather than a stochastic recipe.

The third is that a Gaussian process, from the Bayesian chapter, returns not a point but a predictive distribution, whose variance

$$\sigma^2(x_\star) = k(x_\star, x_\star) - k(x_\star)^\top (K + \sigma^2 I)^{-1} k(x_\star)$$

depends only on where $x_\star$ sits relative to the data, not on the observed labels. The model reports its own ignorance: far from the data the variance climbs back toward the prior. That error bar is delivered by the same solve as the mean, not by a separate procedure trained to imitate uncertainty.

The contrast with a deep network is not a matter of taste. A network’s explanation must be reconstructed after the fact by a saliency method whose faithfulness is itself in question; its uncertainty must be approximated by an ensemble or by dropout; its training run is not reproducible without pinning a dozen sources of nondeterminism. The kernel model supplies all three natively because they are consequences of positive definiteness and convexity. The rest of this chapter turns each fact into a working tool, and is honest about where each one strains: calibration depends on the kernel being right, attribution is only as inspectable as the training set is documented, and every exact statement here is an exact statement about a model small enough to solve exactly.

56.2 Uncertainty you can put in a report

A number without an error bar is not yet a scientific result, and a great deal of what makes kernel methods trusted in the sciences of the next chapter is that they hand back the error bar as part of the answer. We take the native Gaussian-process interval first, see exactly where it can be trusted and where it cannot, and then layer on the distribution-free guarantee that repairs the gap.

56.2.1 The Gaussian-process interval, and its honest limits

The predictive variance above has a clean reading. It starts at the prior variance $k(x_\star, x_\star)$ and subtracts the quadratic form $k(x_\star)^\top (K + \sigma^2 I)^{-1} k(x_\star)$, which is exactly the information the training data supply about $x_\star$. Near a cluster of data the subtraction is large and the posterior tightens; in a gap or beyond the data the subtraction vanishes and the interval balloons back toward the prior. Under a correctly specified model, the intervals $[\mu(x_\star) \pm z_{1-\alpha/2} \sigma(x_\star)]$ are genuine Bayesian credible intervals with the coverage they claim.

The qualifier is the whole story, so we make both halves of it concrete on numbers.

Example (an honest error bar, and a dishonest one)

First the honest behavior. We fit a Gaussian process with an RBF kernel (length scale 1, noise 0.05) to eight points of $\sin x$ on $[0, 6]$ with a deliberate gap in $(2.5, 4.5)$, and read the posterior standard deviation at three places. Then the failure. We draw 40 points from a wiggly mean $\sin(3x)$ on $[0, 1]$ corrupted by heteroscedastic noise whose standard deviation grows from 0.05 at $x = 0$ to 0.5 at $x = 1$, and fit a GP that wrongly assumes a single homoscedastic noise equal to the average, $\hat{\sigma} = 0.292$. We then measure how often the nominal 90% interval actually covers fresh data. All numbers from `checks/ch-accountable-ex1.py`.

1. Watch the band breathe. The posterior standard deviation is 0.049 at a training point ($x = 0.9$), essentially the noise floor; it rises to 0.472 in the middle of the gap ($x = 3.5$); and it is 0.247 just past the data ($x = 6.5$). The model announces exactly where it has no evidence, and it does so from the inputs alone.
2. Now misspecify the noise. Averaged over the whole domain the nominal 90% interval covers 89.1% of fresh points, which looks fine. It is not fine: restricted to the high-noise half of the domain ($x > 0.5$), coverage falls to 79.2%. The single assumed noise is too small where the true noise is large, and the interval silently under-covers there.
3. Read the lesson. The GP error bar is honest about *where* the data are sparse (step 1), because that part depends only on the kernel geometry. It is only as honest about *how noisy* the data are as the noise model you gave it (step 2). Aggregate coverage hid a local failure that a report on the noisy regime would have relied on.

Reading. Two different promises live inside one interval. The width from the kernel geometry is trustworthy and is the source of the “it knows when it is extrapolating” behavior that makes GPs valued in the sciences. The width from the assumed noise model is trustworthy only under correct specification, and a misspecified kernel or noise level can make the nominal coverage a fiction exactly where it matters. This is the gap the next section closes.

Verification artifact. `checks/example-ch-accountable-example-55-1.json` records the example source hash and verification scope.

56.2.2 Conformal prediction: a distribution-free coverage guarantee

The way to stop trusting the model’s self-assessment and start certifying coverage is conformal prediction (Vovk, Gammerman, and Shafer 2005; Lei et al. 2018; Angelopoulos and Bates 2021). It wraps any predictor, kernel or otherwise, and turns its residuals into an interval with a finite-sample coverage guarantee that assumes nothing about the data distribution beyond exchangeability. The split, or inductive, form is a few lines.

Algorithm (Split-conformal prediction for regression)

Input a fitted predictor $\hat{f}$ (trained on data disjoint from the calibration set); a calibration set $\{(x_i, y_i)\}_{i=1}^n$; a miscoverage level α .

Output a prediction interval $C(x)$ with $\mathbb{P}(Y \in C(X)) \geq 1 - \alpha$.

1. Compute the conformity scores $s_i = |y_i - \hat{f}(x_i)|$ on the calibration set.
2. Set $\hat{q}$ to the $\lceil (n+1)(1-\alpha) \rceil$ -th smallest score.
3. Return $C(x) = [\hat{f}(x) - \hat{q}, \hat{f}(x) + \hat{q}]$.

The guarantee is the theorem of Vovk, Gammerman, and Shafer (2005), sharpened by Lei et al. (2018): if the calibration points and the test point are exchangeable, then

$$1 - \alpha \leq \mathbb{P}(Y_{n+1} \in C(X_{n+1})) \leq 1 - \alpha + \frac{1}{n+1},$$

the upper bound holding when the scores are almost surely distinct. Nothing about the shape of the data enters. The predictor $\hat{f}$ can be a kernel ridge fit, a support vector regressor, or a Gaussian-process mean; if it is good the interval is tight, and if it is poor the interval is wide, but the coverage is certified either way. The jackknife+ of Barber et al. (2021) achieves a companion guarantee using leave-one-out residuals, which sits especially well with kernel models because their leave-one-out residuals are closed form, as the next section shows.

Example (conformal repairs the coverage the GP lost)

Same heteroscedastic truth as the failure above. We fit kernel ridge regression (RBF length scale 0.15, ridge 10^{-2}) on a training split, calibrate on $n = 500$ held-out points at $\alpha = 0.10$, and test on 8000 fresh points. We compare the conformal band against a naive Gaussian band $\hat{f}(x) \pm 1.645 \hat{\sigma}$ using a single $\hat{\sigma}$ estimated from the training residuals. All numbers from `checks/ch-accountable-ex2.py`.

1. Read the quantile off the calibration residuals. The rank is $\lceil 501 \cdot 0.9 \rceil = 451$, so $\hat{q}$ is the 451st smallest of the 500 absolute residuals, $\hat{q} = 0.512$. The band is $\hat{f}(x) \pm 0.512$, width 1.025.
2. Certify the coverage. On the fresh test set the conformal band covers 89.9%, inside the guaranteed envelope $[0.900, 0.902]$ up to test-sample noise. The naive Gaussian band, width 0.909, covers only 87.1%: it is narrower and under-covers, because one $\hat{\sigma}$ cannot describe noise that varies across the domain.
3. Note what conformal does and does not fix. Conformal restores *marginal* coverage to the target. It does not by itself equalize coverage across the domain: in the high-noise half the conformal band still covers only 81.0% (the naive band, 76.0%). The guarantee is an average over the input distribution, not a promise at every x .

Reading. A defensible interval for a report is one whose coverage you can certify without believing the model’s noise assumptions, and split conformal delivers exactly that from a held-out set and a single sorted list of residuals. Its honest limitation is that the certificate is marginal: to tighten it where the data are hard, one conditions the score on the input (normalized or locally-weighted conformal), which the exercises pursue. The pairing is the practical recipe of the chapter: let the Gaussian process propose the interval, and let conformal certify it.

Verification artifact. `checks/example-ch-accountable-example-55-2.json` records the example source hash and verification scope.

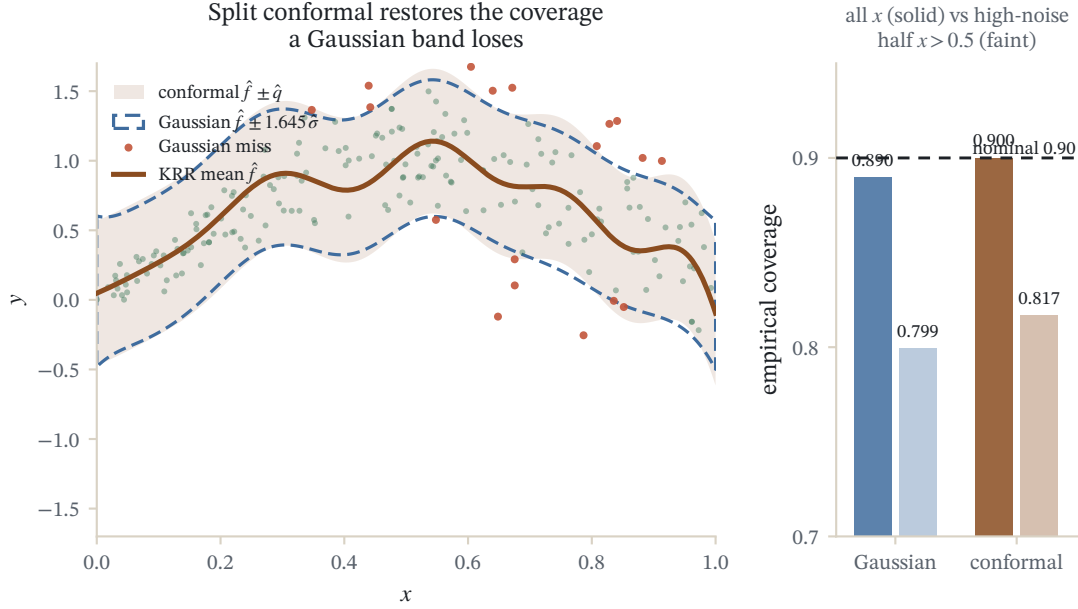


Figure 56.1: Under a misspecified Gaussian process the native predictive interval under-covers, while a split-conformal wrapper restores the nominal 90% coverage with a finite-sample, distribution-free guarantee.

56.3 Explanation for free: the representer theorem as attribution

When a kernel model makes a decision that someone wants to contest, the question is rarely “which pixels” and usually “which prior cases, and how much.” Kernel machines answer that question exactly, because the answer is written into the representer form, and for the convex ones the sensitivity of a prediction to each training point is closed form rather than estimated.

56.3.1 Prediction as a weighted vote over training points

Return to $f(x) = \sum_i \alpha_i k(x_i, x)$. Each training point casts a vote for the prediction at x , of magnitude $|\alpha_i| k(x_i, x)$ and sign $\text{sign}(\alpha_i)$. Two features make this a usable explanation rather than a formula. In the support vector machine the Karush-Kuhn-Tucker conditions force most α_i to be exactly zero, so the prediction depends on a small, inspectable set of support vectors; the explanation is sparse by construction. In kernel ridge and Gaussian-process regression the weights $\alpha = (K + \lambda I)^{-1} y$ are dense, so the credit is spread, but for a local kernel the factor $k(x_i, x)$ concentrates the contribution on the training points near x . Either way, “why this prediction” resolves to named examples with signed weights, and no surrogate model is needed to produce it.

The same RKHS machinery answers the dual question, “which examples represent the data, and which are the edge cases,” through the prototypes and criticisms of Kim, Khanna, and Koyejo (2016). Prototypes are the points whose empirical distribution best matches the data in the sense of maximum mean discrepancy from the mean-embedding chapter; criticisms are the points in regions the prototypes represent worst, found by the witness function. Both are computed from the same Gram matrix the model already uses, and both are consumable by a human reviewer as “these are the typical cases the model has learned, and these are the outliers it may not handle.”

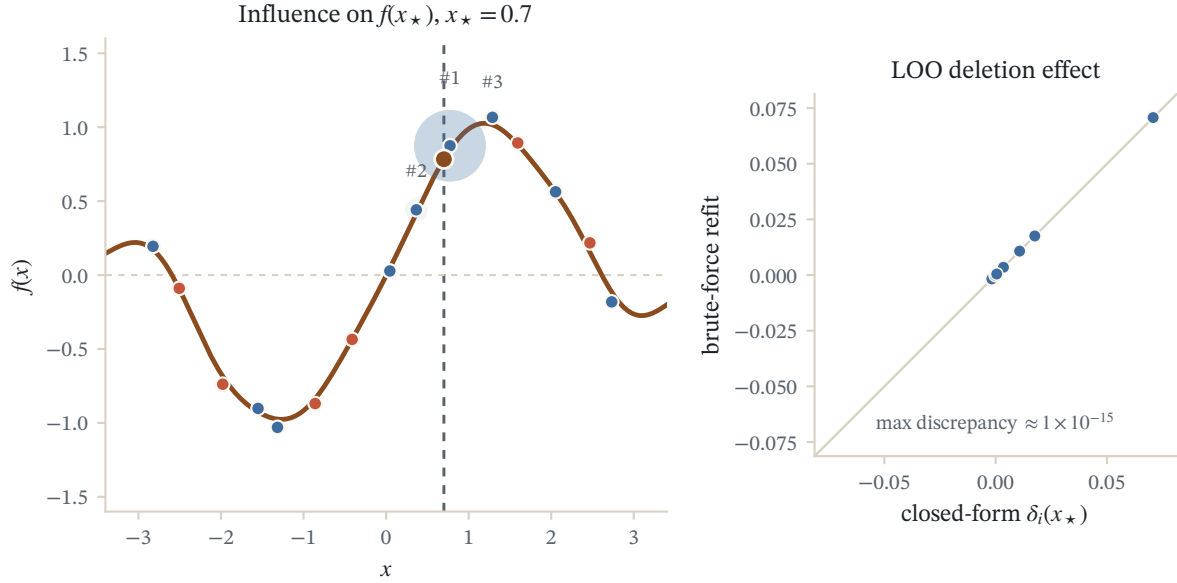


Figure 56.2: Every kernel-ridge prediction is a signed sum over training points; the closed-form leave-one-out and influence match brute-force refits to machine precision, so attribution is exact and free.

56.3.2 Exact influence and closed-form leave-one-out

Beyond “which points support this prediction” is the sharper counterfactual, “how would this prediction change if a given training point had never been collected.” For general models this is the province of influence functions (Koh and Liang 2017), which approximate the effect of up-weighting a training point through the inverse Hessian of the loss. Their derivation needs the empirical risk to be twice differentiable and strictly convex, so that the Hessian is positive definite and invertible. That condition is exactly what a convex kernel machine satisfies and a deep network does not, which is why influence estimates are trustworthy here and demonstrably fragile for deep nets (Basu, Pope, and Feizi 2021). Leaning into that contrast is the point: the tool that is a fragile approximation elsewhere is exact for a kernel ridge model.

Exact, and closed form. Kernel ridge regression is a linear smoother, $\hat{y} = Hy$ with the hat matrix $H = K(K + \lambda I)^{-1}$. The leave-one-out residual at a training point needs no refit,

$$y_i - \hat{f}^{(-i)}(x_i) = \frac{y_i - \hat{y}_i}{1 - H_{ii}},$$

where H_{ii} is the leverage of point i ; and the same block-inverse identity gives the exact change in any test prediction $f(x_*)$ when point i is deleted, at the cost of one rank-one update rather than a refit. Ranking training points by that change identifies, for a specific decision, the data it rests on.

Algorithm (Exact leave-one-out attribution for kernel ridge)

Input Gram matrix K , labels y , ridge λ ; a query x_* with cross-kernel $k_* = [k(x_*, x_i)]_i$.

Output the training points ranked by their influence on $f(x_*)$.

1. Solve once: $G = (K + \lambda I)^{-1}$, $\alpha = Gy$, $H = KG$, $\hat{y} = Hy$.
2. Form the leave-one-out residuals $r_i = (y_i - \hat{y}_i)/(1 - H_{ii})$.
3. For each i , the exact change in the prediction on deleting point i is $\Delta_i = (G k_*)_i r_i$.
4. Return the points sorted by $|\Delta_i|$.

Example (which three points a decision rests on)

Kernel ridge regression on 40 points of $\sin(1.3x)$ with light noise on $[-3, 3]$ (RBF length scale 0.4, ridge 0.1). We ask which training points most determine the prediction at $x_\star = 0.7$, computing the deletion effect both by brute-force refitting and by the rank-one formula of the algorithm. All numbers from `checks/ch-accountable-ex3.py`.

1. Confirm the closed form is exact. The full-model prediction is $f(x_\star) = 0.7249$. The rank-one deletion effect Δ_i agrees with an actual refit-without- i to 1.6×10^{-15} , machine precision: the formula is not an approximation.
2. Rank the responsible points. The three largest $|\Delta_i|$ come from $x_i = 0.781$ (weight $\alpha_i = -1.344$, leverage $H_{ii} = 0.308$, $k(x_i, x_\star) = 0.980$, $\Delta_i = -0.0555$), then $x_i = 0.521$ ($\Delta_i = +0.0325$) and $x_i = 0.963$ ($\Delta_i = +0.0270$). They are the nearest high-weight neighbors of $x_\star$, and deleting the first alone would move the prediction by 0.056.
3. Turn it into an audit statement. The decision at $x_\star$ rests chiefly on three named training points; if any were later found mislabeled or unrepresentative, the exact effect on this prediction is known in advance, at the cost of one solve.

Reading. Data attribution that is a delicate approximation for deep models is, for a kernel ridge model, a closed-form consequence of the hat matrix. The same quantity $1 - H_{ii}$ that yields the leave-one-out error yields the influence of a point on any prediction, so cross-validation and attribution are the same linear algebra. Data-valuation schemes such as Data Shapley (Ghorbani and Zou 2019) build on exactly this notion of a training point's marginal worth.

Verification artifact. `checks/example-ch-accountable-example-55-3.json` records the example source hash and verification scope.

56.4 Auditing a deployed kernel model

A model that was accountable at training time can drift out of validity in deployment, and the questions an auditor asks then are statistical: is the input distribution still the one the model was trained on, are the model's outputs independent of attributes they must not depend on, and can the whole pipeline be reproduced. Each of these is answered by a kernel statistic, and the first two come with p-values.

56.4.1 Is the input still the training distribution? Kernel drift monitoring

The natural monitor is the kernel two-sample test of Gretton et al. (2012), which measures the maximum mean discrepancy between a reference sample (the training inputs) and a recent production window. Writing the mean embedding of a distribution as $\mu_P = \mathbb{E} k(x, \cdot)$, the squared MMD is the RKHS distance between embeddings,

$$\text{MMD}^2(P, Q) = \mathbb{E} k(x, x') + \mathbb{E} k(y, y') - 2 \mathbb{E} k(x, y),$$

with an unbiased U-statistic estimator built directly from Gram blocks. For a characteristic kernel, from the embedding chapter and Sriperumbudur et al. (2010), the embedding is injective, so $\text{MMD} = 0$ if and only if $P = Q$: the test can detect any distributional change, not only a shift in mean or variance. A permutation procedure supplies a finite-sample null and a p-value.

Example (a drift alarm that does not cry wolf)

A reference sample of 200 standard-normal inputs, an RBF kernel at the median-heuristic bandwidth, and 2000 permutations for the null. We test the reference against a fresh unshifted draw and against a window shifted in mean by 0.5. All numbers from `checks/ch-accountable-ex4.py`.

1. Set the bandwidth from the data. The median pairwise distance is 0.946, giving $\gamma = 0.559$ in $k(x, x') = e^{-\gamma(x-x')^2}$.

2. Pass the null case. Against the unshifted draw the statistic is $\text{MMD}^2 = -0.004$ (the unbiased estimator can be negative) with p-value 0.980: no false alarm.
3. Catch the shift. Against the shifted window $\text{MMD}^2 = +0.025$ with p-value 0.001: drift detected at any reasonable level.

Reading. This is a principled replacement for an ad-hoc threshold on some summary statistic: a hypothesis test whose null rejection means the production inputs no longer look like training. It is also the guard on the previous section’s guarantee, because conformal coverage assumes the test point is exchangeable with calibration, and an MMD alarm is precisely a signal that the assumption has broken. Its honest cost is that a characteristic kernel and adequate sample size are needed for power, and that it reports that the distributions differ, not which feature moved.

Verification artifact. checks/example-ch-accountable-example-55-4.json records the example source hash and verification scope.

Kernel MMD drift monitor: statistic vs. permutation null (RBF, $\gamma = 0.54$, $n = 80$, 200 perms)

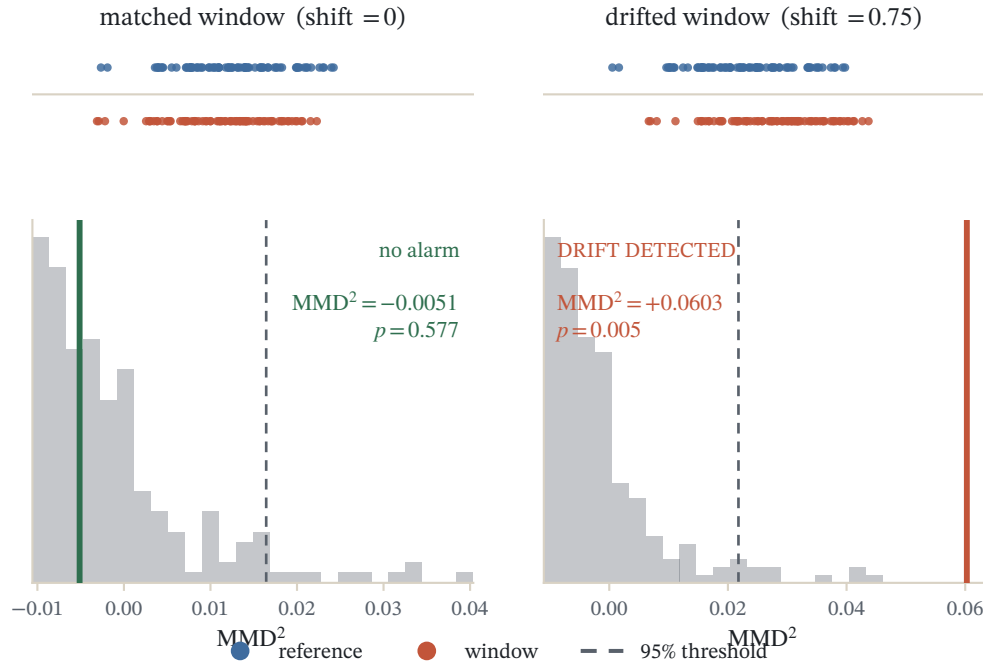


Figure 56.3: MMD drift monitoring: the kernel two-sample statistic between a reference window and a live window, tested against its permutation null, flags a shifted deployment distribution (small p) while a matched window does not.

56.4.2 Independence and fairness audits with HSIC

A different audit asks whether the model’s outputs are statistically independent of an attribute they must not use, a protected characteristic, a batch label, a sensor id. The Hilbert-Schmidt Independence Criterion of Gretton et al. (2005) measures dependence as the squared Hilbert-Schmidt norm of the cross-covariance operator, estimated by

$$\widehat{\text{HSIC}} = \frac{1}{(n-1)^2} \text{tr}(KHLH), \quad H = I - \frac{1}{n} \mathbf{1}\mathbf{1}^\top,$$

with K a kernel on the predictions and L a kernel on the attribute. For characteristic kernels $\text{HSIC} = 0$ if and only if the two are independent, so it detects nonlinear dependence that a correlation would miss, and a permutation null turns it into a test (Gretton et al. 2008).

Example (auditing dependence on a protected attribute)

Two hundred cases, a continuous protected attribute, and two predictors: a biased one whose score leans on the attribute, and a fair one from which that dependence has been removed. We test each with HSIC and a permutation null. All numbers from `checks/ch-accountable-ex5.py`.

1. Test the biased model. $\text{HSIC} = 0.0231$ with p-value 0.000: the hypothesis of independence is decisively rejected, so the predictions do depend on the attribute.
2. Test the fair model. $\text{HSIC} = 0.0006$ with p-value 0.775: independence is not rejected. The same statistic that flagged the first model clears the second.

Reading. A group-fairness audit becomes a hypothesis test with a p-value, and because HSIC can also be added as a training penalty (Fair Kernel Learning, Pérez-Suay et al. 2017, which we meet again over satellite data in the next chapter), the property is both testable and enforceable with one kernel quantity. The honest caveat is that HSIC targets statistical independence, a demographic-parity-style notion, and other fairness definitions require other criteria.

Verification artifact. `checks/example-ch-accountable-example-55-5.json` records the example source hash and verification scope.

56.4.3 Reproducibility and the audit trail

The last audit needs no experiment. Because a kernel machine is the unique optimum of a convex program, its solution is a deterministic function of the data, the kernel, and the hyperparameters: the linear system $(K + \lambda I)\alpha = y$ for ridge and Gaussian processes, a convex quadratic program for the support vector machine. Rerunning the fit reproduces the same parameters to the precision of the arithmetic, with no seed, no batch order, and no early-stopping choice to record. The documentable configuration is short and closed, which is precisely what a reproducible audit trail requires and what a stochastic-gradient training run struggles to provide.

This is where the technical machinery meets the documentation frameworks that high-stakes deployment now expects. Model cards (Mitchell et al. 2019) ask for intended use, performance across conditions, and limitations; a kernel model supplies example-based attributions, feature-relevance tests with p-values, and certified intervals to fill them. Datasheets for datasets (Gebru et al. 2021) ask for data provenance; exact influence and data valuation make each training point's contribution auditable. Risk-management frameworks such as the NIST AI Risk Management Framework (2023) call for measured uncertainty, robustness, and fairness, and conformal coverage, MMD drift tests, and HSIC independence tests are exactly such measurements. Where a regime like the European Union's AI Act designates a system high-risk and demands traceability and technical documentation, the deterministic solve and the explicit representer form make the "how it works and how to reproduce it" section a closed specification. None of this is a legal guarantee, and none of it discharges an obligation; the point is narrower and sound, that kernel methods produce, as a byproduct of their mathematics, the technical evidence these frameworks ask a model to supply.

56.5 Uncertainty that drives the next experiment

The calibrated variance is not only something to report; it is something to act on. When each new label costs a wet-lab experiment or a supercomputer run, the model's own uncertainty is the natural guide to where the next label should be spent, and this closes the loop from accountability to decision. The rule is uncertainty sampling: query where the posterior variance $\sigma^2(x)$ is largest, since that is where a new observation most reduces the model's ignorance, and crucially the variance depends only on the input locations, so the most informative query can be chosen before the expensive label is known. Its acquisition-function generalizations, which trade the mean

against the variance to optimize rather than merely explore, are the subject of the Bayesian optimization chapter (Frazier 2018; Shahriari et al. 2016), and the calibration of a computer model against reality by Kennedy and O’Hagan (2001) is the emulation backdrop the next chapter builds on.

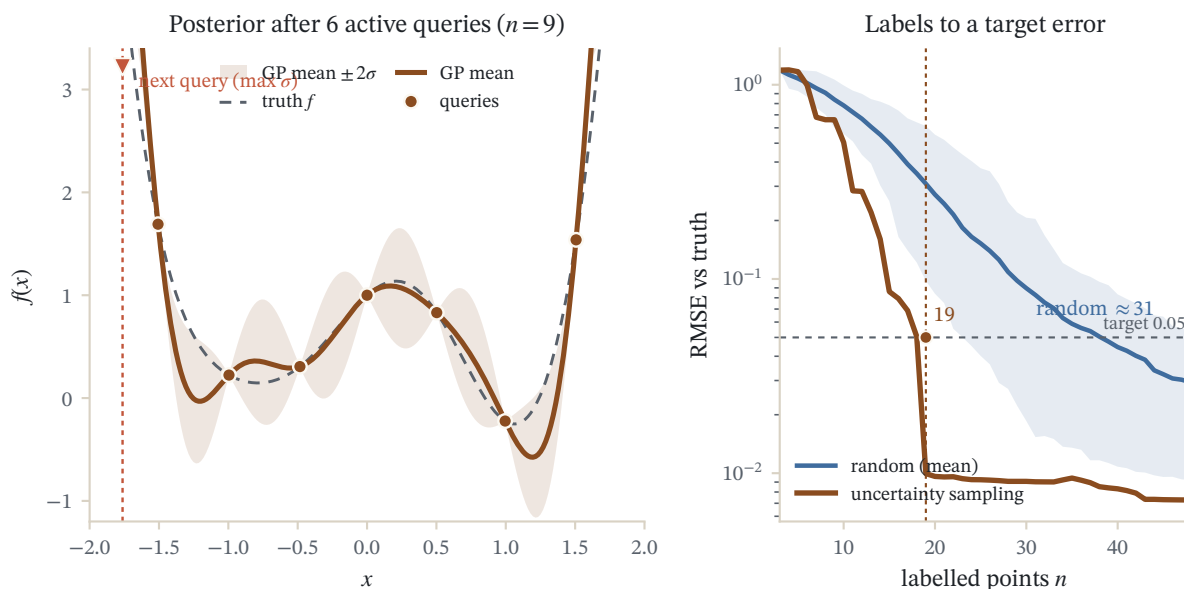


Figure 56.4: Uncertainty-driven active learning queries where the Gaussian-process posterior variance is largest; the variance-guided design reaches a target accuracy in far fewer labels than random sampling.

The next chapter turns this from a demonstration into a working method. In an on-the-fly interatomic potential, a molecular-dynamics simulation runs on the cheap Gaussian-process energy until the predictive variance spikes at a configuration the model has never seen, at which point a single quantum-mechanical calculation is triggered, added to the training set, and the simulation continues. The variance is the whole control signal, and it is the same variance whose honest reading opened this chapter.

56.6 Summary

Accountability is not a layer added to a kernel model; it is a reading of the model’s own structure. The representer theorem makes every prediction an explicit signed sum over named training points, so example-based explanation is exact and free, and for convex kernel machines the influence of a training point and the leave-one-out error are closed-form consequences of one hat matrix, exact where the same tools are fragile approximations for deep networks. The Gaussian-process posterior returns a calibrated variance from the same solve as the mean, honest about where the data are sparse; where its noise assumptions cannot be trusted, split conformal prediction wraps any kernel predictor in an interval with a distribution-free, finite-sample coverage guarantee. Deployment is audited with kernel statistics: the MMD two-sample test flags distributional drift with a characteristic kernel that misses nothing, HSIC turns a fairness check into a hypothesis test, and the deterministic convex solve makes the whole fit reproducible for a documentation trail. Finally, the calibrated variance that made the model reportable also makes it active, choosing the next expensive experiment where it is least sure. Every claim here is exact for a model small enough to solve exactly, and the honest cost of exactness, the cubic solve and the need for a well-chosen kernel, is the same cost the next chapter’s sciences pay gladly for an error bar they can defend.

56.7 Exercises

1. warm-up A kernel ridge model predicts a value with no error bar, and a stakeholder asks "how sure are you." Name two different things that phrase could mean for this model, one answerable from the kernel geometry alone and one requiring an assumption about the noise, and say which of the two the Gaussian-process variance $\sigma^2(x_\star) = k(x_\star, x_\star) - k(x_\star)^\top (K + \sigma^2 I)^{-1} k(x_\star)$ reports honestly and why.

2. computation A calibration set has $n = 19$ absolute residuals, sorted: 0.10, 0.12, For $\alpha = 0.10$, which rank is the conformal quantile $\hat{q}$, and what is the interval? Then show that as $n \rightarrow \infty$ the rank $\lceil (n+1)(1-\alpha) \rceil / n$ tends to the $(1-\alpha)$ empirical quantile, and explain why the finite- n correction always makes the interval at least as wide as the plug-in quantile would. Hint

$\lceil 20 \cdot 0.9 \rceil = 18$, so $\hat{q}$ is the 18th smallest residual and $C(x) = \hat{f}(x) \pm \hat{q}$. The ceiling rounds up, so the finite-sample quantile sits at or above the plug-in one; this is the price of the exact $1 - \alpha$ lower bound.

3. proof Kernel ridge regression is the linear smoother $\hat{y} = Hy$, $H = K(K + \lambda I)^{-1}$. Prove the closed-form leave-one-out identity $y_i - \hat{f}^{(-i)}(x_i) = (y_i - \hat{y}_i) / (1 - H_{ii})$. Hint

Let $\hat{f}^{(-i)}$ be the fit with point i removed and consider the auxiliary problem in which y_i is replaced by $\hat{f}^{(-i)}(x_i)$; its fit is unchanged at i , so $\hat{f}^{(-i)}(x_i) = \sum_j H_{ij} \tilde{y}_j$ with $\tilde{y}_i = \hat{f}^{(-i)}(x_i)$. Solve the resulting scalar equation for $\hat{f}^{(-i)}(x_i)$ and subtract from y_i .

4. exploration Take the influence example (RBF length scale 0.4, ridge 0.1, $x_\star = 0.7$). Predict how the ranked list of the three most influential points changes as the ridge λ grows large, then check with a short script, and connect the change to the leverages H_{ii} and the effective dimension of the practice chapter. Hint

As $\lambda \rightarrow \infty$, $H \rightarrow 0$, the leverages and all Δ_i shrink, and the fit flattens toward a constant; the ranking is dominated by $(Gk_\star)_i$, which for a local kernel still favors the neighbors of $x_\star$ but with far smaller magnitude.

5. challenge Explain precisely why the conformal coverage guarantee requires exchangeability, and construct a deployment scenario (a slow covariate shift over time) in which split conformal silently loses coverage. Then argue why an MMD drift test between the calibration window and the current window is the right guard, and what its rejection tells you to do. Hint

Exchangeability makes the test residual's rank among the calibration residuals uniform; a time trend breaks it so the test residual is stochastically larger, and coverage falls below $1 - \alpha$. An MMD alarm signals the calibration set is no longer representative, so $\hat{q}$ must be recomputed on a fresh window.

6. exploration The MMD drift test used the median-heuristic bandwidth. Investigate its effect on power: rerun the shifted-versus-unshifted test as the bandwidth ranges over an order of magnitude around the median, and describe the trade-off. Then explain why a characteristic kernel is necessary for the guarantee "MMD = 0 iff $P = Q$ " and give a kernel for which it fails. Hint

Too small a bandwidth makes every point look dissimilar and kills power; too large makes every point look identical and also kills power; the median sits near the useful middle. A non-characteristic kernel, such as a polynomial of fixed degree, embeds distinct distributions to the same mean and can give MMD = 0 for $P \neq Q$.

57 Kernels in Science and Space

The previous chapter argued that a kernel machine carries its own accountability: a calibrated error bar, an exact attribution to training data, a reproducible solve, and audit statistics with p-values. This chapter is the payoff, in three domains that demand exactly those properties because a wrong answer with no error bar is unacceptable. In astronomy and space engineering the deliverable is often the uncertainty itself: a planet's mass with a credible interval, a spacecraft-telemetry anomaly score with a false-alarm rate, an orbit prediction with a covariance a collision-avoidance system can trust. In the physical sciences each training label is a quantum-mechanical calculation or a wet-lab experiment, so data efficiency and a variance that tells you where to spend the next one are not conveniences but the method. In Earth observation the product is a global map, and every pixel needs a confidence. Across all three the same three kernel properties recur as requirements, not features: a calibrated posterior covariance, data efficiency when samples are precious, and a kernel that is the explicit, auditable place to write down the physics. We work each domain to a real, cited result, name the kernel and the property that earned it, and state the honest limitation, which turns out to be the same one everywhere.

57.1 Why these domains demand accountable models

It is worth being precise about why these particular sciences adopted kernel methods, because the reasons are the accountability properties of the last chapter rather than raw predictive accuracy. A stellar light curve or a radial-velocity series is short, irregularly sampled, and dominated by a nuisance signal that must be separated from the thing of interest; the model has to report how much of its answer is signal and how much is the model's own uncertainty. A potential-energy surface is smooth and expensive, sampled at a handful of quantum-mechanical points that cost hours each, so the model must interpolate from little data and, critically, must know where it is interpolating badly so a simulation can decide when to pay for another point. A satellite retrieval turns reflectance into a biophysical quantity over millions of pixels, some of which fall outside anything in the training set, and the operational product is only usable if each pixel carries a trustworthy confidence. In every case the Gaussian process and its kernel relatives were adopted because they answer "how sure, from what, and is this still valid" as part of the computation, and because the kernel is where the domain's physics, quasi-periodicity, rotational symmetry, spatial context, enters explicitly rather than being learned opaquely.

57.2 Space and astronomy

Astronomy was among the first fields to make Gaussian processes standard equipment, and the review of Aigrain and Foreman-Mackey (2023) is the map of that adoption across exoplanets, stellar rotation, and asteroseismology. The through-line is that the covariance kernel is a statement of stellar physics, and the posterior is a distribution over the quantity a mission actually needs.

57.2.1 Stellar rotation and exoplanet light curves

A star's brightness is modulated as spots rotate into and out of view, but spots form and decay, so the signal is quasi-periodic rather than strictly periodic. The kernel writes that down directly. The quasi-periodic covariance

$$k(\tau) = A \exp\left(-\frac{\tau^2}{2\ell^2} - \Gamma \sin^2 \frac{\pi\tau}{P}\right),$$

with rotation period P , spot-coherence length ℓ , and modulation depth Γ , combines a periodic term with a decay whose length scale is literally the spot lifetime (Angus et al. 2018). This is the kernel-families construction put to physical use: an interpretable hyperparameter for each physical quantity. The payoff over a classical periodogram is the subject of the last chapter, a full posterior on the period rather than a point estimate, and the celerite method of Foreman-Mackey et al. (2017) makes it affordable by giving the semi-separable covariance an exact likelihood in linear rather than cubic time, so the fully probabilistic model runs on the hundred-thousand-point light curves of Kepler and TESS.

Example (a rotation period, with a credible interval)

We synthesize 120 irregularly sampled brightness measurements over 60 days from a quasi-periodic GP with true period $P = 10$ days, coherence $\ell = 30$ days, and modulation $\Gamma = 2$, plus white noise. We recover P by evaluating the GP log marginal likelihood over a grid and reading off the posterior, and compare with a periodogram peak. All numbers from `checks/ch-highstakes-ex1.py`.

1. Marginalize over the period. With a flat prior on P , the posterior peaks at a maximum-a-posteriori value of 10.10 days with posterior mean also 10.10 days, recovering the true 10.00 days.
2. Report the interval, not just the peak. The 68% credible interval is [10.00, 10.20] days: the deliverable is the period and its uncertainty together.
3. Contrast the periodogram. A Lomb-Scargle-style periodogram returns a single peak at 10.73 days, a point estimate with no native interval and biased here by the evolving spot phase the quasi-periodic kernel models and the sinusoid does not.

Reading. The kernel encodes the physics (quasi-periodicity with a finite coherence time) and the posterior delivers the accountable answer (a period with a credible interval). The honest limitation is that the period posterior is often multimodal, with harmonics of P as competing peaks, so careful sampling matters; and the exact GP is cubic, which is exactly the cost celerite’s semi-separable structure removes for this kernel family.

Verification artifact. `checks/example-ch-highstakes-example-56-1.json` records the example source hash and verification scope.

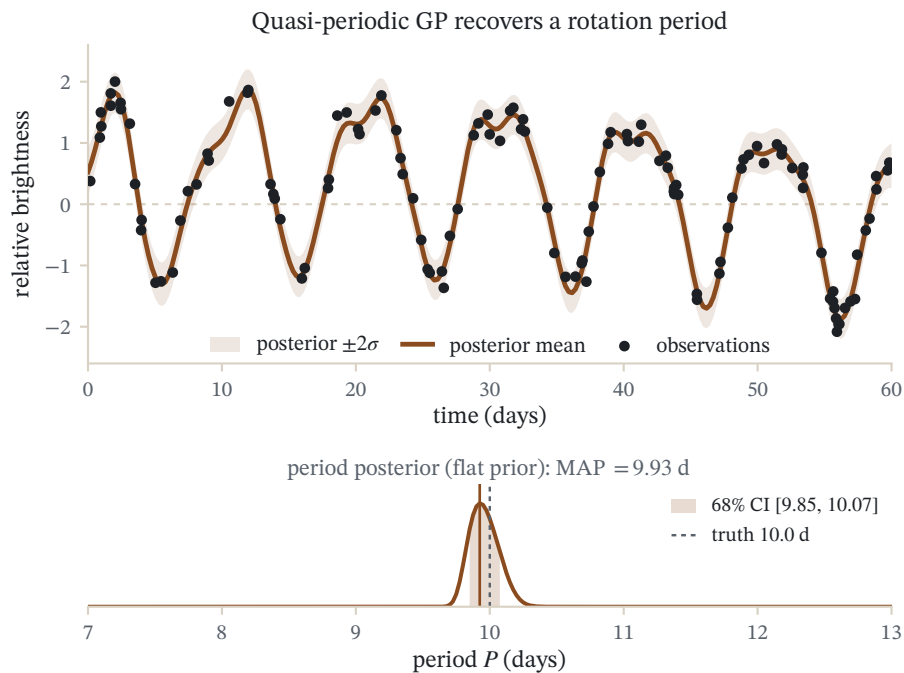


Figure 57.1: A quasi-periodic Gaussian process (a periodic kernel times an RBF decay) fits a noisy stellar light curve and recovers the rotation period with a calibrated interval.

57.2.2 Disentangling a planet from stellar activity

The same quasi-periodic signal that reveals rotation is a nuisance when the goal is a planet's mass from radial velocities, because spots induce apparent velocity shifts that mimic or mask a Keplerian signal. The framework of Rajpaul et al. (2015) models the radial velocity and several activity indicators as different linear combinations of one shared latent quasi-periodic GP and its time derivative, so the activity component is constrained by the indicators and can be subtracted, leaving the planet. Because the fit is a posterior, the planet's mass comes out as a credible interval rather than a point. The CoRoT-7 analysis of Haywood et al. (2014) is the textbook case: treating the activity as a GP whose covariance is inherited from the light curve, they recover the mass of CoRoT-7b as 4.73 ± 0.95 Earth masses. The error bar is the result, and it is an error bar precisely because a kernel model produced it.

57.2.3 Gravitational waves: the noise-weighted inner product is a kernel

The cleanest illustration in all of physics that detection is a projection in an inner-product space is matched filtering, the method behind gravitational-wave detection (Allen et al. 2012). The optimal statistic for a known waveform h buried in noise is the noise-weighted inner product

$$\langle d, h \rangle = 4 \operatorname{Re} \int_0^\infty \frac{\tilde{d}(f) \tilde{h}^*(f)}{S_n(f)} df,$$

where $S_n(f)$ is the noise power spectral density. This is a kernel: it defines the geometry of a whitened Hilbert space in which the data and the template have a well-defined angle, and the detection statistic is the projection of the data onto the template. In white noise it reduces to the plain dot product, which is all we need to see the mechanism on numbers.

Example (finding a chirp by projection)

A chirp template h of 512 samples with rising frequency and a tapered envelope, injected into a white-noise stream at a known arrival lag. We scale the noise so the optimal signal-to-noise ratio $\rho = \|h\|/\sigma$ equals 8, then slide the template and compute the normalized statistic $\rho(t_0) = \langle d, h_{t_0} \rangle / (\sigma \|h\|)$, whose noise has unit variance. All numbers from `checks/ch-highstakes-ex2.py`.

1. Set the difficulty. The template norm is $\|h\| = 7.372$; the optimal SNR of 8 fixes the noise level $\sigma = 0.9215$. The signal is invisible in the raw stream at this noise.
2. Project and read the peak. The matched-filter statistic peaks at 7.37 at exactly the true arrival lag, near the optimal value of 8 (the realized peak is the optimal SNR plus a standard-normal draw).
3. Compare to noise. The loudest pure-noise excursion elsewhere is only 1.71. The projection onto the template lifts the signal far above the noise floor, which is the entire idea of an optimal detector.

Reading. Detection is a projection onto a template in a whitened inner-product space, and that inner product is a kernel in the exact sense the book has used throughout. The optimality is a genuine theorem, the Neyman-Pearson optimal linear detector for a known waveform in stationary Gaussian noise, and the recovered SNR is an auditable, calibrated number. The honest limitation is that the optimality assumes stationary Gaussian noise and a known template, so real pipelines add non-Gaussian-glitch vetoes and must tile the template bank densely, which scales badly in parameter dimension.

Verification artifact. `checks/example-ch-highstakes-example-56-2.json` records the example source hash and verification scope.

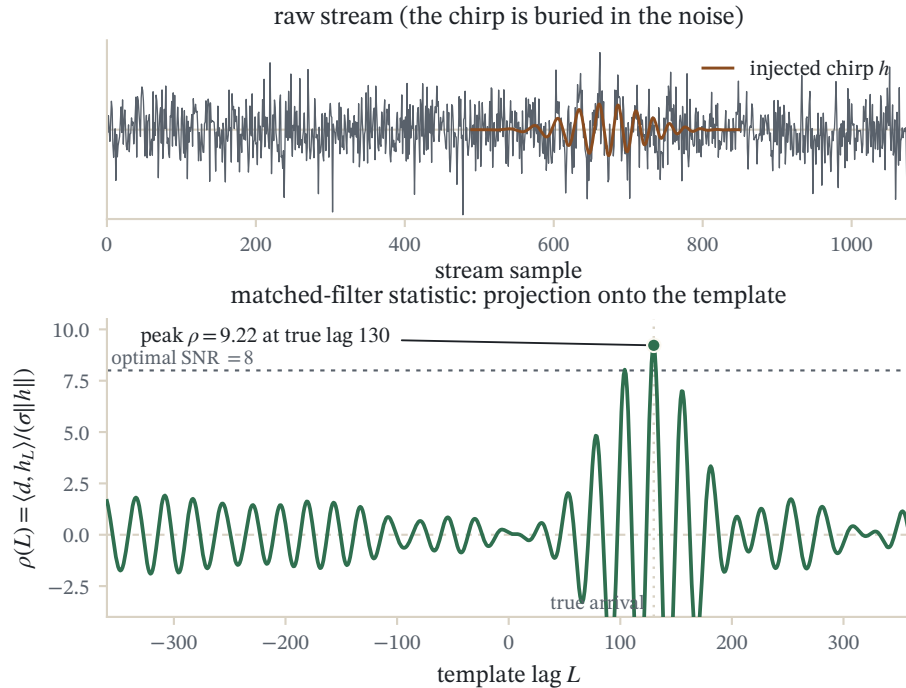


Figure 57.2: Matched filtering as a noise-weighted inner product: correlating a template against noisy data peaks sharply at the true lag, the optimal linear detector for a known signal in coloured noise.

57.2.4 Spacecraft telemetry and orbit uncertainty

Two more space applications make the accountability argument in engineering terms. Spacecraft almost never have labeled fault data, so anomaly detection is naturally a one-class problem: Fujimaki et al. (2005) build a kernel-PCA subspace of nominal telemetry and score a new sample by its reconstruction error in that feature space, a deterministic, reproducible distance rather than a black-box classifier, which is what a mission-review board can act on. And for conjunction assessment, where two objects might collide, a predicted position is useless without a trustworthy covariance; Peng and Bai (2018) learn the residual of a physics-based orbit propagator with a Gaussian process, keeping the interpretable dynamics and adding an input-dependent uncertainty that fuses with the filter covariance. In both, the kernel model is chosen because its output is an auditable uncertainty, not merely a number.

57.3 Molecules and materials

If there is one domain where kernels, not neural networks, are the canonical data-efficient and uncertainty-aware workhorse, it is the machine learning of interatomic potentials and molecular properties, and the review of Deringer et al. (2021) is its reference. The reason is the accountability triple again: data efficiency because each label is a density-functional-theory calculation, a physics prior written into the kernel, and a posterior variance that decides when to compute the next expensive label.

57.3.1 Potentials from a kernel that knows the symmetries

A Gaussian Approximation Potential (Bartók et al. 2010) is Gaussian-process regression of the energy on quantum-mechanical training data, with no fixed functional form; it interpolates the potential-energy surface and is systematically improvable as more data arrive. What made it general is the kernel. The Smooth Overlap of Atomic

Positions descriptor (Bartók, Kondor, and Csányi 2013) defines the similarity between two atomic neighborhoods as the rotationally integrated overlap of their smeared neighbor densities, so the kernel is invariant to rotation, translation, and permutation of like atoms by construction. Those are hard physical symmetries encoded in the kernel itself rather than learned from data, a vivid instance of the book's theme that the kernel is where domain knowledge enters. One such model reproduces density-functional theory across crystalline, liquid, and amorphous silicon and its defects (Bartók et al. 2018), phases where classical force fields fail.

The molecular-property lineage runs in parallel. Kernel ridge regression on the Coulomb-matrix descriptor (Rupp et al. 2012) predicted atomization energies of 7165 small organic molecules with a mean absolute error near 9.9 kcal/mol at a tiny fraction of the quantum cost, and the descriptors that followed, culminating in FCHL (Faber et al. 2018; Christensen et al. 2020), reach chemical accuracy on the QM9 benchmark of Ramakrishnan et al. (2014). The systematic learning curves of Faber et al. (2017) are the evidence for the headline claim: kernel models dominate the small-data regime and reach below hybrid-DFT error. And by learning in the gradient domain with energy conservation imposed exactly, the GDML force fields of Chmiela et al. (2017) reach roughly 0.3 kcal/mol from only a thousand molecular configurations, a data efficiency that comes from pairing the kernel with a physics prior.

Example (a bond length from a handful of points)

The smallest instance of a learned potential. We sample about a dozen points of the Morse potential for the hydrogen molecule (dissociation energy $D_e = 4.75$ eV, equilibrium length $r_e = 0.7416$ angstrom, width $a = 1.942$ inverse angstrom), denser near the well, fit kernel ridge regression with an RBF kernel, and recover the equilibrium bond length as the minimum of the predicted curve. All numbers from `checks/ch-highstakes-ex3.py`.

1. Recover the physical quantity. The minimum of the kernel-ridge curve sits at 0.7425 angstrom, within 0.9 milliangstrom of the true 0.7416, from a dozen points.
2. Read the leave-one-out error as uncertainty. The closed-form leave-one-out errors of the accountability chapter are largest on the steep repulsive wall, 4.365 eV at $r = 2.40$ angstrom, and small in the smooth well, with a median of 0.0234 eV over the set.
3. See where to sample next. The interpolation is hardest exactly where the leave-one-out error is largest, on the repulsive wall, which is precisely where an active learner would place its next expensive point.

Reading. Even at toy scale the pattern of the real force fields is visible: a kernel interpolates a physical curve from few points, and its own leave-one-out error is a map of where it is least trustworthy. Scaled up with the SOAP kernel and a real potential-energy surface, that error map becomes the control signal for the active-learning loop below.

Verification artifact. `checks/example-ch-highstakes-example-56-3.json` records the example source hash and verification scope.

57.3.2 The killer app: variance triggers a calculation

The property that makes kernels indispensable here is that the Gaussian-process posterior variance depends only on where a configuration sits relative to the training data, not on its unknown energy, so it can flag an unfamiliar configuration before the expensive label is computed. The on-the-fly force fields of Vandermause et al. (2020) turn this into a method: a molecular-dynamics simulation runs on the cheap GP energy, and at each step the predictive variance is checked; while it stays low the GP prediction is accepted, and when it spikes at a configuration the model has never seen, a rare event such as a bond breaking, a single density-functional-theory calculation is triggered, added to the training set, and the simulation continues. The same idea in a Bayesian linear-regression form bypasses more than ninety-nine percent of the quantum calculations that a brute-force simulation would require (Jinnouchi et al. 2019). The calibrated variance is the entire control signal, converting a fixed compute budget into an adaptive one, and it is the same variance whose honest reading opened the previous chapter.

Example (uncertainty sampling reaches accuracy sooner)

A one-dimensional toy potential-energy surface with two wells. A Gaussian process (RBF length scale 0.35) starts from three points and, at each step, either places its next sample at the point of maximum posterior variance

(uncertainty sampling) or at a random location. We count how many points each needs to reach a target root-mean-square error of 0.05. All numbers from `checks/ch-highstakes-ex4.py`.

1. Sample where the model is unsure. Uncertainty sampling reaches the target with 19 points total.
2. Sample blindly. Random sampling needs 26.6 points on average over thirty seeds to reach the same error.
3. Count the saving. Active learning uses 0.72 of the random budget, about a quarter fewer expensive evaluations, and the saving grows with the cost and dimension of the label.

Reading. The posterior variance is not decoration; it is a decision rule that spends the experimental budget where it buys the most accuracy. This toy loop is the mechanism of the on-the-fly force fields and, in the next section, of autonomous materials discovery. The honest cost is the cubic GP solve, which forces sparse or low-rank approximations at the scales real simulations reach.

Verification artifact. `checks/example-ch-highstakes-example-56-4.json` records the example source hash and verification scope.

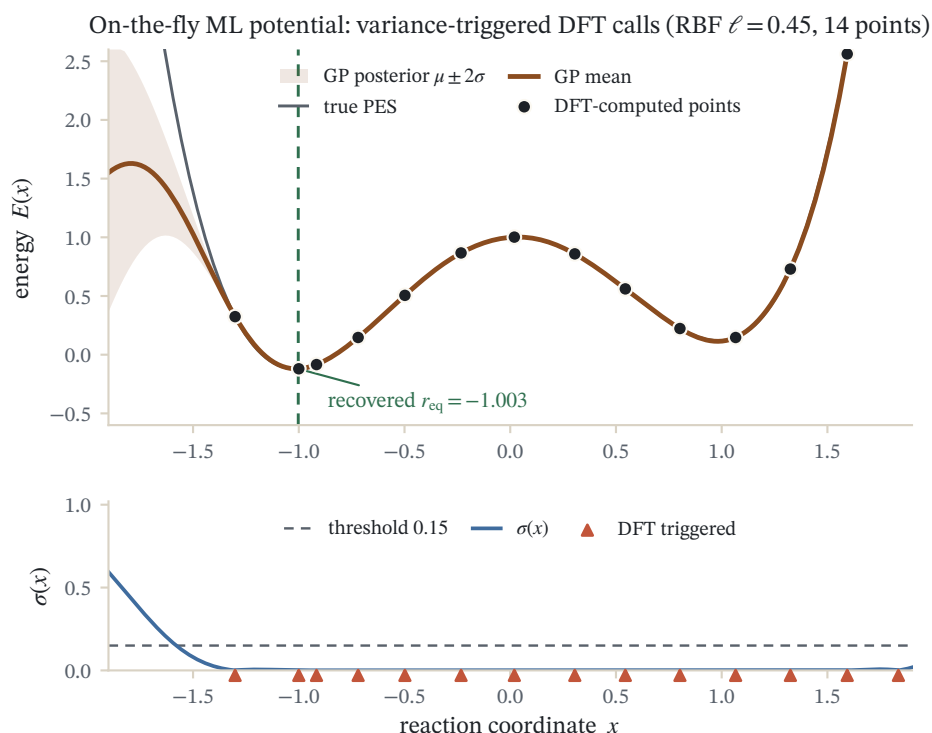


Figure 57.3: An on-the-fly machine-learned interatomic potential: a Gaussian process fits the energy curve and triggers a reference (DFT) call wherever its posterior variance exceeds a threshold, recovering the equilibrium bond length from few evaluations.

57.3.3 Bayesian optimization for discovery

When the goal is not a potential but the best material or reaction, the same posterior drives Bayesian optimization, from its chapter, whose modern form traces to modeling a deterministic simulator as a Gaussian process (Sacks et al. 1989) and whose review is Shahriari et al. (2016). The acquisition function balances exploiting the posterior mean against exploring its variance, so each costly experiment is chosen to be maximally informative. In reaction chemistry, Bayesian optimization over reaction conditions outperformed expert chemists in both efficiency and consistency (Shields et al. 2021). In materials, the adaptive-design loop of Xue et al. (2016) searched roughly eight hundred thousand candidate compositions and, by synthesizing only thirty-six of them chosen by expected improvement, found a new nickel-titanium shape-memory alloy with the smallest thermal hysteresis in the family.

The uncertainty, not just the mean, chose each alloy to make, which is what turned a combinatorial search into a dozen experiments, and the methodology is consolidated in the review of Lookman et al. (2019).

57.4 Earth observation

The third domain turns satellite measurements into maps, and its adoption of kernels tracks the accountability properties once more, through the long line of work surveyed for Gaussian processes by Camps-Valls et al. (2016). Hyperspectral pixels live in hundreds of spectral dimensions but labeled examples are scarce, the small-sample high-dimensional regime where the margin and regularization of a support vector machine shine; Camps-Valls and Bruzzone (2005) made kernel classifiers the remote-sensing standard for exactly this reason. Spatial context enters through the algebra of kernels: a composite kernel $k = \mu k_{\text{spectral}} + (1 - \mu) k_{\text{spatial}}$ is valid because sums of valid kernels are valid (Camps-Valls et al. 2006), and the mixing weight μ is an interpretable, auditable dial for how much neighborhood texture versus pure spectrum drives the decision, a direct use of the Mercer closure properties of the kernel-families chapter.

The flagship is calibrated retrieval. Gaussian-process regression from reflectance to biophysical variables such as leaf area index and chlorophyll content (Verrelst et al. 2012, 2013) delivers a per-pixel predictive variance, which becomes a spatial confidence map: pixels whose spectra fall outside the training distribution, unusual canopies or poor atmospheric correction, are flagged as low-confidence, and for an operational product used in crop monitoring or carbon accounting that confidence is the deliverable, not an extra. The GP matched or beat neural networks on accuracy and supplied the uncertainty for free, and its learned per-band length scales double as a relevance ranking over the spectral bands. Because radiative-transfer models that map biophysical parameters to spectra are slow, the same kernels emulate them: a GP surrogate over a principal-component basis of the model output (Verrelst et al. 2016) runs orders of magnitude faster, is smooth and differentiable for inversion and sensitivity analysis, and flags where it is extrapolating. And the audit tools of the last chapter reappear here in their birthplace: Fair Kernel Learning (Pérez-Suay et al. 2017) uses the HSIC dependence measure to remove a sensor or acquisition bias from a retrieval, the same statistic that turned a fairness check into a hypothesis test.

57.5 What the cases share

Read together, the domains make one argument three times. First, a calibrated posterior covariance is the actual product: a planet mass of 4.73 ± 0.95 Earth masses, a per-pixel confidence map, an orbit covariance a collision-avoidance system trusts, a variance that decides when to run a quantum calculation. Second, data efficiency is a requirement, not a nicety, when a single sample is a space mission, a supercomputer run, or a scarce ground-truth label, and the kernel's smoothness prior is what makes interpolation from few points defensible. Third, the kernel is the explicit, auditable place where the physics is written: a quasi-periodic kernel for evolving spots, a semi-separable kernel for stellar granulation, a rotation-and-permutation-invariant kernel for atomic environments, a composite spatial-spectral kernel for imagery, and a noise-weighted inner product for matched filtering. These are the three structural facts of the accountability chapter cashed out in real science.

The honest limitation is also the same everywhere, and stating it once is the fair close. The exact Gaussian process is cubic in the number of points and assumes a stationary, correctly chosen kernel. That is why every mature application here pairs the method with a scalability structure, the semi-separable celerite covariance, sparse or inducing-point approximations, principal-component bases for emulation, or the Nyström and random-feature methods of the large-scale chapter, and why kernel design and cross-validation are never skipped. The accountability is not free; it costs a solve and a modeling choice, and these sciences pay that cost gladly for an error bar they can defend.

57.6 Summary

Kernel methods earned their place in astronomy, the physical sciences, and Earth observation for the reasons the previous chapter laid out, not despite them. In space and astronomy the quasi-periodic Gaussian process recovers a stellar rotation period with a credible interval, a shared latent GP separates a planet's mass from stellar activity and reports it with an error bar, matched filtering realizes detection as a projection in a kernel inner-product space, and kernel models supply auditable anomaly scores and orbit covariances. In molecules and materials the SOAP kernel writes rotational and permutational symmetry into a Gaussian Approximation Potential, kernel ridge regression reaches below hybrid-DFT error in the small-data regime, and the posterior variance drives on-the-fly force fields and autonomous discovery by triggering an expensive calculation exactly where the model is unsure. In Earth observation kernel classifiers handle the high-dimensional few-label regime, composite kernels fold in spatial context through Mercer closure, and Gaussian-process retrieval turns every pixel into a value with a confidence. The recurring requirement is a calibrated covariance, the recurring enabler is data efficiency, and the recurring design principle is that the kernel holds the physics; the recurring cost is the cubic solve and the need for a well-chosen, stationary kernel, paid with scalability structure. The book's through-line reaches its most consequential form here: fix a positive definite kernel that encodes what you know, and a great deal of high-stakes science becomes a linear-algebra problem whose answer arrives with the error bar attached.

57.7 Exercises

1. warm-up For each application, name the kernel property that earned it its place (calibrated uncertainty, data efficiency, or physics encoded in the kernel), in one phrase: (a) an on-the-fly interatomic potential; (b) matched filtering for gravitational waves; (c) a per-pixel satellite chlorophyll map; (d) recovering a stellar rotation period; (e) a Bayesian-optimization search for a new alloy. Some have more than one; give the dominant one.
2. computation In the matched-filter example the optimal SNR is $\rho = \|h\|/\sigma$ and the normalized statistic at the true lag is ρ plus a standard normal. With $\|h\| = 7.372$ and $\sigma = 0.9215$, verify $\rho = 8$, and compute the probability that a single pure-noise sample of the statistic exceeds the observed peak of 7.37. Why does the loudest of many correlated noise lags still stay far below 7.37? Hint

$7.372/0.9215 = 8.0$. A unit-normal exceeds 7.37 with probability about 9×10^{-14} . The template width makes adjacent lags highly correlated, so the number of effectively independent noise trials is far smaller than the number of lags, and the expected maximum is a few, not eight.

3. proof Show that the composite kernel $k = \mu k_1 + (1 - \mu) k_2$ with $\mu \in [0, 1]$ is positive definite whenever k_1, k_2 are, and explain why the mixing weight μ is therefore a safe, interpretable dial in the spatial-spectral remote-sensing kernel. What goes wrong if μ is allowed outside $[0, 1]$? Hint

A nonnegative combination of positive definite kernels is positive definite, since the Gram matrix is the same nonnegative combination of positive semidefinite matrices. Outside $[0, 1]$ a negative coefficient can destroy positive definiteness, so the "kernel" is no longer a valid inner product.

4. exploration Reproduce the Morse-potential fit and study data efficiency: as you thin the training points from twelve to five, track the recovered bond length and the median leave-one-out error, and identify which points are most costly to remove. Relate your finding to why an active learner samples the repulsive wall first. Hint

Removing points on the steep wall degrades the fit fastest because the curvature is highest there and the kernel must work hardest; the well is nearly flat and tolerates sparser sampling. The leave-one-out error you computed already ranks the wall points as the least redundant.

5. challenge The on-the-fly force field trusts the GP prediction while the posterior variance is below a threshold and calls quantum mechanics when it exceeds it. Frame this as a decision rule with a cost for a wrong

prediction and a cost for a quantum call, and derive how the threshold should depend on the two costs. What failure mode arises if the kernel is misspecified so the variance is not calibrated? Hint

Call quantum mechanics when the expected cost of trusting the GP (roughly the miscoverage probability times the error cost) exceeds the call cost; the threshold falls as the error cost rises. If the variance is miscalibrated low, the rule accepts predictions it should not, and the simulation drifts into unphysical regions unnoticed, the deployment analogue of the previous chapter's under-coverage.

6. exploration The chapter claims the exact GP is cubic and every mature application pairs it with a scalability structure. Pick two, celerite's semi-separable covariance and inducing-point sparse GPs, and explain what structural assumption each exploits and what it gives up, connecting them to the Nyström and random-feature trade-offs of the large-scale chapter. Hint

Celerite restricts the kernel to a sum of (complex) exponentials on one-dimensional inputs, buying an exact linear-time likelihood at the cost of generality; inducing points summarize the data by $m \ll n$ pseudo-points, an approximation whose fidelity is governed by the effective dimension, the same quantity that sets how many Nyström landmarks suffice.

- Aizerman, Braverman, M. A., and L. I. Rozonoer. 1964. *Theoretical Foundations of the Potential Function Method in Pattern Recognition Learning*. Automation and Remote Control 25, 821-837. <https://ci.nii.ac.jp/naid/10021200712/>.
- Álvarez, Mauricio A., Lorenzo Rosasco, and Neil D. Lawrence. 2012. "Kernels for Vector-Valued Functions: A Review." *Foundations and Trends in Machine Learning* 4 (3): 195–266. <https://doi.org/10.1561/22000000036>.
- Andrew, Arora, G., and K. Livescu. 2013. *Deep Canonical Correlation Analysis*. Proceedings of the 30th International Conference on Machine Learning (ICML), PMLR 28(3), 1247-1255. <https://proceedings.mlr.press/v28/andrew13.html>.
- Aronszajn, N. 1950. *Theory of Reproducing Kernels*. Transactions of the American Mathematical Society 68(3), 337-404. <https://doi.org/10.1090/s0002-9947-1950-0051437-7>.
- Arora, Du, S., and R. Wang. 2019. *On Exact Computation with an Infinitely Wide Neural Net*. NeurIPS 32. <https://arxiv.org/pdf/1904.11955v2>.
- Bach, F. R., and M. I. Jordan. 2002. *Kernel Independent Component Analysis*. Journal of Machine Learning Research 3, 1-48. <https://www.jmlr.org/papers/v3/bach02a.html>.
- Baker, C. R. 1973. *Joint Measures and Cross-Covariance Operators*. Transactions of the American Mathematical Society 186, 273-289. <https://doi.org/10.1090/s0002-9947-1973-0336795-3>.
- Bartlett, Jordan, P. L., and J. D. McAuliffe. 2006. *Convexity, Classification, and Risk Bounds*. Journal of the American Statistical Association 101(473), 138-156. <https://doi.org/10.1198/016214505000000907>.
- Bartlett, P. L., and S. Mendelson. 2002. *Rademacher and Gaussian Complexities: Risk Bounds and Structural Results*. Journal of Machine Learning Research 3, 463-482. https://doi.org/10.1007/3-540-44581-1_15.
- Belkin, Hsu, M., and S. Mandal. 2019. *Reconciling Modern Machine-Learning Practice and the Classical Bias-Variance Trade-Off*. Proceedings of the National Academy of Sciences 116(32), 15849-15854. <https://doi.org/10.1073/pnas.1903070116>.
- Belkin, Ma, M., and S. Mandal. 2018. *To Understand Deep Learning We Need to Understand Kernel Learning*. ICML. <https://doi.org/10.48550/arxiv.1802.01396>.
- Belkin, Mikhail, Partha Niyogi, and Vikas Sindhwani. 2006. "Manifold Regularization: A Geometric Framework for Learning from Labeled and Unlabeled Examples." *Journal of Machine Learning Research* 7: 2399–434.

- <https://www.jmlr.org/papers/v7/belkin06a.html>.
- Borgwardt, K. M., and H.-P. Kriegel. 2005. *Shortest-Path Kernels on Graphs*. ICDM, 74-81. <https://doi.org/10.1109/icdm.2005.132>.
- Borovitskiy, Terenin, V., and M. P. Deisenroth. 2020. *Matérn Gaussian Processes on Riemannian Manifolds*. NeurIPS 33. <https://doi.org/10.48550/arxiv.2006.10160>.
- Burnaev, Evgeny, and Vladimir Vovk. 2014. “Efficiency of Conformalized Ridge Regression.” *Proceedings of Machine Learning Research* 35: 605–22. <https://proceedings.mlr.press/v35/burnaev14.html>.
- Caponnetto, A., and E. De Vito. 2007. *Optimal Rates for the Regularized Least-Squares Algorithm*. Foundations of Computational Mathematics 7(3), 331-368. <https://doi.org/10.1007/s10208-006-0196-8>.
- Chaudhuri, Kamalika, Claire Monteleoni, and Anand D. Sarwate. 2011. “Differentially Private Empirical Risk Minimization.” *Journal of Machine Learning Research* 12 (29): 1069–109. <https://www.jmlr.org/papers/v12/chaudhuri11a.html>.
- Chen, Garcia, Y., and L. Cazzanti. 2009. *Similarity-Based Classification: Concepts and Algorithms*. Journal of Machine Learning Research 10, 747-776. <https://doi.org/10.5555/1577069.1577096>.
- Chen, K.-T. 1958. *Integration of Paths, a Faithful Representation of Paths by Noncommutative Formal Power Series*. Transactions of the American Mathematical Society 89(2), 395-407. <https://doi.org/10.2307/1993193>.
- Chizat, Oyallon, L., and F. Bach. 2019. *On Lazy Training in Differentiable Programming*. NeurIPS 32. <https://doi.org/10.48550/arxiv.1812.07956>.
- Cho, Y., and L. K. Saul. 2009. *Kernel Methods for Deep Learning*. NeurIPS 22. <https://escholarship.org/uc/item/9dr6q1w8>.
- Christmann, A., and I. Steinwart. 2010. *Universal Kernels on Non-Standard Input Spaces*. NeurIPS 23, 406-414. https://papers.nips.cc/paper_files/paper/2010/hash/4e0cb6fb5fb446d1c92ede2ed8780188-Abstract.html.
- Chwialkowski, Ramdas, K. P., and A. Gretton. 2015. *Fast Two-Sample Testing with Analytic Representations of Probability Measures*. NeurIPS 28, 1981-1989. <https://doi.org/10.48550/arxiv.1506.04725>.
- Cockayne, Jon, Chris J. Oates, Timothy J. Sullivan, and Mark Girolami. 2019. “Bayesian Probabilistic Numerical Methods.” *SIAM Review* 61 (4): 756–89. <https://doi.org/10.1137/17M1139357>.
- Cover, T. M. 1965. *Geometrical and Statistical Properties of Systems of Linear Inequalities with Applications in Pattern Recognition*. IEEE Transactions on Electronic Computers EC-14(3), 326-334. <https://doi.org/10.1109/pgec.1965.264137>.
- Crammer, Koby, Ofer Dekel, Joseph Keshet, Shai Shalev-Shwartz, and Yoram Singer. 2006. “Online Passive-Aggressive Algorithms.” *Journal of Machine Learning Research* 7: 551–85. <https://www.jmlr.org/papers/v7/crammer06a.html>.
- Craven, P., and G. Wahba. 1979. *Smoothing Noisy Data with Spline Functions: Estimating the Correct Degree of Smoothing by the Method of Generalized Cross-Validation*. Numerische Mathematik 31(4), 377-403. <https://doi.org/10.1007/BF01404567>.
- Cucker, F., and S. Smale. 2002. *On the Mathematical Foundations of Learning*. Bulletin of the American Mathematical Society 39(1), 1-49. <https://doi.org/10.1090/s0273-0979-01-00923-5>.

- Dhillon, Guan, I. S., and B. Kulis. 2004. *Kernel k -Means, Spectral Clustering and Normalized Cuts*. Proceedings of the 10th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 551-556. <https://doi.org/10.1145/1014052.1014118>.
- Drineas, P., and M. W. Mahoney. 2005. *On the Nyström Method for Approximating a Gram Matrix for Improved Kernel-Based Learning*. Journal of Machine Learning Research 6, 2153-2175. <https://jmlr.org/papers/v6/drineas05a.html>.
- Drucker, Burges, H., and V. N. Vapnik. 1997. *Support Vector Regression Machines*. Advances in Neural Information Processing Systems 9, 155-161. https://proceedings.neurips.cc/paper_files/paper/1996/hash/d38901788c533e8286cb6400b40b386d-Abstract.html.
- Durbin, Eddy, R., and G. Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511790492>.
- Eleftheriadis, Stefanos, Tom Nicholson, Marc Deisenroth, and James Hensman. 2017. "Identification of Gaussian Process State Space Models." *Advances in Neural Information Processing Systems* 30. <https://proceedings.neurips.cc/paper/2017/hash/1006ff12c465532f8c574aeaa4461b16-Abstract.html>.
- Farahmand, Amir-massoud, Mohammad Ghavamzadeh, Csaba Szepesvári, and Shie Mannor. 2016. "Regularized Policy Iteration with Nonparametric Function Spaces." *Journal of Machine Learning Research* 17 (139): 1-66. <https://www.jmlr.org/papers/v17/13-016.html>.
- Feng, Xingdong, Xin He, Caixing Wang, Chao Wang, and Jingnan Zhang. 2023. "Towards a Unified Analysis of Kernel-Based Methods Under Covariate Shift." *Advances in Neural Information Processing Systems* 36. https://proceedings.neurips.cc/paper_files/paper/2023/hash/e9b0ae84d6879b30c78cb8537466a4e0-Abstract-Conference.html.
- Fisher, R. A. 1936. *The Use of Multiple Measurements in Taxonomic Problems*. Annals of Eugenics 7(2), 179-188. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>.
- Frazier, P. I. 2018. *A Tutorial on Bayesian Optimization*. arXiv:1807.02811. <https://arxiv.org/abs/1807.02811>.
- Freund, Iyer, Y., and Y. Singer. 2003. *An Efficient Boosting Algorithm for Combining Preferences*. Journal of Machine Learning Research 4, 933-969. <https://www.jmlr.org/papers/v4/freund03a.html>.
- Fukumizu, Bach, K., and A. Gretton. 2007. *Statistical Consistency of Kernel Canonical Correlation Analysis*. Journal of Machine Learning Research 8, 361-383. <https://dl.acm.org/doi/10.5555/1314498.1314512>.
- Fukumizu, Bach, K., and M. I. Jordan. 2004. *Dimensionality Reduction for Supervised Learning with Reproducing Kernel Hilbert Spaces*. Journal of Machine Learning Research 5, 73-99. <https://doi.org/10.21236/ada446572>.
- Fukumizu, Gretton, K., and B. Schölkopf. 2008. *Kernel Measures of Conditional Dependence*. NeurIPS 20, 489-496. <https://proceedings.neurips.cc/paper/2007/hash/3a0772443a0739141292a5429b952fe6-Abstract.html>.
- Gardner, Jacob, Matt Kusner, Zhixiang Xu, Kilian Weinberger, and John Cunningham. 2014. "Bayesian Optimization with Inequality Constraints." *Proceedings of the 31st International Conference on Machine Learning*, 937-45. <https://proceedings.mlr.press/v32/gardner14.html>.
- Gärtner, Flach, T., and S. Wrobel. 2003. *On Graph Kernels: Hardness Results and Efficient Alternatives*. COLT/Kernel Workshop. https://doi.org/10.1007/978-3-540-45167-9_11.
- Gneiting, Tilmann. 2002. "Nonseparable, Stationary Covariance Functions for Space-Time Data." *Journal of the American Statistical Association* 97 (458): 590-600. <https://doi.org/10.1198/016214502760047113>.

- Green, P. J. 1984. *Iteratively Reweighted Least Squares for Maximum Likelihood Estimation, and Some Robust and Resistant Alternatives*. Journal of the Royal Statistical Society, Series B 46(2), 149-192. <https://doi.org/10.1111/j.2517-6161.1984.tb01288.x>.
- Gretton, Borgwardt, A., and A. Smola. 2006. *A Kernel Method for the Two-Sample Problem*. NeurIPS 19. <https://doi.org/10.7551/mitpress/7503.003.0069>.
- Gretton, Borgwardt, A., and A. Smola. 2012. *A Kernel Two-Sample Test*. Journal of Machine Learning Research 13, 723-773. <https://www.jmlr.org/papers/v13/gretton12a.html>.
- Gretton, Bousquet, A., and B. Schölkopf. 2005. *Measuring Statistical Dependence with Hilbert-Schmidt Norms*. Algorithmic Learning Theory (ALT). https://doi.org/10.1007/11564089_7.
- Haasdonk, B. 2005. *Feature Space Interpretation of SVMs with Indefinite Kernels*. IEEE Transactions on Pattern Analysis and Machine Intelligence 27(4), 482-492. <https://doi.org/10.1109/tpami.2005.78>.
- Hambly, B., and T. Lyons. 2010. *Uniqueness for the Signature of a Path of Bounded Variation and the Reduced Path Group*. Annals of Mathematics 171(1), 109-167. <https://doi.org/10.4007/annals.2010.171.109>.
- Hastie, Montanari, T., and R. J. Tibshirani. 2019. *Surprises in High-Dimensional Ridgeless Least Squares Interpolation*. arXiv:1903.08560; Annals of Statistics 50(2), 949-986 (2022). <https://arxiv.org/abs/1903.08560>.
- Haussler, D. 1999. *Convolution Kernels on Discrete Structures*. Technical Report UCSC-CRL-99-10, UC Santa Cruz. <https://stemcellgenomics.ucsc.edu/miscellaneous-texts/>.
- Havlíček, Vojtěch, Antonio D. Córcoles, Kristan Temme, et al. 2019. “Supervised Learning with Quantum-Enhanced Feature Spaces.” *Nature* 567: 209–12. <https://doi.org/10.1038/s41586-019-0980-2>.
- Herbrich, Graepel, R., and K. Obermayer. 2000. *Large Margin Rank Boundaries for Ordinal Regression*. Advances in Large Margin Classifiers, MIT Press, 115-132. <https://doi.org/10.7551/mitpress/1113.003.0010>.
- Jaakkola, Diekhans, T., and D. Haussler. 2000. *A Discriminative Framework for Detecting Remote Protein Homologies*. Journal of Computational Biology 7(1-2), 95-114. <https://doi.org/10.1089/10665270050081405>.
- Jacot, Gabriel, A., and C. Hongler. 2018. *Neural Tangent Kernel: Convergence and Generalization in Neural Networks*. NeurIPS 31. <https://proceedings.neurips.cc/paper/2018/hash/5a4be1fa34e62bb8a6ec6b91d2462f5a-Abstract.html>.
- Joachims, T. 1998. *Text Categorization with Support Vector Machines: Learning with Many Relevant Features*. Proceedings of the European Conference on Machine Learning (ECML), 137-142. <https://doi.org/10.1007/bf0026683>.
- Joachims, T. 1999. *Making Large-Scale SVM Learning Practical*. Advances in Kernel Methods: Support Vector Learning, MIT Press, 169-184. <https://doi.org/10.17877/DE290R-14262>.
- Joachims, T. 2002. *Optimizing Search Engines Using Clickthrough Data*. Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 133-142. <https://doi.org/10.1145/775066.775067>.
- Kandasamy, Kirthivasan, Gautam Dasarathy, Junier Oliva, Jeff Schneider, and Barnabas Poczos. 2017. “Multi-Fidelity Bayesian Optimisation with Continuous Approximations.” *Proceedings of the 34th International Conference on Machine Learning*, 1799–808. <https://proceedings.mlr.press/v70/kandasamy17a.html>.
- Kantorovich, L. V. 1942. *On the Translocation of Masses*. Doklady Akademii Nauk SSSR 37, 199-201 (English

- translation in *Management Science* 5 (1958), 1-4). <https://www.mathnet.ru/eng/zns1/v312/p11>.
- Kashima, Tsuda, H., and A. Inokuchi. 2003. *Marginalized Kernels Between Labeled Graphs*. ICML, 321-328. <https://www.ed.aaai.org/Library/ICML/2003/icml03-044.php>.
- Kimeldorf, G., and G. Wahba. 1971. *Some Results on Tchebycheffian Spline Functions*. *Journal of Mathematical Analysis and Applications* 33(1), 82-95. [https://doi.org/10.1016/0022-247x\(71\)90184-3](https://doi.org/10.1016/0022-247x(71)90184-3).
- Krause, Andreas, Ajit Singh, and Carlos Guestrin. 2008. "Near-Optimal Sensor Placements in Gaussian Processes: Theory, Efficient Algorithms and Empirical Studies." *Journal of Machine Learning Research* 9: 235-84. <https://www.jmlr.org/papers/v9/krause08a.html>.
- Lanckriet, Cristianini, G. R. G., and M. I. Jordan. 2004. *Learning the Kernel Matrix with Semidefinite Programming*. *Journal of Machine Learning Research* 5, 27-72. <https://jmlr.org/papers/v5/lanckriet04a.html>.
- Lanckriet, De Bie, G. R. G., and W. S. Noble. 2004. *A Statistical Framework for Genomic Data Fusion*. *Bioinformatics* 20(16), 2626-2635. <https://doi.org/10.1093/bioinformatics/bth294>.
- LeCun, Jackel, Y., and V. Vapnik. 1995. *Comparison of Learning Algorithms for Handwritten Digit Recognition*. International Conference on Artificial Neural Networks (ICANN), 53-60. <https://yann.lecun.com/exdb/publis/>.
- Leslie, Eskin, C., and W. S. Noble. 2002. *The Spectrum Kernel: A String Kernel for SVM Protein Classification*. *Pacific Symposium on Biocomputing* 7, 564-575. https://doi.org/10.1142/9789812799623_0053.
- Leslie, Eskin, C., and W. S. Noble. 2004. *Mismatch String Kernels for Discriminative Protein Classification*. *Bioinformatics* 20(4). <https://doi.org/10.1093/bioinformatics/btg431>.
- Li, Kovachki, Z., and A. Anandkumar. 2020. *Fourier Neural Operator for Parametric Partial Differential Equations*. ICLR 2021. <https://openreview.net/forum?id=c8P9NQVtmnO>.
- Lindgren, Finn, Håvard Rue, and Johan Lindström. 2011. "An Explicit Link Between Gaussian Fields and Gaussian Markov Random Fields: The Stochastic Partial Differential Equation Approach." *Journal of the Royal Statistical Society: Series B* 73 (4): 423-98. <https://doi.org/10.1111/j.1467-9868.2011.00777.x>.
- Lindgren, Rue, F., and J. Lindström. 2011. *An Explicit Link Between Gaussian Fields and Gaussian Markov Random Fields: The Stochastic Partial Differential Equation Approach*. *Journal of the Royal Statistical Society: Series B* 73(4), 423-498. <https://doi.org/10.1111/j.1467-9868.2011.00777.x>.
- Lodhi, Saunders, H., and C. Watkins. 2002. *Text Classification Using String Kernels*. *Journal of Machine Learning Research* 2, 419-444. <https://www.jmlr.org/papers/v2/lodhi02a.html>.
- Luxburg, U. von. 2007. *A Tutorial on Spectral Clustering*. *Statistics and Computing* 17(4), 395-416. <https://doi.org/10.1007/s11222-007-9033-z>.
- Lyons, T. J. 1998. *Differential Equations Driven by Rough Signals*. *Revista Matemática Iberoamericana* 14(2), 215-310. <https://doi.org/10.4171/rmi/240>.
- Matérn, B. 1960. *Spatial Variation: Stochastic Models and Their Application to Some Problems in Forest Surveys and Other Sampling Investigations*. Meddelanden från Statens Skogsforskningsinstitut 49(5); reprinted 1986 as *Lecture Notes in Statistics* 36, Springer. <https://doi.org/10.1007/978-1-4615-7892-5>.
- Mercer, J. 1909. *Functions of Positive and Negative Type, and Their Connection with the Theory of Integral Equations*. *Philosophical Transactions of the Royal Society A* 209, 415-446. <https://doi.org/10.1098/rspa.1909.0075>.

- Micchelli, Charles A., and Massimiliano Pontil. 2005. "On Learning Vector-Valued Functions." *Neural Computation* 17 (1): 177–204. <https://doi.org/10.1162/0899766052530802>.
- Mika, Rättsch, S., and K.-R. Müller. 1999. *Fisher Discriminant Analysis with Kernels*. Neural Networks for Signal Processing IX, 41–48. <https://doi.org/10.1109/nnsp.1999.788121>.
- Mika, Schölkopf, S., and G. Rättsch. 1999. *Kernel PCA and de-Noising in Feature Spaces*. Advances in Neural Information Processing Systems 11, 536–542. <https://proceedings.neurips.cc/paper/1998/hash/226d1f15ecd35f784d2a20c3ecf56d7f-Abstract.html>.
- Muandet, Fukumizu, K., and B. Schölkopf. 2012. *Learning from Distributions via Support Measure Machines*. NeurIPS 25, 10–18. <https://proceedings.neurips.cc/paper/2012/hash/9bf31c7ff062936a96d3c8bd1f8f2ff3-Abstract.html>.
- Neal, R. M. 1996. *Bayesian Learning for Neural Networks*. Springer, Lecture Notes in Statistics 118. <https://doi.org/10.1007/978-1-4612-0745-0>.
- Newey, Whitney K., and James L. Powell. 1987. "Asymmetric Least Squares Estimation and Testing." *Econometrica* 55 (4): 819–47. <https://doi.org/10.2307/1911031>.
- Neyshabur, Bhojanapalli, B., and N. Srebro. 2018. *A PAC-Bayesian Approach to Spectrally-Normalized Margin Bounds for Neural Networks*. ICLR. <https://arxiv.org/abs/1707.09564>.
- Ng, Jordan, A. Y., and Y. Weiss. 2002. *On Spectral Clustering: Analysis and an Algorithm*. Advances in Neural Information Processing Systems 14, 849–856. <https://proceedings.neurips.cc/paper/2001/hash/801272ee79cfd e7fa5960571fee36b9b-Abstract.html>.
- Novikoff, A. B. J. 1962. *On Convergence Proofs on Perceptrons*. Proceedings of the Symposium on the Mathematical Theory of Automata 12, 615–622, Polytechnic Institute of Brooklyn.
- O'Hagan, A. 1991. *Bayes-Hermite Quadrature*. Journal of Statistical Planning and Inference 29(3), 245–260. [https://doi.org/10.1016/0378-3758\(91\)90002-v](https://doi.org/10.1016/0378-3758(91)90002-v).
- Ober, Sebastian W., Carl E. Rasmussen, and Mark van der Wilk. 2021. "The Promises and Pitfalls of Deep Kernel Learning." *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence* 161: 1206–16. <https://proceedings.mlr.press/v161/ober21a.html>.
- Ong, Mary, C. S., and A. J. Smola. 2004. *Learning with Non-Positive Kernels*. Proceedings of the 21st International Conference on Machine Learning (ICML), 639–646. <https://doi.org/10.1145/1015330.1015443>.
- Osuna, Freund, E., and F. Girosi. 1997. *An Improved Training Algorithm for Support Vector Machines*. Neural Networks for Signal Processing VII (IEEE Workshop), 276–285. <https://doi.org/10.1109/nnsp.1997.622408>.
- Parhi, Rahul, and Robert D. Nowak. 2021. "Banach Space Representer Theorems for Neural Networks and Ridge Splines." *Journal of Machine Learning Research* 22 (43): 1–40. <https://www.jmlr.org/papers/v22/20-583.html>.
- Platt, J. C. 1998. *Sequential Minimal Optimization: A Fast Algorithm for Training Support Vector Machines*. Microsoft Research Technical Report MSR-TR-98-14. <https://www.microsoft.com/en-us/research/?p=152322>.
- Rahimi, A., and B. Recht. 2007. *Random Features for Large-Scale Kernel Machines*. NeurIPS 20. https://proceedings.neurips.cc/paper_files/paper/2007/hash/013a006f03dbc5392effeb8f18fda755-Abstract.html.
- Raissi, Maziar, and George Em Karniadakis. 2017. "Machine Learning of Linear Differential Equations Using Gaussian Processes." *Journal of Computational Physics* 348: 683–93. <https://doi.org/10.1016/j.jcp.2017.07.050>.

- Raskutti, Garvesh, Martin J. Wainwright, and Bin Yu. 2014. “Early Stopping and Non-Parametric Regression: An Optimal Data-Dependent Stopping Rule.” *Journal of Machine Learning Research* 15 (11): 335–66. <https://www.jmlr.org/papers/v15/raskutti14a.html>.
- Rasmussen, C. E., and Z. Ghahramani. 2003. *Bayesian Monte Carlo*. Advances in Neural Information Processing Systems (NeurIPS) 15, 505–512. https://proceedings.neurips.cc/paper_files/paper/2002/hash/24917db15c4e37e421866448c9ab23d8-Abstract.html.
- Rasmussen, C. E., and C. K. I. Williams. 2006. *Gaussian Processes for Machine Learning*. MIT Press. <https://doi.org/10.7551/mitpress/3206.001.0001>.
- Rosasco, Lorenzo, Andrea Caponnetto, Ernesto De Vito, Francesca Odone, and Umberto De Giovannini. 2004. “Learning, Regularization and Ill-Posed Inverse Problems.” *Advances in Neural Information Processing Systems* 17. https://papers.neurips.cc/paper_files/paper/2004/hash/33267e5dc58fad346e92471c43fccdc-Abstract.html.
- Rosenblatt, F. 1958. *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*. Psychological Review 65(6), 386–408. <https://doi.org/10.1037/h0042519>.
- Rowley, Clarence W., Igor Mezić, Shervin Bagheri, Philipp Schlatter, and Dan S. Henningson. 2009. “Spectral Analysis of Nonlinear Flows.” *Journal of Fluid Mechanics* 641: 115–27. <https://doi.org/10.1017/S0022112009992059>.
- Salton, G. 1988. *Automatic Text Processing: The Transformation, Analysis, and Retrieval of Information by Computer*. Addison-Wesley. <https://doi.org/10.5860/choice.27-0351>.
- Salton, Wong, G., and C. S. Yang. 1975. *A Vector Space Model for Automatic Indexing*. Communications of the ACM 18(11), 613–620. <https://doi.org/10.1145/361219.361220>.
- Schölkopf, B., and A. J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press. <https://mitpress.mit.edu/9780262536578/learning-with-kernels/>.
- Schölkopf, Burges, B., and V. Vapnik. 1996. *Incorporating Invariances in Support Vector Learning Machines*. International Conference on Artificial Neural Networks (ICANN), LNCS 1112, 47–52, Springer. https://doi.org/10.1007/3-540-61510-5_12.
- Schölkopf, Platt, B., and R. C. Williamson. 2001. *Estimating the Support of a High-Dimensional Distribution*. Neural Computation 13(7), 1443–1471. <https://doi.org/10.1162/089976601750264965>.
- Schölkopf, Smola, B., and P. L. Bartlett. 2000. *New Support Vector Algorithms*. Neural Computation 12(5), 1207–1245. <https://doi.org/10.1162/089976600300015565>.
- Schölkopf, Smola, B., and K.-R. Müller. 1998. *Nonlinear Component Analysis as a Kernel Eigenvalue Problem*. Neural Computation 10(5), 1299–1319. <https://doi.org/10.1162/089976698300017467>.
- Schrab, Kim, A., and A. Gretton. 2021. *MMD Aggregated Two-Sample Test*. arXiv:2110.15073; Journal of Machine Learning Research 24(194), 1–81 (2023). <https://arxiv.org/abs/2110.15073>.
- Sejdinovic, Sriperumbudur, D., and K. Fukumizu. 2013. *Equivalence of Distance-Based and RKHS-Based Statistics in Hypothesis Testing*. Annals of Statistics 41(5), 2263–2291. <https://doi.org/10.1214/13-aos1140>.
- Serfling, R. J. 1980. *Approximation Theorems of Mathematical Statistics*. Wiley, New York. <https://doi.org/10.1002/9780470316481>.

- Shahriari, Swersky, B., and N. de Freitas. 2016. *Taking the Human Out of the Loop: A Review of Bayesian Optimization*. Proceedings of the IEEE 104(1), 148-175. <https://doi.org/10.1109/jproc.2015.2494218>.
- Shawe-Taylor, J., and N. Cristianini. 2004. *Kernel Methods for Pattern Analysis*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511809682>.
- Simard, LeCun, P. Y., and B. Victorri. 1998. *Transformation Invariance in Pattern Recognition: Tangent Distance and Tangent Propagation*. Neural Networks: Tricks of the Trade, LNCS 1524, 239-274, Springer. https://doi.org/10.1007/3-540-49430-8_13.
- Song, Fukumizu, L., and A. Gretton. 2013. *Kernel Embeddings of Conditional Distributions*. IEEE Signal Processing Magazine 30(4), 98-111. <https://doi.org/10.1109/MSP.2013.2252713>.
- Song, Huang, L., and K. Fukumizu. 2009. *Hilbert Space Embeddings of Conditional Distributions with Applications to Dynamical Systems*. Proceedings of the 26th International Conference on Machine Learning (ICML), 961-968. <https://doi.org/10.1145/1553374.1553497>.
- Srinivas, Krause, N., and M. Seeger. 2010. *Gaussian Process Optimization in the Bandit Setting: No Regret and Experimental Design*. Proceedings of the 27th International Conference on Machine Learning (ICML), 1015-1022. <https://arxiv.org/abs/0912.3995>.
- Sriperumbudur, Fukumizu, B. K., and G. R. G. Lanckriet. 2012. *On the Empirical Estimation of Integral Probability Metrics*. Electronic Journal of Statistics 6, 1550-1599. <https://doi.org/10.1214/12-ejs722>.
- Sriperumbudur, Gretton, B. K., and G. Lanckriet. 2010. *Hilbert Space Embeddings and Metrics on Probability Measures*. Journal of Machine Learning Research 11, 1517-1561. <https://www.jmlr.org/papers/v11/sriperumbudur10a.html>.
- Stein, M. L. 1999. *Interpolation of Spatial Data: Some Theory for Kriging*. Springer. [https://doi.org/10.1016/s0016-7061\(00\)00010-0](https://doi.org/10.1016/s0016-7061(00)00010-0).
- Sui, Yanan, Alkis Gotovos, Joel Burdick, and Andreas Krause. 2015. "Safe Exploration for Optimization with Gaussian Processes." *Proceedings of the 32nd International Conference on Machine Learning*, 997-1005. <https://proceedings.mlr.press/v37/sui15.html>.
- Szabó, Sriperumbudur, Z., and A. Gretton. 2016. *Learning Theory for Distribution Regression*. Journal of Machine Learning Research 17(152), 1-40. <https://www.jmlr.org/papers/v17/14-510.html>.
- Tax, D. M. J., and R. P. W. Duin. 2004. *Support Vector Data Description*. Machine Learning 54(1), 45-66. <https://doi.org/10.1023/b:mach.0000008084.60811.49>.
- Torgerson, W. S. 1952. *Multidimensional Scaling: I. Theory and Method*. Psychometrika 17(4), 401-419. <https://doi.org/10.1007/bf02288916>.
- Tsuda, Kin, K., and K. Asai. 2003. *Marginalized Kernels for Biological Sequences*. Bioinformatics 18(Suppl. 1), S268-S275. https://doi.org/10.1093/bioinformatics/18.suppl_1.s268.
- Vapnik, V. N. 1995. *The Nature of Statistical Learning Theory*. Springer. <https://doi.org/10.1007/978-1-4757-2440-0>.
- Vapnik, V. N. 1998. *Statistical Learning Theory*. Wiley. <https://doi.org/10.1080/00401706.1999.10485951>.
- Vapnik, V. N., and A. Ya. Chervonenkis. 1971. *On the Uniform Convergence of Relative Frequencies of Events to Their Probabilities*. Theory of Probability and Its Applications 16(2), 264-280. <https://doi.org/10.1137/1116025>.

- Villani, C. 2009. *Optimal Transport: Old and New*. Grundlehren der mathematischen Wissenschaften 338, Springer. <https://doi.org/10.1007/978-3-540-71050-9>.
- Von Oswald, Johannes, Eyvind Niklasson, Ettore Randazzo, et al. 2023. “Transformers Learn in-Context by Gradient Descent.” *Proceedings of the 40th International Conference on Machine Learning*, 35151–74. <https://proceedings.mlr.press/v202/von-oswald23a.html>.
- Wahba, G. 1990. *Spline Models for Observational Data*. SIAM, CBMS-NSF Regional Conference Series in Applied Mathematics 59. <https://doi.org/10.1137/1.9781611970128>.
- Wang, Rui, Yuesheng Xu, and Mingsong Yan. 2024. “Sparse Representer Theorems for Learning in Reproducing Kernel Banach Spaces.” *Journal of Machine Learning Research* 25 (93): 1–45. <https://www.jmlr.org/papers/v25/23-0645.html>.
- Watkins, C. 2000. *Dynamic Alignment Kernels*. Advances in Large Margin Classifiers, MIT Press, 39–50. <https://doi.org/10.7551/mitpress/1113.003.0006>.
- Welling, M. 2009. *Herding Dynamical Weights to Learn*. International Conference on Machine Learning (ICML) 26, 1121–1128. <https://doi.org/10.1145/1553374.1553517>.
- Wendland, H. 2005. *Scattered Data Approximation*. Cambridge University Press. <https://doi.org/10.1017/cbo9780511617539>.
- Whittle, P. 1954. *On Stationary Processes in the Plane*. Biometrika 41(3–4), 434–449. <https://doi.org/10.2307/2332724>.
- Williams, C. K. I., and C. E. Rasmussen. 1996. *Gaussian Processes for Regression*. Advances in Neural Information Processing Systems 8, 514–520. <https://proceedings.neurips.cc/paper/1995/hash/7cce53cf90577442771720a370c3c723-Abstract.html>.
- Williams, C. K. I., and M. Seeger. 2001. *Using the Nyström Method to Speed up Kernel Machines*. NeurIPS 13. <https://proceedings.neurips.cc/paper/2000/hash/19de10adbaa1b2ee13f77f679fa1483a-Abstract.html>.
- Wilson, Andrew Gordon, and Hannes Nickisch. 2015. “Kernel Interpolation for Scalable Structured Gaussian Processes.” *Proceedings of the 32nd International Conference on Machine Learning* 37: 1775–84. <https://proceedings.mlr.press/v37/wilson15.html>.
- Wilson, Hu, A. G., and E. P. Xing. 2016. *Deep Kernel Learning*. Proceedings of Machine Learning Research 51, 370–378. <https://proceedings.mlr.press/v51/wilson16.html>.
- Yamanishi, Vert, Y., and M. Kanehisa. 2004. *Protein Network Inference from Multiple Genomic Data: A Supervised Approach*. Bioinformatics 20(Suppl 1), i363–i370. <https://doi.org/10.1093/bioinformatics/bth910>.
- Young, G., and A. S. Householder. 1938. *Discussion of a Set of Points in Terms of Their Mutual Distances*. Psychometrika 3(1), 19–22. <https://doi.org/10.1007/bf02287916>.
- Zhang, Xu, H., and J. Zhang. 2009. *Reproducing Kernel Banach Spaces for Machine Learning*. Journal of Machine Learning Research 10, 2741–2775. <https://doi.org/10.1109/ijcnn.2009.5179093>.

